

Coral

1.7.8

Volume II · Referência e Arquitetura

Sintaxe, bibliotecas, ferramentas, instalação, arquitetura e referência consolidada

EDIÇÃO 1.7.8

Revisão de leitura QoL 1 · outubro de 2026

Português corrente controlado. Contratos verificáveis. Ferramentas integradas.

Coleção oficial: consulte o relatório de qualidade e pendências editoriais.

Como usar este livro

Este volume reúne linguagem, biblioteca, ferramentas, arquitetura e compatibilidade. As assinaturas são extraídas do runtime; lacunas editoriais permanecem registradas no relatório de validação.

Caixas laterais destacam notas, compatibilidade e limites de evidência. Exemplos em Coral são validados pelo compilador da própria árvore antes da release.

Sumário navegável

Como usar este livro 2

Sumário navegável 3

Coral 1.7.8 · Volume II 5

Sobre esta edição 5

Como consultar este volume 5

PARTE I: REFERÊNCIA DA LINGUAGEM 6

Coleções tipadas 12

Nulidade 12

Verificação conservadora 12

Herança múltipla 13

Chamada cooperativa ao pai 13

Editor 14

Cadastro de referência 14

Organização sugerida 14

chame e execute 15

Importação 15

Editor 15

Limites do programa principal 15

Contratos e composição 17

Diagnósticos 17

Editor e ensino 17

Qualificação da distribuição 18

Contratos importados na Coral 1.7.4 18

PARTE II: REFERÊNCIA COMPLETA DA BIBLIOTECA 22

Como consultar a biblioteca 22

Índice de bibliotecas 44

PARTE III: FERRAMENTAS 1014

CLI Coral 1014

VS Code 1015

Depuração 1016

Diagnóstico e logs 1016

Referência do CLI 1017

Configurações 1020

Solução de problemas 1022

Instalação 1023

Realce semântico na 1.7.4 1026

Editor na Coral 1.7.6 1026

PARTE IV: IMPLEMENTAÇÃO E ARQUITETURA 1028

Fundação em Python 1028

Pipeline do compilador 1029

Lexer e expressões 1032

Programa, blocos e AST 1033

Semântica e símbolos 1034

Backend e mapas de linhas 1035

Módulos e construção 1035

LSP 1036

DAP 1038

Extensão e VSIX 1039

Instalação e Manager 1040

Princípios de manutenção 1041

Resolução e build incremental na 1.7.4 1043

Documentos, snapshots e índice seletivo na 1.7.5 1044

PARTE V: COMPATIBILIDADE, LIMITES E INTEROPERABILIDADE 1045

Compatibilidade da 1.7.4 1047

Compatibilidade da 1.7.5 1048

Coral 1.7.8 · Volume II

Coral: Referência e Arquitetura

Sobre esta edição

Este volume incorpora a referência da linguagem, o inventário público extraído do runtime, ferramentas e arquitetura. Assinaturas sem anotação são identificadas como tais. A implementação é incluída para consulta offline e para não inventar contratos ausentes.

Como consultar este volume

Use o sumário para escolher uma parte. Na referência de bibliotecas, escolha primeiro o módulo e depois o símbolo. Leia a assinatura, os parâmetros, o retorno e os erros antes de copiar um exemplo.

Código Coral apresenta instruções da linguagem. **Terminal** apresenta comandos do sistema. **Saída esperada** apresenta o resultado. **Assinatura para consulta** descreve uma chamada e não é um programa completo. **Código Python** pertence à implementação do runtime.

No HTML, ajuste fonte, largura e tema; use Copiar nos exemplos. A implementação canônica fica recolhida e pode ser aberta quando necessário. No PDF, todos os trechos permanecem disponíveis. As fichas incompletas continuam identificadas; o inventário não substitui um contrato documentado.

PARTE I: REFERÊNCIA DA LINGUAGEM

Objetivo desta parte

Consultar a sintaxe e os contratos da linguagem.

Como usar esta referência

Identifique a construção e confira suas condições de uso. Trechos de referência podem depender de variáveis definidas anteriormente.

Referência da linguagem vigente na Coral 1.7.6. As construções abaixo incluem trechos que dependem de variáveis definidas pelo programa. Os capítulos introdutórios e `Livro/Exemplos/` contêm exemplos completos e executáveis.

1. Escopo

A Coral 1.7.6 preserva a compatibilidade da família 1.x. Esta parte reúne os contratos vigentes da linguagem, incluindo programação geral, projetos, objetos, ferramentas e superfícies especializadas.

2. Princípio lexical e de leitura

Código Coral é um subconjunto controlado do português. Palavras com acento são aceitas na forma canônica e várias formas sem acento continuam reconhecidas por compatibilidade. A linguagem não usa processamento de linguagem natural livre: cada construção possui gramática determinística.

3. Blocos

Blocos são delimitados por `fim`. A indentação é formatada oficialmente, mas `fim` permanece o delimitador sintático explícito.

Código Coral

```
defina vida como 0
se vida for menor ou igual a 0 então
    mostre "Fim"
senão
    subtraia 1 de vida
fim
```

Saída esperada

```
Fim
```

4. Estado e operações básicas

Código Coral

```
defina pontos como 10
adicione 5 a pontos
subtraia 2 de pontos
mostre pontos
```

Saída esperada

```
13
```

5. Funções e chamadas naturais

Código Coral

```
crie a função dobro com numero chamada como "dobre {numero}"
    retorne numero vezes 2
fim

defina resultado como dobre 10
```

Este exemplo não imprime texto no terminal.

A forma natural é declarativa e participa da AST de expressões; não há interpretação por IA.

6. Regras, eventos e tempo

Código Coral

```
regra "ferido" quando vida for menor ou igual a 5 então
    defina estado como "ferido"
fim

quando ocorrer "cura" recebendo valor
    adicione valor a vida
fim

a cada 1 segundo faça
    subtraia 1 de energia
fim
```

Este exemplo não imprime texto no terminal.

Regras usam `MotorRegras`. Modos disponíveis: `ao ativar`, `sempre` e `uma vez`.

7. Superfície natural de mundos

Código Coral

```
crie um mundo chamado campanha com nome "Minha Campanha"
crie uma entidade chamada vila no mundo campanha
crie uma entidade chamada bosque no mundo campanha
relacione vila com bosque como "estrada" no mundo campanha
encontre um caminho de vila até bosque no mundo campanha como rota
```

Este exemplo não imprime texto no terminal.

`coral.mundo` representa entidades e relações como grafo dirigido e oferece busca de caminho.

8. Superfície natural de RPG

Código Coral

```
crie um mundo chamado campanha com nome "Campanha"
crie um personagem chamado monstro no mundo campanha com 10 de vida
crie um personagem chamado heroina no mundo campanha com 12 de vida
role um dado de 20 lados usando a semente 42 como ataque
cause 3 de dano a heroina
cure 2 de vida de heroina
crie um combate chamado batalha com heroina e monstro
inicie o combate batalha
```

Este exemplo não imprime texto no terminal.

Rolagens aceitam sementes para reprodutibilidade.

9. Superfície natural de jogos

Código Coral

```
de coral.jogos importe AZUL

crie um jogo chamado jogo com largura 800 e altura 450 sem janela
crie uma cena chamada principal
crie um sprite chamado jogador em 100 e 180 com largura 40 e altura 40
adicione jogador à cena principal
mude jogo para a cena principal

quando o jogo jogo atualizar com dt
    se a tecla "direita" estiver pressionada no jogo jogo então
        mova jogador 180 vezes dt para a direita
    fim
fim

quando o jogo jogo desenhar
    limpe jogo com AZUL
    desenhe o texto "Olá" em 20 e 20 no jogo jogo
fim

execute jogo.executar(max_quadros=2)
```

Este exemplo não imprime texto no terminal.

A implementação gráfica oficial usa Pygame CE; `BackendNulo` permite testes sem janela.

10. Modo Aprender

`coral explicar`, `coral aprender` e `coral rastrear` executam o programa com rastreamento pedagógico de linhas, variáveis, escopos e saída. A saída JSON é consumida pela extensão VS Code.

11. Auditoria de Naturalidade

`coral naturalidade arquivo.coral` não verifica correção semântica. Ele avalia se o código está usando escapes ou superfícies de baixo nível quando já existe forma Coral oficial.

12. Ferramentas

A distribuição inclui formatter, linter, REPL, test runner, documentação automática, LSP, Debug Adapter, templates, cache de compilação, conformidade e gates de release.

13. Compatibilidade

Código válido em Coral 1.0 e 1.1 deve permanecer válido na 1.2, salvo defeito previamente não especificado. As novas construções da 1.2 são aditivas.

14. Fora do escopo

A linha científica/acadêmica, NumPy, PyTorch e Experimentos Coral permanecem reservados para estudo futuro da 1.3 e não fazem parte do contrato da 1.2.

15. Contrato de naturalidade estável

A distribuição 1.2 inclui `coral naturalidade ARQUIVO_OU_PROJETO`. A auditoria é uma ferramenta de estilo: não torna formas interoperáveis inválidas, mas sinaliza quando uma API oficial já possui uma forma Coral mais legível. Em projetos, a pontuação geral é a média por arquivo para não punir artificialmente repositórios maiores. `--json` fornece saída destinada a CI, extensões de editor e outras ferramentas.

O contrato editorial da 1.2 exige que templates, tutoriais e vitrines oficiais priorizem a superfície natural sempre que ela exista.

A tabela resume formas reconhecidas. Os nomes em maiúsculas são espaços reservados explicativos, não palavras que você deve copiar literalmente. Cada bloco precisa de corpo e encerramento adequados.

Construção	Forma de referência
Atribuição	`defina NOME como EXPRESSAO`
Atribuição alternativa	`NOME recebe EXPRESSAO`
Tipo opcional	`defina NOME do tipo TIPO como EXPRESSAO`
Saída	`mostre EXPRESSAO`
Atualização	`adicione VALOR a NOME`, `subtraia VALOR de NOME`
Condicional	`se CONDICAO então ... senão ... fim`
Repetição	`enquanto CONDICAO faça ... fim`
Intervalo	`para NOME de INICIO até FIM faça ... fim`
Coleção	`para cada NOME em EXPRESSAO faça ... fim`
Função	`crie a função NOME com PARAMETROS ... fim`
Chamada natural	`chamada como "PADRAO {parametro}"` no cabeçalho da função
Retorno	`retorne EXPRESSAO`

Construção	Forma de referência
Gerador	`produza EXPRESSAO`, `produza de EXPRESSAO`
Chamada por efeito	`execute EXPRESSAO` ou `chame FUNCAO com ARGUMENTOS`
Importação	`importe MODULO`, `de MODULO importe NOME e OUTRO`
Classe	`crie a classe NOME ... fim`
Teste	`teste "NOME" ... fim`
Verificação	`garanta que CONDICAO`
Exceção	`tente ... se der erro ... finalmente ... fim`
Regra	`regra "NOME" quando CONDICAO então ... fim`
Evento	`quando ocorrer "NOME" recebendo PARAMETRO ... fim`
Temporizador	`a cada INTERVALO segundos faça ... fim`
Escape	`python: INSTRUCAO_PYTHON`

Léxico e blocos

Use arquivos UTF-8 e comentários iniciados com `#`. A política pública de identificadores é única: o primeiro caractere aceita `_`, letras ASCII, `ª`, `º` e letras latinas de Latin-1; os seguintes também aceitam os dígitos ASCII `0` a `9`. Outros alfabetos Unicode não pertencem ao contrato lexical atual. Maiúsculas e minúsculas importam nos nomes de variáveis.

Strings admitem aspas simples ou duplas. Dentro delas, espaços são dados. Escapes como `\n` representam caracteres especiais. Uma expressão numérica usa ponto decimal. A indentação recomendada é de quatro espaços; `fim` é o delimitador explícito.

O que não presumir

A linguagem não interpreta português arbitrário. `leia` aparece em material histórico e sugestões, mas não é uma instrução implementada pelo parser atual; o capítulo de entrada ensina a ponte explícita usada nesta entrega. A sintaxe `de modulo importe nome como apelido` também não é ensinada como válida: use uma atribuição explícita a outro nome após importar.

A gramática `coral-1.0.ebnf` é um artefato histórico. A linha 1.6.1 mantém uma referência sintática versionada gerada a partir do esquema corrente; parser e testes sentinela permanecem a autoridade operacional.

16. Superfície científica e de sistema da 1.3

Código Coral

```
crie um vetor chamado dados com [1, 2, 3]
calcule a média de dados como media_dados

crie uma matriz chamada a com [[2, 0], [0, 4]]
crie um vetor chamado b com [8, 20]
resolva o sistema a com b como solucao
```

Este exemplo não imprime texto no terminal.

Código Coral

```
experimento "amostra" como estudo repetindo 3 vezes usando a semente 10
    retorne 42
fim
salve o experimento estudo em "resultado.json"
defina resumo como estudo.resumo()
mostre resumo["execucoes"]
mostre resumo["sucessos"]
mostre resumo["falhas"]
mostre resumo["passou"]
```

Saída esperada

```
3
3
0
True
```

Os tempos e as medições de memória do resumo variam com a execução e o ambiente. O exemplo confere os campos determinísticos em vez de fixar uma medição de desempenho.

As operações abaixo exigem um processo externo autorizado no ambiente do usuário. São trechos de referência, não uma demonstração isolada:

Referência para leitura

```
obtenha informações do sistema como sistema
leia a variável de ambiente "PATH" como caminho_path
execute o comando "python" com ["--version"] como execucao
```

Uma consulta local pode ser executada sem iniciar outro processo:

Código Coral

```
de coral.sistema importe quantidade_processadores
mostre quantidade_processadores() > 0
```

Saída esperada

```
True
```

A referência a uma porta física exige uma placa conectada e validação de campo. O trecho seguinte ilustra a forma, sem afirmar execução física nesta edição:

Referência para leitura

```
conecte à porta serial "COM3" a 115200 como placa
envie "PING" para placa com quebra de linha
leia uma linha de placa como resposta
feche a conexão placa
```

Essas formas são reconhecidas pelo backend estrutural. A API serial e o backend simulado são testados; a comunicação física continua dependente de evidência em dispositivo real.

Tipagem opcional e herança na Coral 1.7.3

A Coral 1.7.3 amplia as anotações opcionais sem transformar a linguagem em uma linguagem de tipagem obrigatória. Código sem anotações continua válido e dinâmico. Quando há informação suficiente, o analisador consegue apontar incompatibilidades comprovadas sem converter valores automaticamente.

Coleções tipadas

As quatro formas estruturais novas são listas, conjuntos, dicionários e tuplas de posições fixas.

Código Coral

```
defina notas do tipo lista de decimal como [8.0, 9.5]
defina tags do tipo conjunto de texto como {"coral", "linguagem"}
defina idades do tipo dicionário de texto para inteiro como {"Ana": 19}
defina pessoa do tipo tupla de (texto, inteiro) como ("Ana", 19)
```

Este exemplo não imprime texto no terminal.

Tipos podem ser compostos. Por exemplo, `dicionário de texto para lista de inteiro` representa um dicionário cujos valores conhecidos são listas de inteiros.

Nulidade

A forma `T ou nulo` indica que o valor pode ter o tipo informado ou ser nulo.

Código Coral

```
defina apelido do tipo texto ou nulo como nulo
```

Este exemplo não imprime texto no terminal.

A posição dos parênteses importa. `lista de inteiro ou nulo` é uma lista que também pode ser nula. `lista de (inteiro ou nulo)` é uma lista cujos elementos podem ser nulos.

A 1.7.3 não introduz uniões gerais. Formas como `inteiro ou texto` continuam fora do contrato.

Verificação conservadora

As anotações ajudam a análise estática e o editor. Elas não inserem coerções nem validações profundas em tempo de execução. Quando a Coral não consegue provar o tipo de um valor, a informação permanece desconhecida em vez de produzir uma certeza inventada.

Coleções mutáveis são tratadas de modo conservador. Uma `lista de inteiro` não é assumida compatível com `lista de decimal` apenas porque os tipos numéricos individuais possam ter relações úteis em outros contextos.

Herança múltipla

Uma classe pode declarar mais de uma base em ordem explícita:

Código Coral

```
crie a classe Identificavel
    passe
fim
crie a classe Publicavel
    passe
fim
crie a classe Livro herda de Identificavel e Publicavel
    passe
fim
```

Este exemplo não imprime texto no terminal.

A Coral usa a ordem de resolução C3, a mesma regra aplicada pelo backend Python. Em uma hierarquia em diamante, a base comum aparece uma única vez na ordem de resolução.

Bases repetidas, ciclos, bases conhecidas que não são classes e hierarquias comprovadamente inconsistentes recebem diagnóstico antes da execução. Quando uma base é opaca para a análise local, a Coral evita inventar uma ordem estática.

Chamada cooperativa ao pai

Dentro de métodos de instância ou de classe, `chame o método pai` continua a cadeia cooperativa segundo a MRO.

Código Coral

```
crie a classe Raiz
    método nomes retornando lista de texto
        retorne ["Raiz"]
    fim
fim
crie a classe A herda de Raiz
    método nomes retornando lista de texto
        retorne ["A"] mais (chame o método pai nomes)
    fim
fim
```

Este exemplo não imprime texto no terminal.

Construtores podem usar `inicialize a classe pai`. A Coral não chama automaticamente todos os construtores das bases. A cooperação precisa estar explícita nas classes que participam da cadeia.

Editor

O hover apresenta anotações compostas; a hierarquia de tipos mostra todas as bases conhecidas; o autocomplete local considera membros herdados pela MRO C3. Documentos incompletos durante a digitação são tratados conservadoramente. Na Coral 1.7.4, contratos públicos conhecidos permitem analisar tipos e membros entre módulos; vínculos opacos permanecem desconhecidos.

Cadastro de referência

`Exemplos/Tipagem_1_7_3/04_cadastro_tipado.coral` combina os recursos desta versão. O programa mantém uma `lista de Livro`, faz uma busca que retorna `Livro ou nulo`, usa `Livro` com duas bases e inicia a aplicação por `programa principal`.

Programa principal na Coral 1.7.3

A Coral 1.7.3 aceita uma entrada principal opcional para organizar aplicações sem obrigar todo arquivo a usar esse modelo.

Código Coral

```
programa principal
    mostre "Olá"
fim
```

Saída esperada

```
Olá
```

Quando o arquivo é executado diretamente, a Coral inicializa primeiro o módulo inteiro e depois inicia o bloco `programa principal` uma única vez. Isso permite declarar funções e classes abaixo do bloco e utilizá las dentro dele. Quando o mesmo arquivo é importado como módulo, a entrada não é iniciada.

O bloco não recebe parâmetros, não possui anotação de retorno e não cria uma função pública chamada `principal`. Uma função comum com esse nome continua válida e precisa ser chamada normalmente. Também não é permitido devolver um valor da entrada. Um `retorne` vazio pode encerrar o bloco antecipadamente.

Organização sugerida

Código Coral

```
programa principal
    chame apresentar com "Ana"
fim

crie a função apresentar com nome
    mostre "Olá, " mais nome
fim
```

Saída esperada

```
Olá, Ana
```

A entrada é opcional. Scripts lineares continuam aceitos exatamente como antes:

Código Coral

```
mostre "Olá"
```

Saída esperada

```
Olá
```

`chame` e `execute`

`chame nome com argumentos` é a forma corrente da Coral para invocar uma função pelo nome e por seus argumentos, especialmente quando a intenção é realizar uma ação e ignorar o valor devolvido.

`execute EXPRESSAO` executa uma expressão de chamada por efeito. É útil para chamadas explícitas como `execute objeto.metodo(1)` ou quando a forma de expressão é mais adequada. Para consumir um retorno, use a chamada dentro de uma expressão, por exemplo em `defina` ou `mostre`.

Nenhuma dessas formas é usada para iniciar `programa principal`. A própria execução direta do arquivo inicia o bloco automaticamente.

Importação

Um arquivo que contém `programa principal` ainda pode ser usado como módulo. A importação disponibiliza seus símbolos públicos, mas não inicia a aplicação. Isso evita efeitos inesperados em testes, bibliotecas e projetos com vários módulos.

Editor

A extensão Coral reconhece a entrada no outline, hover, dobramento, indentação e realce. Ao pressionar Enter no cabeçalho, o fechamento com `fim` segue a configuração de fechamento automático de blocos. O snippet de entrada principal permanece uma escolha contextual e não substitui automaticamente identificadores digitados pelo usuário.

Limites do programa principal

A entrada não pode ficar dentro de condição, repetição, função, classe, `tente` ou `combinar`. Só pode existir uma entrada principal válida por módulo. REPL e execução interativa de trechos rejeitam esse bloco porque não representam a execução completa do módulo.

Biblioteca base e diagnósticos na Coral 1.7.2

Referência da biblioteca fundamental preservada na edição 1.7.4.

Todas as formas abaixo são expressões. Use `defina resultado como ...`, `mostre ...` ou uma condição. Escrita retorna nulo. As APIs explícitas continuam disponíveis para argumentos opcionais, codificação, ordenação reversa e tipos construídos pelo programa.

Expressão	API equivalente
`o nome do tipo de valor`	`coral.tipos.nome_tipo(valor)`
`o tipo de valor`	`coral.tipos.tipo_de(valor)`
`leia uma linha com valor`	`coral.entrada.ler_linha(valor)`
`existe o caminho valor`	`coral.arquivos.existe(valor)`
`leia o texto de valor`	`coral.arquivos.ler_texto(valor)`
`maiúsculas de valor`	`coral.texto.maiusculas(valor)`
`maiusculas de valor`	`coral.texto.maiusculas(valor)`
`minúsculas de valor`	`coral.texto.minusculas(valor)`
`minusculas de valor`	`coral.texto.minusculas(valor)`
`apare o texto de valor`	`coral.texto.aparar(valor)`
`o primeiro de valor`	`coral.colecoes.primeiro(valor)`
`ordene os valores de valor`	`coral.colecoes.ordenar(valor)`
`os valores únicos de valor`	`coral.colecoes.unicos(valor)`
`os valores unicos de valor`	`coral.colecoes.unicos(valor)`
`conte as ocorrências de valor`	`coral.colecoes.contar(valor)`
`conte as ocorrencias de valor`	`coral.colecoes.contar(valor)`
`serialize json de valor`	`coral.json.para_json(valor)`
`interprete json de valor`	`coral.json.de_json(valor)`
`valor for do tipo inteiro`	`coral.tipos.e_tipo(valor, "inteiro")`
`valor não for do tipo inteiro`	`não e_tipo(valor, "inteiro")`
`converta valor para inteiro`	`coral.conversoes.inteiro(valor)`
`tente converter valor para inteiro, senão nulo`	`coral.conversoes.inteiro(valor, nulo)`
`valor começa com prefixo`	`coral.texto.comeca_com(valor, prefixo)`
`valor termina com sufixo`	`coral.texto.termina_com(valor, sufixo)`
`substitua antigo por novo em valor`	`coral.texto.substituir(valor, antigo, novo)`
`escreva texto em caminho`	`coral.arquivos.escrever_texto(caminho, texto)`

Contratos e composição

Booleano não é inteiro. Decimal corresponde a float; o alias número mantém o sentido atual da API, sem incluir inteiros. O teste aceita os descritores conhecidos por Coral; tipos definidos pelo programa continuam usando e_tipo explicitamente. A forma curta `valor for inteiro` não é aceita. Compare uma variável com `valor for igual a inteiro`.

Consultas prefixadas recebem um operando unário. Para uma expressão composta use parênteses: `o tipo de (a + b)`, `maiúsculas de (primeiro + segundo)`, `tente converter entrada para inteiro, senão (padrao + 1)`. Conversões aceitam inteiro, decimal, texto e booleano. As condições combinam normalmente `e`, `ou` e `não`.

Cada argumento é avaliado uma vez, na ordem da chamada de API equivalente mostrada na tabela. Na escrita o caminho é avaliado antes do texto; na substituição o valor é avaliado antes de antigo e novo. O valor padrão de uma conversão é avaliado mesmo quando a conversão é bem sucedida, conforme a API existente.

Contar ocorrências retorna um dicionário de frequências. Primeiro retorna nulo para coleção vazia. Únicos conserva a ordem e exige itens utilizáveis como chaves. Nenhuma operação altera a coleção original.

Arquivos conservam FileNotFoundError e PermissionError. JSON conserva JSONDecodeError. Conversões conservam ErroConversao, destino e causa; entrada conserva FimDeEntrada. A naturalização não troca esses contratos.

Diagnósticos

Terminal e depurador usam a mesma classificação: R100/R101 entrada, R102 conversão, R110/R111/R112 arquivo, R120 formato, R130 operação, R201 aritmética, R202 valor, R203 tipo, R204 acesso, R205 nome e R001 execução não classificada. A mensagem original é preservada, acompanhada de uma sugestão. O diagnóstico estruturado conserva tipo Python, fonte, linha Python e cadeia de causas em detalhe.

Frames de bibliotecas externas não recebem uma linha Coral por coincidência numérica. Em projetos, mapas persistidos no build localizam o módulo Coral mais profundo. O campo adicional do DAP está em `exceptionInfo.details.diagnostico`.

Editor e ensino

LSP e snippets apresentam as formas aceitas. Formatação preserva a escrita. Ensinar e ModoAprender explicam contratos sem executar a expressão. Naturalidade sugere N112 somente para imports explícitos reconhecidos, sem redefinição conhecida; APIs continuam sendo opções válidas.

Qualificação da distribuição

Na raiz Coral, execute `bash Projeto/tools/validar_cp9_1_7_4.sh`. A execução completa instala dependências num ambiente dedicado, testa uma cópia externa e entrega um ZIP com logs, resultados e hashes. A distribuição original não recebe evidências nem caches. Para uma verificação local sem dependências adicionais use `python3 Projeto/tools/qualificar_1_7_4.py --grupo focados`.

`existe o caminho valor` usa a normalização da API, inclusive expansão de `~`. As formas anteriores `existe o arquivo ...` e `caminho existe` conservam a semântica anterior de existência do sistema, sem essa expansão.

Contratos importados na Coral 1.7.4

O verificador estrutural aceita interfaces públicas de módulos Coral como contexto explícito. Resolve assinaturas de funções, valores com tipos declarados, classes, bases, construtores e métodos. Imports em escopo local são processados na ordem das instruções; um novo import restaura o vínculo naquele escopo.

Aliases e reexportações são seguidos até a origem canônica quando comprováveis. Ciclos sem conclusão e símbolos opacos permanecem desconhecidos. A verificação não infere contratos gerais de módulos Python nem transforma ausência de informação em incompatibilidade. Atributos sem anotação não ganham um contrato garantido por mera atribuição.

Os diagnósticos de incompatibilidade continuam advertências. O suporte não implica renomeação geral de membros qualificados nem análise completa dos efeitos de execução.

Chamadas explícitas em várias linhas

Quando uma chamada tem muitos argumentos, coloque cada argumento numa linha. `chame somar com` abre o corpo porque `com` termina a linha sem argumentos; `fim` encerra essa chamada.

Código Coral

```
crie a função somar com primeiro, segundo
    retorne primeiro mais segundo
fim
defina total como chame somar com
    10
    chame somar com
        2
        3
    fim
fim
mostre total
```

Saída esperada

```
15
```

O primeiro `fim` encerra a chamada interna e o segundo encerra a externa. Os argumentos são avaliados uma vez, na ordem escrita. A versão compacta `chame somar com 10, 5` continua válida.

Cada entrada aceita uma expressão completa. Comentários e linhas vazias não contam como argumentos. Uma vírgula final é opcional. Argumentos nomeados e expansões seguem as regras da chamada compacta. Para uma chamada sem argumentos, use a forma compacta sem `com`; abrir um bloco vazio com `com` é erro.

A chamada em bloco também pode criar objetos e chamar métodos. Ela pode aparecer em atribuições, retornos e argumentos de outra chamada. Se fizer parte de uma expressão maior, use os agrupamentos da linguagem para deixar a continuação explícita. O fechamento deve ocupar sua própria linha.

Use `chame` quando a chamada participa da organização natural do programa; `execute funcao(...)` continua disponível para executar a função por seu efeito. As duas formas invocam a operação; consumir ou ignorar o retorno depende do contexto. O suporte descrito aqui é para chamadas explícitas, não uma nova forma de declarar parâmetros em várias linhas.

Contrato normativo das chamadas multilinha

A abertura exige uma chamada explícita cujo `com` encerre o código da linha sem argumentos. A análise lexical respeita strings, comentários, parênteses e chamadas aninhadas. A construção usa o mesmo nó Chamada e os mesmos argumentos da forma compacta. Não há avaliação adicional, reordenação ou conversão do corpo em uma sequência opaca de texto.

Alvo, argumentos e fechamento conservam suas posições físicas. Diagnósticos e mapas de linha apontam o argumento correspondente, inclusive quando a chamada está numa lista ou em outra chamada. O formatador conserva a forma escolhida e seu resultado deve ser idempotente. Fechamento ausente e bloco vazio são erros do parser estrito. A recuperação de edição não autoriza execução de uma fonte incompleta.

Contrato normativo da consulta de pertencimento

As construções `valor for do tipo Classe` e `valor não for do tipo Classe` são expressões booleanas. A primeira chama semanticamente `coral.tipos.e_tipo(valor, Classe)`; a segunda nega esse resultado. Tipos nativos mantêm a forma atual com seus descritores. Nomes de classes são referências resolvidas por escopo e pelas interfaces importadas, incluindo aliases; não são convertidos em texto para comparação nominal.

O nó ConsultaTipo preserva a expressão consultada, a referência de classe ou o descritor nativo e as coordenadas da fonte. O backend emite o mesmo contrato público de `e_tipo`. Uma instância de subclasse satisfaz a consulta às classes ancestrais válidas, inclusive em herança múltipla. A MRO existente não é substituída.

T206 identifica um nome existente que não representa classe ou tipo; T207 identifica um tipo inexistente e pode sugerir nomes próximos. A análise é conservadora quando não

consegue comprovar a identidade. Tipos nativos conservam sua precedência; evite reutilizar seus nomes para classes locais. Dentro de métodos, classes declaradas posteriormente podem ser referenciadas, mas precisam existir quando a consulta for executada.

Exemplo: uma célula que guarda conteúdo

O objeto abaixo pode guardar grama ou um animal. Seus métodos fazem perguntas sobre esse conteúdo sem precisar comparar textos ou repetir regras de herança.

Código Coral

```
crie a classe Celula
    ao criar uma Celula com conteudo
        defina seu conteudo como conteudo
    fim
    método tem_grama
        retorne seu conteudo for do tipo Grama
    fim
    método tem_herbivoro
        retorne seu conteudo for do tipo Herbivoro
    fim
    método tem_carnivoro
        retorne seu conteudo for do tipo Carnivoro
    fim
fim
crie a classe Grama
fim
crie a classe Animal
fim
crie a classe Herbivoro herda de Animal
fim
crie a classe Carnivoro herda de Animal
fim
crie a classe Onivoro herda de Herbivoro e Carnivoro
fim

defina celula como chame Celula com
    Onivoro()
fim
mostre celula.tem_grama()
mostre celula.tem_herbivoro()
mostre celula.tem_carnivoro()
```

Saída esperada

```
False
True
True
```

As classes consultadas nos métodos podem ser declaradas depois da classe Celula; elas precisam estar disponíveis quando esses métodos forem executados. `Onivoro` pertence às duas classes ancestrais do exemplo. Para a aplicação organizada em módulos e testes, consulte `Exemplos/Analise_1_7_5/Ecossistema`.

Cabeçalhos explícitos da 1.7.6

A gramática reconhece três listas físicas opcionais: parâmetros abertos por `com` de uma declaração e fechados por `fim com`; imports seletivos abertos por `de modulo importe` e fechados por `fim`; bases abertas por `herda de` e fechadas por `fim herança`. Um item ocupa uma linha lógica, que pode conter agrupamentos ou uma chamada multilinha de padrão. Vírgula final em um item é opcional. Comentários e linhas vazias não criam itens.

As listas alimentam os mesmos contratos de assinatura, importação e herança das formas compactas. Não existe continuação universal por indentação. Somente os cabeçalhos reconhecidos abrem essas listas; strings e comentários não abrem nem fecham blocos. Listas vazias ou fechamentos incompatíveis produzem diagnóstico na fonte física.

O coletor conserva linhas físicas; o parser registra as posições dos parâmetros, itens importados e bases. O backend emite código equivalente com mapas de linha, e as ferramentas consultam esses dados para assinatura e navegação. O formatador conserva a escolha entre forma compacta e explícita. Enter respeita a configuração de fechamento de blocos e distingue `fim com` e `fim herança` do `fim` do corpo.

PARTE II: REFERÊNCIA COMPLETA DA BIBLIOTECA

Objetivo desta parte

Encontrar funções, classes e contratos das bibliotecas.

Como usar esta referência

Escolha o módulo. Confira a assinatura e a ficha; consulte a implementação apenas se precisar investigar o comportamento.

Como consultar a biblioteca

As fichas pedagógicas verificadas são as mesmas do Volume I. O inventário seguinte enumera símbolos públicos e suas assinaturas, parâmetros nomeados, valores padrão e anotações de retorno diretamente do runtime.

O inventário não atribui tipos que o código não declara. Descrições ausentes e retornos sem anotação são identificados; a implementação canônica de cada módulo é incorporada para consulta sem rede. O gate editorial distingue essa extração da conclusão de todas as fichas normativas.

`<ausente>` representa um sentinela de argumento omitido; ele não é um valor que o programa deve construir. `None` corresponde a nulo nas expressões Coral. As assinaturas mantêm a notação do runtime, enquanto exemplos e formas naturais usam Coral.

Referência de famílias

A lista abaixo registra as assinaturas implementadas. Parâmetros sem valor padrão são obrigatórios. Chamadas podem usar argumentos nomeados quando a assinatura permite.

coral.arquivos

Caminhos relativos usam a pasta de trabalho. Escrita substitui conteúdo; adicionar_texto acrescenta ao final. A pasta pai deve existir, salvo quando criada explicitamente.

Função	Assinatura
`ler_texto`	`ler_texto(caminho, codificacao='utf-8')`
`escrever_texto`	`escrever_texto(caminho, texto, codificacao='utf-8')`
`adicionar_texto`	`adicionar_texto(caminho, texto, codificacao='utf-8')`
`existe`	`existe(caminho)`

Função	Assinatura
`listar`	`listar(caminho='.')`
`criar_pasta`	`criar_pasta(caminho)`

coral.texto

As funções convertem seu argumento principal para texto. dividir retorna lista; juntar recebe separador e coleção.

Função	Assinatura
`maiusculas`	`maiusculas(valor)`
`minusculas`	`minusculas(valor)`
`aparar`	`aparar(valor)`
`dividir`	`dividir(valor, separador=None)`
`juntar`	`juntar(separador, valores)`
`substituir`	`substituir(valor, antigo, novo)`
`comeca_com`	`comeca_com(valor, prefixo)`
`termina_com`	`termina_com(valor, sufixo)`

coral.matematica

`coral.matematica` reúne matemática escalar e discreta sem NumPy. `media` rejeita coleção vazia. `fatorial`, `combinacoes` e `arranjos` exigem inteiros nos seus domínios. `aproximadamente_igual` usa tolerâncias explícitas; NaN nunca compara como aproximadamente igual. `interpolar_linear` também permite extrapolação quando a fração fica fora de 0 a 1. `log10` e `log2` exigem valor positivo. `raiz_nesima` oferece raiz real para índice inteiro positivo e só admite radicando negativo com índice ímpar. `sinal` e operações geométricas novas exigem valores finitos. Constantes: `pi` e `e`.

Função	Assinatura
`raiz`	`raiz(valor)`
`potencia`	`potencia(base, expoente)`
`absoluto`	`absoluto(valor)`
`arredondar`	`arredondar(valor, casas=0)`
`minimo`	`minimo(valores)`
`maximo`	`maximo(valores)`
`soma`	`soma(valores)`
`media`	`media(valores)`

Função	Assinatura
fatorial	fatorial(n)
combinacoes	combinacoes(n, k)
arranjos	arranjos(n, k)
hipotenusa	hipotenusa(a, b)
aproximadamente_igual	aproximadamente_igual(a, b, tolerancia_relativa=1e-9, tolerancia_absoluta=0.0)
interpolar_linear	interpolar_linear(inicio, fim, fracao)
log10	log10(valor)
log2	log2(valor)
arco_seno	arco_seno(valor)
arco_cosseno	arco_cosseno(valor)
arco_tangente	arco_tangente(valor)
raiz_nesima	raiz_nesima(valor, indice)
produto	produto(valores)
sinal	sinal(valor)
volume_bola3	volume_bola3(raio)

coral.datas

Datas suportam ISO 8601, durações e fusos horários explícitos. Uma data/hora sem fuso não é convertida silenciosamente para outro fuso.

Função	Assinatura
agora	agora(fuso=None)
hoje	hoje(fuso=None)
dias	dias(valor)
segundos	segundos(valor)
minutos	minutos(valor)
horas	horas(valor)
semanas	semanas(valor)
formatar	formatar(valor, formato='%d/%m/%Y')
analisar	analisar(texto, formato=None)
de_iso	de_iso(texto)
para_iso	para_iso(valor, vezespec='auto')

Função	Assinatura
`fuso_horario`	`fuso_horario(nome='UTC')`
`converter_fuso`	`converter_fuso(valor, fuso)`
`diferenca`	`diferenca(final, inicial)`

coral.json

para_json/de_json transformam entre dados e texto; ler_json/escrever_json trabalham com arquivos. Objetos arbitrários precisam de conversão explícita.

Função	Assinatura
`para_json`	`para_json(valor, identar=2)`
`de_json`	`de_json(texto)`
`ler_json`	`ler_json(caminho)`
`escrever_json`	`escrever_json(caminho, valor, identar=2)`

coral.colecoes

ordenar retorna nova lista; unicos preserva a primeira ocorrência; contar devolve frequências; agrupar_pares combina duas sequências até a mais curta.

Função	Assinatura
`ordenar`	`ordenar(valores, reverso=False)`
`unicos`	`unicos(valores)`
`contar`	`contar(valores)`
`primeiro`	`primeiro(valores, padrao=None)`
`agrupar_pares`	`agrupar_pares(chaves, valores)`

coral.aleatorio

As funções convenientes continuam podendo usar o gerador comum, mas a 1.5.8 acrescenta `FonteAleatoria` e `fonte(semente)` para simulações reproduzíveis sem depender do estado aleatório global. Uma mesma fonte explícita produz a mesma sequência quando recebe a mesma semente. Esta API é voltada a simulação, testes e jogos, não a segredos.

Função ou tipo	Assinatura
`FonteAleatoria`	`FonteAleatoria(semente=None)`
`fonte`	`fonte(semente=None)`
`inteiro`	`inteiro(minimo, maximo, fonte=None)`

Função ou tipo	Assinatura
`decimal`	`decimal(minimo=0.0, maximo=1.0, fonte=None)`
`escolher`	`escolher(valores, fonte=None)`
`amostra`	`amostra(valores, quantidade, fonte=None)`
`escolha_ponderada`	`escolha_ponderada(valores, pesos, fonte=None)`
`embaralhar`	`embaralhar(valores, fonte=None)`

Superfícies fundamentais acrescentadas na 1.5.8

Módulo	Operações principais
`coral.entrada`	`ler_linha`, `usar_fonte_entrada`, `FonteEntradaSequencial`
`coral.conversoes`	`inteiro`, `decimal`, `texto`, `booleano`
`coral.formatacao`	`montar_texto`, `formatar_valor`
`coral.tipos`	`tipo_de`, `nome_tipo`, `e_tipo`
`coral.persistencia`	`salvar`, `carregar`, `valor_portatil`, `para_dados`, `de_dados`, `texto_canonico`, `serializar_canonico`
`coral.caminhos`	`normalizar_caminho`, `juntar_caminho`
`coral.assincrono`	`aguardar_com_timeout`, `cancelar_tarefa`, `grupo_tarefas`

A persistência portátil preserva as categorias declaradas como portáteis, incluindo tupla, conjunto, bytes, data, data/hora, duração e caminho, além dos valores básicos. Serializações canônicas mantêm ordem determinística onde a ordem lógica não é significativa.

Assinaturas confirmadas no runtime 1.6.9

Módulo	Assinatura
arquivos	`ler_texto(caminho, codificacao='utf-8')`
arquivos	`escrever_texto(caminho, texto, codificacao='utf-8')`
arquivos	`adicionar_texto(caminho, texto, codificacao='utf-8')`
arquivos	`existe(caminho)`
arquivos	`listar(caminho='.')`
arquivos	`criar_pasta(caminho)`
texto	`maiusculas(valor)`
texto	`minusculas(valor)`
texto	`aparar(valor)`
texto	`dividir(valor, separador=None)`
texto	`juntar(separador, valores)`
texto	`substituir(valor, antigo, novo)`
texto	`comeca_com(valor, prefixo)`
texto	`termina_com(valor, sufixo)`
matematica	`raiz(valor)`
matematica	`potencia(base, expoente)`
matematica	`absoluto(valor)`
matematica	`arredondar(valor, casas=0)`
matematica	`minimo(valores)`
matematica	`maximo(valores)`
matematica	`soma(valores)`
matematica	`media(valores)`
matematica	`fatorial(n)`
matematica	`combinacoes(n, k)`
matematica	`arranjos(n, k)`
matematica	`hipotenusa(a, b)`
matematica	`aproximadamente_igual(a, b, tolerancia_relativa=1e-9, tolerancia_absoluta=0.0)`
matematica	`interpolar_linear(inicio, fim, fracao)`
matematica	`log10(valor)`
matematica	`log2(valor)`

Módulo	Assinatura
matematica	`arco_seno(valor)`
matematica	`arco_cosseno(valor)`
matematica	`arco_tangente(valor)`
matematica	`raiz_nesima(valor, indice)`
matematica	`produto(valores)`
matematica	`sinal(valor)`
matematica	`volume_bola3(raio)`
datas	`agora()`
datas	`hoje()`
datas	`dias(valor)`
datas	`segundos(valor)`
datas	`formatar(valor, formato='%d/%m/%Y')`
json	`para_json(valor, identar=2)`
json	`de_json(texto)`
json	`ler_json(caminho)`
json	`escrever_json(caminho, valor, identar=2)`
colecoes	`ordenar(valores, reverso=False)`
colecoes	`unicos(valores)`
colecoes	`contar(valores)`
colecoes	`primeiro(valores, padrao=None)`
colecoes	`agrupar_pares(chaves, valores)`
aleatorio	`inteiro(minimo, maximo)`
aleatorio	`escolher(valores)`
aleatorio	`embaralhar(valores)`
aleatorio	`decimal()`

APIs acrescentadas na linha 1.3

`coral.numerico`

Na 1.6.1, `espaco_linear` exige que `quantidade` represente um inteiro exato. Valores fracionários não são truncados silenciosamente; formas compatíveis que representam exatamente um inteiro, como `3.0` e `"3"`, continuam aceitas.

Função ou tipo	Assinatura resumida
vetor	vetor(valores, tipo=None)
matriz	matriz(linhas, tipo=None)
zeros	zeros(forma, tipo=float)
uns	uns(forma, tipo=float)
intervalo	intervalo(inicio, fim=None, passo=1, tipo=None)
espaco_linear	espaco_linear(inicio, fim, quantidade=50)
identidade	identidade(ordem)
remodelar	remodelar(valores, forma)
concatenar	concatenar(valores, eixo=0)
selecionar_coluna	selecionar_coluna(registros, campo, ausentes="erro")
media	media(valores)
mediana	mediana(valores)
variancia	variancia(valores, populacional=True)
desvio_padrao	desvio_padrao(valores, populacional=True)
percentil	percentil(valores, percentual)
covariancia	covariancia(a, b, populacional=True)
correlacao	correlacao(a, b)
frequencias	frequencias(valores)
moda	moda(valores)
quartis	quartis(valores)
amplitude	amplitude(valores)
intervalo_interquartil	intervalo_interquartil(valores)
mediana_desvios_absolutos	mediana_desvios_absolutos(valores)
media_ponderada	media_ponderada(valores, pesos)
media_geometrica	media_geometrica(valores)
media_harmonica	media_harmonica(valores)
histograma	histograma(valores, faixas)
erro_padrao	erro_padrao(valores)
assimetria	assimetria(valores)
curtose	curtose(valores)
matriz_covariancia	matriz_covariancia(dados, populacional=False)

Função ou tipo	Assinatura resumida
matriz_correlacao	matriz_correlacao(dados)
regressao_linear	regressao_linear(x, y)
intervalo_bootstrap	intervalo_bootstrap(valores, estatistica, confianca, repeticoes, semente)
teste_permutacao	teste_permutacao(a, b, estatistica, repeticoes, semente)
norma	norma(valores)
normalizar	normalizar(valores)
produto_vetorial	produto_vetorial(a, b)
posto	posto(matriz)
minimos_quadrados	minimos_quadrados(coeficientes, termos)
vetor3	vetor3(x, y, z)
distancia_vetores	distancia_vetores(a, b)
angulo_entre_vetores	angulo_entre_vetores(a, b)
projecao_vetorial	projecao_vetorial(a, sobre)
produto_misto3	produto_misto3(a, b, c)
volume_paralelepipedo3	volume_paralelepipedo3(a, b, c)
rotacionar3	rotacionar3(vetor, eixo, angulo)
transformar_ponto3	transformar_ponto3(ponto, transformacao)
matriz_rotacao3	matriz_rotacao3(eixo, angulo)
matriz_translacao3	matriz_translacao3(dx, dy, dz)
matriz_escala3	matriz_escala3(sx, sy=None, sz=None)
compor_transformacoes3	compor_transformacoes3(*transformacoes)
transformar_vetor3	transformar_vetor3(vetor, transformacao)
distancia_ponto_plano3	distancia_ponto_plano3(ponto, ponto_plano, normal)
projetar_ponto_plano3	projetar_ponto_plano3(ponto, ponto_plano, normal)
intersecao_reta_plano3	intersecao_reta_plano3(origem, direcao, ponto_plano, normal, tolerancia=1e-12)
coordenadas_esfericas3	coordenadas_esfericas3(ponto)
ponto_de_esfericas3	ponto_de_esfericas3(raio, azimute, elevacao)
area_triangulo3	area_triangulo3(a, b, c)
volume_tetraedro3	volume_tetraedro3(a, b, c, d)
produto_escalar	produto_escalar(a, b)

Função ou tipo	Assinatura resumida
`multiplicar_matrizes`	`multiplicar_matrizes(a, b)`
`determinante`	`determinante(m)`
`inversa`	`inversa(m)`
`resolver_sistema`	`resolver_sistema(coeficientes, termos)`
`autovalores`	`autovalores(m)`
`forma`	`forma(valor)`

Erros públicos principais: `DependenciaNumericaAusente` e `ErroNumerico`.

coral.graficos

Função ou tipo	Assinatura resumida
Grafico	especificação imutável de apresentação
Serie3D	série tridimensional imutável para pontos ou trajetórias
Superficie3D	grade tridimensional retangular explícita
linha	linha(x, y, nome=None)
dispersao	dispersao(x, y, nome=None)
barras	barras(categorias, valores, nome=None)
grafico_histograma	grafico_histograma(valores, faixas, nome=None)
grafico_caixa	grafico_caixa(grupos)
grafico_funcao	grafico_funcao(funcao, inicio, fim, amostras=200)
series_de_registros	series_de_registros(registros, campo_x, campo_y)
mapa_calor	mapa_calor(matriz, rotulos_linhas=None, rotulos_colunas=None, nome=None)
mapa_correlacao	mapa_correlacao(dados, rotulos=None, nome=None)
barras_erro	barras_erro(x, y, erros, nome=None)
faixa_incerteza	faixa_incerteza(x, centro, inferior, superior, nome=None)
grafico_experimento	grafico_experimento(resultado, eixo_x, metrica)
painel	painel(graficos, colunas=2, titulo="")
dispersao3d	dispersao3d(x, y, z, nome=None)
trajetoria3d	trajetoria3d(x, y, z, nome=None)
superficie3d	superficie3d(x, y, z, nome=None)
configurar	configurar(grafico, titulo=None, eixos=None, unidades=None, legenda=None, tema=None, ...)
para_svg	para_svg(grafico, largura=800, altura=500)
salvar	salvar(grafico, caminho)

Erros públicos principais: ErroGrafico. SVG usa somente a biblioteca padrão; PNG e PDF exigem Matplotlib opcional.

coral.laboratorio

Função ou tipo	Assinatura resumida
executar_experimento	executar_experimento(nome, funcao, repeticoes=1, parametros=None, semente=None, metricas=None, continuar_em_erro=True)
grade_parametros	grade_parametros(parametros)
planejar_experimento	planejar_experimento(parametros, repeticoes=1, limite=None)
selecionar_medicoes	selecionar_medicoes(resultado, criterio)
resumir_por_parametro	resumir_por_parametro(resultado, nome)
validar_resultado	validar_resultado(resultado)
carregar_resultado_json	carregar_resultado_json(caminho)
comparar_experimentos	comparar_experimentos(a, b)
protocolo_medicao	protocolo_medicao()
diagnosticar_laboratorio	diagnosticar_laboratorio()
ResultadoExperimento.resumo	resumo()
ResultadoExperimento.salvar	salvar(caminho)
ResultadoExperimento.salvar_json	salvar_json(caminho)
ResultadoExperimento.salvar_csv	salvar_csv(caminho)
ResultadoExperimento.salvar_markdown	salvar_markdown(caminho)

coral.sistema

Função ou tipo	Assinatura resumida
informacoes	informacoes()
localizar_programa	localizar_programa(nome)
programa_disponivel	programa_disponivel(nome)
quantidade_processadores	quantidade_processadores()
variavel_ambiente	variavel_ambiente(nome, padrao=None)
variavel_ambiente_obrigatoria	variavel_ambiente_obrigatoria(nome)
ambiente_selecionado	ambiente_selecionado(nomes)
pasta_temporaria	pasta_temporaria()
terminal_interativo	terminal_interativo(canal)
argumentos_programa	argumentos_programa()
caminho_executavel	caminho_executavel()
executar_comando	executar_comando(programa, argumentos=(), pasta=None, timeout=None, ambiente=None, entrada=None, codificacao=None, erros_decodificacao="estrito", limite_saida_bytes=None)
processo_atual	processo_atual()
ResultadoComando.linhas_saida	linhas_saida()
ResultadoComando.linhas_erro	linhas_erro()
ResultadoComando.codigo_saida	codigo_saida()
ResultadoComando.tempo_decorrido	tempo_decorrido()
ResultadoComando.exigir_sucesso	exigir_sucesso()

ResultadoComando preserva codigo, saida, erro, duracao_segundos e sucesso, e acrescenta saida_truncada e erro_truncado. O limite de captura é aplicado durante a leitura dos fluxos; timeout tem prioridade e processos continuam sendo iniciados sem shell.

coral.hardware

Função ou tipo	Assinatura resumida
cpu	cpu()
memoria	memoria()
discos	discos()
bateria	bateria()
diagnosticar_hardware	diagnosticar_hardware()
portas_seriais	portas_seriais()
conectar_serial	conectar_serial(porta, velocidade=115200, timeout=1.0)
ConexaoSerial.enviar	enviar(texto, terminar_linha=False)
ConexaoSerial.ler	ler(quantidade=1)
ConexaoSerial.ler_linha	ler_linha()
ConexaoSerial.fechar	fechar()

A ausência de pyserial produz `DependenciaSerialAusente`. A ausência de psutil reduz os dados disponíveis de inventário sem impedir a importação do pacote.

A tabela abaixo documenta a superfície Python/OO avançada consolidada na linha 1.5 e revisada para o runtime 1.6.1; para código introdutório, prefira as frases naturais mostradas nos capítulos anteriores.

Módulo	Símbolo / assinatura
coral.regras	classe Regra(nome: 'str', condicao: 'Condicao', acao: 'Acao', modo: 'str' = 'ao_ativar', habilitada: 'bool' = True, ativa: 'bool' = False, execucoes: 'int' = 0) -> None
coral.regras	Regra.avaliar(self, contexto: 'Any') -> 'bool'
coral.regras	classe Temporizador(intervalo: 'float', acao: 'Acao', repetir: 'bool' = False, nome: 'str' = 'temporizador', restante: 'float | None' = None, ativo: 'bool' = True) -> None
coral.regras	Temporizador.atualizar(self, contexto: 'Any', dt: 'float') -> 'int'
coral.regras	classe Eventos()
coral.regras	Eventos.emitir(self, nome: 'str', *args, **kwargs) -> 'int'
coral.regras	Eventos.quando(self, nome: 'str', funcao: 'Callable[..., Any]')
coral.regras	classe MotorRegras()
coral.regras	MotorRegras.a_cada(self, segundos: 'float', acao: 'Acao', *, nome: 'str' = 'intervalo') -> 'Temporizador'
coral.regras	MotorRegras.adicionar(self, regra: 'Regra') -> 'Regra'

Módulo	Símbolo / assinatura
coral.regras	`MotorRegras.atualizar(self, contexto: 'Any', dt: 'float') -> 'tuple[str, ...]'`
coral.regras	`MotorRegras.avaliar(self, contexto: 'Any') -> 'tuple[str, ...]'`
coral.regras	`MotorRegras.depois_de(self, segundos: 'float', acao: 'Acao', *, nome: 'str' = 'depois') -> 'Temporizador'`
coral.regras	`MotorRegras.regra(self, nome: 'str', condicao: 'Condicao', acao: 'Acao', *, modo: 'str' = 'ao_ativar') -> 'Regra'`
coral.regras	função `regra(nome: 'str', quando: 'Condicao', entao: 'Acao', *, modo: 'str' = 'ao_ativar') -> 'Regra'`
coral.mundo	classe `Entidade(nome: 'str', atributos: 'dict[str, Any]' = <factory>, tags: 'set[str]' = <factory>, inventario: 'list[Any]' = <factory>) -> None`
coral.mundo	`Entidade.adicionar_tag(self, tag: 'str') -> "'Entidade'"`
coral.mundo	`Entidade.definir(self, nome: 'str', valor: 'Any') -> "'Entidade'"`
coral.mundo	`Entidade.guardar(self, item: 'Any') -> 'Any'`
coral.mundo	`Entidade.obter(self, nome: 'str', padrao: 'Any' = None) -> 'Any'`
coral.mundo	`Entidade.possui_tag(self, tag: 'str') -> 'bool'`
coral.mundo	`Entidade.remover_item(self, item: 'Any') -> 'bool'`
coral.mundo	`Entidade.remover_tag(self, tag: 'str') -> 'None'`
coral.mundo	classe `Relacao(origem: 'Entidade', destino: 'Entidade', tipo: 'str', dados: 'dict[str, Any]' = <factory>) -> None`
coral.mundo	classe `Mundo(nome: 'str' = 'mundo')`
coral.mundo	`Mundo.adicionar(self, entidade: 'Entidade') -> 'Entidade'`
coral.mundo	`Mundo.atualizar(self, dt: 'float' = 0.0) -> 'tuple[str, ...]'`
coral.mundo	`Mundo.buscar(self, nome: 'str') -> 'Entidade | None'`
coral.mundo	`Mundo.caminho(self, origem: 'Entidade | str', destino: 'Entidade | str', *, tipo: 'str | None' = None, bidirecional: 'bool' = True) -> 'tuple[Entidade, ...]'`
coral.mundo	`Mundo.com_tag(self, tag: 'str') -> 'tuple[Entidade, ...]'`
coral.mundo	`Mundo.criar_entidade(self, nome: 'str', **atributos: 'Any') -> 'Entidade'`
coral.mundo	`Mundo.emitir(self, evento: 'str', *args, **kwargs) -> 'int'`
coral.mundo	`Mundo.exigir(self, nome: 'str') -> 'Entidade'`
coral.mundo	`Mundo.para_dict(self) -> 'dict[str, Any]'`
coral.mundo	`Mundo.para_json(self, **kwargs) -> 'str'`

Módulo	Símbolo / assinatura
coral.mundo	`Mundo.relacionados(self, entidade: 'Entidade | str', tipo: 'str | None' = None, *, direcao: 'str' = 'saida') -> 'tuple[Entidade, ...]'`
coral.mundo	`Mundo.relacionar(self, origem: 'Entidade | str', destino: 'Entidade | str', tipo: 'str', **dados: 'Any') -> 'Relacao'`
coral.mundo	`Mundo.remover(self, entidade: 'Entidade | str') -> 'bool'`
coral.mundo	`Mundo.remover_relacao(self, relacao: 'Relacao') -> 'bool'`
coral.mundo	função `criar_mundo(nome: 'str' = 'mundo') -> 'Mundo'`
coral.rpg	classe `Rolagem(expressao: 'str', dados: 'tuple[int, ...]', modificador: 'int' = 0) -> None`
coral.rpg	classe `Personagem(nome: 'str', vida: 'int' = 10, *, vida_max: 'int | None' = None, **atributos: 'Any')`
coral.rpg	`Personagem.adicionar_tag(self, tag: 'str') -> "'Entidade'"`
coral.rpg	`Personagem.curar(self, quantidade: 'int') -> 'int'`
coral.rpg	`Personagem.definir(self, nome: 'str', valor: 'Any') -> "'Entidade'"`
coral.rpg	`Personagem.guardar(self, item: 'Any') -> 'Any'`
coral.rpg	`Personagem.obter(self, nome: 'str', padrao: 'Any' = None) -> 'Any'`
coral.rpg	`Personagem.possui_tag(self, tag: 'str') -> 'bool'`
coral.rpg	`Personagem.receber_dano(self, quantidade: 'int') -> 'int'`
coral.rpg	`Personagem.remover_item(self, item: 'Any') -> 'bool'`
coral.rpg	`Personagem.remover_tag(self, tag: 'str') -> 'None'`
coral.rpg	`Personagem.teste(self, atributo: 'str', dificuldade: 'int' = 10, *, semente: 'int | None' = None) -> 'tuple[Rolagem, bool]'`
coral.rpg	classe `CombateTurnos(participantes: 'Iterable[Personagem]' = ())`
coral.rpg	`CombateTurnos.adicionar(self, personagem: 'Personagem') -> 'Personagem'`
coral.rpg	`CombateTurnos.iniciar(self, *, iniciativas: 'dict[str, int] | None' = None, semente: 'int | None' = None) -> 'tuple[Personagem, ...]'`
coral.rpg	`CombateTurnos.proximo(self) -> 'Personagem | None'`
coral.rpg	função `rolar(expressao: 'str' = '1d20', semente: 'int | None' = None, *, gerador: 'random.Random | None' = None) -> 'Rolagem'`
coral.rpg	função `criar_personagem(mundo: 'Mundo', nome: 'str', vida: 'int' = 10, **atributos: 'Any') -> 'Personagem'`
coral.jogos	classe `Jogo(titulo: 'str', largura: 'int' = 800, altura: 'int' = 450, fps: 'int' = 60, *, backend: 'BackendJogos', tela_cheia: 'bool' = False)`

Módulo	Símbolo / assinatura
coral.jogos	`Jogo.desenhar_circulo(self, x: 'float', y: 'float', raio: 'float', cor: 'Cor' = Cor(vermelho=255, verde=255, azul=255, alfa=255), *, contorno: 'int' = 0) -> 'None'`
coral.jogos	`Jogo.desenhar_retangulo(self, x: 'float', y: 'float', largura: 'float', altura: 'float', cor: 'Cor' = Cor(vermelho=255, verde=255, azul=255, alfa=255), *, contorno: 'int' = 0) -> 'Retangulo'`
coral.jogos	`Jogo.desenhar_texto(self, texto: 'str', x: 'float', y: 'float', tamanho: 'int' = 24, cor: 'Cor' = Cor(vermelho=255, verde=255, azul=255, alfa=255)) -> 'None'`
coral.jogos	`Jogo.executar(self, *, max_quadros: 'int | None' = None) -> 'None'`
coral.jogos	`Jogo.limpar(self, cor: 'Cor' = Cor(vermelho=0, verde=0, azul=0, alfa=255)) -> 'None'`
coral.jogos	`Jogo.mudar_cena(self, cena) -> 'None'`
coral.jogos	`Jogo.parar(self) -> 'None'`
coral.jogos	`Jogo.quando_atualizar(self, funcao: "Callable[['Jogo', float], Any]")`
coral.jogos	`Jogo.quando_desenhar(self, funcao: "Callable[['Jogo'], Any]")`
coral.jogos	`Jogo.quando_encerrar(self, funcao: "Callable[['Jogo'], Any]")`
coral.jogos	`Jogo.quando_iniciar(self, funcao: "Callable[['Jogo'], Any]")`
coral.jogos	`Jogo.tecla_pressionada(self, tecla: 'str') -> 'bool'`
coral.jogos	`Jogo.posicao_mouse(self) -> 'tuple[int, int]'`
coral.jogos	`Jogo.botao_mouse_pressionado(self, botao: 'str' = 'esquerdo') -> 'bool'`
coral.jogos	`Jogo.mouse_clicado(self, botao: 'str' = 'esquerdo') -> 'bool'`
coral.jogos	`Jogo.mouse_solto(self, botao: 'str' = 'esquerdo') -> 'bool'`
coral.jogos	`Jogo.roda_mouse(self) -> 'tuple[int, int]'`
coral.jogos	`Jogo.em_tela_cheia(self) -> 'bool'`
coral.jogos	`Jogo.definir_tela_cheia(self, ativa: 'bool') -> 'None'`
coral.jogos	`Jogo.alternar_tela_cheia(self) -> 'bool'`
coral.jogos	`Jogo.tocar_som(self, caminho: 'str', volume: 'float' = 1.0) -> 'None'`
coral.jogos	classe `Cor(vermelho: 'int', verde: 'int', azul: 'int', alfa: 'int' = 255) -> None`
coral.jogos	`Cor.rgb(self) -> 'tuple[int, int, int]'`
coral.jogos	classe `Retangulo(x: 'float', y: 'float', largura: 'float', altura: 'float') -> None`
coral.jogos	`Retangulo.colide_com(self, outro: "'Retangulo'") -> 'bool'`
coral.jogos	classe `BackendNulo()`

Módulo	Símbolo / assinatura
coral.jogos	`BackendNulo.apresentar(self) -> 'None'`
coral.jogos	`BackendNulo.desenhar_circulo(self, x: 'float', y: 'float', raio: 'float', cor: 'Cor', *, contorno: 'int' = 0) -> 'None'`
coral.jogos	`BackendNulo.desenhar_retangulo(self, retangulo: 'Retangulo', cor: 'Cor', *, contorno: 'int' = 0) -> 'None'`
coral.jogos	`BackendNulo.desenhar_texto(self, texto: 'str', x: 'float', y: 'float', tamanho: 'int', cor: 'Cor') -> 'None'`
coral.jogos	`BackendNulo.encerrar(self) -> 'None'`
coral.jogos	`BackendNulo.iniciar(self, titulo: 'str', largura: 'int', altura: 'int') -> 'None'`
coral.jogos	`BackendNulo.limitar_fps(self, fps: 'int') -> 'float'`
coral.jogos	`BackendNulo.limpar(self, cor: 'Cor') -> 'None'`
coral.jogos	`BackendNulo.processar_eventos(self) -> 'bool'`
coral.jogos	`BackendNulo.tecla_pressionada(self, tecla: 'str') -> 'bool'`
coral.jogos	`BackendNulo.tocar_som(self, caminho: 'str', *, volume: 'float' = 1.0) -> 'None'`
coral.jogos	classe `Assets(raiz: 'str | Path' = 'assets')`
coral.jogos	`Assets.imagem(self, nome: 'str | Path') -> 'Path'`
coral.jogos	`Assets.som(self, nome: 'str | Path') -> 'Path'`
coral.jogos	classe `Sprite(x: 'float', y: 'float', largura: 'float', altura: 'float', imagem: 'str | Path | None' = None, cor: 'Cor' = Cor(vermelho=255, verde=255, azul=255, alfa=255), velocidade_x: 'float' = 0.0, velocidade_y: 'float' = 0.0, visivel: 'bool' = True, nome: 'str' = 'sprite', dados: 'dict[str, Any]' = <factory>, animacao: 'Any' = None) -> None`
coral.jogos	`Sprite.atualizar(self, dt: 'float') -> 'None'`
coral.jogos	`Sprite.colide_com(self, outro: "'Sprite'") -> 'bool'`
coral.jogos	`Sprite.mover(self, dx: 'float', dy: 'float') -> 'None'`
coral.jogos	classe `Cena(nome: 'str')`
coral.jogos	`Cena.adicionar(self, sprite: 'Sprite') -> 'Sprite'`
coral.jogos	`Cena.atualizar(self, jogo, dt: 'float')`
coral.jogos	`Cena.desenhar(self, jogo)`
coral.jogos	`Cena.entrar(self, jogo)`
coral.jogos	`Cena.quando_atualizar(self, f)`
coral.jogos	`Cena.quando_desenhar(self, f)`
coral.jogos	`Cena.quando_entrar(self, f)`

Módulo	Símbolo / assinatura
coral.jogos	`Cena.quando_sair(self, f)`
coral.jogos	`Cena.remover(self, sprite: 'Sprite') -> 'None'`
coral.jogos	`Cena.sair(self, jogo)`
coral.jogos	classe `Animacao(quadros: 'tuple[str | Path, ...]', fps: 'float' = 10.0, repetir: 'bool' = True, _tempo: 'float' = 0.0, _indice: 'int' = 0, finalizada: 'bool' = False) -> None`
coral.jogos	`Animacao.atualizar(self, dt: 'float')`
coral.jogos	`Animacao.reiniciar(self)`
coral.jogos	classe `Camera(x: 'float' = 0.0, y: 'float' = 0.0, largura: 'float' = 800.0, altura: 'float' = 450.0, alvo: 'Sprite | None' = None, suavidade: 'float' = 1.0) -> None`
coral.jogos	`Camera.atualizar(self)`
coral.jogos	`Camera.seguir(self, sprite: 'Sprite', suavidade: 'float' = 1.0)`
coral.jogos	`Camera.tela(self, x: 'float', y: 'float') -> 'tuple[float, float]'`
coral.jogos	função `criar_jogo(titulo: 'str', largura: 'int' = 800, altura: 'int' = 450, fps: 'int' = 60, *, backend=None) -> 'Jogo'`
coral.jogos	função `pygame_disponivel() -> 'bool'`

Constantes de cor em coral.jogos

`PRETO`, `BRANCO`, `VERMELHO`, `VERDE`, `AZUL` e `AMARELO` são objetos `Cor` prontos para uso.

`coral.criptografia` oferece `resumir`, `verificar_resumo`, `autenticar`, `verificar_autenticacao`, `gerar_bytes_seguros`, `gerar_token_seguro`, `comparar_com_seguranca`, `listar_algoritmos` e `explicar_algoritmo`. Texto usa UTF 8 e valores binários preservam bytes. `resumir` e `verificar_resumo` aceitam SHA 256 ou SHA 512; HMAC pertence a `autenticar` e `verificar_autenticacao`.

Resultados podem ser `hex` ou `bytes`. Uma comparação com resumo válido porém diferente retorna falso, enquanto comprimento, tipo ou representação hexadecimal inválidos produzem erro de formato. Chaves vazias são recusadas e mensagens de erro não reproduzem chaves, dados autenticados ou resultados recebidos.

Bytes e tokens seguros usam a fonte criptográfica da plataforma por meio de `secrets`; eles não reutilizam o gerador pseudoaleatório de `coral.aleatorio` ou `coral.procedural`. O catálogo diferencia hash de autenticação e deixa explícito que nenhum desses mecanismos cifra dados.

Código Coral

```
de coral.criptografia importe resumir, autenticar, verificar_autenticacao

defina mensagem como "Coral ç"
defina chave como "chave de exemplo"
defina resumo como resumir(mensagem, "SHA-256")
defina etiqueta como autenticar(mensagem, chave, "sha256")
garanta que verificar_autenticacao(mensagem, chave, etiqueta, "sha256")
mostre resumo
```

Saída esperada

```
feedefbb9a55b0ec479decef5283caad5e5e103d2b3bf75aa5502132470d4e86
```

A 1.6.16 não oferece cifragem simétrica ou assimétrica, derivação de senha, certificados, assinaturas digitais, armazenamento de credenciais ou gestão de chaves. Essas capacidades exigem um roadmap próprio.

coral.arquivos: operações graduais 1.7.7

As cinco operações de `coral.arquivos` complementam as operações materializadas existentes. `coral.formatos.de_csv` conserva o contrato anterior e retorna dados materializados.

Operação	Assinatura	Retorno
ler_linhas	`ler_linhas(caminho, codificacao='utf-8')`	iterador de textos
ler_csv	`ler_csv(caminho, codificacao='utf-8', *, delimitador=',', cabecalho=True)`	iterador de objetos texto/texto ou listas de textos
ler_lotes_csv	`ler_lotes_csv(caminho, tamanho_lote=100, codificacao='utf-8', *, delimitador=',', cabecalho=True)`	iterador de listas de registros
escrever_linhas	`escrever_linhas(caminho, linhas, codificacao='utf-8')`	nulo
escrever_csv	`escrever_csv(caminho, registros, codificacao='utf-8', *, delimitador=',', colunas=None, cabecalho=True)`	nulo

Leitura e fechamento

Os leitores abrem o arquivo somente no primeiro avanço. São de passagem única: percorrer novamente não reinicia a leitura. `ler_linhas` preserva as terminações de linha, inclusive CRLF e a ausência de quebra na última linha. CSV aceita separadores entre aspas e campos multilinha; linhas físicas não equivalem a registros CSV. O processamento CSV é estrito.

Com `cabecalho=True`, a primeira linha CSV define chaves únicas e não vazias; cada registro precisa ter a mesma largura. Com `cabecalho=False`, cada registro é uma lista de textos. O arquivo vazio é válido. `tamanho_lote` exige inteiro positivo, excluindo booleanos; o último lote pode ser menor.

EOF e erros fecham o leitor. Interromper com `pare` exige um bloco `com` ou chamada explícita a `fechar()`/`close()`. Não há garantia de fechamento por coleta de lixo. Quando um leitor alimenta um escritor, o chamador mantém a responsabilidade pelo leitor: envolva a leitura no bloco `com` externo.

Escrita e falhas

A escrita consome gradualmente o iterável e substitui o destino. Uma falha pode deixar conteúdo parcial: não é uma operação transacional. `escrever_linhas` aceita somente textos e não insere quebras; forneça as terminações desejadas.

`escrever_csv` aceita objetos de chaves/valores textuais ou sequências de textos, sem conversão implícita. `colunas` explicita a ordem; para objetos, sua ausência infere a ordem do primeiro registro. Todos os objetos precisam ter exatamente as mesmas chaves, e as sequências a largura correspondente. Configure `cabecalho=False` para omitir a linha de nomes. O delimitador deve ser um caractere válido para CSV. Codificação, caminhos e permissões inválidos causam erros; valide dados antes se precisar evitar escrita parcial.

Projeto aplicado

`Exemplos/Projetos/Importador_Gradual` é um projeto Coral executável. Lê lotes de dois registros, seleciona `ativo` igual a `sim`, apara `nome` e escreve as colunas `nome,valor`. Consulte seu README para os comandos e arquivos esperados. O editor apresenta o retorno como iterador, lote ou lista conforme a operação.

Para sequências de campos, nomes não são inferíveis: sem colunas explícitas, não é emitido cabeçalho mesmo com cabecalho=True. Objetos permitem inferir nomes na ordem do primeiro registro; colunas explícitas definem nomes e ordem.

coral.esquemas: validação declarativa 1.7.8

Na Coral 1.7.8, `coral.esquemas` descreve contratos de cadastro, pesquisa e inventário com quatro chamadas explícitas: `campo`, `esquema`, `validar_registro` e `validar_registros`. Nenhuma forma natural nova foi criada. A validação aceita ou relata dados; não converte textos, não preenche valores padrão e não altera a entrada.

Ausência significa que a chave não existe. Nulo significa que a chave existe com `nulo`. Um campo opcional pode estar ausente e ainda recusar nulo; um campo obrigatório e anulável exige a chave, mas aceita nulo. Inteiro rejeita booleanos; decimal aceita números finitos e rejeita booleanos. Limites são inclusivos.

Código Coral

```
de coral.esquemas importe campo, esquema, validar_registro
de coral.conversoes importe inteiro

defina cadastro como esquema({"idade": campo("inteiro", minimo=0), "apelido":
  campo("texto", obrigatorio=falso)}, permitir_extras=falso)
defina bruto como {"idade": "18"}
mostre validar_registro(bruto, cadastro)["erros"][0]["regra"]
defina convertido como {"idade": inteiro(bruto["idade"])}
mostre validar_registro(convertido, cadastro)["valido"]
mostre validar_registro({"idade": 18, "apelido": nulo}, cadastro)["erros"][0]["regra"]
```

Saída esperada

```
tipo
True
nulo
```

A conversão `inteiro` é explícita e pode falhar antes da validação; preserve esse erro no relatório de importação. JSON também precisa validar dados após `de_json`. Não confunda texto CSV vazio, chave ausente e nulo JSON. O projeto `Exemplos/Projetos/Importador_Validado` combina conversão, validação e relatório JSON.

Código Coral

```
de coral.esquemas importe campo, esquema, validar_registros

defina pesquisa como esquema({"medida": campo("decimal", minimo=0), "grupo":
  campo("texto", permitidos=["controle", "ensaio"])})
defina resultado como validar_registros([{"medida": -1, "grupo": "outro"}, {"medida": 2,
   "grupo": "ensaio"}], pesquisa, limite_erros=1, linha_inicial=2, origem="pesquisa.csv")
mostre resultado["erros"][0]["campo"]
mostre resultado["erros"][0]["regra"]
mostre resultado["erros"][0]["registro"]
mostre resultado["erros"][0]["linha"]
mostre resultado["erros"][0]["origem"]
mostre resultado["verificados"]
mostre resultado["interrompido"]
```

Saída esperada

```
medida
minimo
1
2
pesquisa.csv
1
True
```

Erros contêm `campo`, `regra`, `mensagem`, `registro`, `linha` e `origem`, sem copiar valores sensíveis. A ordem declarada dos campos determina a ordem do diagnóstico; extras recusados vêm depois, na ordem do registro. Tipo incompatível não avalia limites desse campo. `modo="primeiro"` para no primeiro erro; `modo="coletar"` para ao atingir `limite_erros`, inteiro positivo que não aceita booleano. `interrompido` sinaliza a política, sem alegar que o restante foi examinado. `verificados` conta somente registros consumidos.

`linha_inicial` informa a linha lógica do primeiro registro e os seguintes somam 1; o índice de registro começa em 1. CSV com campos multilinha não permite deduzir linha física pelo índice. `origem` é fornecida pelo chamador. A validação consome uma passagem e guarda apenas erros, sem fechar iteradores alheios. Ao usar `ler_csv`, mantenha o leitor dentro de `com`, inclusive quando o limite interromper a validação. Esquema e opções inválidos falham antes do consumo; falhas de leitura/conversão propagam ao chamador.

Consulte as quatro fichas completas no Volume I e na seção da biblioteca coral.esquemas, a referência no Volume II e `Documentacao/Usuario/Coral_Esquemas_1_7_8.md`.

Índice de bibliotecas

- coral
- coral.aleatorio
- coral.api
- coral.arquivos
- coral.assincrono
- coral.caminhos
- coral.colecoes
- coral.conversoes
- coral.criptografia
- coral.datas
- coral.entrada
- coral.erros
- coral.esquemas
- coral.formatacao
- coral.formatos
- coral.graficos
- coral.hardware
- coral.hardware.serial
- coral.inspecao
- coral.jogos
- coral.json
- coral.laboratorio
- coral.matematica
- coral.mundo
- coral.numerico
- coral.persistencia
- coral.procedural
- coral.regras
- coral.rpg
- coral.sistema
- coral.texto
- coral.tipos
- coral.web
- coral.web.html
- coral.web.http
- coral.web.servidor
- coral.web.url

Biblioteca `coral`

`coral.AmostraInspecao`

Snapshot congelado associado a uma posição temporal e sequência.

Assinatura

Assinatura para consulta

```
AmostraInspecao(sequencia: 'int', snapshot: 'SnapshotInspecao', instante: 'float | None'
    = None, diferenca_anterior: 'DiferencaInspecao | None' = None, contrato: 'str' =
  'coral.inspecao.amostra/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
sequencia	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
snapshot	SnapshotInspecao	POSITIONAL_OR_KEYWORD	sim	sem padrão
instante	float | None	POSITIONAL_OR_KEYWORD	não	None
diferenca_anterior	DiferencaInspecao | None	POSITIONAL_OR_KEYWORD	não	None
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.amostra/1'

Retorno declarado

None

Atributo `sequencia`: `int`.

Atributo `snapshot`: `SnapshotInspecao`.

Atributo `instante`: `float | None`.

Atributo `diferenca_anterior`: `DiferencaInspecao | None`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.CONTRATO_AMOSTRA_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.amostra/1'`

`coral.CONTRATO_COMANDO_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.comando/1'`

`coral.CONTRATO_DIAGNOSTICO_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.diagnostico/1'`

`coral.CONTRATO_EVENTO_OBSERVABILIDADE`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.evento/1'`

`coral.CONTRATO_EXPLICACAO_REGRA`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.explicacao_regra/1'`

`coral.CONTRATO_HISTORICO_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.historico/1'`

`coral.CONTRATO_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao/1'`

`coral.CONTRATO_RELATORIO_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.relatorio/1'`

`coral.CONTRATO_RELEASE`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.release/1'`

`coral.CONTRATO_RESPOSTA_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.resposta/1'`

`coral.CONTRATO_SESSAO_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.sessao/1'`

`coral.CONTRATO_VISAO_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.visao/1'`

`coral.DiagnosticoInspecao`

Diagnóstico leve do contrato de inspeção de um objeto.

Assinatura

Assinatura para consulta

```
DiagnosticoInspecao(ok: 'bool', tipo: 'str', nome: 'str | None', assinatura: 'str |
  None', visoes: 'tuple[str, ...]' = (), avisos: 'tuple[str, ...]' = (), erro: 'str |
  None' = None, contrato: 'str' = 'coral.inspecao.diagnostico/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
ok	bool	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
assinatura	str | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
visoes	tuple[str, ...]	POSITIONAL_OR_KEYWORD	não	()
avisos	tuple[str, ...]	POSITIONAL_OR_KEYWORD	não	()
erro	str | None	POSITIONAL_OR_KEYWORD	não	None
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.diagnostico/1'

Retorno declarado

None

Atributo `ok`: `bool`.

Atributo `tipo`: `str`.

Atributo `nome`: `str | None`.

Atributo `assinatura`: `str | None`.

Atributo `visoes`: `tuple[str, ...]`.

Atributo `avisos`: `tuple[str, ...]`.

Atributo `erro`: `str | None`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.DiferencaInspecao`

DiferencaInspecao(antes: 'str', depois: 'str', mudancas: 'tuple[MudancaInspecao, ...]' = (), contrato: 'str' = 'coral.inspecao.diferenca/1')

Assinatura

Assinatura para consulta

```
DiferencaInspecao(antes: 'str', depois: 'str', mudancas: 'tuple[MudancaInspecao, ...]' =
    (), contrato: 'str' = 'coral.inspecao.diferenca/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
antes	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
depois	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
mudancas	tuple[MudancaInspecao, ...]	POSITIONAL_OR_KEYWORD	não	()
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.diferenca/1'

Retorno declarado

None

Atributo `antes`: `str`.

Atributo `depois`: `str`.

Atributo `mudancas`: `tuple[MudancaInspecao, ...]`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_json(self, **kwargs: 'Any') -> 'str'
```

Método público sem descrição própria no runtime.

`coral.EDICAO_LIVRO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'1.7.0'`

`coral.ErroCoral`

Raiz pública dos erros controlados pela linguagem e bibliotecas Coral.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.ErroDeTraducao`

Erro encontrado durante parsing, análise ou geração de código.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.EventoObservabilidade`

Evento neutro emitido pela infraestrutura de observabilidade.

Assinatura

Assinatura para consulta

```
EventoObservabilidade(sequencia: 'int', tipo: 'str', alvo: 'str | None' = None,
  instante: 'float | None' = None, caminho: 'str | None' = None, valor: 'Any' = None,
  anterior: 'Any' = None, detalhes: 'Mapping[str, Any]' = <factory>, contrato: 'str' =
  'coral.inspecao.evento/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
sequencia	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
alvo	str | None	POSITIONAL_OR_KEYWORD	não	None

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
instante	float | None	POSITIONAL_OR_KEYWORD	não	None
caminho	str | None	POSITIONAL_OR_KEYWORD	não	None
valor	Any	POSITIONAL_OR_KEYWORD	não	None
anterior	Any	POSITIONAL_OR_KEYWORD	não	None
detalhes	Mapping[str, Any]	POSITIONAL_OR_KEYWORD	não	<factory>
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.evento/1'

Retorno declarado

None

Atributo `sequencia`: `int`.

Atributo `tipo`: `str`.

Atributo `alvo`: `str | None`.

Atributo `instante`: `float | None`.

Atributo `caminho`: `str | None`.

Atributo `valor`: `Any`.

Atributo `anterior`: `Any`.

Atributo `detalhes`: `Mapping[str, Any]`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura



```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.ExplicacaoRegra`

Explicação somente leitura do último estado observável de uma regra.

Assinatura

Assinatura para consulta

```
ExplicacaoRegra(nome: 'str | None', estado: 'str', mensagem: 'str', dados: 'Mapping[str,
   Any]' = <factory>, contrato: 'str' = 'coral.inspecao.explicacao_regra/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
estado	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
dados	Mapping[str, Any]	POSITIONAL_OR_KEYWORD	não	<factory>
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.explicacao_regra/1'

Retorno declarado

None

Atributo `nome`: `str | None`.

Atributo `estado`: `str`.

Atributo `mensagem`: `str`.

Atributo `dados`: `Mapping[str, Any]`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.HistoricoInspecao`

Histórico curto e limitado de snapshots de um mesmo alvo.

Nenhuma amostragem acontece em segundo plano. O chamador decide quando `` `registrar` `` é chamado, o que preserva determinismo em testes e simulações.

Assinatura

Assinatura para consulta

```
HistoricoInspecao(*, max_amostras: 'int' = 120, intervalo: 'float' = 0.0,
  somente_mudancas: 'bool' = False, nome_alvo: 'str | None' = None) -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
max_amostras	int	KEYWORD_ONLY	não	120
intervalo	float	KEYWORD_ONLY	não	0.0
somente_mudancas	bool	KEYWORD_ONLY	não	False
nome_alvo	str | None	KEYWORD_ONLY	não	None

Retorno declarado

None

Métodos públicos

Referência para leitura

```
ao_evento(self, callback: 'Callable[[EventoObservabilidade], Any]')
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estatisticas_inspecao(self) -> 'dict[str, int]'
```

Contadores leves para medir a própria observabilidade.

Referência para leitura

```
limpar(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
registrar(self, objeto: 'Any', *, instante: 'float | None' = None, forcar: 'bool' =
  False) -> 'AmostraInspecao | None'
```

Método público sem descrição própria no runtime.

`coral.MudancaInspecao`

MudancaInspecao(caminho: 'str', tipo: 'str', antes: 'Any' = None, depois: 'Any' = None)

Assinatura

Assinatura para consulta

```
MudancaInspecao(caminho: 'str', tipo: 'str', antes: 'Any' = None, depois: 'Any' = None)
  -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
caminho	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
antes	Any	POSITIONAL_OR_KEYWORD	não	None
depois	Any	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `caminho`: `str`.

Atributo `tipo`: `str`.

Atributo `antes`: `Any`.

Atributo `depois`: `Any`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.RelatorioInspecao`

Apresentação humana derivada do mesmo snapshot estruturado.

Assinatura

Assinatura para consulta

```
RelatorioInspecao(tipo: 'str', nome: 'str | None', dados: 'Mapping[str, Any]', visao:
  'str | None' = None, assinatura_origem: 'str | None' = None, contrato: 'str' =
  'coral.inspecao.relatorio/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
dados	Mapping[str, Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
visao	str | None	POSITIONAL_OR_KEYWORD	não	None
assinatura_origem	str | None	POSITIONAL_OR_KEYWORD	não	None
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.relatorio/1'

Retorno declarado

None

Atributo `tipo`: `str`.

Atributo `nome`: `str | None`.

Atributo `dados`: `Mapping[str, Any]`.

Atributo `visao`: `str | None`.

Atributo `assinatura_origem`: `str | None`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_markdown(self) -> 'str'
```

Método público sem descrição própria no runtime.

`coral.RespostaInspecao`

Resposta estável para consumidores externos da infraestrutura de inspeção.

Assinatura

Assinatura para consulta

```
RespostaInspecao(operacao: 'str', ok: 'bool', resultado: 'Any' = None, erro: 'str |
  None' = None, contrato: 'str' = 'coral.inspecao.resposta/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
operacao	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
ok	bool	POSITIONAL_OR_KEYWORD	sim	sem padrão
resultado	Any	POSITIONAL_OR_KEYWORD	não	None
erro	str | None	POSITIONAL_OR_KEYWORD	não	None
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.resposta/1'

Retorno declarado

None

Atributo `operacao`: `str`.

Atributo `ok`: `bool`.

Atributo `resultado`: `Any`.

Atributo `erro`: `str | None`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_json(self, **kwargs: 'Any') -> 'str'
```

Método público sem descrição própria no runtime.

`coral.SessaoInspecao`

Ponte estável entre o runtime e consumidores como DAP ou VS Code.

Assinatura

Assinatura para consulta

```
SessaoInspecao(*, max_eventos: 'int' = 500) -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
max_eventos	int	KEYWORD_ONLY	não	500

Retorno declarado

None

Métodos públicos

Referência para leitura

```
adicionar_alvo(self, nome: 'str', objeto: 'Any', *, max_amostras: 'int' = 120,
  intervalo: 'float' = 0.0, somente_mudancas: 'bool' = False) -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
adicionar_vigia(self, alvo: 'str', vigia: 'VigiaInspecao') -> 'VigiaInspecao'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
amostrar(self, *, instante: 'float | None' = None, forcar: 'bool' = False) -> 'dict[str,
   AmostraInspecao | None]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
consultar(self, alvo: 'str', caminho: 'str', padrao: 'Any' = <ausente>) -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
consultar_eventos(self, *, cursor: 'int' = 0, limite: 'int' = 100, alvo: 'str | None' =
  None, tipos: 'tuple[str, ...] | list[str]' = (), caminho: 'str | None' = None) ->
  'dict[str, Any]'
```

Lê a fila limitada sem esconder perda de eventos por overflow.

Referência para leitura

```
historico(self, alvo: 'str') -> 'HistoricoInspecao'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
limpar_eventos(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_json(self, **kwargs: 'Any') -> 'str'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_alvo(self, nome: 'str') -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
snapshot(self, alvo: 'str') -> 'SnapshotInspecao'
```

Método público sem descrição própria no runtime.

`coral.SnapshotInspecao`

SnapshotInspecao(tipo: 'str', nome: 'str | None', dados: 'Mapping[str, Any]' = <factory>, avisos: 'tuple[str, ...]' = (), contrato: 'str' = 'coral.inspecao/1')

Assinatura

Assinatura para consulta

```
SnapshotInspecao(tipo: 'str', nome: 'str | None', dados: 'Mapping[str, Any]' =
  <factory>, avisos: 'tuple[str, ...]' = (), contrato: 'str' = 'coral.inspecao/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
dados	Mapping[str, Any]	POSITIONAL_OR_KEYWORD	não	<factory>
avisos	tuple[str, ...]	POSITIONAL_OR_KEYWORD	não	()
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao/1'

Retorno declarado

None

Atributo `tipo`: `str`.

Atributo `nome`: `str | None`.

Atributo `dados`: `Mapping[str, Any]`.

Atributo `avisos`: `tuple[str, ...]`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
assinatura(self) -> 'str'
```

Hash estável do snapshot normalizado para comparação e cache.

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_json(self, **kwargs: 'Any') -> 'str'
```

Método público sem descrição própria no runtime.

`coral.VersaoSemantica`

VersaoSemantica(major: 'int', minor: 'int', patch: 'int')

Assinatura

Assinatura para consulta

```
VersaoSemantica(major: 'int', minor: 'int', patch: 'int') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
major	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
minor	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
patch	int	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `major`: `int`.

Atributo `minor`: `int`.

Atributo `patch`: `int`.

Métodos públicos

Referência para leitura

```
analisar(valor: 'str') -> "'VersaoSemantica'"
```

Método público sem descrição própria no runtime.

`coral.VigiaInspecao`

Watchpoint somente leitura sobre um caminho do contrato de inspeção.

Assinatura

Assinatura para consulta

```
VigiaInspecao(nome: 'str', caminho: 'str', *, operador: 'str' = 'mudou', valor: 'Any' =
  None, ativo: 'bool' = True) -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
caminho	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
operador	str	KEYWORD_ONLY	não	'mudou'
valor	Any	KEYWORD_ONLY	não	None
ativo	bool	KEYWORD_ONLY	não	True

Retorno declarado

None

Métodos públicos

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
observar(self, objeto: 'Any', *, alvo: 'str | None' = None, instante: 'float | None' =
  None) -> 'EventoObservabilidade | None'
```

Método público sem descrição própria no runtime.

`coral.VisaoInspecao`

Projeção semântica estável de um snapshot de inspeção.

A visão não possui autoridade sobre o domínio e não altera o objeto observado. Ela existe para que DAP, VS Code, logs e futuros overlays consumam um vocabulário menor que o snapshot completo.

Assinatura

Assinatura para consulta

```
VisaoInspecao(visao: 'str', tipo: 'str', nome: 'str | None', dados: 'Mapping[str, Any]'
  = <factory>, assinatura_origem: 'str | None' = None, avisos: 'tuple[str, ...]' = (),
  contrato: 'str' = 'coral.inspecao.visao/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
visao	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
dados	Mapping[str, Any]	POSITIONAL_OR_KEYWORD	não	<factory>
assinatura_origem	str | None	POSITIONAL_OR_KEYWORD	não	None
avisos	tuple[str, ...]	POSITIONAL_OR_KEYWORD	não	()
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.visao/1'

Retorno declarado

None

Atributo `visao`: `str`.

Atributo `tipo`: `str`.

Atributo `nome`: `str | None`.

Atributo `dados`: `Mapping[str, Any]`.

Atributo `assinatura_origem`: `str | None`.

Atributo `avisos`: `tuple[str, ...]`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_json(self, **kwargs: 'Any') -> 'str'
```

Método público sem descrição própria no runtime.

`coral.__version__`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'1.7.8'`

`coral.analisar_expressao_coral`

Retorna a AST Coral de uma expressão.

Assinatura

Assinatura para consulta

```
analisar_expressao_coral(expr: 'str')
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
expr	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.bases_release`

Deriva as bases de patch anterior e estável sem lookup por release.

Assinatura

Assinatura para consulta

```
bases_release(atual: 'str | None' = None, *, ancora_linha_anterior: 'str | None' = None)
   -> 'dict[str, str]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
atual	str | None	POSITIONAL_OR_KEYWORD	não	None
ancora_linha_anterior	str | None	KEYWORD_ONLY	não	None

Retorno declarado

dict[str, str]

`coral.comparar_inspecoes`

Compara snapshots normalizados de forma determinística e somente leitura.

Assinatura

Assinatura para consulta

```
comparar_inspecoes(antes: 'Any', depois: 'Any', *, incluir: 'tuple[str, ...] |
  list[str]' = (), ignorar: 'tuple[str, ...] | list[str]' = ()) -> 'DiferencaInspecao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
antes	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
depois	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
incluir	tuple[str, ...] | list[str]	KEYWORD_ONLY	não	()
ignorar	tuple[str, ...] | list[str]	KEYWORD_ONLY	não	()

Retorno declarado

DiferencaInspecao

`coral.consultar_inspecao`

Consulta um caminho dentro de `dados` sem expor internals do domínio.

Aceita notação por ponto, índices entre colchetes e `* como curinga. Exemplo: entidades[*].nome`.

Assinatura

Assinatura para consulta

```
consultar_inspecao(objeto: 'Any', caminho: 'str', padrao: 'Any' = <ausente>) -> 'Any'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
caminho	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
padrao	Any	POSITIONAL_OR_KEYWORD	não	<ausente>

Retorno declarado

Any

`coral.diagnosticar_inspecao`

Confirma snapshot, JSON seguro e visões disponíveis sem alterar o alvo.

Assinatura

Assinatura para consulta

```
diagnosticar_inspecao(objeto: 'Any') -> 'DiagnosticoInspecao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

DiagnosticoInspecao

`coral.executar_comando_inspecao`

Executa um comando neutro para DAP, VS Code, CLI ou outras ferramentas.

A função não conhece protocolos de transporte. O consumidor envia somente operação, alvo e argumentos e recebe uma resposta JSON segura.

Assinatura

Assinatura para consulta

```
executar_comando_inspecao(sessao: "'SessaoInspecao'", operacao: 'str', *, alvo: 'str |
  None' = None, argumentos: 'Mapping[str, Any] | None' = None, capturar_erros: 'bool' =
  True) -> 'RespostaInspecao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
sessao	'SessaoInspecao'	POSITIONAL_OR_KEYWORD	sim	sem padrão
operacao	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
alvo	str | None	KEYWORD_ONLY	não	None
argumentos	Mapping[str, Any] | None	KEYWORD_ONLY	não	None
capturar_erros	bool	KEYWORD_ONLY	não	True

Retorno declarado

RespostaInspecao

`coral.explicar_regra`

Explica o último estado de uma Regra usando apenas o contrato de inspeção.

Assinatura

Assinatura para consulta

```
explicar_regra(objeto: 'Any') -> 'ExplicacaoRegra'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

ExplicacaoRegra

`coral.identidade_release`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
identidade_release(atual: 'str | None' = None, *, ancora_linha_anterior: 'str | None' =
  None) -> 'dict[str, object]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
atual	str | None	POSITIONAL_OR_KEYWORD	não	None
ancora_linha_anterior	str | None	KEYWORD_ONLY	não	None

Retorno declarado

dict[str, object]

`coral.inspecionar`

Cria um snapshot somente leitura sem exigir um domínio específico.

Assinatura

Assinatura para consulta

```
inspecionar(objeto: 'Any', *, nome: 'str | None' = None, tipo: 'str | None' = None) ->
  'SnapshotInspecao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	KEYWORD_ONLY	não	None
tipo	str | None	KEYWORD_ONLY	não	None

Retorno declarado

SnapshotInspecao

`coral.inspecionar_dict`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
inspecionar_dict(objeto: 'Any', **opcoes: 'Any') -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
opcoes	Any	VAR_KEYWORD	não	sem padrão

Retorno declarado

dict[str, Any]

`coral.pyproject_declara_versao_corrente`

Aceita metadado literal histórico ou versão dinâmica derivada do runtime.

Assinatura

Assinatura para consulta

```
pyproject_declara_versao_corrente(texto: 'str') -> 'bool'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
texto	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

bool

`coral.relatorio_inspecao`

Gera uma apresentação humana sem criar um segundo modelo de estado.

Assinatura

Assinatura para consulta

```
relatorio_inspecao(objeto: 'Any', *, visao: 'str | None' = None, caminhos: 'list[str] |
  tuple[str, ...]' = ()) -> 'RelatorioInspecao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
visao	str | None	KEYWORD_ONLY	não	None
caminhos	list[str] | tuple[str, ...]	KEYWORD_ONLY	não	()

Retorno declarado

RelatorioInspecao

`coral.selecionar_inspecao`

Retorna somente os caminhos solicitados, preservando a ordem pedida.

Assinatura

Assinatura para consulta

```
selecionar_inspecao(objeto: 'Any', caminhos: 'list[str] | tuple[str, ...]') ->
  'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
caminhos	list[str] | tuple[str, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

dict[str, Any]

`coral.traduzir_arquivo`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
traduzir_arquivo(caminho)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
caminho	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.traduzir_codigo`

Compila Coral 1.0 para Python usando somente a arquitetura estrutural.

Assinatura

Assinatura para consulta

```
traduzir_codigo(codigo: 'str', nome_fonte: 'str' = '<memoria>') -> 'str'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
codigo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome_fonte	str	POSITIONAL_OR_KEYWORD	não	'<memoria>'

Retorno declarado

str

`coral.traduzir_expressao`

Traduz uma expressão exclusivamente pelo parser e AST Coral 1.0.

Assinatura

Assinatura para consulta

```
traduzir_expressao(expr: 'str') -> 'str'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
expr	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

str

`coral.versao_anterior`

Deriva a release imediatamente anterior.

Patches são puramente aritméticos. Uma release `` `X.Y.0` `` exige uma âncora explícita porque semver, sozinho, não informa qual foi o último patch da linha anterior.

Assinatura

Assinatura para consulta

```
versao_anterior(atual: 'str | None' = None, *, ancora_linha_anterior: 'str | None' =
  None) -> 'str'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
atual	str | None	POSITIONAL_OR_KEYWORD	não	None
ancora_linha_anterior	str | None	KEYWORD_ONLY	não	None

Retorno declarado

str

`coral.versao_corrente`

Retorna a única identidade corrente publicada pelo runtime sem ciclo de import.

Assinatura

Assinatura para consulta

```
versao_corrente() -> 'str'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

str

`coral.visoes_inspecao`

Lista as visões semânticas disponíveis sem importar módulos de domínio.

Assinatura

Assinatura para consulta

```
visoes_inspecao(objeto: 'Any') -> 'tuple[str, ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

tuple[str, ...]

`coral.visualizar_inspecao`

Produz uma visão de alto nível sobre um snapshot já normalizado.

A função trabalha somente com o contrato `` `coral.inspecao/1` ``. Nenhum módulo de domínio é importado aqui, preservando a direção das dependências.

Assinatura

Assinatura para consulta

```
visualizar_inspecao(objeto: 'Any', visao: 'str') -> 'VisaoInspecao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
visao	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

VisaoInspecao

Biblioteca `coral.aleatorio`

`coral.aleatorio.FonteAleatoria`

Fonte reproduzível quando criada com semente explícita.

Não deve ser usada para senhas, chaves, tokens ou outros segredos.

Assinatura

Assinatura para consulta

```
FonteAleatoria(semente: 'Any' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
semente	Any	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `semente`: `Any`.

Métodos públicos

Referência para leitura

```
amostra(self, valores, quantidade)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
decimal(self, minimo=0.0, maximo=1.0)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
embaralhar(self, valores)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
escolha_ponderada(self, valores, pesos)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
escolher(self, valores)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
inteiro(self, minimo, maximo)
```

Método público sem descrição própria no runtime.

`amostra(...)` · módulo `coral.aleatorio`

O que faz

Escolhe uma quantidade de posições distintas sem reposição.

Assinatura

Assinatura para consulta

```
amostra(valores, quantidade, *, fonte: 'FonteAleatoria | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável	sim	sem padrão	População materializada em lista; valores repetidos continuam possíveis.
quantidade	inteiro	sim	sem padrão	Quantidade entre zero e o tamanho da população.
fonte	FonteAleatoria ou nulo	não	None	Fonte independente explícita; nulo usa o gerador global de random do Python.

Retorno

Lista com os itens selecionados.

Exemplo

Código Coral

```
de coral.aleatorio importe amostra
de coral.aleatorio importe FonteAleatoria
mostre amostra(["Coral"], 1, fonte=FonteAleatoria(7))
```

Resultado

Saída esperada

```
['Coral']
```

Erros

ValueError para quantidade negativa ou maior que a população; TypeError para tipos incompatíveis.

Atenção

Avança o estado da fonte. Não use para senhas, tokens ou chaves.

`decimal(...)` · módulo `coral.aleatorio`

O que faz

Sorteia um número decimal usando distribuição uniforme.

Assinatura

Assinatura para consulta

```
decimal(minimo=0.0, maximo=1.0, *, fonte: 'FonteAleatoria | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
minimo	número real	não	0.0	Primeiro extremo do intervalo.
maximo	número real	não	1.0	Segundo extremo do intervalo.
fonte	FonteAleatoria ou nulo	não	None	Fonte independente explícita; nulo usa o gerador global de random do Python.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.aleatorio importe decimal
de coral.aleatorio importe FonteAleatoria
mostre decimal(2, 2, fonte=FonteAleatoria(7))
```

Resultado

Saída esperada

```
2.0
```

Erros

TypeError para extremos incompatíveis.

Atenção

Avança o estado da fonte. Não use para senhas, tokens ou chaves. Extremos invertidos são aceitos; por arredondamento, o extremo superior pode ocorrer.

`embaralhar(...)` · módulo `coral.aleatorio`

O que faz

Embaralha uma cópia dos itens.

Assinatura

Assinatura para consulta

```
embaralhar(valores, *, fonte: 'FonteAleatoria | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável	sim	sem padrão	Itens que serão copiados para a lista embaralhada.
fonte	FonteAleatoria ou nulo	não	None	Fonte independente explícita; nulo usa o gerador global de random do Python.

Retorno

Lista nova, em ordem sorteada.

Exemplo

Código Coral

```
de coral.aleatorio importe embaralhar
de coral.aleatorio importe FonteAleatoria
mostre embaralhar(["Coral"], fonte=FonteAleatoria(7))
```

Resultado

Saída esperada

```
['Coral']
```

Erros

TypeError para valor não iterável.

Atenção

Avança o estado da fonte. Não use para senhas, tokens ou chaves. A coleção original é preservada.

`escolha_ponderada(...)` · módulo `coral.aleatorio`

O que faz

Escolhe um item com probabilidade proporcional ao peso.

Assinatura

Assinatura para consulta

```
escolha_ponderada(valores, pesos, *, fonte: 'FonteAleatoria | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável não vazio	sim	sem padrão	Itens disponíveis.
pesos	iterável de pesos numéricos	sim	sem padrão	Peso de cada item; mesmos tamanhos, pesos não negativos e soma positiva.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
fonte	FonteAleatoria ou nulo	não	None	Fonte independente explícita; nulo usa o gerador global de random do Python.

Retorno

Elemento selecionado, com seu tipo original.

Exemplo

Código Coral

```
de coral.aleatorio importe escolha_ponderada
de coral.aleatorio importe FonteAleatoria
mostre escolha_ponderada(["Coral"], [1], fonte=FonteAleatoria(7))
```

Resultado

Saída esperada

```
Coral
```

Erros

ValueError para tamanhos incompatíveis, população vazia, pesos negativos ou soma não positiva; erros de random.choices para peso total não finito; TypeError para pesos incompatíveis.

Atenção

Avança o estado da fonte. Não use para senhas, tokens ou chaves.

`escolher(...)` · módulo `coral.aleatorio`

O que faz

Escolhe um elemento de um iterável materializado em lista.

Assinatura

Assinatura para consulta

```
escolher(valores, *, fonte: 'FonteAleatoria | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável não vazio	sim	sem padrão	Elementos disponíveis para escolha.
fonte	FonteAleatoria ou nulo	não	None	Fonte independente explícita; nulo usa o gerador global de random do Python.

Retorno

Elemento selecionado, com seu tipo original.

Exemplo

Código Coral

```
de coral.aleatorio importe escolher
de coral.aleatorio importe FonteAleatoria
mostre escolher(["Coral"], fonte=FonteAleatoria(7))
```

Resultado

Saída esperada

```
Coral
```

Erros

IndexError para coleção vazia; TypeError para valor não iterável.

Atenção

Avança o estado da fonte. Não use para senhas, tokens ou chaves.

`fonte(...)` · módulo `coral.aleatorio`

O que faz

Cria uma fonte pseudoaleatória independente, com semente opcional.

Assinatura

Assinatura para consulta

```
fonte(semente=None) -> 'FonteAleatoria'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
semente	nulo, inteiro, decimal, texto, bytes ou bytearray	não	None	Semente aceita por random.Random; nulo usa a fonte padrão de entropia.

Retorno

FonteAleatoria.

Exemplo

Código Coral

```
de coral.aleatorio importe fonte
defina origem como fonte(7)
mostre origem.inteiro(5, 5)
```

Resultado

Saída esperada

```
5
```

Erros

TypeError para tipo de semente não suportado.

Atenção

A fonte mantém estado e avança a cada sorteio. Uma semente ajuda a repetir experimentos; não produz segredos seguros nem garante resultados iguais entre todas as versões do Python.

`inteiro(...)` · módulo `coral.aleatorio`

O que faz

Sorteia um inteiro no intervalo fechado.

Assinatura

Assinatura para consulta

```
inteiro(minimo, maximo, *, fonte: 'FonteAleatoria | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
minimo	inteiro	sim	sem padrão	Limite inferior incluído.
maximo	inteiro	sim	sem padrão	Limite superior incluído.
fonte	FonteAleatoria ou nulo	não	None	Fonte independente explícita; nulo usa o gerador global de random do Python.

Retorno

Inteiro.

Exemplo

Código Coral

```
de coral.aleatorio importe inteiro
de coral.aleatorio importe FonteAleatoria
mostre inteiro(5, 5, fonte=FonteAleatoria(7))
```

Resultado

Saída esperada

```
5
```

Erros

ValueError para intervalo vazio; TypeError para limites incompatíveis.

Atenção

Avança o estado da fonte. Não use para senhas, tokens ou chaves.

Consultar a implementação canônica de coral.aleatorio

Implementação canônica de `coral.aleatorio`

Código Python

```
"""Superfície pública de coral.aleatorio."""
from .stdlib.aleatorio import *  # noqa: F401,F403

from __future__ import annotations

import random
from dataclasses import dataclass, field
from typing import Any, Iterable, Sequence


@dataclass
class FonteAleatoria:
    """Fonte reproduzível quando criada com semente explícita.

    Não deve ser usada para senhas, chaves, tokens ou outros segredos.
    """

    semente: Any = None
    _gerador: random.Random = field(init=False, repr=False)

    def __post_init__(self):
        self._gerador = random.Random(self.semente)

    def inteiro(self, minimo, maximo):
        return self._gerador.randint(minimo, maximo)

    def decimal(self, minimo=0.0, maximo=1.0):
        return self._gerador.uniform(minimo, maximo)

    def escolher(self, valores):
        return self._gerador.choice(list(valores))

    def amostra(self, valores, quantidade):
        return self._gerador.sample(list(valores), quantidade)

    def escolha_ponderada(self, valores, pesos):
        valores = list(valores)
        pesos = list(pesos)
        if len(valores) != len(pesos) or not valores:
            raise ValueError("valores e pesos precisam ter o mesmo tamanho não vazio")
        if any(float(p) < 0 for p in pesos) or sum(map(float, pesos)) <= 0:
            raise ValueError("pesos precisam ser não negativos e ter soma positiva")
        return self._gerador.choices(valores, weights=pesos, k=1)[0]

    def embaralhar(self, valores):
        copia = list(valores)
        self._gerador.shuffle(copia)
        return copia


def fonte(semente=None) -> FonteAleatoria:
    return FonteAleatoria(semente)


def _g(fonte_aleatoria: FonteAleatoria | None):
    return fonte_aleatoria if fonte_aleatoria is not None else None


def inteiro(minimo, maximo, *, fonte: FonteAleatoria | None = None):
```

```
    return fonte.inteiro(minimo, maximo) if fonte is not None else
      random.randint(minimo, maximo)


def decimal(minimo=0.0, maximo=1.0, *, fonte: FonteAleatoria | None = None):
    return fonte.decimal(minimo, maximo) if fonte is not None else
      random.uniform(minimo, maximo)


def escolher(valores, *, fonte: FonteAleatoria | None = None):
    return fonte.escolher(valores) if fonte is not None else
      random.choice(list(valores))


def amostra(valores, quantidade, *, fonte: FonteAleatoria | None = None):
    return fonte.amostra(valores, quantidade) if fonte is not None else
      random.sample(list(valores), quantidade)


def escolha_ponderada(valores, pesos, *, fonte: FonteAleatoria | None = None):
    if fonte is not None:
        return fonte.escolha_ponderada(valores, pesos)
    valores = list(valores); pesos = list(pesos)
    if len(valores) != len(pesos) or not valores:
        raise ValueError("valores e pesos precisam ter o mesmo tamanho não vazio")
    if any(float(p) < 0 for p in pesos) or sum(map(float, pesos)) <= 0:
        raise ValueError("pesos precisam ser não negativos e ter soma positiva")
    return random.choices(valores, weights=pesos, k=1)[0]


def embaralhar(valores, *, fonte: FonteAleatoria | None = None):
    if fonte is not None:
        return fonte.embaralhar(valores)
    copia = list(valores)
    random.shuffle(copia)
    return copia


__all__ = [
    "FonteAleatoria", "fonte", "inteiro", "decimal", "escolher", "amostra",
    "escolha_ponderada", "embaralhar",
]
```

Biblioteca `coral.api`

`coral.api.API_LATEST_VERSION`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'1.1'`

`coral.api.API_VERSION`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'1.0'`

`coral.api.API_VERSIONS`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`('1.0', '1.1')`

`coral.api.DiagnosticoEstruturado`

DiagnosticoEstruturado(codigo: 'str', nivel: 'str', mensagem: 'str', linha: 'int | None' = None, coluna: 'int | None' = None, sugestao: 'str | None' = None, origem: 'str' = 'coral', trecho: 'str | None' = None, categoria: 'str' = 'codigo', detalhe: 'Mapping[str, Any]' = <factory>)

Assinatura

Assinatura para consulta

```
DiagnosticoEstruturado(codigo: 'str', nivel: 'str', mensagem: 'str', linha: 'int | None'
   = None, coluna: 'int | None' = None, sugestao: 'str | None' = None, origem: 'str' =
  'coral', trecho: 'str | None' = None, categoria: 'str' = 'codigo', detalhe:
  'Mapping[str, Any]' = <factory>) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
codigo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
nivel	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
linha	int | None	POSITIONAL_OR_KEYWORD	não	None
coluna	int | None	POSITIONAL_OR_KEYWORD	não	None
sugestao	str | None	POSITIONAL_OR_KEYWORD	não	None
origem	str	POSITIONAL_OR_KEYWORD	não	'coral'
trecho	str | None	POSITIONAL_OR_KEYWORD	não	None

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
categoria	str	POSITIONAL_OR_KEYWORD	não	'codigo'
detalhe	Mapping[str, Any]	POSITIONAL_OR_KEYWORD	não	<factory>

Retorno declarado

None

Atributo `codigo`: `str`.

Atributo `nivel`: `str`.

Atributo `mensagem`: `str`.

Atributo `linha`: `int | None`.

Atributo `coluna`: `int | None`.

Atributo `sugestao`: `str | None`.

Atributo `origem`: `str`.

Atributo `trecho`: `str | None`.

Atributo `categoria`: `str`.

Atributo `detalhe`: `Mapping[str, Any]`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.api.ErroCoral`

Raiz pública dos erros controlados pela linguagem e bibliotecas Coral.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.api.ErroDeTraducao`

Erro encontrado durante parsing, análise ou geração de código.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.api.ResultadoAnalise`

ResultadoAnalise(diagnosticos: 'tuple[DiagnosticoEstruturado, ...]')

Assinatura

Assinatura para consulta

```
ResultadoAnalise(diagnosticos: 'tuple[DiagnosticoEstruturado, ...]') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
diagnosticos	tuple[DiagnosticoEstruturado, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `diagnosticos`: `tuple[DiagnosticoEstruturado, ...]`.

`coral.api.ResultadoCompilacao`

ResultadoCompilacao(codigo: 'str', mapa_linhas: 'dict[int, int]' = <factory>, folhas_legadas: 'int' = 0, blocos_estruturais: 'int' = 0, folhas_genericas: 'int' = 0, blocos_genericos: 'int' = 0)

Assinatura

Assinatura para consulta

```
ResultadoCompilacao(codigo: 'str', mapa_linhas: 'dict[int, int]' = <factory>,
  folhas_legadas: 'int' = 0, blocos_estruturais: 'int' = 0, folhas_genericas: 'int' = 0,
  blocos_genericos: 'int' = 0) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
codigo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
mapa_linhas	dict[int, int]	POSITIONAL_OR_KEYWORD	não	<factory>
folhas_legadas	int	POSITIONAL_OR_KEYWORD	não	0

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
blocos_estruturais	int	POSITIONAL_OR_KEYWORD	não	0
folhas_genericas	int	POSITIONAL_OR_KEYWORD	não	0
blocos_genericos	int	POSITIONAL_OR_KEYWORD	não	0

Retorno declarado

None

Atributo `codigo`: `str`.

Atributo `mapa_linhas`: `dict[int, int]`.

Atributo `folhas_legadas`: `int`.

Atributo `blocos_estruturais`: `int`.

Atributo `folhas_genericas`: `int`.

Atributo `blocos_genericos`: `int`.

`coral.api.SnapshotAnalise`

SnapshotAnalise(fonte: 'str', nome_fonte: 'str', hash_fonte: 'str', ast: 'Programa | None', simbolos: 'TabelaSimbolos | None', interface_modulo: 'InterfaceModuloAnalise', tipos_conhecidos: 'tuple[tuple[str, str], ...]', diagnosticos: 'tuple[DiagnosticoEstruturado, ...]', versao: 'int | str' = 1, versao_snapshot: 'str' = '1')

Assinatura

Assinatura para consulta

```
SnapshotAnalise(fonte: 'str', nome_fonte: 'str', hash_fonte: 'str', ast: 'Programa |
  None', simbolos: 'TabelaSimbolos | None', interface_modulo: 'InterfaceModuloAnalise',
  tipos_conhecidos: 'tuple[tuple[str, str], ...]', diagnosticos:
  'tuple[DiagnosticoEstruturado, ...]', versao: 'int | str' = 1, versao_snapshot: 'str' =
  '1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
fonte	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome_fonte	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
hash_fonte	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
ast	Programa | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
simbolos	TabelaSimbolos | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
interface_modulo	InterfaceModuloAnalise	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipos_conhecidos	tuple[tuple[str, str], ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
diagnosticos	tuple[DiagnosticoEstruturado, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
versao	int | str	POSITIONAL_OR_KEYWORD	não	1
versao_snapshot	str	POSITIONAL_OR_KEYWORD	não	'1'

Retorno declarado

None

Atributo `fonte`: `str`.

Atributo `nome_fonte`: `str`.

Atributo `hash_fonte`: `str`.

Atributo `ast`: `Programa | None`.

Atributo `simbolos`: `TabelaSimbolos | None`.

Atributo `interface_modulo`: `InterfaceModuloAnalise`.

Atributo `tipos_conhecidos`: `tuple[tuple[str, str], ...]`.

Atributo `diagnosticos`: `tuple[DiagnosticoEstruturado, ...]`.

Atributo `versao`: `int | str`.

Atributo `versao_snapshot`: `str`.

`coral.api.analisar`

API 1.0: mantém o resultado histórico, agora derivado de SnapshotAnalise.

Assinatura

Assinatura para consulta

```
analisar(codigo: 'str', nome_fonte: 'str' = '<api>') -> 'ResultadoAnalise'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
codigo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome_fonte	str	POSITIONAL_OR_KEYWORD	não	'<api>'

Retorno declarado

ResultadoAnalise

`coral.api.analisar_snapshot`

API 1.1: cria uma fotografia reutilizável de análise em um único parse.

Assinatura

Assinatura para consulta

```
analisar_snapshot(codigo: 'str', nome_fonte: 'str' = '<api>', *, versao: 'int | str' =
  1) -> 'SnapshotAnalise'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
codigo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome_fonte	str	POSITIONAL_OR_KEYWORD	não	'<api>'
versao	int | str	KEYWORD_ONLY	não	1

Retorno declarado

SnapshotAnalise

`coral.api.compilar`

Compila Coral para Python e preserva mapa de linhas e metadados.

Assinatura

Assinatura para consulta

```
compilar(codigo: 'str', nome_fonte: 'str' = '<api>') -> 'ResultadoCompilacao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
codigo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome_fonte	str	POSITIONAL_OR_KEYWORD	não	'<api>'

Retorno declarado

ResultadoCompilacao

`coral.api.compilar_de_snapshot`

API 1.1: compila a AST do snapshot sem executar um novo parse.

Assinatura

Assinatura para consulta

```
compilar_de_snapshot(snapshot: 'SnapshotAnalise') -> 'ResultadoCompilacao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
snapshot	SnapshotAnalise	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

ResultadoCompilacao

`coral.api.formatar`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
formatar(codigo: 'str') -> 'str'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
codigo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

str

`coral.api.info_api`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
info_api() -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

dict[str, Any]

`coral.api.parse_expressao`

Analisa uma expressão Coral e retorna sua AST com localização.

Assinatura

Assinatura para consulta

```
parse_expressao(texto: 'str')
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
texto	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.api.parse_programa`

Analisa um programa Coral e retorna sua AST estrutural.

Assinatura

Assinatura para consulta

```
parse_programa(codigo: 'str') -> 'Programa'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
codigo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Programa

Biblioteca `coral.arquivos`

`adicionar_texto(...)` · **módulo** `coral.arquivos`

O que faz

Acrescenta texto ao fim de um arquivo, criando o arquivo se necessário.

Assinatura

Assinatura para consulta

```
adicionar_texto(caminho, texto, codificacao='utf-8')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.
texto	qualquer valor	sim	sem padrão	Conteúdo convertido para texto com str.
codificacao	texto	não	'utf-8'	Codificação usada para gravar, por padrão UTF-8.

Retorno

nulo.

Exemplo

Código Coral

```
de coral.arquivos importe adicionar_texto
execute adicionar_texto("dados.txt", "Coral")
de coral.arquivos importe ler_texto
mostre ler_texto("dados.txt")
```

Resultado

Saída esperada

```
Coral
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento.

Atenção

Escreve no arquivo. Não acrescenta uma quebra de linha automaticamente e não cria pastas ausentes.

`arquivo_temporario(...)` · módulo `coral.arquivos`

O que faz

Cria um arquivo vazio de nome exclusivo e fecha seu descritor.

Assinatura

Assinatura para consulta

```
arquivo_temporario(*, prefixo='coral_', sufixo='', pasta=None) -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
prefixo	texto	não	'coral_'	Prefixo do nome gerado.
sufixo	texto	não	''	Sufixo do nome gerado.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
pasta	caminho nativo ou nulo	não	None	Pasta onde criar; nulo usa a pasta temporária padrão do sistema.

Retorno

Path do arquivo criado.

Exemplo

Código Coral

```
de coral.arquivos importe arquivo_temporario
defina alvo como arquivo_temporario(pasta=".")
mostre alvo.exists()
de coral.arquivos importe remover
execute remover(alvo)
```

Resultado

Saída esperada

```
True
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento.

Atenção

Cria um recurso persistente; o chamador deve removê-lo quando terminar. O nome aleatório não deve ser considerado previsível.

`copiar(...)` · módulo `coral.arquivos`

O que faz

Copia um arquivo preservando metadados, ou uma árvore de pastas quando autorizado.

Assinatura

Assinatura para consulta

```
copiar(caminho, destino, *, recursivo: 'bool' = False) -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.
destino	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho de destino da operação; ~ é expandido.
recursivo	booleano	não	False	Obrigatório como verdadeiro para copiar pasta inteira.

Retorno

Path do destino resultante.

Exemplo

Código Coral

```
de coral.arquivos importe copiar
de coral.arquivos importe escrever_texto, ler_texto
execute escrever_texto("origem.txt", "Coral")
execute copiar("origem.txt", "destino.txt")
mostre ler_texto("destino.txt")
```

Resultado

Saída esperada

```
Coral
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento. IsADirectoryError ao copiar pasta sem recursivo; FileExistsError se o destino de copytree já existe.

Atenção

Pode gravar no destino. As regras de substituição variam com a operação e o sistema operacional; confira antes de usar um destino existente.

`criar_pasta(...)` · módulo `coral.arquivos`

O que faz

Cria uma pasta e suas pastas ancestrais ausentes.

Assinatura

Assinatura para consulta

```
criar_pasta(caminho)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.

Retorno

nulo.

Exemplo

Código Coral

```
de coral.arquivos importe criar_pasta
execute criar_pasta("dados/sub")
de coral.arquivos importe existe
mostre existe("dados/sub")
```

Resultado

Saída esperada

```
True
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento.

Atenção

Aceita pasta já existente. Não substitui um arquivo que já ocupa o mesmo caminho.

`escrever_bytes(...)` · módulo `coral.arquivos`

O que faz

Grava conteúdo binário, criando ou substituindo o arquivo.

Assinatura

Assinatura para consulta

```
escrever_bytes(caminho, dados) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.
dados	bytes, bytearray ou memoryview	sim	sem padrão	Conteúdo binário; texto precisa ser codificado antes.

Retorno

nulo.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_bytes
execute escrever_bytes("dados.bin", "AB".encode())
de coral.arquivos importe tamanho
mostre tamanho("dados.bin")
```

Resultado

Saída esperada

```
2
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento. TypeError se dados não é bytes, bytearray ou memoryview.

Atenção

Lê ou escreve todos os bytes de uma vez. A escrita substitui conteúdo existente e não cria pastas.

`escrever_csv(...)` · **módulo** `coral.arquivos`

O que faz

Nulo; o destino recebe CSV com ordem de colunas explícita ou inferida. Processamento gradual sem materializar a entrada inteira.

Assinatura

Assinatura para consulta

```
escrever_csv(caminho, registros: 'Iterable[Mapping[str, str] | Sequence[str]]',
  codificacao='utf-8', *, delimitador=',', colunas=None, cabecalho=True) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou caminho de arquivo	sim	sem padrão	Caminho do arquivo, relativo à pasta de trabalho ou absoluto.
registros	iterável de objetos ou sequências textuais	sim	sem padrão	Iterável de objetos texto/texto ou sequências de textos de largura uniforme.
codificacao	texto	não	'utf-8'	Codificação textual usada na abertura; padrão utf-8.
delimitador	texto de um caractere	não	','	Caractere separador CSV, inclusive entre campos; padrão vírgula.
colunas	sequência de textos ou nulo	não	None	Sequência ordenada de nomes; nulo infere nomes do primeiro objeto.
cabecalho	booleano	não	True	Se verdadeiro, lê ou escreve a linha de nomes das colunas.

Retorno

Nulo; o destino recebe CSV com ordem de colunas explícita ou inferida.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_csv e ler_csv
execute escrever_csv("saida.csv", [{"nome": "Ana", "valor": "10"}], colunas=["nome",
  "valor"])
com ler_csv("saida.csv") como registros faça
    para cada registro em registros faça
        mostre registro["nome"]
        mostre registro["valor"]
    fim
fim
```

Resultado

Saída esperada

```
Ana
10
```

Erros

Erros de caminho, permissão ou codificação; TypeError para tipos inválidos e ValueError para opções/dados fora do contrato. CSV malformado é rejeitado.

Atenção

Escrita substitui o destino e pode ficar parcial em erro; campos precisam ser textos. O chamador fecha leitores de entrada com com externo. Objetos CSV exigem chaves exatas e sequências largura exata. Para sequências de campos, nomes não são inferíveis: sem colunas explícitas, não é emitido cabeçalho mesmo com cabecalho=True. Objetos permitem inferir nomes na ordem do primeiro registro; colunas explícitas definem nomes e ordem.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md.

`escrever_linhas(...)` · módulo `coral.arquivos`

O que faz

Nulo; o destino recebe os textos sem quebras inseridas. Processamento gradual sem materializar a entrada inteira.

Assinatura

Assinatura para consulta

```
escrever_linhas(caminho, linhas: 'Iterable[str]', codificacao='utf-8') -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou caminho de arquivo	sim	sem padrão	Caminho do arquivo, relativo à pasta de trabalho ou absoluto.
linhas	iterável de textos	sim	sem padrão	Iterável de textos com as terminações desejadas; sem conversão implícita.
codificacao	texto	não	'utf-8'	Codificação textual usada na abertura; padrão utf-8.

Retorno

Nulo; o destino recebe os textos sem quebras inseridas.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_linhas e ler_texto
execute escrever_linhas("saida.txt", ["Ana\n", "Bia"])
mostre ler_texto("saida.txt")
```

Resultado

Saída esperada

```
Ana
Bia
```

Erros

Erros de caminho, permissão ou codificação; TypeError para tipos inválidos e ValueError para opções/dados fora do contrato. CSV malformado é rejeitado.

Atenção

Escrita substitui o destino e pode ficar parcial em erro; campos precisam ser textos. O chamador fecha leitores de entrada com com externo. Objetos CSV exigem chaves exatas e sequências largura exata.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md.

`escrever_texto(...)` · módulo `coral.arquivos`

O que faz

Escreve texto no arquivo escolhido.

Assinatura

Assinatura para consulta

```
escrever_texto(caminho, texto, codificacao='utf-8')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho do arquivo local, interpretado na pasta de trabalho.
texto	qualquer valor convertido com str	sim	sem padrão	Texto a consultar ou transformar.
codificacao	texto de codificação	não	'utf-8'	Codificação de texto usada na leitura ou escrita.

Retorno

Nulo.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_texto
mostre escrever_texto("dados.txt", "Coral")
```

Resultado

Saída esperada

```
None
```

Forma natural

`escreva "Coral" em "dados.txt"` devolve nulo; o caminho é avaliado antes do texto.

Erros

PermissionError para acesso recusado; erros de caminho ou codificação quando a operação não é possível.

Atenção

Cria ou substitui o arquivo, com efeito no sistema de arquivos. Confira o caminho antes de escrever.

`existe(...)` · módulo `coral.arquivos`

O que faz

Verifica se o caminho existe, usando a normalização oficial.

Assinatura

Assinatura para consulta

```
existe(caminho)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho do arquivo local, interpretado na pasta de trabalho.

Retorno

Booleano.

Exemplo

Código Coral

```
de coral.arquivos importe existe
de coral.arquivos importe escrever_texto
execute escrever_texto("dados.txt", "Coral")
mostre existe("dados.txt")
```

Resultado

Saída esperada

```
True
```

Forma natural

`existe o caminho "dados.txt"` é equivalente.

Erros

A leitura pode produzir FileNotFoundError ou PermissionError. Consulte o Cap. 18.

Atenção

O exemplo cria seu próprio arquivo na pasta de trabalho.

`extensao(...)` · módulo `coral.arquivos`

O que faz

Obtém o último sufixo do nome.

Assinatura

Assinatura para consulta

```
extensao(caminho) -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.

Retorno

Texto; vazio quando não há sufixo.

Exemplo

Código Coral

```
de coral.arquivos importe extensao
mostre extensao("dados/arquivo.tar.gz")
```

Resultado

Saída esperada

```
.gz
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento.

Atenção

Operação lexical: não exige que o arquivo exista.

`ler_bytes(...)` · módulo `coral.arquivos`

O que faz

Lê o conteúdo binário integral de um arquivo.

Assinatura

Assinatura para consulta

```
ler_bytes(caminho) -> 'bytes'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.

Retorno

bytes.

Exemplo

Código Coral

```
de coral.arquivos importe ler_bytes
de coral.arquivos importe escrever_texto
execute escrever_texto("dados.txt", "AB")
mostre ler_bytes("dados.txt").hex()
```

Resultado

Saída esperada

```
4142
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento.

Atenção

Lê ou escreve todos os bytes de uma vez. A escrita substitui conteúdo existente e não cria pastas.

`ler_csv(...)` · módulo `coral.arquivos`

O que faz

Iterador de objetos com chaves e valores textuais se cabecalho=True; listas de textos caso contrário. Processamento gradual sem materializar a entrada inteira.

Assinatura

Assinatura para consulta

```
ler_csv(caminho, codificacao='utf-8', *, delimitador=',', cabecalho=True) ->
  'Iterator[dict[str, str] | list[str]]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou caminho de arquivo	sim	sem padrão	Caminho do arquivo, relativo à pasta de trabalho ou absoluto.
codificacao	texto	não	'utf-8'	Codificação textual usada na abertura; padrão utf-8.
delimitador	texto de um caractere	não	','	Caractere separador CSV, inclusive entre campos; padrão vírgula.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
cabecalho	booleano	não	True	Se verdadeiro, lê ou escreve a linha de nomes das colunas.

Retorno

Iterador de objetos com chaves e valores textuais se cabecalho=True; listas de textos caso contrário.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_texto e ler_csv
execute escrever_texto("dados.csv", "nome,valor\nAna,10\n")
com ler_csv("dados.csv") como registros faça
    para cada registro em registros faça
        mostre registro["nome"]
    fim
fim
```

Resultado

Saída esperada

```
Ana
```

Erros

Erros de caminho, permissão ou codificação; TypeError para tipos inválidos e ValueError para opções/dados fora do contrato. CSV malformado é rejeitado.

Atenção

Leitura preguiçosa, passagem única; EOF e erro fecham. Use com ou fechar()/close() ao interromper antes. CSV exige cabeçalhos únicos não vazios e largura coerente; aceita aspas, separadores e campos multilinha.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md.

`ler_linhas(...)` · módulo `coral.arquivos`

O que faz

Iterador de textos, preservando terminações de linha. Processamento gradual sem materializar a entrada inteira.

Assinatura

Assinatura para consulta

```
ler_linhas(caminho, codificacao='utf-8') -> 'Iterator[str]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou caminho de arquivo	sim	sem padrão	Caminho do arquivo, relativo à pasta de trabalho ou absoluto.
codificacao	texto	não	'utf-8'	Codificação textual usada na abertura; padrão utf-8.

Retorno

Iterador de textos, preservando terminações de linha.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_texto e ler_linhas
execute escrever_texto("linhas.txt", "Ana\nBia")
com ler_linhas("linhas.txt") como linhas faça
    para cada linha em linhas faça
        mostre tamanho de linha
    fim
fim
```

Resultado

Saída esperada

```
4
3
```

Erros

Erros de caminho, permissão ou codificação; TypeError para tipos inválidos e ValueError para opções/dados fora do contrato. CSV malformado é rejeitado.

Atenção

Leitura preguiçosa, passagem única; EOF e erro fecham. Use com ou fechar()/close() ao interromper antes. CSV exige cabeçalhos únicos não vazios e largura coerente; aceita aspas, separadores e campos multilinha.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md.

`ler_lotes_csv(...)` · módulo `coral.arquivos`

O que faz

Iterador de listas de registros; lote final possivelmente menor. Processamento gradual sem materializar a entrada inteira.

Assinatura

Assinatura para consulta

```
ler_lotes_csv(caminho, tamanho_lote=100, codificacao='utf-8', *, delimitador=',',
  cabecalho=True) -> 'Iterator[list[dict[str, str] | list[str]]]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou caminho de arquivo	sim	sem padrão	Caminho do arquivo, relativo à pasta de trabalho ou absoluto.
tamanho_lote	inteiro positivo, não booleano	não	100	Máximo de registros por lote: inteiro positivo, exceto booleano.
codificacao	texto	não	'utf-8'	Codificação textual usada na abertura; padrão utf-8.
delimitador	texto de um caractere	não	','	Caractere separador CSV, inclusive entre campos; padrão vírgula.
cabecalho	booleano	não	True	Se verdadeiro, lê ou escreve a linha de nomes das colunas.

Retorno

Iterador de listas de registros; lote final possivelmente menor.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_texto e ler_lotes_csv
execute escrever_texto("dados.csv", "nome,valor\nAna,10\nBia,20\nCaio,30\n")
com ler_lotes_csv("dados.csv", tamanho_lote=2) como lotes faça
    para cada lote em lotes faça
        mostre tamanho de lote
    fim
fim
```

Resultado

Saída esperada

```
2
1
```

Erros

Erros de caminho, permissão ou codificação; TypeError para tipos inválidos e ValueError para opções/dados fora do contrato. CSV malformado é rejeitado.

Atenção

Leitura preguiçosa, passagem única; EOF e erro fecham. Use com ou fechar()/close() ao interromper antes. CSV exige cabeçalhos únicos não vazios e largura coerente; aceita aspas, separadores e campos multilinha.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md.

`ler_texto(...)` · módulo `coral.arquivos`

O que faz

Lê todo o texto de um arquivo.

Assinatura

Assinatura para consulta

```
ler_texto(caminho, codificacao='utf-8')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho do arquivo local, interpretado na pasta de trabalho.
codificacao	texto de codificação	não	'utf-8'	Codificação de texto usada na leitura ou escrita.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.arquivos importe ler_texto
de coral.arquivos importe escrever_texto
execute escrever_texto("dados.txt", "Coral")
mostre ler_texto("dados.txt")
```

Resultado

Saída esperada

```
Coral
```

Forma natural

`leia o texto de "dados.txt"` é equivalente.

Erros

A leitura pode produzir FileNotFoundError ou PermissionError. Consulte o Cap. 18.

Atenção

O exemplo cria seu próprio arquivo na pasta de trabalho.

`listar(...)` · módulo `coral.arquivos`

O que faz

Lista os filhos imediatos de uma pasta.

Assinatura

Assinatura para consulta

```
listar(caminho='.')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	não	'.'	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.

Retorno

Lista de textos com os caminhos dos filhos.

Exemplo

Código Coral

```
de coral.arquivos importe listar
de coral.arquivos importe criar_pasta, escrever_texto
execute criar_pasta("dados")
execute escrever_texto("dados/a.txt", "a")
mostre listar("dados")
```

Resultado

Saída esperada

```
['dados/a.txt']
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento.

Atenção

A ordem é a do sistema de arquivos; use sorted para uma ordem estável. Não percorre subpastas.

`listar_recursivo(...)` · módulo `coral.arquivos`

O que faz

Percorre arquivos e, opcionalmente, pastas a partir de uma raiz.

Assinatura

Assinatura para consulta

```
listar_recursivo(caminho='.', *, profundidade: 'int | None' = None, incluir_pastas:
  'bool' = False)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	não	'.'	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.
profundidade	inteiro não negativo ou nulo	não	None	Limite de descida; zero inclui só arquivos da raiz, nulo não limita a profundidade.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
incluir_pastas	booleano	não	False	Inclui nomes de subpastas que permanecem na travessia.

Retorno

Lista de textos com caminhos.

Exemplo

Código Coral

```
de coral.arquivos importe listar_recursivo
de coral.arquivos importe criar_pasta, escrever_texto
execute criar_pasta("dados/sub")
execute escrever_texto("dados/a.txt", "a")
execute escrever_texto("dados/sub/b.txt", "b")
mostre listar_recursivo("dados", profundidade=0)
```

Resultado

Saída esperada

```
['dados/a.txt']
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento. ValueError para profundidade inválida.

Atenção

Ordena nomes de arquivos e pastas dentro de cada diretório. Não segue links de diretório. os.walk pode ignorar uma raiz inexistente ou erros de acesso e produzir lista vazia; não use como prova de existência.

`metadados(...)` · módulo `coral.arquivos`

O que faz

Consulta dados do caminho e seus atributos no sistema de arquivos.

Assinatura

Assinatura para consulta

```
metadados(caminho) -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.

Retorno

Dicionário com caminho, nome, tamanho, arquivo, pasta, modificado_em e criado_em.

Exemplo

Código Coral

```
de coral.arquivos importe metadados
de coral.arquivos importe escrever_texto
execute escrever_texto("dados.txt", "AB")
defina info como metadados("dados.txt")
mostre info["nome"]
mostre info["tamanho"]
mostre info["arquivo"]
```

Resultado

Saída esperada

```
dados.txt
2
True
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento.

Atenção

Datas são datetime com fuso UTC. criado_em usa st_ctime: no Linux é mudança de metadados, não uma garantia da data de criação. caminho é Path, não texto.

`mover(...)` · módulo `coral.arquivos`

O que faz

Move um arquivo ou pasta; pode copiar e remover a origem entre sistemas de arquivos.

Assinatura

Assinatura para consulta

```
mover(caminho, destino) -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.
destino	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho de destino da operação; ~ é expandido.

Retorno

Path do destino resultante.

Exemplo

Código Coral

```
de coral.arquivos importe mover
de coral.arquivos importe escrever_texto, ler_texto
execute escrever_texto("origem.txt", "Coral")
execute mover("origem.txt", "destino.txt")
mostre ler_texto("destino.txt")
```

Resultado

Saída esperada

```
Coral
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento.

Atenção

Pode gravar no destino. As regras de substituição variam com a operação e o sistema operacional; confira antes de usar um destino existente.

`nome(...)` · módulo `coral.arquivos`

O que faz

Obtém o último componente do caminho.

Assinatura

Assinatura para consulta

```
nome(caminho) -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.arquivos importe nome
mostre nome("dados/arquivo.txt")
```

Resultado

Saída esperada

```
arquivo.txt
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento.

Atenção

Operação lexical: não exige que o arquivo exista.

`pai(...)` · módulo `coral.arquivos`

O que faz

Obtém o caminho da pasta imediatamente superior.

Assinatura

Assinatura para consulta

```
pai(caminho) -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.

Retorno

Path.

Exemplo

Código Coral

```
de coral.arquivos importe pai
mostre pai("dados/arquivo.txt")
```

Resultado

Saída esperada

```
dados
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento.

Atenção

Operação lexical: não exige que o arquivo exista.

`pasta_temporaria(...)` · módulo `coral.arquivos`

O que faz

Cria uma pasta de nome exclusivo.

Assinatura

Assinatura para consulta

```
pasta_temporaria(*, prefixo='coral_', pasta=None) -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
prefixo	texto	não	'coral_'	Prefixo do nome gerado.
pasta	caminho nativo ou nulo	não	None	Pasta onde criar; nulo usa a pasta temporária padrão do sistema.

Retorno

Path da pasta criada.

Exemplo

Código Coral

```
de coral.arquivos importe pasta_temporaria
defina alvo como pasta_temporaria(pasta=".")
mostre alvo.exists()
de coral.arquivos importe remover
execute remover(alvo)
```

Resultado

Saída esperada

```
True
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento.

Atenção

Cria um recurso persistente; o chamador deve removê-lo quando terminar. O nome aleatório não deve ser considerado previsível.

`remover(...)` · módulo `coral.arquivos`

O que faz

Remove um arquivo, link ou pasta.

Assinatura

Assinatura para consulta

```
remover(caminho, *, recursivo: 'bool' = False) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
recursivo	booleano	não	False	Permite remover uma árvore de pastas; sem isso, a pasta precisa estar vazia.

Retorno

nulo.

Exemplo

Código Coral

```
de coral.arquivos importe remover
de coral.arquivos importe escrever_texto, existe
execute escrever_texto("dados.txt", "a")
execute remover("dados.txt")
mostre existe("dados.txt")
```

Resultado

Saída esperada

```
False
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento.

Atenção

A remoção é permanente. Um link simbólico é removido como link, sem apagar sua pasta de destino.

`renomear(...)` · módulo `coral.arquivos`

O que faz

Renomeia um arquivo ou pasta.

Assinatura

Assinatura para consulta

```
renomear(caminho, destino) -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.
destino	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho de destino da operação; ~ é expandido.

Retorno

Path do destino resultante.

Exemplo

Código Coral

```
de coral.arquivos importe renomear
de coral.arquivos importe escrever_texto, ler_texto
execute escrever_texto("origem.txt", "Coral")
execute renomear("origem.txt", "destino.txt")
mostre ler_texto("destino.txt")
```

Resultado

Saída esperada

```
Coral
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento.

Atenção

Pode gravar no destino. As regras de substituição variam com a operação e o sistema operacional; confira antes de usar um destino existente.

`resolver(...)` · módulo `coral.arquivos`

O que faz

Converte o caminho para absoluto e resolve links simbólicos.

Assinatura

Assinatura para consulta

```
resolver(caminho, *, estrito: 'bool' = False) -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.
estrito	booleano	não	False	Se verdadeiro, exige que os componentes existam.

Retorno

Path absoluto.

Exemplo

Código Coral

```
de coral.arquivos importe resolver
mostre resolver("dados.txt").is_absolute()
```

Resultado

Saída esperada

```
True
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento.

Atenção

Com estrito falso, resolve também caminhos que ainda não existem. Não confina o resultado a uma pasta de projeto.

`tamanho(...)` · módulo `coral.arquivos`

O que faz

Obtém st_size do caminho informado.

Assinatura

Assinatura para consulta

```
tamanho(caminho) -> 'int'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local; a biblioteca expande ~. Caminho puro de outro sistema é recusado.

Retorno

Inteiro em bytes.

Exemplo

Código Coral

```
de coral.arquivos importe tamanho
de coral.arquivos importe escrever_texto
execute escrever_texto("dados.txt", "AB")
mostre tamanho("dados.txt")
```

Resultado

Saída esperada

```
2
```

Erros

FileNotFoundError e PermissionError mantêm a classe e recebem mensagem em português. Outros erros do sistema de arquivos preservam seu comportamento.

Atenção

Para pastas, o tamanho de stat não é a soma dos arquivos contidos.

Consultar a implementação canônica de coral.arquivos

Implementação canônica de `coral.arquivos`

Código Python

```
"""Superfície pública de coral.arquivos."""
from .stdlib.arquivos import *  # noqa: F401,F403
from .stdlib.arquivos import __all__

from __future__ import annotations

from datetime import datetime, timezone
from pathlib import Path
import shutil
import tempfile
from typing import Any, Iterable, Iterator, Mapping, Sequence

from coral.caminhos import normalizar_caminho
from . import _arquivos_graduais as _gradual


def _erro_arquivo_em_portugues(exc: OSError, caminho) -> OSError:
    alvo = str(normalizar_caminho(caminho, expandir_usuario=True))
    if isinstance(exc, FileNotFoundError):
        return FileNotFoundError(exc.errno or 2, f"arquivo não encontrado: {alvo}",
          alvo)
    if isinstance(exc, PermissionError):
        return PermissionError(exc.errno or 13, f"permissão negada para o arquivo ou
          pasta: {alvo}", alvo)
    return exc


def _com_erro_traduzido(caminho, operacao):
    try:
        return operacao()
    except (FileNotFoundError, PermissionError) as exc:
        raise _erro_arquivo_em_portugues(exc, caminho) from exc


def _path(caminho) -> Path:
    return normalizar_caminho(caminho, expandir_usuario=True)


def ler_texto(caminho, codificacao="utf-8"):
    path = _path(caminho)
    return _com_erro_traduzido(path, lambda: path.read_text(encoding=codificacao))


def escrever_texto(caminho, texto, codificacao="utf-8"):
    path = _path(caminho)
    _com_erro_traduzido(path, lambda: path.write_text(str(texto), encoding=codificacao))
    return None


def _abrir_gradual(caminho, modo, codificacao):
    path = _path(caminho)
    return _com_erro_traduzido(path, lambda: path.open(modo, encoding=codificacao,
      newline=""))
```

```
def ler_linhas(caminho, codificacao="utf-8") -> Iterator[str]:
    """Lê linhas sob demanda, preservando terminadores. Use com/with ou fechar/close.

    Abre no primeiro avanço e fecha ao terminar ou falhar. É um iterador de
    passagem única; após fechar não reabre. Não há garantia de fechamento por GC.
    """
    return _gradual._Leitor(lambda: _abrir_gradual(caminho, "r", codificacao), iter)


def ler_csv(caminho, codificacao="utf-8", *, delimitador=",", cabecalho=True) ->
  Iterator[dict[str, str] | list[str]]:
    """Lê registros CSV sob demanda, com campos de texto e estrutura estrita.

    Com cabeçalho produz dicionários, sem cabeçalho produz listas. Use com/with
    ou fechar/close se interromper a leitura; abre apenas no primeiro avanço.
    Fecha no fim e em erros, não reabre e não garante fechamento por GC.
    """
    _gradual._opcoes(delimitador, cabecalho)
    return _gradual._Leitor(lambda: _abrir_gradual(caminho, "r", codificacao),
                            lambda arquivo: _gradual._registros(arquivo, delimitador,
                              cabecalho))


def ler_lotes_csv(caminho, tamanho_lote=100, codificacao="utf-8", *, delimitador=",",
  cabecalho=True) -> Iterator[list[dict[str, str] | list[str]]]:
    """Lê lotes CSV de tamanho limitado, incluindo o último lote parcial.

    Recurso de passagem única e abertura preguiçosa. Use com/with ou fechar/close
    em interrupções; fecha no fim e em erros, sem reabrir ou garantia por GC.
    """
    _gradual._opcoes(delimitador, cabecalho)
    if isinstance(tamanho_lote, bool) or not isinstance(tamanho_lote, int) or
      tamanho_lote <= 0:
        raise ValueError("tamanho_lote precisa ser inteiro positivo")
    return _gradual._Leitor(lambda: _abrir_gradual(caminho, "r", codificacao),
                            lambda arquivo: _gradual._lotes(_gradual._registros(arquivo,
                              delimitador, cabecalho), tamanho_lote))


def escrever_linhas(caminho, linhas: Iterable[str], codificacao="utf-8") -> None:
    """Escreve textos gradualmente, sem inserir terminadores ou converter valores.

    Trunca o destino e sempre o fecha. Uma falha pode deixar arquivo parcial.
    """
    with _abrir_gradual(caminho, "w", codificacao) as arquivo:
        for linha in linhas:
            if not isinstance(linha, str):
                raise TypeError("linhas precisam ser texto")
            arquivo.write(linha)


def escrever_csv(caminho, registros: Iterable[Mapping[str, str] | Sequence[str]],
  codificacao="utf-8", *, delimitador=",", colunas=None, cabecalho=True) -> None:
    """Escreve CSV gradualmente com campos estritamente textuais e estrutura exata.

    Infere colunas do primeiro mapeamento ou aceita nomes explícitos. Sequências
    textuais usam sua primeira largura; sem colunas não geram cabeçalho, mesmo
    com cabecalho=True. Trunca e fecha o destino mesmo em erro;
    uma falha pode deixar arquivo parcial. Aspas e novas linhas seguem CSV.
    """
    _gradual._opcoes(delimitador, cabecalho)
```

```
    nomes = _gradual._colunas(colunas) if colunas is not None else None
    with _abrir_gradual(caminho, "w", codificacao) as arquivo:
        _gradual._escrever_csv(arquivo, registros, delimitador, nomes, cabecalho)


def adicionar_texto(caminho, texto, codificacao="utf-8"):
    path = _path(caminho)

    def adicionar():
        with path.open("a", encoding=codificacao) as f:
            return f.write(str(texto))

    _com_erro_traduzido(path, adicionar)
    return None


def ler_bytes(caminho) -> bytes:
    path = _path(caminho)
    return _com_erro_traduzido(path, path.read_bytes)


def escrever_bytes(caminho, dados) -> None:
    path = _path(caminho)
    if not isinstance(dados, (bytes, bytearray, memoryview)):
        raise TypeError("dados precisa ser bytes, bytearray ou memoryview")
    _com_erro_traduzido(path, lambda: path.write_bytes(bytes(dados)))
    return None


def existe(caminho):
    return _path(caminho).exists()


def listar(caminho="."):
    path = _path(caminho)
    return _com_erro_traduzido(path, lambda: [str(p) for p in path.iterdir()])


def listar_recursivo(caminho=".", *, profundidade: int | None = None, incluir_pastas:
  bool = False):
    raiz = _path(caminho)
    if profundidade is not None and (not isinstance(profundidade, int) or profundidade <
       0):
        raise ValueError("profundidade precisa ser inteiro não negativo ou nulo")
    resultado: list[str] = []
    base_nivel = len(raiz.parts)
    try:
        for atual, pastas, arquivos in __import__("os").walk(raiz):
            atual_path = Path(atual)
            nivel = len(atual_path.parts) - base_nivel
            if profundidade is not None and nivel >= profundidade:
                pastas[:] = []
            if incluir_pastas:
                resultado.extend(str(atual_path / nome) for nome in sorted(pastas))
            resultado.extend(str(atual_path / nome) for nome in sorted(arquivos))
    except (FileNotFoundError, PermissionError) as exc:
        raise _erro_arquivo_em_portugues(exc, raiz) from exc
    return resultado


def criar_pasta(caminho):
```

```
    path = _path(caminho)
    return _com_erro_traduzido(path, lambda: path.mkdir(parents=True, exist_ok=True))


def remover(caminho, *, recursivo: bool = False) -> None:
    path = _path(caminho)

    def operar():
        if path.is_dir() and not path.is_symlink():
            if recursivo:
                shutil.rmtree(path)
            else:
                path.rmdir()
        else:
            path.unlink()

    _com_erro_traduzido(path, operar)


def renomear(caminho, destino) -> Path:
    origem = _path(caminho)
    alvo = _path(destino)
    return _com_erro_traduzido(origem, lambda: origem.rename(alvo))


def copiar(caminho, destino, *, recursivo: bool = False) -> Path:
    origem = _path(caminho)
    alvo = _path(destino)

    def operar():
        if origem.is_dir():
            if not recursivo:
                raise IsADirectoryError(f"copiar pasta exige recursivo=True: {origem}")
            return Path(shutil.copytree(origem, alvo))
        return Path(shutil.copy2(origem, alvo))

    return _com_erro_traduzido(origem, operar)


def mover(caminho, destino) -> Path:
    origem = _path(caminho)
    alvo = _path(destino)
    return _com_erro_traduzido(origem, lambda: Path(shutil.move(str(origem),
      str(alvo))))


def tamanho(caminho) -> int:
    path = _path(caminho)
    return _com_erro_traduzido(path, lambda: path.stat().st_size)


def metadados(caminho) -> dict[str, Any]:
    path = _path(caminho)
    st = _com_erro_traduzido(path, path.stat)
    return {
        "caminho": path,
        "nome": path.name,
        "tamanho": st.st_size,
        "arquivo": path.is_file(),
        "pasta": path.is_dir(),
        "modificado_em": datetime.fromtimestamp(st.st_mtime, tz=timezone.utc),
```

```
        "criado_em": datetime.fromtimestamp(st.st_ctime, tz=timezone.utc),
    }


def arquivo_temporario(*, prefixo="coral_", sufixo="", pasta=None) -> Path:
    diretorio = str(_path(pasta)) if pasta is not None else None
    descritor, nome_arquivo = tempfile.mkstemp(prefix=str(prefixo), suffix=str(sufixo),
      dir=diretorio)
    __import__("os").close(descritor)
    return Path(nome_arquivo)


def pasta_temporaria(*, prefixo="coral_", pasta=None) -> Path:
    diretorio = str(_path(pasta)) if pasta is not None else None
    return Path(tempfile.mkdtemp(prefix=str(prefixo), dir=diretorio))


def nome(caminho) -> str:
    return _path(caminho).name


def extensao(caminho) -> str:
    return _path(caminho).suffix


def pai(caminho) -> Path:
    return _path(caminho).parent


def resolver(caminho, *, estrito: bool = False) -> Path:
    return _path(caminho).resolve(strict=bool(estrito))


__all__ = [
    "ler_linhas", "ler_csv", "ler_lotes_csv", "escrever_linhas", "escrever_csv",
    "ler_texto", "escrever_texto", "adicionar_texto", "ler_bytes", "escrever_bytes",
    "existe", "listar", "listar_recursivo", "criar_pasta", "remover", "renomear",
    "copiar", "mover", "tamanho", "metadados", "arquivo_temporario",
    "pasta_temporaria", "nome", "extensao", "pai", "resolver",
]
```

Biblioteca `coral.assincrono`

`coral.assincrono.ErroTempoEsgotado`

Uma operação assíncrona excedeu o limite solicitado.

Assinatura

Assinatura para consulta

```
ErroTempoEsgotado(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.assincrono.aguardar_com_timeout`

Aguarda `esperavel por no máximo segundos`.

O awaitable subjacente é cancelado pela política do `asyncio.wait_for` quando o limite é excedido.

Assinatura

Assinatura para consulta

```
aguardar_com_timeout(esperavel: 'Awaitable[Any]', segundos: 'float')
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
esperavel	Awaitable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
segundos	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.assincrono.cancelar_tarefa`

Solicita cancelamento cooperativo de uma tarefa asyncio.

Retorna `True quando uma solicitação nova foi feita e False` quando a tarefa já havia terminado ou já estava cancelada.

Assinatura

Assinatura para consulta

```
cancelar_tarefa(tarefa: 'Any', motivo: 'str | None' = None) -> 'bool'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tarefa	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
motivo	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

bool

`coral.assincrono.grupo_tarefas`

Aguarda vários awaitables e preserva a ordem dos resultados.

Assinatura

Assinatura para consulta

```
grupo_tarefas(*esperaveis: 'Awaitable[Any]') -> 'tuple[Any, ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
esperaveis	Awaitable[Any]	VAR_POSITIONAL	não	sem padrão

Retorno declarado

tuple[Any, ...]

Consultar a implementação canônica de coral.assincrono

Implementação canônica de `coral.assincrono`

Código Python

```
"""Utilidades assíncronas pequenas e gerais da Coral 1.5.8.

A sintaxe existente de ``aguarde``, tarefas e funções assíncronas continua sendo
a base da linguagem. Este módulo apenas cobre três lacunas práticas sem criar um
segundo modelo de concorrência: timeout, cancelamento e agrupamento.
"""
from __future__ import annotations

import asyncio
import inspect
from typing import Any, Awaitable

from .erros import ErroOperacao, ErroValor


class ErroTempoEsgotado(ErroOperacao, TimeoutError):
    """Uma operação assíncrona excedeu o limite solicitado."""


async def aguardar_com_timeout(esperavel: Awaitable[Any], segundos: float):
    """Aguarda ``esperavel`` por no máximo ``segundos``.

    O awaitable subjacente é cancelado pela política do ``asyncio.wait_for``
    quando o limite é excedido.
    """
    try:
        limite = float(segundos)
    except (TypeError, ValueError) as erro:
        if inspect.iscoroutine(esperavel):
            esperavel.close()
        raise ErroValor("timeout precisa ser um número positivo") from erro
    if limite <= 0:
        if inspect.iscoroutine(esperavel):
            esperavel.close()
        raise ErroValor("timeout precisa ser um número positivo")
    try:
        return await asyncio.wait_for(esperavel, timeout=limite)
    except TimeoutError as erro:
        raise ErroTempoEsgotado(f"operação excedeu o timeout de {limite:g} segundos",
          causa=erro) from erro


def cancelar_tarefa(tarefa: Any, motivo: str | None = None) -> bool:
    """Solicita cancelamento cooperativo de uma tarefa asyncio.

    Retorna ``True`` quando uma solicitação nova foi feita e ``False`` quando a
    tarefa já havia terminado ou já estava cancelada.
    """
    if not isinstance(tarefa, asyncio.Future):
        raise ErroValor("cancelar_tarefa espera uma tarefa assíncrona")
    if tarefa.done():
        return False
    if motivo is None:
        return bool(tarefa.cancel())
    try:
        return bool(tarefa.cancel(str(motivo)))
    except TypeError:  # compatibilidade com runtimes sem mensagem de cancelamento
        return bool(tarefa.cancel())
```

```
async def grupo_tarefas(*esperaveis: Awaitable[Any]) -> tuple[Any, ...]:
    """Aguarda vários awaitables e preserva a ordem dos resultados."""
    if not esperaveis:
        return ()
    resultados = await asyncio.gather(*esperaveis)
    return tuple(resultados)


__all__ = ["ErroTempoEsgotado", "aguardar_com_timeout", "cancelar_tarefa",
  "grupo_tarefas"]

from __future__ import annotations

from typing import Any

from .comum import ErroCoral, ErroOperacaoCoral


class ErroCoralRuntime(ErroCoral):
    """Base dos erros controlados expostos pelo runtime base da Coral."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroEntrada(ErroCoralRuntime):
    """Falha ao obter entrada para um programa Coral."""


class FimDeEntrada(ErroEntrada, EOFError):
    """A fonte de entrada terminou antes de produzir outra linha."""


class ErroConversao(ErroCoralRuntime, ValueError):
    """Um valor não pôde ser convertido para o tipo Coral solicitado."""

    def __init__(
        self,
        valor: Any,
        destino: str,
        mensagem: str | None = None,
        *,
        causa: BaseException | None = None,
    ):
        self.valor = valor
        self.destino = destino
        super().__init__(
            mensagem or f"não foi possível converter o valor para {destino}",
            causa=causa,
        )


class ErroFormato(ErroCoralRuntime, ValueError):
    """Dados ou texto não obedecem ao formato esperado."""


class ErroValor(ErroCoralRuntime, ValueError):
    """O tipo é aceito, mas o valor não é válido para a operação."""
```

```
class ErroOperacao(ErroOperacaoCoral):
    """Falha operacional com preservação opcional da causa técnica."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroArquivo(ErroOperacao):
    """Falha em uma operação com arquivo ou caminho."""


__all__ = [
    "ErroCoralRuntime",
    "ErroEntrada",
    "FimDeEntrada",
    "ErroConversao",
    "ErroArquivo",
    "ErroFormato",
    "ErroValor",
    "ErroOperacao",
]
```

Biblioteca `coral.caminhos`

`coral.caminhos.CaminhoAceito`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`str | os.PathLike[str]`

`caminho_modulo(...)` · **módulo** `coral.caminhos`

O que faz

Obtém a pasta do módulo e, opcionalmente, junta um caminho a ela.

Assinatura

Assinatura para consulta

```
caminho_modulo(relativo: 'CaminhoAceito | None' = None, *, base: 'CaminhoAceito | None'
  = None) -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
relativo	caminho nativo ou nulo	não	None	Parte a juntar; nulo retorna só a pasta do módulo.
base	caminho nativo ou nulo	não	None	Módulo usado como contexto. nulo busca o chamador na pilha; caminho com sufixo é tratado como arquivo.

Retorno

Path.

Exemplo

Código Coral

```
de coral.caminhos importe caminho_modulo
mostre caminho_modulo("dados.txt", base="principal.coral").name
```

Resultado

Saída esperada

```
dados.txt
```

Erros

TypeError ou ValueError para entradas não compatíveis com caminhos.

Atenção

Não exige que o alvo exista. Um argumento absoluto pode substituir a pasta do módulo; não use esta função como mecanismo de contenção.

`caminho_portatil(...)` · módulo `coral.caminhos`

O que faz

Reconstrói um caminho com o estilo explicitamente declarado.

Assinatura

Assinatura para consulta

```
caminho_portatil(texto: 'str', estilo: 'str', *, concreto: 'bool' = True) -> 'PurePath'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
texto	texto	sim	sem padrão	Representação do caminho no estilo informado.
estilo	texto	sim	sem padrão	windows ou posix, sem distinguir maiúsculas de minúsculas.
concreto	booleano	não	True	Permite Path concreto somente quando o estilo é nativo.

Retorno

PureWindowsPath ou PurePosixPath; quando concreto é verdadeiro e o estilo corresponde ao sistema, retorna Path.

Exemplo

Código Coral

```
de coral.caminhos importe caminho_portatil
defina caminho como caminho_portatil("C:/dados/a.txt", "windows", concreto=falso)
mostre caminho.name
```

Resultado

Saída esperada

```
a.txt
```

Erros

ValueError para estilo desconhecido; TypeError para representação incompatível.

Atenção

Um caminho puro de outro sistema conserva sua estrutura e não passa a autorizar operações locais.

`caminho_recurso(...)` · módulo `coral.caminhos`

O que faz

Monta o caminho de um recurso na fonte ou na pasta _recursos do build.

Assinatura

Assinatura para consulta

```
caminho_recurso(relativo: 'CaminhoAceito', *, base: 'CaminhoAceito | None' = None) ->
  'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
relativo	caminho nativo relativo	sim	sem padrão	Recurso dentro do projeto; não pode ser absoluto nem conter componente `..`.
base	caminho nativo ou nulo	não	None	Módulo usado como contexto. nulo busca o chamador na pilha; caminho com sufixo é tratado como arquivo.

Retorno

Path do recurso.

Exemplo

Código Coral

```
de coral.caminhos importe caminho_recurso
de coral.arquivos importe escrever_texto
execute escrever_texto("coral.toml", "")
mostre caminho_recurso("dados.txt", base="principal.coral").name
```

Resultado

Saída esperada

```
dados.txt
```

Erros

ValueError para caminho absoluto, componente .. ou estilo puro não nativo; FileNotFoundError se não encontra raiz de projeto.

Atenção

Não exige que o recurso exista nem verifica a contenção de links simbólicos. No projeto fonte, usa a raiz; no build, raiz/_recursos.

`estilo_caminho(...)` · módulo `coral.caminhos`

O que faz

Identifica o estilo de um caminho puro.

Assinatura

Assinatura para consulta

```
estilo_caminho(valor: 'PurePath') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	PurePath	sim	sem padrão	Objeto de caminho puro a classificar.

Retorno

Texto windows para PureWindowsPath, posix nos demais casos.

Exemplo

Código Coral

```
de coral.caminhos importe estilo_caminho
de pathlib importe PureWindowsPath
mostre estilo_caminho(PureWindowsPath("C:/dados"))
```

Resultado

Saída esperada

```
windows
```

Atenção

O contrato de entrada é PurePath. A função não tenta detectar o estilo de uma string.

`estilo_nativo(...)` · módulo `coral.caminhos`

O que faz

Informa o estilo de caminho do sistema em execução.

Assinatura

Assinatura para consulta

```
estilo_nativo() -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Texto windows em os.name igual a nt; posix nos demais sistemas.

Exemplo

Código Coral

```
de coral.caminhos importe estilo_nativo
mostre estilo_nativo()
```

Resultado

Saída esperada

```
posix
```

Atenção

Resultado verificado no Linux; no Windows o retorno é windows. Isso não constitui qualificação da release no Windows.

`juntar_caminho(...)` · módulo `coral.caminhos`

O que faz

Junta componentes usando Path nativo.

Assinatura

Assinatura para consulta

```
juntar_caminho(*partes: 'Any', expandir_usuario: 'bool' = True) -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
partes	argumentos variáveis	não	sem padrão	Componentes de caminho; objetos PathLike usam os.fspath, os demais usam str.
expandir_usuario	booleano	não	True	Expande ~ no caminho resultante.

Retorno

Path; sem partes, representa a pasta corrente.

Exemplo

Código Coral

```
de coral.caminhos importe juntar_caminho
mostre juntar_caminho("dados", "arquivo.txt", expandir_usuario=falso).name
```

Resultado

Saída esperada

```
arquivo.txt
```

Erros

Erros de Path ou expanduser para entrada incompatível ou usuário sem pasta resolvível.

Atenção

Uma parte absoluta pode substituir componentes anteriores. Esta função monta um caminho; não é uma barreira de contenção.

`normalizar_caminho(...)` · módulo `coral.caminhos`

O que faz

Converte a entrada para Path nativo com opções explícitas.

Assinatura

Assinatura para consulta

```
normalizar_caminho(valor: 'CaminhoAceito', *, expandir_usuario: 'bool' = False,
  resolver: 'bool' = False) -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho interpretado no sistema atual.
expandir_usuario	booleano	não	False	Expande ~ quando verdadeiro.
resolver	booleano	não	False	Resolve links e torna o caminho absoluto quando verdadeiro.

Retorno

Path.

Exemplo

Código Coral

```
de coral.caminhos importe normalizar_caminho
mostre normalizar_caminho("dados/arquivo.txt").name
```

Resultado

Saída esperada

```
arquivo.txt
```

Erros

ValueError para PurePath de outro sistema; TypeError para entrada não compatível com caminho.

Atenção

Texto é sempre interpretado no estilo nativo. A expansão de ~ e a resolução ficam desligadas por padrão.

`raiz_projeto(...)` · módulo `coral.caminhos`

O que faz

Busca a raiz de projeto ou de build subindo a partir do módulo de contexto.

Assinatura

Assinatura para consulta

```
raiz_projeto(*, base: 'CaminhoAceito | None' = None) -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
base	caminho nativo ou nulo	não	None	Módulo usado como contexto. nulo busca o chamador na pilha; caminho com sufixo é tratado como arquivo.

Retorno

Path absoluto da raiz encontrada.

Exemplo

Código Coral

```
de coral.caminhos importe raiz_projeto
de coral.arquivos importe escrever_texto
execute escrever_texto("coral.toml", "")
mostre raiz_projeto(base="principal.coral").is_absolute()
```

Resultado

Saída esperada

```
True
```

Erros

FileNotFoundError se não encontra coral.toml nem .coral_build_manifest.json; erros de normalização para caminho incompatível.

Atenção

A busca reconhece .coral_build_manifest.json antes de coral.toml em cada pasta. Ter a raiz encontrada não valida todo o projeto.

Consultar a implementação canônica de coral.caminhos

Implementação canônica de `coral.caminhos`

Código Python

```
"""Contrato único de caminhos usado pelas APIs públicas da Coral.

Texto recebido por uma operação local é interpretado como caminho nativo do
sistema atual. Objetos ``PurePath`` de outro estilo continuam sendo caminhos
portáteis, mas não são usados silenciosamente em operações locais.
"""
from __future__ import annotations

import os
from pathlib import Path, PurePath, PurePosixPath, PureWindowsPath
from typing import Any


CaminhoAceito = str | os.PathLike[str]


def estilo_caminho(valor: PurePath) -> str:
    if isinstance(valor, PureWindowsPath):
        return "windows"
    return "posix"


def estilo_nativo() -> str:
    return "windows" if os.name == "nt" else "posix"



def normalizar_caminho(
    valor: CaminhoAceito,
    *,
    expandir_usuario: bool = False,
    resolver: bool = False,
) -> Path:
    """Converte uma entrada para ``Path`` nativo com regras explícitas.

    Caminhos puros de outro sistema são recusados em operações locais. Isso
    evita transformar silenciosamente ``C:\\dados`` em um caminho relativo em
    um host POSIX, ou o inverso.
    """

    if isinstance(valor, PurePath) and estilo_caminho(valor) != estilo_nativo():
        raise ValueError(
            f"caminho de estilo {estilo_caminho(valor)} não pode ser usado "
            f"como caminho local em sistema {estilo_nativo()}"
        )
    try:
        caminho = Path(os.fspath(valor))
    except (TypeError, ValueError) as erro:
        raise TypeError("caminho precisa ser texto ou objeto compatível com
          os.PathLike") from erro
    if expandir_usuario:
        caminho = caminho.expanduser()
    if resolver:
        caminho = caminho.resolve()
    return caminho


def juntar_caminho(*partes: Any, expandir_usuario: bool = True) -> Path:
    if not partes:
        caminho = Path()
```

```
    else:
        caminho = Path(*(os.fspath(p) if isinstance(p, os.PathLike) else str(p) for p in
           partes))
    return caminho.expanduser() if expandir_usuario else caminho


def caminho_portatil(texto: str, estilo: str, *, concreto: bool = True) -> PurePath:
    """Reconstrói a categoria caminho sem fingir compatibilidade entre SOs."""

    estilo = str(estilo).casefold()
    if estilo not in {"posix", "windows"}:
        raise ValueError(f"estilo de caminho desconhecido: {estilo!r}")
    puro: PurePath = PureWindowsPath(texto) if estilo == "windows" else
      PurePosixPath(texto)
    if concreto and estilo == estilo_nativo():
        return Path(puro)
    return puro


__all__ = [
    "CaminhoAceito",
    "normalizar_caminho",
    "juntar_caminho",
    "estilo_caminho",
    "estilo_nativo",
    "caminho_portatil",
]


def _arquivo_chamador(base: CaminhoAceito | None = None) -> Path:
    if base is not None:
        caminho = normalizar_caminho(base, resolver=True)
        return caminho if caminho.suffix else caminho / "__coral_contexto__.py"
    import inspect
    este = Path(__file__).resolve()
    for quadro in inspect.stack()[1:]:
        candidato = Path(quadro.filename).resolve()
        if candidato != este:
            return candidato
    return Path.cwd().resolve() / "__coral_contexto__.py"


def _achar_raiz_contexto(arquivo: Path) -> tuple[Path, bool]:
    atual = arquivo.parent if arquivo.suffix else arquivo
    for pasta in (atual, *atual.parents):
        if (pasta / ".coral_build_manifest.json").is_file():
            return pasta, True
        if (pasta / "coral.toml").is_file():
            return pasta, False
    raise FileNotFoundError("raiz de projeto Coral não encontrada a partir do módulo
      atual")


def raiz_projeto(*, base: CaminhoAceito | None = None) -> Path:
    """Retorna a raiz portátil do projeto fonte ou do build movido."""
    raiz, _ = _achar_raiz_contexto(_arquivo_chamador(base))
    return raiz


def caminho_modulo(
    relativo: CaminhoAceito | None = None,
```

```
    *,
    base: CaminhoAceito | None = None,
) -> Path:
    """Retorna a pasta do módulo chamador, opcionalmente acrescida de um caminho."""
    arquivo = _arquivo_chamador(base)
    pasta = arquivo.parent
    return pasta if relativo is None else pasta / normalizar_caminho(relativo)


def caminho_recurso(relativo: CaminhoAceito, *, base: CaminhoAceito | None = None) ->
  Path:
    """Resolve um recurso declarado tanto na fonte quanto em um build movido."""
    arquivo = _arquivo_chamador(base)
    raiz, em_build = _achar_raiz_contexto(arquivo)
    rel = normalizar_caminho(relativo)
    if rel.is_absolute() or ".." in rel.parts:
        raise ValueError("caminho de recurso deve ser relativo e permanecer dentro do
          projeto")
    return raiz / ("_recursos" if em_build else "") / rel


__all__ += ["raiz_projeto", "caminho_modulo", "caminho_recurso"]
```

Biblioteca `coral.colecoes`

`agrupar_pares(...)` · **módulo** `coral.colecoes`

O que faz

Combina chaves e valores por posição para construir um dicionário.

Assinatura

Assinatura para consulta

```
agrupar_pares(chaves, valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
chaves	iterável de chaves hashable	sim	sem padrão	Coleção de chaves.
valores	iterável de valores	sim	sem padrão	Coleção de valores, combinada por posição com as chaves.

Retorno

Dicionário com os pares; chaves repetidas mantêm o último valor.

Exemplo

Código Coral

```
de coral.colecoes importe agrupar_pares
mostre agrupar_pares(["a", "b"], [1, 2])
```

Resultado

Saída esperada

```
{'a': 1, 'b': 2}
```

Atenção

Consome chaves e valores até terminar o iterável mais curto. Chaves repetidas ficam com o último valor correspondente.

`contar(...)` · módulo `coral.colecoes`

O que faz

Conta as ocorrências de cada valor.

Assinatura

Assinatura para consulta

```
contar(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de itens hashable	sim	sem padrão	Coleção de entrada.

Retorno

Dicionário de frequências.

Exemplo

Código Coral

```
de coral.colecoes importe contar
mostre contar([3, 1, 3])
```

Resultado

Saída esperada

```
{3: 2, 1: 1}
```

Forma natural

`conte as ocorrências de [3, 1, 3]` é equivalente para estes argumentos.

Atenção

A origem não é alterada. Chaves e elementos usados na eliminação de duplicatas ou contagem precisam ser utilizáveis como chaves.

`ordenar(...)` · módulo `coral.colecoes`

O que faz

Ordena os valores e produz uma nova lista.

Assinatura

Assinatura para consulta

```
ordenar(valores, reverso=False)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de itens comparáveis	sim	sem padrão	Coleção de entrada.
reverso	booleano	não	False	Se verdadeiro, inverte a ordem da classificação.

Retorno

Lista ordenada.

Exemplo

Código Coral

```
de coral.colecoes importe ordenar
mostre ordenar([3, 1, 2])
```

Resultado

Saída esperada

```
[1, 2, 3]
```

Forma natural

`ordene os valores de [3, 1, 2]` é equivalente para estes argumentos.

Atenção

Retorna uma lista nova; não modifica a coleção original. Requer que os itens possam ser comparados entre si.

`primeiro(...)` · módulo `coral.colecoes`

O que faz

Obtém o primeiro elemento ou o padrão se não houver elemento.

Assinatura

Assinatura para consulta

```
primeiro(valores, padrao=None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável	sim	sem padrão	Coleção de entrada.
padrao	qualquer valor	não	None	Valor devolvido quando a operação não pode obter o resultado normal. Omitir é diferente de fornecer nulo.

Retorno

Elemento ou padrão; por padrão, nulo.

Exemplo

Código Coral

```
de coral.colecoes importe primeiro
mostre primeiro([])
```

Resultado

Saída esperada

```
None
```

Forma natural

`o primeiro de []` é equivalente para estes argumentos.

Atenção

A origem não é alterada. Chaves e elementos usados na eliminação de duplicatas ou contagem precisam ser utilizáveis como chaves.

`unicos(...)` · módulo `coral.colecoes`

O que faz

Conserva a primeira ocorrência de cada valor.

Assinatura

Assinatura para consulta

```
unicos(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de itens hashable	sim	sem padrão	Coleção de entrada.

Retorno

Lista sem duplicatas.

Exemplo

Código Coral

```
de coral.colecoes importe unicos
mostre unicos([3, 1, 3, 2])
```

Resultado

Saída esperada

```
[3, 1, 2]
```

Forma natural

`os valores únicos de [3, 1, 3, 2]` é equivalente para estes argumentos.

Atenção

A origem não é alterada. Chaves e elementos usados na eliminação de duplicatas ou contagem precisam ser utilizáveis como chaves.

Consultar a implementação canônica de coral.colecoes

Implementação canônica de `coral.colecoes`

Código Python

```
"""Superfície pública de coral.colecoes."""
from .stdlib.colecoes import *  # noqa: F401,F403

from collections import Counter

def ordenar(valores, reverso=False): return sorted(valores, reverse=reverso)
def unicos(valores): return list(dict.fromkeys(valores))
def contar(valores): return dict(Counter(valores))
def primeiro(valores, padrao=None): return next(iter(valores), padrao)
def agrupar_pares(chaves, valores): return dict(zip(chaves, valores))
```

Biblioteca `coral.conversoes`

`coral.conversoes.ErroConversao`

Um valor não pôde ser convertido para o tipo Coral solicitado.

Assinatura

Assinatura para consulta

```
ErroConversao(valor: 'Any', destino: 'str', mensagem: 'str | None' = None, *, causa:
  'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
destino	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
mensagem	str | None	POSITIONAL_OR_KEYWORD	não	None
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`booleano(...)` · módulo `coral.conversoes`

O que faz

Converte representações explícitas de verdadeiro e falso.

Assinatura

Assinatura para consulta

```
booleano(valor: 'Any', padrao: 'Any' = <ausente>) -> 'bool | Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	booleano, número 0/1 ou texto booleano explícito	sim	sem padrão	Valor sobre o qual a operação atua.
padrao	qualquer valor	não	<ausente>	Valor devolvido quando a operação não pode obter o resultado normal. Omitir é diferente de fornecer nulo.

Retorno

Booleano, ou o padrão fornecido.

Exemplo

Código Coral

```
de coral.conversoes importe booleano
mostre booleano("sim")
```

Resultado

Saída esperada

```
True
```

Forma natural

`converta "sim" para booleano` é equivalente. O padrão natural usa `tente converter ..., senão ...`.

Erros

ErroConversao se a entrada é inválida e o padrão foi omitido.

Atenção

Aceita verdadeiro/falso, números exatamente 0/1 e textos normalizados: verdadeiro,true,sim,1; falso,false,nao,não,0. Não usa a regra Python de qualquer objeto não vazio ser verdadeiro. O padrão é retornado como foi fornecido, inclusive nulo.

`decimal(...)` · módulo `coral.conversoes`

O que faz

Converte um número ou texto para decimal finito, aceitando ponto ou vírgula decimal.

Assinatura

Assinatura para consulta

```
decimal(valor: 'Any', padrao: 'Any' = <ausente>) -> 'float | Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	inteiro, decimal finito ou texto numérico, exceto booleano	sim	sem padrão	Valor sobre o qual a operação atua.
padrao	qualquer valor	não	<ausente>	Valor devolvido quando a operação não pode obter o resultado normal. Omitir é diferente de fornecer nulo.

Retorno

Decimal finito, ou o padrão fornecido.

Exemplo

Código Coral

```
de coral.conversoes importe decimal
mostre decimal("2,5")
```

Resultado

Saída esperada

```
2.5
```

Forma natural

`converta "2,5" para decimal` é equivalente. O padrão natural usa `tente converter ..., senão ...`.

Erros

ErroConversao se a entrada é inválida e o padrão foi omitido.

Atenção

Aceita ponto ou vírgula decimal, mas rejeita a mistura dos dois no mesmo texto. Rejeita NaN, infinitos e booleanos. O padrão é retornado como foi fornecido, inclusive nulo.

`inteiro(...)` · módulo `coral.conversoes`

O que faz

Converte um inteiro, um decimal exato ou uma representação textual inteira.

Assinatura

Assinatura para consulta

```
inteiro(valor: 'Any', padrao: 'Any' = <ausente>) -> 'int | Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	inteiro, decimal finito sem fração ou texto de inteiro, exceto booleano	sim	sem padrão	Valor sobre o qual a operação atua.
padrao	qualquer valor	não	<ausente>	Valor devolvido quando a operação não pode obter o resultado normal. Omitir é diferente de fornecer nulo.

Retorno

Inteiro, ou o padrão explicitamente fornecido.

Exemplo

Código Coral

```
de coral.conversoes importe inteiro
mostre inteiro("12")
```

Resultado

Saída esperada

```
12
```

Forma natural

`converta "12" para inteiro` é equivalente. O padrão natural usa `tente converter ..., senão ...`.

Erros

ErroConversao se a entrada é inválida e o padrão foi omitido.

Atenção

Não trunca decimais fracionários e rejeita booleanos. Textos admitem sinal e espaços nas extremidades, sem separadores de milhar. O padrão é retornado como foi fornecido, inclusive nulo.

`texto(...)` · módulo `coral.conversoes`

O que faz

Produz texto com a grafia Coral para os valores básicos.

Assinatura

Assinatura para consulta

```
texto(valor: 'Any', padrao: 'Any' = <ausente>) -> 'str | Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor representável por str, incluindo nulo e booleanos	sim	sem padrão	Valor sobre o qual a operação atua.
padrao	qualquer valor	não	<ausente>	Valor devolvido quando a operação não pode obter o resultado normal. Omitir é diferente de fornecer nulo.

Retorno

Texto, ou o padrão se sua produção falhar.

Exemplo

Código Coral

```
de coral.conversoes importe texto
mostre texto(nulo)
```

Resultado

Saída esperada

```
nulo
```

Forma natural

`converta nulo para texto` é equivalente. O padrão natural usa `tente converter ..., senão ...`.

Erros

ErroConversao se a entrada é inválida e o padrão foi omitido.

Atenção

nulo, verdadeiro e falso usam grafia Coral; os demais valores usam str. Se a representação falha, devolve o padrão fornecido, inclusive nulo, ou levanta ErroConversao quando o padrão foi omitido.

Consultar a implementação canônica de coral.conversoes

Implementação canônica de `coral.conversoes`

Código Python

```
from __future__ import annotations

import math
import re
from typing import Any

from .erros import ErroConversao


_AUSENTE = object()
_RE_INTEIRO = re.compile(r"^[+-]?\d+$")
_VERDADEIROS = {"verdadeiro", "true", "sim", "1"}
_FALSOS = {"falso", "false", "nao", "não", "0"}


def _falhar_ou_padrao(valor: Any, destino: str, padrao: Any, causa: BaseException | None
   = None):
    if padrao is not _AUSENTE:
        return padrao
    raise ErroConversao(valor, destino, causa=causa) from causa


def inteiro(valor: Any, padrao: Any = _AUSENTE) -> int | Any:
    """Converte para inteiro de forma estrita, com padrão opcional."""


    try:
        if isinstance(valor, bool):
            raise ValueError("booleano não é inteiro implícito")
        if isinstance(valor, int):
            return valor
        if isinstance(valor, float):
            if not math.isfinite(valor) or not valor.is_integer():
                raise ValueError("decimal não representa inteiro exato")
            return int(valor)
        if isinstance(valor, str):
            texto_valor = valor.strip()
            if not _RE_INTEIRO.fullmatch(texto_valor):
                raise ValueError("texto não representa inteiro")
            return int(texto_valor, 10)
        raise TypeError(f"tipo {type(valor).__name__} não é conversível para inteiro")
    except (TypeError, ValueError, OverflowError) as erro:
        return _falhar_ou_padrao(valor, "inteiro", padrao, erro)


def decimal(valor: Any, padrao: Any = _AUSENTE) -> float | Any:
    """Converte para decimal finito, aceitando ponto ou vírgula decimal em texto."""

    try:
        if isinstance(valor, bool):
            raise ValueError("booleano não é decimal implícito")
        if isinstance(valor, (int, float)):
            resultado = float(valor)
        elif isinstance(valor, str):
            texto_valor = valor.strip()
            if not texto_valor:
                raise ValueError("texto vazio")
            if "," in texto_valor and "." in texto_valor:
                raise ValueError("separador decimal ambíguo")
            resultado = float(texto_valor.replace(",", "."))
```

```
        else:
            raise TypeError(f"tipo {type(valor).__name__} não é conversível para
              decimal")
        if not math.isfinite(resultado):
            raise ValueError("decimal precisa ser finito")
        return resultado
    except (TypeError, ValueError, OverflowError) as erro:
        return _falhar_ou_padrao(valor, "decimal", padrao, erro)


def texto(valor: Any, padrao: Any = _AUSENTE) -> str | Any:
    """Produz texto com grafia coerente para os literais básicos da Coral."""

    try:
        if valor is None:
            return "nulo"
        if valor is True:
            return "verdadeiro"
        if valor is False:
            return "falso"
        return str(valor)
    except Exception as erro:
        return _falhar_ou_padrao(valor, "texto", padrao, erro)


def booleano(valor: Any, padrao: Any = _AUSENTE) -> bool | Any:
    """Converte apenas representações booleanas explícitas e não vazias."""

    try:
        if isinstance(valor, bool):
            return valor
        if isinstance(valor, (int, float)) and not isinstance(valor, bool):
            if valor == 1:
                return True
            if valor == 0:
                return False
            raise ValueError("número precisa ser 0 ou 1")
        if isinstance(valor, str):
            normal = valor.strip().casefold()
            if normal in _VERDADEIROS:
                return True
            if normal in _FALSOS:
                return False
            raise ValueError("texto não representa booleano")
        raise TypeError(f"tipo {type(valor).__name__} não é conversível para booleano")
    except (TypeError, ValueError, OverflowError) as erro:
        return _falhar_ou_padrao(valor, "booleano", padrao, erro)


__all__ = ["ErroConversao", "inteiro", "decimal", "texto", "booleano"]

from __future__ import annotations

from typing import Any

from .comum import ErroCoral, ErroOperacaoCoral


class ErroCoralRuntime(ErroCoral):
    """Base dos erros controlados expostos pelo runtime base da Coral."""

```

```
    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroEntrada(ErroCoralRuntime):
    """Falha ao obter entrada para um programa Coral."""


class FimDeEntrada(ErroEntrada, EOFError):
    """A fonte de entrada terminou antes de produzir outra linha."""


class ErroConversao(ErroCoralRuntime, ValueError):
    """Um valor não pôde ser convertido para o tipo Coral solicitado."""

    def __init__(
        self,
        valor: Any,
        destino: str,
        mensagem: str | None = None,
        *,
        causa: BaseException | None = None,
    ):
        self.valor = valor
        self.destino = destino
        super().__init__(
            mensagem or f"não foi possível converter o valor para {destino}",
            causa=causa,
        )


class ErroFormato(ErroCoralRuntime, ValueError):
    """Dados ou texto não obedecem ao formato esperado."""


class ErroValor(ErroCoralRuntime, ValueError):
    """O tipo é aceito, mas o valor não é válido para a operação."""


class ErroOperacao(ErroOperacaoCoral):
    """Falha operacional com preservação opcional da causa técnica."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroArquivo(ErroOperacao):
    """Falha em uma operação com arquivo ou caminho."""


__all__ = [
    "ErroCoralRuntime",
    "ErroEntrada",
    "FimDeEntrada",
    "ErroConversao",
    "ErroArquivo",
    "ErroFormato",
```

```
    "ErroValor",
    "ErroOperacao",
]
```

Biblioteca `coral.criptografia`

`coral.criptografia.autenticar`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
autenticar(dados, chave, algoritmo='sha256', formato='hex')
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
dados	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
chave	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
algoritmo	sem anotação	POSITIONAL_OR_KEYWORD	não	'sha256'
formato	sem anotação	POSITIONAL_OR_KEYWORD	não	'hex'

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.criptografia.comparar_com_seguranca`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
comparar_com_seguranca(primeiro, segundo) -> 'bool'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
primeiro	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
segundo	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

bool

`coral.criptografia.explicar_algoritmo`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
explicar_algoritmo(nome) -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

dict[str, Any]

`coral.criptografia.gerar_bytes_seguros`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
gerar_bytes_seguros(quantidade=32) -> 'bytes'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
quantidade	sem anotação	POSITIONAL_OR_KEYWORD	não	32

Retorno declarado

bytes

`coral.criptografia.gerar_token_seguro`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
gerar_token_seguro(bytes_de_entropia=32, formato='url')
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
bytes_de_entropia	sem anotação	POSITIONAL_OR_KEYWORD	não	32
formato	sem anotação	POSITIONAL_OR_KEYWORD	não	'url'

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.criptografia.listar_algoritmos`

Lista o catálogo geral, incluindo hashes e variantes HMAC.

Assinatura

Assinatura para consulta

```
listar_algoritmos() -> 'list[str]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

list[str]

`coral.criptografia.resumir`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
resumir(dados, algoritmo='sha256', formato='hex')
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
dados	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
algoritmo	sem anotação	POSITIONAL_OR_KEYWORD	não	'sha256'
formato	sem anotação	POSITIONAL_OR_KEYWORD	não	'hex'

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.criptografia.verificar_autenticacao`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
verificar_autenticacao(dados, chave, esperado, algoritmo='sha256', formato='hex') ->
  'bool'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
dados	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
chave	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
esperado	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
algoritmo	sem anotação	POSITIONAL_OR_KEYWORD	não	'sha256'
formato	sem anotação	POSITIONAL_OR_KEYWORD	não	'hex'

Retorno declarado

bool

`coral.criptografia.verificar_resumo`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
verificar_resumo(dados, esperado, algoritmo='sha256', formato='hex') -> 'bool'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
dados	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
esperado	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
algoritmo	sem anotação	POSITIONAL_OR_KEYWORD	não	'sha256'

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
formato	sem anotação	POSITIONAL_OR_KEYWORD	não	'hex'

Retorno declarado

bool

Consultar a implementação canônica de coral.criptografia

Implementação canônica de `coral.criptografia`

Código Python

```
"""Fundação criptográfica didática refinada na Coral 1.6.16.

A superfície usa somente primitivas criptográficas da biblioteca padrão do
Python. Texto é codificado como UTF 8, bytes são preservados e nenhum objeto
arbitrário é convertido silenciosamente em texto. Hash, HMAC, codificação e
geração de tokens são conceitos distintos e não formam, isoladamente, um
protocolo de segurança completo.
"""
from __future__ import annotations

import hashlib
import hmac
import secrets
from typing import Any

from coral.erros import ErroFormato, ErroValor


_REGISTRO = {
    "sha256": {
        "nome": "sha256",
        "categoria": "hash",
        "finalidade": "produzir um resumo determinístico de dados",
        "algoritmo_base": "sha256",

        "tamanho_bytes": 32,
        "usa_chave": False,
        "deterministico": True,
        "codificacao_texto": "utf-8",
        "formatos_saida": ["hex", "bytes"],
        "operacoes": ["resumir", "verificar_resumo"],
        "usos_apropriados": [
            "integridade não autenticada",
            "identificação de conteúdo",
            "base para protocolos bem especificados",
        ],
        "usos_inadequados": ["cifrar dados", "armazenar senhas diretamente", "autenticar
           sem chave"],
        "exemplo": 'resumir("mensagem", algoritmo="sha256")',
        "advertencia": "Hash não é cifragem e uma primitiva isolada não forma um
          protocolo seguro.",
    },
    "sha512": {
        "nome": "sha512",
        "categoria": "hash",
```

```
        "finalidade": "produzir um resumo determinístico de dados",
        "algoritmo_base": "sha512",
        "tamanho_bytes": 64,
        "usa_chave": False,
        "deterministico": True,
        "codificacao_texto": "utf-8",
        "formatos_saida": ["hex", "bytes"],
        "operacoes": ["resumir", "verificar_resumo"],
        "usos_apropriados": [
            "integridade não autenticada",
            "identificação de conteúdo",
            "base para protocolos bem especificados",
        ],
        "usos_inadequados": ["cifrar dados", "armazenar senhas diretamente", "autenticar
           sem chave"],
        "exemplo": 'resumir("mensagem", algoritmo="sha512")',
        "advertencia": "Hash não é cifragem e uma primitiva isolada não forma um
          protocolo seguro.",
    },
    "hmac-sha256": {
        "nome": "hmac-sha256",
        "categoria": "autenticacao",
        "finalidade": "autenticar integridade de dados usando uma chave secreta",
        "algoritmo_base": "sha256",
        "tamanho_bytes": 32,
        "usa_chave": True,
        "deterministico": True,
        "codificacao_texto": "utf-8",
        "formatos_saida": ["hex", "bytes"],
        "operacoes": ["autenticar", "verificar_autenticacao"],
        "usos_apropriados": ["autenticação de mensagens quando o protocolo especifica
          HMAC"],
        "usos_inadequados": ["cifrar dados", "ocultar conteúdo", "substituir
          gerenciamento de chaves"],
        "exemplo": 'autenticar("mensagem", chave, algoritmo="sha256")',
        "advertencia": "HMAC autentica dados, mas não os torna secretos.",
    },
    "hmac-sha512": {
        "nome": "hmac-sha512",
        "categoria": "autenticacao",
        "finalidade": "autenticar integridade de dados usando uma chave secreta",
        "algoritmo_base": "sha512",
        "tamanho_bytes": 64,
        "usa_chave": True,
        "deterministico": True,
        "codificacao_texto": "utf-8",
        "formatos_saida": ["hex", "bytes"],
        "operacoes": ["autenticar", "verificar_autenticacao"],
        "usos_apropriados": ["autenticação de mensagens quando o protocolo especifica
          HMAC"],
        "usos_inadequados": ["cifrar dados", "ocultar conteúdo", "substituir
          gerenciamento de chaves"],
        "exemplo": 'autenticar("mensagem", chave, algoritmo="sha512")',
        "advertencia": "HMAC autentica dados, mas não os torna secretos.",
    },
}
_HASHES = {"sha256": hashlib.sha256, "sha512": hashlib.sha512}
_HEX = frozenset("0123456789abcdefABCDEF")


def _bytes(dados: Any, papel: str) -> bytes:
```

```
    if isinstance(dados, str):
        return dados.encode("utf-8")
    if isinstance(dados, (bytes, bytearray, memoryview)):
        return bytes(dados)
    raise ErroValor(f"{papel} precisa ser texto UTF 8 ou bytes")


def _texto_configuracao(valor: Any, papel: str, erro=ErroValor) -> str:
    if not isinstance(valor, str):
        raise erro(f"{papel} precisa ser texto")
    return valor.strip().casefold()


def _algoritmo_hash(nome: Any):
    normal = _texto_configuracao(nome, "algoritmo").replace("_", "").replace("-", "")
    aliases = {"sha256": "sha256", "sha512": "sha512"}
    canonico = aliases.get(normal)
    if canonico is None:
        raise ErroValor(
            "algoritmo de hash não registrado para esta operação; use 'sha256' ou
              'sha512'; "
            "para HMAC use autenticar"
        )
    return canonico, _HASHES[canonico]


def _formato_resumo(formato: Any) -> str:
    normal = _texto_configuracao(formato, "formato", ErroFormato)
    aliases = {"hex": "hex", "hexadecimal": "hex", "bytes": "bytes", "bruto": "bytes"}
    canonico = aliases.get(normal)
    if canonico is None:
        raise ErroFormato("formato precisa ser 'hex' ou 'bytes'")
    return canonico


def _formato_token(formato: Any) -> str:
    normal = _texto_configuracao(formato, "formato de token", ErroFormato)
    aliases = {
        "url": "url",
        "urlsafe": "url",
        "url_seguro": "url",
        "url-seguro": "url",
        "hex": "hex",
        "hexadecimal": "hex",
        "bytes": "bytes",
        "bruto": "bytes",
    }
    canonico = aliases.get(normal)
    if canonico is None:
        raise ErroFormato("formato de token precisa ser 'url', 'hex' ou 'bytes'")
    return canonico


def _formatar_resultado(bruto: bytes, formato: Any):
    canonico = _formato_resumo(formato)
    if canonico == "hex":
        return bruto.hex()
    return bruto


def _esperado_bytes(valor: Any, formato: Any, *, tamanho: int, algoritmo: str) -> bytes:
```

```
    canonico = _formato_resumo(formato)
    if canonico == "bytes":
        if not isinstance(valor, (bytes, bytearray, memoryview)):
            raise ErroFormato("resultado esperado no formato bytes precisa ser binário")
        recebido = bytes(valor)
        if len(recebido) != tamanho:
            raise ErroFormato(f"resultado esperado para {algoritmo} precisa ter
              {tamanho} bytes")
        return recebido

    if not isinstance(valor, str):
        raise ErroFormato("resultado hexadecimal esperado precisa ser texto")
    texto = valor.strip()
    caracteres = tamanho * 2
    if len(texto) != caracteres or any(caractere not in _HEX for caractere in texto):
        raise ErroFormato(
            f"resultado hexadecimal esperado para {algoritmo} precisa ter {caracteres}
              dígitos hexadecimais"
        )
    return bytes.fromhex(texto)


def resumir(dados, algoritmo="sha256", formato="hex"):
    _, construtor = _algoritmo_hash(algoritmo)
    bruto = construtor(_bytes(dados, "dados")).digest()
    return _formatar_resultado(bruto, formato)


def verificar_resumo(dados, esperado, algoritmo="sha256", formato="hex") -> bool:
    canonico, construtor = _algoritmo_hash(algoritmo)
    calculado = construtor(_bytes(dados, "dados")).digest()
    recebido = _esperado_bytes(
        esperado,
        formato,
        tamanho=_REGISTRO[canonico]["tamanho_bytes"],
        algoritmo=canonico,
    )
    return hmac.compare_digest(calculado, recebido)


def autenticar(dados, chave, algoritmo="sha256", formato="hex"):
    _, construtor = _algoritmo_hash(algoritmo)
    chave_bytes = _bytes(chave, "chave")
    if not chave_bytes:
        raise ErroValor("chave HMAC não pode ser vazia")
    bruto = hmac.new(chave_bytes, _bytes(dados, "dados"), digestmod=construtor).digest()
    return _formatar_resultado(bruto, formato)


def verificar_autenticacao(dados, chave, esperado, algoritmo="sha256", formato="hex") ->
   bool:
    canonico, construtor = _algoritmo_hash(algoritmo)
    chave_bytes = _bytes(chave, "chave")
    if not chave_bytes:
        raise ErroValor("chave HMAC não pode ser vazia")
    calculado = hmac.new(chave_bytes, _bytes(dados, "dados"),
      digestmod=construtor).digest()
    recebido = _esperado_bytes(
        esperado,
        formato,
        tamanho=_REGISTRO[canonico]["tamanho_bytes"],
```

```
        algoritmo=canonico,
    )
    return hmac.compare_digest(calculado, recebido)


def gerar_bytes_seguros(quantidade=32) -> bytes:
    if isinstance(quantidade, bool) or not isinstance(quantidade, int) or quantidade <
      0:
        raise ErroValor("quantidade precisa ser um inteiro não negativo")
    return secrets.token_bytes(quantidade)


def gerar_token_seguro(bytes_de_entropia=32, formato="url"):
    if isinstance(bytes_de_entropia, bool) or not isinstance(bytes_de_entropia, int) or
      bytes_de_entropia < 0:
        raise ErroValor("bytes_de_entropia precisa ser um inteiro não negativo")
    canonico = _formato_token(formato)
    if canonico == "url":
        return secrets.token_urlsafe(bytes_de_entropia)
    if canonico == "hex":
        return secrets.token_hex(bytes_de_entropia)
    return secrets.token_bytes(bytes_de_entropia)


def comparar_com_seguranca(primeiro, segundo) -> bool:
    a = _bytes(primeiro, "primeiro valor")
    b = _bytes(segundo, "segundo valor")
    return hmac.compare_digest(a, b)


def listar_algoritmos() -> list[str]:
    """Lista o catálogo geral, incluindo hashes e variantes HMAC."""
    return list(_REGISTRO)


def explicar_algoritmo(nome) -> dict[str, Any]:
    chave = _texto_configuracao(nome, "nome do algoritmo").replace("_", "-")
    aliases = {
        "sha-256": "sha256",
        "sha256": "sha256",
        "sha-512": "sha512",
        "sha512": "sha512",
        "hmac-sha-256": "hmac-sha256",
        "hmac-sha256": "hmac-sha256",
        "hmac-sha-512": "hmac-sha512",
        "hmac-sha512": "hmac-sha512",
    }
    canonico = aliases.get(chave)
    if canonico is None:
        raise ErroValor("algoritmo não registrado; use listar_algoritmos para consultar
          os nomes disponíveis")
    item = _REGISTRO[canonico]
    return {k: (list(v) if isinstance(v, list) else v) for k, v in item.items()}


__all__ = [
    "resumir",
    "verificar_resumo",
    "autenticar",
    "verificar_autenticacao",
    "gerar_bytes_seguros",
```

```
    "gerar_token_seguro",
    "comparar_com_seguranca",
    "listar_algoritmos",
    "explicar_algoritmo",
]

from __future__ import annotations

from typing import Any

from .comum import ErroCoral, ErroOperacaoCoral


class ErroCoralRuntime(ErroCoral):
    """Base dos erros controlados expostos pelo runtime base da Coral."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroEntrada(ErroCoralRuntime):
    """Falha ao obter entrada para um programa Coral."""


class FimDeEntrada(ErroEntrada, EOFError):
    """A fonte de entrada terminou antes de produzir outra linha."""


class ErroConversao(ErroCoralRuntime, ValueError):
    """Um valor não pôde ser convertido para o tipo Coral solicitado."""

    def __init__(
        self,
        valor: Any,
        destino: str,
        mensagem: str | None = None,
        *,
        causa: BaseException | None = None,
    ):
        self.valor = valor
        self.destino = destino
        super().__init__(
            mensagem or f"não foi possível converter o valor para {destino}",
            causa=causa,
        )


class ErroFormato(ErroCoralRuntime, ValueError):
    """Dados ou texto não obedecem ao formato esperado."""


class ErroValor(ErroCoralRuntime, ValueError):
    """O tipo é aceito, mas o valor não é válido para a operação."""


class ErroOperacao(ErroOperacaoCoral):
    """Falha operacional com preservação opcional da causa técnica."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
```

```
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroArquivo(ErroOperacao):
    """Falha em uma operação com arquivo ou caminho."""


__all__ = [
    "ErroCoralRuntime",
    "ErroEntrada",
    "FimDeEntrada",
    "ErroConversao",
    "ErroArquivo",
    "ErroFormato",
    "ErroValor",
    "ErroOperacao",
]
```

Biblioteca `coral.datas`

`agora(...)` · módulo `coral.datas`

O que faz

Obtém a data e hora atuais no sistema, com fuso opcional.

Assinatura

Assinatura para consulta

```
agora(fuso: 'str | tzinfo | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
fuso	texto, tzinfo ou nulo	não	None	Fuso explícito; nulo em agora/hoje usa horário local sem informação de fuso.

Retorno

datetime; sem fuso explícito, o resultado é ingênuo, com tzinfo igual a nulo.

Exemplo

Código Coral

```
de coral.datas importe agora
mostre agora("UTC").tzinfo
```

Resultado

Saída esperada

```
UTC
```

Erros

ValueError para nome de fuso desconhecido.

Atenção

O instante varia a cada execução. O exemplo confere o fuso e não fixa uma hora. Nomes IANA exigem dados de fuso disponíveis.

`analisar(...)` · módulo `coral.datas`

O que faz

Converte texto ISO ou texto em formato explícito para data ou data/hora.

Assinatura

Assinatura para consulta

```
analisar(texto: 'str', formato: 'str | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
texto	texto	sim	sem padrão	Texto temporal; espaços externos são removidos.
formato	texto ou nulo	não	None	Formato de strptime; nulo seleciona leitura ISO.

Retorno

date quando o texto ISO não tem T nem espaço; datetime quando há horário ou quando formato é informado.

Exemplo

Código Coral

```
de coral.datas importe analisar
mostre analisar("03/10/2026", "%d/%m/%Y").isoformat()
```

Resultado

Saída esperada

```
2026-10-03T00:00:00
```

Erros

ValueError para texto ou formato temporal inválido.

Atenção

Sem formato, T ou espaço seleciona datetime.fromisoformat; os demais textos usam date.fromisoformat. Z é convertido para +00:00 no ramo de data/hora. Não atribui fuso a texto que não o declara.

`converter_fuso(...)` · módulo `coral.datas`

O que faz

Converte uma data/hora com fuso para outro fuso preservando o instante.

Assinatura

Assinatura para consulta

```
converter_fuso(valor: 'datetime', fuso: 'str | tzinfo') -> 'datetime'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	datetime com tzinfo	sim	sem padrão	Instante a converter; sem fuso não há conversão implícita.
fuso	texto ou tzinfo	sim	sem padrão	Fuso de destino; nomes IANA dependem dos dados de zoneinfo.

Retorno

datetime com o fuso de destino.

Exemplo

Código Coral

```
de coral.datas importe converter_fuso
de coral.datas importe de_iso
mostre converter_fuso(de_iso("2026-10-03T12:00:00+02:00"), "UTC").isoformat()
```

Resultado

Saída esperada

```
2026-10-03T10:00:00+00:00
```

Erros

TypeError para valor que não é datetime; ValueError para datetime sem fuso ou nome de fuso desconhecido.

`de_iso(...)` · módulo `coral.datas`

O que faz

Converte um texto ISO usando as regras de analisar sem formato explícito.

Assinatura

Assinatura para consulta

```
de_iso(texto: 'str')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
texto	texto	sim	sem padrão	Data ou data/hora em formato ISO.

Retorno

date ou datetime, conforme o texto.

Exemplo

Código Coral

```
de coral.datas importe de_iso
mostre de_iso("2026-10-03").isoformat()
```

Resultado

Saída esperada

```
2026-10-03
```

Erros

ValueError para texto temporal inválido.

Atenção

A leitura com horário conserva o deslocamento declarado; sem deslocamento, o datetime resultante não tem fuso.

`dias(...)` · módulo `coral.datas`

O que faz

Cria uma duração medida em dias.

Assinatura

Assinatura para consulta

```
dias(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	inteiro ou decimal	sim	sem padrão	Quantidade de dias; pode ser negativa ou fracionária.

Retorno

timedelta, com precisão de microssegundos.

Exemplo

Código Coral

```
de coral.datas importe dias
mostre dias(1).total_seconds()
```

Resultado

Saída esperada

```
86400.0
```

Erros

TypeError para tipo incompatível; OverflowError para duração fora dos limites de timedelta.

Atenção

Uma duração em dias usa 24 horas por dia. Não representa meses ou anos de calendário nem corrige mudanças de horário de verão.

`diferenca(...)` · módulo `coral.datas`

O que faz

Subtrai o instante ou data inicial do final.

Assinatura

Assinatura para consulta

```
diferenca(final, inicial)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
final	date ou datetime	sim	sem padrão	Data ou instante final.
inicial	date ou datetime compatível com final	sim	sem padrão	Data ou instante inicial.

Retorno

timedelta.

Exemplo

Código Coral

```
de coral.datas importe diferenca
de coral.datas importe de_iso
mostre diferenca(de_iso("2026-10-03"), de_iso("2026-10-01")).days
```

Resultado

Saída esperada

```
2
```

Erros

TypeError para tipos não temporais, mistura date/datetime ou mistura de datetime com e sem fuso.

Atenção

A diferença pode ser negativa e segue as regras de subtração de datetime do Python, inclusive o tratamento de fusos.

`formatar(...)` · módulo `coral.datas`

O que faz

Formata uma data ou data/hora usando strftime.

Assinatura

Assinatura para consulta

```
formatar(valor, formato='%d/%m/%Y')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	date, datetime ou objeto com strftime	sim	sem padrão	Valor temporal a formatar.
formato	texto	não	'%d/%m/%Y'	Diretivas de strftime; o padrão apresenta dia/mês/ano.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.datas importe formatar
de coral.datas importe de_iso
mostre formatar(de_iso("2026-10-03"))
```

Resultado

Saída esperada

```
03/10/2026
```

Erros

AttributeError para objeto sem strftime; TypeError ou ValueError para formato incompatível.

Atenção

Algumas diretivas e nomes de meses/dias dependem do sistema e da configuração de idioma.

`fuso_horario(...)` · módulo `coral.datas`

O que faz

Resolve UTC, Z ou um identificador IANA para objeto de fuso.

Assinatura

Assinatura para consulta

```
fuso_horario(nome: 'str' = 'UTC') -> 'tzinfo'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	texto ou tzinfo	não	'UTC'	Nome de fuso ou objeto tzinfo; UTC e Z são aceitos sem distinguir caixa.

Retorno

tzinfo.

Exemplo

Código Coral

```
de coral.datas importe fuso_horario
mostre fuso_horario("UTC")
```

Resultado

Saída esperada

```
UTC
```

Erros

ValueError para fuso desconhecido ou sem dados disponíveis.

Atenção

Na implementação, nulo resolve para UTC. Nomes IANA dependem de zoneinfo e dos dados de fuso instalados.

`hoje(...)` · módulo `coral.datas`

O que faz

Obtém a data corrente no fuso escolhido.

Assinatura

Assinatura para consulta

```
hoje(fuso: 'str | tzinfo | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
fuso	texto, tzinfo ou nulo	não	None	Fuso explícito; nulo em agora/hoje usa horário local sem informação de fuso.

Retorno

date, sem horário nem informação de fuso.

Exemplo

Código Coral

```
de coral.datas importe hoje
defina data como hoje("UTC")
mostre data.month >= 1
mostre data.month <= 12
```

Resultado

Saída esperada

```
True
True
```

Erros

ValueError para nome de fuso desconhecido.

Atenção

A data pode mudar de um fuso para outro perto da meia noite. Nomes IANA dependem dos dados instalados.

`horas(...)` · módulo `coral.datas`

O que faz

Cria uma duração medida em horas.

Assinatura

Assinatura para consulta

```
horas(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	inteiro ou decimal	sim	sem padrão	Quantidade de horas; pode ser negativa ou fracionária.

Retorno

timedelta, com precisão de microssegundos.

Exemplo

Código Coral

```
de coral.datas importe horas
mostre horas(1).total_seconds()
```

Resultado

Saída esperada

```
3600.0
```

Erros

TypeError para tipo incompatível; OverflowError para duração fora dos limites de timedelta.

Atenção

Uma duração em dias usa 24 horas por dia. Não representa meses ou anos de calendário nem corrige mudanças de horário de verão.

`minutos(...)` · módulo `coral.datas`

O que faz

Cria uma duração medida em minutos.

Assinatura

Assinatura para consulta

```
minutos(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	inteiro ou decimal	sim	sem padrão	Quantidade de minutos; pode ser negativa ou fracionária.

Retorno

timedelta, com precisão de microssegundos.

Exemplo

Código Coral

```
de coral.datas importe minutos
mostre minutos(1).total_seconds()
```

Resultado

Saída esperada

```
60.0
```

Erros

TypeError para tipo incompatível; OverflowError para duração fora dos limites de timedelta.

Atenção

Uma duração em dias usa 24 horas por dia. Não representa meses ou anos de calendário nem corrige mudanças de horário de verão.

`para_iso(...)` · módulo `coral.datas`

O que faz

Serializa uma data ou data/hora para texto ISO.

Assinatura

Assinatura para consulta

```
para_iso(valor, *, vezespec='auto') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	date ou datetime	sim	sem padrão	Valor a serializar.
vezespec	texto	não	'auto'	Precisão do horário: auto, hours, minutes, seconds, milliseconds ou microseconds.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.datas importe para_iso
de coral.datas importe de_iso
mostre para_iso(de_iso("2026-10-03T12:30:45Z"), vezespec="seconds")
```

Resultado

Saída esperada

```
2026-10-03T12:30:45+00:00
```

Erros

TypeError para valor que não é date nem datetime; ValueError para vezespec inválido em datetime.

Atenção

vezespec só é aplicado a datetime. Precisões menores truncam componentes; não arredondam. Uma date ignora vezespec.

`segundos(...)` · módulo `coral.datas`

O que faz

Cria uma duração medida em segundos.

Assinatura

Assinatura para consulta

```
segundos(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	inteiro ou decimal	sim	sem padrão	Quantidade de segundos; pode ser negativa ou fracionária.

Retorno

timedelta, com precisão de microssegundos.

Exemplo

Código Coral

```
de coral.datas importe segundos
mostre segundos(1).total_seconds()
```

Resultado

Saída esperada

```
1.0
```

Erros

TypeError para tipo incompatível; OverflowError para duração fora dos limites de timedelta.

Atenção

Uma duração em dias usa 24 horas por dia. Não representa meses ou anos de calendário nem corrige mudanças de horário de verão.

`semanas(...)` · módulo `coral.datas`

O que faz

Cria uma duração medida em semanas.

Assinatura

Assinatura para consulta

```
semanas(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	inteiro ou decimal	sim	sem padrão	Quantidade de semanas; pode ser negativa ou fracionária.

Retorno

timedelta, com precisão de microssegundos.

Exemplo

Código Coral

```
de coral.datas importe semanas
mostre semanas(1).total_seconds()
```

Resultado

Saída esperada

```
604800.0
```

Erros

TypeError para tipo incompatível; OverflowError para duração fora dos limites de timedelta.

Atenção

Uma duração em dias usa 24 horas por dia. Não representa meses ou anos de calendário nem corrige mudanças de horário de verão.

Consultar a implementação canônica de coral.datas

Implementação canônica de `coral.datas`

Código Python

```
"""Superfície pública de coral.datas."""
from .stdlib.datas import *  # noqa: F401,F403

from __future__ import annotations

from datetime import date, datetime, timedelta, timezone, tzinfo
from zoneinfo import ZoneInfo, ZoneInfoNotFoundError


def agora(fuso: str | tzinfo | None = None):
    return datetime.now(_resolver_fuso(fuso)) if fuso is not None else datetime.now()


def hoje(fuso: str | tzinfo | None = None):
    return agora(fuso).date() if fuso is not None else date.today()


def dias(valor): return timedelta(days=valor)
def segundos(valor): return timedelta(seconds=valor)
def minutos(valor): return timedelta(minutes=valor)
def horas(valor): return timedelta(hours=valor)
def semanas(valor): return timedelta(weeks=valor)



def formatar(valor, formato="%d/%m/%Y"):
    return valor.strftime(formato)


def _resolver_fuso(fuso: str | tzinfo | None) -> tzinfo:
    if isinstance(fuso, tzinfo):
        return fuso
    if fuso is None:
        return timezone.utc
    nome = str(fuso).strip()
    if nome.upper() in {"UTC", "Z"}:
        return timezone.utc
    try:
        return ZoneInfo(nome)
    except ZoneInfoNotFoundError as erro:
        raise ValueError(f"fuso horário desconhecido: {nome}") from erro


def fuso_horario(nome: str = "UTC") -> tzinfo:
    return _resolver_fuso(nome)


def analisar(texto: str, formato: str | None = None):
    bruto = str(texto).strip()
    if formato is not None:
        return datetime.strptime(bruto, formato)
    try:
        if "T" in bruto or " " in bruto:
            return datetime.fromisoformat(bruto.replace("Z", "+00:00"))
        return date.fromisoformat(bruto)
    except ValueError as erro:
        raise ValueError(f"data ou data/hora inválida: {bruto!r}") from erro


def de_iso(texto: str):
```

```
    return analisar(texto)


def para_iso(valor, *, vezespec="auto") -> str:
    if isinstance(valor, datetime):
        return valor.isoformat(timespec=vezespec)
    if isinstance(valor, date):
        return valor.isoformat()
    raise TypeError("valor precisa ser data ou data/hora")


def converter_fuso(valor: datetime, fuso: str | tzinfo) -> datetime:
    if not isinstance(valor, datetime):
        raise TypeError("valor precisa ser data/hora")
    if valor.tzinfo is None:
        raise ValueError("data/hora sem fuso não pode ser convertida implicitamente")
    return valor.astimezone(_resolver_fuso(fuso))


def diferenca(final, inicial):
    if not isinstance(final, (date, datetime)) or not isinstance(inicial, (date,
      datetime)):
        raise TypeError("diferença exige datas ou datas/horas compatíveis")
    return final - inicial


__all__ = [
    "agora", "hoje", "dias", "segundos", "minutos", "horas", "semanas", "formatar",
    "analisar", "de_iso", "para_iso", "fuso_horario", "converter_fuso", "diferenca",
]
```

Biblioteca `coral.entrada`

`coral.entrada.FimDeEntrada`

A fonte de entrada terminou antes de produzir outra linha.

Assinatura

Assinatura para consulta

```
FimDeEntrada(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.entrada.FonteEntrada`

Base class for protocol classes.

Protocol classes are defined as::

class Proto(Protocol): def meth(self) -> int: ...

Such classes are primarily used with static type checkers that recognize structural subtyping (static duck-typing).

For example::

class C: def meth(self) -> int: return 0

def func(x: Proto) -> int: return x.meth()

func(C()) # Passes static type check

See PEP 544 for details. Protocol classes decorated with @typing.runtime_checkable act as simple-minded runtime protocols that check only the presence of given attributes, ignoring their type signatures. Protocol classes can be generic, they are defined as::

class GenProtoT (Protocol): def meth(self) -> T: ...

Assinatura

Assinatura para consulta

```
FonteEntrada(*args, **kwargs)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
args	sem anotação	VAR_POSITIONAL	não	sem padrão
kwargs	sem anotação	VAR_KEYWORD	não	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
ler_linha(self, mensagem: 'str' = '') -> 'str'
```

Método público sem descrição própria no runtime.

`coral.entrada.FonteEntradaFuncao`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
FonteEntradaFuncao(*args, **kwargs)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
args	sem anotação	VAR_POSITIONAL	não	sem padrão
kwargs	sem anotação	VAR_KEYWORD	não	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`FonteEntradaSequencial(...)` · módulo `coral.entrada`

O que faz

Fornece linhas pré definidas para repetir uma leitura de entrada em testes.

Construtor

Assinatura para consulta

```
FonteEntradaSequencial(linhas: 'Iterable[str]') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
linhas	iterável de textos	sim	sem padrão	Linhas fornecidas à fonte em ordem de leitura.

Atributos importantes

Atributo	Significado
`linhas`	Coleção de textos fornecida ao criar a fonte.

Métodos principais

Assinatura para consulta

```
ler_linha(self, mensagem: 'str' = '') -> 'str'
```

Retorno

Instância de FonteEntradaSequencial.

Exemplo

Código Coral

```
de coral.entrada importe FonteEntradaSequencial
defina fonte como FonteEntradaSequencial(["Ana"])
mostre fonte.ler_linha()
```

Resultado

Saída esperada

```
Ana
```

Erros

FimDeEntrada quando não há mais linhas.

`ler_linha(...)` · módulo `coral.entrada`

O que faz

Lê uma linha da fonte explícita, do contexto de entrada ou do terminal.

Assinatura

Assinatura para consulta

```
ler_linha(mensagem: 'str | None' = None, *, fonte: 'FonteEntrada | FonteEntradaFuncao |
  None' = None) -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mensagem	texto ou nulo	não	None	Texto mostrado antes de ler a entrada; nulo equivale a texto vazio.
fonte	FonteEntrada ou FonteEntradaFuncao ou nulo	não	None	Fonte explícita de entrada; omitida, usa o contexto ou o terminal.

Retorno

Texto sem a quebra de linha final.

Exemplo

Código Coral

```
de coral.entrada importe ler_linha
mostre ler_linha("Nome: ")
```

Resultado

Saída esperada

```
Nome: Ana
```

Forma natural

`leia uma linha com "Nome: "` tem o mesmo contrato. Veja o Cap. 22.

Erros

FimDeEntrada quando a fonte termina; não é um texto vazio.

`usar_fonte_entrada(...)` · módulo `coral.entrada`

O que faz

Escolhe uma fonte de entrada apenas durante um bloco de contexto.

Assinatura

Assinatura para consulta

```
usar_fonte_entrada(fonte: 'FonteEntrada | FonteEntradaFuncao')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
fonte	FonteEntrada	sim	sem padrão	Fonte usada temporariamente no contexto de entrada.

Retorno

Gerenciador de contexto; entrar nele devolve a fonte.

Exemplo

Código Coral

```
de coral.entrada importe FonteEntradaSequencial, usar_fonte_entrada, ler_linha
defina fonte como FonteEntradaSequencial(["Ana"])
com usar_fonte_entrada(fonte) faça
    mostre ler_linha()
fim
```

Resultado

Saída esperada

```
Ana
```

Atenção

Restaura a fonte anterior ao sair do bloco, inclusive em caso de erro.

Consultar a implementação canônica de coral.entrada

Implementação canônica de `coral.entrada`

Código Python

```
from __future__ import annotations

import builtins
from collections.abc import Callable, Iterable, Iterator
from contextlib import contextmanager
from contextvars import ContextVar
from dataclasses import dataclass, field
from typing import Protocol, runtime_checkable

from .erros import FimDeEntrada


@runtime_checkable
class FonteEntrada(Protocol):
    def ler_linha(self, mensagem: str = "") -> str:
        ...


FonteEntradaFuncao = Callable[[str], str]
_fonte_contextual: ContextVar[FonteEntrada | FonteEntradaFuncao | None] = ContextVar(
    "coral_fonte_entrada", default=None
)



def _ler_terminal(mensagem: str) -> str:
    # A resolução é feita no momento da chamada para que DAPs e outros hosts
    # possam interceptar builtins.input sem o módulo Coral guardar uma cópia antiga.
    return builtins.input(mensagem)


def _chamar_fonte(fonte: FonteEntrada | FonteEntradaFuncao, mensagem: str) -> str:
    if isinstance(fonte, FonteEntrada):
        return fonte.ler_linha(mensagem)
    return fonte(mensagem)


def ler_linha(
    mensagem: str | None = None,
    *,
    fonte: FonteEntrada | FonteEntradaFuncao | None = None,
) -> str:
    """Lê uma linha usando a fonte explícita, contextual ou o terminal.

    EOF é convertido em ``FimDeEntrada`` para que programas Coral não precisem
    depender diretamente de ``EOFError``.
    """

    prompt = "" if mensagem is None else str(mensagem)
    leitor = fonte or _fonte_contextual.get()
    try:
        valor = _ler_terminal(prompt) if leitor is None else _chamar_fonte(leitor,
          prompt)
    except FimDeEntrada:
        raise
    except EOFError as erro:
        raise FimDeEntrada("a fonte de entrada terminou", causa=erro) from erro
    if valor is None:
        raise FimDeEntrada("a fonte de entrada terminou")
    return str(valor).rstrip("\r\n")
```

```
@contextmanager
def usar_fonte_entrada(fonte: FonteEntrada | FonteEntradaFuncao):
    """Instala temporariamente uma fonte de entrada no contexto corrente."""

    token = _fonte_contextual.set(fonte)
    try:
        yield fonte
    finally:
        _fonte_contextual.reset(token)


@dataclass
class FonteEntradaSequencial:
    """Fonte determinística útil em testes, REPLs e ferramentas."""

    linhas: Iterable[str]
    _iterador: Iterator[str] = field(init=False, repr=False)

    def __post_init__(self) -> None:
        self._iterador = iter(self.linhas)

    def ler_linha(self, mensagem: str = "") -> str:
        del mensagem
        try:
            return str(next(self._iterador)).rstrip("\r\n")
        except StopIteration as erro:
            raise FimDeEntrada("a fonte de entrada terminou", causa=erro) from erro


__all__ = [
    "FonteEntrada",
    "FonteEntradaFuncao",
    "FonteEntradaSequencial",
    "FimDeEntrada",
    "ler_linha",
    "usar_fonte_entrada",
]

from __future__ import annotations

from typing import Any

from .comum import ErroCoral, ErroOperacaoCoral


class ErroCoralRuntime(ErroCoral):
    """Base dos erros controlados expostos pelo runtime base da Coral."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroEntrada(ErroCoralRuntime):
    """Falha ao obter entrada para um programa Coral."""


class FimDeEntrada(ErroEntrada, EOFError):
```

```
    """A fonte de entrada terminou antes de produzir outra linha."""


class ErroConversao(ErroCoralRuntime, ValueError):
    """Um valor não pôde ser convertido para o tipo Coral solicitado."""

    def __init__(
        self,
        valor: Any,
        destino: str,
        mensagem: str | None = None,
        *,
        causa: BaseException | None = None,
    ):
        self.valor = valor
        self.destino = destino
        super().__init__(
            mensagem or f"não foi possível converter o valor para {destino}",
            causa=causa,
        )


class ErroFormato(ErroCoralRuntime, ValueError):
    """Dados ou texto não obedecem ao formato esperado."""


class ErroValor(ErroCoralRuntime, ValueError):
    """O tipo é aceito, mas o valor não é válido para a operação."""


class ErroOperacao(ErroOperacaoCoral):
    """Falha operacional com preservação opcional da causa técnica."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroArquivo(ErroOperacao):
    """Falha em uma operação com arquivo ou caminho."""


__all__ = [
    "ErroCoralRuntime",
    "ErroEntrada",
    "FimDeEntrada",
    "ErroConversao",
    "ErroArquivo",
    "ErroFormato",
    "ErroValor",
    "ErroOperacao",
]
```

Biblioteca `coral.erros`

`coral.erros.ErroArquivo`

Falha em uma operação com arquivo ou caminho.

Assinatura

Assinatura para consulta

```
ErroArquivo(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.erros.ErroConversao`

Um valor não pôde ser convertido para o tipo Coral solicitado.

Assinatura

Assinatura para consulta

```
ErroConversao(valor: 'Any', destino: 'str', mensagem: 'str | None' = None, *, causa:
  'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
destino	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str | None	POSITIONAL_OR_KEYWORD	não	None
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.erros.ErroCoralRuntime`

Base dos erros controlados expostos pelo runtime base da Coral.

Assinatura

Assinatura para consulta

```
ErroCoralRuntime(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.erros.ErroEntrada`

Falha ao obter entrada para um programa Coral.

Assinatura

Assinatura para consulta

```
ErroEntrada(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.erros.ErroFormato`

Dados ou texto não obedecem ao formato esperado.

Assinatura

Assinatura para consulta

```
ErroFormato(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.erros.ErroOperacao`

Falha operacional com preservação opcional da causa técnica.

Assinatura

Assinatura para consulta

```
ErroOperacao(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.erros.ErroValor`

O tipo é aceito, mas o valor não é válido para a operação.

Assinatura

Assinatura para consulta

```
ErroValor(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.erros.FimDeEntrada`

A fonte de entrada terminou antes de produzir outra linha.

Assinatura

Assinatura para consulta

```
FimDeEntrada(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

Consultar a implementação canônica de coral.erros

Implementação canônica de `coral.erros`

Código Python

```
"""Contratos transversais pequenos do runtime Coral.

A camada comum não substitui APIs de domínio. Ela fornece a gramática
arquitetural compartilhada do ciclo 1.5 para erro operacional, resultado e
diagnóstico, preservando os nomes públicos existentes em cada biblioteca.
"""
from __future__ import annotations

from dataclasses import asdict, dataclass, field
from typing import Any, Iterable, Mapping

CONTRATO = "coral.comum/1"
CONTRATO_DIAGNOSTICO = "coral.diagnostico.operacional/1"


class ErroCoral(Exception):
    """Raiz pública dos erros controlados pela linguagem e bibliotecas Coral."""


class ErroOperacaoCoral(RuntimeError, ErroCoral):
    """Falha operacional de um recurso Coral ou de seu backend."""


class ErroDependenciaCoral(ErroOperacaoCoral):
    """Uma capacidade opcional não pode operar por ausência de dependência."""


@dataclass(frozen=True)
class FalhaOperacao:
    codigo: str
    mensagem: str
    dominio: str
    tipo: str = "operacional"
    causa: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def de_excecao(cls, exc: BaseException, *, codigo: str, dominio: str, tipo: str =
      "operacional", detalhe: Mapping[str, Any] | None = None) -> "FalhaOperacao":
        causa = exc.__cause__ if exc.__cause__ is not None else exc
        return cls(codigo, str(exc), dominio, tipo, type(causa).__name__, dict(detalhe
          or {}))

    def para_dict(self) -> dict[str, Any]:
        return asdict(self)


@dataclass(frozen=True)
class ResultadoOperacao:
    """Representação transversal sem substituir resultados ricos de domínio."""
    ok: bool
    valor: Any = None
    falha: FalhaOperacao | None = None
```

```
    metadados: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def sucesso(cls, valor: Any = None, **metadados: Any) -> "ResultadoOperacao":
        return cls(True, valor=valor, metadados=metadados)

    @classmethod
    def erro(cls, falha: FalhaOperacao, **metadados: Any) -> "ResultadoOperacao":
        return cls(False, falha=falha, metadados=metadados)

    def para_dict(self) -> dict[str, Any]:
        return {
            "contrato": CONTRATO,
            "ok": self.ok,
            "valor": self.valor,
            "falha": self.falha.para_dict() if self.falha is not None else None,
            "metadados": dict(self.metadados),
        }


@dataclass(frozen=True)
class DiagnosticoOperacional:
    id: str
    dominio: str
    ok: bool
    mensagem: str
    capacidade: str | None = None
    sugestao: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @property
    def nivel(self) -> str:
        return "info" if self.ok else "erro"

    def para_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "dominio": self.dominio,
            "ok": self.ok,
            "nivel": self.nivel,
            "mensagem": self.mensagem,
            "capacidade": self.capacidade,
            "sugestao": self.sugestao,
            "detalhe": dict(self.detalhe),
        }


def diagnostico(id: str, dominio: str, ok: bool, mensagem: Any, *, capacidade: str |
  None = None, sugestao: str | None = None, detalhe: Mapping[str, Any] | None = None) ->
  DiagnosticoOperacional:
    return DiagnosticoOperacional(str(id), str(dominio), bool(ok), str(mensagem),
      capacidade, sugestao, dict(detalhe or {}))


def agregar_diagnosticos(itens: Iterable[DiagnosticoOperacional | Mapping[str, Any]], *,
   origem: str = "coral") -> dict[str, Any]:
    normalizados: list[dict[str, Any]] = []
    for item in itens:
        if isinstance(item, DiagnosticoOperacional):
            normalizados.append(item.para_dict())
            continue
```

```
        d = dict(item)
        if "id" not in d:
            d["id"] = str(d.get("nome", "diagnostico"))
        if "dominio" not in d:
            d["dominio"] = origem
        d["ok"] = bool(d.get("ok"))
        d.setdefault("nivel", "info" if d["ok"] else "erro")
        d.setdefault("mensagem", str(d.get("detalhe", "")))
        d.setdefault("capacidade", None)
        d.setdefault("sugestao", None)
        detalhe = d.get("detalhe")
        if not isinstance(detalhe, Mapping):
            d["detalhe"] = {"valor": detalhe}
        normalizados.append(d)
    return {
        "contrato": CONTRATO_DIAGNOSTICO,
        "origem": origem,
        "ok": all(x["ok"] for x in normalizados),
        "quantidade": len(normalizados),
        "itens": normalizados,
    }


__all__ = [
    "CONTRATO", "CONTRATO_DIAGNOSTICO", "ErroCoral", "ErroOperacaoCoral",
    "ErroDependenciaCoral", "FalhaOperacao", "ResultadoOperacao",
    "DiagnosticoOperacional", "diagnostico", "agregar_diagnosticos",
]

from __future__ import annotations

from typing import Any

from .comum import ErroCoral, ErroOperacaoCoral


class ErroCoralRuntime(ErroCoral):
    """Base dos erros controlados expostos pelo runtime base da Coral."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroEntrada(ErroCoralRuntime):
    """Falha ao obter entrada para um programa Coral."""


class FimDeEntrada(ErroEntrada, EOFError):
    """A fonte de entrada terminou antes de produzir outra linha."""


class ErroConversao(ErroCoralRuntime, ValueError):
    """Um valor não pôde ser convertido para o tipo Coral solicitado."""

    def __init__(
        self,
        valor: Any,
        destino: str,
        mensagem: str | None = None,
```

```
        *,
        causa: BaseException | None = None,
    ):
        self.valor = valor
        self.destino = destino
        super().__init__(
            mensagem or f"não foi possível converter o valor para {destino}",
            causa=causa,
        )


class ErroFormato(ErroCoralRuntime, ValueError):
    """Dados ou texto não obedecem ao formato esperado."""


class ErroValor(ErroCoralRuntime, ValueError):
    """O tipo é aceito, mas o valor não é válido para a operação."""


class ErroOperacao(ErroOperacaoCoral):
    """Falha operacional com preservação opcional da causa técnica."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroArquivo(ErroOperacao):
    """Falha em uma operação com arquivo ou caminho."""


__all__ = [
    "ErroCoralRuntime",
    "ErroEntrada",
    "FimDeEntrada",
    "ErroConversao",
    "ErroArquivo",
    "ErroFormato",
    "ErroValor",
    "ErroOperacao",
]
```

Biblioteca `coral.esquemas`

`campo(...)` · módulo `coral.esquemas`

O que faz

Cria a especificação declarativa de um campo; não converte nem preenche dados.

Assinatura

Assinatura para consulta

```
campo(tipo: 'str', *, obrigatorio: 'bool' = True, anulavel: 'bool' = False, minimo: 'int
   | float | None' = None, maximo: 'int | float | None' = None, tamanho_minimo: 'int |
  None' = None, tamanho_maximo: 'int | None' = None, permitidos: 'Iterable | None' = None)
   -> 'dict'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo	texto	sim	sem padrão	Tipo canônico: inteiro, decimal, texto, booleano, lista ou dicionario.
obrigatorio	booleano	não	True	Exige presença da chave; verdadeiro por padrão.
anulavel	booleano	não	False	Aceita nulo quando a chave está presente; falso por padrão.
minimo	inteiro ou decimal finito ou nulo	não	None	Limite numérico inferior inclusivo; nulo desativa.
maximo	inteiro ou decimal finito ou nulo	não	None	Limite numérico superior inclusivo; nulo desativa.
tamanho_minimo	inteiro não negativo, exceto booleano, ou nulo	não	None	Comprimento mínimo inclusivo; nulo desativa.
tamanho_maximo	inteiro não negativo, exceto booleano, ou nulo	não	None	Comprimento máximo inclusivo; nulo desativa.
permitidos	iterável de valores ou nulo	não	None	Iterável de valores aceitos; nulo desativa, coleção vazia não aceita nenhum valor não nulo.

Retorno

Dicionário de especificação, copiado e novamente conferido pelo consumidor.

Exemplo

Código Coral

```
de coral.esquemas importe campo
defina idade como campo("inteiro", minimo=0, maximo=120)
mostre idade["tipo"]
```

Resultado

Saída esperada

```
inteiro
```

Forma natural

Não há forma natural específica para esta API. Use a chamada explícita importada de coral.esquemas; não há nova gramática.

Erros

TypeError para tipos de opções inválidos, incluindo limites numéricos não finitos ou booleanos; ValueError para tipo canônico desconhecido, tamanho negativo, regras incompatíveis ou limites contraditórios.

Atenção

Inteiro rejeita booleanos; decimal aceita inteiro/decimal finitos e rejeita booleanos. Regras numéricas só servem para inteiro/decimal; comprimento só para texto/lista/dicionario.

Limites são inclusivos. Ausência e nulo são independentes: opcional permite ausência, anulavel permite nulo presente. Nulo aceito não percorre regras de valor. Especificações não retêm referências mutáveis do chamador.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Esquemas_1_7_8.md; Exemplos/Projetos/Importador_Validado.

`esquema(...)` · módulo `coral.esquemas`

O que faz

Compõe campos em um esquema de registro com ordem declarada de diagnóstico.

Assinatura

Assinatura para consulta

```
esquema(campos: 'Mapping[str, dict]', *, permitir_extras: 'bool' = True) -> 'dict'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
campos	mapeamento texto/especificação de campo	sim	sem padrão	Mapeamento de nomes textuais não vazios para especificações de campo.
permitir_extras	booleano	não	True	Permite chaves fora da declaração; verdadeiro por padrão.

Retorno

Dicionário de esquema com especificações copiadas; sem dados do registro.

Exemplo

Código Coral

```
de coral.esquemas importe campo, esquema, validar_registro
defina cadastro como esquema({"nome": campo("texto", tamanho_minimo=1)},
  permitir_extras=falso)
mostre validar_registro({"nome": "Ana"}, cadastro)["valido"]
```

Resultado

Saída esperada

```
True
```

Forma natural

Não há forma natural específica para esta API. Use a chamada explícita importada de coral.esquemas; não há nova gramática.

Erros

TypeError para campos ou opções de tipo inválido; ValueError para nome vazio ou especificação inválida, incluindo regras e limites contraditórios.

Atenção

A ordem dos campos declarados define a ordem dos erros; extras recusados aparecem depois, na ordem do registro. Não há campos aninhados, preenchimento padrão, conversão ou mutação dos dados. Use especificações produzidas por campo. A conferência ocorre também ao consumir o esquema.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Esquemas_1_7_8.md; Exemplos/Projetos/Importador_Validado.

`validar_registro(...)` · módulo `coral.esquemas`

O que faz

Confere um registro e relata falhas sem incluir o valor sensível nem modificar a entrada.

Assinatura

Assinatura para consulta

```
validar_registro(registro: 'Mapping', esquema: 'Mapping', *, indice: 'int' = 1, linha:
  'int | None' = None, origem: 'str | None' = None, modo: 'str' = 'coletar', limite_erros:
   'int' = 100) -> 'dict'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
registro	mapeamento	sim	sem padrão	Mapeamento com os dados; outra entrada resulta em erro de regra registro.
esquema	mapeamento de esquema	sim	sem padrão	Esquema criado por esquema e reconferido no consumo.
indice	inteiro positivo, exceto booleano	não	1	Identificador lógico do registro, inteiro positivo base 1; padrão 1.
linha	inteiro positivo, exceto booleano, ou nulo	não	None	Linha informada pelo chamador, inteiro positivo ou nulo; não inferida.
origem	texto ou nulo	não	None	Rótulo textual opcional fornecido pelo chamador; nunca inferido.
modo	texto: primeiro ou coletar	não	'coletar'	primeiro interrompe no primeiro erro; coletar usa o limite global.
limite_erros	inteiro positivo, exceto booleano	não	100	Máximo positivo de erros; inteiro excluindo booleano, padrão 100.

Retorno

Dicionário com valido (booleano), erros (lista) e interrompido (booleano). Cada erro contém campo, regra, mensagem, registro (índice base 1), linha e origem; não contém valor.

Exemplo

Código Coral

```
de coral.esquemas importe campo, esquema, validar_registro
defina contrato como esquema({"apelido": campo("texto", obrigatorio=falso)})
mostre validar_registro({}, contrato)["valido"]
defina resultado como validar_registro({"apelido": nulo}, contrato, indice=3, linha=8,
  origem="cadastro.json")
mostre resultado["erros"][0]["regra"]
mostre resultado["erros"][0]["registro"]
mostre resultado["erros"][0]["linha"]
```

Resultado

Saída esperada

```
True
nulo
3
8
```

Forma natural

Não há forma natural específica para esta API. Use a chamada explícita importada de coral.esquemas; não há nova gramática.

Erros

TypeError/ValueError para esquema ou opções inválidos. Dados inválidos são diagnósticos, não exceções: registro, ausente, nulo, tipo, minimo, maximo, tamanho_minimo, tamanho_maximo, permitido e extra.

Atenção

Tipo errado encerra a avaliação daquele campo. Ausente opcional é válido; nulo presente exige anulavel. primeiro e coletar ao alcançar limite marcam interrompido, sem afirmar exame do restante. Erros seguem ordem declarada e depois extras. Não há conversão; nenhum dado de entrada é alterado.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Esquemas_1_7_8.md; Exemplos/Projetos/Importador_Validado.

`validar_registros(...)` · módulo `coral.esquemas`

O que faz

Valida um iterável em passagem única e acumula apenas erros, com limite global.

Assinatura

Assinatura para consulta

```
validar_registros(registros: 'Iterable[Mapping]', esquema: 'Mapping', *, modo: 'str' =
  'coletar', limite_erros: 'int' = 100, linha_inicial: 'int | None' = None, origem: 'str |
   None' = None) -> 'dict'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
registros	iterável de mapeamentos	sim	sem padrão	Iterável de registros; cada item não mapeamento produz erro registro.
esquema	mapeamento de esquema	sim	sem padrão	Esquema validado antes de avançar a entrada.
modo	texto: primeiro ou coletar	não	'coletar'	primeiro interrompe no primeiro erro; coletar usa o limite global.
limite_erros	inteiro positivo, exceto booleano	não	100	Máximo positivo de erros; inteiro excluindo booleano, padrão 100.
linha_inicial	inteiro positivo, exceto booleano, ou nulo	não	None	Linha lógica do primeiro registro; nulo omite linha. O índice seguinte soma 1.
origem	texto ou nulo	não	None	Rótulo textual opcional fornecido pelo chamador; nunca inferido.

Retorno

Dicionário com valido (booleano), erros (lista), verificados (número de registros consumidos) e interrompido (booleano). Os erros têm campo, regra, mensagem, registro, linha e origem.

Exemplo

Código Coral

```
de coral.esquemas importe campo, esquema, validar_registros
defina inventario como esquema({"quantidade": campo("inteiro", minimo=0)})
defina resultado como validar_registros([{"quantidade": -1}, {"quantidade": -2},
  {"quantidade": 4}], inventario, limite_erros=1, linha_inicial=2,
  origem="inventario.csv")
mostre resultado["verificados"]
mostre resultado["interrompido"]
mostre resultado["erros"][0]["linha"]
```

Resultado

Saída esperada

```
1
True
2
```

Forma natural

Não há forma natural específica para esta API. Use a chamada explícita importada de coral.esquemas; não há nova gramática.

Erros

TypeError/ValueError para esquema, iterável ou opções inválidos; esquema e opções falham antes do primeiro consumo. Falhas do iterador de entrada propagam ao chamador. Dados incompatíveis produzem as regras de validar_registro.

Atenção

Índice de registro é base 1 e linha = linha_inicial + índice - 1. Isso é contagem lógica, não linha física de CSV multilinha. origem não é inferida. Limite positivo é global; nenhum registro adicional é lido após parar. Não guarda registros nem fecha iteradores de terceiros; o chamador mantém o contexto com e o fechamento. Consumo não altera dados; conversão é explícita antes da validação.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Esquemas_1_7_8.md; Exemplos/Projetos/Importador_Validado.

Consultar a implementação canônica de coral.esquemas

Implementação canônica de `coral.esquemas`

Código Python

```
"""Superfície pública de coral.esquemas."""
from .stdlib.esquemas import *  # noqa: F401,F403
from .stdlib.esquemas import __all__

"""Validação declarativa sem conversão ou alteração dos registros externos."""
from __future__ import annotations

from collections.abc import Iterable, Mapping
from copy import deepcopy
import math

__all__ = ['campo', 'esquema', 'validar_registro', 'validar_registros']
_TIPOS = {'inteiro', 'decimal', 'texto', 'booleano', 'lista', 'dicionario'}
_CHAVES_CAMPO = {'tipo', 'obrigatorio', 'anulavel', 'minimo', 'maximo',
                 'tamanho_minimo', 'tamanho_maximo', 'permitidos'}


def _numero(valor):
    return isinstance(valor, (int, float)) and not isinstance(valor, bool) and (not
      isinstance(valor, float) or math.isfinite(valor))


def _inteiro_positivo(valor, nome, *, opcional=False):
    if opcional and valor is None:

        return
    if not isinstance(valor, int) or isinstance(valor, bool):
        raise TypeError(f'{nome} deve ser inteiro')
    if valor < 1:
        raise ValueError(f'{nome} deve ser positivo')


def campo(tipo: str, *, obrigatorio: bool = True, anulavel: bool = False,
          minimo: int | float | None = None, maximo: int | float | None = None,
          tamanho_minimo: int | None = None, tamanho_maximo: int | None = None,
          permitidos: Iterable | None = None) -> dict:
    """Define um campo: ausente depende de obrigatorio; nulo depende de anulavel.

    Tipos: inteiro, decimal, texto, booleano, lista, dicionario. Sem coerção.
    Limites são inclusivos; permitidos vazio recusa qualquer valor não nulo.
    Retorna uma cópia independente. Configuração inválida: TypeError/ValueError.
    """
    if not isinstance(tipo, str):
```

```
        raise TypeError('tipo deve ser texto')
    if tipo not in _TIPOS:
        raise ValueError('tipo desconhecido')
    for nome, valor in [('obrigatorio', obrigatorio), ('anulavel', anulavel)]:
        if not isinstance(valor, bool):
            raise TypeError(f'{nome} deve ser booleano')
    for nome, valor in [('minimo', minimo), ('maximo', maximo)]:
        if valor is not None:
            if tipo not in {'inteiro', 'decimal'}:
                raise ValueError('limite numérico requer inteiro ou decimal')
            if not _numero(valor):
                raise TypeError(f'{nome} deve ser número finito, excluindo booleano')
    for nome, valor in [('tamanho_minimo', tamanho_minimo), ('tamanho_maximo',
      tamanho_maximo)]:
        if valor is not None:
            if tipo not in {'texto', 'lista', 'dicionario'}:
                raise ValueError('limite de tamanho requer texto, lista ou dicionario')
            if not isinstance(valor, int) or isinstance(valor, bool):
                raise TypeError(f'{nome} deve ser inteiro')
            if valor < 0:
                raise ValueError(f'{nome} deve ser não negativo')
    if minimo is not None and maximo is not None and minimo > maximo:
        raise ValueError('minimo maior que maximo')
    if tamanho_minimo is not None and tamanho_maximo is not None and tamanho_minimo >
      tamanho_maximo:
        raise ValueError('tamanho_minimo maior que tamanho_maximo')
    return {'tipo': tipo, 'obrigatorio': obrigatorio, 'anulavel': anulavel,
            'minimo': minimo, 'maximo': maximo, 'tamanho_minimo': tamanho_minimo,
            'tamanho_maximo': tamanho_maximo,
            'permitidos': None if permitidos is None else deepcopy(list(permitidos))}


def _copiar_campo(especificacao):
    if not isinstance(especificacao, Mapping):
        raise TypeError('especificação de campo deve ser Mapping')
    if set(especificacao) != _CHAVES_CAMPO:
        raise ValueError('especificação deve conter as chaves produzidas por campo')
    return campo(**especificacao)


def esquema(campos: Mapping[str, dict], *, permitir_extras: bool = True) -> dict:
    """Copia campos declarados em ordem; extras são permitidos por padrão.

    Cada especificação vem de campo. Ausente difere de nulo. Não há defaults,
    campos aninhados ou alteração da entrada. Configuração inválida falha cedo.
    """
    if not isinstance(campos, Mapping):
        raise TypeError('campos deve ser Mapping')
    if not isinstance(permitir_extras, bool):
        raise TypeError('permitir_extras deve ser booleano')
    copia = {}
    for nome, especificacao in campos.items():
        if not isinstance(nome, str):
            raise TypeError('nome de campo deve ser texto')
        if not nome:
            raise ValueError('nome de campo não pode ser vazio')
        copia[nome] = _copiar_campo(especificacao)
    return {'campos': copia, 'permitir_extras': permitir_extras}


def _copiar_esquema(definicao):
```

```
    if not isinstance(definicao, Mapping):
        raise TypeError('esquema deve ser Mapping')
    if set(definicao) != {'campos', 'permitir_extras'}:
        raise ValueError('esquema deve conter as chaves produzidas por esquema')
    return esquema(**definicao)


def _opcoes(modo, limite_erros, origem):
    if not isinstance(modo, str):
        raise TypeError('modo deve ser texto')
    if modo not in {'primeiro', 'coletar'}:
        raise ValueError('modo deve ser primeiro ou coletar')
    _inteiro_positivo(limite_erros, 'limite_erros')
    if origem is not None and not isinstance(origem, str):
        raise TypeError('origem deve ser texto ou None')


def _tipo_correto(valor, tipo):
    if tipo == 'inteiro':
        return isinstance(valor, int) and not isinstance(valor, bool)
    if tipo == 'decimal':
        return _numero(valor)
    return isinstance(valor, {'texto': str, 'booleano': bool, 'lista': list,
      'dicionario': dict}[tipo])


def _erros_registro(registro, definicao):
    if not isinstance(registro, Mapping):
        yield None, 'registro', 'Registro deve ser um Mapping.'
        return
    for nome, c in definicao['campos'].items():
        if nome not in registro:
            if c['obrigatorio']:
                yield nome, 'ausente', 'Campo obrigatório ausente.'
            continue
        valor = registro[nome]
        if valor is None:
            if not c['anulavel']:
                yield nome, 'nulo', 'Campo não permite nulo.'
            continue
        if not _tipo_correto(valor, c['tipo']):
            yield nome, 'tipo', 'Tipo de campo incompatível.'
            continue
        for regra, desc in [('minimo', 'abaixo do mínimo'), ('maximo', 'acima do
          máximo')]:
            limite = c[regra]
            if limite is not None and (valor < limite if regra == 'minimo' else valor >
              limite):
                yield nome, regra, f'Campo {desc}.'
        for regra, desc in [('tamanho_minimo', 'abaixo do mínimo'), ('tamanho_maximo',
          'acima do máximo')]:
            limite = c[regra]
            if limite is not None and (len(valor) < limite if regra == 'tamanho_minimo'
              else len(valor) > limite):
                yield nome, regra, f'Tamanho do campo {desc}.'
        if c['permitidos'] is not None and valor not in c['permitidos']:
            yield nome, 'permitido', 'Campo fora dos valores permitidos.'
    if not definicao['permitir_extras']:
        for nome in registro:
            if nome not in definicao['campos']:
                yield nome, 'extra', 'Campo extra não permitido.'
```

```
def _validar(registro, definicao, indice, linha, origem, modo, limite_erros):
    erros = []
    interrompido = False
    for nome, regra, mensagem in _erros_registro(registro, definicao):
        erros.append({'campo': nome, 'regra': regra, 'mensagem': mensagem,
                      'registro': indice, 'linha': linha, 'origem': origem})
        if modo == 'primeiro' or len(erros) >= limite_erros:
            interrompido = True
            break
    return {'valido': not erros, 'erros': erros, 'interrompido': interrompido}


def validar_registro(registro: Mapping, esquema: Mapping, *, indice: int = 1,
                     linha: int | None = None, origem: str | None = None,
                     modo: str = 'coletar', limite_erros: int = 100) -> dict:
    """Valida sem converter: ausente e nulo têm regras diferentes.

    Retorna valido, erros e interrompido. Erros têm campo, regra, mensagem,
    registro (índice base 1), linha e origem; nunca incluem o valor do campo.
    primeiro para no primeiro erro; coletar para ao atingir limite_erros.
    """
    _opcoes(modo, limite_erros, origem)
    _inteiro_positivo(indice, 'indice')
    _inteiro_positivo(linha, 'linha', opcional=True)
    definicao = _copiar_esquema(esquema)
    return _validar(registro, definicao, indice, linha, origem, modo, limite_erros)


def validar_registros(registros: Iterable[Mapping], esquema: Mapping, *,
                      modo: str = 'coletar', limite_erros: int = 100,
                      linha_inicial: int | None = None, origem: str | None = None) ->
                        dict:
    """Valida uma passagem, sem reter registros nem fechar o iterador do chamador.

    Retorna valido, erros, verificados e interrompido. O limite de erros é global.
    linha_inicial conta registros lógicos, não linhas físicas de CSV multilinha.
    Ausente difere de nulo. Configuração inválida falha antes da iteração.
    """
    _opcoes(modo, limite_erros, origem)
    _inteiro_positivo(linha_inicial, 'linha_inicial', opcional=True)
    definicao = _copiar_esquema(esquema)
    erros = []
    verificados = 0
    interrompido = False
    for indice, registro in enumerate(registros, 1):
        linha = None if linha_inicial is None else linha_inicial + indice - 1
        resultado = _validar(registro, definicao, indice, linha, origem, modo,
          limite_erros - len(erros))
        verificados += 1
        erros.extend(resultado['erros'])
        if resultado['interrompido']:
            interrompido = True
            break
    return {'valido': not erros, 'erros': erros, 'verificados': verificados,
            'interrompido': interrompido}
```

Biblioteca `coral.formatacao`

`coral.formatacao.ErroFormato`

Dados ou texto não obedecem ao formato esperado.

Assinatura

Assinatura para consulta

```
ErroFormato(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`formatar_valor(...)` · módulo `coral.formatacao`

O que faz

Formata um valor básico usando sua grafia Coral.

Assinatura

Assinatura para consulta

```
formatar_valor(valor: 'Any') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor representável como texto Coral	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.formatacao importe formatar_valor
mostre formatar_valor(verdadeiro)
```

Resultado

Saída esperada

```
verdadeiro
```

`montar_texto(...)` · módulo `coral.formatacao`

O que faz

Monta texto a partir de vários valores, separador e final.

Assinatura

Assinatura para consulta

```
montar_texto(*partes: 'Any', separador: 'str' = '', final: 'str' = '') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
partes	argumentos variáveis, representáveis como texto Coral	não	sem padrão	Valores convertidos e reunidos no texto final.
separador	valor representável como texto Coral	não	''	Texto que separa as partes.
final	valor representável como texto Coral	não	''	Texto acrescentado após a última parte.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.formatacao importe montar_texto
mostre montar_texto("Coral", 3, separador=" ")
```

Resultado

Saída esperada

```
Coral 3
```

Consultar a implementação canônica de coral.formatacao

Implementação canônica de `coral.formatacao`

Código Python

```
from __future__ import annotations

import math
import re
from typing import Any

from .erros import ErroConversao


_AUSENTE = object()
_RE_INTEIRO = re.compile(r"^[+-]?\d+$")
_VERDADEIROS = {"verdadeiro", "true", "sim", "1"}
_FALSOS = {"falso", "false", "nao", "não", "0"}


def _falhar_ou_padrao(valor: Any, destino: str, padrao: Any, causa: BaseException | None
   = None):
    if padrao is not _AUSENTE:
        return padrao
    raise ErroConversao(valor, destino, causa=causa) from causa


def inteiro(valor: Any, padrao: Any = _AUSENTE) -> int | Any:
    """Converte para inteiro de forma estrita, com padrão opcional."""


    try:
        if isinstance(valor, bool):
            raise ValueError("booleano não é inteiro implícito")
        if isinstance(valor, int):
            return valor
        if isinstance(valor, float):
            if not math.isfinite(valor) or not valor.is_integer():
                raise ValueError("decimal não representa inteiro exato")
            return int(valor)
        if isinstance(valor, str):
            texto_valor = valor.strip()
            if not _RE_INTEIRO.fullmatch(texto_valor):
                raise ValueError("texto não representa inteiro")
            return int(texto_valor, 10)
        raise TypeError(f"tipo {type(valor).__name__} não é conversível para inteiro")
    except (TypeError, ValueError, OverflowError) as erro:
        return _falhar_ou_padrao(valor, "inteiro", padrao, erro)


def decimal(valor: Any, padrao: Any = _AUSENTE) -> float | Any:
    """Converte para decimal finito, aceitando ponto ou vírgula decimal em texto."""

    try:
        if isinstance(valor, bool):
            raise ValueError("booleano não é decimal implícito")
        if isinstance(valor, (int, float)):
            resultado = float(valor)
        elif isinstance(valor, str):
            texto_valor = valor.strip()
            if not texto_valor:
                raise ValueError("texto vazio")
            if "," in texto_valor and "." in texto_valor:
                raise ValueError("separador decimal ambíguo")
            resultado = float(texto_valor.replace(",", "."))
```

```
        else:
            raise TypeError(f"tipo {type(valor).__name__} não é conversível para
              decimal")
        if not math.isfinite(resultado):
            raise ValueError("decimal precisa ser finito")
        return resultado
    except (TypeError, ValueError, OverflowError) as erro:
        return _falhar_ou_padrao(valor, "decimal", padrao, erro)


def texto(valor: Any, padrao: Any = _AUSENTE) -> str | Any:
    """Produz texto com grafia coerente para os literais básicos da Coral."""

    try:
        if valor is None:
            return "nulo"
        if valor is True:
            return "verdadeiro"
        if valor is False:
            return "falso"
        return str(valor)
    except Exception as erro:
        return _falhar_ou_padrao(valor, "texto", padrao, erro)


def booleano(valor: Any, padrao: Any = _AUSENTE) -> bool | Any:
    """Converte apenas representações booleanas explícitas e não vazias."""

    try:
        if isinstance(valor, bool):
            return valor
        if isinstance(valor, (int, float)) and not isinstance(valor, bool):
            if valor == 1:
                return True
            if valor == 0:
                return False
            raise ValueError("número precisa ser 0 ou 1")
        if isinstance(valor, str):
            normal = valor.strip().casefold()
            if normal in _VERDADEIROS:
                return True
            if normal in _FALSOS:
                return False
            raise ValueError("texto não representa booleano")
        raise TypeError(f"tipo {type(valor).__name__} não é conversível para booleano")
    except (TypeError, ValueError, OverflowError) as erro:
        return _falhar_ou_padrao(valor, "booleano", padrao, erro)


__all__ = ["ErroConversao", "inteiro", "decimal", "texto", "booleano"]

from __future__ import annotations

from typing import Any

from .comum import ErroCoral, ErroOperacaoCoral


class ErroCoralRuntime(ErroCoral):
    """Base dos erros controlados expostos pelo runtime base da Coral."""

```

```
    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroEntrada(ErroCoralRuntime):
    """Falha ao obter entrada para um programa Coral."""


class FimDeEntrada(ErroEntrada, EOFError):
    """A fonte de entrada terminou antes de produzir outra linha."""


class ErroConversao(ErroCoralRuntime, ValueError):
    """Um valor não pôde ser convertido para o tipo Coral solicitado."""

    def __init__(
        self,
        valor: Any,
        destino: str,
        mensagem: str | None = None,
        *,
        causa: BaseException | None = None,
    ):
        self.valor = valor
        self.destino = destino
        super().__init__(
            mensagem or f"não foi possível converter o valor para {destino}",
            causa=causa,
        )


class ErroFormato(ErroCoralRuntime, ValueError):
    """Dados ou texto não obedecem ao formato esperado."""


class ErroValor(ErroCoralRuntime, ValueError):
    """O tipo é aceito, mas o valor não é válido para a operação."""


class ErroOperacao(ErroOperacaoCoral):
    """Falha operacional com preservação opcional da causa técnica."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroArquivo(ErroOperacao):
    """Falha em uma operação com arquivo ou caminho."""


__all__ = [
    "ErroCoralRuntime",
    "ErroEntrada",
    "FimDeEntrada",
    "ErroConversao",
    "ErroArquivo",
    "ErroFormato",
```

```
    "ErroValor",
    "ErroOperacao",
]

from __future__ import annotations

from typing import Any

from .conversoes import texto
from .erros import ErroFormato


def formatar_valor(valor: Any) -> str:
    """Representação textual básica usando convenções de literais Coral."""

    return texto(valor)


def montar_texto(*partes: Any, separador: str = "", final: str = "") -> str:
    """Monta texto convertendo as partes de modo explícito e previsível.

    Não interpreta placeholders nem expressões dentro de strings; portanto não
    introduz interpolação fora do parser/AST da linguagem.
    """

    try:
        sep = texto(separador)
        fim = texto(final)
    except Exception as erro:
        raise ErroFormato("separador ou final inválido", causa=erro) from erro
    return sep.join(formatar_valor(parte) for parte in partes) + fim


__all__ = ["formatar_valor", "montar_texto", "ErroFormato"]
```

Biblioteca `coral.formatos`

`de_base64(...)` · **módulo** `coral.formatos`

O que faz

Decodifica Base64, restaurando o padding final quando ausente.

Assinatura

Assinatura para consulta

```
de_base64(texto, *, url=False) -> 'bytes'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
texto	valor convertido para texto	sim	sem padrão	Texto Base64; espaços só nas extremidades são removidos.
url	booleano	não	False	Aceita também o alfabeto de URLs.

Retorno

bytes.

Exemplo

Código Coral

```
de coral.formatos importe de_base64
mostre de_base64("Q29yYWw").decode("utf-8")
```

Resultado

Saída esperada

```
Coral
```

Erros

ValueError para Base64 inválido ou texto não ASCII.

Atenção

Espaços no interior do conteúdo não são ignorados pela validação estrita.

`de_csv(...)` · módulo `coral.formatos`

O que faz

Lê texto CSV como registros ou listas de células.

Assinatura

Assinatura para consulta

```
de_csv(texto, *, cabecalho=True, delimitador=',') -> 'list[dict[str | None, str |
  list[str] | None] | list[str]]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
texto	valor convertido para texto	sim	sem padrão	Conteúdo CSV.
cabecalho	booleano	não	True	Usa a primeira linha como nomes das colunas.
delimitador	texto de um caractere	não	','	Separador das células.

Retorno

Lista de dicionários quando cabecalho é verdadeiro; lista de listas caso contrário.

Exemplo

Código Coral

```
de coral.formatos importe de_csv
mostre de_csv("nome,nota\nAna,7\n")
```

Resultado

Saída esperada

```
[{'nome': 'Ana', 'nota': '7'}]
```

Erros

TypeError ou csv.Error para configuração incompatível.

Atenção

Não converte automaticamente as células para números. Cabeçalhos repetidos seguem as regras de csv.DictReader; não há validação de esquema.

`de_hex(...)` · módulo `coral.formatos`

O que faz

Decodifica texto hexadecimal, aceitando espaços permitidos por bytes.fromhex.

Assinatura

Assinatura para consulta

```
de_hex(texto) -> 'bytes'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
texto	valor convertido para texto	sim	sem padrão	Representação hexadecimal a decodificar.

Retorno

bytes.

Exemplo

Código Coral

```
de coral.formatos importe de_hex
mostre de_hex("43 6f 72 61 6c").decode("utf-8")
```

Resultado

Saída esperada

```
Coral
```

Erros

ValueError para representação hexadecimal inválida.

`para_base64(...)` · módulo `coral.formatos`

O que faz

Codifica conteúdo em Base64 padrão ou adequado a URLs.

Assinatura

Assinatura para consulta

```
para_base64(dados, *, url=False, sem_padding=False) -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
dados	texto, bytes, bytearray ou memoryview	sim	sem padrão	Conteúdo a codificar; texto é convertido em bytes UTF-8.
url	booleano	não	False	Usa o alfabeto com - e _ no lugar de + e /.
sem_padding	booleano	não	False	Remove os sinais = finais.

Retorno

Texto ASCII.

Exemplo

Código Coral

```
de coral.formatos importe para_base64
mostre para_base64("Coral", sem_padding=verdadeiro)
```

Resultado

Saída esperada

```
Q29yYWw
```

Erros

TypeError para dados incompatíveis.

Atenção

Base64 é codificação reversível; não protege nem cifra dados.

`para_csv(...)` · módulo `coral.formatos`

O que faz

Converte linhas de sequências ou dicionários em CSV.

Assinatura

Assinatura para consulta

```
para_csv(linhas: 'Iterable[Mapping[str, Any] | Iterable[Any]]', *, colunas=None,
  delimitador=',') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
linhas	iterável de dicionários ou de iteráveis	sim	sem padrão	Linhas a serializar; o conteúdo é materializado em memória.
colunas	iterável de nomes ou nulo	não	None	Cabeçalho e ordem das colunas; com dicionários, nulo usa as chaves da primeira linha.
delimitador	texto de um caractere	não	','	Separador das células.

Retorno

Texto com delimitadores e terminadores CRLF; entrada vazia produz texto vazio.

Exemplo

Código Coral

```
de coral.formatos importe para_csv
defina csv como para_csv([["Ana", 7]], colunas=["nome", "nota"])
mostre csv.splitlines()
```

Resultado

Saída esperada

```
['nome,nota', 'Ana,7']
```

Erros

ValueError se uma linha de dicionário possui chave fora das colunas; TypeError ou csv.Error para dados ou delimitadores incompatíveis.

Atenção

Dicionários sempre geram cabeçalho. Sequências só geram cabeçalho quando colunas é fornecido. Valores são serializados pelas regras de csv do Python.

`para_hex(...)` · módulo `coral.formatos`

O que faz

Codifica conteúdo como texto hexadecimal.

Assinatura

Assinatura para consulta

```
para_hex(dados, *, maiusculas=False) -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
dados	texto, bytes, bytearray ou memoryview	sim	sem padrão	Conteúdo a codificar; texto é convertido em bytes UTF-8.
maiusculas	booleano	não	False	Produz letras A a F em maiúsculas.

Retorno

Texto hexadecimal.

Exemplo

Código Coral

```
de coral.formatos importe para_hex
mostre para_hex("Coral", maiusculas=verdadeiro)
```

Resultado

Saída esperada

```
436F72616C
```

Erros

TypeError se dados não é texto nem uma sequência binária aceita.

Consultar a implementação canônica de coral.formatos

Implementação canônica de `coral.formatos`

Código Python

```
"""Superfície pública de coral.formatos."""
from .stdlib.formatos import *  # noqa: F401,F403
from .stdlib.formatos import __all__

from __future__ import annotations

import base64
import binascii
import csv
import io
from typing import Any, Iterable, Mapping


def para_hex(dados, *, maiusculas=False) -> str:
    if isinstance(dados, str):
        dados = dados.encode("utf-8")
    if not isinstance(dados, (bytes, bytearray, memoryview)):
        raise TypeError("hexadecimal aceita texto ou bytes")
    saida = bytes(dados).hex()
    return saida.upper() if maiusculas else saida


def de_hex(texto) -> bytes:
    try:
        return bytes.fromhex(str(texto))
    except ValueError as erro:
        raise ValueError("texto hexadecimal inválido") from erro


def para_base64(dados, *, url=False, sem_padding=False) -> str:
    if isinstance(dados, str):
        dados = dados.encode("utf-8")
    if not isinstance(dados, (bytes, bytearray, memoryview)):
        raise TypeError("Base64 aceita texto ou bytes")
    bruto = bytes(dados)
    codificado = base64.urlsafe_b64encode(bruto) if url else base64.b64encode(bruto)
    texto = codificado.decode("ascii")
    return texto.rstrip("=") if sem_padding else texto


def de_base64(texto, *, url=False) -> bytes:
    bruto = str(texto).strip()
    bruto += "=" * ((4 - len(bruto) % 4) % 4)
    try:
        if url:
            return base64.b64decode(bruto.encode("ascii"), altchars=b"-_",
              validate=True)
        return base64.b64decode(bruto.encode("ascii"), validate=True)
    except (ValueError, binascii.Error, UnicodeEncodeError) as erro:
        raise ValueError("Base64 inválido") from erro
```

```
def para_csv(linhas: Iterable[Mapping[str, Any] | Iterable[Any]], *, colunas=None,
  delimitador=",") -> str:
    itens = list(linhas)
    saida = io.StringIO(newline="")
    if not itens:
        return ""
    if isinstance(itens[0], Mapping):
        nomes = list(colunas) if colunas is not None else list(itens[0].keys())
        escritor = csv.DictWriter(saida, fieldnames=nomes, delimiter=str(delimitador))
        escritor.writeheader()
        escritor.writerows(itens)
    else:
        escritor = csv.writer(saida, delimiter=str(delimitador))
        if colunas is not None:
            escritor.writerow(list(colunas))
        escritor.writerows(itens)
    return saida.getvalue()


def de_csv(texto, *, cabecalho=True, delimitador=",") -> list[dict[str | None, str |
  list[str] | None] | list[str]]:
    entrada = io.StringIO(str(texto), newline="")
    if cabecalho:
        return list(csv.DictReader(entrada, delimiter=str(delimitador)))
    return list(csv.reader(entrada, delimiter=str(delimitador)))


__all__ = ["para_hex", "de_hex", "para_base64", "de_base64", "para_csv", "de_csv"]
```

Biblioteca `coral.graficos`

`coral.graficos.ErroGrafico`

Erro público para especificações ou dados gráficos inválidos.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.graficos.FaixaIncerteza`

Centro e limites absolutos de uma faixa de incerteza.

Assinatura

Assinatura para consulta

```
FaixaIncerteza(x: 'tuple[float, ...]', centro: 'tuple[float, ...]', inferior:
  'tuple[float, ...]', superior: 'tuple[float, ...]', nome: 'str | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
x	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
centro	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
inferior	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
superior	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `x`: `tuple[float, ...]`.

Atributo `centro`: `tuple[float, ...]`.

Atributo `inferior`: `tuple[float, ...]`.

Atributo `superior`: `tuple[float, ...]`.

Atributo `nome`: `str | None`.

`coral.graficos.Grafico`

Especificação imutável de um gráfico Coral 2D ou 3D.

Assinatura

Assinatura para consulta

```
Grafico(tipo: 'str', elementos: 'tuple[Any, ...]', titulo: 'str' = '', rotulo_x: 'str' =
    '', rotulo_y: 'str' = '', rotulo_z: 'str' = '', unidade_x: 'str' = '', unidade_y: 'str'
    = '', unidade_z: 'str' = '', legenda: 'bool' = True, tema: 'str' = 'claro',
   proveniencia: 'tuple[str, ...]' = (), texto_alternativo: 'str' = '', avisos: 'tuple[str,
    ...]' = ()) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
elementos	tuple[Any, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
titulo	str	POSITIONAL_OR_KEYWORD	não	''
rotulo_x	str	POSITIONAL_OR_KEYWORD	não	''
rotulo_y	str	POSITIONAL_OR_KEYWORD	não	''
rotulo_z	str	POSITIONAL_OR_KEYWORD	não	''
unidade_x	str	POSITIONAL_OR_KEYWORD	não	''
unidade_y	str	POSITIONAL_OR_KEYWORD	não	''
unidade_z	str	POSITIONAL_OR_KEYWORD	não	''
legenda	bool	POSITIONAL_OR_KEYWORD	não	True
tema	str	POSITIONAL_OR_KEYWORD	não	'claro'
proveniencia	tuple[str, ...]	POSITIONAL_OR_KEYWORD	não	()
texto_alternativo	str	POSITIONAL_OR_KEYWORD	não	''
avisos	tuple[str, ...]	POSITIONAL_OR_KEYWORD	não	()

Retorno declarado

None

Atributo `tipo`: `str`.

Atributo `elementos`: `tuple[Any, ...]`.

Atributo `titulo`: `str`.

Atributo `rotulo_x`: `str`.

Atributo `rotulo_y`: `str`.

Atributo `rotulo_z`: `str`.

Atributo `unidade_x`: `str`.

Atributo `unidade_y`: `str`.

Atributo `unidade_z`: `str`.

Atributo `legenda`: `bool`.

Atributo `tema`: `str`.

Atributo `proveniencia`: `tuple[str, ...]`.

Atributo `texto_alternativo`: `str`.

Atributo `avisos`: `tuple[str, ...]`.

`coral.graficos.MatrizCores`

Matriz numérica e rótulos para mapas de calor e correlação.

Assinatura

Assinatura para consulta

```
MatrizCores(valores: 'tuple[tuple[float, ...], ...]', rotulos_linhas: 'tuple[str, ...]',
    rotulos_colunas: 'tuple[str, ...]', nome: 'str | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	tuple[tuple[float, ...], ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
rotulos_linhas	tuple[str, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
rotulos_colunas	tuple[str, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `valores`: `tuple[tuple[float, ...], ...]`.

Atributo `rotulos_linhas`: `tuple[str, ...]`.

Atributo `rotulos_colunas`: `tuple[str, ...]`.

Atributo `nome`: `str | None`.

`coral.graficos.PainelGraficos`

Composição imutável de gráficos usando uma grade regular.

Assinatura

Assinatura para consulta

```
PainelGraficos(graficos: "tuple['Grafico', ...]", colunas: 'int', titulo: 'str' = '') ->
    None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
graficos	tuple['Grafico', ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
colunas	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
titulo	str	POSITIONAL_OR_KEYWORD	não	''

Retorno declarado

None

Atributo `graficos`: `tuple['Grafico', ...]`.

Atributo `colunas`: `int`.

Atributo `titulo`: `str`.

`coral.graficos.ResumoCaixa`

Resumo estatístico de um grupo para um gráfico de caixa.

Assinatura

Assinatura para consulta

```
ResumoCaixa(nome: 'str', minimo: 'float', q1: 'float', mediana: 'float', q3: 'float',
  maximo: 'float', bigode_inferior: 'float', bigode_superior: 'float', extremos:
  'tuple[float, ...]', quantidade: 'int') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
minimo	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
q1	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mediana	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
q3	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
maximo	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
bigode_inferior	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
bigode_superior	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
extremos	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
quantidade	int	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `nome`: `str`.

Atributo `minimo`: `float`.

Atributo `q1`: `float`.

Atributo `mediana`: `float`.

Atributo `q3`: `float`.

Atributo `maximo`: `float`.

Atributo `bigode_inferior`: `float`.

Atributo `bigode_superior`: `float`.

Atributo `extremos`: `tuple[float, ...]`.

Atributo `quantidade`: `int`.

`coral.graficos.Serie2D`

Série cartesiana usada por linhas, dispersão e funções.

`None é aceito internamente apenas como separador explícito de segmentos produzidos por :func:grafico_funcao` quando uma amostra falha ou não é finita. Construtores de dados diretos rejeitam valores ausentes.

Assinatura

Assinatura para consulta

```
Serie2D(estilo: 'str', x: 'tuple[float, ...]', y: 'tuple[float | None, ...]', nome: 'str
    | None' = None, origem: 'str' = 'direta') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
estilo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
x	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	tuple[float | None, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None
origem	str	POSITIONAL_OR_KEYWORD	não	'direta'

Retorno declarado

None

Atributo `estilo`: `str`.

Atributo `x`: `tuple[float, ...]`.

Atributo `y`: `tuple[float | None, ...]`.

Atributo `nome`: `str | None`.

Atributo `origem`: `str`.

`coral.graficos.Serie3D`

Série cartesiana tridimensional imutável.

Os três eixos são materializados como tuplas de números finitos. Criar a especificação não projeta, não abre janela e não carrega backend visual.

Assinatura

Assinatura para consulta

```
Serie3D(estilo: 'str', x: 'tuple[float, ...]', y: 'tuple[float, ...]', z: 'tuple[float,
  ...]', nome: 'str | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
estilo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
x	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
z	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `estilo`: `str`.

Atributo `x`: `tuple[float, ...]`.

Atributo `y`: `tuple[float, ...]`.

Atributo `z`: `tuple[float, ...]`.

Atributo `nome`: `str | None`.

`coral.graficos.SerieCategorias`

Série categórica para barras ou histograma.

Assinatura

Assinatura para consulta

```
SerieCategorias(estilo: 'str', categorias: 'tuple[str, ...]', valores: 'tuple[float,
  ...]', limites: 'tuple[tuple[float, float], ...]' = (), nome: 'str | None' = None) ->
  None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
estilo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
categorias	tuple[str, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
valores	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
limites	tuple[tuple[float, float], ...]	POSITIONAL_OR_KEYWORD	não	()

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `estilo`: `str`.

Atributo `categorias`: `tuple[str, ...]`.

Atributo `valores`: `tuple[float, ...]`.

Atributo `limites`: `tuple[tuple[float, float], ...]`.

Atributo `nome`: `str | None`.

`coral.graficos.SerieErro`

Série cartesiana com incerteza inferior e superior por ponto.

Assinatura

Assinatura para consulta

```
SerieErro(x: 'tuple[float, ...]', y: 'tuple[float, ...]', erro_inferior: 'tuple[float,
  ...]', erro_superior: 'tuple[float, ...]', nome: 'str | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
x	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
erro_inferior	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
erro_superior	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `x`: `tuple[float, ...]`.

Atributo `y`: `tuple[float, ...]`.

Atributo `erro_inferior`: `tuple[float, ...]`.

Atributo `erro_superior`: `tuple[float, ...]`.

Atributo `nome`: `str | None`.

`coral.graficos.SeriesRegistros`

Duas séries alinhadas extraídas de registros comuns.

Assinatura

Assinatura para consulta

```
SeriesRegistros(x: 'tuple[float, ...]', y: 'tuple[float, ...]', campo_x: 'str', campo_y:
    'str') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
x	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	tuple[float, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
campo_x	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
campo_y	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `x`: `tuple[float, ...]`.

Atributo `y`: `tuple[float, ...]`.

Atributo `campo_x`: `str`.

Atributo `campo_y`: `str`.

`coral.graficos.Superficie3D`

Grade tridimensional explícita, sem interpolação de pontos ausentes.

Assinatura

Assinatura para consulta

```
Superficie3D(x: 'tuple[tuple[float, ...], ...]', y: 'tuple[tuple[float, ...], ...]', z:
    'tuple[tuple[float, ...], ...]', nome: 'str | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
x	tuple[tuple[float, ...], ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	tuple[tuple[float, ...], ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
z	tuple[tuple[float, ...], ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `x`: `tuple[tuple[float, ...], ...]`.

Atributo `y`: `tuple[tuple[float, ...], ...]`.

Atributo `z`: `tuple[tuple[float, ...], ...]`.

Atributo `nome`: `str | None`.

`coral.graficos.amostrar_grafico`

Reduz explicitamente séries simples, registrando a amostragem na especificação.

A função não é usada automaticamente por `` `para_html` ``. Superfícies, mapas, caixas, barras e faixas exigem redução explícita pelo programa porque sua semântica não é equivalente à seleção uniforme de observações.

Assinatura

Assinatura para consulta

```
amostrar_grafico(grafico: 'Any', max_pontos: 'int') -> 'Any'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
grafico	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
max_pontos	int	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Any

`coral.graficos.barras`

Cria barras categóricas; a escala vertical inclui zero por contrato.

Assinatura

Assinatura para consulta

```
barras(categorias: 'Iterable[Any]', valores: 'Iterable[Any]', nome: 'str | None' = None)
    -> 'Grafico'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
categorias	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
valores	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

Grafico

`coral.graficos.barras_erro`

Cria pontos com barras de erro simétricas ou assimétricas explicitamente fornecidas.

Assinatura

Assinatura para consulta

```
barras_erro(x: 'Iterable[Any]', y: 'Iterable[Any]', erros: 'Iterable[Any]', nome: 'str |
    None' = None) -> 'Grafico'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
x	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
erros	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

Grafico

`configurar(...)` · módulo `coral.graficos`

O que faz

Cria uma cópia do gráfico com as opções solicitadas.

Assinatura

Assinatura para consulta

```
configurar(grafico: 'Grafico', titulo: 'str | None' = None, eixos: 'Any' = None,
  unidades: 'Any' = None, legenda: 'bool | None' = None, tema: 'str | None' = None, *,
  proveniencia: 'Iterable[Any] | None' = None, texto_alternativo: 'str | None' = None) ->
  'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grafico	Grafico	sim	sem padrão	Especificação de gráfico a configurar ou salvar.
titulo	str | None	não	None	Valor titulo usado na operação.
eixos	Any	não	None	Rótulos dos eixos; a quantidade precisa corresponder à dimensionalidade.
unidades	Any	não	None	Unidades dos eixos; acompanham a dimensionalidade do gráfico.
legenda	bool | None	não	None	Se verdadeiro, inclui a legenda.
tema	str | None	não	None	Identificador do tema visual suportado.
proveniencia	Iterable[Any] | None	não	None	Metadados que explicam de onde vêm os dados.
texto_alternativo	str | None	não	None	Descrição textual para acessibilidade.

Retorno

Nova especificação Grafico.

Exemplo

Código Coral

```
de coral.graficos importe configurar
de coral.graficos importe linha
defina figura como configurar(linha([0, 1], [2, 3]), titulo="Medidas")
mostre figura.titulo
```

Resultado

Saída esperada

```
Medidas
```

Erros

ErroGrafico para dados, configuração, limites ou backend inválidos.

Atenção

Criar a especificação não abre janela. Exportar escreve no caminho informado. HTML incorpora dados e exige o backend opcional Plotly; outros formatos têm requisitos próprios.

`dispersao(...)` · módulo `coral.graficos`

O que faz

Cria uma especificação imutável de gráfico de pontos.

Assinatura

Assinatura para consulta

```
dispersao(x: 'Iterable[Any]', y: 'Iterable[Any]', nome: 'str | None' = None) ->
  'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	Iterable[Any]	sim	sem padrão	Coordenadas horizontais da série, ou posição horizontal do objeto.
y	Iterable[Any]	sim	sem padrão	Coordenadas verticais da série, ou posição vertical do objeto.
nome	str | None	não	None	Nome da série ou do objeto.

Retorno

Grafico 2D.

Exemplo

Código Coral

```
de coral.graficos importe dispersao
defina figura como dispersao([0, 1], [2, 3])
mostre figura.elementos[0].estilo
```

Resultado

Saída esperada

```
dispersao
```

Erros

ErroGrafico para dados, configuração, limites ou backend inválidos.

Atenção

Criar a especificação não abre janela. Exportar escreve no caminho informado. HTML incorpora dados e exige o backend opcional Plotly; outros formatos têm requisitos próprios.

`coral.graficos.dispersao3d`

Cria pontos 3D preservando exatamente as coordenadas fornecidas.

Assinatura

Assinatura para consulta

```
dispersao3d(x: 'Iterable[Any]', y: 'Iterable[Any]', z: 'Iterable[Any]', nome: 'str |
  None' = None) -> 'Grafico'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
x	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
z	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

Grafico

`coral.graficos.explorar`

Abre uma prévia local apenas quando solicitada explicitamente.

Em sessão sem interface, falha com orientação para `salvar_html` e não tenta iniciar servidor ou navegador em segundo plano.

Assinatura

Assinatura para consulta

```
explorar(grafico: 'Any', *, limite_pontos: 'int' = 200000) -> 'Path'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
grafico	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
limite_pontos	int	KEYWORD_ONLY	não	200000

Retorno declarado

Path

`coral.graficos.faixa_incerteza`

Cria faixa de limites absolutos sem atribuir significado estatístico à faixa.

Assinatura

Assinatura para consulta

```
faixa_incerteza(x: 'Iterable[Any]', centro: 'Iterable[Any]', inferior: 'Iterable[Any]',
  superior: 'Iterable[Any]', nome: 'str | None' = None) -> 'Grafico'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
x	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
centro	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
inferior	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
superior	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

Grafico

`coral.graficos.grafico_caixa`

Cria caixas de Tukey usando quartis do contrato `coral.numerico`.

Bigodes usam 1,5 × IQR. Valores além dos limites são preservados em `extremos`. Grupos com menos de cinco observações recebem aviso explícito.

Assinatura

Assinatura para consulta

```
grafico_caixa(grupos: 'Any') -> 'Grafico'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
grupos	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Grafico

`coral.graficos.grafico_experimento`

Adapta medições bem sucedidas a uma série comum sem dependência no import do módulo.

Assinatura

Assinatura para consulta

```
grafico_experimento(resultado: 'Any', eixo_x: 'str', metrica: 'str') -> 'Grafico'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
resultado	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
eixo_x	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
metrica	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Grafico

`coral.graficos.grafico_funcao`

Amostra função escalar em intervalo fechado, preservando falhas como lacunas.

Assinatura

Assinatura para consulta

```
grafico_funcao(funcao: 'Callable[[float], Any]', inicio: 'Any', fim: 'Any', amostras:
  'int' = 200) -> 'Grafico'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
funcao	Callable[[float], Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
inicio	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
fim	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
amostras	int	POSITIONAL_OR_KEYWORD	não	200

Retorno declarado

Grafico

`coral.graficos.grafico_histograma`

Cria gráfico a partir do contrato canônico `` `numerico.histograma` ``.

Assinatura

Assinatura para consulta

```
grafico_histograma(valores: 'Iterable[Any]', faixas: 'Any', nome: 'str | None' = None)
  -> 'Grafico'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
faixas	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

Grafico

`linha(...)` · módulo `coral.graficos`

O que faz

Cria uma especificação imutável de gráfico de linhas.

Assinatura

Assinatura para consulta

```
linha(x: 'Iterable[Any]', y: 'Iterable[Any]', nome: 'str | None' = None) -> 'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	Iterable[Any]	sim	sem padrão	Coordenadas horizontais da série, ou posição horizontal do objeto.
y	Iterable[Any]	sim	sem padrão	Coordenadas verticais da série, ou posição vertical do objeto.
nome	str | None	não	None	Nome da série ou do objeto.

Retorno

Grafico 2D.

Exemplo

Código Coral

```
de coral.graficos importe linha
defina figura como linha([0, 1], [2, 3])
mostre figura.elementos[0].estilo
```

Resultado

Saída esperada

```
linha
```

Erros

ErroGrafico para dados, configuração, limites ou backend inválidos.

Atenção

Criar a especificação não abre janela. Exportar escreve no caminho informado. HTML incorpora dados e exige o backend opcional Plotly; outros formatos têm requisitos próprios.

`coral.graficos.mapa_calor`

Representa uma matriz numérica por intensidade de cor, sem recalcular dados.

Assinatura

Assinatura para consulta

```
mapa_calor(matriz: 'Any', rotulos_linhas: 'Iterable[Any] | None' = None,
  rotulos_colunas: 'Iterable[Any] | None' = None, nome: 'str | None' = None) -> 'Grafico'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
matriz	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
rotulos_linhas	Iterable[Any] | None	POSITIONAL_OR_KEYWORD	não	None
rotulos_colunas	Iterable[Any] | None	POSITIONAL_OR_KEYWORD	não	None
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

Grafico

`coral.graficos.mapa_correlacao`

Calcula Pearson em `coral.numerico` e representa a matriz resultante.

Assinatura

Assinatura para consulta

```
mapa_correlacao(dados: 'Any', rotulos: 'Iterable[Any] | None' = None, nome: 'str | None'
   = None) -> 'Grafico'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
dados	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
rotulos	Iterable[Any] | None	POSITIONAL_OR_KEYWORD	não	None
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

Grafico

`coral.graficos.painel`

Combina gráficos em grade sem modificar suas escalas ou especificações.

Assinatura

Assinatura para consulta

```
painel(graficos: 'Iterable[Grafico]', colunas: 'int' = 2, titulo: 'str' = '') ->
  'PainelGraficos'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
graficos	Iterable[Grafico]	POSITIONAL_OR_KEYWORD	sim	sem padrão
colunas	int	POSITIONAL_OR_KEYWORD	não	2
titulo	str	POSITIONAL_OR_KEYWORD	não	''

Retorno declarado

PainelGraficos

`coral.graficos.para_html`

Gera documento HTML interativo autossuficiente, sem abrir rede ou janela.

O limite é de proteção contra materialização acidental. A função nunca amostra dados silenciosamente. Para uma série grande, o programa pode usar `amostrar_grafico` de forma explícita ou escolher um limite maior.

Assinatura

Assinatura para consulta

```
para_html(grafico: 'Any', *, limite_pontos: 'int' = 200000) -> 'str'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
grafico	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
limite_pontos	int	KEYWORD_ONLY	não	200000

Retorno declarado

str

`coral.graficos.para_svg`

Renderiza gráficos 2D como SVG autossuficiente em memória.

Especificações 3D preservam coordenadas sem escolher projeção no núcleo. Para renderização 3D use :func:`salvar`, que carrega Matplotlib sob demanda.

Assinatura

Assinatura para consulta

```
para_svg(grafico: 'Grafico | PainelGraficos', *, largura: 'int' = 800, altura: 'int' =
  500) -> 'str'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
grafico	Grafico | PainelGraficos	POSITIONAL_OR_KEYWORD	sim	sem padrão
largura	int	KEYWORD_ONLY	não	800
altura	int	KEYWORD_ONLY	não	500

Retorno declarado

str

`salvar(...)` · módulo `coral.graficos`

O que faz

Renderiza um gráfico num arquivo escolhido pelo formato de saída.

Assinatura

Assinatura para consulta

```
salvar(grafico: 'Grafico | PainelGraficos', caminho: 'str | Path') -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grafico	Grafico | PainelGraficos	sim	sem padrão	Especificação de gráfico a configurar ou salvar.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	str | Path	sim	sem padrão	Caminho local do arquivo de saída ou da imagem de origem.

Retorno

Path do arquivo salvo.

Exemplo

Código Coral

```
de coral.graficos importe salvar
de coral.graficos importe linha
execute salvar(linha([0, 1], [2, 3]), "figura.svg")
de coral.arquivos importe existe
mostre existe("figura.svg")
```

Resultado

Saída esperada

```
True
```

Erros

ErroGrafico para dados, configuração, limites ou backend inválidos.

Atenção

Criar a especificação não abre janela. Exportar escreve no caminho informado. HTML incorpora dados e exige o backend opcional Plotly; outros formatos têm requisitos próprios.

`salvar_html(...)` · módulo `coral.graficos`

O que faz

Exporta um gráfico interativo autossuficiente, sem CDN.

Assinatura

Assinatura para consulta

```
salvar_html(grafico: 'Any', caminho: 'str | Path', *, limite_pontos: 'int' = 200000) ->
  'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grafico	Any	sim	sem padrão	Especificação de gráfico a configurar ou salvar.
caminho	str | Path	sim	sem padrão	Caminho local do arquivo de saída ou da imagem de origem.
limite_pontos	int	não	200000	Teto explícito de observações aceitas na exportação HTML.

Retorno

Path do HTML salvo.

Exemplo

Código Coral

```
de coral.graficos importe salvar_html
de coral.graficos importe linha
execute salvar_html(linha([0, 1], [2, 3]), "figura.html")
de coral.arquivos importe existe
mostre existe("figura.html")
```

Resultado

Saída esperada

```
True
```

Erros

ErroGrafico para dados, configuração, limites ou backend inválidos.

Atenção

Criar a especificação não abre janela. Exportar escreve no caminho informado. HTML incorpora dados e exige o backend opcional Plotly; outros formatos têm requisitos próprios.

`coral.graficos.series_de_registros`

Extrai dois campos numéricos alinhados de registros comuns.

Registros podem ser mapeamentos ou objetos com atributos. Campo ausente e valor ausente são erros explícitos; nenhuma linha é descartada em silêncio.

Assinatura

Assinatura para consulta

```
series_de_registros(registros: 'Iterable[Any]', campo_x: 'str', campo_y: 'str') ->
  'SeriesRegistros'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
registros	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
campo_x	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
campo_y	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

SeriesRegistros

`coral.graficos.superficie3d`

Cria uma superfície 3D de grade retangular sem interpolação silenciosa.

`x e y podem ser eixos unidimensionais compatíveis com a matriz z ou grades bidimensionais da mesma forma de z`. Valores ausentes e formas irregulares são rejeitados em vez de serem preenchidos automaticamente.

Assinatura

Assinatura para consulta

```
superficie3d(x: 'Any', y: 'Any', z: 'Any', nome: 'str | None' = None) -> 'Grafico'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
x	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
z	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

Grafico

`coral.graficos.trajetoria3d`

Cria uma trajetória 3D na ordem dada, sem reamostrar ou suavizar.

Assinatura

Assinatura para consulta

```
trajetoria3d(x: 'Iterable[Any]', y: 'Iterable[Any]', z: 'Iterable[Any]', nome: 'str |
  None' = None) -> 'Grafico'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
x	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
z	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

Grafico

Consultar a implementação canônica de coral.graficos

Implementação canônica de `coral.graficos`

Código Python

```
"""Backend interativo opcional de :mod:`coral.graficos`.

Este módulo é carregado apenas por chamadas explícitas de exportação HTML.
Plotly permanece opcional e nunca é importado junto com ``coral.graficos``.
A exportação usa JavaScript incorporado no próprio documento para que o arquivo
funcione localmente sem CDN e sem rede.
"""
from __future__ import annotations

from hashlib import sha256
from pathlib import Path
from typing import Any


LIMITE_PONTOS_HTML_PADRAO = 200_000


def _plotly():
    try:
        import plotly.graph_objects as go
        import plotly.io as pio
        from plotly.subplots import make_subplots
    except Exception as exc:
        from .graficos import ErroGrafico

        raise ErroGrafico(
            "Para criar HTML interativo, instale o backend opcional Plotly no ambiente
              da Coral."
        ) from exc
    return go, pio, make_subplots


def _texto_eixo(rotulo: str, unidade: str) -> str:
    if rotulo and unidade:
        return f"{rotulo} ({unidade})"
    return rotulo or unidade


def _tema_layout(tema: str) -> dict[str, Any]:
    from .graficos import ErroGrafico

    if tema == "claro":
        return {"template": "plotly_white"}
    if tema == "escuro":
        return {"template": "plotly_dark"}
    if tema == "alto_contraste":
        return {
            "template": "plotly_white",
            "paper_bgcolor": "white",
            "plot_bgcolor": "white",
            "font": {"color": "black"},
        }
```

```
    raise ErroGrafico("tema gráfico inválido")


def _trace_2d(go: Any, grafico: Any) -> list[Any]:
    from .graficos import (
        FaixaIncerteza,
        MatrizCores,
        ResumoCaixa,
        Serie2D,
        SerieCategorias,
        SerieErro,
    )

    elem = grafico.elementos[0] if grafico.elementos else None
    if grafico.tipo in {"linha", "funcao", "dispersao"} and isinstance(elem, Serie2D):
        modo = "markers" if grafico.tipo == "dispersao" else "lines"
        return [go.Scatter(x=list(elem.x), y=list(elem.y), mode=modo, name=elem.nome)]
    if grafico.tipo in {"barras", "histograma"} and isinstance(elem, SerieCategorias):
        return [go.Bar(x=list(elem.categorias), y=list(elem.valores), name=elem.nome)]
    if grafico.tipo == "caixa":
        saida = []
        for caixa in grafico.elementos:
            if not isinstance(caixa, ResumoCaixa):
                continue
            saida.append(
                go.Box(
                    name=caixa.nome,
                    q1=[caixa.q1],
                    median=[caixa.mediana],
                    q3=[caixa.q3],
                    lowerfence=[caixa.bigode_inferior],
                    upperfence=[caixa.bigode_superior],
                )
            )
            if caixa.extremos:
                saida.append(
                    go.Scatter(
                        x=[caixa.nome] * len(caixa.extremos),
                        y=list(caixa.extremos),
                        mode="markers",
                        name=f"{caixa.nome}: extremos",
                    )
                )
        return saida
    if grafico.tipo in {"mapa_calor", "mapa_correlacao"} and isinstance(elem,
      MatrizCores):
        return [
            go.Heatmap(
                z=[list(linha) for linha in elem.valores],
                x=list(elem.rotulos_colunas),
                y=list(elem.rotulos_linhas),
                name=elem.nome,
                colorbar={"title": elem.nome or "valor"},
            )
        ]
    if grafico.tipo == "barras_erro" and isinstance(elem, SerieErro):
        return [
            go.Scatter(
                x=list(elem.x),
                y=list(elem.y),
                mode="markers",
```

```
                name=elem.nome,
                error_y={
                    "type": "data",
                    "symmetric": False,
                    "array": list(elem.erro_superior),
                    "arrayminus": list(elem.erro_inferior),
                    "visible": True,
                },
            )
        ]
    if grafico.tipo == "faixa_incerteza" and isinstance(elem, FaixaIncerteza):
        nome = elem.nome or "faixa"
        return [
            go.Scatter(
                x=list(elem.x),
                y=list(elem.inferior),
                mode="lines",
                line={"width": 0},
                hoverinfo="skip",
                showlegend=False,
                name=f"{nome}: inferior",
            ),
            go.Scatter(
                x=list(elem.x),
                y=list(elem.superior),
                mode="lines",
                line={"width": 0},
                fill="tonexty",
                name=f"{nome}: faixa",
            ),
            go.Scatter(
                x=list(elem.x),
                y=list(elem.centro),
                mode="lines",
                name=elem.nome,
            ),
        ]
    from .graficos import ErroGrafico
    raise ErroGrafico(f"tipo de gráfico sem renderizador interativo: {grafico.tipo}")


def _trace_3d(go: Any, grafico: Any) -> list[Any]:
    from .graficos import Serie3D, Superficie3D

    elem = grafico.elementos[0] if grafico.elementos else None
    if grafico.tipo in {"dispersao3d", "trajetoria3d"} and isinstance(elem, Serie3D):
        modo = "markers" if grafico.tipo == "dispersao3d" else "lines+markers"
        return [
            go.Scatter3d(
                x=list(elem.x),
                y=list(elem.y),
                z=list(elem.z),
                mode=modo,
                name=elem.nome,
            )
        ]
    if grafico.tipo == "superficie3d" and isinstance(elem, Superficie3D):
        return [
            go.Surface(
                x=[list(linha) for linha in elem.x],
                y=[list(linha) for linha in elem.y],
```

```
                z=[list(linha) for linha in elem.z],
                name=elem.nome,
                showscale=True,
            )
        ]
    from .graficos import ErroGrafico
    raise ErroGrafico(f"tipo de gráfico 3D sem renderizador interativo: {grafico.tipo}")


def _aplicar_layout(figura: Any, grafico: Any) -> None:
    from .graficos import _grafico_eh_3d

    layout = _tema_layout(grafico.tema)
    layout["title"] = {"text": grafico.titulo} if grafico.titulo else None
    layout["showlegend"] = bool(grafico.legenda)
    if _grafico_eh_3d(grafico):
        layout["scene"] = {
            "xaxis_title": _texto_eixo(grafico.rotulo_x, grafico.unidade_x),
            "yaxis_title": _texto_eixo(grafico.rotulo_y, grafico.unidade_y),
            "zaxis_title": _texto_eixo(grafico.rotulo_z, grafico.unidade_z),
            "aspectmode": "data",
        }
    else:
        layout["xaxis_title"] = _texto_eixo(grafico.rotulo_x, grafico.unidade_x)
        layout["yaxis_title"] = _texto_eixo(grafico.rotulo_y, grafico.unidade_y)
    figura.update_layout(**layout)


def _figura_simples(grafico: Any) -> Any:
    from .graficos import Grafico, ErroGrafico, _grafico_eh_3d

    if not isinstance(grafico, Grafico):
        raise ErroGrafico("HTML interativo requer Grafico ou PainelGraficos")
    go, _, _ = _plotly()
    traces = _trace_3d(go, grafico) if _grafico_eh_3d(grafico) else _trace_2d(go,
      grafico)
    figura = go.Figure(data=traces)
    _aplicar_layout(figura, grafico)
    return figura


def _figura_painel(painel: Any) -> Any:
    from .graficos import PainelGraficos, ErroGrafico, _grafico_eh_3d

    if not isinstance(painel, PainelGraficos):
        raise ErroGrafico("painel interativo inválido")
    go, _, make_subplots = _plotly()
    specs = []
    for linha in range(painel.linhas):
        row = []
        for coluna in range(painel.colunas):
            indice = linha * painel.colunas + coluna
            if indice >= len(painel.graficos):
                row.append(None)
            else:
                row.append({"type": "scene" if _grafico_eh_3d(painel.graficos[indice])
                  else "xy"})
        specs.append(row)
    figura = make_subplots(
        rows=painel.linhas, cols=painel.colunas, specs=specs,
        subplot_titles=[g.titulo or g.tipo for g in painel.graficos],
```

```
    )
    for indice, grafico in enumerate(painel.graficos):
        row = indice // painel.colunas + 1
        col = indice % painel.colunas + 1
        traces = _trace_3d(go, grafico) if _grafico_eh_3d(grafico) else _trace_2d(go,
          grafico)
        for trace in traces:
            figura.add_trace(trace, row=row, col=col)
        if _grafico_eh_3d(grafico):
            scene_index = sum(1 for g in painel.graficos[: indice + 1] if
              _grafico_eh_3d(g))
            scene_key = "scene" if scene_index == 1 else f"scene{scene_index}"
            figura.layout[scene_key].update(
                xaxis_title=_texto_eixo(grafico.rotulo_x, grafico.unidade_x),
                yaxis_title=_texto_eixo(grafico.rotulo_y, grafico.unidade_y),
                zaxis_title=_texto_eixo(grafico.rotulo_z, grafico.unidade_z),
                aspectmode="data",
            )
        else:
            figura.update_xaxes(title_text=_texto_eixo(grafico.rotulo_x,
              grafico.unidade_x), row=row, col=col)
            figura.update_yaxes(title_text=_texto_eixo(grafico.rotulo_y,
              grafico.unidade_y), row=row, col=col)
    figura.update_layout(title=painel.titulo or None)
    return figura


def construir_figura(grafico: Any) -> Any:
    """Converte uma especificação Coral em uma figura Plotly sem abrir janela."""
    from .graficos import Grafico, PainelGraficos, ErroGrafico

    if isinstance(grafico, Grafico):
        return _figura_simples(grafico)
    if isinstance(grafico, PainelGraficos):
        return _figura_painel(grafico)
    raise ErroGrafico("HTML interativo requer Grafico ou PainelGraficos")


def _identificador(grafico: Any) -> str:
    resumo = repr(grafico).encode("utf-8", "strict")
    return "coral-grafico-" + sha256(resumo).hexdigest()[:20]


def _texto_acessivel(grafico: Any) -> str:
    from .graficos import Grafico, PainelGraficos

    if isinstance(grafico, Grafico):
        partes: list[str] = []
        # Preserve o título como metadado textual mesmo quando há texto
        # alternativo. Plotly pode escapar Unicode em seu JSON embutido;
        # manter o título aqui garante acessibilidade e fidelidade editorial
        # no HTML autossuficiente.
        if grafico.titulo:
            partes.append(grafico.titulo)
        if grafico.texto_alternativo:
            if grafico.texto_alternativo != grafico.titulo:
                partes.append(grafico.texto_alternativo)
        elif not grafico.titulo:
            partes.append(f"Gráfico {grafico.tipo}")
        if grafico.avisos:
            partes.append("Avisos: " + "; ".join(grafico.avisos))
```

```
        if grafico.proveniencia:
            partes.append("Proveniência: " + "; ".join(grafico.proveniencia))
        return " ".join(partes)
    if isinstance(grafico, PainelGraficos):
        partes = [grafico.titulo or f"Painel com {len(grafico.graficos)} gráficos"]
        for indice, filho in enumerate(grafico.graficos, start=1):
            partes.append(f"Gráfico {indice}: {_texto_acessivel(filho)}")
        return " ".join(partes)
    return "Gráfico Coral"


def _metadados_acessiveis(grafico: Any) -> str:
    from html import escape

    return (
        '<p class="coral-texto-alternativo" '
        'style="position:absolute;left:-10000px;width:1px;height:1px;overflow:hidden">'
        + escape(_texto_acessivel(grafico))
        + "</p>"
    )


def _quantidade_pontos(grafico: Any) -> int:
    from .graficos import (
        FaixaIncerteza,
        Grafico,
        MatrizCores,
        PainelGraficos,
        ResumoCaixa,
        Serie2D,
        Serie3D,
        SerieCategorias,
        SerieErro,
        Superficie3D,
    )

    if isinstance(grafico, PainelGraficos):
        return sum(_quantidade_pontos(filho) for filho in grafico.graficos)
    if not isinstance(grafico, Grafico):
        return 0
    total = 0
    for elem in grafico.elementos:
        if isinstance(elem, (Serie2D, Serie3D, SerieErro, FaixaIncerteza)):
            total += len(elem.x)
        elif isinstance(elem, SerieCategorias):
            total += len(elem.categorias)
        elif isinstance(elem, MatrizCores):
            total += sum(len(linha) for linha in elem.valores)
        elif isinstance(elem, Superficie3D):
            total += sum(len(linha) for linha in elem.z)
        elif isinstance(elem, ResumoCaixa):
            total += elem.quantidade
    return total


def _validar_limite_pontos(limite_pontos: Any) -> int:
    from .graficos import ErroGrafico

    if isinstance(limite_pontos, bool) or not isinstance(limite_pontos, int) or
      limite_pontos <= 0:
        raise ErroGrafico("limite_pontos deve ser inteiro positivo")
```

```
    return limite_pontos


def _indices_amostra(quantidade: int, max_pontos: int) -> tuple[int, ...]:
    if quantidade <= max_pontos:
        return tuple(range(quantidade))
    if max_pontos == 2:
        return (0, quantidade - 1)
    return tuple(round(i * (quantidade - 1) / (max_pontos - 1)) for i in
      range(max_pontos))


def amostrar_grafico(grafico: Any, max_pontos: int) -> Any:
    """Reduz explicitamente séries simples, registrando a amostragem na especificação.

    A função não é usada automaticamente por ``para_html``. Superfícies, mapas,
    caixas, barras e faixas exigem redução explícita pelo programa porque sua
    semântica não é equivalente à seleção uniforme de observações.
    """
    from dataclasses import replace
    from .graficos import ErroGrafico, Grafico, Serie2D, Serie3D

    if isinstance(max_pontos, bool) or not isinstance(max_pontos, int) or max_pontos <
      2:
        raise ErroGrafico("max_pontos deve ser inteiro maior ou igual a 2")
    if not isinstance(grafico, Grafico) or grafico.tipo not in {"linha", "dispersao",
      "dispersao3d", "trajetoria3d"}:
        raise ErroGrafico(
            "amostrar_grafico aceita linha, dispersao, dispersao3d ou trajetoria3d; "
            "reduza outros formatos explicitamente antes de criar o gráfico"
        )
    if len(grafico.elementos) != 1:
        raise ErroGrafico("amostrar_grafico requer uma série simples")
    elem = grafico.elementos[0]
    quantidade = len(elem.x)
    if quantidade <= max_pontos:
        return grafico
    indices = _indices_amostra(quantidade, max_pontos)
    if isinstance(elem, Serie2D):
        novo = Serie2D(elem.estilo, tuple(elem.x[i] for i in indices), tuple(elem.y[i]
          for i in indices), elem.nome, elem.origem)
    elif isinstance(elem, Serie3D):
        novo = Serie3D(
            elem.estilo,
            tuple(elem.x[i] for i in indices),
            tuple(elem.y[i] for i in indices),
            tuple(elem.z[i] for i in indices),
            elem.nome,
        )
    else:
        raise ErroGrafico("amostrar_grafico encontrou uma série incompatível")
    aviso = f"amostragem uniforme explícita: {quantidade} → {len(indices)} pontos"
    return replace(
        grafico,
        elementos=(novo,),
        avisos=tuple(grafico.avisos) + (aviso,),
        proveniencia=tuple(grafico.proveniencia) +
          (f"amostragem_uniforme:{quantidade}:{len(indices)}",),
    )
```

```
def para_html(grafico: Any, *, limite_pontos: int = LIMITE_PONTOS_HTML_PADRAO) -> str:
    """Gera documento HTML interativo autossuficiente, sem abrir rede ou janela.

    O limite é de proteção contra materialização acidental. A função nunca
    amostra dados silenciosamente. Para uma série grande, o programa pode usar
    ``amostrar_grafico`` de forma explícita ou escolher um limite maior.
    """
    from html import escape
    from .graficos import ErroGrafico

    limite = _validar_limite_pontos(limite_pontos)
    quantidade = _quantidade_pontos(grafico)
    if quantidade > limite:
        raise ErroGrafico(
            f"gráfico possui {quantidade} observações/células, acima do limite HTML de
              {limite}; "
            "reduza os dados explicitamente, use amostrar_grafico quando aplicável ou
              escolha limite_pontos maior"
        )
    _, pio, _ = _plotly()
    figura = construir_figura(grafico)
    div_id = _identificador(grafico)
    html = pio.to_html(
        figura,
        include_plotlyjs=True,
        full_html=True,
        auto_play=False,
        div_id=div_id,
        config={"responsive": True, "displaylogo": False, "scrollZoom": True},
    )
    texto_acessivel = _texto_acessivel(grafico)
    # Quando o autor fornece texto alternativo, ele é o rótulo ARIA canônico.
    # Título, avisos e proveniência continuam disponíveis no metadado textual
    # oculto abaixo, evitando duplicação desnecessária para leitores de tela.
    rotulo_aria = getattr(grafico, "texto_alternativo", None) or texto_acessivel
    alvo = f'id="{div_id}"'
    html = html.replace(
        alvo,
        alvo + ' role="img" aria-label="' + escape(rotulo_aria, quote=True) + '"',
        1,
    )
    marcador = "<body>"
    if marcador in html:
        html = html.replace(
            marcador,
            marcador
            + '\n<div data-coral-contrato="coral.graficos.html/1" '
            + 'data-coral-offline="true" data-coral-contem-dados="true">'
            + '<p class="coral-aviso-dados"><strong>Aviso:</strong> este arquivo HTML
              contém os dados do gráfico incorporados. '
            + 'Revise dados sensíveis antes de compartilhar.</p>'
            + _metadados_acessiveis(grafico)
            + "</div>",
            1,
        )
    return html


def salvar_html(
    grafico: Any,
    caminho: str | Path,
```

```
    *,
    limite_pontos: int = LIMITE_PONTOS_HTML_PADRAO,
) -> Path:
    """Salva HTML interativo autossuficiente usando Plotly somente nesta chamada."""
    from .graficos import ErroGrafico

    destino = Path(caminho)
    if destino.suffix.lower() not in {".html", ".htm"}:
        raise ErroGrafico("salvar_html requer um caminho terminado em .html ou .htm")
    texto = para_html(grafico, limite_pontos=limite_pontos)
    destino.parent.mkdir(parents=True, exist_ok=True)
    temporario = destino.with_name(destino.name + ".tmp-coral")
    temporario.write_text(texto, encoding="utf-8", newline="\n")
    temporario.replace(destino)
    return destino


def _sessao_grafica_disponivel() -> bool:
    import os
    import platform

    if platform.system() in {"Windows", "Darwin"}:
        return True
    return bool(os.environ.get("DISPLAY") or os.environ.get("WAYLAND_DISPLAY"))


def explorar(grafico: Any, *, limite_pontos: int = LIMITE_PONTOS_HTML_PADRAO) -> Path:
    """Abre uma prévia local apenas quando solicitada explicitamente.

    Em sessão sem interface, falha com orientação para ``salvar_html`` e não
    tenta iniciar servidor ou navegador em segundo plano.
    """
    import tempfile
    import webbrowser
    from .graficos import ErroGrafico

    if not _sessao_grafica_disponivel():
        raise ErroGrafico(
            "explorar requer uma sessão gráfica; use salvar_html para gerar um arquivo
              interativo em ambiente sem interface"
        )
    pasta = Path(tempfile.gettempdir()) / "coral" / "graficos_interativos"
    pasta.mkdir(parents=True, exist_ok=True)
    destino = pasta / f"{_identificador(grafico)}.html"
    salvar_html(grafico, destino, limite_pontos=limite_pontos)
    try:
        abriu = webbrowser.open(destino.resolve().as_uri(), new=2)
    except Exception as exc:
        raise ErroGrafico(
            f"não foi possível abrir a prévia interativa; o HTML foi salvo em {destino}.
               Use salvar_html para escolher outro caminho"
        ) from exc
    if not abriu:
        raise ErroGrafico(
            f"nenhum navegador aceitou a prévia; o HTML foi salvo em {destino}. Use
              salvar_html para escolher outro caminho"
        )
    return destino

"""Contratos transversais pequenos do runtime Coral.
```

```
A camada comum não substitui APIs de domínio. Ela fornece a gramática
arquitetural compartilhada do ciclo 1.5 para erro operacional, resultado e
diagnóstico, preservando os nomes públicos existentes em cada biblioteca.
"""
from __future__ import annotations

from dataclasses import asdict, dataclass, field
from typing import Any, Iterable, Mapping

CONTRATO = "coral.comum/1"
CONTRATO_DIAGNOSTICO = "coral.diagnostico.operacional/1"


class ErroCoral(Exception):
    """Raiz pública dos erros controlados pela linguagem e bibliotecas Coral."""


class ErroOperacaoCoral(RuntimeError, ErroCoral):
    """Falha operacional de um recurso Coral ou de seu backend."""


class ErroDependenciaCoral(ErroOperacaoCoral):
    """Uma capacidade opcional não pode operar por ausência de dependência."""


@dataclass(frozen=True)
class FalhaOperacao:
    codigo: str
    mensagem: str
    dominio: str
    tipo: str = "operacional"
    causa: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def de_excecao(cls, exc: BaseException, *, codigo: str, dominio: str, tipo: str =
      "operacional", detalhe: Mapping[str, Any] | None = None) -> "FalhaOperacao":
        causa = exc.__cause__ if exc.__cause__ is not None else exc
        return cls(codigo, str(exc), dominio, tipo, type(causa).__name__, dict(detalhe
          or {}))

    def para_dict(self) -> dict[str, Any]:
        return asdict(self)


@dataclass(frozen=True)
class ResultadoOperacao:
    """Representação transversal sem substituir resultados ricos de domínio."""
    ok: bool
    valor: Any = None
    falha: FalhaOperacao | None = None
    metadados: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def sucesso(cls, valor: Any = None, **metadados: Any) -> "ResultadoOperacao":
        return cls(True, valor=valor, metadados=metadados)

    @classmethod
    def erro(cls, falha: FalhaOperacao, **metadados: Any) -> "ResultadoOperacao":
        return cls(False, falha=falha, metadados=metadados)
```

```
    def para_dict(self) -> dict[str, Any]:
        return {
            "contrato": CONTRATO,
            "ok": self.ok,
            "valor": self.valor,
            "falha": self.falha.para_dict() if self.falha is not None else None,
            "metadados": dict(self.metadados),
        }


@dataclass(frozen=True)
class DiagnosticoOperacional:
    id: str
    dominio: str
    ok: bool
    mensagem: str
    capacidade: str | None = None
    sugestao: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @property
    def nivel(self) -> str:
        return "info" if self.ok else "erro"

    def para_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "dominio": self.dominio,
            "ok": self.ok,
            "nivel": self.nivel,
            "mensagem": self.mensagem,
            "capacidade": self.capacidade,
            "sugestao": self.sugestao,
            "detalhe": dict(self.detalhe),
        }


def diagnostico(id: str, dominio: str, ok: bool, mensagem: Any, *, capacidade: str |
  None = None, sugestao: str | None = None, detalhe: Mapping[str, Any] | None = None) ->
  DiagnosticoOperacional:
    return DiagnosticoOperacional(str(id), str(dominio), bool(ok), str(mensagem),
      capacidade, sugestao, dict(detalhe or {}))


def agregar_diagnosticos(itens: Iterable[DiagnosticoOperacional | Mapping[str, Any]], *,
   origem: str = "coral") -> dict[str, Any]:
    normalizados: list[dict[str, Any]] = []
    for item in itens:
        if isinstance(item, DiagnosticoOperacional):
            normalizados.append(item.para_dict())
            continue
        d = dict(item)
        if "id" not in d:
            d["id"] = str(d.get("nome", "diagnostico"))
        if "dominio" not in d:
            d["dominio"] = origem
        d["ok"] = bool(d.get("ok"))
        d.setdefault("nivel", "info" if d["ok"] else "erro")
        d.setdefault("mensagem", str(d.get("detalhe", "")))
        d.setdefault("capacidade", None)
        d.setdefault("sugestao", None)
```

```
        detalhe = d.get("detalhe")
        if not isinstance(detalhe, Mapping):
            d["detalhe"] = {"valor": detalhe}
        normalizados.append(d)
    return {
        "contrato": CONTRATO_DIAGNOSTICO,
        "origem": origem,
        "ok": all(x["ok"] for x in normalizados),
        "quantidade": len(normalizados),
        "itens": normalizados,
    }


__all__ = [
    "CONTRATO", "CONTRATO_DIAGNOSTICO", "ErroCoral", "ErroOperacaoCoral",
    "ErroDependenciaCoral", "FalhaOperacao", "ResultadoOperacao",
    "DiagnosticoOperacional", "diagnostico", "agregar_diagnosticos",
]

"""Gráficos científicos da Coral.
A camada pública constrói especificações imutáveis para séries, matrizes,
incerteza e painéis. SVG continua disponível somente com a biblioteca padrão.
PNG e PDF usam Matplotlib como backend opcional e ele é carregado apenas quando
a exportação correspondente é solicitada. Importar este módulo não abre janela,
não acessa rede e não carrega Matplotlib, Plotly, NumPy ou Laboratório.
Operações estatísticas pertencem a :mod:`coral.numerico`. Construtores que
precisam de estatística importam Numérico localmente, mantendo cálculo e
apresentação como responsabilidades separadas. O adaptador de experimentos
importa Laboratório apenas durante a chamada explícita de ``grafico_experimento``.
"""
from __future__ import annotations

from dataclasses import dataclass, replace
from html import escape
import json
import math
from pathlib import Path
from collections.abc import Mapping, Sequence
from typing import Any, Callable, Iterable

from .comum import ErroCoral


class ErroGrafico(ValueError, ErroCoral):
    """Erro público para especificações ou dados gráficos inválidos."""


@dataclass(frozen=True)
class Serie2D:
    """Série cartesiana usada por linhas, dispersão e funções.

    ``None`` é aceito internamente apenas como separador explícito de segmentos
    produzidos por :func:`grafico_funcao` quando uma amostra falha ou não é
    finita. Construtores de dados diretos rejeitam valores ausentes.
    """

    estilo: str
    x: tuple[float, ...]
    y: tuple[float | None, ...]
    nome: str | None = None
    origem: str = "direta"
```

```
@dataclass(frozen=True)
class Serie3D:
    """Série cartesiana tridimensional imutável.

    Os três eixos são materializados como tuplas de números finitos. Criar a
    especificação não projeta, não abre janela e não carrega backend visual.
    """

    estilo: str
    x: tuple[float, ...]
    y: tuple[float, ...]
    z: tuple[float, ...]
    nome: str | None = None


@dataclass(frozen=True)
class Superficie3D:
    """Grade tridimensional explícita, sem interpolação de pontos ausentes."""

    x: tuple[tuple[float, ...], ...]
    y: tuple[tuple[float, ...], ...]
    z: tuple[tuple[float, ...], ...]
    nome: str | None = None

    @property
    def forma(self) -> tuple[int, int]:
        return (len(self.z), len(self.z[0]))


@dataclass(frozen=True)
class SerieCategorias:
    """Série categórica para barras ou histograma."""

    estilo: str
    categorias: tuple[str, ...]
    valores: tuple[float, ...]
    limites: tuple[tuple[float, float], ...] = ()
    nome: str | None = None


@dataclass(frozen=True)
class ResumoCaixa:
    """Resumo estatístico de um grupo para um gráfico de caixa."""

    nome: str
    minimo: float
    q1: float
    mediana: float
    q3: float
    maximo: float
    bigode_inferior: float
    bigode_superior: float
    extremos: tuple[float, ...]
    quantidade: int


@dataclass(frozen=True)
class SeriesRegistros:
    """Duas séries alinhadas extraídas de registros comuns."""
```

```
    x: tuple[float, ...]
    y: tuple[float, ...]
    campo_x: str
    campo_y: str

    @property
    def quantidade(self) -> int:
        return len(self.x)


@dataclass(frozen=True)
class MatrizCores:
    """Matriz numérica e rótulos para mapas de calor e correlação."""

    valores: tuple[tuple[float, ...], ...]
    rotulos_linhas: tuple[str, ...]
    rotulos_colunas: tuple[str, ...]
    nome: str | None = None


@dataclass(frozen=True)
class SerieErro:
    """Série cartesiana com incerteza inferior e superior por ponto."""

    x: tuple[float, ...]
    y: tuple[float, ...]
    erro_inferior: tuple[float, ...]
    erro_superior: tuple[float, ...]
    nome: str | None = None


@dataclass(frozen=True)
class FaixaIncerteza:
    """Centro e limites absolutos de uma faixa de incerteza."""

    x: tuple[float, ...]
    centro: tuple[float, ...]
    inferior: tuple[float, ...]
    superior: tuple[float, ...]
    nome: str | None = None


@dataclass(frozen=True)
class PainelGraficos:
    """Composição imutável de gráficos usando uma grade regular."""

    graficos: tuple["Grafico", ...]
    colunas: int
    titulo: str = ""

    @property
    def linhas(self) -> int:
        return (len(self.graficos) + self.colunas - 1) // self.colunas


@dataclass(frozen=True)
class Grafico:
    """Especificação imutável de um gráfico Coral 2D ou 3D."""

    tipo: str
```

```
    elementos: tuple[Any, ...]
    titulo: str = ""
    rotulo_x: str = ""
    rotulo_y: str = ""
    rotulo_z: str = ""
    unidade_x: str = ""
    unidade_y: str = ""
    unidade_z: str = ""
    legenda: bool = True
    tema: str = "claro"
    proveniencia: tuple[str, ...] = ()
    texto_alternativo: str = ""
    avisos: tuple[str, ...] = ()


_TEMAS = {
    "claro": {
        "fundo": "#ffffff", "texto": "#202124", "grade": "#d9dce1",
        "eixo": "#59636e", "serie": "#1565c0", "serie2": "#c62828",
        "preenchimento": "#90caf9",
    },
    "escuro": {
        "fundo": "#15181d", "texto": "#f1f3f4", "grade": "#3c4043",
        "eixo": "#bdc1c6", "serie": "#8ab4f8", "serie2": "#f28b82",
        "preenchimento": "#3f6f9f",
    },
    "alto_contraste": {
        "fundo": "#ffffff", "texto": "#000000", "grade": "#777777",
        "eixo": "#000000", "serie": "#0057b8", "serie2": "#a00000",
        "preenchimento": "#d6e8ff",
    },
}


def _numero_finito(valor: Any, contexto: str) -> float:
    if isinstance(valor, bool):
        raise ErroGrafico(f"{contexto} requer número real finito; booleano não é
          aceito")
    try:
        numero = float(valor)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroGrafico(f"{contexto} requer número real finito") from exc
    if not math.isfinite(numero):
        raise ErroGrafico(f"{contexto} não aceita valor ausente, NaN ou infinito")
    return numero


def _serie_numerica(valores: Iterable[Any], contexto: str, *, permitir_vazia: bool =
  False) -> tuple[float, ...]:
    if isinstance(valores, (str, bytes, bytearray)):
        raise ErroGrafico(f"{contexto} requer uma coleção de números")
    try:
        itens = tuple(valores)
    except TypeError as exc:
        raise ErroGrafico(f"{contexto} requer uma coleção de números") from exc
    if not itens and not permitir_vazia:
        raise ErroGrafico(f"{contexto} requer ao menos um valor")
    return tuple(_numero_finito(item, contexto) for item in itens)


def _pares_xy(x: Iterable[Any], y: Iterable[Any], contexto: str) -> tuple[tuple[float,
```

```
  ...], tuple[float, ...]]:
    xs = _serie_numerica(x, f"{contexto}: eixo x")
    ys = _serie_numerica(y, f"{contexto}: eixo y")
    if len(xs) != len(ys):
        raise ErroGrafico(f"{contexto} requer x e y com o mesmo comprimento")
    return xs, ys


def _grafico_cartesiano(estilo: str, x: Iterable[Any], y: Iterable[Any], *, nome: str |
  None = None) -> Grafico:
    xs, ys = _pares_xy(x, y, estilo)
    return Grafico(tipo=estilo, elementos=(Serie2D(estilo, xs, ys, nome),))


def linha(x: Iterable[Any], y: Iterable[Any], nome: str | None = None) -> Grafico:
    """Cria uma série de linha 2D sem abrir interface gráfica."""
    return _grafico_cartesiano("linha", x, y, nome=nome)


def dispersao(x: Iterable[Any], y: Iterable[Any], nome: str | None = None) -> Grafico:
    """Cria uma série de pontos 2D sem conectar observações."""
    return _grafico_cartesiano("dispersao", x, y, nome=nome)


def _triplas_xyz(
    x: Iterable[Any], y: Iterable[Any], z: Iterable[Any], contexto: str
) -> tuple[tuple[float, ...], tuple[float, ...], tuple[float, ...]]:
    xs = _serie_numerica(x, f"{contexto}: eixo x")
    ys = _serie_numerica(y, f"{contexto}: eixo y")
    zs = _serie_numerica(z, f"{contexto}: eixo z")
    if len(xs) != len(ys) or len(xs) != len(zs):
        raise ErroGrafico(f"{contexto} requer x, y e z com o mesmo comprimento")
    return xs, ys, zs


def dispersao3d(
    x: Iterable[Any], y: Iterable[Any], z: Iterable[Any], nome: str | None = None
) -> Grafico:
    """Cria pontos 3D preservando exatamente as coordenadas fornecidas."""
    xs, ys, zs = _triplas_xyz(x, y, z, "dispersao3d")
    return Grafico(tipo="dispersao3d", elementos=(Serie3D("dispersao3d", xs, ys, zs,
      nome),))


def trajetoria3d(
    x: Iterable[Any], y: Iterable[Any], z: Iterable[Any], nome: str | None = None
) -> Grafico:
    """Cria uma trajetória 3D na ordem dada, sem reamostrar ou suavizar."""
    xs, ys, zs = _triplas_xyz(x, y, z, "trajetoria3d")
    return Grafico(tipo="trajetoria3d", elementos=(Serie3D("trajetoria3d", xs, ys, zs,
      nome),))


def _grade_superficie3d(x: Any, y: Any, z: Any) -> Superficie3D:
    zs = _matriz_numerica_retangular(z, "superficie3d: z")
    linhas, colunas = len(zs), len(zs[0])

    def eh_matriz(valor: Any) -> bool:
        if isinstance(valor, (str, bytes, bytearray)):
            return False
        try:
```

```
            itens = tuple(valor)
        except TypeError:
            return False
        return bool(itens) and all(
            isinstance(item, Sequence) and not isinstance(item, (str, bytes, bytearray))
            for item in itens
        )

    if eh_matriz(x) or eh_matriz(y):
        if not (eh_matriz(x) and eh_matriz(y)):
            raise ErroGrafico("superficie3d requer x e y ambos como eixos ou ambos como
              grades")
        xs = _matriz_numerica_retangular(x, "superficie3d: x")
        ys = _matriz_numerica_retangular(y, "superficie3d: y")
        if len(xs) != linhas or len(ys) != linhas or len(xs[0]) != colunas or len(ys[0])
           != colunas:
            raise ErroGrafico("superficie3d requer grades x, y e z com a mesma forma")
        return Superficie3D(xs, ys, zs)

    xs_eixo = _serie_numerica(x, "superficie3d: eixo x")
    ys_eixo = _serie_numerica(y, "superficie3d: eixo y")
    if len(xs_eixo) != colunas or len(ys_eixo) != linhas:
        raise ErroGrafico(
            "superficie3d requer len(x) igual ao número de colunas de z e len(y) igual
              ao número de linhas"
        )
    xs = tuple(tuple(xs_eixo[j] for j in range(colunas)) for _ in range(linhas))
    ys = tuple(tuple(ys_eixo[i] for _ in range(colunas)) for i in range(linhas))
    return Superficie3D(xs, ys, zs)


def superficie3d(x: Any, y: Any, z: Any, nome: str | None = None) -> Grafico:
    """Cria uma superfície 3D de grade retangular sem interpolação silenciosa.

    ``x`` e ``y`` podem ser eixos unidimensionais compatíveis com a matriz ``z``
    ou grades bidimensionais da mesma forma de ``z``. Valores ausentes e formas
    irregulares são rejeitados em vez de serem preenchidos automaticamente.
    """
    grade = _grade_superficie3d(x, y, z)
    if nome is not None:
        grade = replace(grade, nome=str(nome))
    return Grafico(tipo="superficie3d", elementos=(grade,))


def barras(categorias: Iterable[Any], valores: Iterable[Any], nome: str | None = None)
  -> Grafico:
    """Cria barras categóricas; a escala vertical inclui zero por contrato."""
    if isinstance(categorias, (str, bytes, bytearray)):
        raise ErroGrafico("barras requer uma coleção de categorias")
    try:
        cats_brutas = tuple(categorias)
    except TypeError as exc:
        raise ErroGrafico("barras requer uma coleção de categorias") from exc
    ys = _serie_numerica(valores, "barras")
    if not cats_brutas:
        raise ErroGrafico("barras requer ao menos uma categoria")
    if len(cats_brutas) != len(ys):
        raise ErroGrafico("barras requer categorias e valores com o mesmo comprimento")
    cats: list[str] = []
    for item in cats_brutas:
        if item is None:
```

```
            raise ErroGrafico("barras não aceita categoria ausente")
        texto = str(item).strip()
        if not texto:
            raise ErroGrafico("barras não aceita categoria vazia")
        cats.append(texto)
    serie = SerieCategorias("barras", tuple(cats), ys, nome=nome)
    return Grafico(tipo="barras", elementos=(serie,))


def series_de_registros(registros: Iterable[Any], campo_x: str, campo_y: str) ->
  SeriesRegistros:
    """Extrai dois campos numéricos alinhados de registros comuns.

    Registros podem ser mapeamentos ou objetos com atributos. Campo ausente e
    valor ausente são erros explícitos; nenhuma linha é descartada em silêncio.
    """
    if not isinstance(campo_x, str) or not campo_x.strip() or not isinstance(campo_y,
      str) or not campo_y.strip():
        raise ErroGrafico("series_de_registros requer nomes de campos não vazios")
    if isinstance(registros, (str, bytes, bytearray)):
        raise ErroGrafico("series_de_registros requer uma coleção de registros")
    try:
        itens = tuple(registros)
    except TypeError as exc:
        raise ErroGrafico("series_de_registros requer uma coleção de registros") from
          exc
    if not itens:
        raise ErroGrafico("series_de_registros requer ao menos um registro")

    xs: list[float] = []
    ys: list[float] = []
    for indice, registro in enumerate(itens):
        if isinstance(registro, Mapping):
            if campo_x not in registro or campo_y not in registro:
                faltantes = [campo for campo in (campo_x, campo_y) if campo not in
                  registro]
                raise ErroGrafico(
                    f"registro {indice} não possui campo(s): {', '.join(faltantes)}"
                )
            vx, vy = registro[campo_x], registro[campo_y]
        else:
            faltantes = [campo for campo in (campo_x, campo_y) if not hasattr(registro,
              campo)]
            if faltantes:
                raise ErroGrafico(
                    f"registro {indice} não possui campo(s): {', '.join(faltantes)}"
                )
            vx, vy = getattr(registro, campo_x), getattr(registro, campo_y)
        xs.append(_numero_finito(vx, f"registro {indice}, campo {campo_x}"))
        ys.append(_numero_finito(vy, f"registro {indice}, campo {campo_y}"))
    return SeriesRegistros(tuple(xs), tuple(ys), campo_x, campo_y)


def grafico_histograma(valores: Iterable[Any], faixas: Any, nome: str | None = None) ->
  Grafico:
    """Cria gráfico a partir do contrato canônico ``numerico.histograma``."""
    # Importação local mantém `import coral.graficos` livre de NumPy.
    from . import numerico

    try:
        contagens, bordas = numerico.histograma(valores, faixas)
```

```
    except Exception as exc:
        if isinstance(exc, ErroCoral):
            raise
        raise ErroGrafico(f"não foi possível construir o histograma: {exc}") from exc
    cont = tuple(float(v) for v in contagens.tolist())
    edges = tuple(float(v) for v in bordas.tolist())
    limites = tuple(zip(edges[:-1], edges[1:]))
    categorias = tuple(f"{a:g}–{b:g}" for a, b in limites)
    serie = SerieCategorias("histograma", categorias, cont, limites=limites, nome=nome)
    return Grafico(tipo="histograma", elementos=(serie,))


def _normalizar_grupos(grupos: Any) -> tuple[tuple[str, tuple[Any, ...]], ...]:
    if isinstance(grupos, Mapping):
        if not grupos:
            raise ErroGrafico("grafico_caixa requer ao menos um grupo")
        saida = []
        for nome, valores in grupos.items():
            if valores is None or isinstance(valores, (str, bytes, bytearray)):
                raise ErroGrafico(f"grupo {nome!r} requer uma coleção numérica")
            try:
                materializados = tuple(valores)
            except TypeError as exc:
                raise ErroGrafico(f"grupo {nome!r} requer uma coleção numérica") from
                  exc
            saida.append((str(nome), materializados))
        return tuple(saida)

    if isinstance(grupos, (str, bytes, bytearray)):
        raise ErroGrafico("grafico_caixa requer grupos numéricos")
    try:
        itens = tuple(grupos)
    except TypeError as exc:
        raise ErroGrafico("grafico_caixa requer grupos numéricos") from exc
    if not itens:
        raise ErroGrafico("grafico_caixa requer ao menos um grupo")

    primeiro = itens[0]
    eh_grupo_unico = not isinstance(primeiro, (Sequence, Mapping)) or
      isinstance(primeiro, (str, bytes, bytearray))
    if eh_grupo_unico:
        return (("Dados", itens),)

    saida = []
    for indice, valores in enumerate(itens, start=1):
        if isinstance(valores, (str, bytes, bytearray)):
            raise ErroGrafico(f"grupo {indice} requer uma coleção numérica")
        try:
            materializados = tuple(valores)
        except TypeError as exc:
            raise ErroGrafico(f"grupo {indice} requer uma coleção numérica") from exc
        saida.append((f"Grupo {indice}", materializados))
    return tuple(saida)


def grafico_caixa(grupos: Any) -> Grafico:
    """Cria caixas de Tukey usando quartis do contrato ``coral.numerico``.

    Bigodes usam 1,5 × IQR. Valores além dos limites são preservados em
    ``extremos``. Grupos com menos de cinco observações recebem aviso explícito.
    """
```

```
    from . import numerico

    resumos: list[ResumoCaixa] = []
    avisos: list[str] = []
    for nome, valores_brutos in _normalizar_grupos(grupos):
        valores = _serie_numerica(valores_brutos, f"grafico_caixa: grupo {nome}")
        q1, med, q3 = (float(v) for v in numerico.quartis(valores))
        ordenados = sorted(valores)
        iqr = q3 - q1
        limite_inf = q1 - 1.5 * iqr
        limite_sup = q3 + 1.5 * iqr
        internos = [v for v in ordenados if limite_inf <= v <= limite_sup]
        if not internos:  # defensivo; mediana e quartis garantem que normalmente há
          internos.
            internos = ordenados
        extremos = tuple(v for v in ordenados if v < limite_inf or v > limite_sup)
        resumos.append(ResumoCaixa(
            nome=nome,
            minimo=ordenados[0],
            q1=q1,
            mediana=med,
            q3=q3,
            maximo=ordenados[-1],
            bigode_inferior=internos[0],
            bigode_superior=internos[-1],
            extremos=extremos,
            quantidade=len(ordenados),
        ))
        if len(ordenados) < 5:
            avisos.append(f"grupo {nome!r} possui apenas {len(ordenados)} observações")
    return Grafico(tipo="caixa", elementos=tuple(resumos), avisos=tuple(avisos))


def _rotulos_eixos(valor: Any, contexto: str) -> tuple[str, str, str]:
    if valor is None:
        return "", "", ""
    if isinstance(valor, Mapping):
        return str(valor.get("x", "")), str(valor.get("y", "")), str(valor.get("z", ""))
    if isinstance(valor, (str, bytes, bytearray)):
        raise ErroGrafico(f"{contexto} requer par/trio (x, y[, z]) ou mapeamento com x,
          y e z")
    try:
        itens = tuple(valor)
    except TypeError as exc:
        raise ErroGrafico(f"{contexto} requer par/trio (x, y[, z]) ou mapeamento com x,
          y e z") from exc
    if len(itens) not in {2, 3}:
        raise ErroGrafico(f"{contexto} requer dois ou três valores")
    if len(itens) == 2:
        return str(itens[0]), str(itens[1]), ""
    return str(itens[0]), str(itens[1]), str(itens[2])


def configurar(
    grafico: Grafico,
    titulo: str | None = None,
    eixos: Any = None,
    unidades: Any = None,
    legenda: bool | None = None,
    tema: str | None = None,
    *,
```

```
    proveniencia: Iterable[Any] | None = None,
    texto_alternativo: str | None = None,
) -> Grafico:
    """Retorna cópia configurada sem alterar dados ou estatísticas do gráfico."""
    if not isinstance(grafico, Grafico):
        raise ErroGrafico("configurar requer uma especificação Grafico")
    rx, ry, rz = _rotulos_eixos(eixos, "eixos") if eixos is not None else
      (grafico.rotulo_x, grafico.rotulo_y, grafico.rotulo_z)
    ux, uy, uz = _rotulos_eixos(unidades, "unidades") if unidades is not None else
      (grafico.unidade_x, grafico.unidade_y, grafico.unidade_z)
    if tema is None:
        tema_novo = grafico.tema
    else:
        tema_novo = str(tema).strip().lower().replace(" ", "_")
        if tema_novo not in _TEMAS:
            raise ErroGrafico("tema desconhecido; use claro, escuro ou alto_contraste")
    if legenda is not None and not isinstance(legenda, bool):
        raise ErroGrafico("legenda deve ser verdadeiro ou falso")
    if proveniencia is None:
        prov = grafico.proveniencia
    else:
        if isinstance(proveniencia, (str, bytes, bytearray)):
            prov = (str(proveniencia),)
        else:
            try:
                prov = tuple(str(v) for v in proveniencia)
            except TypeError as exc:
                raise ErroGrafico("proveniência deve ser texto ou coleção de textos")
                  from exc
    return replace(
        grafico,
        titulo=grafico.titulo if titulo is None else str(titulo),
        rotulo_x=rx,
        rotulo_y=ry,
        rotulo_z=rz,
        unidade_x=ux,
        unidade_y=uy,
        unidade_z=uz,
        legenda=grafico.legenda if legenda is None else legenda,
        tema=tema_novo,
        proveniencia=prov,
        texto_alternativo=grafico.texto_alternativo if texto_alternativo is None else
          str(texto_alternativo),
    )


def grafico_funcao(funcao: Callable[[float], Any], inicio: Any, fim: Any, amostras: int
  = 200) -> Grafico:
    """Amostra função escalar em intervalo fechado, preservando falhas como lacunas."""
    if not callable(funcao):
        raise ErroGrafico("grafico_funcao requer uma função chamável")
    a = _numero_finito(inicio, "grafico_funcao: início")
    b = _numero_finito(fim, "grafico_funcao: fim")
    if a == b:
        raise ErroGrafico("grafico_funcao requer início e fim diferentes")
    if isinstance(amostras, bool):
        raise ErroGrafico("amostras deve ser inteiro entre 2 e 100000")
    try:
        n = int(amostras)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroGrafico("amostras deve ser inteiro entre 2 e 100000") from exc
```

```
    try:
        exato = amostras == n
    except Exception:
        exato = False
    if not exato or not 2 <= n <= 100_000:
        raise ErroGrafico("amostras deve ser inteiro entre 2 e 100000")
    passo = (b - a) / (n - 1)
    xs: list[float] = []
    ys: list[float | None] = []
    validos = 0
    for i in range(n):
        x = a + passo * i
        xs.append(x)
        try:
            y_bruto = funcao(x)
            y = _numero_finito(y_bruto, "grafico_funcao: retorno")
        except Exception:
            y = None
        ys.append(y)
        validos += y is not None
    if validos == 0:
        raise ErroGrafico("grafico_funcao não obteve nenhuma amostra finita")
    serie = Serie2D("linha", tuple(xs), tuple(ys), origem="funcao")
    avisos = () if validos == n else (f"{n - validos} amostra(s) inválida(s) foram
      representadas como lacunas",)
    return Grafico(tipo="funcao", elementos=(serie,), avisos=avisos)


def _matriz_numerica_retangular(matriz: Any, contexto: str) -> tuple[tuple[float, ...],
  ...]:
    if isinstance(matriz, (str, bytes, bytearray)):
        raise ErroGrafico(f"{contexto} requer matriz numérica bidimensional")
    try:
        linhas_brutas = tuple(matriz)
    except TypeError as exc:
        raise ErroGrafico(f"{contexto} requer matriz numérica bidimensional") from exc
    if not linhas_brutas:
        raise ErroGrafico(f"{contexto} requer ao menos uma linha")
    linhas: list[tuple[float, ...]] = []
    largura: int | None = None
    for i, linha in enumerate(linhas_brutas):
        if isinstance(linha, (str, bytes, bytearray)):
            raise ErroGrafico(f"{contexto}: linha {i} não é uma coleção numérica")
        try:
            valores = tuple(linha)
        except TypeError as exc:
            raise ErroGrafico(f"{contexto}: linha {i} não é uma coleção numérica") from
              exc
        if not valores:
            raise ErroGrafico(f"{contexto} não aceita linha vazia")
        if largura is None:
            largura = len(valores)
        elif len(valores) != largura:
            raise ErroGrafico(f"{contexto} requer matriz retangular")
        linhas.append(tuple(_numero_finito(v, f"{contexto}: linha {i}") for v in
          valores))
    return tuple(linhas)


def _rotulos_matriz(rotulos: Any, quantidade: int, contexto: str) -> tuple[str, ...]:
    if rotulos is None:
```

```
        return tuple(str(i + 1) for i in range(quantidade))
    if isinstance(rotulos, (str, bytes, bytearray)):
        raise ErroGrafico(f"{contexto} requer uma coleção de rótulos")
    try:
        itens = tuple(str(v) for v in rotulos)
    except TypeError as exc:
        raise ErroGrafico(f"{contexto} requer uma coleção de rótulos") from exc
    if len(itens) != quantidade:
        raise ErroGrafico(f"{contexto} requer exatamente {quantidade} rótulo(s)")
    return itens


def mapa_calor(
    matriz: Any,
    rotulos_linhas: Iterable[Any] | None = None,
    rotulos_colunas: Iterable[Any] | None = None,
    nome: str | None = None,
) -> Grafico:
    """Representa uma matriz numérica por intensidade de cor, sem recalcular dados."""
    valores = _matriz_numerica_retangular(matriz, "mapa_calor")
    nl, nc = len(valores), len(valores[0])
    linhas = _rotulos_matriz(rotulos_linhas, nl, "rotulos_linhas")
    colunas = _rotulos_matriz(rotulos_colunas, nc, "rotulos_colunas")
    return Grafico(
        tipo="mapa_calor",
        elementos=(MatrizCores(valores, linhas, colunas, nome),),
    )


def mapa_correlacao(dados: Any, rotulos: Iterable[Any] | None = None, nome: str | None =
   None) -> Grafico:
    """Calcula Pearson em ``coral.numerico`` e representa a matriz resultante."""
    from . import numerico

    try:
        matriz = numerico.matriz_correlacao(dados)
    except Exception as exc:
        if isinstance(exc, ErroCoral):
            raise
        raise ErroGrafico(f"não foi possível calcular a matriz de correlação: {exc}")
          from exc
    valores = tuple(tuple(float(v) for v in linha) for linha in matriz.tolist())
    n = len(valores)
    nomes = _rotulos_matriz(rotulos, n, "rotulos")
    return Grafico(
        tipo="mapa_correlacao",
        elementos=(MatrizCores(valores, nomes, nomes, nome),),
        proveniencia=("coral.numerico.matriz_correlacao",),
    )


def _normalizar_erros(erros: Iterable[Any], quantidade: int) -> tuple[tuple[float, ...],
   tuple[float, ...]]:
    if isinstance(erros, (str, bytes, bytearray)):
        raise ErroGrafico("barras_erro requer coleção de erros")
    try:
        itens = tuple(erros)
    except TypeError as exc:
        raise ErroGrafico("barras_erro requer coleção de erros") from exc
    if len(itens) != quantidade:
        raise ErroGrafico("barras_erro requer um erro por ponto")
```

```
    inferiores: list[float] = []
    superiores: list[float] = []
    for i, item in enumerate(itens):
        if isinstance(item, Sequence) and not isinstance(item, (str, bytes, bytearray)):
            par = tuple(item)
            if len(par) != 2:
                raise ErroGrafico(f"erro {i} deve ser valor ou par (inferior,
                  superior)")
            ei = _numero_finito(par[0], f"erro {i}")
            es = _numero_finito(par[1], f"erro {i}")
        else:
            ei = es = _numero_finito(item, f"erro {i}")
        if ei < 0 or es < 0:
            raise ErroGrafico("barras_erro não aceita incerteza negativa")
        inferiores.append(ei)
        superiores.append(es)
    return tuple(inferiores), tuple(superiores)


def barras_erro(x: Iterable[Any], y: Iterable[Any], erros: Iterable[Any], nome: str |
  None = None) -> Grafico:
    """Cria pontos com barras de erro simétricas ou assimétricas explicitamente
      fornecidas."""
    xs, ys = _pares_xy(x, y, "barras_erro")
    inferiores, superiores = _normalizar_erros(erros, len(xs))
    return Grafico(
        tipo="barras_erro",
        elementos=(SerieErro(xs, ys, inferiores, superiores, nome),),
    )


def faixa_incerteza(
    x: Iterable[Any],
    centro: Iterable[Any],
    inferior: Iterable[Any],
    superior: Iterable[Any],
    nome: str | None = None,
) -> Grafico:
    """Cria faixa de limites absolutos sem atribuir significado estatístico à faixa."""
    xs, centros = _pares_xy(x, centro, "faixa_incerteza")
    inf = _serie_numerica(inferior, "faixa_incerteza: inferior")
    sup = _serie_numerica(superior, "faixa_incerteza: superior")
    if len(inf) != len(xs) or len(sup) != len(xs):
        raise ErroGrafico("faixa_incerteza requer x, centro, inferior e superior com o
          mesmo comprimento")
    for i, (a, c, b) in enumerate(zip(inf, centros, sup)):
        if a > c or c > b:
            raise ErroGrafico(f"faixa_incerteza requer inferior <= centro <= superior no
               ponto {i}")
    aviso = "A faixa representa os limites fornecidos pelo programa; não implica
      intervalo de confiança."
    return Grafico(
        tipo="faixa_incerteza",
        elementos=(FaixaIncerteza(xs, centros, inf, sup, nome),),
        avisos=(aviso,),
    )


def _valor_medicao_grafico(medicao: Any, chave: str, *, eixo: bool = False) -> Any:
    if chave == "repeticao":
        return medicao.repeticao
```

```
    if chave == "semente":
        return medicao.semente
    if chave in {"tempo_segundos", "memoria_atual_bytes", "memoria_pico_bytes",
      "retorno"}:
        return getattr(medicao, chave)
    if chave.startswith("parametros."):
        return medicao.parametros.get(chave.split(".", 1)[1])
    if chave.startswith("metricas."):
        return medicao.metricas.get(chave.split(".", 1)[1])
    if chave in medicao.parametros:
        return medicao.parametros[chave]
    if chave in medicao.metricas:
        return medicao.metricas[chave]
    tipo = "eixo" if eixo else "métrica"
    raise ErroGrafico(f"{tipo} {chave!r} não existe na medição")


def grafico_experimento(resultado: Any, eixo_x: str, metrica: str) -> Grafico:
    """Adapta medições bem sucedidas a uma série comum sem dependência no import do
      módulo."""
    from .laboratorio import ResultadoExperimento

    if not isinstance(resultado, ResultadoExperimento):
        raise ErroGrafico("grafico_experimento requer ResultadoExperimento")
    if not isinstance(eixo_x, str) or not eixo_x.strip() or not isinstance(metrica, str)
       or not metrica.strip():
        raise ErroGrafico("eixo_x e metrica devem ser textos não vazios")
    xs: list[float] = []
    ys: list[float] = []
    falhas = 0
    sementes: list[str] = []
    for medicao in resultado.medicoes:
        if medicao.erro is not None:
            falhas += 1
            continue
        vx = _valor_medicao_grafico(medicao, eixo_x, eixo=True)
        vy = _valor_medicao_grafico(medicao, metrica)
        xs.append(_numero_finito(vx, f"grafico_experimento: eixo {eixo_x}"))
        ys.append(_numero_finito(vy, f"grafico_experimento: métrica {metrica}"))
        if medicao.semente is not None:
            sementes.append(str(medicao.semente))
    if not xs:
        raise ErroGrafico("grafico_experimento não encontrou medições bem sucedidas e
          numéricas")

    protocolo = resultado.metadados.get("protocolo_medicao", {})
    unidade_y = ""
    if metrica == "tempo_segundos":
        unidade_y = str(protocolo.get("tempo_unidade", "segundos"))
    elif metrica in {"memoria_atual_bytes", "memoria_pico_bytes"}:
        unidade_y = str(protocolo.get("memoria_unidade", "bytes"))
    avisos = () if falhas == 0 else (f"{falhas} medição(ões) com falha foram omitidas",)
    proveniencia = [f"experimento:{resultado.nome}"]
    if sementes:
        proveniencia.append("sementes:" + ",".join(sementes))
    if protocolo:
        proveniencia.append("protocolo:" + json.dumps(protocolo, ensure_ascii=False,
          sort_keys=True))
    serie = Serie2D("dispersao", tuple(xs), tuple(ys), nome=resultado.nome,
      origem="experimento")
    return Grafico(
```

```
        tipo="dispersao",
        elementos=(serie,),
        rotulo_x=eixo_x,
        rotulo_y=metrica,
        unidade_y=unidade_y,
        proveniencia=tuple(proveniencia),
        avisos=avisos,
    )


def painel(graficos: Iterable[Grafico], colunas: int = 2, titulo: str = "") ->
  PainelGraficos:
    """Combina gráficos em grade sem modificar suas escalas ou especificações."""
    if isinstance(graficos, (str, bytes, bytearray)):
        raise ErroGrafico("painel requer uma coleção de gráficos")
    try:
        itens = tuple(graficos)
    except TypeError as exc:
        raise ErroGrafico("painel requer uma coleção de gráficos") from exc
    if not itens or not all(isinstance(g, Grafico) for g in itens):
        raise ErroGrafico("painel requer ao menos um Grafico válido")
    if isinstance(colunas, bool):
        raise ErroGrafico("colunas deve ser inteiro positivo")
    try:
        n = int(colunas)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroGrafico("colunas deve ser inteiro positivo") from exc
    if n != colunas or n <= 0:
        raise ErroGrafico("colunas deve ser inteiro positivo")
    return PainelGraficos(itens, min(n, len(itens)), str(titulo))


def _limites_cartesianos(grafico: Grafico) -> tuple[float, float, float, float]:
    xs: list[float] = []
    ys: list[float] = []
    for elem in grafico.elementos:
        if isinstance(elem, Serie2D):
            for x, y in zip(elem.x, elem.y):
                if y is not None:
                    xs.append(float(x)); ys.append(float(y))
        elif isinstance(elem, SerieErro):
            xs.extend(elem.x)
            ys.extend(y - ei for y, ei in zip(elem.y, elem.erro_inferior))
            ys.extend(y + es for y, es in zip(elem.y, elem.erro_superior))
        elif isinstance(elem, FaixaIncerteza):
            xs.extend(elem.x)
            ys.extend(elem.inferior)
            ys.extend(elem.superior)
    if not xs or not ys:
        raise ErroGrafico("gráfico cartesiano sem pontos finitos")
    xmin, xmax = min(xs), max(xs)
    ymin, ymax = min(ys), max(ys)
    if xmin == xmax:
        delta = max(abs(xmin) * 0.05, 1.0)
        xmin -= delta; xmax += delta
    if ymin == ymax:
        delta = max(abs(ymin) * 0.05, 1.0)
        ymin -= delta; ymax += delta
    return xmin, xmax, ymin, ymax
```

```
def _expandir_intervalo(a: float, b: float, *, incluir_zero: bool = False) ->
  tuple[float, float]:
    if incluir_zero:
        a, b = min(a, 0.0), max(b, 0.0)
    if a == b:
        d = max(abs(a) * 0.05, 1.0)
        return a - d, b + d
    margem = (b - a) * 0.05
    return a - margem, b + margem


def _texto_eixo(rotulo: str, unidade: str) -> str:
    if rotulo and unidade:
        return f"{rotulo} ({unidade})"
    return rotulo or unidade


def _ticks(a: float, b: float, quantidade: int = 5) -> tuple[float, ...]:
    if quantidade <= 1:
        return (a,)
    return tuple(a + (b - a) * i / (quantidade - 1) for i in range(quantidade))


def _svg_inicio(grafico: Grafico, largura: int, altura: int) -> list[str]:
    tema = _TEMAS[grafico.tema]
    alt = grafico.texto_alternativo or grafico.titulo or f"Gráfico {grafico.tipo}"
    metadados = {
        "contrato": "coral.graficos.svg/1",
        "tipo": grafico.tipo,
        "proveniencia": list(grafico.proveniencia),
        "avisos": list(grafico.avisos),
    }
    return [
        '<?xml version="1.0" encoding="UTF-8"?>',
        f'<svg xmlns="http://www.w3.org/2000/svg" width="{largura}" height="{altura}"
          viewBox="0 0 {largura} {altura}" role="img">',
        f'<title>{escape(alt)}</title>',
        f'<desc>{escape(alt)}</desc>',
        f'<metadata>{escape(json.dumps(metadados, ensure_ascii=False,
          sort_keys=True))}</metadata>',
        f'<rect x="0" y="0" width="{largura}" height="{altura}"
          fill="{tema["fundo"]}"/>',
    ]


def _render_cartesiano(grafico: Grafico, largura: int, altura: int) -> list[str]:
    tema = _TEMAS[grafico.tema]
    esquerda, direita, topo, fundo = 82.0, largura - 32.0, 54.0, altura - 72.0
    xmin, xmax, ymin, ymax = _limites_cartesianos(grafico)
    xmin, xmax = _expandir_intervalo(xmin, xmax)
    ymin, ymax = _expandir_intervalo(ymin, ymax)
    sx = lambda x: esquerda + (x - xmin) / (xmax - xmin) * (direita - esquerda)
    sy = lambda y: fundo - (y - ymin) / (ymax - ymin) * (fundo - topo)
    out: list[str] = []
    for tick in _ticks(ymin, ymax):
        py = sy(tick)
        out.append(f'<line x1="{esquerda:.2f}" y1="{py:.2f}" x2="{direita:.2f}"
          y2="{py:.2f}" stroke="{tema["grade"]}" stroke-width="1"/>')
        out.append(f'<text x="{esquerda - 8:.2f}" y="{py + 4:.2f}" text-anchor="end"
          font-size="11" fill="{tema["texto"]}">{tick:.4g}</text>')
    for tick in _ticks(xmin, xmax):
```

```
        px = sx(tick)
        out.append(f'<line x1="{px:.2f}" y1="{topo:.2f}" x2="{px:.2f}" y2="{fundo:.2f}"
          stroke="{tema["grade"]}" stroke-width="1"/>')
        out.append(f'<text x="{px:.2f}" y="{fundo + 20:.2f}" text-anchor="middle"
          font-size="11" fill="{tema["texto"]}">{tick:.4g}</text>')
    out.append(f'<line x1="{esquerda}" y1="{fundo}" x2="{direita}" y2="{fundo}"
      stroke="{tema["eixo"]}" stroke-width="1.5"/>')
    out.append(f'<line x1="{esquerda}" y1="{topo}" x2="{esquerda}" y2="{fundo}"
      stroke="{tema["eixo"]}" stroke-width="1.5"/>')

    for indice, elem in enumerate(grafico.elementos):
        if not isinstance(elem, Serie2D):
            continue
        cor = tema["serie"] if indice % 2 == 0 else tema["serie2"]
        if elem.estilo == "linha":
            comandos: list[str] = []
            novo = True
            for x, y in zip(elem.x, elem.y):
                if y is None:
                    novo = True
                    continue
                comandos.append(f'{"M" if novo else "L"} {sx(x):.2f} {sy(y):.2f}')
                novo = False
            if comandos:
                out.append(f'<path d="{" ".join(comandos)}" fill="none" stroke="{cor}"
                  stroke-width="2"/>')
        if elem.estilo == "dispersao":
            for x, y in zip(elem.x, elem.y):
                if y is not None:
                    out.append(f'<circle cx="{sx(x):.2f}" cy="{sy(y):.2f}" r="3.5"
                      fill="{cor}"/>')
    return out


def _render_incerteza(grafico: Grafico, largura: int, altura: int) -> list[str]:
    tema = _TEMAS[grafico.tema]
    esquerda, direita, topo, fundo = 82.0, largura - 32.0, 54.0, altura - 72.0
    xmin, xmax, ymin, ymax = _limites_cartesianos(grafico)
    xmin, xmax = _expandir_intervalo(xmin, xmax)
    ymin, ymax = _expandir_intervalo(ymin, ymax)
    sx = lambda x: esquerda + (x - xmin) / (xmax - xmin) * (direita - esquerda)
    sy = lambda y: fundo - (y - ymin) / (ymax - ymin) * (fundo - topo)
    out: list[str] = []
    for tick in _ticks(ymin, ymax):
        py = sy(tick)
        out.append(f'<line x1="{esquerda:.2f}" y1="{py:.2f}" x2="{direita:.2f}"
          y2="{py:.2f}" stroke="{tema["grade"]}" stroke-width="1"/>')
    out.append(f'<line x1="{esquerda}" y1="{fundo}" x2="{direita}" y2="{fundo}"
      stroke="{tema["eixo"]}" stroke-width="1.5"/>')
    out.append(f'<line x1="{esquerda}" y1="{topo}" x2="{esquerda}" y2="{fundo}"
      stroke="{tema["eixo"]}" stroke-width="1.5"/>')
    elem = grafico.elementos[0]
    if isinstance(elem, SerieErro):
        for x, y, ei, es in zip(elem.x, elem.y, elem.erro_inferior, elem.erro_superior):
            px, py = sx(x), sy(y)
            pa, pb = sy(y - ei), sy(y + es)
            out.append(f'<line x1="{px:.2f}" y1="{pb:.2f}" x2="{px:.2f}" y2="{pa:.2f}"
              stroke="{tema["serie"]}"/>')
            out.append(f'<line x1="{px-5:.2f}" y1="{pb:.2f}" x2="{px+5:.2f}"
              y2="{pb:.2f}" stroke="{tema["serie"]}"/>')
            out.append(f'<line x1="{px-5:.2f}" y1="{pa:.2f}" x2="{px+5:.2f}"
```

```
              y2="{pa:.2f}" stroke="{tema["serie"]}"/>')
            out.append(f'<circle cx="{px:.2f}" cy="{py:.2f}" r="3"
              fill="{tema["serie"]}"/>')
    elif isinstance(elem, FaixaIncerteza):
        superiores = [(sx(x), sy(y)) for x, y in zip(elem.x, elem.superior)]
        inferiores = [(sx(x), sy(y)) for x, y in reversed(tuple(zip(elem.x,
          elem.inferior)))]
        pontos = " ".join(f"{x:.2f},{y:.2f}" for x, y in superiores + inferiores)
        out.append(f'<polygon points="{pontos}" fill="{tema["preenchimento"]}"
          fill-opacity="0.55" stroke="none"/>')
        centro = " ".join(f"{sx(x):.2f},{sy(y):.2f}" for x, y in zip(elem.x,
          elem.centro))
        out.append(f'<polyline points="{centro}" fill="none" stroke="{tema["serie"]}"
          stroke-width="2"/>')
    return out


def _cor_celula(valor: float, minimo: float, maximo: float) -> str:
    if minimo == maximo:
        t = 0.5
    else:
        t = (valor - minimo) / (maximo - minimo)
    t = max(0.0, min(1.0, t))
    # Escala sequencial clara e determinística, sem semântica categórica implícita.
    r = round(245 - 185 * t)
    g = round(248 - 115 * t)
    b = round(252 - 35 * t)
    return f"#{r:02x}{g:02x}{b:02x}"


def _render_mapa(grafico: Grafico, largura: int, altura: int) -> list[str]:
    tema = _TEMAS[grafico.tema]
    elem = grafico.elementos[0]
    if not isinstance(elem, MatrizCores):
        raise ErroGrafico("especificação de mapa inválida")
    nl, nc = len(elem.valores), len(elem.valores[0])
    esquerda, direita, topo, fundo = 110.0, largura - 50.0, 58.0, altura - 90.0
    cw = (direita - esquerda) / nc
    ch = (fundo - topo) / nl
    todos = [v for linha in elem.valores for v in linha]
    minimo, maximo = min(todos), max(todos)
    out: list[str] = []
    for i, linha in enumerate(elem.valores):
        for j, valor in enumerate(linha):
            x, y = esquerda + j * cw, topo + i * ch
            cor = _cor_celula(valor, minimo, maximo)
            out.append(f'<rect x="{x:.2f}" y="{y:.2f}" width="{cw:.2f}"
              height="{ch:.2f}" fill="{cor}" stroke="{tema["fundo"]}" stroke-width="1"/>')
            if nl <= 12 and nc <= 12:
                texto_cor = "#000000" if (valor - minimo) <= (maximo - minimo) * 0.62
                  else "#ffffff"
                out.append(f'<text x="{x+cw/2:.2f}" y="{y+ch/2+4:.2f}"
                  text-anchor="middle" font-size="10"
                  fill="{texto_cor}">{valor:.3g}</text>')
    for j, rotulo in enumerate(elem.rotulos_colunas):
        x = esquerda + (j + 0.5) * cw
        out.append(f'<text x="{x:.2f}" y="{fundo+20:.2f}" text-anchor="middle"
          font-size="10" fill="{tema["texto"]}">{escape(rotulo)}</text>')
    for i, rotulo in enumerate(elem.rotulos_linhas):
        y = topo + (i + 0.5) * ch + 4
        out.append(f'<text x="{esquerda-8:.2f}" y="{y:.2f}" text-anchor="end"
```

```
          font-size="10" fill="{tema["texto"]}">{escape(rotulo)}</text>')
    return out


def _render_barras(grafico: Grafico, largura: int, altura: int) -> list[str]:
    tema = _TEMAS[grafico.tema]
    serie = grafico.elementos[0]
    if not isinstance(serie, SerieCategorias):
        raise ErroGrafico("especificação de barras inválida")
    esquerda, direita, topo, fundo = 82.0, largura - 32.0, 54.0, altura - 88.0
    ymin, ymax = _expandir_intervalo(min(serie.valores), max(serie.valores),
      incluir_zero=True)
    sy = lambda y: fundo - (y - ymin) / (ymax - ymin) * (fundo - topo)
    base = sy(0.0)
    passo = (direita - esquerda) / len(serie.valores)
    bw = passo * 0.68
    out: list[str] = []
    for tick in _ticks(ymin, ymax):
        py = sy(tick)
        out.append(f'<line x1="{esquerda:.2f}" y1="{py:.2f}" x2="{direita:.2f}"
          y2="{py:.2f}" stroke="{tema["grade"]}" stroke-width="1"/>')
        out.append(f'<text x="{esquerda - 8:.2f}" y="{py + 4:.2f}" text-anchor="end"
          font-size="11" fill="{tema["texto"]}">{tick:.4g}</text>')
    for i, (categoria, valor) in enumerate(zip(serie.categorias, serie.valores)):
        cx = esquerda + passo * (i + 0.5)
        yv = sy(valor)
        y = min(base, yv); h = max(abs(base - yv), 0.8)
        out.append(f'<rect x="{cx - bw/2:.2f}" y="{y:.2f}" width="{bw:.2f}"
          height="{h:.2f}" fill="{tema["preenchimento"]}" stroke="{tema["serie"]}"/>')
        if len(serie.categorias) <= 20 or i % max(1, len(serie.categorias)//10) == 0:
            out.append(f'<text x="{cx:.2f}" y="{fundo + 20:.2f}" text-anchor="middle"
              font-size="10" fill="{tema["texto"]}">{escape(categoria)}</text>')
    out.append(f'<line x1="{esquerda}" y1="{base:.2f}" x2="{direita}" y2="{base:.2f}"
      stroke="{tema["eixo"]}" stroke-width="1.5"/>')
    out.append(f'<line x1="{esquerda}" y1="{topo}" x2="{esquerda}" y2="{fundo}"
      stroke="{tema["eixo"]}" stroke-width="1.5"/>')
    return out


def _render_caixa(grafico: Grafico, largura: int, altura: int) -> list[str]:
    tema = _TEMAS[grafico.tema]
    caixas = [e for e in grafico.elementos if isinstance(e, ResumoCaixa)]
    if not caixas:
        raise ErroGrafico("especificação de caixa sem grupos")
    esquerda, direita, topo, fundo = 82.0, largura - 32.0, 54.0, altura - 88.0
    todos = [v for c in caixas for v in (c.minimo, c.maximo)]
    ymin, ymax = _expandir_intervalo(min(todos), max(todos))
    sy = lambda y: fundo - (y - ymin) / (ymax - ymin) * (fundo - topo)
    passo = (direita - esquerda) / len(caixas)
    bw = min(passo * 0.42, 70.0)
    out: list[str] = []
    for tick in _ticks(ymin, ymax):
        py = sy(tick)
        out.append(f'<line x1="{esquerda:.2f}" y1="{py:.2f}" x2="{direita:.2f}"
          y2="{py:.2f}" stroke="{tema["grade"]}" stroke-width="1"/>')
        out.append(f'<text x="{esquerda - 8:.2f}" y="{py + 4:.2f}" text-anchor="end"
          font-size="11" fill="{tema["texto"]}">{tick:.4g}</text>')
    for i, c in enumerate(caixas):
        x = esquerda + passo * (i + 0.5)
        out.append(f'<line x1="{x:.2f}" y1="{sy(c.bigode_superior):.2f}" x2="{x:.2f}"
          y2="{sy(c.bigode_inferior):.2f}" stroke="{tema["serie"]}"/>')
```

```
        out.append(f'<line x1="{x-bw/3:.2f}" y1="{sy(c.bigode_superior):.2f}"
          x2="{x+bw/3:.2f}" y2="{sy(c.bigode_superior):.2f}" stroke="{tema["serie"]}"/>')
        out.append(f'<line x1="{x-bw/3:.2f}" y1="{sy(c.bigode_inferior):.2f}"
          x2="{x+bw/3:.2f}" y2="{sy(c.bigode_inferior):.2f}" stroke="{tema["serie"]}"/>')
        yq3, yq1 = sy(c.q3), sy(c.q1)
        out.append(f'<rect x="{x-bw/2:.2f}" y="{min(yq3,yq1):.2f}" width="{bw:.2f}"
          height="{max(abs(yq1-yq3),0.8):.2f}" fill="{tema["preenchimento"]}"
          stroke="{tema["serie"]}"/>')
        out.append(f'<line x1="{x-bw/2:.2f}" y1="{sy(c.mediana):.2f}" x2="{x+bw/2:.2f}"
          y2="{sy(c.mediana):.2f}" stroke="{tema["serie2"]}" stroke-width="2"/>')
        for extremo in c.extremos:
            out.append(f'<circle cx="{x:.2f}" cy="{sy(extremo):.2f}" r="3" fill="none"
              stroke="{tema["serie2"]}"/>')
        out.append(f'<text x="{x:.2f}" y="{fundo+20:.2f}" text-anchor="middle"
          font-size="10" fill="{tema["texto"]}">{escape(c.nome)}</text>')
    out.append(f'<line x1="{esquerda}" y1="{fundo}" x2="{direita}" y2="{fundo}"
      stroke="{tema["eixo"]}" stroke-width="1.5"/>')
    out.append(f'<line x1="{esquerda}" y1="{topo}" x2="{esquerda}" y2="{fundo}"
      stroke="{tema["eixo"]}" stroke-width="1.5"/>')
    return out


def _grafico_eh_3d(grafico: Grafico) -> bool:
    return grafico.tipo in {"dispersao3d", "trajetoria3d", "superficie3d"}


def _contem_3d(grafico: Grafico | PainelGraficos) -> bool:
    if isinstance(grafico, Grafico):
        return _grafico_eh_3d(grafico)
    return any(_grafico_eh_3d(filho) for filho in grafico.graficos)


def para_svg(grafico: Grafico | PainelGraficos, *, largura: int = 800, altura: int =
  500) -> str:
    """Renderiza gráficos 2D como SVG autossuficiente em memória.

    Especificações 3D preservam coordenadas sem escolher projeção no núcleo. Para
    renderização 3D use :func:`salvar`, que carrega Matplotlib sob demanda.
    """
    if _contem_3d(grafico):
        raise ErroGrafico("para_svg não projeta gráficos 3D; use salvar com .svg, .png
          ou .pdf e Matplotlib")
    if isinstance(grafico, PainelGraficos):
        if largura < 320 or altura < 240:
            raise ErroGrafico("SVG requer largura >= 320 e altura >= 240")
        linhas = grafico.linhas
        cw, ch = largura / grafico.colunas, altura / linhas
        out = [
            '<?xml version="1.0" encoding="UTF-8"?>',
            f'<svg xmlns="http://www.w3.org/2000/svg" width="{largura}"
              height="{altura}" viewBox="0 0 {largura} {altura}" role="img">',
            f'<title>{escape(grafico.titulo or "Painel de gráficos")}</title>',
        ]
        for i, filho in enumerate(grafico.graficos):
            col, row = i % grafico.colunas, i // grafico.colunas
            filho_svg = para_svg(filho, largura=800, altura=500)
            ini = filho_svg.find(">", filho_svg.find("<svg")) + 1
            fim = filho_svg.rfind("</svg>")
            corpo = filho_svg[ini:fim]
            out.append(f'<svg x="{col*cw:.2f}" y="{row*ch:.2f}" width="{cw:.2f}"
              height="{ch:.2f}" viewBox="0 0 800 500">{corpo}</svg>')
```

```
        out.append('</svg>')
        return "\n".join(out) + "\n"
    if not isinstance(grafico, Grafico):
        raise ErroGrafico("para_svg requer Grafico ou PainelGraficos")
    if largura < 320 or altura < 240:
        raise ErroGrafico("SVG requer largura >= 320 e altura >= 240")
    if grafico.tema not in _TEMAS:
        raise ErroGrafico("tema gráfico inválido")
    tema = _TEMAS[grafico.tema]
    out = _svg_inicio(grafico, largura, altura)
    if grafico.titulo:
        out.append(f'<text x="{largura/2:.2f}" y="30" text-anchor="middle"
          font-size="18" font-weight="600"
          fill="{tema["texto"]}">{escape(grafico.titulo)}</text>')
    if grafico.tipo in {"linha", "dispersao", "funcao"}:
        out.extend(_render_cartesiano(grafico, largura, altura))
    elif grafico.tipo in {"barras", "histograma"}:
        out.extend(_render_barras(grafico, largura, altura))
    elif grafico.tipo == "caixa":
        out.extend(_render_caixa(grafico, largura, altura))
    elif grafico.tipo in {"mapa_calor", "mapa_correlacao"}:
        out.extend(_render_mapa(grafico, largura, altura))
    elif grafico.tipo in {"barras_erro", "faixa_incerteza"}:
        out.extend(_render_incerteza(grafico, largura, altura))
    else:
        raise ErroGrafico(f"tipo de gráfico sem renderizador SVG: {grafico.tipo}")

    eixo_x = _texto_eixo(grafico.rotulo_x, grafico.unidade_x)
    eixo_y = _texto_eixo(grafico.rotulo_y, grafico.unidade_y)
    if eixo_x:
        out.append(f'<text x="{largura/2:.2f}" y="{altura-18}" text-anchor="middle"
          font-size="13" fill="{tema["texto"]}">{escape(eixo_x)}</text>')
    if eixo_y:
        out.append(f'<text x="18" y="{altura/2:.2f}" text-anchor="middle" font-size="13"
           fill="{tema["texto"]}" transform="rotate(-90 18
          {altura/2:.2f})">{escape(eixo_y)}</text>')
    if grafico.avisos:
        out.append(f'<desc>{escape("; ".join(grafico.avisos))}</desc>')
    out.append('</svg>')
    return "\n".join(out) + "\n"


def _desenhar_matplotlib(ax: Any, grafico: Grafico) -> None:
    elem = grafico.elementos[0] if grafico.elementos else None
    if grafico.tipo == "dispersao3d" and isinstance(elem, Serie3D):
        ax.scatter(elem.x, elem.y, elem.z, label=elem.nome)
    elif grafico.tipo == "trajetoria3d" and isinstance(elem, Serie3D):
        ax.plot(elem.x, elem.y, elem.z, label=elem.nome)
    elif grafico.tipo == "superficie3d" and isinstance(elem, Superficie3D):
        # A renderização usa a própria malha fornecida. Não há interpolação,
        # triangulação ou reamostragem de pontos ausentes.
        for xr, yr, zr in zip(elem.x, elem.y, elem.z):
            ax.plot(xr, yr, zr)
        linhas, colunas = elem.forma
        for j in range(colunas):
            ax.plot(
                [elem.x[i][j] for i in range(linhas)],
                [elem.y[i][j] for i in range(linhas)],
                [elem.z[i][j] for i in range(linhas)],
            )
    elif grafico.tipo in {"linha", "funcao", "dispersao"} and isinstance(elem, Serie2D):
```

```
        xs, ys = [], []
        for x, y in zip(elem.x, elem.y):
            if y is None:
                if xs and grafico.tipo != "dispersao":
                    ax.plot(xs, ys, label=elem.nome)
                xs, ys = [], []
                continue
            xs.append(x); ys.append(y)
        if xs:
            if grafico.tipo == "dispersao":
                ax.scatter(xs, ys, label=elem.nome)
            else:
                ax.plot(xs, ys, label=elem.nome)
    elif grafico.tipo in {"barras", "histograma"} and isinstance(elem, SerieCategorias):
        ax.bar(elem.categorias, elem.valores, label=elem.nome)
    elif grafico.tipo == "caixa":
        caixas = [e for e in grafico.elementos if isinstance(e, ResumoCaixa)]
        dados = [{"label": c.nome, "whislo": c.bigode_inferior, "q1": c.q1,
                  "med": c.mediana, "q3": c.q3, "whishi": c.bigode_superior,
                  "fliers": list(c.extremos)} for c in caixas]
        ax.bxp(dados, showfliers=True)
    elif grafico.tipo in {"mapa_calor", "mapa_correlacao"} and isinstance(elem,
      MatrizCores):
        imagem = ax.imshow(elem.valores, aspect="auto")
        ax.figure.colorbar(imagem, ax=ax)
        ax.set_xticks(range(len(elem.rotulos_colunas)), elem.rotulos_colunas)
        ax.set_yticks(range(len(elem.rotulos_linhas)), elem.rotulos_linhas)
    elif grafico.tipo == "barras_erro" and isinstance(elem, SerieErro):
        ax.errorbar(elem.x, elem.y, yerr=[elem.erro_inferior, elem.erro_superior],
          fmt="o", label=elem.nome)
    elif grafico.tipo == "faixa_incerteza" and isinstance(elem, FaixaIncerteza):
        ax.fill_between(elem.x, elem.inferior, elem.superior, alpha=0.25)
        ax.plot(elem.x, elem.centro, label=elem.nome)
    else:
        raise ErroGrafico(f"tipo de gráfico sem renderizador Matplotlib:
          {grafico.tipo}")
    if grafico.titulo:
        ax.set_title(grafico.titulo)
    ax.set_xlabel(_texto_eixo(grafico.rotulo_x, grafico.unidade_x))
    ax.set_ylabel(_texto_eixo(grafico.rotulo_y, grafico.unidade_y))
    if _grafico_eh_3d(grafico):
        ax.set_zlabel(_texto_eixo(grafico.rotulo_z, grafico.unidade_z))
    if (
        grafico.legenda
        and grafico.tipo in {"linha", "funcao", "dispersao", "barras", "histograma",
          "barras_erro", "faixa_incerteza", "dispersao3d", "trajetoria3d"}
        and getattr(elem, "nome", None)
    ):
        ax.legend()


def _salvar_matplotlib(grafico: Grafico | PainelGraficos, destino: Path) -> Path:
    try:
        import matplotlib.pyplot as plt
    except Exception as exc:
        raise ErroGrafico(
            "Para salvar gráficos 3D, PNG ou PDF, instale o backend opcional Matplotlib
              no ambiente da Coral."
        ) from exc
    if isinstance(grafico, PainelGraficos):
        fig = plt.figure(figsize=(6 * grafico.colunas, 4 * grafico.linhas))
```

```
        for i, filho in enumerate(grafico.graficos):
            projecao = "3d" if _grafico_eh_3d(filho) else None
            ax = fig.add_subplot(grafico.linhas, grafico.colunas, i + 1,
              projection=projecao)
            _desenhar_matplotlib(ax, filho)
        if grafico.titulo:
            fig.suptitle(grafico.titulo)
    elif isinstance(grafico, Grafico):
        fig = plt.figure(figsize=(8, 5))
        ax = fig.add_subplot(111, projection="3d" if _grafico_eh_3d(grafico) else None)
        _desenhar_matplotlib(ax, grafico)
    else:
        raise ErroGrafico("salvar requer Grafico ou PainelGraficos")
    fig.tight_layout()
    destino.parent.mkdir(parents=True, exist_ok=True)
    temporario = destino.with_name(destino.stem + ".tmp-coral" + destino.suffix)
    fig.savefig(temporario, format=destino.suffix.lower().lstrip("."))
    plt.close(fig)
    temporario.replace(destino)
    return destino


from ._graficos_interativos import amostrar_grafico, explorar, para_html, salvar_html

def salvar(grafico: Grafico | PainelGraficos, caminho: str | Path) -> Path:
    """Salva 2D em SVG pelo núcleo; 3D e PNG/PDF usam Matplotlib sob demanda."""
    destino = Path(caminho)
    sufixo = destino.suffix.lower()
    if sufixo == ".svg":
        if _contem_3d(grafico):
            return _salvar_matplotlib(grafico, destino)
        destino.parent.mkdir(parents=True, exist_ok=True)
        texto = para_svg(grafico)
        temporario = destino.with_name(destino.name + ".tmp-coral")
        temporario.write_text(texto, encoding="utf-8", newline="\n")
        temporario.replace(destino)
        return destino
    if sufixo in {".png", ".pdf"}:
        return _salvar_matplotlib(grafico, destino)
    raise ErroGrafico("Formato gráfico desconhecido. Use .svg, .png ou .pdf")


__all__ = [
    "ErroGrafico",
    "Serie2D",
    "Serie3D",
    "Superficie3D",
    "SerieCategorias",
    "ResumoCaixa",
    "SeriesRegistros",
    "MatrizCores",
    "SerieErro",
    "FaixaIncerteza",
    "PainelGraficos",
    "Grafico",
    "linha",
    "dispersao",
    "dispersao3d",
    "trajetoria3d",
    "superficie3d",
    "barras",
```

```
    "grafico_histograma",
    "grafico_caixa",
    "configurar",
    "grafico_funcao",
    "series_de_registros",
    "mapa_calor",
    "mapa_correlacao",
    "barras_erro",
    "faixa_incerteza",
    "grafico_experimento",
    "painel",
    "para_svg",
    "para_html",
    "salvar_html",
    "explorar",
    "amostrar_grafico",
    "salvar",
]
```

Biblioteca `coral.hardware`

`coral.hardware.BackendSerial`

Contrato mínimo de backend serial usado por `ConexaoSerial`.

Assinatura

Assinatura para consulta

```
BackendSerial(*args, **kwargs)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
args	sem anotação	VAR_POSITIONAL	não	sem padrão
kwargs	sem anotação	VAR_KEYWORD	não	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
abrir(self, porta: 'str', velocidade: 'int', timeout: 'float | None') ->
  'TransporteSerial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
listar_portas(self) -> 'tuple[PortaSerial, ...]'
```

Método público sem descrição própria no runtime.

`coral.hardware.BackendSerialReal`

Adaptador do pyserial, carregado apenas quando realmente necessário.

Assinatura

Assinatura para consulta

```
BackendSerialReal()
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
abrir(self, porta: 'str', velocidade: 'int', timeout: 'float | None') ->
  'TransporteSerial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
listar_portas(self) -> 'tuple[PortaSerial, ...]'
```

Método público sem descrição própria no runtime.

`coral.hardware.BackendSerialSimulado`

Backend serial em memória, sem espera real e sem dependências externas.

Portas precisam ser registradas explicitamente. Bytes injetados com :meth:`injetar_recebimento` são consumidos por `ler/ler_linha; bytes escritos ficam disponíveis em :meth:`dados_enviados`.

Assinatura

Assinatura para consulta

```
BackendSerialSimulado() -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

None

Métodos públicos

Referência para leitura

```
abrir(self, porta: 'str', velocidade: 'int', timeout: 'float | None') ->
  'TransporteSerial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
dados_enviados(self, dispositivo: 'str', *, limpar: 'bool' = False) -> 'bytes'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
desconectar(self, dispositivo: 'str') -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
injetar_recebimento(self, dispositivo: 'str', dados: 'bytes | bytearray | str', *,
  codificacao: 'str' = 'utf-8') -> 'int'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
limpar(self, dispositivo: 'str | None' = None) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
listar_portas(self) -> 'tuple[PortaSerial, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
reconectar(self, dispositivo: 'str') -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
registrar_porta(self, dispositivo: 'str' = 'SIM0', *, descricao: 'str' = 'Porta serial
  simulada', fabricante: 'str | None' = 'Coral', vid: 'int | None' = None, pid: 'int |
  None' = None) -> 'PortaSerial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_porta(self, dispositivo: 'str') -> 'None'
```

Método público sem descrição própria no runtime.

`coral.hardware.Bateria`

Bateria(percentual: 'float | None', conectada: 'bool | None', segundos_restantes: 'int | None')

Assinatura

Assinatura para consulta

```
Bateria(percentual: 'float | None', conectada: 'bool | None', segundos_restantes: 'int |
   None') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
percentual	float | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
conectada	bool | None	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
segundos_restantes	int | None	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `percentual`: `float | None`.

Atributo `conectada`: `bool | None`.

Atributo `segundos_restantes`: `int | None`.

`coral.hardware.CPU`

CPU(nome: 'str', nucleos_logicos: 'int | None', nucleos_fisicos: 'int | None', frequencia_mhz: 'float | None')

Assinatura

Assinatura para consulta

```
CPU(nome: 'str', nucleos_logicos: 'int | None', nucleos_fisicos: 'int | None',
  frequencia_mhz: 'float | None') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
nucleos_logicos	int | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
nucleos_fisicos	int | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
frequencia_mhz	float | None	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `nome`: `str`.

Atributo `nucleos_logicos`: `int | None`.

Atributo `nucleos_fisicos`: `int | None`.

Atributo `frequencia_mhz`: `float | None`.

`coral.hardware.ConexaoSerial`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
ConexaoSerial(porta: 'str', velocidade: 'int' = 115200, timeout: 'float | None' = 1.0,
  *, codificacao: 'str' = 'utf-8', backend: 'BackendSerial | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
porta	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
velocidade	int	POSITIONAL_OR_KEYWORD	não	115200
timeout	float | None	POSITIONAL_OR_KEYWORD	não	1.0
codificacao	str	KEYWORD_ONLY	não	'utf-8'
backend	BackendSerial | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
enviar(self, texto: 'Any', *, terminar_linha: 'bool' = False) -> 'int'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
enviar_bytes(self, dados: 'bytes | bytearray') -> 'int'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
fechar(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
ler(self, quantidade: 'int' = 1) -> 'bytes'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
ler_linha(self) -> 'str'
```

Método público sem descrição própria no runtime.

`coral.hardware.ConexaoSerialFechada`

Uma operação foi solicitada depois do fechamento da conexão.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.hardware.DependenciaHardwareAusente`

Uma capacidade opcional não pode operar por ausência de dependência.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.hardware.DependenciaSerialAusente`

pyserial não está disponível para o backend real.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.hardware.DesconexaoSerial`

A porta deixou de estar conectada durante a sessão.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.hardware.Disco`

Disco(dispositivo: 'str', ponto_montagem: 'str', sistema_arquivos: 'str', total_bytes: 'int | None' = None, livre_bytes: 'int | None' = None)

Assinatura

Assinatura para consulta

```
Disco(dispositivo: 'str', ponto_montagem: 'str', sistema_arquivos: 'str', total_bytes:
  'int | None' = None, livre_bytes: 'int | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
dispositivo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
ponto_montagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
sistema_arquivos	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
total_bytes	int | None	POSITIONAL_OR_KEYWORD	não	None
livre_bytes	int | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `dispositivo`: `str`.

Atributo `ponto_montagem`: `str`.

Atributo `sistema_arquivos`: `str`.

Atributo `total_bytes`: `int | None`.

Atributo `livre_bytes`: `int | None`.

`coral.hardware.ErroConfiguracaoSerial`

Parâmetro serial inválido antes da abertura da porta.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.hardware.ErroSerial`

Erro público base da camada serial Coral.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.hardware.Memoria`

Memoria(total_bytes: 'int | None', disponivel_bytes: 'int | None', percentual_usado: 'float | None')

Assinatura

Assinatura para consulta

```
Memoria(total_bytes: 'int | None', disponivel_bytes: 'int | None', percentual_usado:
  'float | None') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
total_bytes	int | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
disponivel_bytes	int | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
percentual_usado	float | None	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `total_bytes`: `int | None`.

Atributo `disponivel_bytes`: `int | None`.

Atributo `percentual_usado`: `float | None`.

`coral.hardware.PortaSerial`

PortaSerial(dispositivo: 'str', descricao: 'str' = '', fabricante: 'str | None' = None, vid: 'int | None' = None, pid: 'int | None' = None)

Assinatura

Assinatura para consulta

```
PortaSerial(dispositivo: 'str', descricao: 'str' = '', fabricante: 'str | None' = None,
  vid: 'int | None' = None, pid: 'int | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
dispositivo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
descricao	str	POSITIONAL_OR_KEYWORD	não	''
fabricante	str | None	POSITIONAL_OR_KEYWORD	não	None
vid	int | None	POSITIONAL_OR_KEYWORD	não	None
pid	int | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `dispositivo`: `str`.

Atributo `descricao`: `str`.

Atributo `fabricante`: `str | None`.

Atributo `vid`: `int | None`.

Atributo `pid`: `int | None`.

`coral.hardware.auditar_hardware`

Inventário observável e estado dos gates de coesão de Hardware 1.4.9.

Assinatura

Assinatura para consulta

```
auditar_hardware() -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

dict[str, Any]

`coral.hardware.bateria`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
bateria() -> 'Bateria | None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

Bateria | None

`coral.hardware.conectar_serial`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
conectar_serial(porta: 'str', velocidade: 'int' = 115200, timeout: 'float | None' = 1.0,
    *, codificacao: 'str' = 'utf-8', backend: 'BackendSerial | None' = None) ->
  'ConexaoSerial'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
porta	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
velocidade	int	POSITIONAL_OR_KEYWORD	não	115200
timeout	float | None	POSITIONAL_OR_KEYWORD	não	1.0
codificacao	str	KEYWORD_ONLY	não	'utf-8'
backend	BackendSerial | None	KEYWORD_ONLY	não	None

Retorno declarado

ConexaoSerial

`coral.hardware.cpu`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
cpu() -> 'CPU'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

CPU

`coral.hardware.diagnosticar_hardware`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
diagnosticar_hardware() -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

dict[str, Any]

`coral.hardware.discos`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
discos() -> 'tuple[Disco, ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

tuple[Disco, ...]

`coral.hardware.memoria`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
memoria() -> 'Memoria'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

Memoria

`coral.hardware.portas_seriais`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
portas_seriais(*, backend: 'BackendSerial | None' = None) -> 'tuple[PortaSerial, ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
backend	BackendSerial | None	KEYWORD_ONLY	não	None

Retorno declarado

tuple[PortaSerial, ...]

`coral.hardware.psutil_disponivel`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
psutil_disponivel() -> 'bool'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

bool

`coral.hardware.serial_disponivel`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
serial_disponivel() -> 'bool'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

bool

`coral.hardware.usar_backend_serial`

Usa temporariamente um backend serial como padrão do contexto atual.

Isso permite executar inclusive a sintaxe natural Coral contra o backend simulado, sem alterar chamadas existentes nem depender de estado global compartilhado entre threads/tarefas.

Assinatura

Assinatura para consulta

```
usar_backend_serial(backend: 'BackendSerial')
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
backend	BackendSerial	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Consultar a implementação canônica de coral.hardware

Implementação canônica de `coral.hardware`

Código Python

```
"""Contratos transversais pequenos do runtime Coral.

A camada comum não substitui APIs de domínio. Ela fornece a gramática
arquitetural compartilhada do ciclo 1.5 para erro operacional, resultado e
diagnóstico, preservando os nomes públicos existentes em cada biblioteca.
"""
from __future__ import annotations

from dataclasses import asdict, dataclass, field
from typing import Any, Iterable, Mapping

CONTRATO = "coral.comum/1"
CONTRATO_DIAGNOSTICO = "coral.diagnostico.operacional/1"


class ErroCoral(Exception):
    """Raiz pública dos erros controlados pela linguagem e bibliotecas Coral."""


class ErroOperacaoCoral(RuntimeError, ErroCoral):
    """Falha operacional de um recurso Coral ou de seu backend."""


class ErroDependenciaCoral(ErroOperacaoCoral):

    """Uma capacidade opcional não pode operar por ausência de dependência."""


@dataclass(frozen=True)
class FalhaOperacao:
```

```
    codigo: str
    mensagem: str
    dominio: str
    tipo: str = "operacional"
    causa: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def de_excecao(cls, exc: BaseException, *, codigo: str, dominio: str, tipo: str =
      "operacional", detalhe: Mapping[str, Any] | None = None) -> "FalhaOperacao":
        causa = exc.__cause__ if exc.__cause__ is not None else exc
        return cls(codigo, str(exc), dominio, tipo, type(causa).__name__, dict(detalhe
          or {}))

    def para_dict(self) -> dict[str, Any]:
        return asdict(self)


@dataclass(frozen=True)
class ResultadoOperacao:
    """Representação transversal sem substituir resultados ricos de domínio."""
    ok: bool
    valor: Any = None
    falha: FalhaOperacao | None = None
    metadados: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def sucesso(cls, valor: Any = None, **metadados: Any) -> "ResultadoOperacao":
        return cls(True, valor=valor, metadados=metadados)

    @classmethod
    def erro(cls, falha: FalhaOperacao, **metadados: Any) -> "ResultadoOperacao":
        return cls(False, falha=falha, metadados=metadados)

    def para_dict(self) -> dict[str, Any]:
        return {
            "contrato": CONTRATO,
            "ok": self.ok,
            "valor": self.valor,
            "falha": self.falha.para_dict() if self.falha is not None else None,
            "metadados": dict(self.metadados),
        }


@dataclass(frozen=True)
class DiagnosticoOperacional:
    id: str
    dominio: str
    ok: bool
    mensagem: str
    capacidade: str | None = None
    sugestao: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @property
    def nivel(self) -> str:
        return "info" if self.ok else "erro"

    def para_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
```

```
            "dominio": self.dominio,
            "ok": self.ok,
            "nivel": self.nivel,
            "mensagem": self.mensagem,
            "capacidade": self.capacidade,
            "sugestao": self.sugestao,
            "detalhe": dict(self.detalhe),
        }


def diagnostico(id: str, dominio: str, ok: bool, mensagem: Any, *, capacidade: str |
  None = None, sugestao: str | None = None, detalhe: Mapping[str, Any] | None = None) ->
  DiagnosticoOperacional:
    return DiagnosticoOperacional(str(id), str(dominio), bool(ok), str(mensagem),
      capacidade, sugestao, dict(detalhe or {}))


def agregar_diagnosticos(itens: Iterable[DiagnosticoOperacional | Mapping[str, Any]], *,
   origem: str = "coral") -> dict[str, Any]:
    normalizados: list[dict[str, Any]] = []
    for item in itens:
        if isinstance(item, DiagnosticoOperacional):
            normalizados.append(item.para_dict())
            continue
        d = dict(item)
        if "id" not in d:
            d["id"] = str(d.get("nome", "diagnostico"))
        if "dominio" not in d:
            d["dominio"] = origem
        d["ok"] = bool(d.get("ok"))
        d.setdefault("nivel", "info" if d["ok"] else "erro")
        d.setdefault("mensagem", str(d.get("detalhe", "")))
        d.setdefault("capacidade", None)
        d.setdefault("sugestao", None)
        detalhe = d.get("detalhe")
        if not isinstance(detalhe, Mapping):
            d["detalhe"] = {"valor": detalhe}
        normalizados.append(d)
    return {
        "contrato": CONTRATO_DIAGNOSTICO,
        "origem": origem,
        "ok": all(x["ok"] for x in normalizados),
        "quantidade": len(normalizados),
        "itens": normalizados,
    }


__all__ = [
    "CONTRATO", "CONTRATO_DIAGNOSTICO", "ErroCoral", "ErroOperacaoCoral",
    "ErroDependenciaCoral", "FalhaOperacao", "ResultadoOperacao",
    "DiagnosticoOperacional", "diagnostico", "agregar_diagnosticos",
]

"""Informações e comunicação de hardware para Coral 1.3.

`psutil` e `pyserial` são dependências opcionais. O pacote pode ser importado
sem elas e expõe diagnósticos claros para cada capacidade.
"""
from __future__ import annotations

from dataclasses import asdict, dataclass
```

```
import os
import platform
from typing import Any

from ..comum import ErroDependenciaCoral, agregar_diagnosticos, diagnostico


class DependenciaHardwareAusente(ErroDependenciaCoral):
    pass


@dataclass(frozen=True)
class CPU:
    nome: str
    nucleos_logicos: int | None
    nucleos_fisicos: int | None
    frequencia_mhz: float | None


@dataclass(frozen=True)
class Memoria:
    total_bytes: int | None
    disponivel_bytes: int | None
    percentual_usado: float | None


@dataclass(frozen=True)
class Disco:
    dispositivo: str
    ponto_montagem: str
    sistema_arquivos: str
    total_bytes: int | None = None
    livre_bytes: int | None = None


@dataclass(frozen=True)
class Bateria:
    percentual: float | None
    conectada: bool | None
    segundos_restantes: int | None


def psutil_disponivel() -> bool:
    try:
        import psutil  # noqa: F401
        return True
    except Exception:
        return False


def serial_disponivel() -> bool:
    try:
        import serial  # noqa: F401
        return True
    except Exception:
        return False


def cpu() -> CPU:
    nome = platform.processor() or platform.machine() or "desconhecido"
    fallback = CPU(nome, os.cpu_count(), None, None)
```

```
    if not psutil_disponivel():
        return fallback
    try:
        import psutil
        freq = psutil.cpu_freq()
        atual = getattr(freq, "current", None) if freq else None
        return CPU(nome, psutil.cpu_count(True), psutil.cpu_count(False), float(atual)
          if atual is not None else None)
    except Exception:
        return fallback


def memoria() -> Memoria:
    fallback = Memoria(None, None, None)
    if not psutil_disponivel():
        return fallback
    try:
        import psutil
        m = psutil.virtual_memory()
        return Memoria(int(m.total), int(m.available), float(m.percent))
    except Exception:
        return fallback


def discos() -> tuple[Disco, ...]:
    if not psutil_disponivel():
        return ()
    try:
        import psutil
        particoes = psutil.disk_partitions(all=False)
    except Exception:
        return ()
    saida=[]
    for p in particoes:
        total=livre=None
        try:
            u=psutil.disk_usage(p.mountpoint); total=int(u.total); livre=int(u.free)
        except Exception:
            pass
        saida.append(Disco(str(p.device),str(p.mountpoint),str(p.fstype),total,livre))
    return tuple(saida)


def bateria() -> Bateria | None:
    if not psutil_disponivel():
        return None
    try:
        import psutil
        b=psutil.sensors_battery()
    except Exception:
        return None
    if b is None:
        return None
    try:
        ignorados = {getattr(psutil, "POWER_TIME_UNLIMITED", -2), getattr(psutil,
          "POWER_TIME_UNKNOWN", -1), None}
        secs = None if getattr(b, "secsleft", None) in ignorados else int(b.secsleft)
        return Bateria(float(b.percent), bool(b.power_plugged), secs)
    except (TypeError, ValueError, OverflowError):
        return None
```

```
def diagnosticar_hardware() -> dict[str, Any]:
    c=cpu(); m=memoria(); b=bateria()
    ps_ok = psutil_disponivel(); serial_ok = serial_disponivel()
    dados = {
        "psutil": {"disponivel": ps_ok, "instalacao": "python -m pip install psutil"},
        "serial": {"disponivel": serial_ok, "instalacao": "python -m pip install
          pyserial",
                   "backend_simulado": {"disponivel": True, "classe":
                     "BackendSerialSimulado", "dependencia_externa": False}},
        "cpu": asdict(c), "memoria": asdict(m), "bateria": asdict(b) if b else None,
        "discos": [asdict(d) for d in discos()],
    }
    itens = [
        diagnostico("hardware.psutil", "hardware", ps_ok, "psutil disponível." if ps_ok
          else "psutil ausente; informações avançadas usarão modo degradado.",
          capacidade="psutil", sugestao=None if ps_ok else "Instale a capacidade Hardware
          pelo Coral Setup."),
        diagnostico("hardware.serial.real", "hardware", serial_ok, "pyserial disponível
          para portas reais." if serial_ok else "pyserial ausente; o backend serial
          simulado continua disponível.", capacidade="pyserial", sugestao=None if
          serial_ok else "Instale a capacidade Serial pelo Coral Setup."),
        diagnostico("hardware.serial.simulado", "hardware", True, "Backend serial
          simulado disponível sem dependência externa.",
          capacidade="hardware.serial.simulado"),
    ]
    dados["diagnostico"] = agregar_diagnosticos(itens, origem="hardware")
    return dados


def auditar_hardware() -> dict[str, Any]:
    """Inventário observável e estado dos gates de coesão de Hardware 1.4.9."""
    superficie = [
        "DependenciaHardwareAusente", "CPU", "Memoria", "Disco", "Bateria",
        "psutil_disponivel", "serial_disponivel", "cpu", "memoria", "discos", "bateria",
        "diagnosticar_hardware", "auditar_hardware", "ErroSerial",
          "DependenciaSerialAusente", "ErroConfiguracaoSerial",
        "ConexaoSerialFechada", "DesconexaoSerial", "BackendSerialReal",
          "BackendSerialSimulado",
        "usar_backend_serial", "ConexaoSerial", "PortaSerial", "portas_seriais",
          "conectar_serial",
    ]
    return {
        "contrato": "coral.hardware.auditoria/1",
        "superficie_publica": superficie,
        "classificacao": {
            "dominio": ["CPU", "Memoria", "Disco", "Bateria", "PortaSerial",
              "ConexaoSerial"],
            "adaptadores": ["BackendSerialReal", "BackendSerialSimulado"],
            "compatibilidade": ["DependenciaHardwareAusente"],
            "internos": ["_serial"],
        },
        "dependencias_opcionais": ["psutil", "pyserial"],
        "backends": ["real", "simulado"],
        "estado_observavel": ["ConexaoSerial.aberta", "PortaSerial"],
        "fronteira_erros": ["ErroSerial", "DependenciaSerialAusente",
          "ErroConfiguracaoSerial", "ConexaoSerialFechada", "DesconexaoSerial"],
        "gates": {
            "I": {"estado": "aprovado", "evidencia": "inventário público e classificação
               versionados"},
            "T": {"estado": "aprovado", "evidencia": "sucesso/falha do backend simulado,
```

```
              dependências e regressão automatizados"},
            "C": {"estado": "aprovado", "evidencia": "erros públicos Coral delimitam a
              fronteira serial"},
            "X": {"estado": "aprovado", "evidencia": "diagnóstico, capacidades e matriz
              de pontes usam contratos comuns"},
            "D": {"estado": "aprovado", "evidencia": "referência 1.4.9 e relatório de
              gates"},
            "E": {"estado": "aprovado", "evidencia": "LSP, Ensinar, Manager e
              componentes consomem coral.pontes/1"},
            "U": {"estado": "nao_aplicavel", "justificativa": "A API de Hardware
              auditada não possui interface gráfica própria; validações visuais permanecem
               na suíte de campo."},
            "R": {"estado": "aprovado", "evidencia": "aliases e exceções públicas
              históricas preservados"},
        },
    }


from .serial import (
    ErroSerial, DependenciaSerialAusente, ErroConfiguracaoSerial,
    ConexaoSerialFechada, DesconexaoSerial, PortaSerial, BackendSerial,
    BackendSerialReal, BackendSerialSimulado, usar_backend_serial, ConexaoSerial,
      portas_seriais, conectar_serial,
)

__all__ = [
    "DependenciaHardwareAusente", "CPU", "Memoria", "Disco", "Bateria",
    "psutil_disponivel", "serial_disponivel", "cpu", "memoria", "discos", "bateria",
    "diagnosticar_hardware", "auditar_hardware", "ErroSerial",
      "DependenciaSerialAusente", "ErroConfiguracaoSerial",
    "ConexaoSerialFechada", "DesconexaoSerial", "BackendSerial", "BackendSerialReal",
    "BackendSerialSimulado", "usar_backend_serial", "ConexaoSerial", "PortaSerial",
      "portas_seriais", "conectar_serial",
]

"""Comunicação serial opcional para Coral 1.3.

O backend real usa pyserial quando disponível. ``BackendSerialSimulado`` oferece
um transporte determinístico para testes, aulas e CI sem depender de dispositivo
físico. Ambos obedecem ao mesmo contrato consumido por :class:`ConexaoSerial`.
"""
from __future__ import annotations

from dataclasses import dataclass
from contextlib import contextmanager
from contextvars import ContextVar
import codecs
from numbers import Real
from typing import Any, Protocol, runtime_checkable

from ..comum import ErroDependenciaCoral, ErroOperacaoCoral
from ..tempo_eventos import validar_duracao


class ErroSerial(ErroOperacaoCoral):
    """Erro público base da camada serial Coral."""


class DependenciaSerialAusente(ErroSerial, ErroDependenciaCoral):
    """pyserial não está disponível para o backend real."""
```

```
class ErroConfiguracaoSerial(ErroSerial):
    """Parâmetro serial inválido antes da abertura da porta."""


class ConexaoSerialFechada(ErroSerial):
    """Uma operação foi solicitada depois do fechamento da conexão."""


class DesconexaoSerial(ErroSerial):
    """A porta deixou de estar conectada durante a sessão."""


def _serial():
    """Carrega pyserial sob demanda.

    A função permanece separada também por compatibilidade com integrações que
    já a substituíam em testes da linha Coral 1.3.
    """
    try:
        import serial
        return serial
    except Exception as exc:
        raise DependenciaSerialAusente(
            "A comunicação serial da Coral requer pyserial. "
            "Instale com: python -m pip install pyserial"
        ) from exc


@dataclass(frozen=True)
class PortaSerial:
    dispositivo: str
    descricao: str = ""
    fabricante: str | None = None
    vid: int | None = None
    pid: int | None = None


def _validar_porta(porta: Any) -> str:
    if not isinstance(porta, str) or not porta.strip():
        raise ErroConfiguracaoSerial("A porta serial deve ser um texto não vazio.")
    return porta.strip()


def _validar_velocidade(velocidade: Any) -> int:
    if isinstance(velocidade, bool) or not isinstance(velocidade, int) or velocidade <=
      0:
        raise ErroConfiguracaoSerial("A velocidade serial deve ser um inteiro positivo
          em bauds.")
    return velocidade


def _validar_timeout(timeout: Any) -> float | None:
    if timeout is not None and not isinstance(timeout, Real):
        raise ErroConfiguracaoSerial("O timeout serial deve ser um número finito não
          negativo ou nulo.")
    try:
        return validar_duracao(timeout, "timeout serial", permitir_none=True)
    except ValueError as exc:
        raise ErroConfiguracaoSerial("O timeout serial deve ser um número finito não
          negativo ou nulo.") from exc
```

```
def _validar_quantidade(quantidade: Any) -> int:
    if isinstance(quantidade, bool) or not isinstance(quantidade, int) or quantidade <
      0:
        raise ErroConfiguracaoSerial("A quantidade de bytes deve ser um inteiro não
          negativo.")
    return quantidade


def _validar_codificacao(codificacao: Any) -> str:
    if not isinstance(codificacao, str) or not codificacao.strip():
        raise ErroConfiguracaoSerial("A codificação serial deve ser um texto válido.")
    try:
        codecs.lookup(codificacao)
    except LookupError as exc:
        raise ErroConfiguracaoSerial(f"Codificação serial desconhecida: {codificacao}")
          from exc
    return codificacao


@runtime_checkable
class TransporteSerial(Protocol):
    @property
    def is_open(self) -> bool: ...
    def write(self, dados: bytes) -> int: ...
    def read(self, quantidade: int = 1) -> bytes: ...
    def readline(self) -> bytes: ...
    def close(self) -> None: ...


@runtime_checkable
class BackendSerial(Protocol):
    """Contrato mínimo de backend serial usado por ``ConexaoSerial``."""

    def abrir(self, porta: str, velocidade: int, timeout: float | None) ->
      TransporteSerial: ...
    def listar_portas(self) -> tuple[PortaSerial, ...]: ...


class BackendSerialReal:
    """Adaptador do pyserial, carregado apenas quando realmente necessário."""

    def abrir(self, porta: str, velocidade: int, timeout: float | None) ->
      TransporteSerial:
        serial = _serial()
        try:
            return serial.Serial(porta, velocidade, timeout=timeout)
        except Exception as exc:
            # Erros do pyserial não fazem parte da API pública Coral.
            raise ErroSerial(f"Não foi possível abrir a porta serial {porta!r}: {exc}")
              from exc

    def listar_portas(self) -> tuple[PortaSerial, ...]:
        try:
            from serial.tools import list_ports
        except Exception as exc:
            raise DependenciaSerialAusente(
                "Para listar portas seriais instale pyserial: python -m pip install
                  pyserial"
            ) from exc
```

```
        try:
            portas = list_ports.comports()
        except Exception as exc:
            raise ErroSerial(f"Não foi possível listar as portas seriais: {exc}") from
              exc
        return tuple(
            PortaSerial(
                p.device,
                p.description or "",
                getattr(p, "manufacturer", None),
                getattr(p, "vid", None),
                getattr(p, "pid", None),
            )
            for p in portas
        )


@dataclass
class _EstadoPortaSimulada:
    info: PortaSerial
    conectada: bool = True
    recebimento: bytearray | None = None
    enviados: bytearray | None = None
    geracao: int = 0

    def __post_init__(self) -> None:
        if self.recebimento is None:
            self.recebimento = bytearray()
        if self.enviados is None:
            self.enviados = bytearray()


class _TransporteSerialSimulado:
    def __init__(self, backend: "BackendSerialSimulado", estado: _EstadoPortaSimulada,
                 velocidade: int, timeout: float | None):
        self._backend = backend
        self._estado = estado
        self.velocidade = velocidade
        self.timeout = timeout
        self._fechada = False
        self._geracao = estado.geracao

    @property
    def is_open(self) -> bool:
        return not self._fechada and self._estado.conectada and self._geracao ==
          self._estado.geracao

    @property
    def desconectada(self) -> bool:
        return not self._estado.conectada or self._geracao != self._estado.geracao

    def _garantir_operavel(self) -> None:
        if self._fechada:
            raise ConexaoSerialFechada("A conexão serial já foi fechada.")
        if not self._estado.conectada:
            raise DesconexaoSerial(f"A porta serial simulada
              {self._estado.info.dispositivo!r} foi desconectada.")

    def write(self, dados: bytes) -> int:
        self._garantir_operavel()
        valor = bytes(dados)
```

```
        assert self._estado.enviados is not None
        self._estado.enviados.extend(valor)
        return len(valor)

    def read(self, quantidade: int = 1) -> bytes:
        self._garantir_operavel()
        if quantidade <= 0:
            return b""
        assert self._estado.recebimento is not None
        n = min(quantidade, len(self._estado.recebimento))
        valor = bytes(self._estado.recebimento[:n])
        del self._estado.recebimento[:n]
        # Sem dados, representa timeout de leitura do pyserial com b"". O
        # backend simulado não dorme: isso mantém testes determinísticos.
        return valor

    def readline(self) -> bytes:
        self._garantir_operavel()
        assert self._estado.recebimento is not None
        if not self._estado.recebimento:
            return b""
        try:
            fim = self._estado.recebimento.index(10) + 1
        except ValueError:
            fim = len(self._estado.recebimento)
        valor = bytes(self._estado.recebimento[:fim])
        del self._estado.recebimento[:fim]
        return valor

    def close(self) -> None:
        self._fechada = True


class BackendSerialSimulado:
    """Backend serial em memória, sem espera real e sem dependências externas.

    Portas precisam ser registradas explicitamente. Bytes injetados com
    :meth:`injetar_recebimento` são consumidos por ``ler``/``ler_linha``;
    bytes escritos ficam disponíveis em :meth:`dados_enviados`.
    """

    def __init__(self) -> None:
        self._portas: dict[str, _EstadoPortaSimulada] = {}

    def registrar_porta(self, dispositivo: str = "SIM0", *, descricao: str = "Porta
      serial simulada",
                        fabricante: str | None = "Coral", vid: int | None = None,
                        pid: int | None = None) -> PortaSerial:
        nome = _validar_porta(dispositivo)
        if nome in self._portas:
            raise ErroConfiguracaoSerial(f"A porta serial simulada {nome!r} já existe.")
        info = PortaSerial(nome, str(descricao), fabricante, vid, pid)
        self._portas[nome] = _EstadoPortaSimulada(info)
        return info

    def remover_porta(self, dispositivo: str) -> None:
        nome = _validar_porta(dispositivo)
        estado = self._portas.get(nome)
        if estado is None:
            raise ErroSerial(f"Porta serial simulada inexistente: {nome!r}.")
        estado.conectada = False
```

```
        estado.geracao += 1
        del self._portas[nome]

    def desconectar(self, dispositivo: str) -> None:
        estado = self._obter(dispositivo)
        if estado.conectada:
            estado.geracao += 1
        estado.conectada = False

    def reconectar(self, dispositivo: str) -> None:
        estado = self._obter(dispositivo)
        estado.conectada = True

    def injetar_recebimento(self, dispositivo: str, dados: bytes | bytearray | str,
                            *, codificacao: str = "utf-8") -> int:
        estado = self._obter(dispositivo)
        if not estado.conectada:
            raise DesconexaoSerial(f"A porta serial simulada {estado.info.dispositivo!r}
               está desconectada.")
        if isinstance(dados, str):
            dados = dados.encode(_validar_codificacao(codificacao))
        elif not isinstance(dados, (bytes, bytearray)):
            raise ErroConfiguracaoSerial("Dados simulados devem ser texto ou bytes.")
        valor = bytes(dados)
        assert estado.recebimento is not None
        estado.recebimento.extend(valor)
        return len(valor)

    def dados_enviados(self, dispositivo: str, *, limpar: bool = False) -> bytes:
        estado = self._obter(dispositivo)
        assert estado.enviados is not None
        valor = bytes(estado.enviados)
        if limpar:
            estado.enviados.clear()
        return valor

    def limpar(self, dispositivo: str | None = None) -> None:
        estados = self._portas.values() if dispositivo is None else
          (self._obter(dispositivo),)
        for estado in estados:
            assert estado.recebimento is not None and estado.enviados is not None
            estado.recebimento.clear()
            estado.enviados.clear()

    def listar_portas(self) -> tuple[PortaSerial, ...]:
        return tuple(estado.info for estado in self._portas.values() if
          estado.conectada)

    def abrir(self, porta: str, velocidade: int, timeout: float | None) ->
      TransporteSerial:
        estado = self._obter(porta)
        if not estado.conectada:
            raise DesconexaoSerial(f"A porta serial simulada {estado.info.dispositivo!r}
               está desconectada.")
        return _TransporteSerialSimulado(self, estado, velocidade, timeout)

    def _obter(self, dispositivo: str) -> _EstadoPortaSimulada:
        nome = _validar_porta(dispositivo)
        try:
            return self._portas[nome]
        except KeyError as exc:
```

```
            raise ErroSerial(f"Porta serial simulada inexistente: {nome!r}.") from exc


_BACKEND_CONTEXTO: ContextVar[BackendSerial | None] = ContextVar("coral_backend_serial",
   default=None)


@contextmanager
def usar_backend_serial(backend: BackendSerial):
    """Usa temporariamente um backend serial como padrão do contexto atual.

    Isso permite executar inclusive a sintaxe natural Coral contra o backend
    simulado, sem alterar chamadas existentes nem depender de estado global
    compartilhado entre threads/tarefas.
    """
    escolhido = _backend(backend)
    token = _BACKEND_CONTEXTO.set(escolhido)
    try:
        yield escolhido
    finally:
        _BACKEND_CONTEXTO.reset(token)


def _backend(backend: BackendSerial | None) -> BackendSerial:
    if backend is None:
        backend = _BACKEND_CONTEXTO.get()
    if backend is None:
        return BackendSerialReal()
    if not isinstance(backend, BackendSerial):
        raise ErroConfiguracaoSerial("Backend serial inválido: esperado objeto com
          abrir() e listar_portas().")
    return backend


def portas_seriais(*, backend: BackendSerial | None = None) -> tuple[PortaSerial, ...]:
    return _backend(backend).listar_portas()


class ConexaoSerial:
    def __init__(self, porta: str, velocidade: int = 115200, timeout: float | None =
      1.0,
                 *, codificacao: str = "utf-8", backend: BackendSerial | None = None):
        self.porta = _validar_porta(porta)
        self.velocidade = _validar_velocidade(velocidade)
        self.timeout = _validar_timeout(timeout)
        self.codificacao = _validar_codificacao(codificacao)
        self.backend = _backend(backend)
        try:
            self._conexao = self.backend.abrir(self.porta, self.velocidade,
              self.timeout)
        except ErroSerial:
            raise
        except Exception as exc:
            raise ErroSerial(f"Não foi possível abrir a porta serial {self.porta!r}:
              {exc}") from exc
        self._fechada = False

    @property
    def aberta(self) -> bool:
        return not self._fechada and bool(getattr(self._conexao, "is_open", False))
```

```
    def _garantir_aberta(self) -> None:
        if self._fechada:
            raise ConexaoSerialFechada("A conexão serial já foi fechada.")
        if bool(getattr(self._conexao, "desconectada", False)):
            raise DesconexaoSerial(f"A porta serial {self.porta!r} foi desconectada.")
        if not bool(getattr(self._conexao, "is_open", False)):
            raise ConexaoSerialFechada("A conexão serial não está aberta.")

    def enviar_bytes(self, dados: bytes | bytearray) -> int:
        self._garantir_aberta()
        if not isinstance(dados, (bytes, bytearray)):
            raise ErroConfiguracaoSerial("enviar_bytes requer bytes ou bytearray.")
        try:
            return int(self._conexao.write(bytes(dados)))
        except ErroSerial:
            raise
        except Exception as exc:
            raise ErroSerial(f"Falha ao escrever na porta serial {self.porta!r}: {exc}")
               from exc

    def enviar(self, texto: Any, *, terminar_linha: bool = False) -> int:
        valor = str(texto)
        if terminar_linha:
            valor += "\n"
        return self.enviar_bytes(valor.encode(self.codificacao))

    def ler(self, quantidade: int = 1) -> bytes:
        n = _validar_quantidade(quantidade)
        self._garantir_aberta()
        try:
            return bytes(self._conexao.read(n))
        except ErroSerial:
            raise
        except Exception as exc:
            raise ErroSerial(f"Falha ao ler a porta serial {self.porta!r}: {exc}") from
              exc

    def ler_linha(self) -> str:
        self._garantir_aberta()
        try:
            dados = bytes(self._conexao.readline())
        except ErroSerial:
            raise
        except Exception as exc:
            raise ErroSerial(f"Falha ao ler a porta serial {self.porta!r}: {exc}") from
              exc
        return dados.decode(self.codificacao, errors="replace").rstrip("\r\n")

    def fechar(self) -> None:
        if self._fechada:
            return
        try:
            self._conexao.close()
        except ErroSerial:
            raise
        except Exception as exc:
            raise ErroSerial(f"Falha ao fechar a porta serial {self.porta!r}: {exc}")
              from exc
        finally:
            self._fechada = True
```

```
    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc, tb):
        self.fechar()
        return False


def conectar_serial(porta: str, velocidade: int = 115200, timeout: float | None = 1.0,
                    *, codificacao: str = "utf-8", backend: BackendSerial | None = None)
                       -> ConexaoSerial:
    return ConexaoSerial(porta, velocidade, timeout, codificacao=codificacao,
      backend=backend)


__all__ = [
    "ErroSerial", "DependenciaSerialAusente", "ErroConfiguracaoSerial",
    "ConexaoSerialFechada", "DesconexaoSerial", "PortaSerial", "BackendSerial",
    "BackendSerialReal", "BackendSerialSimulado", "usar_backend_serial",
      "ConexaoSerial",
    "portas_seriais", "conectar_serial",
]
```

Biblioteca `coral.hardware.serial`

`coral.hardware.serial.BackendSerial`

Contrato mínimo de backend serial usado por `ConexaoSerial`.

Assinatura

Assinatura para consulta

```
BackendSerial(*args, **kwargs)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
args	sem anotação	VAR_POSITIONAL	não	sem padrão
kwargs	sem anotação	VAR_KEYWORD	não	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
abrir(self, porta: 'str', velocidade: 'int', timeout: 'float | None') ->
  'TransporteSerial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
listar_portas(self) -> 'tuple[PortaSerial, ...]'
```

Método público sem descrição própria no runtime.

`coral.hardware.serial.BackendSerialReal`

Adaptador do pyserial, carregado apenas quando realmente necessário.

Assinatura

Assinatura para consulta

```
BackendSerialReal()
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
abrir(self, porta: 'str', velocidade: 'int', timeout: 'float | None') ->
  'TransporteSerial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
listar_portas(self) -> 'tuple[PortaSerial, ...]'
```

Método público sem descrição própria no runtime.

`coral.hardware.serial.BackendSerialSimulado`

Backend serial em memória, sem espera real e sem dependências externas.

Portas precisam ser registradas explicitamente. Bytes injetados com :meth:`injetar_recebimento` são consumidos por `ler/ler_linha; bytes escritos ficam disponíveis em :meth:`dados_enviados`.

Assinatura

Assinatura para consulta

```
BackendSerialSimulado() -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

None

Métodos públicos

Referência para leitura

```
abrir(self, porta: 'str', velocidade: 'int', timeout: 'float | None') ->
  'TransporteSerial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
dados_enviados(self, dispositivo: 'str', *, limpar: 'bool' = False) -> 'bytes'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
desconectar(self, dispositivo: 'str') -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
injetar_recebimento(self, dispositivo: 'str', dados: 'bytes | bytearray | str', *,
  codificacao: 'str' = 'utf-8') -> 'int'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
limpar(self, dispositivo: 'str | None' = None) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
listar_portas(self) -> 'tuple[PortaSerial, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
reconectar(self, dispositivo: 'str') -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
registrar_porta(self, dispositivo: 'str' = 'SIM0', *, descricao: 'str' = 'Porta serial
  simulada', fabricante: 'str | None' = 'Coral', vid: 'int | None' = None, pid: 'int |
  None' = None) -> 'PortaSerial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_porta(self, dispositivo: 'str') -> 'None'
```

Método público sem descrição própria no runtime.

`coral.hardware.serial.ConexaoSerial`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
ConexaoSerial(porta: 'str', velocidade: 'int' = 115200, timeout: 'float | None' = 1.0,
  *, codificacao: 'str' = 'utf-8', backend: 'BackendSerial | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
porta	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
velocidade	int	POSITIONAL_OR_KEYWORD	não	115200
timeout	float | None	POSITIONAL_OR_KEYWORD	não	1.0
codificacao	str	KEYWORD_ONLY	não	'utf-8'
backend	BackendSerial | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
enviar(self, texto: 'Any', *, terminar_linha: 'bool' = False) -> 'int'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
enviar_bytes(self, dados: 'bytes | bytearray') -> 'int'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
fechar(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
ler(self, quantidade: 'int' = 1) -> 'bytes'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
ler_linha(self) -> 'str'
```

Método público sem descrição própria no runtime.

`coral.hardware.serial.ConexaoSerialFechada`

Uma operação foi solicitada depois do fechamento da conexão.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

`with_traceback`

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.hardware.serial.DependenciaSerialAusente`

pyserial não está disponível para o backend real.

Métodos públicos

Referência para leitura

`add_note`

Exception.add_note(note) -- add a note to the exception

Referência para leitura

`with_traceback`

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.hardware.serial.DesconexaoSerial`

A porta deixou de estar conectada durante a sessão.

Métodos públicos

Referência para leitura

`add_note`

Exception.add_note(note) -- add a note to the exception

Referência para leitura

`with_traceback`

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.hardware.serial.ErroConfiguracaoSerial`

Parâmetro serial inválido antes da abertura da porta.

Métodos públicos

Referência para leitura

`add_note`

Exception.add_note(note) -- add a note to the exception

Referência para leitura

`with_traceback`

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.hardware.serial.ErroSerial`

Erro público base da camada serial Coral.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.hardware.serial.PortaSerial`

PortaSerial(dispositivo: 'str', descricao: 'str' = '', fabricante: 'str | None' = None, vid: 'int | None' = None, pid: 'int | None' = None)

Assinatura

Assinatura para consulta

```
PortaSerial(dispositivo: 'str', descricao: 'str' = '', fabricante: 'str | None' = None,
  vid: 'int | None' = None, pid: 'int | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
dispositivo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
descricao	str	POSITIONAL_OR_KEYWORD	não	''
fabricante	str | None	POSITIONAL_OR_KEYWORD	não	None
vid	int | None	POSITIONAL_OR_KEYWORD	não	None
pid	int | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `dispositivo`: `str`.

Atributo `descricao`: `str`.

Atributo `fabricante`: `str | None`.

Atributo `vid`: `int | None`.

Atributo `pid`: `int | None`.

`coral.hardware.serial.conectar_serial`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
conectar_serial(porta: 'str', velocidade: 'int' = 115200, timeout: 'float | None' = 1.0,
    *, codificacao: 'str' = 'utf-8', backend: 'BackendSerial | None' = None) ->
  'ConexaoSerial'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
porta	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
velocidade	int	POSITIONAL_OR_KEYWORD	não	115200
timeout	float | None	POSITIONAL_OR_KEYWORD	não	1.0
codificacao	str	KEYWORD_ONLY	não	'utf-8'
backend	BackendSerial | None	KEYWORD_ONLY	não	None

Retorno declarado

ConexaoSerial

`coral.hardware.serial.portas_seriais`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
portas_seriais(*, backend: 'BackendSerial | None' = None) -> 'tuple[PortaSerial, ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
backend	BackendSerial | None	KEYWORD_ONLY	não	None

Retorno declarado

tuple[PortaSerial, ...]

`coral.hardware.serial.usar_backend_serial`

Usa temporariamente um backend serial como padrão do contexto atual.

Isso permite executar inclusive a sintaxe natural Coral contra o backend simulado, sem alterar chamadas existentes nem depender de estado global compartilhado entre threads/tarefas.

Assinatura

Assinatura para consulta

```
usar_backend_serial(backend: 'BackendSerial')
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
backend	BackendSerial	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Consultar a implementação canônica de coral.hardware.serial

Implementação canônica de `coral.hardware.serial`

Código Python

```
"""Contratos transversais pequenos do runtime Coral.

A camada comum não substitui APIs de domínio. Ela fornece a gramática
arquitetural compartilhada do ciclo 1.5 para erro operacional, resultado e
diagnóstico, preservando os nomes públicos existentes em cada biblioteca.
"""
from __future__ import annotations

from dataclasses import asdict, dataclass, field
from typing import Any, Iterable, Mapping

CONTRATO = "coral.comum/1"
CONTRATO_DIAGNOSTICO = "coral.diagnostico.operacional/1"


class ErroCoral(Exception):
    """Raiz pública dos erros controlados pela linguagem e bibliotecas Coral."""


class ErroOperacaoCoral(RuntimeError, ErroCoral):
    """Falha operacional de um recurso Coral ou de seu backend."""


class ErroDependenciaCoral(ErroOperacaoCoral):

    """Uma capacidade opcional não pode operar por ausência de dependência."""


@dataclass(frozen=True)
class FalhaOperacao:
    codigo: str
    mensagem: str
    dominio: str
    tipo: str = "operacional"
```

```
    causa: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def de_excecao(cls, exc: BaseException, *, codigo: str, dominio: str, tipo: str =
      "operacional", detalhe: Mapping[str, Any] | None = None) -> "FalhaOperacao":
        causa = exc.__cause__ if exc.__cause__ is not None else exc
        return cls(codigo, str(exc), dominio, tipo, type(causa).__name__, dict(detalhe
          or {}))

    def para_dict(self) -> dict[str, Any]:
        return asdict(self)


@dataclass(frozen=True)
class ResultadoOperacao:
    """Representação transversal sem substituir resultados ricos de domínio."""
    ok: bool
    valor: Any = None
    falha: FalhaOperacao | None = None
    metadados: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def sucesso(cls, valor: Any = None, **metadados: Any) -> "ResultadoOperacao":
        return cls(True, valor=valor, metadados=metadados)

    @classmethod
    def erro(cls, falha: FalhaOperacao, **metadados: Any) -> "ResultadoOperacao":
        return cls(False, falha=falha, metadados=metadados)

    def para_dict(self) -> dict[str, Any]:
        return {
            "contrato": CONTRATO,
            "ok": self.ok,
            "valor": self.valor,
            "falha": self.falha.para_dict() if self.falha is not None else None,
            "metadados": dict(self.metadados),
        }


@dataclass(frozen=True)
class DiagnosticoOperacional:
    id: str
    dominio: str
    ok: bool
    mensagem: str
    capacidade: str | None = None
    sugestao: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @property
    def nivel(self) -> str:
        return "info" if self.ok else "erro"

    def para_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "dominio": self.dominio,
            "ok": self.ok,
            "nivel": self.nivel,
            "mensagem": self.mensagem,
```

```
            "capacidade": self.capacidade,
            "sugestao": self.sugestao,
            "detalhe": dict(self.detalhe),
        }


def diagnostico(id: str, dominio: str, ok: bool, mensagem: Any, *, capacidade: str |
  None = None, sugestao: str | None = None, detalhe: Mapping[str, Any] | None = None) ->
  DiagnosticoOperacional:
    return DiagnosticoOperacional(str(id), str(dominio), bool(ok), str(mensagem),
      capacidade, sugestao, dict(detalhe or {}))


def agregar_diagnosticos(itens: Iterable[DiagnosticoOperacional | Mapping[str, Any]], *,
   origem: str = "coral") -> dict[str, Any]:
    normalizados: list[dict[str, Any]] = []
    for item in itens:
        if isinstance(item, DiagnosticoOperacional):
            normalizados.append(item.para_dict())
            continue
        d = dict(item)
        if "id" not in d:
            d["id"] = str(d.get("nome", "diagnostico"))
        if "dominio" not in d:
            d["dominio"] = origem
        d["ok"] = bool(d.get("ok"))
        d.setdefault("nivel", "info" if d["ok"] else "erro")
        d.setdefault("mensagem", str(d.get("detalhe", "")))
        d.setdefault("capacidade", None)
        d.setdefault("sugestao", None)
        detalhe = d.get("detalhe")
        if not isinstance(detalhe, Mapping):
            d["detalhe"] = {"valor": detalhe}
        normalizados.append(d)
    return {
        "contrato": CONTRATO_DIAGNOSTICO,
        "origem": origem,
        "ok": all(x["ok"] for x in normalizados),
        "quantidade": len(normalizados),
        "itens": normalizados,
    }


__all__ = [
    "CONTRATO", "CONTRATO_DIAGNOSTICO", "ErroCoral", "ErroOperacaoCoral",
    "ErroDependenciaCoral", "FalhaOperacao", "ResultadoOperacao",
    "DiagnosticoOperacional", "diagnostico", "agregar_diagnosticos",
]

"""Comunicação serial opcional para Coral 1.3.

O backend real usa pyserial quando disponível. ``BackendSerialSimulado`` oferece
um transporte determinístico para testes, aulas e CI sem depender de dispositivo
físico. Ambos obedecem ao mesmo contrato consumido por :class:`ConexaoSerial`.
"""
from __future__ import annotations

from dataclasses import dataclass
from contextlib import contextmanager
from contextvars import ContextVar
import codecs
```

```
from numbers import Real
from typing import Any, Protocol, runtime_checkable

from ..comum import ErroDependenciaCoral, ErroOperacaoCoral
from ..tempo_eventos import validar_duracao


class ErroSerial(ErroOperacaoCoral):
    """Erro público base da camada serial Coral."""


class DependenciaSerialAusente(ErroSerial, ErroDependenciaCoral):
    """pyserial não está disponível para o backend real."""


class ErroConfiguracaoSerial(ErroSerial):
    """Parâmetro serial inválido antes da abertura da porta."""


class ConexaoSerialFechada(ErroSerial):
    """Uma operação foi solicitada depois do fechamento da conexão."""


class DesconexaoSerial(ErroSerial):
    """A porta deixou de estar conectada durante a sessão."""


def _serial():
    """Carrega pyserial sob demanda.

    A função permanece separada também por compatibilidade com integrações que
    já a substituíam em testes da linha Coral 1.3.
    """
    try:
        import serial
        return serial
    except Exception as exc:
        raise DependenciaSerialAusente(
            "A comunicação serial da Coral requer pyserial. "
            "Instale com: python -m pip install pyserial"
        ) from exc


@dataclass(frozen=True)
class PortaSerial:
    dispositivo: str
    descricao: str = ""
    fabricante: str | None = None
    vid: int | None = None
    pid: int | None = None


def _validar_porta(porta: Any) -> str:
    if not isinstance(porta, str) or not porta.strip():
        raise ErroConfiguracaoSerial("A porta serial deve ser um texto não vazio.")
    return porta.strip()


def _validar_velocidade(velocidade: Any) -> int:
    if isinstance(velocidade, bool) or not isinstance(velocidade, int) or velocidade <=
      0:
```

```
        raise ErroConfiguracaoSerial("A velocidade serial deve ser um inteiro positivo
          em bauds.")
    return velocidade


def _validar_timeout(timeout: Any) -> float | None:
    if timeout is not None and not isinstance(timeout, Real):
        raise ErroConfiguracaoSerial("O timeout serial deve ser um número finito não
          negativo ou nulo.")
    try:
        return validar_duracao(timeout, "timeout serial", permitir_none=True)
    except ValueError as exc:
        raise ErroConfiguracaoSerial("O timeout serial deve ser um número finito não
          negativo ou nulo.") from exc


def _validar_quantidade(quantidade: Any) -> int:
    if isinstance(quantidade, bool) or not isinstance(quantidade, int) or quantidade <
      0:
        raise ErroConfiguracaoSerial("A quantidade de bytes deve ser um inteiro não
          negativo.")
    return quantidade


def _validar_codificacao(codificacao: Any) -> str:
    if not isinstance(codificacao, str) or not codificacao.strip():
        raise ErroConfiguracaoSerial("A codificação serial deve ser um texto válido.")
    try:
        codecs.lookup(codificacao)
    except LookupError as exc:
        raise ErroConfiguracaoSerial(f"Codificação serial desconhecida: {codificacao}")
          from exc
    return codificacao


@runtime_checkable
class TransporteSerial(Protocol):
    @property
    def is_open(self) -> bool: ...
    def write(self, dados: bytes) -> int: ...
    def read(self, quantidade: int = 1) -> bytes: ...
    def readline(self) -> bytes: ...
    def close(self) -> None: ...


@runtime_checkable
class BackendSerial(Protocol):
    """Contrato mínimo de backend serial usado por ``ConexaoSerial``."""

    def abrir(self, porta: str, velocidade: int, timeout: float | None) ->
      TransporteSerial: ...
    def listar_portas(self) -> tuple[PortaSerial, ...]: ...


class BackendSerialReal:
    """Adaptador do pyserial, carregado apenas quando realmente necessário."""

    def abrir(self, porta: str, velocidade: int, timeout: float | None) ->
      TransporteSerial:
        serial = _serial()
        try:
```

```
            return serial.Serial(porta, velocidade, timeout=timeout)
        except Exception as exc:
            # Erros do pyserial não fazem parte da API pública Coral.
            raise ErroSerial(f"Não foi possível abrir a porta serial {porta!r}: {exc}")
              from exc

    def listar_portas(self) -> tuple[PortaSerial, ...]:
        try:
            from serial.tools import list_ports
        except Exception as exc:
            raise DependenciaSerialAusente(
                "Para listar portas seriais instale pyserial: python -m pip install
                  pyserial"
            ) from exc
        try:
            portas = list_ports.comports()
        except Exception as exc:
            raise ErroSerial(f"Não foi possível listar as portas seriais: {exc}") from
              exc
        return tuple(
            PortaSerial(
                p.device,
                p.description or "",
                getattr(p, "manufacturer", None),
                getattr(p, "vid", None),
                getattr(p, "pid", None),
            )
            for p in portas
        )


@dataclass
class _EstadoPortaSimulada:
    info: PortaSerial
    conectada: bool = True
    recebimento: bytearray | None = None
    enviados: bytearray | None = None
    geracao: int = 0

    def __post_init__(self) -> None:
        if self.recebimento is None:
            self.recebimento = bytearray()
        if self.enviados is None:
            self.enviados = bytearray()


class _TransporteSerialSimulado:
    def __init__(self, backend: "BackendSerialSimulado", estado: _EstadoPortaSimulada,
                 velocidade: int, timeout: float | None):
        self._backend = backend
        self._estado = estado
        self.velocidade = velocidade
        self.timeout = timeout
        self._fechada = False
        self._geracao = estado.geracao

    @property
    def is_open(self) -> bool:
        return not self._fechada and self._estado.conectada and self._geracao ==
          self._estado.geracao
```

```
    @property
    def desconectada(self) -> bool:
        return not self._estado.conectada or self._geracao != self._estado.geracao

    def _garantir_operavel(self) -> None:
        if self._fechada:
            raise ConexaoSerialFechada("A conexão serial já foi fechada.")
        if not self._estado.conectada:
            raise DesconexaoSerial(f"A porta serial simulada
              {self._estado.info.dispositivo!r} foi desconectada.")

    def write(self, dados: bytes) -> int:
        self._garantir_operavel()
        valor = bytes(dados)
        assert self._estado.enviados is not None
        self._estado.enviados.extend(valor)
        return len(valor)

    def read(self, quantidade: int = 1) -> bytes:
        self._garantir_operavel()
        if quantidade <= 0:
            return b""
        assert self._estado.recebimento is not None
        n = min(quantidade, len(self._estado.recebimento))
        valor = bytes(self._estado.recebimento[:n])
        del self._estado.recebimento[:n]
        # Sem dados, representa timeout de leitura do pyserial com b"". O
        # backend simulado não dorme: isso mantém testes determinísticos.
        return valor

    def readline(self) -> bytes:
        self._garantir_operavel()
        assert self._estado.recebimento is not None
        if not self._estado.recebimento:
            return b""
        try:
            fim = self._estado.recebimento.index(10) + 1
        except ValueError:
            fim = len(self._estado.recebimento)
        valor = bytes(self._estado.recebimento[:fim])
        del self._estado.recebimento[:fim]
        return valor

    def close(self) -> None:
        self._fechada = True


class BackendSerialSimulado:
    """Backend serial em memória, sem espera real e sem dependências externas.

    Portas precisam ser registradas explicitamente. Bytes injetados com
    :meth:`injetar_recebimento` são consumidos por ``ler``/``ler_linha``;
    bytes escritos ficam disponíveis em :meth:`dados_enviados`.
    """

    def __init__(self) -> None:
        self._portas: dict[str, _EstadoPortaSimulada] = {}

    def registrar_porta(self, dispositivo: str = "SIM0", *, descricao: str = "Porta
      serial simulada",
                        fabricante: str | None = "Coral", vid: int | None = None,
```

```
                    pid: int | None = None) -> PortaSerial:
        nome = _validar_porta(dispositivo)
        if nome in self._portas:
            raise ErroConfiguracaoSerial(f"A porta serial simulada {nome!r} já existe.")
        info = PortaSerial(nome, str(descricao), fabricante, vid, pid)
        self._portas[nome] = _EstadoPortaSimulada(info)
        return info

    def remover_porta(self, dispositivo: str) -> None:
        nome = _validar_porta(dispositivo)
        estado = self._portas.get(nome)
        if estado is None:
            raise ErroSerial(f"Porta serial simulada inexistente: {nome!r}.")
        estado.conectada = False
        estado.geracao += 1
        del self._portas[nome]

    def desconectar(self, dispositivo: str) -> None:
        estado = self._obter(dispositivo)
        if estado.conectada:
            estado.geracao += 1
        estado.conectada = False

    def reconectar(self, dispositivo: str) -> None:
        estado = self._obter(dispositivo)
        estado.conectada = True

    def injetar_recebimento(self, dispositivo: str, dados: bytes | bytearray | str,
                            *, codificacao: str = "utf-8") -> int:
        estado = self._obter(dispositivo)
        if not estado.conectada:
            raise DesconexaoSerial(f"A porta serial simulada {estado.info.dispositivo!r}
               está desconectada.")
        if isinstance(dados, str):
            dados = dados.encode(_validar_codificacao(codificacao))
        elif not isinstance(dados, (bytes, bytearray)):
            raise ErroConfiguracaoSerial("Dados simulados devem ser texto ou bytes.")
        valor = bytes(dados)
        assert estado.recebimento is not None
        estado.recebimento.extend(valor)
        return len(valor)

    def dados_enviados(self, dispositivo: str, *, limpar: bool = False) -> bytes:
        estado = self._obter(dispositivo)
        assert estado.enviados is not None
        valor = bytes(estado.enviados)
        if limpar:
            estado.enviados.clear()
        return valor

    def limpar(self, dispositivo: str | None = None) -> None:
        estados = self._portas.values() if dispositivo is None else
          (self._obter(dispositivo),)
        for estado in estados:
            assert estado.recebimento is not None and estado.enviados is not None
            estado.recebimento.clear()
            estado.enviados.clear()

    def listar_portas(self) -> tuple[PortaSerial, ...]:
        return tuple(estado.info for estado in self._portas.values() if
          estado.conectada)
```

```
    def abrir(self, porta: str, velocidade: int, timeout: float | None) ->
      TransporteSerial:
        estado = self._obter(porta)
        if not estado.conectada:
            raise DesconexaoSerial(f"A porta serial simulada {estado.info.dispositivo!r}
               está desconectada.")
        return _TransporteSerialSimulado(self, estado, velocidade, timeout)

    def _obter(self, dispositivo: str) -> _EstadoPortaSimulada:
        nome = _validar_porta(dispositivo)
        try:
            return self._portas[nome]
        except KeyError as exc:
            raise ErroSerial(f"Porta serial simulada inexistente: {nome!r}.") from exc


_BACKEND_CONTEXTO: ContextVar[BackendSerial | None] = ContextVar("coral_backend_serial",
   default=None)


@contextmanager
def usar_backend_serial(backend: BackendSerial):
    """Usa temporariamente um backend serial como padrão do contexto atual.

    Isso permite executar inclusive a sintaxe natural Coral contra o backend
    simulado, sem alterar chamadas existentes nem depender de estado global
    compartilhado entre threads/tarefas.
    """
    escolhido = _backend(backend)
    token = _BACKEND_CONTEXTO.set(escolhido)
    try:
        yield escolhido
    finally:
        _BACKEND_CONTEXTO.reset(token)


def _backend(backend: BackendSerial | None) -> BackendSerial:
    if backend is None:
        backend = _BACKEND_CONTEXTO.get()
    if backend is None:
        return BackendSerialReal()
    if not isinstance(backend, BackendSerial):
        raise ErroConfiguracaoSerial("Backend serial inválido: esperado objeto com
          abrir() e listar_portas().")
    return backend


def portas_seriais(*, backend: BackendSerial | None = None) -> tuple[PortaSerial, ...]:
    return _backend(backend).listar_portas()


class ConexaoSerial:
    def __init__(self, porta: str, velocidade: int = 115200, timeout: float | None =
      1.0,
                 *, codificacao: str = "utf-8", backend: BackendSerial | None = None):
        self.porta = _validar_porta(porta)
        self.velocidade = _validar_velocidade(velocidade)
        self.timeout = _validar_timeout(timeout)
        self.codificacao = _validar_codificacao(codificacao)
        self.backend = _backend(backend)
```

```
        try:
            self._conexao = self.backend.abrir(self.porta, self.velocidade,
              self.timeout)
        except ErroSerial:
            raise
        except Exception as exc:
            raise ErroSerial(f"Não foi possível abrir a porta serial {self.porta!r}:
              {exc}") from exc
        self._fechada = False

    @property
    def aberta(self) -> bool:
        return not self._fechada and bool(getattr(self._conexao, "is_open", False))

    def _garantir_aberta(self) -> None:
        if self._fechada:
            raise ConexaoSerialFechada("A conexão serial já foi fechada.")
        if bool(getattr(self._conexao, "desconectada", False)):
            raise DesconexaoSerial(f"A porta serial {self.porta!r} foi desconectada.")
        if not bool(getattr(self._conexao, "is_open", False)):
            raise ConexaoSerialFechada("A conexão serial não está aberta.")

    def enviar_bytes(self, dados: bytes | bytearray) -> int:
        self._garantir_aberta()
        if not isinstance(dados, (bytes, bytearray)):
            raise ErroConfiguracaoSerial("enviar_bytes requer bytes ou bytearray.")
        try:
            return int(self._conexao.write(bytes(dados)))
        except ErroSerial:
            raise
        except Exception as exc:
            raise ErroSerial(f"Falha ao escrever na porta serial {self.porta!r}: {exc}")
               from exc

    def enviar(self, texto: Any, *, terminar_linha: bool = False) -> int:
        valor = str(texto)
        if terminar_linha:
            valor += "\n"
        return self.enviar_bytes(valor.encode(self.codificacao))

    def ler(self, quantidade: int = 1) -> bytes:
        n = _validar_quantidade(quantidade)
        self._garantir_aberta()
        try:
            return bytes(self._conexao.read(n))
        except ErroSerial:
            raise
        except Exception as exc:
            raise ErroSerial(f"Falha ao ler a porta serial {self.porta!r}: {exc}") from
              exc

    def ler_linha(self) -> str:
        self._garantir_aberta()
        try:
            dados = bytes(self._conexao.readline())
        except ErroSerial:
            raise
        except Exception as exc:
            raise ErroSerial(f"Falha ao ler a porta serial {self.porta!r}: {exc}") from
              exc
        return dados.decode(self.codificacao, errors="replace").rstrip("\r\n")
```

```
    def fechar(self) -> None:
        if self._fechada:
            return
        try:
            self._conexao.close()
        except ErroSerial:
            raise
        except Exception as exc:
            raise ErroSerial(f"Falha ao fechar a porta serial {self.porta!r}: {exc}")
              from exc
        finally:
            self._fechada = True

    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc, tb):
        self.fechar()
        return False


def conectar_serial(porta: str, velocidade: int = 115200, timeout: float | None = 1.0,
                    *, codificacao: str = "utf-8", backend: BackendSerial | None = None)
                       -> ConexaoSerial:
    return ConexaoSerial(porta, velocidade, timeout, codificacao=codificacao,
      backend=backend)


__all__ = [
    "ErroSerial", "DependenciaSerialAusente", "ErroConfiguracaoSerial",
    "ConexaoSerialFechada", "DesconexaoSerial", "PortaSerial", "BackendSerial",
    "BackendSerialReal", "BackendSerialSimulado", "usar_backend_serial",
      "ConexaoSerial",
    "portas_seriais", "conectar_serial",
]

"""Tempo, eventos e cancelamento compartilhados do runtime Coral.

Este módulo materializa a segunda parte da coesão 1.5 sem substituir as APIs
históricas de ``coral.regras``. Ele fornece fontes de tempo injetáveis,
validação uniforme de durações, eventos síncronos/enfileirados/propagados e um
token cooperativo de cancelamento reutilizável por bibliotecas e ferramentas.
"""
from __future__ import annotations

from collections import deque
from dataclasses import dataclass, field
import math
import time
from typing import Any, Callable, Mapping, Protocol, runtime_checkable

from .comum import ErroOperacaoCoral

CONTRATO = "coral.tempo_eventos/1"


def validar_duracao(valor: Any, nome: str = "duração", *, permitir_none: bool = False,
                    permitir_zero: bool = True) -> float | None:
    """Normaliza uma duração pública para segundos finitos não negativos."""
    if valor is None and permitir_none:
```

```
        return None
    if isinstance(valor, bool):
        raise ValueError(f"{nome} deve ser um número finito maior ou igual a zero")
    try:
        numero = float(valor)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ValueError(f"{nome} deve ser um número finito maior ou igual a zero") from
           exc
    if not math.isfinite(numero) or numero < 0 or (numero == 0 and not permitir_zero):
        operador = "maior que zero" if not permitir_zero else "maior ou igual a zero"
        raise ValueError(f"{nome} deve ser um número finito {operador}")
    return numero


def normalizar_dt(valor: Any, *, compatibilidade_negativo_zero: bool = True) -> float:
    """Normaliza avanço de simulação.

    A linha 1.2 já definia que ``dt`` negativo não volta temporizadores. Por
    compatibilidade, a política padrão converte valores negativos finitos em
    zero; novos consumidores podem optar por rejeitá-los.
    """
    if isinstance(valor, bool):
        raise ValueError("avanço de tempo deve ser um número finito")
    try:
        numero = float(valor)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ValueError("avanço de tempo deve ser um número finito") from exc
    if not math.isfinite(numero):
        raise ValueError("avanço de tempo deve ser um número finito")
    if numero < 0:
        if compatibilidade_negativo_zero:
            return 0.0
        raise ValueError("avanço de tempo não pode ser negativo")
    return numero


@runtime_checkable
class FonteTempo(Protocol):
    def agora(self) -> float: ...
    def passo(self) -> float: ...


class RelogioMonotonico:
    """Relógio real baseado em ``perf_counter`` e apropriado para durações."""
    def __init__(self, fonte: Callable[[], float] = time.perf_counter):
        self._fonte = fonte
        self._ultimo = float(fonte())

    def agora(self) -> float:
        return float(self._fonte())

    def passo(self) -> float:
        atual = self.agora()
        dt = normalizar_dt(atual - self._ultimo)
        self._ultimo = atual
        return dt


class RelogioSimulado:
    """Relógio controlado pelo programa, sem dormir nem consultar o sistema."""
    def __init__(self, inicial: float = 0.0):
```

```
        self._agora = float(validar_duracao(inicial, "tempo inicial"))
        self._ultimo = self._agora

    def agora(self) -> float:
        return self._agora

    def avancar(self, dt: Any) -> float:
        self._agora += normalizar_dt(dt, compatibilidade_negativo_zero=False)
        return self._agora

    def passo(self) -> float:
        dt = normalizar_dt(self._agora - self._ultimo,
          compatibilidade_negativo_zero=False)
        self._ultimo = self._agora
        return dt

    def definir(self, instante: Any) -> float:
        novo = float(validar_duracao(instante, "instante"))
        if novo < self._agora:
            raise ValueError("o relógio simulado não pode retroceder")
        self._agora = novo
        return novo


class OperacaoCancelada(ErroOperacaoCoral):
    """Operação cooperativa interrompida antes de uma fase segura."""


class TokenCancelamento:
    def __init__(self):
        self._cancelado = False
        self._motivo: str | None = None

    @property
    def cancelado(self) -> bool:
        return self._cancelado

    @property
    def motivo(self) -> str | None:
        return self._motivo

    def cancelar(self, motivo: str = "Operação cancelada pelo usuário.") -> None:
        self._cancelado = True
        self._motivo = str(motivo)

    def verificar(self) -> None:
        if self._cancelado:
            raise OperacaoCancelada(self._motivo or "Operação cancelada.")


@dataclass
class Evento:
    nome: str
    args: tuple[Any, ...] = ()
    kwargs: Mapping[str, Any] = field(default_factory=dict)
    origem: str | None = None
    sequencia: int = 0
    instante: float | None = None
    cancelavel: bool = False
    cancelado: bool = False
    motivo_cancelamento: str | None = None
```

```
    consumido: bool = False
    metadados: Mapping[str, Any] = field(default_factory=dict)
    causa_sequencia: int | None = None

    def __post_init__(self):
        self.nome = str(self.nome)
        self.args = tuple(self.args)
        self.kwargs = dict(self.kwargs)
        self.origem = None if self.origem is None else str(self.origem)
        self.metadados = dict(self.metadados)

    def __getattr__(self, nome: str) -> Any:
        payload = self.__dict__.get("kwargs", {})
        if nome in payload:
            return payload[nome]
        raise AttributeError(nome)

    def cancelar(self, motivo: str | None = None) -> None:
        if not self.cancelavel:
            raise ValueError("este evento não é cancelável")
        self.cancelado = True
        self.motivo_cancelamento = None if motivo is None else str(motivo)

    def consumir(self) -> None:
        self.consumido = True

    def estado_inspecao(self) -> dict[str, Any]:
        """Estado observável do evento sem executar ouvintes nem alterar a fila."""
        return {
            "nome": self.nome,
            "args": self.args,
            "kwargs": dict(self.kwargs),
            "origem": self.origem,
            "sequencia": self.sequencia,
            "instante": self.instante,
            "cancelavel": self.cancelavel,
            "cancelado": self.cancelado,
            "motivo_cancelamento": self.motivo_cancelamento,
            "consumido": self.consumido,
            "metadados": dict(self.metadados),
            "causa_sequencia": self.causa_sequencia,
        }


@dataclass
class _InscricaoEvento:
    funcao: Callable[..., Any]
    prioridade: int
    ordem: int
    receber_evento: bool = False
    max_execucoes: int | None = None
    cooldown: float = 0.0
    execucoes: int = 0
    ultimo_disparo: float | None = None


_PRIORIDADES_EVENTO = {
    "baixa": 0,
    "normal": 100,
    "alta": 200,
    "critica": 300,
```

```
    "crítica": 300,
}


def _normalizar_prioridade_evento(prioridade: str | int) -> int:
    if isinstance(prioridade, bool):
        raise ValueError("prioridade de evento desconhecida: use baixa, normal, alta ou
          crítica")
    if isinstance(prioridade, int):
        return prioridade
    nome = str(prioridade).strip().casefold()
    if nome not in _PRIORIDADES_EVENTO:
        raise ValueError(
            f"prioridade de evento desconhecida: {prioridade!r}; use baixa, normal, alta
               ou crítica"
        )
    return _PRIORIDADES_EVENTO[nome]


class Eventos:
    """Barramento local compatível com ``coral.regras.Eventos``.

    ``emitir`` mantém a chamada histórica de ouvintes via ``*args`` e
    ``**kwargs``. Inscrições novas podem receber o objeto :class:`Evento`
    completo e declarar prioridade sem alterar consumidores antigos.
    """
    def __init__(
        self,
        *,
        pai: "Eventos | None" = None,
        relogio: FonteTempo | None = None,
        fonte_instante: Callable[[], float] | None = None,
    ):
        self._ouvintes: dict[str, list[_InscricaoEvento]] = {}
        self._fila: deque[Evento] = deque()
        self._pai = pai
        self._relogio = relogio
        self._fonte_instante = fonte_instante
        self._sequencia = 0
        self._ordem_inscricao = 0
        self._pilha_processamento: list[Evento] = []

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "pendentes": len(self._fila),
            "fila": tuple(self._fila),
            "tipos_ouvintes": tuple(sorted(self._ouvintes)),
            "quantidade_ouvintes": sum(len(itens) for itens in self._ouvintes.values()),
            "ouvintes_por_tipo": {
                nome: len(itens)
                for nome, itens in sorted(self._ouvintes.items())
            },
            "sequencia": self._sequencia,
            "possui_pai": self._pai is not None,
            "possui_fonte_instante": self.possui_fonte_instante,
        }

    @property
    def pendentes(self) -> int:
        return len(self._fila)
```

```
    @property
    def pai(self) -> "Eventos | None":
        return self._pai

    @property
    def relogio(self) -> FonteTempo | None:
        return self._relogio

    def definir_fonte_instante(self, fonte: Callable[[], float] | None) -> None:
        self._fonte_instante = fonte

    def _instante_atual(self) -> float | None:
        if self._relogio is not None:
            return float(self._relogio.agora())
        if self._fonte_instante is not None:
            return float(self._fonte_instante())
        return None

    @property
    def possui_fonte_instante(self) -> bool:
        return self._relogio is not None or self._fonte_instante is not None

    def definir_pai(self, pai: "Eventos | None") -> None:
        if pai is self:
            raise ValueError("um barramento de eventos não pode propagar para si mesmo")
        atual = pai
        while atual is not None:
            if atual is self:
                raise ValueError("ciclo de propagação de eventos não permitido")
            atual = atual.pai
        self._pai = pai

    def quando(
        self,
        nome: str,
        funcao: Callable[..., Any],
        *,
        prioridade: str | int = "normal",
        receber_evento: bool = False,
        max_execucoes: int | None = None,
        cooldown: float = 0.0,
    ):
        valor_prioridade = _normalizar_prioridade_evento(prioridade)
        if max_execucoes is not None:
            if isinstance(max_execucoes, bool) or not isinstance(max_execucoes, int) or
              max_execucoes <= 0:
                raise ValueError("máximo de execuções de evento deve ser um inteiro
                  positivo ou nulo")
        try:
            valor_cooldown = float(validar_duracao(cooldown, "cooldown de evento"))
        except ValueError as exc:
            raise ValueError("cooldown de evento deve ser um número finito maior ou
              igual a zero") from exc
        if valor_cooldown > 0 and self._relogio is None and self._fonte_instante is
          None:
            raise ValueError("cooldown de evento exige uma fonte de tempo")
        self._ordem_inscricao += 1
        inscricao = _InscricaoEvento(
            funcao=funcao,
            prioridade=valor_prioridade,
            ordem=self._ordem_inscricao,
```

```
            receber_evento=bool(receber_evento),
            max_execucoes=max_execucoes,
            cooldown=valor_cooldown,
        )
        self._ouvintes.setdefault(str(nome), []).append(inscricao)
        return funcao

    def remover(self, nome: str, funcao: Callable[..., Any]) -> bool:
        ouvintes = self._ouvintes.get(str(nome))
        if not ouvintes:
            return False
        for indice, inscricao in enumerate(ouvintes):
            if inscricao.funcao is funcao or inscricao.funcao == funcao:
                del ouvintes[indice]
                if not ouvintes:
                    self._ouvintes.pop(str(nome), None)
                return True
        return False

    def _novo_evento(
        self,
        nome: str,
        args: tuple[Any, ...],
        kwargs: Mapping[str, Any],
        origem: str | None,
        *,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
    ) -> Evento:
        self._sequencia += 1
        instante = self._instante_atual()
        return Evento(
            str(nome),
            tuple(args),
            dict(kwargs),
            None if origem is None else str(origem),
            self._sequencia,
            instante=instante,
            cancelavel=bool(cancelavel),
            metadados={} if metadados is None else dict(metadados),
            causa_sequencia=(self._pilha_processamento[-1].sequencia if
              self._pilha_processamento else None),
        )

    def _snapshot_ouvintes(self, nome: str) -> tuple[_InscricaoEvento, ...]:
        ouvintes = tuple(self._ouvintes.get(nome, ()))
        return tuple(sorted(ouvintes, key=lambda item: (-item.prioridade, item.ordem)))

    def _emitir_local(self, evento: Evento) -> int:
        ouvintes = self._snapshot_ouvintes(evento.nome)
        for inscricao in ouvintes:
            if inscricao.max_execucoes is not None and inscricao.execucoes >=
              inscricao.max_execucoes:
                continue
            if inscricao.cooldown > 0 and inscricao.ultimo_disparo is not None:
                if evento.instante is None:
                    raise ValueError("cooldown de evento exige instante lógico")
                decorrido = float(evento.instante) - inscricao.ultimo_disparo
                if decorrido < inscricao.cooldown and not math.isclose(
                    decorrido, inscricao.cooldown, rel_tol=1e-12, abs_tol=1e-15
                ):
```

```
                continue
            if inscricao.receber_evento:
                inscricao.funcao(evento)
            else:
                inscricao.funcao(*evento.args, **dict(evento.kwargs))
            inscricao.execucoes += 1
            if evento.instante is not None:
                inscricao.ultimo_disparo = float(evento.instante)
        return len(ouvintes)

    def emitir(self, nome: str, *args, **kwargs) -> int:
        evento = self._novo_evento(nome, args, kwargs, None)
        return self._emitir_local(evento)

    def emitir_evento(
        self,
        nome: str,
        *args,
        origem: str | None = None,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
        **kwargs,
    ) -> Evento:
        evento = self._novo_evento(
            nome,
            args,
            kwargs,
            origem,
            cancelavel=cancelavel,
            metadados=metadados,
        )
        self._emitir_local(evento)
        return evento

    def enfileirar(self, nome: str, *args, origem: str | None = None, **kwargs) ->
      Evento:
        evento = self._novo_evento(nome, args, kwargs, origem)
        self._fila.append(evento)
        return evento

    def enfileirar_evento(
        self,
        nome: str,
        *args,
        origem: str | None = None,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
        **kwargs,
    ) -> Evento:
        evento = self._novo_evento(
            nome,
            args,
            kwargs,
            origem,
            cancelavel=cancelavel,
            metadados=metadados,
        )
        self._fila.append(evento)
        return evento

    def processar(self, limite: int | None = None) -> tuple[Evento, ...]:
```

```
        if limite is not None and (isinstance(limite, bool) or not isinstance(limite,
          int) or limite < 0):
            raise ValueError("limite de eventos deve ser inteiro não negativo ou nulo")
        processados: list[Evento] = []
        maximo = len(self._fila) if limite is None else min(limite, len(self._fila))
        for _ in range(maximo):
            evento = self._fila.popleft()
            self._pilha_processamento.append(evento)
            try:
                self._emitir_local(evento)
            finally:
                self._pilha_processamento.pop()
            processados.append(evento)
        return tuple(processados)

    def _emitir_propagado_objeto(self, evento: Evento) -> int:
        total = self._emitir_local(evento)
        if self._pai is not None and not evento.consumido:
            total += self._pai._emitir_propagado_objeto(evento)
        return total

    def emitir_propagado(self, nome: str, *args, origem: str | None = None, **kwargs) ->
       int:
        evento = self._novo_evento(nome, args, kwargs, origem)
        return self._emitir_propagado_objeto(evento)

    def emitir_evento_propagado(
        self,
        nome: str,
        *args,
        origem: str | None = None,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
        **kwargs,
    ) -> Evento:
        evento = self._novo_evento(
            nome,
            args,
            kwargs,
            origem,
            cancelavel=cancelavel,
            metadados=metadados,
        )
        self._emitir_propagado_objeto(evento)
        return evento


__all__ = [
    "CONTRATO", "validar_duracao", "normalizar_dt", "FonteTempo",
    "RelogioMonotonico", "RelogioSimulado", "OperacaoCancelada",
    "TokenCancelamento", "Evento", "Eventos",
]
```

Biblioteca `coral.inspecao`

`coral.inspecao.AmostraInspecao`

Snapshot congelado associado a uma posição temporal e sequência.

Assinatura

Assinatura para consulta

```
AmostraInspecao(sequencia: 'int', snapshot: 'SnapshotInspecao', instante: 'float | None'
   = None, diferenca_anterior: 'DiferencaInspecao | None' = None, contrato: 'str' =
  'coral.inspecao.amostra/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
sequencia	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
snapshot	SnapshotInspecao	POSITIONAL_OR_KEYWORD	sim	sem padrão
instante	float | None	POSITIONAL_OR_KEYWORD	não	None
diferenca_anterior	DiferencaInspecao | None	POSITIONAL_OR_KEYWORD	não	None
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.amostra/1'

Retorno declarado

None

Atributo `sequencia`: `int`.

Atributo `snapshot`: `SnapshotInspecao`.

Atributo `instante`: `float | None`.

Atributo `diferenca_anterior`: `DiferencaInspecao | None`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.inspecao.CONTRATO_AMOSTRA_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.amostra/1'`

`coral.inspecao.CONTRATO_COMANDO_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.comando/1'`

`coral.inspecao.CONTRATO_DIAGNOSTICO_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.diagnostico/1'`

`coral.inspecao.CONTRATO_EVENTO_OBSERVABILIDADE`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.evento/1'`

`coral.inspecao.CONTRATO_EXPLICACAO_REGRA`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.explicacao_regra/1'`

`coral.inspecao.CONTRATO_HISTORICO_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.historico/1'`

`coral.inspecao.CONTRATO_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao/1'`

`coral.inspecao.CONTRATO_RELATORIO_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.relatorio/1'`

`coral.inspecao.CONTRATO_RESPOSTA_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.resposta/1'`

`coral.inspecao.CONTRATO_SESSAO_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.sessao/1'`

`coral.inspecao.CONTRATO_VISAO_INSPECAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.inspecao.visao/1'`

`coral.inspecao.DiagnosticoInspecao`

Diagnóstico leve do contrato de inspeção de um objeto.

Assinatura

Assinatura para consulta

```
DiagnosticoInspecao(ok: 'bool', tipo: 'str', nome: 'str | None', assinatura: 'str |
  None', visoes: 'tuple[str, ...]' = (), avisos: 'tuple[str, ...]' = (), erro: 'str |
  None' = None, contrato: 'str' = 'coral.inspecao.diagnostico/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
ok	bool	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
assinatura	str | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
visoes	tuple[str, ...]	POSITIONAL_OR_KEYWORD	não	()

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
avisos	tuple[str, ...]	POSITIONAL_OR_KEYWORD	não	()
erro	str | None	POSITIONAL_OR_KEYWORD	não	None
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.diagnostico/1'

Retorno declarado

None

Atributo `ok`: `bool`.

Atributo `tipo`: `str`.

Atributo `nome`: `str | None`.

Atributo `assinatura`: `str | None`.

Atributo `visoes`: `tuple[str, ...]`.

Atributo `avisos`: `tuple[str, ...]`.

Atributo `erro`: `str | None`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.inspecao.DiferencaInspecao`

DiferencaInspecao(antes: 'str', depois: 'str', mudancas: 'tuple[MudancaInspecao, ...]' = (), contrato: 'str' = 'coral.inspecao.diferenca/1')

Assinatura

Assinatura para consulta

```
DiferencaInspecao(antes: 'str', depois: 'str', mudancas: 'tuple[MudancaInspecao, ...]' =
    (), contrato: 'str' = 'coral.inspecao.diferenca/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
antes	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
depois	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
mudancas	tuple[MudancaInspecao, ...]	POSITIONAL_OR_KEYWORD	não	()
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.diferenca/1'

Retorno declarado

None

Atributo `antes`: `str`.

Atributo `depois`: `str`.

Atributo `mudancas`: `tuple[MudancaInspecao, ...]`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_json(self, **kwargs: 'Any') -> 'str'
```

Método público sem descrição própria no runtime.

`coral.inspecao.EventoObservabilidade`

Evento neutro emitido pela infraestrutura de observabilidade.

Assinatura

Assinatura para consulta

```
EventoObservabilidade(sequencia: 'int', tipo: 'str', alvo: 'str | None' = None,
  instante: 'float | None' = None, caminho: 'str | None' = None, valor: 'Any' = None,
  anterior: 'Any' = None, detalhes: 'Mapping[str, Any]' = <factory>, contrato: 'str' =
  'coral.inspecao.evento/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
sequencia	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
alvo	str | None	POSITIONAL_OR_KEYWORD	não	None
instante	float | None	POSITIONAL_OR_KEYWORD	não	None
caminho	str | None	POSITIONAL_OR_KEYWORD	não	None
valor	Any	POSITIONAL_OR_KEYWORD	não	None
anterior	Any	POSITIONAL_OR_KEYWORD	não	None
detalhes	Mapping[str, Any]	POSITIONAL_OR_KEYWORD	não	<factory>
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.evento/1'

Retorno declarado

None

Atributo `sequencia`: `int`.

Atributo `tipo`: `str`.

Atributo `alvo`: `str | None`.

Atributo `instante`: `float | None`.

Atributo `caminho`: `str | None`.

Atributo `valor`: `Any`.

Atributo `anterior`: `Any`.

Atributo `detalhes`: `Mapping[str, Any]`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.inspecao.ExplicacaoRegra`

Explicação somente leitura do último estado observável de uma regra.

Assinatura

Assinatura para consulta

```
ExplicacaoRegra(nome: 'str | None', estado: 'str', mensagem: 'str', dados: 'Mapping[str,
    Any]' = <factory>, contrato: 'str' = 'coral.inspecao.explicacao_regra/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
estado	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
dados	Mapping[str, Any]	POSITIONAL_OR_KEYWORD	não	<factory>
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.explicacao_regra/1'

Retorno declarado

None

Atributo `nome`: `str | None`.

Atributo `estado`: `str`.

Atributo `mensagem`: `str`.

Atributo `dados`: `Mapping[str, Any]`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.inspecao.HistoricoInspecao`

Histórico curto e limitado de snapshots de um mesmo alvo.

Nenhuma amostragem acontece em segundo plano. O chamador decide quando `registrar` é chamado, o que preserva determinismo em testes e simulações.

Assinatura

Assinatura para consulta

```
HistoricoInspecao(*, max_amostras: 'int' = 120, intervalo: 'float' = 0.0,
  somente_mudancas: 'bool' = False, nome_alvo: 'str | None' = None) -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
max_amostras	int	KEYWORD_ONLY	não	120
intervalo	float	KEYWORD_ONLY	não	0.0
somente_mudancas	bool	KEYWORD_ONLY	não	False
nome_alvo	str | None	KEYWORD_ONLY	não	None

Retorno declarado

None

Métodos públicos

Referência para leitura

```
ao_evento(self, callback: 'Callable[[EventoObservabilidade], Any]')
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estatisticas_inspecao(self) -> 'dict[str, int]'
```

Contadores leves para medir a própria observabilidade.

Referência para leitura

```
limpar(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
registrar(self, objeto: 'Any', *, instante: 'float | None' = None, forcar: 'bool' =
  False) -> 'AmostraInspecao | None'
```

Método público sem descrição própria no runtime.

`coral.inspecao.Inspecionavel`

Base class for protocol classes.

Protocol classes are defined as::

class Proto(Protocol): def meth(self) -> int: ...

Such classes are primarily used with static type checkers that recognize structural subtyping (static duck-typing).

For example::

class C: def meth(self) -> int: return 0

def func(x: Proto) -> int: return x.meth()

func(C()) # Passes static type check

See PEP 544 for details. Protocol classes decorated with @typing.runtime_checkable act as simple-minded runtime protocols that check only the presence of given attributes, ignoring their type signatures. Protocol classes can be generic, they are defined as::

class GenProtoT (Protocol): def meth(self) -> T: ...

Assinatura

Assinatura para consulta

```
Inspecionavel(*args, **kwargs)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
args	sem anotação	VAR_POSITIONAL	não	sem padrão
kwargs	sem anotação	VAR_KEYWORD	não	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
estado_inspecao(self) -> 'Mapping[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.inspecao.LIMITE_COLECAO_PADRAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`100`

`coral.inspecao.LIMITE_REPR_PADRAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`500`

`coral.inspecao.MudancaInspecao`

MudancaInspecao(caminho: 'str', tipo: 'str', antes: 'Any' = None, depois: 'Any' = None)

Assinatura

Assinatura para consulta

```
MudancaInspecao(caminho: 'str', tipo: 'str', antes: 'Any' = None, depois: 'Any' = None)
  -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
caminho	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
antes	Any	POSITIONAL_OR_KEYWORD	não	None
depois	Any	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `caminho`: `str`.

Atributo `tipo`: `str`.

Atributo `antes`: `Any`.

Atributo `depois`: `Any`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.inspecao.PROFUNDIDADE_PADRAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`7`

`coral.inspecao.RelatorioInspecao`

Apresentação humana derivada do mesmo snapshot estruturado.

Assinatura

Assinatura para consulta

```
RelatorioInspecao(tipo: 'str', nome: 'str | None', dados: 'Mapping[str, Any]', visao:
  'str | None' = None, assinatura_origem: 'str | None' = None, contrato: 'str' =
  'coral.inspecao.relatorio/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
dados	Mapping[str, Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
visao	str | None	POSITIONAL_OR_KEYWORD	não	None
assinatura_origem	str | None	POSITIONAL_OR_KEYWORD	não	None
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.relatorio/1'

Retorno declarado

None

Atributo `tipo`: `str`.

Atributo `nome`: `str | None`.

Atributo `dados`: `Mapping[str, Any]`.

Atributo `visao`: `str | None`.

Atributo `assinatura_origem`: `str | None`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_markdown(self) -> 'str'
```

Método público sem descrição própria no runtime.

`coral.inspecao.RespostaInspecao`

Resposta estável para consumidores externos da infraestrutura de inspeção.

Assinatura

Assinatura para consulta

```
RespostaInspecao(operacao: 'str', ok: 'bool', resultado: 'Any' = None, erro: 'str |
  None' = None, contrato: 'str' = 'coral.inspecao.resposta/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
operacao	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
ok	bool	POSITIONAL_OR_KEYWORD	sim	sem padrão
resultado	Any	POSITIONAL_OR_KEYWORD	não	None
erro	str | None	POSITIONAL_OR_KEYWORD	não	None
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.resposta/1'

Retorno declarado

None

Atributo `operacao`: `str`.

Atributo `ok`: `bool`.

Atributo `resultado`: `Any`.

Atributo `erro`: `str | None`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_json(self, **kwargs: 'Any') -> 'str'
```

Método público sem descrição própria no runtime.

`coral.inspecao.SessaoInspecao`

Ponte estável entre o runtime e consumidores como DAP ou VS Code.

Assinatura

Assinatura para consulta

```
SessaoInspecao(*, max_eventos: 'int' = 500) -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
max_eventos	int	KEYWORD_ONLY	não	500

Retorno declarado

None

Métodos públicos

Referência para leitura

```
adicionar_alvo(self, nome: 'str', objeto: 'Any', *, max_amostras: 'int' = 120,
  intervalo: 'float' = 0.0, somente_mudancas: 'bool' = False) -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
adicionar_vigia(self, alvo: 'str', vigia: 'VigiaInspecao') -> 'VigiaInspecao'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
amostrar(self, *, instante: 'float | None' = None, forcar: 'bool' = False) -> 'dict[str,
   AmostraInspecao | None]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
consultar(self, alvo: 'str', caminho: 'str', padrao: 'Any' = <ausente>) -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
consultar_eventos(self, *, cursor: 'int' = 0, limite: 'int' = 100, alvo: 'str | None' =
  None, tipos: 'tuple[str, ...] | list[str]' = (), caminho: 'str | None' = None) ->
  'dict[str, Any]'
```

Lê a fila limitada sem esconder perda de eventos por overflow.

Referência para leitura

```
historico(self, alvo: 'str') -> 'HistoricoInspecao'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
limpar_eventos(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_json(self, **kwargs: 'Any') -> 'str'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_alvo(self, nome: 'str') -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
snapshot(self, alvo: 'str') -> 'SnapshotInspecao'
```

Método público sem descrição própria no runtime.

`coral.inspecao.SnapshotInspecao`

SnapshotInspecao(tipo: 'str', nome: 'str | None', dados: 'Mapping[str, Any]' = <factory>, avisos: 'tuple[str, ...]' = (), contrato: 'str' = 'coral.inspecao/1')

Assinatura

Assinatura para consulta

```
SnapshotInspecao(tipo: 'str', nome: 'str | None', dados: 'Mapping[str, Any]' =
  <factory>, avisos: 'tuple[str, ...]' = (), contrato: 'str' = 'coral.inspecao/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
dados	Mapping[str, Any]	POSITIONAL_OR_KEYWORD	não	<factory>
avisos	tuple[str, ...]	POSITIONAL_OR_KEYWORD	não	()
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao/1'

Retorno declarado

None

Atributo `tipo`: `str`.

Atributo `nome`: `str | None`.

Atributo `dados`: `Mapping[str, Any]`.

Atributo `avisos`: `tuple[str, ...]`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
assinatura(self) -> 'str'
```

Hash estável do snapshot normalizado para comparação e cache.

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_json(self, **kwargs: 'Any') -> 'str'
```

Método público sem descrição própria no runtime.

`coral.inspecao.VigiaInspecao`

Watchpoint somente leitura sobre um caminho do contrato de inspeção.

Assinatura

Assinatura para consulta

```
VigiaInspecao(nome: 'str', caminho: 'str', *, operador: 'str' = 'mudou', valor: 'Any' =
  None, ativo: 'bool' = True) -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
caminho	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
operador	str	KEYWORD_ONLY	não	'mudou'
valor	Any	KEYWORD_ONLY	não	None
ativo	bool	KEYWORD_ONLY	não	True

Retorno declarado

None

Métodos públicos

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
observar(self, objeto: 'Any', *, alvo: 'str | None' = None, instante: 'float | None' =
  None) -> 'EventoObservabilidade | None'
```

Método público sem descrição própria no runtime.

`coral.inspecao.VisaoInspecao`

Projeção semântica estável de um snapshot de inspeção.

A visão não possui autoridade sobre o domínio e não altera o objeto observado. Ela existe para que DAP, VS Code, logs e futuros overlays consumam um vocabulário menor que o snapshot completo.

Assinatura

Assinatura para consulta

```
VisaoInspecao(visao: 'str', tipo: 'str', nome: 'str | None', dados: 'Mapping[str, Any]'
  = <factory>, assinatura_origem: 'str | None' = None, avisos: 'tuple[str, ...]' = (),
  contrato: 'str' = 'coral.inspecao.visao/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
visao	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
dados	Mapping[str, Any]	POSITIONAL_OR_KEYWORD	não	<factory>
assinatura_origem	str | None	POSITIONAL_OR_KEYWORD	não	None
avisos	tuple[str, ...]	POSITIONAL_OR_KEYWORD	não	()
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.inspecao.visao/1'

Retorno declarado

None

Atributo `visao`: `str`.

Atributo `tipo`: `str`.

Atributo `nome`: `str | None`.

Atributo `dados`: `Mapping[str, Any]`.

Atributo `assinatura_origem`: `str | None`.

Atributo `avisos`: `tuple[str, ...]`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_json(self, **kwargs: 'Any') -> 'str'
```

Método público sem descrição própria no runtime.

`coral.inspecao.comparar_inspecoes`

Compara snapshots normalizados de forma determinística e somente leitura.

Assinatura

Assinatura para consulta

```
comparar_inspecoes(antes: 'Any', depois: 'Any', *, incluir: 'tuple[str, ...] |
  list[str]' = (), ignorar: 'tuple[str, ...] | list[str]' = ()) -> 'DiferencaInspecao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
antes	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
depois	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
incluir	tuple[str, ...] | list[str]	KEYWORD_ONLY	não	()
ignorar	tuple[str, ...] | list[str]	KEYWORD_ONLY	não	()

Retorno declarado

DiferencaInspecao

`coral.inspecao.consultar_inspecao`

Consulta um caminho dentro de `dados` sem expor internals do domínio.

Aceita notação por ponto, índices entre colchetes e `* como curinga. Exemplo: entidades[*].nome`.

Assinatura

Assinatura para consulta

```
consultar_inspecao(objeto: 'Any', caminho: 'str', padrao: 'Any' = <ausente>) -> 'Any'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
caminho	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
padrao	Any	POSITIONAL_OR_KEYWORD	não	<ausente>

Retorno declarado

Any

`coral.inspecao.diagnosticar_inspecao`

Confirma snapshot, JSON seguro e visões disponíveis sem alterar o alvo.

Assinatura

Assinatura para consulta

```
diagnosticar_inspecao(objeto: 'Any') -> 'DiagnosticoInspecao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

DiagnosticoInspecao

`coral.inspecao.executar_comando_inspecao`

Executa um comando neutro para DAP, VS Code, CLI ou outras ferramentas.

A função não conhece protocolos de transporte. O consumidor envia somente operação, alvo e argumentos e recebe uma resposta JSON segura.

Assinatura

Assinatura para consulta

```
executar_comando_inspecao(sessao: "'SessaoInspecao'", operacao: 'str', *, alvo: 'str |
  None' = None, argumentos: 'Mapping[str, Any] | None' = None, capturar_erros: 'bool' =
  True) -> 'RespostaInspecao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
sessao	'SessaoInspecao'	POSITIONAL_OR_KEYWORD	sim	sem padrão
operacao	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
alvo	str | None	KEYWORD_ONLY	não	None
argumentos	Mapping[str, Any] | None	KEYWORD_ONLY	não	None
capturar_erros	bool	KEYWORD_ONLY	não	True

Retorno declarado

RespostaInspecao

`coral.inspecao.explicar_regra`

Explica o último estado de uma Regra usando apenas o contrato de inspeção.

Assinatura

Assinatura para consulta

```
explicar_regra(objeto: 'Any') -> 'ExplicacaoRegra'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

ExplicacaoRegra

`coral.inspecao.inspecionar`

Cria um snapshot somente leitura sem exigir um domínio específico.

Assinatura

Assinatura para consulta

```
inspecionar(objeto: 'Any', *, nome: 'str | None' = None, tipo: 'str | None' = None) ->
  'SnapshotInspecao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	KEYWORD_ONLY	não	None
tipo	str | None	KEYWORD_ONLY	não	None

Retorno declarado

SnapshotInspecao

`coral.inspecao.inspecionar_dict`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
inspecionar_dict(objeto: 'Any', **opcoes: 'Any') -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
opcoes	Any	VAR_KEYWORD	não	sem padrão

Retorno declarado

dict[str, Any]

`coral.inspecao.normalizar_valor`

Converte um valor de inspeção para uma forma JSON segura e limitada.

Assinatura

Assinatura para consulta

```
normalizar_valor(valor: 'Any', *, profundidade: 'int' = 7, limite_colecao: 'int' = 100,
  _vistos: 'set[int] | None' = None) -> 'Any'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
profundidade	int	KEYWORD_ONLY	não	7

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
limite_colecao	int	KEYWORD_ONLY	não	100
_vistos	set[int] | None	KEYWORD_ONLY	não	None

Retorno declarado

Any

`coral.inspecao.relatorio_inspecao`

Gera uma apresentação humana sem criar um segundo modelo de estado.

Assinatura

Assinatura para consulta

```
relatorio_inspecao(objeto: 'Any', *, visao: 'str | None' = None, caminhos: 'list[str] |
  tuple[str, ...]' = ()) -> 'RelatorioInspecao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
visao	str | None	KEYWORD_ONLY	não	None
caminhos	list[str] | tuple[str, ...]	KEYWORD_ONLY	não	()

Retorno declarado

RelatorioInspecao

`coral.inspecao.selecionar_inspecao`

Retorna somente os caminhos solicitados, preservando a ordem pedida.

Assinatura

Assinatura para consulta

```
selecionar_inspecao(objeto: 'Any', caminhos: 'list[str] | tuple[str, ...]') ->
  'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
caminhos	list[str] | tuple[str, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

dict[str, Any]

`coral.inspecao.visoes_inspecao`

Lista as visões semânticas disponíveis sem importar módulos de domínio.

Assinatura

Assinatura para consulta

```
visoes_inspecao(objeto: 'Any') -> 'tuple[str, ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

tuple[str, ...]

`coral.inspecao.visualizar_inspecao`

Produz uma visão de alto nível sobre um snapshot já normalizado.

A função trabalha somente com o contrato `` `coral.inspecao/1` ``. Nenhum módulo de domínio é importado aqui, preservando a direção das dependências.

Assinatura

Assinatura para consulta

```
visualizar_inspecao(objeto: 'Any', visao: 'str') -> 'VisaoInspecao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
visao	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

VisaoInspecao

Biblioteca `coral.jogos`

`coral.jogos.AMARELO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`Cor(vermelho=255, verde=215, azul=0, alfa=255)`

`coral.jogos.AZUL`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`Cor(vermelho=38, verde=139, azul=210, alfa=255)`

`coral.jogos.AcaoApresentacao`

AcaoApresentacao(acao: 'Callable[[], Any]', nome: 'str | None' = None)

Assinatura

Assinatura para consulta

```
AcaoApresentacao(acao: 'Callable[[], Any]', nome: 'str | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
acao	Callable[[], Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `acao`: `Callable[[], Any]`.

Atributo `nome`: `str | None`.

Métodos públicos

Referência para leitura

```
avancar(self, dt: 'float') -> 'float'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
reiniciar(self) -> 'None'
```

Método público sem descrição própria no runtime.

`coral.jogos.AdaptadorEventosJogo`

Publica acontecimentos de Jogos no barramento reativo do próprio Jogo.

O adaptador é deliberadamente opcional e depende apenas da API pública de `Jogo/Sprite`. O núcleo reativo não importa Jogos e este módulo não importa Pygame.

Assinatura

Assinatura para consulta

```
AdaptadorEventosJogo(jogo: 'Jogo', *, teclas: 'Iterable[str]' = (), botoes_mouse:
  'Iterable[str]' = (), observar_mouse: 'bool' = False)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
jogo	Jogo	POSITIONAL_OR_KEYWORD	sim	sem padrão
teclas	Iterable[str]	KEYWORD_ONLY	não	()
botoes_mouse	Iterable[str]	KEYWORD_ONLY	não	()
observar_mouse	bool	KEYWORD_ONLY	não	False

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
desligar(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
ligar(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
observar_colisao(self, primeiro: 'Sprite', segundo: 'Sprite', *, nome: 'str | None' =
  None) -> '_ColisaoObservada'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
publicar_transicao_visual(self, sprite: 'Sprite', transicao: 'str', *, estado: 'str |
  None' = None)
```

Método público sem descrição própria no runtime.

`coral.jogos.Animacao`

Animacao(quadros: 'tuple[str | Path | QuadroSprite, ...]', fps: 'float' = 10.0, repetir: 'bool' = True, _tempo: 'float' = 0.0, _indice: 'int' = 0, finalizada: 'bool' = False, duracoes: 'tuple[float, ...] | None' = None, velocidade: 'float' = 1.0, modo: 'str' = 'normal', indice_inicial: 'int | None' = None, marcadores: 'dict[str, int]' = <factory>)

Assinatura

Assinatura para consulta

```
Animacao(quadros: 'tuple[str | Path | QuadroSprite, ...]', fps: 'float' = 10.0, repetir:
   'bool' = True, _tempo: 'float' = 0.0, _indice: 'int' = 0, finalizada: 'bool' = False,
  duracoes: 'tuple[float, ...] | None' = None, velocidade: 'float' = 1.0, modo: 'str' =
  'normal', indice_inicial: 'int | None' = None, marcadores: 'dict[str, int]' = <factory>)
   -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
quadros	tuple[str | Path | QuadroSprite, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
fps	float	POSITIONAL_OR_KEYWORD	não	10.0
repetir	bool	POSITIONAL_OR_KEYWORD	não	True
_tempo	float	POSITIONAL_OR_KEYWORD	não	0.0
_indice	int	POSITIONAL_OR_KEYWORD	não	0
finalizada	bool	POSITIONAL_OR_KEYWORD	não	False
duracoes	tuple[float, ...] | None	POSITIONAL_OR_KEYWORD	não	None
velocidade	float	POSITIONAL_OR_KEYWORD	não	1.0
modo	str	POSITIONAL_OR_KEYWORD	não	'normal'
indice_inicial	int | None	POSITIONAL_OR_KEYWORD	não	None
marcadores	dict[str, int]	POSITIONAL_OR_KEYWORD	não	<factory>

Retorno declarado

None

Atributo `quadros`: `tuple[str | Path | QuadroSprite, ...]`.

Atributo `fps`: `float`.

Atributo `repetir`: `bool`.

Atributo `finalizada`: `bool`.

Atributo `duracoes`: `tuple[float, ...] | None`.

Atributo `velocidade`: `float`.

Atributo `modo`: `str`.

Atributo `indice_inicial`: `int | None`.

Atributo `marcadores`: `dict[str, int]`.

Métodos públicos

Referência para leitura

```
ao_completar_ciclo(self, callback: 'Callable') -> 'Callable'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
ao_finalizar(self, callback: 'Callable') -> 'Callable'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
ao_marcador(self, nome: 'str', callback: 'Callable') -> 'Callable'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
atualizar(self, dt: 'float')
```

Método público sem descrição própria no runtime.

Referência para leitura

```
buscar_quadro(self, indice: 'int') -> "'Animacao'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
buscar_tempo(self, segundos: 'float') -> "'Animacao'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
duplicar(self) -> "'Animacao'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
marcar(self, nome: 'str', quadro: 'int') -> "'Animacao'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
reiniciar(self)
```

Método público sem descrição própria no runtime.

`coral.jogos.AnimacaoTransformacao`

AnimacaoTransformacao(canais: 'tuple[CanalAnimado, ...]', tempo: 'float' = 0.0, estado: 'str' = 'ativo')

Assinatura

Assinatura para consulta

```
AnimacaoTransformacao(canais: 'tuple[CanalAnimado, ...]', tempo: 'float' = 0.0, estado:
  'str' = 'ativo') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
canais	tuple[CanalAnimado, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
tempo	float	POSITIONAL_OR_KEYWORD	não	0.0
estado	str	POSITIONAL_OR_KEYWORD	não	'ativo'

Retorno declarado

None

Atributo `canais`: `tuple[CanalAnimado, ...]`.

Atributo `tempo`: `float`.

Atributo `estado`: `str`.

Métodos públicos

Referência para leitura

```
avancar(self, dt: 'float') -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
encerrar(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
pausar(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
reiniciar(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
retomar(self) -> 'None'
```

Método público sem descrição própria no runtime.

`coral.jogos.AnimacoesDirecionais`

Conjunto nomeado de animações para direções de movimento.

A classe não decide como uma entidade se move. Ela apenas seleciona a animação correspondente à direção informada e a liga ao Sprite visual.

Assinatura

Assinatura para consulta

```
AnimacoesDirecionais(animacoes: 'dict[str, Animacao]') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
animacoes	dict[str, Animacao]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `animacoes`: `dict[str, Animacao]`.

Métodos públicos

Referência para leitura

```
ligar(self, movimento: 'MovimentoDirecional', sprite: 'Sprite') ->
  'VinculoAnimacaoMovimento'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para(self, direcao: 'str') -> 'Animacao'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
usar(self, sprite: 'Sprite', direcao: 'str', *, reiniciar: 'bool' = True) -> 'Animacao'
```

Método público sem descrição própria no runtime.

`coral.jogos.Assets`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
Assets(raiz: 'str | Path' = 'assets')
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
raiz	str | Path	POSITIONAL_OR_KEYWORD	não	'assets'

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
imagem(self, nome: 'str | Path') -> 'Path'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
som(self, nome: 'str | Path') -> 'Path'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
spritesheet(self, nome: 'str | Path', *, largura_imagem: 'int', altura_imagem: 'int',
  largura_quadro: 'int', altura_quadro: 'int', margem: 'int' = 0, espacamento: 'int' = 0)
  -> 'Spritesheet'
```

Método público sem descrição própria no runtime.

`coral.jogos.BRANCO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`Cor(vermelho=255, verde=255, azul=255, alfa=255)`

`BackendNulo(...)` · módulo `coral.jogos`

O que faz

Simula entrada e desenho sem criar uma janela.

Construtor

Assinatura para consulta

```
BackendNulo(*, dt_fixo: 'float | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
dt_fixo	float | None	não	None	Intervalo de tempo fixo do backend, quando fornecido.

Atributos importantes

Atributo	Significado
`quadros`	Quantidade de quadros apresentados na simulação.

Métodos principais

Assinatura para consulta

```
definir_teclas(self, *teclas: 'str') -> 'None'
```

Referência para leitura

```
definir_mouse(self, *, posicao: 'tuple[int, int]' = (0, 0), pressionados=(),
  clicados=(), soltos=(), roda: 'tuple[int, int]' = (0, 0)) -> 'None'
```

Referência para leitura

```
processar_eventos(self) -> 'bool'
```

Retorno

Instância de BackendNulo.

Exemplo

Código Coral

```
de coral.jogos importe BackendNulo
defina backend como BackendNulo()
mostre backend.quadros
```

Resultado

Saída esperada

```
0
```

Atenção

A criação do objeto de dados não inicia a execução de um jogo. A simulação sem janela não comprova comportamento de uma janela ou dispositivo físico.

`coral.jogos.CONTRATO_OVERLAY_DEBUG`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.jogos.debug.overlay/1'`

`coral.jogos.CamadaApresentacao`

Camada 2D ordenada que pode pertencer ao mundo ou à tela.

A lista de elementos é estável: elementos com a mesma camada mantêm a ordem de inserção. `` `espaco='tela'` `` ignora a câmera e é apropriado para HUD. Paralaxe só afeta camadas de mundo.

Assinatura

Assinatura para consulta

```
CamadaApresentacao(nome: 'str', ordem: 'int' = 0, visivel: 'bool' = True, opacidade:
  'float' = 1.0, espaco: 'str' = 'mundo', paralaxe_x: 'float' = 1.0, paralaxe_y: 'float' =
   1.0, elementos: 'list[Any]' = <factory>) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
ordem	int	POSITIONAL_OR_KEYWORD	não	0
visivel	bool	POSITIONAL_OR_KEYWORD	não	True
opacidade	float	POSITIONAL_OR_KEYWORD	não	1.0
espaco	str	POSITIONAL_OR_KEYWORD	não	'mundo'
paralaxe_x	float	POSITIONAL_OR_KEYWORD	não	1.0
paralaxe_y	float	POSITIONAL_OR_KEYWORD	não	1.0
elementos	list[Any]	POSITIONAL_OR_KEYWORD	não	<factory>

Retorno declarado

None

Atributo `nome`: `str`.

Atributo `ordem`: `int`.

Atributo `visivel`: `bool`.

Atributo `opacidade`: `float`.

Atributo `espaco`: `str`.

Atributo `paralaxe_x`: `float`.

Atributo `paralaxe_y`: `float`.

Atributo `elementos`: `list[Any]`.

Métodos públicos

Referência para leitura

```
adicionar(self, elemento: 'Any') -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
definir_opacidade(self, opacidade: 'float') -> "'CamadaApresentacao'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
definir_paralaxe(self, x: 'float' = 1.0, y: 'float | None' = None) ->
  "'CamadaApresentacao'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
definir_visibilidade(self, visivel: 'bool') -> "'CamadaApresentacao'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover(self, elemento: 'Any') -> 'bool'
```

Método público sem descrição própria no runtime.

`Camera(...)` · módulo `coral.jogos`

O que faz

Controla a posição e a transformação da região visualizada.

Construtor

Assinatura para consulta

```
Camera(x: 'float' = 0.0, y: 'float' = 0.0, largura: 'float' = 800.0, altura: 'float' =
  450.0, alvo: 'Any | None' = None, suavidade: 'float' = 1.0, zoom: 'float' = 1.0,
  rotacao: 'float' = 0.0, zoom_minimo: 'float | None' = None, zoom_maximo: 'float | None'
  = None, deslocamento_visual_x: 'float' = 0.0, deslocamento_visual_y: 'float' = 0.0,
  constante_tempo: 'float | None' = None, zona_morta: 'tuple[float, float] | None' = None,
   enquadramento_x: 'float' = 0.0, enquadramento_y: 'float' = 0.0, antecipacao: 'float' =
  0.0, limites: 'Retangulo | None' = None, travar_x: 'bool' = False, travar_y: 'bool' =
  False) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	float	não	0.0	Coordenadas horizontais da série, ou posição horizontal do objeto.
y	float	não	0.0	Coordenadas verticais da série, ou posição vertical do objeto.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
largura	float	não	800.0	Largura em unidades da apresentação.
altura	float	não	450.0	Altura em unidades da apresentação.
alvo	Any | None	não	None	Objeto que a câmera acompanha.
suavidade	float	não	1.0	Fator de suavização usado no acompanhamento.
zoom	float	não	1.0	Fator de ampliação da câmera.
rotacao	float	não	0.0	Ângulo de rotação da apresentação.
zoom_minimo	float | None	não	None	Limite inferior permitido para zoom.
zoom_maximo	float | None	não	None	Limite superior permitido para zoom.
deslocamento_visual_x	float	não	0.0	Deslocamento horizontal adicional da apresentação.
deslocamento_visual_y	float	não	0.0	Deslocamento vertical adicional da apresentação.
constante_tempo	float | None	não	None	Constante temporal da suavização quando configurada.
zona_morta	tuple[float, float] | None	não	None	Região em que o alvo pode se mover sem deslocar a câmera.
enquadramento_x	float	não	0.0	Deslocamento horizontal desejado no enquadramento.
enquadramento_y	float	não	0.0	Deslocamento vertical desejado no enquadramento.
antecipacao	float	não	0.0	Fator de antecipação do movimento do alvo.
limites	Retangulo | None	não	None	Retângulo que limita o deslocamento da câmera.
travar_x	bool	não	False	Impede acompanhamento horizontal.
travar_y	bool	não	False	Impede acompanhamento vertical.

Atributos importantes

Atributo	Significado
`x`	Posição horizontal da câmera.
y	Posição vertical da câmera.
`largura`	Largura da área visualizada.

Atributo	Significado
`altura`	Altura da área visualizada.
`zoom`	Fator de ampliação.

Métodos principais

Assinatura para consulta

```
seguir(self, sprite: 'Sprite', suavidade: 'float' = 1.0, *, constante_tempo: 'float |
  None' = None, zona_morta: 'tuple[float, float] | None' = None, enquadramento:
  'tuple[float, float] | None' = None, antecipacao: 'float | None' = None, travar_x: 'bool
   | None' = None, travar_y: 'bool | None' = None)
```

Referência para leitura

```
atualizar(self, dt: 'float | None' = None)
```

Referência para leitura

```
mundo_para_tela(self, x: 'float', y: 'float') -> 'tuple[float, float]'
```

Referência para leitura

```
tela_para_mundo(self, x: 'float', y: 'float') -> 'tuple[float, float]'
```

Retorno

Instância de Camera.

Exemplo

Código Coral

```
de coral.jogos importe Camera
defina camera como Camera(0, 0, 64, 64)
mostre camera.largura
```

Resultado

Saída esperada

```
64.0
```

Atenção

A criação do objeto de dados não inicia a execução de um jogo. A simulação sem janela não comprova comportamento de uma janela ou dispositivo físico.

`coral.jogos.CanalAnimado`

CanalAnimado(canal: 'str', base: 'float', amplitude: 'float', frequencia: 'float', fase: 'float' = 0.0, modo: 'str' = 'absoluto', onda: 'str | Callable[[float], float]' = 'seno')

Assinatura

Assinatura para consulta

```
CanalAnimado(canal: 'str', base: 'float', amplitude: 'float', frequencia: 'float', fase:
   'float' = 0.0, modo: 'str' = 'absoluto', onda: 'str | Callable[[float], float]' =
  'seno') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
canal	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
base	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
amplitude	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
frequencia	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
fase	float	POSITIONAL_OR_KEYWORD	não	0.0
modo	str	POSITIONAL_OR_KEYWORD	não	'absoluto'
onda	str | Callable[[float], float]	POSITIONAL_OR_KEYWORD	não	'seno'

Retorno declarado

None

Atributo `canal`: `str`.

Atributo `base`: `float`.

Atributo `amplitude`: `float`.

Atributo `frequencia`: `float`.

Atributo `fase`: `float`.

Atributo `modo`: `str`.

Atributo `onda`: `str | Callable[[float], float]`.

Métodos públicos

Referência para leitura

```
valor(self, tempo: 'float', *, referencia: 'float' = 0.0) -> 'float'
```

Método público sem descrição própria no runtime.

`coral.jogos.Cena`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
Cena(nome: 'str')
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
adicionar(self, sprite: 'Sprite', *, camada: 'str | CamadaApresentacao | None' = None)
  -> 'Sprite'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
adicionar_em_camada(self, elemento: 'Any', camada: 'str | CamadaApresentacao') -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
atualizar(self, jogo, dt: 'float')
```

Método público sem descrição própria no runtime.

Referência para leitura

```
camada(self, nome: 'str') -> 'CamadaApresentacao'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
criar_camada(self, nome: 'str', *, ordem: 'int' = 0, visivel: 'bool' = True, opacidade:
  'float' = 1.0, espaco: 'str' = 'mundo', paralaxe: 'float | tuple[float, float]' = 1.0)
  -> 'CamadaApresentacao'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
desenhar(self, jogo)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
entrar(self, jogo)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
quando_atualizar(self, f)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
quando_desenhar(self, f)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
quando_entrar(self, f)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
quando_sair(self, f)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover(self, sprite: 'Sprite') -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_de_camada(self, elemento: 'Any', camada: 'str | CamadaApresentacao | None' =
  None) -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
representar(self, entidade: 'Any', sprite: 'Any', *, atributo_x: 'str' = 'x',
  atributo_y: 'str' = 'y') -> 'RepresentacaoEntidade'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
sair(self, jogo)
```

Método público sem descrição própria no runtime.

`coral.jogos.ClipeAnimacao`

Reprodutor headless de uma ou mais faixas de quadros chave.

Assinatura

Assinatura para consulta

```
ClipeAnimacao(faixas: 'tuple[FaixaAnimacao, ...]', velocidade: 'float' = 1.0, modo:
  'str' = 'normal', repetir: 'bool' = False, tempo: 'float' = 0.0, estado: 'str' =
  'ativo') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
faixas	tuple[FaixaAnimacao, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
velocidade	float	POSITIONAL_OR_KEYWORD	não	1.0
modo	str	POSITIONAL_OR_KEYWORD	não	'normal'
repetir	bool	POSITIONAL_OR_KEYWORD	não	False
tempo	float	POSITIONAL_OR_KEYWORD	não	0.0
estado	str	POSITIONAL_OR_KEYWORD	não	'ativo'

Retorno declarado

None

Atributo `faixas`: `tuple[FaixaAnimacao, ...]`.

Atributo `velocidade`: `float`.

Atributo `modo`: `str`.

Atributo `repetir`: `bool`.

Atributo `tempo`: `float`.

Atributo `estado`: `str`.

Métodos públicos

Referência para leitura

```
avancar(self, dt: 'float') -> 'dict[str, float]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
buscar(self, instante: 'float') -> "'ClipeAnimacao'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
pausar(self) -> "'ClipeAnimacao'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
reiniciar(self) -> "'ClipeAnimacao'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
retomar(self) -> "'ClipeAnimacao'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
valor(self, faixa: 'str') -> 'float'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
valores(self) -> 'dict[str, float]'
```

Método público sem descrição própria no runtime.

`coral.jogos.ComandoComposicao`

ComandoComposicao(operacao: 'str', valor: 'float | None' = None)

Assinatura

Assinatura para consulta

```
ComandoComposicao(operacao: 'str', valor: 'float | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
operacao	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
valor	float | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `operacao`: `str`.

Atributo `valor`: `float | None`.

`coral.jogos.ComposicaoTemporal`

ComposicaoTemporal(raiz: 'NoApresentacao', repeticoes: 'int | None' = 1, nome: 'str | None' = None, estado: 'str' = 'ativo')

Assinatura

Assinatura para consulta

```
ComposicaoTemporal(raiz: 'NoApresentacao', repeticoes: 'int | None' = 1, nome: 'str |
  None' = None, estado: 'str' = 'ativo') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
raiz	NoApresentacao	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
repeticoes	int | None	POSITIONAL_OR_KEYWORD	não	1
nome	str | None	POSITIONAL_OR_KEYWORD	não	None
estado	str	POSITIONAL_OR_KEYWORD	não	'ativo'

Retorno declarado

None

Atributo `raiz`: `NoApresentacao`.

Atributo `repeticoes`: `int | None`.

Atributo `nome`: `str | None`.

Atributo `estado`: `str`.

Métodos públicos

Referência para leitura

```
avancar(self, dt: 'float') -> "'ComposicaoTemporal'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
cancelar(self) -> "'ComposicaoTemporal'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
pausar(self) -> "'ComposicaoTemporal'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
reiniciar(self) -> "'ComposicaoTemporal'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
retomar(self) -> "'ComposicaoTemporal'"
```

Método público sem descrição própria no runtime.

`coral.jogos.CondicaoTransicaoAnimacao`

CondicaoTransicaoAnimacao(condicao: 'Callable[[Sprite], bool]', transicao: 'TransicaoEstadoAnimacao', ordem: 'int' = 0)

Assinatura

Assinatura para consulta

```
CondicaoTransicaoAnimacao(condicao: 'Callable[[Sprite], bool]', transicao:
  'TransicaoEstadoAnimacao', ordem: 'int' = 0) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
condicao	Callable[[Sprite], bool]	POSITIONAL_OR_KEYWORD	sim	sem padrão
transicao	TransicaoEstadoAnimacao	POSITIONAL_OR_KEYWORD	sim	sem padrão
ordem	int	POSITIONAL_OR_KEYWORD	não	0

Retorno declarado

None

Atributo `condicao`: `Callable[[Sprite], bool]`.

Atributo `transicao`: `TransicaoEstadoAnimacao`.

Atributo `ordem`: `int`.

`coral.jogos.ConfiguracaoDebugJogo`

Configuração persistente e serializável do modo de debug de Jogo.

Assinatura

Assinatura para consulta

```
ConfiguracaoDebugJogo(ativo: 'bool' = False, categorias: 'frozenset[str]' =
  frozenset({'camera', 'colisor', 'limites', 'metricas', 'pivo'}), incluir_invisiveis:
  'bool' = False, estilo: 'EstiloOverlayDebug' = <factory>) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
ativo	bool	POSITIONAL_OR_KEYWORD	não	False
categorias	frozenset[str]	POSITIONAL_OR_KEYWORD	não	frozenset({'camera', 'colisor', 'limites', 'metricas', 'pivo'})
incluir_invisiveis	bool	POSITIONAL_OR_KEYWORD	não	False
estilo	EstiloOverlayDebug	POSITIONAL_OR_KEYWORD	não	<factory>

Retorno declarado

None

Atributo `ativo`: `bool`.

Atributo `categorias`: `frozenset[str]`.

Atributo `incluir_invisiveis`: `bool`.

Atributo `estilo`: `EstiloOverlayDebug`.

Métodos públicos

Referência para leitura

```
com_alteracoes(self, *, ativo: 'bool | None' = None, categorias: 'Any' = None,
  incluir_invisiveis: 'bool | None' = None, estilo: 'EstiloOverlayDebug | None' = None) ->
   "'ConfiguracaoDebugJogo'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.jogos.Cor`

Cor(vermelho: 'int', verde: 'int', azul: 'int', alfa: 'int' = 255)

Assinatura

Assinatura para consulta

```
Cor(vermelho: 'int', verde: 'int', azul: 'int', alfa: 'int' = 255) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
vermelho	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
verde	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
azul	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
alfa	int	POSITIONAL_OR_KEYWORD	não	255

Retorno declarado

None

Atributo `vermelho`: `int`.

Atributo `verde`: `int`.

Atributo `azul`: `int`.

Atributo `alfa`: `int`.

Métodos públicos

Referência para leitura

```
rgb(self) -> 'tuple[int, int, int]'
```

Método público sem descrição própria no runtime.

`coral.jogos.DependenciaJogosAusente`

Uma capacidade opcional não pode operar por ausência de dependência.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.jogos.EfeitoCamera`

Efeito visual temporal que não altera o estado lógico da câmera.

Assinatura

Assinatura para consulta

```
EfeitoCamera(tipo: 'str', duracao: 'float', intensidade: 'float' = 1.0, x: 'float' =
  0.0, y: 'float' = 0.0, zoom: 'float' = 0.0, rotacao: 'float' = 0.0, semente: 'int' = 0,
  frequencia: 'float' = 24.0, curva: 'str | Callable[[float], float]' = 'linear', tempo:
  'float' = 0.0, estado: 'str' = 'ativo') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
duracao	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
intensidade	float	POSITIONAL_OR_KEYWORD	não	1.0
x	float	POSITIONAL_OR_KEYWORD	não	0.0
y	float	POSITIONAL_OR_KEYWORD	não	0.0
zoom	float	POSITIONAL_OR_KEYWORD	não	0.0

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
rotacao	float	POSITIONAL_OR_KEYWORD	não	0.0
semente	int	POSITIONAL_OR_KEYWORD	não	0
frequencia	float	POSITIONAL_OR_KEYWORD	não	24.0
curva	str | Callable[[float], float]	POSITIONAL_OR_KEYWORD	não	'linear'
tempo	float	POSITIONAL_OR_KEYWORD	não	0.0
estado	str	POSITIONAL_OR_KEYWORD	não	'ativo'

Retorno declarado

None

Atributo `tipo`: `str`.

Atributo `duracao`: `float`.

Atributo `intensidade`: `float`.

Atributo `x`: `float`.

Atributo `y`: `float`.

Atributo `zoom`: `float`.

Atributo `rotacao`: `float`.

Atributo `semente`: `int`.

Atributo `frequencia`: `float`.

Atributo `curva`: `str | Callable[[float], float]`.

Atributo `tempo`: `float`.

Atributo `estado`: `str`.

Métodos públicos

Referência para leitura

```
amostra(self) -> 'tuple[float, float, float, float]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
avancar(self, dt: 'float') -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
cancelar(self) -> "'EfeitoCamera'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
pausar(self) -> "'EfeitoCamera'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
retomar(self) -> "'EfeitoCamera'"
```

Método público sem descrição própria no runtime.

`coral.jogos.ErroAssetCoral`

Common base class for all non-exit exceptions.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.jogos.ErroJogosCoral`

Common base class for all non-exit exceptions.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.jogos.EsperaApresentacao`

EsperaApresentacao(duracao: 'float', nome: 'str | None' = None, tempo: 'float' = 0.0)

Assinatura

Assinatura para consulta

```
EsperaApresentacao(duracao: 'float', nome: 'str | None' = None, tempo: 'float' = 0.0) ->
    None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
duracao	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None
tempo	float	POSITIONAL_OR_KEYWORD	não	0.0

Retorno declarado

None

Atributo `duracao`: `float`.

Atributo `nome`: `str | None`.

Atributo `tempo`: `float`.

Métodos públicos

Referência para leitura

```
avancar(self, dt: 'float') -> 'float'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
reiniciar(self) -> 'None'
```

Método público sem descrição própria no runtime.

`coral.jogos.EsperaConclusao`

EsperaConclusao(alvo: 'Any', nome: 'str | None' = None)

Assinatura

Assinatura para consulta

```
EsperaConclusao(alvo: 'Any', nome: 'str | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
alvo	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `alvo`: `Any`.

Atributo `nome`: `str | None`.

Métodos públicos

Referência para leitura

```
avancar(self, dt: 'float') -> 'float'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
reiniciar(self) -> 'None'
```

Método público sem descrição própria no runtime.

`coral.jogos.EstadosAnimacao`

Agrupa estados visuais sem assumir autoridade sobre estado lógico.

`` `estado_inicial` `` é opt in para preservar a compatibilidade histórica. Eventos de entrada e saída pertencem à apresentação e não substituem eventos de domínio do jogo.

Assinatura

Assinatura para consulta

```
EstadosAnimacao(estados: 'dict[str, Animacao | AnimacoesDirecionais]', *,
  estado_inicial: 'str | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
estados	dict[str, Animacao | AnimacoesDirecionais]	POSITIONAL_OR_KEYWORD	sim	sem padrão
estado_inicial	str | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
ao_entrar(self, estado: 'str', callback: 'Callable') -> 'Callable'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
ao_sair(self, estado: 'str', callback: 'Callable') -> 'Callable'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
ligar(self, movimento: 'MovimentoDirecional', sprite: 'Sprite', *, estado_inicial: 'str
  | None' = None, direcao_inicial: 'str | None' = None) ->
  'VinculoEstadosAnimacaoMovimento'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para(self, estado: 'str') -> 'Animacao | AnimacoesDirecionais'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
usar(self, sprite: 'Sprite', estado: 'str', *, direcao: 'str | None' = None, reiniciar:
  'bool' = True) -> 'Animacao'
```

Método público sem descrição própria no runtime.

`coral.jogos.EstiloCelula`

EstiloCelula(cor: 'Cor | None' = None, imagem: 'str | QuadroSprite | None' = None, texto: 'str | None' = None, cor_texto: 'Cor' = Cor(vermelho=255, verde=255, azul=255, alfa=255), tamanho_texto: 'int' = 16, contorno: 'int' = 0, opacidade: 'float' = 1.0)

Assinatura

Assinatura para consulta

```
EstiloCelula(cor: 'Cor | None' = None, imagem: 'str | QuadroSprite | None' = None,
  texto: 'str | None' = None, cor_texto: 'Cor' = Cor(vermelho=255, verde=255, azul=255,
  alfa=255), tamanho_texto: 'int' = 16, contorno: 'int' = 0, opacidade: 'float' = 1.0) ->
  None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
cor	Cor | None	POSITIONAL_OR_KEYWORD	não	None
imagem	str | QuadroSprite | None	POSITIONAL_OR_KEYWORD	não	None
texto	str | None	POSITIONAL_OR_KEYWORD	não	None
cor_texto	Cor	POSITIONAL_OR_KEYWORD	não	Cor(vermelho=255, verde=255, azul=255, alfa=255)
tamanho_texto	int	POSITIONAL_OR_KEYWORD	não	16
contorno	int	POSITIONAL_OR_KEYWORD	não	0

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
opacidade	float	POSITIONAL_OR_KEYWORD	não	1.0

Retorno declarado

None

Atributo `cor`: `Cor | None`.

Atributo `imagem`: `str | QuadroSprite | None`.

Atributo `texto`: `str | None`.

Atributo `cor_texto`: `Cor`.

Atributo `tamanho_texto`: `int`.

Atributo `contorno`: `int`.

Atributo `opacidade`: `float`.

`coral.jogos.EstiloOverlayDebug`

Estilo de desenho desacoplado da geração geométrica do overlay.

Assinatura

Assinatura para consulta

```
EstiloOverlayDebug(limites: 'Any' = None, colisor: 'Any' = None, pivo: 'Any' = None,
  camera: 'Any' = None, camera_centro: 'Any' = None, rotulo: 'Any' = None, metricas: 'Any'
   = None, espessura: 'int' = 1, tamanho_pivo: 'int' = 3, tamanho_texto: 'int' = 14,
  opacidade: 'float' = 1.0) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
limites	Any	POSITIONAL_OR_KEYWORD	não	None
colisor	Any	POSITIONAL_OR_KEYWORD	não	None
pivo	Any	POSITIONAL_OR_KEYWORD	não	None
camera	Any	POSITIONAL_OR_KEYWORD	não	None
camera_centro	Any	POSITIONAL_OR_KEYWORD	não	None
rotulo	Any	POSITIONAL_OR_KEYWORD	não	None

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
metricas	Any	POSITIONAL_OR_KEYWORD	não	None
espessura	int	POSITIONAL_OR_KEYWORD	não	1
tamanho_pivo	int	POSITIONAL_OR_KEYWORD	não	3
tamanho_texto	int	POSITIONAL_OR_KEYWORD	não	14
opacidade	float	POSITIONAL_OR_KEYWORD	não	1.0

Retorno declarado

None

Atributo `limites`: `Any`.

Atributo `colisor`: `Any`.

Atributo `pivo`: `Any`.

Atributo `camera`: `Any`.

Atributo `camera_centro`: `Any`.

Atributo `rotulo`: `Any`.

Atributo `metricas`: `Any`.

Atributo `espessura`: `int`.

Atributo `tamanho_pivo`: `int`.

Atributo `tamanho_texto`: `int`.

Atributo `opacidade`: `float`.

Métodos públicos

Referência para leitura

```
cor_para(self, categoria: 'str')
```

Método público sem descrição própria no runtime.

`coral.jogos.FaixaAnimacao`

Faixa temporal imutável de quadros chave para uma propriedade.

Assinatura

Assinatura para consulta

```
FaixaAnimacao(nome: 'str', quadros: 'tuple[QuadroChave, ...]') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
quadros	tuple[QuadroChave, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `nome`: `str`.

Atributo `quadros`: `tuple[QuadroChave, ...]`.

Métodos públicos

Referência para leitura

```
valor_em(self, instante: 'float') -> 'float'
```

Método público sem descrição própria no runtime.

`coral.jogos.FormaTransformada`

FormaTransformada(vertices: 'tuple[tuple[float, float], ...]')

Assinatura

Assinatura para consulta

```
FormaTransformada(vertices: 'tuple[tuple[float, float], ...]') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
vertices	tuple[tuple[float, float], ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `vertices`: `tuple[tuple[float, float], ...]`.

`coral.jogos.GravadorComposicao`

Grava comandos explícitos de uma composição para replay determinístico.

Assinatura

Assinatura para consulta

```
GravadorComposicao(composicao: 'ComposicaoTemporal', comandos: 'list[ComandoComposicao]'
   = <factory>) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
composicao	ComposicaoTemporal	POSITIONAL_OR_KEYWORD	sim	sem padrão
comandos	list[ComandoComposicao]	POSITIONAL_OR_KEYWORD	não	<factory>

Retorno declarado

None

Atributo `composicao`: `ComposicaoTemporal`.

Atributo `comandos`: `list[ComandoComposicao]`.

Métodos públicos

Referência para leitura

```
avancar(self, dt: 'float') -> "'GravadorComposicao'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
cancelar(self) -> "'GravadorComposicao'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
pausar(self) -> "'GravadorComposicao'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
reiniciar(self) -> "'GravadorComposicao'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
reproduzir_em(self, composicao: 'ComposicaoTemporal') -> 'ComposicaoTemporal'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
retomar(self) -> "'GravadorComposicao'"
```

Método público sem descrição própria no runtime.

`coral.jogos.Jogo`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
Jogo(titulo: 'str', largura: 'int' = 800, altura: 'int' = 450, fps: 'int' = 60, *,
  backend: 'BackendJogos', mundo=None, relogio: 'FonteTempo | None' = None, eventos:
  'Eventos | None' = None, tela_cheia: 'bool' = False)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
titulo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
largura	int	POSITIONAL_OR_KEYWORD	não	800
altura	int	POSITIONAL_OR_KEYWORD	não	450
fps	int	POSITIONAL_OR_KEYWORD	não	60
backend	BackendJogos	KEYWORD_ONLY	sim	sem padrão
mundo	sem anotação	KEYWORD_ONLY	não	None
relogio	FonteTempo | None	KEYWORD_ONLY	não	None
eventos	Eventos | None	KEYWORD_ONLY	não	None
tela_cheia	bool	KEYWORD_ONLY	não	False

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
alternar_debug(self, *categorias: 'str') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
alternar_tela_cheia(self) -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
ativar_debug(self, *categorias: 'str', incluir_invisiveis: 'bool | None' = None,
  estilo=None)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
botao_mouse_pressionado(self, botao: 'str' = 'esquerdo') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
cancelar_transicao_cena(self) -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
configurar_debug(self, *, ativo: 'bool | None' = None, categorias=None,
  incluir_invisiveis: 'bool | None' = None, estilo=None)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
debug_ativo(self) -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
definir_tela_cheia(self, ativa: 'bool') -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
desativar_debug(self)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
desenhar_circulo(self, x: 'float', y: 'float', raio: 'float', cor: 'Cor' =
  Cor(vermelho=255, verde=255, azul=255, alfa=255), *, contorno: 'int' = 0, opacidade:
  'float' = 1.0) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
desenhar_debug(self)
```

Desenha o modo persistente atual e devolve o overlay combinado.

Referência para leitura

```
desenhar_imagem(self, caminho: 'str | QuadroSprite', x: 'float', y: 'float', largura:
  'float | None' = None, altura: 'float | None' = None, *, escala: 'float' = 1.0, rotacao:
   'float' = 0.0, espelhar_horizontalmente: 'bool' = False, espelhar_verticalmente: 'bool'
   = False, origem: 'str' = 'topo_esquerdo', pivo: 'tuple[float, float] | None' = None,
  opacidade: 'float' = 1.0) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
desenhar_linha(self, x1: 'float', y1: 'float', x2: 'float', y2: 'float', cor: 'Cor' =
  Cor(vermelho=255, verde=255, azul=255, alfa=255), *, espessura: 'int' = 1, opacidade:
  'float' = 1.0) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
desenhar_overlay_debug(self, objeto=None, **opcoes)
```

Desenha um overlay de debug usando somente a API pública de Jogos.

Referência para leitura

```
desenhar_poligono(self, vertices, cor: 'Cor' = Cor(vermelho=255, verde=255, azul=255,
  alfa=255), *, contorno: 'int' = 1, opacidade: 'float' = 1.0) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
desenhar_ponto(self, x: 'float', y: 'float', cor: 'Cor' = Cor(vermelho=255, verde=255,
  azul=255, alfa=255), *, tamanho: 'int' = 1, opacidade: 'float' = 1.0) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
desenhar_retangulo(self, x: 'float', y: 'float', largura: 'float', altura: 'float', cor:
   'Cor' = Cor(vermelho=255, verde=255, azul=255, alfa=255), *, contorno: 'int' = 0,
  opacidade: 'float' = 1.0) -> 'Retangulo'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
desenhar_texto(self, texto: 'str', x: 'float', y: 'float', tamanho: 'int' = 24, cor:
  'Cor' = Cor(vermelho=255, verde=255, azul=255, alfa=255), *, origem: 'str' =
  'topo_esquerdo', opacidade: 'float' = 1.0) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
em_tela_cheia(self) -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
entrada_bloqueada_por_transicao(self) -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estado_debug(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
executar(self, *, max_quadros: 'int | None' = None) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
historico_inspecao(self)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
limpar(self, cor: 'Cor' = Cor(vermelho=0, verde=0, azul=0, alfa=255)) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
medir_texto(self, texto: 'str', tamanho: 'int' = 24) -> 'tuple[float, float]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
monitorar_inspecao(self, *, max_amostras: 'int' = 120, intervalo: 'float' = 0.1,
  somente_mudancas: 'bool' = True)
```

Ativa histórico curto de observabilidade no próprio laço do jogo.

Referência para leitura

```
mouse_clicado(self, botao: 'str' = 'esquerdo') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
mouse_solto(self, botao: 'str' = 'esquerdo') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
mudar_cena(self, cena) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
mudar_cena_com_transicao(self, cena, *, tipo: 'str' = 'desaparecer', duracao: 'float' =
  0.4, curva: 'str | Callable[[float], float]' = 'suave', bloquear_entrada: 'bool' = True,
   atualizar_anterior: 'bool' = True, atualizar_proxima: 'bool' = True, direcao: 'str' =
  'direita') -> 'TransicaoCena'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
observabilidade_ativa(self) -> 'bool'
```

Indica se há trabalho opcional de inspeção ou overlay por quadro.

Referência para leitura

```
parar(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
parar_monitoramento_inspecao(self)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
posicao_mouse(self) -> 'tuple[int, int]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
posicao_mouse_logica(self, *, limitar: 'bool' = False, modo: 'str' = 'ajustar') ->
  'tuple[float, float]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
quando_atualizar(self, funcao: "Callable[['Jogo', float], Any]")
```

Método público sem descrição própria no runtime.

Referência para leitura

```
quando_desenhar(self, funcao: "Callable[['Jogo'], Any]")
```

Método público sem descrição própria no runtime.

Referência para leitura

```
quando_encerrar(self, funcao: "Callable[['Jogo'], Any]")
```

Método público sem descrição própria no runtime.

Referência para leitura

```
quando_iniciar(self, funcao: "Callable[['Jogo'], Any]")
```

Método público sem descrição própria no runtime.

Referência para leitura

```
redimensionar_janela(self, largura: 'int', altura: 'int') -> 'tuple[int, int]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_mundo(self)
```

Desassocia o Mundo atual e remove somente a propagação criada pelo Jogo.

Referência para leitura

```
roda_mouse(self) -> 'tuple[int, int]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
tamanho_fisico(self) -> 'tuple[int, int]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
tamanho_logico(self) -> 'tuple[int, int]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
tecla_pressionada(self, tecla: 'str') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
tocar_som(self, caminho: 'str', volume: 'float' = 1.0) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
usar_mundo(self, mundo)
```

Associa um Mundo ao jogo e propaga eventos do Mundo para o Jogo.

Referência para leitura

```
viewport(self, *, modo: 'str' = 'ajustar') -> 'Viewport'
```

Método público sem descrição própria no runtime.

`coral.jogos.MetricasJogo`

Métricas operacionais leves e determinísticas do laço principal.

Assinatura

Assinatura para consulta

```
MetricasJogo(quadros: 'int' = 0, delta_logico: 'float' = 0.0, tempo_quadro: 'float' =
  0.0, fps_instantaneo: 'float' = 0.0, fps_medio: 'float' = 0.0) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
quadros	int	POSITIONAL_OR_KEYWORD	não	0
delta_logico	float	POSITIONAL_OR_KEYWORD	não	0.0
tempo_quadro	float	POSITIONAL_OR_KEYWORD	não	0.0
fps_instantaneo	float	POSITIONAL_OR_KEYWORD	não	0.0
fps_medio	float	POSITIONAL_OR_KEYWORD	não	0.0

Retorno declarado

None

Atributo `quadros`: `int`.

Atributo `delta_logico`: `float`.

Atributo `tempo_quadro`: `float`.

Atributo `fps_instantaneo`: `float`.

Atributo `fps_medio`: `float`.

Métodos públicos

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
registrar(self, delta_logico: 'float', tempo_quadro: 'float') -> 'None'
```

Método público sem descrição própria no runtime.

`coral.jogos.OverlayDebug`

OverlayDebug(primitivas: 'tuple[PrimitivaDebug, ...]' = (), assinatura_origem: 'str | None' = None, contrato: 'str' = 'coral.jogos.debug.overlay/1')

Assinatura

Assinatura para consulta

```
OverlayDebug(primitivas: 'tuple[PrimitivaDebug, ...]' = (), assinatura_origem: 'str |
  None' = None, contrato: 'str' = 'coral.jogos.debug.overlay/1') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
primitivas	tuple[PrimitivaDebug, ...]	POSITIONAL_OR_KEYWORD	não	()
assinatura_origem	str | None	POSITIONAL_OR_KEYWORD	não	None
contrato	str	POSITIONAL_OR_KEYWORD	não	'coral.jogos.debug.overlay/1'

Retorno declarado

None

Atributo `primitivas`: `tuple[PrimitivaDebug, ...]`.

Atributo `assinatura_origem`: `str | None`.

Atributo `contrato`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_json(self, **kwargs: 'Any') -> 'str'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
por_categoria(self, categoria: 'str') -> 'tuple[PrimitivaDebug, ...]'
```

Método público sem descrição própria no runtime.

`coral.jogos.PRETO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`Cor(vermelho=0, verde=0, azul=0, alfa=255)`

`coral.jogos.ParaleloApresentacao`

ParaleloApresentacao(etapas: 'tuple[NoApresentacao, ...]', nome: 'str | None' = None)

Assinatura

Assinatura para consulta

```
ParaleloApresentacao(etapas: 'tuple[NoApresentacao, ...]', nome: 'str | None' = None) ->
    None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
etapas	tuple[NoApresentacao, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `etapas`: `tuple[NoApresentacao, ...]`.

Atributo `nome`: `str | None`.

Métodos públicos

Referência para leitura

```
avancar(self, dt: 'float') -> 'float'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
reiniciar(self) -> 'None'
```

Método público sem descrição própria no runtime.

`coral.jogos.PrimitivaDebug`

PrimitivaDebug(tipo: 'str', categoria: 'str', dados: 'Mapping[str, Any]' = <factory>, rotulo: 'str | None' = None, espaco: 'str' = 'mundo')

Assinatura

Assinatura para consulta

```
PrimitivaDebug(tipo: 'str', categoria: 'str', dados: 'Mapping[str, Any]' = <factory>,
  rotulo: 'str | None' = None, espaco: 'str' = 'mundo') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
categoria	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
dados	Mapping[str, Any]	POSITIONAL_OR_KEYWORD	não	<factory>
rotulo	str | None	POSITIONAL_OR_KEYWORD	não	None
espaco	str	POSITIONAL_OR_KEYWORD	não	'mundo'

Retorno declarado

None

Atributo `tipo`: `str`.

Atributo `categoria`: `str`.

Atributo `dados`: `Mapping[str, Any]`.

Atributo `rotulo`: `str | None`.

Atributo `espaco`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.jogos.QuadroChave`

Valor numérico associado a um instante de uma faixa temporal.

Assinatura

Assinatura para consulta

```
QuadroChave(instante: 'float', valor: 'float', curva: 'str | Callable[[float], float]' =
   'linear') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
instante	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
valor	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
curva	str | Callable[[float], float]	POSITIONAL_OR_KEYWORD	não	'linear'

Retorno declarado

None

Atributo `instante`: `float`.

Atributo `valor`: `float`.

Atributo `curva`: `str | Callable[[float], float]`.

`coral.jogos.QuadroSprite`

Região imutável de uma imagem usada como quadro de sprite.

Assinatura

Assinatura para consulta

```
QuadroSprite(caminho: 'Path', x: 'int', y: 'int', largura: 'int', altura: 'int', indice:
    'int | None' = None, linha: 'int | None' = None, coluna: 'int | None' = None, nome:
   'str | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
caminho	Path	POSITIONAL_OR_KEYWORD	sim	sem padrão
x	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
largura	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
altura	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
indice	int | None	POSITIONAL_OR_KEYWORD	não	None

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
linha	int | None	POSITIONAL_OR_KEYWORD	não	None
coluna	int | None	POSITIONAL_OR_KEYWORD	não	None
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `caminho`: `Path`.

Atributo `x`: `int`.

Atributo `y`: `int`.

Atributo `largura`: `int`.

Atributo `altura`: `int`.

Atributo `indice`: `int | None`.

Atributo `linha`: `int | None`.

Atributo `coluna`: `int | None`.

Atributo `nome`: `str | None`.

`RenderizadorMapa2D(...)` · módulo `coral.jogos`

O que faz

Converte as células de um mapa em comandos de desenho 2D.

Construtor

Assinatura para consulta

```
RenderizadorMapa2D(mapa: 'Mapa', *, camadas: 'Iterable[str] | None' = None, camera:
  'Camera | None' = None, resolvedor_estilo: 'ResolvedorEstiloMapa | None' = None,
  area_visivel: 'tuple[float, float, float, float] | None' = None) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mapa	Mapa	sim	sem padrão	Mapa que deve ser visualizado.
camadas	Iterable[str] | None	não	None	Nomes das camadas a considerar.
camera	Camera | None	não	None	Câmera que controla a região e transformação da visualização.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
resolvedor_estilo	ResolvedorEstiloMapa | None	não	None	Função que define a aparência de uma célula.
area_visivel	tuple[float, float, float, float] | None	não	None	Retângulo de mundo que delimita a consulta visual.

Atributos importantes

Atributo	Significado
`mapa`	Mapa consultado pelo renderizador.
`camera`	Câmera fornecida, quando houver.
`area_visivel`	Limite explícito de consulta quando usado.

Métodos principais

Assinatura para consulta

```
celulas_visiveis(self) -> 'tuple[Any, ...]'
```

Referência para leitura

```
desenhar(self, backend: 'BackendJogos') -> 'int'
```

Retorno

Instância de RenderizadorMapa2D.

Exemplo

Código Coral

```
de coral.jogos importe RenderizadorMapa2D, BackendNulo
crie um mapa chamado sala com topologia quadrada
adicione uma camada chamada terreno ao mapa sala
coloque o bloco "parede" na célula (1, 1) da camada terreno do mapa sala
defina renderizador como RenderizadorMapa2D(sala, area_visivel=(0, 0, 64, 64))
mostre renderizador.desenhar(BackendNulo())
```

Resultado

Saída esperada

```
1
```

Atenção

A criação do objeto de dados não inicia a execução de um jogo. A simulação sem janela não comprova comportamento de uma janela ou dispositivo físico.

`coral.jogos.RepresentacaoEntidade`

Liga uma entidade de mundo a um Sprite sem acoplar coral.mundo a jogos.

A entidade é a fonte de verdade para a posição. O adaptador apenas lê atributos nomeados e atualiza o sprite visual correspondente.

Assinatura

Assinatura para consulta

```
RepresentacaoEntidade(entidade: 'Any', sprite: 'Sprite', atributo_x: 'str' = 'x',
  atributo_y: 'str' = 'y') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
entidade	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
sprite	Sprite	POSITIONAL_OR_KEYWORD	sim	sem padrão
atributo_x	str	POSITIONAL_OR_KEYWORD	não	'x'
atributo_y	str	POSITIONAL_OR_KEYWORD	não	'y'

Retorno declarado

None

Atributo `entidade`: `Any`.

Atributo `sprite`: `Sprite`.

Atributo `atributo_x`: `str`.

Atributo `atributo_y`: `str`.

Métodos públicos

Referência para leitura

```
sincronizar(self) -> 'Sprite'
```

Método público sem descrição própria no runtime.

`coral.jogos.Retangulo`

Retangulo(x: 'float', y: 'float', largura: 'float', altura: 'float')

Assinatura

Assinatura para consulta

```
Retangulo(x: 'float', y: 'float', largura: 'float', altura: 'float') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
x	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
largura	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
altura	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `x`: `float`.

Atributo `y`: `float`.

Atributo `largura`: `float`.

Atributo `altura`: `float`.

Métodos públicos

Referência para leitura

```
colide_com(self, outro: "'Retangulo'") -> 'bool'
```

Método público sem descrição própria no runtime.

`coral.jogos.SequenciaApresentacao`

SequenciaApresentacao(etapas: 'tuple[NoApresentacao, ...]', nome: 'str | None' = None)

Assinatura

Assinatura para consulta

```
SequenciaApresentacao(etapas: 'tuple[NoApresentacao, ...]', nome: 'str | None' = None)
  -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
etapas	tuple[NoApresentacao, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `etapas`: `tuple[NoApresentacao, ...]`.

Atributo `nome`: `str | None`.

Métodos públicos

Referência para leitura

```
avancar(self, dt: 'float') -> 'float'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
reiniciar(self) -> 'None'
```

Método público sem descrição própria no runtime.

`Sprite(...)` · módulo `coral.jogos`

O que faz

Mantém posição, tamanho, apresentação e dados de um elemento visual.

Construtor

Assinatura para consulta

```
Sprite(x: 'float', y: 'float', largura: 'float', altura: 'float', imagem: 'str | Path |
  QuadroSprite | None' = None, cor: 'Cor' = Cor(vermelho=255, verde=255, azul=255,
  alfa=255), velocidade_x: 'float' = 0.0, velocidade_y: 'float' = 0.0, visivel: 'bool' =
  True, nome: 'str' = 'sprite', dados: 'dict[str, Any]' = <factory>, animacao: 'Any' =
  None, escala: 'float' = 1.0, rotacao: 'float' = 0.0, espelhado_horizontalmente: 'bool' =
   False, espelhado_verticalmente: 'bool' = False, origem: 'str' = 'topo_esquerdo', pivo:
  'tuple[float, float] | None' = None, opacidade: 'float' = 1.0, colisao_acompanha_escala:
   'bool' = False, colisao_acompanha_rotacao: 'bool' = False) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	float	sim	sem padrão	Coordenadas horizontais da série, ou posição horizontal do objeto.
y	float	sim	sem padrão	Coordenadas verticais da série, ou posição vertical do objeto.
largura	float	sim	sem padrão	Largura em unidades da apresentação.
altura	float	sim	sem padrão	Altura em unidades da apresentação.
imagem	str | Path | QuadroSprite | None	não	None	Imagem ou quadro do sprite; nulo usa sua representação de cor.
cor	Cor	não	Cor(vermelho=255, verde=255, azul=255, alfa=255)	Cor de apresentação do sprite.
velocidade_x	float	não	0.0	Velocidade horizontal.
velocidade_y	float	não	0.0	Velocidade vertical.
visivel	bool	não	True	Se verdadeiro, o sprite participa da apresentação.
nome	str	não	'sprite'	Nome da série ou do objeto.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
dados	dict[str, Any]	não	<factory>	Dicionário livre de dados associados ao sprite; a fábrica cria um dicionário independente.
animacao	Any	não	None	Animação associada ao sprite.
escala	float	não	1.0	Fator de escala visual.
rotacao	float	não	0.0	Ângulo de rotação da apresentação.
espelhado_horizontalmente	bool	não	False	Se verdadeiro, inverte horizontalmente a imagem.
espelhado_verticalmente	bool	não	False	Se verdadeiro, inverte verticalmente a imagem.
origem	str	não	'topo_esquerdo'	Ponto de referência da posição visual.
pivo	tuple[float, float] | None	não	None	Ponto usado como pivô de transformações.
opacidade	float	não	1.0	Intensidade de opacidade entre zero e um.
colisao_acompanha_escala	bool	não	False	Se verdadeiro, a forma de colisão acompanha a escala visual.
colisao_acompanha_rotacao	bool	não	False	Se verdadeiro, a forma de colisão acompanha a rotação visual.

Atributos importantes

Atributo	Significado
`x`	Posição horizontal.
`y`	Posição vertical.
`largura`	Largura da forma base.
`altura`	Altura da forma base.
`dados`	Dados associados ao objeto.

Métodos principais

Assinatura para consulta

```
mover(self, dx: 'float', dy: 'float') -> 'None'
```

Referência para leitura

```
atualizar(self, dt: 'float') -> 'None'
```

Referência para leitura

```
colide_com(self, outro: "'Sprite'") -> 'bool'
```

Retorno

Instância de Sprite.

Exemplo

Código Coral

```
de coral.jogos importe Sprite
defina jogador como Sprite(0, 0, 10, 10)
mostre jogador.largura
```

Resultado

Saída esperada

```
10
```

Atenção

A criação do objeto de dados não inicia a execução de um jogo. A simulação sem janela não comprova comportamento de uma janela ou dispositivo físico.

`coral.jogos.Spritesheet`

Descrição headless de uma spritesheet ou atlas de imagens.

Assinatura

Assinatura para consulta

```
Spritesheet(caminho: 'str | Path', *, largura_imagem: 'int', altura_imagem: 'int') ->
  'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
caminho	str | Path	POSITIONAL_OR_KEYWORD	sim	sem padrão
largura_imagem	int	KEYWORD_ONLY	sim	sem padrão
altura_imagem	int	KEYWORD_ONLY	sim	sem padrão

Retorno declarado

None

Métodos públicos

Referência para leitura

```
animacao(self, inicio: 'int', fim: 'int', *, fps: 'float' = 10.0, repetir: 'bool' =
  True)
```

Cria uma Animacao usando um intervalo inclusivo de quadros.

Referência para leitura

```
animacao_da_linha(self, linha: 'int', coluna_inicial: 'int' = 0, coluna_final: 'int |
  None' = None, *, fps: 'float' = 10.0, repetir: 'bool' = True)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
animacoes_direcionais(self, linhas: 'dict[str, int]', coluna_inicial: 'int' = 0,
  coluna_final: 'int | None' = None, *, fps: 'float' = 10.0, repetir: 'bool' = True)
```

Cria um grupo de animações a partir de linhas nomeadas da grade.

Referência para leitura

```
em_grade(caminho: 'str | Path', *, largura_imagem: 'int', altura_imagem: 'int',
  largura_quadro: 'int', altura_quadro: 'int', margem: 'int' = 0, espacamento: 'int' = 0)
  -> "'Spritesheet'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
quadro(self, indice: 'int') -> 'QuadroSprite'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
quadro_em(self, linha: 'int', coluna: 'int') -> 'QuadroSprite'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
quadros_da_linha(self, linha: 'int', coluna_inicial: 'int' = 0, coluna_final: 'int |
  None' = None) -> 'tuple[QuadroSprite, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
quadros_de(self, inicio: 'int', fim: 'int') -> 'tuple[QuadroSprite, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
regiao(self, x: 'int', y: 'int', largura: 'int', altura: 'int', *, nome: 'str | None' =
  None) -> 'QuadroSprite'
```

Método público sem descrição própria no runtime.

`coral.jogos.TextoApresentacao`

Texto 2D reutilizável para camadas de mundo ou de tela.

A classe cuida somente de apresentação. Não cria widgets, não captura entrada e não altera estado lógico. A quebra de linha usa a métrica do backend por meio de `` `Jogo.medir_texto` `` e, portanto, também funciona em execução headless.

Assinatura

Assinatura para consulta

```
TextoApresentacao(texto: 'str', x: 'float' = 0.0, y: 'float' = 0.0, tamanho: 'int' = 24,
   cor: 'Cor' = Cor(vermelho=255, verde=255, azul=255, alfa=255), largura_maxima: 'float |
   None' = None, alinhamento_horizontal: 'str' = 'esquerda', alinhamento_vertical: 'str' =
   'topo', espacamento_linhas: 'float' = 1.0, sombra: 'Cor | None' = None,
  deslocamento_sombra: 'tuple[float, float]' = (2.0, 2.0), contorno: 'Cor | None' = None,
  espessura_contorno: 'int' = 1, opacidade: 'float' = 1.0, origem: 'str | None' = None,
  pivo: 'tuple[float, float] | None' = None, ancora_viewport: 'str | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
texto	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
x	float	POSITIONAL_OR_KEYWORD	não	0.0
y	float	POSITIONAL_OR_KEYWORD	não	0.0
tamanho	int	POSITIONAL_OR_KEYWORD	não	24
cor	Cor	POSITIONAL_OR_KEYWORD	não	Cor(vermelho=255, verde=255, azul=255, alfa=255)
largura_maxima	float | None	POSITIONAL_OR_KEYWORD	não	None
alinhamento_horizontal	str	POSITIONAL_OR_KEYWORD	não	'esquerda'
alinhamento_vertical	str	POSITIONAL_OR_KEYWORD	não	'topo'
espacamento_linhas	float	POSITIONAL_OR_KEYWORD	não	1.0
sombra	Cor | None	POSITIONAL_OR_KEYWORD	não	None
deslocamento_sombra	tuple[float, float]	POSITIONAL_OR_KEYWORD	não	(2.0, 2.0)
contorno	Cor | None	POSITIONAL_OR_KEYWORD	não	None
espessura_contorno	int	POSITIONAL_OR_KEYWORD	não	1
opacidade	float	POSITIONAL_OR_KEYWORD	não	1.0
origem	str | None	POSITIONAL_OR_KEYWORD	não	None

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
pivo	tuple[float, float] | None	POSITIONAL_OR_KEYWORD	não	None
ancora_viewport	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `texto`: `str`.

Atributo `x`: `float`.

Atributo `y`: `float`.

Atributo `tamanho`: `int`.

Atributo `cor`: `Cor`.

Atributo `largura_maxima`: `float | None`.

Atributo `alinhamento_horizontal`: `str`.

Atributo `alinhamento_vertical`: `str`.

Atributo `espacamento_linhas`: `float`.

Atributo `sombra`: `Cor | None`.

Atributo `deslocamento_sombra`: `tuple[float, float]`.

Atributo `contorno`: `Cor | None`.

Atributo `espessura_contorno`: `int`.

Atributo `opacidade`: `float`.

Atributo `origem`: `str | None`.

Atributo `pivo`: `tuple[float, float] | None`.

Atributo `ancora_viewport`: `str | None`.

Métodos públicos

Referência para leitura

```
desenhar(self, jogo, cena: "'Cena'", camada: 'CamadaApresentacao') -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
linhas(self, jogo) -> 'tuple[str, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
medir(self, jogo) -> 'tuple[float, float]'
```

Método público sem descrição própria no runtime.

`coral.jogos.TransicaoCena`

Coordena apresentação temporal entre duas cenas sem possuir o jogo.

O objeto é headless e determinístico. A troca efetiva continua sendo feita por `` `Jogo.mudar_cena` `` quando o progresso cruza o ponto de troca.

Assinatura

Assinatura para consulta

```
TransicaoCena(origem: 'Any', destino: 'Any', tipo: 'str' = 'desaparecer', duracao:
  'float' = 0.4, curva: 'str | Callable[[float], float]' = 'suave', bloquear_entrada:
  'bool' = True, atualizar_anterior: 'bool' = True, atualizar_proxima: 'bool' = True,
  direcao: 'str' = 'direita', tempo: 'float' = 0.0, estado: 'str' = 'ativo') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
origem	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
destino	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipo	str	POSITIONAL_OR_KEYWORD	não	'desaparecer'
duracao	float	POSITIONAL_OR_KEYWORD	não	0.4
curva	str | Callable[[float], float]	POSITIONAL_OR_KEYWORD	não	'suave'
bloquear_entrada	bool	POSITIONAL_OR_KEYWORD	não	True
atualizar_anterior	bool	POSITIONAL_OR_KEYWORD	não	True
atualizar_proxima	bool	POSITIONAL_OR_KEYWORD	não	True
direcao	str	POSITIONAL_OR_KEYWORD	não	'direita'
tempo	float	POSITIONAL_OR_KEYWORD	não	0.0

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
estado	str	POSITIONAL_OR_KEYWORD	não	'ativo'

Retorno declarado

None

Atributo `origem`: `Any`.

Atributo `destino`: `Any`.

Atributo `tipo`: `str`.

Atributo `duracao`: `float`.

Atributo `curva`: `str | Callable[[float], float]`.

Atributo `bloquear_entrada`: `bool`.

Atributo `atualizar_anterior`: `bool`.

Atributo `atualizar_proxima`: `bool`.

Atributo `direcao`: `str`.

Atributo `tempo`: `float`.

Atributo `estado`: `str`.

Métodos públicos

Referência para leitura

```
amostra(self) -> 'dict[str, Any]'
```

Estado visual independente de backend para renderers e testes.

Referência para leitura

```
ao_concluir(self, callback: 'Callable') -> 'Callable'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
avancar(self, dt: 'float') -> 'bool'
```

Avança e informa se a troca de cena deve ocorrer neste passo.

Referência para leitura

```
cancelar(self) -> "'TransicaoCena'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
pausar(self) -> "'TransicaoCena'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
retomar(self) -> "'TransicaoCena'"
```

Método público sem descrição própria no runtime.

`coral.jogos.TransicaoEstadoAnimacao`

Política declarativa para uma transição visual.

Assinatura

Assinatura para consulta

```
TransicaoEstadoAnimacao(estado: 'str', prioridade: 'int' = 0, tempo_minimo: 'float' =
  0.0, bloquear_interrupcao: 'float' = 0.0, retorno_automatico: 'str | None' = None,
  reiniciar: 'bool' = True) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
estado	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
prioridade	int	POSITIONAL_OR_KEYWORD	não	0
tempo_minimo	float	POSITIONAL_OR_KEYWORD	não	0.0
bloquear_interrupcao	float	POSITIONAL_OR_KEYWORD	não	0.0
retorno_automatico	str | None	POSITIONAL_OR_KEYWORD	não	None
reiniciar	bool	POSITIONAL_OR_KEYWORD	não	True

Retorno declarado

None

Atributo `estado`: `str`.

Atributo `prioridade`: `int`.

Atributo `tempo_minimo`: `float`.

Atributo `bloquear_interrupcao`: `float`.

Atributo `retorno_automatico`: `str | None`.

Atributo `reiniciar`: `bool`.

`coral.jogos.TransicoesEstadosAnimacao`

Transições visuais por evento, condição ou chamada manual.

Continua opt in e não substitui máquinas de estado de agentes, RPG ou simulação. Valores `str` no dicionário mantêm o contrato histórico.

Assinatura

Assinatura para consulta

```
TransicoesEstadosAnimacao(estados: 'EstadosAnimacao', transicoes: 'dict[str, str |
  TransicaoEstadoAnimacao | dict[str, Any]]', *, reiniciar_ao_retornar: 'bool' = True)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
estados	EstadosAnimacao	POSITIONAL_OR_KEYWORD	sim	sem padrão
transicoes	dict[str, str | TransicaoEstadoAnimacao | dict[str, Any]]	POSITIONAL_OR_KEYWORD	sim	sem padrão
reiniciar_ao_retornar	bool	KEYWORD_ONLY	não	True

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
acionar(self, sprite: 'Sprite', evento: 'str') -> 'Animacao | None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
ligar(self, movimento: 'MovimentoDirecional', sprite: 'Sprite') ->
  'VinculoTransicoesEstadosMovimento'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
ligar_eventos(self, eventos: 'Any', sprite: 'Sprite') -> 'VinculoTransicoesEventos'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para(self, evento: 'str') -> 'str'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_transicao(self, evento: 'str') -> 'TransicaoEstadoAnimacao'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
quando(self, condicao: 'Callable[[Sprite], bool]', estado: 'str', *, prioridade: 'int' =
   0, tempo_minimo: 'float' = 0.0, bloquear_interrupcao: 'float' = 0.0,
  retorno_automatico: 'str | None' = None, reiniciar: 'bool' = True) ->
  'CondicaoTransicaoAnimacao'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
transicionar(self, sprite: 'Sprite', estado: 'str', **kwargs) -> 'Animacao | None'
```

Método público sem descrição própria no runtime.

`coral.jogos.VERDE`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`Cor(vermelho=46, verde=160, azul=67, alfa=255)`

`coral.jogos.VERMELHO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`Cor(vermelho=220, verde=50, azul=47, alfa=255)`

`coral.jogos.Viewport`

Transformação entre a superfície física e a área lógica de Jogos.

O modo `` `ajustar` preserva a proporção lógica e centraliza a área útil. O modo `esticar` `` ocupa toda a superfície física usando escalas independentes. A classe não desenha nem altera estado de jogo, apenas converte coordenadas.

Assinatura

Assinatura para consulta

```
Viewport(largura_logica: 'float', altura_logica: 'float', largura_fisica: 'float',
  altura_fisica: 'float', modo: 'str' = 'ajustar') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
largura_logica	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
altura_logica	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
largura_fisica	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
altura_fisica	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
modo	str	POSITIONAL_OR_KEYWORD	não	'ajustar'

Retorno declarado

None

Atributo `largura_logica`: `float`.

Atributo `altura_logica`: `float`.

Atributo `largura_fisica`: `float`.

Atributo `altura_fisica`: `float`.

Atributo `modo`: `str`.

Métodos públicos

Referência para leitura

```
ancorar(self, largura: 'float', altura: 'float', *, ancora: 'str' = 'superior_esquerda',
    margem_x: 'float' = 0.0, margem_y: 'float | None' = None) -> 'tuple[float, float]'
```

Retorna o canto superior esquerdo de um elemento na área lógica.

Referência para leitura

```
contem_tela(self, x: 'float', y: 'float') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
logica_para_tela(self, x: 'float', y: 'float') -> 'tuple[float, float]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
tela_para_logica(self, x: 'float', y: 'float', *, limitar: 'bool' = False) ->
  'tuple[float, float]'
```

Método público sem descrição própria no runtime.

`coral.jogos.VinculoAnimacaoMovimento`

VinculoAnimacaoMovimento(grupo: "'AnimacoesDirecionais'", movimento: 'MovimentoDirecional', sprite: 'Sprite', direcoes_compativeis: 'tuple[str, ...]', direcao_visual: 'str | None' = None)

Assinatura

Assinatura para consulta

```
VinculoAnimacaoMovimento(grupo: "'AnimacoesDirecionais'", movimento:
  'MovimentoDirecional', sprite: 'Sprite', direcoes_compativeis: 'tuple[str, ...]',
  direcao_visual: 'str | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
grupo	'AnimacoesDirecionais'	POSITIONAL_OR_KEYWORD	sim	sem padrão
movimento	MovimentoDirecional	POSITIONAL_OR_KEYWORD	sim	sem padrão
sprite	Sprite	POSITIONAL_OR_KEYWORD	sim	sem padrão
direcoes_compativeis	tuple[str, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
direcao_visual	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `grupo`: `'AnimacoesDirecionais'`.

Atributo `movimento`: `MovimentoDirecional`.

Atributo `sprite`: `Sprite`.

Atributo `direcoes_compativeis`: `tuple[str, ...]`.

Atributo `direcao_visual`: `str | None`.

`coral.jogos.VinculoClipe`

VinculoClipe(alvo: 'Any', clipe: 'ClipeAnimacao', mapeamento: 'dict[str, str]', modo: 'str' = 'absoluto', restaurar_ao_final: 'bool' = False, composicao: 'str' = 'substituir')

Assinatura

Assinatura para consulta

```
VinculoClipe(alvo: 'Any', clipe: 'ClipeAnimacao', mapeamento: 'dict[str, str]', modo:
  'str' = 'absoluto', restaurar_ao_final: 'bool' = False, composicao: 'str' =
  'substituir') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
alvo	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
clipe	ClipeAnimacao	POSITIONAL_OR_KEYWORD	sim	sem padrão
mapeamento	dict[str, str]	POSITIONAL_OR_KEYWORD	sim	sem padrão
modo	str	POSITIONAL_OR_KEYWORD	não	'absoluto'
restaurar_ao_final	bool	POSITIONAL_OR_KEYWORD	não	False
composicao	str	POSITIONAL_OR_KEYWORD	não	'substituir'

Retorno declarado

None

Atributo `alvo`: `Any`.

Atributo `clipe`: `ClipeAnimacao`.

Atributo `mapeamento`: `dict[str, str]`.

Atributo `modo`: `str`.

Atributo `restaurar_ao_final`: `bool`.

Atributo `composicao`: `str`.

Métodos públicos

Referência para leitura

```
atualizar(self, dt: 'float') -> 'dict[str, float]'
```

Método público sem descrição própria no runtime.

`coral.jogos.VinculoComposicaoEventos`

Controla uma composição por um evento sem acoplar o barramento a Jogos.

Assinatura

Assinatura para consulta

```
VinculoComposicaoEventos(composicao: 'ComposicaoTemporal', eventos: 'Any', evento:
  'str', modo: 'str' = 'reiniciar') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
composicao	ComposicaoTemporal	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
eventos	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
evento	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
modo	str	POSITIONAL_OR_KEYWORD	não	'reiniciar'

Retorno declarado

None

Atributo `composicao`: `ComposicaoTemporal`.

Atributo `eventos`: `Any`.

Atributo `evento`: `str`.

Atributo `modo`: `str`.

Atributo `ativo`: `bool`.

Métodos públicos

Referência para leitura

```
desligar(self) -> 'None'
```

Método público sem descrição própria no runtime.

`coral.jogos.VinculoEstadosAnimacaoMovimento`

Mantém estado visual e direção como dimensões independentes.

O vínculo é deliberadamente visual: ele não representa o estado lógico da entidade. O histórico de estado e os eventos de entrada e saída existem apenas para coordenar apresentação.

Assinatura

Assinatura para consulta

```
VinculoEstadosAnimacaoMovimento(grupo: "'EstadosAnimacao'", movimento:
  'MovimentoDirecional', sprite: 'Sprite', estado_atual: 'str | None' = None,
  estado_anterior: 'str | None' = None, direcao_visual: 'str | None' = None,
  ultima_direcao_visual: 'str | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
grupo	'EstadosAnimacao'	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
movimento	MovimentoDirecional	POSITIONAL_OR_KEYWORD	sim	sem padrão
sprite	Sprite	POSITIONAL_OR_KEYWORD	sim	sem padrão
estado_atual	str | None	POSITIONAL_OR_KEYWORD	não	None
estado_anterior	str | None	POSITIONAL_OR_KEYWORD	não	None
direcao_visual	str | None	POSITIONAL_OR_KEYWORD	não	None
ultima_direcao_visual	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `grupo`: `'EstadosAnimacao'`.

Atributo `movimento`: `MovimentoDirecional`.

Atributo `sprite`: `Sprite`.

Atributo `estado_atual`: `str | None`.

Atributo `estado_anterior`: `str | None`.

Atributo `direcao_visual`: `str | None`.

Atributo `ultima_direcao_visual`: `str | None`.

Métodos públicos

Referência para leitura

```
usar(self, estado: 'str', *, direcao: 'str | None' = None, reiniciar: 'bool' = True) ->
  'Animacao'
```

Método público sem descrição própria no runtime.

`coral.jogos.VinculoTempoApresentacao`

Avança uma composição a partir de uma fonte de tempo explicitamente escolhida.

A fonte precisa apenas oferecer `agora(). O vínculo observa diferenças entre instantes e nunca chama passo()` nem avança o relógio por conta própria, evitando tomar autoridade do tempo de Mundo ou Simulação.

Assinatura

Assinatura para consulta

```
VinculoTempoApresentacao(composicao: 'ComposicaoTemporal', fonte: 'Any') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
composicao	ComposicaoTemporal	POSITIONAL_OR_KEYWORD	sim	sem padrão
fonte	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `composicao`: `ComposicaoTemporal`.

Atributo `fonte`: `Any`.

Métodos públicos

Referência para leitura

```
atualizar(self) -> 'float'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
ressincronizar(self) -> "'VinculoTempoApresentacao'"
```

Método público sem descrição própria no runtime.

`coral.jogos.VinculoTransicoesEstadosMovimento`

Executa a máquina visual opt in de um Sprite.

Assinatura

Assinatura para consulta

```
VinculoTransicoesEstadosMovimento(grupo: "'TransicoesEstadosAnimacao'", movimento:
  'MovimentoDirecional', sprite: 'Sprite', vinculo_estados:
  'VinculoEstadosAnimacaoMovimento', ultimo_evento: 'str | None' = None, tempo_no_estado:
  'float' = 0.0, bloqueio_restante: 'float' = 0.0, prioridade_atual: 'int' = 0,
  _estava_movendo: 'bool' = False, _ultimo_estado: 'str | None' = None,
  _retorno_automatico: 'str | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
grupo	'TransicoesEstadosAnimacao'	POSITIONAL_OR_KEYWORD	sim	sem padrão
movimento	MovimentoDirecional	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
sprite	Sprite	POSITIONAL_OR_KEYWORD	sim	sem padrão
vinculo_estados	VinculoEstadosAnimacaoMovimento	POSITIONAL_OR_KEYWORD	sim	sem padrão
ultimo_evento	str | None	POSITIONAL_OR_KEYWORD	não	None
tempo_no_estado	float	POSITIONAL_OR_KEYWORD	não	0.0
bloqueio_restante	float	POSITIONAL_OR_KEYWORD	não	0.0
prioridade_atual	int	POSITIONAL_OR_KEYWORD	não	0
_estava_movendo	bool	POSITIONAL_OR_KEYWORD	não	False
_ultimo_estado	str | None	POSITIONAL_OR_KEYWORD	não	None
_retorno_automatico	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `grupo`: `'TransicoesEstadosAnimacao'`.

Atributo `movimento`: `MovimentoDirecional`.

Atributo `sprite`: `Sprite`.

Atributo `vinculo_estados`: `VinculoEstadosAnimacaoMovimento`.

Atributo `ultimo_evento`: `str | None`.

Atributo `tempo_no_estado`: `float`.

Atributo `bloqueio_restante`: `float`.

Atributo `prioridade_atual`: `int`.

Métodos públicos

Referência para leitura

```
acionar(self, evento: 'str') -> 'Animacao | None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
atualizar(self, dt: 'float') -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
transicionar(self, estado: 'str', *, prioridade: 'int' = 0, tempo_minimo: 'float' = 0.0,
    bloquear_interrupcao: 'float' = 0.0, retorno_automatico: 'str | None' = None,
  reiniciar: 'bool' = True) -> 'Animacao | None'
```

Método público sem descrição própria no runtime.

`coral.jogos.VinculoTransicoesEventos`

Escuta um barramento genérico e encaminha eventos declarados para transições visuais.

Assinatura

Assinatura para consulta

```
VinculoTransicoesEventos(grupo: "'TransicoesEstadosAnimacao'", eventos: 'Any', sprite:
  'Sprite', vinculo_movimento: 'VinculoTransicoesEstadosMovimento') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
grupo	'TransicoesEstadosAnimacao'	POSITIONAL_OR_KEYWORD	sim	sem padrão
eventos	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
sprite	Sprite	POSITIONAL_OR_KEYWORD	sim	sem padrão
vinculo_movimento	VinculoTransicoesEstadosMovimento	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `grupo`: `'TransicoesEstadosAnimacao'`.

Atributo `eventos`: `Any`.

Atributo `sprite`: `Sprite`.

Atributo `vinculo_movimento`: `VinculoTransicoesEstadosMovimento`.

Atributo `ativo`: `bool`.

Métodos públicos

Referência para leitura

```
desligar(self) -> 'None'
```

Método público sem descrição própria no runtime.

`coral.jogos.adaptar_eventos`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
adaptar_eventos(jogo: 'Jogo', *, teclas: 'Iterable[str]' = (), botoes_mouse:
  'Iterable[str]' = (), observar_mouse: 'bool' = False) -> 'AdaptadorEventosJogo'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
jogo	Jogo	POSITIONAL_OR_KEYWORD	sim	sem padrão
teclas	Iterable[str]	KEYWORD_ONLY	não	()
botoes_mouse	Iterable[str]	KEYWORD_ONLY	não	()
observar_mouse	bool	KEYWORD_ONLY	não	False

Retorno declarado

AdaptadorEventosJogo

`coral.jogos.animar_propriedade`

Anime uma propriedade pública suportada até um valor em tempo explícito.

Esta função é a ponte compacta usada pela superfície em português corrente. Ela reutiliza os adaptadores seguros do CP6 em vez de usar `setattr` irrestrito.

Assinatura

Assinatura para consulta

```
animar_propriedade(alvo: 'Any', propriedade: 'str', valor: 'float', duracao: 'float', *,
   curva: 'str | Callable[[float], float]' = 'suave', modo: 'str' = 'normal') ->
  "'VinculoClipe'"
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
alvo	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
propriedade	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
duracao	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
curva	str | Callable[[float], float]	KEYWORD_ONLY	não	'suave'
modo	str	KEYWORD_ONLY	não	'normal'

Retorno declarado

'VinculoClipe'

`coral.jogos.aplicar_campo_em_sprites`

Avalia um campo 2D na posição dos sprites e grava o valor em `dados`.

Assinatura

Assinatura para consulta

```
aplicar_campo_em_sprites(campo: 'CampoProcedural', sprites: 'Iterable[Sprite]', *,
  chave: 'str' = 'valor_procedural', usar_centro: 'bool' = False, limite_sprites: 'int' =
  10000) -> 'int'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
campo	CampoProcedural	POSITIONAL_OR_KEYWORD	sim	sem padrão
sprites	Iterable[Sprite]	POSITIONAL_OR_KEYWORD	sim	sem padrão
chave	str	KEYWORD_ONLY	não	'valor_procedural'
usar_centro	bool	KEYWORD_ONLY	não	False
limite_sprites	int	KEYWORD_ONLY	não	10000

Retorno declarado

int

`coral.jogos.avaliar_curva_temporal`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
avaliar_curva_temporal(curva: 'str | Callable[[float], float]', progresso: 'float', *,
  extrapolar: 'bool' = False) -> 'float'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
curva	str | Callable[[float], float]	POSITIONAL_OR_KEYWORD	sim	sem padrão
progresso	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
extrapolar	bool	KEYWORD_ONLY	não	False

Retorno declarado

float

`carregar_spritesheet(...)` · módulo `coral.jogos`

O que faz

Define uma folha de sprites dividida em quadros numa grade declarada.

Assinatura

Assinatura para consulta

```
carregar_spritesheet(caminho: 'str | Path', largura_imagem: 'int', altura_imagem: 'int',
   largura_quadro: 'int', altura_quadro: 'int', *, margem: 'int' = 0, espacamento: 'int' =
   0) -> 'Spritesheet'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	str | Path	sim	sem padrão	Caminho local do arquivo de saída ou da imagem de origem.
largura_imagem	int	sim	sem padrão	Largura total da imagem declarada.
altura_imagem	int	sim	sem padrão	Altura total da imagem declarada.
largura_quadro	int	sim	sem padrão	Largura de cada quadro da grade.
altura_quadro	int	sim	sem padrão	Altura de cada quadro da grade.
margem	int	não	0	Margem entre a borda da imagem e a grade.
espacamento	int	não	0	Separação entre quadros.

Retorno

Spritesheet.

Exemplo

Código Coral

```
de coral.jogos importe carregar_spritesheet
defina folha como carregar_spritesheet("heroi.png", 96, 64, 32, 32)
mostre len(folha)
```

Resultado

Saída esperada

```
6
```

Atenção

O exemplo descreve a grade. A renderização da imagem exige o recurso correspondente; não está sendo aberta uma janela.

`coral.jogos.celula_para_tela`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
celula_para_tela(mapa: 'Mapa', camera: 'Camera | None', celula: 'Any', *, ancora: 'str'
  = 'centro', viewport: 'Viewport | None' = None) -> 'tuple[float, float]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mapa	Mapa	POSITIONAL_OR_KEYWORD	sim	sem padrão
camera	Camera | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
celula	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
ancora	str	KEYWORD_ONLY	não	'centro'
viewport	Viewport | None	KEYWORD_ONLY	não	None

Retorno declarado

tuple[float, float]

`criar_jogo(...)` · módulo `coral.jogos`

O que faz

Cria um jogo com dimensões, backend e fontes explicitamente configuráveis.

Assinatura

Assinatura para consulta

```
criar_jogo(titulo: 'str', largura: 'int' = 800, altura: 'int' = 450, fps: 'int' = 60, *,
   backend=None, mundo=None, relogio=None, eventos=None, tela_cheia: 'bool' = False) ->
  'Jogo'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
titulo	str	sim	sem padrão	Valor titulo usado na operação.
largura	int	não	800	Largura em unidades da apresentação.
altura	int	não	450	Altura em unidades da apresentação.
fps	int	não	60	Frequência alvo de quadros por segundo.
backend	valor dinâmico	não	None	Backend que realiza desenho e entrada; BackendNulo permite executar sem janela.
mundo	valor dinâmico	não	None	Mundo associado ao jogo, quando houver.
relogio	valor dinâmico	não	None	Fonte de tempo que o jogo usa.
eventos	valor dinâmico	não	None	Motor de eventos do jogo.
tela_cheia	bool	não	False	Se verdadeiro, pede modo de tela cheia ao backend.

Retorno

Jogo, ainda sem iniciar seu laço.

Exemplo

Código Coral

```
de coral.jogos importe criar_jogo, BackendNulo
defina jogo como criar_jogo("Teste", backend=BackendNulo())
execute jogo.executar(max_quadros=2)
mostre jogo.backend.quadros
```

Resultado

Saída esperada

```
2
```

`coral.jogos.curva_cubica`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
curva_cubica(progresso: 'float') -> 'float'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
progresso	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

float

`coral.jogos.curva_degrau`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
curva_degrau(progresso: 'float') -> 'float'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
progresso	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

float

`coral.jogos.curva_entrada_saida_suave`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
curva_entrada_saida_suave(progresso: 'float') -> 'float'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
progresso	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

float

`coral.jogos.curva_entrada_suave`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
curva_entrada_suave(progresso: 'float') -> 'float'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
progresso	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

float

`coral.jogos.curva_linear`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
curva_linear(progresso: 'float') -> 'float'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
progresso	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

float

`coral.jogos.curva_quadratica`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
curva_quadratica(progresso: 'float') -> 'float'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
progresso	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

float

`coral.jogos.curva_saida_suave`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
curva_saida_suave(progresso: 'float') -> 'float'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
progresso	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

float

`coral.jogos.desenhar_overlay_debug`

Renderiza primitivas de debug pela API pública de desenho.

Assinatura

Assinatura para consulta

```
desenhar_overlay_debug(alvo: 'Any', overlay_ou_objeto: 'Any', *, camera: 'Any | None' =
  None, estilo: 'EstiloOverlayDebug | None' = None, limites: 'bool' = True, colisores:
  'bool' = True, pivos: 'bool' = True, rotulos: 'bool' = False, incluir_invisiveis: 'bool'
   = False, mostrar_camera: 'bool' = True) -> 'OverlayDebug'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
alvo	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
overlay_ou_objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
camera	Any | None	KEYWORD_ONLY	não	None
estilo	EstiloOverlayDebug | None	KEYWORD_ONLY	não	None
limites	bool	KEYWORD_ONLY	não	True
colisores	bool	KEYWORD_ONLY	não	True
pivos	bool	KEYWORD_ONLY	não	True
rotulos	bool	KEYWORD_ONLY	não	False
incluir_invisiveis	bool	KEYWORD_ONLY	não	False
mostrar_camera	bool	KEYWORD_ONLY	não	True

Retorno declarado

OverlayDebug

`coral.jogos.diagnosticar_jogos`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
diagnosticar_jogos() -> 'dict[str, object]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

dict[str, object]

`coral.jogos.esperar`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
esperar(duracao: 'float', *, nome: 'str | None' = None) -> 'EsperaApresentacao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
duracao	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	KEYWORD_ONLY	não	None

Retorno declarado

EsperaApresentacao

`coral.jogos.esperar_conclusao`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
esperar_conclusao(alvo: 'Any', *, nome: 'str | None' = None) -> 'EsperaConclusao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
alvo	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str | None	KEYWORD_ONLY	não	None

Retorno declarado

EsperaConclusao

`coral.jogos.executar`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
executar(acao: 'Callable[[], Any]', *, nome: 'str | None' = None) -> 'AcaoApresentacao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
acao	Callable[[], Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str | None	KEYWORD_ONLY	não	None

Retorno declarado

AcaoApresentacao

`coral.jogos.forma_espacial_sprite`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
forma_espacial_sprite(sprite: 'Any') -> 'Poligono'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
sprite	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Poligono

`coral.jogos.formas_colidem`

Colisão convexa por SAT. Toque de borda não conta como colisão.

Assinatura

Assinatura para consulta

```
formas_colidem(a: 'FormaTransformada', b: 'FormaTransformada') -> 'bool'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	FormaTransformada	POSITIONAL_OR_KEYWORD	sim	sem padrão
b	FormaTransformada	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

bool

`coral.jogos.gerar_overlay_debug`

Gera primitivas de debug sem desenhar e sem depender de backend gráfico.

Assinatura

Assinatura para consulta

```
gerar_overlay_debug(objeto: 'Any', *, limites: 'bool' = True, colisores: 'bool' = True,
  camera: 'bool' = True, pivos: 'bool' = True, rotulos: 'bool' = False,
  incluir_invisiveis: 'bool' = False) -> 'OverlayDebug'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
limites	bool	KEYWORD_ONLY	não	True
colisores	bool	KEYWORD_ONLY	não	True
camera	bool	KEYWORD_ONLY	não	True
pivos	bool	KEYWORD_ONLY	não	True
rotulos	bool	KEYWORD_ONLY	não	False
incluir_invisiveis	bool	KEYWORD_ONLY	não	False

Retorno declarado

OverlayDebug

`coral.jogos.gerar_overlay_metricas_debug`

Converte métricas operacionais em texto de tela, sem backend gráfico.

Assinatura

Assinatura para consulta

```
gerar_overlay_metricas_debug(metricas: 'Any', *, x: 'float' = 8.0, y: 'float' = 8.0,
  espacamento: 'float' = 18.0) -> 'OverlayDebug'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
metricas	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
x	float	KEYWORD_ONLY	não	8.0
y	float	KEYWORD_ONLY	não	8.0
espacamento	float	KEYWORD_ONLY	não	18.0

Retorno declarado

OverlayDebug

`coral.jogos.interpolar`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
interpolar(inicio: 'float', fim: 'float', progresso: 'float', *, curva: 'str |
  Callable[[float], float]' = 'linear', extrapolar: 'bool' = False) -> 'float'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
inicio	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
fim	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
progresso	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
curva	str | Callable[[float], float]	KEYWORD_ONLY	não	'linear'
extrapolar	bool	KEYWORD_ONLY	não	False

Retorno declarado

float

`coral.jogos.ligar_clipe`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
ligar_clipe(alvo: 'Any', clipe: 'ClipeAnimacao', mapeamento: 'dict[str, str]', *, modo:
  'str' = 'absoluto', restaurar_ao_final: 'bool' = False, composicao: 'str' =
  'substituir', permitir_conflito: 'bool' = False) -> 'VinculoClipe'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
alvo	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
clipe	ClipeAnimacao	POSITIONAL_OR_KEYWORD	sim	sem padrão
mapeamento	dict[str, str]	POSITIONAL_OR_KEYWORD	sim	sem padrão
modo	str	KEYWORD_ONLY	não	'absoluto'
restaurar_ao_final	bool	KEYWORD_ONLY	não	False
composicao	str	KEYWORD_ONLY	não	'substituir'
permitir_conflito	bool	KEYWORD_ONLY	não	False

Retorno declarado

VinculoClipe

`coral.jogos.ligar_evento`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
ligar_evento(composicao: 'ComposicaoTemporal', eventos: 'Any', evento: 'str', *, modo:
  'str' = 'reiniciar') -> 'VinculoComposicaoEventos'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
composicao	ComposicaoTemporal	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
eventos	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
evento	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
modo	str	KEYWORD_ONLY	não	'reiniciar'

Retorno declarado

VinculoComposicaoEventos

`coral.jogos.mundo_para_tela`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
mundo_para_tela(camera: 'Camera | None', coordenada: 'Any', *, viewport: 'Viewport |
  None' = None) -> 'tuple[float, float]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
camera	Camera | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
coordenada	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
viewport	Viewport | None	KEYWORD_ONLY	não	None

Retorno declarado

tuple[float, float]

`coral.jogos.normalizar_categorias_debug`

Normaliza categorias públicas do modo de debug persistente.

Assinatura

Assinatura para consulta

```
normalizar_categorias_debug(categorias: 'Any' = None) -> 'frozenset[str]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
categorias	Any	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

frozenset[str]

`coral.jogos.normalizar_progresso`

Normaliza progresso temporal; extrapolação só ocorre quando explícita.

Assinatura

Assinatura para consulta

```
normalizar_progresso(progresso: 'float', *, extrapolar: 'bool' = False) -> 'float'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
progresso	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
extrapolar	bool	KEYWORD_ONLY	não	False

Retorno declarado

float

`coral.jogos.onda_cosseno`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
onda_cosseno(fase: 'float') -> 'float'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
fase	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

float

`coral.jogos.onda_pulso`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
onda_pulso(fase: 'float') -> 'float'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
fase	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

float

`coral.jogos.onda_seno`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
onda_seno(fase: 'float') -> 'float'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
fase	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

float

`coral.jogos.onda_serra`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
onda_serra(fase: 'float') -> 'float'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
fase	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

float

`coral.jogos.onda_triangular`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
onda_triangular(fase: 'float') -> 'float'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
fase	float	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

float

`coral.jogos.paralelo`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
paralelo(*etapas: 'NoApresentacao', nome: 'str | None' = None) -> 'ParaleloApresentacao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
etapas	NoApresentacao	VAR_POSITIONAL	não	sem padrão
nome	str | None	KEYWORD_ONLY	não	None

Retorno declarado

ParaleloApresentacao

`coral.jogos.popular_cena_procedural`

Cria sprites procedurais e os adiciona a uma cena existente.

Assinatura

Assinatura para consulta

```
popular_cena_procedural(cena: 'Cena', *, pontos: 'Iterable[Sequence[float]] | None' =
  None, receita: 'ReceitaProcedural | Mapping[str, Any] | None' = None, camada: 'Any' =
  None, limite_custo: 'int' = 1000000, **opcoes_sprite: 'Any') -> 'tuple[Sprite, ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
cena	Cena	POSITIONAL_OR_KEYWORD	sim	sem padrão
pontos	Iterable[Sequence[float]] | None	KEYWORD_ONLY	não	None
receita	ReceitaProcedural | Mapping[str, Any] | None	KEYWORD_ONLY	não	None

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
camada	Any	KEYWORD_ONLY	não	None
limite_custo	int	KEYWORD_ONLY	não	1000000
opcoes_sprite	Any	VAR_KEYWORD	não	sem padrão

Retorno declarado

tuple[Sprite, ...]

`coral.jogos.posicao_espacial_sprite`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
posicao_espacial_sprite(sprite: 'Any')
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
sprite	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.jogos.pygame_disponivel`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
pygame_disponivel() -> 'bool'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

bool

`coral.jogos.registrar_propriedade_animavel`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
registrar_propriedade_animavel(tipo: 'type', nome: 'str', obter: 'Callable[[Any],
  float]', definir: 'Callable[[Any, float], None]') -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tipo	type	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
obter	Callable[[Any], float]	POSITIONAL_OR_KEYWORD	sim	sem padrão
definir	Callable[[Any, float], None]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

`coral.jogos.reproduzir_ao_contrario`

Reinicie uma animação ou clipe no modo reverso, quando suportado.

Assinatura

Assinatura para consulta

```
reproduzir_ao_contrario(animacao: 'Any') -> 'Any'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
animacao	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Any

`coral.jogos.resolver_curva_temporal`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
resolver_curva_temporal(curva: 'str | Callable[[float], float]') -> 'Callable[[float],
  float]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
curva	str | Callable[[float], float]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Callable[[float], float]

`coral.jogos.sequencia`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
sequencia(*etapas: 'NoApresentacao', nome: 'str | None' = None) ->
  'SequenciaApresentacao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
etapas	NoApresentacao	VAR_POSITIONAL	não	sem padrão
nome	str | None	KEYWORD_ONLY	não	None

Retorno declarado

SequenciaApresentacao

`coral.jogos.sprites_colidem`

Colisão 2D de Jogos usando sobreposição interior da baseline.

A escolha por `sobrepoe` preserva o contrato histórico de Jogos em que encostar apenas pelas bordas não conta como colisão.

Assinatura

Assinatura para consulta

```
sprites_colidem(a: 'Any', b: 'Any') -> 'bool'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
b	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

bool

`coral.jogos.sprites_de_pontos`

Converte pontos 2D determinísticos em sprites sem abrir janela.

Assinatura

Assinatura para consulta

```
sprites_de_pontos(pontos: 'Iterable[Sequence[float]]', *, largura: 'float' = 16.0,
  altura: 'float' = 16.0, imagem: 'Any' = None, cor: 'Cor' = Cor(vermelho=255, verde=255,
  azul=255, alfa=255), nome_base: 'str' = 'procedural', dados: 'Mapping[str, Any] | None'
  = None, campos: 'Mapping[str, CampoProcedural] | None' = None, fabrica: 'Callable[[int,
  float, float, dict[str, Any]], Sprite] | None' = None, limite_sprites: 'int' = 10000) ->
   'tuple[Sprite, ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
pontos	Iterable[Sequence[float]]	POSITIONAL_OR_KEYWORD	sim	sem padrão
largura	float	KEYWORD_ONLY	não	16.0
altura	float	KEYWORD_ONLY	não	16.0
imagem	Any	KEYWORD_ONLY	não	None
cor	Cor	KEYWORD_ONLY	não	Cor(vermelho=255, verde=255, azul=255, alfa=255)
nome_base	str	KEYWORD_ONLY	não	'procedural'
dados	Mapping[str, Any] | None	KEYWORD_ONLY	não	None
campos	Mapping[str, CampoProcedural] | None	KEYWORD_ONLY	não	None
fabrica	Callable[[int, float, float, dict[str, Any]], Sprite] | None	KEYWORD_ONLY	não	None
limite_sprites	int	KEYWORD_ONLY	não	10000

Retorno declarado

tuple[Sprite, ...]

`coral.jogos.sprites_de_receita`

Reconstrói uma receita de pontos e a adapta para sprites.

Assinatura

Assinatura para consulta

```
sprites_de_receita(receita: 'ReceitaProcedural | Mapping[str, Any]', *, limite_custo:
  'int' = 1000000, **opcoes_sprite: 'Any') -> 'tuple[Sprite, ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
receita	ReceitaProcedural | Mapping[str, Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
limite_custo	int	KEYWORD_ONLY	não	1000000
opcoes_sprite	Any	VAR_KEYWORD	não	sem padrão

Retorno declarado

tuple[Sprite, ...]

`coral.jogos.tela_para_celula`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
tela_para_celula(mapa: 'Mapa', camera: 'Camera | None', x: 'float', y: 'float', *,
  viewport: 'Viewport | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mapa	Mapa	POSITIONAL_OR_KEYWORD	sim	sem padrão
camera	Camera | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
x	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
viewport	Viewport | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.jogos.tela_para_mundo`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
tela_para_mundo(camera: 'Camera | None', x: 'float', y: 'float', *, viewport: 'Viewport
  | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
camera	Camera | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
x	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
viewport	Viewport | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.jogos.usar_fonte_tempo`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
usar_fonte_tempo(composicao: 'ComposicaoTemporal', fonte: 'Any') ->
  'VinculoTempoApresentacao'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
composicao	ComposicaoTemporal	POSITIONAL_OR_KEYWORD	sim	sem padrão
fonte	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

VinculoTempoApresentacao

Consultar a implementação canônica de coral.jogos

Implementação canônica de `coral.jogos`

Código Python

```
"""Contratos transversais pequenos do runtime Coral.

A camada comum não substitui APIs de domínio. Ela fornece a gramática
arquitetural compartilhada do ciclo 1.5 para erro operacional, resultado e
diagnóstico, preservando os nomes públicos existentes em cada biblioteca.
"""
from __future__ import annotations

from dataclasses import asdict, dataclass, field
from typing import Any, Iterable, Mapping

CONTRATO = "coral.comum/1"
CONTRATO_DIAGNOSTICO = "coral.diagnostico.operacional/1"


class ErroCoral(Exception):
    """Raiz pública dos erros controlados pela linguagem e bibliotecas Coral."""


class ErroOperacaoCoral(RuntimeError, ErroCoral):
    """Falha operacional de um recurso Coral ou de seu backend."""


class ErroDependenciaCoral(ErroOperacaoCoral):
    """Uma capacidade opcional não pode operar por ausência de dependência."""


@dataclass(frozen=True)
class FalhaOperacao:
    codigo: str
    mensagem: str
    dominio: str
    tipo: str = "operacional"
    causa: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def de_excecao(cls, exc: BaseException, *, codigo: str, dominio: str, tipo: str =
      "operacional", detalhe: Mapping[str, Any] | None = None) -> "FalhaOperacao":
        causa = exc.__cause__ if exc.__cause__ is not None else exc
        return cls(codigo, str(exc), dominio, tipo, type(causa).__name__, dict(detalhe
          or {}))

    def para_dict(self) -> dict[str, Any]:
        return asdict(self)


@dataclass(frozen=True)
class ResultadoOperacao:
    """Representação transversal sem substituir resultados ricos de domínio."""
    ok: bool
    valor: Any = None
    falha: FalhaOperacao | None = None
    metadados: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def sucesso(cls, valor: Any = None, **metadados: Any) -> "ResultadoOperacao":
        return cls(True, valor=valor, metadados=metadados)
```

```
    @classmethod
    def erro(cls, falha: FalhaOperacao, **metadados: Any) -> "ResultadoOperacao":
        return cls(False, falha=falha, metadados=metadados)

    def para_dict(self) -> dict[str, Any]:
        return {
            "contrato": CONTRATO,
            "ok": self.ok,
            "valor": self.valor,
            "falha": self.falha.para_dict() if self.falha is not None else None,
            "metadados": dict(self.metadados),
        }


@dataclass(frozen=True)
class DiagnosticoOperacional:
    id: str
    dominio: str
    ok: bool
    mensagem: str
    capacidade: str | None = None
    sugestao: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @property
    def nivel(self) -> str:
        return "info" if self.ok else "erro"

    def para_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "dominio": self.dominio,
            "ok": self.ok,
            "nivel": self.nivel,
            "mensagem": self.mensagem,
            "capacidade": self.capacidade,
            "sugestao": self.sugestao,
            "detalhe": dict(self.detalhe),
        }


def diagnostico(id: str, dominio: str, ok: bool, mensagem: Any, *, capacidade: str |
  None = None, sugestao: str | None = None, detalhe: Mapping[str, Any] | None = None) ->
  DiagnosticoOperacional:
    return DiagnosticoOperacional(str(id), str(dominio), bool(ok), str(mensagem),
      capacidade, sugestao, dict(detalhe or {}))


def agregar_diagnosticos(itens: Iterable[DiagnosticoOperacional | Mapping[str, Any]], *,
   origem: str = "coral") -> dict[str, Any]:
    normalizados: list[dict[str, Any]] = []
    for item in itens:
        if isinstance(item, DiagnosticoOperacional):
            normalizados.append(item.para_dict())
            continue
        d = dict(item)
        if "id" not in d:
            d["id"] = str(d.get("nome", "diagnostico"))
        if "dominio" not in d:
            d["dominio"] = origem
        d["ok"] = bool(d.get("ok"))
```

```
        d.setdefault("nivel", "info" if d["ok"] else "erro")
        d.setdefault("mensagem", str(d.get("detalhe", "")))
        d.setdefault("capacidade", None)
        d.setdefault("sugestao", None)
        detalhe = d.get("detalhe")
        if not isinstance(detalhe, Mapping):
            d["detalhe"] = {"valor": detalhe}
        normalizados.append(d)
    return {
        "contrato": CONTRATO_DIAGNOSTICO,
        "origem": origem,
        "ok": all(x["ok"] for x in normalizados),
        "quantidade": len(normalizados),
        "itens": normalizados,
    }


__all__ = [
    "CONTRATO", "CONTRATO_DIAGNOSTICO", "ErroCoral", "ErroOperacaoCoral",
    "ErroDependenciaCoral", "FalhaOperacao", "ResultadoOperacao",
    "DiagnosticoOperacional", "diagnostico", "agregar_diagnosticos",
]

from __future__ import annotations
from ..comum import diagnostico
from .objetos import Assets, Sprite, Cena, CamadaApresentacao, TextoApresentacao,
  RepresentacaoEntidade, ErroAssetCoral, Animacao, AnimacoesDirecionais,
  VinculoAnimacaoMovimento, EstadosAnimacao, VinculoEstadosAnimacaoMovimento,
  TransicaoEstadoAnimacao, CondicaoTransicaoAnimacao, TransicoesEstadosAnimacao,
  VinculoTransicoesEstadosMovimento, VinculoTransicoesEventos, EfeitoCamera, Camera,
  VinculoClipe, ligar_clipe, registrar_propriedade_animavel
from .spritesheet import QuadroSprite, Spritesheet, carregar_spritesheet
from .transformacoes import FormaTransformada, formas_colidem, CanalAnimado,
  AnimacaoTransformacao, onda_seno, onda_cosseno, onda_triangular, onda_serra, onda_pulso,
   normalizar_progresso, curva_linear, curva_entrada_suave, curva_saida_suave,
  curva_entrada_saida_suave, curva_quadratica, curva_cubica, curva_degrau,
  resolver_curva_temporal, avaliar_curva_temporal, interpolar, QuadroChave, FaixaAnimacao,
   ClipeAnimacao
from .espacial import forma_espacial_sprite, posicao_espacial_sprite, sprites_colidem
from .eventos import AdaptadorEventosJogo, adaptar_eventos
from .mapas import EstiloCelula, RenderizadorMapa2D, celula_para_tela, mundo_para_tela,
  tela_para_celula, tela_para_mundo
from .viewport import Viewport
from .debug import (
    CONTRATO_OVERLAY_DEBUG, PrimitivaDebug, OverlayDebug, EstiloOverlayDebug,
    ConfiguracaoDebugJogo, normalizar_categorias_debug, gerar_overlay_debug,
    gerar_overlay_metricas_debug, desenhar_overlay_debug,
)
from .base import (
    Jogo, TransicaoCena, MetricasJogo, Cor, Retangulo, BackendNulo, ErroJogosCoral,
      DependenciaJogosAusente,
    PRETO, BRANCO, VERMELHO, VERDE, AZUL, AMARELO,
)

from .objetos import animar_propriedade, reproduzir_ao_contrario
from .composicao import (
    AcaoApresentacao, EsperaApresentacao, EsperaConclusao, SequenciaApresentacao,
    ParaleloApresentacao, ComposicaoTemporal, VinculoTempoApresentacao,
    VinculoComposicaoEventos, ComandoComposicao, GravadorComposicao, executar,
    esperar, esperar_conclusao, sequencia, paralelo, usar_fonte_tempo, ligar_evento,
)
```

```
from . import persistencia_apresentacao as _persistencia_apresentacao

from .procedural import (
    sprites_de_pontos, sprites_de_receita, popular_cena_procedural,
      aplicar_campo_em_sprites,
)

def criar_jogo(titulo: str, largura: int = 800, altura: int = 450, fps: int = 60, *,
  backend=None, mundo=None, relogio=None, eventos=None, tela_cheia: bool = False) -> Jogo:
    if backend is None:
        from .pygame_backend import PygameBackend
        backend=PygameBackend()
    return Jogo(titulo, largura, altura, fps, backend=backend, mundo=mundo,
      relogio=relogio, eventos=eventos, tela_cheia=tela_cheia)


def pygame_disponivel() -> bool:
    try:
        import pygame  # noqa
        return True
    except ModuleNotFoundError:
        return False


def diagnosticar_jogos() -> dict[str, object]:
    disponivel = pygame_disponivel()
    mensagem = (
        "pygame-ce disponível para backend gráfico de Jogos."
        if disponivel
        else "pygame-ce não está disponível; BackendNulo continua funcional sem janela."
    )
    sugestao = None if disponivel else "Instale a capacidade Jogos pelo Coral Setup ou
      Gerenciador de Componentes."
    return {
        "disponivel": disponivel,
        "dependencia": "pygame-ce",
        "backend_nulo": True,
        "diagnostico": diagnostico(
            "jogos.pygame",
            "jogos",
            disponivel,
            mensagem,
            capacidade="pygame-ce",
            sugestao=sugestao,
            detalhe={"backend_nulo": True},
        ).para_dict(),
    }


__all__ = [
    'Jogo','TransicaoCena','MetricasJogo','Cor','Retangulo','BackendNulo','ErroJogosCora
      l','DependenciaJogosAusente',
    'PRETO','BRANCO','VERMELHO','VERDE','AZUL','AMARELO','criar_jogo','pygame_disponivel
      ','diagnosticar_jogos',
    'Assets','Sprite','Cena','CamadaApresentacao','TextoApresentacao','RepresentacaoEnti
      dade','ErroAssetCoral','Animacao','AnimacoesDirecionais','VinculoAnimacaoMovimento',
      'EstadosAnimacao','VinculoEstadosAnimacaoMovimento','TransicaoEstadoAnimacao','Condi
      caoTransicaoAnimacao','TransicoesEstadosAnimacao','VinculoTransicoesEstadosMovimento
      ','VinculoTransicoesEventos','EfeitoCamera','Camera','VinculoClipe','ligar_clipe','r
      egistrar_propriedade_animavel',
```

```
    'QuadroSprite','Spritesheet','carregar_spritesheet',
    'FormaTransformada','formas_colidem','CanalAnimado','AnimacaoTransformacao','onda_se
      no','onda_cosseno','onda_triangular','onda_serra','onda_pulso',
    'normalizar_progresso','curva_linear','curva_entrada_suave','curva_saida_suave','cur
      va_entrada_saida_suave','curva_quadratica','curva_cubica','curva_degrau','resolver_c
      urva_temporal','avaliar_curva_temporal','interpolar','QuadroChave','FaixaAnimacao','
      ClipeAnimacao',
    'AcaoApresentacao','EsperaApresentacao','EsperaConclusao','SequenciaApresentacao','P
      araleloApresentacao','ComposicaoTemporal','VinculoTempoApresentacao','VinculoComposi
      caoEventos','ComandoComposicao','GravadorComposicao','executar','esperar','esperar_c
      onclusao','sequencia','paralelo','usar_fonte_tempo','ligar_evento',
    'forma_espacial_sprite','posicao_espacial_sprite','sprites_colidem',
    'AdaptadorEventosJogo','adaptar_eventos','Viewport',
    'CONTRATO_OVERLAY_DEBUG','PrimitivaDebug','OverlayDebug','EstiloOverlayDebug','Confi
      guracaoDebugJogo','normalizar_categorias_debug','gerar_overlay_debug','gerar_overlay
      _metricas_debug','desenhar_overlay_debug',
    'EstiloCelula','RenderizadorMapa2D','celula_para_tela','mundo_para_tela','tela_para_
      celula','tela_para_mundo',
    'animar_propriedade','reproduzir_ao_contrario',
    'sprites_de_pontos','sprites_de_receita','popular_cena_procedural','aplicar_campo_em
      _sprites'
]

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Callable, Protocol, Any
import math
from ..comum import ErroDependenciaCoral
from ..regras import MotorRegras
from ..tempo_eventos import Eventos, FonteTempo, normalizar_dt, validar_duracao
from .spritesheet import QuadroSprite
from .viewport import Viewport
from .transformacoes import avaliar_curva_temporal


class ErroJogosCoral(Exception):
    pass


class DependenciaJogosAusente(ErroJogosCoral, ErroDependenciaCoral):
    pass


@dataclass(frozen=True)
class Cor:
    vermelho: int
    verde: int
    azul: int
    alfa: int = 255

    def __post_init__(self):
        for nome, valor in (("vermelho", self.vermelho), ("verde", self.verde), ("azul",
           self.azul), ("alfa", self.alfa)):
            if not 0 <= int(valor) <= 255:
                raise ValueError(f"{nome} deve estar entre 0 e 255")

    def rgb(self) -> tuple[int, int, int]:
        return (self.vermelho, self.verde, self.azul)
```

```
PRETO = Cor(0, 0, 0)
BRANCO = Cor(255, 255, 255)
VERMELHO = Cor(220, 50, 47)
VERDE = Cor(46, 160, 67)
AZUL = Cor(38, 139, 210)
AMARELO = Cor(255, 215, 0)


_ORIGENS_IMAGEM = {"topo_esquerdo", "centro", "centro_inferior"}


def _normalizar_opacidade(opacidade: float = 1.0) -> float:
    if isinstance(opacidade, bool):
        raise ValueError("opacidade deve ser um número finito entre 0 e 1")
    try:
        valor = float(opacidade)
    except (TypeError, ValueError) as exc:
        raise ValueError("opacidade deve ser um número finito entre 0 e 1") from exc
    if not math.isfinite(valor) or not 0.0 <= valor <= 1.0:
        raise ValueError("opacidade deve ser um número finito entre 0 e 1")
    return valor


def _normalizar_origem_desenho(origem: str = "topo_esquerdo") -> str:
    valor = str(origem).strip().casefold().replace(" ", "_")
    if valor not in _ORIGENS_IMAGEM:
        opcoes = ", ".join(sorted(_ORIGENS_IMAGEM))
        raise ValueError(f"origem inválida; use uma de: {opcoes}")
    return valor


def _normalizar_transformacao_imagem(
    *,
    escala: float = 1.0,
    rotacao: float = 0.0,
    espelhar_horizontalmente: bool = False,
    espelhar_verticalmente: bool = False,
    origem: str = "topo_esquerdo",
    pivo: tuple[float, float] | None = None,
) -> tuple[float, float, bool, bool, str, tuple[float, float] | None]:
    if isinstance(escala, bool):
        raise ValueError("escala da imagem deve ser um número finito positivo")
    try:
        escala_normalizada = float(escala)
    except (TypeError, ValueError) as exc:
        raise ValueError("escala da imagem deve ser um número finito positivo") from exc
    if not math.isfinite(escala_normalizada) or escala_normalizada <= 0:
        raise ValueError("escala da imagem deve ser um número finito positivo")

    if isinstance(rotacao, bool):
        raise ValueError("rotação da imagem deve ser um número finito em graus")
    try:
        rotacao_normalizada = float(rotacao)
    except (TypeError, ValueError) as exc:
        raise ValueError("rotação da imagem deve ser um número finito em graus") from
          exc
    if not math.isfinite(rotacao_normalizada):
        raise ValueError("rotação da imagem deve ser um número finito em graus")

    origem_normalizada = str(origem).strip().casefold().replace(" ", "_")
    if origem_normalizada not in _ORIGENS_IMAGEM:
```

```
        opcoes = ", ".join(sorted(_ORIGENS_IMAGEM))
        raise ValueError(f"origem da imagem inválida; use uma de: {opcoes}")

    pivo_normalizado = None
    if pivo is not None:
        try:
            pivo_x, pivo_y = pivo
        except (TypeError, ValueError) as exc:
            raise ValueError("pivô da imagem deve conter exatamente dois números
              finitos") from exc
        try:
            pivo_x = float(pivo_x)
            pivo_y = float(pivo_y)
        except (TypeError, ValueError) as exc:
            raise ValueError("pivô da imagem deve conter exatamente dois números
              finitos") from exc
        if not math.isfinite(pivo_x) or not math.isfinite(pivo_y):
            raise ValueError("pivô da imagem deve conter exatamente dois números
              finitos")
        pivo_normalizado = (pivo_x, pivo_y)

    return (
        escala_normalizada,
        rotacao_normalizada,
        bool(espelhar_horizontalmente),
        bool(espelhar_verticalmente),
        origem_normalizada,
        pivo_normalizado,
    )


def _transformacao_imagem_neutra(
    escala: float,
    rotacao: float,
    espelhar_horizontalmente: bool,
    espelhar_verticalmente: bool,
    origem: str,
    pivo: tuple[float, float] | None,
) -> bool:
    return (
        escala == 1.0
        and rotacao == 0.0
        and not espelhar_horizontalmente
        and not espelhar_verticalmente
        and origem == "topo_esquerdo"
        and pivo is None
    )


@dataclass
class Retangulo:
    x: float
    y: float
    largura: float
    altura: float

    def colide_com(self, outro: "Retangulo") -> bool:
        return not (
            self.x + self.largura <= outro.x or
            outro.x + outro.largura <= self.x or
            self.y + self.altura <= outro.y or
```

```
            outro.y + outro.altura <= self.y
        )


class BackendJogos(Protocol):
    def iniciar(self, titulo: str, largura: int, altura: int, *, tela_cheia: bool =
      False) -> None: ...
    def processar_eventos(self) -> bool: ...
    def tecla_pressionada(self, tecla: str) -> bool: ...
    def posicao_mouse(self) -> tuple[int, int]: ...
    def botao_mouse_pressionado(self, botao: str) -> bool: ...
    def mouse_clicado(self, botao: str) -> bool: ...
    def mouse_solto(self, botao: str) -> bool: ...
    def roda_mouse(self) -> tuple[int, int]: ...
    def definir_tela_cheia(self, ativa: bool) -> None: ...
    def definir_tamanho_janela(self, largura: int, altura: int) -> None: ...
    def tamanho_fisico(self) -> tuple[int, int]: ...
    def definir_recorte(self, recorte: Retangulo | None) -> None: ...
    def limpar(self, cor: Cor) -> None: ...
    def desenhar_retangulo(self, retangulo: Retangulo, cor: Cor, *, contorno: int = 0,
      opacidade: float = 1.0) -> None: ...
    def desenhar_circulo(self, x: float, y: float, raio: float, cor: Cor, *, contorno:
      int = 0, opacidade: float = 1.0) -> None: ...
    def desenhar_linha(self, x1: float, y1: float, x2: float, y2: float, cor: Cor, *,
      espessura: int = 1, opacidade: float = 1.0) -> None: ...
    def desenhar_poligono(self, vertices, cor: Cor, *, contorno: int = 1, opacidade:
      float = 1.0) -> None: ...
    def desenhar_ponto(self, x: float, y: float, cor: Cor, *, tamanho: int = 1,
      opacidade: float = 1.0) -> None: ...
    def desenhar_texto(self, texto: str, x: float, y: float, tamanho: int, cor: Cor, *,
      origem: str = "topo_esquerdo", opacidade: float = 1.0) -> None: ...
    def medir_texto(self, texto: str, tamanho: int) -> tuple[float, float]: ...
    def desenhar_imagem(
        self,
        caminho: str | QuadroSprite,
        x: float,
        y: float,
        largura: float | None = None,
        altura: float | None = None,
        *,
        escala: float = 1.0,
        rotacao: float = 0.0,
        espelhar_horizontalmente: bool = False,
        espelhar_verticalmente: bool = False,
        origem: str = "topo_esquerdo",
        pivo: tuple[float, float] | None = None,
        opacidade: float = 1.0,
    ) -> None: ...
    def apresentar(self) -> None: ...
    def tocar_som(self, caminho: str, *, volume: float = 1.0) -> None: ...
    def limitar_fps(self, fps: int) -> float: ...
    def encerrar(self) -> None: ...


class BackendNulo:
    """Backend determinístico para testes, servidores e execução sem interface
      gráfica."""
    def __init__(self, *, dt_fixo: float | None = None):
        if dt_fixo is not None:
            try:
                dt_fixo = validar_duracao(dt_fixo, "dt_fixo")
```

```
            except ValueError as exc:
                raise ValueError("dt_fixo deve ser um número finito maior ou igual a
                  zero") from exc
        self.comandos: list[tuple[str, tuple[Any, ...]]] = []
        self.quadros = 0
        self.ativo = True
        self.teclas: set[str] = set()
        self.iniciado = False
        self.dt_fixo = None if dt_fixo is None else float(dt_fixo)
        self._roteiro_teclas: list[set[str]] = []
        self._posicao_mouse = (0, 0)
        self._botoes_mouse: set[str] = set()
        self._cliques_mouse: set[str] = set()
        self._solturas_mouse: set[str] = set()
        self._roda_mouse = (0, 0)
        self._roteiro_mouse: list[dict[str, Any]] = []
        self._tela_cheia = False
        self._tamanho_fisico = (800, 450)
        self._recorte: Retangulo | None = None
        self._quadro_processado = 0

    def definir_teclas(self, *teclas: str) -> None:
        """Substitui o estado atual de teclado do backend sem janela."""
        self.teclas = {str(tecla).casefold() for tecla in teclas}

    def programar_teclas(self, *quadros) -> None:
        """Programa estados de teclado por quadro para testes determinísticos.

        Cada item pode ser um iterável de nomes de teclas. O primeiro estado é
        aplicado no primeiro quadro processado, o segundo no próximo e assim por diante.
        """
        self._roteiro_teclas = [{str(t).casefold() for t in quadro} for quadro in
          quadros]
        self._quadro_processado = 0

    def definir_mouse(
        self,
        *,
        posicao: tuple[int, int] = (0, 0),
        pressionados=(),
        clicados=(),
        soltos=(),
        roda: tuple[int, int] = (0, 0),
    ) -> None:
        self._posicao_mouse = _normalizar_posicao_mouse(posicao)
        self._botoes_mouse = {_normalizar_botao_mouse(botao) for botao in pressionados}
        self._cliques_mouse = {_normalizar_botao_mouse(botao) for botao in clicados}
        self._solturas_mouse = {_normalizar_botao_mouse(botao) for botao in soltos}
        self._roda_mouse = _normalizar_roda_mouse(roda)

    def programar_mouse(self, *quadros: dict[str, Any]) -> None:
        self._roteiro_mouse = [dict(quadro) for quadro in quadros]
        self._quadro_processado = 0

    def iniciar(self, titulo: str, largura: int, altura: int, *, tela_cheia: bool =
      False) -> None:
        # Um backend encerrado pode ser reutilizado em outra execução do mesmo jogo.
        self.ativo = True
        self.iniciado = True
        self._tela_cheia = bool(tela_cheia)
        self._tamanho_fisico = (int(largura), int(altura))
```

```
        self._quadro_processado = 0
        argumentos = (titulo, largura, altura, True) if tela_cheia else (titulo,
          largura, altura)
        self.comandos.append(("iniciar", argumentos))

    def processar_eventos(self) -> bool:
        if not self.ativo:
            return False
        if self._quadro_processado < len(self._roteiro_teclas):
            self.teclas = set(self._roteiro_teclas[self._quadro_processado])
        if self._quadro_processado < len(self._roteiro_mouse):
            quadro = self._roteiro_mouse[self._quadro_processado]
            self.definir_mouse(
                posicao=quadro.get("posicao", self._posicao_mouse),
                pressionados=quadro.get("pressionados", ()),
                clicados=quadro.get("clicados", ()),
                soltos=quadro.get("soltos", ()),
                roda=quadro.get("roda", (0, 0)),
            )
        else:
            self._cliques_mouse.clear()
            self._solturas_mouse.clear()
            self._roda_mouse = (0, 0)
        self._quadro_processado += 1
        return True

    def tecla_pressionada(self, tecla: str) -> bool:
        return tecla.casefold() in {t.casefold() for t in self.teclas}

    def posicao_mouse(self) -> tuple[int, int]:
        return self._posicao_mouse

    def botao_mouse_pressionado(self, botao: str) -> bool:
        return _normalizar_botao_mouse(botao) in self._botoes_mouse

    def mouse_clicado(self, botao: str) -> bool:
        return _normalizar_botao_mouse(botao) in self._cliques_mouse

    def mouse_solto(self, botao: str) -> bool:
        return _normalizar_botao_mouse(botao) in self._solturas_mouse

    def roda_mouse(self) -> tuple[int, int]:
        return self._roda_mouse

    def definir_tela_cheia(self, ativa: bool) -> None:
        self._tela_cheia = bool(ativa)
        self.comandos.append(("tela_cheia", (self._tela_cheia,)))

    def definir_tamanho_janela(self, largura: int, altura: int) -> None:
        largura, altura = _normalizar_tamanho_janela(largura, altura)
        self._tamanho_fisico = (largura, altura)
        self.comandos.append(("redimensionar", self._tamanho_fisico))

    def tamanho_fisico(self) -> tuple[int, int]:
        return self._tamanho_fisico

    def definir_recorte(self, recorte: Retangulo | None) -> None:
        if recorte is not None and not isinstance(recorte, Retangulo):
            raise TypeError("recorte precisa ser Retangulo ou None")
        self._recorte = recorte
        self.comandos.append(("recorte", (recorte,)))
```

```
    def limpar(self, cor: Cor) -> None:
        self.comandos.append(("limpar", (cor,)))

    def desenhar_retangulo(self, retangulo: Retangulo, cor: Cor, *, contorno: int = 0,
      opacidade: float = 1.0) -> None:
        opacidade = _normalizar_opacidade(opacidade)
        argumentos = (retangulo, cor, contorno) if opacidade == 1.0 else (retangulo,
          cor, contorno, opacidade)
        self.comandos.append(("retangulo", argumentos))

    def desenhar_circulo(self, x: float, y: float, raio: float, cor: Cor, *, contorno:
      int = 0, opacidade: float = 1.0) -> None:
        opacidade = _normalizar_opacidade(opacidade)
        argumentos = (x, y, raio, cor, contorno) if opacidade == 1.0 else (x, y, raio,
          cor, contorno, opacidade)
        self.comandos.append(("circulo", argumentos))

    def desenhar_linha(self, x1: float, y1: float, x2: float, y2: float, cor: Cor, *,
      espessura: int = 1, opacidade: float = 1.0) -> None:
        self.comandos.append(("linha", (x1, y1, x2, y2, cor, int(espessura),
          _normalizar_opacidade(opacidade))))

    def desenhar_poligono(self, vertices, cor: Cor, *, contorno: int = 1, opacidade:
      float = 1.0) -> None:
        pontos = tuple((float(x), float(y)) for x, y in vertices)
        if len(pontos) < 2:
            raise ValueError("um polígono precisa de ao menos dois vértices")
        if isinstance(contorno, bool) or int(contorno) < 0:
            raise ValueError("contorno do polígono deve ser inteiro maior ou igual a
              zero")
        self.comandos.append((
            "poligono",
            (pontos, cor, int(contorno), _normalizar_opacidade(opacidade)),
        ))

    def desenhar_ponto(self, x: float, y: float, cor: Cor, *, tamanho: int = 1,
      opacidade: float = 1.0) -> None:
        self.comandos.append(("ponto", (x, y, cor, int(tamanho),
          _normalizar_opacidade(opacidade))))

    def desenhar_texto(self, texto: str, x: float, y: float, tamanho: int, cor: Cor, *,
      origem: str = "topo_esquerdo", opacidade: float = 1.0) -> None:
        origem = _normalizar_origem_desenho(origem)
        opacidade = _normalizar_opacidade(opacidade)
        argumentos = (texto, x, y, tamanho, cor) if origem == "topo_esquerdo" and
          opacidade == 1.0 else (texto, x, y, tamanho, cor, origem, opacidade)
        self.comandos.append(("texto", argumentos))

    def medir_texto(self, texto: str, tamanho: int) -> tuple[float, float]:
        if isinstance(tamanho, bool) or int(tamanho) <= 0:
            raise ValueError("tamanho do texto deve ser um inteiro positivo")
        tamanho = int(tamanho)
        # Métrica determinística para headless. A razão 0,6 é deliberadamente
        # simples: serve ao layout previsível sem fingir uma fonte específica.
        return (len(str(texto)) * tamanho * 0.6, float(tamanho))

    def desenhar_imagem(
        self,
        caminho: str | QuadroSprite,
        x: float,
```

```
        y: float,
        largura: float | None = None,
        altura: float | None = None,
        *,
        escala: float = 1.0,
        rotacao: float = 0.0,
        espelhar_horizontalmente: bool = False,
        espelhar_verticalmente: bool = False,
        origem: str = "topo_esquerdo",
        pivo: tuple[float, float] | None = None,
        opacidade: float = 1.0,
    ) -> None:
        opacidade = _normalizar_opacidade(opacidade)
        transformacao = _normalizar_transformacao_imagem(
            escala=escala,
            rotacao=rotacao,
            espelhar_horizontalmente=espelhar_horizontalmente,
            espelhar_verticalmente=espelhar_verticalmente,
            origem=origem,
            pivo=pivo,
        )
        if _transformacao_imagem_neutra(*transformacao):
            if isinstance(caminho, QuadroSprite):
                argumentos = (str(caminho.caminho), caminho.retangulo, x, y, largura,
                  altura)
                if opacidade != 1.0:
                    argumentos = (*argumentos, opacidade)
                self.comandos.append(("imagem_quadro", argumentos))
                return
            argumentos = (str(caminho), x, y, largura, altura)
            if opacidade != 1.0:
                argumentos = (*argumentos, opacidade)
            self.comandos.append(("imagem", argumentos))
            return

        comando = "imagem_quadro_transformada" if isinstance(caminho, QuadroSprite) else
           "imagem_transformada"
        origem_imagem = (str(caminho.caminho), caminho.retangulo) if isinstance(caminho,
           QuadroSprite) else str(caminho)
        argumentos = (origem_imagem, x, y, largura, altura, *transformacao)
        if opacidade != 1.0:
            argumentos = (*argumentos, opacidade)
        self.comandos.append((comando, argumentos))

    def apresentar(self) -> None:
        self.quadros += 1
        self.comandos.append(("apresentar", ()))

    def tocar_som(self, caminho: str, *, volume: float = 1.0) -> None:
        self.comandos.append(("som", (str(caminho), float(volume))))

    def limitar_fps(self, fps: int) -> float:
        return self.dt_fixo if self.dt_fixo is not None else 1.0 / max(1, fps)

    def encerrar(self) -> None:
        self.ativo = False
        self.comandos.append(("encerrar", ()))


@dataclass
class TransicaoCena:
```

```
    """Coordena apresentação temporal entre duas cenas sem possuir o jogo.

    O objeto é headless e determinístico. A troca efetiva continua sendo feita
    por ``Jogo.mudar_cena`` quando o progresso cruza o ponto de troca.
    """

    origem: Any
    destino: Any
    tipo: str = "desaparecer"
    duracao: float = 0.4
    curva: str | Callable[[float], float] = "suave"
    bloquear_entrada: bool = True
    atualizar_anterior: bool = True
    atualizar_proxima: bool = True
    direcao: str = "direita"
    tempo: float = 0.0
    estado: str = "ativo"
    _trocada: bool = field(default=False, init=False, repr=False)
    _callbacks_conclusao: list[Callable] = field(default_factory=list, init=False,
      repr=False)

    def __post_init__(self) -> None:
        self.tipo = str(self.tipo).strip().casefold().replace(" ", "_")
        aliases = {
            "fade": "desaparecer",
            "fade_out": "desaparecer",
            "fade_in": "aparecer",
            "slide": "deslizar",
            "reveal": "revelar",
        }
        self.tipo = aliases.get(self.tipo, self.tipo)
        permitidos = {"desaparecer", "aparecer", "escurecer", "clarear", "deslizar",
          "revelar", "personalizada"}
        if self.tipo not in permitidos:
            raise ValueError(f"tipo de transição de cena inválido: {self.tipo!r}")
        self.duracao = validar_duracao(self.duracao, "duração da transição")
        if self.duracao <= 0:
            raise ValueError("duração da transição deve ser positiva")
        self.tempo = validar_duracao(self.tempo, "tempo da transição")
        self.direcao = str(self.direcao).strip().casefold()
        if self.direcao not in {"direita", "esquerda", "cima", "baixo"}:
            raise ValueError("direção da transição deve ser direita, esquerda, cima ou
              baixo")
        self.estado = str(self.estado).strip().casefold()
        if self.estado not in {"ativo", "pausado", "cancelado", "finalizado"}:
            raise ValueError("estado inválido da transição de cena")
        # Valida a curva sem depender de backend gráfico.
        avaliar_curva_temporal(self.curva, 0.0)

    @property
    def progresso(self) -> float:
        return min(1.0, max(0.0, self.tempo / self.duracao))

    @property
    def progresso_curvo(self) -> float:
        return avaliar_curva_temporal(self.curva, self.progresso)

    @property
    def ativa(self) -> bool:
        return self.estado in {"ativo", "pausado"}
```

```
    @property
    def finalizada(self) -> bool:
        return self.estado == "finalizado"

    @property
    def fase(self) -> str:
        if self.finalizada:
            return "concluida"
        return "saida" if self.progresso < 0.5 else "entrada"

    @property
    def entrada_bloqueada(self) -> bool:
        return bool(self.bloquear_entrada and self.ativa)

    def ao_concluir(self, callback: Callable) -> Callable:
        if not callable(callback):
            raise TypeError("callback de conclusão precisa ser chamável")
        self._callbacks_conclusao.append(callback)
        return callback

    def pausar(self) -> "TransicaoCena":
        if self.estado == "ativo":
            self.estado = "pausado"
        return self

    def retomar(self) -> "TransicaoCena":
        if self.estado == "pausado":
            self.estado = "ativo"
        return self

    def cancelar(self) -> "TransicaoCena":
        if self.estado not in {"cancelado", "finalizado"}:
            self.estado = "cancelado"
        return self

    def avancar(self, dt: float) -> bool:
        """Avança e informa se a troca de cena deve ocorrer neste passo."""
        dt = validar_duracao(dt, "dt da transição")
        if self.estado != "ativo":
            return False
        anterior = self.progresso
        self.tempo = min(self.duracao, self.tempo + dt)
        atual = self.progresso
        trocar = not self._trocada and anterior < 0.5 <= atual
        if trocar:
            self._trocada = True
        if self.tempo >= self.duracao:
            self.estado = "finalizado"
            for callback in tuple(self._callbacks_conclusao):
                callback(self)
        return trocar

    def amostra(self) -> dict[str, Any]:
        """Estado visual independente de backend para renderers e testes."""
        p = self.progresso_curvo
        triangular = 1.0 - abs(2.0 * p - 1.0)
        if self.tipo == "aparecer":
            opacidade = 1.0 - p
        elif self.tipo == "desaparecer":
            opacidade = triangular
        elif self.tipo in {"escurecer", "clarear"}:
```

```
            opacidade = triangular
        else:
            opacidade = 0.0
        eixo_x = 1.0 if self.direcao == "direita" else -1.0 if self.direcao ==
          "esquerda" else 0.0
        eixo_y = 1.0 if self.direcao == "baixo" else -1.0 if self.direcao == "cima" else
           0.0
        if p < 0.5:
            fase_local = p * 2.0
            deslocamento_saida = (eixo_x * -fase_local, eixo_y * -fase_local)
            deslocamento_entrada = (eixo_x, eixo_y)
            revelacao = 0.0
        else:
            fase_local = (p - 0.5) * 2.0
            deslocamento_saida = (eixo_x * -1.0, eixo_y * -1.0)
            deslocamento_entrada = (eixo_x * (1.0 - fase_local), eixo_y * (1.0 -
              fase_local))
            revelacao = fase_local
        return {
            "tipo": self.tipo,
            "progresso": p,
            "progresso_fase": fase_local,
            "fase": self.fase,
            "opacidade": opacidade,
            "cor": BRANCO if self.tipo == "clarear" else PRETO,
            "deslocamento_saida": deslocamento_saida,
            "deslocamento_entrada": deslocamento_entrada,
            "revelacao": revelacao,
        }


@dataclass
class MetricasJogo:
    """Métricas operacionais leves e determinísticas do laço principal."""

    quadros: int = 0
    delta_logico: float = 0.0
    tempo_quadro: float = 0.0
    fps_instantaneo: float = 0.0
    fps_medio: float = 0.0

    @property
    def tempo_quadro_ms(self) -> float:
        return self.tempo_quadro * 1000.0

    def registrar(self, delta_logico: float, tempo_quadro: float) -> None:
        delta = normalizar_dt(delta_logico)
        tempo = normalizar_dt(tempo_quadro)
        fps = 0.0 if tempo <= 0.0 else 1.0 / tempo
        self.quadros += 1
        self.delta_logico = delta
        self.tempo_quadro = tempo
        self.fps_instantaneo = fps
        if self.quadros == 1:
            self.fps_medio = fps
        else:
            self.fps_medio = self.fps_medio * 0.9 + fps * 0.1

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "quadros": self.quadros,
```

```
            "delta_logico": self.delta_logico,
            "tempo_quadro": self.tempo_quadro,
            "tempo_quadro_ms": self.tempo_quadro_ms,
            "fps_instantaneo": self.fps_instantaneo,
            "fps_medio": self.fps_medio,
        }


class Jogo:
    def __init__(self, titulo: str, largura: int = 800, altura: int = 450, fps: int =
      60, *, backend: BackendJogos, mundo=None, relogio: FonteTempo | None = None,
      eventos: Eventos | None = None, tela_cheia: bool = False):
        if largura <= 0 or altura <= 0:
            raise ValueError("largura e altura devem ser positivas")
        if fps <= 0:
            raise ValueError("fps deve ser positivo")
        self.titulo = titulo
        self.largura = int(largura)
        self.altura = int(altura)
        self.fps = int(fps)
        self._tela_cheia = bool(tela_cheia)
        self.backend = backend
        self.rodando = False
        self._ao_iniciar: list[Callable[["Jogo"], Any]] = []
        self._ao_atualizar: list[Callable[["Jogo", float], Any]] = []
        self._ao_desenhar: list[Callable[["Jogo"], Any]] = []
        self._ao_encerrar: list[Callable[["Jogo"], Any]] = []
        self.cena_atual = None
        self._transicao_cena: TransicaoCena | None = None
        self._deslocamento_apresentacao_x = 0.0
        self._deslocamento_apresentacao_y = 0.0
        self._recorte_apresentacao: Retangulo | None = None
        self.mundo = None
        self.relogio = relogio
        self.eventos = eventos if eventos is not None else Eventos()
        self.regras = MotorRegras(relogio=relogio, eventos=self.eventos)
        self.metricas = MetricasJogo()
        from .debug import ConfiguracaoDebugJogo
        self._debug = ConfiguracaoDebugJogo()
        self._historico_inspecao = None
        self._tempo_inspecao = 0.0
        if mundo is not None:
            self.usar_mundo(mundo)

    def estado_inspecao(self) -> dict[str, Any]:
        transicao = self._transicao_cena
        return {
            "titulo": self.titulo,
            "rodando": self.rodando,
            "fps_alvo": self.fps,
            "tamanho_logico": (self.largura, self.altura),
            "tamanho_fisico": self.tamanho_fisico(),
            "tela_cheia": self._tela_cheia,
            "cena_atual": getattr(self.cena_atual, "nome", None),
            "cena": self.cena_atual,
            "transicao_cena": None if transicao is None else transicao.amostra(),
            "mundo": self.mundo,
            "regras": self.regras,
            "eventos": self.eventos,
            "metricas": self.metricas,
            "debug": self._debug,
```

```
            "observabilidade_ativa": self.observabilidade_ativa(),
            "monitor_inspecao": (
                None
                if self._historico_inspecao is None
                else self._historico_inspecao.estado_inspecao()
            ),
        }

    def estado_debug(self) -> dict[str, Any]:
        return self._debug.estado_inspecao()

    def observabilidade_ativa(self) -> bool:
        """Indica se há trabalho opcional de inspeção ou overlay por quadro."""
        return bool(self._debug.ativo or self._historico_inspecao is not None)

    def debug_ativo(self) -> bool:
        return bool(self._debug.ativo)

    def configurar_debug(
        self,
        *,
        ativo: bool | None = None,
        categorias=None,
        incluir_invisiveis: bool | None = None,
        estilo=None,
    ):
        self._debug = self._debug.com_alteracoes(
            ativo=ativo,
            categorias=categorias,
            incluir_invisiveis=incluir_invisiveis,
            estilo=estilo,
        )
        return self._debug

    def ativar_debug(
        self,
        *categorias: str,
        incluir_invisiveis: bool | None = None,
        estilo=None,
    ):
        novas_categorias = None if not categorias else categorias
        return self.configurar_debug(
            ativo=True,
            categorias=novas_categorias,
            incluir_invisiveis=incluir_invisiveis,
            estilo=estilo,
        )

    def desativar_debug(self):
        return self.configurar_debug(ativo=False)

    def alternar_debug(self, *categorias: str) -> bool:
        if categorias:
            self.configurar_debug(categorias=categorias)
        self.configurar_debug(ativo=not self._debug.ativo)
        return self._debug.ativo

    def desenhar_debug(self):
        """Desenha o modo persistente atual e devolve o overlay combinado."""
        from .debug import (
            OverlayDebug,
```

```
            desenhar_overlay_debug,
            gerar_overlay_debug,
            gerar_overlay_metricas_debug,
        )

        if not self._debug.ativo:
            return OverlayDebug()

        categorias = self._debug.categorias
        objeto = self.cena_atual if self.cena_atual is not None else self
        cena_overlay = gerar_overlay_debug(
            objeto,
            limites="limites" in categorias,
            colisores="colisor" in categorias,
            camera="camera" in categorias,
            pivos="pivo" in categorias,
            rotulos="rotulo" in categorias,
            incluir_invisiveis=self._debug.incluir_invisiveis,
        )
        primitivas = list(cena_overlay.primitivas)
        if "metricas" in categorias:
            primitivas.extend(gerar_overlay_metricas_debug(self.metricas).primitivas)

        combinado = OverlayDebug(
            primitivas=tuple(primitivas),
            assinatura_origem=cena_overlay.assinatura_origem,
        )
        camera = getattr(self.cena_atual, "camera", None) if self.cena_atual is not None
           else None
        return desenhar_overlay_debug(
            self,
            combinado,
            camera=camera,
            estilo=self._debug.estilo,
        )

    def monitorar_inspecao(
        self,
        *,
        max_amostras: int = 120,
        intervalo: float = 0.1,
        somente_mudancas: bool = True,
    ):
        """Ativa histórico curto de observabilidade no próprio laço do jogo."""
        from ..inspecao import HistoricoInspecao

        self._historico_inspecao = HistoricoInspecao(
            max_amostras=max_amostras,
            intervalo=intervalo,
            somente_mudancas=somente_mudancas,
            nome_alvo=self.titulo,
        )
        self._tempo_inspecao = 0.0
        return self._historico_inspecao

    def parar_monitoramento_inspecao(self):
        anterior = self._historico_inspecao
        self._historico_inspecao = None
        return anterior

    def historico_inspecao(self):
```

```
        return self._historico_inspecao

    def quando_iniciar(self, funcao: Callable[["Jogo"], Any]):
        self._ao_iniciar.append(funcao); return funcao

    def quando_atualizar(self, funcao: Callable[["Jogo", float], Any]):
        self._ao_atualizar.append(funcao); return funcao

    def quando_desenhar(self, funcao: Callable[["Jogo"], Any]):
        self._ao_desenhar.append(funcao); return funcao

    def quando_encerrar(self, funcao: Callable[["Jogo"], Any]):
        self._ao_encerrar.append(funcao); return funcao

    def parar(self) -> None:
        self.rodando = False

    def remover_mundo(self):
        """Desassocia o Mundo atual e remove somente a propagação criada pelo Jogo."""
        anterior = self.mundo
        if anterior is not None and anterior.regras.eventos.pai is self.eventos:
            anterior.regras.eventos.definir_pai(None)
        self.mundo = None
        return anterior

    def usar_mundo(self, mundo):
        """Associa um Mundo ao jogo e propaga eventos do Mundo para o Jogo."""
        if mundo is self.mundo:
            return mundo
        self.remover_mundo()
        self.mundo = mundo
        if mundo is not None:
            mundo.regras.eventos.definir_pai(self.eventos)
        return mundo

    def tecla_pressionada(self, tecla: str) -> bool:
        if self.entrada_bloqueada_por_transicao():
            return False
        return self.backend.tecla_pressionada(tecla)

    def posicao_mouse(self) -> tuple[int, int]:
        return self.backend.posicao_mouse()

    def tamanho_logico(self) -> tuple[int, int]:
        return self.largura, self.altura

    def tamanho_fisico(self) -> tuple[int, int]:
        if hasattr(self.backend, "tamanho_fisico"):
            largura, altura = self.backend.tamanho_fisico()
            return int(largura), int(altura)
        return self.tamanho_logico()

    def viewport(self, *, modo: str = "ajustar") -> Viewport:
        largura_fisica, altura_fisica = self.tamanho_fisico()
        return Viewport(self.largura, self.altura, largura_fisica, altura_fisica,
          modo=modo)

    def posicao_mouse_logica(self, *, limitar: bool = False, modo: str = "ajustar") ->
      tuple[float, float]:
        x, y = self.posicao_mouse()
        return self.viewport(modo=modo).tela_para_logica(x, y, limitar=limitar)
```

```
    def redimensionar_janela(self, largura: int, altura: int) -> tuple[int, int]:
        largura, altura = _normalizar_tamanho_janela(largura, altura)
        if not hasattr(self.backend, "definir_tamanho_janela"):
            raise ErroJogosCoral("o backend atual não oferece redimensionamento de
              janela")
        self.backend.definir_tamanho_janela(largura, altura)
        return self.tamanho_fisico()

    def botao_mouse_pressionado(self, botao: str = "esquerdo") -> bool:
        if self.entrada_bloqueada_por_transicao():
            return False
        return self.backend.botao_mouse_pressionado(_normalizar_botao_mouse(botao))

    def mouse_clicado(self, botao: str = "esquerdo") -> bool:
        if self.entrada_bloqueada_por_transicao():
            return False
        return self.backend.mouse_clicado(_normalizar_botao_mouse(botao))

    def mouse_solto(self, botao: str = "esquerdo") -> bool:
        if self.entrada_bloqueada_por_transicao():
            return False
        return self.backend.mouse_solto(_normalizar_botao_mouse(botao))

    def roda_mouse(self) -> tuple[int, int]:
        if self.entrada_bloqueada_por_transicao():
            return (0, 0)
        return self.backend.roda_mouse()

    def em_tela_cheia(self) -> bool:
        return self._tela_cheia

    def definir_tela_cheia(self, ativa: bool) -> None:
        ativa = bool(ativa)
        if ativa == self._tela_cheia:
            return
        if not hasattr(self.backend, "definir_tela_cheia"):
            raise ErroJogosCoral("o backend atual não oferece alternância de tela
              cheia")
        self.backend.definir_tela_cheia(ativa)
        self._tela_cheia = ativa

    def alternar_tela_cheia(self) -> bool:
        self.definir_tela_cheia(not self._tela_cheia)
        return self._tela_cheia

    def limpar(self, cor: Cor = PRETO) -> None:
        if self._deslocamento_apresentacao_x or self._deslocamento_apresentacao_y or
          self._recorte_apresentacao is not None:
            self.desenhar_retangulo(0, 0, self.largura, self.altura, cor)
            return
        self.backend.limpar(cor)

    def _deslocar_apresentacao(self, x: float, y: float) -> tuple[float, float]:
        return (
            float(x) + self._deslocamento_apresentacao_x,
            float(y) + self._deslocamento_apresentacao_y,
        )

    def desenhar_retangulo(self, x: float, y: float, largura: float, altura: float, cor:
       Cor = BRANCO, *, contorno: int = 0, opacidade: float = 1.0) -> Retangulo:
```

```
        x, y = self._deslocar_apresentacao(x, y)
        r=Retangulo(x,y,largura,altura)
        opacidade = _normalizar_opacidade(opacidade)
        self.backend.desenhar_retangulo(r,cor,contorno=contorno,opacidade=opacidade)
        return r

    def desenhar_circulo(self, x: float, y: float, raio: float, cor: Cor = BRANCO, *,
      contorno: int = 0, opacidade: float = 1.0) -> None:
        x, y = self._deslocar_apresentacao(x, y)
        self.backend.desenhar_circulo(x,y,raio,cor,contorno=contorno,opacidade=_normaliz
          ar_opacidade(opacidade))

    def desenhar_linha(self, x1: float, y1: float, x2: float, y2: float, cor: Cor =
      BRANCO, *, espessura: int = 1, opacidade: float = 1.0) -> None:
        if isinstance(espessura, bool) or int(espessura) < 1:
            raise ValueError("espessura da linha deve ser um inteiro positivo")
        x1, y1 = self._deslocar_apresentacao(x1, y1)
        x2, y2 = self._deslocar_apresentacao(x2, y2)
        self.backend.desenhar_linha(x1,y1,x2,y2,cor,espessura=int(espessura),opacidade=_
          normalizar_opacidade(opacidade))

    def desenhar_poligono(self, vertices, cor: Cor = BRANCO, *, contorno: int = 1,
      opacidade: float = 1.0) -> None:
        pontos = tuple(self._deslocar_apresentacao(x, y) for x, y in vertices)
        if len(pontos) < 2:
            raise ValueError("um polígono precisa de ao menos dois vértices")
        if isinstance(contorno, bool) or int(contorno) < 0:
            raise ValueError("contorno do polígono deve ser inteiro maior ou igual a
              zero")
        desenhar = getattr(self.backend, "desenhar_poligono", None)
        if callable(desenhar):
            desenhar(
                pontos,
                cor,
                contorno=int(contorno),
                opacidade=_normalizar_opacidade(opacidade),
            )
            return
        if int(contorno) == 0:
            raise ErroJogosCoral("o backend atual não oferece preenchimento de
              polígonos")
        for indice, (x1, y1) in enumerate(pontos):
            x2, y2 = pontos[(indice + 1) % len(pontos)]
            self.backend.desenhar_linha(
                x1,
                y1,
                x2,
                y2,
                cor,
                espessura=max(1, int(contorno)),
                opacidade=_normalizar_opacidade(opacidade),
            )

    def desenhar_overlay_debug(self, objeto=None, **opcoes):
        """Desenha um overlay de debug usando somente a API pública de Jogos."""
        from .debug import desenhar_overlay_debug

        if objeto is None:
            objeto = self.cena_atual if self.cena_atual is not None else self
        camera = opcoes.pop("camera", None)
        if camera is None and self.cena_atual is not None:
```

```
            camera = getattr(self.cena_atual, "camera", None)
        return desenhar_overlay_debug(self, objeto, camera=camera, **opcoes)

    def desenhar_ponto(self, x: float, y: float, cor: Cor = BRANCO, *, tamanho: int = 1,
       opacidade: float = 1.0) -> None:
        if isinstance(tamanho, bool) or int(tamanho) < 1:
            raise ValueError("tamanho do ponto deve ser um inteiro positivo")
        x, y = self._deslocar_apresentacao(x, y)
        self.backend.desenhar_ponto(x,y,cor,tamanho=int(tamanho),opacidade=_normalizar_o
          pacidade(opacidade))

    def desenhar_texto(self, texto: str, x: float, y: float, tamanho: int = 24, cor: Cor
       = BRANCO, *, origem: str = "topo_esquerdo", opacidade: float = 1.0) -> None:
        x, y = self._deslocar_apresentacao(x, y)
        self.backend.desenhar_texto(texto,x,y,tamanho,cor,origem=_normalizar_origem_dese
          nho(origem),opacidade=_normalizar_opacidade(opacidade))

    def medir_texto(self, texto: str, tamanho: int = 24) -> tuple[float, float]:
        if isinstance(tamanho, bool) or int(tamanho) <= 0:
            raise ValueError("tamanho do texto deve ser um inteiro positivo")
        medir = getattr(self.backend, "medir_texto", None)
        if callable(medir):
            largura, altura = medir(str(texto), int(tamanho))
            return float(largura), float(altura)
        return (len(str(texto)) * int(tamanho) * 0.6, float(int(tamanho)))

    def desenhar_imagem(
        self,
        caminho: str | QuadroSprite,
        x: float,
        y: float,
        largura: float | None = None,
        altura: float | None = None,
        *,
        escala: float = 1.0,
        rotacao: float = 0.0,
        espelhar_horizontalmente: bool = False,
        espelhar_verticalmente: bool = False,
        origem: str = "topo_esquerdo",
        pivo: tuple[float, float] | None = None,
        opacidade: float = 1.0,
    ) -> None:
        opacidade = _normalizar_opacidade(opacidade)
        if (largura is None) != (altura is None):
            raise ValueError("largura e altura devem ser informadas juntas ao
              redimensionar uma imagem")
        if largura is not None and largura <= 0:
            raise ValueError("largura da imagem deve ser positiva")
        if altura is not None and altura <= 0:
            raise ValueError("altura da imagem deve ser positiva")
        x, y = self._deslocar_apresentacao(x, y)
        transformacao = _normalizar_transformacao_imagem(
            escala=escala,
            rotacao=rotacao,
            espelhar_horizontalmente=espelhar_horizontalmente,
            espelhar_verticalmente=espelhar_verticalmente,
            origem=origem,
            pivo=pivo,
        )
        self.backend.desenhar_imagem(
            caminho,
```

```
            x,
            y,
            largura,
            altura,
            escala=transformacao[0],
            rotacao=transformacao[1],
            espelhar_horizontalmente=transformacao[2],
            espelhar_verticalmente=transformacao[3],
            origem=transformacao[4],
            pivo=transformacao[5],
            opacidade=opacidade,
        )

    def tocar_som(self, caminho: str, volume: float = 1.0) -> None:
        if not hasattr(self.backend, "tocar_som"):
            raise ErroJogosCoral("o backend atual não oferece áudio")
        self.backend.tocar_som(caminho, volume=max(0.0,min(1.0,float(volume))))

    def mudar_cena(self, cena) -> None:
        if self.cena_atual is cena:
            return
        if self.cena_atual is not None:
            self.cena_atual.sair(self)
        self.cena_atual = cena
        if cena is not None:
            cena.entrar(self)

    @property
    def transicao_cena(self) -> TransicaoCena | None:
        return self._transicao_cena

    def entrada_bloqueada_por_transicao(self) -> bool:
        return bool(self._transicao_cena is not None and
          self._transicao_cena.entrada_bloqueada)

    def mudar_cena_com_transicao(
        self,
        cena,
        *,
        tipo: str = "desaparecer",
        duracao: float = 0.4,
        curva: str | Callable[[float], float] = "suave",
        bloquear_entrada: bool = True,
        atualizar_anterior: bool = True,
        atualizar_proxima: bool = True,
        direcao: str = "direita",
    ) -> TransicaoCena:
        if self.cena_atual is cena and self._transicao_cena is None:
            raise ValueError("a cena de destino já é a cena atual")
        if self._transicao_cena is not None and self._transicao_cena.ativa:
            self._transicao_cena.cancelar()
        transicao = TransicaoCena(
            self.cena_atual,
            cena,
            tipo=tipo,
            duracao=duracao,
            curva=curva,
            bloquear_entrada=bloquear_entrada,
            atualizar_anterior=atualizar_anterior,
            atualizar_proxima=atualizar_proxima,
            direcao=direcao,
```

```
        )
        self._transicao_cena = transicao
        return transicao

    def cancelar_transicao_cena(self) -> bool:
        if self._transicao_cena is None or not self._transicao_cena.ativa:
            return False
        self._transicao_cena.cancelar()
        return True

    def _atualizar_transicao_cena(self, dt: float) -> None:
        transicao = self._transicao_cena
        if transicao is None or not transicao.ativa:
            if self.cena_atual is not None:
                self.cena_atual.atualizar(self, dt)
            return
        # Antes da troca, somente a cena anterior pode ser atualizada; depois,
        # somente a próxima. Isso preserva o ciclo entrar/sair das cenas.
        if not transicao._trocada:
            if transicao.atualizar_anterior and self.cena_atual is not None:
                self.cena_atual.atualizar(self, dt)
        elif transicao.atualizar_proxima and self.cena_atual is not None:
            self.cena_atual.atualizar(self, dt)
        trocar = transicao.avancar(dt)
        if trocar:
            self.mudar_cena(transicao.destino)
        if transicao.finalizada:
            self._transicao_cena = None

    def _definir_contexto_apresentacao(
        self,
        *,
        deslocamento: tuple[float, float] = (0.0, 0.0),
        recorte: Retangulo | None = None,
    ) -> None:
        self._deslocamento_apresentacao_x = float(deslocamento[0])
        self._deslocamento_apresentacao_y = float(deslocamento[1])
        self._recorte_apresentacao = recorte
        definir_recorte = getattr(self.backend, "definir_recorte", None)
        if callable(definir_recorte):
            definir_recorte(recorte)

    def _limpar_contexto_apresentacao(self) -> None:
        self._definir_contexto_apresentacao(deslocamento=(0.0, 0.0), recorte=None)

    def _recorte_revelacao(self, transicao: TransicaoCena, fracao: float) -> Retangulo:
        fracao = max(0.0, min(1.0, float(fracao)))
        if transicao.direcao == "direita":
            return Retangulo(0, 0, self.largura * fracao, self.altura)
        if transicao.direcao == "esquerda":
            largura = self.largura * fracao
            return Retangulo(self.largura - largura, 0, largura, self.altura)
        if transicao.direcao == "baixo":
            return Retangulo(0, 0, self.largura, self.altura * fracao)
        altura = self.altura * fracao
        return Retangulo(0, self.altura - altura, self.largura, altura)

    def _desenhar_cena_com_contexto(
        self,
        cena,
        *,
```

```
        deslocamento: tuple[float, float] = (0.0, 0.0),
        recorte: Retangulo | None = None,
    ) -> None:
        """Desenha uma cena com transformação puramente de apresentação.

        O contexto não altera sprites, câmera, callbacks registrados nem a
        autoridade de ``cena_atual``. Ele existe somente durante o desenho e é
        restaurado mesmo quando um callback de desenho lança exceção.
        """
        if cena is None:
            return
        self._definir_contexto_apresentacao(deslocamento=deslocamento, recorte=recorte)
        try:
            cena.desenhar(self)
        finally:
            self._limpar_contexto_apresentacao()

    def _desenhar_cena_atual(self) -> None:
        transicao = self._transicao_cena
        if transicao is None or not transicao.ativa:
            self._desenhar_cena_com_contexto(self.cena_atual)
            return

        amostra = transicao.amostra()
        if transicao.tipo == "deslizar":
            saida = amostra["deslocamento_saida"]
            entrada = amostra["deslocamento_entrada"]
            self._desenhar_cena_com_contexto(
                transicao.origem,
                deslocamento=(saida[0] * self.largura, saida[1] * self.altura),
            )
            self._desenhar_cena_com_contexto(
                transicao.destino,
                deslocamento=(entrada[0] * self.largura, entrada[1] * self.altura),
            )
            return

        if transicao.tipo == "revelar":
            # A cena anterior permanece como fundo; a próxima é revelada por
            # recorte. Nenhuma das duas precisa ter suas coordenadas mutadas.
            self._desenhar_cena_com_contexto(transicao.origem)
            recorte = self._recorte_revelacao(transicao, amostra["revelacao"])
            self._desenhar_cena_com_contexto(transicao.destino, recorte=recorte)
            return

        # Fades e transições por sobreposição preservam o contrato histórico:
        # desenha-se somente a cena que é autoridade naquele instante e o
        # efeito visual é aplicado depois por ``_desenhar_transicao_cena``.
        self._desenhar_cena_com_contexto(self.cena_atual)

    def _desenhar_transicao_cena(self) -> None:
        transicao = self._transicao_cena
        if transicao is None or not transicao.ativa:
            return
        amostra = transicao.amostra()
        if transicao.tipo in {"desaparecer", "aparecer", "escurecer", "clarear"}:
            opacidade = float(amostra["opacidade"])
            if opacidade > 0.0:
                self.desenhar_retangulo(
                    0,
                    0,
```

```
                    self.largura,
                    self.altura,
                    amostra["cor"],
                    opacidade=opacidade,
                )

    def executar(self, *, max_quadros: int | None = None) -> None:
        if max_quadros is not None:
            if isinstance(max_quadros, bool) or not isinstance(max_quadros, int):
                raise TypeError("max_quadros deve ser inteiro ou None")
            if max_quadros < 0:
                raise ValueError("max_quadros deve ser maior ou igual a zero")
        if self._tela_cheia:
            self.backend.iniciar(self.titulo, self.largura, self.altura,
              tela_cheia=True)
        else:
            self.backend.iniciar(self.titulo, self.largura, self.altura)
        self.rodando=True
        quadros=0
        try:
            for f in self._ao_iniciar: f(self)
            while self.rodando and (max_quadros is None or quadros < max_quadros) and
              self.backend.processar_eventos():
                dt_backend = normalizar_dt(self.backend.limitar_fps(self.fps))
                dt = self.relogio.passo() if self.relogio is not None else dt_backend
                dt = normalizar_dt(dt)
                self.metricas.registrar(dt, dt_backend)
                self.regras.atualizar(self, dt)
                if self.mundo is not None:
                    self.mundo.atualizar(dt)
                for f in self._ao_atualizar: f(self,dt)
                if self._transicao_cena is not None and self._transicao_cena.ativa:
                    self._atualizar_transicao_cena(dt)
                elif self.cena_atual is not None:
                    self.cena_atual.atualizar(self, dt)
                if self._historico_inspecao is not None:
                    self._tempo_inspecao += dt
                    self._historico_inspecao.registrar(
                        self,
                        instante=self._tempo_inspecao,
                    )
                for f in self._ao_desenhar: f(self)
                self._desenhar_cena_atual()
                self._desenhar_transicao_cena()
                if self._debug.ativo:
                    self.desenhar_debug()
                self.backend.apresentar()
                quadros += 1
        finally:
            self.rodando=False
            try:
                for f in self._ao_encerrar: f(self)
            finally:
                self.backend.encerrar()


def _jogo_para_dados(jogo: Jogo) -> dict[str, Any]:
    return {
        "titulo": jogo.titulo,
        "largura": jogo.largura,
        "altura": jogo.altura,
```

```
        "fps": jogo.fps,
        "tela_cheia": jogo.em_tela_cheia(),
        "mundo": jogo.mundo,
    }


_BOTOES_MOUSE = {
    "esquerdo": "esquerdo",
    "esquerda": "esquerdo",
    "left": "esquerdo",
    "meio": "meio",
    "central": "meio",
    "middle": "meio",
    "direito": "direito",
    "direita": "direito",
    "right": "direito",
}


def _normalizar_tamanho_janela(largura: int, altura: int) -> tuple[int, int]:
    if isinstance(largura, bool) or isinstance(altura, bool):
        raise ValueError("largura e altura da janela devem ser inteiros positivos")
    try:
        largura = int(largura)
        altura = int(altura)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ValueError("largura e altura da janela devem ser inteiros positivos") from
           exc
    if largura <= 0 or altura <= 0:
        raise ValueError("largura e altura da janela devem ser inteiros positivos")
    return largura, altura


def _normalizar_botao_mouse(botao: str) -> str:
    nome = str(botao).strip().casefold()
    normalizado = _BOTOES_MOUSE.get(nome)
    if normalizado is None:
        raise ValueError("botão do mouse deve ser esquerdo, meio/central ou direito")
    return normalizado


def _normalizar_posicao_mouse(posicao) -> tuple[int, int]:
    try:
        x, y = posicao
    except (TypeError, ValueError) as exc:
        raise ValueError("posição do mouse deve conter x e y") from exc
    return (int(x), int(y))


def _normalizar_roda_mouse(roda) -> tuple[int, int]:
    try:
        x, y = roda
    except (TypeError, ValueError) as exc:
        raise ValueError("roda do mouse deve conter deslocamento horizontal e vertical")
           from exc
    return (int(x), int(y))


def _jogo_de_dados(dados: dict[str, Any]) -> Jogo:
    obrigatorios = ("titulo", "largura", "altura", "fps")
    if any(chave not in dados for chave in obrigatorios):
```

```
        raise ValueError("payload de Jogo incompleto")
    return Jogo(
        str(dados["titulo"]),
        int(dados["largura"]),
        int(dados["altura"]),
        int(dados["fps"]),
        backend=BackendNulo(),
        mundo=dados.get("mundo"),
        tela_cheia=bool(dados.get("tela_cheia", False)),
    )


from ..persistencia import registrar_adaptador as _registrar_adaptador
_registrar_adaptador(Jogo, "coral.jogos/Jogo", _jogo_para_dados, _jogo_de_dados)

from __future__ import annotations

from dataclasses import dataclass, field
import math
from typing import Any, Callable, Protocol


def _numero_finito(valor: Any, nome: str) -> float:
    if isinstance(valor, bool):
        raise ValueError(f"{nome} deve ser um número finito")
    try:
        numero = float(valor)
    except (TypeError, ValueError) as exc:
        raise ValueError(f"{nome} deve ser um número finito") from exc
    if not math.isfinite(numero):
        raise ValueError(f"{nome} deve ser um número finito")
    return numero


class NoApresentacao(Protocol):
    nome: str | None

    @property
    def finalizado(self) -> bool: ...

    @property
    def progresso(self) -> float: ...

    def avancar(self, dt: float) -> float: ...

    def reiniciar(self) -> None: ...


@dataclass
class AcaoApresentacao:
    acao: Callable[[], Any]
    nome: str | None = None
    _executada: bool = field(default=False, init=False, repr=False)

    def __post_init__(self) -> None:
        if not callable(self.acao):
            raise TypeError("ação de apresentação precisa ser chamável")
        if self.nome is not None:
            self.nome = str(self.nome).strip() or None

    @property
```

```
    def finalizado(self) -> bool:
        return self._executada

    @property
    def progresso(self) -> float:
        return 1.0 if self._executada else 0.0

    def avancar(self, dt: float) -> float:
        dt = _numero_finito(dt, "dt da ação")
        if dt < 0:
            raise ValueError("dt da ação deve ser maior ou igual a zero")
        if not self._executada:
            self.acao()
            self._executada = True
        return dt

    def reiniciar(self) -> None:
        self._executada = False


@dataclass
class EsperaApresentacao:
    duracao: float
    nome: str | None = None
    tempo: float = 0.0

    def __post_init__(self) -> None:
        self.duracao = _numero_finito(self.duracao, "duração da espera")
        self.tempo = _numero_finito(self.tempo, "tempo da espera")
        if self.duracao < 0 or self.tempo < 0:
            raise ValueError("duração e tempo da espera devem ser não negativos")
        self.tempo = min(self.tempo, self.duracao)
        if self.nome is not None:
            self.nome = str(self.nome).strip() or None

    @property
    def finalizado(self) -> bool:
        return self.tempo >= self.duracao

    @property
    def progresso(self) -> float:
        if self.duracao == 0:
            return 1.0
        return min(1.0, self.tempo / self.duracao)

    def avancar(self, dt: float) -> float:
        dt = _numero_finito(dt, "dt da espera")
        if dt < 0:
            raise ValueError("dt da espera deve ser maior ou igual a zero")
        if self.finalizado:
            return dt
        restante = self.duracao - self.tempo
        consumido = min(restante, dt)
        self.tempo += consumido
        return dt - consumido

    def reiniciar(self) -> None:
        self.tempo = 0.0


@dataclass
```

```
class EsperaConclusao:
    alvo: Any
    nome: str | None = None

    def __post_init__(self) -> None:
        if self.nome is not None:
            self.nome = str(self.nome).strip() or None
        if not callable(self.alvo) and not (
            hasattr(self.alvo, "finalizado") or hasattr(self.alvo, "finalizada")
        ):
            raise TypeError("espera de conclusão requer chamável ou alvo com estado
              finalizado")

    def _concluido(self) -> bool:
        if callable(self.alvo):
            return bool(self.alvo())
        if hasattr(self.alvo, "finalizado"):
            return bool(getattr(self.alvo, "finalizado"))
        return bool(getattr(self.alvo, "finalizada"))

    @property
    def finalizado(self) -> bool:
        return self._concluido()

    @property
    def progresso(self) -> float:
        if self.finalizado:
            return 1.0
        valor = getattr(self.alvo, "progresso", 0.0)
        try:
            numero = float(valor)
        except (TypeError, ValueError):
            return 0.0
        if not math.isfinite(numero):
            return 0.0
        return min(1.0, max(0.0, numero))

    def avancar(self, dt: float) -> float:
        dt = _numero_finito(dt, "dt da espera por conclusão")
        if dt < 0:
            raise ValueError("dt da espera por conclusão deve ser maior ou igual a
              zero")
        return dt if self.finalizado else 0.0

    def reiniciar(self) -> None:
        # O alvo possui sua própria autoridade temporal; a composição não o
        # reinicia implicitamente.
        return None


def _validar_nos(itens: tuple[NoApresentacao, ...], nome: str) -> tuple[NoApresentacao,
  ...]:
    if not itens:
        raise ValueError(f"{nome} precisa de ao menos uma etapa")
    for item in itens:
        for membro in ("avancar", "reiniciar", "finalizado", "progresso"):
            if not hasattr(item, membro):
                raise TypeError(f"etapa inválida em {nome}: falta {membro}")
    return itens
```

```
@dataclass
class SequenciaApresentacao:
    etapas: tuple[NoApresentacao, ...]
    nome: str | None = None
    _indice: int = field(default=0, init=False, repr=False)

    def __post_init__(self) -> None:
        self.etapas = _validar_nos(tuple(self.etapas), "sequência")
        if self.nome is not None:
            self.nome = str(self.nome).strip() or None

    @property
    def finalizado(self) -> bool:
        return self._indice >= len(self.etapas)

    @property
    def progresso(self) -> float:
        total = len(self.etapas)
        if self.finalizado:
            return 1.0
        return (self._indice + self.etapas[self._indice].progresso) / total

    @property
    def etapa_atual(self) -> NoApresentacao | None:
        return None if self.finalizado else self.etapas[self._indice]

    def avancar(self, dt: float) -> float:
        restante = _numero_finito(dt, "dt da sequência")
        if restante < 0:
            raise ValueError("dt da sequência deve ser maior ou igual a zero")
        while not self.finalizado:
            etapa = self.etapas[self._indice]
            restante = etapa.avancar(restante)
            if not etapa.finalizado:
                return 0.0
            self._indice += 1
            # Uma etapa instantânea pode deixar todo o dt disponível para a
            # próxima; o laço é finito porque a sequência tem tamanho finito.
        return restante

    def reiniciar(self) -> None:
        self._indice = 0
        for etapa in self.etapas:
            etapa.reiniciar()


@dataclass
class ParaleloApresentacao:
    etapas: tuple[NoApresentacao, ...]
    nome: str | None = None

    def __post_init__(self) -> None:
        self.etapas = _validar_nos(tuple(self.etapas), "paralelo")
        if self.nome is not None:
            self.nome = str(self.nome).strip() or None

    @property
    def finalizado(self) -> bool:
        return all(etapa.finalizado for etapa in self.etapas)

    @property
```

```
    def progresso(self) -> float:
        return sum(etapa.progresso for etapa in self.etapas) / len(self.etapas)

    def avancar(self, dt: float) -> float:
        dt = _numero_finito(dt, "dt do paralelo")
        if dt < 0:
            raise ValueError("dt do paralelo deve ser maior ou igual a zero")
        if self.finalizado:
            return dt
        sobras = [etapa.avancar(dt) for etapa in self.etapas if not etapa.finalizado]
        if self.finalizado:
            # O ramo mais lento determina quanto tempo o bloco paralelo
            # consumiu; por isso a menor sobra é a sobra global.
            return min(sobras, default=dt)
        return 0.0

    def reiniciar(self) -> None:
        for etapa in self.etapas:
            etapa.reiniciar()


@dataclass
class ComposicaoTemporal:
    raiz: NoApresentacao
    repeticoes: int | None = 1
    nome: str | None = None
    estado: str = "ativo"
    _ciclos_concluidos: int = field(default=0, init=False, repr=False)

    def __post_init__(self) -> None:
        _validar_nos((self.raiz,), "composição")
        if self.repeticoes is not None:
            if isinstance(self.repeticoes, bool) or int(self.repeticoes) < 1:
                raise ValueError("repetições deve ser inteiro positivo ou None")
            self.repeticoes = int(self.repeticoes)
        if self.nome is not None:
            self.nome = str(self.nome).strip() or None
        self.estado = str(self.estado).strip().casefold()
        if self.estado not in {"ativo", "pausado", "cancelado", "finalizado"}:
            raise ValueError("estado inválido da composição temporal")

    @property
    def finalizado(self) -> bool:
        return self.estado == "finalizado"

    @property
    def cancelado(self) -> bool:
        return self.estado == "cancelado"

    @property
    def ciclos_concluidos(self) -> int:
        return self._ciclos_concluidos

    @property
    def progresso(self) -> float:
        if self.finalizado:
            return 1.0
        if self.repeticoes is None:
            return self.raiz.progresso
        return min(1.0, (self._ciclos_concluidos + self.raiz.progresso) /
          self.repeticoes)
```

```
    @property
    def etapa_atual(self) -> str | None:
        atual = self.raiz
        while isinstance(atual, SequenciaApresentacao):
            atual = atual.etapa_atual
            if atual is None:
                return None
        return getattr(atual, "nome", None)

    def pausar(self) -> "ComposicaoTemporal":
        if self.estado == "ativo":
            self.estado = "pausado"
        return self

    def retomar(self) -> "ComposicaoTemporal":
        if self.estado == "pausado":
            self.estado = "ativo"
        return self

    def cancelar(self) -> "ComposicaoTemporal":
        if not self.finalizado:
            self.estado = "cancelado"
        return self

    def reiniciar(self) -> "ComposicaoTemporal":
        self.raiz.reiniciar()
        self._ciclos_concluidos = 0
        self.estado = "ativo"
        return self

    def avancar(self, dt: float) -> "ComposicaoTemporal":
        restante = _numero_finito(dt, "dt da composição")
        if restante < 0:
            raise ValueError("dt da composição deve ser maior ou igual a zero")
        if self.estado != "ativo":
            return self
        while self.estado == "ativo":
            restante = self.raiz.avancar(restante)
            if not self.raiz.finalizado:
                break
            self._ciclos_concluidos += 1
            if self.repeticoes is not None and self._ciclos_concluidos >=
              self.repeticoes:
                self.estado = "finalizado"
                break
            self.raiz.reiniciar()
            if restante <= 0:
                break
        return self


def executar(acao: Callable[[], Any], *, nome: str | None = None) -> AcaoApresentacao:
    return AcaoApresentacao(acao, nome=nome)


def esperar(duracao: float, *, nome: str | None = None) -> EsperaApresentacao:
    return EsperaApresentacao(duracao, nome=nome)


def esperar_conclusao(alvo: Any, *, nome: str | None = None) -> EsperaConclusao:
```

```
    return EsperaConclusao(alvo, nome=nome)


def sequencia(*etapas: NoApresentacao, nome: str | None = None) ->
  SequenciaApresentacao:
    return SequenciaApresentacao(tuple(etapas), nome=nome)


def paralelo(*etapas: NoApresentacao, nome: str | None = None) -> ParaleloApresentacao:
    return ParaleloApresentacao(tuple(etapas), nome=nome)


@dataclass
class VinculoTempoApresentacao:
    """Avança uma composição a partir de uma fonte de tempo explicitamente escolhida.

    A fonte precisa apenas oferecer ``agora()``. O vínculo observa diferenças
    entre instantes e nunca chama ``passo()`` nem avança o relógio por conta
    própria, evitando tomar autoridade do tempo de Mundo ou Simulação.
    """

    composicao: ComposicaoTemporal
    fonte: Any
    _ultimo: float = field(init=False, repr=False)

    def __post_init__(self) -> None:
        agora = getattr(self.fonte, "agora", None)
        if not callable(agora):
            raise TypeError("fonte de tempo da apresentação precisa oferecer agora()")
        self._ultimo = _numero_finito(agora(), "instante inicial da apresentação")

    def atualizar(self) -> float:
        atual = _numero_finito(self.fonte.agora(), "instante da apresentação")
        if atual < self._ultimo:
            raise ValueError("fonte de tempo da apresentação não pode retroceder")
        dt = atual - self._ultimo
        self._ultimo = atual
        self.composicao.avancar(dt)
        return dt

    def ressincronizar(self) -> "VinculoTempoApresentacao":
        self._ultimo = _numero_finito(self.fonte.agora(), "instante da apresentação")
        return self


@dataclass
class VinculoComposicaoEventos:
    """Controla uma composição por um evento sem acoplar o barramento a Jogos."""

    composicao: ComposicaoTemporal
    eventos: Any
    evento: str
    modo: str = "reiniciar"
    ativo: bool = field(default=True, init=False)
    _ouvinte: Callable[..., Any] = field(init=False, repr=False)

    def __post_init__(self) -> None:
        quando = getattr(self.eventos, "quando", None)
        remover = getattr(self.eventos, "remover", None)
        if not callable(quando) or not callable(remover):
            raise TypeError("vínculo de composição requer barramento com quando() e
```

```
            remover()")
        self.evento = str(self.evento).strip()
        if not self.evento:
            raise ValueError("nome do evento da composição não pode ser vazio")
        self.modo = str(self.modo).strip().casefold()
        if self.modo not in {"reiniciar", "retomar", "pausar", "cancelar"}:
            raise ValueError("modo do vínculo deve ser reiniciar, retomar, pausar ou
              cancelar")

        def ouvinte(*_args, **_kwargs):
            if not self.ativo:
                return
            getattr(self.composicao, self.modo)()

        self._ouvinte = ouvinte
        quando(self.evento, ouvinte)

    def desligar(self) -> None:
        if not self.ativo:
            return
        self.eventos.remover(self.evento, self._ouvinte)
        self.ativo = False


@dataclass(frozen=True)
class ComandoComposicao:
    operacao: str
    valor: float | None = None

    def __post_init__(self) -> None:
        operacao = str(self.operacao).strip().casefold()
        if operacao not in {"avancar", "pausar", "retomar", "cancelar", "reiniciar"}:
            raise ValueError("operação de replay de apresentação desconhecida")
        valor = self.valor
        if operacao == "avancar":
            if valor is None:
                raise ValueError("comando avançar precisa de dt")
            valor = _numero_finito(valor, "dt do comando de apresentação")
            if valor < 0:
                raise ValueError("dt do comando de apresentação não pode ser negativo")
        elif valor is not None:
            raise ValueError("somente avançar aceita valor no comando de apresentação")
        object.__setattr__(self, "operacao", operacao)
        object.__setattr__(self, "valor", valor)


@dataclass
class GravadorComposicao:
    """Grava comandos explícitos de uma composição para replay determinístico."""

    composicao: ComposicaoTemporal
    comandos: list[ComandoComposicao] = field(default_factory=list)

    def _registrar(self, operacao: str, valor: float | None = None) ->
      "GravadorComposicao":
        comando = ComandoComposicao(operacao, valor)
        self.comandos.append(comando)
        if operacao == "avancar":
            self.composicao.avancar(float(comando.valor))
        else:
            getattr(self.composicao, operacao)()
```

```
        return self

    def avancar(self, dt: float) -> "GravadorComposicao":
        return self._registrar("avancar", dt)

    def pausar(self) -> "GravadorComposicao":
        return self._registrar("pausar")

    def retomar(self) -> "GravadorComposicao":
        return self._registrar("retomar")

    def cancelar(self) -> "GravadorComposicao":
        return self._registrar("cancelar")

    def reiniciar(self) -> "GravadorComposicao":
        return self._registrar("reiniciar")

    def reproduzir_em(self, composicao: ComposicaoTemporal) -> ComposicaoTemporal:
        composicao.reiniciar()
        for comando in self.comandos:
            if comando.operacao == "avancar":
                composicao.avancar(float(comando.valor))
            else:
                getattr(composicao, comando.operacao)()
        return composicao


def usar_fonte_tempo(composicao: ComposicaoTemporal, fonte: Any) ->
  VinculoTempoApresentacao:
    return VinculoTempoApresentacao(composicao, fonte)


def ligar_evento(
    composicao: ComposicaoTemporal,
    eventos: Any,
    evento: str,
    *,
    modo: str = "reiniciar",
) -> VinculoComposicaoEventos:
    return VinculoComposicaoEventos(composicao, eventos, evento, modo=modo)

"""Primitivas neutras de overlay de debug para Coral Jogos 1.5.14.

Este módulo não depende de Pygame. Ele transforma snapshots de inspeção em
instruções geométricas estáveis que podem ser consumidas por qualquer backend.
"""
from __future__ import annotations

from dataclasses import dataclass, field
import json
import unicodedata
from typing import Any, Mapping

from ..inspecao import SnapshotInspecao, inspecionar, normalizar_valor


CONTRATO_OVERLAY_DEBUG = "coral.jogos.debug.overlay/1"

_CATEGORIAS_DEBUG_CANONICAS = frozenset({
    "limites",
    "colisor",
```

```
    "pivo",
    "camera",
    "rotulo",
    "metricas",
})
_ALIASES_CATEGORIAS_DEBUG = {
    "limite": "limites",
    "limites": "limites",
    "colisao": "colisor",
    "colisoes": "colisor",
    "colisor": "colisor",
    "colisores": "colisor",
    "pivo": "pivo",
    "pivos": "pivo",
    "camera": "camera",
    "cameras": "camera",
    "rotulo": "rotulo",
    "rotulos": "rotulo",
    "metrica": "metricas",
    "metricas": "metricas",
    "fps": "metricas",
}
_CATEGORIAS_DEBUG_PADRAO = frozenset({
    "limites", "colisor", "pivo", "camera", "metricas"
})


def normalizar_categorias_debug(categorias: Any = None) -> frozenset[str]:
    """Normaliza categorias públicas do modo de debug persistente."""
    if categorias is None:
        return _CATEGORIAS_DEBUG_PADRAO
    if isinstance(categorias, str):
        categorias = (categorias,)
    saida: set[str] = set()
    for categoria in categorias:
        chave = str(categoria).strip().casefold().replace(" ", "_")
        chave = "".join(
            caractere
            for caractere in unicodedata.normalize("NFKD", chave)
            if not unicodedata.combining(caractere)
        )
        canonica = _ALIASES_CATEGORIAS_DEBUG.get(chave)
        if canonica is None:
            opcoes = ", ".join(sorted(_CATEGORIAS_DEBUG_CANONICAS))
            raise ValueError(f"categoria de debug inválida: {categoria!r}; use uma de:
              {opcoes}")
        saida.add(canonica)
    return frozenset(saida)


@dataclass(frozen=True)
class PrimitivaDebug:
    tipo: str
    categoria: str
    dados: Mapping[str, Any] = field(default_factory=dict)
    rotulo: str | None = None
    espaco: str = "mundo"

    def para_dict(self) -> dict[str, Any]:
        return {
            "tipo": self.tipo,
```

```
            "categoria": self.categoria,
            "rotulo": self.rotulo,
            "espaco": self.espaco,
            "dados": normalizar_valor(self.dados),
        }


@dataclass(frozen=True)
class OverlayDebug:
    primitivas: tuple[PrimitivaDebug, ...] = ()
    assinatura_origem: str | None = None
    contrato: str = CONTRATO_OVERLAY_DEBUG

    @property
    def quantidade(self) -> int:
        return len(self.primitivas)

    def por_categoria(self, categoria: str) -> tuple[PrimitivaDebug, ...]:
        alvo = str(categoria).strip().casefold()
        return tuple(p for p in self.primitivas if p.categoria.casefold() == alvo)

    def para_dict(self) -> dict[str, Any]:
        return {
            "contrato": self.contrato,
            "quantidade": self.quantidade,
            "assinatura_origem": self.assinatura_origem,
            "primitivas": [p.para_dict() for p in self.primitivas],
        }

    def para_json(self, **kwargs: Any) -> str:
        opcoes = {"ensure_ascii": False, "sort_keys": True}
        opcoes.update(kwargs)
        return json.dumps(self.para_dict(), **opcoes)


def _numero(valor: Any, padrao: float = 0.0) -> float:
    try:
        return float(valor)
    except (TypeError, ValueError, OverflowError):
        return float(padrao)


def _retangulo(dados: Mapping[str, Any] | None) -> dict[str, float] | None:
    if not isinstance(dados, Mapping):
        return None
    chaves = ("x", "y", "largura", "altura")
    if not all(chave in dados for chave in chaves):
        return None
    return {chave: _numero(dados[chave]) for chave in chaves}


def _ponto_posicao(dados: Mapping[str, Any]) -> tuple[float, float] | None:
    posicao = dados.get("posicao")
    if not isinstance(posicao, Mapping) or "x" not in posicao or "y" not in posicao:
        return None
    return (_numero(posicao["x"]), _numero(posicao["y"]))


def _primitivas_sprite(
    dados: Mapping[str, Any],
    *,
```

```
    limites: bool,
    colisores: bool,
    pivos: bool,
    rotulos: bool,
) -> list[PrimitivaDebug]:
    saida: list[PrimitivaDebug] = []
    nome = str(dados.get("nome") or "sprite")

    if limites:
        retangulo = _retangulo(dados.get("limites_visuais"))
        if retangulo is not None:
            saida.append(PrimitivaDebug(
                "retangulo",
                "limites",
                retangulo,
                rotulo=nome,
            ))

    if colisores:
        colisao = dados.get("colisao")
        if isinstance(colisao, Mapping):
            vertices = colisao.get("vertices")
            if isinstance(vertices, (list, tuple)) and len(vertices) >= 2:
                pontos = [
                    {"x": _numero(v[0]), "y": _numero(v[1])}
                    for v in vertices
                    if isinstance(v, (list, tuple)) and len(v) >= 2
                ]
                if len(pontos) >= 2:
                    saida.append(PrimitivaDebug(
                        "poligono",
                        "colisor",
                        {
                            "vertices": pontos,
                            "fechado": True,
                            "acompanha_escala": bool(colisao.get("acompanha_escala",
                              False)),
                            "acompanha_rotacao": bool(colisao.get("acompanha_rotacao",
                              False)),
                        },
                        rotulo=nome,
                    ))

    posicao = _ponto_posicao(dados)
    if pivos and posicao is not None:
        saida.append(PrimitivaDebug(
            "ponto",
            "pivo",
            {"x": posicao[0], "y": posicao[1], "tamanho": 5},
            rotulo=nome,
        ))

    if rotulos and posicao is not None:
        saida.append(PrimitivaDebug(
            "texto",
            "rotulo",
            {"texto": nome, "x": posicao[0] + 6.0, "y": posicao[1] - 10.0},
            rotulo=nome,
        ))

    return saida
```

```
def _primitivas_camera(dados: Mapping[str, Any], *, camera: bool) ->
  list[PrimitivaDebug]:
    if not camera:
        return []
    origem = dados.get("camera") if isinstance(dados.get("camera"), Mapping) else dados
    if not isinstance(origem, Mapping):
        return []

    saida: list[PrimitivaDebug] = []
    vertices = origem.get("vertices_area_visivel")
    if isinstance(vertices, (list, tuple)) and len(vertices) >= 4:
        pontos = [
            {"x": _numero(v[0]), "y": _numero(v[1])}
            for v in vertices
            if isinstance(v, (list, tuple)) and len(v) >= 2
        ]
        if len(pontos) >= 4:
            saida.append(PrimitivaDebug(
                "poligono",
                "camera",
                {"vertices": pontos, "fechado": True},
                rotulo="camera",
            ))
    else:
        area = _retangulo(origem.get("area_visivel"))
        if area is not None:
            saida.append(PrimitivaDebug(
                "retangulo",
                "camera",
                area,
                rotulo="camera",
            ))

    centro = origem.get("centro")
    if isinstance(centro, (list, tuple)) and len(centro) >= 2:
        saida.append(PrimitivaDebug(
            "ponto",
            "camera_centro",
            {"x": _numero(centro[0]), "y": _numero(centro[1]), "tamanho": 5},
            rotulo="camera",
        ))
    return saida


def gerar_overlay_debug(
    objeto: Any,
    *,
    limites: bool = True,
    colisores: bool = True,
    camera: bool = True,
    pivos: bool = True,
    rotulos: bool = False,
    incluir_invisiveis: bool = False,
) -> OverlayDebug:
    """Gera primitivas de debug sem desenhar e sem depender de backend gráfico."""
    snapshot = objeto if isinstance(objeto, SnapshotInspecao) else inspecionar(objeto)
    dados = snapshot.para_dict()["dados"]
    primitivas: list[PrimitivaDebug] = []
```

```
    cena = dados.get("cena")
    if isinstance(cena, Mapping):
        dados = cena

    sprites = dados.get("sprites")
    if isinstance(sprites, (list, tuple)):
        for sprite in sprites:
            if not isinstance(sprite, Mapping):
                continue
            if not incluir_invisiveis and sprite.get("visivel") is False:
                continue
            primitivas.extend(_primitivas_sprite(
                sprite,
                limites=limites,
                colisores=colisores,
                pivos=pivos,
                rotulos=rotulos,
            ))
        primitivas.extend(_primitivas_camera(dados, camera=camera))
    else:
        if "limites_visuais" in dados or "colisao" in dados:
            primitivas.extend(_primitivas_sprite(
                dados,
                limites=limites,
                colisores=colisores,
                pivos=pivos,
                rotulos=rotulos,
            ))
        primitivas.extend(_primitivas_camera(dados, camera=camera))

    return OverlayDebug(
        primitivas=tuple(primitivas),
        assinatura_origem=snapshot.assinatura(),
    )


@dataclass(frozen=True)
class EstiloOverlayDebug:
    """Estilo de desenho desacoplado da geração geométrica do overlay."""

    limites: Any = None
    colisor: Any = None
    pivo: Any = None
    camera: Any = None
    camera_centro: Any = None
    rotulo: Any = None
    metricas: Any = None
    espessura: int = 1
    tamanho_pivo: int = 3
    tamanho_texto: int = 14
    opacidade: float = 1.0

    def cor_para(self, categoria: str):
        from .base import AMARELO, AZUL, BRANCO, VERDE, VERMELHO

        padrao = {
            "limites": AMARELO,
            "colisor": VERMELHO,
            "pivo": AZUL,
            "camera": VERDE,
            "camera_centro": VERDE,
```

```
            "rotulo": BRANCO,
            "metricas": BRANCO,
        }
        valor = getattr(self, str(categoria).casefold(), None)
        return padrao.get(str(categoria).casefold(), BRANCO) if valor is None else valor



@dataclass(frozen=True)
class ConfiguracaoDebugJogo:
    """Configuração persistente e serializável do modo de debug de Jogo."""

    ativo: bool = False
    categorias: frozenset[str] = _CATEGORIAS_DEBUG_PADRAO
    incluir_invisiveis: bool = False
    estilo: EstiloOverlayDebug = field(default_factory=EstiloOverlayDebug)

    def __post_init__(self):
        object.__setattr__(self, "categorias",
          normalizar_categorias_debug(self.categorias))

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "ativo": bool(self.ativo),
            "categorias": sorted(self.categorias),
            "incluir_invisiveis": bool(self.incluir_invisiveis),
        }

    def com_alteracoes(
        self,
        *,
        ativo: bool | None = None,
        categorias: Any = None,
        incluir_invisiveis: bool | None = None,
        estilo: EstiloOverlayDebug | None = None,
    ) -> "ConfiguracaoDebugJogo":
        return ConfiguracaoDebugJogo(
            ativo=self.ativo if ativo is None else bool(ativo),
            categorias=self.categorias if categorias is None else
              normalizar_categorias_debug(categorias),
            incluir_invisiveis=(
                self.incluir_invisiveis
                if incluir_invisiveis is None
                else bool(incluir_invisiveis)
            ),
            estilo=self.estilo if estilo is None else estilo,
        )


def gerar_overlay_metricas_debug(
    metricas: Any,
    *,
    x: float = 8.0,
    y: float = 8.0,
    espacamento: float = 18.0,
) -> OverlayDebug:
    """Converte métricas operacionais em texto de tela, sem backend gráfico."""
    metodo = getattr(metricas, "estado_inspecao", None)
    dados = metodo() if callable(metodo) else metricas
    if not isinstance(dados, Mapping):
        raise TypeError("métricas de debug precisam ser um mapeamento ou expor
```

```
          estado_inspecao()")

    linhas = (
        f"FPS {float(dados.get('fps_instantaneo', 0.0)):.1f}",
        f"Quadro {float(dados.get('tempo_quadro_ms', 0.0)):.2f} ms",
        f"Delta {float(dados.get('delta_logico', 0.0)):.4f} s",
        f"Quadros {int(dados.get('quadros', 0))}",
    )
    primitivas = tuple(
        PrimitivaDebug(
            "texto",
            "metricas",
            {"texto": texto, "x": float(x), "y": float(y) + indice *
              float(espacamento)},
            rotulo="metricas",
            espaco="tela",
        )
        for indice, texto in enumerate(linhas)
    )
    return OverlayDebug(primitivas=primitivas)

def _ponto_renderizacao(x: Any, y: Any, camera: Any | None) -> tuple[float, float]:
    x = _numero(x)
    y = _numero(y)
    if camera is None:
        return x, y
    transformar = getattr(camera, "mundo_para_tela", None)
    if callable(transformar):
        tx, ty = transformar(x, y)
        return float(tx), float(ty)
    return x, y


def _vertices_renderizacao(vertices: Any, camera: Any | None) -> tuple[tuple[float,
  float], ...]:
    if not isinstance(vertices, (list, tuple)):
        return ()
    saida = []
    for vertice in vertices:
        if isinstance(vertice, Mapping):
            x, y = vertice.get("x"), vertice.get("y")
        elif isinstance(vertice, (list, tuple)) and len(vertice) >= 2:
            x, y = vertice[0], vertice[1]
        else:
            continue
        saida.append(_ponto_renderizacao(x, y, camera))
    return tuple(saida)


def desenhar_overlay_debug(
    alvo: Any,
    overlay_ou_objeto: Any,
    *,
    camera: Any | None = None,
    estilo: EstiloOverlayDebug | None = None,
    limites: bool = True,
    colisores: bool = True,
    pivos: bool = True,
    rotulos: bool = False,
    incluir_invisiveis: bool = False,
    mostrar_camera: bool = True,
```

```
) -> OverlayDebug:
    """Renderiza primitivas de debug pela API pública de desenho."""
    overlay = (
        overlay_ou_objeto
        if isinstance(overlay_ou_objeto, OverlayDebug)
        else gerar_overlay_debug(
            overlay_ou_objeto,
            limites=limites,
            colisores=colisores,
            camera=mostrar_camera,
            pivos=pivos,
            rotulos=rotulos,
            incluir_invisiveis=incluir_invisiveis,
        )
    )
    estilo = estilo or EstiloOverlayDebug()

    desenhar_retangulo = getattr(alvo, "desenhar_retangulo", None)
    desenhar_poligono = getattr(alvo, "desenhar_poligono", None)
    desenhar_linha = getattr(alvo, "desenhar_linha", None)
    desenhar_ponto = getattr(alvo, "desenhar_ponto", None)
    desenhar_texto = getattr(alvo, "desenhar_texto", None)

    if not all(callable(x) for x in (desenhar_retangulo, desenhar_linha, desenhar_ponto,
       desenhar_texto)):
        raise TypeError("alvo de overlay precisa oferecer a API pública de desenho de
          Jogos")

    from .base import Retangulo

    for primitiva in overlay.primitivas:
        dados = primitiva.dados
        cor = estilo.cor_para(primitiva.categoria)
        opacidade = float(estilo.opacidade)
        camera_primitiva = camera if primitiva.espaco == "mundo" else None

        if primitiva.tipo == "retangulo":
            x = _numero(dados.get("x"))
            y = _numero(dados.get("y"))
            largura = _numero(dados.get("largura"))
            altura = _numero(dados.get("altura"))
            if camera_primitiva is None:
                if hasattr(alvo, "backend"):
                    desenhar_retangulo(
                        x, y, largura, altura, cor,
                        contorno=max(1, int(estilo.espessura)),
                        opacidade=opacidade,
                    )
                else:
                    desenhar_retangulo(
                        Retangulo(x, y, largura, altura), cor,
                        contorno=max(1, int(estilo.espessura)),
                        opacidade=opacidade,
                    )
            else:
                vertices = (
                    _ponto_renderizacao(x, y, camera_primitiva),
                    _ponto_renderizacao(x + largura, y, camera_primitiva),
                    _ponto_renderizacao(x + largura, y + altura, camera_primitiva),
                    _ponto_renderizacao(x, y + altura, camera_primitiva),
                )
```

```
                if callable(desenhar_poligono):
                    desenhar_poligono(
                        vertices, cor,
                        contorno=max(1, int(estilo.espessura)),
                        opacidade=opacidade,
                    )
                else:
                    for i, (x1, y1) in enumerate(vertices):
                        x2, y2 = vertices[(i + 1) % len(vertices)]
                        desenhar_linha(
                            x1, y1, x2, y2, cor,
                            espessura=max(1, int(estilo.espessura)),
                            opacidade=opacidade,
                        )
            continue

        if primitiva.tipo == "poligono":
            vertices = _vertices_renderizacao(dados.get("vertices"), camera_primitiva)
            if len(vertices) < 2:
                continue
            if callable(desenhar_poligono):
                desenhar_poligono(
                    vertices, cor,
                    contorno=max(1, int(estilo.espessura)),
                    opacidade=opacidade,
                )
            else:
                limite = len(vertices) if dados.get("fechado", True) else len(vertices)
                  - 1
                for i in range(max(0, limite)):
                    x1, y1 = vertices[i]
                    x2, y2 = vertices[(i + 1) % len(vertices)]
                    desenhar_linha(
                        x1, y1, x2, y2, cor,
                        espessura=max(1, int(estilo.espessura)),
                        opacidade=opacidade,
                    )
            continue

        if primitiva.tipo == "ponto":
            x, y = _ponto_renderizacao(dados.get("x"), dados.get("y"), camera_primitiva)
            tamanho = int(dados.get("tamanho") or estilo.tamanho_pivo)
            desenhar_ponto(
                x, y, cor,
                tamanho=max(1, tamanho),
                opacidade=opacidade,
            )
            continue

        if primitiva.tipo == "texto":
            x, y = _ponto_renderizacao(dados.get("x"), dados.get("y"), camera_primitiva)
            desenhar_texto(
                str(dados.get("texto") or primitiva.rotulo or ""),
                x, y,
                max(1, int(estilo.tamanho_texto)),
                cor,
                opacidade=opacidade,
            )

    return overlay
```

```
__all__ = [
    "CONTRATO_OVERLAY_DEBUG",
    "PrimitivaDebug",
    "OverlayDebug",
    "gerar_overlay_debug",
    "EstiloOverlayDebug",
    "ConfiguracaoDebugJogo",
    "normalizar_categorias_debug",
    "gerar_overlay_metricas_debug",
    "desenhar_overlay_debug",
]

"""Adaptação 2D de ``coral.jogos`` para a baseline espacial da Coral.

A dependência é deliberadamente unidirecional: Jogos conhece a baseline,
mas ``coral._espacial`` nunca importa ou conhece sprites, cenas ou Pygame.
"""
from __future__ import annotations

from typing import Any

from coral._espacial import Poligono, poligono, ponto, posicao, sobrepoe


def _validar_sprite(sprite: Any) -> None:
    if not all(hasattr(sprite, nome) for nome in ("x", "y", "forma_colisao")):
        raise TypeError("adaptação espacial de jogos espera um Sprite compatível")


def posicao_espacial_sprite(sprite: Any):
    _validar_sprite(sprite)
    return posicao(sprite.x, sprite.y)


def forma_espacial_sprite(sprite: Any) -> Poligono:
    _validar_sprite(sprite)
    forma = sprite.forma_colisao
    vertices = getattr(forma, "vertices", None)
    if not vertices:
        raise TypeError("o colisor do sprite não fornece vértices 2D")
    return poligono(*(ponto(x, y) for x, y in vertices))


def sprites_colidem(a: Any, b: Any) -> bool:
    """Colisão 2D de Jogos usando sobreposição interior da baseline.

    A escolha por ``sobrepoe`` preserva o contrato histórico de Jogos em que
    encostar apenas pelas bordas não conta como colisão.
    """
    return sobrepoe(forma_espacial_sprite(a), forma_espacial_sprite(b))


__all__ = ["forma_espacial_sprite", "posicao_espacial_sprite", "sprites_colidem"]

from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Iterable

from .base import Jogo, _normalizar_botao_mouse
```

```
from .objetos import Sprite


@dataclass
class _ColisaoObservada:
    primeiro: Sprite
    segundo: Sprite
    nome: str
    ativa: bool = False


class AdaptadorEventosJogo:
    """Publica acontecimentos de Jogos no barramento reativo do próprio Jogo.

    O adaptador é deliberadamente opcional e depende apenas da API pública de
    ``Jogo``/``Sprite``. O núcleo reativo não importa Jogos e este módulo não
    importa Pygame.
    """

    def __init__(
        self,
        jogo: Jogo,
        *,
        teclas: Iterable[str] = (),
        botoes_mouse: Iterable[str] = (),
        observar_mouse: bool = False,
    ):
        if not isinstance(jogo, Jogo):
            raise TypeError("o adaptador de eventos espera um Jogo")
        self.jogo = jogo
        self.teclas = tuple(dict.fromkeys(str(t).strip().casefold() for t in teclas if
          str(t).strip()))
        self.botoes_mouse = tuple(dict.fromkeys(_normalizar_botao_mouse(b) for b in
          botoes_mouse))
        self.observar_mouse = bool(observar_mouse)
        self._teclas_anteriores = {tecla: bool(jogo.tecla_pressionada(tecla)) for tecla
          in self.teclas}
        self._posicao_mouse_anterior: tuple[int, int] | None = None
        self._colisoes: list[_ColisaoObservada] = []
        self._ativo = True
        jogo.quando_atualizar(self._atualizar)

    @property
    def ativo(self) -> bool:
        return self._ativo

    def desligar(self) -> None:
        self._ativo = False

    def ligar(self) -> None:
        self._ativo = True

    def observar_colisao(self, primeiro: Sprite, segundo: Sprite, *, nome: str | None =
      None) -> _ColisaoObservada:
        if not isinstance(primeiro, Sprite) or not isinstance(segundo, Sprite):
            raise TypeError("observação de colisão espera dois sprites")
        nome_colisao = str(nome).strip() if nome is not None else
          f"{primeiro.nome}:{segundo.nome}"
        if not nome_colisao:
            raise ValueError("o nome da colisão não pode ser vazio")
        observada = _ColisaoObservada(
```

```
            primeiro=primeiro,
            segundo=segundo,
            nome=nome_colisao,
            ativa=primeiro.colide_com(segundo),
        )
        self._colisoes.append(observada)
        return observada

    def publicar_transicao_visual(self, sprite: Sprite, transicao: str, *, estado: str |
       None = None):
        if not isinstance(sprite, Sprite):
            raise TypeError("transição visual espera um Sprite")
        nome_transicao = str(transicao).strip()
        if not nome_transicao:
            raise ValueError("o nome da transição visual não pode ser vazio")
        return self.jogo.eventos.emitir_evento(
            "transicao_visual",
            origem="jogos",
            metadados={"dominio": "jogos"},
            jogo=self.jogo,
            sprite=sprite,
            transicao=nome_transicao,
            estado=None if estado is None else str(estado),
        )

    def _emitir(self, nome: str, **dados: Any):
        return self.jogo.eventos.emitir_evento(
            nome,
            origem="jogos",
            metadados={"dominio": "jogos"},
            jogo=self.jogo,
            **dados,
        )

    def _atualizar_teclado(self) -> None:
        for tecla in self.teclas:
            atual = bool(self.jogo.tecla_pressionada(tecla))
            anterior = self._teclas_anteriores.get(tecla, False)
            if atual and not anterior:
                self._emitir("tecla_pressionada", tecla=tecla)
            elif anterior and not atual:
                self._emitir("tecla_solta", tecla=tecla)
            self._teclas_anteriores[tecla] = atual

    def _atualizar_mouse(self) -> None:
        posicao = self.jogo.posicao_mouse()
        if self.observar_mouse:
            if self._posicao_mouse_anterior is not None and posicao !=
              self._posicao_mouse_anterior:
                self._emitir("mouse_movido", x=posicao[0], y=posicao[1])
            self._posicao_mouse_anterior = posicao
            roda = self.jogo.roda_mouse()
            if roda != (0, 0):
                self._emitir("roda_mouse", horizontal=roda[0], vertical=roda[1])
        for botao in self.botoes_mouse:
            if self.jogo.mouse_clicado(botao):
                self._emitir("mouse_clicado", botao=botao, x=posicao[0], y=posicao[1])
            if self.jogo.mouse_solto(botao):
                self._emitir("mouse_solto", botao=botao, x=posicao[0], y=posicao[1])

    def _atualizar_colisoes(self) -> None:
```

```
        for observada in self._colisoes:
            atual = observada.primeiro.colide_com(observada.segundo)
            if atual and not observada.ativa:
                self._emitir(
                    "colisao_iniciada",
                    nome_colisao=observada.nome,
                    primeiro=observada.primeiro,
                    segundo=observada.segundo,
                )
            elif observada.ativa and not atual:
                self._emitir(
                    "colisao_encerrada",
                    nome_colisao=observada.nome,
                    primeiro=observada.primeiro,
                    segundo=observada.segundo,
                )
            observada.ativa = atual

    def _atualizar(self, jogo: Jogo, dt: float) -> None:
        if not self._ativo:
            return
        self._emitir("jogo_atualizado", dt=float(dt))
        self._atualizar_teclado()
        self._atualizar_mouse()
        self._atualizar_colisoes()


def adaptar_eventos(
    jogo: Jogo,
    *,
    teclas: Iterable[str] = (),
    botoes_mouse: Iterable[str] = (),
    observar_mouse: bool = False,
) -> AdaptadorEventosJogo:
    return AdaptadorEventosJogo(
        jogo,
        teclas=teclas,
        botoes_mouse=botoes_mouse,
        observar_mouse=observar_mouse,
    )


__all__ = ["AdaptadorEventosJogo", "adaptar_eventos"]

"""Conversão e visualização 2D opcional de mapas Coral.

O módulo é consumidor de ``coral.mundo``. O núcleo de mapas nunca importa
Jogos, câmera, backend gráfico ou Pygame.
"""

from __future__ import annotations

from dataclasses import dataclass
import math
from typing import Any, Callable, Iterable, Mapping

from coral._espacial import posicao
from coral._mapas import (
    CamadaMapa,
    ConteudoCelula,
    ErroMapaCoral,
```

```
    Mapa,
    TopologiaHexagonal,
    TopologiaQuadrada,
    TopologiaVoxel,
)

from .base import BRANCO, BackendJogos, Cor, Retangulo
from .objetos import Camera
from .viewport import Viewport
from .spritesheet import QuadroSprite


@dataclass(frozen=True)
class EstiloCelula:
    cor: Cor | None = None
    imagem: str | QuadroSprite | None = None
    texto: str | None = None
    cor_texto: Cor = BRANCO
    tamanho_texto: int = 16
    contorno: int = 0
    opacidade: float = 1.0


ResolvedorEstiloMapa = Callable[[Any, ConteudoCelula, CamadaMapa, Any], Any]


def _exigir_2d(mapa: Mapa) -> None:
    if isinstance(mapa.topologia, TopologiaVoxel) or mapa.topologia.dimensionalidade !=
      2:
        raise ErroMapaCoral(
            'Jogos 2D não projeta mapas voxel. Use o mapa voxel sem renderização ou um
              adaptador 3D específico.',
            codigo='mapa.projecao_voxel_nao_suportada',
            contexto={'mapa': mapa.nome, 'topologia': mapa.topologia.nome},
        )


def tela_para_mundo(camera: Camera | None, x: float, y: float, *, viewport: Viewport |
  None = None):
    if camera is None:
        raise ErroMapaCoral(
            'Converter coordenadas de tela para mundo exige uma câmera de Jogos.',
            codigo='mapa.conversao_tela_sem_camera',
        )
    if viewport is not None:
        x, y = viewport.tela_para_logica(x, y)
    wx, wy = camera.tela_para_mundo(x, y)
    return posicao(wx, wy)


def mundo_para_tela(camera: Camera | None, coordenada: Any, *, viewport: Viewport | None
   = None) -> tuple[float, float]:
    if camera is None:
        raise ErroMapaCoral(
            'Converter coordenadas de mundo para tela exige uma câmera de Jogos.',
            codigo='mapa.conversao_tela_sem_camera',
        )
    if not hasattr(coordenada, 'dimensoes') or coordenada.dimensoes != 2:
        raise ErroMapaCoral('Jogos 2D exige uma coordenada de mundo 2D.',
          codigo='mapa.dimensionalidade_incompativel')
    tela = camera.mundo_para_tela(coordenada.x, coordenada.y)
```

```
    return viewport.logica_para_tela(*tela) if viewport is not None else tela


def tela_para_celula(mapa: Mapa, camera: Camera | None, x: float, y: float, *, viewport:
   Viewport | None = None):
    _exigir_2d(mapa)
    return mapa.topologia.mundo_para_celula(tela_para_mundo(camera, x, y,
      viewport=viewport))


def celula_para_tela(
    mapa: Mapa,
    camera: Camera | None,
    celula: Any,
    *,
    ancora: str = 'centro',
    viewport: Viewport | None = None,
) -> tuple[float, float]:
    _exigir_2d(mapa)
    mundo = mapa.topologia.celula_para_mundo(celula, ancora=ancora)
    return mundo_para_tela(camera, mundo, viewport=viewport)


def _normalizar_estilo(valor: Any) -> EstiloCelula | None:
    if valor is None:
        return None
    if isinstance(valor, EstiloCelula):
        return valor
    if isinstance(valor, Cor):
        return EstiloCelula(cor=valor)
    if isinstance(valor, (str, QuadroSprite)):
        if isinstance(valor, QuadroSprite):
            return EstiloCelula(imagem=valor)
        return EstiloCelula(texto=valor)
    if isinstance(valor, Mapping):
        return EstiloCelula(
            cor=valor.get('cor'),
            imagem=valor.get('imagem', valor.get('quadro')),
            texto=None if valor.get('texto') is None else str(valor.get('texto')),
            cor_texto=valor.get('cor_texto', BRANCO),
            tamanho_texto=int(valor.get('tamanho_texto', 16)),
            contorno=int(valor.get('contorno', 0)),
            opacidade=float(valor.get('opacidade', 1.0)),
        )
    raise TypeError('resolvedor de estilo deve retornar EstiloCelula, Cor, texto,
      quadro, dicionário ou nulo')


def _resolver_padrao(valor: Any, conteudo: ConteudoCelula, camada: CamadaMapa, celula:
  Any) -> EstiloCelula:
    if isinstance(valor, Cor):
        return EstiloCelula(cor=valor)
    return EstiloCelula(texto=str(valor))


def _area_mundo(camera: Camera | None, area_visivel: tuple[float, float, float, float] |
   None) -> tuple[float, float, float, float]:
    if camera is not None:
        area = camera.area_visivel
        return float(area.x), float(area.y), float(area.largura), float(area.altura)
    if area_visivel is None:
```

```
        raise ErroMapaCoral(
            'Renderizar um mapa sem câmera exige uma área visível explícita.',
            codigo='mapa.area_visivel_ausente',
        )
    if len(area_visivel) != 4:
        raise ErroMapaCoral('A área visível deve conter x, y, largura e altura.',
          codigo='mapa.area_visivel_invalida')
    x, y, largura, altura = (float(v) for v in area_visivel)
    if not all(math.isfinite(v) for v in (x, y, largura, altura)) or largura <= 0 or
      altura <= 0:
        raise ErroMapaCoral('A área visível precisa ser finita e positiva.',
          codigo='mapa.area_visivel_invalida')
    return x, y, largura, altura


def _para_tela(camera: Camera | None, area: tuple[float, float, float, float], x: float,
   y: float) -> tuple[float, float]:
    if camera is not None:
        return camera.mundo_para_tela(x, y)
    return x - area[0], y - area[1]


def _intervalo_limitado(inicio: int, fim: int, minimo: int | None, maximo: int | None)
  -> range:
    if minimo is not None:
        inicio = max(inicio, minimo)
    if maximo is not None:
        fim = min(fim, maximo)
    return range(inicio, fim + 1) if fim >= inicio else range(0)


def _candidatas_quadradas(topologia: TopologiaQuadrada, area: tuple[float, float, float,
   float]):
    x, y, largura, altura = area
    c0 = math.floor((x - topologia.origem[0]) / topologia.largura_celula) - 1
    c1 = math.floor((x + largura - topologia.origem[0]) / topologia.largura_celula) + 1
    l0 = math.floor((y - topologia.origem[1]) / topologia.altura_celula) - 1
    l1 = math.floor((y + altura - topologia.origem[1]) / topologia.altura_celula) + 1
    limites = topologia.limites
    minc = minl = maxc = maxl = None
    if limites is not None:
        (minc, minl), (maxc, maxl) = limites
    for coluna in _intervalo_limitado(c0, c1, minc, maxc):
        for linha in _intervalo_limitado(l0, l1, minl, maxl):
            yield topologia.normalizar((coluna, linha))


def _candidatas_hexagonais(topologia: TopologiaHexagonal, area: tuple[float, float,
  float, float]):
    x, y, largura, altura = area
    pontos = [
        posicao(x, y), posicao(x + largura, y), posicao(x, y + altura),
        posicao(x + largura, y + altura), posicao(x + largura / 2, y + altura / 2),
    ]
    celulas = []
    for ponto_mundo in pontos:
        try:
            celulas.append(topologia.mundo_para_celula(ponto_mundo))
        except ErroMapaCoral as exc:
            if exc.codigo != 'mapa.celula_fora_dos_limites':
                raise
```

```
            # Para mapas finitos, cantos externos ainda podem cercar a área útil.
    if not celulas and topologia.limites is not None:
        (minq, minr), (maxq, maxr) = topologia.limites
    elif celulas:
        minq = min(c.q for c in celulas) - 2
        maxq = max(c.q for c in celulas) + 2
        minr = min(c.r for c in celulas) - 2
        maxr = max(c.r for c in celulas) + 2
        if topologia.limites is not None:
            (lminq, lminr), (lmaxq, lmaxr) = topologia.limites
            minq, minr = max(minq, lminq), max(minr, lminr)
            maxq, maxr = min(maxq, lmaxq), min(maxr, lmaxr)
    else:
        return
    margem = topologia.tamanho_celula * 2
    for q in range(minq, maxq + 1):
        for r in range(minr, maxr + 1):
            celula = topologia.normalizar((q, r))
            centro = topologia.celula_para_mundo(celula)
            if x - margem <= centro.x <= x + largura + margem and y - margem <= centro.y
               <= y + altura + margem:
                yield celula


def _vertices_hex(topologia: TopologiaHexagonal, centro_x: float, centro_y: float) ->
  tuple[tuple[float, float], ...]:
    inicio = 30.0 if topologia.orientacao == 'topo_pontudo' else 0.0
    return tuple(
        (
            centro_x + topologia.tamanho_celula * math.cos(math.radians(inicio + 60 *
              i)),
            centro_y + topologia.tamanho_celula * math.sin(math.radians(inicio + 60 *
              i)),
        )
        for i in range(6)
    )


class RenderizadorMapa2D:
    def __init__(
        self,
        mapa: Mapa,
        *,
        camadas: Iterable[str] | None = None,
        camera: Camera | None = None,
        resolvedor_estilo: ResolvedorEstiloMapa | None = None,
        area_visivel: tuple[float, float, float, float] | None = None,
    ) -> None:
        _exigir_2d(mapa)
        self.mapa = mapa
        self.camera = camera
        self.area_visivel = area_visivel
        self.resolvedor_estilo = resolvedor_estilo or _resolver_padrao
        self._camadas = None if camadas is None else tuple(str(nome) for nome in
          camadas)

    @property
    def camadas(self) -> tuple[CamadaMapa, ...]:
        if self._camadas is None:
            return self.mapa.camadas
        return tuple(self.mapa.exigir_camada(nome) for nome in self._camadas)
```

```
    def celulas_visiveis(self) -> tuple[Any, ...]:
        area = _area_mundo(self.camera, self.area_visivel)
        topologia = self.mapa.topologia
        if isinstance(topologia, TopologiaQuadrada):
            return tuple(_candidatas_quadradas(topologia, area))
        if isinstance(topologia, TopologiaHexagonal):
            return tuple(_candidatas_hexagonais(topologia, area))
        _exigir_2d(self.mapa)
        return ()

    def desenhar(self, backend: BackendJogos) -> int:
        area = _area_mundo(self.camera, self.area_visivel)
        candidatas = self.celulas_visiveis()
        desenhadas = 0
        for camada in self.camadas:
            for celula in candidatas:
                conteudo = camada.conteudo(celula)
                if conteudo is None:
                    continue
                estilo = _normalizar_estilo(self.resolvedor_estilo(conteudo.valor,
                  conteudo, camada, celula))
                if estilo is None:
                    continue
                self._desenhar_celula(backend, celula, estilo, area)
                desenhadas += 1
        return desenhadas

    def _desenhar_celula(self, backend: BackendJogos, celula: Any, estilo: EstiloCelula,
       area) -> None:
        topologia = self.mapa.topologia
        if isinstance(topologia, TopologiaQuadrada):
            origem = topologia.celula_para_mundo(celula, ancora='origem')
            x, y = _para_tela(self.camera, area, origem.x, origem.y)
            largura, altura = topologia.largura_celula, topologia.altura_celula
            cx, cy = x + largura / 2, y + altura / 2
            if estilo.cor is not None:
                backend.desenhar_retangulo(
                    Retangulo(x, y, largura, altura), estilo.cor,
                    contorno=estilo.contorno, opacidade=estilo.opacidade,
                )
        elif isinstance(topologia, TopologiaHexagonal):
            centro = topologia.celula_para_mundo(celula)
            cx, cy = _para_tela(self.camera, area, centro.x, centro.y)
            vertices = _vertices_hex(topologia, cx, cy)
            cor_linha = estilo.cor or estilo.cor_texto
            espessura = max(1, estilo.contorno or 1)
            for i, (x1, y1) in enumerate(vertices):
                x2, y2 = vertices[(i + 1) % len(vertices)]
                backend.desenhar_linha(x1, y1, x2, y2, cor_linha, espessura=espessura,
                  opacidade=estilo.opacidade)
            if topologia.orientacao == 'topo_pontudo':
                largura, altura = math.sqrt(3) * topologia.tamanho_celula, 2 *
                  topologia.tamanho_celula
            else:
                largura, altura = 2 * topologia.tamanho_celula, math.sqrt(3) *
                  topologia.tamanho_celula
        else:
            _exigir_2d(self.mapa)
            return
```

```
        if estilo.imagem is not None:
            backend.desenhar_imagem(
                estilo.imagem, cx, cy, largura, altura,
                origem='centro', opacidade=estilo.opacidade,
            )
        if estilo.texto is not None:
            backend.desenhar_texto(
                estilo.texto, cx, cy, estilo.tamanho_texto, estilo.cor_texto,
                origem='centro', opacidade=estilo.opacidade,
            )


__all__ = [
    'EstiloCelula', 'RenderizadorMapa2D', 'ResolvedorEstiloMapa',
    'celula_para_tela', 'mundo_para_tela', 'tela_para_celula', 'tela_para_mundo',
]

from __future__ import annotations
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any, Callable
import math
from .base import Retangulo, Cor, BRANCO, _normalizar_transformacao_imagem
from .spritesheet import QuadroSprite, Spritesheet
from .transformacoes import avaliar_curva_temporal, ClipeAnimacao, FaixaAnimacao,
  QuadroChave
from coral.direcoes import MovimentoDirecional


class ErroAssetCoral(Exception):
    pass


class Assets:
    def __init__(self, raiz: str | Path = "assets"):
        self.raiz = Path(raiz).expanduser().resolve()

    def _resolver(self, relativo: str | Path) -> Path:
        alvo = (self.raiz / relativo).resolve()
        try:
            alvo.relative_to(self.raiz)
        except ValueError as exc:
            raise ErroAssetCoral("o asset precisa permanecer dentro da pasta de assets")
               from exc
        if not alvo.is_file():
            raise ErroAssetCoral(f"asset não encontrado: {alvo}")
        return alvo

    def imagem(self, nome: str | Path) -> Path:
        return self._resolver(nome)

    def som(self, nome: str | Path) -> Path:
        return self._resolver(nome)

    def spritesheet(
        self,
        nome: str | Path,
        *,
        largura_imagem: int,
        altura_imagem: int,
        largura_quadro: int,
```

```
        altura_quadro: int,
        margem: int = 0,
        espacamento: int = 0,
    ) -> Spritesheet:
        return Spritesheet.em_grade(
            self._resolver(nome),
            largura_imagem=largura_imagem,
            altura_imagem=altura_imagem,
            largura_quadro=largura_quadro,
            altura_quadro=altura_quadro,
            margem=margem,
            espacamento=espacamento,
        )




def _numero_finito(valor, nome: str) -> float:
    if isinstance(valor, bool):
        raise ValueError(f"{nome} deve ser um número finito")
    try:
        numero=float(valor)
    except (TypeError,ValueError) as exc:
        raise ValueError(f"{nome} deve ser um número finito") from exc
    if not math.isfinite(numero):
        raise ValueError(f"{nome} deve ser um número finito")
    return numero

def _percentual(valor, nome: str) -> float:
    numero=_numero_finito(valor,nome)
    if numero < 0:
        raise ValueError(f"{nome} deve ser não negativo")
    return numero

@dataclass
class Sprite:
    x: float
    y: float
    largura: float
    altura: float
    imagem: str | Path | QuadroSprite | None = None
    cor: Cor = BRANCO
    velocidade_x: float = 0.0
    velocidade_y: float = 0.0
    visivel: bool = True
    nome: str = "sprite"
    dados: dict[str, Any] = field(default_factory=dict)
    animacao: Any = None
    escala: float = 1.0
    rotacao: float = 0.0
    espelhado_horizontalmente: bool = False
    espelhado_verticalmente: bool = False
    origem: str = "topo_esquerdo"
    pivo: tuple[float, float] | None = None
    opacidade: float = 1.0
    colisao_acompanha_escala: bool = False
    colisao_acompanha_rotacao: bool = False
    _automacoes_transformacao: list[Any] = field(default_factory=list, init=False,
      repr=False)
    _vinculos_clipe: list[Any] = field(default_factory=list, init=False, repr=False)
    _vinculos_transicoes_animacao: list[Any] = field(default_factory=list, init=False,
```

```
      repr=False)

    def __post_init__(self) -> None:
        transformacao = _normalizar_transformacao_imagem(
            escala=self.escala,
            rotacao=self.rotacao,
            espelhar_horizontalmente=self.espelhado_horizontalmente,
            espelhar_verticalmente=self.espelhado_verticalmente,
            origem=self.origem,
            pivo=self.pivo,
        )
        self.escala, self.rotacao = transformacao[0], transformacao[1]
        self.espelhado_horizontalmente, self.espelhado_verticalmente = transformacao[2],
           transformacao[3]
        self.origem, self.pivo = transformacao[4], transformacao[5]
        self.definir_opacidade(self.opacidade)
        self.colisao_acompanha_escala = bool(self.colisao_acompanha_escala)
        self.colisao_acompanha_rotacao = bool(self.colisao_acompanha_rotacao)

    def estado_inspecao(self) -> dict[str, Any]:
        limites = self.limites_visuais
        forma = self.forma_colisao
        return {
            "nome": self.nome,
            "posicao": {"x": self.x, "y": self.y},
            "tamanho": {"largura": self.largura, "altura": self.altura},
            "velocidade": {"x": self.velocidade_x, "y": self.velocidade_y},
            "visivel": self.visivel,
            "transformacao": {
                "escala": self.escala,
                "rotacao": self.rotacao,
                "opacidade": self.opacidade,
                "origem": self.origem,
                "pivo": self.pivo,
                "espelhado_horizontalmente": self.espelhado_horizontalmente,
                "espelhado_verticalmente": self.espelhado_verticalmente,
            },
            "colisao": {
                "acompanha_escala": self.colisao_acompanha_escala,
                "acompanha_rotacao": self.colisao_acompanha_rotacao,
                "vertices": tuple(forma.vertices),
            },
            "limites_visuais": {
                "x": limites.x,
                "y": limites.y,
                "largura": limites.largura,
                "altura": limites.altura,
            },
            "automacoes_ativas": sum(
                getattr(item, "estado", None) == "ativo"
                for item in self._automacoes_transformacao
            ),
            "clipes_ativos": sum(
                not getattr(item, "finalizado", False)
                for item in self._vinculos_clipe
            ),
            "animacao": self.animacao,
            "transicoes_animacao": tuple(
                {
                    "estado_atual": getattr(item, "estado_atual", None),
                    "estado_anterior": getattr(item, "estado_anterior", None),
```

```
                "tempo_no_estado": getattr(item, "tempo_no_estado", None),
                "bloqueio_restante": getattr(item, "bloqueio_restante", None),
                "prioridade_atual": getattr(item, "prioridade_atual", None),
                "ultimo_evento": getattr(item, "ultimo_evento", None),
            }
            for item in self._vinculos_transicoes_animacao
        ),
    }

@property
def retangulo(self) -> Retangulo:
    # Compatibilidade histórica: esta propriedade ignora transformações visuais.
    return Retangulo(self.x, self.y, self.largura, self.altura)

@property
def limites_visuais(self) -> Retangulo:
    """Retângulo conservador da imagem após escala, espelho e rotação.

    O cálculo usa o mesmo contrato de pivô do backend gráfico e permite
    culling sem depender de Pygame nem do tamanho físico da janela.
    """
    px, py = self._pivo_local()
    escala = float(self.escala)
    largura = float(self.largura) * escala
    altura = float(self.altura) * escala
    pivo_x, pivo_y = px * escala, py * escala
    pontos = [
        [0.0, 0.0],
        [largura, 0.0],
        [largura, altura],
        [0.0, altura],
    ]
    if self.espelhado_horizontalmente:
        for ponto in pontos:
            ponto[0] = largura - ponto[0]
        pivo_x = largura - pivo_x
    if self.espelhado_verticalmente:
        for ponto in pontos:
            ponto[1] = altura - ponto[1]
        pivo_y = altura - pivo_y
    angulo = math.radians(float(self.rotacao))
    coseno, seno = math.cos(angulo), math.sin(angulo)
    globais = []
    for local_x, local_y in pontos:
        dx, dy = local_x - pivo_x, local_y - pivo_y
        rx = dx * coseno + dy * seno
        ry = -dx * seno + dy * coseno
        globais.append((float(self.x) + rx, float(self.y) + ry))
    xs = [p[0] for p in globais]
    ys = [p[1] for p in globais]
    minimo_x, maximo_x = min(xs), max(xs)
    minimo_y, maximo_y = min(ys), max(ys)
    return Retangulo(minimo_x, minimo_y, maximo_x - minimo_x, maximo_y - minimo_y)

def _pivo_local(self) -> tuple[float, float]:
    if self.pivo is not None:
        return self.pivo
    if self.origem == "centro":
        return (self.largura / 2.0, self.altura / 2.0)
    if self.origem == "centro_inferior":
        return (self.largura / 2.0, self.altura)
```

```
        return (0.0, 0.0)

    @property
    def forma_colisao(self):
        from .transformacoes import FormaTransformada
        if not self.colisao_acompanha_escala and not self.colisao_acompanha_rotacao:
            r = self.retangulo
            return FormaTransformada(((r.x, r.y), (r.x+r.largura, r.y), (r.x+r.largura,
              r.y+r.altura), (r.x, r.y+r.altura)))
        px, py = self._pivo_local()
        escala = self.escala if self.colisao_acompanha_escala else 1.0
        angulo = math.radians(self.rotacao if self.colisao_acompanha_rotacao else 0.0)
        coseno, seno = math.cos(angulo), math.sin(angulo)
        vertices=[]
        for lx, ly in
          ((0.0,0.0),(self.largura,0.0),(self.largura,self.altura),(0.0,self.altura)):
            dx=(lx-px)*escala; dy=(ly-py)*escala
            rx = dx*coseno + dy*seno
            ry = -dx*seno + dy*coseno
            vertices.append((float(self.x)+rx,float(self.y)+ry))
        return FormaTransformada(tuple(vertices))

    def mover(self, dx: float, dy: float) -> None:
        self.x += dx
        self.y += dy

    def definir_escala(self, escala: float):
        self.escala = _normalizar_transformacao_imagem(escala=escala,
          rotacao=self.rotacao, origem=self.origem, pivo=self.pivo)[0]
        return self

    def aumentar(self, percentual: float):
        percentual = _percentual(percentual, "aumento")
        return self.definir_escala(self.escala * (1.0 + percentual / 100.0))

    def diminuir(self, percentual: float):
        percentual = _percentual(percentual, "redução")
        if percentual >= 100:
            raise ValueError("redução da escala deve ser menor que 100 por cento")
        return self.definir_escala(self.escala * (1.0 - percentual / 100.0))

    def definir_rotacao(self, graus: float):
        self.rotacao = _normalizar_transformacao_imagem(escala=self.escala,
          rotacao=graus, origem=self.origem, pivo=self.pivo)[1]
        return self

    def girar(self, graus: float):
        return self.definir_rotacao(self.rotacao + _numero_finito(graus, "rotação"))

    def definir_opacidade(self, opacidade: float):
        opacidade = _numero_finito(opacidade, "opacidade")
        if not 0.0 <= opacidade <= 1.0:
            raise ValueError("opacidade deve estar entre 0 e 1")
        self.opacidade = opacidade
        return self

    def alternar_espelhamento(self, eixo: str):
        eixo = str(eixo).strip().casefold()
        if eixo in {"horizontal", "horizontalmente"}:
            self.espelhado_horizontalmente = not self.espelhado_horizontalmente
        elif eixo in {"vertical", "verticalmente"}:
```

```
            self.espelhado_verticalmente = not self.espelhado_verticalmente
        else:
            raise ValueError("eixo deve ser horizontal ou vertical")
        return self

    def remover_espelhamento(self, eixo: str):
        eixo = str(eixo).strip().casefold()
        if eixo in {"horizontal", "horizontalmente"}:
            self.espelhado_horizontalmente = False
        elif eixo in {"vertical", "verticalmente"}:
            self.espelhado_verticalmente = False
        else:
            raise ValueError("eixo deve ser horizontal ou vertical")
        return self

    def restaurar_transformacoes(self):
        self.escala = 1.0
        self.rotacao = 0.0
        self.espelhado_horizontalmente = False
        self.espelhado_verticalmente = False
        self.origem = "topo_esquerdo"
        self.pivo = None
        return self

    def configurar_colisao(self, *, acompanhar_escala: bool | None = None,
      acompanhar_rotacao: bool | None = None):
        if acompanhar_escala is not None:
            self.colisao_acompanha_escala = bool(acompanhar_escala)
        if acompanhar_rotacao is not None:
            self.colisao_acompanha_rotacao = bool(acompanhar_rotacao)
        return self

    def manter_colisao_fixa(self):
        self.colisao_acompanha_escala = False
        self.colisao_acompanha_rotacao = False
        return self

    def animar_transformacoes(self, **canais):
        from .transformacoes import AnimacaoTransformacao, CanalAnimado
        criados=[]
        for nome, configuracao in canais.items():
            if configuracao is None:
                continue
            if not isinstance(configuracao, dict):
                raise TypeError("cada canal da automação deve usar um dicionário de
                  configuração")
            atual = float(getattr(self, nome))
            cfg=dict(configuracao)
            base=cfg.pop("base", atual)
            criados.append(CanalAnimado(nome, base, cfg.pop("amplitude",0.0),
              cfg.pop("frequencia",1.0), fase=cfg.pop("fase",0.0),
              modo=cfg.pop("modo","absoluto"), onda=cfg.pop("onda","seno")))
            if cfg:
                raise TypeError(f"opções desconhecidas na automação de {nome}: {',
                  '.join(sorted(cfg))}")
        automacao=AnimacaoTransformacao(tuple(criados))
        self._automacoes_transformacao.append(automacao)
        return automacao

    def _aplicar_automacao(self, automacao, dt: float):
        if automacao.estado != "ativo":
```

```
            return
        automacao.avancar(dt)
        valores=[]
        for canal in automacao.canais:
            referencia=float(getattr(self, canal.canal)) if canal.modo == "relativo"
              else 0.0
            valores.append((canal, canal.valor(automacao.tempo, referencia=referencia)))
        # Valide todos os valores antes de mutar qualquer canal.
        for canal, valor in valores:
            if canal.canal == "escala" and valor <= 0:
                raise ValueError("automação de escala produziria valor não positivo")
            if canal.canal == "opacidade" and not 0.0 <= valor <= 1.0:
                raise ValueError("automação de opacidade produziria valor fora de 0 a
                  1")
        for canal, valor in valores:
            if canal.canal == "escala": self.definir_escala(valor)
            elif canal.canal == "rotacao": self.definir_rotacao(valor)
            elif canal.canal == "opacidade": self.definir_opacidade(valor)
            else: setattr(self, canal.canal, valor)

    def pausar_automacao(self, automacao=None):
        for item in (self._automacoes_transformacao if automacao is None else
          [automacao]): item.pausar()

    def retomar_automacao(self, automacao=None):
        for item in (self._automacoes_transformacao if automacao is None else
          [automacao]): item.retomar()

    def reiniciar_automacao(self, automacao=None):
        for item in (self._automacoes_transformacao if automacao is None else
          [automacao]): item.reiniciar()

    def remover_automacao(self, automacao, *, restaurar_base: bool = False):
        if automacao not in self._automacoes_transformacao:
            return False
        if restaurar_base:
            for canal in automacao.canais:
                if canal.canal == "escala": self.definir_escala(canal.base)
                elif canal.canal == "rotacao": self.definir_rotacao(canal.base)
                elif canal.canal == "opacidade": self.definir_opacidade(canal.base)
                else: setattr(self, canal.canal, canal.base)
        automacao.encerrar()
        self._automacoes_transformacao.remove(automacao)
        return True

    def animar_com_clipe(
        self,
        clipe: ClipeAnimacao,
        mapeamento: dict[str, str],
        *,
        modo: str = "absoluto",
        restaurar_ao_final: bool = False,
        composicao: str = "substituir",
        permitir_conflito: bool = False,
    ):
        return ligar_clipe(
            self,
            clipe,
            mapeamento,
            modo=modo,
            restaurar_ao_final=restaurar_ao_final,
```

```
            composicao=composicao,
            permitir_conflito=permitir_conflito,
        )

    def atualizar(self, dt: float) -> None:
        self.x += self.velocidade_x * dt
        self.y += self.velocidade_y * dt
        if self.animacao is not None:
            self.imagem = self.animacao.atualizar(dt)
        for vinculo_estado in tuple(self._vinculos_transicoes_animacao):
            vinculo_estado.atualizar(dt)
        for automacao in tuple(self._automacoes_transformacao):
            self._aplicar_automacao(automacao, dt)
        for vinculo in tuple(self._vinculos_clipe):
            vinculo.atualizar(dt)
            if vinculo.finalizado:
                self._vinculos_clipe.remove(vinculo)

    def colide_com(self, outro: "Sprite") -> bool:
        if not isinstance(outro, Sprite):
            raise TypeError("colisão de sprite espera outro Sprite")
        from .espacial import sprites_colidem
        return sprites_colidem(self, outro)


@dataclass
class RepresentacaoEntidade:
    """Liga uma entidade de mundo a um Sprite sem acoplar coral.mundo a jogos.

    A entidade é a fonte de verdade para a posição. O adaptador apenas lê
    atributos nomeados e atualiza o sprite visual correspondente.
    """

    entidade: Any
    sprite: Sprite
    atributo_x: str = "x"
    atributo_y: str = "y"

    def __post_init__(self) -> None:
        self.atributo_x = str(self.atributo_x)
        self.atributo_y = str(self.atributo_y)
        if not self.atributo_x or not self.atributo_y:
            raise ValueError("os nomes dos atributos de posição não podem ser vazios")

    def _obter(self, nome: str):
        ausente = object()
        obter = getattr(self.entidade, "obter", None)
        if callable(obter):
            valor = obter(nome, ausente)
        else:
            valor = getattr(self.entidade, nome, ausente)
        if valor is ausente:
            raise ValueError(f"a entidade não possui o atributo {nome!r} usado pela
              representação")
        if isinstance(valor, bool) or not isinstance(valor, (int, float)) or not
          math.isfinite(float(valor)):
            raise TypeError(f"o atributo {nome!r} usado pela representação precisa ser
              um número finito")
        return valor

    @property
```

```
    def x(self) -> float:
        return float(self.sprite.x)

    @property
    def y(self) -> float:
        return float(self.sprite.y)

    @property
    def largura(self) -> float:
        return float(self.sprite.largura)

    @property
    def altura(self) -> float:
        return float(self.sprite.altura)

    @property
    def velocidade_x(self) -> float:
        return float(getattr(self.sprite, "velocidade_x", 0.0))

    @property
    def velocidade_y(self) -> float:
        return float(getattr(self.sprite, "velocidade_y", 0.0))

    def sincronizar(self) -> Sprite:
        self.sprite.x = self._obter(self.atributo_x)
        self.sprite.y = self._obter(self.atributo_y)
        return self.sprite


@dataclass
class CamadaApresentacao:
    """Camada 2D ordenada que pode pertencer ao mundo ou à tela.

    A lista de elementos é estável: elementos com a mesma camada mantêm a
    ordem de inserção. ``espaco='tela'`` ignora a câmera e é apropriado para
    HUD. Paralaxe só afeta camadas de mundo.
    """

    nome: str
    ordem: int = 0
    visivel: bool = True
    opacidade: float = 1.0
    espaco: str = "mundo"
    paralaxe_x: float = 1.0
    paralaxe_y: float = 1.0
    elementos: list[Any] = field(default_factory=list)

    def __post_init__(self) -> None:
        self.nome = str(self.nome).strip()
        if not self.nome:
            raise ValueError("camada precisa de nome")
        self.ordem = int(self.ordem)
        self.visivel = bool(self.visivel)
        self.opacidade = _numero_finito(self.opacidade, "opacidade da camada")
        if not 0.0 <= self.opacidade <= 1.0:
            raise ValueError("opacidade da camada deve estar entre 0 e 1")
        self.espaco = str(self.espaco).strip().casefold()
        if self.espaco not in {"mundo", "tela"}:
            raise ValueError("espaço da camada deve ser mundo ou tela")
        self.paralaxe_x = _numero_finito(self.paralaxe_x, "paralaxe x")
        self.paralaxe_y = _numero_finito(self.paralaxe_y, "paralaxe y")
```

```
    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "nome": self.nome,
            "ordem": self.ordem,
            "visivel": self.visivel,
            "opacidade": self.opacidade,
            "espaco": self.espaco,
            "paralaxe": {"x": self.paralaxe_x, "y": self.paralaxe_y},
            "quantidade_elementos": len(self.elementos),
        }

    @property
    def presa_a_tela(self) -> bool:
        return self.espaco == "tela"

    def adicionar(self, elemento: Any) -> Any:
        if elemento not in self.elementos:
            self.elementos.append(elemento)
        return elemento

    def remover(self, elemento: Any) -> bool:
        if elemento not in self.elementos:
            return False
        self.elementos.remove(elemento)
        return True

    def definir_visibilidade(self, visivel: bool) -> "CamadaApresentacao":
        self.visivel = bool(visivel)
        return self

    def definir_opacidade(self, opacidade: float) -> "CamadaApresentacao":
        opacidade = _numero_finito(opacidade, "opacidade da camada")
        if not 0.0 <= opacidade <= 1.0:
            raise ValueError("opacidade da camada deve estar entre 0 e 1")
        self.opacidade = opacidade
        return self

    def definir_paralaxe(self, x: float = 1.0, y: float | None = None) ->
      "CamadaApresentacao":
        self.paralaxe_x = _numero_finito(x, "paralaxe x")
        self.paralaxe_y = self.paralaxe_x if y is None else _numero_finito(y, "paralaxe
          y")
        return self


_ANCORAS_VIEWPORT = {
    "topo_esquerdo": (0.0, 0.0),
    "topo": (0.5, 0.0),
    "topo_direito": (1.0, 0.0),
    "esquerda": (0.0, 0.5),
    "centro": (0.5, 0.5),
    "direita": (1.0, 0.5),
    "base_esquerda": (0.0, 1.0),
    "base": (0.5, 1.0),
    "base_direita": (1.0, 1.0),
}


def _normalizar_ancora_viewport(valor: str | None) -> str | None:
    if valor is None:
```

```
        return None
    nome = str(valor).strip().casefold().replace(" ", "_")
    aliases = {
        "inferior_esquerdo": "base_esquerda",
        "inferior": "base",
        "inferior_direito": "base_direita",
        "centro_esquerdo": "esquerda",
        "centro_direito": "direita",
    }
    nome = aliases.get(nome, nome)
    if nome not in _ANCORAS_VIEWPORT:
        raise ValueError("âncora de viewport inválida")
    return nome


@dataclass
class TextoApresentacao:
    """Texto 2D reutilizável para camadas de mundo ou de tela.

    A classe cuida somente de apresentação. Não cria widgets, não captura
    entrada e não altera estado lógico. A quebra de linha usa a métrica do
    backend por meio de ``Jogo.medir_texto`` e, portanto, também funciona em
    execução headless.
    """

    texto: str
    x: float = 0.0
    y: float = 0.0
    tamanho: int = 24
    cor: Cor = BRANCO
    largura_maxima: float | None = None
    alinhamento_horizontal: str = "esquerda"
    alinhamento_vertical: str = "topo"
    espacamento_linhas: float = 1.0
    sombra: Cor | None = None
    deslocamento_sombra: tuple[float, float] = (2.0, 2.0)
    contorno: Cor | None = None
    espessura_contorno: int = 1
    opacidade: float = 1.0
    origem: str | None = None
    pivo: tuple[float, float] | None = None
    ancora_viewport: str | None = None

    def __post_init__(self) -> None:
        self.texto = str(self.texto)
        self.x = _numero_finito(self.x, "x do texto")
        self.y = _numero_finito(self.y, "y do texto")
        if isinstance(self.tamanho, bool) or int(self.tamanho) <= 0:
            raise ValueError("tamanho do texto deve ser um inteiro positivo")
        self.tamanho = int(self.tamanho)
        if self.largura_maxima is not None:
            self.largura_maxima = _numero_finito(self.largura_maxima, "largura máxima do
               texto")
            if self.largura_maxima <= 0:
                raise ValueError("largura máxima do texto deve ser positiva")
        self.alinhamento_horizontal =
          str(self.alinhamento_horizontal).strip().casefold()
        self.alinhamento_vertical = str(self.alinhamento_vertical).strip().casefold()
        if self.alinhamento_horizontal not in {"esquerda", "centro", "direita"}:
            raise ValueError("alinhamento horizontal deve ser esquerda, centro ou
              direita")
```

```
        if self.alinhamento_vertical not in {"topo", "centro", "base"}:
            raise ValueError("alinhamento vertical deve ser topo, centro ou base")
        self.espacamento_linhas = _numero_finito(self.espacamento_linhas, "espaçamento
          entre linhas")
        if self.espacamento_linhas <= 0:
            raise ValueError("espaçamento entre linhas deve ser positivo")
        self.opacidade = _numero_finito(self.opacidade, "opacidade do texto")
        if not 0.0 <= self.opacidade <= 1.0:
            raise ValueError("opacidade do texto deve estar entre 0 e 1")
        if isinstance(self.espessura_contorno, bool) or int(self.espessura_contorno) <
          0:
            raise ValueError("espessura do contorno deve ser um inteiro não negativo")
        self.espessura_contorno = int(self.espessura_contorno)
        if len(self.deslocamento_sombra) != 2:
            raise ValueError("deslocamento da sombra precisa conter x e y")
        self.deslocamento_sombra = (
            _numero_finito(self.deslocamento_sombra[0], "deslocamento x da sombra"),
            _numero_finito(self.deslocamento_sombra[1], "deslocamento y da sombra"),
        )
        if self.pivo is not None:
            if len(self.pivo) != 2:
                raise ValueError("pivô do texto precisa conter x e y")
            self.pivo = (
                _numero_finito(self.pivo[0], "pivô x do texto"),
                _numero_finito(self.pivo[1], "pivô y do texto"),
            )
        if self.origem is not None:
            origem = str(self.origem).strip().casefold().replace(" ", "_")
            if origem not in {"topo_esquerdo", "centro", "centro_inferior"}:
                raise ValueError("origem do texto inválida")
            self.origem = origem
        self.ancora_viewport = _normalizar_ancora_viewport(self.ancora_viewport)

    def _quebrar_palavra(self, jogo, palavra: str, limite: float) -> list[str]:
        partes: list[str] = []
        atual = ""
        for caractere in palavra:
            candidato = atual + caractere
            if atual and jogo.medir_texto(candidato, self.tamanho)[0] > limite:
                partes.append(atual)
                atual = caractere
            else:
                atual = candidato
        if atual or not partes:
            partes.append(atual)
        return partes

    def linhas(self, jogo) -> tuple[str, ...]:
        if self.largura_maxima is None:
            return tuple(self.texto.split("\n"))
        limite = float(self.largura_maxima)
        resultado: list[str] = []
        for paragrafo in self.texto.split("\n"):
            if not paragrafo:
                resultado.append("")
                continue
            linha = ""
            for palavra in paragrafo.split():
                if jogo.medir_texto(palavra, self.tamanho)[0] > limite:
                    if linha:
                        resultado.append(linha)
```

```
                    linha = ""
                partes = self._quebrar_palavra(jogo, palavra, limite)
                resultado.extend(partes[:-1])
                linha = partes[-1]
                continue
            candidato = palavra if not linha else f"{linha} {palavra}"
            if linha and jogo.medir_texto(candidato, self.tamanho)[0] > limite:
                resultado.append(linha)
                linha = palavra
            else:
                linha = candidato
        resultado.append(linha)
    return tuple(resultado or [""])

def medir(self, jogo) -> tuple[float, float]:
    linhas = self.linhas(jogo)
    medidas = [jogo.medir_texto(linha, self.tamanho) for linha in linhas]
    altura_linha = jogo.medir_texto("Ag", self.tamanho)[1]
    largura_conteudo = max((largura for largura, _ in medidas), default=0.0)
    largura = float(self.largura_maxima) if self.largura_maxima is not None else
      largura_conteudo
    altura = altura_linha + max(0, len(linhas) - 1) * altura_linha *
      self.espacamento_linhas
    return largura, altura

def _referencia(self, jogo, cena: "Cena", camada: CamadaApresentacao) ->
  tuple[float, float]:
    if self.ancora_viewport is not None:
        fx, fy = _ANCORAS_VIEWPORT[self.ancora_viewport]
        return jogo.largura * fx + self.x, jogo.altura * fy + self.y
    if camada.presa_a_tela or cena.camera is None:
        return self.x, self.y
    if camada.paralaxe_x == 1.0 and camada.paralaxe_y == 1.0:
        return cena.camera.tela(self.x, self.y)
    return cena.camera.mundo_para_tela_paralaxe(
        self.x, self.y, camada.paralaxe_x, camada.paralaxe_y
    )

def _origem_bloco(self, jogo, cena: "Cena", camada: CamadaApresentacao) ->
  tuple[float, float, float, float]:
    largura, altura = self.medir(jogo)
    x, y = self._referencia(jogo, cena, camada)
    if self.pivo is not None:
        return x - self.pivo[0], y - self.pivo[1], largura, altura
    origem = self.origem
    if origem == "centro":
        return x - largura / 2.0, y - altura / 2.0, largura, altura
    if origem == "centro_inferior":
        return x - largura / 2.0, y - altura, largura, altura
    if origem == "topo_esquerdo":
        return x, y, largura, altura
    if self.alinhamento_horizontal == "centro":
        x -= largura / 2.0
    elif self.alinhamento_horizontal == "direita":
        x -= largura
    if self.alinhamento_vertical == "centro":
        y -= altura / 2.0
    elif self.alinhamento_vertical == "base":
        y -= altura
    return x, y, largura, altura
```

```
    def _desenhar_linha(self, jogo, texto: str, x: float, y: float, opacidade: float) ->
       None:
        if self.sombra is not None:
            jogo.desenhar_texto(
                texto, x + self.deslocamento_sombra[0], y + self.deslocamento_sombra[1],
                self.tamanho, self.sombra, opacidade=opacidade,
            )
        if self.contorno is not None and self.espessura_contorno > 0:
            e = self.espessura_contorno
            for dx, dy in ((-e, 0), (e, 0), (0, -e), (0, e), (-e, -e), (-e, e), (e, -e),
               (e, e)):
                jogo.desenhar_texto(texto, x + dx, y + dy, self.tamanho, self.contorno,
                  opacidade=opacidade)
        jogo.desenhar_texto(texto, x, y, self.tamanho, self.cor, opacidade=opacidade)

    def desenhar(self, jogo, cena: "Cena", camada: CamadaApresentacao) -> None:
        linhas = self.linhas(jogo)
        x0, y0, largura_bloco, _ = self._origem_bloco(jogo, cena, camada)
        altura_linha = jogo.medir_texto("Ag", self.tamanho)[1]
        opacidade = self.opacidade * camada.opacidade
        for indice, linha in enumerate(linhas):
            largura_linha = jogo.medir_texto(linha, self.tamanho)[0]
            if self.alinhamento_horizontal == "centro":
                x = x0 + (largura_bloco - largura_linha) / 2.0
            elif self.alinhamento_horizontal == "direita":
                x = x0 + largura_bloco - largura_linha
            else:
                x = x0
            y = y0 + indice * altura_linha * self.espacamento_linhas
            self._desenhar_linha(jogo, linha, x, y, opacidade)


class Cena:
    def __init__(self, nome: str):
        self.nome = nome
        self.sprites: list[Sprite] = []
        self._representacoes: list[RepresentacaoEntidade] = []
        self._camadas: list[CamadaApresentacao] = []
        self._camada_por_sprite: dict[int, CamadaApresentacao] = {}
        self._cache_camadas_chave: tuple[tuple[int, int], ...] | None = None
        self._cache_camadas_ordenadas: tuple[CamadaApresentacao, ...] = ()
        self.camera = None
        self._ao_entrar: list[Callable] = []
        self._ao_sair: list[Callable] = []
        self._ao_atualizar: list[Callable] = []
        self._ao_desenhar: list[Callable] = []

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "nome": self.nome,
            "quantidade_sprites": len(self.sprites),
            "sprites": tuple(self.sprites),
            "quantidade_representacoes": len(self._representacoes),
            "representacoes": tuple(
                {
                    "entidade": getattr(item.entidade, "nome",
                      type(item.entidade).__name__),
                    "sprite": getattr(item.sprite, "nome", type(item.sprite).__name__),
                    "atributo_x": item.atributo_x,
                    "atributo_y": item.atributo_y,
                }
```

```
                for item in self._representacoes
            ),
            "camadas": tuple(self.camadas),
            "camera": self.camera,
        }

    @property
    def representacoes(self) -> tuple[RepresentacaoEntidade, ...]:
        return tuple(self._representacoes)

    @property
    def camadas(self) -> tuple[CamadaApresentacao, ...]:
        # Ordenação estável é recalculada somente quando a composição ou a
        # ordem pública das camadas muda. A chave mantém atribuições diretas
        # a ``camada.ordem`` compatíveis sem exigir setter especial.
        chave = tuple((id(camada), int(camada.ordem)) for camada in self._camadas)
        if chave != self._cache_camadas_chave:
            self._cache_camadas_chave = chave
            self._cache_camadas_ordenadas = tuple(sorted(self._camadas, key=lambda
              camada: camada.ordem))
        return self._cache_camadas_ordenadas

    def criar_camada(
        self,
        nome: str,
        *,
        ordem: int = 0,
        visivel: bool = True,
        opacidade: float = 1.0,
        espaco: str = "mundo",
        paralaxe: float | tuple[float, float] = 1.0,
    ) -> CamadaApresentacao:
        nome_normalizado = str(nome).strip()
        if any(camada.nome == nome_normalizado for camada in self._camadas):
            raise ValueError(f"a camada {nome_normalizado!r} já existe nesta cena")
        if isinstance(paralaxe, (tuple, list)):
            if len(paralaxe) != 2:
                raise ValueError("paralaxe em tupla precisa ter x e y")
            paralaxe_x, paralaxe_y = paralaxe
        else:
            paralaxe_x = paralaxe_y = paralaxe
        camada = CamadaApresentacao(
            nome_normalizado,
            ordem=ordem,
            visivel=visivel,
            opacidade=opacidade,
            espaco=espaco,
            paralaxe_x=paralaxe_x,
            paralaxe_y=paralaxe_y,
        )
        self._camadas.append(camada)
        return camada

    def camada(self, nome: str) -> CamadaApresentacao:
        nome = str(nome).strip()
        for camada in self._camadas:
            if camada.nome == nome:
                return camada
        raise ValueError(f"camada {nome!r} não existe nesta cena")

    def adicionar(self, sprite: Sprite, *, camada: str | CamadaApresentacao | None =
```

```
      None) -> Sprite:
        if sprite not in self.sprites:
            self.sprites.append(sprite)
        if camada is not None:
            self.adicionar_em_camada(sprite, camada)
        return sprite

    def adicionar_em_camada(
        self,
        elemento: Any,
        camada: str | CamadaApresentacao,
    ) -> Any:
        alvo = self.camada(camada) if isinstance(camada, str) else camada
        if alvo not in self._camadas:
            raise ValueError("a camada precisa pertencer a esta cena")
        if isinstance(elemento, Sprite):
            if elemento not in self.sprites:
                self.sprites.append(elemento)
            anterior = self._camada_por_sprite.get(id(elemento))
            if anterior is not None and anterior is not alvo:
                anterior.remover(elemento)
            self._camada_por_sprite[id(elemento)] = alvo
        return alvo.adicionar(elemento)

    def remover_de_camada(self, elemento: Any, camada: str | CamadaApresentacao | None =
       None) -> bool:
        if camada is None and isinstance(elemento, Sprite):
            alvo = self._camada_por_sprite.get(id(elemento))
        else:
            alvo = self.camada(camada) if isinstance(camada, str) else camada
        if alvo is None or alvo not in self._camadas:
            return False
        removido = alvo.remover(elemento)
        if removido and isinstance(elemento, Sprite):
            self._camada_por_sprite.pop(id(elemento), None)
        return removido

    def representar(
        self,
        entidade: Any,
        sprite: Any,
        *,
        atributo_x: str = "x",
        atributo_y: str = "y",
    ) -> RepresentacaoEntidade:
        if any(r.sprite is sprite for r in self._representacoes):
            raise ValueError("o sprite já representa uma entidade nesta cena")
        representacao = RepresentacaoEntidade(
            entidade,
            sprite,
            atributo_x=atributo_x,
            atributo_y=atributo_y,
        )
        representacao.sincronizar()
        self.adicionar(sprite)
        self._representacoes.append(representacao)
        return representacao

    def remover(self, sprite: Sprite) -> None:
        if sprite in self.sprites:
            self.sprites.remove(sprite)
```

```
        camada = self._camada_por_sprite.pop(id(sprite), None)
        if camada is not None:
            camada.remover(sprite)
        self._representacoes[:] = [r for r in self._representacoes if r.sprite is not
          sprite]

    def quando_entrar(self, f): self._ao_entrar.append(f); return f
    def quando_sair(self, f): self._ao_sair.append(f); return f
    def quando_atualizar(self, f): self._ao_atualizar.append(f); return f
    def quando_desenhar(self, f): self._ao_desenhar.append(f); return f

    def entrar(self, jogo):
        for f in self._ao_entrar: f(jogo, self)

    def sair(self, jogo):
        for f in self._ao_sair: f(jogo, self)

    def atualizar(self, jogo, dt: float):
        for sprite in tuple(self.sprites):
            sprite.atualizar(dt)
        for representacao in tuple(self._representacoes):
            representacao.sincronizar()
        if self.camera is not None:
            self.camera.atualizar(dt)
        for f in self._ao_atualizar: f(jogo, self, dt)

    def _posicao_sprite_na_camada(
        self,
        sprite: Sprite,
        camada: CamadaApresentacao | None,
    ) -> tuple[float, float]:
        if camada is not None and camada.presa_a_tela:
            return float(sprite.x), float(sprite.y)
        if self.camera is None:
            return float(sprite.x), float(sprite.y)
        if camada is None or (camada.paralaxe_x == 1.0 and camada.paralaxe_y == 1.0):
            return self.camera.tela(sprite.x, sprite.y)
        return self.camera.mundo_para_tela_paralaxe(
            sprite.x,
            sprite.y,
            camada.paralaxe_x,
            camada.paralaxe_y,
        )

    def _sprite_visivel_na_camada(self, sprite: Sprite, camada: CamadaApresentacao |
      None) -> bool:
        if not sprite.visivel:
            return False
        if self.camera is None or (camada is not None and camada.presa_a_tela):
            return True
        if camada is not None and (camada.paralaxe_x != 1.0 or camada.paralaxe_y !=
          1.0):
            # Culling conservador: paralaxe diferente não produz falso negativo.
            return True
        return self.camera.visivel(sprite)

    def _desenhar_sprite(self, jogo, sprite: Sprite, camada: CamadaApresentacao | None =
       None) -> None:
        if not self._sprite_visivel_na_camada(sprite, camada):
            return
        x, y = self._posicao_sprite_na_camada(sprite, camada)
```

```
        opacidade_camada = 1.0 if camada is None else camada.opacidade
        opacidade = sprite.opacidade * opacidade_camada
        if sprite.imagem is not None and hasattr(jogo, "desenhar_imagem"):
            jogo.desenhar_imagem(
                sprite.imagem,
                x,
                y,
                sprite.largura,
                sprite.altura,
                escala=sprite.escala,
                rotacao=sprite.rotacao,
                espelhar_horizontalmente=sprite.espelhado_horizontalmente,
                espelhar_verticalmente=sprite.espelhado_verticalmente,
                origem=sprite.origem,
                pivo=sprite.pivo,
                opacidade=opacidade,
            )
        else:
            jogo.desenhar_retangulo(
                x,
                y,
                sprite.largura,
                sprite.altura,
                sprite.cor,
                opacidade=opacidade,
            )

    def _desenhar_elemento(self, jogo, elemento: Any, camada: CamadaApresentacao) ->
      None:
        if isinstance(elemento, Sprite):
            self._desenhar_sprite(jogo, elemento, camada)
            return
        desenhar = getattr(elemento, "desenhar", None)
        if callable(desenhar):
            desenhar(jogo, self, camada)
            return
        if callable(elemento):
            elemento(jogo, self, camada)
            return
        raise TypeError("elemento de camada precisa ser Sprite, chamável ou possuir
          desenhar()")

    def desenhar(self, jogo):
        # Sprites sem camada preservam exatamente a ordem histórica.
        for sprite in tuple(self.sprites):
            if id(sprite) in self._camada_por_sprite:
                continue
            self._desenhar_sprite(jogo, sprite)
        # sorted é estável; camadas de mesma ordem preservam criação.
        for camada in self.camadas:
            if not camada.visivel or camada.opacidade <= 0.0:
                continue
            for elemento in tuple(camada.elementos):
                self._desenhar_elemento(jogo, elemento, camada)
        for f in self._ao_desenhar: f(jogo, self)


@dataclass
class Animacao:
    quadros: tuple[str | Path | QuadroSprite, ...]
    fps: float = 10.0
```

```
    repetir: bool = True
    _tempo: float = 0.0
    _indice: int = 0
    finalizada: bool = False
    duracoes: tuple[float, ...] | None = None
    velocidade: float = 1.0
    modo: str = "normal"
    indice_inicial: int | None = None
    marcadores: dict[str, int] = field(default_factory=dict)
    _sentido: int = field(default=1, init=False, repr=False)
    _ciclos: int = field(default=0, init=False, repr=False)
    _callbacks_marcador: dict[str, list[Callable]] = field(default_factory=dict,
      init=False, repr=False)
    _callbacks_ciclo: list[Callable] = field(default_factory=list, init=False,
      repr=False)
    _callbacks_final: list[Callable] = field(default_factory=list, init=False,
      repr=False)

    def __post_init__(self):
        self.quadros = tuple(self.quadros)
        if not self.quadros:
            raise ValueError("uma animação precisa de ao menos um quadro")
        try:
            self.fps = _numero_finito(self.fps, "fps da animação")
        except ValueError as exc:
            raise ValueError("fps da animação deve ser um número finito positivo") from
              exc
        if self.fps <= 0:
            raise ValueError("fps da animação deve ser um número finito positivo")
        self.velocidade = _numero_finito(self.velocidade, "velocidade da animação")
        if self.velocidade <= 0:
            raise ValueError("velocidade da animação deve ser positiva")
        if self.duracoes is None:
            self.duracoes = tuple(1.0 / self.fps for _ in self.quadros)
        else:
            self.duracoes = tuple(_numero_finito(v, "duração de quadro") for v in
              self.duracoes)
            if len(self.duracoes) != len(self.quadros):
                raise ValueError("duracoes deve possuir um valor para cada quadro")
            if any(v <= 0 for v in self.duracoes):
                raise ValueError("durações de quadro devem ser positivas")
        self.modo = str(self.modo).strip().casefold()
        aliases = {"ida_e_volta": "ida_volta", "pingpong": "ida_volta", "ping_pong":
          "ida_volta"}
        self.modo = aliases.get(self.modo, self.modo)
        if self.modo not in {"normal", "reverso", "ida_volta"}:
            raise ValueError("modo da animação deve ser normal, reverso ou ida_volta")
        if self.indice_inicial is None:
            inicial = len(self.quadros) - 1 if self.modo == "reverso" else
              int(self._indice)
        else:
            inicial = int(self.indice_inicial)
        if inicial < 0 or inicial >= len(self.quadros):
            raise IndexError("índice inicial da animação não existe")
        self._indice = inicial
        self._tempo = max(0.0, _numero_finito(self._tempo, "tempo da animação"))
        self.finalizada = bool(self.finalizada)
        self._sentido = -1 if self.modo == "reverso" else 1
        normalizados: dict[str, int] = {}
        for nome, indice in dict(self.marcadores).items():
            chave = str(nome).strip()
```

```
            if not chave:
                raise ValueError("marcador de animação precisa de nome")
            indice = int(indice)
            if indice < 0 or indice >= len(self.quadros):
                raise IndexError(f"quadro do marcador {chave!r} não existe")
            normalizados[chave] = indice
        self.marcadores = normalizados

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "quadros_total": len(self.quadros),
            "indice_atual": self._indice,
            "quadro_atual": self.quadro_atual,
            "fps": self.fps,
            "duracao": self.duracao,
            "tempo_atual": self.tempo_atual,
            "repetir": self.repetir,
            "finalizada": self.finalizada,
            "velocidade": self.velocidade,
            "modo": self.modo,
            "sentido": self._sentido,
            "ciclos": self._ciclos,
            "marcadores": dict(self.marcadores),
        }

    @property
    def quadro_atual(self):
        return self.quadros[self._indice]

    @property
    def indice_atual(self) -> int:
        return self._indice

    @property
    def ciclos(self) -> int:
        return self._ciclos

    @property
    def duracao(self) -> float:
        return sum(self.duracoes or ())

    @property
    def tempo_atual(self) -> float:
        return sum((self.duracoes or ())[: self._indice]) + self._tempo

    def marcar(self, nome: str, quadro: int) -> "Animacao":
        nome = str(nome).strip()
        if not nome:
            raise ValueError("marcador de animação precisa de nome")
        quadro = int(quadro)
        if quadro < 0 or quadro >= len(self.quadros):
            raise IndexError("quadro do marcador não existe")
        self.marcadores[nome] = quadro
        return self

    def ao_marcador(self, nome: str, callback: Callable) -> Callable:
        nome = str(nome).strip()
        if nome not in self.marcadores:
            raise KeyError(f"marcador inexistente: {nome!r}")
        self._callbacks_marcador.setdefault(nome, []).append(callback)
        return callback
```

```
    def ao_completar_ciclo(self, callback: Callable) -> Callable:
        self._callbacks_ciclo.append(callback)
        return callback

    def ao_finalizar(self, callback: Callable) -> Callable:
        self._callbacks_final.append(callback)
        return callback

    def _emitir_marcadores(self) -> None:
        for nome, indice in self.marcadores.items():
            if indice == self._indice:
                for callback in tuple(self._callbacks_marcador.get(nome, ())):
                    callback(self, nome)

    def _emitir_ciclo(self) -> None:
        self._ciclos += 1
        for callback in tuple(self._callbacks_ciclo):
            callback(self, self._ciclos)

    def _finalizar(self) -> None:
        if self.finalizada:
            return
        self.finalizada = True
        self._tempo = 0.0
        for callback in tuple(self._callbacks_final):
            callback(self)

    def reiniciar(self):
        self._tempo = 0.0
        self.finalizada = False
        self._ciclos = 0
        if self.modo == "reverso":
            self._indice = len(self.quadros) - 1 if self.indice_inicial is None else
              int(self.indice_inicial)
            self._sentido = -1
        else:
            self._indice = 0 if self.indice_inicial is None else
              int(self.indice_inicial)
            self._sentido = 1
        return self

    def buscar_quadro(self, indice: int) -> "Animacao":
        indice = int(indice)
        if indice < 0 or indice >= len(self.quadros):
            raise IndexError("quadro da animação não existe")
        self._indice = indice
        self._tempo = 0.0
        self.finalizada = False
        return self

    def buscar_tempo(self, segundos: float) -> "Animacao":
        segundos = _numero_finito(segundos, "tempo buscado da animação")
        if segundos < 0:
            raise ValueError("tempo buscado não pode ser negativo")
        if self.duracao <= 0:
            return self.buscar_quadro(0)
        t = min(segundos, self.duracao)
        acumulado = 0.0
        for indice, duracao in enumerate(self.duracoes or ()):
            fim = acumulado + duracao
```

```
            if t < fim or indice == len(self.quadros) - 1:
                self._indice = indice
                self._tempo = min(duracao, max(0.0, t - acumulado))
                self.finalizada = False
                return self
            acumulado = fim
        return self

    def duplicar(self) -> "Animacao":
        copia = Animacao(
            self.quadros,
            fps=self.fps,
            repetir=self.repetir,
            duracoes=self.duracoes,
            velocidade=self.velocidade,
            modo=self.modo,
            indice_inicial=self.indice_inicial,
            marcadores=dict(self.marcadores),
        )
        copia._callbacks_marcador = {nome: list(callbacks) for nome, callbacks in
          self._callbacks_marcador.items()}
        copia._callbacks_ciclo = list(self._callbacks_ciclo)
        copia._callbacks_final = list(self._callbacks_final)
        return copia

    def _avancar_indice(self) -> None:
        ultimo = len(self.quadros) - 1
        if self.modo == "normal":
            if self._indice < ultimo:
                self._indice += 1
                self._emitir_marcadores()
            elif self.repetir:
                self._emitir_ciclo()
                self._indice = 0
                self._emitir_marcadores()
            else:
                self._finalizar()
            return
        if self.modo == "reverso":
            if self._indice > 0:
                self._indice -= 1
                self._emitir_marcadores()
            elif self.repetir:
                self._emitir_ciclo()
                self._indice = ultimo
                self._emitir_marcadores()
            else:
                self._finalizar()
            return
        if self._sentido > 0:
            if self._indice < ultimo:
                self._indice += 1
                self._emitir_marcadores()
            else:
                self._sentido = -1
                if ultimo == 0:
                    if self.repetir:
                        self._emitir_ciclo()
                    else:
                        self._finalizar()
                else:
```

```
                    self._indice -= 1
                    self._emitir_marcadores()
        else:
            if self._indice > 0:
                self._indice -= 1
                self._emitir_marcadores()
            else:
                self._emitir_ciclo()
                if self.repetir:
                    self._sentido = 1
                    if ultimo > 0:
                        self._indice += 1
                        self._emitir_marcadores()
                else:
                    self._finalizar()

    def atualizar(self, dt: float):
        if self.finalizada:
            return self.quadro_atual
        dt = _numero_finito(dt, "avanço da animação")
        if dt < 0:
            raise ValueError("avanço da animação não pode ser negativo")
        self._tempo += dt * self.velocidade
        while not self.finalizada:
            duracao = (self.duracoes or ())[self._indice]
            tolerancia = max(1e-15, duracao * 1e-12)
            if self._tempo + tolerancia < duracao:
                break
            self._tempo -= duracao
            if abs(self._tempo) <= tolerancia:
                self._tempo = 0.0
            self._avancar_indice()
        return self.quadro_atual


@dataclass
class VinculoAnimacaoMovimento:
    grupo: "AnimacoesDirecionais"
    movimento: MovimentoDirecional
    sprite: Sprite
    direcoes_compativeis: tuple[str, ...]
    direcao_visual: str | None = None

    def __post_init__(self) -> None:
        if not self.direcoes_compativeis:
            raise ValueError("nenhuma direção da animação é compatível com o movimento")
        self.movimento.ao_mudar_direcao(self._ao_mudar_direcao)
        if self.movimento.direcao_olhar is not None:
            self._ao_mudar_direcao(self.movimento, self.movimento.direcao_olhar)

    def _ao_mudar_direcao(self, movimento: MovimentoDirecional, direcao: str | None) ->
      None:
        if direcao is None:
            return
        vetor = movimento.direcoes.direcao(direcao)
        visual = movimento.direcoes.resolver(
            vetor.x,
            vetor.y,
            candidatas=self.direcoes_compativeis,
        ).nome
        self.grupo.usar(self.sprite, visual)
```

```
        self.direcao_visual = visual


@dataclass
class AnimacoesDirecionais:
    """Conjunto nomeado de animações para direções de movimento.

    A classe não decide como uma entidade se move. Ela apenas seleciona a
    animação correspondente à direção informada e a liga ao Sprite visual.
    """

    animacoes: dict[str, Animacao]

    def __post_init__(self) -> None:
        normalizadas: dict[str, Animacao] = {}
        for direcao, animacao in self.animacoes.items():
            nome = str(direcao).strip().casefold()
            if not nome:
                raise ValueError("o nome de uma direção não pode ser vazio")
            if not isinstance(animacao, Animacao):
                raise TypeError("cada direção precisa apontar para uma Animacao")
            normalizadas[nome] = animacao
        if not normalizadas:
            raise ValueError("uma animação direcional precisa de ao menos uma direção")
        self.animacoes = normalizadas

    @property
    def direcoes(self) -> tuple[str, ...]:
        return tuple(self.animacoes)

    def para(self, direcao: str) -> Animacao:
        nome = str(direcao).strip().casefold()
        try:
            return self.animacoes[nome]
        except KeyError as exc:
            raise ValueError(f"direção {direcao!r} não existe nesta animação
              direcional") from exc

    def usar(self, sprite: Sprite, direcao: str, *, reiniciar: bool = True) -> Animacao:
        if not isinstance(sprite, Sprite):
            raise TypeError("uma animação direcional só pode ser usada em um Sprite")
        animacao = self.para(direcao)
        mudou = sprite.animacao is not animacao
        if mudou:
            if reiniciar:
                animacao.reiniciar()
            sprite.animacao = animacao
        sprite.imagem = animacao.quadro_atual
        return animacao

    def ligar(self, movimento: MovimentoDirecional, sprite: Sprite) ->
      VinculoAnimacaoMovimento:
        if not isinstance(movimento, MovimentoDirecional):
            raise TypeError("a animação direcional precisa de um MovimentoDirecional")
        if not isinstance(sprite, Sprite):
            raise TypeError("a animação direcional só pode ser ligada a um Sprite")
        nomes_movimento = set(movimento.direcoes.nomes)
        compativeis = tuple(nome for nome in self.direcoes if nome in nomes_movimento)
        if not compativeis:
            raise ValueError("nenhuma direção da animação é compatível com o movimento")
        return VinculoAnimacaoMovimento(self, movimento, sprite, compativeis)
```

```
@dataclass
class VinculoEstadosAnimacaoMovimento:
    """Mantém estado visual e direção como dimensões independentes.

    O vínculo é deliberadamente visual: ele não representa o estado lógico da
    entidade. O histórico de estado e os eventos de entrada e saída existem
    apenas para coordenar apresentação.
    """

    grupo: "EstadosAnimacao"
    movimento: MovimentoDirecional
    sprite: Sprite
    estado_atual: str | None = None
    estado_anterior: str | None = None
    direcao_visual: str | None = None
    ultima_direcao_visual: str | None = None

    def __post_init__(self) -> None:
        if not isinstance(self.movimento, MovimentoDirecional):
            raise TypeError("os estados de animação precisam de um MovimentoDirecional")
        if not isinstance(self.sprite, Sprite):
            raise TypeError("os estados de animação só podem ser ligados a um Sprite")
        self.movimento.ao_mudar_direcao(self._ao_mudar_direcao)

    def _resolver_direcao(self, recurso: AnimacoesDirecionais, direcao: str) -> str:
        nome = str(direcao).strip().casefold()
        if nome in recurso.direcoes:
            return nome
        try:
            vetor = self.movimento.direcoes.direcao(nome)
        except ValueError as exc:
            raise ValueError(f"direção {direcao!r} não existe no movimento ligado") from
               exc
        candidatas = tuple(nome for nome in recurso.direcoes if nome in
          self.movimento.direcoes.nomes)
        if not candidatas:
            raise ValueError("nenhuma direção deste estado é compatível com o movimento
              ligado")
        return self.movimento.direcoes.resolver(
            vetor.x,
            vetor.y,
            candidatas=candidatas,
        ).nome

    def usar(
        self,
        estado: str,
        *,
        direcao: str | None = None,
        reiniciar: bool = True,
    ) -> Animacao:
        nome_estado = self.grupo._nome_estado(estado)
        recurso = self.grupo.para(nome_estado)
        anterior = self.estado_atual
        mudou_estado = anterior != nome_estado
        if mudou_estado and anterior is not None:
            self.grupo._emitir_saida(anterior, self.sprite, nome_estado)
            self.estado_anterior = anterior
```

```
        if isinstance(recurso, Animacao):
            mudou_animacao = self.sprite.animacao is not recurso
            if mudou_animacao:
                if reiniciar:
                    recurso.reiniciar()
                self.sprite.animacao = recurso
            self.sprite.imagem = recurso.quadro_atual
            self.estado_atual = nome_estado
            self.direcao_visual = None
            if mudou_estado:
                self.grupo._emitir_entrada(nome_estado, self.sprite, anterior)
            return recurso

        escolhida = direcao or self.movimento.direcao_olhar or
          self.ultima_direcao_visual
        if escolhida is None:
            raise ValueError("um estado com animação direcional precisa de uma direção
              de olhar")
        visual = self._resolver_direcao(recurso, escolhida)
        animacao = recurso.usar(self.sprite, visual, reiniciar=reiniciar)
        self.estado_atual = nome_estado
        self.direcao_visual = visual
        self.ultima_direcao_visual = visual
        if mudou_estado:
            self.grupo._emitir_entrada(nome_estado, self.sprite, anterior)
        return animacao

    def _ao_mudar_direcao(self, movimento: MovimentoDirecional, direcao: str | None) ->
      None:
        if direcao is None or self.estado_atual is None:
            return
        recurso = self.grupo.para(self.estado_atual)
        if isinstance(recurso, AnimacoesDirecionais):
            # Mudar apenas a direção visual não reinicia o estado inteiro.
            self.usar(self.estado_atual, direcao=direcao, reiniciar=False)


class EstadosAnimacao:
    """Agrupa estados visuais sem assumir autoridade sobre estado lógico.

    ``estado_inicial`` é opt in para preservar a compatibilidade histórica.
    Eventos de entrada e saída pertencem à apresentação e não substituem
    eventos de domínio do jogo.
    """

    def __init__(
        self,
        estados: dict[str, Animacao | AnimacoesDirecionais],
        *,
        estado_inicial: str | None = None,
    ):
        normalizados: dict[str, Animacao | AnimacoesDirecionais] = {}
        for estado, recurso in estados.items():
            nome = self._nome_estado(estado)
            if nome in normalizados:
                raise ValueError(f"o estado {nome!r} foi declarado mais de uma vez")
            if not isinstance(recurso, (Animacao, AnimacoesDirecionais)):
                raise TypeError("cada estado precisa apontar para uma Animacao ou
                  AnimacoesDirecionais")
            normalizados[nome] = recurso
        if not normalizados:
```

```
            raise ValueError("um conjunto de estados de animação precisa de ao menos um
              estado")
        self._estados = normalizados
        self._vinculos: list[VinculoEstadosAnimacaoMovimento] = []
        self._ao_entrar: dict[str, list[Callable]] = {}
        self._ao_sair: dict[str, list[Callable]] = {}
        self.estado_inicial = None if estado_inicial is None else
          self._nome_estado(estado_inicial)
        if self.estado_inicial is not None:
            self.para(self.estado_inicial)

    @staticmethod
    def _nome_estado(estado: str) -> str:
        nome = str(estado).strip().casefold()
        if not nome:
            raise ValueError("o nome de um estado de animação não pode ser vazio")
        return nome

    @property
    def estados(self) -> tuple[str, ...]:
        return tuple(self._estados)

    def para(self, estado: str) -> Animacao | AnimacoesDirecionais:
        nome = self._nome_estado(estado)
        try:
            return self._estados[nome]
        except KeyError as exc:
            raise ValueError(f"estado {estado!r} não existe neste conjunto de
              animações") from exc

    def ao_entrar(self, estado: str, callback: Callable) -> Callable:
        nome = self._nome_estado(estado)
        self.para(nome)
        if not callable(callback):
            raise TypeError("callback de entrada precisa ser chamável")
        self._ao_entrar.setdefault(nome, []).append(callback)
        return callback

    def ao_sair(self, estado: str, callback: Callable) -> Callable:
        nome = self._nome_estado(estado)
        self.para(nome)
        if not callable(callback):
            raise TypeError("callback de saída precisa ser chamável")
        self._ao_sair.setdefault(nome, []).append(callback)
        return callback

    def _emitir_entrada(self, estado: str, sprite: Sprite, anterior: str | None) ->
      None:
        for callback in tuple(self._ao_entrar.get(estado, ())):
            callback(sprite, estado, anterior)

    def _emitir_saida(self, estado: str, sprite: Sprite, proximo: str) -> None:
        for callback in tuple(self._ao_sair.get(estado, ())):
            callback(sprite, estado, proximo)

    def _vinculo_do(self, sprite: Sprite) -> VinculoEstadosAnimacaoMovimento | None:
        for vinculo in reversed(self._vinculos):
            if vinculo.sprite is sprite:
                return vinculo
        return None
```

```
    def ligar(
        self,
        movimento: MovimentoDirecional,
        sprite: Sprite,
        *,
        estado_inicial: str | None = None,
        direcao_inicial: str | None = None,
    ) -> VinculoEstadosAnimacaoMovimento:
        existente = self._vinculo_do(sprite)
        if existente is not None:
            if existente.movimento is movimento:
                return existente
            raise ValueError("o sprite já possui estes estados ligados a outro
              movimento")
        vinculo = VinculoEstadosAnimacaoMovimento(self, movimento, sprite)
        self._vinculos.append(vinculo)
        inicial = self.estado_inicial if estado_inicial is None else
          self._nome_estado(estado_inicial)
        if inicial is not None:
            vinculo.usar(inicial, direcao=direcao_inicial)
        return vinculo

    def usar(
        self,
        sprite: Sprite,
        estado: str,
        *,
        direcao: str | None = None,
        reiniciar: bool = True,
    ) -> Animacao:
        if not isinstance(sprite, Sprite):
            raise TypeError("um estado de animação só pode ser usado em um Sprite")
        vinculo = self._vinculo_do(sprite)
        if vinculo is not None:
            return vinculo.usar(estado, direcao=direcao, reiniciar=reiniciar)

        recurso = self.para(estado)
        if isinstance(recurso, Animacao):
            mudou = sprite.animacao is not recurso
            if mudou:
                if reiniciar:
                    recurso.reiniciar()
                sprite.animacao = recurso
            sprite.imagem = recurso.quadro_atual
            return recurso
        if direcao is None:
            raise ValueError("um estado com animação direcional precisa de uma direção
              ou movimento ligado")
        return recurso.usar(sprite, direcao, reiniciar=reiniciar)


@dataclass(frozen=True)
class TransicaoEstadoAnimacao:
    """Política declarativa para uma transição visual."""

    estado: str
    prioridade: int = 0
    tempo_minimo: float = 0.0
    bloquear_interrupcao: float = 0.0
    retorno_automatico: str | None = None
    reiniciar: bool = True
```

```
    def __post_init__(self) -> None:
        object.__setattr__(self, "prioridade", int(self.prioridade))
        tempo = _numero_finito(self.tempo_minimo, "tempo mínimo da transição")
        bloqueio = _numero_finito(self.bloquear_interrupcao, "bloqueio de interrupção")
        if tempo < 0 or bloqueio < 0:
            raise ValueError("tempos da transição não podem ser negativos")
        object.__setattr__(self, "tempo_minimo", tempo)
        object.__setattr__(self, "bloquear_interrupcao", bloqueio)
        retorno = self.retorno_automatico
        if retorno is not None:
            retorno = str(retorno).strip().casefold()
            if not retorno:
                retorno = None
        object.__setattr__(self, "retorno_automatico", retorno)
        object.__setattr__(self, "reiniciar", bool(self.reiniciar))


@dataclass(frozen=True)
class CondicaoTransicaoAnimacao:
    condicao: Callable[[Sprite], bool]
    transicao: TransicaoEstadoAnimacao
    ordem: int = 0

    def __post_init__(self) -> None:
        if not callable(self.condicao):
            raise TypeError("condição de transição precisa ser chamável")


@dataclass
class VinculoTransicoesEstadosMovimento:
    """Executa a máquina visual opt in de um Sprite."""

    grupo: "TransicoesEstadosAnimacao"
    movimento: MovimentoDirecional
    sprite: Sprite
    vinculo_estados: VinculoEstadosAnimacaoMovimento
    ultimo_evento: str | None = None
    tempo_no_estado: float = 0.0
    bloqueio_restante: float = 0.0
    prioridade_atual: int = 0
    _estava_movendo: bool = False
    _ultimo_estado: str | None = None
    _retorno_automatico: str | None = None

    def __post_init__(self) -> None:
        if not isinstance(self.movimento, MovimentoDirecional):
            raise TypeError("as transições de estados precisam de um
              MovimentoDirecional")
        if not isinstance(self.sprite, Sprite):
            raise TypeError("as transições de estados só podem ser ligadas a um Sprite")
        self._estava_movendo = self.movimento.direcao_movimento is not None
        self._ultimo_estado = self.vinculo_estados.estado_atual
        self.movimento.ao_mudar_direcao(self._ao_mudar_direcao)
        self.sprite._vinculos_transicoes_animacao.append(self)

    @property
    def estado_atual(self) -> str | None:
        return self.vinculo_estados.estado_atual

    @property
```

```
    def estado_anterior(self) -> str | None:
        return self.vinculo_estados.estado_anterior

    @property
    def pode_ser_interrompido(self) -> bool:
        return self.bloqueio_restante <= 0.0

    def _sincronizar_estado_externo(self) -> None:
        atual = self.vinculo_estados.estado_atual
        if atual != self._ultimo_estado:
            self.tempo_no_estado = 0.0
            self.bloqueio_restante = 0.0
            self.prioridade_atual = 0
            self._retorno_automatico = None
            self._ultimo_estado = atual

    def _resolver_retorno(self, retorno: str | None, anterior: str | None) -> str |
      None:
        if retorno is None:
            return None
        if retorno in {"anterior", "estado_anterior"}:
            return anterior
        self.grupo.estados_animacao.para(retorno)
        return retorno

    def _aplicar(self, transicao: TransicaoEstadoAnimacao, *, evento: str | None = None)
       -> Animacao | None:
        self._sincronizar_estado_externo()
        if self.tempo_no_estado + 1e-12 < transicao.tempo_minimo:
            return None
        if self.bloqueio_restante > 0.0 and transicao.prioridade <=
          self.prioridade_atual:
            return None
        estado_destino = self.grupo.estados_animacao._nome_estado(transicao.estado)
        self.grupo.estados_animacao.para(estado_destino)
        anterior = self.vinculo_estados.estado_atual
        animacao = self.vinculo_estados.usar(
            estado_destino,
            reiniciar=transicao.reiniciar,
        )
        self.tempo_no_estado = 0.0
        self.bloqueio_restante = transicao.bloquear_interrupcao
        self.prioridade_atual = transicao.prioridade
        self._retorno_automatico = self._resolver_retorno(transicao.retorno_automatico,
          anterior)
        self._ultimo_estado = estado_destino
        if evento is not None:
            self.ultimo_evento = evento
        return animacao

    def transicionar(
        self,
        estado: str,
        *,
        prioridade: int = 0,
        tempo_minimo: float = 0.0,
        bloquear_interrupcao: float = 0.0,
        retorno_automatico: str | None = None,
        reiniciar: bool = True,
    ) -> Animacao | None:
        return self._aplicar(
```

```
            self.grupo._criar_transicao(
                estado,
                prioridade=prioridade,
                tempo_minimo=tempo_minimo,
                bloquear_interrupcao=bloquear_interrupcao,
                retorno_automatico=retorno_automatico,
                reiniciar=reiniciar,
            )
        )

    def acionar(self, evento: str) -> Animacao | None:
        nome_evento = self.grupo._nome_evento(evento)
        transicao = self.grupo.para_transicao(nome_evento)
        return self._aplicar(transicao, evento=nome_evento)

    def atualizar(self, dt: float) -> None:
        dt = _numero_finito(dt, "dt das transições de animação")
        if dt < 0:
            raise ValueError("dt das transições de animação não pode ser negativo")
        self._sincronizar_estado_externo()
        self.tempo_no_estado += dt
        self.bloqueio_restante = max(0.0, self.bloqueio_restante - dt)

        # Retorno automático ocorre somente quando a animação visual terminou.
        if self._retorno_automatico is not None:
            atual = self.sprite.animacao
            if isinstance(atual, Animacao) and atual.finalizada:
                retorno = self._retorno_automatico
                self._retorno_automatico = None
                self._aplicar(
                    self.grupo._criar_transicao(
                        retorno,
                        prioridade=self.prioridade_atual,
                        reiniciar=self.grupo.reiniciar_ao_retornar,
                    )
                )
                return

        candidatas: list[CondicaoTransicaoAnimacao] = []
        for item in self.grupo._condicoes:
            if bool(item.condicao(self.sprite)):
                candidatas.append(item)
        candidatas.sort(key=lambda item: (-item.transicao.prioridade, item.ordem))
        for item in candidatas:
            if self._aplicar(item.transicao) is not None:
                break

    def _ao_mudar_direcao(self, movimento: MovimentoDirecional, direcao: str | None) ->
      None:
        movendo = direcao is not None
        if movendo and not self._estava_movendo and "mover" in self.grupo.eventos:
            self.acionar("mover")
        elif not movendo and self._estava_movendo and "parar" in self.grupo.eventos:
            self.acionar("parar")
        self._estava_movendo = movendo


@dataclass
class VinculoTransicoesEventos:
    """Escuta um barramento genérico e encaminha eventos declarados para transições
      visuais."""
```

```
    grupo: "TransicoesEstadosAnimacao"
    eventos: Any
    sprite: Sprite
    vinculo_movimento: VinculoTransicoesEstadosMovimento
    _ouvintes: dict[str, Callable[..., Any]] = field(default_factory=dict, init=False)
    ativo: bool = field(default=True, init=False)

    def __post_init__(self) -> None:
        quando = getattr(self.eventos, "quando", None)
        remover = getattr(self.eventos, "remover", None)
        if not callable(quando) or not callable(remover):
            raise TypeError("o vínculo de eventos precisa de um barramento de eventos
              compatível")
        for nome_evento in self.grupo.eventos:
            def ouvinte(*_args, _evento=nome_evento, **_kwargs):
                if self.ativo:
                    self.vinculo_movimento.acionar(_evento)
            quando(nome_evento, ouvinte)
            self._ouvintes[nome_evento] = ouvinte

    def desligar(self) -> None:
        if not self.ativo:
            return
        remover = self.eventos.remover
        for nome_evento, ouvinte in tuple(self._ouvintes.items()):
            remover(nome_evento, ouvinte)
        self._ouvintes.clear()
        self.ativo = False


class TransicoesEstadosAnimacao:
    """Transições visuais por evento, condição ou chamada manual.

    Continua opt in e não substitui máquinas de estado de agentes, RPG ou
    simulação. Valores ``str`` no dicionário mantêm o contrato histórico.
    """

    def __init__(
        self,
        estados: EstadosAnimacao,
        transicoes: dict[str, str | TransicaoEstadoAnimacao | dict[str, Any]],
        *,
        reiniciar_ao_retornar: bool = True,
    ):
        if not isinstance(estados, EstadosAnimacao):
            raise TypeError("as transições precisam de um EstadosAnimacao")
        self.estados_animacao = estados
        normalizadas: dict[str, TransicaoEstadoAnimacao] = {}
        for evento, definicao in transicoes.items():
            nome_evento = self._nome_evento(evento)
            if nome_evento in normalizadas:
                raise ValueError(f"o evento {nome_evento!r} foi declarado mais de uma
                  vez")
            normalizadas[nome_evento] = self._normalizar_transicao(definicao)
            estados.para(normalizadas[nome_evento].estado)
        if not normalizadas:
            raise ValueError("um conjunto de transições precisa declarar ao menos um
              evento")
        self._transicoes = normalizadas
        self._vinculos: list[VinculoTransicoesEstadosMovimento] = []
```

```
        self._vinculos_eventos: list[VinculoTransicoesEventos] = []
        self._condicoes: list[CondicaoTransicaoAnimacao] = []
        self._ordem_condicao = 0
        self.reiniciar_ao_retornar = bool(reiniciar_ao_retornar)

    @staticmethod
    def _nome_evento(evento: str) -> str:
        nome = str(evento).strip().casefold()
        if not nome:
            raise ValueError("o nome de um evento de transição não pode ser vazio")
        return nome

    def _criar_transicao(self, estado: str, **kwargs) -> TransicaoEstadoAnimacao:
        nome_estado = self.estados_animacao._nome_estado(estado)
        self.estados_animacao.para(nome_estado)
        retorno = kwargs.get("retorno_automatico")
        if retorno not in {None, "anterior", "estado_anterior"}:
            kwargs["retorno_automatico"] = self.estados_animacao._nome_estado(retorno)
            self.estados_animacao.para(kwargs["retorno_automatico"])
        return TransicaoEstadoAnimacao(nome_estado, **kwargs)

    def _normalizar_transicao(
        self, definicao: str | TransicaoEstadoAnimacao | dict[str, Any]
    ) -> TransicaoEstadoAnimacao:
        if isinstance(definicao, TransicaoEstadoAnimacao):
            estado = self.estados_animacao._nome_estado(definicao.estado)
            retorno = definicao.retorno_automatico
            if retorno not in {None, "anterior", "estado_anterior"}:
                retorno = self.estados_animacao._nome_estado(retorno)
                self.estados_animacao.para(retorno)
            return TransicaoEstadoAnimacao(
                estado,
                prioridade=definicao.prioridade,
                tempo_minimo=definicao.tempo_minimo,
                bloquear_interrupcao=definicao.bloquear_interrupcao,
                retorno_automatico=retorno,
                reiniciar=definicao.reiniciar,
            )
        if isinstance(definicao, str):
            return self._criar_transicao(definicao)
        if isinstance(definicao, dict):
            dados = dict(definicao)
            try:
                estado = dados.pop("estado")
            except KeyError as exc:
                raise ValueError("transição declarativa precisa informar estado") from
                  exc
            conhecidos = {"prioridade", "tempo_minimo", "bloquear_interrupcao",
              "retorno_automatico", "reiniciar"}
            extras = set(dados) - conhecidos
            if extras:
                raise TypeError(f"opções desconhecidas da transição: {',
                  '.join(sorted(extras))}")
            return self._criar_transicao(estado, **dados)
        raise TypeError("transição precisa ser estado, TransicaoEstadoAnimacao ou
          dicionário")

    @property
    def eventos(self) -> tuple[str, ...]:
        return tuple(self._transicoes)
```

```
    def para(self, evento: str) -> str:
        return self.para_transicao(evento).estado

    def para_transicao(self, evento: str) -> TransicaoEstadoAnimacao:
        nome = self._nome_evento(evento)
        try:
            return self._transicoes[nome]
        except KeyError as exc:
            raise ValueError(f"o evento {evento!r} não foi declarado nestas transições")
               from exc

    def quando(
        self,
        condicao: Callable[[Sprite], bool],
        estado: str,
        *,
        prioridade: int = 0,
        tempo_minimo: float = 0.0,
        bloquear_interrupcao: float = 0.0,
        retorno_automatico: str | None = None,
        reiniciar: bool = True,
    ) -> CondicaoTransicaoAnimacao:
        transicao = self._criar_transicao(
            estado,
            prioridade=prioridade,
            tempo_minimo=tempo_minimo,
            bloquear_interrupcao=bloquear_interrupcao,
            retorno_automatico=retorno_automatico,
            reiniciar=reiniciar,
        )
        item = CondicaoTransicaoAnimacao(condicao, transicao, self._ordem_condicao)
        self._ordem_condicao += 1
        self._condicoes.append(item)
        return item

    def _vinculo_do(self, sprite: Sprite) -> VinculoTransicoesEstadosMovimento | None:
        for vinculo in reversed(self._vinculos):
            if vinculo.sprite is sprite:
                return vinculo
        return None

    def ligar(self, movimento: MovimentoDirecional, sprite: Sprite) ->
      VinculoTransicoesEstadosMovimento:
        existente = self._vinculo_do(sprite)
        if existente is not None:
            if existente.movimento is movimento:
                return existente
            raise ValueError("o sprite já possui estas transições ligadas a outro
              movimento")
        vinculo_estados = self.estados_animacao.ligar(movimento, sprite)
        vinculo = VinculoTransicoesEstadosMovimento(self, movimento, sprite,
          vinculo_estados)
        self._vinculos.append(vinculo)
        return vinculo

    def _vinculo_eventos_do(self, eventos: Any, sprite: Sprite) ->
      VinculoTransicoesEventos | None:
        for vinculo in reversed(self._vinculos_eventos):
            if vinculo.eventos is eventos and vinculo.sprite is sprite and
              vinculo.ativo:
                return vinculo
```

```
        return None

    def ligar_eventos(self, eventos: Any, sprite: Sprite) -> VinculoTransicoesEventos:
        existente = self._vinculo_eventos_do(eventos, sprite)
        if existente is not None:
            return existente
        vinculo_movimento = self._vinculo_do(sprite)
        if vinculo_movimento is None:
            raise ValueError("ligue primeiro estas transições a um movimento do sprite")
        vinculo = VinculoTransicoesEventos(self, eventos, sprite, vinculo_movimento)
        self._vinculos_eventos.append(vinculo)
        return vinculo

    def acionar(self, sprite: Sprite, evento: str) -> Animacao | None:
        vinculo = self._vinculo_do(sprite)
        if vinculo is None:
            raise ValueError("o sprite não possui estas transições ligadas a um
              movimento")
        return vinculo.acionar(evento)

    def transicionar(self, sprite: Sprite, estado: str, **kwargs) -> Animacao | None:
        vinculo = self._vinculo_do(sprite)
        if vinculo is None:
            raise ValueError("o sprite não possui estas transições ligadas a um
              movimento")
        return vinculo.transicionar(estado, **kwargs)


@dataclass
class EfeitoCamera:
    """Efeito visual temporal que não altera o estado lógico da câmera."""

    tipo: str
    duracao: float
    intensidade: float = 1.0
    x: float = 0.0
    y: float = 0.0
    zoom: float = 0.0
    rotacao: float = 0.0
    semente: int = 0
    frequencia: float = 24.0
    curva: str | Callable[[float], float] = "linear"
    tempo: float = 0.0
    estado: str = "ativo"
    _fase_x: float = field(init=False, repr=False)
    _fase_y: float = field(init=False, repr=False)

    def __post_init__(self) -> None:
        self.tipo = str(self.tipo).strip().casefold()
        if self.tipo not in {"tremor", "deslocamento", "zoom", "rotacao"}:
            raise ValueError(f"tipo de efeito de câmera inválido: {self.tipo!r}")
        self.duracao = _numero_finito(self.duracao, "duração do efeito")
        self.intensidade = _numero_finito(self.intensidade, "intensidade do efeito")
        self.x = _numero_finito(self.x, "x do efeito")
        self.y = _numero_finito(self.y, "y do efeito")
        self.zoom = _numero_finito(self.zoom, "zoom do efeito")
        self.rotacao = _numero_finito(self.rotacao, "rotação do efeito")
        self.frequencia = _numero_finito(self.frequencia, "frequência do efeito")
        self.tempo = _numero_finito(self.tempo, "tempo do efeito")
        if self.duracao <= 0:
            raise ValueError("duração do efeito deve ser positiva")
```

```
        if self.frequencia < 0:
            raise ValueError("frequência do efeito não pode ser negativa")
        if self.tempo < 0:
            raise ValueError("tempo do efeito não pode ser negativo")
        self.estado = str(self.estado).strip().casefold()
        if self.estado not in {"ativo", "pausado", "cancelado", "finalizado"}:
            raise ValueError("estado inválido do efeito de câmera")
        self.semente = int(self.semente)
        self._fase_x = (self.semente * 0.6180339887498949) % 1.0
        self._fase_y = (self.semente * 0.4142135623730950 + 0.2718281828459045) % 1.0
        self._valor_curva(0.0)

    @property
    def progresso(self) -> float:
        return min(1.0, max(0.0, self.tempo / self.duracao))

    @property
    def ativo(self) -> bool:
        return self.estado in {"ativo", "pausado"}

    def _valor_curva(self, progresso: float) -> float:
        return 1.0 - avaliar_curva_temporal(self.curva, progresso)

    def amostra(self) -> tuple[float, float, float, float]:
        if not self.ativo:
            return (0.0, 0.0, 0.0, 0.0)
        ganho = self._valor_curva(self.progresso) * self.intensidade
        if self.tipo == "tremor":
            fase = self.tempo * self.frequencia
            dx = math.sin(math.tau * (fase + self._fase_x)) * ganho
            dy = math.sin(math.tau * (fase * 1.371 + self._fase_y)) * ganho
            return (dx, dy, 0.0, 0.0)
        if self.tipo == "deslocamento":
            return (self.x * ganho, self.y * ganho, 0.0, 0.0)
        if self.tipo == "zoom":
            return (0.0, 0.0, self.zoom * ganho, 0.0)
        return (0.0, 0.0, 0.0, self.rotacao * ganho)

    def avancar(self, dt: float) -> None:
        dt = _numero_finito(dt, "dt do efeito")
        if dt < 0:
            raise ValueError("dt do efeito não pode ser negativo")
        if self.estado != "ativo":
            return
        self.tempo = min(self.duracao, self.tempo + dt)
        if self.tempo >= self.duracao:
            self.estado = "finalizado"

    def pausar(self) -> "EfeitoCamera":
        if self.estado == "ativo":
            self.estado = "pausado"
        return self

    def retomar(self) -> "EfeitoCamera":
        if self.estado == "pausado":
            self.estado = "ativo"
        return self

    def cancelar(self) -> "EfeitoCamera":
        if self.estado not in {"finalizado", "cancelado"}:
            self.estado = "cancelado"
```

```
        return self


@dataclass
class Camera:
    """Câmera 2D headless com transformação inversível de mundo e tela.

    ``x`` e ``y`` preservam a semântica histórica de canto superior esquerdo
    quando ``zoom == 1`` e ``rotacao == 0``. Zoom e rotação acontecem ao redor
    do centro lógico da câmera. O deslocamento visual é temporário e não altera
    ``x``/``y``, permitindo efeitos sem mover a autoridade lógica da câmera.
    """

    x: float = 0.0
    y: float = 0.0
    largura: float = 800.0
    altura: float = 450.0
    alvo: Any | None = None
    suavidade: float = 1.0
    zoom: float = 1.0
    rotacao: float = 0.0
    zoom_minimo: float | None = None
    zoom_maximo: float | None = None
    deslocamento_visual_x: float = 0.0
    deslocamento_visual_y: float = 0.0
    constante_tempo: float | None = None
    zona_morta: tuple[float, float] | None = None
    enquadramento_x: float = 0.0
    enquadramento_y: float = 0.0
    antecipacao: float = 0.0
    limites: Retangulo | None = None
    travar_x: bool = False
    travar_y: bool = False
    _efeitos: list[EfeitoCamera] = field(default_factory=list, init=False, repr=False)
    _vinculos_clipe: list[Any] = field(default_factory=list, init=False, repr=False)
    _cache_rotacao: float | None = field(default=None, init=False, repr=False)
    _cache_trig: tuple[float, float, float, float] = field(default=(1.0, 0.0, 1.0, 0.0),
       init=False, repr=False)
    _cache_area_chave: tuple[float, ...] | None = field(default=None, init=False,
      repr=False)
    _cache_area_limites: tuple[float, float, float, float] | None = field(default=None,
      init=False, repr=False)

    def __post_init__(self) -> None:
        self.x = _numero_finito(self.x, "x da câmera")
        self.y = _numero_finito(self.y, "y da câmera")
        self.largura = _numero_finito(self.largura, "largura da câmera")
        self.altura = _numero_finito(self.altura, "altura da câmera")
        if self.largura <= 0 or self.altura <= 0:
            raise ValueError("largura e altura da câmera devem ser positivas")
        self.suavidade = max(0.0, min(1.0, _numero_finito(self.suavidade, "suavidade da
          câmera")))
        self.rotacao = _numero_finito(self.rotacao, "rotação da câmera")
        self.deslocamento_visual_x = _numero_finito(self.deslocamento_visual_x,
          "deslocamento visual x")
        self.deslocamento_visual_y = _numero_finito(self.deslocamento_visual_y,
          "deslocamento visual y")
        self.enquadramento_x = _numero_finito(self.enquadramento_x, "enquadramento x")
        self.enquadramento_y = _numero_finito(self.enquadramento_y, "enquadramento y")
        self.antecipacao = _numero_finito(self.antecipacao, "antecipação da câmera")
        if self.antecipacao < 0:
```

```
            raise ValueError("antecipação da câmera não pode ser negativa")
        if self.constante_tempo is not None:
            self.definir_constante_tempo(self.constante_tempo)
        if self.zona_morta is not None:
            self.definir_zona_morta(*self.zona_morta)
        self.travar_x = bool(self.travar_x)
        self.travar_y = bool(self.travar_y)
        limites_iniciais = self.limites
        self.limites = None
        self.definir_limites_zoom(self.zoom_minimo, self.zoom_maximo)
        self.definir_zoom(self.zoom)
        if limites_iniciais is not None:
            self.definir_limites(limites_iniciais)

    def estado_inspecao(self) -> dict[str, Any]:
        dx, dy, zoom_visual, rotacao_visual = self._transformacao_visual()
        limites = self.limites
        alvo = self.alvo
        area_visivel = self.area_visivel
        vertices_area_visivel = tuple(
            self.tela_para_mundo(x, y)
            for x, y in (
                (0.0, 0.0),
                (self.largura, 0.0),
                (self.largura, self.altura),
                (0.0, self.altura),
            )
        )
        return {
            "posicao": {"x": self.x, "y": self.y},
            "centro": self.centro,
            "tamanho": {"largura": self.largura, "altura": self.altura},
            "area_visivel": {
                "x": area_visivel.x,
                "y": area_visivel.y,
                "largura": area_visivel.largura,
                "altura": area_visivel.altura,
            },
            "vertices_area_visivel": vertices_area_visivel,
            "zoom": self.zoom,
            "rotacao": self.rotacao,
            "transformacao_visual": {
                "deslocamento_x": dx,
                "deslocamento_y": dy,
                "zoom": zoom_visual,
                "rotacao": rotacao_visual,
            },
            "alvo": getattr(alvo, "nome", type(alvo).__name__ if alvo is not None else
              None),
            "seguimento": {
                "suavidade": self.suavidade,
                "constante_tempo": self.constante_tempo,
                "zona_morta": self.zona_morta,
                "enquadramento": (self.enquadramento_x, self.enquadramento_y),
                "antecipacao": self.antecipacao,
                "travar_x": self.travar_x,
                "travar_y": self.travar_y,
            },
            "limites": None if limites is None else {
                "x": limites.x,
                "y": limites.y,
```

```
                "largura": limites.largura,
                "altura": limites.altura,
            },
            "efeitos_ativos": len(self._efeitos),
        }

    @property
    def centro(self) -> tuple[float, float]:
        return (self.x + self.largura / 2.0, self.y + self.altura / 2.0)

    def definir_centro(self, x: float, y: float) -> "Camera":
        centro_x = _numero_finito(x, "x do centro da câmera")
        centro_y = _numero_finito(y, "y do centro da câmera")
        if self.limites is not None:
            centro_x, centro_y = self._limitar_centro(centro_x, centro_y)
        self.x = centro_x - self.largura / 2.0
        self.y = centro_y - self.altura / 2.0
        return self

    def definir_limites_zoom(
        self,
        minimo: float | None = None,
        maximo: float | None = None,
    ) -> "Camera":
        if minimo is not None:
            minimo = _numero_finito(minimo, "zoom mínimo")
            if minimo <= 0:
                raise ValueError("zoom mínimo deve ser positivo")
        if maximo is not None:
            maximo = _numero_finito(maximo, "zoom máximo")
            if maximo <= 0:
                raise ValueError("zoom máximo deve ser positivo")
        if minimo is not None and maximo is not None and minimo > maximo:
            raise ValueError("zoom mínimo não pode ser maior que zoom máximo")
        self.zoom_minimo = minimo
        self.zoom_maximo = maximo
        if hasattr(self, "zoom"):
            self.zoom = self._limitar_zoom(self.zoom)
        return self

    def _limitar_zoom(self, valor: float) -> float:
        zoom = _numero_finito(valor, "zoom da câmera")
        if zoom <= 0:
            raise ValueError("zoom da câmera deve ser positivo")
        if self.zoom_minimo is not None:
            zoom = max(zoom, self.zoom_minimo)
        if self.zoom_maximo is not None:
            zoom = min(zoom, self.zoom_maximo)
        return zoom

    def definir_zoom(self, valor: float) -> "Camera":
        self.zoom = self._limitar_zoom(valor)
        if self.limites is not None:
            self._aplicar_limites()
        return self

    def definir_rotacao(self, graus: float) -> "Camera":
        self.rotacao = _numero_finito(graus, "rotação da câmera")
        if self.limites is not None:
            self._aplicar_limites()
        return self
```

```
    def girar(self, graus: float) -> "Camera":
        return self.definir_rotacao(self.rotacao + _numero_finito(graus, "giro da
          câmera"))

    def definir_deslocamento_visual(self, x: float = 0.0, y: float = 0.0) -> "Camera":
        self.deslocamento_visual_x = _numero_finito(x, "deslocamento visual x")
        self.deslocamento_visual_y = _numero_finito(y, "deslocamento visual y")
        return self

    def limpar_deslocamento_visual(self) -> "Camera":
        return self.definir_deslocamento_visual(0.0, 0.0)

    def animar_com_clipe(
        self,
        clipe: ClipeAnimacao,
        mapeamento: dict[str, str],
        *,
        modo: str = "absoluto",
        restaurar_ao_final: bool = False,
        composicao: str = "substituir",
        permitir_conflito: bool = False,
    ):
        return ligar_clipe(
            self,
            clipe,
            mapeamento,
            modo=modo,
            restaurar_ao_final=restaurar_ao_final,
            composicao=composicao,
            permitir_conflito=permitir_conflito,
        )

    @property
    def efeitos(self) -> tuple[EfeitoCamera, ...]:
        return tuple(self._efeitos)

    def adicionar_efeito(self, efeito: EfeitoCamera) -> EfeitoCamera:
        if not isinstance(efeito, EfeitoCamera):
            raise TypeError("efeito precisa ser EfeitoCamera")
        self._efeitos.append(efeito)
        return efeito

    def tremer(
        self,
        intensidade: float = 4.0,
        duracao: float = 0.2,
        *,
        semente: int = 0,
        frequencia: float = 24.0,
        curva: str | Callable[[float], float] = "linear",
    ) -> EfeitoCamera:
        return self.adicionar_efeito(EfeitoCamera(
            "tremor",
            duracao,
            intensidade=intensidade,
            semente=semente,
            frequencia=frequencia,
            curva=curva,
        ))
```

```
    def impulsionar_deslocamento(
        self,
        x: float,
        y: float,
        duracao: float = 0.2,
        *,
        curva: str | Callable[[float], float] = "linear",
    ) -> EfeitoCamera:
        return self.adicionar_efeito(EfeitoCamera("deslocamento", duracao, x=x, y=y,
          curva=curva))

    def impulsionar_zoom(
        self,
        delta: float,
        duracao: float = 0.2,
        *,
        curva: str | Callable[[float], float] = "linear",
    ) -> EfeitoCamera:
        return self.adicionar_efeito(EfeitoCamera("zoom", duracao, zoom=delta,
          curva=curva))

    def impulsionar_rotacao(
        self,
        graus: float,
        duracao: float = 0.2,
        *,
        curva: str | Callable[[float], float] = "linear",
    ) -> EfeitoCamera:
        return self.adicionar_efeito(EfeitoCamera("rotacao", duracao, rotacao=graus,
          curva=curva))

    def aproximar_temporariamente(
        self,
        delta_zoom: float,
        duracao: float = 0.2,
        *,
        curva: str | Callable[[float], float] = "suave",
    ) -> EfeitoCamera:
        return self.impulsionar_zoom(delta_zoom, duracao, curva=curva)

    def pausar_efeitos(self) -> None:
        for efeito in self._efeitos:
            efeito.pausar()

    def retomar_efeitos(self) -> None:
        for efeito in self._efeitos:
            efeito.retomar()

    def cancelar_efeitos(self) -> None:
        for efeito in self._efeitos:
            efeito.cancelar()
        self._efeitos.clear()

    def _avancar_efeitos(self, dt: float | None) -> None:
        if dt is None:
            return
        for efeito in tuple(self._efeitos):
            efeito.avancar(dt)
        self._efeitos[:] = [efeito for efeito in self._efeitos if efeito.ativo]

    def _transformacao_visual(self) -> tuple[float, float, float, float]:
```

```
        dx = self.deslocamento_visual_x
        dy = self.deslocamento_visual_y
        zoom = self.zoom
        rotacao = self.rotacao
        for efeito in self._efeitos:
            ex, ey, ez, er = efeito.amostra()
            dx += ex
            dy += ey
            zoom += ez
            rotacao += er
        zoom = self._limitar_zoom(max(1e-12, zoom))
        return dx, dy, zoom, rotacao

    def definir_constante_tempo(self, segundos: float | None) -> "Camera":
        if segundos is None:
            self.constante_tempo = None
            return self
        valor = _numero_finito(segundos, "constante de tempo da câmera")
        if valor <= 0:
            raise ValueError("constante de tempo da câmera deve ser positiva")
        self.constante_tempo = valor
        return self

    def definir_zona_morta(self, largura: float = 0.0, altura: float = 0.0) -> "Camera":
        largura = _numero_finito(largura, "largura da zona morta")
        altura = _numero_finito(altura, "altura da zona morta")
        if largura < 0 or altura < 0:
            raise ValueError("dimensões da zona morta não podem ser negativas")
        self.zona_morta = (largura, altura) if largura or altura else None
        return self

    def definir_enquadramento(self, x: float = 0.0, y: float = 0.0) -> "Camera":
        self.enquadramento_x = _numero_finito(x, "enquadramento x")
        self.enquadramento_y = _numero_finito(y, "enquadramento y")
        return self

    def definir_antecipacao(self, segundos: float = 0.0) -> "Camera":
        segundos = _numero_finito(segundos, "antecipação da câmera")
        if segundos < 0:
            raise ValueError("antecipação da câmera não pode ser negativa")
        self.antecipacao = segundos
        return self

    def definir_limites(self, limites: Retangulo | None) -> "Camera":
        if limites is not None and not isinstance(limites, Retangulo):
            raise TypeError("limites da câmera devem ser Retangulo ou None")
        self.limites = limites
        self._aplicar_limites()
        return self

    def travar_eixos(self, *, x: bool | None = None, y: bool | None = None) -> "Camera":
        if x is not None:
            self.travar_x = bool(x)
        if y is not None:
            self.travar_y = bool(y)
        return self

    def seguir(
        self,
        sprite: Sprite,
        suavidade: float = 1.0,
```

```
        *,
        constante_tempo: float | None = None,
        zona_morta: tuple[float, float] | None = None,
        enquadramento: tuple[float, float] | None = None,
        antecipacao: float | None = None,
        travar_x: bool | None = None,
        travar_y: bool | None = None,
    ):
        self.alvo = sprite
        self.suavidade = max(0.0, min(1.0, _numero_finito(suavidade, "suavidade da
          câmera")))
        if constante_tempo is not None:
            self.definir_constante_tempo(constante_tempo)
        if zona_morta is not None:
            self.definir_zona_morta(*zona_morta)
        if enquadramento is not None:
            self.definir_enquadramento(*enquadramento)
        if antecipacao is not None:
            self.definir_antecipacao(antecipacao)
        self._geometria_alvo(sprite)
        self.travar_eixos(x=travar_x, y=travar_y)
        return self

    def seguir_representacao(self, representacao: RepresentacaoEntidade, **opcoes) ->
      "Camera":
        if not isinstance(representacao, RepresentacaoEntidade):
            raise TypeError("seguir representação espera RepresentacaoEntidade")
        return self.seguir(representacao, **opcoes)

    def _valor_alvo(self, nome: str, padrao: float = 0.0) -> float:
        if self.alvo is None:
            return float(padrao)
        ausente = object()
        valor = getattr(self.alvo, nome, ausente)
        if valor is ausente:
            obter = getattr(self.alvo, "obter", None)
            if callable(obter):
                valor = obter(nome, ausente)
        if valor is ausente:
            return float(padrao)
        return _numero_finito(valor, f"{nome} do alvo da câmera")

    def _geometria_alvo(self, alvo: Any | None = None) -> tuple[float, float, float,
      float]:
        anterior = self.alvo
        if alvo is not None:
            self.alvo = alvo
        try:
            if self.alvo is None:
                raise ValueError("a câmera precisa de um alvo")
            return (
                self._valor_alvo("x"),
                self._valor_alvo("y"),
                self._valor_alvo("largura", 0.0),
                self._valor_alvo("altura", 0.0),
            )
        finally:
            if alvo is not None:
                self.alvo = anterior

    def _delta_tela_para_mundo(self, dx: float, dy: float) -> tuple[float, float]:
```

```
        dx /= self.zoom
        dy /= self.zoom
        angulo = math.radians(self.rotacao)
        cos_a = math.cos(angulo)
        sin_a = math.sin(angulo)
        return dx * cos_a - dy * sin_a, dx * sin_a + dy * cos_a

    def _centro_desejado_do_alvo(self) -> tuple[float, float]:
        if self.alvo is None:
            return self.centro
        alvo_x, alvo_y, alvo_largura, alvo_altura = self._geometria_alvo()
        alvo_x += alvo_largura / 2.0
        alvo_y += alvo_altura / 2.0
        alvo_x += self._valor_alvo("velocidade_x", 0.0) * self.antecipacao
        alvo_y += self._valor_alvo("velocidade_y", 0.0) * self.antecipacao
        tela_x, tela_y = self._mundo_para_tela_base(alvo_x, alvo_y)
        ancora_x = self.largura / 2.0 + self.enquadramento_x +
          self.deslocamento_visual_x
        ancora_y = self.altura / 2.0 + self.enquadramento_y + self.deslocamento_visual_y
        zona_largura, zona_altura = self.zona_morta or (0.0, 0.0)
        metade_x = zona_largura / 2.0
        metade_y = zona_altura / 2.0

        if tela_x < ancora_x - metade_x:
            delta_x = tela_x - (ancora_x - metade_x)
        elif tela_x > ancora_x + metade_x:
            delta_x = tela_x - (ancora_x + metade_x)
        else:
            delta_x = 0.0
        if tela_y < ancora_y - metade_y:
            delta_y = tela_y - (ancora_y - metade_y)
        elif tela_y > ancora_y + metade_y:
            delta_y = tela_y - (ancora_y + metade_y)
        else:
            delta_y = 0.0
        mundo_dx, mundo_dy = self._delta_tela_para_mundo(delta_x, delta_y)
        centro_x, centro_y = self.centro
        return centro_x + mundo_dx, centro_y + mundo_dy

    def _meias_extensoes_visiveis(self) -> tuple[float, float]:
        angulo = math.radians(self.rotacao)
        cos_a = abs(math.cos(angulo))
        sin_a = abs(math.sin(angulo))
        meia_largura = self.largura / (2.0 * self.zoom)
        meia_altura = self.altura / (2.0 * self.zoom)
        return (
            meia_largura * cos_a + meia_altura * sin_a,
            meia_largura * sin_a + meia_altura * cos_a,
        )

    def _limitar_centro(self, x: float, y: float) -> tuple[float, float]:
        if self.limites is None:
            return x, y
        metade_x, metade_y = self._meias_extensoes_visiveis()
        minimo_x = self.limites.x + metade_x
        maximo_x = self.limites.x + self.limites.largura - metade_x
        minimo_y = self.limites.y + metade_y
        maximo_y = self.limites.y + self.limites.altura - metade_y
        if minimo_x > maximo_x:
            x = self.limites.x + self.limites.largura / 2.0
        else:
```

```
            x = min(max(x, minimo_x), maximo_x)
        if minimo_y > maximo_y:
            y = self.limites.y + self.limites.altura / 2.0
        else:
            y = min(max(y, minimo_y), maximo_y)
        return x, y

    def _aplicar_limites(self) -> None:
        if self.limites is None:
            return
        x, y = self._limitar_centro(*self.centro)
        self.x = x - self.largura / 2.0
        self.y = y - self.altura / 2.0

    def centralizar_no_alvo(self) -> "Camera":
        if self.alvo is None:
            return self
        alvo_x, alvo_y, alvo_largura, alvo_altura = self._geometria_alvo()
        alvo_x += alvo_largura / 2.0
        alvo_y += alvo_altura / 2.0
        alvo_x += self._valor_alvo("velocidade_x", 0.0) * self.antecipacao
        alvo_y += self._valor_alvo("velocidade_y", 0.0) * self.antecipacao
        mundo_offset = self._delta_tela_para_mundo(self.enquadramento_x,
          self.enquadramento_y)
        centro_x = alvo_x - mundo_offset[0]
        centro_y = alvo_y - mundo_offset[1]
        atual_x, atual_y = self.centro
        if self.travar_x:
            centro_x = atual_x
        if self.travar_y:
            centro_y = atual_y
        centro_x, centro_y = self._limitar_centro(centro_x, centro_y)
        return self.definir_centro(centro_x, centro_y)

    def enquadrar_alvos(
        self,
        alvos: list[Sprite] | tuple[Sprite, ...],
        *,
        margem: float = 0.0,
        ajustar_zoom: bool = False,
    ) -> "Camera":
        itens = tuple(alvos)
        if not itens:
            raise ValueError("enquadramento exige ao menos um alvo")
        margem = _numero_finito(margem, "margem de enquadramento")
        if margem < 0:
            raise ValueError("margem de enquadramento não pode ser negativa")
        limites = [sprite.limites_visuais for sprite in itens]
        esquerda = min(r.x for r in limites)
        topo = min(r.y for r in limites)
        direita = max(r.x + r.largura for r in limites)
        base = max(r.y + r.altura for r in limites)
        largura_mundo = direita - esquerda
        altura_mundo = base - topo
        if ajustar_zoom:
            disponivel_x = max(1e-12, self.largura - 2.0 * margem)
            disponivel_y = max(1e-12, self.altura - 2.0 * margem)
            candidatos = []
            if largura_mundo > 0:
                candidatos.append(disponivel_x / largura_mundo)
            if altura_mundo > 0:
```

```
                candidatos.append(disponivel_y / altura_mundo)
            if candidatos:
                self.definir_zoom(min(candidatos))
        centro_x = (esquerda + direita) / 2.0
        centro_y = (topo + base) / 2.0
        atual_x, atual_y = self.centro
        if self.travar_x:
            centro_x = atual_x
        if self.travar_y:
            centro_y = atual_y
        centro_x, centro_y = self._limitar_centro(centro_x, centro_y)
        return self.definir_centro(centro_x, centro_y)

    def atualizar(self, dt: float | None = None):
        self._avancar_efeitos(dt)
        if self.alvo is None:
            self._aplicar_limites()
            if dt is not None:
                self._atualizar_vinculos_clipe(dt)
            return
        desejado_x, desejado_y = self._centro_desejado_do_alvo()
        atual_x, atual_y = self.centro
        if self.travar_x:
            desejado_x = atual_x
        if self.travar_y:
            desejado_y = atual_y
        if self.constante_tempo is not None and dt is not None:
            dt = _numero_finito(dt, "dt da câmera")
            if dt < 0:
                raise ValueError("dt da câmera não pode ser negativo")
            alpha = 1.0 - math.exp(-dt / self.constante_tempo) if dt else 0.0
        else:
            alpha = self.suavidade
        centro_x = atual_x + (desejado_x - atual_x) * alpha
        centro_y = atual_y + (desejado_y - atual_y) * alpha
        centro_x, centro_y = self._limitar_centro(centro_x, centro_y)
        self.definir_centro(centro_x, centro_y)
        if dt is not None:
            self._atualizar_vinculos_clipe(dt)

    def _atualizar_vinculos_clipe(self, dt: float) -> None:
        for vinculo in tuple(self._vinculos_clipe):
            vinculo.atualizar(dt)
            if vinculo.finalizado:
                self._vinculos_clipe.remove(vinculo)

    def _mundo_para_tela_base(self, x: float, y: float) -> tuple[float, float]:
        mundo_x = _numero_finito(x, "x de mundo")
        mundo_y = _numero_finito(y, "y de mundo")
        centro_x, centro_y = self.centro
        dx = mundo_x - centro_x
        dy = mundo_y - centro_y
        cos_a, sin_a, _, _ = self._trig_rotacao(self.rotacao)
        rx = dx * cos_a - dy * sin_a
        ry = dx * sin_a + dy * cos_a
        return (
            self.largura / 2.0 + rx * self.zoom + self.deslocamento_visual_x,
            self.altura / 2.0 + ry * self.zoom + self.deslocamento_visual_y,
        )

    def _trig_rotacao(self, rotacao: float) -> tuple[float, float, float, float]:
```

```
        rotacao = float(rotacao)
        if self._cache_rotacao != rotacao:
            angulo = math.radians(rotacao)
            cos_pos = math.cos(angulo)
            sin_pos = math.sin(angulo)
            self._cache_rotacao = rotacao
            self._cache_trig = (cos_pos, -sin_pos, cos_pos, sin_pos)
        return self._cache_trig

    def mundo_para_tela(self, x: float, y: float) -> tuple[float, float]:
        mundo_x = _numero_finito(x, "x de mundo")
        mundo_y = _numero_finito(y, "y de mundo")
        visual_x, visual_y, zoom, rotacao = self._transformacao_visual()
        centro_x, centro_y = self.centro
        dx = mundo_x - centro_x
        dy = mundo_y - centro_y
        cos_a, sin_a, _, _ = self._trig_rotacao(rotacao)
        rx = dx * cos_a - dy * sin_a
        ry = dx * sin_a + dy * cos_a
        return (
            self.largura / 2.0 + rx * zoom + visual_x,
            self.altura / 2.0 + ry * zoom + visual_y,
        )

    def mundo_para_tela_paralaxe(
        self,
        x: float,
        y: float,
        paralaxe_x: float = 1.0,
        paralaxe_y: float | None = None,
    ) -> tuple[float, float]:
        """Projeta um ponto usando uma fração do deslocamento da câmera.

        Zoom, rotação e efeitos visuais continuam pertencendo à câmera; apenas
        a translação do centro lógico recebe o fator de paralaxe.
        """
        mundo_x = _numero_finito(x, "x de mundo")
        mundo_y = _numero_finito(y, "y de mundo")
        fator_x = _numero_finito(paralaxe_x, "paralaxe x")
        fator_y = fator_x if paralaxe_y is None else _numero_finito(paralaxe_y,
          "paralaxe y")
        visual_x, visual_y, zoom, rotacao = self._transformacao_visual()
        centro_x, centro_y = self.centro
        dx = mundo_x - centro_x * fator_x
        dy = mundo_y - centro_y * fator_y
        cos_a, sin_a, _, _ = self._trig_rotacao(rotacao)
        rx = dx * cos_a - dy * sin_a
        ry = dx * sin_a + dy * cos_a
        return (
            self.largura / 2.0 + rx * zoom + visual_x,
            self.altura / 2.0 + ry * zoom + visual_y,
        )

    def tela_para_mundo(self, x: float, y: float) -> tuple[float, float]:
        tela_x = _numero_finito(x, "x de tela")
        tela_y = _numero_finito(y, "y de tela")
        visual_x, visual_y, zoom, rotacao = self._transformacao_visual()
        dx = (tela_x - self.largura / 2.0 - visual_x) / zoom
        dy = (tela_y - self.altura / 2.0 - visual_y) / zoom
        _, _, cos_a, sin_a = self._trig_rotacao(rotacao)
        rx = dx * cos_a - dy * sin_a
```

```
        ry = dx * sin_a + dy * cos_a
        centro_x, centro_y = self.centro
        return centro_x + rx, centro_y + ry

    def _limites_area_visivel(self) -> tuple[float, float, float, float]:
        visual_x, visual_y, zoom, rotacao = self._transformacao_visual()
        centro_x, centro_y = self.centro
        chave = (
            centro_x, centro_y, float(self.largura), float(self.altura),
            visual_x, visual_y, zoom, rotacao,
        )
        if chave == self._cache_area_chave and self._cache_area_limites is not None:
            return self._cache_area_limites

        _, _, cos_a, sin_a = self._trig_rotacao(rotacao)
        pontos = []
        for tela_x, tela_y in (
            (0.0, 0.0),
            (self.largura, 0.0),
            (self.largura, self.altura),
            (0.0, self.altura),
        ):
            dx = (tela_x - self.largura / 2.0 - visual_x) / zoom
            dy = (tela_y - self.altura / 2.0 - visual_y) / zoom
            rx = dx * cos_a - dy * sin_a
            ry = dx * sin_a + dy * cos_a
            pontos.append((centro_x + rx, centro_y + ry))
        xs = [p[0] for p in pontos]
        ys = [p[1] for p in pontos]
        limites = (min(xs), min(ys), max(xs), max(ys))
        self._cache_area_chave = chave
        self._cache_area_limites = limites
        return limites

    @property
    def area_visivel(self) -> Retangulo:
        minimo_x, minimo_y, maximo_x, maximo_y = self._limites_area_visivel()
        return Retangulo(minimo_x, minimo_y, maximo_x - minimo_x, maximo_y - minimo_y)

    def visivel(self, objeto: Sprite | Retangulo, *, margem: float = 0.0) -> bool:
        margem = _numero_finito(margem, "margem de culling")
        if margem < 0:
            raise ValueError("margem de culling não pode ser negativa")
        if isinstance(objeto, Sprite):
            alvo = objeto.limites_visuais
        elif isinstance(objeto, Retangulo):
            alvo = objeto
        else:
            raise TypeError("culling da câmera aceita Sprite ou Retangulo")
        minimo_x, minimo_y, maximo_x, maximo_y = self._limites_area_visivel()
        minimo_x -= margem
        minimo_y -= margem
        maximo_x += margem
        maximo_y += margem
        return not (
            maximo_x <= alvo.x or
            alvo.x + alvo.largura <= minimo_x or
            maximo_y <= alvo.y or
            alvo.y + alvo.altura <= minimo_y
        )
```

```
    def tela(self, x: float, y: float) -> tuple[float, float]:
        """Compatibilidade histórica: converte coordenadas de mundo para tela."""
        return self.mundo_para_tela(x, y)

_ADAPTADORES_ANIMAVEIS: dict[type, dict[str, tuple[Callable[[Any], float],
  Callable[[Any, float], None]]]] = {}


def registrar_propriedade_animavel(
    tipo: type,
    nome: str,
    obter: Callable[[Any], float],
    definir: Callable[[Any, float], None],
) -> None:
    if not isinstance(tipo, type):
        raise TypeError("tipo do adaptador animável precisa ser uma classe")
    nome = str(nome).strip()
    if not nome:
        raise ValueError("propriedade animável precisa de nome")
    if not callable(obter) or not callable(definir):
        raise TypeError("adaptador animável exige funções de leitura e escrita")
    _ADAPTADORES_ANIMAVEIS.setdefault(tipo, {})[nome] = (obter, definir)


def _adaptador_animavel(alvo: Any, propriedade: str):
    nome = str(propriedade).strip()
    if isinstance(alvo, Sprite):
        def numero_atributo(chave: str):
            return (
                lambda obj: _numero_finito(getattr(obj, chave), chave),
                lambda obj, valor: setattr(obj, chave, _numero_finito(valor, chave)),
            )
        adaptadores = {
            "x": numero_atributo("x"),
            "y": numero_atributo("y"),
            "escala": (lambda obj: float(obj.escala), lambda obj, valor:
              obj.definir_escala(valor)),
            "rotacao": (lambda obj: float(obj.rotacao), lambda obj, valor:
              obj.definir_rotacao(valor)),
            "opacidade": (lambda obj: float(obj.opacidade), lambda obj, valor:
              obj.definir_opacidade(valor)),
            "pivo_x": (
                lambda obj: float(obj._pivo_local()[0]),
                lambda obj, valor: setattr(obj, "pivo", (_numero_finito(valor, "pivô
                  x"), obj._pivo_local()[1])),
            ),
            "pivo_y": (
                lambda obj: float(obj._pivo_local()[1]),
                lambda obj, valor: setattr(obj, "pivo", (obj._pivo_local()[0],
                  _numero_finito(valor, "pivô y"))),
            ),
        }
        if nome in adaptadores:
            return adaptadores[nome]
    if isinstance(alvo, Camera):
        adaptadores = {
            "zoom": (lambda obj: float(obj.zoom), lambda obj, valor:
              obj.definir_zoom(valor)),
            "rotacao": (lambda obj: float(obj.rotacao), lambda obj, valor:
              obj.definir_rotacao(valor)),
        }
```

```
        if nome in adaptadores:
            return adaptadores[nome]
    for tipo, adaptadores in _ADAPTADORES_ANIMAVEIS.items():
        if isinstance(alvo, tipo) and nome in adaptadores:
            return adaptadores[nome]
    raise ValueError(f"propriedade {nome!r} não possui adaptador animável para
      {type(alvo).__name__}")


def animar_propriedade(
    alvo: Any,
    propriedade: str,
    valor: float,
    duracao: float,
    *,
    curva: str | Callable[[float], float] = "suave",
    modo: str = "normal",
) -> "VinculoClipe":
    """Anime uma propriedade pública suportada até um valor em tempo explícito.

    Esta função é a ponte compacta usada pela superfície em português corrente.
    Ela reutiliza os adaptadores seguros do CP6 em vez de usar ``setattr`` irrestrito.
    """
    nome = str(propriedade).strip()
    obter, _ = _adaptador_animavel(alvo, nome)
    inicio = _numero_finito(obter(alvo), f"valor inicial de {nome}")
    destino = _numero_finito(valor, f"valor final de {nome}")
    duracao = _numero_finito(duracao, "duração da animação")
    if duracao <= 0:
        raise ValueError("duração da animação deve ser positiva")
    faixa = FaixaAnimacao(
        nome,
        (QuadroChave(0.0, inicio, curva), QuadroChave(duracao, destino)),
    )
    clipe = ClipeAnimacao((faixa,), modo=modo)
    return ligar_clipe(alvo, clipe, {nome: nome})


def reproduzir_ao_contrario(animacao: Any) -> Any:
    """Reinicie uma animação ou clipe no modo reverso, quando suportado."""
    if not hasattr(animacao, "modo") or not callable(getattr(animacao, "reiniciar",
      None)):
        raise TypeError("reprodução ao contrário exige uma animação ou clipe
          reiniciável")
    animacao.modo = "reverso"
    if hasattr(animacao, "_sentido"):
        animacao._sentido = -1
    animacao.reiniciar()
    return animacao


@dataclass
class VinculoClipe:
    alvo: Any
    clipe: ClipeAnimacao
    mapeamento: dict[str, str]
    modo: str = "absoluto"
    restaurar_ao_final: bool = False
    composicao: str = "substituir"
    _iniciais: dict[str, float] = field(default_factory=dict, init=False, repr=False)
    _ultimos: dict[str, float] = field(default_factory=dict, init=False, repr=False)
```

```
    _restaurado: bool = field(default=False, init=False, repr=False)

    def __post_init__(self) -> None:
        if not isinstance(self.clipe, ClipeAnimacao):
            raise TypeError("vínculo exige ClipeAnimacao")
        self.mapeamento = {str(faixa): str(prop) for faixa, prop in
          dict(self.mapeamento).items()}
        if not self.mapeamento:
            raise ValueError("vínculo de clipe precisa mapear ao menos uma faixa")
        nomes_faixa = {faixa.nome for faixa in self.clipe.faixas}
        ausentes = set(self.mapeamento) - nomes_faixa
        if ausentes:
            raise ValueError(f"faixas inexistentes no clipe: {',
              '.join(sorted(ausentes))}")
        self.modo = str(self.modo).strip().casefold()
        if self.modo not in {"absoluto", "relativo"}:
            raise ValueError("modo do vínculo deve ser absoluto ou relativo")
        self.composicao = str(self.composicao).strip().casefold()
        if self.composicao not in {"substituir", "aditiva"}:
            raise ValueError("composição do vínculo deve ser substituir ou aditiva")
        for propriedade in self.mapeamento.values():
            obter, _ = _adaptador_animavel(self.alvo, propriedade)
            self._iniciais[propriedade] = _numero_finito(obter(self.alvo), f"valor
              inicial de {propriedade}")
            self._ultimos[propriedade] = 0.0

    @property
    def propriedades(self) -> tuple[str, ...]:
        return tuple(dict.fromkeys(self.mapeamento.values()))

    @property
    def finalizado(self) -> bool:
        return self.clipe.finalizado and (not self.restaurar_ao_final or
          self._restaurado)

    def _aplicar(self, valores: dict[str, float]) -> None:
        for faixa, propriedade in self.mapeamento.items():
            valor = _numero_finito(valores[faixa], f"valor da faixa {faixa}")
            obter, definir = _adaptador_animavel(self.alvo, propriedade)
            if self.modo == "relativo":
                valor = self._iniciais[propriedade] + valor
            if self.composicao == "aditiva":
                atual = _numero_finito(obter(self.alvo), propriedade)
                delta = valor - self._ultimos[propriedade]
                definir(self.alvo, atual + delta)
                self._ultimos[propriedade] = valor
            else:
                definir(self.alvo, valor)

    def atualizar(self, dt: float) -> dict[str, float]:
        valores = self.clipe.avancar(dt)
        self._aplicar(valores)
        if self.clipe.finalizado and self.restaurar_ao_final and not self._restaurado:
            for propriedade, valor in self._iniciais.items():
                _, definir = _adaptador_animavel(self.alvo, propriedade)
                definir(self.alvo, valor)
            self._restaurado = True
        return valores


def ligar_clipe(
```

```
    alvo: Any,
    clipe: ClipeAnimacao,
    mapeamento: dict[str, str],
    *,
    modo: str = "absoluto",
    restaurar_ao_final: bool = False,
    composicao: str = "substituir",
    permitir_conflito: bool = False,
) -> VinculoClipe:
    novo = VinculoClipe(
        alvo,
        clipe,
        mapeamento,
        modo=modo,
        restaurar_ao_final=restaurar_ao_final,
        composicao=composicao,
    )
    existentes = getattr(alvo, "_vinculos_clipe", None)
    if existentes is None:
        try:
            setattr(alvo, "_vinculos_clipe", [])
            existentes = getattr(alvo, "_vinculos_clipe")
        except Exception as exc:
            raise TypeError("alvo não permite armazenar vínculos de animação") from exc
    sobrepostas = set(novo.propriedades)
    for atual in existentes:
        conflito = sobrepostas.intersection(atual.propriedades)
        if conflito and novo.composicao != "aditiva" and atual.composicao != "aditiva"
          and not permitir_conflito:
            raise ValueError(f"propriedades já animadas por outro vínculo: {',
              '.join(sorted(conflito))}")
    if isinstance(alvo, Sprite) and not permitir_conflito:
        if "x" in sobrepostas and alvo.velocidade_x != 0 and novo.composicao !=
          "aditiva":
            raise ValueError("animação absoluta de x conflita com velocidade_x; declare
              composição ou permita conflito")
        if "y" in sobrepostas and alvo.velocidade_y != 0 and novo.composicao !=
          "aditiva":
            raise ValueError("animação absoluta de y conflita com velocidade_y; declare
              composição ou permita conflito")
    existentes.append(novo)
    return novo

"""Pontes explícitas entre geração procedural e Jogos.

O núcleo de ``coral.procedural`` não importa Jogos. Este adaptador vive no
consumidor gráfico e continua utilizável com ``BackendNulo`` sem Pygame.
"""
from __future__ import annotations

from copy import deepcopy
from typing import Any, Callable, Iterable, Mapping, Sequence

from coral.procedural import (
    CampoProcedural,
    ReceitaProcedural,
    anotar_pontos,
    reconstruir_receita,
)

from .objetos import Cena, Sprite
```

```
from .base import BRANCO, Cor

CONTRATO = "coral.jogos.procedural/1"


def _inteiro_positivo(valor: Any, nome: str) -> int:
    if isinstance(valor, bool) or not isinstance(valor, int) or valor < 1:
        raise ValueError(f"{nome} precisa ser inteiro positivo")
    return valor


def _pontos_2d(pontos: Iterable[Sequence[float]], limite: int) -> tuple[tuple[float,
  float], ...]:
    saida: list[tuple[float, float]] = []
    for indice, ponto in enumerate(pontos):
        if indice >= limite:
            raise ValueError(
                "quantidade de sprites procedurais excede o limite; aumente o limite
                  explicitamente se aceitar o custo"
            )
        if isinstance(ponto, (str, bytes)):
            raise ValueError("cada ponto de jogo precisa conter x e y")
        coords = tuple(ponto)
        if len(coords) != 2:
            raise ValueError("cada ponto de jogo precisa conter exatamente x e y")
        try:
            x, y = float(coords[0]), float(coords[1])
        except (TypeError, ValueError, OverflowError) as exc:
            raise ValueError("coordenadas de jogo precisam ser numéricas") from exc
        import math
        if not math.isfinite(x) or not math.isfinite(y):
            raise ValueError("coordenadas de jogo precisam ser finitas")
        saida.append((x, y))
    return tuple(saida)


def sprites_de_pontos(
    pontos: Iterable[Sequence[float]],
    *,
    largura: float = 16.0,
    altura: float = 16.0,
    imagem: Any = None,
    cor: Cor = BRANCO,
    nome_base: str = "procedural",
    dados: Mapping[str, Any] | None = None,
    campos: Mapping[str, CampoProcedural] | None = None,
    fabrica: Callable[[int, float, float, dict[str, Any]], Sprite] | None = None,
    limite_sprites: int = 10_000,
) -> tuple[Sprite, ...]:
    """Converte pontos 2D determinísticos em sprites sem abrir janela."""
    limite_sprites = _inteiro_positivo(limite_sprites, "limite de sprites")
    pontos_n = _pontos_2d(pontos, limite_sprites)
    anotados = None
    if campos:
        anotados = anotar_pontos(pontos_n, campos, limite_pontos=limite_sprites)

    base = dict(dados or {})
    resultado: list[Sprite] = []
    for indice, (x, y) in enumerate(pontos_n):
        dados_sprite = deepcopy(base)
        dados_sprite["procedural"] = {"contrato": CONTRATO, "indice": indice, "x": x,
```

```
          "y": y}
        if anotados is not None:
            for chave, valor in anotados[indice].items():
                if chave not in {"indice", "x", "y"}:
                    dados_sprite[chave] = valor
        if fabrica is not None:
            sprite = fabrica(indice, x, y, dados_sprite)
            if not isinstance(sprite, Sprite):
                raise TypeError("fábrica procedural de sprites precisa retornar Sprite")
        else:
            sprite = Sprite(
                x,
                y,
                largura,
                altura,
                imagem=imagem,
                cor=cor,
                nome=f"{nome_base}_{indice}",
                dados=dados_sprite,
            )
        resultado.append(sprite)
    return tuple(resultado)


def sprites_de_receita(
    receita: ReceitaProcedural | Mapping[str, Any],
    *,
    limite_custo: int = 1_000_000,
    **opcoes_sprite: Any,
) -> tuple[Sprite, ...]:
    """Reconstrói uma receita de pontos e a adapta para sprites."""
    produto = reconstruir_receita(receita, limite_custo=limite_custo)
    if isinstance(produto, CampoProcedural):
        raise ValueError("receita de campo não produz pontos; use uma receita de
          distribuição para criar sprites")
    try:
        return sprites_de_pontos(produto, **opcoes_sprite)
    except TypeError as exc:
        raise ValueError("receita procedural não produziu uma coleção de pontos 2D")
          from exc


def popular_cena_procedural(
    cena: Cena,
    *,
    pontos: Iterable[Sequence[float]] | None = None,
    receita: ReceitaProcedural | Mapping[str, Any] | None = None,
    camada: Any = None,
    limite_custo: int = 1_000_000,
    **opcoes_sprite: Any,
) -> tuple[Sprite, ...]:
    """Cria sprites procedurais e os adiciona a uma cena existente."""
    if not isinstance(cena, Cena):
        raise TypeError("popular_cena_procedural exige uma Cena")
    if (pontos is None) == (receita is None):
        raise ValueError("informe pontos ou receita, mas não ambos")
    if receita is not None:
        sprites = sprites_de_receita(receita, limite_custo=limite_custo,
          **opcoes_sprite)
    else:
        sprites = sprites_de_pontos(pontos, **opcoes_sprite)  # type: ignore[arg-type]
```

```
    for sprite in sprites:
        cena.adicionar(sprite, camada=camada)
    return sprites


def aplicar_campo_em_sprites(
    campo: CampoProcedural,
    sprites: Iterable[Sprite],
    *,
    chave: str = "valor_procedural",
    usar_centro: bool = False,
    limite_sprites: int = 10_000,
) -> int:
    """Avalia um campo 2D na posição dos sprites e grava o valor em ``dados``."""
    if not isinstance(campo, CampoProcedural):
        raise TypeError("aplicar_campo_em_sprites exige CampoProcedural")
    chave = str(chave).strip()
    if not chave:
        raise ValueError("chave de dados precisa ser texto não vazio")
    limite_sprites = _inteiro_positivo(limite_sprites, "limite de sprites")
    quantidade = 0
    for sprite in sprites:
        if quantidade >= limite_sprites:
            raise ValueError(
                "aplicação de campo excede o limite de sprites; aumente o limite
                  explicitamente se aceitar o custo"
            )
        if not isinstance(sprite, Sprite):
            raise TypeError("coleção precisa conter apenas Sprite")
        x = float(sprite.x) + (float(sprite.largura) / 2.0 if usar_centro else 0.0)
        y = float(sprite.y) + (float(sprite.altura) / 2.0 if usar_centro else 0.0)
        sprite.dados[chave] = campo.valor(x, y)
        quantidade += 1
    return quantidade


__all__ = [
    "CONTRATO",
    "sprites_de_pontos",
    "sprites_de_receita",
    "popular_cena_procedural",
    "aplicar_campo_em_sprites",
]

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path


@dataclass(frozen=True, slots=True)
class QuadroSprite:
    """Região imutável de uma imagem usada como quadro de sprite."""

    caminho: Path
    x: int
    y: int
    largura: int
    altura: int
    indice: int | None = None
    linha: int | None = None
```

```
    coluna: int | None = None
    nome: str | None = None

    def __post_init__(self) -> None:
        if self.x < 0 or self.y < 0:
            raise ValueError("x e y do quadro não podem ser negativos")
        if self.largura <= 0 or self.altura <= 0:
            raise ValueError("largura e altura do quadro devem ser positivas")

    @property
    def retangulo(self) -> tuple[int, int, int, int]:
        return (self.x, self.y, self.largura, self.altura)


class Spritesheet:
    """Descrição headless de uma spritesheet ou atlas de imagens."""

    def __init__(self, caminho: str | Path, *, largura_imagem: int, altura_imagem: int)
      -> None:
        if largura_imagem <= 0 or altura_imagem <= 0:
            raise ValueError("as dimensões da imagem devem ser positivas")
        self.caminho = Path(caminho)
        self.largura_imagem = int(largura_imagem)
        self.altura_imagem = int(altura_imagem)
        self.largura_quadro: int | None = None
        self.altura_quadro: int | None = None
        self.margem = 0
        self.espacamento = 0
        self.colunas = 0
        self.linhas = 0
        self._quadros: tuple[QuadroSprite, ...] = ()

    @classmethod
    def em_grade(
        cls,
        caminho: str | Path,
        *,
        largura_imagem: int,
        altura_imagem: int,
        largura_quadro: int,
        altura_quadro: int,
        margem: int = 0,
        espacamento: int = 0,
    ) -> "Spritesheet":
        if largura_quadro <= 0 or altura_quadro <= 0:
            raise ValueError("as dimensões do quadro devem ser positivas")
        if margem < 0 or espacamento < 0:
            raise ValueError("margem e espaçamento não podem ser negativos")

        folha = cls(caminho, largura_imagem=largura_imagem, altura_imagem=altura_imagem)
        folha.largura_quadro = int(largura_quadro)
        folha.altura_quadro = int(altura_quadro)
        folha.margem = int(margem)
        folha.espacamento = int(espacamento)

        util_largura = folha.largura_imagem - (2 * folha.margem)
        util_altura = folha.altura_imagem - (2 * folha.margem)
        passo_x = folha.largura_quadro + folha.espacamento
        passo_y = folha.altura_quadro + folha.espacamento

        folha.colunas = (util_largura + folha.espacamento) // passo_x
```

```
        folha.linhas = (util_altura + folha.espacamento) // passo_y
        if folha.colunas <= 0 or folha.linhas <= 0:
            raise ValueError("a spritesheet não contém nenhum quadro completo")

        quadros: list[QuadroSprite] = []
        indice = 0
        for linha in range(folha.linhas):
            y = folha.margem + linha * passo_y
            for coluna in range(folha.colunas):
                x = folha.margem + coluna * passo_x
                if x + folha.largura_quadro > folha.largura_imagem:
                    continue
                if y + folha.altura_quadro > folha.altura_imagem:
                    continue
                quadros.append(
                    QuadroSprite(
                        folha.caminho,
                        x,
                        y,
                        folha.largura_quadro,
                        folha.altura_quadro,
                        indice=indice,
                        linha=linha,
                        coluna=coluna,
                    )
                )
                indice += 1
        folha._quadros = tuple(quadros)
        return folha

    def __len__(self) -> int:
        return len(self._quadros)

    def quadro(self, indice: int) -> QuadroSprite:
        if indice < 0 or indice >= len(self._quadros):
            raise IndexError(f"quadro {indice} não existe na spritesheet")
        return self._quadros[indice]

    def quadro_em(self, linha: int, coluna: int) -> QuadroSprite:
        if self.colunas <= 0 or self.linhas <= 0:
            raise ValueError("a spritesheet não foi configurada como grade")
        if linha < 0 or linha >= self.linhas or coluna < 0 or coluna >= self.colunas:
            raise IndexError(f"posição ({linha}, {coluna}) não existe na spritesheet")
        return self.quadro(linha * self.colunas + coluna)

    def quadros_de(self, inicio: int, fim: int) -> tuple[QuadroSprite, ...]:
        if inicio < 0 or fim < inicio:
            raise ValueError("intervalo de quadros inválido")
        if fim >= len(self):
            raise IndexError(f"quadro {fim} não existe na spritesheet")
        return self._quadros[inicio : fim + 1]

    def quadros_da_linha(
        self,
        linha: int,
        coluna_inicial: int = 0,
        coluna_final: int | None = None,
    ) -> tuple[QuadroSprite, ...]:
        if self.colunas <= 0 or self.linhas <= 0:
            raise ValueError("a spritesheet não foi configurada como grade")
        if linha < 0 or linha >= self.linhas:
```

```
            raise IndexError(f"linha {linha} não existe na spritesheet")
        if coluna_final is None:
            coluna_final = self.colunas - 1
        if coluna_inicial < 0 or coluna_final < coluna_inicial or coluna_final >=
          self.colunas:
            raise IndexError("intervalo de colunas não existe na spritesheet")
        inicio = linha * self.colunas + coluna_inicial
        fim = linha * self.colunas + coluna_final
        return self.quadros_de(inicio, fim)

    def animacao_da_linha(
        self,
        linha: int,
        coluna_inicial: int = 0,
        coluna_final: int | None = None,
        *,
        fps: float = 10.0,
        repetir: bool = True,
    ):
        from .objetos import Animacao

        return Animacao(
            self.quadros_da_linha(linha, coluna_inicial, coluna_final),
            fps=fps,
            repetir=repetir,
        )

    def animacao(self, inicio: int, fim: int, *, fps: float = 10.0, repetir: bool =
      True):
        """Cria uma Animacao usando um intervalo inclusivo de quadros."""
        from .objetos import Animacao

        return Animacao(self.quadros_de(inicio, fim), fps=fps, repetir=repetir)

    def animacoes_direcionais(
        self,
        linhas: dict[str, int],
        coluna_inicial: int = 0,
        coluna_final: int | None = None,
        *,
        fps: float = 10.0,
        repetir: bool = True,
    ):
        """Cria um grupo de animações a partir de linhas nomeadas da grade."""
        from .objetos import Animacao, AnimacoesDirecionais

        animacoes = {
            str(direcao): Animacao(
                self.quadros_da_linha(int(linha), coluna_inicial, coluna_final),
                fps=fps,
                repetir=repetir,
            )
            for direcao, linha in linhas.items()
        }
        return AnimacoesDirecionais(animacoes)

    def regiao(
        self,
        x: int,
        y: int,
        largura: int,
```

```
        altura: int,
        *,
        nome: str | None = None,
    ) -> QuadroSprite:
        quadro = QuadroSprite(self.caminho, x, y, largura, altura, nome=nome)
        if quadro.x + quadro.largura > self.largura_imagem or quadro.y + quadro.altura >
           self.altura_imagem:
            raise ValueError("a região solicitada fica fora dos limites da spritesheet")
        return quadro


def carregar_spritesheet(
    caminho: str | Path,
    largura_imagem: int,
    altura_imagem: int,
    largura_quadro: int,
    altura_quadro: int,
    *,
    margem: int = 0,
    espacamento: int = 0,
) -> Spritesheet:
    """Atalho funcional para criar uma spritesheet em grade uniforme."""

    return Spritesheet.em_grade(
        caminho,
        largura_imagem=largura_imagem,
        altura_imagem=altura_imagem,
        largura_quadro=largura_quadro,
        altura_quadro=altura_quadro,
        margem=margem,
        espacamento=espacamento,
    )

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Callable, Any
import math


def _numero_finito(valor: Any, nome: str) -> float:
    if isinstance(valor, bool):
        raise ValueError(f"{nome} deve ser um número finito")
    try:
        numero = float(valor)
    except (TypeError, ValueError) as exc:
        raise ValueError(f"{nome} deve ser um número finito") from exc
    if not math.isfinite(numero):
        raise ValueError(f"{nome} deve ser um número finito")
    return numero


def onda_seno(fase: float) -> float:
    return math.sin(2.0 * math.pi * float(fase))


def onda_cosseno(fase: float) -> float:
    return math.cos(2.0 * math.pi * float(fase))


def onda_triangular(fase: float) -> float:
```

```
    f = float(fase) % 1.0
    return 1.0 - 4.0 * abs(f - 0.5)


def onda_serra(fase: float) -> float:
    return 2.0 * (float(fase) % 1.0) - 1.0


def onda_pulso(fase: float) -> float:
    return 1.0 if (float(fase) % 1.0) < 0.5 else -1.0


ONDAS: dict[str, Callable[[float], float]] = {
    "seno": onda_seno,
    "cosseno": onda_cosseno,
    "triangular": onda_triangular,
    "serra": onda_serra,
    "pulso": onda_pulso,
}


def resolver_onda(onda: str | Callable[[float], float]) -> Callable[[float], float]:
    if callable(onda):
        return onda
    nome = str(onda).strip().casefold()
    try:
        return ONDAS[nome]
    except KeyError as exc:
        raise ValueError(f"onda desconhecida: {onda!r}") from exc


def normalizar_progresso(progresso: float, *, extrapolar: bool = False) -> float:
    """Normaliza progresso temporal; extrapolação só ocorre quando explícita."""
    valor = _numero_finito(progresso, "progresso temporal")
    if extrapolar:
        return valor
    return min(1.0, max(0.0, valor))


def curva_linear(progresso: float) -> float:
    return float(progresso)


def curva_entrada_suave(progresso: float) -> float:
    p = float(progresso)
    return p * p


def curva_saida_suave(progresso: float) -> float:
    p = float(progresso)
    return 1.0 - (1.0 - p) * (1.0 - p)


def curva_entrada_saida_suave(progresso: float) -> float:
    p = float(progresso)
    return p * p * (3.0 - 2.0 * p)


def curva_quadratica(progresso: float) -> float:
    p = float(progresso)
    return p * p
```

```
def curva_cubica(progresso: float) -> float:
    p = float(progresso)
    return p * p * p


def curva_degrau(progresso: float) -> float:
    return 1.0 if float(progresso) >= 1.0 else 0.0


CURVAS_TEMPORAIS: dict[str, Callable[[float], float]] = {
    "linear": curva_linear,
    "entrada": curva_entrada_suave,
    "entrada_suave": curva_entrada_suave,
    "saida": curva_saida_suave,
    "saida_suave": curva_saida_suave,
    "suave": curva_entrada_saida_suave,
    "entrada_saida": curva_entrada_saida_suave,
    "entrada_saida_suave": curva_entrada_saida_suave,
    "quadratica": curva_quadratica,
    "cúbica": curva_cubica,
    "cubica": curva_cubica,
    "degrau": curva_degrau,
    "constante": curva_degrau,
}


def resolver_curva_temporal(curva: str | Callable[[float], float]) -> Callable[[float],
  float]:
    if callable(curva):
        return curva
    nome = str(curva).strip().casefold()
    try:
        return CURVAS_TEMPORAIS[nome]
    except KeyError as exc:
        raise ValueError(f"curva temporal desconhecida: {curva!r}") from exc


def avaliar_curva_temporal(
    curva: str | Callable[[float], float],
    progresso: float,
    *,
    extrapolar: bool = False,
) -> float:
    p = normalizar_progresso(progresso, extrapolar=extrapolar)
    valor = _numero_finito(resolver_curva_temporal(curva)(p), "resultado da curva
      temporal")
    if extrapolar:
        return valor
    return min(1.0, max(0.0, valor))


def interpolar(
    inicio: float,
    fim: float,
    progresso: float,
    *,
    curva: str | Callable[[float], float] = "linear",
    extrapolar: bool = False,
) -> float:
```

```
    a = _numero_finito(inicio, "início da interpolação")
    b = _numero_finito(fim, "fim da interpolação")
    p = avaliar_curva_temporal(curva, progresso, extrapolar=extrapolar)
    return a + (b - a) * p


@dataclass(frozen=True)
class FormaTransformada:
    vertices: tuple[tuple[float, float], ...]

    def __post_init__(self) -> None:
        if len(self.vertices) < 3:
            raise ValueError("uma forma transformada precisa de ao menos três vértices")
        normalizados: list[tuple[float, float]] = []
        for x, y in self.vertices:
            normalizados.append((_numero_finito(x, "x do vértice"), _numero_finito(y, "y
               do vértice")))
        object.__setattr__(self, "vertices", tuple(normalizados))

    @property
    def limites(self) -> tuple[float, float, float, float]:
        xs = [p[0] for p in self.vertices]
        ys = [p[1] for p in self.vertices]
        return min(xs), min(ys), max(xs), max(ys)


def _eixos(forma: FormaTransformada):
    vertices = forma.vertices
    for i, atual in enumerate(vertices):
        proximo = vertices[(i + 1) % len(vertices)]
        dx = proximo[0] - atual[0]
        dy = proximo[1] - atual[1]
        comprimento = math.hypot(dx, dy)
        if comprimento == 0:
            continue
        yield (-dy / comprimento, dx / comprimento)


def _projecao(forma: FormaTransformada, eixo: tuple[float, float]) -> tuple[float,
  float]:
    valores = [x * eixo[0] + y * eixo[1] for x, y in forma.vertices]
    return min(valores), max(valores)


def formas_colidem(a: FormaTransformada, b: FormaTransformada) -> bool:
    """Colisão convexa por SAT. Toque de borda não conta como colisão."""
    for eixo in (*tuple(_eixos(a)), *tuple(_eixos(b))):
        amin, amax = _projecao(a, eixo)
        bmin, bmax = _projecao(b, eixo)
        if amax <= bmin or bmax <= amin:
            return False
    return True


@dataclass
class CanalAnimado:
    canal: str
    base: float
    amplitude: float
    frequencia: float
    fase: float = 0.0
```

```
    modo: str = "absoluto"
    onda: str | Callable[[float], float] = "seno"

    def __post_init__(self) -> None:
        self.canal = str(self.canal).strip().casefold()
        if self.canal not in {"x", "y", "escala", "rotacao", "opacidade"}:
            raise ValueError(f"canal de transformação inválido: {self.canal!r}")
        self.base = _numero_finito(self.base, "base")
        self.amplitude = _numero_finito(self.amplitude, "amplitude")
        self.frequencia = _numero_finito(self.frequencia, "frequência")
        self.fase = _numero_finito(self.fase, "fase")
        if self.frequencia < 0:
            raise ValueError("frequência deve ser maior ou igual a zero")
        self.modo = str(self.modo).strip().casefold()
        if self.modo not in {"absoluto", "relativo"}:
            raise ValueError("modo deve ser 'absoluto' ou 'relativo'")
        resolver_onda(self.onda)

    def valor(self, tempo: float, *, referencia: float = 0.0) -> float:
        fase = (self.fase + self.frequencia * tempo) % 1.0
        bruto = resolver_onda(self.onda)(fase)
        onda = _numero_finito(bruto, "resultado da onda")
        valor = self.base + self.amplitude * onda
        if self.modo == "relativo":
            valor += referencia
        return _numero_finito(valor, f"valor do canal {self.canal}")


@dataclass
class AnimacaoTransformacao:
    canais: tuple[CanalAnimado, ...]
    tempo: float = 0.0
    estado: str = "ativo"

    def __post_init__(self) -> None:
        self.canais = tuple(self.canais)
        if not self.canais:
            raise ValueError("uma automação precisa de ao menos um canal")
        self.tempo = _numero_finito(self.tempo, "tempo")
        if self.tempo < 0:
            raise ValueError("tempo deve ser maior ou igual a zero")
        self.estado = str(self.estado).strip().casefold()
        if self.estado not in {"ativo", "pausado", "encerrado"}:
            raise ValueError("estado inválido da automação")

    def pausar(self) -> None:
        if self.estado != "encerrado":
            self.estado = "pausado"

    def retomar(self) -> None:
        if self.estado != "encerrado":
            self.estado = "ativo"

    def reiniciar(self) -> None:
        self.tempo = 0.0
        if self.estado == "encerrado":
            self.estado = "ativo"

    def encerrar(self) -> None:
        self.estado = "encerrado"
```

```
    def avancar(self, dt: float) -> None:
        dt = _numero_finito(dt, "dt da automação")
        if dt < 0:
            raise ValueError("dt da automação deve ser maior ou igual a zero")
        if self.estado == "ativo":
            self.tempo += dt


@dataclass(frozen=True)
class QuadroChave:
    """Valor numérico associado a um instante de uma faixa temporal."""

    instante: float
    valor: float
    curva: str | Callable[[float], float] = "linear"

    def __post_init__(self) -> None:
        instante = _numero_finito(self.instante, "instante do quadro chave")
        valor = _numero_finito(self.valor, "valor do quadro chave")
        if instante < 0:
            raise ValueError("instante do quadro chave não pode ser negativo")
        resolver_curva_temporal(self.curva)
        object.__setattr__(self, "instante", instante)
        object.__setattr__(self, "valor", valor)


@dataclass(frozen=True)
class FaixaAnimacao:
    """Faixa temporal imutável de quadros chave para uma propriedade."""

    nome: str
    quadros: tuple[QuadroChave, ...]

    def __post_init__(self) -> None:
        nome = str(self.nome).strip()
        if not nome:
            raise ValueError("faixa de animação precisa de nome")
        quadros = tuple(self.quadros)
        if not quadros:
            raise ValueError("faixa de animação precisa de ao menos um quadro chave")
        if not all(isinstance(q, QuadroChave) for q in quadros):
            raise TypeError("faixa de animação aceita apenas QuadroChave")
        instantes = [q.instante for q in quadros]
        if instantes != sorted(instantes):
            raise ValueError("quadros chave devem estar em ordem temporal")
        if len(set(instantes)) != len(instantes):
            raise ValueError("uma faixa não pode ter dois quadros chave no mesmo
              instante")
        object.__setattr__(self, "nome", nome)
        object.__setattr__(self, "quadros", quadros)

    @property
    def duracao(self) -> float:
        return self.quadros[-1].instante

    def valor_em(self, instante: float) -> float:
        t = _numero_finito(instante, "instante consultado")
        if t <= self.quadros[0].instante:
            return self.quadros[0].valor
        if t >= self.quadros[-1].instante:
            return self.quadros[-1].valor
```

```
        for atual, proximo in zip(self.quadros, self.quadros[1:]):
            if atual.instante <= t <= proximo.instante:
                intervalo = proximo.instante - atual.instante
                progresso = 0.0 if intervalo == 0 else (t - atual.instante) / intervalo
                return interpolar(atual.valor, proximo.valor, progresso,
                  curva=atual.curva)
        return self.quadros[-1].valor


@dataclass
class ClipeAnimacao:
    """Reprodutor headless de uma ou mais faixas de quadros chave."""

    faixas: tuple[FaixaAnimacao, ...]
    velocidade: float = 1.0
    modo: str = "normal"
    repetir: bool = False
    tempo: float = 0.0
    estado: str = "ativo"
    _sentido: int = field(default=1, init=False, repr=False)

    def __post_init__(self) -> None:
        self.faixas = tuple(self.faixas)
        if not self.faixas:
            raise ValueError("clipe precisa de ao menos uma faixa")
        if not all(isinstance(f, FaixaAnimacao) for f in self.faixas):
            raise TypeError("clipe aceita apenas FaixaAnimacao")
        nomes = [f.nome for f in self.faixas]
        if len(set(nomes)) != len(nomes):
            raise ValueError("nomes de faixa precisam ser únicos dentro do clipe")
        self.velocidade = _numero_finito(self.velocidade, "velocidade do clipe")
        if self.velocidade <= 0:
            raise ValueError("velocidade do clipe deve ser positiva")
        self.modo = str(self.modo).strip().casefold()
        aliases = {"ida_e_volta": "ida_volta", "pingpong": "ida_volta", "ping_pong":
          "ida_volta"}
        self.modo = aliases.get(self.modo, self.modo)
        if self.modo not in {"normal", "reverso", "ida_volta"}:
            raise ValueError("modo do clipe deve ser normal, reverso ou ida_volta")
        self.estado = str(self.estado).strip().casefold()
        if self.estado not in {"ativo", "pausado", "finalizado"}:
            raise ValueError("estado inválido do clipe")
        self.tempo = _numero_finito(self.tempo, "tempo do clipe")
        if self.tempo < 0:
            raise ValueError("tempo do clipe não pode ser negativo")
        if self.modo == "reverso" and self.tempo == 0.0:
            self.tempo = self.duracao
            self._sentido = -1
        elif self.modo == "reverso":
            self._sentido = -1
        self.tempo = min(self.duracao, self.tempo)

    @property
    def duracao(self) -> float:
        return max(f.duracao for f in self.faixas)

    @property
    def finalizado(self) -> bool:
        return self.estado == "finalizado"

    @property
```

```
    def progresso(self) -> float:
        if self.duracao == 0:
            return 1.0
        return self.tempo / self.duracao

    def valor(self, faixa: str) -> float:
        nome = str(faixa)
        for item in self.faixas:
            if item.nome == nome:
                return item.valor_em(self.tempo)
        raise KeyError(f"faixa inexistente no clipe: {faixa!r}")

    def valores(self) -> dict[str, float]:
        return {faixa.nome: faixa.valor_em(self.tempo) for faixa in self.faixas}

    def pausar(self) -> "ClipeAnimacao":
        if not self.finalizado:
            self.estado = "pausado"
        return self

    def retomar(self) -> "ClipeAnimacao":
        if not self.finalizado:
            self.estado = "ativo"
        return self

    def reiniciar(self) -> "ClipeAnimacao":
        self.estado = "ativo"
        if self.modo == "reverso":
            self.tempo = self.duracao
            self._sentido = -1
        else:
            self.tempo = 0.0
            self._sentido = 1
        return self

    def buscar(self, instante: float) -> "ClipeAnimacao":
        instante = _numero_finito(instante, "instante do clipe")
        self.tempo = min(self.duracao, max(0.0, instante))
        return self

    def _avancar_normal(self, passo: float) -> None:
        novo = self.tempo + passo
        if self.duracao == 0:
            self.tempo = 0.0
            self.estado = "finalizado"
            return
        if novo < self.duracao:
            self.tempo = novo
            return
        if self.repetir:
            self.tempo = novo % self.duracao
        else:
            self.tempo = self.duracao
            self.estado = "finalizado"

    def _avancar_reverso(self, passo: float) -> None:
        novo = self.tempo - passo
        if self.duracao == 0:
            self.tempo = 0.0
            self.estado = "finalizado"
            return
```

```
        if novo > 0:
            self.tempo = novo
            return
        if self.repetir:
            restante = (-novo) % self.duracao
            self.tempo = self.duracao - restante if restante else self.duracao
        else:
            self.tempo = 0.0
            self.estado = "finalizado"

    def _avancar_ida_volta(self, passo: float) -> None:
        if self.duracao == 0:
            self.tempo = 0.0
            self.estado = "finalizado"
            return
        restante = passo
        while restante > 0 and self.estado == "ativo":
            limite = self.duracao if self._sentido > 0 else 0.0
            distancia = abs(limite - self.tempo)
            if restante < distancia:
                self.tempo += restante * self._sentido
                restante = 0.0
                continue
            self.tempo = limite
            restante -= distancia
            if self._sentido > 0:
                self._sentido = -1
            elif self.repetir:
                self._sentido = 1
            else:
                self.estado = "finalizado"

    def avancar(self, dt: float) -> dict[str, float]:
        dt = _numero_finito(dt, "dt do clipe")
        if dt < 0:
            raise ValueError("dt do clipe não pode ser negativo")
        if self.estado != "ativo" or dt == 0:
            return self.valores()
        passo = dt * self.velocidade
        if self.modo == "normal":
            self._avancar_normal(passo)
        elif self.modo == "reverso":
            self._avancar_reverso(passo)
        else:
            self._avancar_ida_volta(passo)
        return self.valores()

from __future__ import annotations

from dataclasses import dataclass
import math


_MODOS_VIEWPORT = {"ajustar", "esticar"}
_ANCORAS = {
    "superior_esquerda", "superior_centro", "superior_direita",
    "centro_esquerda", "centro", "centro_direita",
    "inferior_esquerda", "inferior_centro", "inferior_direita",
}
```

```
def _dimensao(valor: float, nome: str) -> float:
    if isinstance(valor, bool):
        raise ValueError(f"{nome} deve ser um número finito positivo")
    try:
        numero = float(valor)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ValueError(f"{nome} deve ser um número finito positivo") from exc
    if not math.isfinite(numero) or numero <= 0:
        raise ValueError(f"{nome} deve ser um número finito positivo")
    return numero


@dataclass(frozen=True)
class Viewport:
    """Transformação entre a superfície física e a área lógica de Jogos.

    O modo ``ajustar`` preserva a proporção lógica e centraliza a área útil.
    O modo ``esticar`` ocupa toda a superfície física usando escalas independentes.
    A classe não desenha nem altera estado de jogo, apenas converte coordenadas.
    """

    largura_logica: float
    altura_logica: float
    largura_fisica: float
    altura_fisica: float
    modo: str = "ajustar"

    def __post_init__(self) -> None:
        object.__setattr__(self, "largura_logica", _dimensao(self.largura_logica,
          "largura lógica"))
        object.__setattr__(self, "altura_logica", _dimensao(self.altura_logica, "altura
          lógica"))
        object.__setattr__(self, "largura_fisica", _dimensao(self.largura_fisica,
          "largura física"))
        object.__setattr__(self, "altura_fisica", _dimensao(self.altura_fisica, "altura
          física"))
        modo = str(self.modo).strip().casefold().replace(" ", "_")
        if modo not in _MODOS_VIEWPORT:
            raise ValueError("modo do viewport deve ser ajustar ou esticar")
        object.__setattr__(self, "modo", modo)

    @property
    def escala_x(self) -> float:
        if self.modo == "esticar":
            return self.largura_fisica / self.largura_logica
        return min(self.largura_fisica / self.largura_logica, self.altura_fisica /
          self.altura_logica)

    @property
    def escala_y(self) -> float:
        if self.modo == "esticar":
            return self.altura_fisica / self.altura_logica
        return self.escala_x

    @property
    def deslocamento_x(self) -> float:
        return (self.largura_fisica - self.largura_logica * self.escala_x) / 2.0

    @property
    def deslocamento_y(self) -> float:
        return (self.altura_fisica - self.altura_logica * self.escala_y) / 2.0
```

```
    @property
    def area_fisica_util(self) -> tuple[float, float, float, float]:
        return (
            self.deslocamento_x,
            self.deslocamento_y,
            self.largura_logica * self.escala_x,
            self.altura_logica * self.escala_y,
        )

    def logica_para_tela(self, x: float, y: float) -> tuple[float, float]:
        return (
            self.deslocamento_x + float(x) * self.escala_x,
            self.deslocamento_y + float(y) * self.escala_y,
        )

    def tela_para_logica(self, x: float, y: float, *, limitar: bool = False) ->
      tuple[float, float]:
        lx = (float(x) - self.deslocamento_x) / self.escala_x
        ly = (float(y) - self.deslocamento_y) / self.escala_y
        if limitar:
            lx = min(max(lx, 0.0), self.largura_logica)
            ly = min(max(ly, 0.0), self.altura_logica)
        return lx, ly

    def contem_tela(self, x: float, y: float) -> bool:
        lx, ly = self.tela_para_logica(x, y)
        return 0.0 <= lx <= self.largura_logica and 0.0 <= ly <= self.altura_logica

    def ancorar(
        self,
        largura: float,
        altura: float,
        *,
        ancora: str = "superior_esquerda",
        margem_x: float = 0.0,
        margem_y: float | None = None,
    ) -> tuple[float, float]:
        """Retorna o canto superior esquerdo de um elemento na área lógica."""
        largura = _dimensao(largura, "largura do elemento")
        altura = _dimensao(altura, "altura do elemento")
        ancora = str(ancora).strip().casefold().replace(" ", "_")
        if ancora not in _ANCORAS:
            raise ValueError("âncora do viewport é inválida")
        mx = float(margem_x)
        my = mx if margem_y is None else float(margem_y)
        if not math.isfinite(mx) or not math.isfinite(my):
            raise ValueError("margens do viewport devem ser números finitos")

        if ancora.endswith("esquerda"):
            x = mx
        elif ancora.endswith("direita"):
            x = self.largura_logica - largura - mx
        else:
            x = (self.largura_logica - largura) / 2.0

        if ancora.startswith("superior"):
            y = my
        elif ancora.startswith("inferior"):
            y = self.altura_logica - altura - my
        else:
```

```
            y = (self.altura_logica - altura) / 2.0
        return x, y


__all__ = ["Viewport"]
```

Biblioteca `coral.json`

`de_json(...)` · módulo `coral.json`

O que faz

Interpreta um texto JSON e reconstrói seus valores.

Assinatura

Assinatura para consulta

```
de_json(texto)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
texto	str, bytes ou bytearray com JSON	sim	sem padrão	Texto a consultar ou transformar.

Retorno

Valor compatível com JSON: lista, dicionário, texto, número, booleano ou nulo.

Exemplo

Código Coral

```
de coral.json importe de_json
mostre de_json("[1, 2]")
```

Resultado

Saída esperada

```
[1, 2]
```

Forma natural

`interprete json de "[1, 2]"` é equivalente.

Erros

JSONDecodeError para texto inválido.

`escrever_json(...)` · módulo `coral.json`

O que faz

Serializa um valor JSON e grava em UTF-8, criando ou substituindo o arquivo.

Assinatura

Assinatura para consulta

```
escrever_json(caminho, valor, identar=2)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local do arquivo; ~ é expandido.
valor	valor serializável pelo json do Python	sim	sem padrão	Conteúdo a serializar.
identar	inteiro, texto ou nulo	não	2	Indentação; nulo gera representação compacta conforme json.dumps.

Retorno

nulo.

Exemplo

Código Coral

```
de coral.json importe escrever_json
execute escrever_json("dados.json", {"nome": "Ana"})
de coral.json importe ler_json
mostre ler_json("dados.json")["nome"]
```

Resultado

Saída esperada

```
Ana
```

Erros

TypeError para objeto não serializável; ValueError para referência circular; erros de arquivo para destino inválido.

Atenção

Sobrescreve o conteúdo existente, preserva caracteres Unicode e não cria pastas. O serializador padrão permite NaN e Infinity; isso não é JSON estrito em todos os consumidores.

`ler_json(...)` · módulo `coral.json`

O que faz

Lê um arquivo UTF-8 e decodifica seu conteúdo JSON.

Assinatura

Assinatura para consulta

```
ler_json(caminho)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho local do arquivo; ~ é expandido.

Retorno

Valor Python correspondente ao JSON: dicionário, lista, texto, número, booleano ou nulo.

Exemplo

Código Coral

```
de coral.json importe ler_json
de coral.json importe escrever_json
execute escrever_json("dados.json", {"nome": "Ana"})
mostre ler_json("dados.json")["nome"]
```

Resultado

Saída esperada

```
Ana
```

Erros

JSONDecodeError para JSON malformado; FileNotFoundError, PermissionError e erros de decodificação UTF-8 para arquivo inválido.

Atenção

Materializa o arquivo e o valor JSON em memória. Datas e classes não são reconstruídas automaticamente.

`para_json(...)` · módulo `coral.json`

O que faz

Transforma valores compatíveis com JSON em texto JSON.

Assinatura

Assinatura para consulta

```
para_json(valor, identar=2)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	valor serializável pelo json do Python	sim	sem padrão	Valor sobre o qual a operação atua.
identar	inteiro, texto ou nulo	não	2	Quantidade de espaços de recuo; nulo produz a representação sem recuo.

Retorno

Texto JSON.

Exemplo

Código Coral

```
de coral.json importe para_json
mostre para_json([1, 2])
```

Resultado

Saída esperada

```
[
  1,
  2
]
```

Forma natural

`serialize json de [1, 2]` é equivalente.

Erros

TypeError para objetos não serializáveis.

Atenção

Preserva Unicode. Chaves compatíveis não textuais são convertidas para texto. NaN e Infinity são permitidos pelo serializador padrão, embora consumidores de JSON estrito possam recusá-los.

Consultar a implementação canônica de coral.json

Implementação canônica de `coral.json`

Código Python

```
"""Superfície pública de coral.json."""
from .stdlib.json import *  # noqa: F401,F403

from __future__ import annotations

from datetime import datetime, timezone
from pathlib import Path
import shutil
import tempfile
from typing import Any, Iterable, Iterator, Mapping, Sequence

from coral.caminhos import normalizar_caminho
from . import _arquivos_graduais as _gradual


def _erro_arquivo_em_portugues(exc: OSError, caminho) -> OSError:
    alvo = str(normalizar_caminho(caminho, expandir_usuario=True))
    if isinstance(exc, FileNotFoundError):
        return FileNotFoundError(exc.errno or 2, f"arquivo não encontrado: {alvo}",
          alvo)
    if isinstance(exc, PermissionError):
        return PermissionError(exc.errno or 13, f"permissão negada para o arquivo ou
          pasta: {alvo}", alvo)
    return exc



def _com_erro_traduzido(caminho, operacao):
    try:
        return operacao()
    except (FileNotFoundError, PermissionError) as exc:
        raise _erro_arquivo_em_portugues(exc, caminho) from exc

```

```
def _path(caminho) -> Path:
    return normalizar_caminho(caminho, expandir_usuario=True)


def ler_texto(caminho, codificacao="utf-8"):
    path = _path(caminho)
    return _com_erro_traduzido(path, lambda: path.read_text(encoding=codificacao))


def escrever_texto(caminho, texto, codificacao="utf-8"):
    path = _path(caminho)
    _com_erro_traduzido(path, lambda: path.write_text(str(texto), encoding=codificacao))
    return None


def _abrir_gradual(caminho, modo, codificacao):
    path = _path(caminho)
    return _com_erro_traduzido(path, lambda: path.open(modo, encoding=codificacao,
      newline=""))


def ler_linhas(caminho, codificacao="utf-8") -> Iterator[str]:
    """Lê linhas sob demanda, preservando terminadores. Use com/with ou fechar/close.

    Abre no primeiro avanço e fecha ao terminar ou falhar. É um iterador de
    passagem única; após fechar não reabre. Não há garantia de fechamento por GC.
    """
    return _gradual._Leitor(lambda: _abrir_gradual(caminho, "r", codificacao), iter)


def ler_csv(caminho, codificacao="utf-8", *, delimitador=",", cabecalho=True) ->
  Iterator[dict[str, str] | list[str]]:
    """Lê registros CSV sob demanda, com campos de texto e estrutura estrita.

    Com cabeçalho produz dicionários, sem cabeçalho produz listas. Use com/with
    ou fechar/close se interromper a leitura; abre apenas no primeiro avanço.
    Fecha no fim e em erros, não reabre e não garante fechamento por GC.
    """
    _gradual._opcoes(delimitador, cabecalho)
    return _gradual._Leitor(lambda: _abrir_gradual(caminho, "r", codificacao),
                            lambda arquivo: _gradual._registros(arquivo, delimitador,
                              cabecalho))


def ler_lotes_csv(caminho, tamanho_lote=100, codificacao="utf-8", *, delimitador=",",
  cabecalho=True) -> Iterator[list[dict[str, str] | list[str]]]:
    """Lê lotes CSV de tamanho limitado, incluindo o último lote parcial.

    Recurso de passagem única e abertura preguiçosa. Use com/with ou fechar/close
    em interrupções; fecha no fim e em erros, sem reabrir ou garantia por GC.
    """
    _gradual._opcoes(delimitador, cabecalho)
    if isinstance(tamanho_lote, bool) or not isinstance(tamanho_lote, int) or
      tamanho_lote <= 0:
        raise ValueError("tamanho_lote precisa ser inteiro positivo")
    return _gradual._Leitor(lambda: _abrir_gradual(caminho, "r", codificacao),
                            lambda arquivo: _gradual._lotes(_gradual._registros(arquivo,
                              delimitador, cabecalho), tamanho_lote))
```

```
def escrever_linhas(caminho, linhas: Iterable[str], codificacao="utf-8") -> None:
    """Escreve textos gradualmente, sem inserir terminadores ou converter valores.

    Trunca o destino e sempre o fecha. Uma falha pode deixar arquivo parcial.
    """
    with _abrir_gradual(caminho, "w", codificacao) as arquivo:
        for linha in linhas:
            if not isinstance(linha, str):
                raise TypeError("linhas precisam ser texto")
            arquivo.write(linha)


def escrever_csv(caminho, registros: Iterable[Mapping[str, str] | Sequence[str]],
  codificacao="utf-8", *, delimitador=",", colunas=None, cabecalho=True) -> None:
    """Escreve CSV gradualmente com campos estritamente textuais e estrutura exata.

    Infere colunas do primeiro mapeamento ou aceita nomes explícitos. Sequências
    textuais usam sua primeira largura; sem colunas não geram cabeçalho, mesmo
    com cabecalho=True. Trunca e fecha o destino mesmo em erro;
    uma falha pode deixar arquivo parcial. Aspas e novas linhas seguem CSV.
    """
    _gradual._opcoes(delimitador, cabecalho)
    nomes = _gradual._colunas(colunas) if colunas is not None else None
    with _abrir_gradual(caminho, "w", codificacao) as arquivo:
        _gradual._escrever_csv(arquivo, registros, delimitador, nomes, cabecalho)


def adicionar_texto(caminho, texto, codificacao="utf-8"):
    path = _path(caminho)

    def adicionar():
        with path.open("a", encoding=codificacao) as f:
            return f.write(str(texto))

    _com_erro_traduzido(path, adicionar)
    return None


def ler_bytes(caminho) -> bytes:
    path = _path(caminho)
    return _com_erro_traduzido(path, path.read_bytes)


def escrever_bytes(caminho, dados) -> None:
    path = _path(caminho)
    if not isinstance(dados, (bytes, bytearray, memoryview)):
        raise TypeError("dados precisa ser bytes, bytearray ou memoryview")
    _com_erro_traduzido(path, lambda: path.write_bytes(bytes(dados)))
    return None


def existe(caminho):
    return _path(caminho).exists()


def listar(caminho="."):
    path = _path(caminho)
    return _com_erro_traduzido(path, lambda: [str(p) for p in path.iterdir()])


def listar_recursivo(caminho=".", *, profundidade: int | None = None, incluir_pastas:
```

```
  bool = False):
    raiz = _path(caminho)
    if profundidade is not None and (not isinstance(profundidade, int) or profundidade <
       0):
        raise ValueError("profundidade precisa ser inteiro não negativo ou nulo")
    resultado: list[str] = []
    base_nivel = len(raiz.parts)
    try:
        for atual, pastas, arquivos in __import__("os").walk(raiz):
            atual_path = Path(atual)
            nivel = len(atual_path.parts) - base_nivel
            if profundidade is not None and nivel >= profundidade:
                pastas[:] = []
            if incluir_pastas:
                resultado.extend(str(atual_path / nome) for nome in sorted(pastas))
            resultado.extend(str(atual_path / nome) for nome in sorted(arquivos))
    except (FileNotFoundError, PermissionError) as exc:
        raise _erro_arquivo_em_portugues(exc, raiz) from exc
    return resultado


def criar_pasta(caminho):
    path = _path(caminho)
    return _com_erro_traduzido(path, lambda: path.mkdir(parents=True, exist_ok=True))


def remover(caminho, *, recursivo: bool = False) -> None:
    path = _path(caminho)

    def operar():
        if path.is_dir() and not path.is_symlink():
            if recursivo:
                shutil.rmtree(path)
            else:
                path.rmdir()
        else:
            path.unlink()

    _com_erro_traduzido(path, operar)


def renomear(caminho, destino) -> Path:
    origem = _path(caminho)
    alvo = _path(destino)
    return _com_erro_traduzido(origem, lambda: origem.rename(alvo))


def copiar(caminho, destino, *, recursivo: bool = False) -> Path:
    origem = _path(caminho)
    alvo = _path(destino)

    def operar():
        if origem.is_dir():
            if not recursivo:
                raise IsADirectoryError(f"copiar pasta exige recursivo=True: {origem}")
            return Path(shutil.copytree(origem, alvo))
        return Path(shutil.copy2(origem, alvo))

    return _com_erro_traduzido(origem, operar)
```

```
def mover(caminho, destino) -> Path:
    origem = _path(caminho)
    alvo = _path(destino)
    return _com_erro_traduzido(origem, lambda: Path(shutil.move(str(origem),
      str(alvo))))


def tamanho(caminho) -> int:
    path = _path(caminho)
    return _com_erro_traduzido(path, lambda: path.stat().st_size)


def metadados(caminho) -> dict[str, Any]:
    path = _path(caminho)
    st = _com_erro_traduzido(path, path.stat)
    return {
        "caminho": path,
        "nome": path.name,
        "tamanho": st.st_size,
        "arquivo": path.is_file(),
        "pasta": path.is_dir(),
        "modificado_em": datetime.fromtimestamp(st.st_mtime, tz=timezone.utc),
        "criado_em": datetime.fromtimestamp(st.st_ctime, tz=timezone.utc),
    }


def arquivo_temporario(*, prefixo="coral_", sufixo="", pasta=None) -> Path:
    diretorio = str(_path(pasta)) if pasta is not None else None
    descritor, nome_arquivo = tempfile.mkstemp(prefix=str(prefixo), suffix=str(sufixo),
      dir=diretorio)
    __import__("os").close(descritor)
    return Path(nome_arquivo)


def pasta_temporaria(*, prefixo="coral_", pasta=None) -> Path:
    diretorio = str(_path(pasta)) if pasta is not None else None
    return Path(tempfile.mkdtemp(prefix=str(prefixo), dir=diretorio))


def nome(caminho) -> str:
    return _path(caminho).name


def extensao(caminho) -> str:
    return _path(caminho).suffix


def pai(caminho) -> Path:
    return _path(caminho).parent


def resolver(caminho, *, estrito: bool = False) -> Path:
    return _path(caminho).resolve(strict=bool(estrito))


__all__ = [
    "ler_linhas", "ler_csv", "ler_lotes_csv", "escrever_linhas", "escrever_csv",
    "ler_texto", "escrever_texto", "adicionar_texto", "ler_bytes", "escrever_bytes",
    "existe", "listar", "listar_recursivo", "criar_pasta", "remover", "renomear",
    "copiar", "mover", "tamanho", "metadados", "arquivo_temporario",
    "pasta_temporaria", "nome", "extensao", "pai", "resolver",
```

```
]

import json as _json

from .arquivos import ler_texto, escrever_texto


def para_json(valor, identar=2):
    return _json.dumps(valor, ensure_ascii=False, indent=identar)


def de_json(texto):
    try:
        return _json.loads(texto)
    except _json.JSONDecodeError as exc:
        raise _json.JSONDecodeError(f"JSON malformado: {exc.msg}", exc.doc, exc.pos)
          from exc


def ler_json(caminho):
    return de_json(ler_texto(caminho, "utf-8"))


def escrever_json(caminho, valor, identar=2):
    escrever_texto(caminho, para_json(valor, identar), "utf-8")
```

Biblioteca `coral.laboratorio`

`coral.laboratorio.CONTRATO_RESULTADO_JSON`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.laboratorio.resultado/1'`

`coral.laboratorio.Medicao`

Medicao(repeticao: 'int', parametros: 'dict[str, Any]', semente: 'int | None', tempo_segundos: 'float', memoria_atual_bytes: 'int', memoria_pico_bytes: 'int', retorno: 'Any' = None, erro: 'str | None' = None, metricas: 'dict[str, Any]' = <factory>)

Assinatura

Assinatura para consulta

```
Medicao(repeticao: 'int', parametros: 'dict[str, Any]', semente: 'int | None',
  tempo_segundos: 'float', memoria_atual_bytes: 'int', memoria_pico_bytes: 'int', retorno:
   'Any' = None, erro: 'str | None' = None, metricas: 'dict[str, Any]' = <factory>) ->
  None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
repeticao	int	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
parametros	dict[str, Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
semente	int | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
tempo_segundos	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
memoria_atual_bytes	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
memoria_pico_bytes	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
retorno	Any	POSITIONAL_OR_KEYWORD	não	None
erro	str | None	POSITIONAL_OR_KEYWORD	não	None
metricas	dict[str, Any]	POSITIONAL_OR_KEYWORD	não	<factory>

Retorno declarado

None

Atributo `repeticao`: `int`.

Atributo `parametros`: `dict[str, Any]`.

Atributo `semente`: `int | None`.

Atributo `tempo_segundos`: `float`.

Atributo `memoria_atual_bytes`: `int`.

Atributo `memoria_pico_bytes`: `int`.

Atributo `retorno`: `Any`.

Atributo `erro`: `str | None`.

Atributo `metricas`: `dict[str, Any]`.

`coral.laboratorio.ResultadoExperimento`

ResultadoExperimento(nome: 'str', medicoes: 'list[Medicao]', metadados: 'dict[str, Any]')

Assinatura

Assinatura para consulta

```
ResultadoExperimento(nome: 'str', medicoes: 'list[Medicao]', metadados: 'dict[str,
  Any]') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
medicoes	list[Medicao]	POSITIONAL_OR_KEYWORD	sim	sem padrão
metadados	dict[str, Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `nome`: `str`.

Atributo `medicoes`: `list[Medicao]`.

Atributo `metadados`: `dict[str, Any]`.

Métodos públicos

Referência para leitura

```
adicionar_metadados(self, nome: 'str', valor: 'Any') -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
resultado_operacao(self) -> 'ResultadoOperacao'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
resumo(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
retornos(self, *, incluir_falhas: 'bool' = False) -> 'list[Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
salvar(self, caminho: 'str | Path') -> 'Path'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
salvar_csv(self, caminho: 'str | Path') -> 'Path'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
salvar_json(self, caminho: 'str | Path') -> 'Path'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
salvar_markdown(self, caminho: 'str | Path') -> 'Path'
```

Método público sem descrição própria no runtime.

`coral.laboratorio.VERSAO_RESULTADO_JSON`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`1`

`coral.laboratorio.carregar_resultado_json`

Carrega relatório JSON do laboratório e recusa contratos ou esquemas desconhecidos.

Assinatura

Assinatura para consulta

```
carregar_resultado_json(caminho: 'str | Path') -> 'ResultadoExperimento'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
caminho	str | Path	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

ResultadoExperimento

`coral.laboratorio.comparar_experimentos`

Compara dois experimentos de forma estritamente descritiva.

A função não calcula p valor, intervalo de confiança nem significância estatística. Diferenças numéricas são sempre apresentadas como `` `b - a` ``.

Assinatura

Assinatura para consulta

```
comparar_experimentos(a: 'ResultadoExperimento', b: 'ResultadoExperimento') ->
  'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	ResultadoExperimento	POSITIONAL_OR_KEYWORD	sim	sem padrão
b	ResultadoExperimento	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

dict[str, Any]

`coral.laboratorio.diagnosticar_laboratorio`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
diagnosticar_laboratorio() -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

dict[str, Any]

`coral.laboratorio.executar_experimento`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
executar_experimento(nome: 'str', funcao: 'Callable[..., Any]', *, repeticoes: 'int' =
  1, parametros: 'Mapping[str, Iterable[Any]] | None' = None, semente: 'int | None' =
  None, metricas: 'Callable[[Any], Mapping[str, Any]] | None' = None, continuar_em_erro:
  'bool' = True, relogio: 'FonteTempo | None' = None) -> 'ResultadoExperimento'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
funcao	Callable[..., Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
repeticoes	int	KEYWORD_ONLY	não	1
parametros	Mapping[str, Iterable[Any]] | None	KEYWORD_ONLY	não	None

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
semente	int | None	KEYWORD_ONLY	não	None
metricas	Callable[[Any], Mapping[str, Any]] | None	KEYWORD_ONLY	não	None
continuar_em_erro	bool	KEYWORD_ONLY	não	True
relogio	FonteTempo | None	KEYWORD_ONLY	não	None

Retorno declarado

ResultadoExperimento

`coral.laboratorio.executar_experimento_procedural`

Executa uma receita procedural como fator explícito de um experimento.

`` `funcao recebe primeiro o produto reconstruído e depois os parâmetros declarados pelo estudo. A semente da receita aparece em cada medição como semente_procedural` `` e a receita base é preservada nos metadados.

Assinatura

Assinatura para consulta

```
executar_experimento_procedural(nome: 'str', receita: 'Any', funcao: 'Callable[...,
  Any]', *, sementes: 'Iterable[Any] | None' = None, parametros: 'Mapping[str,
  Iterable[Any]] | None' = None, repeticoes: 'int' = 1, semente_laboratorio: 'int | None'
  = None, metricas: 'Callable[[Any], Mapping[str, Any]] | None' = None, continuar_em_erro:
   'bool' = True, relogio: 'FonteTempo | None' = None, limite_custo_receita: 'int' =
  1000000, limite_execucoes: 'int | None' = None, limite_custo_total: 'int | None' = None)
   -> 'ResultadoExperimento'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
receita	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
funcao	Callable[..., Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
sementes	Iterable[Any] | None	KEYWORD_ONLY	não	None
parametros	Mapping[str, Iterable[Any]] | None	KEYWORD_ONLY	não	None
repeticoes	int	KEYWORD_ONLY	não	1

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
semente_laboratorio	int | None	KEYWORD_ONLY	não	None
metricas	Callable[[Any], Mapping[str, Any]] | None	KEYWORD_ONLY	não	None
continuar_em_erro	bool	KEYWORD_ONLY	não	True
relogio	FonteTempo | None	KEYWORD_ONLY	não	None
limite_custo_receita	int	KEYWORD_ONLY	não	1000000
limite_execucoes	int | None	KEYWORD_ONLY	não	None
limite_custo_total	int | None	KEYWORD_ONLY	não	None

Retorno declarado

ResultadoExperimento

`coral.laboratorio.grade_parametros`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
grade_parametros(parametros: 'Mapping[str, Iterable[Any]] | None') -> 'list[dict[str,
  Any]]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
parametros	Mapping[str, Iterable[Any]] | None	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

list[dict[str, Any]]

`coral.laboratorio.planejar_experimento`

Calcula o tamanho de uma execução experimental sem materializar a grade completa.

Iteráveis sem `len` são percorridos para contagem, mas seus itens não são mantidos em memória. A função nunca executa a função que será estudada.

Assinatura

Assinatura para consulta

```
planejar_experimento(parametros: 'Mapping[str, Iterable[Any]] | None', repeticoes: 'int'
   = 1, limite: 'int | None' = None) -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
parametros	Mapping[str, Iterable[Any]] | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
repeticoes	int	POSITIONAL_OR_KEYWORD	não	1
limite	int | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

dict[str, Any]

`coral.laboratorio.planejar_experimento_procedural`

Planeja um estudo procedural sem reconstruir nem executar a receita.

O custo combina a quantidade de execuções com o pior caso declarado pela própria receita. Isso permite rejeitar grades enormes antes da materialização.

Assinatura

Assinatura para consulta

```
planejar_experimento_procedural(receita: 'Any', *, sementes: 'Iterable[Any] | None' =
  None, parametros: 'Mapping[str, Iterable[Any]] | None' = None, repeticoes: 'int' = 1,
  limite_execucoes: 'int | None' = None, limite_custo_total: 'int | None' = None) ->
  'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
receita	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
sementes	Iterable[Any] | None	KEYWORD_ONLY	não	None
parametros	Mapping[str, Iterable[Any]] | None	KEYWORD_ONLY	não	None
repeticoes	int	KEYWORD_ONLY	não	1
limite_execucoes	int | None	KEYWORD_ONLY	não	None
limite_custo_total	int | None	KEYWORD_ONLY	não	None

Retorno declarado

dict[str, Any]

`coral.laboratorio.protocolo_medicao`

Descreve o protocolo usado pelo runtime sem prometer estabilidade de benchmark.

Assinatura

Assinatura para consulta

```
protocolo_medicao(relogio: 'FonteTempo | None' = None) -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
relogio	FonteTempo | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

dict[str, Any]

`coral.laboratorio.resumir_por_parametro`

Agrupa medições pelo valor de um parâmetro, preservando a ordem de aparição.

Assinatura

Assinatura para consulta

```
resumir_por_parametro(resultado: 'ResultadoExperimento', nome: 'str') -> 'dict[str,
  Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
resultado	ResultadoExperimento	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

dict[str, Any]

`coral.laboratorio.selecionar_medicoes`

Seleciona medições preservando ordem e sem modificar o resultado original.

Assinatura

Assinatura para consulta

```
selecionar_medicoes(resultado: 'ResultadoExperimento', criterio: 'Callable[[Medicao],
  bool] | Mapping[str, Any]') -> 'list[Medicao]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
resultado	ResultadoExperimento	POSITIONAL_OR_KEYWORD	sim	sem padrão
criterio	Callable[[Medicao], bool] | Mapping[str, Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

list[Medicao]

`coral.laboratorio.validar_resultado`

Valida a estrutura e a portabilidade de um resultado sem modificá lo.

Assinatura

Assinatura para consulta

```
validar_resultado(resultado: 'Any') -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
resultado	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

dict[str, Any]

Consultar a implementação canônica de coral.laboratorio

Implementação canônica de `coral.laboratorio`

Código Python

```
"""Contratos transversais pequenos do runtime Coral.

A camada comum não substitui APIs de domínio. Ela fornece a gramática
arquitetural compartilhada do ciclo 1.5 para erro operacional, resultado e
diagnóstico, preservando os nomes públicos existentes em cada biblioteca.
"""
from __future__ import annotations

from dataclasses import asdict, dataclass, field
from typing import Any, Iterable, Mapping

CONTRATO = "coral.comum/1"
CONTRATO_DIAGNOSTICO = "coral.diagnostico.operacional/1"


class ErroCoral(Exception):
    """Raiz pública dos erros controlados pela linguagem e bibliotecas Coral."""


class ErroOperacaoCoral(RuntimeError, ErroCoral):
    """Falha operacional de um recurso Coral ou de seu backend."""


class ErroDependenciaCoral(ErroOperacaoCoral):
    """Uma capacidade opcional não pode operar por ausência de dependência."""


@dataclass(frozen=True)
class FalhaOperacao:
    codigo: str
    mensagem: str
    dominio: str
    tipo: str = "operacional"
    causa: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def de_excecao(cls, exc: BaseException, *, codigo: str, dominio: str, tipo: str =
      "operacional", detalhe: Mapping[str, Any] | None = None) -> "FalhaOperacao":
        causa = exc.__cause__ if exc.__cause__ is not None else exc
        return cls(codigo, str(exc), dominio, tipo, type(causa).__name__, dict(detalhe
          or {}))

    def para_dict(self) -> dict[str, Any]:
        return asdict(self)


@dataclass(frozen=True)
class ResultadoOperacao:
    """Representação transversal sem substituir resultados ricos de domínio."""
    ok: bool
    valor: Any = None
    falha: FalhaOperacao | None = None
    metadados: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def sucesso(cls, valor: Any = None, **metadados: Any) -> "ResultadoOperacao":
        return cls(True, valor=valor, metadados=metadados)
```

```
    @classmethod
    def erro(cls, falha: FalhaOperacao, **metadados: Any) -> "ResultadoOperacao":
        return cls(False, falha=falha, metadados=metadados)

    def para_dict(self) -> dict[str, Any]:
        return {
            "contrato": CONTRATO,
            "ok": self.ok,
            "valor": self.valor,
            "falha": self.falha.para_dict() if self.falha is not None else None,
            "metadados": dict(self.metadados),
        }


@dataclass(frozen=True)
class DiagnosticoOperacional:
    id: str
    dominio: str
    ok: bool
    mensagem: str
    capacidade: str | None = None
    sugestao: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @property
    def nivel(self) -> str:
        return "info" if self.ok else "erro"

    def para_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "dominio": self.dominio,
            "ok": self.ok,
            "nivel": self.nivel,
            "mensagem": self.mensagem,
            "capacidade": self.capacidade,
            "sugestao": self.sugestao,
            "detalhe": dict(self.detalhe),
        }


def diagnostico(id: str, dominio: str, ok: bool, mensagem: Any, *, capacidade: str |
  None = None, sugestao: str | None = None, detalhe: Mapping[str, Any] | None = None) ->
  DiagnosticoOperacional:
    return DiagnosticoOperacional(str(id), str(dominio), bool(ok), str(mensagem),
      capacidade, sugestao, dict(detalhe or {}))


def agregar_diagnosticos(itens: Iterable[DiagnosticoOperacional | Mapping[str, Any]], *,
   origem: str = "coral") -> dict[str, Any]:
    normalizados: list[dict[str, Any]] = []
    for item in itens:
        if isinstance(item, DiagnosticoOperacional):
            normalizados.append(item.para_dict())
            continue
        d = dict(item)
        if "id" not in d:
            d["id"] = str(d.get("nome", "diagnostico"))
        if "dominio" not in d:
            d["dominio"] = origem
        d["ok"] = bool(d.get("ok"))
```

```
        d.setdefault("nivel", "info" if d["ok"] else "erro")
        d.setdefault("mensagem", str(d.get("detalhe", "")))
        d.setdefault("capacidade", None)
        d.setdefault("sugestao", None)
        detalhe = d.get("detalhe")
        if not isinstance(detalhe, Mapping):
            d["detalhe"] = {"valor": detalhe}
        normalizados.append(d)
    return {
        "contrato": CONTRATO_DIAGNOSTICO,
        "origem": origem,
        "ok": all(x["ok"] for x in normalizados),
        "quantidade": len(normalizados),
        "itens": normalizados,
    }


__all__ = [
    "CONTRATO", "CONTRATO_DIAGNOSTICO", "ErroCoral", "ErroOperacaoCoral",
    "ErroDependenciaCoral", "FalhaOperacao", "ResultadoOperacao",
    "DiagnosticoOperacional", "diagnostico", "agregar_diagnosticos",
]

"""Experimentos reprodutíveis, medições e análise para Coral."""
from __future__ import annotations

from dataclasses import asdict, dataclass, field
from itertools import product
import csv
import hashlib
import inspect
import json
import math
from numbers import Integral
from pathlib import Path
import platform
import random
import statistics
import time
import tracemalloc
from typing import Any, Callable, Iterable, Mapping

from . import __version__ as CORAL_VERSION
from .comum import FalhaOperacao, ResultadoOperacao, diagnostico
from .tempo_eventos import FonteTempo


CONTRATO_RESULTADO_JSON = "coral.laboratorio.resultado/1"
VERSAO_RESULTADO_JSON = 1


def protocolo_medicao(relogio: FonteTempo | None = None) -> dict[str, Any]:
    """Descreve o protocolo usado pelo runtime sem prometer estabilidade de
      benchmark."""
    return {
        "relogio": "time.perf_counter" if relogio is None else type(relogio).__name__,
        "tempo_unidade": "segundos",
        "memoria": "tracemalloc",
        "memoria_unidade": "bytes",
        "aquecimento_execucoes": 0,
        "observacao": (
```

```
            "Tempo e memória dependem da máquina, do sistema, da carga e do runtime. "
            "Compare cenários executados sob o mesmo protocolo; não trate um valor
              isolado como universal."
        ),
    }


@dataclass(frozen=True)
class Medicao:
    repeticao: int
    parametros: dict[str, Any]
    semente: int | None
    tempo_segundos: float
    memoria_atual_bytes: int
    memoria_pico_bytes: int
    retorno: Any = None
    erro: str | None = None
    metricas: dict[str, Any] = field(default_factory=dict)


@dataclass
class ResultadoExperimento:
    nome: str
    medicoes: list[Medicao]
    metadados: dict[str, Any]

    @property
    def passou(self) -> bool:
        return all(m.erro is None for m in self.medicoes)

    def adicionar_metadados(self, nome: str, valor: Any) -> None:
        if not isinstance(nome, str) or not nome.strip():
            raise ValueError("O nome do metadado deve ser um texto não vazio.")
        chave = nome.strip()
        para_dict = getattr(valor, "para_dict", None)
        if callable(para_dict):
            valor = para_dict()
        from .persistencia import valor_portatil

        self.metadados[chave] = valor_portatil(valor)

    def retornos(self, *, incluir_falhas: bool = False) -> list[Any]:
        if incluir_falhas:
            return [m.retorno for m in self.medicoes]
        return [m.retorno for m in self.medicoes if m.erro is None]

    def resumo(self) -> dict[str, Any]:
        tempos = [m.tempo_segundos for m in self.medicoes if m.erro is None]
        picos = [m.memoria_pico_bytes for m in self.medicoes if m.erro is None]
        return {
            "nome": self.nome,
            "execucoes": len(self.medicoes),
            "sucessos": sum(m.erro is None for m in self.medicoes),
            "falhas": sum(m.erro is not None for m in self.medicoes),
            "tempo_medio_segundos": statistics.fmean(tempos) if tempos else None,
            "tempo_mediano_segundos": statistics.median(tempos) if tempos else None,
            "memoria_pico_media_bytes": statistics.fmean(picos) if picos else None,
            "passou": self.passou,
        }

    def para_dict(self) -> dict[str, Any]:
```

```
        return {
            "nome": self.nome,
            "metadados": self.metadados,
            "resumo": self.resumo(),
            "medicoes": [asdict(m) for m in self.medicoes],
        }

    def resultado_operacao(self) -> ResultadoOperacao:
        if self.passou:
            return ResultadoOperacao.sucesso(
                self, dominio="laboratorio", execucoes=len(self.medicoes)
            )
        erros = [m.erro for m in self.medicoes if m.erro is not None]
        falha = FalhaOperacao(
            "laboratorio.experimento.falhas",
            f"O experimento registrou {len(erros)} falha(s) em {len(self.medicoes)}
              execução(ões).",
            "laboratorio",
            "falha_como_dado",
            detalhe={"erros": erros},
        )
        return ResultadoOperacao.erro(falha, execucoes=len(self.medicoes))

    def salvar_json(self, caminho: str | Path) -> Path:
        from .persistencia import valor_portatil

        diagnostico_resultado = validar_resultado(self)
        erros_estruturais = [
            item for item in diagnostico_resultado["erros"] if item["codigo"] !=
              "portabilidade"
        ]
        if erros_estruturais:
            mensagens = "; ".join(item["mensagem"] for item in erros_estruturais)
            raise ValueError("Resultado de experimento inválido: " + mensagens)

        p = Path(caminho)
        p.parent.mkdir(parents=True, exist_ok=True)
        documento = valor_portatil(self.para_dict())
        documento["contrato"] = CONTRATO_RESULTADO_JSON
        documento["versao_esquema"] = VERSAO_RESULTADO_JSON
        texto = json.dumps(documento, ensure_ascii=False, indent=2, allow_nan=False)
        p.write_text(texto + "\n", encoding="utf-8")
        return p

    def salvar_csv(self, caminho: str | Path) -> Path:
        p = Path(caminho)
        p.parent.mkdir(parents=True, exist_ok=True)
        chaves = sorted({k for m in self.medicoes for k in m.parametros})
        metricas = sorted({k for m in self.medicoes for k in m.metricas})
        reservados = {
            "repeticao",
            "semente",
            "tempo_segundos",
            "memoria_atual_bytes",
            "memoria_pico_bytes",
            "erro",
        }
        conflitos = (
            (set(chaves) & reservados)
            | (set(metricas) & reservados)
            | (set(chaves) & set(metricas))
```

```
        )
        if conflitos:
            raise ValueError(
                "Nomes de parâmetros ou métricas colidem com colunas reservadas do CSV:
                  "
                + ", ".join(sorted(map(str, conflitos)))
            )
        campos = [
            "repeticao",
            "semente",
            *chaves,
            "tempo_segundos",
            "memoria_atual_bytes",
            "memoria_pico_bytes",
            "erro",
            *metricas,
        ]
        with p.open("w", encoding="utf-8", newline="") as f:
            w = csv.DictWriter(f, fieldnames=campos)
            w.writeheader()
            for m in self.medicoes:
                linha = {
                    "repeticao": m.repeticao,
                    "semente": m.semente,
                    "tempo_segundos": m.tempo_segundos,
                    "memoria_atual_bytes": m.memoria_atual_bytes,
                    "memoria_pico_bytes": m.memoria_pico_bytes,
                    "erro": m.erro or "",
                }
                linha.update(m.parametros)
                linha.update(m.metricas)
                w.writerow(linha)
        return p

    def salvar(self, caminho: str | Path) -> Path:
        p = Path(caminho)
        sufixo = p.suffix.casefold()
        if sufixo == ".json":
            return self.salvar_json(p)
        if sufixo == ".csv":
            return self.salvar_csv(p)
        if sufixo in {".md", ".markdown"}:
            return self.salvar_markdown(p)
        raise ValueError("Formato de relatório desconhecido. Use .json, .csv ou .md")

    def salvar_markdown(self, caminho: str | Path) -> Path:
        p = Path(caminho)
        p.parent.mkdir(parents=True, exist_ok=True)
        r = self.resumo()
        linhas = [
            f"# Experimento: {self.nome}",
            "",
            f"- Execuções: {r['execucoes']}",
            f"- Sucessos: {r['sucessos']}",
            f"- Falhas: {r['falhas']}",
            f"- Tempo médio: {r['tempo_medio_segundos']} s",
            f"- Tempo mediano: {r['tempo_mediano_segundos']} s",
            f"- Memória de pico média: {r['memoria_pico_media_bytes']} bytes",
            "",
            "## Metadados",
            "",
```

```
        ]
        linhas += [f"- **{k}**: {v}" for k, v in self.metadados.items()]
        p.write_text("\n".join(linhas) + "\n", encoding="utf-8")
        return p


def _metadados(
    funcao: Callable[..., Any] | None = None, relogio: FonteTempo | None = None
) -> dict[str, Any]:
    d = {
        "coral": CORAL_VERSION,
        "python": platform.python_version(),
        "sistema": platform.system(),
        "arquitetura": platform.machine(),
        "protocolo_medicao": protocolo_medicao(relogio),
    }
    if funcao is not None:
        try:
            fonte = inspect.getsource(funcao).encode("utf-8")
            d["funcao"] = getattr(funcao, "__qualname__", getattr(funcao, "__name__",
              str(funcao)))
            d["hash_funcao_sha256"] = hashlib.sha256(fonte).hexdigest()
        except Exception:
            pass
    try:
        import numpy as np

        d["numpy"] = np.__version__
    except Exception:
        pass
    return d


def grade_parametros(parametros: Mapping[str, Iterable[Any]] | None) -> list[dict[str,
  Any]]:
    if not parametros:
        return [{}]
    nomes = list(parametros)
    valores = [list(parametros[n]) for n in nomes]
    vazios = [str(nome) for nome, itens in zip(nomes, valores) if not itens]
    if vazios:
        raise ValueError("Parâmetros sem valores na grade: " + ", ".join(vazios))
    return [dict(zip(nomes, comb)) for comb in product(*valores)]


def _inteiro_exato(valor: Any, nome: str, *, permitir_none: bool = False) -> int | None:
    if valor is None and permitir_none:
        return None
    if isinstance(valor, bool):
        raise ValueError(
            f"{nome} deve ser um inteiro" + (" ou None" if permitir_none else "
              positivo")
        )
    if isinstance(valor, Integral):
        return int(valor)
    if isinstance(valor, str):
        texto = valor.strip()
        if texto and texto.lstrip("+-").isdigit():
            return int(texto)
    raise ValueError(
        f"{nome} deve ser um inteiro" + (" ou None" if permitir_none else " positivo")
```

```
    )


def _contar_iteravel(valores: Iterable[Any], nome: str) -> int:
    try:
        quantidade = len(valores)  # type: ignore[arg-type]
    except (TypeError, AttributeError):
        quantidade = sum(1 for _ in valores)
    if quantidade < 1:
        raise ValueError(f"Parâmetro sem valores na grade: {nome}")
    return quantidade


def planejar_experimento(
    parametros: Mapping[str, Iterable[Any]] | None,
    repeticoes: int = 1,
    limite: int | None = None,
) -> dict[str, Any]:
    """Calcula o tamanho de uma execução experimental sem materializar a grade completa.

    Iteráveis sem ``len`` são percorridos para contagem, mas seus itens não são
    mantidos em memória. A função nunca executa a função que será estudada.
    """
    total_repeticoes = _inteiro_exato(repeticoes, "repeticoes")
    assert total_repeticoes is not None
    if total_repeticoes < 1:
        raise ValueError("repeticoes deve ser maior ou igual a 1")
    limite_validado = _inteiro_exato(limite, "limite", permitir_none=True)
    if limite_validado is not None and limite_validado < 1:
        raise ValueError("limite deve ser maior ou igual a 1")

    dimensoes: dict[str, int] = {}
    combinacoes = 1
    if parametros:
        for nome, valores in parametros.items():
            if not isinstance(nome, str) or not nome:
                raise ValueError("Nomes de parâmetros devem ser textos não vazios.")
            quantidade = _contar_iteravel(valores, nome)
            dimensoes[nome] = quantidade
            combinacoes *= quantidade

    execucoes = combinacoes * total_repeticoes
    excesso = None if limite_validado is None else max(0, execucoes - limite_validado)
    return {
        "parametros": list(dimensoes),
        "dimensoes": dimensoes,
        "combinacoes": combinacoes,
        "repeticoes": total_repeticoes,
        "execucoes_previstas": execucoes,
        "limite": limite_validado,
        "excede_limite": bool(excesso),
        "excesso": excesso,
        "materializou_grade": False,
    }


def _valor_criterio(medicao: Medicao, chave: str) -> tuple[bool, Any]:
    if chave.startswith("parametros."):
        nome = chave.split(".", 1)[1]
        return (nome in medicao.parametros, medicao.parametros.get(nome))
    if chave.startswith("metricas."):
```

```
        nome = chave.split(".", 1)[1]
        return (nome in medicao.metricas, medicao.metricas.get(nome))
    if chave in {
        "repeticao",
        "semente",
        "tempo_segundos",
        "memoria_atual_bytes",
        "memoria_pico_bytes",
        "retorno",
        "erro",
    }:
        return True, getattr(medicao, chave)
    if chave in medicao.parametros:
        return True, medicao.parametros[chave]
    if chave in medicao.metricas:
        return True, medicao.metricas[chave]
    return False, None


def selecionar_medicoes(
    resultado: ResultadoExperimento,
    criterio: Callable[[Medicao], bool] | Mapping[str, Any],
) -> list[Medicao]:
    """Seleciona medições preservando ordem e sem modificar o resultado original."""
    if not isinstance(resultado, ResultadoExperimento):
        raise TypeError("resultado deve ser ResultadoExperimento")

    if callable(criterio):
        return [medicao for medicao in resultado.medicoes if bool(criterio(medicao))]
    if isinstance(criterio, Mapping):
        pares = list(criterio.items())
        if not all(isinstance(chave, str) and chave for chave, _ in pares):
            raise ValueError("As chaves do critério precisam ser textos não vazios.")

        def corresponde(medicao: Medicao) -> bool:
            for chave, esperado in pares:
                existe, atual = _valor_criterio(medicao, chave)
                if not existe or atual != esperado:
                    return False
            return True

        return [medicao for medicao in resultado.medicoes if corresponde(medicao)]
    raise TypeError("criterio deve ser uma função ou um mapeamento")


def _chave_agrupamento(valor: Any) -> str:
    from .persistencia import ErroPersistencia, valor_portatil

    try:
        portatil = valor_portatil(valor)
        return json.dumps(
            portatil,
            ensure_ascii=False,
            sort_keys=True,
            separators=(",", ":"),
            allow_nan=False,
        )
    except (ErroPersistencia, TypeError, ValueError) as exc:
        raise ValueError(
            "O parâmetro contém valor não agrupável pela política portátil da Coral."
        ) from exc
```

```
def _resumo_medicoes(medicoes: list[Medicao]) -> dict[str, Any]:
    sucessos = [m for m in medicoes if m.erro is None]
    tempos = [m.tempo_segundos for m in sucessos]
    atuais = [m.memoria_atual_bytes for m in sucessos]
    picos = [m.memoria_pico_bytes for m in sucessos]
    return {
        "execucoes": len(medicoes),
        "sucessos": len(sucessos),
        "falhas": len(medicoes) - len(sucessos),
        "tempo_medio_segundos": statistics.fmean(tempos) if tempos else None,
        "tempo_mediano_segundos": statistics.median(tempos) if tempos else None,
        "memoria_atual_media_bytes": statistics.fmean(atuais) if atuais else None,
        "memoria_pico_media_bytes": statistics.fmean(picos) if picos else None,
    }


def resumir_por_parametro(resultado: ResultadoExperimento, nome: str) -> dict[str, Any]:
    """Agrupa medições pelo valor de um parâmetro, preservando a ordem de aparição."""
    if not isinstance(resultado, ResultadoExperimento):
        raise TypeError("resultado deve ser ResultadoExperimento")
    if not isinstance(nome, str) or not nome:
        raise ValueError("nome deve ser um texto não vazio")
    if not resultado.medicoes:
        return {"parametro": nome, "grupos": [], "execucoes": 0}

    ausentes = [indice for indice, medicao in enumerate(resultado.medicoes) if nome not
      in medicao.parametros]
    if ausentes:
        raise ValueError(
            f"O parâmetro {nome!r} está ausente em {len(ausentes)} medição(ões)."
        )

    grupos: dict[str, dict[str, Any]] = {}
    ordem: list[str] = []
    for medicao in resultado.medicoes:
        valor = medicao.parametros[nome]
        chave = _chave_agrupamento(valor)
        if chave not in grupos:
            grupos[chave] = {"valor": valor, "medicoes": []}
            ordem.append(chave)
        grupos[chave]["medicoes"].append(medicao)

    saida = []
    for chave in ordem:
        grupo = grupos[chave]
        resumo = _resumo_medicoes(grupo["medicoes"])
        saida.append({"valor": grupo["valor"], **resumo})
    return {"parametro": nome, "grupos": saida, "execucoes": len(resultado.medicoes)}



def _valores_configuracao(resultado: ResultadoExperimento) -> dict[str, list[Any]]:
    """Resume valores de parâmetros em ordem de primeira aparição."""
    nomes: list[str] = []
    vistos_nomes: set[str] = set()
    for medicao in resultado.medicoes:
        for nome in medicao.parametros:
            if nome not in vistos_nomes:
                vistos_nomes.add(nome)
```

```
                nomes.append(nome)

    saida: dict[str, list[Any]] = {}
    for nome in nomes:
        valores: list[Any] = []
        chaves: set[str] = set()
        for medicao in resultado.medicoes:
            if nome not in medicao.parametros:
                continue
            valor = medicao.parametros[nome]
            chave = _chave_agrupamento(valor)
            if chave not in chaves:
                chaves.add(chave)
                valores.append(valor)
        saida[nome] = valores
    return saida


def _diferenca_descritiva(a: Any, b: Any) -> dict[str, Any]:
    diferenca = None
    if a is not None and b is not None:
        diferenca = b - a
    return {"a": a, "b": b, "diferenca_b_menos_a": diferenca}


def comparar_experimentos(
    a: ResultadoExperimento,
    b: ResultadoExperimento,
) -> dict[str, Any]:
    """Compara dois experimentos de forma estritamente descritiva.

    A função não calcula p valor, intervalo de confiança nem significância
    estatística. Diferenças numéricas são sempre apresentadas como ``b - a``.
    """
    if not isinstance(a, ResultadoExperimento) or not isinstance(b,
      ResultadoExperimento):
        raise TypeError("a e b devem ser ResultadoExperimento")

    for rotulo, resultado in (("a", a), ("b", b)):
        diagnostico_resultado = validar_resultado(resultado)
        erros_estruturais = [
            item
            for item in diagnostico_resultado["erros"]
            if item["codigo"] != "portabilidade"
        ]
        if erros_estruturais:
            mensagens = "; ".join(item["mensagem"] for item in erros_estruturais)
            raise ValueError(f"Resultado {rotulo} inválido: {mensagens}")

    resumo_a = a.resumo()
    resumo_b = b.resumo()
    parametros_a = _valores_configuracao(a)
    parametros_b = _valores_configuracao(b)

    sementes_a = []
    sementes_b = []
    for medicao in a.medicoes:
        if medicao.semente not in sementes_a:
            sementes_a.append(medicao.semente)
    for medicao in b.medicoes:
        if medicao.semente not in sementes_b:
```

```
            sementes_b.append(medicao.semente)

    chaves_metadados = list(dict.fromkeys([*a.metadados.keys(), *b.metadados.keys()]))
    metadados_diferentes = [
        {
            "nome": chave,
            "presente_a": chave in a.metadados,
            "presente_b": chave in b.metadados,
            "a": a.metadados.get(chave),
            "b": b.metadados.get(chave),
        }
        for chave in chaves_metadados
        if a.metadados.get(chave) != b.metadados.get(chave)
        or (chave in a.metadados) != (chave in b.metadados)
    ]

    campos_numericos = (
        "execucoes",
        "sucessos",
        "falhas",
        "tempo_medio_segundos",
        "tempo_mediano_segundos",
        "memoria_pico_media_bytes",
    )
    metricas = {
        campo: _diferenca_descritiva(resumo_a[campo], resumo_b[campo])
        for campo in campos_numericos
    }

    return {
        "a": {"nome": a.nome, "resumo": resumo_a},
        "b": {"nome": b.nome, "resumo": resumo_b},
        "configuracao": {
            "nomes_iguais": a.nome == b.nome,
            "parametros": {
                "a": parametros_a,
                "b": parametros_b,
                "iguais": parametros_a == parametros_b,
            },
            "sementes": {
                "a": sementes_a,
                "b": sementes_b,
                "iguais": sementes_a == sementes_b,
            },
            "metadados": {
                "iguais": not metadados_diferentes,
                "diferentes": metadados_diferentes,
            },
        },
        "metricas": metricas,
        "significancia_estatistica": None,
        "interpretacao": (
            "Comparação descritiva. Diferenças numéricas usam b - a e não demonstram "
            "significância estatística, causalidade ou superioridade geral."
        ),
    }


def _erro_validacao(codigo: str, caminho: str, mensagem: str) -> dict[str, str]:
    return {"codigo": codigo, "caminho": caminho, "mensagem": mensagem}
```

```
def validar_resultado(resultado: Any) -> dict[str, Any]:
    """Valida a estrutura e a portabilidade de um resultado sem modificá lo."""
    erros: list[dict[str, str]] = []
    avisos: list[dict[str, str]] = []
    if not isinstance(resultado, ResultadoExperimento):
        erros.append(
            _erro_validacao(
                "tipo_resultado",
                "$",
                "O valor precisa ser uma instância de ResultadoExperimento.",
            )
        )
        return {"valido": False, "erros": erros, "avisos": avisos}

    if not isinstance(resultado.nome, str) or not resultado.nome.strip():
        erros.append(_erro_validacao("nome", "$.nome", "O nome deve ser um texto não
          vazio."))
    if not isinstance(resultado.metadados, dict) or not all(
        isinstance(chave, str) for chave in resultado.metadados
    ):
        erros.append(
            _erro_validacao(
                "metadados", "$.metadados", "Metadados devem ser um dicionário com
                  chaves de texto."
            )
        )
    if not isinstance(resultado.medicoes, list):
        erros.append(_erro_validacao("medicoes", "$.medicoes", "Medições devem formar
          uma lista."))
        return {"valido": False, "erros": erros, "avisos": avisos}

    for indice, medicao in enumerate(resultado.medicoes):
        base = f"$.medicoes[{indice}]"
        if not isinstance(medicao, Medicao):
            erros.append(_erro_validacao("tipo_medicao", base, "Item não é uma
              Medicao."))
            continue
        if isinstance(medicao.repeticao, bool) or not isinstance(medicao.repeticao,
          Integral) or medicao.repeticao < 1:
            erros.append(_erro_validacao("repeticao", base + ".repeticao", "Repetição
              deve ser inteira positiva."))
        if medicao.semente is not None and (
            isinstance(medicao.semente, bool) or not isinstance(medicao.semente,
              Integral)
        ):
            erros.append(_erro_validacao("semente", base + ".semente", "Semente deve ser
               inteira ou None."))
        if not isinstance(medicao.tempo_segundos, (int, float)) or
          isinstance(medicao.tempo_segundos, bool) or not
          math.isfinite(float(medicao.tempo_segundos)) or medicao.tempo_segundos < 0:
            erros.append(_erro_validacao("tempo", base + ".tempo_segundos", "Tempo deve
              ser finito e não negativo."))
        for campo in ("memoria_atual_bytes", "memoria_pico_bytes"):
            valor = getattr(medicao, campo)
            if isinstance(valor, bool) or not isinstance(valor, Integral) or valor < 0:
                erros.append(_erro_validacao("memoria", base + "." + campo, "Memória
                  deve ser inteira e não negativa."))
        if not isinstance(medicao.parametros, dict) or not all(isinstance(chave, str)
          for chave in medicao.parametros):
            erros.append(_erro_validacao("parametros", base + ".parametros", "Parâmetros
```

```
                devem ter chaves de texto."))
        if not isinstance(medicao.metricas, dict) or not all(isinstance(chave, str) for
          chave in medicao.metricas):
            erros.append(_erro_validacao("metricas", base + ".metricas", "Métricas devem
               ter chaves de texto."))
        if medicao.erro is not None and not isinstance(medicao.erro, str):
            erros.append(_erro_validacao("erro", base + ".erro", "Erro deve ser texto ou
               None."))

    from .persistencia import ErroPersistencia, valor_portatil

    try:
        valor_portatil(resultado.para_dict())
    except ErroPersistencia as exc:
        erros.append(_erro_validacao("portabilidade", "$", str(exc)))

    return {"valido": not erros, "erros": erros, "avisos": avisos}


def _restaurar_documento_portatil(documento: dict[str, Any]) -> dict[str, Any]:
    from .persistencia import CONTRATO, TIPO_VALOR_BASE, VERSAO_ESQUEMA, de_dados

    envelope = {
        "contrato": CONTRATO,
        "versao_esquema": VERSAO_ESQUEMA,
        "tipo": TIPO_VALOR_BASE,
        "dados": {"valor": documento},
    }
    restaurado = de_dados(envelope)
    if not isinstance(restaurado, dict):
        raise ValueError("Documento do laboratório não resultou em um objeto.")
    return restaurado


def carregar_resultado_json(caminho: str | Path) -> ResultadoExperimento:
    """Carrega relatório JSON do laboratório e recusa contratos ou esquemas
      desconhecidos."""
    p = Path(caminho)
    try:
        bruto = json.loads(p.read_text(encoding="utf-8"))
    except (OSError, UnicodeError, json.JSONDecodeError) as exc:
        raise ValueError(f"Não foi possível ler relatório JSON do laboratório: {exc}")
          from exc
    if not isinstance(bruto, dict):
        raise ValueError("Relatório JSON do laboratório deve ser um objeto.")

    contrato = bruto.get("contrato")
    versao = bruto.get("versao_esquema")
    if contrato is None and versao is None:
        # Formato histórico conhecido, anterior à 1.6.10.
        if not {"nome", "metadados", "medicoes"}.issubset(bruto):
            raise ValueError("Formato JSON do laboratório desconhecido.")
    else:
        if contrato != CONTRATO_RESULTADO_JSON:
            raise ValueError(f"Contrato JSON do laboratório desconhecido: {contrato!r}")
        if versao != VERSAO_RESULTADO_JSON:
            raise ValueError(f"Versão de esquema do laboratório desconhecida:
              {versao!r}")

    dados_codificados = {k: v for k, v in bruto.items() if k not in {"contrato",
      "versao_esquema", "resumo"}}
```

```
    dados = _restaurar_documento_portatil(dados_codificados)
    nome = dados.get("nome")
    metadados = dados.get("metadados")
    medicoes_brutas = dados.get("medicoes")
    if not isinstance(nome, str) or not isinstance(metadados, dict) or not
      isinstance(medicoes_brutas, list):
        raise ValueError("Estrutura do relatório JSON do laboratório é inválida.")

    medicoes: list[Medicao] = []
    for indice, item in enumerate(medicoes_brutas):
        if not isinstance(item, dict):
            raise ValueError(f"Medição {indice} do relatório é inválida.")
        try:
            medicoes.append(_medicao_de_dados(item))
        except (KeyError, TypeError, ValueError) as exc:
            raise ValueError(f"Medição {indice} do relatório é inválida: {exc}") from
              exc

    resultado = ResultadoExperimento(nome, medicoes, metadados)
    diagnostico_resultado = validar_resultado(resultado)
    if not diagnostico_resultado["valido"]:
        mensagens = "; ".join(item["mensagem"] for item in
          diagnostico_resultado["erros"])
        raise ValueError("Relatório JSON do laboratório inconsistente: " + mensagens)
    return resultado


def executar_experimento(
    nome: str,
    funcao: Callable[..., Any],
    *,
    repeticoes: int = 1,
    parametros: Mapping[str, Iterable[Any]] | None = None,
    semente: int | None = None,
    metricas: Callable[[Any], Mapping[str, Any]] | None = None,
    continuar_em_erro: bool = True,
    relogio: FonteTempo | None = None,
) -> ResultadoExperimento:
    total_repeticoes = _inteiro_exato(repeticoes, "repeticoes")
    assert total_repeticoes is not None
    if total_repeticoes < 1:
        raise ValueError("repeticoes deve ser maior ou igual a 1")
    semente_base = _inteiro_exato(semente, "semente", permitir_none=True)

    grade = grade_parametros(parametros)
    medicoes: list[Medicao] = []
    indice = 0
    estado_random = random.getstate()
    np = None
    estado_numpy = None
    try:
        try:
            import numpy as np_mod

            np = np_mod
            estado_numpy = np.random.get_state()
        except Exception:
            np = None
            estado_numpy = None

        for param in grade:
```

```
            for rep in range(total_repeticoes):
                seed = None if semente_base is None else semente_base + indice
                if seed is not None:
                    random.seed(seed)
                    if np is not None:
                        np.random.seed(seed % (2**32 - 1))

                rastreio_preexistente = tracemalloc.is_tracing()
                if not rastreio_preexistente:
                    tracemalloc.start()
                agora = time.perf_counter if relogio is None else relogio.agora
                inicio = float(agora())
                retorno = None
                erro = None
                mets = {}
                propagar: Exception | None = None
                try:
                    retorno = funcao(**param)
                    if metricas is not None:
                        mets = dict(metricas(retorno))
                except Exception as exc:
                    erro = f"{type(exc).__name__}: {exc}"
                    if not continuar_em_erro:
                        propagar = exc
                finally:
                    duracao = max(0.0, float(agora()) - inicio)
                    if tracemalloc.is_tracing():
                        atual, pico = tracemalloc.get_traced_memory()
                    else:
                        atual, pico = 0, 0
                    if not rastreio_preexistente and tracemalloc.is_tracing():
                        tracemalloc.stop()
                if propagar is not None:
                    raise propagar
                medicoes.append(
                    Medicao(
                        rep + 1,
                        dict(param),
                        seed,
                        duracao,
                        int(atual),
                        int(pico),
                        retorno,
                        erro,
                        mets,
                    )
                )
                indice += 1
    finally:
        random.setstate(estado_random)
        if np is not None and estado_numpy is not None:
            try:
                np.random.set_state(estado_numpy)
            except Exception:
                pass
    return ResultadoExperimento(str(nome), medicoes, _metadados(funcao, relogio))


def _normalizar_receita_procedural(receita: Any):
    from .procedural import ReceitaProcedural
```

```
    if isinstance(receita, ReceitaProcedural):
        return receita
    if isinstance(receita, Mapping):
        return ReceitaProcedural.de_dados(receita)
    raise TypeError("receita procedural precisa ser ReceitaProcedural ou dados
      portáteis")


def _sementes_procedurais(receita: Any, sementes: Iterable[Any] | None) -> tuple[Any,
  ...]:
    from .procedural import semente as criar_semente

    if sementes is None:
        return (receita.semente.valor,)
    valores = tuple(criar_semente(item).valor for item in sementes)
    if not valores:
        raise ValueError("sementes procedurais precisa conter ao menos um valor")
    return valores


def _grade_procedural(
    receita: Any,
    parametros: Mapping[str, Iterable[Any]] | None,
    sementes: Iterable[Any] | None,
) -> dict[str, tuple[Any, ...]]:
    grade: dict[str, tuple[Any, ...]] = {}
    for nome, valores in (parametros or {}).items():
        if nome == "semente_procedural":
            raise ValueError("semente_procedural é reservada pela integração
              procedural")
        itens = tuple(valores)
        if not itens:
            raise ValueError(f"Parâmetro sem valores na grade: {nome}")
        grade[nome] = itens
    grade["semente_procedural"] = _sementes_procedurais(receita, sementes)
    return grade


def planejar_experimento_procedural(
    receita: Any,
    *,
    sementes: Iterable[Any] | None = None,
    parametros: Mapping[str, Iterable[Any]] | None = None,
    repeticoes: int = 1,
    limite_execucoes: int | None = None,
    limite_custo_total: int | None = None,
) -> dict[str, Any]:
    """Planeja um estudo procedural sem reconstruir nem executar a receita.

    O custo combina a quantidade de execuções com o pior caso declarado pela
    própria receita. Isso permite rejeitar grades enormes antes da materialização.
    """
    from .procedural import diagnosticar_custo_receita

    receita_n = _normalizar_receita_procedural(receita)
    grade = _grade_procedural(receita_n, parametros, sementes)
    plano = planejar_experimento(grade, repeticoes=repeticoes, limite=limite_execucoes)
    custo_receita = diagnosticar_custo_receita(receita_n)
    custo_total = plano["execucoes_previstas"] * custo_receita["custo_maximo_estimado"]
    excede_custo = False
    limite_custo_n = None
```

```
    if limite_custo_total is not None:
        limite_custo_n = _inteiro_exato(limite_custo_total, "limite de custo total")
        assert limite_custo_n is not None
        if limite_custo_n < 1:
            raise ValueError("limite de custo total deve ser maior ou igual a 1")
        excede_custo = custo_total > limite_custo_n
    return {
        "contrato": "coral.laboratorio.procedural.plano/1",
        "receita": receita_n.para_dados(),
        "plano_experimento": plano,
        "custo_por_execucao": custo_receita,
        "custo_maximo_total": custo_total,
        "limite_custo_total": limite_custo_n,
        "excede_limite_custo": excede_custo,
        "executou_receita": False,
    }


def executar_experimento_procedural(
    nome: str,
    receita: Any,
    funcao: Callable[..., Any],
    *,
    sementes: Iterable[Any] | None = None,
    parametros: Mapping[str, Iterable[Any]] | None = None,
    repeticoes: int = 1,
    semente_laboratorio: int | None = None,
    metricas: Callable[[Any], Mapping[str, Any]] | None = None,
    continuar_em_erro: bool = True,
    relogio: FonteTempo | None = None,
    limite_custo_receita: int = 1_000_000,
    limite_execucoes: int | None = None,
    limite_custo_total: int | None = None,
) -> ResultadoExperimento:
    """Executa uma receita procedural como fator explícito de um experimento.

    ``funcao`` recebe primeiro o produto reconstruído e depois os parâmetros
    declarados pelo estudo. A semente da receita aparece em cada medição como
    ``semente_procedural`` e a receita base é preservada nos metadados.
    """
    from .procedural import ReceitaProcedural, reconstruir_receita

    if not callable(funcao):
        raise TypeError("função do experimento procedural precisa ser chamável")
    receita_n = _normalizar_receita_procedural(receita)
    plano = planejar_experimento_procedural(
        receita_n,
        sementes=sementes,
        parametros=parametros,
        repeticoes=repeticoes,
        limite_execucoes=limite_execucoes,
        limite_custo_total=limite_custo_total,
    )
    plano_base = plano["plano_experimento"]
    if plano_base["excede_limite"]:
        raise ValueError("experimento procedural excede o limite de execuções")
    if plano["excede_limite_custo"]:
        raise ValueError("experimento procedural excede o limite de custo total")

    grade = _grade_procedural(receita_n, parametros, sementes)
    parametros_receita = receita_n.parametros
```

```
    versao_receita = receita_n.versao_algoritmo
    algoritmo = receita_n.algoritmo

    def executar_uma(*, semente_procedural: Any, **parametros_estudo: Any):
        receita_execucao = ReceitaProcedural(
            algoritmo,
            semente_raiz=semente_procedural,
            parametros=parametros_receita,
            versao_algoritmo=versao_receita,
        )
        produto = reconstruir_receita(receita_execucao,
          limite_custo=limite_custo_receita)
        return funcao(produto, **parametros_estudo)

    resultado = executar_experimento(
        nome,
        executar_uma,
        repeticoes=repeticoes,
        parametros=grade,
        semente=semente_laboratorio,
        metricas=metricas,
        continuar_em_erro=continuar_em_erro,
        relogio=relogio,
    )
    resultado.adicionar_metadados(
        "procedural",
        {
            "contrato": "coral.laboratorio.procedural/1",
            "receita_base": receita_n.para_dados(),
            "sementes": list(grade["semente_procedural"]),
            "limite_custo_receita": limite_custo_receita,
            "custo_maximo_total": plano["custo_maximo_total"],
        },
    )
    return resultado


def diagnosticar_laboratorio() -> dict[str, Any]:
    dados = {
        "disponivel": True,
        "coral": CORAL_VERSION,
        "recursos": [
            "tempo",
            "memoria",
            "sementes",
            "parametros",
            "planejamento",
            "selecao",
            "resumos_por_parametro",
            "json_recuperavel",
            "comparacao_descritiva",
            "experimentos_procedurais",
            "csv",
            "markdown",
        ],
        "protocolo_medicao": protocolo_medicao(),
    }
    dados["diagnostico"] = diagnostico(
        "laboratorio.medicao",
        "laboratorio",
        True,
```

```
        "Laboratório disponível para planejar, executar, analisar e recuperar
          experimentos.",
        capacidade="laboratorio",
        detalhe={"recursos": list(dados["recursos"])},
    ).para_dict()
    return dados


__all__ = [
    "CONTRATO_RESULTADO_JSON",
    "VERSAO_RESULTADO_JSON",
    "Medicao",
    "ResultadoExperimento",
    "grade_parametros",
    "planejar_experimento",
    "selecionar_medicoes",
    "resumir_por_parametro",
    "validar_resultado",
    "carregar_resultado_json",
    "comparar_experimentos",
    "executar_experimento",
    "planejar_experimento_procedural",
    "executar_experimento_procedural",
    "diagnosticar_laboratorio",
    "protocolo_medicao",
]


def _medicao_para_dados(m: Medicao) -> dict[str, Any]:
    return {
        "repeticao": m.repeticao,
        "parametros": dict(m.parametros),
        "semente": m.semente,
        "tempo_segundos": m.tempo_segundos,
        "memoria_atual_bytes": m.memoria_atual_bytes,
        "memoria_pico_bytes": m.memoria_pico_bytes,
        "retorno": m.retorno,
        "erro": m.erro,
        "metricas": dict(m.metricas),
    }


def _medicao_de_dados(d: dict[str, Any]) -> Medicao:
    return Medicao(
        int(d["repeticao"]),
        dict(d.get("parametros", {})),
        d.get("semente"),
        float(d["tempo_segundos"]),
        int(d["memoria_atual_bytes"]),
        int(d["memoria_pico_bytes"]),
        retorno=d.get("retorno"),
        erro=d.get("erro"),
        metricas=dict(d.get("metricas", {})),
    )


def _resultado_para_dados(r: ResultadoExperimento) -> dict[str, Any]:
    from .persistencia import para_dados

    return {
        "nome": r.nome,
```

```
        "medicoes": [para_dados(m) for m in r.medicoes],
        "metadados": dict(r.metadados),
    }


def _resultado_de_dados(d: dict[str, Any]) -> ResultadoExperimento:
    medicoes = d.get("medicoes", [])
    if (
        not isinstance(d.get("nome"), str)
        or not isinstance(medicoes, list)
        or not all(isinstance(x, Medicao) for x in medicoes)
    ):
        raise ValueError("payload de ResultadoExperimento inválido")
    metadados = d.get("metadados", {})
    if not isinstance(metadados, dict):
        raise ValueError("metadados inválidos")
    return ResultadoExperimento(d["nome"], list(medicoes), dict(metadados))


from .persistencia import registrar_adaptador as _registrar_adaptador

_registrar_adaptador(Medicao, "coral.laboratorio/Medicao", _medicao_para_dados,
  _medicao_de_dados)
_registrar_adaptador(
    ResultadoExperimento,
    "coral.laboratorio/ResultadoExperimento",
    _resultado_para_dados,
    _resultado_de_dados,
)

"""Tempo, eventos e cancelamento compartilhados do runtime Coral.

Este módulo materializa a segunda parte da coesão 1.5 sem substituir as APIs
históricas de ``coral.regras``. Ele fornece fontes de tempo injetáveis,
validação uniforme de durações, eventos síncronos/enfileirados/propagados e um
token cooperativo de cancelamento reutilizável por bibliotecas e ferramentas.
"""
from __future__ import annotations

from collections import deque
from dataclasses import dataclass, field
import math
import time
from typing import Any, Callable, Mapping, Protocol, runtime_checkable

from .comum import ErroOperacaoCoral

CONTRATO = "coral.tempo_eventos/1"


def validar_duracao(valor: Any, nome: str = "duração", *, permitir_none: bool = False,
                    permitir_zero: bool = True) -> float | None:
    """Normaliza uma duração pública para segundos finitos não negativos."""
    if valor is None and permitir_none:
        return None
    if isinstance(valor, bool):
        raise ValueError(f"{nome} deve ser um número finito maior ou igual a zero")
    try:
        numero = float(valor)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ValueError(f"{nome} deve ser um número finito maior ou igual a zero") from
```

```
            exc
    if not math.isfinite(numero) or numero < 0 or (numero == 0 and not permitir_zero):
        operador = "maior que zero" if not permitir_zero else "maior ou igual a zero"
        raise ValueError(f"{nome} deve ser um número finito {operador}")
    return numero


def normalizar_dt(valor: Any, *, compatibilidade_negativo_zero: bool = True) -> float:
    """Normaliza avanço de simulação.

    A linha 1.2 já definia que ``dt`` negativo não volta temporizadores. Por
    compatibilidade, a política padrão converte valores negativos finitos em
    zero; novos consumidores podem optar por rejeitá-los.
    """
    if isinstance(valor, bool):
        raise ValueError("avanço de tempo deve ser um número finito")
    try:
        numero = float(valor)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ValueError("avanço de tempo deve ser um número finito") from exc
    if not math.isfinite(numero):
        raise ValueError("avanço de tempo deve ser um número finito")
    if numero < 0:
        if compatibilidade_negativo_zero:
            return 0.0
        raise ValueError("avanço de tempo não pode ser negativo")
    return numero


@runtime_checkable
class FonteTempo(Protocol):
    def agora(self) -> float: ...
    def passo(self) -> float: ...


class RelogioMonotonico:
    """Relógio real baseado em ``perf_counter`` e apropriado para durações."""
    def __init__(self, fonte: Callable[[], float] = time.perf_counter):
        self._fonte = fonte
        self._ultimo = float(fonte())

    def agora(self) -> float:
        return float(self._fonte())

    def passo(self) -> float:
        atual = self.agora()
        dt = normalizar_dt(atual - self._ultimo)
        self._ultimo = atual
        return dt


class RelogioSimulado:
    """Relógio controlado pelo programa, sem dormir nem consultar o sistema."""
    def __init__(self, inicial: float = 0.0):
        self._agora = float(validar_duracao(inicial, "tempo inicial"))
        self._ultimo = self._agora

    def agora(self) -> float:
        return self._agora

    def avancar(self, dt: Any) -> float:
```

```
        self._agora += normalizar_dt(dt, compatibilidade_negativo_zero=False)
        return self._agora

    def passo(self) -> float:
        dt = normalizar_dt(self._agora - self._ultimo,
          compatibilidade_negativo_zero=False)
        self._ultimo = self._agora
        return dt

    def definir(self, instante: Any) -> float:
        novo = float(validar_duracao(instante, "instante"))
        if novo < self._agora:
            raise ValueError("o relógio simulado não pode retroceder")
        self._agora = novo
        return novo


class OperacaoCancelada(ErroOperacaoCoral):
    """Operação cooperativa interrompida antes de uma fase segura."""


class TokenCancelamento:
    def __init__(self):
        self._cancelado = False
        self._motivo: str | None = None

    @property
    def cancelado(self) -> bool:
        return self._cancelado

    @property
    def motivo(self) -> str | None:
        return self._motivo

    def cancelar(self, motivo: str = "Operação cancelada pelo usuário.") -> None:
        self._cancelado = True
        self._motivo = str(motivo)

    def verificar(self) -> None:
        if self._cancelado:
            raise OperacaoCancelada(self._motivo or "Operação cancelada.")


@dataclass
class Evento:
    nome: str
    args: tuple[Any, ...] = ()
    kwargs: Mapping[str, Any] = field(default_factory=dict)
    origem: str | None = None
    sequencia: int = 0
    instante: float | None = None
    cancelavel: bool = False
    cancelado: bool = False
    motivo_cancelamento: str | None = None
    consumido: bool = False
    metadados: Mapping[str, Any] = field(default_factory=dict)
    causa_sequencia: int | None = None

    def __post_init__(self):
        self.nome = str(self.nome)
        self.args = tuple(self.args)
```

```
        self.kwargs = dict(self.kwargs)
        self.origem = None if self.origem is None else str(self.origem)
        self.metadados = dict(self.metadados)

    def __getattr__(self, nome: str) -> Any:
        payload = self.__dict__.get("kwargs", {})
        if nome in payload:
            return payload[nome]
        raise AttributeError(nome)

    def cancelar(self, motivo: str | None = None) -> None:
        if not self.cancelavel:
            raise ValueError("este evento não é cancelável")
        self.cancelado = True
        self.motivo_cancelamento = None if motivo is None else str(motivo)

    def consumir(self) -> None:
        self.consumido = True

    def estado_inspecao(self) -> dict[str, Any]:
        """Estado observável do evento sem executar ouvintes nem alterar a fila."""
        return {
            "nome": self.nome,
            "args": self.args,
            "kwargs": dict(self.kwargs),
            "origem": self.origem,
            "sequencia": self.sequencia,
            "instante": self.instante,
            "cancelavel": self.cancelavel,
            "cancelado": self.cancelado,
            "motivo_cancelamento": self.motivo_cancelamento,
            "consumido": self.consumido,
            "metadados": dict(self.metadados),
            "causa_sequencia": self.causa_sequencia,
        }


@dataclass
class _InscricaoEvento:
    funcao: Callable[..., Any]
    prioridade: int
    ordem: int
    receber_evento: bool = False
    max_execucoes: int | None = None
    cooldown: float = 0.0
    execucoes: int = 0
    ultimo_disparo: float | None = None


_PRIORIDADES_EVENTO = {
    "baixa": 0,
    "normal": 100,
    "alta": 200,
    "critica": 300,
    "crítica": 300,
}


def _normalizar_prioridade_evento(prioridade: str | int) -> int:
    if isinstance(prioridade, bool):
        raise ValueError("prioridade de evento desconhecida: use baixa, normal, alta ou
```

```
          crítica")
    if isinstance(prioridade, int):
        return prioridade
    nome = str(prioridade).strip().casefold()
    if nome not in _PRIORIDADES_EVENTO:
        raise ValueError(
            f"prioridade de evento desconhecida: {prioridade!r}; use baixa, normal, alta
               ou crítica"
        )
    return _PRIORIDADES_EVENTO[nome]


class Eventos:
    """Barramento local compatível com ``coral.regras.Eventos``.

    ``emitir`` mantém a chamada histórica de ouvintes via ``*args`` e
    ``**kwargs``. Inscrições novas podem receber o objeto :class:`Evento`
    completo e declarar prioridade sem alterar consumidores antigos.
    """
    def __init__(
        self,
        *,
        pai: "Eventos | None" = None,
        relogio: FonteTempo | None = None,
        fonte_instante: Callable[[], float] | None = None,
    ):
        self._ouvintes: dict[str, list[_InscricaoEvento]] = {}
        self._fila: deque[Evento] = deque()
        self._pai = pai
        self._relogio = relogio
        self._fonte_instante = fonte_instante
        self._sequencia = 0
        self._ordem_inscricao = 0
        self._pilha_processamento: list[Evento] = []

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "pendentes": len(self._fila),
            "fila": tuple(self._fila),
            "tipos_ouvintes": tuple(sorted(self._ouvintes)),
            "quantidade_ouvintes": sum(len(itens) for itens in self._ouvintes.values()),
            "ouvintes_por_tipo": {
                nome: len(itens)
                for nome, itens in sorted(self._ouvintes.items())
            },
            "sequencia": self._sequencia,
            "possui_pai": self._pai is not None,
            "possui_fonte_instante": self.possui_fonte_instante,
        }

    @property
    def pendentes(self) -> int:
        return len(self._fila)

    @property
    def pai(self) -> "Eventos | None":
        return self._pai

    @property
    def relogio(self) -> FonteTempo | None:
        return self._relogio
```

```
    def definir_fonte_instante(self, fonte: Callable[[], float] | None) -> None:
        self._fonte_instante = fonte

    def _instante_atual(self) -> float | None:
        if self._relogio is not None:
            return float(self._relogio.agora())
        if self._fonte_instante is not None:
            return float(self._fonte_instante())
        return None

    @property
    def possui_fonte_instante(self) -> bool:
        return self._relogio is not None or self._fonte_instante is not None

    def definir_pai(self, pai: "Eventos | None") -> None:
        if pai is self:
            raise ValueError("um barramento de eventos não pode propagar para si mesmo")
        atual = pai
        while atual is not None:
            if atual is self:
                raise ValueError("ciclo de propagação de eventos não permitido")
            atual = atual.pai
        self._pai = pai

    def quando(
        self,
        nome: str,
        funcao: Callable[..., Any],
        *,
        prioridade: str | int = "normal",
        receber_evento: bool = False,
        max_execucoes: int | None = None,
        cooldown: float = 0.0,
    ):
        valor_prioridade = _normalizar_prioridade_evento(prioridade)
        if max_execucoes is not None:
            if isinstance(max_execucoes, bool) or not isinstance(max_execucoes, int) or
              max_execucoes <= 0:
                raise ValueError("máximo de execuções de evento deve ser um inteiro
                  positivo ou nulo")
        try:
            valor_cooldown = float(validar_duracao(cooldown, "cooldown de evento"))
        except ValueError as exc:
            raise ValueError("cooldown de evento deve ser um número finito maior ou
              igual a zero") from exc
        if valor_cooldown > 0 and self._relogio is None and self._fonte_instante is
          None:
            raise ValueError("cooldown de evento exige uma fonte de tempo")
        self._ordem_inscricao += 1
        inscricao = _InscricaoEvento(
            funcao=funcao,
            prioridade=valor_prioridade,
            ordem=self._ordem_inscricao,
            receber_evento=bool(receber_evento),
            max_execucoes=max_execucoes,
            cooldown=valor_cooldown,
        )
        self._ouvintes.setdefault(str(nome), []).append(inscricao)
        return funcao
```

```
    def remover(self, nome: str, funcao: Callable[..., Any]) -> bool:
        ouvintes = self._ouvintes.get(str(nome))
        if not ouvintes:
            return False
        for indice, inscricao in enumerate(ouvintes):
            if inscricao.funcao is funcao or inscricao.funcao == funcao:
                del ouvintes[indice]
                if not ouvintes:
                    self._ouvintes.pop(str(nome), None)
                return True
        return False

    def _novo_evento(
        self,
        nome: str,
        args: tuple[Any, ...],
        kwargs: Mapping[str, Any],
        origem: str | None,
        *,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
    ) -> Evento:
        self._sequencia += 1
        instante = self._instante_atual()
        return Evento(
            str(nome),
            tuple(args),
            dict(kwargs),
            None if origem is None else str(origem),
            self._sequencia,
            instante=instante,
            cancelavel=bool(cancelavel),
            metadados={} if metadados is None else dict(metadados),
            causa_sequencia=(self._pilha_processamento[-1].sequencia if
              self._pilha_processamento else None),
        )

    def _snapshot_ouvintes(self, nome: str) -> tuple[_InscricaoEvento, ...]:
        ouvintes = tuple(self._ouvintes.get(nome, ()))
        return tuple(sorted(ouvintes, key=lambda item: (-item.prioridade, item.ordem)))

    def _emitir_local(self, evento: Evento) -> int:
        ouvintes = self._snapshot_ouvintes(evento.nome)
        for inscricao in ouvintes:
            if inscricao.max_execucoes is not None and inscricao.execucoes >=
              inscricao.max_execucoes:
                continue
            if inscricao.cooldown > 0 and inscricao.ultimo_disparo is not None:
                if evento.instante is None:
                    raise ValueError("cooldown de evento exige instante lógico")
                decorrido = float(evento.instante) - inscricao.ultimo_disparo
                if decorrido < inscricao.cooldown and not math.isclose(
                    decorrido, inscricao.cooldown, rel_tol=1e-12, abs_tol=1e-15
                ):
                    continue
            if inscricao.receber_evento:
                inscricao.funcao(evento)
            else:
                inscricao.funcao(*evento.args, **dict(evento.kwargs))
            inscricao.execucoes += 1
            if evento.instante is not None:
```

```
                inscricao.ultimo_disparo = float(evento.instante)
        return len(ouvintes)

    def emitir(self, nome: str, *args, **kwargs) -> int:
        evento = self._novo_evento(nome, args, kwargs, None)
        return self._emitir_local(evento)

    def emitir_evento(
        self,
        nome: str,
        *args,
        origem: str | None = None,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
        **kwargs,
    ) -> Evento:
        evento = self._novo_evento(
            nome,
            args,
            kwargs,
            origem,
            cancelavel=cancelavel,
            metadados=metadados,
        )
        self._emitir_local(evento)
        return evento

    def enfileirar(self, nome: str, *args, origem: str | None = None, **kwargs) ->
      Evento:
        evento = self._novo_evento(nome, args, kwargs, origem)
        self._fila.append(evento)
        return evento

    def enfileirar_evento(
        self,
        nome: str,
        *args,
        origem: str | None = None,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
        **kwargs,
    ) -> Evento:
        evento = self._novo_evento(
            nome,
            args,
            kwargs,
            origem,
            cancelavel=cancelavel,
            metadados=metadados,
        )
        self._fila.append(evento)
        return evento

    def processar(self, limite: int | None = None) -> tuple[Evento, ...]:
        if limite is not None and (isinstance(limite, bool) or not isinstance(limite,
          int) or limite < 0):
            raise ValueError("limite de eventos deve ser inteiro não negativo ou nulo")
        processados: list[Evento] = []
        maximo = len(self._fila) if limite is None else min(limite, len(self._fila))
        for _ in range(maximo):
            evento = self._fila.popleft()
```

```
            self._pilha_processamento.append(evento)
            try:
                self._emitir_local(evento)
            finally:
                self._pilha_processamento.pop()
            processados.append(evento)
        return tuple(processados)

    def _emitir_propagado_objeto(self, evento: Evento) -> int:
        total = self._emitir_local(evento)
        if self._pai is not None and not evento.consumido:
            total += self._pai._emitir_propagado_objeto(evento)
        return total

    def emitir_propagado(self, nome: str, *args, origem: str | None = None, **kwargs) ->
       int:
        evento = self._novo_evento(nome, args, kwargs, origem)
        return self._emitir_propagado_objeto(evento)

    def emitir_evento_propagado(
        self,
        nome: str,
        *args,
        origem: str | None = None,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
        **kwargs,
    ) -> Evento:
        evento = self._novo_evento(
            nome,
            args,
            kwargs,
            origem,
            cancelavel=cancelavel,
            metadados=metadados,
        )
        self._emitir_propagado_objeto(evento)
        return evento


__all__ = [
    "CONTRATO", "validar_duracao", "normalizar_dt", "FonteTempo",
    "RelogioMonotonico", "RelogioSimulado", "OperacaoCancelada",
    "TokenCancelamento", "Evento", "Eventos",
]
```

Biblioteca `coral.matematica`

`absoluto(...)` · **módulo** `coral.matematica`

O que faz

Obtém a magnitude sem o sinal.

Assinatura

Assinatura para consulta

```
absoluto(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número ou objeto com __abs__	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Número.

Exemplo

Código Coral

```
de coral.matematica importe absoluto
mostre absoluto(-3)
```

Resultado

Saída esperada

```
3
```

Erros

Entradas fora do domínio da operação produzem erro. Média não aceita ausência de observações; raiz real não aceita número negativo.

`aproximadamente_igual(...)` · módulo `coral.matematica`

O que faz

Compara dois números usando tolerâncias relativa e absoluta.

Assinatura

Assinatura para consulta

```
aproximadamente_igual(a, b, tolerancia_relativa=1e-09, tolerancia_absoluta=0.0)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	real, exceto booleano	sim	sem padrão	Primeiro número comparado.
b	real, exceto booleano	sim	sem padrão	Segundo número comparado.
tolerancia_relativa	real finito não negativo	não	1e-09	Tolerância proporcional à maior magnitude comparada.
tolerancia_absoluta	real finito não negativo	não	0.0	Tolerância mínima absoluta, útil perto de zero.

Retorno

Booleano.

Exemplo

Código Coral

```
de coral.matematica importe aproximadamente_igual
mostre aproximadamente_igual(0.1 + 0.2, 0.3)
```

Resultado

Saída esperada

```
True
```

Erros

TypeError para tipo incompatível; ValueError para tolerância negativa, não finita ou conversão fora do intervalo suportado.

Atenção

NaN nunca é aproximadamente igual. Infinitos são iguais somente ao mesmo infinito. O padrão relativo é 1e-9 e o absoluto é 0.0.

`arco_cosseno(...)` · módulo `coral.matematica`

O que faz

Calcula o arco cosseno.

Assinatura

Assinatura para consulta

```
arco_cosseno(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real finito, exceto booleano	sim	sem padrão	Número finito sobre o qual a operação atua.

Retorno

Ângulo decimal em radianos.

Exemplo

Código Coral

```
de coral.matematica importe arco_cosseno
mostre arco_cosseno(1)
```

Resultado

Saída esperada

```
0.0
```

Erros

TypeError para tipo incompatível ou booleano; ValueError para valor não finito ou fora de -1 a 1.

Atenção

Usa aproximação em ponto flutuante.

`arco_seno(...)` · módulo `coral.matematica`

O que faz

Calcula o arco seno.

Assinatura

Assinatura para consulta

```
arco_seno(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real finito, exceto booleano	sim	sem padrão	Número finito sobre o qual a operação atua.

Retorno

Ângulo decimal em radianos.

Exemplo

Código Coral

```
de coral.matematica importe arco_seno
mostre arco_seno(0)
```

Resultado

Saída esperada

```
0.0
```

Erros

TypeError para tipo incompatível ou booleano; ValueError para valor não finito ou fora de -1 a 1.

Atenção

Usa aproximação em ponto flutuante.

`arco_tangente(...)` · módulo `coral.matematica`

O que faz

Calcula o arco tangente.

Assinatura

Assinatura para consulta

```
arco_tangente(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real finito, exceto booleano	sim	sem padrão	Número finito sobre o qual a operação atua.

Retorno

Ângulo decimal em radianos.

Exemplo

Código Coral

```
de coral.matematica importe arco_tangente
mostre arco_tangente(0)
```

Resultado

Saída esperada

```
0.0
```

Erros

TypeError para tipo incompatível ou booleano; ValueError para valor não finito.

Atenção

Usa aproximação em ponto flutuante.

`arranjos(...)` · módulo `coral.matematica`

O que faz

Conta escolhas ordenadas de k elementos distintos entre n, sem repetição.

Assinatura

Assinatura para consulta

```
arranjos(n, k)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
n	inteiro, exceto booleano	sim	sem padrão	Quantidade inteira não negativa.
k	inteiro, exceto booleano	sim	sem padrão	Quantidade escolhida, entre zero e n.

Retorno

Inteiro não negativo.

Exemplo

Código Coral

```
de coral.matematica importe arranjos
mostre arranjos(5, 2)
```

Resultado

Saída esperada

```
20
```

Erros

TypeError para tipos não inteiros ou booleanos; ValueError se n é negativo ou k está fora de 0 <= k <= n.

`arredondar(...)` · módulo `coral.matematica`

O que faz

Arredonda para a quantidade de casas decimais informada.

Assinatura

Assinatura para consulta

```
arredondar(valor, casas=0)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real	sim	sem padrão	Número sobre o qual a operação atua.
casas	inteiro	não	0	Casas decimais; valores negativos arredondam dezenas, centenas e assim por diante.

Retorno

Número arredondado; com casas explícitas, o tipo segue round do Python.

Exemplo

Código Coral

```
de coral.matematica importe arredondar
mostre arredondar(2.675, 2)
```

Resultado

Saída esperada

```
2.67
```

Erros

TypeError para tipos incompatíveis.

Atenção

Usa round do Python: empates vão ao par. A representação binária pode produzir resultados como 2.675 arredondado para 2.67.

`combinacoes(...)` · módulo `coral.matematica`

O que faz

Conta escolhas de k elementos distintos entre n, sem considerar a ordem.

Assinatura

Assinatura para consulta

```
combinacoes(n, k)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
n	inteiro, exceto booleano	sim	sem padrão	Quantidade inteira não negativa.
k	inteiro, exceto booleano	sim	sem padrão	Quantidade escolhida, entre zero e n.

Retorno

Inteiro não negativo.

Exemplo

Código Coral

```
de coral.matematica importe combinacoes
mostre combinacoes(5, 2)
```

Resultado

Saída esperada

```
10
```

Erros

TypeError para tipos não inteiros ou booleanos; ValueError se n é negativo ou k está fora de 0 <= k <= n.

`cosseno(...)` · módulo `coral.matematica`

O que faz

Calcula o cosseno de um ângulo em radianos.

Assinatura

Assinatura para consulta

```
cosseno(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real	sim	sem padrão	Ângulo em radianos.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.matematica importe cosseno
mostre cosseno(0)
```

Resultado

Saída esperada

```
1.0
```

Erros

TypeError para tipo incompatível; ValueError para infinito.

Atenção

Delega a math do Python. Resultados trigonométricos têm precisão limitada de ponto flutuante.

`exp(...)` · módulo `coral.matematica`

O que faz

Calcula e elevado ao valor.

Assinatura

Assinatura para consulta

```
exp(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real	sim	sem padrão	Número sobre o qual a operação atua.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.matematica importe exp
mostre exp(0)
```

Resultado

Saída esperada

```
1.0
```

Erros

TypeError para tipo incompatível; OverflowError quando o resultado excede o intervalo de ponto flutuante.

`fatorial(...)` · módulo `coral.matematica`

O que faz

Calcula o produto dos inteiros de 1 até n.

Assinatura

Assinatura para consulta

```
fatorial(n)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
n	inteiro, exceto booleano	sim	sem padrão	Quantidade inteira não negativa.

Retorno

Inteiro; fatorial(0) é 1.

Exemplo

Código Coral

```
de coral.matematica importe fatorial
mostre fatorial(5)
```

Resultado

Saída esperada

```
120
```

Erros

TypeError se n não é inteiro ou é booleano; ValueError se n é negativo.

`finito(...)` · módulo `coral.matematica`

O que faz

Verifica se um número não é NaN nem infinito.

Assinatura

Assinatura para consulta

```
finito(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real	sim	sem padrão	Número sobre o qual a operação atua.

Retorno

Booleano.

Exemplo

Código Coral

```
de coral.matematica importe finito
mostre finito(42)
```

Resultado

Saída esperada

```
True
```

Erros

TypeError para tipo incompatível; OverflowError para inteiro grande demais na conversão interna.

`graus(...)` · módulo `coral.matematica`

O que faz

Converte radianos para graus.

Assinatura

Assinatura para consulta

```
graus(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real	sim	sem padrão	Ângulo em radianos.

Retorno

Número decimal em graus.

Exemplo

Código Coral

```
de coral.matematica importe graus
mostre graus(0)
```

Resultado

Saída esperada

```
0.0
```

Erros

TypeError para tipo incompatível.

`hipotenusa(...)` · módulo `coral.matematica`

O que faz

Calcula a distância euclidiana de dois componentes com math.hypot.

Assinatura

Assinatura para consulta

```
hipotenusa(a, b)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	real finito, exceto booleano	sim	sem padrão	Primeiro componente.
b	real finito, exceto booleano	sim	sem padrão	Segundo componente.

Retorno

Número decimal não negativo.

Exemplo

Código Coral

```
de coral.matematica importe hipotenusa
mostre hipotenusa(3, 4)
```

Resultado

Saída esperada

```
5.0
```

Erros

TypeError para tipo incompatível; ValueError para entrada não finita ou fora do intervalo numérico suportado.

`interpolar_linear(...)` · módulo `coral.matematica`

O que faz

Interpola ou extrapola entre dois números finitos.

Assinatura

Assinatura para consulta

```
interpolar_linear(inicio, fim, fracao)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
inicio	real finito, exceto booleano	sim	sem padrão	Valor na fração zero.
fim	real finito, exceto booleano	sim	sem padrão	Valor na fração um.
fracao	real finito, exceto booleano	sim	sem padrão	Posição relativa; fora de zero a um, extrapola.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.matematica importe interpolar_linear
mostre interpolar_linear(10, 20, 0.25)
```

Resultado

Saída esperada

```
12.5
```

Erros

TypeError para tipos incompatíveis; ValueError para entradas não finitas ou fora do intervalo suportado.

Atenção

Não limita a fração. Frações zero e um retornam exatamente os extremos convertidos para decimal; outras operações podem transbordar.

`limitar(...)` · módulo `coral.matematica`

O que faz

Restringe um valor ao intervalo fechado informado.

Assinatura

Assinatura para consulta

```
limitar(valor, minimo, maximo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	valor comparável	sim	sem padrão	Valor a limitar.
minimo	valor comparável	sim	sem padrão	Extremo inferior incluído.
maximo	valor comparável	sim	sem padrão	Extremo superior incluído.

Retorno

Valor limitado, com o tipo do argumento ou extremo selecionado.

Exemplo

Código Coral

```
de coral.matematica importe limitar
mostre limitar(12, 0, 10)
```

Resultado

Saída esperada

```
10
```

Erros

ValueError se mínimo é maior que máximo; TypeError se os valores não podem ser comparados.

Atenção

Não converte os argumentos para decimal nem rejeita automaticamente NaN.

`log(...)` · módulo `coral.matematica`

O que faz

Calcula o logaritmo natural ou na base informada.

Assinatura

Assinatura para consulta

```
log(valor, base=None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número positivo	sim	sem padrão	Argumento do logaritmo.
base	número positivo diferente de 1 ou nulo	não	None	Base do logaritmo; nulo seleciona o logaritmo natural.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.matematica importe log
mostre log(8, 2)
```

Resultado

Saída esperada

```
3.0
```

Erros

ValueError para domínio inválido; ZeroDivisionError para base 1; TypeError para tipos incompatíveis.

`log10(...)` · módulo `coral.matematica`

O que faz

Calcula o logaritmo na base 10.

Assinatura

Assinatura para consulta

```
log10(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	real finito positivo, exceto booleano	sim	sem padrão	Argumento do logaritmo.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.matematica importe log10
mostre log10(100)
```

Resultado

Saída esperada

```
2.0
```

Erros

TypeError para tipo incompatível ou booleano; ValueError para valor não positivo, não finito ou conversão fora do intervalo suportado.

`log2(...)` · módulo `coral.matematica`

O que faz

Calcula o logaritmo na base 2.

Assinatura

Assinatura para consulta

```
log2(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	real finito positivo, exceto booleano	sim	sem padrão	Argumento do logaritmo.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.matematica importe log2
mostre log2(8)
```

Resultado

Saída esperada

```
3.0
```

Erros

TypeError para tipo incompatível ou booleano; ValueError para valor não positivo, não finito ou conversão fora do intervalo suportado.

`maximo(...)` · módulo `coral.matematica`

O que faz

Obtém o maior item.

Assinatura

Assinatura para consulta

```
maximo(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de valores compatíveis	sim	sem padrão	Valores consumidos pela operação.

Retorno

Item selecionado, com seu tipo original.

Exemplo

Código Coral

```
de coral.matematica importe maximo
mostre maximo([3, 1, 2])
```

Resultado

Saída esperada

```
3
```

Erros

ValueError para coleção vazia; TypeError se os itens não podem ser comparados.

`mdc(...)` · módulo `coral.matematica`

O que faz

Calcula o máximo divisor comum dos argumentos inteiros.

Assinatura

Assinatura para consulta

```
mdc(*valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	argumentos inteiros variáveis	não	sem padrão	Números inteiros usados no cálculo; esta função recebe argumentos separados, não uma lista única.

Retorno

Inteiro não negativo; sem argumentos, zero.

Exemplo

Código Coral

```
de coral.matematica importe mdc
mostre mdc(12, 18)
```

Resultado

Saída esperada

```
6
```

Erros

TypeError para argumento que não suporta interpretação inteira.

Atenção

Delega a math do Python e aceita booleanos como inteiros, diferentemente de fatorial e combinacoes.

`media(...)` · módulo `coral.matematica`

O que faz

Calcula a média aritmética de observações.

Assinatura

Assinatura para consulta

```
media(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável não vazio de valores numéricos	sim	sem padrão	Coleção de entrada.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.matematica importe media
mostre media([2, 4, 6])
```

Resultado

Saída esperada

```
4.0
```

Erros

Entradas fora do domínio da operação produzem erro. Média não aceita ausência de observações; raiz real não aceita número negativo.

`minimo(...)` · módulo `coral.matematica`

O que faz

Obtém o menor item.

Assinatura

Assinatura para consulta

```
minimo(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de valores compatíveis	sim	sem padrão	Valores consumidos pela operação.

Retorno

Item selecionado, com seu tipo original.

Exemplo

Código Coral

```
de coral.matematica importe minimo
mostre minimo([3, 1, 2])
```

Resultado

Saída esperada

```
1
```

Erros

ValueError para coleção vazia; TypeError se os itens não podem ser comparados.

`mmc(...)` · módulo `coral.matematica`

O que faz

Calcula o mínimo múltiplo comum dos argumentos inteiros.

Assinatura

Assinatura para consulta

```
mmc(*valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	argumentos inteiros variáveis	não	sem padrão	Números inteiros usados no cálculo; esta função recebe argumentos separados, não uma lista única.

Retorno

Inteiro não negativo; sem argumentos, um. Se há um zero, retorna zero.

Exemplo

Código Coral

```
de coral.matematica importe mmc
mostre mmc(4, 6)
```

Resultado

Saída esperada

```
12
```

Erros

TypeError para argumento que não suporta interpretação inteira.

Atenção

Delega a math do Python e aceita booleanos como inteiros, diferentemente de fatorial e combinacoes.

`piso(...)` · módulo `coral.matematica`

O que faz

Obtém o maior inteiro menor ou igual ao valor.

Assinatura

Assinatura para consulta

```
piso(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real	sim	sem padrão	Número sobre o qual a operação atua.

Retorno

Inteiro.

Exemplo

Código Coral

```
de coral.matematica importe piso
mostre piso(-1.2)
```

Resultado

Saída esperada

```
-2
```

Erros

TypeError para tipo incompatível; ValueError para NaN; OverflowError para infinito.

`potencia(...)` · módulo `coral.matematica`

O que faz

Eleva a base ao expoente.

Assinatura

Assinatura para consulta

```
potencia(base, expoente)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
base	valor compatível com potência	sim	sem padrão	Base da potência.
expoente	expoente compatível com a base	sim	sem padrão	Expoente aplicado à base.

Retorno

Número produzido pela potência; o tipo depende dos operandos.

Exemplo

Código Coral

```
de coral.matematica importe potencia
mostre potencia(2, 3)
```

Resultado

Saída esperada

```
8
```

Erros

Entradas fora do domínio da operação produzem erro. Média não aceita ausência de observações; raiz real não aceita número negativo.

`produto(...)` · módulo `coral.matematica`

O que faz

Multiplica os valores de uma coleção.

Assinatura

Assinatura para consulta

```
produto(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de valores compatíveis	sim	sem padrão	Valores consumidos pela operação.

Retorno

Produto numérico; coleção vazia produz inteiro 1.

Exemplo

Código Coral

```
de coral.matematica importe produto
mostre produto([2, 3, 4])
```

Resultado

Saída esperada

```
24
```

Erros

TypeError para itens cuja multiplicação não é suportada.

`radianos(...)` · módulo `coral.matematica`

O que faz

Converte graus para radianos.

Assinatura

Assinatura para consulta

```
radianos(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real	sim	sem padrão	Ângulo em graus.

Retorno

Número decimal em radianos.

Exemplo

Código Coral

```
de coral.matematica importe radianos
mostre radianos(0)
```

Resultado

Saída esperada

```
0.0
```

Erros

TypeError para tipo incompatível.

`raiz(...)` · módulo `coral.matematica`

O que faz

Calcula a raiz quadrada real.

Assinatura

Assinatura para consulta

```
raiz(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real não negativo	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.matematica importe raiz
mostre raiz(9)
```

Resultado

Saída esperada

```
3.0
```

Erros

Entradas fora do domínio da operação produzem erro. Média não aceita ausência de observações; raiz real não aceita número negativo.

`raiz_nesima(...)` · módulo `coral.matematica`

O que faz

Calcula a raiz real de índice inteiro positivo.

Assinatura

Assinatura para consulta

```
raiz_nesima(valor, indice)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real finito, exceto booleano	sim	sem padrão	Número finito sobre o qual a operação atua.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
indice	inteiro positivo, exceto booleano	sim	sem padrão	Índice da raiz; valor negativo exige índice ímpar.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.matematica importe raiz_nesima
mostre raiz_nesima(-8, 3)
```

Resultado

Saída esperada

```
-2.0
```

Erros

TypeError para tipo incompatível ou booleano; ValueError para índice não positivo, número não finito ou valor negativo com índice par.

Atenção

Usa potência de ponto flutuante e pode ter erro de arredondamento.

`seno(...)` · módulo `coral.matematica`

O que faz

Calcula o seno de um ângulo em radianos.

Assinatura

Assinatura para consulta

```
seno(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real	sim	sem padrão	Ângulo em radianos.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.matematica importe seno
mostre seno(0)
```

Resultado

Saída esperada

```
0.0
```

Erros

TypeError para tipo incompatível; ValueError para infinito.

Atenção

Delega a math do Python. Resultados trigonométricos têm precisão limitada de ponto flutuante.

`sinal(...)` · módulo `coral.matematica`

O que faz

Obtém o sinal de um real finito.

Assinatura

Assinatura para consulta

```
sinal(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real finito, exceto booleano	sim	sem padrão	Número finito sobre o qual a operação atua.

Retorno

Inteiro -1, 0 ou 1.

Exemplo

Código Coral

```
de coral.matematica importe sinal
mostre sinal(-2)
```

Resultado

Saída esperada

```
-1
```

Erros

TypeError para tipo incompatível ou booleano; ValueError para número não finito ou fora do intervalo suportado.

`soma(...)` · módulo `coral.matematica`

O que faz

Soma os valores da coleção.

Assinatura

Assinatura para consulta

```
soma(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de valores somáveis	sim	sem padrão	Coleção de entrada.

Retorno

Número.

Exemplo

Código Coral

```
de coral.matematica importe soma
mostre soma([2, 4, 6])
```

Resultado

Saída esperada

```
12
```

Erros

Entradas fora do domínio da operação produzem erro. Média não aceita ausência de observações; raiz real não aceita número negativo.

`tangente(...)` · módulo `coral.matematica`

O que faz

Calcula a tangente de um ângulo em radianos.

Assinatura

Assinatura para consulta

```
tangente(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real	sim	sem padrão	Ângulo em radianos.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.matematica importe tangente
mostre tangente(0)
```

Resultado

Saída esperada

```
0.0
```

Erros

TypeError para tipo incompatível; ValueError para infinito.

Atenção

Delega a math do Python. Resultados trigonométricos têm precisão limitada de ponto flutuante.

`teto(...)` · módulo `coral.matematica`

O que faz

Obtém o menor inteiro maior ou igual ao valor.

Assinatura

Assinatura para consulta

```
teto(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real	sim	sem padrão	Número sobre o qual a operação atua.

Retorno

Inteiro.

Exemplo

Código Coral

```
de coral.matematica importe teto
mostre teto(-1.2)
```

Resultado

Saída esperada

```
-1
```

Erros

TypeError para tipo incompatível; ValueError para NaN; OverflowError para infinito.

`truncar(...)` · módulo `coral.matematica`

O que faz

Descarta a parte fracionária em direção a zero.

Assinatura

Assinatura para consulta

```
truncar(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real	sim	sem padrão	Número sobre o qual a operação atua.

Retorno

Inteiro.

Exemplo

Código Coral

```
de coral.matematica importe truncar
mostre truncar(-1.9)
```

Resultado

Saída esperada

```
-1
```

Erros

TypeError para tipo incompatível; ValueError para NaN; OverflowError para infinito.

`volume_bola3(...)` · módulo `coral.matematica`

O que faz

Calcula o volume de uma bola euclidiana em três dimensões.

Assinatura

Assinatura para consulta

```
volume_bola3(raio)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
raio	real finito não negativo, exceto booleano	sim	sem padrão	Raio da bola.

Retorno

Número decimal, na unidade cúbica do raio.

Exemplo

Código Coral

```
de coral.matematica importe volume_bola3
mostre volume_bola3(0)
```

Resultado

Saída esperada

```
0.0
```

Erros

TypeError para tipo incompatível ou booleano; ValueError para raio negativo ou não finito; OverflowError se a potência excede o intervalo suportado.

Consultar a implementação canônica de coral.matematica

Implementação canônica de `coral.matematica`

Código Python

```
"""Superfície pública de coral.matematica."""
from .stdlib.matematica import *  # noqa: F401,F403

from __future__ import annotations

import math
from numbers import Integral, Real


def _inteiro_exato(valor, nome: str) -> int:
    if isinstance(valor, bool) or not isinstance(valor, Integral):
        raise TypeError(f"{nome} deve ser um inteiro")
    return int(valor)


def _real(valor, nome: str) -> float:
    if isinstance(valor, bool) or not isinstance(valor, Real):
        raise TypeError(f"{nome} deve ser um número real")
    try:
        return float(valor)
    except OverflowError as exc:
        raise ValueError(f"{nome} está fora do intervalo numérico suportado") from exc



def _real_finito(valor, nome: str) -> float:
    numero = _real(valor, nome)
    if not math.isfinite(numero):
        raise ValueError(f"{nome} deve ser finito")
    return numero


def raiz(valor): return math.sqrt(valor)
def potencia(base, expoente): return base ** expoente
def absoluto(valor): return abs(valor)
def arredondar(valor, casas=0): return round(valor, casas)
def minimo(valores): return min(valores)
def maximo(valores): return max(valores)
def soma(valores): return sum(valores)
def media(valores):
    valores=list(valores)
    if not valores: raise ValueError("não é possível calcular média de uma coleção
      vazia")
    return sum(valores)/len(valores)


def fatorial(n):
    """Fatorial de um inteiro não negativo."""
    n = _inteiro_exato(n, "n")
    if n < 0:
        raise ValueError("n deve ser maior ou igual a zero")
```

```
    return math.factorial(n)


def combinacoes(n, k):
    """Quantidade de escolhas de k elementos sem considerar ordem."""
    n = _inteiro_exato(n, "n")
    k = _inteiro_exato(k, "k")
    if n < 0:
        raise ValueError("n deve ser maior ou igual a zero")
    if k < 0 or k > n:
        raise ValueError("k deve satisfazer 0 <= k <= n")
    return math.comb(n, k)


def arranjos(n, k):
    """Quantidade de escolhas ordenadas de k elementos sem repetição."""
    n = _inteiro_exato(n, "n")
    k = _inteiro_exato(k, "k")
    if n < 0:
        raise ValueError("n deve ser maior ou igual a zero")
    if k < 0 or k > n:
        raise ValueError("k deve satisfazer 0 <= k <= n")
    return math.perm(n, k)


def hipotenusa(a, b):
    """Comprimento euclidiano dos catetos a e b, calculado de forma estável."""
    return math.hypot(_real_finito(a, "a"), _real_finito(b, "b"))


def aproximadamente_igual(a, b, tolerancia_relativa=1e-9, tolerancia_absoluta=0.0):
    """Compara números reais com tolerâncias explícitas."""
    a = _real(a, "a")
    b = _real(b, "b")
    relativa = _real_finito(tolerancia_relativa, "tolerancia_relativa")
    absoluta = _real_finito(tolerancia_absoluta, "tolerancia_absoluta")
    if relativa < 0 or absoluta < 0:
        raise ValueError("as tolerâncias não podem ser negativas")
    if math.isnan(a) or math.isnan(b):
        return False
    return math.isclose(a, b, rel_tol=relativa, abs_tol=absoluta)


def interpolar_linear(inicio, fim, fracao):
    """Interpola ou extrapola linearmente entre dois valores reais finitos."""
    inicio = _real_finito(inicio, "inicio")
    fim = _real_finito(fim, "fim")
    fracao = _real_finito(fracao, "fracao")
    if fracao == 0.0:
        return inicio
    if fracao == 1.0:
        return fim
    if 0.0 < fracao < 1.0:
        return inicio * (1.0 - fracao) + fim * fracao
    return inicio + (fim - inicio) * fracao


def seno(valor): return math.sin(valor)
def cosseno(valor): return math.cos(valor)
def tangente(valor): return math.tan(valor)
def arco_seno(valor):
```

```
    valor = _real_finito(valor, "valor")
    if valor < -1 or valor > 1:
        raise ValueError("valor deve estar entre -1 e 1 para arco_seno")
    return math.asin(valor)

def arco_cosseno(valor):
    valor = _real_finito(valor, "valor")
    if valor < -1 or valor > 1:
        raise ValueError("valor deve estar entre -1 e 1 para arco_cosseno")
    return math.acos(valor)

def arco_tangente(valor): return math.atan(_real_finito(valor, "valor"))
def piso(valor): return math.floor(valor)
def teto(valor): return math.ceil(valor)
def truncar(valor): return math.trunc(valor)
def log(valor, base=None): return math.log(valor) if base is None else math.log(valor,
  base)
def log10(valor):
    valor = _real_finito(valor, "valor")
    if valor <= 0:
        raise ValueError("valor deve ser positivo para log10")
    return math.log10(valor)
def log2(valor):
    valor = _real_finito(valor, "valor")
    if valor <= 0:
        raise ValueError("valor deve ser positivo para log2")
    return math.log2(valor)
def exp(valor): return math.exp(valor)
def graus(valor): return math.degrees(valor)
def radianos(valor): return math.radians(valor)
def finito(valor): return math.isfinite(valor)
def mdc(*valores): return math.gcd(*valores)
def mmc(*valores): return math.lcm(*valores)


def raiz_nesima(valor, indice):
    """Raiz real de índice inteiro positivo."""
    valor = _real_finito(valor, "valor")
    indice = _inteiro_exato(indice, "indice")
    if indice <= 0:
        raise ValueError("indice deve ser um inteiro positivo")
    if valor == 0:
        return 0.0
    if valor < 0:
        if indice % 2 == 0:
            raise ValueError("valor negativo exige indice ímpar para raiz real")
        return -math.pow(-valor, 1.0 / indice)
    return math.pow(valor, 1.0 / indice)


def produto(valores):
    """Multiplica os itens de uma coleção; a coleção vazia tem produto 1."""
    return math.prod(valores)


def sinal(valor):
    """Devolve -1, 0 ou 1 para um número real finito."""
    valor = _real_finito(valor, "valor")
    if valor > 0:
        return 1
    if valor < 0:
```

```
        return -1
    return 0


def volume_bola3(raio):
    """Volume de uma bola euclidiana tridimensional de raio não negativo."""
    raio = _real_finito(raio, "raio")
    if raio < 0:
        raise ValueError("raio não pode ser negativo")
    return (4.0 / 3.0) * math.pi * raio ** 3


def limitar(valor, minimo, maximo):
    if minimo > maximo:
        raise ValueError("mínimo não pode ser maior que máximo")
    return min(max(valor, minimo), maximo)


pi=math.pi
e=math.e

__all__ = [
    "raiz", "potencia", "absoluto", "arredondar", "minimo", "maximo", "soma", "media",
    "fatorial", "combinacoes", "arranjos", "hipotenusa", "aproximadamente_igual",
    "interpolar_linear", "seno", "cosseno", "tangente", "arco_seno", "arco_cosseno",
    "arco_tangente", "piso", "teto", "truncar", "log", "log10", "log2", "exp", "graus",
    "radianos", "raiz_nesima", "produto", "sinal", "volume_bola3", "limitar", "finito",
    "mdc", "mmc", "pi", "e",
]
```

Biblioteca `coral.mundo`

`coral.mundo.BlocoMapa`

Conveniência para terreno, material, obstáculo ou tile.

Assinatura

Assinatura para consulta

```
BlocoMapa(identificador: 'str', propriedades: 'Mapping[str, Any] | None' = None) ->
  'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
identificador	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
propriedades	Mapping[str, Any] | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `identificador`: `str`.

`coral.mundo.CamadaMapa`

Camada nomeada, ordenada e esparsa de um mapa.

Assinatura

Assinatura para consulta

```
CamadaMapa(nome: 'str', *, ordem: 'int' = 0, propriedades: 'Mapping[str, Any] | None' =
  None, participa_travessia: 'bool' = True, _sequencia: 'int' = 0) -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
ordem	int	KEYWORD_ONLY	não	0
propriedades	Mapping[str, Any] | None	KEYWORD_ONLY	não	None
participa_travessia	bool	KEYWORD_ONLY	não	True
_sequencia	int	KEYWORD_ONLY	não	0

Retorno declarado

None

Métodos públicos

Referência para leitura

```
conteudo(self, celula: 'CelulaMapa') -> 'ConteudoCelula | None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
possui(self, celula: 'CelulaMapa') -> 'bool'
```

Método público sem descrição própria no runtime.

`coral.mundo.CelulaHexagonal`

CelulaHexagonal(q: 'int', r: 'int')

Assinatura

Assinatura para consulta

```
CelulaHexagonal(q: 'int', r: 'int') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
q	int	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
r	int	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `q`: `int`.

Atributo `r`: `int`.

Métodos públicos

Referência para leitura

```
como_tupla(self) -> 'tuple[int, int]'
```

Método público sem descrição própria no runtime.

`coral.mundo.CelulaQuadrada`

CelulaQuadrada(coluna: 'int', linha: 'int')

Assinatura

Assinatura para consulta

```
CelulaQuadrada(coluna: 'int', linha: 'int') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
coluna	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
linha	int	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `coluna`: `int`.

Atributo `linha`: `int`.

Métodos públicos

Referência para leitura

```
como_tupla(self) -> 'tuple[int, int]'
```

Método público sem descrição própria no runtime.

`coral.mundo.CelulaVoxel`

CelulaVoxel(x: 'int', y: 'int', z: 'int')

Assinatura

Assinatura para consulta

```
CelulaVoxel(x: 'int', y: 'int', z: 'int') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
x	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
z	int	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `x`: `int`.

Atributo `y`: `int`.

Atributo `z`: `int`.

Métodos públicos

Referência para leitura

```
como_tupla(self) -> 'tuple[int, int, int]'
```

Método público sem descrição própria no runtime.

`coral.mundo.ConteudoCelula`

Valor imutável associado a uma célula, com travessia opcional.

Assinatura

Assinatura para consulta

```
ConteudoCelula(valor: 'Any', propriedades: 'Mapping[str, Any] | None' = None, *,
  travessia: 'bool | None' = None, custo: 'float | int | None' = None, motivo: 'str |
  None' = None) -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
propriedades	Mapping[str, Any] | None	POSITIONAL_OR_KEYWORD	não	None
travessia	bool | None	KEYWORD_ONLY	não	None

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
custo	float | int | None	KEYWORD_ONLY	não	None
motivo	str | None	KEYWORD_ONLY	não	None

Retorno declarado

None

Atributo `travessia_permitida`: `bool | None`.

Atributo `custo_travessia`: `float | None`.

Atributo `motivo_travessia`: `str | None`.

Métodos públicos

Referência para leitura

```
com_propriedades(self, propriedades: 'Mapping[str, Any]') -> "'ConteudoCelula'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
com_travessia(self, *, permitida: 'bool | None' = None, custo: 'float | int | None' =
  None, motivo: 'str | None' = None) -> "'ConteudoCelula'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
com_valor(self, valor: 'Any') -> "'ConteudoCelula'"
```

Método público sem descrição própria no runtime.

`coral.mundo.Entidade`

Entidade(nome: 'str', atributos: 'dict[str, Any]' = <factory>, tags: 'set[str]' = <factory>, inventario: 'list[Any]' = <factory>)

Assinatura

Assinatura para consulta

```
Entidade(nome: 'str', atributos: 'dict[str, Any]' = <factory>, tags: 'set[str]' =
  <factory>, inventario: 'list[Any]' = <factory>) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
atributos	dict[str, Any]	POSITIONAL_OR_KEYWORD	não	<factory>

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tags	set[str]	POSITIONAL_OR_KEYWORD	não	<factory>
inventario	list[Any]	POSITIONAL_OR_KEYWORD	não	<factory>

Retorno declarado

None

Atributo `nome`: `str`.

Atributo `atributos`: `dict[str, Any]`.

Atributo `tags`: `set[str]`.

Atributo `inventario`: `list[Any]`.

Métodos públicos

Referência para leitura

```
adicionar_tag(self, tag: 'str') -> "'Entidade'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
definir(self, nome: 'str', valor: 'Any') -> "'Entidade'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
guardar(self, item: 'Any') -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
obter(self, nome: 'str', padrao: 'Any' = None) -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
possui_tag(self, tag: 'str') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_item(self, item: 'Any') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_tag(self, tag: 'str') -> 'None'
```

Método público sem descrição própria no runtime.

`coral.mundo.ErroMapaCoral`

Erro de domínio para operações inválidas de mapas Coral.

Assinatura

Assinatura para consulta

```
ErroMapaCoral(mensagem: 'str', *, codigo: 'str' = 'mapa.invalido', contexto: 'dict[str,
  Any] | None' = None) -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
codigo	str	KEYWORD_ONLY	não	'mapa.invalido'
contexto	dict[str, Any] | None	KEYWORD_ONLY	não	None

Retorno declarado

None

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.mundo.Mapa`

Estrutura espacial discreta genérica, independente de visualização.

Assinatura

Assinatura para consulta

```
Mapa(nome: 'str', topologia: 'Any', *, propriedades: 'Mapping[str, Any] | None' = None,
  eventos: 'Eventos | None' = None) -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
topologia	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
propriedades	Mapping[str, Any] | None	KEYWORD_ONLY	não	None
eventos	Eventos | None	KEYWORD_ONLY	não	None

Retorno declarado

None

Métodos públicos

Referência para leitura

```
adicionar_camada(self, nome: 'str', *, ordem: 'int | None' = None, propriedades:
  'Mapping[str, Any] | None' = None, participa_travessia: 'bool' = True) -> 'CamadaMapa'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
adicionar_regiao(self, regiao: 'Any') -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
aplicar_lote(self, operacoes: 'Iterable[Mapping[str, Any]]') -> 'tuple[MudancaMapa,
  ...]'
```

Valida todo o lote antes de aplicar qualquer escrita.

Referência para leitura

```
avaliar_travessia(self, origem: 'object', destino: 'object') -> 'ResultadoPassagem'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
bloquear_travessia(self, camada: 'str', celula: 'object', *, motivo: 'str | None' =
  None, causa: 'Any' = None) -> 'MudancaMapa'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
buscar_camada(self, nome: 'str') -> 'CamadaMapa | None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
buscar_regiao(self, nome: 'str') -> 'Any | None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
celulas_da_regiao(self, nome: 'str', *, limites: 'object' = None) -> 'tuple[CelulaMapa,
  ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
conteudo_em(self, camada: 'str', celula: 'object') -> 'ConteudoCelula | None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
contexto_em(self, celula: 'object', *, origem: 'object | None' = None) ->
  'ContextoLocalMapa'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
criar_regiao(self, nome: 'str', *, celulas: 'Iterable[object] | None' = None, forma:
  'Any' = None, referencia: 'str' = 'centro', propriedades: 'Mapping[str, Any] | None' =
  None) -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
definir(self, camada: 'str', celula: 'object', valor: 'Any', *, propriedades:
  'Mapping[str, Any] | None' = None, travessia: 'bool | None' = None, custo: 'float | int
  | None' = None, motivo: 'str | None' = None, causa: 'Any' = None) -> 'MudancaMapa'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
definir_travessia(self, camada: 'str', celula: 'object', *, permitida: 'bool | None' =
  None, custo: 'float | int | None' = None, motivo: 'str | None' = None, causa: 'Any' =
  None) -> 'MudancaMapa'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
exigir_camada(self, nome: 'str') -> 'CamadaMapa'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
exigir_regiao(self, nome: 'str') -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
obter(self, camada: 'str', celula: 'object', padrao: 'Any' = None) -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
permitir_travessia(self, camada: 'str', celula: 'object', *, custo: 'float | int | None'
   = None, motivo: 'str | None' = None, causa: 'Any' = None) -> 'MudancaMapa'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
possui_valor(self, camada: 'str', celula: 'object') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
regioes_em(self, celula_ou_posicao: 'object') -> 'tuple[Any, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover(self, camada: 'str', celula: 'object', *, causa: 'Any' = None) -> 'MudancaMapa |
   None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_camada(self, nome: 'str') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_regiao(self, nome: 'str') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
tentar_travessia(self, origem: 'object', destino: 'object', *, causa: 'Any' = None) ->
  'ResultadoPassagem'
```

Avalia uma tentativa e publica bloqueio apenas quando a ação é explícita.

Referência para leitura

```
vizinhos(self, celula: 'object', **opcoes: 'Any') -> 'tuple[CelulaMapa, ...]'
```

Método público sem descrição própria no runtime.

`coral.mundo.MudancaMapa`

MudancaMapa(mapa: 'str', camada: 'str', celula: 'CelulaMapa', operacao: 'str', anterior: 'ConteudoCelula | None', novo: 'ConteudoCelula | None', causa: 'Any' = None, aplicada: 'bool' = True, cancelada: 'bool' = False, motivo_cancelamento: 'str | None' = None, evento_sequencia: 'int | None' = None)

Assinatura

Assinatura para consulta

```
MudancaMapa(mapa: 'str', camada: 'str', celula: 'CelulaMapa', operacao: 'str', anterior:
   'ConteudoCelula | None', novo: 'ConteudoCelula | None', causa: 'Any' = None, aplicada:
  'bool' = True, cancelada: 'bool' = False, motivo_cancelamento: 'str | None' = None,
  evento_sequencia: 'int | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mapa	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
camada	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
celula	CelulaMapa	POSITIONAL_OR_KEYWORD	sim	sem padrão
operacao	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
anterior	ConteudoCelula | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
novo	ConteudoCelula | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	Any	POSITIONAL_OR_KEYWORD	não	None
aplicada	bool	POSITIONAL_OR_KEYWORD	não	True
cancelada	bool	POSITIONAL_OR_KEYWORD	não	False
motivo_cancelamento	str | None	POSITIONAL_OR_KEYWORD	não	None
evento_sequencia	int | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `mapa`: `str`.

Atributo `camada`: `str`.

Atributo `celula`: `CelulaMapa`.

Atributo `operacao`: `str`.

Atributo `anterior`: `ConteudoCelula | None`.

Atributo `novo`: `ConteudoCelula | None`.

Atributo `causa`: `Any`.

Atributo `aplicada`: `bool`.

Atributo `cancelada`: `bool`.

Atributo `motivo_cancelamento`: `str | None`.

Atributo `evento_sequencia`: `int | None`.

`coral.mundo.Mundo`

Mundo leve para jogos, RPGs e simulações Coral.

A API mantém entidades e relações como um grafo dirigido. Relações podem ser consultadas nos dois sentidos e o método `caminho` usa BFS.

Assinatura

Assinatura para consulta

```
Mundo(nome: 'str' = 'mundo', *, relogio: 'FonteTempo | None' = None, eventos: 'Eventos |
    None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	não	'mundo'
relogio	FonteTempo | None	KEYWORD_ONLY	não	None
eventos	Eventos | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
adicionar(self, entidade: 'Entidade') -> 'Entidade'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
adicionar_mapa(self, mapa: 'Mapa') -> 'Mapa'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
atualizar(self, dt: 'float' = 0.0) -> 'tuple[str, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
atualizar_por_relogio(self) -> 'tuple[str, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
atualizar_simulacao(self, dt_externo: 'float')
```

Atualiza a simulação respeitando pausa e escala temporal.

Referência para leitura

```
avancar_tempo(self, duracao: 'float')
```

Avança a simulação e depois atualiza regras e temporizadores pelo mesmo delta.

Referência para leitura

```
buscar(self, nome: 'str') -> 'Entidade | None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
buscar_combate(self: 'Mundo', nome: 'str')
```

Método público sem descrição própria no runtime.

Referência para leitura

```
buscar_mapa(self, nome: 'str') -> 'Mapa | None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
caminho(self, origem: 'Entidade | str', destino: 'Entidade | str', *, tipo: 'str | None'
   = None, bidirecional: 'bool' = True) -> 'tuple[Entidade, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
com_tag(self, tag: 'str') -> 'tuple[Entidade, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
criar_entidade(self, nome: 'str', **atributos: 'Any') -> 'Entidade'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
criar_mapa(self, nome: 'str', topologia, *, propriedades: 'dict[str, Any] | None' =
  None) -> 'Mapa'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
criar_simulacao(self, *, tempo=None)
```

Cria ou devolve a simulação temporal do mundo sem exigir Jogos.

Referência para leitura

```
emitir(self, evento: 'str', *args, **kwargs) -> 'int'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
emitir_evento(self, evento: 'str', *args, cancelavel: 'bool' = False, metadados:
  'dict[str, Any] | None' = None, **kwargs)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
emitir_evento_propagado(self, evento: 'str', *args, cancelavel: 'bool' = False,
  metadados: 'dict[str, Any] | None' = None, **kwargs)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
emitir_propagado(self, evento: 'str', *args, **kwargs) -> 'int'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
enfileirar_evento(self, evento: 'str', *args, **kwargs)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
exigir(self, nome: 'str') -> 'Entidade'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
exigir_mapa(self, nome: 'str') -> 'Mapa'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
observar_regioes_mapa(self, nome: 'str', alvo: 'Any', mapa: 'Mapa | str', regioes,
  resolvedor, *, identificador_alvo: 'Any' = None) -> 'ObservadorRegiaoMapa'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_json(self, **kwargs) -> 'str'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
processar_eventos(self, limite: 'int | None' = None)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
registrar_combate(self: 'Mundo', nome: 'str', combate: 'CombateTurnos')
```

Método público sem descrição própria no runtime.

Referência para leitura

```
relacionados(self, entidade: 'Entidade | str', tipo: 'str | None' = None, *, direcao:
  'str' = 'saida') -> 'tuple[Entidade, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
relacionar(self, origem: 'Entidade | str', destino: 'Entidade | str', tipo: 'str',
  **dados: 'Any') -> 'Relacao'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover(self, entidade: 'Entidade | str') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_mapa(self, mapa: 'Mapa | str') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_relacao(self, relacao: 'Relacao') -> 'bool'
```

Método público sem descrição própria no runtime.

`coral.mundo.ObservadorRegiaoMapa`

Reconhece transições de entrada e saída entre avaliações sucessivas.

Assinatura

Assinatura para consulta

```
ObservadorRegiaoMapa(nome: 'str', alvo: 'Any', mapa: 'Mapa', regioes:
  'Iterable[RegiaoMapa | str]', resolvedor: 'ResolvedorLocalizacao', eventos: 'Eventos',
  *, identificador_alvo: 'Any' = None) -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
alvo	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
mapa	Mapa	POSITIONAL_OR_KEYWORD	sim	sem padrão
regioes	Iterable[RegiaoMapa | str]	POSITIONAL_OR_KEYWORD	sim	sem padrão
resolvedor	ResolvedorLocalizacao	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
eventos	Eventos	POSITIONAL_OR_KEYWORD	sim	sem padrão
identificador_alvo	Any	KEYWORD_ONLY	não	None

Retorno declarado

None

Atributo `nome`: `str`.

Atributo `alvo`: `Any`.

Atributo `mapa`: `Mapa`.

Atributo `regioes`: `tuple[RegiaoMapa, ...]`.

Atributo `resolvedor`: `ResolvedorLocalizacao`.

Atributo `eventos`: `Eventos`.

Atributo `identificador_alvo`: `Any`.

Métodos públicos

Referência para leitura

```
avaliar(self, contexto: 'Any') -> 'bool'
```

Método público sem descrição própria no runtime.

`coral.mundo.RegiaoMapa`

Região por conjunto de células ou por forma espacial oficial.

Assinatura

Assinatura para consulta

```
RegiaoMapa(mapa: 'Any', nome: 'str', *, celulas: 'Iterable[object] | None' = None,
  forma: 'Any' = None, referencia: 'str' = 'centro', propriedades: 'Mapping[str, Any] |
  None' = None) -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mapa	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
celulas	Iterable[object] | None	KEYWORD_ONLY	não	None
forma	Any	KEYWORD_ONLY	não	None

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
referencia	str	KEYWORD_ONLY	não	'centro'
propriedades	Mapping[str, Any] | None	KEYWORD_ONLY	não	None

Retorno declarado

None

Métodos públicos

Referência para leitura

```
celulas(self, *, limites: 'object' = None) -> 'tuple[CelulaMapa, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
contem(self, celula_ou_posicao: 'object') -> 'bool'
```

Método público sem descrição própria no runtime.

`coral.mundo.Relacao`

Relacao(origem: 'Entidade', destino: 'Entidade', tipo: 'str', dados: 'dict[str, Any]' = <factory>)

Assinatura

Assinatura para consulta

```
Relacao(origem: 'Entidade', destino: 'Entidade', tipo: 'str', dados: 'dict[str, Any]' =
  <factory>) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
origem	Entidade	POSITIONAL_OR_KEYWORD	sim	sem padrão
destino	Entidade	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
dados	dict[str, Any]	POSITIONAL_OR_KEYWORD	não	<factory>

Retorno declarado

None

Atributo `origem`: `Entidade`.

Atributo `destino`: `Entidade`.

Atributo `tipo`: `str`.

Atributo `dados`: `dict[str, Any]`.

`coral.mundo.ResultadoPassagem`

ResultadoPassagem(permitida: 'bool', custo: 'float', motivo: 'str | None', camadas_responsaveis: 'tuple[str, ...]', origem: 'CelulaMapa', destino: 'CelulaMapa')

Assinatura

Assinatura para consulta

```
ResultadoPassagem(permitida: 'bool', custo: 'float', motivo: 'str | None',
  camadas_responsaveis: 'tuple[str, ...]', origem: 'CelulaMapa', destino: 'CelulaMapa') ->
   None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
permitida	bool	POSITIONAL_OR_KEYWORD	sim	sem padrão
custo	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
motivo	str | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
camadas_responsaveis	tuple[str, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
origem	CelulaMapa	POSITIONAL_OR_KEYWORD	sim	sem padrão
destino	CelulaMapa	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `permitida`: `bool`.

Atributo `custo`: `float`.

Atributo `motivo`: `str | None`.

Atributo `camadas_responsaveis`: `tuple[str, ...]`.

Atributo `origem`: `CelulaMapa`.

Atributo `destino`: `CelulaMapa`.

`coral.mundo.TopologiaHexagonal`

TopologiaHexagonal(tamanho_celula: 'float' = 1.0, orientacao: 'str' = 'topo_plano', origem: 'tuple[float, float]' = (0.0, 0.0), limites_configurados: 'object' = None)

Assinatura

Assinatura para consulta

```
TopologiaHexagonal(tamanho_celula: 'float' = 1.0, orientacao: 'str' = 'topo_plano',
  origem: 'tuple[float, float]' = (0.0, 0.0), limites_configurados: 'object' = None) ->
  None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tamanho_celula	float	POSITIONAL_OR_KEYWORD	não	1.0
orientacao	str	POSITIONAL_OR_KEYWORD	não	'topo_plano'
origem	tuple[float, float]	POSITIONAL_OR_KEYWORD	não	(0.0, 0.0)
limites_configurados	object	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `tamanho_celula`: `float`.

Atributo `orientacao`: `str`.

Atributo `origem`: `tuple[float, float]`.

Atributo `limites_configurados`: `object`.

Métodos públicos

Referência para leitura

```
adjacentes(self, primeira: 'object', segunda: 'object', **opcoes: 'object') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
celula_para_mundo(self, celula: 'object', *, ancora: 'str' = 'centro') ->
  'ValorEspacial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
dentro_dos_limites(self, celula: 'object') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
mundo_para_celula(self, coordenada: 'object') -> 'CelulaHexagonal'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
normalizar(self, celula: 'object') -> 'C'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
validar(self, celula: 'object') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
vizinhos(self, celula: 'object', **_: 'object') -> 'tuple[CelulaHexagonal, ...]'
```

Método público sem descrição própria no runtime.

`coral.mundo.TopologiaQuadrada`

TopologiaQuadrada(largura_celula: 'float' = 1.0, altura_celula: 'float' = 1.0, origem: 'tuple[float, float]' = (0.0, 0.0), limites_configurados: 'object' = None)

Assinatura

Assinatura para consulta

```
TopologiaQuadrada(largura_celula: 'float' = 1.0, altura_celula: 'float' = 1.0, origem:
  'tuple[float, float]' = (0.0, 0.0), limites_configurados: 'object' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
largura_celula	float	POSITIONAL_OR_KEYWORD	não	1.0
altura_celula	float	POSITIONAL_OR_KEYWORD	não	1.0
origem	tuple[float, float]	POSITIONAL_OR_KEYWORD	não	(0.0, 0.0)
limites_configurados	object	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `largura_celula`: `float`.

Atributo `altura_celula`: `float`.

Atributo `origem`: `tuple[float, float]`.

Atributo `limites_configurados`: `object`.

Métodos públicos

Referência para leitura

```
adjacentes(self, primeira: 'object', segunda: 'object', **opcoes: 'object') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
celula_para_mundo(self, celula: 'object', *, ancora: 'str' = 'centro') ->
  'ValorEspacial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
dentro_dos_limites(self, celula: 'object') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
mundo_para_celula(self, coordenada: 'object') -> 'CelulaQuadrada'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
normalizar(self, celula: 'object') -> 'C'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
validar(self, celula: 'object') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
vizinhos(self, celula: 'object', *, incluir_diagonais: 'bool' = False) ->
  'tuple[CelulaQuadrada, ...]'
```

Método público sem descrição própria no runtime.

`coral.mundo.TopologiaVoxel`

TopologiaVoxel(tamanho_x: 'float' = 1.0, tamanho_y: 'float' = 1.0, tamanho_z: 'float' = 1.0, origem: 'tuple[float, float, float]' = (0.0, 0.0, 0.0), limites_configurados: 'object' = None)

Assinatura

Assinatura para consulta

```
TopologiaVoxel(tamanho_x: 'float' = 1.0, tamanho_y: 'float' = 1.0, tamanho_z: 'float' =
  1.0, origem: 'tuple[float, float, float]' = (0.0, 0.0, 0.0), limites_configurados:
  'object' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tamanho_x	float	POSITIONAL_OR_KEYWORD	não	1.0

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tamanho_y	float	POSITIONAL_OR_KEYWORD	não	1.0
tamanho_z	float	POSITIONAL_OR_KEYWORD	não	1.0
origem	tuple[float, float, float]	POSITIONAL_OR_KEYWORD	não	(0.0, 0.0, 0.0)
limites_configurados	object	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `tamanho_x`: `float`.

Atributo `tamanho_y`: `float`.

Atributo `tamanho_z`: `float`.

Atributo `origem`: `tuple[float, float, float]`.

Atributo `limites_configurados`: `object`.

Métodos públicos

Referência para leitura

```
adjacentes(self, primeira: 'object', segunda: 'object', **opcoes: 'object') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
celula_para_mundo(self, celula: 'object', *, ancora: 'str' = 'centro') ->
  'ValorEspacial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
dentro_dos_limites(self, celula: 'object') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
mundo_para_celula(self, coordenada: 'object') -> 'CelulaVoxel'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
normalizar(self, celula: 'object') -> 'C'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
validar(self, celula: 'object') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
vizinhos(self, celula: 'object', **_: 'object') -> 'tuple[CelulaVoxel, ...]'
```

Método público sem descrição própria no runtime.

`coral.mundo.criar_mundo`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
criar_mundo(nome: 'str' = 'mundo') -> 'Mundo'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	não	'mundo'

Retorno declarado

Mundo

Consultar a implementação canônica de coral.mundo

Implementação canônica de `coral.mundo`

Código Python

```
"""Modelo discreto e genérico de mapas Coral.

O núcleo é headless e não conhece cenas, sprites, câmeras ou personagens.
Células representam localizações discretas; posições contínuas permanecem na
baseline espacial oficial da Coral.
"""

from __future__ import annotations

from collections.abc import Iterable, Mapping
from dataclasses import dataclass, field
import math
from numbers import Integral
from types import MappingProxyType
from typing import Any

from coral.tempo_eventos import Eventos


class ErroMapaCoral(ValueError):
    """Erro de domínio para operações inválidas de mapas Coral."""

    def __init__(self, mensagem: str, *, codigo: str = "mapa.invalido", contexto:
      dict[str, Any] | None = None) -> None:
```

```
        super().__init__(mensagem)
        self.codigo = codigo
        self.contexto = dict(contexto or {})


def _coordenada_inteira(valor: object, *, nome: str) -> int:
    if isinstance(valor, bool) or not isinstance(valor, Integral):
        raise ErroMapaCoral(
            f'A coordenada "{nome}" deve ser inteira; recebido {valor!r}.',
            codigo="mapa.coordenada_nao_inteira",
            contexto={"coordenada": nome, "valor": valor},
        )
    return int(valor)


@dataclass(frozen=True, slots=True)
class CelulaQuadrada:
    coluna: int
    linha: int

    def __post_init__(self) -> None:
        object.__setattr__(self, "coluna", _coordenada_inteira(self.coluna,
          nome="coluna"))
        object.__setattr__(self, "linha", _coordenada_inteira(self.linha, nome="linha"))

    def como_tupla(self) -> tuple[int, int]:
        return self.coluna, self.linha

    def __iter__(self):
        return iter(self.como_tupla())


@dataclass(frozen=True, slots=True)
class CelulaHexagonal:
    q: int
    r: int

    def __post_init__(self) -> None:
        object.__setattr__(self, "q", _coordenada_inteira(self.q, nome="q"))
        object.__setattr__(self, "r", _coordenada_inteira(self.r, nome="r"))

    def como_tupla(self) -> tuple[int, int]:
        return self.q, self.r

    def __iter__(self):
        return iter(self.como_tupla())


@dataclass(frozen=True, slots=True)
class CelulaVoxel:
    x: int
    y: int
    z: int

    def __post_init__(self) -> None:
        object.__setattr__(self, "x", _coordenada_inteira(self.x, nome="x"))
        object.__setattr__(self, "y", _coordenada_inteira(self.y, nome="y"))
        object.__setattr__(self, "z", _coordenada_inteira(self.z, nome="z"))

    def como_tupla(self) -> tuple[int, int, int]:
        return self.x, self.y, self.z
```

```
    def __iter__(self):
        return iter(self.como_tupla())


CelulaMapa = CelulaQuadrada | CelulaHexagonal | CelulaVoxel


def _sequencia_coordenadas(valor: object, *, dimensoes: int, nome_topologia: str) ->
  tuple[object, ...]:
    if isinstance(valor, (str, bytes, bytearray)):
        raise ErroMapaCoral(
            f"Uma célula de topologia {nome_topologia} precisa de {dimensoes}
              coordenadas inteiras.",
            codigo="mapa.celula_dimensao_incorreta",
            contexto={"topologia": nome_topologia, "dimensoes": dimensoes, "valor":
              valor},
        )
    try:
        coordenadas = tuple(valor)  # type: ignore[arg-type]
    except TypeError as exc:
        raise ErroMapaCoral(
            f"Uma célula de topologia {nome_topologia} precisa de {dimensoes}
              coordenadas inteiras.",
            codigo="mapa.celula_dimensao_incorreta",
            contexto={"topologia": nome_topologia, "dimensoes": dimensoes, "valor":
              valor},
        ) from exc
    if len(coordenadas) != dimensoes:
        raise ErroMapaCoral(
            f"Uma célula de topologia {nome_topologia} precisa de {dimensoes}
              coordenadas; recebido {len(coordenadas)}.",
            codigo="mapa.celula_dimensao_incorreta",
            contexto={"topologia": nome_topologia, "dimensoes": dimensoes,
              "quantidade_recebida": len(coordenadas)},
        )
    return coordenadas


def _clonar_portatil(valor: Any, *, campo: str) -> Any:
    """Valida e clona um valor pelo contrato portátil existente da Coral."""
    from coral.persistencia import ErroPersistencia, de_dados, valor_portatil

    try:
        codificado = valor_portatil(valor)
        return de_dados({"valor": codificado})["valor"]
    except ErroPersistencia as exc:
        raise ErroMapaCoral(
            f"O {campo} não é portátil para um mapa Coral: {exc}",
            codigo="mapa.valor_nao_portatil",
            contexto={"campo": campo, "tipo": type(valor).__name__},
        ) from exc


def _copiar_propriedades(propriedades: Mapping[str, Any] | None, *, campo: str =
  "propriedades") -> dict[str, Any]:
    if propriedades is None:
        return {}
    if not isinstance(propriedades, Mapping):
        raise ErroMapaCoral(
            f"{campo.capitalize()} precisam ser um dicionário com chaves de texto.",
```

```
            codigo="mapa.propriedades_invalidas",
            contexto={"campo": campo, "tipo": type(propriedades).__name__},
        )
    bruto = dict(propriedades)
    if any(not isinstance(chave, str) for chave in bruto):
        raise ErroMapaCoral(
            f"{campo.capitalize()} exigem chaves de texto.",
            codigo="mapa.propriedades_invalidas",
            contexto={"campo": campo},
        )
    clonadas = _clonar_portatil(bruto, campo=campo)
    assert isinstance(clonadas, dict)
    return clonadas


def _validar_travessia(
    permitida: bool | None,
    custo: float | int | None,
    motivo: str | None,
) -> tuple[bool | None, float | None, str | None]:
    if permitida is not None and not isinstance(permitida, bool):
        raise ErroMapaCoral(
            "A política de travessia precisa ser verdadeira, falsa ou não declarada.",
            codigo="mapa.travessia_invalida",
        )
    custo_normalizado = None
    if custo is not None:
        if isinstance(custo, bool) or not isinstance(custo, (int, float)):
            raise ErroMapaCoral("O custo de travessia precisa ser numérico.",
              codigo="mapa.custo_travessia_invalido")
        custo_normalizado = float(custo)
        if custo_normalizado < 0 or not math.isfinite(custo_normalizado):
            raise ErroMapaCoral(
                "O custo de travessia precisa ser finito e maior ou igual a zero.",
                codigo="mapa.custo_travessia_invalido",
                contexto={"custo": custo},
            )
    if motivo is not None and not isinstance(motivo, str):
        raise ErroMapaCoral("O motivo de travessia precisa ser texto.",
          codigo="mapa.travessia_invalida")
    return permitida, custo_normalizado, motivo


@dataclass(frozen=True, slots=True, init=False)
class BlocoMapa:
    """Conveniência para terreno, material, obstáculo ou tile."""

    identificador: str
    _propriedades: dict[str, Any] = field(repr=False, compare=True, hash=False)

    def __init__(self, identificador: str, propriedades: Mapping[str, Any] | None =
      None) -> None:
        nome = str(identificador).strip()
        if not nome:
            raise ErroMapaCoral("Um bloco precisa de um identificador não vazio.",
              codigo="mapa.bloco_invalido")
        object.__setattr__(self, "identificador", nome)
        object.__setattr__(self, "_propriedades", _copiar_propriedades(propriedades,
          campo="propriedades do bloco"))

    @property
```

```
    def propriedades(self) -> Mapping[str, Any]:
        return MappingProxyType(_clonar_portatil(self._propriedades, campo="propriedades
           do bloco"))


@dataclass(frozen=True, slots=True, init=False)
class ConteudoCelula:
    """Valor imutável associado a uma célula, com travessia opcional."""

    _valor: Any = field(repr=False, compare=True, hash=False)
    _propriedades: dict[str, Any] = field(repr=False, compare=True, hash=False)
    travessia_permitida: bool | None
    custo_travessia: float | None
    motivo_travessia: str | None

    def __init__(
        self,
        valor: Any,
        propriedades: Mapping[str, Any] | None = None,
        *,
        travessia: bool | None = None,
        custo: float | int | None = None,
        motivo: str | None = None,
    ) -> None:
        permitida, custo_normalizado, motivo_normalizado = _validar_travessia(travessia,
           custo, motivo)
        object.__setattr__(self, "_valor", _clonar_portatil(valor, campo="valor da
          célula"))
        object.__setattr__(self, "_propriedades", _copiar_propriedades(propriedades))
        object.__setattr__(self, "travessia_permitida", permitida)
        object.__setattr__(self, "custo_travessia", custo_normalizado)
        object.__setattr__(self, "motivo_travessia", motivo_normalizado)

    @property
    def valor(self) -> Any:
        return _clonar_portatil(self._valor, campo="valor da célula")

    @property
    def propriedades(self) -> Mapping[str, Any]:
        return MappingProxyType(_clonar_portatil(self._propriedades,
          campo="propriedades"))

    @property
    def declarou_travessia(self) -> bool:
        return self.travessia_permitida is not None or self.custo_travessia is not None
          or self.motivo_travessia is not None

    def com_valor(self, valor: Any) -> "ConteudoCelula":
        return ConteudoCelula(
            valor,
            self._propriedades,
            travessia=self.travessia_permitida,
            custo=self.custo_travessia,
            motivo=self.motivo_travessia,
        )

    def com_propriedades(self, propriedades: Mapping[str, Any]) -> "ConteudoCelula":
        return ConteudoCelula(
            self._valor,
            propriedades,
            travessia=self.travessia_permitida,
```

```
            custo=self.custo_travessia,
            motivo=self.motivo_travessia,
        )

    def com_travessia(
        self,
        *,
        permitida: bool | None = None,
        custo: float | int | None = None,
        motivo: str | None = None,
    ) -> "ConteudoCelula":
        return ConteudoCelula(self._valor, self._propriedades, travessia=permitida,
          custo=custo, motivo=motivo)


@dataclass(frozen=True, slots=True)
class MudancaMapa:
    mapa: str
    camada: str
    celula: CelulaMapa
    operacao: str
    anterior: ConteudoCelula | None
    novo: ConteudoCelula | None
    causa: Any = None
    aplicada: bool = True
    cancelada: bool = False
    motivo_cancelamento: str | None = None
    evento_sequencia: int | None = None


@dataclass(frozen=True, slots=True)
class ResultadoPassagem:
    permitida: bool
    custo: float
    motivo: str | None
    camadas_responsaveis: tuple[str, ...]
    origem: CelulaMapa
    destino: CelulaMapa


@dataclass(frozen=True, slots=True)
class ContextoLocalMapa:
    mapa: "Mapa"
    celula: CelulaMapa
    camadas_presentes: tuple[str, ...]
    conteudos: Mapping[str, ConteudoCelula]
    regioes: tuple[Any, ...]
    travessia: ResultadoPassagem | None


class CamadaMapa:
    """Camada nomeada, ordenada e esparsa de um mapa."""

    __slots__ = ("nome", "ordem", "participa_travessia", "_propriedades", "_conteudos",
      "_sequencia")

    def __init__(
        self,
        nome: str,
        *,
        ordem: int = 0,
```

```
        propriedades: Mapping[str, Any] | None = None,
        participa_travessia: bool = True,
        _sequencia: int = 0,
    ) -> None:
        nome_normalizado = str(nome).strip()
        if not nome_normalizado:
            raise ErroMapaCoral("Uma camada precisa de um nome não vazio.",
              codigo="mapa.camada_invalida")
        if isinstance(ordem, bool) or not isinstance(ordem, Integral):
            raise ErroMapaCoral("A ordem de uma camada precisa ser inteira.",
              codigo="mapa.ordem_camada_invalida")
        if not isinstance(participa_travessia, bool):
            raise ErroMapaCoral("participa_travessia precisa ser verdadeiro ou falso.",
              codigo="mapa.travessia_invalida")
        self.nome = nome_normalizado
        self.ordem = int(ordem)
        self.participa_travessia = participa_travessia
        self._propriedades = _copiar_propriedades(propriedades, campo=f"propriedades da
          camada {nome_normalizado}")
        self._conteudos: dict[CelulaMapa, ConteudoCelula] = {}
        self._sequencia = int(_sequencia)

    @property
    def propriedades(self) -> Mapping[str, Any]:
        return MappingProxyType(_clonar_portatil(self._propriedades,
          campo=f"propriedades da camada {self.nome}"))

    @property
    def quantidade(self) -> int:
        return len(self._conteudos)

    @property
    def celulas(self) -> tuple[CelulaMapa, ...]:
        return tuple(self._conteudos)

    def possui(self, celula: CelulaMapa) -> bool:
        return celula in self._conteudos

    def conteudo(self, celula: CelulaMapa) -> ConteudoCelula | None:
        return self._conteudos.get(celula)

    def _definir(self, celula: CelulaMapa, conteudo: ConteudoCelula) -> ConteudoCelula |
       None:
        anterior = self._conteudos.get(celula)
        self._conteudos[celula] = conteudo
        return anterior

    def _remover(self, celula: CelulaMapa) -> ConteudoCelula | None:
        return self._conteudos.pop(celula, None)


class Mapa:
    """Estrutura espacial discreta genérica, independente de visualização."""

    def __init__(
        self,
        nome: str,
        topologia: Any,
        *,
        propriedades: Mapping[str, Any] | None = None,
        eventos: Eventos | None = None,
```

```
    ) -> None:
        nome_normalizado = str(nome).strip()
        if not nome_normalizado:
            raise ErroMapaCoral("Um mapa precisa de um nome não vazio.",
              codigo="mapa.nome_invalido")
        if not all(hasattr(topologia, atributo) for atributo in ("normalizar",
          "vizinhos", "dimensionalidade", "nome")):
            raise ErroMapaCoral(
                "A topologia informada não implementa o contrato de mapas Coral.",
                codigo="mapa.topologia_desconhecida",
                contexto={"tipo": type(topologia).__name__},
            )
        self.nome = nome_normalizado
        self.topologia = topologia
        self._propriedades = _copiar_propriedades(propriedades, campo="propriedades do
          mapa")
        self._camadas: dict[str, CamadaMapa] = {}
        self._regioes: dict[str, Any] = {}
        self._proxima_sequencia = 0
        self._eventos: Eventos | None = eventos
        self._rastreamento: Any = None


    @property
    def eventos(self) -> Eventos | None:
        """Barramento associado, sem criar runtime oculto quando ausente."""
        return self._eventos

    def _vincular_runtime(self, eventos: Eventos | None, rastreamento: Any = None) ->
      None:
        self._eventos = eventos
        self._rastreamento = rastreamento

    def _registrar_evento_reativo(self, evento: Any, decisao: str) -> None:
        rastreamento = self._rastreamento
        if rastreamento is None:
            return
        rastreamento.registrar(
            sequencia=evento.sequencia,
            instante=evento.instante,
            tipo="evento",
            nome=evento.nome,
            decisao=decisao,
            causa_sequencia=evento.causa_sequencia,
            detalhes={
                "origem": evento.origem,
                "cancelado": evento.cancelado,
                "mapa": self.nome,
            },
        )

    @staticmethod
    def _valor_evento(conteudo: ConteudoCelula | None) -> Any:
        return None if conteudo is None else conteudo.valor

    def _dados_mudanca_evento(self, mudanca: MudancaMapa) -> dict[str, Any]:
        return {
            "mapa": self.nome,
            "camada": mudanca.camada,
            "celula": list(mudanca.celula.como_tupla()),
            "operacao": mudanca.operacao,
```

```
            "valor_anterior": self._valor_evento(mudanca.anterior),
            "valor_novo": self._valor_evento(mudanca.novo),
            "causa": mudanca.causa,
        }

    def _autorizar_mudanca(self, mudanca: MudancaMapa) -> MudancaMapa:
        if self._eventos is None:
            return mudanca
        evento = self._eventos.emitir_evento(
            "mapa.valor_antes_de_mudar",
            origem=self.nome,
            cancelavel=True,
            **self._dados_mudanca_evento(mudanca),
        )
        self._registrar_evento_reativo(evento, "cancelado" if evento.cancelado else
          "autorizado")
        if not evento.cancelado:
            return MudancaMapa(
                mudanca.mapa, mudanca.camada, mudanca.celula, mudanca.operacao,
                mudanca.anterior, mudanca.novo, mudanca.causa,
                evento_sequencia=evento.sequencia,
            )
        return MudancaMapa(
            mudanca.mapa, mudanca.camada, mudanca.celula, mudanca.operacao,
            mudanca.anterior, mudanca.novo, mudanca.causa,
            aplicada=False, cancelada=True,
            motivo_cancelamento=evento.motivo_cancelamento,
            evento_sequencia=evento.sequencia,
        )

    def _publicar_mudanca(self, mudanca: MudancaMapa) -> None:
        if self._eventos is None or not mudanca.aplicada:
            return
        evento = self._eventos.emitir_evento(
            "mapa.valor_mudou",
            origem=self.nome,
            **self._dados_mudanca_evento(mudanca),
        )
        self._registrar_evento_reativo(evento, "emitido")

    @property
    def propriedades(self) -> Mapping[str, Any]:
        return MappingProxyType(_clonar_portatil(self._propriedades, campo="propriedades
           do mapa"))

    @property
    def camadas(self) -> tuple[CamadaMapa, ...]:
        return tuple(sorted(self._camadas.values(), key=lambda camada: (camada.ordem,
          camada._sequencia)))

    @property
    def regioes(self) -> tuple[Any, ...]:
        return tuple(self._regioes.values())

    def adicionar_camada(
        self,
        nome: str,
        *,
        ordem: int | None = None,
        propriedades: Mapping[str, Any] | None = None,
        participa_travessia: bool = True,
```

```
    ) -> CamadaMapa:
        nome_normalizado = str(nome).strip()
        if nome_normalizado in self._camadas:
            raise ErroMapaCoral(
                f'Já existe uma camada chamada "{nome_normalizado}" no mapa
                  "{self.nome}".',
                codigo="mapa.camada_duplicada",
                contexto={"mapa": self.nome, "camada": nome_normalizado},
            )
        ordem_real = self._proxima_sequencia if ordem is None else ordem
        camada = CamadaMapa(
            nome_normalizado,
            ordem=ordem_real,
            propriedades=propriedades,
            participa_travessia=participa_travessia,
            _sequencia=self._proxima_sequencia,
        )
        self._proxima_sequencia += 1
        self._camadas[nome_normalizado] = camada
        return camada

    def buscar_camada(self, nome: str) -> CamadaMapa | None:
        return self._camadas.get(str(nome))

    def exigir_camada(self, nome: str) -> CamadaMapa:
        camada = self.buscar_camada(nome)
        if camada is None:
            raise ErroMapaCoral(
                f'A camada "{nome}" não existe no mapa "{self.nome}".',
                codigo="mapa.camada_inexistente",
                contexto={"mapa": self.nome, "camada": str(nome)},
            )
        return camada

    def remover_camada(self, nome: str) -> bool:
        return self._camadas.pop(str(nome), None) is not None

    def adicionar_regiao(self, regiao: Any) -> Any:
        from .regioes import RegiaoMapa

        if not isinstance(regiao, RegiaoMapa) or regiao.mapa is not self:
            raise ErroMapaCoral("A região precisa pertencer ao mapa em que será
              adicionada.", codigo="mapa.regiao_invalida")
        if regiao.nome in self._regioes and self._regioes[regiao.nome] is not regiao:
            raise ErroMapaCoral(
                f'Já existe uma região chamada "{regiao.nome}" no mapa "{self.nome}".',
                codigo="mapa.regiao_duplicada",
            )
        self._regioes[regiao.nome] = regiao
        return regiao

    def criar_regiao(
        self,
        nome: str,
        *,
        celulas: Iterable[object] | None = None,
        forma: Any = None,
        referencia: str = "centro",
        propriedades: Mapping[str, Any] | None = None,
    ) -> Any:
        from .regioes import RegiaoMapa
```

```
        return self.adicionar_regiao(
            RegiaoMapa(self, nome, celulas=celulas, forma=forma, referencia=referencia,
              propriedades=propriedades)
        )

    def buscar_regiao(self, nome: str) -> Any | None:
        return self._regioes.get(str(nome))

    def exigir_regiao(self, nome: str) -> Any:
        regiao = self.buscar_regiao(nome)
        if regiao is None:
            raise ErroMapaCoral(
                f'A região "{nome}" não existe no mapa "{self.nome}".',
                codigo="mapa.regiao_inexistente",
            )
        return regiao

    def remover_regiao(self, nome: str) -> bool:
        return self._regioes.pop(str(nome), None) is not None

    def regioes_em(self, celula_ou_posicao: object) -> tuple[Any, ...]:
        return tuple(regiao for regiao in self._regioes.values() if
          regiao.contem(celula_ou_posicao))

    def celulas_da_regiao(self, nome: str, *, limites: object = None) ->
      tuple[CelulaMapa, ...]:
        return self.exigir_regiao(nome).celulas(limites=limites)

    def possui_valor(self, camada: str, celula: object) -> bool:
        alvo = self.exigir_camada(camada)
        normalizada = self.topologia.normalizar(celula)
        return alvo.possui(normalizada)

    def conteudo_em(self, camada: str, celula: object) -> ConteudoCelula | None:
        alvo = self.exigir_camada(camada)
        normalizada = self.topologia.normalizar(celula)
        return alvo.conteudo(normalizada)

    def obter(self, camada: str, celula: object, padrao: Any = None) -> Any:
        conteudo = self.conteudo_em(camada, celula)
        return padrao if conteudo is None else conteudo.valor

    def definir(
        self,
        camada: str,
        celula: object,
        valor: Any,
        *,
        propriedades: Mapping[str, Any] | None = None,
        travessia: bool | None = None,
        custo: float | int | None = None,
        motivo: str | None = None,
        causa: Any = None,
    ) -> MudancaMapa:
        alvo = self.exigir_camada(camada)
        normalizada = self.topologia.normalizar(celula)
        extras = propriedades is not None or travessia is not None or custo is not None
          or motivo is not None
        if isinstance(valor, ConteudoCelula):
            if extras:
```

```
                raise ErroMapaCoral(
                    "Ao fornecer ConteudoCelula, altere suas propriedades ou travessia
                      antes de defini lo no mapa.",
                    codigo="mapa.conteudo_ambiguo",
                )
            conteudo = ConteudoCelula(
                valor.valor,
                valor.propriedades,
                travessia=valor.travessia_permitida,
                custo=valor.custo_travessia,
                motivo=valor.motivo_travessia,
            )
        else:
            conteudo = ConteudoCelula(valor, propriedades, travessia=travessia,
              custo=custo, motivo=motivo)
        causa_clonada = _clonar_causa(causa)
        anterior = alvo.conteudo(normalizada)
        proposta = self._autorizar_mudanca(
            MudancaMapa(self.nome, alvo.nome, normalizada, "definir", anterior,
              conteudo, causa_clonada)
        )
        if not proposta.aplicada:
            return proposta
        alvo._definir(normalizada, conteudo)
        self._publicar_mudanca(proposta)
        return proposta

    def remover(self, camada: str, celula: object, *, causa: Any = None) -> MudancaMapa
      | None:
        alvo = self.exigir_camada(camada)
        normalizada = self.topologia.normalizar(celula)
        causa_clonada = _clonar_causa(causa)
        anterior = alvo.conteudo(normalizada)
        if anterior is None:
            return None
        proposta = self._autorizar_mudanca(
            MudancaMapa(self.nome, alvo.nome, normalizada, "remover", anterior, None,
              causa_clonada)
        )
        if not proposta.aplicada:
            return proposta
        alvo._remover(normalizada)
        self._publicar_mudanca(proposta)
        return proposta

    def definir_travessia(
        self,
        camada: str,
        celula: object,
        *,
        permitida: bool | None = None,
        custo: float | int | None = None,
        motivo: str | None = None,
        causa: Any = None,
    ) -> MudancaMapa:
        alvo = self.exigir_camada(camada)
        normalizada = self.topologia.normalizar(celula)
        anterior = alvo.conteudo(normalizada)
        if anterior is None:
            raise ErroMapaCoral(
                "Não é possível definir travessia em uma célula sem conteúdo nessa
```

```
                camada.",
                codigo="mapa.conteudo_ausente",
                contexto={"mapa": self.nome, "camada": camada, "celula":
                  normalizada.como_tupla()},
            )
        novo = anterior.com_travessia(permitida=permitida, custo=custo, motivo=motivo)
        causa_clonada = _clonar_causa(causa)
        proposta = self._autorizar_mudanca(
            MudancaMapa(self.nome, alvo.nome, normalizada, "travessia", anterior, novo,
              causa_clonada)
        )
        if not proposta.aplicada:
            return proposta
        alvo._definir(normalizada, novo)
        self._publicar_mudanca(proposta)
        return proposta

    def bloquear_travessia(self, camada: str, celula: object, *, motivo: str | None =
      None, causa: Any = None) -> MudancaMapa:
        return self.definir_travessia(camada, celula, permitida=False, motivo=motivo,
          causa=causa)

    def permitir_travessia(
        self,
        camada: str,
        celula: object,
        *,
        custo: float | int | None = None,
        motivo: str | None = None,
        causa: Any = None,
    ) -> MudancaMapa:
        return self.definir_travessia(camada, celula, permitida=True, custo=custo,
          motivo=motivo, causa=causa)

    def avaliar_travessia(self, origem: object, destino: object) -> ResultadoPassagem:
        origem_normalizada = self.topologia.normalizar(origem)
        destino_normalizado = self.topologia.normalizar(destino)
        custo_total = 0.0
        responsaveis: list[str] = []
        bloqueios: list[tuple[str, str | None]] = []

        for camada in self.camadas:
            if not camada.participa_travessia:
                continue
            conteudo = camada.conteudo(destino_normalizado)
            if conteudo is None or not conteudo.declarou_travessia:
                continue
            responsaveis.append(camada.nome)
            if conteudo.custo_travessia is not None:
                custo_total += conteudo.custo_travessia
            if conteudo.travessia_permitida is False:
                bloqueios.append((camada.nome, conteudo.motivo_travessia))

        if bloqueios:
            primeiro = bloqueios[0]
            motivo = primeiro[1] or f'A camada "{primeiro[0]}" bloqueia a travessia.'
            return ResultadoPassagem(False, custo_total, motivo, tuple(responsaveis),
              origem_normalizada, destino_normalizado)
        return ResultadoPassagem(True, custo_total, None, tuple(responsaveis),
          origem_normalizada, destino_normalizado)
```

```
    def tentar_travessia(self, origem: object, destino: object, *, causa: Any = None) ->
       ResultadoPassagem:
        """Avalia uma tentativa e publica bloqueio apenas quando a ação é explícita."""
        resultado = self.avaliar_travessia(origem, destino)
        if resultado.permitida or self._eventos is None:
            return resultado
        causa_clonada = _clonar_causa(causa)
        evento = self._eventos.emitir_evento(
            "mapa.travessia_bloqueada",
            origem=self.nome,
            mapa=self.nome,
            origem_celula=list(resultado.origem.como_tupla()),
            destino=list(resultado.destino.como_tupla()),
            custo=resultado.custo,
            motivo=resultado.motivo,
            camadas_responsaveis=list(resultado.camadas_responsaveis),
            causa=causa_clonada,
        )
        self._registrar_evento_reativo(evento, "emitido")
        return resultado

    def contexto_em(self, celula: object, *, origem: object | None = None) ->
      ContextoLocalMapa:
        normalizada = self.topologia.normalizar(celula)
        presentes: list[str] = []
        conteudos: dict[str, ConteudoCelula] = {}
        for camada in self.camadas:
            conteudo = camada.conteudo(normalizada)
            if conteudo is not None:
                presentes.append(camada.nome)
                conteudos[camada.nome] = conteudo
        travessia = None if origem is None else self.avaliar_travessia(origem,
          normalizada)
        return ContextoLocalMapa(
            self,
            normalizada,
            tuple(presentes),
            MappingProxyType(conteudos),
            self.regioes_em(normalizada),
            travessia,
        )

    def vizinhos(self, celula: object, **opcoes: Any) -> tuple[CelulaMapa, ...]:
        return self.topologia.vizinhos(celula, **opcoes)

    def aplicar_lote(self, operacoes: Iterable[Mapping[str, Any]]) -> tuple[MudancaMapa,
       ...]:
        """Valida todo o lote antes de aplicar qualquer escrita."""
        try:
            entradas = list(operacoes)
        except TypeError as exc:
            raise ErroMapaCoral("Um lote precisa ser uma coleção de operações.",
              codigo="mapa.lote_invalido") from exc

        simulacao: dict[tuple[str, CelulaMapa], ConteudoCelula | None] = {}
        preparados: list[tuple[str, CamadaMapa, CelulaMapa, ConteudoCelula | None,
          ConteudoCelula | None, Any]] = []

        for indice, operacao in enumerate(entradas):
            if not isinstance(operacao, Mapping):
                raise ErroMapaCoral(
```

```
                f"A operação {indice} do lote precisa ser um registro.",
                codigo="mapa.lote_invalido",
                contexto={"indice": indice},
            )
        tipo = str(operacao.get("operacao", "definir"))
        nome_camada = operacao.get("camada")
        if not isinstance(nome_camada, str):
            raise ErroMapaCoral(
                f"A operação {indice} do lote não informa uma camada válida.",
                codigo="mapa.lote_invalido",
                contexto={"indice": indice},
            )
        alvo = self.exigir_camada(nome_camada)
        if "celula" not in operacao:
            raise ErroMapaCoral(
                f"A operação {indice} do lote não informa uma célula.",
                codigo="mapa.lote_invalido",
                contexto={"indice": indice},
            )
        celula = self.topologia.normalizar(operacao["celula"])
        chave = (alvo.nome, celula)
        anterior = simulacao[chave] if chave in simulacao else alvo.conteudo(celula)
        causa = _clonar_causa(operacao.get("causa"))

        if tipo == "definir":
            if "valor" not in operacao:
                raise ErroMapaCoral(
                    f"A operação {indice} de definição não informa um valor.",
                    codigo="mapa.lote_invalido",
                    contexto={"indice": indice},
                )
            novo = ConteudoCelula(
                operacao["valor"],
                operacao.get("propriedades"),
                travessia=operacao.get("travessia"),
                custo=operacao.get("custo"),
                motivo=operacao.get("motivo"),
            )
        elif tipo == "remover":
            novo = None
        elif tipo == "travessia":
            if anterior is None:
                raise ErroMapaCoral(
                    f"A operação {indice} tenta configurar travessia sem conteúdo.",
                    codigo="mapa.conteudo_ausente",
                    contexto={"indice": indice},
                )
            novo = anterior.com_travessia(
                permitida=operacao.get("permitida"),
                custo=operacao.get("custo"),
                motivo=operacao.get("motivo"),
            )
        else:
            raise ErroMapaCoral(
                f'Operação de lote desconhecida: "{tipo}".',
                codigo="mapa.lote_invalido",
                contexto={"indice": indice, "operacao": tipo},
            )

        simulacao[chave] = novo
        preparados.append((tipo, alvo, celula, anterior, novo, causa))
```

```
        mudancas: list[MudancaMapa] = []
        for tipo, alvo, celula, anterior, novo, causa in preparados:
            if novo is None:
                alvo._remover(celula)
            else:
                alvo._definir(celula, novo)
            mudancas.append(MudancaMapa(self.nome, alvo.nome, celula, tipo, anterior,
              novo, causa))

        resultado = tuple(mudancas)
        if self._eventos is not None and resultado:
            evento = self._eventos.emitir_evento(
                "mapa.lote_aplicado",
                origem=self.nome,
                mapa=self.nome,
                quantidade=len(resultado),
                mudancas=[self._dados_mudanca_evento(mudanca) for mudanca in resultado],
            )
            self._registrar_evento_reativo(evento, "emitido")
        return resultado


def _clonar_causa(causa: Any) -> Any:
    if causa is None:
        return None
    return _clonar_portatil(causa, campo="causa da mudança")


def _bloco_para_dados(bloco: BlocoMapa) -> dict[str, Any]:
    return {"identificador": bloco.identificador, "propriedades":
      dict(bloco.propriedades)}


def _bloco_de_dados(dados: dict[str, Any]) -> BlocoMapa:
    identificador = dados.get("identificador")
    propriedades = dados.get("propriedades", {})
    if not isinstance(identificador, str) or not isinstance(propriedades, dict):
        raise ValueError("payload de BlocoMapa inválido")
    return BlocoMapa(identificador, propriedades)


from coral.persistencia import registrar_adaptador as _registrar_adaptador

_registrar_adaptador(BlocoMapa, "coral.mundo/BlocoMapa", _bloco_para_dados,
  _bloco_de_dados)


__all__ = [
    "BlocoMapa",
    "CamadaMapa",
    "CelulaHexagonal",
    "CelulaMapa",
    "CelulaQuadrada",
    "CelulaVoxel",
    "ConteudoCelula",
    "ContextoLocalMapa",
    "ErroMapaCoral",
    "Mapa",
    "MudancaMapa",
    "ResultadoPassagem",
```

```
]

"""Observação reativa de regiões discretas sem acoplar mapas a entidades."""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Callable, Iterable

from coral.tempo_eventos import Eventos

from .modelo import Mapa
from .regioes import RegiaoMapa

ResolvedorLocalizacao = Callable[[Any, Any], Any]


def _identificador_portatil(alvo: Any, explicito: Any = None) -> Any:
    from coral.persistencia import ErroPersistencia, de_dados, valor_portatil

    valor = explicito
    if valor is None:
        if isinstance(alvo, (str, int, float, bool)) or alvo is None:
            valor = alvo
        else:
            valor = getattr(alvo, "nome", None)
    if valor is None:
        return None
    try:
        return de_dados({"valor": valor_portatil(valor)})["valor"]
    except ErroPersistencia:
        return str(valor)


@dataclass
class ObservadorRegiaoMapa:
    """Reconhece transições de entrada e saída entre avaliações sucessivas."""

    nome: str
    alvo: Any
    mapa: Mapa
    regioes: tuple[RegiaoMapa, ...]
    resolvedor: ResolvedorLocalizacao
    eventos: Eventos
    identificador_alvo: Any = None
    _anteriores: tuple[str, ...] | None = field(default=None, init=False, repr=False)

    def __init__(
        self,
        nome: str,
        alvo: Any,
        mapa: Mapa,
        regioes: Iterable[RegiaoMapa | str],
        resolvedor: ResolvedorLocalizacao,
        eventos: Eventos,
        *,
        identificador_alvo: Any = None,
    ) -> None:
        if not callable(resolvedor):
            raise TypeError("resolvedor de região precisa ser chamável")
        normalizadas: list[RegiaoMapa] = []
```

```
        for item in regioes:
            regiao = mapa.exigir_regiao(item) if isinstance(item, str) else item
            if not isinstance(regiao, RegiaoMapa) or regiao.mapa is not mapa:
                raise ValueError("todas as regiões observadas precisam pertencer ao mapa
                   informado")
            if regiao not in normalizadas:
                normalizadas.append(regiao)
        if not normalizadas:
            raise ValueError("informe ao menos uma região para observação")
        self.nome = str(nome)
        self.alvo = alvo
        self.mapa = mapa
        self.regioes = tuple(normalizadas)
        self.resolvedor = resolvedor
        self.eventos = eventos
        self.identificador_alvo = _identificador_portatil(alvo, identificador_alvo)
        self._anteriores = None

    @property
    def regioes_anteriores(self) -> tuple[str, ...]:
        return () if self._anteriores is None else self._anteriores

    def avaliar(self, contexto: Any) -> bool:
        localizacao = self.resolvedor(self.alvo, contexto)
        atuais = tuple(regiao.nome for regiao in self.regioes if
          regiao.contem(localizacao))
        anteriores = self._anteriores
        self._anteriores = atuais
        if anteriores is None:
            return False

        conjunto_anterior = set(anteriores)
        conjunto_atual = set(atuais)
        saiu = tuple(nome for nome in anteriores if nome not in conjunto_atual)
        entrou = tuple(nome for nome in atuais if nome not in conjunto_anterior)
        disparou = bool(saiu or entrou)

        for nome_regiao in saiu:
            evento = self.eventos.emitir_evento(
                "mapa.regiao_saiu",
                origem=self.mapa.nome,
                mapa=self.mapa.nome,
                regiao=nome_regiao,
                alvo=self.identificador_alvo,
            )
            self.mapa._registrar_evento_reativo(evento, "emitido")
        for nome_regiao in entrou:
            evento = self.eventos.emitir_evento(
                "mapa.regiao_entrou",
                origem=self.mapa.nome,
                mapa=self.mapa.nome,
                regiao=nome_regiao,
                alvo=self.identificador_alvo,
            )
            self.mapa._registrar_evento_reativo(evento, "emitido")
        return disparou


__all__ = ["ObservadorRegiaoMapa", "ResolvedorLocalizacao"]

"""Regiões discretas e geométricas associadas a mapas Coral."""
```

```
from __future__ import annotations

from collections.abc import Iterable, Mapping
from itertools import product
from types import MappingProxyType
from typing import Any

from coral._espacial import Caixa, Circulo, Esfera, Poligono, Regiao2D, Regiao3D,
  Retangulo, ValorEspacial, contem

from .modelo import CelulaMapa, ErroMapaCoral, _copiar_propriedades, _clonar_portatil,
  _coordenada_inteira, _sequencia_coordenadas

_FORMAS_2D = (Retangulo, Circulo, Poligono, Regiao2D)
_FORMAS_3D = (Caixa, Esfera, Regiao3D)


class RegiaoMapa:
    """Região por conjunto de células ou por forma espacial oficial."""

    def __init__(
        self,
        mapa: Any,
        nome: str,
        *,
        celulas: Iterable[object] | None = None,
        forma: Any = None,
        referencia: str = "centro",
        propriedades: Mapping[str, Any] | None = None,
    ) -> None:
        nome_normalizado = str(nome).strip()
        if not nome_normalizado:
            raise ErroMapaCoral("Uma região precisa de um nome não vazio.",
              codigo="mapa.regiao_invalida")
        if (celulas is None) == (forma is None):
            raise ErroMapaCoral(
                "Uma região precisa usar células ou uma forma espacial, mas não as duas
                  ao mesmo tempo.",
                codigo="mapa.regiao_invalida",
            )
        if referencia not in {"centro", "origem"}:
            raise ErroMapaCoral(
                'A referência da região precisa ser "centro" ou "origem".',
                codigo="mapa.regiao_referencia_invalida",
            )
        self.mapa = mapa
        self.nome = nome_normalizado
        self.referencia = referencia
        self._propriedades = _copiar_propriedades(propriedades, campo=f"propriedades da
          região {nome_normalizado}")
        self._forma = None
        self._celulas: tuple[CelulaMapa, ...] = ()
        self._indice: frozenset[CelulaMapa] = frozenset()

        if celulas is not None:
            normalizadas = {mapa.topologia.normalizar(celula) for celula in celulas}
            self._celulas = tuple(sorted(normalizadas, key=lambda celula:
              celula.como_tupla()))
            self._indice = frozenset(self._celulas)
        else:
```

```
            dimensionalidade = getattr(forma, "dimensoes", None)
            permitida = _FORMAS_2D if mapa.topologia.dimensionalidade == 2 else
              _FORMAS_3D
            if dimensionalidade != mapa.topologia.dimensionalidade or not
              isinstance(forma, permitida):
                raise ErroMapaCoral(
                    f'A forma da região "{nome_normalizado}" é incompatível com o mapa
                      {mapa.topologia.dimensionalidade}D.',
                    codigo="mapa.regiao_dimensionalidade_incompativel",
                    contexto={"mapa": mapa.nome, "dimensionalidade":
                      mapa.topologia.dimensionalidade},
                )
            self._forma = forma

    @property
    def tipo(self) -> str:
        return "celulas" if self._forma is None else "forma"

    @property
    def forma(self) -> Any:
        return self._forma

    @property
    def propriedades(self) -> Mapping[str, Any]:
        return MappingProxyType(_clonar_portatil(self._propriedades,
          campo=f"propriedades da região {self.nome}"))

    @property
    def celulas_explicitas(self) -> tuple[CelulaMapa, ...]:
        return self._celulas

    def contem(self, celula_ou_posicao: object) -> bool:
        if isinstance(celula_ou_posicao, ValorEspacial):
            if celula_ou_posicao.dimensoes != self.mapa.topologia.dimensionalidade:
                raise ErroMapaCoral(
                    f'A posição consultada é {celula_ou_posicao.dimensoes}D, mas o mapa
                      "{self.mapa.nome}" é {self.mapa.topologia.dimensionalidade}D.',
                    codigo="mapa.regiao_dimensionalidade_incompativel",
                )
            if self._forma is None:
                celula = self.mapa.topologia.mundo_para_celula(celula_ou_posicao)
                return celula in self._indice
            return bool(contem(self._forma, celula_ou_posicao))

        celula = self.mapa.topologia.normalizar(celula_ou_posicao)
        if self._forma is None:
            return celula in self._indice
        ponto = self.mapa.topologia.celula_para_mundo(celula, ancora=self.referencia)
        return bool(contem(self._forma, ponto))

    def celulas(self, *, limites: object = None) -> tuple[CelulaMapa, ...]:
        if self._forma is None:
            if limites is None:
                return self._celulas
            minimos, maximos = _normalizar_limites_enumeracao(limites,
              self.mapa.topologia)
            return tuple(
                celula
                for celula in self._celulas
                if all(a <= valor <= b for valor, a, b in zip(celula.como_tupla(),
                  minimos, maximos, strict=True))
```

```
            )

        limites_efetivos = limites if limites is not None else
          self.mapa.topologia.limites
        if limites_efetivos is None:
            raise ErroMapaCoral(
                f'A região por forma "{self.nome}" está em um mapa não limitado. Informe
                   limites para enumerar suas células.',
                codigo="mapa.enumeracao_potencialmente_infinita",
            )
        minimos, maximos = _normalizar_limites_enumeracao(limites_efetivos,
          self.mapa.topologia)
        resultado = []
        intervalos = [range(inicio, fim + 1) for inicio, fim in zip(minimos, maximos,
          strict=True)]
        for coordenadas in product(*intervalos):
            try:
                celula = self.mapa.topologia.normalizar(coordenadas)
            except ErroMapaCoral as exc:
                if exc.codigo == "mapa.celula_fora_dos_limites":
                    continue
                raise
            if self.contem(celula):
                resultado.append(celula)
        return tuple(resultado)


def _normalizar_limites_enumeracao(limites: object, topologia: Any) -> tuple[tuple[int,
  ...], tuple[int, ...]]:
    try:
        minimo_bruto, maximo_bruto = tuple(limites)  # type: ignore[misc]
    except (TypeError, ValueError) as exc:
        raise ErroMapaCoral(
            "Limites de enumeração devem usar ((mínimos...), (máximos...)).",
            codigo="mapa.limites_invalidos",
        ) from exc
    dimensoes = topologia.dimensionalidade
    minimos_brutos = _sequencia_coordenadas(minimo_bruto, dimensoes=dimensoes,
      nome_topologia=topologia.nome)
    maximos_brutos = _sequencia_coordenadas(maximo_bruto, dimensoes=dimensoes,
      nome_topologia=topologia.nome)
    minimos = tuple(_coordenada_inteira(v, nome=f"mínimo {i}") for i, v in
      enumerate(minimos_brutos))
    maximos = tuple(_coordenada_inteira(v, nome=f"máximo {i}") for i, v in
      enumerate(maximos_brutos))
    if any(a > b for a, b in zip(minimos, maximos, strict=True)):
        raise ErroMapaCoral("Limites de enumeração estão invertidos.",
          codigo="mapa.limites_invalidos")
    return minimos, maximos


__all__ = ["RegiaoMapa"]

"""Topologias discretas oficiais da Coral 1.5.7."""

from __future__ import annotations

from dataclasses import dataclass, field
import math
from typing import Generic, Iterable, TypeVar
```

```
from coral._espacial import ValorEspacial, posicao

from .modelo import CelulaHexagonal, CelulaQuadrada, CelulaVoxel, ErroMapaCoral,
  _sequencia_coordenadas

C = TypeVar("C", CelulaQuadrada, CelulaHexagonal, CelulaVoxel)


def _numero_positivo_finito(valor: object, *, nome: str) -> float:
    if isinstance(valor, bool) or not isinstance(valor, (int, float)):
        raise ErroMapaCoral(
            f'O parâmetro "{nome}" deve ser um número positivo e finito.',
            codigo="mapa.escala_invalida",
            contexto={"parametro": nome, "valor": valor},
        )
    numero = float(valor)
    if numero <= 0 or not math.isfinite(numero):
        raise ErroMapaCoral(
            f'O parâmetro "{nome}" deve ser maior que zero e finito; recebido
              {valor!r}.',
            codigo="mapa.escala_invalida",
            contexto={"parametro": nome, "valor": valor},
        )
    return numero


def _origem_finita(valores: Iterable[object], *, dimensoes: int) -> tuple[float, ...]:
    try:
        origem = tuple(float(v) for v in valores)
    except (TypeError, ValueError) as exc:
        raise ErroMapaCoral(
            f"A origem da topologia precisa ter {dimensoes} números finitos.",
            codigo="mapa.origem_invalida",
        ) from exc
    if len(origem) != dimensoes or any(not math.isfinite(v) for v in origem):
        raise ErroMapaCoral(
            f"A origem da topologia precisa ter {dimensoes} números finitos.",
            codigo="mapa.origem_invalida",
            contexto={"dimensoes": dimensoes, "origem": origem},
        )
    return origem


def _exigir_posicao_mundo(valor: object, *, dimensoes: int) -> ValorEspacial:
    if not isinstance(valor, ValorEspacial):
        raise ErroMapaCoral(
            "Conversões de mundo exigem uma posição ou ponto espacial oficial da
              Coral.",
            codigo="mapa.coordenada_mundo_invalida",
        )
    if valor.dimensoes != dimensoes:
        raise ErroMapaCoral(
            f"A coordenada de mundo é {valor.dimensoes}D, mas a topologia é
              {dimensoes}D.",
            codigo="mapa.dimensionalidade_incompativel",
        )
    return valor


def _validar_ancora(ancora: str, *, permite_origem: bool) -> str:
    aceitas = {"centro", "origem"} if permite_origem else {"centro"}
```

```
    if ancora not in aceitas:
        opcoes = '"centro" ou "origem"' if permite_origem else '"centro"'
        raise ErroMapaCoral(f"A âncora dessa topologia deve ser {opcoes}.",
          codigo="mapa.ancora_invalida")
    return ancora


@dataclass(frozen=True, slots=True)
class _LimitesInteiros:
    minimos: tuple[int, ...]
    maximos: tuple[int, ...]

    def __post_init__(self) -> None:
        if len(self.minimos) != len(self.maximos):
            raise ErroMapaCoral("Os limites mínimo e máximo usam dimensões diferentes.",
               codigo="mapa.limites_invalidos")
        if any(a > b for a, b in zip(self.minimos, self.maximos, strict=True)):
            raise ErroMapaCoral(
                "Cada limite mínimo deve ser menor ou igual ao limite máximo
                  correspondente.",
                codigo="mapa.limites_invalidos",
                contexto={"minimos": self.minimos, "maximos": self.maximos},
            )

    def contem(self, coordenadas: tuple[int, ...]) -> bool:
        return all(a <= valor <= b for valor, a, b in zip(coordenadas, self.minimos,
          self.maximos, strict=True))


def _criar_limites(limites: object, *, dimensoes: int, nome_topologia: str) ->
  _LimitesInteiros | None:
    if limites is None:
        return None
    try:
        minimo_bruto, maximo_bruto = tuple(limites)  # type: ignore[misc]
    except (TypeError, ValueError) as exc:
        raise ErroMapaCoral(
            "Limites devem usar o formato ((mínimos...), (máximos...)).",
            codigo="mapa.limites_invalidos",
        ) from exc
    minimos = _sequencia_coordenadas(minimo_bruto, dimensoes=dimensoes,
      nome_topologia=nome_topologia)
    maximos = _sequencia_coordenadas(maximo_bruto, dimensoes=dimensoes,
      nome_topologia=nome_topologia)
    from .modelo import _coordenada_inteira

    return _LimitesInteiros(
        tuple(_coordenada_inteira(v, nome=f"mínimo {i}") for i, v in
          enumerate(minimos)),
        tuple(_coordenada_inteira(v, nome=f"máximo {i}") for i, v in
          enumerate(maximos)),
    )


class _TopologiaBase(Generic[C]):
    dimensionalidade: int
    identificador_persistente: str
    nome: str
    _limites: _LimitesInteiros | None

    @property
```

```
    def limitada(self) -> bool:
        return self._limites is not None

    @property
    def limites(self) -> tuple[tuple[int, ...], tuple[int, ...]] | None:
        if self._limites is None:
            return None
        return self._limites.minimos, self._limites.maximos

    def _coordenadas(self, celula: C) -> tuple[int, ...]:
        return celula.como_tupla()

    def dentro_dos_limites(self, celula: object) -> bool:
        normalizada = self._normalizar_sem_limites(celula)
        return self._limites is None or
          self._limites.contem(self._coordenadas(normalizada))

    def normalizar(self, celula: object) -> C:
        normalizada = self._normalizar_sem_limites(celula)
        if self._limites is not None and not
          self._limites.contem(self._coordenadas(normalizada)):
            raise ErroMapaCoral(
                f'A célula {self._coordenadas(normalizada)} está fora dos limites da
                  topologia {self.nome}.',
                codigo="mapa.celula_fora_dos_limites",
                contexto={"topologia": self.nome, "celula":
                  self._coordenadas(normalizada), "limites": self.limites},
            )
        return normalizada

    def validar(self, celula: object) -> bool:
        self.normalizar(celula)
        return True

    def vizinhos(self, celula: object, **opcoes: object) -> tuple[C, ...]:
        raise NotImplementedError

    def adjacentes(self, primeira: object, segunda: object, **opcoes: object) -> bool:
        a = self.normalizar(primeira)
        b = self.normalizar(segunda)
        return b in self.vizinhos(a, **opcoes)

    def _normalizar_sem_limites(self, celula: object) -> C:
        raise NotImplementedError

    def _filtrar_limites(self, celulas: Iterable[C]) -> tuple[C, ...]:
        if self._limites is None:
            return tuple(celulas)
        return tuple(c for c in celulas if self._limites.contem(self._coordenadas(c)))


@dataclass(frozen=True, slots=True)
class TopologiaQuadrada(_TopologiaBase[CelulaQuadrada]):
    largura_celula: float = 1.0
    altura_celula: float = 1.0
    origem: tuple[float, float] = (0.0, 0.0)
    limites_configurados: object = field(default=None, repr=False, compare=False)
    _limites: _LimitesInteiros | None = field(init=False, repr=False, compare=True)

    dimensionalidade = 2
    identificador_persistente = "coral.mundo/topologia-quadrada/1"
```

```
    nome = "quadrada"

    def __post_init__(self) -> None:
        object.__setattr__(self, "largura_celula",
          _numero_positivo_finito(self.largura_celula, nome="largura_celula"))
        object.__setattr__(self, "altura_celula",
          _numero_positivo_finito(self.altura_celula, nome="altura_celula"))
        object.__setattr__(self, "origem", _origem_finita(self.origem, dimensoes=2))
        object.__setattr__(self, "_limites", _criar_limites(self.limites_configurados,
          dimensoes=2, nome_topologia=self.nome))

    def _normalizar_sem_limites(self, celula: object) -> CelulaQuadrada:
        if isinstance(celula, CelulaQuadrada):
            return celula
        if isinstance(celula, (CelulaHexagonal, CelulaVoxel)):
            raise ErroMapaCoral(
                f'A célula {celula.como_tupla()} não pertence a uma topologia quadrada
                  2D.',
                codigo="mapa.celula_topologia_incompativel",
            )
        coluna, linha = _sequencia_coordenadas(celula, dimensoes=2,
          nome_topologia=self.nome)
        return CelulaQuadrada(coluna, linha)  # type: ignore[arg-type]

    def vizinhos(self, celula: object, *, incluir_diagonais: bool = False) ->
      tuple[CelulaQuadrada, ...]:
        c = self.normalizar(celula)
        offsets = ((0, -1), (1, 0), (0, 1), (-1, 0))
        if incluir_diagonais:
            offsets += ((1, -1), (1, 1), (-1, 1), (-1, -1))
        return self._filtrar_limites(CelulaQuadrada(c.coluna + dc, c.linha + dl) for dc,
           dl in offsets)

    def celula_para_mundo(self, celula: object, *, ancora: str = "centro") ->
      ValorEspacial:
        c = self.normalizar(celula)
        _validar_ancora(ancora, permite_origem=True)
        deslocamento = 0.5 if ancora == "centro" else 0.0
        return posicao(
            self.origem[0] + (c.coluna + deslocamento) * self.largura_celula,
            self.origem[1] + (c.linha + deslocamento) * self.altura_celula,
        )

    def mundo_para_celula(self, coordenada: object) -> CelulaQuadrada:
        p = _exigir_posicao_mundo(coordenada, dimensoes=2)
        coluna = math.floor((p.x - self.origem[0]) / self.largura_celula)
        linha = math.floor((p.y - self.origem[1]) / self.altura_celula)
        return self.normalizar((coluna, linha))


@dataclass(frozen=True, slots=True)
class TopologiaHexagonal(_TopologiaBase[CelulaHexagonal]):
    tamanho_celula: float = 1.0
    orientacao: str = "topo_plano"
    origem: tuple[float, float] = (0.0, 0.0)
    limites_configurados: object = field(default=None, repr=False, compare=False)
    _limites: _LimitesInteiros | None = field(init=False, repr=False, compare=True)

    dimensionalidade = 2
    identificador_persistente = "coral.mundo/topologia-hexagonal/1"
    nome = "hexagonal"
```

```
    def __post_init__(self) -> None:
        object.__setattr__(self, "tamanho_celula",
          _numero_positivo_finito(self.tamanho_celula, nome="tamanho_celula"))
        if self.orientacao not in {"topo_plano", "topo_pontudo"}:
            raise ErroMapaCoral(
                'A orientação hexagonal deve ser "topo_plano" ou "topo_pontudo".',
                codigo="mapa.orientacao_hexagonal_invalida",
                contexto={"orientacao": self.orientacao},
            )
        object.__setattr__(self, "origem", _origem_finita(self.origem, dimensoes=2))
        object.__setattr__(self, "_limites", _criar_limites(self.limites_configurados,
          dimensoes=2, nome_topologia=self.nome))

    def _normalizar_sem_limites(self, celula: object) -> CelulaHexagonal:
        if isinstance(celula, CelulaHexagonal):
            return celula
        if isinstance(celula, (CelulaQuadrada, CelulaVoxel)):
            raise ErroMapaCoral(
                f'A célula {celula.como_tupla()} não pertence a uma topologia hexagonal
                  axial.',
                codigo="mapa.celula_topologia_incompativel",
            )
        q, r = _sequencia_coordenadas(celula, dimensoes=2, nome_topologia=self.nome)
        return CelulaHexagonal(q, r)  # type: ignore[arg-type]

    def vizinhos(self, celula: object, **_: object) -> tuple[CelulaHexagonal, ...]:
        c = self.normalizar(celula)
        offsets = ((1, 0), (1, -1), (0, -1), (-1, 0), (-1, 1), (0, 1))
        return self._filtrar_limites(CelulaHexagonal(c.q + dq, c.r + dr) for dq, dr in
          offsets)

    def celula_para_mundo(self, celula: object, *, ancora: str = "centro") ->
      ValorEspacial:
        c = self.normalizar(celula)
        _validar_ancora(ancora, permite_origem=False)
        s = self.tamanho_celula
        if self.orientacao == "topo_pontudo":
            x = s * math.sqrt(3.0) * (c.q + c.r / 2.0)
            y = s * 1.5 * c.r
        else:
            x = s * 1.5 * c.q
            y = s * math.sqrt(3.0) * (c.r + c.q / 2.0)
        return posicao(self.origem[0] + x, self.origem[1] + y)

    def mundo_para_celula(self, coordenada: object) -> CelulaHexagonal:
        p = _exigir_posicao_mundo(coordenada, dimensoes=2)
        x = (p.x - self.origem[0]) / self.tamanho_celula
        y = (p.y - self.origem[1]) / self.tamanho_celula
        if self.orientacao == "topo_pontudo":
            q = math.sqrt(3.0) / 3.0 * x - y / 3.0
            r = 2.0 / 3.0 * y
        else:
            q = 2.0 / 3.0 * x
            r = -x / 3.0 + math.sqrt(3.0) / 3.0 * y
        rq, rr = _arredondar_axial(q, r)
        return self.normalizar((rq, rr))


@dataclass(frozen=True, slots=True)
class TopologiaVoxel(_TopologiaBase[CelulaVoxel]):
```

```
    tamanho_x: float = 1.0
    tamanho_y: float = 1.0
    tamanho_z: float = 1.0
    origem: tuple[float, float, float] = (0.0, 0.0, 0.0)
    limites_configurados: object = field(default=None, repr=False, compare=False)
    _limites: _LimitesInteiros | None = field(init=False, repr=False, compare=True)

    dimensionalidade = 3
    identificador_persistente = "coral.mundo/topologia-voxel/1"
    nome = "voxel"

    def __post_init__(self) -> None:
        object.__setattr__(self, "tamanho_x", _numero_positivo_finito(self.tamanho_x,
          nome="tamanho_x"))
        object.__setattr__(self, "tamanho_y", _numero_positivo_finito(self.tamanho_y,
          nome="tamanho_y"))
        object.__setattr__(self, "tamanho_z", _numero_positivo_finito(self.tamanho_z,
          nome="tamanho_z"))
        object.__setattr__(self, "origem", _origem_finita(self.origem, dimensoes=3))
        object.__setattr__(self, "_limites", _criar_limites(self.limites_configurados,
          dimensoes=3, nome_topologia=self.nome))

    def _normalizar_sem_limites(self, celula: object) -> CelulaVoxel:
        if isinstance(celula, CelulaVoxel):
            return celula
        if isinstance(celula, (CelulaQuadrada, CelulaHexagonal)):
            raise ErroMapaCoral(
                f'A célula {celula.como_tupla()} não pertence a uma topologia voxel
                  3D.',
                codigo="mapa.celula_topologia_incompativel",
            )
        x, y, z = _sequencia_coordenadas(celula, dimensoes=3, nome_topologia=self.nome)
        return CelulaVoxel(x, y, z)  # type: ignore[arg-type]

    def vizinhos(self, celula: object, **_: object) -> tuple[CelulaVoxel, ...]:
        c = self.normalizar(celula)
        offsets = ((1, 0, 0), (-1, 0, 0), (0, 1, 0), (0, -1, 0), (0, 0, 1), (0, 0, -1))
        return self._filtrar_limites(CelulaVoxel(c.x + dx, c.y + dy, c.z + dz) for dx,
          dy, dz in offsets)

    def celula_para_mundo(self, celula: object, *, ancora: str = "centro") ->
      ValorEspacial:
        c = self.normalizar(celula)
        _validar_ancora(ancora, permite_origem=True)
        deslocamento = 0.5 if ancora == "centro" else 0.0
        return posicao(
            self.origem[0] + (c.x + deslocamento) * self.tamanho_x,
            self.origem[1] + (c.y + deslocamento) * self.tamanho_y,
            self.origem[2] + (c.z + deslocamento) * self.tamanho_z,
        )

    def mundo_para_celula(self, coordenada: object) -> CelulaVoxel:
        p = _exigir_posicao_mundo(coordenada, dimensoes=3)
        return self.normalizar(
            (
                math.floor((p.x - self.origem[0]) / self.tamanho_x),
                math.floor((p.y - self.origem[1]) / self.tamanho_y),
                math.floor((p.z - self.origem[2]) / self.tamanho_z),
            )
        )
```

```
def _arredondar_axial(q: float, r: float) -> tuple[int, int]:
    x, z = q, r
    y = -x - z
    rx, ry, rz = round(x), round(y), round(z)
    dx, dy, dz = abs(rx - x), abs(ry - y), abs(rz - z)
    if dx > dy and dx > dz:
        rx = -ry - rz
    elif dy > dz:
        ry = -rx - rz
    else:
        rz = -rx - ry
    return int(rx), int(rz)


__all__ = ["TopologiaHexagonal", "TopologiaQuadrada", "TopologiaVoxel"]

from __future__ import annotations

from collections import deque
from dataclasses import dataclass, field
import json
from typing import Any, Iterable

from ._mapas import (
    BlocoMapa,
    CamadaMapa,
    CelulaHexagonal,
    CelulaQuadrada,
    CelulaVoxel,
    ConteudoCelula,
    ErroMapaCoral,
    Mapa,
    MudancaMapa,
    ObservadorRegiaoMapa,
    RegiaoMapa,
    ResultadoPassagem,
    TopologiaHexagonal,
    TopologiaQuadrada,
    TopologiaVoxel,
)
from .regras import MotorRegras
from .tempo_eventos import Eventos, FonteTempo


@dataclass(eq=False)
class Entidade:
    nome: str
    atributos: dict[str, Any] = field(default_factory=dict)
    tags: set[str] = field(default_factory=set)
    inventario: list[Any] = field(default_factory=list)

    def definir(self, nome: str, valor: Any) -> "Entidade":
        self.atributos[str(nome)] = valor
        return self

    def obter(self, nome: str, padrao: Any = None) -> Any:
        return self.atributos.get(str(nome), padrao)

    def adicionar_tag(self, tag: str) -> "Entidade":
        self.tags.add(str(tag))
```

```
        return self

    def remover_tag(self, tag: str) -> None:
        self.tags.discard(str(tag))

    def possui_tag(self, tag: str) -> bool:
        return str(tag) in self.tags

    def guardar(self, item: Any) -> Any:
        self.inventario.append(item)
        return item

    def remover_item(self, item: Any) -> bool:
        try:
            self.inventario.remove(item)
            return True
        except ValueError:
            return False

    def __getitem__(self, chave: str) -> Any:
        return self.atributos[chave]

    def __setitem__(self, chave: str, valor: Any) -> None:
        self.atributos[chave] = valor

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "nome": self.nome,
            "atributos": dict(self.atributos),
            "tags": tuple(sorted(self.tags)),
            "inventario": tuple(self.inventario),
            "quantidade_itens": len(self.inventario),
        }

    def __repr__(self) -> str:
        return f"Entidade({self.nome!r})"


@dataclass(frozen=True)
class Relacao:
    origem: Entidade
    destino: Entidade
    tipo: str
    dados: dict[str, Any] = field(default_factory=dict, compare=False)


class Mundo:
    """Mundo leve para jogos, RPGs e simulações Coral.

    A API mantém entidades e relações como um grafo dirigido. Relações podem
    ser consultadas nos dois sentidos e o método ``caminho`` usa BFS.
    """

    def __init__(self, nome: str = "mundo", *, relogio: FonteTempo | None = None,
      eventos: Eventos | None = None):
        self.nome = str(nome)
        self._entidades: dict[str, Entidade] = {}
        self._relacoes: list[Relacao] = []
        self._mapas: dict[str, Mapa] = {}
        self.regras = MotorRegras(relogio=relogio, eventos=eventos)
        self._simulacao = None
```

```
    @property
    def entidades(self) -> tuple[Entidade, ...]:
        return tuple(self._entidades.values())

    @property
    def relacoes(self) -> tuple[Relacao, ...]:
        return tuple(self._relacoes)

    @property
    def mapas(self) -> tuple[Mapa, ...]:
        return tuple(self._mapas.values())

    @property
    def simulacao(self):
        """Simulação temporal associada, quando criada explicitamente."""
        return self._simulacao

    def criar_simulacao(self, *, tempo=None):
        """Cria ou devolve a simulação temporal do mundo sem exigir Jogos."""
        from .simulacao import Simulacao, TempoMundo
        from .tempo_eventos import RelogioSimulado

        if self._simulacao is not None:
            if tempo is not None and self._simulacao.tempo is not tempo:
                raise ValueError("o mundo já possui uma simulação com outro tempo")
            return self._simulacao
        if tempo is None:
            relogio_existente = self.regras.relogio
            if isinstance(relogio_existente, RelogioSimulado):
                tempo = TempoMundo(eventos=self.regras.eventos,
                  relogio=relogio_existente)
            else:
                tempo = TempoMundo(self.regras.instante_atual(),
                  eventos=self.regras.eventos)
        if not isinstance(tempo, TempoMundo):
            raise TypeError("tempo precisa ser TempoMundo")
        self.regras.relogio = tempo.relogio
        self._simulacao = Simulacao(tempo=tempo)
        return self._simulacao

    def avancar_tempo(self, duracao: float):
        """Avança a simulação e depois atualiza regras e temporizadores pelo mesmo
          delta."""
        simulacao = self.criar_simulacao()
        inicio = simulacao.tempo.instante
        resultados = simulacao.avancar(duracao, contexto=self)
        aplicado = simulacao.tempo.instante - inicio
        self.regras.atualizar(self, aplicado)
        return resultados

    def atualizar_simulacao(self, dt_externo: float):
        """Atualiza a simulação respeitando pausa e escala temporal."""
        simulacao = self.criar_simulacao()
        inicio = simulacao.tempo.instante
        resultados = simulacao.atualizar(dt_externo, contexto=self)
        aplicado = simulacao.tempo.instante - inicio
        if aplicado:
            self.regras.atualizar(self, aplicado)
        return resultados
```

```
    def adicionar_mapa(self, mapa: Mapa) -> Mapa:
        if not isinstance(mapa, Mapa):
            raise ErroMapaCoral("Mundo só pode adicionar objetos Mapa.",
              codigo="mapa.mapa_invalido")
        existente = self._mapas.get(mapa.nome)
        if existente is not None and existente is not mapa:
            raise ErroMapaCoral(
                f'Já existe um mapa chamado "{mapa.nome}" no mundo "{self.nome}".',
                codigo="mapa.mapa_duplicado",
                contexto={"mundo": self.nome, "mapa": mapa.nome},
            )
        self._mapas[mapa.nome] = mapa
        mapa._vincular_runtime(self.regras.eventos, self.regras.rastreamento)
        return mapa

    def criar_mapa(self, nome: str, topologia, *, propriedades: dict[str, Any] | None =
      None) -> Mapa:
        return self.adicionar_mapa(Mapa(nome, topologia, propriedades=propriedades))

    def buscar_mapa(self, nome: str) -> Mapa | None:
        return self._mapas.get(str(nome))

    def exigir_mapa(self, nome: str) -> Mapa:
        mapa = self.buscar_mapa(nome)
        if mapa is None:
            raise ErroMapaCoral(
                f'O mapa "{nome}" não existe no mundo "{self.nome}".',
                codigo="mapa.mapa_inexistente",
                contexto={"mundo": self.nome, "mapa": str(nome)},
            )
        return mapa

    def remover_mapa(self, mapa: Mapa | str) -> bool:
        nome = mapa if isinstance(mapa, str) else mapa.nome
        existente = self._mapas.get(str(nome))
        if existente is None or (isinstance(mapa, Mapa) and existente is not mapa):
            return False
        del self._mapas[str(nome)]
        existente._vincular_runtime(None, None)
        return True

    def observar_regioes_mapa(
        self,
        nome: str,
        alvo: Any,
        mapa: Mapa | str,
        regioes,
        resolvedor,
        *,
        identificador_alvo: Any = None,
    ) -> ObservadorRegiaoMapa:
        mapa_real = self.exigir_mapa(mapa) if isinstance(mapa, str) else mapa
        if self._mapas.get(mapa_real.nome) is not mapa_real:
            raise ErroMapaCoral(
                "O mapa observado precisa pertencer ao mundo.",
                codigo="mapa.mapa_inexistente",
                contexto={"mundo": self.nome, "mapa": mapa_real.nome},
            )
        return self.regras.observar_regioes_mapa(
            nome, alvo, mapa_real, regioes, resolvedor,
              identificador_alvo=identificador_alvo
```

```
        )

    def adicionar(self, entidade: Entidade) -> Entidade:
        if entidade.nome in self._entidades and self._entidades[entidade.nome] is not
          entidade:
            raise ValueError(f"já existe uma entidade chamada {entidade.nome!r}")
        self._entidades[entidade.nome] = entidade
        if hasattr(entidade, "_definir_mundo"):
            entidade._definir_mundo(self)
        return entidade

    def criar_entidade(self, nome: str, **atributos: Any) -> Entidade:
        entidade = Entidade(str(nome), dict(atributos))
        return self.adicionar(entidade)

    def buscar(self, nome: str) -> Entidade | None:
        return self._entidades.get(str(nome))

    def exigir(self, nome: str) -> Entidade:
        entidade = self.buscar(nome)
        if entidade is None:
            raise KeyError(f"entidade não encontrada: {nome}")
        return entidade

    def remover(self, entidade: Entidade | str) -> bool:
        if isinstance(entidade, str):
            entidade = self.buscar(entidade)
        if entidade is None or self._entidades.get(entidade.nome) is not entidade:
            return False
        del self._entidades[entidade.nome]
        self._relacoes[:] = [r for r in self._relacoes if r.origem is not entidade and
          r.destino is not entidade]
        if hasattr(entidade, "_definir_mundo"):
            entidade._definir_mundo(None)
        return True

    def relacionar(self, origem: Entidade | str, destino: Entidade | str, tipo: str,
      **dados: Any) -> Relacao:
        if isinstance(origem, str): origem = self.exigir(origem)
        if isinstance(destino, str): destino = self.exigir(destino)
        if self._entidades.get(origem.nome) is not origem or
          self._entidades.get(destino.nome) is not destino:
            raise ValueError("as duas entidades precisam pertencer ao mundo")
        relacao = Relacao(origem, destino, str(tipo), dict(dados))
        self._relacoes.append(relacao)
        return relacao

    def remover_relacao(self, relacao: Relacao) -> bool:
        try:
            self._relacoes.remove(relacao)
            return True
        except ValueError:
            return False

    def relacionados(self, entidade: Entidade | str, tipo: str | None = None, *,
      direcao: str = "saida") -> tuple[Entidade, ...]:
        if isinstance(entidade, str): entidade = self.exigir(entidade)
        direcao = direcao.casefold()
        if direcao not in {"saida", "entrada", "ambas"}:
            raise ValueError("direção deve ser 'saida', 'entrada' ou 'ambas'")
        resultado=[]
```

```
        for r in self._relacoes:
            if tipo is not None and r.tipo != tipo:
                continue
            if direcao in {"saida", "ambas"} and r.origem is entidade:
                resultado.append(r.destino)
            if direcao in {"entrada", "ambas"} and r.destino is entidade:
                resultado.append(r.origem)
        # preserva ordem e remove duplicatas por identidade
        vistos=set(); unicos=[]
        for e in resultado:
            if id(e) not in vistos:
                vistos.add(id(e)); unicos.append(e)
        return tuple(unicos)

    def caminho(self, origem: Entidade | str, destino: Entidade | str, *, tipo: str |
      None = None, bidirecional: bool = True) -> tuple[Entidade, ...]:
        if isinstance(origem, str): origem = self.exigir(origem)
        if isinstance(destino, str): destino = self.exigir(destino)
        if origem is destino:
            return (origem,)
        fila=deque([origem]); anterior={id(origem):None}; por_id={id(origem):origem}
        while fila:
            atual=fila.popleft()
            vizinhos=self.relacionados(atual,tipo,direcao="ambas" if bidirecional else
              "saida")
            for vizinho in vizinhos:
                chave=id(vizinho)
                if chave in anterior: continue
                anterior[chave]=id(atual); por_id[chave]=vizinho
                if vizinho is destino:
                    ids=[]; cursor=chave
                    while cursor is not None:
                        ids.append(cursor); cursor=anterior[cursor]
                    return tuple(por_id[i] for i in reversed(ids))
                fila.append(vizinho)
        return ()

    def com_tag(self, tag: str) -> tuple[Entidade, ...]:
        return tuple(e for e in self._entidades.values() if e.possui_tag(tag))

    def atualizar(self, dt: float = 0.0) -> tuple[str, ...]:
        return self.regras.atualizar(self, dt)

    def atualizar_por_relogio(self) -> tuple[str, ...]:
        return self.regras.atualizar_por_relogio(self)

    def emitir(self, evento: str, *args, **kwargs) -> int:
        return self.regras.eventos.emitir(evento, *args, **kwargs)

    def emitir_evento(
        self,
        evento: str,
        *args,
        cancelavel: bool = False,
        metadados: dict[str, Any] | None = None,
        **kwargs,
    ):
        return self.regras.eventos.emitir_evento(
            evento,
            *args,
            origem=self.nome,
```

```
            cancelavel=cancelavel,
            metadados=metadados,
            **kwargs,
        )

    def enfileirar_evento(self, evento: str, *args, **kwargs):
        return self.regras.eventos.enfileirar(evento, *args, origem=self.nome, **kwargs)

    def processar_eventos(self, limite: int | None = None):
        return self.regras.eventos.processar(limite)

    def emitir_propagado(self, evento: str, *args, **kwargs) -> int:
        return self.regras.eventos.emitir_propagado(evento, *args, origem=self.nome,
          **kwargs)

    def emitir_evento_propagado(
        self,
        evento: str,
        *args,
        cancelavel: bool = False,
        metadados: dict[str, Any] | None = None,
        **kwargs,
    ):
        return self.regras.eventos.emitir_evento_propagado(
            evento,
            *args,
            origem=self.nome,
            cancelavel=cancelavel,
            metadados=metadados,
            **kwargs,
        )

    def estado_inspecao(self) -> dict[str, Any]:
        simulacao = self._simulacao
        return {
            "nome": self.nome,
            "quantidade_entidades": len(self._entidades),
            "entidades": tuple(self._entidades.values()),
            "quantidade_relacoes": len(self._relacoes),
            "relacoes": tuple(
                {
                    "origem": relacao.origem.nome,
                    "destino": relacao.destino.nome,
                    "tipo": relacao.tipo,
                    "dados": dict(relacao.dados),
                }
                for relacao in self._relacoes
            ),
            "mapas": tuple(self._mapas),
            "simulacao_ativa": simulacao is not None,
            "regras": self.regras,
        }

    def para_dict(self) -> dict[str, Any]:
        return {
            "nome": self.nome,
            "entidades": [
                {"nome": e.nome, "atributos": dict(e.atributos), "tags": sorted(e.tags),
                   "inventario": list(e.inventario)}
                for e in self._entidades.values()
            ],
```

```
            "relacoes": [
                {"origem": r.origem.nome, "destino": r.destino.nome, "tipo": r.tipo,
                  "dados": dict(r.dados)}
                for r in self._relacoes
            ],
        }

    def para_json(self, **kwargs) -> str:
        opcoes={"ensure_ascii":False,"indent":2}; opcoes.update(kwargs)
        return json.dumps(self.para_dict(), **opcoes)

    def __iter__(self):
        return iter(self._entidades.values())

    def __len__(self) -> int:
        return len(self._entidades)


def criar_mundo(nome: str = "mundo") -> Mundo:
    return Mundo(nome)


__all__ = [
    "Mundo", "Entidade", "Relacao", "criar_mundo",
    "Mapa", "CamadaMapa", "ConteudoCelula", "BlocoMapa", "MudancaMapa", "RegiaoMapa",
      "ResultadoPassagem", "ObservadorRegiaoMapa",
    "CelulaQuadrada", "CelulaHexagonal", "CelulaVoxel",
    "TopologiaQuadrada", "TopologiaHexagonal", "TopologiaVoxel", "ErroMapaCoral",
]


# Adaptadores de persistência são registrados no módulo do domínio para manter
# o núcleo ``coral.persistencia`` desacoplado de Mundo.
def _entidade_para_dados(entidade: Entidade) -> dict[str, Any]:
    return {
        "nome": entidade.nome,
        "atributos": dict(entidade.atributos),
        "tags": sorted(entidade.tags),
        "inventario": list(entidade.inventario),
    }


def _entidade_de_dados(dados: dict[str, Any]) -> Entidade:
    if not isinstance(dados.get("nome"), str):
        raise ValueError("nome de entidade inválido")
    atributos = dados.get("atributos", {})
    tags = dados.get("tags", [])
    inventario = dados.get("inventario", [])
    if not isinstance(atributos, dict) or not isinstance(tags, list) or not
      isinstance(inventario, list):
        raise ValueError("payload de entidade inválido")
    entidade = Entidade(dados["nome"], dict(atributos))
    entidade.tags.update(str(x) for x in tags)
    entidade.inventario.extend(inventario)
    return entidade


def _mundo_para_dados(mundo: Mundo) -> dict[str, Any]:
    from .persistencia import extensoes_para_dados, para_dados
    dados = {
        "nome": mundo.nome,
```

```
        "entidades": [para_dados(e) for e in mundo.entidades],
        "relacoes": [
            {"origem": r.origem.nome, "destino": r.destino.nome, "tipo": r.tipo,
              "dados": dict(r.dados)}
            for r in mundo.relacoes
        ],
    }
    if mundo.mapas:
        dados["mapas"] = [para_dados(mapa) for mapa in mundo.mapas]
    extensoes = extensoes_para_dados(mundo)
    if extensoes:
        dados["extensoes"] = extensoes
    return dados


def _mundo_de_dados(dados: dict[str, Any]) -> Mundo:
    from .persistencia import ErroPersistencia
    nome = dados.get("nome")
    entidades = dados.get("entidades", [])
    relacoes = dados.get("relacoes", [])
    mapas = dados.get("mapas", [])
    extensoes = dados.get("extensoes", {})
    if not isinstance(nome, str) or not isinstance(entidades, list) or not
      isinstance(relacoes, list) or not isinstance(mapas, list) or not
      isinstance(extensoes, dict):
        raise ErroPersistencia("Payload de Mundo inválido.")
    mundo = Mundo(nome)
    for entidade in entidades:
        if not isinstance(entidade, Entidade):
            raise ErroPersistencia("Mundo contém item que não é Entidade.")
        mundo.adicionar(entidade)
    for indice, relacao in enumerate(relacoes):
        if not isinstance(relacao, dict):
            raise ErroPersistencia(f"Relação inválida no índice {indice}.")
        origem = mundo.buscar(relacao.get("origem")) if
          isinstance(relacao.get("origem"), str) else None
        destino = mundo.buscar(relacao.get("destino")) if
          isinstance(relacao.get("destino"), str) else None
        if origem is None or destino is None:
            raise ErroPersistencia(f"Relação quebrada no índice {indice}: origem ou
              destino inexistente.")
        tipo = relacao.get("tipo")
        rel_dados = relacao.get("dados", {})
        if not isinstance(tipo, str) or not isinstance(rel_dados, dict):
            raise ErroPersistencia(f"Relação inválida no índice {indice}.")
        mundo.relacionar(origem, destino, tipo, **rel_dados)

    for indice, mapa in enumerate(mapas):
        if not isinstance(mapa, Mapa):
            raise ErroPersistencia(f"Mapa inválido no índice {indice} do Mundo.")
        mundo.adicionar_mapa(mapa)

    from .persistencia import aplicar_extensoes_persistentes
    aplicar_extensoes_persistentes(mundo, extensoes)
    return mundo


from .persistencia import registrar_adaptador as _registrar_adaptador
_registrar_adaptador(Entidade, "coral.mundo/Entidade", _entidade_para_dados,
  _entidade_de_dados)
_registrar_adaptador(Mundo, "coral.mundo/Mundo", _mundo_para_dados, _mundo_de_dados)
```

```
from __future__ import annotations

from dataclasses import dataclass
import math
from typing import Any, Callable

from .tempo_eventos import Evento, Eventos, FonteTempo, normalizar_dt, validar_duracao
from .rastreamento_reativo import ErroCicloReativo, RastreamentoReativo
from .observadores_reativos import ObservadorEspacial, ObservadorMudanca
from .sequencias_reativas import SequenciaDoisPassos


Condicao = Callable[[Any], bool]
Acao = Callable[[Any], Any]
MODOS_REGRA = frozenset({"ao_ativar", "sempre", "uma_vez"})


_PRIORIDADES_REGRA = {
    "baixa": 0,
    "normal": 100,
    "alta": 200,
    "critica": 300,
    "crítica": 300,
}


def _normalizar_prioridade_regra(prioridade: str | int) -> int:
    if isinstance(prioridade, bool):
        raise ValueError("prioridade de regra desconhecida: use baixa, normal, alta ou
          crítica")
    if isinstance(prioridade, int):
        return prioridade
    nome = str(prioridade).strip().casefold()
    if nome not in _PRIORIDADES_REGRA:
        raise ValueError(
            f"prioridade de regra desconhecida: {prioridade!r}; use baixa, normal, alta
              ou crítica"
        )
    return _PRIORIDADES_REGRA[nome]


@dataclass
class Regra:
    nome: str
    condicao: Condicao
    acao: Acao
    modo: str = "ao_ativar"
    habilitada: bool = True
    ativa: bool = False
    execucoes: int = 0
    prioridade: str | int = "normal"
    max_execucoes: int | None = None
    cooldown: float = 0.0
    ultimo_disparo: float | None = None
    motivo_ultima_supressao: str | None = None
    ultima_condicao: bool | None = None

    def __post_init__(self):
        self.prioridade = _normalizar_prioridade_regra(self.prioridade)
        if self.max_execucoes is not None:
```

```
            if (
                isinstance(self.max_execucoes, bool)
                or not isinstance(self.max_execucoes, int)
                or self.max_execucoes <= 0
            ):
                raise ValueError("máximo de execuções deve ser um inteiro positivo ou
                  nulo")
        try:
            self.cooldown = float(validar_duracao(self.cooldown, "cooldown"))
        except ValueError as exc:
            raise ValueError("cooldown deve ser um número finito maior ou igual a zero")
               from exc

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "nome": self.nome,
            "habilitada": self.habilitada,
            "ativa": self.ativa,
            "modo": self.modo,
            "prioridade": self.prioridade,
            "execucoes": self.execucoes,
            "max_execucoes": self.max_execucoes,
            "cooldown": self.cooldown,
            "ultimo_disparo": self.ultimo_disparo,
            "ultima_condicao": self.ultima_condicao,
            "motivo_ultima_supressao": self.motivo_ultima_supressao,
        }

    def avaliar(self, contexto: Any, *, instante: float | None = None) -> bool:
        self.motivo_ultima_supressao = None
        if not self.habilitada:
            self.motivo_ultima_supressao = "desabilitada"
            return False
        agora = bool(self.condicao(contexto))
        self.ultima_condicao = agora
        executar = False
        if self.modo == "sempre":
            executar = agora
        elif self.modo == "uma_vez":
            executar = agora and self.execucoes == 0
        elif self.modo == "ao_ativar":
            executar = agora and not self.ativa
        else:
            raise ValueError(f"modo de regra desconhecido: {self.modo}")
        self.ativa = agora
        if not executar:
            self.motivo_ultima_supressao = "condicao_inativa" if not agora else "modo"
            return False
        if self.max_execucoes is not None and self.execucoes >= self.max_execucoes:
            self.motivo_ultima_supressao = "limite_execucoes"
            return False
        if self.cooldown > 0 and self.ultimo_disparo is not None:
            if instante is None:
                raise ValueError("regra com cooldown exige instante lógico")
            decorrido = float(instante) - self.ultimo_disparo
            if decorrido < self.cooldown and not math.isclose(decorrido, self.cooldown,
              rel_tol=1e-12, abs_tol=1e-15):
                self.motivo_ultima_supressao = "cooldown"
                return False
        self.acao(contexto)
        self.execucoes += 1
```

```
        if instante is not None:
            self.ultimo_disparo = float(instante)
        return True


@dataclass(frozen=True)
class GrupoRegras:
    nome: str
    regras: tuple[Regra, ...]

    def habilitar(self) -> None:
        for regra in self.regras:
            regra.habilitada = True

    def desabilitar(self) -> None:
        for regra in self.regras:
            regra.habilitada = False


@dataclass
class Debounce:
    intervalo: float
    acao: Acao
    nome: str = "debounce"
    restante: float | None = None
    pendente: bool = False
    contexto_mais_recente: Any = None

    def __post_init__(self):
        try:
            self.intervalo = float(validar_duracao(self.intervalo, "intervalo de
              debounce"))
        except ValueError as exc:
            raise ValueError("intervalo de debounce deve ser um número finito maior ou
              igual a zero") from exc

    def notificar(self, contexto: Any = None) -> None:
        self.contexto_mais_recente = contexto
        self.restante = self.intervalo
        self.pendente = True

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "nome": self.nome,
            "intervalo": self.intervalo,
            "restante": self.restante,
            "pendente": self.pendente,
        }

    def atualizar(self, dt: float) -> bool:
        if not self.pendente:
            return False
        passo = normalizar_dt(dt, compatibilidade_negativo_zero=True)
        restante = self.intervalo if self.restante is None else float(self.restante)
        if passo < restante and not math.isclose(passo, restante, rel_tol=1e-12,
          abs_tol=1e-15):
            self.restante = restante - passo
            return False
        contexto = self.contexto_mais_recente
        self.pendente = False
        self.restante = None
```

```
        self.acao(contexto)
        return True


@dataclass
class _AtivacaoRegraDurante:
    regra: Regra
    fim: float


@dataclass
class _AtivacaoRegraAte:
    regra: Regra
    condicao: Condicao


@dataclass
class Temporizador:
    intervalo: float
    acao: Acao
    repetir: bool = False
    nome: str = "temporizador"
    restante: float | None = None
    ativo: bool = True

    def __post_init__(self):
        try:
            self.intervalo = float(validar_duracao(self.intervalo, "intervalo",
              permitir_zero=not self.repetir))
        except ValueError as exc:
            # Preserva mensagens públicas da linha 1.3 para os contratos antigos.
            if self.repetir and "maior que zero" in str(exc):
                raise ValueError("intervalo repetido deve ser maior que zero") from exc
            if "finito" in str(exc) and not str(exc).endswith("maior que zero"):
                raise ValueError("intervalo deve ser um número finito") from exc
            raise ValueError("intervalo não pode ser negativo") from exc
        if self.restante is None:
            self.restante = self.intervalo
        else:
            try:
                self.restante = float(validar_duracao(self.restante, "tempo restante"))
            except ValueError as exc:
                raise ValueError("tempo restante deve ser um número finito") from exc

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "nome": self.nome,
            "intervalo": self.intervalo,
            "restante": self.restante,
            "repetir": self.repetir,
            "ativo": self.ativo,
        }

    def atualizar(self, contexto: Any, dt: float) -> int:
        if not self.ativo:
            return 0
        # Compatibilidade 1.2: avanços negativos não voltam o relógio do motor.
        passo = normalizar_dt(dt, compatibilidade_negativo_zero=True)
        restante_antes = float(self.restante)
        if passo < restante_antes:
            self.restante = restante_antes - passo
```

```
            return 0

        if not self.repetir:
            self.acao(contexto)
            self.restante = max(0.0, restante_antes - passo)
            self.ativo = False
            return 1

        # Calcula os intervalos vencidos de uma só vez. nextafter corrige o caso
        # 0.1 + 0.1 + 0.1, que na 1.3.6 podia perder o disparo do limite exato.
        excedente = passo - restante_antes
        quociente = excedente / self.intervalo
        adicionais = math.floor(math.nextafter(quociente, math.inf))
        previstos = 1 + max(0, adicionais)
        disparos = 0
        for _ in range(previstos):
            if not self.ativo:
                break
            self.acao(contexto)
            disparos += 1
        if self.ativo:
            self.restante = restante_antes - passo + disparos * self.intervalo
            # Evita expor resíduos negativos minúsculos depois de um limite exato.
            if math.isclose(self.restante, self.intervalo, rel_tol=1e-12, abs_tol=0.0):
                self.restante = self.intervalo
            elif self.restante < 0 and math.isclose(self.restante, 0.0, rel_tol=0.0,
              abs_tol=math.ulp(max(1.0, abs(passo))) * 4):
                self.restante = 0.0
        return disparos


class MotorRegras:
    def __init__(
        self,
        *,
        relogio: FonteTempo | None = None,
        eventos: Eventos | None = None,
        rastreamento: RastreamentoReativo | str | None = None,
    ):
        self.regras: list[Regra] = []
        self.temporizadores: list[Temporizador] = []
        self.debounces: list[Debounce] = []
        self.observadores: list[ObservadorMudanca | ObservadorEspacial] = []
        self.grupos: dict[str, GrupoRegras] = {}
        self.sequencias: list[SequenciaDoisPassos] = []
        self._ativacoes_durante: list[_AtivacaoRegraDurante] = []
        self._ativacoes_ate: list[_AtivacaoRegraAte] = []
        self.relogio = relogio
        self._tempo_logico = float(relogio.agora()) if relogio is not None else 0.0
        self.eventos = eventos if eventos is not None else Eventos(relogio=relogio)
        self.eventos.definir_fonte_instante(self._instante_atual)
        if rastreamento is None:
            self.rastreamento = RastreamentoReativo("desligado")
        elif isinstance(rastreamento, str):
            self.rastreamento = RastreamentoReativo(rastreamento)
        else:
            self.rastreamento = rastreamento

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "tempo_logico": self._tempo_logico,
```

```
            "regras": tuple(self.regras),
            "quantidade_regras": len(self.regras),
            "quantidade_temporizadores": len(self.temporizadores),
            "temporizadores": tuple(self.temporizadores),
            "quantidade_debounces": len(self.debounces),
            "debounces": tuple(self.debounces),
            "quantidade_observadores": len(self.observadores),
            "quantidade_sequencias": len(self.sequencias),
            "eventos_pendentes": self.eventos.pendentes,
            "rastreamento": {
                "modo": self.rastreamento.modo,
                "quantidade_entradas": len(self.rastreamento.historico),
            },
        }

    def adicionar(self, regra: Regra) -> Regra:
        if any(r.nome == regra.nome for r in self.regras):
            raise ValueError(f"já existe uma regra chamada {regra.nome!r}")
        self.regras.append(regra)
        return regra

    def regra(
        self,
        nome: str,
        condicao: Condicao,
        acao: Acao,
        *,
        modo: str = "ao_ativar",
        prioridade: str | int = "normal",
        max_execucoes: int | None = None,
        cooldown: float = 0.0,
    ) -> Regra:
        return self.adicionar(
            Regra(
                nome,
                condicao,
                acao,
                modo=modo,
                prioridade=prioridade,
                max_execucoes=max_execucoes,
                cooldown=cooldown,
            )
        )

    def obter_regra(self, nome: str) -> Regra:
        procurado = str(nome)
        for regra in self.regras:
            if regra.nome == procurado:
                return regra
        raise ValueError(f"regra inexistente: {procurado!r}")

    def habilitar_regra(self, nome: str) -> Regra:
        regra = self.obter_regra(nome)
        regra.habilitada = True
        return regra

    def desabilitar_regra(self, nome: str) -> Regra:
        regra = self.obter_regra(nome)
        regra.habilitada = False
        return regra
```

```
    def criar_grupo(self, nome: str, regras: list[str] | tuple[str, ...]) ->
      GrupoRegras:
        nome_grupo = str(nome)
        if nome_grupo in self.grupos:
            raise ValueError(f"já existe um grupo chamado {nome_grupo!r}")
        grupo = GrupoRegras(nome_grupo, tuple(self.obter_regra(item) for item in
          regras))
        self.grupos[nome_grupo] = grupo
        return grupo

    def obter_grupo(self, nome: str) -> GrupoRegras:
        procurado = str(nome)
        try:
            return self.grupos[procurado]
        except KeyError as exc:
            raise ValueError(f"grupo inexistente: {procurado!r}") from exc

    def habilitar_grupo(self, nome: str) -> GrupoRegras:
        grupo = self.obter_grupo(nome)
        grupo.habilitar()
        return grupo

    def desabilitar_grupo(self, nome: str) -> GrupoRegras:
        grupo = self.obter_grupo(nome)
        grupo.desabilitar()
        return grupo

    def observar_mudanca(self, nome: str, leitor, acao) -> ObservadorMudanca:
        item = ObservadorMudanca(nome, leitor, acao)
        self.observadores.append(item)
        return item

    def observar_passar_de(self, nome: str, leitor, limite, acao) -> ObservadorMudanca:
        item = ObservadorMudanca.passar_de(nome, leitor, limite, acao)
        self.observadores.append(item)
        return item

    def observar_cair_abaixo_de(self, nome: str, leitor, limite, acao) ->
      ObservadorMudanca:
        item = ObservadorMudanca.cair_abaixo_de(nome, leitor, limite, acao)
        self.observadores.append(item)
        return item

    def observar_entrada_regiao(self, nome: str, leitor, regiao_alvo, acao, **opcoes) ->
       ObservadorEspacial:
        item = ObservadorEspacial.entrar_em_regiao(nome, leitor, regiao_alvo, acao,
          **opcoes)
        self.observadores.append(item)
        return item

    def observar_saida_regiao(self, nome: str, leitor, regiao_alvo, acao, **opcoes) ->
      ObservadorEspacial:
        item = ObservadorEspacial.sair_de_regiao(nome, leitor, regiao_alvo, acao,
          **opcoes)
        self.observadores.append(item)
        return item

    def observar_proximidade(self, nome: str, leitor, referencia, limite, acao,
      **opcoes) -> ObservadorEspacial:
        item = ObservadorEspacial.chegar_a_menos_de(nome, leitor, referencia, limite,
          acao, **opcoes)
```

```
        self.observadores.append(item)
        return item

    def observar_toque(self, nome: str, leitor, referencia, acao, **opcoes) ->
      ObservadorEspacial:
        item = ObservadorEspacial.tocar(nome, leitor, referencia, acao, **opcoes)
        self.observadores.append(item)
        return item

    def observar_intersecao(self, nome: str, leitor, referencia, acao, **opcoes) ->
      ObservadorEspacial:
        item = ObservadorEspacial.intersectar(nome, leitor, referencia, acao, **opcoes)
        self.observadores.append(item)
        return item

    def observar_regioes_mapa(
        self,
        nome: str,
        alvo: Any,
        mapa,
        regioes,
        resolvedor,
        *,
        identificador_alvo: Any = None,
    ):
        """Observa transições discretas usando o mesmo barramento do motor."""
        from ._mapas.observacao import ObservadorRegiaoMapa

        item = ObservadorRegiaoMapa(
            nome,
            alvo,
            mapa,
            regioes,
            resolvedor,
            self.eventos,
            identificador_alvo=identificador_alvo,
        )
        self.observadores.append(item)
        return item

    def sequencia(
        self,
        nome: str,
        primeiro: str,
        segundo: str,
        acao,
        *,
        janela: float | None = None,
    ) -> SequenciaDoisPassos:
        item = SequenciaDoisPassos(
            self.eventos,
            primeiro,
            segundo,
            acao,
            janela=janela,
            nome=nome,
        )
        self.sequencias.append(item)
        return item

    def debounce(self, segundos: float, acao: Acao, *, nome: str = "debounce") ->
```

```
      Debounce:
        item = Debounce(segundos, acao, nome=nome)
        self.debounces.append(item)
        return item

    def ativar_regra_durante(self, nome: str, segundos: float) -> Regra:
        duracao = float(validar_duracao(segundos, "duração da regra temporária"))
        regra = self.obter_regra(nome)
        regra.habilitada = True
        self._ativacoes_durante = [a for a in self._ativacoes_durante if a.regra is not
          regra]
        self._ativacoes_durante.append(_AtivacaoRegraDurante(regra,
          self._instante_atual() + duracao))
        return regra

    def ativar_regra_ate(self, nome: str, condicao: Condicao) -> Regra:
        regra = self.obter_regra(nome)
        regra.habilitada = True
        self._ativacoes_ate = [a for a in self._ativacoes_ate if a.regra is not regra]
        self._ativacoes_ate.append(_AtivacaoRegraAte(regra, condicao))
        return regra

    def _atualizar_ativacoes(self, contexto: Any) -> None:
        instante = self._instante_atual()
        durante_ativas: list[_AtivacaoRegraDurante] = []
        for ativacao in self._ativacoes_durante:
            if instante > ativacao.fim or math.isclose(instante, ativacao.fim,
              rel_tol=1e-12, abs_tol=1e-15):
                ativacao.regra.habilitada = False
            else:
                durante_ativas.append(ativacao)
        self._ativacoes_durante = durante_ativas

        ate_ativas: list[_AtivacaoRegraAte] = []
        for ativacao in self._ativacoes_ate:
            if bool(ativacao.condicao(contexto)):
                ativacao.regra.habilitada = False
            else:
                ate_ativas.append(ativacao)
        self._ativacoes_ate = ate_ativas

    def depois_de(self, segundos: float, acao: Acao, *, nome: str = "depois") ->
      Temporizador:
        t = Temporizador(segundos, acao, False, nome)
        self.temporizadores.append(t)
        return t

    def a_cada(self, segundos: float, acao: Acao, *, nome: str = "intervalo") ->
      Temporizador:
        t = Temporizador(segundos, acao, True, nome)
        self.temporizadores.append(t)
        return t

    def _instante_atual(self) -> float:
        if self.relogio is not None:
            return float(self.relogio.agora())
        return self._tempo_logico

    def instante_atual(self) -> float:
        return self._instante_atual()
```

```
    def avaliar(self, contexto: Any) -> tuple[str, ...]:
        self._atualizar_ativacoes(contexto)
        executadas = []
        instante = self._instante_atual()
        ordenadas = sorted(
            enumerate(tuple(self.regras)),
            key=lambda item: (-int(item[1].prioridade), item[0]),
        )
        for _ordem, r in ordenadas:
            executou = r.avaliar(contexto, instante=instante)
            if executou:
                executadas.append(r.nome)
            self.rastreamento.registrar(
                sequencia=None,
                instante=instante,
                tipo="regra",
                nome=r.nome,
                decisao="executada" if executou else (r.motivo_ultima_supressao or
                  "nao_executada"),
                prioridade=int(r.prioridade),
                resultado_condicao=r.ultima_condicao,
                detalhes={"execucoes": r.execucoes, "modo": r.modo},
            )
        for observador in tuple(self.observadores):
            if observador.avaliar(contexto):
                executadas.append(observador.nome)
        return tuple(executadas)

    def atualizar(self, contexto: Any, dt: float | None = None) -> tuple[str, ...]:
        if dt is None:
            if self.relogio is None:
                raise ValueError("informe dt ou configure um relógio no MotorRegras")
            dt = self.relogio.passo()
        passo = normalizar_dt(dt, compatibilidade_negativo_zero=True)
        if self.relogio is None:
            self._tempo_logico += passo
        executadas = list(self.avaliar(contexto))
        for t in tuple(self.temporizadores):
            if t.atualizar(contexto, passo):
                executadas.append(t.nome)
        self.temporizadores[:] = [t for t in self.temporizadores if t.ativo]
        for debounce in tuple(self.debounces):
            if debounce.atualizar(passo):
                executadas.append(debounce.nome)
        return tuple(executadas)


    def processar_ciclo(self, contexto: Any = None, *, limite_reacoes: int = 1000) ->
      tuple[Evento, ...]:
        if isinstance(limite_reacoes, bool) or not isinstance(limite_reacoes, int) or
          limite_reacoes <= 0:
            raise ValueError("limite de reações deve ser um inteiro positivo")
        processados: list[Evento] = []
        nomes_recentes: list[str] = []
        while self.eventos.pendentes:
            if len(processados) >= limite_reacoes:
                cadeia = " → ".join(f'"{nome}"' for nome in nomes_recentes[-6:])
                self.rastreamento.registrar(
                    sequencia=processados[-1].sequencia if processados else None,
                    instante=processados[-1].instante if processados else None,
                    tipo="ciclo",
```

```
                    nome="ciclo_reativo",
                    decisao="interrompido",
                    detalhes={"limite": limite_reacoes, "cadeia_recente":
                      tuple(nomes_recentes[-6:])},
                )
                detalhe = f" Cadeia recente: {cadeia}." if cadeia else ""
                raise ErroCicloReativo(
                    f"Ciclo reativo interrompido após {limite_reacoes}
                      reações.{detalhe}"
                )
            evento = self.eventos.processar(1)[0]
            processados.append(evento)
            nomes_recentes.append(evento.nome)
            self.rastreamento.registrar(
                sequencia=evento.sequencia,
                instante=evento.instante,
                tipo="evento",
                nome=evento.nome,
                decisao="processado",
                causa_sequencia=evento.causa_sequencia,
                detalhes={"origem": evento.origem, "cancelado": evento.cancelado,
                  "consumido": evento.consumido},
            )
        return tuple(processados)

    def emitir_evento(self, nome: str, *args, **kwargs) -> int:
        """Emite um evento de regra pelo barramento compartilhado, sem conhecer
          consumidores."""
        return self.eventos.emitir_propagado(str(nome), *args, **kwargs)

    def atualizar_por_relogio(self, contexto: Any) -> tuple[str, ...]:
        return self.atualizar(contexto, None)


def regra(
    nome: str,
    quando: Condicao,
    entao: Acao,
    *,
    modo: str = "ao_ativar",
    prioridade: str | int = "normal",
    max_execucoes: int | None = None,
    cooldown: float = 0.0,
) -> Regra:
    return Regra(
        nome,
        quando,
        entao,
        modo=modo,
        prioridade=prioridade,
        max_execucoes=max_execucoes,
        cooldown=cooldown,
    )


__all__ = [
    "Condicao", "Acao", "MODOS_REGRA", "Regra", "GrupoRegras", "Debounce",
      "Temporizador",
    "Eventos", "MotorRegras", "ObservadorMudanca", "ObservadorEspacial",
      "SequenciaDoisPassos", "regra",
]
```

```
"""Tempo, eventos e cancelamento compartilhados do runtime Coral.

Este módulo materializa a segunda parte da coesão 1.5 sem substituir as APIs
históricas de ``coral.regras``. Ele fornece fontes de tempo injetáveis,
validação uniforme de durações, eventos síncronos/enfileirados/propagados e um
token cooperativo de cancelamento reutilizável por bibliotecas e ferramentas.
"""
from __future__ import annotations

from collections import deque
from dataclasses import dataclass, field
import math
import time
from typing import Any, Callable, Mapping, Protocol, runtime_checkable

from .comum import ErroOperacaoCoral

CONTRATO = "coral.tempo_eventos/1"


def validar_duracao(valor: Any, nome: str = "duração", *, permitir_none: bool = False,
                    permitir_zero: bool = True) -> float | None:
    """Normaliza uma duração pública para segundos finitos não negativos."""
    if valor is None and permitir_none:
        return None
    if isinstance(valor, bool):
        raise ValueError(f"{nome} deve ser um número finito maior ou igual a zero")
    try:
        numero = float(valor)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ValueError(f"{nome} deve ser um número finito maior ou igual a zero") from
           exc
    if not math.isfinite(numero) or numero < 0 or (numero == 0 and not permitir_zero):
        operador = "maior que zero" if not permitir_zero else "maior ou igual a zero"
        raise ValueError(f"{nome} deve ser um número finito {operador}")
    return numero


def normalizar_dt(valor: Any, *, compatibilidade_negativo_zero: bool = True) -> float:
    """Normaliza avanço de simulação.

    A linha 1.2 já definia que ``dt`` negativo não volta temporizadores. Por
    compatibilidade, a política padrão converte valores negativos finitos em
    zero; novos consumidores podem optar por rejeitá-los.
    """
    if isinstance(valor, bool):
        raise ValueError("avanço de tempo deve ser um número finito")
    try:
        numero = float(valor)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ValueError("avanço de tempo deve ser um número finito") from exc
    if not math.isfinite(numero):
        raise ValueError("avanço de tempo deve ser um número finito")
    if numero < 0:
        if compatibilidade_negativo_zero:
            return 0.0
        raise ValueError("avanço de tempo não pode ser negativo")
    return numero
```

```
@runtime_checkable
class FonteTempo(Protocol):
    def agora(self) -> float: ...
    def passo(self) -> float: ...


class RelogioMonotonico:
    """Relógio real baseado em ``perf_counter`` e apropriado para durações."""
    def __init__(self, fonte: Callable[[], float] = time.perf_counter):
        self._fonte = fonte
        self._ultimo = float(fonte())

    def agora(self) -> float:
        return float(self._fonte())

    def passo(self) -> float:
        atual = self.agora()
        dt = normalizar_dt(atual - self._ultimo)
        self._ultimo = atual
        return dt


class RelogioSimulado:
    """Relógio controlado pelo programa, sem dormir nem consultar o sistema."""
    def __init__(self, inicial: float = 0.0):
        self._agora = float(validar_duracao(inicial, "tempo inicial"))
        self._ultimo = self._agora

    def agora(self) -> float:
        return self._agora

    def avancar(self, dt: Any) -> float:
        self._agora += normalizar_dt(dt, compatibilidade_negativo_zero=False)
        return self._agora

    def passo(self) -> float:
        dt = normalizar_dt(self._agora - self._ultimo,
          compatibilidade_negativo_zero=False)
        self._ultimo = self._agora
        return dt

    def definir(self, instante: Any) -> float:
        novo = float(validar_duracao(instante, "instante"))
        if novo < self._agora:
            raise ValueError("o relógio simulado não pode retroceder")
        self._agora = novo
        return novo


class OperacaoCancelada(ErroOperacaoCoral):
    """Operação cooperativa interrompida antes de uma fase segura."""


class TokenCancelamento:
    def __init__(self):
        self._cancelado = False
        self._motivo: str | None = None

    @property
    def cancelado(self) -> bool:
        return self._cancelado
```

```
    @property
    def motivo(self) -> str | None:
        return self._motivo

    def cancelar(self, motivo: str = "Operação cancelada pelo usuário.") -> None:
        self._cancelado = True
        self._motivo = str(motivo)

    def verificar(self) -> None:
        if self._cancelado:
            raise OperacaoCancelada(self._motivo or "Operação cancelada.")


@dataclass
class Evento:
    nome: str
    args: tuple[Any, ...] = ()
    kwargs: Mapping[str, Any] = field(default_factory=dict)
    origem: str | None = None
    sequencia: int = 0
    instante: float | None = None
    cancelavel: bool = False
    cancelado: bool = False
    motivo_cancelamento: str | None = None
    consumido: bool = False
    metadados: Mapping[str, Any] = field(default_factory=dict)
    causa_sequencia: int | None = None

    def __post_init__(self):
        self.nome = str(self.nome)
        self.args = tuple(self.args)
        self.kwargs = dict(self.kwargs)
        self.origem = None if self.origem is None else str(self.origem)
        self.metadados = dict(self.metadados)

    def __getattr__(self, nome: str) -> Any:
        payload = self.__dict__.get("kwargs", {})
        if nome in payload:
            return payload[nome]
        raise AttributeError(nome)

    def cancelar(self, motivo: str | None = None) -> None:
        if not self.cancelavel:
            raise ValueError("este evento não é cancelável")
        self.cancelado = True
        self.motivo_cancelamento = None if motivo is None else str(motivo)

    def consumir(self) -> None:
        self.consumido = True

    def estado_inspecao(self) -> dict[str, Any]:
        """Estado observável do evento sem executar ouvintes nem alterar a fila."""
        return {
            "nome": self.nome,
            "args": self.args,
            "kwargs": dict(self.kwargs),
            "origem": self.origem,
            "sequencia": self.sequencia,
            "instante": self.instante,
            "cancelavel": self.cancelavel,
```

```
            "cancelado": self.cancelado,
            "motivo_cancelamento": self.motivo_cancelamento,
            "consumido": self.consumido,
            "metadados": dict(self.metadados),
            "causa_sequencia": self.causa_sequencia,
        }


@dataclass
class _InscricaoEvento:
    funcao: Callable[..., Any]
    prioridade: int
    ordem: int
    receber_evento: bool = False
    max_execucoes: int | None = None
    cooldown: float = 0.0
    execucoes: int = 0
    ultimo_disparo: float | None = None


_PRIORIDADES_EVENTO = {
    "baixa": 0,
    "normal": 100,
    "alta": 200,
    "critica": 300,
    "crítica": 300,
}


def _normalizar_prioridade_evento(prioridade: str | int) -> int:
    if isinstance(prioridade, bool):
        raise ValueError("prioridade de evento desconhecida: use baixa, normal, alta ou
          crítica")
    if isinstance(prioridade, int):
        return prioridade
    nome = str(prioridade).strip().casefold()
    if nome not in _PRIORIDADES_EVENTO:
        raise ValueError(
            f"prioridade de evento desconhecida: {prioridade!r}; use baixa, normal, alta
               ou crítica"
        )
    return _PRIORIDADES_EVENTO[nome]


class Eventos:
    """Barramento local compatível com ``coral.regras.Eventos``.

    ``emitir`` mantém a chamada histórica de ouvintes via ``*args`` e
    ``**kwargs``. Inscrições novas podem receber o objeto :class:`Evento`
    completo e declarar prioridade sem alterar consumidores antigos.
    """
    def __init__(
        self,
        *,
        pai: "Eventos | None" = None,
        relogio: FonteTempo | None = None,
        fonte_instante: Callable[[], float] | None = None,
    ):
        self._ouvintes: dict[str, list[_InscricaoEvento]] = {}
        self._fila: deque[Evento] = deque()
        self._pai = pai
```

```
        self._relogio = relogio
        self._fonte_instante = fonte_instante
        self._sequencia = 0
        self._ordem_inscricao = 0
        self._pilha_processamento: list[Evento] = []

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "pendentes": len(self._fila),
            "fila": tuple(self._fila),
            "tipos_ouvintes": tuple(sorted(self._ouvintes)),
            "quantidade_ouvintes": sum(len(itens) for itens in self._ouvintes.values()),
            "ouvintes_por_tipo": {
                nome: len(itens)
                for nome, itens in sorted(self._ouvintes.items())
            },
            "sequencia": self._sequencia,
            "possui_pai": self._pai is not None,
            "possui_fonte_instante": self.possui_fonte_instante,
        }

    @property
    def pendentes(self) -> int:
        return len(self._fila)

    @property
    def pai(self) -> "Eventos | None":
        return self._pai

    @property
    def relogio(self) -> FonteTempo | None:
        return self._relogio

    def definir_fonte_instante(self, fonte: Callable[[], float] | None) -> None:
        self._fonte_instante = fonte

    def _instante_atual(self) -> float | None:
        if self._relogio is not None:
            return float(self._relogio.agora())
        if self._fonte_instante is not None:
            return float(self._fonte_instante())
        return None

    @property
    def possui_fonte_instante(self) -> bool:
        return self._relogio is not None or self._fonte_instante is not None

    def definir_pai(self, pai: "Eventos | None") -> None:
        if pai is self:
            raise ValueError("um barramento de eventos não pode propagar para si mesmo")
        atual = pai
        while atual is not None:
            if atual is self:
                raise ValueError("ciclo de propagação de eventos não permitido")
            atual = atual.pai
        self._pai = pai

    def quando(
        self,
        nome: str,
        funcao: Callable[..., Any],
```

```
        *,
        prioridade: str | int = "normal",
        receber_evento: bool = False,
        max_execucoes: int | None = None,
        cooldown: float = 0.0,
    ):
        valor_prioridade = _normalizar_prioridade_evento(prioridade)
        if max_execucoes is not None:
            if isinstance(max_execucoes, bool) or not isinstance(max_execucoes, int) or
              max_execucoes <= 0:
                raise ValueError("máximo de execuções de evento deve ser um inteiro
                  positivo ou nulo")
        try:
            valor_cooldown = float(validar_duracao(cooldown, "cooldown de evento"))
        except ValueError as exc:
            raise ValueError("cooldown de evento deve ser um número finito maior ou
              igual a zero") from exc
        if valor_cooldown > 0 and self._relogio is None and self._fonte_instante is
          None:
            raise ValueError("cooldown de evento exige uma fonte de tempo")
        self._ordem_inscricao += 1
        inscricao = _InscricaoEvento(
            funcao=funcao,
            prioridade=valor_prioridade,
            ordem=self._ordem_inscricao,
            receber_evento=bool(receber_evento),
            max_execucoes=max_execucoes,
            cooldown=valor_cooldown,
        )
        self._ouvintes.setdefault(str(nome), []).append(inscricao)
        return funcao

    def remover(self, nome: str, funcao: Callable[..., Any]) -> bool:
        ouvintes = self._ouvintes.get(str(nome))
        if not ouvintes:
            return False
        for indice, inscricao in enumerate(ouvintes):
            if inscricao.funcao is funcao or inscricao.funcao == funcao:
                del ouvintes[indice]
                if not ouvintes:
                    self._ouvintes.pop(str(nome), None)
                return True
        return False

    def _novo_evento(
        self,
        nome: str,
        args: tuple[Any, ...],
        kwargs: Mapping[str, Any],
        origem: str | None,
        *,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
    ) -> Evento:
        self._sequencia += 1
        instante = self._instante_atual()
        return Evento(
            str(nome),
            tuple(args),
            dict(kwargs),
            None if origem is None else str(origem),
```

```
            self._sequencia,
            instante=instante,
            cancelavel=bool(cancelavel),
            metadados={} if metadados is None else dict(metadados),
            causa_sequencia=(self._pilha_processamento[-1].sequencia if
              self._pilha_processamento else None),
        )

    def _snapshot_ouvintes(self, nome: str) -> tuple[_InscricaoEvento, ...]:
        ouvintes = tuple(self._ouvintes.get(nome, ()))
        return tuple(sorted(ouvintes, key=lambda item: (-item.prioridade, item.ordem)))

    def _emitir_local(self, evento: Evento) -> int:
        ouvintes = self._snapshot_ouvintes(evento.nome)
        for inscricao in ouvintes:
            if inscricao.max_execucoes is not None and inscricao.execucoes >=
              inscricao.max_execucoes:
                continue
            if inscricao.cooldown > 0 and inscricao.ultimo_disparo is not None:
                if evento.instante is None:
                    raise ValueError("cooldown de evento exige instante lógico")
                decorrido = float(evento.instante) - inscricao.ultimo_disparo
                if decorrido < inscricao.cooldown and not math.isclose(
                    decorrido, inscricao.cooldown, rel_tol=1e-12, abs_tol=1e-15
                ):
                    continue
            if inscricao.receber_evento:
                inscricao.funcao(evento)
            else:
                inscricao.funcao(*evento.args, **dict(evento.kwargs))
            inscricao.execucoes += 1
            if evento.instante is not None:
                inscricao.ultimo_disparo = float(evento.instante)
        return len(ouvintes)

    def emitir(self, nome: str, *args, **kwargs) -> int:
        evento = self._novo_evento(nome, args, kwargs, None)
        return self._emitir_local(evento)

    def emitir_evento(
        self,
        nome: str,
        *args,
        origem: str | None = None,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
        **kwargs,
    ) -> Evento:
        evento = self._novo_evento(
            nome,
            args,
            kwargs,
            origem,
            cancelavel=cancelavel,
            metadados=metadados,
        )
        self._emitir_local(evento)
        return evento

    def enfileirar(self, nome: str, *args, origem: str | None = None, **kwargs) ->
      Evento:
```

```
        evento = self._novo_evento(nome, args, kwargs, origem)
        self._fila.append(evento)
        return evento

    def enfileirar_evento(
        self,
        nome: str,
        *args,
        origem: str | None = None,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
        **kwargs,
    ) -> Evento:
        evento = self._novo_evento(
            nome,
            args,
            kwargs,
            origem,
            cancelavel=cancelavel,
            metadados=metadados,
        )
        self._fila.append(evento)
        return evento

    def processar(self, limite: int | None = None) -> tuple[Evento, ...]:
        if limite is not None and (isinstance(limite, bool) or not isinstance(limite,
          int) or limite < 0):
            raise ValueError("limite de eventos deve ser inteiro não negativo ou nulo")
        processados: list[Evento] = []
        maximo = len(self._fila) if limite is None else min(limite, len(self._fila))
        for _ in range(maximo):
            evento = self._fila.popleft()
            self._pilha_processamento.append(evento)
            try:
                self._emitir_local(evento)
            finally:
                self._pilha_processamento.pop()
            processados.append(evento)
        return tuple(processados)

    def _emitir_propagado_objeto(self, evento: Evento) -> int:
        total = self._emitir_local(evento)
        if self._pai is not None and not evento.consumido:
            total += self._pai._emitir_propagado_objeto(evento)
        return total

    def emitir_propagado(self, nome: str, *args, origem: str | None = None, **kwargs) ->
       int:
        evento = self._novo_evento(nome, args, kwargs, origem)
        return self._emitir_propagado_objeto(evento)

    def emitir_evento_propagado(
        self,
        nome: str,
        *args,
        origem: str | None = None,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
        **kwargs,
    ) -> Evento:
        evento = self._novo_evento(
```

```
            nome,
            args,
            kwargs,
            origem,
            cancelavel=cancelavel,
            metadados=metadados,
        )
        self._emitir_propagado_objeto(evento)
        return evento


__all__ = [
    "CONTRATO", "validar_duracao", "normalizar_dt", "FonteTempo",
    "RelogioMonotonico", "RelogioSimulado", "OperacaoCancelada",
    "TokenCancelamento", "Evento", "Eventos",
]
```

Biblioteca `coral.numerico`

`coral.numerico.DependenciaNumericaAusente`

Uma capacidade opcional não pode operar por ausência de dependência.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.numerico.ErroNumerico`

Erro público para entradas numéricas inválidas ou operações incompatíveis.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.numerico.ResultadoBootstrap`

Intervalo percentil obtido por reamostragem bootstrap.

Assinatura

Assinatura para consulta

```
ResultadoBootstrap(estimativa: 'float', inferior: 'float', superior: 'float', confianca:
   'float', repeticoes: 'int', semente: 'int', tamanho_amostra: 'int', custo_estimado:
  'int', metodo: 'str' = 'bootstrap_percentil') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
estimativa	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
inferior	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
superior	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
confianca	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
repeticoes	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
semente	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
tamanho_amostra	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
custo_estimado	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
metodo	str	POSITIONAL_OR_KEYWORD	não	'bootstrap_percentil'

Retorno declarado

None

Atributo `estimativa`: `float`.

Atributo `inferior`: `float`.

Atributo `superior`: `float`.

Atributo `confianca`: `float`.

Atributo `repeticoes`: `int`.

Atributo `semente`: `int`.

Atributo `tamanho_amostra`: `int`.

Atributo `custo_estimado`: `int`.

Atributo `metodo`: `str`.

`coral.numerico.ResultadoMinimosQuadrados`

Resultado estruturado de :func:`minimos_quadrados`.

`residuos` segue a convenção do NumPy: pode ser vazio quando o sistema não possui graus de liberdade para estimar a soma dos resíduos.

Assinatura

Assinatura para consulta

```
ResultadoMinimosQuadrados(solucao: 'Any', residuos: 'Any', posto: 'int',
  valores_singulares: 'Any') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
solucao	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
residuos	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
posto	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
valores_singulares	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `solucao`: `Any`.

Atributo `residuos`: `Any`.

Atributo `posto`: `int`.

Atributo `valores_singulares`: `Any`.

`coral.numerico.ResultadoRegressaoLinear`

Ajuste linear simples com resíduos e coeficiente de determinação.

Assinatura

Assinatura para consulta

```
ResultadoRegressaoLinear(inclinacao: 'float', intercepto: 'float', residuos: 'Any',
  r_quadrado: 'float', quantidade: 'int') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
inclinacao	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
intercepto	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
residuos	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
r_quadrado	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
quantidade	int	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `inclinacao`: `float`.

Atributo `intercepto`: `float`.

Atributo `residuos`: `Any`.

Atributo `r_quadrado`: `float`.

Atributo `quantidade`: `int`.

`coral.numerico.ResultadoTestePermutacao`

Resultado de um teste de permutação bilateral entre duas amostras.

Assinatura

Assinatura para consulta

```
ResultadoTestePermutacao(diferenca_observada: 'float', valor_p: 'float', repeticoes:
  'int', semente: 'int', tamanho_a: 'int', tamanho_b: 'int', custo_estimado: 'int',
  metodo: 'str' = 'permutacao_bilateral') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
diferenca_observada	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
valor_p	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
repeticoes	int	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
semente	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
tamanho_a	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
tamanho_b	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
custo_estimado	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
metodo	str	POSITIONAL_OR_KEYWORD	não	'permutacao_bilateral'

Retorno declarado

None

Atributo `diferenca_observada`: `float`.

Atributo `valor_p`: `float`.

Atributo `repeticoes`: `int`.

Atributo `semente`: `int`.

Atributo `tamanho_a`: `int`.

Atributo `tamanho_b`: `int`.

Atributo `custo_estimado`: `int`.

Atributo `metodo`: `str`.

`coral.numerico.amplitude`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
amplitude(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.angulo_entre_vetores`

Ângulo em radianos entre vetores não nulos da mesma dimensão.

Assinatura

Assinatura para consulta

```
angulo_entre_vetores(a, b)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
b	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.area_triangulo3`

Área do triângulo definido por três pontos 3D.

Assinatura

Assinatura para consulta

```
area_triangulo3(a, b, c)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
b	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
c	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.assimetria`

Momento padronizado de ordem 3, sem correção de viés amostral.

Assinatura

Assinatura para consulta

```
assimetria(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.autovalores`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
autovalores(m)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
m	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.compor_transformacoes3`

Compõe transformações 3D na ordem em que são aplicadas ao ponto.

Assinatura

Assinatura para consulta

```
compor_transformacoes3(*transformacoes)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
transformacoes	sem anotação	VAR_POSITIONAL	não	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.concatenar`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
concatenar(valores, eixo=0)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
eixo	sem anotação	POSITIONAL_OR_KEYWORD	não	0

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.coordenadas_esfericas3`

Converte (x, y, z) em (raio, azimute, elevação), em radianos.

Assinatura

Assinatura para consulta

```
coordenadas_esfericas3(ponto)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
ponto	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.correlacao`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
correlacao(a, b)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
b	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.covariancia`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
covariancia(a, b, populacional=True)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
b	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
populacional	sem anotação	POSITIONAL_OR_KEYWORD	não	True

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.curtose`

Curtose em excesso pelo quarto momento padronizado, isto é, normal = 0.

Assinatura

Assinatura para consulta

```
curtose(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.desvio_padrao`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
desvio_padrao(valores, populacional=True)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
populacional	sem anotação	POSITIONAL_OR_KEYWORD	não	True

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.determinante`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
determinante(m)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
m	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.diagnosticar_numerico`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
diagnosticar_numerico() -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

dict[str, Any]

`coral.numerico.distancia_ponto_plano3`

Distância euclidiana não negativa entre um ponto e um plano 3D.

Assinatura

Assinatura para consulta

```
distancia_ponto_plano3(ponto, ponto_plano, normal)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
ponto	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
ponto_plano	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
normal	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.distancia_vetores`

Distância euclidiana entre vetores finitos da mesma dimensão.

Assinatura

Assinatura para consulta

```
distancia_vetores(a, b)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
b	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.erro_padrao`

Calcula o erro padrão da média usando o desvio amostral.

Requer ao menos duas observações finitas. A convenção usa `n - 1 no desvio padrão e divide o resultado por sqrt(n)`.

Assinatura

Assinatura para consulta

```
erro_padrao(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.espaco_linear`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
espaco_linear(inicio, fim, quantidade=50)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
inicio	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
fim	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
quantidade	sem anotação	POSITIONAL_OR_KEYWORD	não	50

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.forma`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
forma(valor) -> 'tuple[int, ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

tuple[int, ...]

`coral.numerico.frequencias`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
frequencias(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.histograma`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
histograma(valores, faixas)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
faixas	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.identidade`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
identidade(ordem)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
ordem	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.intersecao_reta_plano3`

Interseção reta plano; retorna None para paralelas distintas.

Assinatura

Assinatura para consulta

```
intersecao_reta_plano3(origem, direcao, ponto_plano, normal, *, tolerancia=1e-12)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
origem	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
direcao	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
ponto_plano	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
normal	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tolerancia	sem anotação	KEYWORD_ONLY	não	1e-12

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.intervalo`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
intervalo(inicio, fim=None, passo=1, tipo=None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
inicio	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
fim	sem anotação	POSITIONAL_OR_KEYWORD	não	None
passo	sem anotação	POSITIONAL_OR_KEYWORD	não	1
tipo	sem anotação	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.intervalo_bootstrap`

Intervalo bootstrap percentil reproduzível com gerador aleatório local.

Assinatura

Assinatura para consulta

```
intervalo_bootstrap(valores, estatistica, confianca, repeticoes, semente)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
estatistica	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
confianca	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
repeticoes	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
semente	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.intervalo_interquartil`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
intervalo_interquartil(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.inversa`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
inversa(m)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
m	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.matriz`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
matriz(linhas: 'Iterable[Iterable[Any]]', tipo: 'Any | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
linhas	Iterable[Iterable[Any]]	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipo	Any | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.matriz_correlacao`

Matriz de correlação de Pearson para colunas observadas conjuntamente.

Assinatura

Assinatura para consulta

```
matriz_correlacao(dados)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
dados	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.matriz_covariancia`

Matriz de covariância com linhas como observações e colunas como variáveis.

Por padrão usa a convenção amostral, dividindo por `n - 1. Defina populacional=True para dividir por n`.

Assinatura

Assinatura para consulta

```
matriz_covariancia(dados, populacional=False)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
dados	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
populacional	sem anotação	POSITIONAL_OR_KEYWORD	não	False

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.matriz_escala3`

Matriz afim 4 × 4 de escala uniforme ou independente por eixo.

Assinatura

Assinatura para consulta

```
matriz_escala3(sx, sy=None, sz=None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
sx	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
sy	sem anotação	POSITIONAL_OR_KEYWORD	não	None
sz	sem anotação	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.matriz_rotacao3`

Matriz 3 × 3 de Rodrigues para rotação em torno de um eixo 3D.

Assinatura

Assinatura para consulta

```
matriz_rotacao3(eixo, angulo)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
eixo	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
angulo	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.matriz_translacao3`

Matriz afim 4 × 4 de translação 3D em convenção de vetor coluna.

Assinatura

Assinatura para consulta

```
matriz_translacao3(dx, dy, dz)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
dx	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
dy	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
dz	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.maximo`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
maximo(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.media`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
media(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.media_geometrica`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
media_geometrica(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.media_harmonica`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
media_harmonica(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.media_ponderada`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
media_ponderada(valores, pesos)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
pesos	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.mediana`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
mediana(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.mediana_desvios_absolutos`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
mediana_desvios_absolutos(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.minimo`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
minimo(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.minimos_quadrados`

Resolve um ajuste linear por mínimos quadrados sem exigir sistema quadrado.

Assinatura

Assinatura para consulta

```
minimos_quadrados(coeficientes, termos)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
coeficientes	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
termos	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.moda`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
moda(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.multiplicar_matrizes`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
multiplicar_matrizes(a, b)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
b	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.norma`

Norma euclidiana de um vetor finito e não vazio.

Assinatura

Assinatura para consulta

```
norma(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.normalizar`

Devolve uma cópia do vetor com norma euclidiana igual a um.

Assinatura

Assinatura para consulta

```
normalizar(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.numpy_disponivel`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
numpy_disponivel() -> 'bool'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

bool

`coral.numerico.percentil`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
percentil(valores, percentual)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
percentual	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.ponto_de_esfericas3`

Converte (raio, azimute, elevação) para ponto cartesiano 3D.

Assinatura

Assinatura para consulta

```
ponto_de_esfericas3(raio, azimute, elevacao)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
raio	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
azimute	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
elevacao	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.posto`

Posto numérico de uma matriz pela tolerância padrão da decomposição SVD do NumPy.

Assinatura

Assinatura para consulta

```
posto(matriz)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
matriz	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.produto_escalar`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
produto_escalar(a, b)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
b	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.produto_misto3`

Produto misto orientado de três vetores tridimensionais.

Assinatura

Assinatura para consulta

```
produto_misto3(a, b, c)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
b	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
c	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.produto_vetorial`

Produto vetorial exclusivo de vetores tridimensionais.

Assinatura

Assinatura para consulta

```
produto_vetorial(a, b)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
b	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.projecao_vetorial`

Projeta `a` sobre um vetor não nulo, preservando a dimensão.

Assinatura

Assinatura para consulta

```
projecao_vetorial(a, sobre)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
sobre	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.projetar_ponto_plano3`

Projeção ortogonal de um ponto sobre um plano 3D.

Assinatura

Assinatura para consulta

```
projetar_ponto_plano3(ponto, ponto_plano, normal)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
ponto	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
ponto_plano	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
normal	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.quartis`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
quartis(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.regressao_linear`

Ajusta `y = intercepto + inclinacao * x` por mínimos quadrados simples.

Assinatura

Assinatura para consulta

```
regressao_linear(x, y)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
x	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.remodelar`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
remodelar(valores, forma)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
forma	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.resolver_sistema`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
resolver_sistema(coeficientes, termos)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
coeficientes	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
termos	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.rotacionar3`

Rotaciona um vetor 3D em torno de um eixo não nulo, em radianos.

Usa a fórmula de Rodrigues e preserva a norma dentro da precisão do backend numérico. O eixo define orientação pela regra da mão direita.

Assinatura

Assinatura para consulta

```
rotacionar3(vetor, eixo, angulo)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
vetor	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
eixo	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
angulo	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.selecionar_coluna`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
selecionar_coluna(registros, campo, ausentes='erro')
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
registros	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
campo	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
ausentes	sem anotação	POSITIONAL_OR_KEYWORD	não	'erro'

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.soma`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
soma(valores)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.teste_permutacao`

Teste de permutação bilateral para diferença de uma estatística entre grupos.

Assinatura

Assinatura para consulta

```
teste_permutacao(a, b, estatistica, repeticoes, semente)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
b	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
estatistica	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
repeticoes	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
semente	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.transformar_ponto3`

Aplica uma transformação afim homogênea 4 × 4 a um ponto 3D.

A convenção pública usa vetor coluna `[x, y, z, 1]. A translação vive na última coluna e a última linha deve ser [0, 0, 0, 1]` dentro de tolerância, rejeitando matrizes projetivas nesta API.

Assinatura

Assinatura para consulta

```
transformar_ponto3(ponto, transformacao)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
ponto	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
transformacao	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.transformar_vetor3`

Aplica a parte linear de uma transformação afim, ignorando translação.

Assinatura

Assinatura para consulta

```
transformar_vetor3(vetor, transformacao)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
vetor	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
transformacao	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.transposta`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
transposta(valor)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.uns`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
uns(forma, tipo=<class 'float'>)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
forma	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipo	sem anotação	POSITIONAL_OR_KEYWORD	não	float

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.variancia`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
variancia(valores, populacional=True)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
populacional	sem anotação	POSITIONAL_OR_KEYWORD	não	True

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.vetor`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
vetor(valores: 'Iterable[Any]', tipo: 'Any | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valores	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipo	Any | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.vetor3`

Constrói um vetor numérico tridimensional `` `(x, y, z)` ``.

Assinatura

Assinatura para consulta

```
vetor3(x, y, z)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
x	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
y	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
z	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.volume_paralelepipedo3`

Volume do paralelepípedo gerado por três vetores 3D.

Assinatura

Assinatura para consulta

```
volume_paralelepipedo3(a, b, c)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
b	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
c	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.volume_tetraedro3`

Volume do tetraedro definido por quatro pontos 3D.

Assinatura

Assinatura para consulta

```
volume_tetraedro3(a, b, c, d)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
a	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
b	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
c	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
d	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.numerico.zeros`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
zeros(forma, tipo=<class 'float'>)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
forma	sem anotação	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipo	sem anotação	POSITIONAL_OR_KEYWORD	não	float

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Consultar a implementação canônica de coral.numerico

Implementação canônica de `coral.numerico`

Código Python

```
"""Contratos transversais pequenos do runtime Coral.

A camada comum não substitui APIs de domínio. Ela fornece a gramática
arquitetural compartilhada do ciclo 1.5 para erro operacional, resultado e
diagnóstico, preservando os nomes públicos existentes em cada biblioteca.
"""
from __future__ import annotations

from dataclasses import asdict, dataclass, field
from typing import Any, Iterable, Mapping

CONTRATO = "coral.comum/1"
CONTRATO_DIAGNOSTICO = "coral.diagnostico.operacional/1"


class ErroCoral(Exception):
    """Raiz pública dos erros controlados pela linguagem e bibliotecas Coral."""


class ErroOperacaoCoral(RuntimeError, ErroCoral):
    """Falha operacional de um recurso Coral ou de seu backend."""


class ErroDependenciaCoral(ErroOperacaoCoral):
    """Uma capacidade opcional não pode operar por ausência de dependência."""


@dataclass(frozen=True)
class FalhaOperacao:
    codigo: str
    mensagem: str
    dominio: str
    tipo: str = "operacional"
    causa: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def de_excecao(cls, exc: BaseException, *, codigo: str, dominio: str, tipo: str =
      "operacional", detalhe: Mapping[str, Any] | None = None) -> "FalhaOperacao":
        causa = exc.__cause__ if exc.__cause__ is not None else exc
        return cls(codigo, str(exc), dominio, tipo, type(causa).__name__, dict(detalhe
          or {}))

    def para_dict(self) -> dict[str, Any]:
        return asdict(self)


@dataclass(frozen=True)
class ResultadoOperacao:
    """Representação transversal sem substituir resultados ricos de domínio."""
    ok: bool
    valor: Any = None
    falha: FalhaOperacao | None = None
    metadados: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def sucesso(cls, valor: Any = None, **metadados: Any) -> "ResultadoOperacao":
        return cls(True, valor=valor, metadados=metadados)
```

```
    @classmethod
    def erro(cls, falha: FalhaOperacao, **metadados: Any) -> "ResultadoOperacao":
        return cls(False, falha=falha, metadados=metadados)

    def para_dict(self) -> dict[str, Any]:
        return {
            "contrato": CONTRATO,
            "ok": self.ok,
            "valor": self.valor,
            "falha": self.falha.para_dict() if self.falha is not None else None,
            "metadados": dict(self.metadados),
        }


@dataclass(frozen=True)
class DiagnosticoOperacional:
    id: str
    dominio: str
    ok: bool
    mensagem: str
    capacidade: str | None = None
    sugestao: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @property
    def nivel(self) -> str:
        return "info" if self.ok else "erro"

    def para_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "dominio": self.dominio,
            "ok": self.ok,
            "nivel": self.nivel,
            "mensagem": self.mensagem,
            "capacidade": self.capacidade,
            "sugestao": self.sugestao,
            "detalhe": dict(self.detalhe),
        }


def diagnostico(id: str, dominio: str, ok: bool, mensagem: Any, *, capacidade: str |
  None = None, sugestao: str | None = None, detalhe: Mapping[str, Any] | None = None) ->
  DiagnosticoOperacional:
    return DiagnosticoOperacional(str(id), str(dominio), bool(ok), str(mensagem),
      capacidade, sugestao, dict(detalhe or {}))


def agregar_diagnosticos(itens: Iterable[DiagnosticoOperacional | Mapping[str, Any]], *,
   origem: str = "coral") -> dict[str, Any]:
    normalizados: list[dict[str, Any]] = []
    for item in itens:
        if isinstance(item, DiagnosticoOperacional):
            normalizados.append(item.para_dict())
            continue
        d = dict(item)
        if "id" not in d:
            d["id"] = str(d.get("nome", "diagnostico"))
        if "dominio" not in d:
            d["dominio"] = origem
        d["ok"] = bool(d.get("ok"))
```

```
        d.setdefault("nivel", "info" if d["ok"] else "erro")
        d.setdefault("mensagem", str(d.get("detalhe", "")))
        d.setdefault("capacidade", None)
        d.setdefault("sugestao", None)
        detalhe = d.get("detalhe")
        if not isinstance(detalhe, Mapping):
            d["detalhe"] = {"valor": detalhe}
        normalizados.append(d)
    return {
        "contrato": CONTRATO_DIAGNOSTICO,
        "origem": origem,
        "ok": all(x["ok"] for x in normalizados),
        "quantidade": len(normalizados),
        "itens": normalizados,
    }


__all__ = [
    "CONTRATO", "CONTRATO_DIAGNOSTICO", "ErroCoral", "ErroOperacaoCoral",
    "ErroDependenciaCoral", "FalhaOperacao", "ResultadoOperacao",
    "DiagnosticoOperacional", "diagnostico", "agregar_diagnosticos",
]

"""Computação numérica oficial da Coral.

NumPy é opcional: importar ``coral.numerico`` não torna NumPy uma dependência
obrigatória do compilador. As operações que precisam dele emitem um diagnóstico
em português com instrução de instalação.
"""
from __future__ import annotations

from dataclasses import dataclass
import math
import operator
import random
from collections.abc import Mapping
from typing import Any, Iterable, Sequence

from .comum import ErroCoral, ErroDependenciaCoral, diagnostico


class DependenciaNumericaAusente(ErroDependenciaCoral):
    pass


class ErroNumerico(ValueError, ErroCoral):
    """Erro público para entradas numéricas inválidas ou operações incompatíveis."""
    pass


@dataclass(frozen=True)
class ResultadoMinimosQuadrados:
    """Resultado estruturado de :func:`minimos_quadrados`.

    ``residuos`` segue a convenção do NumPy: pode ser vazio quando o sistema
    não possui graus de liberdade para estimar a soma dos resíduos.
    """

    solucao: Any
    residuos: Any
    posto: int
```

```
    valores_singulares: Any


@dataclass(frozen=True)
class ResultadoRegressaoLinear:
    """Ajuste linear simples com resíduos e coeficiente de determinação."""

    inclinacao: float
    intercepto: float
    residuos: Any
    r_quadrado: float
    quantidade: int


@dataclass(frozen=True)
class ResultadoBootstrap:
    """Intervalo percentil obtido por reamostragem bootstrap."""

    estimativa: float
    inferior: float
    superior: float
    confianca: float
    repeticoes: int
    semente: int
    tamanho_amostra: int
    custo_estimado: int
    metodo: str = "bootstrap_percentil"


@dataclass(frozen=True)
class ResultadoTestePermutacao:
    """Resultado de um teste de permutação bilateral entre duas amostras."""

    diferenca_observada: float
    valor_p: float
    repeticoes: int
    semente: int
    tamanho_a: int
    tamanho_b: int
    custo_estimado: int
    metodo: str = "permutacao_bilateral"


_LIMITE_TRABALHO_REAMOSTRAGEM = 5_000_000



def _numpy():
    try:
        import numpy as np
        return np
    except Exception as exc:
        raise DependenciaNumericaAusente(
            "A computação numérica da Coral requer NumPy. "
            "Instale com: python -m pip install numpy"
        ) from exc


def numpy_disponivel() -> bool:
    try:
        import numpy  # noqa: F401
```

```
        return True
    except Exception:
        return False


def diagnosticar_numerico() -> dict[str, Any]:
    dados: dict[str, Any] = {"disponivel": numpy_disponivel(), "dependencia": "numpy"}
    if dados["disponivel"]:
        import numpy as np
        dados["versao"] = np.__version__
        mensagem = "NumPy disponível para computação numérica."
        sugestao = None
    else:
        dados["instalacao"] = "python -m pip install numpy"
        mensagem = "NumPy não está disponível neste ambiente."
        sugestao = "Instale a capacidade Numérico pelo Coral Setup ou Gerenciador de
          Componentes."
    dados["diagnostico"] = diagnostico(
        "numerico.numpy", "numerico", dados["disponivel"], mensagem,
        capacidade="numpy", sugestao=sugestao,
        detalhe={k: v for k, v in dados.items() if k != "diagnostico"},
    ).para_dict()
    return dados


def vetor(valores: Iterable[Any], tipo: Any | None = None):
    np = _numpy()
    try:
        a = np.asarray(list(valores), dtype=tipo)
    except (TypeError, ValueError) as exc:
        raise ErroNumerico(f"não foi possível construir o vetor: {exc}") from exc
    if a.ndim != 1:
        raise ErroNumerico(f"vetor requer uma dimensão; recebido objeto com {a.ndim}
          dimensões")
    return a


def matriz(linhas: Iterable[Iterable[Any]], tipo: Any | None = None):
    np = _numpy()
    try:
        a = np.asarray([list(l) for l in linhas], dtype=tipo)
    except (TypeError, ValueError) as exc:
        raise ErroNumerico(f"não foi possível construir a matriz: {exc}") from exc
    if a.ndim != 2:
        raise ErroNumerico(f"matriz requer duas dimensões; recebido objeto com {a.ndim}
          dimensões")
    return a


def zeros(forma, tipo=float):
    try:
        return _numpy().zeros(forma, dtype=tipo)
    except (TypeError, ValueError) as exc:
        raise ErroNumerico(f"forma inválida para zeros: {exc}") from exc


def uns(forma, tipo=float):
    try:
        return _numpy().ones(forma, dtype=tipo)
    except (TypeError, ValueError) as exc:
        raise ErroNumerico(f"forma inválida para uns: {exc}") from exc
```

```
def intervalo(inicio, fim=None, passo=1, tipo=None):
    np = _numpy()
    if fim is None:
        inicio, fim = 0, inicio
    try:
        if passo == 0:
            raise ErroNumerico("intervalo requer passo diferente de zero")
        return np.arange(inicio, fim, passo, dtype=tipo)
    except ErroNumerico:
        raise
    except (TypeError, ValueError, ZeroDivisionError, OverflowError) as exc:
        raise ErroNumerico(f"intervalo inválido: {exc}") from exc


def espaco_linear(inicio, fim, quantidade=50):
    if isinstance(quantidade, bool):
        raise ErroNumerico("quantidade do espaço linear deve ser um inteiro não
          negativo")
    try:
        q = int(quantidade)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroNumerico("quantidade do espaço linear deve ser um inteiro não
          negativo") from exc
    if isinstance(quantidade, str):
        texto = quantidade.strip()
        if not texto or not texto.lstrip("+-").isdigit():
            raise ErroNumerico("quantidade do espaço linear deve ser um inteiro não
              negativo")
    else:
        try:
            exata = quantidade == q
        except Exception:
            exata = False
        if not exata:
            raise ErroNumerico("quantidade do espaço linear deve ser um inteiro não
              negativo")
    if q < 0:
        raise ErroNumerico("quantidade do espaço linear deve ser maior ou igual a zero")
    try:
        return _numpy().linspace(inicio, fim, q)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroNumerico(f"espaço linear inválido: {exc}") from exc


def soma(valores):
    return _numpy().sum(valores)


def _materializar_array(valores, operacao: str, *, tipo=None):
    np = _numpy()
    bruto = valores
    if not isinstance(valores, np.ndarray) and not isinstance(valores, (str, bytes,
      bytearray)):
        try:
            bruto = list(valores)
        except TypeError:
            bruto = valores
    try:
        return np.asarray(bruto, dtype=tipo)
```

```
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroNumerico(f"{operacao} requer valores numéricos compatíveis: {exc}")
          from exc


def _exigir_nao_vazio(valores, operacao: str):
    a = _materializar_array(valores, operacao)
    if a.size == 0:
        raise ErroNumerico(f"{operacao} requer ao menos um valor")
    return a


def _exigir_dados_finitos(valores, operacao: str):
    np = _numpy()
    a = _exigir_nao_vazio(valores, operacao)
    if a.dtype == object:
        for item in a.flat:
            if item is None:
                raise ErroNumerico(f"{operacao} não aceita valores ausentes")
    try:
        finitos = np.isfinite(a)
    except TypeError as exc:
        raise ErroNumerico(f"{operacao} requer valores numéricos") from exc
    if not bool(np.all(finitos)):
        raise ErroNumerico(f"{operacao} não aceita NaN ou infinito")
    return a


def _serie_real_finita(valores, operacao: str, *, minimo_itens: int = 1):
    a = _materializar_array(valores, operacao, tipo=float)
    if a.ndim != 1:
        raise ErroNumerico(f"{operacao} requer uma sequência unidimensional")
    if a.size < minimo_itens:
        unidade = "valor" if minimo_itens == 1 else "valores"
        raise ErroNumerico(f"{operacao} requer ao menos {minimo_itens} {unidade}")
    if not bool(_numpy().all(_numpy().isfinite(a))):
        raise ErroNumerico(f"{operacao} não aceita valores ausentes, NaN ou infinito")
    return a




def _vetor_real_finito(valores, operacao: str, *, dimensao: int | None = None):
    np = _numpy()
    a = _materializar_array(valores, operacao, tipo=float)
    if a.ndim != 1:
        raise ErroNumerico(f"{operacao} requer um vetor unidimensional")
    if a.size == 0:
        raise ErroNumerico(f"{operacao} requer ao menos um componente")
    if dimensao is not None and a.size != dimensao:
        raise ErroNumerico(f"{operacao} requer vetor com exatamente {dimensao}
          componentes")
    if not bool(np.all(np.isfinite(a))):
        raise ErroNumerico(f"{operacao} não aceita NaN ou infinito")
    return a


def _matriz_real_finita(valores, operacao: str):
    np = _numpy()
    a = _materializar_array(valores, operacao, tipo=float)
    if a.ndim != 2:
```

```
        raise ErroNumerico(f"{operacao} requer uma matriz bidimensional")
    if 0 in a.shape:
        raise ErroNumerico(f"{operacao} requer matriz com linhas e colunas não vazias")
    if not bool(np.all(np.isfinite(a))):
        raise ErroNumerico(f"{operacao} não aceita NaN ou infinito")
    return a


def _percentil_linear_serie(a, percentual: float):
    np = _numpy()
    ordenados = np.sort(a)
    if ordenados.size == 1:
        return ordenados[0]
    posicao = (ordenados.size - 1) * (percentual / 100.0)
    inferior = int(math.floor(posicao))
    superior = int(math.ceil(posicao))
    if inferior == superior:
        return ordenados[inferior]
    fracao = posicao - inferior
    # Combinação convexa evita o overflow intermediário de (b - a) * f.
    return (1.0 - fracao) * ordenados[inferior] + fracao * ordenados[superior]


def _centralizar_escalado(a, operacao: str):
    np = _numpy()
    escala = float(np.max(np.abs(a)))
    if escala == 0.0:
        return np.zeros_like(a, dtype=float), 0.0
    normalizado = a / escala
    centro = float(np.mean(normalizado))
    desvios = normalizado - centro
    if not bool(np.all(np.isfinite(desvios))):
        raise ErroNumerico(f"{operacao} produziu intermediários fora do intervalo
          numérico suportado")
    return desvios, escala


def _exigir_resultado_finito(valor, operacao: str):
    try:
        finito = math.isfinite(float(valor))
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroNumerico(f"{operacao} produziu resultado numérico inválido") from exc
    if not finito:
        raise ErroNumerico(f"{operacao} produziu resultado fora do intervalo numérico
          suportado")
    return valor


def _inteiro_exato(valor, nome: str, *, minimo: int | None = None) -> int:
    if isinstance(valor, bool):
        raise ErroNumerico(f"{nome} deve ser um inteiro")
    try:
        inteiro = operator.index(valor)
    except TypeError as exc:
        raise ErroNumerico(f"{nome} deve ser um inteiro") from exc
    if minimo is not None and inteiro < minimo:
        raise ErroNumerico(f"{nome} deve ser maior ou igual a {minimo}")
    return inteiro


def _eh_ausente(valor) -> bool:
```

```
    if valor is None:
        return True
    if isinstance(valor, float):
        return math.isnan(valor)
    try:
        return bool(valor.is_nan())
    except (AttributeError, TypeError, ValueError):
        return False


def identidade(ordem):
    n = _inteiro_exato(ordem, "ordem", minimo=0)
    return _numpy().eye(n)


def remodelar(valores, forma):
    np = _numpy()
    try:
        return np.asarray(valores).reshape(forma)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroNumerico(f"não foi possível remodelar os valores: {exc}") from exc


def concatenar(valores, eixo=0):
    np = _numpy()
    try:
        partes = list(valores)
    except TypeError as exc:
        raise ErroNumerico("concatenar requer uma coleção de arrays") from exc
    if not partes:
        raise ErroNumerico("concatenar requer ao menos uma coleção")
    try:
        return np.concatenate(partes, axis=eixo)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroNumerico(f"não foi possível concatenar no eixo {eixo}: {exc}") from
          exc


def selecionar_coluna(registros, campo, ausentes="erro"):
    if not isinstance(campo, str) or not campo:
        raise ErroNumerico("campo deve ser um nome de campo não vazio")
    if ausentes not in {"erro", "ignorar"}:
        raise ErroNumerico("ausentes deve ser 'erro' ou 'ignorar'")
    try:
        itens = list(registros)
    except TypeError as exc:
        raise ErroNumerico("selecionar_coluna requer uma coleção de registros") from exc
    saida = []
    for indice, registro in enumerate(itens):
        encontrado = False
        valor = None
        if isinstance(registro, Mapping):
            if campo in registro:
                valor = registro[campo]
                encontrado = True
        elif hasattr(registro, campo):
            valor = getattr(registro, campo)
            encontrado = True
        if not encontrado or _eh_ausente(valor):
            if ausentes == "ignorar":
                continue
```

```
            raise ErroNumerico(f"registro {indice} não possui valor utilizável para o
              campo '{campo}'")
        saida.append(valor)
    return saida


def media(valores):
    return _numpy().mean(_exigir_dados_finitos(valores, "média"))


def mediana(valores):
    return _numpy().median(_exigir_dados_finitos(valores, "mediana"))


def variancia(valores, populacional=True):
    a = _exigir_dados_finitos(valores, "variância")
    if not populacional and a.size < 2:
        raise ErroNumerico("variância amostral requer ao menos dois valores")
    return _numpy().var(a, ddof=0 if populacional else 1)


def desvio_padrao(valores, populacional=True):
    a = _exigir_dados_finitos(valores, "desvio padrão")
    if not populacional and a.size < 2:
        raise ErroNumerico("desvio padrão amostral requer ao menos dois valores")
    return _numpy().std(a, ddof=0 if populacional else 1)


def minimo(valores):
    return _numpy().min(_exigir_dados_finitos(valores, "mínimo"))


def maximo(valores):
    return _numpy().max(_exigir_dados_finitos(valores, "máximo"))


def percentil(valores, percentual):
    a = _serie_real_finita(valores, "percentil")
    try:
        p = float(percentual)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroNumerico("percentual deve ser um número entre 0 e 100") from exc
    if not math.isfinite(p) or p < 0 or p > 100:
        raise ErroNumerico("percentual deve estar entre 0 e 100")
    return _exigir_resultado_finito(_percentil_linear_serie(a, p), "percentil")


def covariancia(a, b, populacional=True):
    np = _numpy()
    x = _serie_real_finita(a, "covariância", minimo_itens=1 if populacional else 2)
    y = _serie_real_finita(b, "covariância", minimo_itens=1 if populacional else 2)
    if x.size != y.size:
        raise ErroNumerico("covariância requer amostras com o mesmo comprimento")
    if not populacional and x.size < 2:
        raise ErroNumerico("covariância amostral requer ao menos dois pares")
    dx, escala_x = _centralizar_escalado(x, "covariância")
    dy, escala_y = _centralizar_escalado(y, "covariância")
    divisor = x.size if populacional else x.size - 1
    normalizada = float(np.sum(dx * dy) / divisor)
    try:
        resultado = normalizada * escala_x * escala_y
```

```
    except OverflowError as exc:
        raise ErroNumerico("covariância produziu resultado fora do intervalo numérico
          suportado") from exc
    return _exigir_resultado_finito(resultado, "covariância")


def correlacao(a, b):
    np = _numpy()
    x = _serie_real_finita(a, "correlação", minimo_itens=2)
    y = _serie_real_finita(b, "correlação", minimo_itens=2)
    if x.size != y.size:
        raise ErroNumerico("correlação requer amostras com o mesmo comprimento")
    dx, _ = _centralizar_escalado(x, "correlação")
    dy, _ = _centralizar_escalado(y, "correlação")
    sx = float(np.sum(dx * dx))
    sy = float(np.sum(dy * dy))
    if sx == 0 or sy == 0:
        raise ErroNumerico("correlação é indefinida quando uma amostra tem variância
          zero")
    resultado = float(np.sum(dx * dy) / math.sqrt(sx * sy))
    return _exigir_resultado_finito(resultado, "correlação")


def frequencias(valores):
    try:
        itens = list(valores)
    except TypeError as exc:
        raise ErroNumerico("frequências requer uma coleção de valores") from exc
    if not itens:
        raise ErroNumerico("frequências requer ao menos um valor")
    contagens = {}
    for valor in itens:
        if _eh_ausente(valor):
            raise ErroNumerico("frequências não aceita valores ausentes ou NaN")
        try:
            if not isinstance(valor, (str, bytes, bytearray)) and not
              math.isfinite(valor):
                raise ErroNumerico("frequências não aceita infinito")
        except TypeError:
            pass
        try:
            contagens[valor] = contagens.get(valor, 0) + 1
        except TypeError as exc:
            raise ErroNumerico("frequências requer valores comparáveis e utilizáveis
              como categorias") from exc
    return contagens


def moda(valores):
    contagens = frequencias(valores)
    maior = max(contagens.values())
    return [valor for valor, quantidade in contagens.items() if quantidade == maior]


def quartis(valores):
    a = _serie_real_finita(valores, "quartis")
    return tuple(_exigir_resultado_finito(_percentil_linear_serie(a, p), "quartis") for
      p in (25.0, 50.0, 75.0))


def amplitude(valores):
```

```
    np = _numpy()
    a = _serie_real_finita(valores, "amplitude")
    with np.errstate(over="ignore", invalid="ignore"):
        resultado = np.max(a) - np.min(a)
    return _exigir_resultado_finito(resultado, "amplitude")


def intervalo_interquartil(valores):
    q1, _, q3 = quartis(valores)
    return q3 - q1


def mediana_desvios_absolutos(valores):
    np = _numpy()
    a = _serie_real_finita(valores, "mediana dos desvios absolutos")
    centro = np.median(a)
    return np.median(np.abs(a - centro))


def media_ponderada(valores, pesos):
    np = _numpy()
    a = _serie_real_finita(valores, "média ponderada")
    w = _serie_real_finita(pesos, "média ponderada")
    if a.size != w.size:
        raise ErroNumerico("média ponderada requer valores e pesos com o mesmo
          comprimento")
    if bool(np.any(w < 0)):
        raise ErroNumerico("média ponderada requer pesos não negativos")
    maior_peso = float(np.max(w))
    if maior_peso <= 0:
        raise ErroNumerico("média ponderada requer soma de pesos positiva")
    normalizados = w / maior_peso
    total = float(np.sum(normalizados))
    coeficientes = normalizados / total
    resultado = float(np.sum(a * coeficientes))
    return _exigir_resultado_finito(resultado, "média ponderada")


def media_geometrica(valores):
    np = _numpy()
    a = _serie_real_finita(valores, "média geométrica")
    if bool(np.any(a <= 0)):
        raise ErroNumerico("média geométrica requer valores estritamente positivos")
    return np.exp(np.mean(np.log(a)))


def media_harmonica(valores):
    np = _numpy()
    a = _serie_real_finita(valores, "média harmônica")
    if bool(np.any(a <= 0)):
        raise ErroNumerico("média harmônica requer valores estritamente positivos")
    escala = float(np.min(a))
    denominador = float(np.sum(escala / a))
    resultado = escala * (a.size / denominador)
    return _exigir_resultado_finito(resultado, "média harmônica")


def histograma(valores, faixas):
    np = _numpy()
    a = _serie_real_finita(valores, "histograma")
    bins = faixas
```

```
    if isinstance(faixas, bool):
        raise ErroNumerico("faixas do histograma deve ser inteiro positivo ou sequência
          de bordas")
    try:
        quantidade = operator.index(faixas)
    except TypeError:
        try:
            bordas = np.asarray(list(faixas), dtype=float)
        except (TypeError, ValueError, OverflowError) as exc:
            raise ErroNumerico("faixas do histograma deve ser inteiro positivo ou
              sequência de bordas") from exc
        if bordas.ndim != 1 or bordas.size < 2:
            raise ErroNumerico("bordas do histograma requerem ao menos dois valores")
        if not bool(np.all(np.isfinite(bordas))):
            raise ErroNumerico("bordas do histograma devem ser finitas")
        if not bool(np.all(np.diff(bordas) > 0)):
            raise ErroNumerico("bordas do histograma devem estar em ordem estritamente
              crescente")
        bins = bordas
    else:
        if quantidade <= 0:
            raise ErroNumerico("quantidade de faixas do histograma deve ser positiva")
        bins = quantidade
    try:
        return np.histogram(a, bins=bins)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroNumerico(f"não foi possível construir o histograma: {exc}") from exc




def norma(valores):
    """Norma euclidiana de um vetor finito e não vazio."""
    np = _numpy()
    a = _vetor_real_finito(valores, "norma")
    try:
        valor = float(np.linalg.norm(a, ord=2))
    except (TypeError, ValueError, np.linalg.LinAlgError, OverflowError) as exc:
        raise ErroNumerico(f"não foi possível calcular a norma: {exc}") from exc
    if not math.isfinite(valor):
        raise ErroNumerico("norma produziu resultado fora do intervalo numérico
          suportado")
    return valor


def normalizar(valores):
    """Devolve uma cópia do vetor com norma euclidiana igual a um."""
    a = _vetor_real_finito(valores, "normalizar")
    comprimento = norma(a)
    if comprimento == 0.0:
        raise ErroNumerico("normalizar não é definido para vetor de norma zero")
    resultado = a / comprimento
    if not bool(_numpy().all(_numpy().isfinite(resultado))):
        raise ErroNumerico("normalizar produziu resultado fora do intervalo numérico
          suportado")
    return resultado


def produto_vetorial(a, b):
    """Produto vetorial exclusivo de vetores tridimensionais."""
    np = _numpy()
```

```
    va = _vetor_real_finito(a, "produto vetorial", dimensao=3)
    vb = _vetor_real_finito(b, "produto vetorial", dimensao=3)
    resultado = np.cross(va, vb)
    if not bool(np.all(np.isfinite(resultado))):
        raise ErroNumerico("produto vetorial produziu resultado fora do intervalo
          numérico suportado")
    return resultado


def posto(matriz):
    """Posto numérico de uma matriz pela tolerância padrão da decomposição SVD do
      NumPy."""
    np = _numpy()
    a = _matriz_real_finita(matriz, "posto")
    try:
        return int(np.linalg.matrix_rank(a))
    except (TypeError, ValueError, np.linalg.LinAlgError, OverflowError) as exc:
        raise ErroNumerico(f"não foi possível calcular o posto: {exc}") from exc


def minimos_quadrados(coeficientes, termos):
    """Resolve um ajuste linear por mínimos quadrados sem exigir sistema quadrado."""
    np = _numpy()
    a = _matriz_real_finita(coeficientes, "mínimos quadrados")
    b = _materializar_array(termos, "mínimos quadrados", tipo=float)
    if b.ndim not in (1, 2):
        raise ErroNumerico("mínimos quadrados requer termos em vetor ou matriz
          bidimensional")
    if b.size == 0:
        raise ErroNumerico("mínimos quadrados requer termos não vazios")
    if b.shape[0] != a.shape[0]:
        raise ErroNumerico("mínimos quadrados requer a mesma quantidade de linhas em
          coeficientes e termos")
    if not bool(np.all(np.isfinite(b))):
        raise ErroNumerico("mínimos quadrados não aceita NaN ou infinito")
    try:
        solucao, residuos, rank, singulares = np.linalg.lstsq(a, b, rcond=None)
    except (TypeError, ValueError, np.linalg.LinAlgError, OverflowError) as exc:
        raise ErroNumerico(f"não foi possível calcular mínimos quadrados: {exc}") from
          exc
    if not bool(np.all(np.isfinite(solucao))) or not
      bool(np.all(np.isfinite(singulares))):
        raise ErroNumerico("mínimos quadrados produziu resultado fora do intervalo
          numérico suportado")
    if residuos.size and not bool(np.all(np.isfinite(residuos))):
        raise ErroNumerico("mínimos quadrados produziu resíduos fora do intervalo
          numérico suportado")
    return ResultadoMinimosQuadrados(solucao, residuos, int(rank), singulares)


def vetor3(x, y, z):
    """Constrói um vetor numérico tridimensional ``(x, y, z)``."""
    return _vetor_real_finito([x, y, z], "vetor3", dimensao=3)


def distancia_vetores(a, b):
    """Distância euclidiana entre vetores finitos da mesma dimensão."""
    va = _vetor_real_finito(a, "distância entre vetores")
    vb = _vetor_real_finito(b, "distância entre vetores")
    if va.shape != vb.shape:
        raise ErroNumerico("distância entre vetores requer vetores da mesma dimensão")
```

```
    return norma(va - vb)


def angulo_entre_vetores(a, b):
    """Ângulo em radianos entre vetores não nulos da mesma dimensão."""
    va = _vetor_real_finito(a, "ângulo entre vetores")
    vb = _vetor_real_finito(b, "ângulo entre vetores")
    if va.shape != vb.shape:
        raise ErroNumerico("ângulo entre vetores requer vetores da mesma dimensão")
    na, nb = norma(va), norma(vb)
    if na == 0.0 or nb == 0.0:
        raise ErroNumerico("ângulo entre vetores não é definido para vetor de norma
          zero")
    cosseno = float(_numpy().dot(va, vb) / (na * nb))
    cosseno = max(-1.0, min(1.0, cosseno))
    return math.acos(cosseno)


def projecao_vetorial(a, sobre):
    """Projeta ``a`` sobre um vetor não nulo, preservando a dimensão."""
    va = _vetor_real_finito(a, "projeção vetorial")
    vb = _vetor_real_finito(sobre, "projeção vetorial")
    if va.shape != vb.shape:
        raise ErroNumerico("projeção vetorial requer vetores da mesma dimensão")
    denominador = float(_numpy().dot(vb, vb))
    if denominador == 0.0:
        raise ErroNumerico("projeção vetorial não é definida sobre vetor de norma zero")
    resultado = (float(_numpy().dot(va, vb)) / denominador) * vb
    if not bool(_numpy().all(_numpy().isfinite(resultado))):
        raise ErroNumerico("projeção vetorial produziu resultado fora do intervalo
          numérico suportado")
    return resultado


def produto_misto3(a, b, c):
    """Produto misto orientado de três vetores tridimensionais."""
    va = _vetor_real_finito(a, "produto misto 3D", dimensao=3)
    vb = _vetor_real_finito(b, "produto misto 3D", dimensao=3)
    vc = _vetor_real_finito(c, "produto misto 3D", dimensao=3)
    valor = float(_numpy().dot(va, produto_vetorial(vb, vc)))
    if not math.isfinite(valor):
        raise ErroNumerico("produto misto 3D produziu resultado fora do intervalo
          numérico suportado")
    return valor


def volume_paralelepipedo3(a, b, c):
    """Volume do paralelepípedo gerado por três vetores 3D."""
    return abs(produto_misto3(a, b, c))


def rotacionar3(vetor, eixo, angulo):
    """Rotaciona um vetor 3D em torno de um eixo não nulo, em radianos.

    Usa a fórmula de Rodrigues e preserva a norma dentro da precisão do
    backend numérico. O eixo define orientação pela regra da mão direita.
    """
    np = _numpy()
    v = _vetor_real_finito(vetor, "rotação 3D", dimensao=3)
    k = _vetor_real_finito(eixo, "rotação 3D", dimensao=3)
    try:
```

```
        theta = float(angulo)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroNumerico("rotação 3D requer ângulo numérico finito em radianos") from
          exc
    if not math.isfinite(theta):
        raise ErroNumerico("rotação 3D requer ângulo numérico finito em radianos")
    comprimento_eixo = norma(k)
    if comprimento_eixo == 0.0:
        raise ErroNumerico("rotação 3D não é definida para eixo de norma zero")
    unidade = k / comprimento_eixo
    c = math.cos(theta)
    sen = math.sin(theta)
    resultado = v * c + np.cross(unidade, v) * sen + unidade * float(np.dot(unidade, v))
       * (1.0 - c)
    if not bool(np.all(np.isfinite(resultado))):
        raise ErroNumerico("rotação 3D produziu resultado fora do intervalo numérico
          suportado")
    return resultado


def transformar_ponto3(ponto, transformacao):
    """Aplica uma transformação afim homogênea 4 × 4 a um ponto 3D.

    A convenção pública usa vetor coluna ``[x, y, z, 1]``. A translação vive
    na última coluna e a última linha deve ser ``[0, 0, 0, 1]`` dentro de
    tolerância, rejeitando matrizes projetivas nesta API.
    """
    np = _numpy()
    p = _vetor_real_finito(ponto, "transformação de ponto 3D", dimensao=3)
    m = _matriz_real_finita(transformacao, "transformação de ponto 3D")
    if m.shape != (4, 4):
        raise ErroNumerico("transformação de ponto 3D requer matriz homogênea 4 × 4")
    if not bool(np.allclose(m[3], np.array([0.0, 0.0, 0.0, 1.0]), rtol=0.0,
      atol=1e-12)):
        raise ErroNumerico("transformação de ponto 3D requer última linha afim [0, 0, 0,
           1]")
    homogeneo = np.array([p[0], p[1], p[2], 1.0], dtype=float)
    resultado = m @ homogeneo
    if not bool(np.all(np.isfinite(resultado))):
        raise ErroNumerico("transformação de ponto 3D produziu resultado fora do
          intervalo numérico suportado")
    if not math.isclose(float(resultado[3]), 1.0, rel_tol=0.0, abs_tol=1e-12):
        raise ErroNumerico("transformação afim 3D produziu coordenada homogênea
          inválida")
    return resultado[:3].copy()




def matriz_rotacao3(eixo, angulo):
    """Matriz 3 × 3 de Rodrigues para rotação em torno de um eixo 3D."""
    np = _numpy()
    k = _vetor_real_finito(eixo, "matriz de rotação 3D", dimensao=3)
    try:
        theta = float(angulo)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroNumerico("matriz de rotação 3D requer ângulo numérico finito em
          radianos") from exc
    if not math.isfinite(theta):
        raise ErroNumerico("matriz de rotação 3D requer ângulo numérico finito em
          radianos")
```

```
    comprimento = norma(k)
    if comprimento == 0.0:
        raise ErroNumerico("matriz de rotação 3D não é definida para eixo de norma
          zero")
    x, y, z = (k / comprimento).tolist()
    c, sen, t = math.cos(theta), math.sin(theta), 1.0 - math.cos(theta)
    resultado = np.array([
        [t*x*x+c, t*x*y-sen*z, t*x*z+sen*y],
        [t*x*y+sen*z, t*y*y+c, t*y*z-sen*x],
        [t*x*z-sen*y, t*y*z+sen*x, t*z*z+c],
    ], dtype=float)
    if not bool(np.all(np.isfinite(resultado))):
        raise ErroNumerico("matriz de rotação 3D produziu resultado fora do intervalo
          numérico suportado")
    return resultado

def matriz_translacao3(dx, dy, dz):
    """Matriz afim 4 × 4 de translação 3D em convenção de vetor coluna."""
    np = _numpy()
    d = _vetor_real_finito([dx, dy, dz], "matriz de translação 3D", dimensao=3)
    resultado = np.eye(4, dtype=float)
    resultado[:3, 3] = d
    return resultado

def matriz_escala3(sx, sy=None, sz=None):
    """Matriz afim 4 × 4 de escala uniforme ou independente por eixo."""
    np = _numpy()
    if sy is None and sz is None:
        fatores = _vetor_real_finito([sx, sx, sx], "matriz de escala 3D", dimensao=3)
    elif sy is None or sz is None:
        raise ErroNumerico("matriz de escala 3D requer um fator uniforme ou três
          fatores")
    else:
        fatores = _vetor_real_finito([sx, sy, sz], "matriz de escala 3D", dimensao=3)
    resultado = np.eye(4, dtype=float)
    resultado[0, 0], resultado[1, 1], resultado[2, 2] = fatores.tolist()
    return resultado

def _transformacao_afim3(transformacao, contexto: str):
    np = _numpy()
    m = _matriz_real_finita(transformacao, contexto)
    if m.shape != (4, 4):
        raise ErroNumerico(f"{contexto} requer matriz homogênea 4 × 4")
    if not bool(np.allclose(m[3], np.array([0.0, 0.0, 0.0, 1.0]), rtol=0.0,
      atol=1e-12)):
        raise ErroNumerico(f"{contexto} requer última linha afim [0, 0, 0, 1]")
    return m

def compor_transformacoes3(*transformacoes):
    """Compõe transformações 3D na ordem em que são aplicadas ao ponto."""
    np = _numpy()
    resultado = np.eye(4, dtype=float)
    for indice, transformacao in enumerate(transformacoes, start=1):
        atual = _transformacao_afim3(transformacao, f"composição 3D: transformação
          {indice}")
        resultado = atual @ resultado
    if not bool(np.all(np.isfinite(resultado))):
        raise ErroNumerico("composição 3D produziu resultado fora do intervalo numérico
          suportado")
    return resultado
```

```
def transformar_vetor3(vetor, transformacao):
    """Aplica a parte linear de uma transformação afim, ignorando translação."""
    np = _numpy()
    v = _vetor_real_finito(vetor, "transformação de vetor 3D", dimensao=3)
    m = _transformacao_afim3(transformacao, "transformação de vetor 3D")
    resultado = m[:3, :3] @ v
    if not bool(np.all(np.isfinite(resultado))):
        raise ErroNumerico("transformação de vetor 3D produziu resultado fora do
          intervalo numérico suportado")
    return resultado

def distancia_ponto_plano3(ponto, ponto_plano, normal):
    """Distância euclidiana não negativa entre um ponto e um plano 3D."""
    p = _vetor_real_finito(ponto, "distância ponto plano 3D", dimensao=3)
    q = _vetor_real_finito(ponto_plano, "distância ponto plano 3D", dimensao=3)
    n = _vetor_real_finito(normal, "distância ponto plano 3D", dimensao=3)
    comprimento = norma(n)
    if comprimento == 0.0:
        raise ErroNumerico("distância ponto plano 3D requer normal de norma não nula")
    valor = abs(float(_numpy().dot(p - q, n))) / comprimento
    if not math.isfinite(valor):
        raise ErroNumerico("distância ponto plano 3D produziu resultado fora do
          intervalo numérico suportado")
    return valor

def projetar_ponto_plano3(ponto, ponto_plano, normal):
    """Projeção ortogonal de um ponto sobre um plano 3D."""
    np = _numpy()
    p = _vetor_real_finito(ponto, "projeção ponto plano 3D", dimensao=3)
    q = _vetor_real_finito(ponto_plano, "projeção ponto plano 3D", dimensao=3)
    n = _vetor_real_finito(normal, "projeção ponto plano 3D", dimensao=3)
    denominador = float(np.dot(n, n))
    if denominador == 0.0:
        raise ErroNumerico("projeção ponto plano 3D requer normal de norma não nula")
    resultado = p - (float(np.dot(p - q, n)) / denominador) * n
    if not bool(np.all(np.isfinite(resultado))):
        raise ErroNumerico("projeção ponto plano 3D produziu resultado fora do intervalo
           numérico suportado")
    return resultado

def intersecao_reta_plano3(origem, direcao, ponto_plano, normal, *, tolerancia=1e-12):
    """Interseção reta plano; retorna None para paralelas distintas."""
    np = _numpy()
    o = _vetor_real_finito(origem, "interseção reta plano 3D", dimensao=3)
    d = _vetor_real_finito(direcao, "interseção reta plano 3D", dimensao=3)
    q = _vetor_real_finito(ponto_plano, "interseção reta plano 3D", dimensao=3)
    n = _vetor_real_finito(normal, "interseção reta plano 3D", dimensao=3)
    if norma(d) == 0.0:
        raise ErroNumerico("interseção reta plano 3D requer direção de norma não nula")
    if norma(n) == 0.0:
        raise ErroNumerico("interseção reta plano 3D requer normal de norma não nula")
    try:
        tol = float(tolerancia)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroNumerico("interseção reta plano 3D requer tolerância finita não
          negativa") from exc
    if not math.isfinite(tol) or tol < 0.0:
        raise ErroNumerico("interseção reta plano 3D requer tolerância finita não
          negativa")
    denominador, numerador = float(np.dot(d, n)), float(np.dot(q - o, n))
    if abs(denominador) <= tol * max(1.0, norma(d) * norma(n)):
```

```
        if abs(numerador) <= tol * max(1.0, norma(q - o) * norma(n)):
            raise ErroNumerico("interseção reta plano 3D não é única porque a reta
              pertence ao plano")
        return None
    resultado = o + (numerador / denominador) * d
    if not bool(np.all(np.isfinite(resultado))):
        raise ErroNumerico("interseção reta plano 3D produziu resultado fora do
          intervalo numérico suportado")
    return resultado

def coordenadas_esfericas3(ponto):
    """Converte (x, y, z) em (raio, azimute, elevação), em radianos."""
    p = _vetor_real_finito(ponto, "coordenadas esféricas 3D", dimensao=3)
    x, y, z = map(float, p)
    raio = norma(p)
    if raio == 0.0:
        return (0.0, 0.0, 0.0)
    return (float(raio), float(math.atan2(y, x)), float(math.atan2(z, math.hypot(x,
      y))))

def ponto_de_esfericas3(raio, azimute, elevacao):
    """Converte (raio, azimute, elevação) para ponto cartesiano 3D."""
    np = _numpy()
    valores = []
    for nome, valor in (("raio", raio), ("azimute", azimute), ("elevação", elevacao)):
        try:
            numero = float(valor)
        except (TypeError, ValueError, OverflowError) as exc:
            raise ErroNumerico(f"coordenadas esféricas 3D requer {nome} finito") from
              exc
        if not math.isfinite(numero):
            raise ErroNumerico(f"coordenadas esféricas 3D requer {nome} finito")
        valores.append(numero)
    r, az, el = valores
    if r < 0.0:
        raise ErroNumerico("coordenadas esféricas 3D requer raio não negativo")
    cos_el = math.cos(el)
    resultado = np.array([r*cos_el*math.cos(az), r*cos_el*math.sin(az), r*math.sin(el)],
       dtype=float)
    if not bool(np.all(np.isfinite(resultado))):
        raise ErroNumerico("coordenadas esféricas 3D produziram resultado fora do
          intervalo numérico suportado")
    return resultado

def area_triangulo3(a, b, c):
    """Área do triângulo definido por três pontos 3D."""
    pa = _vetor_real_finito(a, "área de triângulo 3D", dimensao=3)
    pb = _vetor_real_finito(b, "área de triângulo 3D", dimensao=3)
    pc = _vetor_real_finito(c, "área de triângulo 3D", dimensao=3)
    valor = 0.5 * norma(produto_vetorial(pb - pa, pc - pa))
    if not math.isfinite(valor):
        raise ErroNumerico("área de triângulo 3D produziu resultado fora do intervalo
          numérico suportado")
    return valor

def volume_tetraedro3(a, b, c, d):
    """Volume do tetraedro definido por quatro pontos 3D."""
    pa = _vetor_real_finito(a, "volume de tetraedro 3D", dimensao=3)
    pb = _vetor_real_finito(b, "volume de tetraedro 3D", dimensao=3)
    pc = _vetor_real_finito(c, "volume de tetraedro 3D", dimensao=3)
    pd = _vetor_real_finito(d, "volume de tetraedro 3D", dimensao=3)
```

```
    valor = abs(produto_misto3(pb - pa, pc - pa, pd - pa)) / 6.0
    if not math.isfinite(valor):
        raise ErroNumerico("volume de tetraedro 3D produziu resultado fora do intervalo
          numérico suportado")
    return valor

def erro_padrao(valores):
    """Calcula o erro padrão da média usando o desvio amostral.

    Requer ao menos duas observações finitas. A convenção usa ``n - 1`` no
    desvio padrão e divide o resultado por ``sqrt(n)``.
    """
    np = _numpy()
    a = _serie_real_finita(valores, "erro padrão", minimo_itens=2)
    desvios, escala = _centralizar_escalado(a, "erro padrão")
    if escala == 0.0:
        return 0.0
    variancia_amostral_normalizada = float(np.sum(desvios * desvios) / (a.size - 1))
    resultado = math.sqrt(variancia_amostral_normalizada) * escala / math.sqrt(a.size)
    return float(_exigir_resultado_finito(resultado, "erro padrão"))


def assimetria(valores):
    """Momento padronizado de ordem 3, sem correção de viés amostral."""
    np = _numpy()
    a = _serie_real_finita(valores, "assimetria", minimo_itens=2)
    desvios, _ = _centralizar_escalado(a, "assimetria")
    m2 = float(np.mean(desvios ** 2))
    if m2 == 0.0:
        raise ErroNumerico("assimetria é indefinida para variância zero")
    m3 = float(np.mean(desvios ** 3))
    resultado = m3 / (m2 ** 1.5)
    return float(_exigir_resultado_finito(resultado, "assimetria"))


def curtose(valores):
    """Curtose em excesso pelo quarto momento padronizado, isto é, normal = 0."""
    np = _numpy()
    a = _serie_real_finita(valores, "curtose", minimo_itens=2)
    desvios, _ = _centralizar_escalado(a, "curtose")
    m2 = float(np.mean(desvios ** 2))
    if m2 == 0.0:
        raise ErroNumerico("curtose é indefinida para variância zero")
    m4 = float(np.mean(desvios ** 4))
    resultado = (m4 / (m2 * m2)) - 3.0
    return float(_exigir_resultado_finito(resultado, "curtose"))


def _matriz_observacoes(dados, operacao: str, *, minimo_linhas: int = 2):
    m = _matriz_real_finita(dados, operacao)
    if m.shape[0] < minimo_linhas:
        raise ErroNumerico(f"{operacao} requer ao menos {minimo_linhas} observações")
    if m.shape[1] < 1:
        raise ErroNumerico(f"{operacao} requer ao menos uma variável")
    return m


def _normalizar_colunas_centradas(m, operacao: str):
    np = _numpy()
    escalas = np.max(np.abs(m), axis=0)
    normalizada = np.zeros_like(m, dtype=float)
```

```
    nao_zero = escalas != 0.0
    normalizada[:, nao_zero] = m[:, nao_zero] / escalas[nao_zero]
    centros = np.mean(normalizada, axis=0)
    desvios = normalizada - centros
    if not bool(np.all(np.isfinite(desvios))):
        raise ErroNumerico(f"{operacao} produziu intermediários fora do intervalo
          numérico suportado")
    return desvios, escalas


def matriz_covariancia(dados, populacional=False):
    """Matriz de covariância com linhas como observações e colunas como variáveis.

    Por padrão usa a convenção amostral, dividindo por ``n - 1``. Defina
    ``populacional=True`` para dividir por ``n``.
    """
    np = _numpy()
    m = _matriz_observacoes(dados, "matriz de covariância")
    if not isinstance(populacional, bool):
        raise ErroNumerico("populacional deve ser verdadeiro ou falso")
    if not populacional and m.shape[0] < 2:
        raise ErroNumerico("matriz de covariância amostral requer ao menos duas
          observações")
    desvios, escalas = _normalizar_colunas_centradas(m, "matriz de covariância")
    divisor = m.shape[0] if populacional else m.shape[0] - 1
    base = (desvios.T @ desvios) / divisor
    with np.errstate(over="ignore", invalid="ignore"):
        resultado = base * np.outer(escalas, escalas)
    if not bool(np.all(np.isfinite(resultado))):
        raise ErroNumerico("matriz de covariância produziu resultado fora do intervalo
          numérico suportado")
    return resultado


def matriz_correlacao(dados):
    """Matriz de correlação de Pearson para colunas observadas conjuntamente."""
    np = _numpy()
    m = _matriz_observacoes(dados, "matriz de correlação")
    desvios, _ = _normalizar_colunas_centradas(m, "matriz de correlação")
    somas_quadrados = np.sum(desvios * desvios, axis=0)
    if bool(np.any(somas_quadrados == 0.0)):
        indices = [str(i) for i, valor in enumerate(somas_quadrados) if valor == 0.0]
        raise ErroNumerico(
            "matriz de correlação é indefinida para variável com variância zero "
            f"(coluna(s) {', '.join(indices)})"
        )
    denominadores = np.sqrt(np.outer(somas_quadrados, somas_quadrados))
    resultado = (desvios.T @ desvios) / denominadores
    resultado = np.clip(resultado, -1.0, 1.0)
    np.fill_diagonal(resultado, 1.0)
    if not bool(np.all(np.isfinite(resultado))):
        raise ErroNumerico("matriz de correlação produziu resultado numérico inválido")
    return resultado


def regressao_linear(x, y):
    """Ajusta ``y = intercepto + inclinacao * x`` por mínimos quadrados simples."""
    np = _numpy()
    xs = _serie_real_finita(x, "regressão linear: x", minimo_itens=2)
    ys = _serie_real_finita(y, "regressão linear: y", minimo_itens=2)
    if xs.size != ys.size:
```

```
        raise ErroNumerico("regressão linear requer x e y com o mesmo comprimento")

    dx, escala_x = _centralizar_escalado(xs, "regressão linear")
    dy, escala_y = _centralizar_escalado(ys, "regressão linear")
    soma_x2 = float(np.sum(dx * dx))
    if soma_x2 == 0.0 or escala_x == 0.0:
        raise ErroNumerico("regressão linear é indefinida quando x é constante")

    razao_normalizada = float(np.sum(dx * dy) / soma_x2) if escala_y != 0.0 else 0.0
    inclinacao = (escala_y / escala_x) * razao_normalizada if escala_y != 0.0 else 0.0
    media_x = float(np.mean(xs / escala_x) * escala_x)
    if escala_y == 0.0:
        media_y = 0.0
    else:
        media_y = float(np.mean(ys / escala_y) * escala_y)
    intercepto = media_y - inclinacao * media_x
    _exigir_resultado_finito(inclinacao, "regressão linear")
    _exigir_resultado_finito(intercepto, "regressão linear")

    if escala_y == 0.0:
        residuos = np.zeros_like(ys, dtype=float)
        r_quadrado = 1.0
    else:
        residuos_normalizados = dy - razao_normalizada * dx
        residuos = residuos_normalizados * escala_y
        if not bool(np.all(np.isfinite(residuos))):
            raise ErroNumerico("regressão linear produziu resíduos fora do intervalo
              numérico suportado")
        sse = float(np.sum(residuos_normalizados * residuos_normalizados))
        sst = float(np.sum(dy * dy))
        r_quadrado = 1.0 if sst == 0.0 else 1.0 - (sse / sst)
        r_quadrado = min(1.0, max(0.0, float(r_quadrado)))

    return ResultadoRegressaoLinear(
        inclinacao=float(inclinacao),
        intercepto=float(intercepto),
        residuos=residuos.copy(),
        r_quadrado=float(r_quadrado),
        quantidade=int(xs.size),
    )


def _semente_inteira(semente, operacao: str) -> int:
    if isinstance(semente, bool):
        raise ErroNumerico(f"{operacao} requer semente inteira")
    try:
        valor = operator.index(semente)
    except TypeError as exc:
        raise ErroNumerico(f"{operacao} requer semente inteira") from exc
    return int(valor)


def _avaliar_estatistica(estatistica, valores, operacao: str) -> float:
    if not callable(estatistica):
        raise ErroNumerico(f"{operacao} requer uma estatística chamável")
    try:
        bruto = estatistica(list(valores))
    except ErroCoral:
        raise
    except Exception as exc:
        raise ErroNumerico(f"{operacao}: a estatística falhou: {exc}") from exc
```

```
    try:
        numero = float(bruto)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroNumerico(f"{operacao}: a estatística deve devolver um número real")
          from exc
    return float(_exigir_resultado_finito(numero, operacao))


def _validar_reamostragem(repeticoes, tamanho: int, operacao: str) -> tuple[int, int]:
    r = _inteiro_exato(repeticoes, "repeticoes", minimo=1)
    custo = r * tamanho
    if custo > _LIMITE_TRABALHO_REAMOSTRAGEM:
        raise ErroNumerico(
            f"{operacao} excede o limite de trabalho: {custo} observações reamostradas;
              "
            f"limite {_LIMITE_TRABALHO_REAMOSTRAGEM}"
        )
    return r, custo


def intervalo_bootstrap(valores, estatistica, confianca, repeticoes, semente):
    """Intervalo bootstrap percentil reproduzível com gerador aleatório local."""
    np = _numpy()
    a = _serie_real_finita(valores, "intervalo bootstrap", minimo_itens=2)
    try:
        c = float(confianca)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ErroNumerico("confiança do bootstrap deve ser um número entre 0 e 1") from
           exc
    if not math.isfinite(c) or not (0.0 < c < 1.0):
        raise ErroNumerico("confiança do bootstrap deve estar estritamente entre 0 e 1")
    seed = _semente_inteira(semente, "intervalo bootstrap")
    r, custo = _validar_reamostragem(repeticoes, int(a.size), "intervalo bootstrap")
    dados = [float(v) for v in a]
    estimativa = _avaliar_estatistica(estatistica, dados, "intervalo bootstrap")
    gerador = random.Random(seed)
    amostras = []
    n = len(dados)
    for _ in range(r):
        reamostra = [dados[gerador.randrange(n)] for _ in range(n)]
        amostras.append(_avaliar_estatistica(estatistica, reamostra, "intervalo
          bootstrap"))
    ordenadas = np.asarray(sorted(amostras), dtype=float)
    cauda = (1.0 - c) / 2.0
    inferior = float(_percentil_linear_serie(ordenadas, cauda * 100.0))
    superior = float(_percentil_linear_serie(ordenadas, (1.0 - cauda) * 100.0))
    return ResultadoBootstrap(
        estimativa=estimativa,
        inferior=inferior,
        superior=superior,
        confianca=c,
        repeticoes=r,
        semente=seed,
        tamanho_amostra=n,
        custo_estimado=custo,
    )


def teste_permutacao(a, b, estatistica, repeticoes, semente):
    """Teste de permutação bilateral para diferença de uma estatística entre grupos."""
    amostra_a = _serie_real_finita(a, "teste de permutação: a", minimo_itens=1)
```

```
    amostra_b = _serie_real_finita(b, "teste de permutação: b", minimo_itens=1)
    dados_a = [float(v) for v in amostra_a]
    dados_b = [float(v) for v in amostra_b]
    total = len(dados_a) + len(dados_b)
    seed = _semente_inteira(semente, "teste de permutação")
    r, custo = _validar_reamostragem(repeticoes, total, "teste de permutação")
    observado = (
        _avaliar_estatistica(estatistica, dados_a, "teste de permutação")
        - _avaliar_estatistica(estatistica, dados_b, "teste de permutação")
    )
    combinado = dados_a + dados_b
    n_a = len(dados_a)
    gerador = random.Random(seed)
    extremos = 0
    tolerancia = 1e-15 * max(1.0, abs(observado))
    for _ in range(r):
        permutados = combinado.copy()
        gerador.shuffle(permutados)
        diferenca = (
            _avaliar_estatistica(estatistica, permutados[:n_a], "teste de permutação")
            - _avaliar_estatistica(estatistica, permutados[n_a:], "teste de permutação")
        )
        if abs(diferenca) + tolerancia >= abs(observado):
            extremos += 1
    valor_p = (extremos + 1.0) / (r + 1.0)
    return ResultadoTestePermutacao(
        diferenca_observada=float(observado),
        valor_p=float(valor_p),
        repeticoes=r,
        semente=seed,
        tamanho_a=len(dados_a),
        tamanho_b=len(dados_b),
        custo_estimado=custo,
    )


def transposta(valor):
    return _numpy().transpose(valor)


def produto_escalar(a, b):
    try:
        return _numpy().dot(a, b)
    except ValueError as exc:
        raise ErroNumerico(f"dimensões incompatíveis para produto escalar: {exc}") from
          exc


def multiplicar_matrizes(a, b):
    try:
        return _numpy().matmul(a, b)
    except ValueError as exc:
        raise ErroNumerico(f"dimensões incompatíveis para multiplicação de matrizes:
          {exc}") from exc


def determinante(m):
    np = _numpy()
    try:
        return np.linalg.det(m)
    except (TypeError, ValueError, np.linalg.LinAlgError) as exc:
```

```
        raise ErroNumerico(f"não foi possível calcular o determinante: {exc}") from exc


def inversa(m):
    np = _numpy()
    try:
        return np.linalg.inv(m)
    except (ValueError, np.linalg.LinAlgError) as exc:
        raise ErroNumerico(f"não foi possível inverter a matriz: {exc}") from exc


def resolver_sistema(coeficientes, termos):
    np = _numpy()
    try:
        return np.linalg.solve(coeficientes, termos)
    except (ValueError, np.linalg.LinAlgError) as exc:
        raise ErroNumerico(f"não foi possível resolver o sistema linear: {exc}") from
          exc


def autovalores(m):
    np = _numpy()
    try:
        return np.linalg.eigvals(m)
    except (TypeError, ValueError, np.linalg.LinAlgError) as exc:
        raise ErroNumerico(f"não foi possível calcular autovalores: {exc}") from exc


def forma(valor) -> tuple[int, ...]:
    return tuple(_numpy().shape(valor))


__all__ = [
    "DependenciaNumericaAusente", "ErroNumerico", "ResultadoMinimosQuadrados",
    "ResultadoRegressaoLinear", "ResultadoBootstrap", "ResultadoTestePermutacao",
    "numpy_disponivel", "diagnosticar_numerico",
    "vetor", "matriz", "zeros", "uns", "intervalo", "espaco_linear",
    "identidade", "remodelar", "concatenar", "selecionar_coluna",
    "soma", "media", "mediana", "variancia", "desvio_padrao", "minimo", "maximo",
    "percentil", "covariancia", "correlacao", "frequencias", "moda", "quartis",
    "amplitude", "intervalo_interquartil", "mediana_desvios_absolutos",
    "media_ponderada", "media_geometrica", "media_harmonica", "histograma",
    "erro_padrao", "assimetria", "curtose", "matriz_covariancia", "matriz_correlacao",
    "regressao_linear", "intervalo_bootstrap", "teste_permutacao",
    "norma", "normalizar", "produto_vetorial", "posto", "minimos_quadrados",
    "vetor3", "distancia_vetores", "angulo_entre_vetores", "projecao_vetorial",
    "produto_misto3", "volume_paralelepipedo3", "rotacionar3", "transformar_ponto3",
    "matriz_rotacao3", "matriz_translacao3", "matriz_escala3",
    "compor_transformacoes3", "transformar_vetor3",
    "distancia_ponto_plano3", "projetar_ponto_plano3", "intersecao_reta_plano3",
    "coordenadas_esfericas3", "ponto_de_esfericas3", "area_triangulo3",
      "volume_tetraedro3",
    "transposta", "produto_escalar", "multiplicar_matrizes",
    "determinante", "inversa", "resolver_sistema", "autovalores", "forma",
]
```

Biblioteca `coral.persistencia`

`coral.persistencia.CONTRATO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.persistencia/1'`

`coral.persistencia.ErroPersistencia`

Falha de validação, codificação ou reconstrução persistente.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.persistencia.VERSAO_ESCRITA`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`1`

`coral.persistencia.VERSAO_ESQUEMA`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`1`

`coral.persistencia.VERSOES_LEITURA`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`(1,)`

`coral.persistencia.aplicar_extensoes_persistentes`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
aplicar_extensoes_persistentes(objeto: 'Any', extensoes: 'Any') -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
extensoes	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

`coral.persistencia.carregar`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
carregar(caminho: 'str | Path') -> 'Any'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
caminho	str | Path	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Any

`coral.persistencia.carregar_como`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
carregar_como(caminho: 'str | Path', tipo_esperado: 'type', nome_tipo: 'str | None' =
  None) -> 'Any'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
caminho	str | Path	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipo_esperado	type	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome_tipo	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

Any

`coral.persistencia.de_dados`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
de_dados(documento: 'dict[str, Any]') -> 'Any'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
documento	dict[str, Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Any

`coral.persistencia.extensoes_para_dados`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
extensoes_para_dados(objeto: 'Any') -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

dict[str, Any]

`coral.persistencia.para_dados`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
para_dados(objeto: 'Any') -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

dict[str, Any]

`coral.persistencia.politica_persistencia`

Descreve a política executável do formato persistente corrente.

Assinatura

Assinatura para consulta

```
politica_persistencia() -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

dict[str, Any]

`coral.persistencia.registrar_adaptador`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
registrar_adaptador(tipo_python: 'type', identificador: 'str', para_dados:
  'Callable[[Any], dict[str, Any]]', de_dados: 'Callable[[dict[str, Any]], Any]') ->
  'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tipo_python	type	POSITIONAL_OR_KEYWORD	sim	sem padrão
identificador	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
para_dados	Callable[[Any], dict[str, Any]]	POSITIONAL_OR_KEYWORD	sim	sem padrão
de_dados	Callable[[dict[str, Any]], Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

`coral.persistencia.registrar_extensao_persistente`

Registra uma extensão de domínio sem acoplar o tipo hospedeiro ao módulo dono.

Assinatura

Assinatura para consulta

```
registrar_extensao_persistente(tipo_python: 'type', identificador: 'str', para_dados:
  'Callable[[Any], Any]', aplicar_dados: 'Callable[[Any, Any], None]') -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tipo_python	type	POSITIONAL_OR_KEYWORD	sim	sem padrão
identificador	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
para_dados	Callable[[Any], Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
aplicar_dados	Callable[[Any, Any], None]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

`coral.persistencia.salvar`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
salvar(objeto: 'Any', caminho: 'str | Path') -> 'Path'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
caminho	str | Path	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Path

`coral.persistencia.serializar_canonico`

Versão UTF 8 explícita da serialização canônica.

Assinatura

Assinatura para consulta

```
serializar_canonico(valor: 'Any') -> 'bytes'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

bytes

`coral.persistencia.texto_canonico`

Serialização JSON canônica para hash, cache, replay e equivalência.

Assinatura

Assinatura para consulta

```
texto_canonico(valor: 'Any') -> 'str'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

str

`coral.persistencia.valor_portatil`

Normaliza um valor isolado segundo o contrato portátil da Coral.

Diferente de :func:`para_dados`, não exige adaptador no objeto raiz. É usada por metadados e outras estruturas auxiliares que precisam validar portabilidade sem criar um novo documento persistente.

Assinatura

Assinatura para consulta

```
valor_portatil(valor: 'Any') -> 'Any'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Any

Consultar a implementação canônica de coral.persistencia

Implementação canônica de `coral.persistencia`

Código Python

```
"""Contrato único de caminhos usado pelas APIs públicas da Coral.

Texto recebido por uma operação local é interpretado como caminho nativo do
sistema atual. Objetos ``PurePath`` de outro estilo continuam sendo caminhos
portáteis, mas não são usados silenciosamente em operações locais.
"""
from __future__ import annotations

import os
from pathlib import Path, PurePath, PurePosixPath, PureWindowsPath
from typing import Any


CaminhoAceito = str | os.PathLike[str]


def estilo_caminho(valor: PurePath) -> str:
    if isinstance(valor, PureWindowsPath):
        return "windows"
    return "posix"


def estilo_nativo() -> str:
    return "windows" if os.name == "nt" else "posix"



def normalizar_caminho(
    valor: CaminhoAceito,
    *,
    expandir_usuario: bool = False,
    resolver: bool = False,
) -> Path:
    """Converte uma entrada para ``Path`` nativo com regras explícitas.

    Caminhos puros de outro sistema são recusados em operações locais. Isso
    evita transformar silenciosamente ``C:\\dados`` em um caminho relativo em
    um host POSIX, ou o inverso.
    """

    if isinstance(valor, PurePath) and estilo_caminho(valor) != estilo_nativo():
        raise ValueError(
            f"caminho de estilo {estilo_caminho(valor)} não pode ser usado "
            f"como caminho local em sistema {estilo_nativo()}"
        )
    try:
        caminho = Path(os.fspath(valor))
    except (TypeError, ValueError) as erro:
        raise TypeError("caminho precisa ser texto ou objeto compatível com
          os.PathLike") from erro
    if expandir_usuario:
        caminho = caminho.expanduser()
    if resolver:
        caminho = caminho.resolve()
```

```
    return caminho


def juntar_caminho(*partes: Any, expandir_usuario: bool = True) -> Path:
    if not partes:
        caminho = Path()
    else:
        caminho = Path(*(os.fspath(p) if isinstance(p, os.PathLike) else str(p) for p in
           partes))
    return caminho.expanduser() if expandir_usuario else caminho


def caminho_portatil(texto: str, estilo: str, *, concreto: bool = True) -> PurePath:
    """Reconstrói a categoria caminho sem fingir compatibilidade entre SOs."""

    estilo = str(estilo).casefold()
    if estilo not in {"posix", "windows"}:
        raise ValueError(f"estilo de caminho desconhecido: {estilo!r}")
    puro: PurePath = PureWindowsPath(texto) if estilo == "windows" else
      PurePosixPath(texto)
    if concreto and estilo == estilo_nativo():
        return Path(puro)
    return puro


__all__ = [
    "CaminhoAceito",
    "normalizar_caminho",
    "juntar_caminho",
    "estilo_caminho",
    "estilo_nativo",
    "caminho_portatil",
]


def _arquivo_chamador(base: CaminhoAceito | None = None) -> Path:
    if base is not None:
        caminho = normalizar_caminho(base, resolver=True)
        return caminho if caminho.suffix else caminho / "__coral_contexto__.py"
    import inspect
    este = Path(__file__).resolve()
    for quadro in inspect.stack()[1:]:
        candidato = Path(quadro.filename).resolve()
        if candidato != este:
            return candidato
    return Path.cwd().resolve() / "__coral_contexto__.py"


def _achar_raiz_contexto(arquivo: Path) -> tuple[Path, bool]:
    atual = arquivo.parent if arquivo.suffix else arquivo
    for pasta in (atual, *atual.parents):
        if (pasta / ".coral_build_manifest.json").is_file():
            return pasta, True
        if (pasta / "coral.toml").is_file():
            return pasta, False
    raise FileNotFoundError("raiz de projeto Coral não encontrada a partir do módulo
      atual")


def raiz_projeto(*, base: CaminhoAceito | None = None) -> Path:
    """Retorna a raiz portátil do projeto fonte ou do build movido."""
```

```
    raiz, _ = _achar_raiz_contexto(_arquivo_chamador(base))
    return raiz


def caminho_modulo(
    relativo: CaminhoAceito | None = None,
    *,
    base: CaminhoAceito | None = None,
) -> Path:
    """Retorna a pasta do módulo chamador, opcionalmente acrescida de um caminho."""
    arquivo = _arquivo_chamador(base)
    pasta = arquivo.parent
    return pasta if relativo is None else pasta / normalizar_caminho(relativo)


def caminho_recurso(relativo: CaminhoAceito, *, base: CaminhoAceito | None = None) ->
  Path:
    """Resolve um recurso declarado tanto na fonte quanto em um build movido."""
    arquivo = _arquivo_chamador(base)
    raiz, em_build = _achar_raiz_contexto(arquivo)
    rel = normalizar_caminho(relativo)
    if rel.is_absolute() or ".." in rel.parts:
        raise ValueError("caminho de recurso deve ser relativo e permanecer dentro do
          projeto")
    return raiz / ("_recursos" if em_build else "") / rel


__all__ += ["raiz_projeto", "caminho_modulo", "caminho_recurso"]

"""Persistência portátil e versionada da Coral.

O contrato ``coral.persistencia/1`` armazena somente dados semânticos
reconstruíveis. Estado de execução, handles e objetos arbitrários são
recusados explicitamente.
"""
from __future__ import annotations

from dataclasses import dataclass
import base64
from datetime import date, datetime, timedelta
import importlib
import json
import math
from pathlib import Path, PurePath
from typing import Any, Callable

from .caminhos import caminho_portatil, estilo_caminho, normalizar_caminho

CONTRATO = "coral.persistencia/1"
VERSAO_ESQUEMA = 1
VERSOES_LEITURA = (1,)
VERSAO_ESCRITA = 1

TIPO_VALOR_BASE = "coral.base/Valor"
TIPO_TUPLA = "coral.base/Tupla"
TIPO_CONJUNTO = "coral.base/Conjunto"
TIPO_BYTES = "coral.base/Bytes"
TIPO_DATA = "coral.base/Data"
TIPO_DATA_HORA = "coral.base/DataHora"
TIPO_DURACAO = "coral.base/Duracao"
TIPO_CAMINHO = "coral.base/Caminho"
```

```
_TIPOS_BASE = {
    TIPO_VALOR_BASE, TIPO_TUPLA, TIPO_CONJUNTO, TIPO_BYTES,
    TIPO_DATA, TIPO_DATA_HORA, TIPO_DURACAO, TIPO_CAMINHO,
}


class ErroPersistencia(ValueError):
    """Falha de validação, codificação ou reconstrução persistente."""


def politica_persistencia() -> dict[str, Any]:
    """Descreve a política executável do formato persistente corrente."""
    return {
        "contrato": CONTRATO,
        "leitura": list(VERSOES_LEITURA),
        "escrita": VERSAO_ESCRITA,
        "migracao": "recusa_explicita_de_esquema_desconhecido",
        "compatibilidade": "esquema_1_estavel",
    }


@dataclass(frozen=True)
class _Adaptador:
    tipo_python: type
    identificador: str
    para_dados: Callable[[Any], dict[str, Any]]
    de_dados: Callable[[dict[str, Any]], Any]


@dataclass(frozen=True)
class _ExtensaoPersistente:
    tipo_python: type
    identificador: str
    para_dados: Callable[[Any], Any]
    aplicar_dados: Callable[[Any, Any], None]


_POR_TIPO: dict[type, _Adaptador] = {}
_POR_ID: dict[str, _Adaptador] = {}
_EXTENSOES_POR_TIPO: dict[type, list[_ExtensaoPersistente]] = {}
_EXTENSOES_POR_ID: dict[str, _ExtensaoPersistente] = {}
_MODULOS_CONHECIDOS = {
    "coral.mundo/Entidade": "coral.mundo",
    "coral.mundo/Mundo": "coral.mundo",
    "coral.mundo/Mapa": "coral.mundo",
    "coral.agentes/Recurso": "coral.agentes",
    "coral.agentes/Modificador": "coral.agentes",
    "coral.agentes/Atributo": "coral.agentes",
    "coral.agentes/Efeito": "coral.agentes",
    "coral.agentes/Agente": "coral.agentes",
    "coral.turnos/FilaTurnos": "coral.turnos",
    "coral.simulacao/TempoMundo": "coral.simulacao",
    "coral.simulacao/FaseCiclo": "coral.simulacao",
    "coral.simulacao/Ciclo": "coral.simulacao",
    "coral.procedural/Semente": "coral.procedural",
    "coral.procedural/GeradorProcedural": "coral.procedural",
    "coral.procedural/ReceitaProcedural": "coral.procedural",
    "coral.mundo/BlocoMapa": "coral.mundo",
    "coral.rpg/Personagem": "coral.rpg",
    "coral.rpg/CombateTurnos": "coral.rpg",
    "coral.rpg/EstadoRPG": "coral.rpg",
```

```
    "coral.jogos/Jogo": "coral.jogos",
    "coral.jogos/Camera": "coral.jogos",
    "coral.jogos/Animacao": "coral.jogos",
    "coral.jogos/ClipeAnimacao": "coral.jogos",
    "coral.jogos/CamadaApresentacao": "coral.jogos",
    "coral.jogos/ComposicaoTemporal": "coral.jogos",
    "coral.jogos/ComandoComposicao": "coral.jogos",
    "coral.laboratorio/Medicao": "coral.laboratorio",
    "coral.laboratorio/ResultadoExperimento": "coral.laboratorio",
}


def registrar_adaptador(tipo_python: type, identificador: str,
                        para_dados: Callable[[Any], dict[str, Any]],
                        de_dados: Callable[[dict[str, Any]], Any]) -> None:
    identificador = str(identificador)
    if not identificador or "/" not in identificador:
        raise ValueError("Identificador de persistência inválido.")
    existente = _POR_ID.get(identificador)
    if existente is not None and existente.tipo_python is not tipo_python:
        raise ValueError("Identificador de persistência já registrado: " +
          identificador)
    adaptador = _Adaptador(tipo_python, identificador, para_dados, de_dados)
    _POR_TIPO[tipo_python] = adaptador
    _POR_ID[identificador] = adaptador


def registrar_extensao_persistente(
    tipo_python: type,
    identificador: str,
    para_dados: Callable[[Any], Any],
    aplicar_dados: Callable[[Any, Any], None],
) -> None:
    """Registra uma extensão de domínio sem acoplar o tipo hospedeiro ao módulo dono."""
    identificador = str(identificador)
    if not identificador or "/" not in identificador:
        raise ValueError("Identificador de extensão persistente inválido.")
    existente = _EXTENSOES_POR_ID.get(identificador)
    if existente is not None:
        if existente.tipo_python is not tipo_python:
            raise ValueError("Identificador de extensão persistente já registrado: " +
              identificador)
        lista = _EXTENSOES_POR_TIPO.get(tipo_python, [])
        _EXTENSOES_POR_TIPO[tipo_python] = [e for e in lista if e.identificador !=
          identificador]
    extensao = _ExtensaoPersistente(tipo_python, identificador, para_dados,
      aplicar_dados)
    _EXTENSOES_POR_ID[identificador] = extensao
    _EXTENSOES_POR_TIPO.setdefault(tipo_python, []).append(extensao)


def extensoes_para_dados(objeto: Any) -> dict[str, Any]:
    saida: dict[str, Any] = {}
    for tipo, extensoes in tuple(_EXTENSOES_POR_TIPO.items()):
        if not isinstance(objeto, tipo):
            continue
        for extensao in tuple(extensoes):
            payload = extensao.para_dados(objeto)
            if payload is not None:
                saida[extensao.identificador] = payload
    return saida
```

```
def _garantir_extensao(identificador: str) -> None:
    if identificador in _EXTENSOES_POR_ID:
        return
    if identificador.startswith("coral.") and "/" in identificador:
        importlib.import_module(identificador.split("/", 1)[0])


def aplicar_extensoes_persistentes(objeto: Any, extensoes: Any) -> None:
    if extensoes is None:
        return
    if not isinstance(extensoes, dict):
        raise ErroPersistencia("Extensões persistentes precisam ser um objeto.")
    for identificador, payload in extensoes.items():
        if not isinstance(identificador, str):
            raise ErroPersistencia("Identificador de extensão persistente inválido.")
        _garantir_extensao(identificador)
        extensao = _EXTENSOES_POR_ID.get(identificador)
        if extensao is None:
            raise ErroPersistencia("Extensão persistente desconhecida: " +
              identificador)
        if not isinstance(objeto, extensao.tipo_python):
            raise ErroPersistencia(
                f"Extensão {identificador} não é compatível com
                  {type(objeto).__name__}."
            )
        extensao.aplicar_dados(objeto, payload)


def _erro(caminho: str, mensagem: str) -> ErroPersistencia:
    return ErroPersistencia(f"{mensagem} (campo: {caminho})")


def _adaptador_tipo(objeto: Any) -> _Adaptador | None:
    adaptador = _POR_TIPO.get(type(objeto))
    if adaptador is not None:
        return adaptador
    # Subclasses só usam adaptador de classe base quando não existe um mais
    # específico. Isso mantém tipos de domínio explícitos como Personagem.
    for classe, candidato in _POR_TIPO.items():
        if isinstance(objeto, classe):
            return candidato
    return None


def _garantir_registro(identificador: str) -> None:
    if identificador in _POR_ID:
        return
    modulo = _MODULOS_CONHECIDOS.get(identificador)
    if modulo:
        importlib.import_module(modulo)


def _envelope_base(identificador: str, dados: dict[str, Any]) -> dict[str, Any]:
    return {
        "contrato": CONTRATO,
        "versao_esquema": VERSAO_ESQUEMA,
        "tipo": identificador,
        "dados": dados,
    }
```

```
def _chave_canonica_codificada(valor: Any) -> str:
    return json.dumps(
        valor, ensure_ascii=False, sort_keys=True, separators=(",", ":"),
          allow_nan=False
    )


def _tipo_base_direto(valor: Any) -> bool:
    return isinstance(valor, (tuple, set, frozenset, bytes, bytearray, datetime, date,
      timedelta, PurePath))


def _codificar(valor: Any, caminho: str) -> Any:
    if valor is None or isinstance(valor, (bool, str, int)):
        return valor
    if isinstance(valor, float):
        if not math.isfinite(valor):
            raise _erro(caminho, "Float não finito não é portátil")
        return valor
    if isinstance(valor, datetime):
        return _envelope_base(TIPO_DATA_HORA, {
            "iso": valor.isoformat(),
            "fold": int(valor.fold),
        })
    if isinstance(valor, date):
        return _envelope_base(TIPO_DATA, {"iso": valor.isoformat()})
    if isinstance(valor, timedelta):
        return _envelope_base(TIPO_DURACAO, {
            "dias": valor.days,
            "segundos": valor.seconds,
            "microssegundos": valor.microseconds,
        })
    if isinstance(valor, (bytes, bytearray)):
        return _envelope_base(TIPO_BYTES, {
            "base64": base64.b64encode(bytes(valor)).decode("ascii"),
            "mutavel": isinstance(valor, bytearray),
        })
    if isinstance(valor, PurePath):
        return _envelope_base(TIPO_CAMINHO, {
            "texto": str(valor),
            "estilo": estilo_caminho(valor),
            "concreto": isinstance(valor, Path),
        })
    if isinstance(valor, tuple):
        return _envelope_base(TIPO_TUPLA, {
            "itens": [_codificar(item, f"{caminho}[{indice}]") for indice, item in
              enumerate(valor)]
        })
    if isinstance(valor, (set, frozenset)):
        itens = [_codificar(item, f"{caminho}{{item}}") for item in valor]
        itens.sort(key=_chave_canonica_codificada)
        return _envelope_base(TIPO_CONJUNTO, {
            "itens": itens,
            "imutavel": isinstance(valor, frozenset),
        })
    adaptador = _adaptador_tipo(valor)
    if adaptador is not None:
        bruto = adaptador.para_dados(valor)
        if not isinstance(bruto, dict):
```

```
            raise _erro(caminho, f"Adaptador {adaptador.identificador} não retornou um
              dicionário")
        return {
            "contrato": CONTRATO,
            "versao_esquema": VERSAO_ESQUEMA,
            "tipo": adaptador.identificador,
            "dados": _codificar(bruto, caminho + ".dados"),
        }
    if isinstance(valor, list):
        return [_codificar(item, f"{caminho}[{indice}]") for indice, item in
          enumerate(valor)]
    if isinstance(valor, dict):
        saida = {}
        for chave, item in valor.items():
            if not isinstance(chave, str):
                raise _erro(caminho, "Dicionários persistentes exigem chaves de texto")
            saida[chave] = _codificar(item, f"{caminho}.{chave}")
        return saida
    raise _erro(caminho, f"Objeto não portátil: {type(valor).__name__}")


def _validar_envelope(documento: dict[str, Any], caminho: str) -> tuple[str, dict[str,
  Any]]:
    if documento.get("contrato") != CONTRATO:
        raise _erro(caminho, "Contrato de persistência incompatível")
    versao = documento.get("versao_esquema")
    if versao not in VERSOES_LEITURA:
        raise _erro(caminho, f"Versão de esquema incompatível: {versao!r}")
    identificador = documento.get("tipo")
    if not isinstance(identificador, str) or not identificador:
        raise _erro(caminho + ".tipo", "Tipo persistente ausente")
    dados = documento.get("dados")
    if not isinstance(dados, dict):
        raise _erro(caminho + ".dados", "Payload persistente deve ser um objeto")
    return identificador, dados


def _decodificar_base(identificador: str, dados: dict[str, Any], caminho: str) -> Any:
    if identificador == TIPO_VALOR_BASE:
        if set(dados) != {"valor"}:
            raise _erro(caminho + ".dados", "Envelope de valor base inválido")
        return _decodificar(dados["valor"], caminho + ".dados.valor")
    if identificador == TIPO_TUPLA:
        itens = dados.get("itens")
        if not isinstance(itens, list):
            raise _erro(caminho + ".dados.itens", "Tupla persistente exige lista de
              itens")
        return tuple(_decodificar(v, f"{caminho}.dados.itens[{i}]") for i, v in
          enumerate(itens))
    if identificador == TIPO_CONJUNTO:
        itens = dados.get("itens")
        if not isinstance(itens, list):
            raise _erro(caminho + ".dados.itens", "Conjunto persistente exige lista de
              itens")
        valores = [_decodificar(v, f"{caminho}.dados.itens[{i}]") for i, v in
          enumerate(itens)]
        try:
            return frozenset(valores) if dados.get("imutavel") else set(valores)
        except TypeError as erro:
            raise _erro(caminho, "Conjunto persistente contém valor não hashable") from
              erro
```

```
    if identificador == TIPO_BYTES:
        bruto = dados.get("base64")
        if not isinstance(bruto, str):
            raise _erro(caminho + ".dados.base64", "Bytes persistentes exigem base64
              textual")
        try:
            valor = base64.b64decode(bruto.encode("ascii"), validate=True)
        except Exception as erro:
            raise _erro(caminho + ".dados.base64", "Base64 inválido") from erro
        return bytearray(valor) if dados.get("mutavel") else valor
    if identificador == TIPO_DATA:
        bruto = dados.get("iso")
        if not isinstance(bruto, str):
            raise _erro(caminho + ".dados.iso", "Data persistente exige ISO 8601")
        try:
            return date.fromisoformat(bruto)
        except ValueError as erro:
            raise _erro(caminho + ".dados.iso", "Data ISO 8601 inválida") from erro
    if identificador == TIPO_DATA_HORA:
        bruto = dados.get("iso")
        if not isinstance(bruto, str):
            raise _erro(caminho + ".dados.iso", "Data/hora persistente exige ISO 8601")
        try:
            valor = datetime.fromisoformat(bruto)
            return valor.replace(fold=int(dados.get("fold", 0)))
        except (TypeError, ValueError) as erro:
            raise _erro(caminho + ".dados.iso", "Data/hora ISO 8601 inválida") from erro
    if identificador == TIPO_DURACAO:
        try:
            return timedelta(
                days=int(dados["dias"]),
                seconds=int(dados["segundos"]),
                microseconds=int(dados["microssegundos"]),
            )
        except (KeyError, TypeError, ValueError, OverflowError) as erro:
            raise _erro(caminho + ".dados", "Duração persistente inválida") from erro
    if identificador == TIPO_CAMINHO:
        texto = dados.get("texto")
        estilo = dados.get("estilo")
        if not isinstance(texto, str) or not isinstance(estilo, str):
            raise _erro(caminho + ".dados", "Caminho persistente inválido")
        try:
            return caminho_portatil(texto, estilo, concreto=bool(dados.get("concreto",
              True)))
        except ValueError as erro:
            raise _erro(caminho + ".dados", str(erro)) from erro
    raise _erro(caminho + ".tipo", "Tipo base persistente desconhecido: " +
      identificador)


def _decodificar(valor: Any, caminho: str) -> Any:
    if valor is None or isinstance(valor, (bool, str, int)):
        return valor
    if isinstance(valor, float):
        if not math.isfinite(valor):
            raise _erro(caminho, "Float não finito não é portátil")
        return valor
    if isinstance(valor, list):
        return [_decodificar(item, f"{caminho}[{indice}]") for indice, item in
          enumerate(valor)]
    if isinstance(valor, dict):
```

```
        if valor.get("contrato") == CONTRATO:
            identificador, dados = _validar_envelope(valor, caminho)
            if identificador in _TIPOS_BASE:
                return _decodificar_base(identificador, dados, caminho)
            _garantir_registro(identificador)
            adaptador = _POR_ID.get(identificador)
            if adaptador is None:
                raise _erro(caminho + ".tipo", "Tipo persistente desconhecido: " +
                  identificador)
            payload = _decodificar(dados, caminho + ".dados")
            try:
                return adaptador.de_dados(payload)
            except ErroPersistencia:
                raise
            except Exception as exc:
                raise _erro(caminho, f"Falha ao reconstruir {identificador}:
                  {type(exc).__name__}: {exc}") from exc
        saida = {}
        for chave, item in valor.items():
            if not isinstance(chave, str):
                raise _erro(caminho, "Dicionários persistentes exigem chaves de texto")
            saida[chave] = _decodificar(item, f"{caminho}.{chave}")
        return saida
    raise _erro(caminho, f"Valor persistente inválido: {type(valor).__name__}")


def valor_portatil(valor: Any) -> Any:
    """Normaliza um valor isolado segundo o contrato portátil da Coral.

    Diferente de :func:`para_dados`, não exige adaptador no objeto raiz. É
    usada por metadados e outras estruturas auxiliares que precisam validar
    portabilidade sem criar um novo documento persistente.
    """
    return _codificar(valor, "$")


def para_dados(objeto: Any) -> dict[str, Any]:
    documento = _codificar(objeto, "$")
    if _adaptador_tipo(objeto) is not None or _tipo_base_direto(objeto):
        if isinstance(documento, dict) and documento.get("contrato") == CONTRATO:
            return documento
    return _envelope_base(TIPO_VALOR_BASE, {"valor": documento})


def de_dados(documento: dict[str, Any]) -> Any:
    if not isinstance(documento, dict):
        raise ErroPersistencia("Documento persistente deve ser um objeto JSON.")
    return _decodificar(documento, "$")


def texto_canonico(valor: Any) -> str:
    """Serialização JSON canônica para hash, cache, replay e equivalência."""

    documento = _codificar(valor, "$")
    return json.dumps(
        documento, ensure_ascii=False, sort_keys=True, separators=(",", ":"),
          allow_nan=False
    )


def serializar_canonico(valor: Any) -> bytes:
```

```
    """Versão UTF 8 explícita da serialização canônica."""

    return texto_canonico(valor).encode("utf-8")


def salvar(objeto: Any, caminho: str | Path) -> Path:
    destino = normalizar_caminho(caminho, expandir_usuario=True)
    destino.parent.mkdir(parents=True, exist_ok=True)
    documento = para_dados(objeto)
    try:
        texto = json.dumps(documento, ensure_ascii=False, indent=2, allow_nan=False)
    except (TypeError, ValueError) as exc:
        raise ErroPersistencia("Não foi possível converter o documento para JSON: " +
          str(exc)) from exc
    destino.write_text(texto + "\n", encoding="utf-8")
    return destino


def carregar(caminho: str | Path) -> Any:
    origem = normalizar_caminho(caminho, expandir_usuario=True)
    try:
        documento = json.loads(origem.read_text(encoding="utf-8"))
    except (OSError, UnicodeError, json.JSONDecodeError) as exc:
        raise ErroPersistencia(f"Arquivo persistente inválido: {type(exc).__name__}:
          {exc}") from exc
    return de_dados(documento)


def carregar_como(caminho: str | Path, tipo_esperado: type, nome_tipo: str | None =
  None) -> Any:
    objeto = carregar(caminho)
    if not isinstance(objeto, tipo_esperado):
        esperado = nome_tipo or getattr(tipo_esperado, "__name__", str(tipo_esperado))
        raise ErroPersistencia(
            f"Tipo persistido incompatível: esperado {esperado}, recebido
              {type(objeto).__name__}."
        )
    return objeto


__all__ = [
    "CONTRATO", "VERSAO_ESQUEMA", "VERSOES_LEITURA", "VERSAO_ESCRITA",
    "politica_persistencia", "ErroPersistencia", "registrar_adaptador",
    "registrar_extensao_persistente", "extensoes_para_dados",
      "aplicar_extensoes_persistentes",
    "valor_portatil", "para_dados", "de_dados", "texto_canonico", "serializar_canonico",
    "salvar", "carregar", "carregar_como",
]
```

Biblioteca `coral.procedural`

`coral.procedural.CONTRATO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.procedural/1'`

`coral.procedural.CONTRATO_RECEITA`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'coral.procedural.receita/1'`

`coral.procedural.CampoProcedural`

Campo numérico consultável por coordenadas e combinável por composição.

A quantidade de coordenadas aceita pode ser declarada. O intervalo informado é descritivo e usado para diagnóstico; cada valor ainda é validado como finito no momento da consulta.

Assinatura

Assinatura para consulta

```
CampoProcedural(funcao: 'Callable[..., float]', *, nome: 'str' = 'campo', metadados:
  'Mapping[str, Any] | None' = None, dimensoes_min: 'int' = 1, dimensoes_max: 'int | None'
   = None, intervalo: 'tuple[float, float] | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
funcao	Callable[..., float]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str	KEYWORD_ONLY	não	'campo'
metadados	Mapping[str, Any] | None	KEYWORD_ONLY	não	None
dimensoes_min	int	KEYWORD_ONLY	não	1
dimensoes_max	int | None	KEYWORD_ONLY	não	None
intervalo	tuple[float, float] | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
limitar(self, minimo: 'float', maximo: 'float') -> "'CampoProcedural'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
mascara(self, *, minimo: 'float | None' = None, maximo: 'float | None' = None, dentro:
  'float' = 1.0, fora: 'float' = 0.0) -> "'CampoProcedural'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
multiplicar(self, outro: "'CampoProcedural | float'") -> "'CampoProcedural'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
normalizar(self, origem_min: 'float', origem_max: 'float', destino_min: 'float' = 0.0,
  destino_max: 'float' = 1.0) -> "'CampoProcedural'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
somar(self, outro: "'CampoProcedural | float'") -> "'CampoProcedural'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
transformar(self, funcao: 'Callable[[float], float]', *, nome: 'str | None' = None,
  intervalo: 'tuple[float, float] | None' = None, operacao: 'str' = 'transformar') ->
  "'CampoProcedural'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
valor(self, *coordenadas: 'float') -> 'float'
```

Método público sem descrição própria no runtime.

`coral.procedural.GeradorProcedural`

Gerador sem estado para consultas determinísticas por chave ou contexto.

Assinatura

Assinatura para consulta

```
GeradorProcedural(semente_raiz: 'Any' = 0, *, namespace: 'str' = 'gerador') -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
semente_raiz	Any	POSITIONAL_OR_KEYWORD	não	0
namespace	str	KEYWORD_ONLY	não	'gerador'

Retorno declarado

None

Métodos públicos

Referência para leitura

```
amostra(self, valores: 'Iterable[Any]', quantidade: 'int', *chaves: 'Any') ->
  'list[Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
decimal(self, *chaves: 'Any', minimo: 'float' = 0.0, maximo: 'float' = 1.0) -> 'float'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
derivar(self, rotulo: 'Any') -> "'GeradorProcedural'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
escolha_ponderada(self, valores: 'Iterable[Any]', pesos: 'Iterable[float]', *chaves:
  'Any') -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
escolher(self, valores: 'Iterable[Any]', *chaves: 'Any') -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
fonte(self, *chaves: 'Any') -> 'FonteAleatoria'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
inteiro(self, minimo: 'int', maximo: 'int', *chaves: 'Any') -> 'int'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
normal(self, *chaves: 'Any', media: 'float' = 0.0, desvio: 'float' = 1.0) -> 'float'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
triangular(self, *chaves: 'Any', minimo: 'float' = 0.0, maximo: 'float' = 1.0, moda:
  'float | None' = None) -> 'float'
```

Método público sem descrição própria no runtime.

`coral.procedural.ReceitaProcedural`

Descrição portátil e reconstruível de uma operação procedural suportada.

A receita não guarda callbacks nem objetos de execução. Apenas algoritmo, versão, semente e parâmetros portáteis entram na representação.

Assinatura

Assinatura para consulta

```
ReceitaProcedural(algoritmo: 'str', *, semente_raiz: 'Any' = 0, parametros:
  'Mapping[str, Any] | None' = None, versao_algoritmo: 'int' = 1) -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
algoritmo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
semente_raiz	Any	KEYWORD_ONLY	não	0
parametros	Mapping[str, Any] | None	KEYWORD_ONLY	não	None
versao_algoritmo	int	KEYWORD_ONLY	não	1

Retorno declarado

None

Métodos públicos

Referência para leitura

```
de_dados(dados: 'Mapping[str, Any]') -> "'ReceitaProcedural'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
de_json(texto: 'str') -> "'ReceitaProcedural'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_dados(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
para_json(self, *, identar: 'int | None' = None) -> 'str'
```

Método público sem descrição própria no runtime.

`coral.procedural.Semente`

Semente estável capaz de derivar sub sementes independentes.

Assinatura

Assinatura para consulta

```
Semente(valor: 'Any' = 0) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	Any	POSITIONAL_OR_KEYWORD	não	0

Retorno declarado

None

Atributo `valor`: `Any`.

Métodos públicos

Referência para leitura

```
derivar(self, *rotulos: 'Any') -> "'Semente'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
fonte(self, *rotulos: 'Any') -> 'FonteAleatoria'
```

Método público sem descrição própria no runtime.

`coral.procedural.SequenciaProcedural`

Sequência indexada cuja posição não depende das consultas anteriores.

Assinatura

Assinatura para consulta

```
SequenciaProcedural(semente_raiz: 'Any' = 0, gerador: 'Callable[[GeradorProcedural,
  int], Any] | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
semente_raiz	Any	POSITIONAL_OR_KEYWORD	não	0
gerador	Callable[[GeradorProcedural, int], Any] | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
obter(self, indice: 'int') -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
trecho(self, inicio: 'int', quantidade: 'int', *, limite_itens: 'int' = 1000000) ->
  'tuple[Any, ...]'
```

Método público sem descrição própria no runtime.

`coral.procedural.amostrar_campo`

Materializa amostras portáteis de um campo sem depender de Numérico ou Laboratório.

Cada registro contém `indice`, as coordenadas nomeadas e o valor calculado. O limite é aplicado durante a leitura para também proteger iteráveis sem tamanho.

Assinatura

Assinatura para consulta

```
amostrar_campo(campo: 'CampoProcedural', coordenadas: 'Iterable[Sequence[float]]', *,
  nomes_eixos: 'Sequence[str] | None' = None, nome_valor: 'str' = 'valor',
  limite_amostras: 'int' = 100000) -> 'tuple[dict[str, Any], ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
campo	CampoProcedural	POSITIONAL_OR_KEYWORD	sim	sem padrão
coordenadas	Iterable[Sequence[float]]	POSITIONAL_OR_KEYWORD	sim	sem padrão
nomes_eixos	Sequence[str] | None	KEYWORD_ONLY	não	None
nome_valor	str	KEYWORD_ONLY	não	'valor'
limite_amostras	int	KEYWORD_ONLY	não	100000

Retorno declarado

tuple[dict[str, Any], ...]

`coral.procedural.anotar_pontos`

Anexa valores de campos a pontos 2D em registros gerais e portáteis.

Assinatura

Assinatura para consulta

```
anotar_pontos(pontos: 'Iterable[Sequence[float]]', campos: 'Mapping[str,
  CampoProcedural]', *, limite_pontos: 'int' = 100000) -> 'tuple[dict[str, Any], ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
pontos	Iterable[Sequence[float]]	POSITIONAL_OR_KEYWORD	sim	sem padrão
campos	Mapping[str, CampoProcedural]	POSITIONAL_OR_KEYWORD	sim	sem padrão
limite_pontos	int	KEYWORD_ONLY	não	100000

Retorno declarado

tuple[dict[str, Any], ...]

`coral.procedural.campo_constante`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
campo_constante(valor: 'float', *, nome: 'str' = 'constante') -> 'CampoProcedural'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str	KEYWORD_ONLY	não	'constante'

Retorno declarado

CampoProcedural

`coral.procedural.campo_ruido`

Cria ruído de valor suave para 1D, 2D ou 3D sem dependências externas.

Assinatura

Assinatura para consulta

```
campo_ruido(semente_raiz: 'Any' = 0, *, escala: 'float' = 1.0, oitavas: 'int' = 1,
  persistencia: 'float' = 0.5, lacunaridade: 'float' = 2.0, nome: 'str' = 'ruido') ->
  'CampoProcedural'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
semente_raiz	Any	POSITIONAL_OR_KEYWORD	não	0
escala	float	KEYWORD_ONLY	não	1.0
oitavas	int	KEYWORD_ONLY	não	1
persistencia	float	KEYWORD_ONLY	não	0.5
lacunaridade	float	KEYWORD_ONLY	não	2.0
nome	str	KEYWORD_ONLY	não	'ruido'

Retorno declarado

CampoProcedural

`coral.procedural.derivar_semente`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
derivar_semente(valor: 'Any', *rotulos: 'Any') -> 'Semente'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
rotulos	Any	VAR_POSITIONAL	não	sem padrão

Retorno declarado

Semente

`coral.procedural.diagnosticar_custo_receita`

Estima o trabalho máximo conhecido antes de reconstruir a receita.

Assinatura

Assinatura para consulta

```
diagnosticar_custo_receita(receita: 'ReceitaProcedural | Mapping[str, Any]') ->
  'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
receita	ReceitaProcedural | Mapping[str, Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

dict[str, Any]

`coral.procedural.distribuir_pontos`

Distribui pontos 2D determinísticos sob limites finitos de trabalho.

`` `limite_tentativas_total` `` é um orçamento preventivo: configurações cujo pior caso declarado exceda o limite falham antes de materializar pontos. O chamador pode elevar explicitamente o limite quando aceitar o custo.

Assinatura

Assinatura para consulta

```
distribuir_pontos(largura: 'float', altura: 'float', quantidade: 'int', *, semente_raiz:
   'Any' = 0, margem: 'float' = 0.0, distancia_minima: 'float' = 0.0, restricao:
  'Callable[[float, float], bool] | None' = None, max_tentativas_por_ponto: 'int' = 200,
  limite_tentativas_total: 'int' = 1000000) -> 'tuple[tuple[float, float], ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
largura	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
altura	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
quantidade	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
semente_raiz	Any	KEYWORD_ONLY	não	0
margem	float	KEYWORD_ONLY	não	0.0
distancia_minima	float	KEYWORD_ONLY	não	0.0
restricao	Callable[[float, float], bool] | None	KEYWORD_ONLY	não	None
max_tentativas_por_ponto	int	KEYWORD_ONLY	não	200
limite_tentativas_total	int	KEYWORD_ONLY	não	1000000

Retorno declarado

tuple[tuple[float, float], ...]

`coral.procedural.matriz_campo`

Amostra um campo 2D como matriz, pronta para mapas, jogos e mapas de calor.

Assinatura

Assinatura para consulta

```
matriz_campo(campo: 'CampoProcedural', colunas: 'int', linhas: 'int', *, origem:
  'Sequence[float]' = (0.0, 0.0), passo: 'Sequence[float]' = (1.0, 1.0), limite_celulas:
  'int' = 1000000) -> 'tuple[tuple[float, ...], ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
campo	CampoProcedural	POSITIONAL_OR_KEYWORD	sim	sem padrão
colunas	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
linhas	int	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
origem	Sequence[float]	KEYWORD_ONLY	não	(0.0, 0.0)
passo	Sequence[float]	KEYWORD_ONLY	não	(1.0, 1.0)
limite_celulas	int	KEYWORD_ONLY	não	1000000

Retorno declarado

tuple[tuple[float, ...], ...]

`coral.procedural.proveniencia_procedural`

Descreve origem procedural sem importar domínios consumidores.

Assinatura

Assinatura para consulta

```
proveniencia_procedural(objeto: 'Any') -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
objeto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

dict[str, Any]

`coral.procedural.receita_procedural`

Cria uma receita portátil para um dos algoritmos declarativos suportados.

Assinatura

Assinatura para consulta

```
receita_procedural(algoritmo: 'str', *, semente_raiz: 'Any' = 0, **parametros: 'Any') ->
   'ReceitaProcedural'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
algoritmo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
semente_raiz	Any	KEYWORD_ONLY	não	0
parametros	Any	VAR_KEYWORD	não	sem padrão

Retorno declarado

ReceitaProcedural

`coral.procedural.reconstruir_receita`

Reconstrói uma receita após validar versão e orçamento explícito.

Assinatura

Assinatura para consulta

```
reconstruir_receita(receita: 'ReceitaProcedural | Mapping[str, Any]', *, limite_custo:
  'int' = 1000000) -> 'Any'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
receita	ReceitaProcedural | Mapping[str, Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
limite_custo	int	KEYWORD_ONLY	não	1000000

Retorno declarado

Any

`coral.procedural.semente`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
semente(valor: 'Any' = 0) -> 'Semente'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	Any	POSITIONAL_OR_KEYWORD	não	0

Retorno declarado

Semente

Consultar a implementação canônica de coral.procedural

Implementação canônica de `coral.procedural`

Código Python

```
"""Fundação determinística de geração procedural da Coral.

O módulo é deliberadamente independente de Jogos, RPG, Laboratório, Gráficos e
interfaces gráficas. O contrato ``coral.procedural/1`` preserva sementes estáveis,
subsementes independentes e consultas determinísticas por chave, índice ou
coordenada. A Coral 1.6.15 refina validação e previsibilidade sem redesenhar a
superfície pública existente.
"""
from __future__ import annotations

from dataclasses import dataclass
from copy import deepcopy
import hashlib
import json
import math
from typing import Any, Callable, Iterable, Mapping, Sequence

from .stdlib.aleatorio import FonteAleatoria

CONTRATO = "coral.procedural/1"
CONTRATO_RECEITA = "coral.procedural.receita/1"
VERSAO_RECEITA = 1
LIMITE_CUSTO_PADRAO = 1_000_000
LIMITE_TRECHO_PADRAO = 1_000_000



_SEMENTE_TIPOS = (type(None), bool, int, float, str, bytes)


def _normalizar_semente(valor: Any) -> Any:
    if not isinstance(valor, _SEMENTE_TIPOS):
        raise TypeError("semente procedural deve ser nula, booleana, inteira, decimal,
          texto ou bytes")
    if isinstance(valor, float) and not math.isfinite(valor):
        raise ValueError("semente decimal precisa ser finita")
    return valor


def _material(valor: Any) -> bytes:
    valor = _normalizar_semente(valor)
    if valor is None:
        payload = {"tipo": "nulo", "valor": None}
    elif isinstance(valor, bool):
        payload = {"tipo": "booleano", "valor": valor}
    elif isinstance(valor, int):
        payload = {"tipo": "inteiro", "valor": str(valor)}
    elif isinstance(valor, float):
        payload = {"tipo": "decimal", "valor": valor.hex()}
    elif isinstance(valor, str):
        payload = {"tipo": "texto", "valor": valor}
    else:
        payload = {"tipo": "bytes", "valor": valor.hex()}
    return json.dumps(payload, ensure_ascii=False, sort_keys=True, separators=(",",
      ":")).encode("utf-8")


def _rotulo_bytes(rotulo: Any) -> bytes:
    if isinstance(rotulo, bytes):
        return b"b:" + rotulo
```

```
    if isinstance(rotulo, (str, int, bool, float)) or rotulo is None:
        return b"v:" + _material(rotulo)
    if isinstance(rotulo, tuple):
        return b"t:[" + b"|".join(_rotulo_bytes(item) for item in rotulo) + b"]"
    if isinstance(rotulo, list):
        return b"l:[" + b"|".join(_rotulo_bytes(item) for item in rotulo) + b"]"
    raise TypeError("rótulo procedural precisa ser portátil e determinístico")


@dataclass(frozen=True)
class Semente:
    """Semente estável capaz de derivar sub sementes independentes."""

    valor: Any = 0

    def __post_init__(self) -> None:
        object.__setattr__(self, "valor", _normalizar_semente(self.valor))

    @property
    def inteiro(self) -> int:
        digest = hashlib.sha256(b"coral.procedural/1\0" +
          _material(self.valor)).digest()
        return int.from_bytes(digest[:16], "big", signed=False)

    @property
    def hexadecimal(self) -> str:
        return f"{self.inteiro:032x}"

    def derivar(self, *rotulos: Any) -> "Semente":
        estado = hashlib.sha256(b"coral.procedural/derivar/1\0" + _material(self.valor))
        for rotulo in rotulos:
            bruto = _rotulo_bytes(rotulo)
            estado.update(len(bruto).to_bytes(8, "big"))
            estado.update(bruto)
        return Semente(int.from_bytes(estado.digest()[:16], "big", signed=False))

    def fonte(self, *rotulos: Any) -> FonteAleatoria:
        alvo = self.derivar(*rotulos) if rotulos else self
        return FonteAleatoria(alvo.inteiro)


def semente(valor: Any = 0) -> Semente:
    return valor if isinstance(valor, Semente) else Semente(valor)


def derivar_semente(valor: Any, *rotulos: Any) -> Semente:
    return semente(valor).derivar(*rotulos)


def _numero_finito(valor: Any, nome: str) -> float:
    if isinstance(valor, bool):
        raise ValueError(f"{nome} precisa ser um número finito")
    try:
        numero = float(valor)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ValueError(f"{nome} precisa ser um número finito") from exc
    if not math.isfinite(numero):
        raise ValueError(f"{nome} precisa ser um número finito")
    return numero
```

```
def _inteiro_exato(valor: Any, nome: str, *, minimo: int | None = None) -> int:
    if isinstance(valor, bool):
        raise ValueError(f"{nome} precisa ser inteiro")
    if isinstance(valor, int):
        numero = valor
    elif isinstance(valor, float) and math.isfinite(valor) and valor.is_integer():
        numero = int(valor)
    else:
        raise ValueError(f"{nome} precisa ser inteiro")
    if minimo is not None and numero < minimo:
        if minimo == 0:
            raise ValueError(f"{nome} precisa ser inteiro não negativo")
        if minimo == 1:
            raise ValueError(f"{nome} precisa ser inteiro positivo")
        raise ValueError(f"{nome} precisa ser inteiro maior ou igual a {minimo}")
    return numero


def _pesos_validos(valores: Iterable[Any], pesos: Iterable[float]) -> tuple[tuple[Any,
  ...], tuple[float, ...]]:
    valores_t = tuple(valores)
    pesos_brutos = tuple(pesos)
    if not valores_t or len(valores_t) != len(pesos_brutos):
        raise ValueError("valores e pesos precisam ter o mesmo tamanho não vazio")
    pesos_t = tuple(_numero_finito(peso, "peso") for peso in pesos_brutos)
    if any(peso < 0 for peso in pesos_t):
        raise ValueError("pesos precisam ser não negativos")
    if math.fsum(pesos_t) <= 0:
        raise ValueError("pesos precisam ter soma positiva")
    return valores_t, pesos_t


class GeradorProcedural:
    """Gerador sem estado para consultas determinísticas por chave ou contexto."""

    def __init__(self, semente_raiz: Any = 0, *, namespace: str = "gerador") -> None:
        self.semente = semente(semente_raiz)
        self.namespace = str(namespace)

    def derivar(self, rotulo: Any) -> "GeradorProcedural":
        return GeradorProcedural(self.semente.derivar(self.namespace, rotulo),
          namespace=str(rotulo))

    def fonte(self, *chaves: Any) -> FonteAleatoria:
        return self.semente.fonte(self.namespace, *chaves)

    def decimal(self, *chaves: Any, minimo: float = 0.0, maximo: float = 1.0) -> float:
        minimo = _numero_finito(minimo, "mínimo")
        maximo = _numero_finito(maximo, "máximo")
        if maximo < minimo:
            raise ValueError("máximo não pode ser menor que mínimo")
        return self.fonte(*chaves).decimal(minimo, maximo)

    def inteiro(self, minimo: int, maximo: int, *chaves: Any) -> int:
        if isinstance(minimo, bool) or isinstance(maximo, bool):
            raise ValueError("limites inteiros não aceitam booleanos")
        minimo = int(minimo)
        maximo = int(maximo)
        if maximo < minimo:
            raise ValueError("máximo não pode ser menor que mínimo")
        return self.fonte(*chaves).inteiro(minimo, maximo)
```

```
    def escolher(self, valores: Iterable[Any], *chaves: Any) -> Any:
        valores = tuple(valores)
        if not valores:
            raise ValueError("não é possível escolher em uma coleção vazia")
        return self.fonte(*chaves).escolher(valores)

    def escolha_ponderada(self, valores: Iterable[Any], pesos: Iterable[float], *chaves:
       Any) -> Any:
        valores_t, pesos_t = _pesos_validos(valores, pesos)
        return self.fonte(*chaves).escolha_ponderada(valores_t, pesos_t)

    def amostra(self, valores: Iterable[Any], quantidade: int, *chaves: Any) ->
      list[Any]:
        return self.fonte(*chaves).amostra(tuple(valores), quantidade)

    def normal(self, *chaves: Any, media: float = 0.0, desvio: float = 1.0) -> float:
        media = _numero_finito(media, "média")
        desvio = _numero_finito(desvio, "desvio")
        if desvio < 0:
            raise ValueError("desvio precisa ser maior ou igual a zero")
        if desvio == 0:
            return media
        fonte = self.fonte(*chaves)
        # Box Muller explícito para evitar estado interno e cache de gauss().
        u1 = max(fonte.decimal(), 2.0 ** -53)
        u2 = fonte.decimal()
        z = math.sqrt(-2.0 * math.log(u1)) * math.cos(2.0 * math.pi * u2)
        return media + z * desvio

    def triangular(
        self,
        *chaves: Any,
        minimo: float = 0.0,
        maximo: float = 1.0,
        moda: float | None = None,
    ) -> float:
        minimo = _numero_finito(minimo, "mínimo")
        maximo = _numero_finito(maximo, "máximo")
        if maximo < minimo:
            raise ValueError("máximo não pode ser menor que mínimo")
        if minimo == maximo:
            return minimo
        moda = (minimo + maximo) / 2.0 if moda is None else _numero_finito(moda, "moda")
        if not minimo <= moda <= maximo:
            raise ValueError("moda precisa estar entre mínimo e máximo")
        u = self.decimal(*chaves)
        corte = (moda - minimo) / (maximo - minimo)
        if u <= corte:
            return minimo + math.sqrt(u * (maximo - minimo) * (moda - minimo))
        return maximo - math.sqrt((1.0 - u) * (maximo - minimo) * (maximo - moda))


class SequenciaProcedural:
    """Sequência indexada cuja posição não depende das consultas anteriores."""

    def __init__(self, semente_raiz: Any = 0, gerador: Callable[[GeradorProcedural,
      int], Any] | None = None):
        self.gerador = GeradorProcedural(semente_raiz, namespace="sequencia")
        self._funcao = gerador
```

```
    def obter(self, indice: int) -> Any:
        if isinstance(indice, bool):
            raise ValueError("índice procedural precisa ser inteiro")
        indice = int(indice)
        if indice < 0:
            raise ValueError("índice procedural não pode ser negativo")
        if self._funcao is None:
            return self.gerador.decimal(indice)
        return self._funcao(self.gerador, indice)

    def trecho(
        self,
        inicio: int,
        quantidade: int,
        *,
        limite_itens: int = LIMITE_TRECHO_PADRAO,
    ) -> tuple[Any, ...]:
        inicio = _inteiro_exato(inicio, "início", minimo=0)
        quantidade = _inteiro_exato(quantidade, "quantidade", minimo=0)
        limite_itens = _inteiro_exato(limite_itens, "limite de itens", minimo=1)
        if quantidade > limite_itens:
            raise ValueError(
                "trecho procedural excede o limite de itens; aumente o limite
                  explicitamente se aceitar o custo"
            )
        return tuple(self.obter(inicio + i) for i in range(quantidade))

    def __getitem__(self, indice: int) -> Any:
        return self.obter(indice)


class CampoProcedural:
    """Campo numérico consultável por coordenadas e combinável por composição.

    A quantidade de coordenadas aceita pode ser declarada. O intervalo informado é
    descritivo e usado para diagnóstico; cada valor ainda é validado como finito no
    momento da consulta.
    """

    def __init__(
        self,
        funcao: Callable[..., float],
        *,
        nome: str = "campo",
        metadados: Mapping[str, Any] | None = None,
        dimensoes_min: int = 1,
        dimensoes_max: int | None = None,
        intervalo: tuple[float, float] | None = None,
    ):
        if not callable(funcao):
            raise TypeError("campo procedural exige uma função")
        dimensoes_min = _inteiro_exato(dimensoes_min, "dimensões mínimas", minimo=1)
        if dimensoes_max is not None:
            dimensoes_max = _inteiro_exato(dimensoes_max, "dimensões máximas", minimo=1)
            if dimensoes_max < dimensoes_min:
                raise ValueError("dimensões máximas não podem ser menores que dimensões
                  mínimas")
        intervalo_n = None
        if intervalo is not None:
            if not isinstance(intervalo, (tuple, list)) or len(intervalo) != 2:
                raise ValueError("intervalo declarado precisa ter mínimo e máximo")
```

```
            inicio = _numero_finito(intervalo[0], "mínimo do intervalo")
            fim = _numero_finito(intervalo[1], "máximo do intervalo")
            if fim < inicio:
                raise ValueError("máximo do intervalo não pode ser menor que mínimo")
            intervalo_n = (inicio, fim)

        self._funcao = funcao
        self.nome = str(nome)
        self.dimensoes_min = dimensoes_min
        self.dimensoes_max = dimensoes_max
        self.intervalo = intervalo_n
        self.metadados = dict(metadados or {})
        self.metadados.setdefault("dimensoes_min", dimensoes_min)
        self.metadados.setdefault("dimensoes_max", dimensoes_max)
        if intervalo_n is not None:
            self.metadados.setdefault("intervalo", list(intervalo_n))

    def valor(self, *coordenadas: float) -> float:
        quantidade = len(coordenadas)
        if quantidade < self.dimensoes_min:
            raise ValueError(f"campo procedural exige ao menos {self.dimensoes_min}
              coordenada(s)")
        if self.dimensoes_max is not None and quantidade > self.dimensoes_max:
            raise ValueError(f"campo procedural aceita no máximo {self.dimensoes_max}
              coordenada(s)")
        coords = tuple(_numero_finito(v, "coordenada") for v in coordenadas)
        return _numero_finito(self._funcao(*coords), "valor do campo")

    __call__ = valor

    def _dimensoes_com(self, outro: "CampoProcedural") -> tuple[int, int | None]:
        minimo = max(self.dimensoes_min, outro.dimensoes_min)
        if self.dimensoes_max is None:
            maximo = outro.dimensoes_max
        elif outro.dimensoes_max is None:
            maximo = self.dimensoes_max
        else:
            maximo = min(self.dimensoes_max, outro.dimensoes_max)
        if maximo is not None and maximo < minimo:
            raise ValueError("campos não aceitam uma quantidade de dimensões em comum")
        return minimo, maximo

    def transformar(
        self,
        funcao: Callable[[float], float],
        *,
        nome: str | None = None,
        intervalo: tuple[float, float] | None = None,
        operacao: str = "transformar",
    ) -> "CampoProcedural":
        if not callable(funcao):
            raise TypeError("transformação de campo exige uma função")
        metadados = dict(self.metadados)
        metadados["operacao"] = operacao
        return CampoProcedural(
            lambda *c: funcao(self.valor(*c)),
            nome=nome or self.nome,
            metadados=metadados,
            dimensoes_min=self.dimensoes_min,
            dimensoes_max=self.dimensoes_max,
            intervalo=intervalo,
```

```
        )

    def limitar(self, minimo: float, maximo: float) -> "CampoProcedural":
        minimo = _numero_finito(minimo, "mínimo")
        maximo = _numero_finito(maximo, "máximo")
        if maximo < minimo:
            raise ValueError("máximo não pode ser menor que mínimo")
        return self.transformar(
            lambda v: min(max(v, minimo), maximo),
            nome=f"{self.nome}.limitado",
            intervalo=(minimo, maximo),
            operacao="limitar",
        )

    def normalizar(
        self,
        origem_min: float,
        origem_max: float,
        destino_min: float = 0.0,
        destino_max: float = 1.0,
    ) -> "CampoProcedural":
        origem_min = _numero_finito(origem_min, "origem mínima")
        origem_max = _numero_finito(origem_max, "origem máxima")
        destino_min = _numero_finito(destino_min, "destino mínimo")
        destino_max = _numero_finito(destino_max, "destino máximo")
        if origem_max == origem_min:
            raise ValueError("faixa de origem não pode ter tamanho zero")
        escala = (destino_max - destino_min) / (origem_max - origem_min)
        intervalo = (min(destino_min, destino_max), max(destino_min, destino_max))
        return self.transformar(
            lambda v: destino_min + (v - origem_min) * escala,
            nome=f"{self.nome}.normalizado",
            intervalo=intervalo,
            operacao="normalizar",
        )

    def somar(self, outro: "CampoProcedural | float") -> "CampoProcedural":
        if isinstance(outro, CampoProcedural):
            dimensoes_min, dimensoes_max = self._dimensoes_com(outro)
            metadados = {"operacao": "somar", "entradas": [self.nome, outro.nome]}
            intervalo = None
            if self.intervalo is not None and outro.intervalo is not None:
                intervalo = (self.intervalo[0] + outro.intervalo[0], self.intervalo[1] +
                   outro.intervalo[1])
            return CampoProcedural(
                lambda *c: self.valor(*c) + outro.valor(*c),
                nome=f"{self.nome}+{outro.nome}",
                metadados=metadados,
                dimensoes_min=dimensoes_min,
                dimensoes_max=dimensoes_max,
                intervalo=intervalo,
            )
        numero = _numero_finito(outro, "parcela")
        intervalo = None if self.intervalo is None else (self.intervalo[0] + numero,
          self.intervalo[1] + numero)
        return CampoProcedural(
            lambda *c: self.valor(*c) + numero,
            nome=f"{self.nome}+constante",
            metadados={"operacao": "somar_constante", "entrada": self.nome},
            dimensoes_min=self.dimensoes_min,
            dimensoes_max=self.dimensoes_max,
```

```
            intervalo=intervalo,
        )

    def multiplicar(self, outro: "CampoProcedural | float") -> "CampoProcedural":
        if isinstance(outro, CampoProcedural):
            dimensoes_min, dimensoes_max = self._dimensoes_com(outro)
            metadados = {"operacao": "multiplicar", "entradas": [self.nome, outro.nome]}
            intervalo = None
            if self.intervalo is not None and outro.intervalo is not None:
                candidatos = [a * b for a in self.intervalo for b in outro.intervalo]
                intervalo = (min(candidatos), max(candidatos))
            return CampoProcedural(
                lambda *c: self.valor(*c) * outro.valor(*c),
                nome=f"{self.nome}*{outro.nome}",
                metadados=metadados,
                dimensoes_min=dimensoes_min,
                dimensoes_max=dimensoes_max,
                intervalo=intervalo,
            )
        numero = _numero_finito(outro, "fator")
        intervalo = None
        if self.intervalo is not None:
            extremos = (self.intervalo[0] * numero, self.intervalo[1] * numero)
            intervalo = (min(extremos), max(extremos))
        return CampoProcedural(
            lambda *c: self.valor(*c) * numero,
            nome=f"{self.nome}*constante",
            metadados={"operacao": "multiplicar_constante", "entrada": self.nome},
            dimensoes_min=self.dimensoes_min,
            dimensoes_max=self.dimensoes_max,
            intervalo=intervalo,
        )

    def mascara(
        self,
        *,
        minimo: float | None = None,
        maximo: float | None = None,
        dentro: float = 1.0,
        fora: float = 0.0,
    ) -> "CampoProcedural":
        minimo_n = None if minimo is None else _numero_finito(minimo, "mínimo")
        maximo_n = None if maximo is None else _numero_finito(maximo, "máximo")
        dentro_n = _numero_finito(dentro, "valor dentro")
        fora_n = _numero_finito(fora, "valor fora")
        if minimo_n is not None and maximo_n is not None and maximo_n < minimo_n:
            raise ValueError("máximo não pode ser menor que mínimo")

        def avaliar(*coords: float) -> float:
            valor = self.valor(*coords)
            ok = (minimo_n is None or valor >= minimo_n) and (maximo_n is None or valor
              <= maximo_n)
            return dentro_n if ok else fora_n

        metadados = {
            "operacao": "mascara",
            "entrada": self.nome,
            "minimo": minimo_n,
            "maximo": maximo_n,
            "dentro": dentro_n,
            "fora": fora_n,
```

```
        }
        return CampoProcedural(
            avaliar,
            nome=f"{self.nome}.mascara",
            metadados=metadados,
            dimensoes_min=self.dimensoes_min,
            dimensoes_max=self.dimensoes_max,
            intervalo=(min(dentro_n, fora_n), max(dentro_n, fora_n)),
        )

    def __add__(self, outro: "CampoProcedural | float") -> "CampoProcedural":
        return self.somar(outro)

    def __mul__(self, outro: "CampoProcedural | float") -> "CampoProcedural":
        return self.multiplicar(outro)


def campo_constante(valor: float, *, nome: str = "constante") -> CampoProcedural:
    numero = _numero_finito(valor, "valor")
    return CampoProcedural(lambda *_: numero, nome=nome, intervalo=(numero, numero))


def _suavizar(t: float) -> float:
    return t * t * (3.0 - 2.0 * t)


def _interpolar(a: float, b: float, t: float) -> float:
    return a + (b - a) * t


def _cantos(dimensoes: int):
    total = 1 << dimensoes
    for mascara in range(total):
        yield tuple((mascara >> eixo) & 1 for eixo in range(dimensoes))


def _ruido_valor_ponto(gerador: GeradorProcedural, coordenadas: tuple[float, ...]) ->
  float:
    bases = tuple(math.floor(v) for v in coordenadas)
    fracoes = tuple(_suavizar(v - base) for v, base in zip(coordenadas, bases))
    valores: dict[tuple[int, ...], float] = {}
    for canto in _cantos(len(coordenadas)):
        chave = tuple(base + delta for base, delta in zip(bases, canto))
        valores[canto] = gerador.decimal(*chave, minimo=-1.0, maximo=1.0)

    total = 0.0
    for canto, valor in valores.items():
        peso = 1.0
        for eixo, bit in enumerate(canto):
            t = fracoes[eixo]
            peso *= t if bit else (1.0 - t)
        total += valor * peso
    return total


def campo_ruido(
    semente_raiz: Any = 0,
    *,
    escala: float = 1.0,
    oitavas: int = 1,
    persistencia: float = 0.5,
```

```
    lacunaridade: float = 2.0,
    nome: str = "ruido",
) -> CampoProcedural:
    """Cria ruído de valor suave para 1D, 2D ou 3D sem dependências externas."""
    escala = _numero_finito(escala, "escala")
    persistencia = _numero_finito(persistencia, "persistência")
    lacunaridade = _numero_finito(lacunaridade, "lacunaridade")
    if escala <= 0 or lacunaridade <= 0 or persistencia < 0:
        raise ValueError("escala e lacunaridade precisam ser positivas e persistência
          não negativa")
    if isinstance(oitavas, bool) or int(oitavas) != oitavas or int(oitavas) <= 0:
        raise ValueError("oitavas precisa ser um inteiro positivo")
    oitavas = int(oitavas)
    raiz = semente(semente_raiz)

    def avaliar(*coords: float) -> float:
        if not 1 <= len(coords) <= 3:
            raise ValueError("ruído procedural suporta de uma a três dimensões")
        frequencia = 1.0 / escala
        amplitude = 1.0
        soma = 0.0
        peso = 0.0
        for oitava in range(oitavas):
            gerador = GeradorProcedural(raiz.derivar("ruido", oitava),
              namespace="grade")
            ponto = tuple(c * frequencia for c in coords)
            soma += _ruido_valor_ponto(gerador, ponto) * amplitude
            peso += amplitude
            frequencia *= lacunaridade
            amplitude *= persistencia
        return 0.0 if peso == 0 else soma / peso

    return CampoProcedural(
        avaliar,
        nome=nome,
        metadados={
            "tipo": "ruido_valor",
            "escala": escala,
            "oitavas": oitavas,
            "persistencia": persistencia,
            "lacunaridade": lacunaridade,
            "semente": raiz.hexadecimal,
        },
        dimensoes_min=1,
        dimensoes_max=3,
        intervalo=(-1.0, 1.0),
    )


def distribuir_pontos(
    largura: float,
    altura: float,
    quantidade: int,
    *,
    semente_raiz: Any = 0,
    margem: float = 0.0,
    distancia_minima: float = 0.0,
    restricao: Callable[[float, float], bool] | None = None,
    max_tentativas_por_ponto: int = 200,
    limite_tentativas_total: int = 1_000_000,
) -> tuple[tuple[float, float], ...]:
```

```
    """Distribui pontos 2D determinísticos sob limites finitos de trabalho.

    ``limite_tentativas_total`` é um orçamento preventivo: configurações cujo
    pior caso declarado exceda o limite falham antes de materializar pontos.
    O chamador pode elevar explicitamente o limite quando aceitar o custo.
    """
    largura = _numero_finito(largura, "largura")
    altura = _numero_finito(altura, "altura")
    margem = _numero_finito(margem, "margem")
    distancia_minima = _numero_finito(distancia_minima, "distância mínima")
    if largura <= 0 or altura <= 0:
        raise ValueError("largura e altura precisam ser positivas")
    if margem < 0 or margem * 2 >= largura or margem * 2 >= altura:
        raise ValueError("margem inválida para a área informada")
    if distancia_minima < 0:
        raise ValueError("distância mínima não pode ser negativa")
    quantidade = _inteiro_exato(quantidade, "quantidade", minimo=0)
    max_tentativas_por_ponto = _inteiro_exato(
        max_tentativas_por_ponto, "máximo de tentativas por ponto", minimo=1
    )
    limite_tentativas_total = _inteiro_exato(
        limite_tentativas_total, "limite total de tentativas", minimo=1
    )
    if restricao is not None and not callable(restricao):
        raise TypeError("restrição espacial precisa ser uma função")

    pior_caso = quantidade * max_tentativas_por_ponto
    if pior_caso > limite_tentativas_total:
        raise ValueError(
            "distribuição excede o limite total de tentativas; aumente o limite
              explicitamente se aceitar o custo"
        )

    largura_util = largura - 2.0 * margem
    altura_util = altura - 2.0 * margem
    diagonal_util = math.hypot(largura_util, altura_util)
    if quantidade > 1 and distancia_minima > diagonal_util:
        raise ValueError("distância mínima torna impossível distribuir mais de um ponto
          nessa área")

    fonte = semente(semente_raiz).fonte("pontos")
    pontos: list[tuple[float, float]] = []
    distancia2 = distancia_minima * distancia_minima
    for _ in range(quantidade):
        aceito: tuple[float, float] | None = None
        for _tentativa in range(max_tentativas_por_ponto):
            x = fonte.decimal(margem, largura - margem)
            y = fonte.decimal(margem, altura - margem)
            if restricao is not None and not restricao(x, y):
                continue
            if distancia2 and any((x - px) ** 2 + (y - py) ** 2 < distancia2 for px, py
              in pontos):
                continue
            aceito = (x, y)
            break
        if aceito is None:
            raise RuntimeError("não foi possível satisfazer as restrições da
              distribuição procedural")
        pontos.append(aceito)
    return tuple(pontos)
```

```
def amostrar_campo(
    campo: CampoProcedural,
    coordenadas: Iterable[Sequence[float]],
    *,
    nomes_eixos: Sequence[str] | None = None,
    nome_valor: str = "valor",
    limite_amostras: int = 100_000,
) -> tuple[dict[str, Any], ...]:
    """Materializa amostras portáteis de um campo sem depender de Numérico ou
      Laboratório.

    Cada registro contém ``indice``, as coordenadas nomeadas e o valor calculado.
    O limite é aplicado durante a leitura para também proteger iteráveis sem tamanho.
    """
    if not isinstance(campo, CampoProcedural):
        raise TypeError("amostrar_campo exige um CampoProcedural")
    limite_amostras = _inteiro_exato(limite_amostras, "limite de amostras", minimo=1)
    nome_valor = str(nome_valor).strip()
    if not nome_valor:
        raise ValueError("nome do valor precisa ser texto não vazio")

    nomes = None if nomes_eixos is None else tuple(str(n).strip() for n in nomes_eixos)
    if nomes is not None and (not nomes or any(not n for n in nomes) or len(set(nomes))
      != len(nomes)):
        raise ValueError("nomes dos eixos precisam ser textos não vazios e distintos")
    if nomes is not None and nome_valor in nomes:
        raise ValueError("nome do valor não pode repetir um nome de eixo")

    saida: list[dict[str, Any]] = []
    for indice, coordenada in enumerate(coordenadas):
        if indice >= limite_amostras:
            raise ValueError(
                "amostragem procedural excede o limite de amostras; aumente o limite
                  explicitamente se aceitar o custo"
            )
        if isinstance(coordenada, (str, bytes)):
            raise ValueError("cada coordenada precisa ser uma sequência numérica")
        try:
            coords = tuple(_numero_finito(v, "coordenada") for v in coordenada)
        except TypeError as exc:
            raise ValueError("cada coordenada precisa ser uma sequência numérica") from
              exc
        if not coords:
            raise ValueError("cada amostra precisa informar ao menos uma coordenada")
        eixos = nomes
        if eixos is None:
            padrao = ("x", "y", "z", "w")
            eixos = tuple(padrao[i] if i < len(padrao) else f"eixo_{i + 1}" for i in
              range(len(coords)))
        if len(eixos) != len(coords):
            raise ValueError("quantidade de nomes de eixos não corresponde às
              coordenadas")
        registro = {"indice": indice}
        registro.update(zip(eixos, coords))
        registro[nome_valor] = campo.valor(*coords)
        saida.append(registro)
    return tuple(saida)


def matriz_campo(
```

```
    campo: CampoProcedural,
    colunas: int,
    linhas: int,
    *,
    origem: Sequence[float] = (0.0, 0.0),
    passo: Sequence[float] = (1.0, 1.0),
    limite_celulas: int = 1_000_000,
) -> tuple[tuple[float, ...], ...]:
    """Amostra um campo 2D como matriz, pronta para mapas, jogos e mapas de calor."""
    if not isinstance(campo, CampoProcedural):
        raise TypeError("matriz_campo exige um CampoProcedural")
    colunas = _inteiro_exato(colunas, "colunas", minimo=1)
    linhas = _inteiro_exato(linhas, "linhas", minimo=1)
    limite_celulas = _inteiro_exato(limite_celulas, "limite de células", minimo=1)
    if colunas * linhas > limite_celulas:
        raise ValueError(
            "matriz procedural excede o limite de células; aumente o limite
              explicitamente se aceitar o custo"
        )
    if len(origem) != 2 or len(passo) != 2:
        raise ValueError("origem e passo precisam conter x e y")
    ox, oy = (_numero_finito(v, "origem") for v in origem)
    px, py = (_numero_finito(v, "passo") for v in passo)
    if px == 0 or py == 0:
        raise ValueError("passo da matriz procedural não pode ser zero")
    return tuple(
        tuple(campo.valor(ox + x * px, oy + y * py) for x in range(colunas))
        for y in range(linhas)
    )


def anotar_pontos(
    pontos: Iterable[Sequence[float]],
    campos: Mapping[str, CampoProcedural],
    *,
    limite_pontos: int = 100_000,
) -> tuple[dict[str, Any], ...]:
    """Anexa valores de campos a pontos 2D em registros gerais e portáteis."""
    limite_pontos = _inteiro_exato(limite_pontos, "limite de pontos", minimo=1)
    campos_n = dict(campos)
    if not campos_n:
        raise ValueError("anotar_pontos exige ao menos um campo")
    for nome, campo in campos_n.items():
        if not isinstance(nome, str) or not nome.strip():
            raise ValueError("nomes dos campos precisam ser textos não vazios")
        if nome in {"indice", "x", "y"}:
            raise ValueError(f"nome de campo reservado: {nome}")
        if not isinstance(campo, CampoProcedural):
            raise TypeError(f"campo {nome!r} precisa ser CampoProcedural")

    saida: list[dict[str, Any]] = []
    for indice, ponto in enumerate(pontos):
        if indice >= limite_pontos:
            raise ValueError(
                "anotação procedural excede o limite de pontos; aumente o limite
                  explicitamente se aceitar o custo"
            )
        if isinstance(ponto, (str, bytes)):
            raise ValueError("ponto procedural precisa conter x e y")
        coords = tuple(ponto)
        if len(coords) != 2:
```

```
            raise ValueError("ponto procedural precisa conter exatamente x e y")
        x = _numero_finito(coords[0], "x")
        y = _numero_finito(coords[1], "y")
        registro = {"indice": indice, "x": x, "y": y}
        for nome, campo in campos_n.items():
            registro[nome] = campo.valor(x, y)
        saida.append(registro)
    return tuple(saida)


def proveniencia_procedural(objeto: Any) -> dict[str, Any]:
    """Descreve origem procedural sem importar domínios consumidores."""
    if isinstance(objeto, ReceitaProcedural):
        return {
            "contrato": "coral.procedural.proveniencia/1",
            "tipo": "receita",
            "reconstruivel": True,
            "receita": objeto.para_dados(),
        }
    if isinstance(objeto, Semente):
        return {
            "contrato": "coral.procedural.proveniencia/1",
            "tipo": "semente",
            "reconstruivel": True,
            "semente": _semente_portatil(objeto),
            "hexadecimal": objeto.hexadecimal,
        }
    if isinstance(objeto, GeradorProcedural):
        return {
            "contrato": "coral.procedural.proveniencia/1",
            "tipo": "gerador",
            "reconstruivel": True,
            "namespace": objeto.namespace,
            "semente": _semente_portatil(objeto.semente),
        }
    if isinstance(objeto, CampoProcedural):
        return {
            "contrato": "coral.procedural.proveniencia/1",
            "tipo": "campo",
            "reconstruivel": False,
            "nome": objeto.nome,
            "dimensoes_min": objeto.dimensoes_min,
            "dimensoes_max": objeto.dimensoes_max,
            "intervalo": None if objeto.intervalo is None else list(objeto.intervalo),
            "metadados": deepcopy(objeto.metadados),
        }
    raise TypeError("objeto não possui proveniência procedural conhecida")


_ALGORITMOS_RECEITA = {"campo_constante", "campo_ruido", "distribuir_pontos"}


def _semente_portatil(valor: Semente) -> dict[str, Any]:
    bruto = valor.valor
    if bruto is None:
        return {"tipo": "nulo", "valor": None}
    if isinstance(bruto, bool):
        return {"tipo": "booleano", "valor": bruto}
    if isinstance(bruto, int):
        return {"tipo": "inteiro", "valor": str(bruto)}
    if isinstance(bruto, float):
```

```
        return {"tipo": "decimal", "valor": bruto.hex()}
    if isinstance(bruto, str):
        return {"tipo": "texto", "valor": bruto}
    if isinstance(bruto, bytes):
        return {"tipo": "bytes", "valor": bruto.hex()}
    raise TypeError("semente de receita não é portátil")


def _semente_portatil_ler(dados: Any) -> Semente:
    if not isinstance(dados, dict):
        raise ValueError("semente da receita precisa ser um objeto")
    tipo = dados.get("tipo")
    valor = dados.get("valor")
    if tipo == "nulo" and valor is None:
        return Semente(None)
    if tipo == "booleano" and isinstance(valor, bool):
        return Semente(valor)
    if tipo == "inteiro" and isinstance(valor, str):
        try:
            return Semente(int(valor))
        except ValueError as exc:
            raise ValueError("semente inteira da receita é inválida") from exc
    if tipo == "decimal" and isinstance(valor, str):
        try:
            numero = float.fromhex(valor)
        except ValueError as exc:
            raise ValueError("semente decimal da receita é inválida") from exc
        return Semente(numero)
    if tipo == "texto" and isinstance(valor, str):
        return Semente(valor)
    if tipo == "bytes" and isinstance(valor, str):
        try:
            return Semente(bytes.fromhex(valor))
        except ValueError as exc:
            raise ValueError("semente bytes da receita é inválida") from exc
    raise ValueError("semente da receita possui tipo ou valor inválido")


def _parametros_receita(algoritmo: str, parametros: Mapping[str, Any]) -> dict[str,
  Any]:
    if not isinstance(parametros, Mapping):
        raise TypeError("parâmetros da receita precisam ser um mapa")
    p = dict(parametros)

    if algoritmo == "campo_constante":
        extras = set(p) - {"valor", "nome"}
        if extras:
            raise ValueError("campo_constante recebeu parâmetro de receita desconhecido:
               " + sorted(extras)[0])
        if "valor" not in p:
            raise ValueError("receita campo_constante exige o parâmetro valor")
        return {
            "valor": _numero_finito(p["valor"], "valor"),
            "nome": str(p.get("nome", "constante")),
        }

    if algoritmo == "campo_ruido":
        extras = set(p) - {"escala", "oitavas", "persistencia", "lacunaridade", "nome"}
        if extras:
            raise ValueError("campo_ruido recebeu parâmetro de receita desconhecido: " +
               sorted(extras)[0])
```

```
        escala = _numero_finito(p.get("escala", 1.0), "escala")
        oitavas = _inteiro_exato(p.get("oitavas", 1), "oitavas", minimo=1)
        persistencia = _numero_finito(p.get("persistencia", 0.5), "persistência")
        lacunaridade = _numero_finito(p.get("lacunaridade", 2.0), "lacunaridade")
        if escala <= 0 or lacunaridade <= 0 or persistencia < 0:
            raise ValueError("escala e lacunaridade precisam ser positivas e
              persistência não negativa")
        return {
            "escala": escala,
            "oitavas": oitavas,
            "persistencia": persistencia,
            "lacunaridade": lacunaridade,
            "nome": str(p.get("nome", "ruido")),
        }

    if algoritmo == "distribuir_pontos":
        extras = set(p) - {
            "largura", "altura", "quantidade", "margem", "distancia_minima",
            "max_tentativas_por_ponto", "limite_tentativas_total",
        }
        if extras:
            extra = sorted(extras)[0]
            if extra == "restricao":
                raise ValueError("receita portátil não aceita callback de restrição")
            raise ValueError("distribuir_pontos recebeu parâmetro de receita
              desconhecido: " + extra)
        faltantes = [nome for nome in ("largura", "altura", "quantidade") if nome not in
           p]
        if faltantes:
            raise ValueError("receita distribuir_pontos exige: " + ", ".join(faltantes))
        largura = _numero_finito(p["largura"], "largura")
        altura = _numero_finito(p["altura"], "altura")
        margem = _numero_finito(p.get("margem", 0.0), "margem")
        distancia = _numero_finito(p.get("distancia_minima", 0.0), "distância mínima")
        quantidade = _inteiro_exato(p["quantidade"], "quantidade", minimo=0)
        tentativas = _inteiro_exato(
            p.get("max_tentativas_por_ponto", 200), "máximo de tentativas por ponto",
              minimo=1
        )
        limite = _inteiro_exato(
            p.get("limite_tentativas_total", LIMITE_CUSTO_PADRAO), "limite total de
              tentativas", minimo=1
        )
        if largura <= 0 or altura <= 0:
            raise ValueError("largura e altura precisam ser positivas")
        if margem < 0 or margem * 2 >= largura or margem * 2 >= altura:
            raise ValueError("margem inválida para a área informada")
        if distancia < 0:
            raise ValueError("distância mínima não pode ser negativa")
        if quantidade * tentativas > limite:
            raise ValueError(
                "distribuição excede o limite total de tentativas; aumente o limite
                  explicitamente se aceitar o custo"
            )
        return {
            "largura": largura,
            "altura": altura,
            "quantidade": quantidade,
            "margem": margem,
            "distancia_minima": distancia,
            "max_tentativas_por_ponto": tentativas,
```

```
            "limite_tentativas_total": limite,
        }

    raise ValueError("algoritmo de receita procedural desconhecido: " + algoritmo)


class ReceitaProcedural:
    """Descrição portátil e reconstruível de uma operação procedural suportada.

    A receita não guarda callbacks nem objetos de execução. Apenas algoritmo, versão,
    semente e parâmetros portáteis entram na representação.
    """

    __slots__ = ("algoritmo", "versao_algoritmo", "semente", "_parametros")

    def __init__(
        self,
        algoritmo: str,
        *,
        semente_raiz: Any = 0,
        parametros: Mapping[str, Any] | None = None,
        versao_algoritmo: int = VERSAO_RECEITA,
    ) -> None:
        algoritmo = str(algoritmo).strip()
        if algoritmo not in _ALGORITMOS_RECEITA:
            raise ValueError("algoritmo de receita procedural desconhecido: " +
              algoritmo)
        versao_algoritmo = _inteiro_exato(versao_algoritmo, "versão do algoritmo",
          minimo=1)
        if versao_algoritmo != VERSAO_RECEITA:
            raise ValueError(
                f"versão de receita procedural não suportada: {versao_algoritmo};
                  esperada {VERSAO_RECEITA}"
            )
        self.algoritmo = algoritmo
        self.versao_algoritmo = versao_algoritmo
        self.semente = semente(semente_raiz)
        self._parametros = _parametros_receita(algoritmo, parametros or {})

    @property
    def contrato(self) -> str:
        return CONTRATO_RECEITA

    @property
    def parametros(self) -> dict[str, Any]:
        return deepcopy(self._parametros)

    def para_dados(self) -> dict[str, Any]:
        return {
            "contrato": CONTRATO_RECEITA,
            "versao_algoritmo": self.versao_algoritmo,
            "algoritmo": self.algoritmo,
            "semente": _semente_portatil(self.semente),
            "parametros": self.parametros,
        }

    def para_json(self, *, identar: int | None = None) -> str:
        return json.dumps(
            self.para_dados(),
            ensure_ascii=False,
            sort_keys=True,
```

```
            separators=(",", ":") if identar is None else None,
            indent=identar,
            allow_nan=False,
        )

    @classmethod
    def de_dados(cls, dados: Mapping[str, Any]) -> "ReceitaProcedural":
        if not isinstance(dados, Mapping):
            raise ValueError("receita procedural precisa ser um objeto")
        if dados.get("contrato") != CONTRATO_RECEITA:
            raise ValueError("contrato de receita procedural inválido")
        algoritmo = dados.get("algoritmo")
        if not isinstance(algoritmo, str):
            raise ValueError("algoritmo da receita procedural precisa ser texto")
        versao = dados.get("versao_algoritmo")
        semente_lida = _semente_portatil_ler(dados.get("semente"))
        parametros = dados.get("parametros")
        if not isinstance(parametros, Mapping):
            raise ValueError("parâmetros da receita procedural precisam ser um objeto")
        return cls(
            algoritmo,
            semente_raiz=semente_lida,
            parametros=parametros,
            versao_algoritmo=versao,
        )

    @classmethod
    def de_json(cls, texto: str) -> "ReceitaProcedural":
        if not isinstance(texto, str):
            raise TypeError("JSON da receita procedural precisa ser texto")
        try:
            dados = json.loads(texto)
        except json.JSONDecodeError as exc:
            raise ValueError("JSON da receita procedural é inválido") from exc
        return cls.de_dados(dados)

    def __eq__(self, outro: object) -> bool:
        return isinstance(outro, ReceitaProcedural) and self.para_dados() ==
          outro.para_dados()

    def __repr__(self) -> str:
        return f"ReceitaProcedural({self.algoritmo!r}, versao={self.versao_algoritmo})"


def receita_procedural(algoritmo: str, *, semente_raiz: Any = 0, **parametros: Any) ->
  ReceitaProcedural:
    """Cria uma receita portátil para um dos algoritmos declarativos suportados."""
    return ReceitaProcedural(algoritmo, semente_raiz=semente_raiz,
      parametros=parametros)


def diagnosticar_custo_receita(receita: ReceitaProcedural | Mapping[str, Any]) ->
  dict[str, Any]:
    """Estima o trabalho máximo conhecido antes de reconstruir a receita."""
    if not isinstance(receita, ReceitaProcedural):
        receita = ReceitaProcedural.de_dados(receita)
    if receita.algoritmo == "distribuir_pontos":
        p = receita._parametros
        custo = p["quantidade"] * p["max_tentativas_por_ponto"]
        unidade = "tentativas"
    else:
```

```
        custo = 1
        unidade = "construções"
    return {
        "contrato": "coral.procedural.custo/1",
        "algoritmo": receita.algoritmo,
        "versao_algoritmo": receita.versao_algoritmo,
        "custo_maximo_estimado": custo,
        "unidade": unidade,
    }


def reconstruir_receita(
    receita: ReceitaProcedural | Mapping[str, Any],
    *,
    limite_custo: int = LIMITE_CUSTO_PADRAO,
) -> Any:
    """Reconstrói uma receita após validar versão e orçamento explícito."""
    if not isinstance(receita, ReceitaProcedural):
        receita = ReceitaProcedural.de_dados(receita)
    limite_custo = _inteiro_exato(limite_custo, "limite de custo", minimo=1)
    diagnostico = diagnosticar_custo_receita(receita)
    if diagnostico["custo_maximo_estimado"] > limite_custo:
        raise ValueError(
            "receita procedural excede o limite de custo; aumente o limite
              explicitamente se aceitar o trabalho"
        )
    p = receita._parametros
    if receita.algoritmo == "campo_constante":
        return campo_constante(p["valor"], nome=p["nome"])
    if receita.algoritmo == "campo_ruido":
        return campo_ruido(
            receita.semente,
            escala=p["escala"],
            oitavas=p["oitavas"],
            persistencia=p["persistencia"],
            lacunaridade=p["lacunaridade"],
            nome=p["nome"],
        )
    if receita.algoritmo == "distribuir_pontos":
        return distribuir_pontos(
            p["largura"],
            p["altura"],
            p["quantidade"],
            semente_raiz=receita.semente,
            margem=p["margem"],
            distancia_minima=p["distancia_minima"],
            max_tentativas_por_ponto=p["max_tentativas_por_ponto"],
            limite_tentativas_total=p["limite_tentativas_total"],
        )
    raise ValueError("algoritmo de receita procedural desconhecido: " +
      receita.algoritmo)


__all__ = [
    "CONTRATO",
    "Semente",
    "semente",
    "derivar_semente",
    "GeradorProcedural",
    "SequenciaProcedural",
    "CampoProcedural",
```

```
    "campo_constante",
    "campo_ruido",
    "distribuir_pontos",
    "amostrar_campo",
    "matriz_campo",
    "anotar_pontos",
    "proveniencia_procedural",
    "CONTRATO_RECEITA",
    "ReceitaProcedural",
    "receita_procedural",
    "diagnosticar_custo_receita",
    "reconstruir_receita",
]

# Adaptadores persistentes para a parte declarativa da fundação procedural.
def _semente_para_dados(valor: Semente) -> dict[str, Any]:
    bruto = valor.valor
    if isinstance(bruto, bytes):
        return {"tipo": "bytes", "valor": bruto.hex()}
    return {"tipo": type(bruto).__name__, "valor": bruto}


def _semente_de_dados(dados: dict[str, Any]) -> Semente:
    tipo = dados.get("tipo")
    valor = dados.get("valor")
    if tipo == "bytes":
        if not isinstance(valor, str):
            raise ValueError("semente bytes persistida inválida")
        valor = bytes.fromhex(valor)
    return Semente(valor)


def _gerador_para_dados(valor: GeradorProcedural) -> dict[str, Any]:
    return {"semente": valor.semente, "namespace": valor.namespace}


def _gerador_de_dados(dados: dict[str, Any]) -> GeradorProcedural:
    raiz = dados["semente"]
    if not isinstance(raiz, Semente):
        raise ValueError("semente procedural persistida inválida")
    return GeradorProcedural(raiz, namespace=dados.get("namespace", "gerador"))


from .persistencia import registrar_adaptador as _registrar_adaptador

_registrar_adaptador(Semente, "coral.procedural/Semente", _semente_para_dados,
  _semente_de_dados)
_registrar_adaptador(GeradorProcedural, "coral.procedural/GeradorProcedural",
  _gerador_para_dados, _gerador_de_dados)


def _receita_para_dados(valor: ReceitaProcedural) -> dict[str, Any]:
    return {"receita": valor.para_dados()}


def _receita_de_dados(dados: dict[str, Any]) -> ReceitaProcedural:
    receita = dados.get("receita")
    if not isinstance(receita, dict):
        raise ValueError("receita procedural persistida inválida")
    return ReceitaProcedural.de_dados(receita)
```

```
_registrar_adaptador(ReceitaProcedural, "coral.procedural/ReceitaProcedural",
  _receita_para_dados, _receita_de_dados)

from __future__ import annotations

import random
from dataclasses import dataclass, field
from typing import Any, Iterable, Sequence


@dataclass
class FonteAleatoria:
    """Fonte reproduzível quando criada com semente explícita.

    Não deve ser usada para senhas, chaves, tokens ou outros segredos.
    """

    semente: Any = None
    _gerador: random.Random = field(init=False, repr=False)

    def __post_init__(self):
        self._gerador = random.Random(self.semente)

    def inteiro(self, minimo, maximo):
        return self._gerador.randint(minimo, maximo)

    def decimal(self, minimo=0.0, maximo=1.0):
        return self._gerador.uniform(minimo, maximo)

    def escolher(self, valores):
        return self._gerador.choice(list(valores))

    def amostra(self, valores, quantidade):
        return self._gerador.sample(list(valores), quantidade)

    def escolha_ponderada(self, valores, pesos):
        valores = list(valores)
        pesos = list(pesos)
        if len(valores) != len(pesos) or not valores:
            raise ValueError("valores e pesos precisam ter o mesmo tamanho não vazio")
        if any(float(p) < 0 for p in pesos) or sum(map(float, pesos)) <= 0:
            raise ValueError("pesos precisam ser não negativos e ter soma positiva")
        return self._gerador.choices(valores, weights=pesos, k=1)[0]

    def embaralhar(self, valores):
        copia = list(valores)
        self._gerador.shuffle(copia)
        return copia


def fonte(semente=None) -> FonteAleatoria:
    return FonteAleatoria(semente)


def _g(fonte_aleatoria: FonteAleatoria | None):
    return fonte_aleatoria if fonte_aleatoria is not None else None


def inteiro(minimo, maximo, *, fonte: FonteAleatoria | None = None):
    return fonte.inteiro(minimo, maximo) if fonte is not None else
```

```
      random.randint(minimo, maximo)


def decimal(minimo=0.0, maximo=1.0, *, fonte: FonteAleatoria | None = None):
    return fonte.decimal(minimo, maximo) if fonte is not None else
      random.uniform(minimo, maximo)


def escolher(valores, *, fonte: FonteAleatoria | None = None):
    return fonte.escolher(valores) if fonte is not None else
      random.choice(list(valores))


def amostra(valores, quantidade, *, fonte: FonteAleatoria | None = None):
    return fonte.amostra(valores, quantidade) if fonte is not None else
      random.sample(list(valores), quantidade)


def escolha_ponderada(valores, pesos, *, fonte: FonteAleatoria | None = None):
    if fonte is not None:
        return fonte.escolha_ponderada(valores, pesos)
    valores = list(valores); pesos = list(pesos)
    if len(valores) != len(pesos) or not valores:
        raise ValueError("valores e pesos precisam ter o mesmo tamanho não vazio")
    if any(float(p) < 0 for p in pesos) or sum(map(float, pesos)) <= 0:
        raise ValueError("pesos precisam ser não negativos e ter soma positiva")
    return random.choices(valores, weights=pesos, k=1)[0]


def embaralhar(valores, *, fonte: FonteAleatoria | None = None):
    if fonte is not None:
        return fonte.embaralhar(valores)
    copia = list(valores)
    random.shuffle(copia)
    return copia


__all__ = [
    "FonteAleatoria", "fonte", "inteiro", "decimal", "escolher", "amostra",
    "escolha_ponderada", "embaralhar",
]
```

Biblioteca `coral.regras`

`coral.regras.Acao`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
Acao(*args, **kwargs)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
args	sem anotação	VAR_POSITIONAL	não	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
kwargs	sem anotação	VAR_KEYWORD	não	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.regras.Condicao`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
Condicao(*args, **kwargs)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
args	sem anotação	VAR_POSITIONAL	não	sem padrão
kwargs	sem anotação	VAR_KEYWORD	não	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

`coral.regras.Debounce`

Debounce(intervalo: 'float', acao: 'Acao', nome: 'str' = 'debounce', restante: 'float | None' = None, pendente: 'bool' = False, contexto_mais_recente: 'Any' = None)

Assinatura

Assinatura para consulta

```
Debounce(intervalo: 'float', acao: 'Acao', nome: 'str' = 'debounce', restante: 'float |
  None' = None, pendente: 'bool' = False, contexto_mais_recente: 'Any' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
intervalo	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
acao	Acao	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str	POSITIONAL_OR_KEYWORD	não	'debounce'

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
restante	float | None	POSITIONAL_OR_KEYWORD	não	None
pendente	bool	POSITIONAL_OR_KEYWORD	não	False
contexto_mais_recente	Any	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `intervalo`: `float`.

Atributo `acao`: `Acao`.

Atributo `nome`: `str`.

Atributo `restante`: `float | None`.

Atributo `pendente`: `bool`.

Atributo `contexto_mais_recente`: `Any`.

Métodos públicos

Referência para leitura

```
atualizar(self, dt: 'float') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
notificar(self, contexto: 'Any' = None) -> 'None'
```

Método público sem descrição própria no runtime.

`coral.regras.Eventos`

Barramento local compatível com `coral.regras.Eventos`.

`emitir mantém a chamada histórica de ouvintes via *args e **kwargs. Inscrições novas podem receber o objeto :class:Evento` completo e declarar prioridade sem alterar consumidores antigos.

Assinatura

Assinatura para consulta

```
Eventos(*, pai: "'Eventos | None'" = None, relogio: 'FonteTempo | None' = None,
  fonte_instante: 'Callable[[], float] | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
pai	'Eventos | None'	KEYWORD_ONLY	não	None
relogio	FonteTempo | None	KEYWORD_ONLY	não	None
fonte_instante	Callable[[], float] | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
definir_fonte_instante(self, fonte: 'Callable[[], float] | None') -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
definir_pai(self, pai: "'Eventos | None'") -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
emitir(self, nome: 'str', *args, **kwargs) -> 'int'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
emitir_evento(self, nome: 'str', *args, origem: 'str | None' = None, cancelavel: 'bool'
  = False, metadados: 'Mapping[str, Any] | None' = None, **kwargs) -> 'Evento'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
emitir_evento_propagado(self, nome: 'str', *args, origem: 'str | None' = None,
  cancelavel: 'bool' = False, metadados: 'Mapping[str, Any] | None' = None, **kwargs) ->
  'Evento'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
emitir_propagado(self, nome: 'str', *args, origem: 'str | None' = None, **kwargs) ->
  'int'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
enfileirar(self, nome: 'str', *args, origem: 'str | None' = None, **kwargs) -> 'Evento'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
enfileirar_evento(self, nome: 'str', *args, origem: 'str | None' = None, cancelavel:
  'bool' = False, metadados: 'Mapping[str, Any] | None' = None, **kwargs) -> 'Evento'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
processar(self, limite: 'int | None' = None) -> 'tuple[Evento, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
quando(self, nome: 'str', funcao: 'Callable[..., Any]', *, prioridade: 'str | int' =
  'normal', receber_evento: 'bool' = False, max_execucoes: 'int | None' = None, cooldown:
  'float' = 0.0)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover(self, nome: 'str', funcao: 'Callable[..., Any]') -> 'bool'
```

Método público sem descrição própria no runtime.

`coral.regras.GrupoRegras`

GrupoRegras(nome: 'str', regras: 'tuple[Regra, ...]')

Assinatura

Assinatura para consulta

```
GrupoRegras(nome: 'str', regras: 'tuple[Regra, ...]') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
regras	tuple[Regra, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `nome`: `str`.

Atributo `regras`: `tuple[Regra, ...]`.

Métodos públicos

Referência para leitura

```
desabilitar(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
habilitar(self) -> 'None'
```

Método público sem descrição própria no runtime.

`coral.regras.MODOS_REGRA`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`frozenset({'ao_ativar', 'sempre', 'uma_vez'})`

`coral.regras.MotorRegras`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
MotorRegras(*, relogio: 'FonteTempo | None' = None, eventos: 'Eventos | None' = None,
  rastreamento: 'RastreamentoReativo | str | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
relogio	FonteTempo | None	KEYWORD_ONLY	não	None
eventos	Eventos | None	KEYWORD_ONLY	não	None
rastreamento	RastreamentoReativo | str | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
a_cada(self, segundos: 'float', acao: 'Acao', *, nome: 'str' = 'intervalo') ->
  'Temporizador'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
adicionar(self, regra: 'Regra') -> 'Regra'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
ativar_regra_ate(self, nome: 'str', condicao: 'Condicao') -> 'Regra'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
ativar_regra_durante(self, nome: 'str', segundos: 'float') -> 'Regra'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
atualizar(self, contexto: 'Any', dt: 'float | None' = None) -> 'tuple[str, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
atualizar_por_relogio(self, contexto: 'Any') -> 'tuple[str, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
avaliar(self, contexto: 'Any') -> 'tuple[str, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
criar_grupo(self, nome: 'str', regras: 'list[str] | tuple[str, ...]') -> 'GrupoRegras'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
debounce(self, segundos: 'float', acao: 'Acao', *, nome: 'str' = 'debounce') ->
  'Debounce'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
depois_de(self, segundos: 'float', acao: 'Acao', *, nome: 'str' = 'depois') ->
  'Temporizador'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
desabilitar_grupo(self, nome: 'str') -> 'GrupoRegras'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
desabilitar_regra(self, nome: 'str') -> 'Regra'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
emitir_evento(self, nome: 'str', *args, **kwargs) -> 'int'
```

Emite um evento de regra pelo barramento compartilhado, sem conhecer consumidores.

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
habilitar_grupo(self, nome: 'str') -> 'GrupoRegras'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
habilitar_regra(self, nome: 'str') -> 'Regra'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
instante_atual(self) -> 'float'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
observar_cair_abaixo_de(self, nome: 'str', leitor, limite, acao) -> 'ObservadorMudanca'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
observar_entrada_regiao(self, nome: 'str', leitor, regiao_alvo, acao, **opcoes) ->
  'ObservadorEspacial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
observar_intersecao(self, nome: 'str', leitor, referencia, acao, **opcoes) ->
  'ObservadorEspacial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
observar_mudanca(self, nome: 'str', leitor, acao) -> 'ObservadorMudanca'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
observar_passar_de(self, nome: 'str', leitor, limite, acao) -> 'ObservadorMudanca'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
observar_proximidade(self, nome: 'str', leitor, referencia, limite, acao, **opcoes) ->
  'ObservadorEspacial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
observar_regioes_mapa(self, nome: 'str', alvo: 'Any', mapa, regioes, resolvedor, *,
  identificador_alvo: 'Any' = None)
```

Observa transições discretas usando o mesmo barramento do motor.

Referência para leitura

```
observar_saida_regiao(self, nome: 'str', leitor, regiao_alvo, acao, **opcoes) ->
  'ObservadorEspacial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
observar_toque(self, nome: 'str', leitor, referencia, acao, **opcoes) ->
  'ObservadorEspacial'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
obter_grupo(self, nome: 'str') -> 'GrupoRegras'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
obter_regra(self, nome: 'str') -> 'Regra'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
processar_ciclo(self, contexto: 'Any' = None, *, limite_reacoes: 'int' = 1000) ->
  'tuple[Evento, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
regra(self, nome: 'str', condicao: 'Condicao', acao: 'Acao', *, modo: 'str' =
  'ao_ativar', prioridade: 'str | int' = 'normal', max_execucoes: 'int | None' = None,
  cooldown: 'float' = 0.0) -> 'Regra'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
sequencia(self, nome: 'str', primeiro: 'str', segundo: 'str', acao, *, janela: 'float |
  None' = None) -> 'SequenciaDoisPassos'
```

Método público sem descrição própria no runtime.

`coral.regras.ObservadorEspacial`

ObservadorEspacial(nome: 'str', leitor: 'LeitorValor', referencia: 'Any', acao: 'Callable[[Any, Any, Any], Any]', relacao: 'str', transicao: 'str' = 'entrar', limite: 'Any' = None, obter: 'Callable[[Any], Any] | None' = None, obter_referencia: 'Callable[[Any], Any] | None' = None)

Assinatura

Assinatura para consulta

```
ObservadorEspacial(nome: 'str', leitor: 'LeitorValor', referencia: 'Any', acao:
  'Callable[[Any, Any, Any], Any]', relacao: 'str', transicao: 'str' = 'entrar', limite:
  'Any' = None, obter: 'Callable[[Any], Any] | None' = None, obter_referencia:
  'Callable[[Any], Any] | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
leitor	LeitorValor	POSITIONAL_OR_KEYWORD	sim	sem padrão
referencia	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
acao	Callable[[Any, Any, Any], Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
relacao	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
transicao	str	POSITIONAL_OR_KEYWORD	não	'entrar'
limite	Any	POSITIONAL_OR_KEYWORD	não	None
obter	Callable[[Any], Any] | None	POSITIONAL_OR_KEYWORD	não	None
obter_referencia	Callable[[Any], Any] | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `nome`: `str`.

Atributo `leitor`: `LeitorValor`.

Atributo `referencia`: `Any`.

Atributo `acao`: `Callable[[Any, Any, Any], Any]`.

Atributo `relacao`: `str`.

Atributo `transicao`: `str`.

Atributo `limite`: `Any`.

Atributo `obter`: `Callable[[Any], Any] | None`.

Atributo `obter_referencia`: `Callable[[Any], Any] | None`.

Métodos públicos

Referência para leitura

```
avaliar(self, contexto: 'Any') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
chegar_a_menos_de(nome, leitor, referencia, limite, acao, *, obter=None,
  obter_referencia=None)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
entrar_em_regiao(nome, leitor, regiao_alvo, acao, *, obter=None, obter_referencia=None)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
intersectar(nome, leitor, referencia, acao, *, obter=None, obter_referencia=None)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
sair_de_regiao(nome, leitor, regiao_alvo, acao, *, obter=None, obter_referencia=None)
```

Método público sem descrição própria no runtime.

Referência para leitura

```
tocar(nome, leitor, referencia, acao, *, obter=None, obter_referencia=None)
```

Método público sem descrição própria no runtime.

`coral.regras.ObservadorMudanca`

ObservadorMudanca(nome: 'str', leitor: 'LeitorValor', acao: 'AcaoMudanca', modo: 'str' = 'mudanca', limite: 'Any' = None)

Assinatura

Assinatura para consulta

```
ObservadorMudanca(nome: 'str', leitor: 'LeitorValor', acao: 'AcaoMudanca', modo: 'str' =
   'mudanca', limite: 'Any' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
leitor	LeitorValor	POSITIONAL_OR_KEYWORD	sim	sem padrão
acao	AcaoMudanca	POSITIONAL_OR_KEYWORD	sim	sem padrão
modo	str	POSITIONAL_OR_KEYWORD	não	'mudanca'
limite	Any	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `nome`: `str`.

Atributo `leitor`: `LeitorValor`.

Atributo `acao`: `AcaoMudanca`.

Atributo `modo`: `str`.

Atributo `limite`: `Any`.

Métodos públicos

Referência para leitura

```
avaliar(self, contexto: 'Any') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
cair_abaixo_de(nome: 'str', leitor: 'LeitorValor', limite: 'Any', acao: 'AcaoMudanca')
  -> "'ObservadorMudanca'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
passar_de(nome: 'str', leitor: 'LeitorValor', limite: 'Any', acao: 'AcaoMudanca') ->
  "'ObservadorMudanca'"
```

Método público sem descrição própria no runtime.

`coral.regras.Regra`

Regra(nome: 'str', condicao: 'Condicao', acao: 'Acao', modo: 'str' = 'ao_ativar', habilitada: 'bool' = True, ativa: 'bool' = False, execucoes: 'int' = 0, prioridade: 'str | int' = 'normal', max_execucoes: 'int | None' = None, cooldown: 'float' = 0.0, ultimo_disparo: 'float | None' = None, motivo_ultima_supressao: 'str | None' = None, ultima_condicao: 'bool | None' = None)

Assinatura

Assinatura para consulta

```
Regra(nome: 'str', condicao: 'Condicao', acao: 'Acao', modo: 'str' = 'ao_ativar',
  habilitada: 'bool' = True, ativa: 'bool' = False, execucoes: 'int' = 0, prioridade: 'str
   | int' = 'normal', max_execucoes: 'int | None' = None, cooldown: 'float' = 0.0,
  ultimo_disparo: 'float | None' = None, motivo_ultima_supressao: 'str | None' = None,
  ultima_condicao: 'bool | None' = None) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
condicao	Condicao	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
acao	Acao	POSITIONAL_OR_KEYWORD	sim	sem padrão
modo	str	POSITIONAL_OR_KEYWORD	não	'ao_ativar'
habilitada	bool	POSITIONAL_OR_KEYWORD	não	True
ativa	bool	POSITIONAL_OR_KEYWORD	não	False
execucoes	int	POSITIONAL_OR_KEYWORD	não	0
prioridade	str | int	POSITIONAL_OR_KEYWORD	não	'normal'
max_execucoes	int | None	POSITIONAL_OR_KEYWORD	não	None
cooldown	float	POSITIONAL_OR_KEYWORD	não	0.0
ultimo_disparo	float | None	POSITIONAL_OR_KEYWORD	não	None
motivo_ultima_supressao	str | None	POSITIONAL_OR_KEYWORD	não	None
ultima_condicao	bool | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

None

Atributo `nome`: `str`.

Atributo `condicao`: `Condicao`.

Atributo `acao`: `Acao`.

Atributo `modo`: `str`.

Atributo `habilitada`: `bool`.

Atributo `ativa`: `bool`.

Atributo `execucoes`: `int`.

Atributo `prioridade`: `str | int`.

Atributo `max_execucoes`: `int | None`.

Atributo `cooldown`: `float`.

Atributo `ultimo_disparo`: `float | None`.

Atributo `motivo_ultima_supressao`: `str | None`.

Atributo `ultima_condicao`: `bool | None`.

Métodos públicos

Referência para leitura

```
avaliar(self, contexto: 'Any', *, instante: 'float | None' = None) -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.regras.SequenciaDoisPassos`

SequenciaDoisPassos(eventos: 'Eventos', primeiro: 'str', segundo: 'str', acao: 'AcaoSequencia', janela: 'float | None' = None, nome: 'str' = 'sequencia')

Assinatura

Assinatura para consulta

```
SequenciaDoisPassos(eventos: 'Eventos', primeiro: 'str', segundo: 'str', acao:
  'AcaoSequencia', janela: 'float | None' = None, nome: 'str' = 'sequencia') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
eventos	Eventos	POSITIONAL_OR_KEYWORD	sim	sem padrão
primeiro	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
segundo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
acao	AcaoSequencia	POSITIONAL_OR_KEYWORD	sim	sem padrão
janela	float | None	POSITIONAL_OR_KEYWORD	não	None

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	não	'sequencia'

Retorno declarado

None

Atributo `eventos`: `Eventos`.

Atributo `primeiro`: `str`.

Atributo `segundo`: `str`.

Atributo `acao`: `AcaoSequencia`.

Atributo `janela`: `float | None`.

Atributo `nome`: `str`.

Métodos públicos

Referência para leitura

```
resetar(self) -> 'None'
```

Método público sem descrição própria no runtime.

`coral.regras.Temporizador`

Temporizador(intervalo: 'float', acao: 'Acao', repetir: 'bool' = False, nome: 'str' = 'temporizador', restante: 'float | None' = None, ativo: 'bool' = True)

Assinatura

Assinatura para consulta

```
Temporizador(intervalo: 'float', acao: 'Acao', repetir: 'bool' = False, nome: 'str' =
  'temporizador', restante: 'float | None' = None, ativo: 'bool' = True) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
intervalo	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
acao	Acao	POSITIONAL_OR_KEYWORD	sim	sem padrão
repetir	bool	POSITIONAL_OR_KEYWORD	não	False
nome	str	POSITIONAL_OR_KEYWORD	não	'temporizador'

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
restante	float | None	POSITIONAL_OR_KEYWORD	não	None
ativo	bool	POSITIONAL_OR_KEYWORD	não	True

Retorno declarado

None

Atributo `intervalo`: `float`.

Atributo `acao`: `Acao`.

Atributo `repetir`: `bool`.

Atributo `nome`: `str`.

Atributo `restante`: `float | None`.

Atributo `ativo`: `bool`.

Métodos públicos

Referência para leitura

```
atualizar(self, contexto: 'Any', dt: 'float') -> 'int'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.regras.regra`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
regra(nome: 'str', quando: 'Condicao', entao: 'Acao', *, modo: 'str' = 'ao_ativar',
  prioridade: 'str | int' = 'normal', max_execucoes: 'int | None' = None, cooldown:
  'float' = 0.0) -> 'Regra'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
quando	Condicao	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
entao	Acao	POSITIONAL_OR_KEYWORD	sim	sem padrão
modo	str	KEYWORD_ONLY	não	'ao_ativar'
prioridade	str | int	KEYWORD_ONLY	não	'normal'
max_execucoes	int | None	KEYWORD_ONLY	não	None
cooldown	float	KEYWORD_ONLY	não	0.0

Retorno declarado

Regra

Consultar a implementação canônica de coral.regras

Implementação canônica de `coral.regras`

Código Python

```
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Callable

from .tempo_eventos import Eventos


LeitorValor = Callable[[Any], Any]
AcaoMudanca = Callable[[Any, Any, Any], Any]

_NAO_LIDO = object()


@dataclass
class ObservadorMudanca:
    nome: str
    leitor: LeitorValor
    acao: AcaoMudanca
    modo: str = "mudanca"
    limite: Any = None
    _anterior: Any = field(default=_NAO_LIDO, init=False, repr=False)

    def __post_init__(self) -> None:

        self.nome = str(self.nome)
        if self.modo not in {"mudanca", "passar_de", "cair_abaixo_de"}:
            raise ValueError(f"modo de observação desconhecido: {self.modo}")

    @classmethod
    def passar_de(
        cls,
        nome: str,
        leitor: LeitorValor,
```

```
        limite: Any,
        acao: AcaoMudanca,
    ) -> "ObservadorMudanca":
        return cls(nome, leitor, acao, modo="passar_de", limite=limite)

    @classmethod
    def cair_abaixo_de(
        cls,
        nome: str,
        leitor: LeitorValor,
        limite: Any,
        acao: AcaoMudanca,
    ) -> "ObservadorMudanca":
        return cls(nome, leitor, acao, modo="cair_abaixo_de", limite=limite)

    @property
    def inicializado(self) -> bool:
        return self._anterior is not _NAO_LIDO

    @property
    def valor_anterior(self) -> Any:
        return None if self._anterior is _NAO_LIDO else self._anterior

    def avaliar(self, contexto: Any) -> bool:
        novo = self.leitor(contexto)
        if self._anterior is _NAO_LIDO:
            self._anterior = novo
            return False

        anterior = self._anterior
        self._anterior = novo
        if self.modo == "mudanca":
            disparar = novo != anterior
        elif self.modo == "passar_de":
            disparar = anterior <= self.limite < novo
        else:
            disparar = anterior >= self.limite > novo

        if not disparar:
            return False
        self.acao(anterior, novo, contexto)
        return True


def _resolver_referencia(valor, contexto):
    return valor(contexto) if callable(valor) else valor


@dataclass
class ObservadorEspacial:
    nome: str
    leitor: LeitorValor
    referencia: Any
    acao: Callable[[Any, Any, Any], Any]
    relacao: str
    transicao: str = "entrar"
    limite: Any = None
    obter: Callable[[Any], Any] | None = None
    obter_referencia: Callable[[Any], Any] | None = None
    _anterior: bool | None = field(default=None, init=False, repr=False)
```

```
    @classmethod
    def entrar_em_regiao(cls, nome, leitor, regiao_alvo, acao, *, obter=None,
      obter_referencia=None):
        return cls(nome, leitor, regiao_alvo, acao, "dentro", "entrar", obter=obter,
          obter_referencia=obter_referencia)

    @classmethod
    def sair_de_regiao(cls, nome, leitor, regiao_alvo, acao, *, obter=None,
      obter_referencia=None):
        return cls(nome, leitor, regiao_alvo, acao, "dentro", "sair", obter=obter,
          obter_referencia=obter_referencia)

    @classmethod
    def chegar_a_menos_de(cls, nome, leitor, referencia, limite, acao, *, obter=None,
      obter_referencia=None):
        return cls(nome, leitor, referencia, acao, "perto", "entrar", limite=limite,
          obter=obter, obter_referencia=obter_referencia)

    @classmethod
    def tocar(cls, nome, leitor, referencia, acao, *, obter=None,
      obter_referencia=None):
        return cls(nome, leitor, referencia, acao, "toca", "entrar", obter=obter,
          obter_referencia=obter_referencia)

    @classmethod
    def intersectar(cls, nome, leitor, referencia, acao, *, obter=None,
      obter_referencia=None):
        return cls(nome, leitor, referencia, acao, "intersecta", "entrar", obter=obter,
          obter_referencia=obter_referencia)

    def _estado_atual(self, contexto: Any) -> tuple[bool, Any, Any]:
        from ._espacial import esta_dentro, esta_perto, intersecta, resolver_espacial,
          toca

        alvo_bruto = self.leitor(contexto)
        referencia_bruta = _resolver_referencia(self.referencia, contexto)
        alvo = resolver_espacial(alvo_bruto, obter=self.obter)
        referencia = resolver_espacial(referencia_bruta, obter=self.obter_referencia)
        if self.relacao == "dentro":
            atual = esta_dentro(alvo, referencia)
        elif self.relacao == "perto":
            atual = esta_perto(alvo, referencia, self.limite)
        elif self.relacao == "toca":
            atual = toca(alvo, referencia)
        elif self.relacao == "intersecta":
            atual = intersecta(alvo, referencia)
        else:
            raise ValueError(f"relação espacial desconhecida: {self.relacao}")
        return bool(atual), alvo, referencia

    def avaliar(self, contexto: Any) -> bool:
        atual, alvo, referencia = self._estado_atual(contexto)
        anterior = self._anterior
        self._anterior = atual
        if anterior is None:
            return False
        disparar = (not anterior and atual) if self.transicao == "entrar" else (anterior
           and not atual)
        if not disparar:
            return False
        self.acao(alvo, referencia, contexto)
```

```
        return True


class EstadoObservavel:
    def __init__(
        self,
        nome: str,
        valor_inicial: Any,
        *,
        eventos: Eventos | None = None,
        enfileirar: bool = False,
    ) -> None:
        self.nome = str(nome)
        self._valor = valor_inicial
        self.eventos = eventos if eventos is not None else Eventos()
        self.enfileirar = bool(enfileirar)

    @property
    def valor(self) -> Any:
        return self._valor

    def mudar(self, novo: Any) -> bool:
        anterior = self._valor
        if novo == anterior:
            return False
        self._valor = novo
        dados = {
            "estado": self.nome,
            "anterior": anterior,
            "novo": novo,
        }
        if self.enfileirar:
            self.eventos.enfileirar_evento("estado_mudou", origem=self.nome, **dados)
        else:
            self.eventos.emitir_evento("estado_mudou", origem=self.nome, **dados)
        return True

    def __eq__(self, outro: object) -> bool:
        if isinstance(outro, EstadoObservavel):
            return self._valor == outro._valor
        return self._valor == outro

    def __repr__(self) -> str:
        return f"EstadoObservavel({self.nome!r}, {self._valor!r})"


__all__ = [
    "LeitorValor",
    "AcaoMudanca",
    "ObservadorMudanca",
    "ObservadorEspacial",
    "EstadoObservavel",
]

"""Rastreamento estruturado e proteção do ciclo reativo da Coral 1.5.6."""
from __future__ import annotations

from dataclasses import asdict, dataclass, field
from typing import Any, Mapping

from .comum import ErroOperacaoCoral
```

```
MODOS_RASTREAMENTO = frozenset({"desligado", "resumido", "detalhado"})


class ErroCicloReativo(ErroOperacaoCoral):
    """O ciclo reativo excedeu o limite seguro configurado."""


@dataclass(frozen=True)
class EntradaRastreamentoReativo:
    sequencia: int | None
    instante: float | None
    tipo: str
    nome: str
    decisao: str
    causa_sequencia: int | None = None
    prioridade: int | None = None
    condicao: str | None = None
    resultado_condicao: bool | None = None
    detalhes: Mapping[str, Any] = field(default_factory=dict)

    def como_dict(self) -> dict[str, Any]:
        dados = asdict(self)
        dados["detalhes"] = dict(self.detalhes)
        return dados


class RastreamentoReativo:
    def __init__(self, modo: str = "desligado"):
        nome = str(modo).strip().casefold()
        if nome not in MODOS_RASTREAMENTO:
            raise ValueError("modo de rastreamento deve ser desligado, resumido ou
              detalhado")
        self.modo = nome
        self._entradas: list[EntradaRastreamentoReativo] = []

    @property
    def historico(self) -> tuple[EntradaRastreamentoReativo, ...]:
        return tuple(self._entradas)

    def limpar(self) -> None:
        self._entradas.clear()

    def registrar(
        self,
        *,
        sequencia: int | None,
        instante: float | None,
        tipo: str,
        nome: str,
        decisao: str,
        causa_sequencia: int | None = None,
        prioridade: int | None = None,
        condicao: str | None = None,
        resultado_condicao: bool | None = None,
        detalhes: Mapping[str, Any] | None = None,
    ) -> EntradaRastreamentoReativo | None:
        if self.modo == "desligado":
            return None
        entrada = EntradaRastreamentoReativo(
            sequencia=sequencia,
```

```
            instante=instante,
            tipo=str(tipo),
            nome=str(nome),
            decisao=str(decisao),
            causa_sequencia=causa_sequencia,
            prioridade=prioridade,
            condicao=condicao if self.modo == "detalhado" else None,
            resultado_condicao=resultado_condicao if self.modo == "detalhado" else None,
            detalhes=dict(detalhes or {}) if self.modo == "detalhado" else {},
        )
        self._entradas.append(entrada)
        return entrada

    def como_dados(self) -> list[dict[str, Any]]:
        return [entrada.como_dict() for entrada in self._entradas]

    def resumo_texto(self) -> str:
        if not self._entradas:
            return "Nenhum evento ou regra foi registrado no histórico reativo."
        linhas: list[str] = []
        for entrada in self._entradas:
            if entrada.tipo == "regra":
                if entrada.decisao == "executada":
                    texto = f'Regra "{entrada.nome}": executada'
                else:
                    texto = f'Regra "{entrada.nome}": não executada ({entrada.decisao})'
                if entrada.resultado_condicao is not None:
                    texto += "; condição=" + ("verdadeiro" if entrada.resultado_condicao
                       else "falso")
                linhas.append(texto)
                continue
            texto = f"{entrada.tipo.capitalize()} {entrada.nome}: {entrada.decisao}"
            if entrada.sequencia is not None:
                texto += f" [sequência {entrada.sequencia}]"
            linhas.append(texto)
        return "\n".join(linhas)


__all__ = [
    "MODOS_RASTREAMENTO",
    "ErroCicloReativo",
    "EntradaRastreamentoReativo",
    "RastreamentoReativo",
]

from __future__ import annotations

from dataclasses import dataclass
import math
from typing import Any, Callable

from .tempo_eventos import Evento, Eventos, FonteTempo, normalizar_dt, validar_duracao
from .rastreamento_reativo import ErroCicloReativo, RastreamentoReativo
from .observadores_reativos import ObservadorEspacial, ObservadorMudanca
from .sequencias_reativas import SequenciaDoisPassos


Condicao = Callable[[Any], bool]
Acao = Callable[[Any], Any]
MODOS_REGRA = frozenset({"ao_ativar", "sempre", "uma_vez"})
```

```
_PRIORIDADES_REGRA = {
    "baixa": 0,
    "normal": 100,
    "alta": 200,
    "critica": 300,
    "crítica": 300,
}


def _normalizar_prioridade_regra(prioridade: str | int) -> int:
    if isinstance(prioridade, bool):
        raise ValueError("prioridade de regra desconhecida: use baixa, normal, alta ou
          crítica")
    if isinstance(prioridade, int):
        return prioridade
    nome = str(prioridade).strip().casefold()
    if nome not in _PRIORIDADES_REGRA:
        raise ValueError(
            f"prioridade de regra desconhecida: {prioridade!r}; use baixa, normal, alta
              ou crítica"
        )
    return _PRIORIDADES_REGRA[nome]


@dataclass
class Regra:
    nome: str
    condicao: Condicao
    acao: Acao
    modo: str = "ao_ativar"
    habilitada: bool = True
    ativa: bool = False
    execucoes: int = 0
    prioridade: str | int = "normal"
    max_execucoes: int | None = None
    cooldown: float = 0.0
    ultimo_disparo: float | None = None
    motivo_ultima_supressao: str | None = None
    ultima_condicao: bool | None = None

    def __post_init__(self):
        self.prioridade = _normalizar_prioridade_regra(self.prioridade)
        if self.max_execucoes is not None:
            if (
                isinstance(self.max_execucoes, bool)
                or not isinstance(self.max_execucoes, int)
                or self.max_execucoes <= 0
            ):
                raise ValueError("máximo de execuções deve ser um inteiro positivo ou
                  nulo")
        try:
            self.cooldown = float(validar_duracao(self.cooldown, "cooldown"))
        except ValueError as exc:
            raise ValueError("cooldown deve ser um número finito maior ou igual a zero")
               from exc

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "nome": self.nome,
            "habilitada": self.habilitada,
```

```
            "ativa": self.ativa,
            "modo": self.modo,
            "prioridade": self.prioridade,
            "execucoes": self.execucoes,
            "max_execucoes": self.max_execucoes,
            "cooldown": self.cooldown,
            "ultimo_disparo": self.ultimo_disparo,
            "ultima_condicao": self.ultima_condicao,
            "motivo_ultima_supressao": self.motivo_ultima_supressao,
        }

    def avaliar(self, contexto: Any, *, instante: float | None = None) -> bool:
        self.motivo_ultima_supressao = None
        if not self.habilitada:
            self.motivo_ultima_supressao = "desabilitada"
            return False
        agora = bool(self.condicao(contexto))
        self.ultima_condicao = agora
        executar = False
        if self.modo == "sempre":
            executar = agora
        elif self.modo == "uma_vez":
            executar = agora and self.execucoes == 0
        elif self.modo == "ao_ativar":
            executar = agora and not self.ativa
        else:
            raise ValueError(f"modo de regra desconhecido: {self.modo}")
        self.ativa = agora
        if not executar:
            self.motivo_ultima_supressao = "condicao_inativa" if not agora else "modo"
            return False
        if self.max_execucoes is not None and self.execucoes >= self.max_execucoes:
            self.motivo_ultima_supressao = "limite_execucoes"
            return False
        if self.cooldown > 0 and self.ultimo_disparo is not None:
            if instante is None:
                raise ValueError("regra com cooldown exige instante lógico")
            decorrido = float(instante) - self.ultimo_disparo
            if decorrido < self.cooldown and not math.isclose(decorrido, self.cooldown,
              rel_tol=1e-12, abs_tol=1e-15):
                self.motivo_ultima_supressao = "cooldown"
                return False
        self.acao(contexto)
        self.execucoes += 1
        if instante is not None:
            self.ultimo_disparo = float(instante)
        return True


@dataclass(frozen=True)
class GrupoRegras:
    nome: str
    regras: tuple[Regra, ...]

    def habilitar(self) -> None:
        for regra in self.regras:
            regra.habilitada = True

    def desabilitar(self) -> None:
        for regra in self.regras:
            regra.habilitada = False
```

```
@dataclass
class Debounce:
    intervalo: float
    acao: Acao
    nome: str = "debounce"
    restante: float | None = None
    pendente: bool = False
    contexto_mais_recente: Any = None

    def __post_init__(self):
        try:
            self.intervalo = float(validar_duracao(self.intervalo, "intervalo de
              debounce"))
        except ValueError as exc:
            raise ValueError("intervalo de debounce deve ser um número finito maior ou
              igual a zero") from exc

    def notificar(self, contexto: Any = None) -> None:
        self.contexto_mais_recente = contexto
        self.restante = self.intervalo
        self.pendente = True

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "nome": self.nome,
            "intervalo": self.intervalo,
            "restante": self.restante,
            "pendente": self.pendente,
        }

    def atualizar(self, dt: float) -> bool:
        if not self.pendente:
            return False
        passo = normalizar_dt(dt, compatibilidade_negativo_zero=True)
        restante = self.intervalo if self.restante is None else float(self.restante)
        if passo < restante and not math.isclose(passo, restante, rel_tol=1e-12,
          abs_tol=1e-15):
            self.restante = restante - passo
            return False
        contexto = self.contexto_mais_recente
        self.pendente = False
        self.restante = None
        self.acao(contexto)
        return True


@dataclass
class _AtivacaoRegraDurante:
    regra: Regra
    fim: float


@dataclass
class _AtivacaoRegraAte:
    regra: Regra
    condicao: Condicao


@dataclass
```

```
class Temporizador:
    intervalo: float
    acao: Acao
    repetir: bool = False
    nome: str = "temporizador"
    restante: float | None = None
    ativo: bool = True

    def __post_init__(self):
        try:
            self.intervalo = float(validar_duracao(self.intervalo, "intervalo",
              permitir_zero=not self.repetir))
        except ValueError as exc:
            # Preserva mensagens públicas da linha 1.3 para os contratos antigos.
            if self.repetir and "maior que zero" in str(exc):
                raise ValueError("intervalo repetido deve ser maior que zero") from exc
            if "finito" in str(exc) and not str(exc).endswith("maior que zero"):
                raise ValueError("intervalo deve ser um número finito") from exc
            raise ValueError("intervalo não pode ser negativo") from exc
        if self.restante is None:
            self.restante = self.intervalo
        else:
            try:
                self.restante = float(validar_duracao(self.restante, "tempo restante"))
            except ValueError as exc:
                raise ValueError("tempo restante deve ser um número finito") from exc

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "nome": self.nome,
            "intervalo": self.intervalo,
            "restante": self.restante,
            "repetir": self.repetir,
            "ativo": self.ativo,
        }

    def atualizar(self, contexto: Any, dt: float) -> int:
        if not self.ativo:
            return 0
        # Compatibilidade 1.2: avanços negativos não voltam o relógio do motor.
        passo = normalizar_dt(dt, compatibilidade_negativo_zero=True)
        restante_antes = float(self.restante)
        if passo < restante_antes:
            self.restante = restante_antes - passo
            return 0

        if not self.repetir:
            self.acao(contexto)
            self.restante = max(0.0, restante_antes - passo)
            self.ativo = False
            return 1

        # Calcula os intervalos vencidos de uma só vez. nextafter corrige o caso
        # 0.1 + 0.1 + 0.1, que na 1.3.6 podia perder o disparo do limite exato.
        excedente = passo - restante_antes
        quociente = excedente / self.intervalo
        adicionais = math.floor(math.nextafter(quociente, math.inf))
        previstos = 1 + max(0, adicionais)
        disparos = 0
        for _ in range(previstos):
            if not self.ativo:
```

```
                break
            self.acao(contexto)
            disparos += 1
        if self.ativo:
            self.restante = restante_antes - passo + disparos * self.intervalo
            # Evita expor resíduos negativos minúsculos depois de um limite exato.
            if math.isclose(self.restante, self.intervalo, rel_tol=1e-12, abs_tol=0.0):
                self.restante = self.intervalo
            elif self.restante < 0 and math.isclose(self.restante, 0.0, rel_tol=0.0,
              abs_tol=math.ulp(max(1.0, abs(passo))) * 4):
                self.restante = 0.0
        return disparos


class MotorRegras:
    def __init__(
        self,
        *,
        relogio: FonteTempo | None = None,
        eventos: Eventos | None = None,
        rastreamento: RastreamentoReativo | str | None = None,
    ):
        self.regras: list[Regra] = []
        self.temporizadores: list[Temporizador] = []
        self.debounces: list[Debounce] = []
        self.observadores: list[ObservadorMudanca | ObservadorEspacial] = []
        self.grupos: dict[str, GrupoRegras] = {}
        self.sequencias: list[SequenciaDoisPassos] = []
        self._ativacoes_durante: list[_AtivacaoRegraDurante] = []
        self._ativacoes_ate: list[_AtivacaoRegraAte] = []
        self.relogio = relogio
        self._tempo_logico = float(relogio.agora()) if relogio is not None else 0.0
        self.eventos = eventos if eventos is not None else Eventos(relogio=relogio)
        self.eventos.definir_fonte_instante(self._instante_atual)
        if rastreamento is None:
            self.rastreamento = RastreamentoReativo("desligado")
        elif isinstance(rastreamento, str):
            self.rastreamento = RastreamentoReativo(rastreamento)
        else:
            self.rastreamento = rastreamento

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "tempo_logico": self._tempo_logico,
            "regras": tuple(self.regras),
            "quantidade_regras": len(self.regras),
            "quantidade_temporizadores": len(self.temporizadores),
            "temporizadores": tuple(self.temporizadores),
            "quantidade_debounces": len(self.debounces),
            "debounces": tuple(self.debounces),
            "quantidade_observadores": len(self.observadores),
            "quantidade_sequencias": len(self.sequencias),
            "eventos_pendentes": self.eventos.pendentes,
            "rastreamento": {
                "modo": self.rastreamento.modo,
                "quantidade_entradas": len(self.rastreamento.historico),
            },
        }

    def adicionar(self, regra: Regra) -> Regra:
        if any(r.nome == regra.nome for r in self.regras):
```

```
            raise ValueError(f"já existe uma regra chamada {regra.nome!r}")
        self.regras.append(regra)
        return regra

    def regra(
        self,
        nome: str,
        condicao: Condicao,
        acao: Acao,
        *,
        modo: str = "ao_ativar",
        prioridade: str | int = "normal",
        max_execucoes: int | None = None,
        cooldown: float = 0.0,
    ) -> Regra:
        return self.adicionar(
            Regra(
                nome,
                condicao,
                acao,
                modo=modo,
                prioridade=prioridade,
                max_execucoes=max_execucoes,
                cooldown=cooldown,
            )
        )

    def obter_regra(self, nome: str) -> Regra:
        procurado = str(nome)
        for regra in self.regras:
            if regra.nome == procurado:
                return regra
        raise ValueError(f"regra inexistente: {procurado!r}")

    def habilitar_regra(self, nome: str) -> Regra:
        regra = self.obter_regra(nome)
        regra.habilitada = True
        return regra

    def desabilitar_regra(self, nome: str) -> Regra:
        regra = self.obter_regra(nome)
        regra.habilitada = False
        return regra

    def criar_grupo(self, nome: str, regras: list[str] | tuple[str, ...]) ->
      GrupoRegras:
        nome_grupo = str(nome)
        if nome_grupo in self.grupos:
            raise ValueError(f"já existe um grupo chamado {nome_grupo!r}")
        grupo = GrupoRegras(nome_grupo, tuple(self.obter_regra(item) for item in
          regras))
        self.grupos[nome_grupo] = grupo
        return grupo

    def obter_grupo(self, nome: str) -> GrupoRegras:
        procurado = str(nome)
        try:
            return self.grupos[procurado]
        except KeyError as exc:
            raise ValueError(f"grupo inexistente: {procurado!r}") from exc
```

```
    def habilitar_grupo(self, nome: str) -> GrupoRegras:
        grupo = self.obter_grupo(nome)
        grupo.habilitar()
        return grupo

    def desabilitar_grupo(self, nome: str) -> GrupoRegras:
        grupo = self.obter_grupo(nome)
        grupo.desabilitar()
        return grupo

    def observar_mudanca(self, nome: str, leitor, acao) -> ObservadorMudanca:
        item = ObservadorMudanca(nome, leitor, acao)
        self.observadores.append(item)
        return item

    def observar_passar_de(self, nome: str, leitor, limite, acao) -> ObservadorMudanca:
        item = ObservadorMudanca.passar_de(nome, leitor, limite, acao)
        self.observadores.append(item)
        return item

    def observar_cair_abaixo_de(self, nome: str, leitor, limite, acao) ->
      ObservadorMudanca:
        item = ObservadorMudanca.cair_abaixo_de(nome, leitor, limite, acao)
        self.observadores.append(item)
        return item

    def observar_entrada_regiao(self, nome: str, leitor, regiao_alvo, acao, **opcoes) ->
       ObservadorEspacial:
        item = ObservadorEspacial.entrar_em_regiao(nome, leitor, regiao_alvo, acao,
          **opcoes)
        self.observadores.append(item)
        return item

    def observar_saida_regiao(self, nome: str, leitor, regiao_alvo, acao, **opcoes) ->
      ObservadorEspacial:
        item = ObservadorEspacial.sair_de_regiao(nome, leitor, regiao_alvo, acao,
          **opcoes)
        self.observadores.append(item)
        return item

    def observar_proximidade(self, nome: str, leitor, referencia, limite, acao,
      **opcoes) -> ObservadorEspacial:
        item = ObservadorEspacial.chegar_a_menos_de(nome, leitor, referencia, limite,
          acao, **opcoes)
        self.observadores.append(item)
        return item

    def observar_toque(self, nome: str, leitor, referencia, acao, **opcoes) ->
      ObservadorEspacial:
        item = ObservadorEspacial.tocar(nome, leitor, referencia, acao, **opcoes)
        self.observadores.append(item)
        return item

    def observar_intersecao(self, nome: str, leitor, referencia, acao, **opcoes) ->
      ObservadorEspacial:
        item = ObservadorEspacial.intersectar(nome, leitor, referencia, acao, **opcoes)
        self.observadores.append(item)
        return item

    def observar_regioes_mapa(
        self,
```

```
        nome: str,
        alvo: Any,
        mapa,
        regioes,
        resolvedor,
        *,
        identificador_alvo: Any = None,
    ):
        """Observa transições discretas usando o mesmo barramento do motor."""
        from ._mapas.observacao import ObservadorRegiaoMapa

        item = ObservadorRegiaoMapa(
            nome,
            alvo,
            mapa,
            regioes,
            resolvedor,
            self.eventos,
            identificador_alvo=identificador_alvo,
        )
        self.observadores.append(item)
        return item

    def sequencia(
        self,
        nome: str,
        primeiro: str,
        segundo: str,
        acao,
        *,
        janela: float | None = None,
    ) -> SequenciaDoisPassos:
        item = SequenciaDoisPassos(
            self.eventos,
            primeiro,
            segundo,
            acao,
            janela=janela,
            nome=nome,
        )
        self.sequencias.append(item)
        return item

    def debounce(self, segundos: float, acao: Acao, *, nome: str = "debounce") ->
      Debounce:
        item = Debounce(segundos, acao, nome=nome)
        self.debounces.append(item)
        return item

    def ativar_regra_durante(self, nome: str, segundos: float) -> Regra:
        duracao = float(validar_duracao(segundos, "duração da regra temporária"))
        regra = self.obter_regra(nome)
        regra.habilitada = True
        self._ativacoes_durante = [a for a in self._ativacoes_durante if a.regra is not
          regra]
        self._ativacoes_durante.append(_AtivacaoRegraDurante(regra,
          self._instante_atual() + duracao))
        return regra

    def ativar_regra_ate(self, nome: str, condicao: Condicao) -> Regra:
        regra = self.obter_regra(nome)
```

```
        regra.habilitada = True
        self._ativacoes_ate = [a for a in self._ativacoes_ate if a.regra is not regra]
        self._ativacoes_ate.append(_AtivacaoRegraAte(regra, condicao))
        return regra

    def _atualizar_ativacoes(self, contexto: Any) -> None:
        instante = self._instante_atual()
        durante_ativas: list[_AtivacaoRegraDurante] = []
        for ativacao in self._ativacoes_durante:
            if instante > ativacao.fim or math.isclose(instante, ativacao.fim,
              rel_tol=1e-12, abs_tol=1e-15):
                ativacao.regra.habilitada = False
            else:
                durante_ativas.append(ativacao)
        self._ativacoes_durante = durante_ativas

        ate_ativas: list[_AtivacaoRegraAte] = []
        for ativacao in self._ativacoes_ate:
            if bool(ativacao.condicao(contexto)):
                ativacao.regra.habilitada = False
            else:
                ate_ativas.append(ativacao)
        self._ativacoes_ate = ate_ativas

    def depois_de(self, segundos: float, acao: Acao, *, nome: str = "depois") ->
      Temporizador:
        t = Temporizador(segundos, acao, False, nome)
        self.temporizadores.append(t)
        return t

    def a_cada(self, segundos: float, acao: Acao, *, nome: str = "intervalo") ->
      Temporizador:
        t = Temporizador(segundos, acao, True, nome)
        self.temporizadores.append(t)
        return t

    def _instante_atual(self) -> float:
        if self.relogio is not None:
            return float(self.relogio.agora())
        return self._tempo_logico

    def instante_atual(self) -> float:
        return self._instante_atual()

    def avaliar(self, contexto: Any) -> tuple[str, ...]:
        self._atualizar_ativacoes(contexto)
        executadas = []
        instante = self._instante_atual()
        ordenadas = sorted(
            enumerate(tuple(self.regras)),
            key=lambda item: (-int(item[1].prioridade), item[0]),
        )
        for _ordem, r in ordenadas:
            executou = r.avaliar(contexto, instante=instante)
            if executou:
                executadas.append(r.nome)
            self.rastreamento.registrar(
                sequencia=None,
                instante=instante,
                tipo="regra",
                nome=r.nome,
```

```
                decisao="executada" if executou else (r.motivo_ultima_supressao or
                  "nao_executada"),
                prioridade=int(r.prioridade),
                resultado_condicao=r.ultima_condicao,
                detalhes={"execucoes": r.execucoes, "modo": r.modo},
            )
        for observador in tuple(self.observadores):
            if observador.avaliar(contexto):
                executadas.append(observador.nome)
        return tuple(executadas)

    def atualizar(self, contexto: Any, dt: float | None = None) -> tuple[str, ...]:
        if dt is None:
            if self.relogio is None:
                raise ValueError("informe dt ou configure um relógio no MotorRegras")
            dt = self.relogio.passo()
        passo = normalizar_dt(dt, compatibilidade_negativo_zero=True)
        if self.relogio is None:
            self._tempo_logico += passo
        executadas = list(self.avaliar(contexto))
        for t in tuple(self.temporizadores):
            if t.atualizar(contexto, passo):
                executadas.append(t.nome)
        self.temporizadores[:] = [t for t in self.temporizadores if t.ativo]
        for debounce in tuple(self.debounces):
            if debounce.atualizar(passo):
                executadas.append(debounce.nome)
        return tuple(executadas)


    def processar_ciclo(self, contexto: Any = None, *, limite_reacoes: int = 1000) ->
      tuple[Evento, ...]:
        if isinstance(limite_reacoes, bool) or not isinstance(limite_reacoes, int) or
          limite_reacoes <= 0:
            raise ValueError("limite de reações deve ser um inteiro positivo")
        processados: list[Evento] = []
        nomes_recentes: list[str] = []
        while self.eventos.pendentes:
            if len(processados) >= limite_reacoes:
                cadeia = " → ".join(f'"{nome}"' for nome in nomes_recentes[-6:])
                self.rastreamento.registrar(
                    sequencia=processados[-1].sequencia if processados else None,
                    instante=processados[-1].instante if processados else None,
                    tipo="ciclo",
                    nome="ciclo_reativo",
                    decisao="interrompido",
                    detalhes={"limite": limite_reacoes, "cadeia_recente":
                      tuple(nomes_recentes[-6:])},
                )
                detalhe = f" Cadeia recente: {cadeia}." if cadeia else ""
                raise ErroCicloReativo(
                    f"Ciclo reativo interrompido após {limite_reacoes}
                      reações.{detalhe}"
                )
            evento = self.eventos.processar(1)[0]
            processados.append(evento)
            nomes_recentes.append(evento.nome)
            self.rastreamento.registrar(
                sequencia=evento.sequencia,
                instante=evento.instante,
                tipo="evento",
```

```
                nome=evento.nome,
                decisao="processado",
                causa_sequencia=evento.causa_sequencia,
                detalhes={"origem": evento.origem, "cancelado": evento.cancelado,
                  "consumido": evento.consumido},
            )
        return tuple(processados)

    def emitir_evento(self, nome: str, *args, **kwargs) -> int:
        """Emite um evento de regra pelo barramento compartilhado, sem conhecer
          consumidores."""
        return self.eventos.emitir_propagado(str(nome), *args, **kwargs)

    def atualizar_por_relogio(self, contexto: Any) -> tuple[str, ...]:
        return self.atualizar(contexto, None)


def regra(
    nome: str,
    quando: Condicao,
    entao: Acao,
    *,
    modo: str = "ao_ativar",
    prioridade: str | int = "normal",
    max_execucoes: int | None = None,
    cooldown: float = 0.0,
) -> Regra:
    return Regra(
        nome,
        quando,
        entao,
        modo=modo,
        prioridade=prioridade,
        max_execucoes=max_execucoes,
        cooldown=cooldown,
    )


__all__ = [
    "Condicao", "Acao", "MODOS_REGRA", "Regra", "GrupoRegras", "Debounce",
      "Temporizador",
    "Eventos", "MotorRegras", "ObservadorMudanca", "ObservadorEspacial",
      "SequenciaDoisPassos", "regra",
]

from __future__ import annotations

from dataclasses import dataclass, field
import math
from typing import Any, Callable

from .tempo_eventos import Evento, Eventos, validar_duracao


AcaoSequencia = Callable[[Evento, Evento], Any]


@dataclass
class SequenciaDoisPassos:
    eventos: Eventos
    primeiro: str
```

```
    segundo: str
    acao: AcaoSequencia
    janela: float | None = None
    nome: str = "sequencia"
    _evento_primeiro: Evento | None = field(default=None, init=False, repr=False)

    def __post_init__(self) -> None:
        self.primeiro = str(self.primeiro)
        self.segundo = str(self.segundo)
        self.nome = str(self.nome)
        if self.janela is not None:
            self.janela = float(validar_duracao(self.janela, "janela da sequência"))
            if not self.eventos.possui_fonte_instante:
                raise ValueError("sequência com janela temporal exige fonte de tempo")
        self.eventos.quando(self.primeiro, self._receber_primeiro, receber_evento=True)
        self.eventos.quando(self.segundo, self._receber_segundo, receber_evento=True)

    def _receber_primeiro(self, evento: Evento) -> None:
        self._evento_primeiro = evento

    def _dentro_da_janela(self, primeiro: Evento, segundo: Evento) -> bool:
        if self.janela is None:
            return True
        if primeiro.instante is None or segundo.instante is None:
            return False
        decorrido = segundo.instante - primeiro.instante
        return decorrido < self.janela or math.isclose(
            decorrido,
            self.janela,
            rel_tol=1e-12,
            abs_tol=1e-15,
        )

    def _receber_segundo(self, evento: Evento) -> None:
        primeiro = self._evento_primeiro
        if primeiro is None:
            return
        self._evento_primeiro = None
        if self._dentro_da_janela(primeiro, evento):
            self.acao(primeiro, evento)

    def resetar(self) -> None:
        self._evento_primeiro = None


__all__ = ["AcaoSequencia", "SequenciaDoisPassos"]

"""Tempo, eventos e cancelamento compartilhados do runtime Coral.

Este módulo materializa a segunda parte da coesão 1.5 sem substituir as APIs
históricas de ``coral.regras``. Ele fornece fontes de tempo injetáveis,
validação uniforme de durações, eventos síncronos/enfileirados/propagados e um
token cooperativo de cancelamento reutilizável por bibliotecas e ferramentas.
"""
from __future__ import annotations

from collections import deque
from dataclasses import dataclass, field
import math
import time
from typing import Any, Callable, Mapping, Protocol, runtime_checkable
```

```
from .comum import ErroOperacaoCoral

CONTRATO = "coral.tempo_eventos/1"


def validar_duracao(valor: Any, nome: str = "duração", *, permitir_none: bool = False,
                    permitir_zero: bool = True) -> float | None:
    """Normaliza uma duração pública para segundos finitos não negativos."""
    if valor is None and permitir_none:
        return None
    if isinstance(valor, bool):
        raise ValueError(f"{nome} deve ser um número finito maior ou igual a zero")
    try:
        numero = float(valor)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ValueError(f"{nome} deve ser um número finito maior ou igual a zero") from
           exc
    if not math.isfinite(numero) or numero < 0 or (numero == 0 and not permitir_zero):
        operador = "maior que zero" if not permitir_zero else "maior ou igual a zero"
        raise ValueError(f"{nome} deve ser um número finito {operador}")
    return numero


def normalizar_dt(valor: Any, *, compatibilidade_negativo_zero: bool = True) -> float:
    """Normaliza avanço de simulação.

    A linha 1.2 já definia que ``dt`` negativo não volta temporizadores. Por
    compatibilidade, a política padrão converte valores negativos finitos em
    zero; novos consumidores podem optar por rejeitá-los.
    """
    if isinstance(valor, bool):
        raise ValueError("avanço de tempo deve ser um número finito")
    try:
        numero = float(valor)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ValueError("avanço de tempo deve ser um número finito") from exc
    if not math.isfinite(numero):
        raise ValueError("avanço de tempo deve ser um número finito")
    if numero < 0:
        if compatibilidade_negativo_zero:
            return 0.0
        raise ValueError("avanço de tempo não pode ser negativo")
    return numero


@runtime_checkable
class FonteTempo(Protocol):
    def agora(self) -> float: ...
    def passo(self) -> float: ...


class RelogioMonotonico:
    """Relógio real baseado em ``perf_counter`` e apropriado para durações."""
    def __init__(self, fonte: Callable[[], float] = time.perf_counter):
        self._fonte = fonte
        self._ultimo = float(fonte())

    def agora(self) -> float:
        return float(self._fonte())
```

```
    def passo(self) -> float:
        atual = self.agora()
        dt = normalizar_dt(atual - self._ultimo)
        self._ultimo = atual
        return dt


class RelogioSimulado:
    """Relógio controlado pelo programa, sem dormir nem consultar o sistema."""
    def __init__(self, inicial: float = 0.0):
        self._agora = float(validar_duracao(inicial, "tempo inicial"))
        self._ultimo = self._agora

    def agora(self) -> float:
        return self._agora

    def avancar(self, dt: Any) -> float:
        self._agora += normalizar_dt(dt, compatibilidade_negativo_zero=False)
        return self._agora

    def passo(self) -> float:
        dt = normalizar_dt(self._agora - self._ultimo,
          compatibilidade_negativo_zero=False)
        self._ultimo = self._agora
        return dt

    def definir(self, instante: Any) -> float:
        novo = float(validar_duracao(instante, "instante"))
        if novo < self._agora:
            raise ValueError("o relógio simulado não pode retroceder")
        self._agora = novo
        return novo


class OperacaoCancelada(ErroOperacaoCoral):
    """Operação cooperativa interrompida antes de uma fase segura."""


class TokenCancelamento:
    def __init__(self):
        self._cancelado = False
        self._motivo: str | None = None

    @property
    def cancelado(self) -> bool:
        return self._cancelado

    @property
    def motivo(self) -> str | None:
        return self._motivo

    def cancelar(self, motivo: str = "Operação cancelada pelo usuário.") -> None:
        self._cancelado = True
        self._motivo = str(motivo)

    def verificar(self) -> None:
        if self._cancelado:
            raise OperacaoCancelada(self._motivo or "Operação cancelada.")


@dataclass
```

```
class Evento:
    nome: str
    args: tuple[Any, ...] = ()
    kwargs: Mapping[str, Any] = field(default_factory=dict)
    origem: str | None = None
    sequencia: int = 0
    instante: float | None = None
    cancelavel: bool = False
    cancelado: bool = False
    motivo_cancelamento: str | None = None
    consumido: bool = False
    metadados: Mapping[str, Any] = field(default_factory=dict)
    causa_sequencia: int | None = None

    def __post_init__(self):
        self.nome = str(self.nome)
        self.args = tuple(self.args)
        self.kwargs = dict(self.kwargs)
        self.origem = None if self.origem is None else str(self.origem)
        self.metadados = dict(self.metadados)

    def __getattr__(self, nome: str) -> Any:
        payload = self.__dict__.get("kwargs", {})
        if nome in payload:
            return payload[nome]
        raise AttributeError(nome)

    def cancelar(self, motivo: str | None = None) -> None:
        if not self.cancelavel:
            raise ValueError("este evento não é cancelável")
        self.cancelado = True
        self.motivo_cancelamento = None if motivo is None else str(motivo)

    def consumir(self) -> None:
        self.consumido = True

    def estado_inspecao(self) -> dict[str, Any]:
        """Estado observável do evento sem executar ouvintes nem alterar a fila."""
        return {
            "nome": self.nome,
            "args": self.args,
            "kwargs": dict(self.kwargs),
            "origem": self.origem,
            "sequencia": self.sequencia,
            "instante": self.instante,
            "cancelavel": self.cancelavel,
            "cancelado": self.cancelado,
            "motivo_cancelamento": self.motivo_cancelamento,
            "consumido": self.consumido,
            "metadados": dict(self.metadados),
            "causa_sequencia": self.causa_sequencia,
        }


@dataclass
class _InscricaoEvento:
    funcao: Callable[..., Any]
    prioridade: int
    ordem: int
    receber_evento: bool = False
    max_execucoes: int | None = None
```

```
    cooldown: float = 0.0
    execucoes: int = 0
    ultimo_disparo: float | None = None


_PRIORIDADES_EVENTO = {
    "baixa": 0,
    "normal": 100,
    "alta": 200,
    "critica": 300,
    "crítica": 300,
}


def _normalizar_prioridade_evento(prioridade: str | int) -> int:
    if isinstance(prioridade, bool):
        raise ValueError("prioridade de evento desconhecida: use baixa, normal, alta ou
          crítica")
    if isinstance(prioridade, int):
        return prioridade
    nome = str(prioridade).strip().casefold()
    if nome not in _PRIORIDADES_EVENTO:
        raise ValueError(
            f"prioridade de evento desconhecida: {prioridade!r}; use baixa, normal, alta
               ou crítica"
        )
    return _PRIORIDADES_EVENTO[nome]


class Eventos:
    """Barramento local compatível com ``coral.regras.Eventos``.

    ``emitir`` mantém a chamada histórica de ouvintes via ``*args`` e
    ``**kwargs``. Inscrições novas podem receber o objeto :class:`Evento`
    completo e declarar prioridade sem alterar consumidores antigos.
    """
    def __init__(
        self,
        *,
        pai: "Eventos | None" = None,
        relogio: FonteTempo | None = None,
        fonte_instante: Callable[[], float] | None = None,
    ):
        self._ouvintes: dict[str, list[_InscricaoEvento]] = {}
        self._fila: deque[Evento] = deque()
        self._pai = pai
        self._relogio = relogio
        self._fonte_instante = fonte_instante
        self._sequencia = 0
        self._ordem_inscricao = 0
        self._pilha_processamento: list[Evento] = []

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "pendentes": len(self._fila),
            "fila": tuple(self._fila),
            "tipos_ouvintes": tuple(sorted(self._ouvintes)),
            "quantidade_ouvintes": sum(len(itens) for itens in self._ouvintes.values()),
            "ouvintes_por_tipo": {
                nome: len(itens)
                for nome, itens in sorted(self._ouvintes.items())
```

```
            },
            "sequencia": self._sequencia,
            "possui_pai": self._pai is not None,
            "possui_fonte_instante": self.possui_fonte_instante,
        }

    @property
    def pendentes(self) -> int:
        return len(self._fila)

    @property
    def pai(self) -> "Eventos | None":
        return self._pai

    @property
    def relogio(self) -> FonteTempo | None:
        return self._relogio

    def definir_fonte_instante(self, fonte: Callable[[], float] | None) -> None:
        self._fonte_instante = fonte

    def _instante_atual(self) -> float | None:
        if self._relogio is not None:
            return float(self._relogio.agora())
        if self._fonte_instante is not None:
            return float(self._fonte_instante())
        return None

    @property
    def possui_fonte_instante(self) -> bool:
        return self._relogio is not None or self._fonte_instante is not None

    def definir_pai(self, pai: "Eventos | None") -> None:
        if pai is self:
            raise ValueError("um barramento de eventos não pode propagar para si mesmo")
        atual = pai
        while atual is not None:
            if atual is self:
                raise ValueError("ciclo de propagação de eventos não permitido")
            atual = atual.pai
        self._pai = pai

    def quando(
        self,
        nome: str,
        funcao: Callable[..., Any],
        *,
        prioridade: str | int = "normal",
        receber_evento: bool = False,
        max_execucoes: int | None = None,
        cooldown: float = 0.0,
    ):
        valor_prioridade = _normalizar_prioridade_evento(prioridade)
        if max_execucoes is not None:
            if isinstance(max_execucoes, bool) or not isinstance(max_execucoes, int) or
              max_execucoes <= 0:
                raise ValueError("máximo de execuções de evento deve ser um inteiro
                  positivo ou nulo")
        try:
            valor_cooldown = float(validar_duracao(cooldown, "cooldown de evento"))
        except ValueError as exc:
```

```
            raise ValueError("cooldown de evento deve ser um número finito maior ou
              igual a zero") from exc
        if valor_cooldown > 0 and self._relogio is None and self._fonte_instante is
          None:
            raise ValueError("cooldown de evento exige uma fonte de tempo")
        self._ordem_inscricao += 1
        inscricao = _InscricaoEvento(
            funcao=funcao,
            prioridade=valor_prioridade,
            ordem=self._ordem_inscricao,
            receber_evento=bool(receber_evento),
            max_execucoes=max_execucoes,
            cooldown=valor_cooldown,
        )
        self._ouvintes.setdefault(str(nome), []).append(inscricao)
        return funcao

    def remover(self, nome: str, funcao: Callable[..., Any]) -> bool:
        ouvintes = self._ouvintes.get(str(nome))
        if not ouvintes:
            return False
        for indice, inscricao in enumerate(ouvintes):
            if inscricao.funcao is funcao or inscricao.funcao == funcao:
                del ouvintes[indice]
                if not ouvintes:
                    self._ouvintes.pop(str(nome), None)
                return True
        return False

    def _novo_evento(
        self,
        nome: str,
        args: tuple[Any, ...],
        kwargs: Mapping[str, Any],
        origem: str | None,
        *,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
    ) -> Evento:
        self._sequencia += 1
        instante = self._instante_atual()
        return Evento(
            str(nome),
            tuple(args),
            dict(kwargs),
            None if origem is None else str(origem),
            self._sequencia,
            instante=instante,
            cancelavel=bool(cancelavel),
            metadados={} if metadados is None else dict(metadados),
            causa_sequencia=(self._pilha_processamento[-1].sequencia if
              self._pilha_processamento else None),
        )

    def _snapshot_ouvintes(self, nome: str) -> tuple[_InscricaoEvento, ...]:
        ouvintes = tuple(self._ouvintes.get(nome, ()))
        return tuple(sorted(ouvintes, key=lambda item: (-item.prioridade, item.ordem)))

    def _emitir_local(self, evento: Evento) -> int:
        ouvintes = self._snapshot_ouvintes(evento.nome)
        for inscricao in ouvintes:
```

```
            if inscricao.max_execucoes is not None and inscricao.execucoes >=
              inscricao.max_execucoes:
                continue
            if inscricao.cooldown > 0 and inscricao.ultimo_disparo is not None:
                if evento.instante is None:
                    raise ValueError("cooldown de evento exige instante lógico")
                decorrido = float(evento.instante) - inscricao.ultimo_disparo
                if decorrido < inscricao.cooldown and not math.isclose(
                    decorrido, inscricao.cooldown, rel_tol=1e-12, abs_tol=1e-15
                ):
                    continue
            if inscricao.receber_evento:
                inscricao.funcao(evento)
            else:
                inscricao.funcao(*evento.args, **dict(evento.kwargs))
            inscricao.execucoes += 1
            if evento.instante is not None:
                inscricao.ultimo_disparo = float(evento.instante)
        return len(ouvintes)

    def emitir(self, nome: str, *args, **kwargs) -> int:
        evento = self._novo_evento(nome, args, kwargs, None)
        return self._emitir_local(evento)

    def emitir_evento(
        self,
        nome: str,
        *args,
        origem: str | None = None,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
        **kwargs,
    ) -> Evento:
        evento = self._novo_evento(
            nome,
            args,
            kwargs,
            origem,
            cancelavel=cancelavel,
            metadados=metadados,
        )
        self._emitir_local(evento)
        return evento

    def enfileirar(self, nome: str, *args, origem: str | None = None, **kwargs) ->
      Evento:
        evento = self._novo_evento(nome, args, kwargs, origem)
        self._fila.append(evento)
        return evento

    def enfileirar_evento(
        self,
        nome: str,
        *args,
        origem: str | None = None,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
        **kwargs,
    ) -> Evento:
        evento = self._novo_evento(
            nome,
```

```
            args,
            kwargs,
            origem,
            cancelavel=cancelavel,
            metadados=metadados,
        )
        self._fila.append(evento)
        return evento

    def processar(self, limite: int | None = None) -> tuple[Evento, ...]:
        if limite is not None and (isinstance(limite, bool) or not isinstance(limite,
          int) or limite < 0):
            raise ValueError("limite de eventos deve ser inteiro não negativo ou nulo")
        processados: list[Evento] = []
        maximo = len(self._fila) if limite is None else min(limite, len(self._fila))
        for _ in range(maximo):
            evento = self._fila.popleft()
            self._pilha_processamento.append(evento)
            try:
                self._emitir_local(evento)
            finally:
                self._pilha_processamento.pop()
            processados.append(evento)
        return tuple(processados)

    def _emitir_propagado_objeto(self, evento: Evento) -> int:
        total = self._emitir_local(evento)
        if self._pai is not None and not evento.consumido:
            total += self._pai._emitir_propagado_objeto(evento)
        return total

    def emitir_propagado(self, nome: str, *args, origem: str | None = None, **kwargs) ->
       int:
        evento = self._novo_evento(nome, args, kwargs, origem)
        return self._emitir_propagado_objeto(evento)

    def emitir_evento_propagado(
        self,
        nome: str,
        *args,
        origem: str | None = None,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
        **kwargs,
    ) -> Evento:
        evento = self._novo_evento(
            nome,
            args,
            kwargs,
            origem,
            cancelavel=cancelavel,
            metadados=metadados,
        )
        self._emitir_propagado_objeto(evento)
        return evento


__all__ = [
    "CONTRATO", "validar_duracao", "normalizar_dt", "FonteTempo",
    "RelogioMonotonico", "RelogioSimulado", "OperacaoCancelada",
    "TokenCancelamento", "Evento", "Eventos",
```

```
]
```

Biblioteca `coral.rpg`

`coral.rpg.CombateTurnos`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
CombateTurnos(participantes: 'Iterable[Personagem]' = (), *, nome: 'str | None' = None,
  encerramento: 'str' = 'ultimo_vivo')
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
participantes	Iterable[Personagem]	POSITIONAL_OR_KEYWORD	não	()
nome	str | None	KEYWORD_ONLY	não	None
encerramento	str	KEYWORD_ONLY	não	'ultimo_vivo'

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
adicionar(self, personagem: 'Personagem') -> 'Personagem'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
atacar(self, atacante: 'Personagem', alvo: 'Personagem', dano: 'int', *, defesa: 'int |
  None' = None, chance_critico: 'float' = 0.0, multiplicador_critico: 'float' = 2.0,
  exigir_turno: 'bool' = True) -> 'ResultadoAtaque'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
encerrar(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
iniciar(self, *, iniciativas: 'dict[str, int] | None' = None, semente: 'int | None' =
  None) -> 'tuple[Personagem, ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
pausar(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
proximo(self) -> 'Personagem | None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
retomar(self) -> 'None'
```

Método público sem descrição própria no runtime.

`coral.rpg.EstadoRPG`

Agregado persistente do domínio RPG sobre um Mundo genérico.

Assinatura

Assinatura para consulta

```
EstadoRPG(mundo: 'Mundo') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mundo	Mundo	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `mundo`: `Mundo`.

`coral.rpg.Personagem`

Conveniência de RPG construída sobre :class:`coral.agentes.Agente`.

Assinatura

Assinatura para consulta

```
Personagem(nome: 'str', vida: 'int' = 10, *, vida_max: 'int | None' = None, **atributos:
    'Any')
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
vida	int	POSITIONAL_OR_KEYWORD	não	10
vida_max	int | None	KEYWORD_ONLY	não	None

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
atributos	Any	VAR_KEYWORD	não	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
adicionar_atributo(self, atributo: 'Atributo') -> 'Atributo'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
adicionar_estado(self, estado: 'str') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
adicionar_habilidade(self, habilidade: 'Habilidade') -> 'Habilidade'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
adicionar_imunidade_efeito(self, nome: 'str') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
adicionar_recurso(self, recurso: 'Recurso') -> 'Recurso'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
adicionar_tag(self, tag: 'str') -> "'Entidade'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
aplicar_efeito(self, efeito: 'Efeito') -> 'Efeito | None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
atributo(self, nome: 'str') -> 'Atributo | None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
atualizar(self, dt: 'float' = 0.0) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
curar(self, quantidade: 'int') -> 'int | float'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
decidir(self, contexto: 'Any' = None) -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
definir(self, nome: 'str', valor: 'Any') -> "'Entidade'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
definir_atributo(self, nome: 'str', base: 'float' = 0.0) -> 'Atributo'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
definir_politica_acao(self, politica: 'PoliticaAcao | None') -> "'Agente'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
definir_posicao(self, x: 'float', y: 'float') -> "'Agente'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
definir_recurso(self, nome: 'str', valor: 'float' = 0.0, *, minimo: 'float' = 0.0,
  maximo: 'float | None' = None, unidade: 'str | None' = None) -> 'Recurso'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
exigir_recurso(self, nome: 'str') -> 'Recurso'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
guardar(self, item: 'Any') -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
imune_a(self, efeito: 'str | Efeito') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
obter(self, nome: 'str', padrao: 'Any' = None) -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
possui_estado(self, estado: 'str') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
possui_tag(self, tag: 'str') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
receber_dano(self, quantidade: 'int') -> 'int | float'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
recurso(self, nome: 'str') -> 'Recurso | None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_efeito(self, efeito: 'Efeito') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_estado(self, estado: 'str') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_imunidade_efeito(self, nome: 'str') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_item(self, item: 'Any') -> 'bool'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
remover_tag(self, tag: 'str') -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
teste(self, atributo: 'str', dificuldade: 'int' = 10, *, semente: 'int | None' = None)
  -> 'tuple[Rolagem, bool]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
usar_habilidade(self, nome: 'str', alvo: 'Any' = None, contexto: 'Any' = None) -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
valor_atributo(self, nome: 'str', padrao: 'Any' = None) -> 'Any'
```

Método público sem descrição própria no runtime.

`coral.rpg.ResultadoAtaque`

ResultadoAtaque(atacante: "'Personagem'", alvo: "'Personagem'", dano_base: 'int', defesa: 'int', critico: 'bool', multiplicador_critico: 'float', dano_calculado: 'int', dano_aplicado: 'int | float', cancelado: 'bool' = False)

Assinatura

Assinatura para consulta

```
ResultadoAtaque(atacante: "'Personagem'", alvo: "'Personagem'", dano_base: 'int',
  defesa: 'int', critico: 'bool', multiplicador_critico: 'float', dano_calculado: 'int',
  dano_aplicado: 'int | float', cancelado: 'bool' = False) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
atacante	'Personagem'	POSITIONAL_OR_KEYWORD	sim	sem padrão
alvo	'Personagem'	POSITIONAL_OR_KEYWORD	sim	sem padrão
dano_base	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
defesa	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
critico	bool	POSITIONAL_OR_KEYWORD	sim	sem padrão
multiplicador_critico	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
dano_calculado	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
dano_aplicado	int | float	POSITIONAL_OR_KEYWORD	sim	sem padrão
cancelado	bool	POSITIONAL_OR_KEYWORD	não	False

Retorno declarado

None

Atributo `atacante`: `'Personagem'`.

Atributo `alvo`: `'Personagem'`.

Atributo `dano_base`: `int`.

Atributo `defesa`: `int`.

Atributo `critico`: `bool`.

Atributo `multiplicador_critico`: `float`.

Atributo `dano_calculado`: `int`.

Atributo `dano_aplicado`: `int | float`.

Atributo `cancelado`: `bool`.

`coral.rpg.Rolagem`

Rolagem(expressao: 'str', dados: 'tuple[int, ...]', modificador: 'int' = 0)

Assinatura

Assinatura para consulta

```
Rolagem(expressao: 'str', dados: 'tuple[int, ...]', modificador: 'int' = 0) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
expressao	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
dados	tuple[int, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
modificador	int	POSITIONAL_OR_KEYWORD	não	0

Retorno declarado

None

Atributo `expressao`: `str`.

Atributo `dados`: `tuple[int, ...]`.

Atributo `modificador`: `int`.

`coral.rpg.buscar_combate`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
buscar_combate(mundo: 'Mundo', nome: 'str') -> "'CombateTurnos | None'"
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mundo	Mundo	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

'CombateTurnos | None'

`coral.rpg.carregar_estado_rpg`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
carregar_estado_rpg(caminho: 'str | Path') -> 'Mundo'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
caminho	str	Path	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Mundo

`coral.rpg.combates_do_mundo`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
combates_do_mundo(mundo: 'Mundo') -> "tuple['CombateTurnos', ...]"
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mundo	Mundo	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

tuple['CombateTurnos', ...]

`coral.rpg.criar_personagem`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
criar_personagem(mundo: 'Mundo', nome: 'str', vida: 'int' = 10, **atributos: 'Any') ->
  'Personagem'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mundo	Mundo	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
vida	int	POSITIONAL_OR_KEYWORD	não	10
atributos	Any	VAR_KEYWORD	não	sem padrão

Retorno declarado

Personagem

`coral.rpg.estado_rpg`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
estado_rpg(mundo: 'Mundo') -> 'EstadoRPG'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mundo	Mundo	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

EstadoRPG

`coral.rpg.registrar_combate`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
registrar_combate(mundo: 'Mundo', nome: 'str', combate: "'CombateTurnos'") ->
  "'CombateTurnos'"
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mundo	Mundo	POSITIONAL_OR_KEYWORD	sim	sem padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
combate	'CombateTurnos'	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

'CombateTurnos'

`coral.rpg.rolar`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
rolar(expressao: 'str' = '1d20', semente: 'int | None' = None, *, gerador:
  'random.Random | None' = None) -> 'Rolagem'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
expressao	str	POSITIONAL_OR_KEYWORD	não	'1d20'
semente	int | None	POSITIONAL_OR_KEYWORD	não	None
gerador	random.Random | None	KEYWORD_ONLY	não	None

Retorno declarado

Rolagem

`coral.rpg.salvar_estado_rpg`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
salvar_estado_rpg(mundo: 'Mundo', caminho: 'str | Path') -> 'Path'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mundo	Mundo	POSITIONAL_OR_KEYWORD	sim	sem padrão
caminho	str | Path	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Path

Consultar a implementação canônica de coral.rpg

Implementação canônica de `coral.rpg`

Código Python

```
"""Contratos genéricos de agentes, recursos, atributos e efeitos.

Este módulo não depende de RPG nem de Jogos. Ele reúne os mecanismos neutros
introduzidos na Coral 1.5.10 para que vida, bateria, combustível, estoque,
robôs, personagens e outras entidades compartilhem a mesma fundação.
"""
from __future__ import annotations

from dataclasses import dataclass, field
import math
from typing import Any, Callable, Iterable, Mapping

from .mundo import Entidade, Mundo
from .tempo_eventos import Eventos, normalizar_dt, validar_duracao


def _numero_finito(valor: Any, nome: str) -> float:
    if isinstance(valor, bool):
        raise ValueError(f"{nome} deve ser um número finito")
    try:
        numero = float(valor)
    except (TypeError, ValueError) as exc:
        raise ValueError(f"{nome} deve ser um número finito") from exc
    if not math.isfinite(numero):
        raise ValueError(f"{nome} deve ser um número finito")
    return numero


def _nome_publico(valor: Any, campo: str = "nome") -> str:
    nome = str(valor).strip()
    if not nome:
        raise ValueError(f"{campo} não pode ser vazio")
    return nome


@dataclass(frozen=True)
class MudancaRecurso:
    recurso: str
    anterior: float
    atual: float
```

```
    solicitado: float
    aplicado: float
    origem: str | None = None


class Recurso:
    """Valor limitado reutilizável como vida, energia, bateria ou estoque."""

    def __init__(
        self,
        nome: str,
        valor: float = 0.0,
        *,
        minimo: float = 0.0,
        maximo: float | None = None,
        unidade: str | None = None,
    ) -> None:
        self.nome = _nome_publico(nome)
        self.minimo = _numero_finito(minimo, "mínimo do recurso")
        self.maximo = None if maximo is None else _numero_finito(maximo, "máximo do
          recurso")
        if self.maximo is not None and self.maximo < self.minimo:
            raise ValueError("máximo do recurso não pode ser menor que o mínimo")
        self.unidade = None if unidade is None else str(unidade)
        self.eventos = Eventos()
        self._valor = self._limitar(_numero_finito(valor, "valor do recurso"))

    def _limitar(self, valor: float) -> float:
        valor = max(self.minimo, float(valor))
        if self.maximo is not None:
            valor = min(self.maximo, valor)
        return valor

    @property
    def valor(self) -> float:
        return self._valor

    @property
    def cheio(self) -> bool:
        return self.maximo is not None and math.isclose(self.valor, self.maximo,
          rel_tol=1e-12, abs_tol=1e-15)

    @property
    def vazio(self) -> bool:
        return math.isclose(self.valor, self.minimo, rel_tol=1e-12, abs_tol=1e-15)

    @property
    def fracao(self) -> float | None:
        if self.maximo is None or math.isclose(self.maximo, self.minimo):
            return None
        return (self.valor - self.minimo) / (self.maximo - self.minimo)

    def definir(self, valor: float, *, origem: str | None = None) -> MudancaRecurso:
        solicitado = _numero_finito(valor, "valor do recurso")
        anterior = self._valor
        atual = self._limitar(solicitado)
        self._valor = atual
        mudanca = MudancaRecurso(
            self.nome,
            anterior,
            atual,
```

```
            solicitado - anterior,
            atual - anterior,
            None if origem is None else str(origem),
        )
        if not math.isclose(mudanca.aplicado, 0.0, rel_tol=0.0, abs_tol=1e-15):
            self.eventos.emitir("recurso.alterado", recurso=self, mudanca=mudanca)
        return mudanca

    def alterar(self, quantidade: float, *, origem: str | None = None) ->
      MudancaRecurso:
        quantidade = _numero_finito(quantidade, "alteração do recurso")
        return self.definir(self.valor + quantidade, origem=origem)

    def consumir(self, quantidade: float, *, origem: str | None = None) -> float:
        quantidade = _numero_finito(quantidade, "consumo do recurso")
        if quantidade < 0:
            raise ValueError("consumo do recurso não pode ser negativo")
        return -self.alterar(-quantidade, origem=origem).aplicado

    def recuperar(self, quantidade: float, *, origem: str | None = None) -> float:
        quantidade = _numero_finito(quantidade, "recuperação do recurso")
        if quantidade < 0:
            raise ValueError("recuperação do recurso não pode ser negativa")
        return self.alterar(quantidade, origem=origem).aplicado

    def pode_consumir(self, quantidade: float) -> bool:
        quantidade = _numero_finito(quantidade, "consumo do recurso")
        if quantidade < 0:
            return False
        return self.valor - quantidade >= self.minimo - 1e-12

    def __float__(self) -> float:
        return self.valor

    def __repr__(self) -> str:
        limite = "∞" if self.maximo is None else f"{self.maximo:g}"
        return f"Recurso({self.nome!r}, {self.valor:g}/{limite})"


_OPERACOES_MODIFICADOR = frozenset({"somar", "multiplicar", "substituir"})


@dataclass(frozen=True)
class Modificador:
    nome: str
    valor: float
    operacao: str = "somar"
    prioridade: int = 0
    origem: str | None = None
    persistente: bool = True

    def __post_init__(self) -> None:
        object.__setattr__(self, "nome", _nome_publico(self.nome))
        object.__setattr__(self, "valor", _numero_finito(self.valor, "valor do
          modificador"))
        operacao = str(self.operacao).strip().casefold()
        if operacao not in _OPERACOES_MODIFICADOR:
            raise ValueError("operação do modificador deve ser somar, multiplicar ou
              substituir")
        object.__setattr__(self, "operacao", operacao)
        if isinstance(self.prioridade, bool):
```

```
            raise ValueError("prioridade do modificador deve ser inteira")
        try:
            prioridade = int(self.prioridade)
        except (TypeError, ValueError) as exc:
            raise ValueError("prioridade do modificador deve ser inteira") from exc
        object.__setattr__(self, "prioridade", prioridade)
        if self.origem is not None:
            object.__setattr__(self, "origem", str(self.origem))


class Atributo:
    """Valor base com composição determinística de modificadores rastreáveis."""

    def __init__(self, nome: str, base: float = 0.0) -> None:
        self.nome = _nome_publico(nome)
        self.base = _numero_finito(base, "valor base do atributo")
        self._modificadores: list[tuple[int, Modificador]] = []
        self._proxima_ordem = 0

    @property
    def modificadores(self) -> tuple[Modificador, ...]:
        return tuple(item[1] for item in sorted(self._modificadores, key=lambda item:
          (item[1].prioridade, item[0])))

    @property
    def valor(self) -> float:
        atual = self.base
        for modificador in self.modificadores:
            if modificador.operacao == "somar":
                atual += modificador.valor
            elif modificador.operacao == "multiplicar":
                atual *= modificador.valor
            else:
                atual = modificador.valor
        return atual

    def definir_base(self, valor: float) -> "Atributo":
        self.base = _numero_finito(valor, "valor base do atributo")
        return self

    def adicionar(self, modificador: Modificador) -> Modificador:
        if not isinstance(modificador, Modificador):
            raise TypeError("atributo aceita apenas Modificador")
        self._modificadores.append((self._proxima_ordem, modificador))
        self._proxima_ordem += 1
        return modificador

    def modificar(
        self,
        nome: str,
        valor: float,
        *,
        operacao: str = "somar",
        prioridade: int = 0,
        origem: str | None = None,
        persistente: bool = True,
    ) -> Modificador:
        return self.adicionar(Modificador(nome, valor, operacao, prioridade, origem,
          persistente))

    def remover(self, modificador: Modificador) -> bool:
```

```
        for indice, (_, atual) in enumerate(self._modificadores):
            if atual is modificador:
                del self._modificadores[indice]
                return True
        return False

    def remover_origem(self, origem: str) -> int:
        origem = str(origem)
        antes = len(self._modificadores)
        self._modificadores[:] = [item for item in self._modificadores if item[1].origem
           != origem]
        return antes - len(self._modificadores)

    def explicar(self) -> dict[str, Any]:
        atual = self.base
        passos: list[dict[str, Any]] = []
        for modificador in self.modificadores:
            anterior = atual
            if modificador.operacao == "somar":
                atual += modificador.valor
            elif modificador.operacao == "multiplicar":
                atual *= modificador.valor
            else:
                atual = modificador.valor
            passos.append({
                "nome": modificador.nome,
                "operacao": modificador.operacao,
                "valor": modificador.valor,
                "prioridade": modificador.prioridade,
                "origem": modificador.origem,
                "anterior": anterior,
                "resultado": atual,
            })
        return {"atributo": self.nome, "base": self.base, "passos": passos, "valor":
          atual}

    def __float__(self) -> float:
        return self.valor

    def __repr__(self) -> str:
        return f"Atributo({self.nome!r}, base={self.base:g}, valor={self.valor:g})"


_MODOS_EMPILHAMENTO = frozenset({"empilhar", "renovar", "substituir", "ignorar"})


@dataclass
class Efeito:
    """Condição temporal neutra que pode aplicar modificadores a um agente."""

    nome: str
    duracao: float | None = None
    modificadores: Mapping[str, Iterable[Modificador]] = field(default_factory=dict)
    empilhamento: str = "renovar"
    persistente: bool = True
    origem: str | None = None
    restante: float | None = None
    _aplicados: list[tuple[str, Modificador]] = field(default_factory=list, init=False,
      repr=False)

    def __post_init__(self) -> None:
```

```
        self.nome = _nome_publico(self.nome)
        if self.duracao is not None:
            self.duracao = float(validar_duracao(self.duracao, "duração do efeito"))
        if self.restante is None:
            self.restante = self.duracao
        elif self.duracao is None:
            raise ValueError("efeito sem duração não pode declarar tempo restante")
        else:
            self.restante = float(validar_duracao(self.restante, "tempo restante do
              efeito"))
            self.restante = min(self.restante, self.duracao)
        self.empilhamento = str(self.empilhamento).strip().casefold()
        if self.empilhamento not in _MODOS_EMPILHAMENTO:
            raise ValueError("empilhamento deve ser empilhar, renovar, substituir ou
              ignorar")
        if self.origem is not None:
            self.origem = str(self.origem)
        normalizados: dict[str, tuple[Modificador, ...]] = {}
        for nome_atributo, itens in dict(self.modificadores).items():
            lista = tuple(itens)
            if not all(isinstance(item, Modificador) for item in lista):
                raise TypeError("efeitos aceitam apenas modificadores formais")
            normalizados[str(nome_atributo)] = lista
        self.modificadores = normalizados

    @property
    def ativo(self) -> bool:
        return self.restante is None or self.restante > 0.0

    def renovar(self) -> None:
        self.restante = self.duracao

    def avancar(self, dt: float) -> bool:
        if self.restante is None:
            return False
        passo = normalizar_dt(dt, compatibilidade_negativo_zero=True)
        self.restante = max(0.0, self.restante - passo)
        return self.restante <= 0.0


AcaoHabilidade = Callable[["Agente", Any, Any], Any]
RequisitoHabilidade = Callable[["Agente", Any, Any], bool]
PoliticaAcao = Callable[["Agente", Any], Any]


class Habilidade:
    """Ação reutilizável com custos de recursos e recarga por tempo simulado."""

    def __init__(
        self,
        nome: str,
        acao: AcaoHabilidade | None = None,
        *,
        custos: Mapping[str, float] | None = None,
        recarga: float = 0.0,
        metadados: Mapping[str, Any] | None = None,
        requisitos: Iterable[RequisitoHabilidade] = (),
    ) -> None:
        self.nome = _nome_publico(nome)
        self.acao = acao
        self.custos = {str(k): _numero_finito(v, f"custo de {k}") for k, v in
```

```
          dict(custos or {}).items()}
        if any(valor < 0 for valor in self.custos.values()):
            raise ValueError("custos de habilidade não podem ser negativos")
        self.recarga = float(validar_duracao(recarga, "recarga da habilidade"))
        self.restante_recarga = 0.0
        self.metadados = dict(metadados or {})
        self.requisitos = tuple(requisitos)
        if not all(callable(requisito) for requisito in self.requisitos):
            raise TypeError("requisitos de habilidade precisam ser chamáveis")

    @property
    def pronta(self) -> bool:
        return self.restante_recarga <= 0.0

    def _recursos_indisponiveis(self, agente: "Agente") -> tuple[str, ...]:
        indisponiveis: list[str] = []
        for nome, custo in self.custos.items():
            recurso = agente.recurso(nome)
            if recurso is None or not recurso.pode_consumir(custo):
                indisponiveis.append(nome)
        return tuple(indisponiveis)

    def pode_usar(self, agente: "Agente", alvo: Any = None, contexto: Any = None) ->
      bool:
        if not isinstance(agente, Agente) or not self.pronta:
            return False
        if self._recursos_indisponiveis(agente):
            return False
        return all(bool(requisito(agente, alvo, contexto)) for requisito in
          self.requisitos)

    def usar(self, agente: "Agente", alvo: Any = None, contexto: Any = None) -> Any:
        if not isinstance(agente, Agente):
            raise TypeError("habilidade exige um Agente")
        if not self.pronta:
            agente._emitir("agente.habilidade.falhou", habilidade=self,
              motivo="recarga")
            raise RuntimeError(f"habilidade {self.nome!r} ainda está em recarga")
        indisponiveis = self._recursos_indisponiveis(agente)
        if indisponiveis:
            agente._emitir(
                "agente.habilidade.falhou",
                habilidade=self,
                motivo="recursos_insuficientes",
                recursos=indisponiveis,
            )
            raise RuntimeError("recursos insuficientes para habilidade: " + ",
              ".join(indisponiveis))
        if not all(bool(requisito(agente, alvo, contexto)) for requisito in
          self.requisitos):
            agente._emitir("agente.habilidade.falhou", habilidade=self,
              motivo="requisito")
            raise RuntimeError(f"requisitos da habilidade {self.nome!r} não foram
              atendidos")
        preparacao = agente._emitir_cancelavel(
            "agente.habilidade.antes", habilidade=self, alvo=alvo, contexto=contexto
        )
        if preparacao.cancelado:
            agente._emitir(
                "agente.habilidade.falhou",
                habilidade=self,
```

```
                motivo="cancelada",
                motivo_cancelamento=preparacao.motivo_cancelamento,
            )
            return None
        for nome, custo in self.custos.items():
            agente.exigir_recurso(nome).consumir(custo,
              origem=f"habilidade:{self.nome}")
        resultado = self.acao(agente, alvo, contexto) if self.acao is not None else None
        self.restante_recarga = self.recarga
        agente._emitir("agente.habilidade.usada", habilidade=self, alvo=alvo,
          resultado=resultado)
        return resultado

    def atualizar(self, dt: float) -> None:
        passo = normalizar_dt(dt, compatibilidade_negativo_zero=True)
        self.restante_recarga = max(0.0, self.restante_recarga - passo)


class Agente(Entidade):
    """Entidade com identidade persistente, recursos, atributos e estado."""

    def __init__(
        self,
        nome: str,
        *,
        identificador: str | None = None,
        atributos: Mapping[str, Any] | None = None,
        estados: Iterable[str] = (),
        politica_acao: PoliticaAcao | None = None,
    ) -> None:
        super().__init__(_nome_publico(nome), dict(atributos or {}))
        self.identificador = _nome_publico(identificador if identificador is not None
          else nome, "identificador")
        self.estados: set[str] = {str(item) for item in estados}
        self.recursos: dict[str, Recurso] = {}
        self.atributos_formais: dict[str, Atributo] = {}
        self.habilidades: dict[str, Habilidade] = {}
        self.efeitos: list[Efeito] = []
        self.imunidades_efeitos: set[str] = set()
        self.eventos = Eventos()
        self._mundo: Mundo | None = None
        self._politica_acao: PoliticaAcao | None = None
        self._propagadores_recursos: dict[int, Callable[..., Any]] = {}
        self.definir_politica_acao(politica_acao)

    @property
    def mundo(self) -> Mundo | None:
        return self._mundo

    def _definir_mundo(self, mundo: Mundo | None):
        self._mundo = mundo
        self.eventos.definir_pai(None if mundo is None else mundo.regras.eventos)
        return self

    def _payload_evento(self, dados: Mapping[str, Any]) -> dict[str, Any]:
        payload = {"agente": self, **dict(dados)}
        if self._mundo is not None:
            payload.setdefault("mundo", self._mundo)
        return payload

    def _emitir(self, nome: str, **dados: Any) -> None:
```

```
        self.eventos.emitir_propagado(
            nome, origem=self.identificador, **self._payload_evento(dados)
        )

    def _emitir_cancelavel(self, nome: str, **dados: Any):
        return self.eventos.emitir_evento_propagado(
            nome,
            origem=self.identificador,
            cancelavel=True,
            **self._payload_evento(dados),
        )

    @property
    def politica_acao(self) -> PoliticaAcao | None:
        return self._politica_acao

    def definir_politica_acao(self, politica: PoliticaAcao | None) -> "Agente":
        if politica is not None and not callable(politica):
            raise TypeError("política de ação do agente precisa ser chamável ou nula")
        self._politica_acao = politica
        return self

    def decidir(self, contexto: Any = None) -> Any:
        if self._politica_acao is None:
            raise RuntimeError(f"agente {self.nome!r} não possui política de ação")
        decisao = self._politica_acao(self, contexto)
        self._emitir("agente.decisao", decisao=decisao, contexto=contexto)
        return decisao

    def adicionar_recurso(self, recurso: Recurso) -> Recurso:
        if not isinstance(recurso, Recurso):
            raise TypeError("agente aceita apenas Recurso")
        existente = self.recursos.get(recurso.nome)
        if existente is not None and existente is not recurso:
            raise ValueError(f"já existe um recurso chamado {recurso.nome!r}")
        self.recursos[recurso.nome] = recurso
        chave = id(recurso)
        if chave not in self._propagadores_recursos:
            def propagar(*, recurso: Recurso, mudanca: MudancaRecurso) -> None:
                self._emitir("agente.recurso.alterado", recurso=recurso,
                  mudanca=mudanca)

            recurso.eventos.quando("recurso.alterado", propagar)
            self._propagadores_recursos[chave] = propagar
        return recurso

    def definir_recurso(
        self,
        nome: str,
        valor: float = 0.0,
        *,
        minimo: float = 0.0,
        maximo: float | None = None,
        unidade: str | None = None,
    ) -> Recurso:
        return self.adicionar_recurso(Recurso(nome, valor, minimo=minimo, maximo=maximo,
           unidade=unidade))

    def recurso(self, nome: str) -> Recurso | None:
        return self.recursos.get(str(nome))
```

```
    def exigir_recurso(self, nome: str) -> Recurso:
        recurso = self.recurso(nome)
        if recurso is None:
            raise KeyError(f"recurso não encontrado: {nome}")
        return recurso

    def adicionar_atributo(self, atributo: Atributo) -> Atributo:
        if not isinstance(atributo, Atributo):
            raise TypeError("agente aceita apenas Atributo")
        existente = self.atributos_formais.get(atributo.nome)
        if existente is not None and existente is not atributo:
            raise ValueError(f"já existe um atributo formal chamado {atributo.nome!r}")
        self.atributos_formais[atributo.nome] = atributo
        return atributo

    def definir_atributo(self, nome: str, base: float = 0.0) -> Atributo:
        return self.adicionar_atributo(Atributo(nome, base))

    def atributo(self, nome: str) -> Atributo | None:
        return self.atributos_formais.get(str(nome))

    def valor_atributo(self, nome: str, padrao: Any = None) -> Any:
        atributo = self.atributo(nome)
        if atributo is not None:
            return atributo.valor
        return self.obter(nome, padrao)

    def adicionar_habilidade(self, habilidade: Habilidade) -> Habilidade:
        if not isinstance(habilidade, Habilidade):
            raise TypeError("agente aceita apenas Habilidade")
        existente = self.habilidades.get(habilidade.nome)
        if existente is not None and existente is not habilidade:
            raise ValueError(f"já existe uma habilidade chamada {habilidade.nome!r}")
        self.habilidades[habilidade.nome] = habilidade
        return habilidade

    def usar_habilidade(self, nome: str, alvo: Any = None, contexto: Any = None) -> Any:
        try:
            habilidade = self.habilidades[str(nome)]
        except KeyError as exc:
            raise KeyError(f"habilidade não encontrada: {nome}") from exc
        return habilidade.usar(self, alvo, contexto)

    def adicionar_estado(self, estado: str) -> bool:
        estado = _nome_publico(estado, "estado")
        antes = len(self.estados)
        self.estados.add(estado)
        mudou = len(self.estados) != antes
        if mudou:
            self._emitir("agente.estado.adicionado", estado=estado)
        return mudou

    def remover_estado(self, estado: str) -> bool:
        estado = str(estado)
        if estado not in self.estados:
            return False
        self.estados.remove(estado)
        self._emitir("agente.estado.removido", estado=estado)
        return True

    def possui_estado(self, estado: str) -> bool:
```

```
        return str(estado) in self.estados

    def adicionar_imunidade_efeito(self, nome: str) -> bool:
        nome = _nome_publico(nome, "imunidade")
        antes = len(self.imunidades_efeitos)
        self.imunidades_efeitos.add(nome)
        return len(self.imunidades_efeitos) != antes

    def remover_imunidade_efeito(self, nome: str) -> bool:
        nome = str(nome)
        if nome not in self.imunidades_efeitos:
            return False
        self.imunidades_efeitos.remove(nome)
        return True

    def imune_a(self, efeito: str | Efeito) -> bool:
        nome = efeito.nome if isinstance(efeito, Efeito) else str(efeito)
        return nome in self.imunidades_efeitos

    def aplicar_efeito(self, efeito: Efeito) -> Efeito | None:
        if not isinstance(efeito, Efeito):
            raise TypeError("agente aceita apenas Efeito")
        if self.imune_a(efeito):
            self._emitir("agente.efeito.bloqueado", efeito=efeito, motivo="imunidade")
            return None
        existentes = [atual for atual in self.efeitos if atual.nome == efeito.nome and
          atual.ativo]
        if existentes and efeito.empilhamento == "ignorar":
            return existentes[-1]
        if existentes and efeito.empilhamento == "renovar":
            existentes[-1].renovar()
            self._emitir("agente.efeito.renovado", efeito=existentes[-1])
            return existentes[-1]
        if existentes and efeito.empilhamento == "substituir":
            for atual in tuple(existentes):
                self.remover_efeito(atual)

        token_origem = efeito.origem or f"efeito:{efeito.nome}:{id(efeito)}"
        efeito._aplicados.clear()
        for nome_atributo, modificadores in efeito.modificadores.items():
            atributo = self.atributo(nome_atributo)
            if atributo is None:
                atributo = self.definir_atributo(nome_atributo,
                  float(self.obter(nome_atributo, 0.0) or 0.0))
            for modificador in modificadores:
                aplicado = Modificador(
                    modificador.nome,
                    modificador.valor,
                    modificador.operacao,
                    modificador.prioridade,
                    token_origem,
                    False,
                )
                atributo.adicionar(aplicado)
                efeito._aplicados.append((nome_atributo, aplicado))
        self.efeitos.append(efeito)
        self._emitir("agente.efeito.aplicado", efeito=efeito)
        return efeito

    def remover_efeito(self, efeito: Efeito) -> bool:
        try:
```

```
            self.efeitos.remove(efeito)
        except ValueError:
            return False
        for nome_atributo, modificador in tuple(efeito._aplicados):
            atributo = self.atributo(nome_atributo)
            if atributo is not None:
                atributo.remover(modificador)
        efeito._aplicados.clear()
        self._emitir("agente.efeito.removido", efeito=efeito)
        return True

    def atualizar(self, dt: float = 0.0) -> None:
        passo = normalizar_dt(dt, compatibilidade_negativo_zero=True)
        for habilidade in tuple(self.habilidades.values()):
            habilidade.atualizar(passo)
        expirados = [efeito for efeito in tuple(self.efeitos) if efeito.avancar(passo)]
        for efeito in expirados:
            self.remover_efeito(efeito)

    @property
    def posicao(self) -> tuple[float, float] | None:
        x = self.obter("x", None)
        y = self.obter("y", None)
        if x is None or y is None:
            return None
        return (_numero_finito(x, "posição x"), _numero_finito(y, "posição y"))

    def definir_posicao(self, x: float, y: float) -> "Agente":
        self.definir("x", _numero_finito(x, "posição x"))
        self.definir("y", _numero_finito(y, "posição y"))
        return self


def criar_agente(
    mundo: Mundo,
    nome: str,
    *,
    identificador: str | None = None,
    atributos: Mapping[str, Any] | None = None,
    estados: Iterable[str] = (),
) -> Agente:
    """Cria um agente genérico e o adiciona a um Mundo sem acoplar Mundo a agentes."""
    if not isinstance(mundo, Mundo):
        raise TypeError("criar agente exige um Mundo")
    return mundo.adicionar(
        Agente(nome, identificador=identificador, atributos=atributos, estados=estados)
    )


# Persistência

def _recurso_para_dados(recurso: Recurso) -> dict[str, Any]:
    return {
        "nome": recurso.nome,
        "valor": recurso.valor,
        "minimo": recurso.minimo,
        "maximo": recurso.maximo,
        "unidade": recurso.unidade,
    }
```

```
def _recurso_de_dados(dados: dict[str, Any]) -> Recurso:
    return Recurso(
        dados["nome"],
        dados.get("valor", 0.0),
        minimo=dados.get("minimo", 0.0),
        maximo=dados.get("maximo"),
        unidade=dados.get("unidade"),
    )


def _modificador_para_dados(modificador: Modificador) -> dict[str, Any]:
    return {
        "nome": modificador.nome,
        "valor": modificador.valor,
        "operacao": modificador.operacao,
        "prioridade": modificador.prioridade,
        "origem": modificador.origem,
        "persistente": modificador.persistente,
    }


def _modificador_de_dados(dados: dict[str, Any]) -> Modificador:
    return Modificador(
        dados["nome"],
        dados["valor"],
        dados.get("operacao", "somar"),
        dados.get("prioridade", 0),
        dados.get("origem"),
        bool(dados.get("persistente", True)),
    )


def _atributo_para_dados(atributo: Atributo) -> dict[str, Any]:
    return {
        "nome": atributo.nome,
        "base": atributo.base,
        "modificadores": [m for m in atributo.modificadores if m.persistente],
    }


def _atributo_de_dados(dados: dict[str, Any]) -> Atributo:
    atributo = Atributo(dados["nome"], dados.get("base", 0.0))
    for modificador in dados.get("modificadores", []):
        if not isinstance(modificador, Modificador):
            raise ValueError("atributo persistente possui modificador inválido")
        atributo.adicionar(modificador)
    return atributo


def _efeito_para_dados(efeito: Efeito) -> dict[str, Any]:
    return {
        "nome": efeito.nome,
        "duracao": efeito.duracao,
        "restante": efeito.restante,
        "modificadores": {nome: list(itens) for nome, itens in
          efeito.modificadores.items()},
        "empilhamento": efeito.empilhamento,
        "persistente": efeito.persistente,
        "origem": efeito.origem,
    }
```

```
def _efeito_de_dados(dados: dict[str, Any]) -> Efeito:
    return Efeito(
        dados["nome"],
        dados.get("duracao"),
        modificadores=dados.get("modificadores", {}),
        empilhamento=dados.get("empilhamento", "renovar"),
        persistente=bool(dados.get("persistente", True)),
        origem=dados.get("origem"),
        restante=dados.get("restante"),
    )


def _agente_para_dados(agente: Agente) -> dict[str, Any]:
    return {
        "nome": agente.nome,
        "identificador": agente.identificador,
        "atributos": dict(agente.atributos),
        "tags": sorted(agente.tags),
        "inventario": list(agente.inventario),
        "estados": sorted(agente.estados),
        "imunidades_efeitos": sorted(agente.imunidades_efeitos),
        "recursos": list(agente.recursos.values()),
        "atributos_formais": list(agente.atributos_formais.values()),
        "efeitos": [efeito for efeito in agente.efeitos if efeito.persistente],
    }


def _agente_de_dados(dados: dict[str, Any]) -> Agente:
    agente = Agente(
        dados["nome"],
        identificador=dados.get("identificador"),
        atributos=dados.get("atributos", {}),
        estados=dados.get("estados", ()),
    )
    agente.tags.update(str(item) for item in dados.get("tags", []))
    agente.inventario.extend(dados.get("inventario", []))
    agente.imunidades_efeitos.update(str(item) for item in
      dados.get("imunidades_efeitos", []))
    for recurso in dados.get("recursos", []):
        if not isinstance(recurso, Recurso):
            raise ValueError("agente persistente possui recurso inválido")
        agente.adicionar_recurso(recurso)
    for atributo in dados.get("atributos_formais", []):
        if not isinstance(atributo, Atributo):
            raise ValueError("agente persistente possui atributo inválido")
        agente.adicionar_atributo(atributo)
    for efeito in dados.get("efeitos", []):
        if not isinstance(efeito, Efeito):
            raise ValueError("agente persistente possui efeito inválido")
        agente.aplicar_efeito(efeito)
    return agente


from .persistencia import registrar_adaptador as _registrar_adaptador

_registrar_adaptador(Recurso, "coral.agentes/Recurso", _recurso_para_dados,
  _recurso_de_dados)
_registrar_adaptador(Modificador, "coral.agentes/Modificador", _modificador_para_dados,
  _modificador_de_dados)
_registrar_adaptador(Atributo, "coral.agentes/Atributo", _atributo_para_dados,
```

```
  _atributo_de_dados)
_registrar_adaptador(Efeito, "coral.agentes/Efeito", _efeito_para_dados,
  _efeito_de_dados)
_registrar_adaptador(Agente, "coral.agentes/Agente", _agente_para_dados,
  _agente_de_dados)


__all__ = [
    "Agente",
    "Atributo",
    "Efeito",
    "Habilidade",
    "Modificador",
    "MudancaRecurso",
    "Recurso",
    "criar_agente",
]

from __future__ import annotations

from collections import deque
from dataclasses import dataclass, field
import json
from typing import Any, Iterable

from ._mapas import (
    BlocoMapa,
    CamadaMapa,
    CelulaHexagonal,
    CelulaQuadrada,
    CelulaVoxel,
    ConteudoCelula,
    ErroMapaCoral,
    Mapa,
    MudancaMapa,
    ObservadorRegiaoMapa,
    RegiaoMapa,
    ResultadoPassagem,
    TopologiaHexagonal,
    TopologiaQuadrada,
    TopologiaVoxel,
)
from .regras import MotorRegras
from .tempo_eventos import Eventos, FonteTempo


@dataclass(eq=False)
class Entidade:
    nome: str
    atributos: dict[str, Any] = field(default_factory=dict)
    tags: set[str] = field(default_factory=set)
    inventario: list[Any] = field(default_factory=list)

    def definir(self, nome: str, valor: Any) -> "Entidade":
        self.atributos[str(nome)] = valor
        return self

    def obter(self, nome: str, padrao: Any = None) -> Any:
        return self.atributos.get(str(nome), padrao)

    def adicionar_tag(self, tag: str) -> "Entidade":
```

```
        self.tags.add(str(tag))
        return self

    def remover_tag(self, tag: str) -> None:
        self.tags.discard(str(tag))

    def possui_tag(self, tag: str) -> bool:
        return str(tag) in self.tags

    def guardar(self, item: Any) -> Any:
        self.inventario.append(item)
        return item

    def remover_item(self, item: Any) -> bool:
        try:
            self.inventario.remove(item)
            return True
        except ValueError:
            return False

    def __getitem__(self, chave: str) -> Any:
        return self.atributos[chave]

    def __setitem__(self, chave: str, valor: Any) -> None:
        self.atributos[chave] = valor

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "nome": self.nome,
            "atributos": dict(self.atributos),
            "tags": tuple(sorted(self.tags)),
            "inventario": tuple(self.inventario),
            "quantidade_itens": len(self.inventario),
        }

    def __repr__(self) -> str:
        return f"Entidade({self.nome!r})"


@dataclass(frozen=True)
class Relacao:
    origem: Entidade
    destino: Entidade
    tipo: str
    dados: dict[str, Any] = field(default_factory=dict, compare=False)


class Mundo:
    """Mundo leve para jogos, RPGs e simulações Coral.

    A API mantém entidades e relações como um grafo dirigido. Relações podem
    ser consultadas nos dois sentidos e o método ``caminho`` usa BFS.
    """

    def __init__(self, nome: str = "mundo", *, relogio: FonteTempo | None = None,
      eventos: Eventos | None = None):
        self.nome = str(nome)
        self._entidades: dict[str, Entidade] = {}
        self._relacoes: list[Relacao] = []
        self._mapas: dict[str, Mapa] = {}
        self.regras = MotorRegras(relogio=relogio, eventos=eventos)
```

```
        self._simulacao = None

    @property
    def entidades(self) -> tuple[Entidade, ...]:
        return tuple(self._entidades.values())

    @property
    def relacoes(self) -> tuple[Relacao, ...]:
        return tuple(self._relacoes)

    @property
    def mapas(self) -> tuple[Mapa, ...]:
        return tuple(self._mapas.values())

    @property
    def simulacao(self):
        """Simulação temporal associada, quando criada explicitamente."""
        return self._simulacao

    def criar_simulacao(self, *, tempo=None):
        """Cria ou devolve a simulação temporal do mundo sem exigir Jogos."""
        from .simulacao import Simulacao, TempoMundo
        from .tempo_eventos import RelogioSimulado

        if self._simulacao is not None:
            if tempo is not None and self._simulacao.tempo is not tempo:
                raise ValueError("o mundo já possui uma simulação com outro tempo")
            return self._simulacao
        if tempo is None:
            relogio_existente = self.regras.relogio
            if isinstance(relogio_existente, RelogioSimulado):
                tempo = TempoMundo(eventos=self.regras.eventos,
                  relogio=relogio_existente)
            else:
                tempo = TempoMundo(self.regras.instante_atual(),
                  eventos=self.regras.eventos)
        if not isinstance(tempo, TempoMundo):
            raise TypeError("tempo precisa ser TempoMundo")
        self.regras.relogio = tempo.relogio
        self._simulacao = Simulacao(tempo=tempo)
        return self._simulacao

    def avancar_tempo(self, duracao: float):
        """Avança a simulação e depois atualiza regras e temporizadores pelo mesmo
          delta."""
        simulacao = self.criar_simulacao()
        inicio = simulacao.tempo.instante
        resultados = simulacao.avancar(duracao, contexto=self)
        aplicado = simulacao.tempo.instante - inicio
        self.regras.atualizar(self, aplicado)
        return resultados

    def atualizar_simulacao(self, dt_externo: float):
        """Atualiza a simulação respeitando pausa e escala temporal."""
        simulacao = self.criar_simulacao()
        inicio = simulacao.tempo.instante
        resultados = simulacao.atualizar(dt_externo, contexto=self)
        aplicado = simulacao.tempo.instante - inicio
        if aplicado:
            self.regras.atualizar(self, aplicado)
        return resultados
```

```
    def adicionar_mapa(self, mapa: Mapa) -> Mapa:
        if not isinstance(mapa, Mapa):
            raise ErroMapaCoral("Mundo só pode adicionar objetos Mapa.",
              codigo="mapa.mapa_invalido")
        existente = self._mapas.get(mapa.nome)
        if existente is not None and existente is not mapa:
            raise ErroMapaCoral(
                f'Já existe um mapa chamado "{mapa.nome}" no mundo "{self.nome}".',
                codigo="mapa.mapa_duplicado",
                contexto={"mundo": self.nome, "mapa": mapa.nome},
            )
        self._mapas[mapa.nome] = mapa
        mapa._vincular_runtime(self.regras.eventos, self.regras.rastreamento)
        return mapa

    def criar_mapa(self, nome: str, topologia, *, propriedades: dict[str, Any] | None =
      None) -> Mapa:
        return self.adicionar_mapa(Mapa(nome, topologia, propriedades=propriedades))

    def buscar_mapa(self, nome: str) -> Mapa | None:
        return self._mapas.get(str(nome))

    def exigir_mapa(self, nome: str) -> Mapa:
        mapa = self.buscar_mapa(nome)
        if mapa is None:
            raise ErroMapaCoral(
                f'O mapa "{nome}" não existe no mundo "{self.nome}".',
                codigo="mapa.mapa_inexistente",
                contexto={"mundo": self.nome, "mapa": str(nome)},
            )
        return mapa

    def remover_mapa(self, mapa: Mapa | str) -> bool:
        nome = mapa if isinstance(mapa, str) else mapa.nome
        existente = self._mapas.get(str(nome))
        if existente is None or (isinstance(mapa, Mapa) and existente is not mapa):
            return False
        del self._mapas[str(nome)]
        existente._vincular_runtime(None, None)
        return True

    def observar_regioes_mapa(
        self,
        nome: str,
        alvo: Any,
        mapa: Mapa | str,
        regioes,
        resolvedor,
        *,
        identificador_alvo: Any = None,
    ) -> ObservadorRegiaoMapa:
        mapa_real = self.exigir_mapa(mapa) if isinstance(mapa, str) else mapa
        if self._mapas.get(mapa_real.nome) is not mapa_real:
            raise ErroMapaCoral(
                "O mapa observado precisa pertencer ao mundo.",
                codigo="mapa.mapa_inexistente",
                contexto={"mundo": self.nome, "mapa": mapa_real.nome},
            )
        return self.regras.observar_regioes_mapa(
            nome, alvo, mapa_real, regioes, resolvedor,
```

```
            identificador_alvo=identificador_alvo
        )

    def adicionar(self, entidade: Entidade) -> Entidade:
        if entidade.nome in self._entidades and self._entidades[entidade.nome] is not
          entidade:
            raise ValueError(f"já existe uma entidade chamada {entidade.nome!r}")
        self._entidades[entidade.nome] = entidade
        if hasattr(entidade, "_definir_mundo"):
            entidade._definir_mundo(self)
        return entidade

    def criar_entidade(self, nome: str, **atributos: Any) -> Entidade:
        entidade = Entidade(str(nome), dict(atributos))
        return self.adicionar(entidade)

    def buscar(self, nome: str) -> Entidade | None:
        return self._entidades.get(str(nome))

    def exigir(self, nome: str) -> Entidade:
        entidade = self.buscar(nome)
        if entidade is None:
            raise KeyError(f"entidade não encontrada: {nome}")
        return entidade

    def remover(self, entidade: Entidade | str) -> bool:
        if isinstance(entidade, str):
            entidade = self.buscar(entidade)
        if entidade is None or self._entidades.get(entidade.nome) is not entidade:
            return False
        del self._entidades[entidade.nome]
        self._relacoes[:] = [r for r in self._relacoes if r.origem is not entidade and
          r.destino is not entidade]
        if hasattr(entidade, "_definir_mundo"):
            entidade._definir_mundo(None)
        return True

    def relacionar(self, origem: Entidade | str, destino: Entidade | str, tipo: str,
      **dados: Any) -> Relacao:
        if isinstance(origem, str): origem = self.exigir(origem)
        if isinstance(destino, str): destino = self.exigir(destino)
        if self._entidades.get(origem.nome) is not origem or
          self._entidades.get(destino.nome) is not destino:
            raise ValueError("as duas entidades precisam pertencer ao mundo")
        relacao = Relacao(origem, destino, str(tipo), dict(dados))
        self._relacoes.append(relacao)
        return relacao

    def remover_relacao(self, relacao: Relacao) -> bool:
        try:
            self._relacoes.remove(relacao)
            return True
        except ValueError:
            return False

    def relacionados(self, entidade: Entidade | str, tipo: str | None = None, *,
      direcao: str = "saida") -> tuple[Entidade, ...]:
        if isinstance(entidade, str): entidade = self.exigir(entidade)
        direcao = direcao.casefold()
        if direcao not in {"saida", "entrada", "ambas"}:
            raise ValueError("direção deve ser 'saida', 'entrada' ou 'ambas'")
```

```
        resultado=[]
        for r in self._relacoes:
            if tipo is not None and r.tipo != tipo:
                continue
            if direcao in {"saida", "ambas"} and r.origem is entidade:
                resultado.append(r.destino)
            if direcao in {"entrada", "ambas"} and r.destino is entidade:
                resultado.append(r.origem)
        # preserva ordem e remove duplicatas por identidade
        vistos=set(); unicos=[]
        for e in resultado:
            if id(e) not in vistos:
                vistos.add(id(e)); unicos.append(e)
        return tuple(unicos)

    def caminho(self, origem: Entidade | str, destino: Entidade | str, *, tipo: str |
      None = None, bidirecional: bool = True) -> tuple[Entidade, ...]:
        if isinstance(origem, str): origem = self.exigir(origem)
        if isinstance(destino, str): destino = self.exigir(destino)
        if origem is destino:
            return (origem,)
        fila=deque([origem]); anterior={id(origem):None}; por_id={id(origem):origem}
        while fila:
            atual=fila.popleft()
            vizinhos=self.relacionados(atual,tipo,direcao="ambas" if bidirecional else
              "saida")
            for vizinho in vizinhos:
                chave=id(vizinho)
                if chave in anterior: continue
                anterior[chave]=id(atual); por_id[chave]=vizinho
                if vizinho is destino:
                    ids=[]; cursor=chave
                    while cursor is not None:
                        ids.append(cursor); cursor=anterior[cursor]
                    return tuple(por_id[i] for i in reversed(ids))
                fila.append(vizinho)
        return ()

    def com_tag(self, tag: str) -> tuple[Entidade, ...]:
        return tuple(e for e in self._entidades.values() if e.possui_tag(tag))

    def atualizar(self, dt: float = 0.0) -> tuple[str, ...]:
        return self.regras.atualizar(self, dt)

    def atualizar_por_relogio(self) -> tuple[str, ...]:
        return self.regras.atualizar_por_relogio(self)

    def emitir(self, evento: str, *args, **kwargs) -> int:
        return self.regras.eventos.emitir(evento, *args, **kwargs)

    def emitir_evento(
        self,
        evento: str,
        *args,
        cancelavel: bool = False,
        metadados: dict[str, Any] | None = None,
        **kwargs,
    ):
        return self.regras.eventos.emitir_evento(
            evento,
            *args,
```

```
            origem=self.nome,
            cancelavel=cancelavel,
            metadados=metadados,
            **kwargs,
        )

    def enfileirar_evento(self, evento: str, *args, **kwargs):
        return self.regras.eventos.enfileirar(evento, *args, origem=self.nome, **kwargs)

    def processar_eventos(self, limite: int | None = None):
        return self.regras.eventos.processar(limite)

    def emitir_propagado(self, evento: str, *args, **kwargs) -> int:
        return self.regras.eventos.emitir_propagado(evento, *args, origem=self.nome,
          **kwargs)

    def emitir_evento_propagado(
        self,
        evento: str,
        *args,
        cancelavel: bool = False,
        metadados: dict[str, Any] | None = None,
        **kwargs,
    ):
        return self.regras.eventos.emitir_evento_propagado(
            evento,
            *args,
            origem=self.nome,
            cancelavel=cancelavel,
            metadados=metadados,
            **kwargs,
        )

    def estado_inspecao(self) -> dict[str, Any]:
        simulacao = self._simulacao
        return {
            "nome": self.nome,
            "quantidade_entidades": len(self._entidades),
            "entidades": tuple(self._entidades.values()),
            "quantidade_relacoes": len(self._relacoes),
            "relacoes": tuple(
                {
                    "origem": relacao.origem.nome,
                    "destino": relacao.destino.nome,
                    "tipo": relacao.tipo,
                    "dados": dict(relacao.dados),
                }
                for relacao in self._relacoes
            ),
            "mapas": tuple(self._mapas),
            "simulacao_ativa": simulacao is not None,
            "regras": self.regras,
        }

    def para_dict(self) -> dict[str, Any]:
        return {
            "nome": self.nome,
            "entidades": [
                {"nome": e.nome, "atributos": dict(e.atributos), "tags": sorted(e.tags),
                   "inventario": list(e.inventario)}
                for e in self._entidades.values()
```

```
            ],
            "relacoes": [
                {"origem": r.origem.nome, "destino": r.destino.nome, "tipo": r.tipo,
                  "dados": dict(r.dados)}
                for r in self._relacoes
            ],
        }

    def para_json(self, **kwargs) -> str:
        opcoes={"ensure_ascii":False,"indent":2}; opcoes.update(kwargs)
        return json.dumps(self.para_dict(), **opcoes)

    def __iter__(self):
        return iter(self._entidades.values())

    def __len__(self) -> int:
        return len(self._entidades)


def criar_mundo(nome: str = "mundo") -> Mundo:
    return Mundo(nome)


__all__ = [
    "Mundo", "Entidade", "Relacao", "criar_mundo",
    "Mapa", "CamadaMapa", "ConteudoCelula", "BlocoMapa", "MudancaMapa", "RegiaoMapa",
      "ResultadoPassagem", "ObservadorRegiaoMapa",
    "CelulaQuadrada", "CelulaHexagonal", "CelulaVoxel",
    "TopologiaQuadrada", "TopologiaHexagonal", "TopologiaVoxel", "ErroMapaCoral",
]


# Adaptadores de persistência são registrados no módulo do domínio para manter
# o núcleo ``coral.persistencia`` desacoplado de Mundo.
def _entidade_para_dados(entidade: Entidade) -> dict[str, Any]:
    return {
        "nome": entidade.nome,
        "atributos": dict(entidade.atributos),
        "tags": sorted(entidade.tags),
        "inventario": list(entidade.inventario),
    }


def _entidade_de_dados(dados: dict[str, Any]) -> Entidade:
    if not isinstance(dados.get("nome"), str):
        raise ValueError("nome de entidade inválido")
    atributos = dados.get("atributos", {})
    tags = dados.get("tags", [])
    inventario = dados.get("inventario", [])
    if not isinstance(atributos, dict) or not isinstance(tags, list) or not
      isinstance(inventario, list):
        raise ValueError("payload de entidade inválido")
    entidade = Entidade(dados["nome"], dict(atributos))
    entidade.tags.update(str(x) for x in tags)
    entidade.inventario.extend(inventario)
    return entidade


def _mundo_para_dados(mundo: Mundo) -> dict[str, Any]:
    from .persistencia import extensoes_para_dados, para_dados
    dados = {
```

```
        "nome": mundo.nome,
        "entidades": [para_dados(e) for e in mundo.entidades],
        "relacoes": [
            {"origem": r.origem.nome, "destino": r.destino.nome, "tipo": r.tipo,
              "dados": dict(r.dados)}
            for r in mundo.relacoes
        ],
    }
    if mundo.mapas:
        dados["mapas"] = [para_dados(mapa) for mapa in mundo.mapas]
    extensoes = extensoes_para_dados(mundo)
    if extensoes:
        dados["extensoes"] = extensoes
    return dados


def _mundo_de_dados(dados: dict[str, Any]) -> Mundo:
    from .persistencia import ErroPersistencia
    nome = dados.get("nome")
    entidades = dados.get("entidades", [])
    relacoes = dados.get("relacoes", [])
    mapas = dados.get("mapas", [])
    extensoes = dados.get("extensoes", {})
    if not isinstance(nome, str) or not isinstance(entidades, list) or not
      isinstance(relacoes, list) or not isinstance(mapas, list) or not
      isinstance(extensoes, dict):
        raise ErroPersistencia("Payload de Mundo inválido.")
    mundo = Mundo(nome)
    for entidade in entidades:
        if not isinstance(entidade, Entidade):
            raise ErroPersistencia("Mundo contém item que não é Entidade.")
        mundo.adicionar(entidade)
    for indice, relacao in enumerate(relacoes):
        if not isinstance(relacao, dict):
            raise ErroPersistencia(f"Relação inválida no índice {indice}.")
        origem = mundo.buscar(relacao.get("origem")) if
          isinstance(relacao.get("origem"), str) else None
        destino = mundo.buscar(relacao.get("destino")) if
          isinstance(relacao.get("destino"), str) else None
        if origem is None or destino is None:
            raise ErroPersistencia(f"Relação quebrada no índice {indice}: origem ou
              destino inexistente.")
        tipo = relacao.get("tipo")
        rel_dados = relacao.get("dados", {})
        if not isinstance(tipo, str) or not isinstance(rel_dados, dict):
            raise ErroPersistencia(f"Relação inválida no índice {indice}.")
        mundo.relacionar(origem, destino, tipo, **rel_dados)

    for indice, mapa in enumerate(mapas):
        if not isinstance(mapa, Mapa):
            raise ErroPersistencia(f"Mapa inválido no índice {indice} do Mundo.")
        mundo.adicionar_mapa(mapa)

    from .persistencia import aplicar_extensoes_persistentes
    aplicar_extensoes_persistentes(mundo, extensoes)
    return mundo


from .persistencia import registrar_adaptador as _registrar_adaptador
_registrar_adaptador(Entidade, "coral.mundo/Entidade", _entidade_para_dados,
  _entidade_de_dados)
```

```
_registrar_adaptador(Mundo, "coral.mundo/Mundo", _mundo_para_dados, _mundo_de_dados)

from __future__ import annotations

from dataclasses import dataclass
import random
import math
import re
from pathlib import Path
from typing import Any, Iterable
from weakref import WeakKeyDictionary

from .mundo import Mundo
from .agentes import Agente, Atributo, Efeito, Recurso
from .turnos import FilaTurnos


_REGISTRO_COMBATES: "WeakKeyDictionary[Mundo, dict[str, CombateTurnos]]" =
  WeakKeyDictionary()


def _registro_combates(mundo: Mundo, *, criar: bool = False) -> dict[str,
  "CombateTurnos"]:
    if not isinstance(mundo, Mundo):
        raise TypeError("o registro de combates precisa de um Mundo")
    registro = _REGISTRO_COMBATES.get(mundo)
    if registro is None and criar:
        registro = {}
        _REGISTRO_COMBATES[mundo] = registro
    return registro if registro is not None else {}


def registrar_combate(mundo: Mundo, nome: str, combate: "CombateTurnos") ->
  "CombateTurnos":
    if not isinstance(combate, CombateTurnos):
        raise TypeError("o registro RPG aceita apenas CombateTurnos")
    nome = str(nome)
    if not nome:
        raise ValueError("o nome do combate não pode ser vazio")
    registro = _registro_combates(mundo, criar=True)
    existente = registro.get(nome)
    if existente is not None and existente is not combate:
        raise ValueError(f"já existe um combate chamado {nome!r}")
    registro[nome] = combate
    return combate


def buscar_combate(mundo: Mundo, nome: str) -> "CombateTurnos | None":
    return _registro_combates(mundo).get(str(nome))


def combates_do_mundo(mundo: Mundo) -> tuple["CombateTurnos", ...]:
    return tuple(_registro_combates(mundo).values())


@dataclass(frozen=True)
class Rolagem:
    expressao: str
    dados: tuple[int, ...]
    modificador: int = 0
```

```
    @property
    def total(self) -> int:
        return sum(self.dados) + self.modificador

    def __int__(self) -> int:
        return self.total

    def __repr__(self) -> str:
        mod = f"{self.modificador:+d}" if self.modificador else ""
        return f"Rolagem({self.expressao!r}, dados={self.dados}{',
          mod='+str(self.modificador) if self.modificador else ''}, total={self.total})"


@dataclass(frozen=True)
class ResultadoAtaque:
    atacante: "Personagem"
    alvo: "Personagem"
    dano_base: int
    defesa: int
    critico: bool
    multiplicador_critico: float
    dano_calculado: int
    dano_aplicado: int | float
    cancelado: bool = False


def rolar(expressao: str = "1d20", semente: int | None = None, *, gerador: random.Random
   | None = None) -> Rolagem:
    texto=str(expressao).strip().casefold().replace(" ","")
    m=re.fullmatch(r"(?:(\d+))?d(\d+)([+-]\d+)?",texto)
    if not m:
        raise ValueError("rolagem deve usar a forma NdM, por exemplo 1d20, 3d6 ou
          2d8+4")
    quantidade=int(m.group(1) or 1); lados=int(m.group(2)); modificador=int(m.group(3)
      or 0)
    if quantidade < 1 or quantidade > 1000:
        raise ValueError("quantidade de dados deve ficar entre 1 e 1000")
    if lados < 2:
        raise ValueError("um dado precisa ter ao menos 2 lados")
    rng=gerador or random.Random(semente)
    dados=tuple(rng.randint(1,lados) for _ in range(quantidade))
    return Rolagem(texto,dados,modificador)


class Personagem(Agente):
    """Conveniência de RPG construída sobre :class:`coral.agentes.Agente`."""

    def __init__(self, nome: str, vida: int = 10, *, vida_max: int | None = None,
      **atributos: Any):
        super().__init__(str(nome), atributos=dict(atributos))
        limite = max(1, int(vida if vida_max is None else vida_max))
        self.definir_recurso("vida", max(0, min(int(vida), limite)), minimo=0,
          maximo=limite)

    @property
    def vida(self) -> int | float:
        valor = self.exigir_recurso("vida").valor
        return int(valor) if float(valor).is_integer() else valor

    @vida.setter
    def vida(self, valor: int | float) -> None:
```

```
        self.exigir_recurso("vida").definir(valor, origem="rpg.vida")

    @property
    def vida_max(self) -> int | float:
        maximo = self.exigir_recurso("vida").maximo
        if maximo is None:
            return self.vida
        return int(maximo) if float(maximo).is_integer() else maximo

    @vida_max.setter
    def vida_max(self, valor: int | float) -> None:
        recurso = self.exigir_recurso("vida")
        limite = max(1.0, float(valor))
        recurso.maximo = limite
        recurso.definir(recurso.valor, origem="rpg.vida_max")

    @property
    def vivo(self) -> bool:
        return self.vida > 0

    def receber_dano(self, quantidade: int) -> int | float:
        dano=max(0,int(quantidade))
        antes=self.vida
        if dano and self._mundo is not None:
            antes_evento = self._mundo.emitir_evento_propagado(
                "rpg.dano.antes",
                personagem=self,
                quantidade=dano,
                vida_anterior=antes,
                mundo=self._mundo,
                cancelavel=True,
            )
            if antes_evento.cancelado:
                return 0
        aplicado = self.exigir_recurso("vida").consumir(dano, origem="rpg.dano")
        if aplicado and self._mundo is not None:
            self._mundo.emitir_evento_propagado(
                "rpg.dano",
                personagem=self,
                quantidade=aplicado,
                vida_anterior=antes,
                vida_atual=self.vida,
                mundo=self._mundo,
            )
        return int(aplicado) if float(aplicado).is_integer() else aplicado

    def curar(self, quantidade: int) -> int | float:
        cura=max(0,int(quantidade))
        antes=self.vida
        aplicado = self.exigir_recurso("vida").recuperar(cura, origem="rpg.cura")
        if aplicado and self._mundo is not None:
            self._mundo.emitir_evento_propagado(
                "rpg.cura",
                personagem=self,
                quantidade=aplicado,
                vida_anterior=antes,
                vida_atual=self.vida,
                mundo=self._mundo,
            )
        return int(aplicado) if float(aplicado).is_integer() else aplicado
```

```
    def teste(self, atributo: str, dificuldade: int = 10, *, semente: int | None = None)
       -> tuple[Rolagem,bool]:
        bonus=int(self.valor_atributo(atributo,0) or 0)
        resultado=rolar(f"1d20{bonus:+d}" if bonus else "1d20",semente=semente)
        return resultado, resultado.total >= int(dificuldade)


class CombateTurnos:
    def __init__(
        self,
        participantes: Iterable[Personagem] = (),
        *,
        nome: str | None = None,
        encerramento: str = "ultimo_vivo",
    ):
        self.fila = FilaTurnos(participantes)
        self.participantes = self.fila.participantes
        self._gerador = random.Random()
        if not all(isinstance(p, Personagem) for p in self.participantes):
            raise TypeError("combate por turnos aceita apenas Personagem")
        self.nome = str(nome) if nome is not None else None
        encerramento = str(encerramento).strip().casefold()
        if encerramento not in {"ultimo_vivo", "sem_vivos", "manual"}:
            raise ValueError("encerramento do combate deve ser ultimo_vivo, sem_vivos ou
               manual")
        self.encerramento = encerramento
        self._encerrado_manual = False
        contextos=[p.mundo for p in self.participantes]
        self.mundo: Mundo | None = (
            contextos[0]
            if contextos and contextos[0] is not None and all(m is contextos[0] for m in
               contextos[1:])
            else None
        )
        self._encerramento_emitido=False
        if self.nome is not None and self.mundo is not None:
            registrar_combate(self.mundo, self.nome, self)

    @property
    def ordem(self) -> list[Personagem]:
        return self.fila.ordem

    @ordem.setter
    def ordem(self, valor: Iterable[Personagem]) -> None:
        self.fila.ordem = list(valor)

    @property
    def indice(self) -> int:
        return self.fila.indice

    @indice.setter
    def indice(self, valor: int) -> None:
        self.fila.indice = int(valor)

    @property
    def rodada(self) -> int:
        return self.fila.rodada

    @rodada.setter
    def rodada(self, valor: int) -> None:
        self.fila.rodada = int(valor)
```

```
    @property
    def iniciado(self) -> bool:
        return self.fila.iniciada

    @iniciado.setter
    def iniciado(self, valor: bool) -> None:
        self.fila.iniciada = bool(valor)

    def pausar(self) -> None:
        self.fila.pausar()

    def retomar(self) -> None:
        self.fila.retomar()

    @property
    def pausado(self) -> bool:
        return self.fila.pausada

    def _mundo_compartilhado(self) -> Mundo | None:
        contextos=[p.mundo for p in self.participantes]
        com_mundo=[m for m in contextos if m is not None]
        if not com_mundo:
            return None
        if len(com_mundo) != len(contextos) or any(m is not com_mundo[0] for m in
          com_mundo[1:]):
            raise ValueError("participantes de combate contextual precisam pertencer ao
              mesmo mundo")
        return com_mundo[0]

    def _emitir(self, evento: str, *, personagem: Personagem | None = None) -> int:
        if self.mundo is None:
            return 0
        payload={"combate":self,"rodada":self.rodada,"mundo":self.mundo}
        if personagem is not None:
            payload["personagem"]=personagem
        return self.mundo.emitir_propagado(evento, **payload)

    def _emitir_encerramento_se_necessario(self) -> None:
        if self.iniciado and self.terminou and not self._encerramento_emitido:
            self._encerramento_emitido=True
            self._emitir("rpg.combate.encerrado", personagem=self.atual)

    def adicionar(self, personagem: Personagem) -> Personagem:
        if not isinstance(personagem, Personagem):
            raise TypeError("combate por turnos aceita apenas Personagem")
        self.fila.adicionar(personagem)
        return personagem

    def iniciar(self, *, iniciativas: dict[str,int] | None = None, semente: int | None =
       None) -> tuple[Personagem,...]:
        self.mundo=self._mundo_compartilhado()
        if self.nome is not None and self.mundo is not None:
            registrar_combate(self.mundo, self.nome, self)
        self._encerramento_emitido=False
        self._encerrado_manual = False
        self._gerador = random.Random(semente)
        rng = self._gerador
        iniciativas=iniciativas or {}
        pares=[]
        for p in self.participantes:
```

```
            valor=int(iniciativas[p.nome]) if p.nome in iniciativas else
              rng.randint(1,20)+int(p.obter('iniciativa',0) or 0)
            pares.append((valor,p))
        pares.sort(key=lambda x:(-x[0],x[1].nome.casefold()))
        self.fila.iniciar(ordem=[p for _,p in pares if p.vivo])
        if self.mundo is not None:
            self._emitir("rpg.combate.iniciado")
            if self.atual is not None:
                self._emitir("rpg.combate.turno", personagem=self.atual)
        return tuple(self.ordem)

    @property
    def atual(self) -> Personagem | None:
        if not self.iniciado or not self.ordem: return None
        vivos=[p for p in self.ordem if p.vivo]
        if not vivos: return None
        if self.indice >= len(self.ordem): self.indice=0
        # pula mortos sem reordenar a rodada
        tentativas=0
        while not self.ordem[self.indice].vivo and tentativas < len(self.ordem):
            self.indice=(self.indice+1)%len(self.ordem); tentativas+=1
        return self.ordem[self.indice] if self.ordem[self.indice].vivo else None

    def proximo(self) -> Personagem | None:
        if not self.ordem:
            return None
        if self.pausado:
            return self.atual
        atual = self.fila.avancar(aceitar=lambda personagem: personagem.vivo)
        if atual is None:
            self._emitir_encerramento_se_necessario()
            return None
        if self.terminou:
            self._emitir_encerramento_se_necessario()
        else:
            self._emitir("rpg.combate.turno", personagem=atual)
        return atual

    def atacar(
        self,
        atacante: Personagem,
        alvo: Personagem,
        dano: int,
        *,
        defesa: int | None = None,
        chance_critico: float = 0.0,
        multiplicador_critico: float = 2.0,
        exigir_turno: bool = True,
    ) -> ResultadoAtaque:
        if not self.iniciado:
            raise RuntimeError("combate ainda não foi iniciado")
        if self.pausado:
            raise RuntimeError("combate está pausado")
        if atacante not in self.participantes or alvo not in self.participantes:
            raise ValueError("atacante e alvo precisam participar do combate")
        if exigir_turno and self.atual is not atacante:
            raise RuntimeError("não é o turno deste personagem")
        if not atacante.vivo:
            raise RuntimeError("personagem derrotado não pode atacar")
        if not alvo.vivo:
            raise RuntimeError("alvo derrotado não pode receber ataque")
```

```
        dano_base = max(0, int(dano))
        defesa_final = max(0, int(alvo.valor_atributo("defesa", 0) if defesa is None
          else defesa))
        chance = float(chance_critico)
        multiplicador = float(multiplicador_critico)
        if not math.isfinite(chance) or not 0.0 <= chance <= 1.0:
            raise ValueError("chance de crítico deve ficar entre 0 e 1")
        if not math.isfinite(multiplicador) or multiplicador < 1.0:
            raise ValueError("multiplicador de crítico deve ser finito e maior ou igual
              a 1")
        critico = self._gerador.random() < chance
        bruto = dano_base * (multiplicador if critico else 1.0)
        calculado = max(0, int(round(bruto - defesa_final)))
        if self.mundo is not None:
            evento = self.mundo.emitir_evento_propagado(
                "rpg.combate.ataque.antes",
                combate=self,
                atacante=atacante,
                alvo=alvo,
                dano_base=dano_base,
                defesa=defesa_final,
                critico=critico,
                dano_calculado=calculado,
                rodada=self.rodada,
                mundo=self.mundo,
                cancelavel=True,
            )
            if evento.cancelado:
                return ResultadoAtaque(
                    atacante, alvo, dano_base, defesa_final, critico, multiplicador,
                      calculado, 0, True
                )
        aplicado = alvo.receber_dano(calculado)
        resultado = ResultadoAtaque(
            atacante, alvo, dano_base, defesa_final, critico, multiplicador, calculado,
              aplicado, False
        )
        if self.mundo is not None:
            self.mundo.emitir_propagado(
                "rpg.combate.ataque",
                combate=self,
                resultado=resultado,
                atacante=atacante,
                alvo=alvo,
                quantidade=aplicado,
                rodada=self.rodada,
                mundo=self.mundo,
            )
        self._emitir_encerramento_se_necessario()
        return resultado

    def encerrar(self) -> None:
        self._encerrado_manual = True
        self._emitir_encerramento_se_necessario()

    @property
    def terminou(self) -> bool:
        if self._encerrado_manual:
            return True
        vivos = sum(1 for p in self.participantes if p.vivo)
        if self.encerramento == "manual":
```

```
            return False
        if self.encerramento == "sem_vivos":
            return vivos == 0
        return vivos <= 1


def criar_personagem(mundo: Mundo, nome: str, vida: int = 10, **atributos: Any) ->
  Personagem:
    p=Personagem(nome,vida,**atributos); mundo.adicionar(p); return p


def _personagem_para_dados(personagem: Personagem) -> dict[str, Any]:
    return {
        "nome": personagem.nome,
        "identificador": personagem.identificador,
        "vida": personagem.vida,
        "vida_max": personagem.vida_max,
        "atributos": dict(personagem.atributos),
        "tags": sorted(personagem.tags),
        "inventario": list(personagem.inventario),
        "estados": sorted(personagem.estados),
        "imunidades_efeitos": sorted(personagem.imunidades_efeitos),
        "recursos": [recurso for nome, recurso in personagem.recursos.items() if nome !=
           "vida"],
        "atributos_formais": list(personagem.atributos_formais.values()),
        "efeitos": [efeito for efeito in personagem.efeitos if efeito.persistente],
    }


def _personagem_de_dados(dados: dict[str, Any]) -> Personagem:
    atributos = dados.get("atributos", {})
    if not isinstance(dados.get("nome"), str) or not isinstance(atributos, dict):
        raise ValueError("payload de Personagem inválido")
    p = Personagem(
        dados["nome"],
        vida=int(dados.get("vida", 10)),
        vida_max=int(dados.get("vida_max", dados.get("vida", 10))),
        **atributos,
    )
    if dados.get("identificador") is not None:
        p.identificador = str(dados["identificador"])
    tags = dados.get("tags", [])
    inventario = dados.get("inventario", [])
    estados = dados.get("estados", [])
    recursos = dados.get("recursos", [])
    atributos_formais = dados.get("atributos_formais", [])
    efeitos = dados.get("efeitos", [])
    if not all(isinstance(item, list) for item in (tags, inventario, estados, recursos,
      atributos_formais, efeitos)):
        raise ValueError("payload de Personagem inválido")
    p.tags.update(str(x) for x in tags)
    p.inventario.extend(inventario)
    p.estados.update(str(x) for x in estados)
    p.imunidades_efeitos.update(str(x) for x in dados.get("imunidades_efeitos", []))
    for recurso in recursos:
        if not isinstance(recurso, Recurso):
            raise ValueError("payload de Personagem contém recurso inválido")
        p.adicionar_recurso(recurso)
    for atributo in atributos_formais:
        if not isinstance(atributo, Atributo):
            raise ValueError("payload de Personagem contém atributo inválido")
```

```
        p.adicionar_atributo(atributo)
    for efeito in efeitos:
        if not isinstance(efeito, Efeito):
            raise ValueError("payload de Personagem contém efeito inválido")
        p.aplicar_efeito(efeito)
    return p


def _combate_para_dados(combate: CombateTurnos) -> dict[str, Any]:
    return {
        "nome": combate.nome,
        "encerramento": combate.encerramento,
        "encerrado_manual": combate._encerrado_manual,
        "participantes": list(combate.participantes),
        "ordem": [p.nome for p in combate.ordem],
        "indice": combate.indice,
        "rodada": combate.rodada,
        "iniciado": combate.iniciado,
        "pausado": combate.pausado,
        "estado_aleatorio": combate._gerador.getstate(),
    }


def _combate_de_dados(dados: dict[str, Any]) -> CombateTurnos:
    from .persistencia import ErroPersistencia

    participantes = dados.get("participantes", [])
    nomes_ordem = dados.get("ordem", [])
    if not isinstance(participantes, list) or not all(isinstance(p, Personagem) for p in
       participantes):
        raise ErroPersistencia("Combate isolado possui participantes inválidos.")
    if not isinstance(nomes_ordem, list) or not all(isinstance(nome, str) for nome in
      nomes_ordem):
        raise ErroPersistencia("Combate isolado possui ordem inválida.")

    por_nome: dict[str, Personagem] = {}
    for personagem in participantes:
        if personagem.nome in por_nome:
            raise ErroPersistencia(f"Combate isolado possui participantes duplicados
              chamados {personagem.nome!r}.")
        por_nome[personagem.nome] = personagem

    ordem: list[Personagem] = []
    for nome in nomes_ordem:
        personagem = por_nome.get(nome)
        if personagem is None:
            raise ErroPersistencia(f"Combate isolado referencia participante ausente
              {nome!r} na ordem.")
        ordem.append(personagem)

    combate = CombateTurnos(
        participantes,
        nome=dados.get("nome"),
        encerramento=dados.get("encerramento", "ultimo_vivo"),
    )
    combate._encerrado_manual = bool(dados.get("encerrado_manual", False))
    combate.ordem = ordem
    try:
        combate.indice = int(dados.get("indice", 0))
        combate.rodada = int(dados.get("rodada", 0))
    except (TypeError, ValueError) as exc:
```

```
        raise ErroPersistencia("Combate isolado possui estado numérico inválido.") from
          exc
    combate.iniciado = bool(dados.get("iniciado", False))
    combate.fila.pausada = bool(dados.get("pausado", False))
    estado_aleatorio = dados.get("estado_aleatorio")
    if estado_aleatorio is not None:
        try:
            combate._gerador.setstate(estado_aleatorio)
        except (TypeError, ValueError) as exc:
            raise ErroPersistencia("Combate isolado possui estado aleatório inválido.")
              from exc
    if combate.ordem and not (0 <= combate.indice < len(combate.ordem)):
        raise ErroPersistencia("Combate isolado possui índice de turno inválido.")
    if not combate.ordem and combate.indice != 0:
        raise ErroPersistencia("Combate isolado possui índice sem ordem de turnos.")
    return combate


def _restaurar_combates_contextuais(mundo: Mundo, combates: Any) -> None:
    from .persistencia import ErroPersistencia
    if not isinstance(combates, list):
        raise ErroPersistencia("Payload de combates RPG inválido.")

    def resolver_personagem(nome_entidade: Any, nome_combate: str, campo: str) ->
      Personagem:
        if not isinstance(nome_entidade, str):
            raise ErroPersistencia(f"Combate {nome_combate!r} possui referência inválida
               em {campo}.")
        entidade = mundo.buscar(nome_entidade)
        if entidade is None:
            raise ErroPersistencia(
                f"Combate {nome_combate!r} referencia entidade inexistente
                  {nome_entidade!r} em {campo}."
            )
        if not isinstance(entidade, Personagem):
            raise ErroPersistencia(f"Combate {nome_combate!r} exige Personagem em
              {campo}: {nome_entidade!r}.")
        return entidade

    for indice_combate, bruto in enumerate(combates):
        if not isinstance(bruto, dict) or not isinstance(bruto.get("nome"), str):
            raise ErroPersistencia(f"Combate inválido no índice {indice_combate}.")
        nome_combate = bruto["nome"]
        nomes_participantes = bruto.get("participantes", [])
        nomes_ordem = bruto.get("ordem", [])
        if not isinstance(nomes_participantes, list) or not isinstance(nomes_ordem,
          list):
            raise ErroPersistencia(f"Combate {nome_combate!r} possui participantes ou
              ordem inválidos.")
        participantes = [resolver_personagem(n, nome_combate, "participantes") for n in
          nomes_participantes]
        ordem = [resolver_personagem(n, nome_combate, "ordem") for n in nomes_ordem]
        combate = CombateTurnos(
            participantes,
            nome=nome_combate,
            encerramento=bruto.get("encerramento", "ultimo_vivo"),
        )
        combate._encerrado_manual = bool(bruto.get("encerrado_manual", False))
        combate.ordem = ordem
        try:
            combate.indice = int(bruto.get("indice", 0))
```

```
            combate.rodada = int(bruto.get("rodada", 0))
        except (TypeError, ValueError) as exc:
            raise ErroPersistencia(f"Combate {nome_combate!r} possui estado numérico
              inválido.") from exc
        combate.iniciado = bool(bruto.get("iniciado", False))
        combate.fila.pausada = bool(bruto.get("pausado", False))
        estado_aleatorio = bruto.get("estado_aleatorio")
        if estado_aleatorio is not None:
            try:
                combate._gerador.setstate(estado_aleatorio)
            except (TypeError, ValueError) as exc:
                raise ErroPersistencia(f"Combate {nome_combate!r} possui estado
                  aleatório inválido.") from exc
        if combate.ordem and not (0 <= combate.indice < len(combate.ordem)):
            raise ErroPersistencia(f"Combate {nome_combate!r} possui índice de turno
              inválido.")
        if not combate.ordem and combate.indice != 0:
            raise ErroPersistencia(f"Combate {nome_combate!r} possui índice sem ordem de
               turnos.")


@dataclass(frozen=True)
class EstadoRPG:
    """Agregado persistente do domínio RPG sobre um Mundo genérico."""

    mundo: Mundo

    def __post_init__(self) -> None:
        if not isinstance(self.mundo, Mundo):
            raise TypeError("EstadoRPG precisa de um Mundo")


def estado_rpg(mundo: Mundo) -> EstadoRPG:
    return EstadoRPG(mundo)


def _combate_contextual_para_dados(combate: CombateTurnos) -> dict[str, Any]:
    return {
        "nome": combate.nome,
        "encerramento": combate.encerramento,
        "encerrado_manual": combate._encerrado_manual,
        "participantes": [p.nome for p in combate.participantes],
        "ordem": [p.nome for p in combate.ordem],
        "indice": combate.indice,
        "rodada": combate.rodada,
        "iniciado": combate.iniciado,
        "pausado": combate.pausado,
        "estado_aleatorio": combate._gerador.getstate(),
    }


def _estado_rpg_para_dados(estado: EstadoRPG) -> dict[str, Any]:
    return {"mundo": estado.mundo}


def _estado_rpg_de_dados(dados: dict[str, Any]) -> EstadoRPG:
    from .persistencia import ErroPersistencia
    mundo = dados.get("mundo")
    if not isinstance(mundo, Mundo):
        raise ErroPersistencia("Payload de EstadoRPG inválido.")
    return EstadoRPG(mundo)
```

```
def salvar_estado_rpg(mundo: Mundo, caminho: str | Path) -> Path:
    from .persistencia import salvar
    return salvar(estado_rpg(mundo), caminho)


def carregar_estado_rpg(caminho: str | Path) -> Mundo:
    from .persistencia import carregar_como
    return carregar_como(caminho, EstadoRPG, "EstadoRPG").mundo


def _combates_extensao_para_dados(mundo: Mundo):
    combates = combates_do_mundo(mundo)
    if not combates:
        return None
    return [_combate_contextual_para_dados(c) for c in combates]


def _aplicar_combates_extensao(mundo: Mundo, dados: Any) -> None:
    _restaurar_combates_contextuais(mundo, dados)


from .persistencia import registrar_extensao_persistente as
  _registrar_extensao_persistente
_registrar_extensao_persistente(Mundo, "coral.rpg/combates",
  _combates_extensao_para_dados, _aplicar_combates_extensao)


# Compatibilidade com a superfície histórica: a implementação pertence a RPG.
def _compat_registrar_combate(self: Mundo, nome: str, combate: CombateTurnos):
    return registrar_combate(self, nome, combate)


def _compat_buscar_combate(self: Mundo, nome: str):
    return buscar_combate(self, nome)


if not hasattr(Mundo, "registrar_combate"):
    setattr(Mundo, "registrar_combate", _compat_registrar_combate)
if not hasattr(Mundo, "buscar_combate"):
    setattr(Mundo, "buscar_combate", _compat_buscar_combate)
if not hasattr(Mundo, "combates"):
    setattr(Mundo, "combates", property(lambda self: combates_do_mundo(self)))


from .persistencia import registrar_adaptador as _registrar_adaptador
_registrar_adaptador(Personagem, "coral.rpg/Personagem", _personagem_para_dados,
  _personagem_de_dados)
_registrar_adaptador(CombateTurnos, "coral.rpg/CombateTurnos", _combate_para_dados,
  _combate_de_dados)
_registrar_adaptador(EstadoRPG, "coral.rpg/EstadoRPG", _estado_rpg_para_dados,
  _estado_rpg_de_dados)


__all__ = [
    "Rolagem", "ResultadoAtaque", "rolar", "Personagem", "CombateTurnos",
      "criar_personagem",
    "registrar_combate", "buscar_combate", "combates_do_mundo",
    "EstadoRPG", "estado_rpg", "salvar_estado_rpg", "carregar_estado_rpg",
]
```

```
"""Fila genérica de turnos e escalonamento da Coral 1.5.10."""
from __future__ import annotations

from collections.abc import Callable, Iterable, Mapping
from typing import Any


class FilaTurnos:
    """Fila circular persistível sem conhecimento de RPG ou combate.

    A fila pode representar turnos, atendimento, produção ou escalonamento.
    Participantes são objetos arbitrários portáteis para a persistência Coral.
    """

    def __init__(self, participantes: Iterable[Any] = ()) -> None:
        self.participantes: list[Any] = []
        for participante in participantes:
            if participante not in self.participantes:
                self.participantes.append(participante)
        self.ordem: list[Any] = []
        self.indice = 0
        self.rodada = 0
        self.iniciada = False
        self.pausada = False
        self.historico: list[dict[str, Any]] = []

    @staticmethod
    def identidade(participante: Any) -> str:
        for nome in ("identificador", "nome"):
            valor = getattr(participante, nome, None)
            if valor is not None:
                return str(valor)
        return repr(participante)

    def adicionar(self, participante: Any) -> Any:
        if participante not in self.participantes:
            self.participantes.append(participante)
            if self.iniciada:
                self.ordem.append(participante)
            self.historico.append({"acao": "adicionar", "participante":
              self.identidade(participante)})
        return participante

    def remover(self, participante: Any) -> bool:
        if participante not in self.participantes:
            return False
        self.participantes.remove(participante)
        if participante in self.ordem:
            posicao = self.ordem.index(participante)
            del self.ordem[posicao]
            if not self.ordem:
                self.indice = 0
            elif posicao < self.indice:
                self.indice -= 1
            elif self.indice >= len(self.ordem):
                self.indice = 0
        self.historico.append({"acao": "remover", "participante":
          self.identidade(participante)})
        return True
```

```
    def iniciar(
        self,
        *,
        ordem: Iterable[Any] | None = None,
        prioridades: Mapping[Any, float] | Callable[[Any], float] | None = None,
    ) -> tuple[Any, ...]:
        if ordem is not None and prioridades is not None:
            raise ValueError("use ordem explícita ou prioridades, não ambos")
        if ordem is not None:
            escolhida = list(ordem)
            if any(p not in self.participantes for p in escolhida):
                raise ValueError("ordem só pode conter participantes da fila")
            if len({id(p) for p in escolhida}) != len(escolhida):
                raise ValueError("ordem não pode repetir participante")
        elif prioridades is not None:
            def prioridade(participante: Any) -> float:
                if callable(prioridades):
                    return float(prioridades(participante))
                try:
                    return float(prioridades[participante])
                except (KeyError, TypeError):
                    identidade = self.identidade(participante)
                    try:
                        return float(prioridades[identidade])
                    except KeyError:
                        return 0.0
            enumerados = list(enumerate(self.participantes))
            enumerados.sort(key=lambda item: (-prioridade(item[1]), item[0]))
            escolhida = [participante for _, participante in enumerados]
        else:
            escolhida = list(self.participantes)
        self.ordem = escolhida
        self.indice = 0
        self.rodada = 1 if self.ordem else 0
        self.iniciada = True
        self.pausada = False
        self.historico.append({"acao": "iniciar", "ordem": [self.identidade(p) for p in
          self.ordem]})
        return tuple(self.ordem)

    @property
    def atual(self) -> Any | None:
        if not self.iniciada or not self.ordem:
            return None
        if self.indice >= len(self.ordem):
            self.indice = 0
        return self.ordem[self.indice]

    def avancar(self, *, aceitar: Callable[[Any], bool] | None = None) -> Any | None:
        if self.pausada or not self.ordem:
            return self.atual
        aceitar = aceitar or (lambda _participante: True)
        for _ in range(len(self.ordem)):
            self.indice = (self.indice + 1) % len(self.ordem)
            if self.indice == 0:
                self.rodada += 1
            participante = self.ordem[self.indice]
            if aceitar(participante):
                self.historico.append({
                    "acao": "avancar",
                    "participante": self.identidade(participante),
```

```
                    "rodada": self.rodada,
                })
                return participante
        return None

    def pausar(self) -> None:
        if not self.pausada:
            self.pausada = True
            self.historico.append({"acao": "pausar"})

    def retomar(self) -> None:
        if self.pausada:
            self.pausada = False
            self.historico.append({"acao": "retomar"})

    def limpar_historico(self) -> None:
        self.historico.clear()


def _fila_para_dados(fila: FilaTurnos) -> dict[str, Any]:
    indices = {id(participante): indice for indice, participante in
      enumerate(fila.participantes)}
    try:
        ordem = [indices[id(participante)] for participante in fila.ordem]
    except KeyError as exc:
        raise ValueError("fila possui participante na ordem que não pertence à lista
          principal") from exc
    return {
        "participantes": list(fila.participantes),
        "ordem": ordem,
        "indice": fila.indice,
        "rodada": fila.rodada,
        "iniciada": fila.iniciada,
        "pausada": fila.pausada,
        "historico": list(fila.historico),
    }


def _fila_de_dados(dados: dict[str, Any]) -> FilaTurnos:
    participantes = dados.get("participantes", [])
    ordem_indices = dados.get("ordem", [])
    if not isinstance(participantes, list) or not isinstance(ordem_indices, list):
        raise ValueError("payload de FilaTurnos inválido")
    fila = FilaTurnos(participantes)
    try:
        fila.ordem = [participantes[int(indice)] for indice in ordem_indices]
        fila.indice = int(dados.get("indice", 0))
        fila.rodada = int(dados.get("rodada", 0))
    except (TypeError, ValueError, IndexError) as exc:
        raise ValueError("payload de FilaTurnos possui estado inválido") from exc
    fila.iniciada = bool(dados.get("iniciada", False))
    fila.pausada = bool(dados.get("pausada", False))
    historico = dados.get("historico", [])
    if not isinstance(historico, list):
        raise ValueError("histórico de FilaTurnos inválido")
    fila.historico = list(historico)
    if fila.ordem and not (0 <= fila.indice < len(fila.ordem)):
        raise ValueError("índice de FilaTurnos inválido")
    if not fila.ordem and fila.indice != 0:
        raise ValueError("FilaTurnos vazia não pode possuir índice de turno")
    return fila
```

```
from .persistencia import registrar_adaptador as _registrar_adaptador

_registrar_adaptador(FilaTurnos, "coral.turnos/FilaTurnos", _fila_para_dados,
  _fila_de_dados)

__all__ = ["FilaTurnos"]
```

Biblioteca `coral.sistema`

`coral.sistema.ComandoFalhou`

Código de saída não zero preservando o resultado completo do processo.

Assinatura

Assinatura para consulta

```
ComandoFalhou(resultado: "'ResultadoComando'")
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
resultado	'ResultadoComando'	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.sistema.ErroSistema`

Falha operacional do sistema hospedeiro traduzida para a superfície Coral.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.sistema.FalhaSistemaOperacional`

Demais falhas do SO, mantendo compatibilidade com captura por OSError.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.sistema.InformacoesSistema`

InformacoesSistema(sistema: 'str', versao: 'str', arquitetura: 'str', maquina: 'str', processador: 'str', python: 'str', pid: 'int', pasta_atual: 'str')

Assinatura

Assinatura para consulta

```
InformacoesSistema(sistema: 'str', versao: 'str', arquitetura: 'str', maquina: 'str',
  processador: 'str', python: 'str', pid: 'int', pasta_atual: 'str') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
sistema	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
versao	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
arquitetura	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
maquina	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
processador	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
python	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
pid	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
pasta_atual	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `sistema`: `str`.

Atributo `versao`: `str`.

Atributo `arquitetura`: `str`.

Atributo `maquina`: `str`.

Atributo `processador`: `str`.

Atributo `python`: `str`.

Atributo `pid`: `int`.

Atributo `pasta_atual`: `str`.

Métodos públicos

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

`coral.sistema.ProgramaNaoEncontrado`

Executável ausente, compatível com o FileNotFoundError exposto até a 1.4.2.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.sistema.ResultadoComando`

ResultadoComando(comando: 'tuple[str, ...]', codigo: 'int', saida: 'str', erro: 'str', duracao_segundos: 'float', saida_truncada: 'bool' = False, erro_truncado: 'bool' = False)

Assinatura

Assinatura para consulta

```
ResultadoComando(comando: 'tuple[str, ...]', codigo: 'int', saida: 'str', erro: 'str',
  duracao_segundos: 'float', saida_truncada: 'bool' = False, erro_truncado: 'bool' =
  False) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
comando	tuple[str, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
codigo	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
saida	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
erro	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
duracao_segundos	float	POSITIONAL_OR_KEYWORD	sim	sem padrão
saida_truncada	bool	POSITIONAL_OR_KEYWORD	não	False
erro_truncado	bool	POSITIONAL_OR_KEYWORD	não	False

Retorno declarado

None

Atributo `comando`: `tuple[str, ...]`.

Atributo `codigo`: `int`.

Atributo `saida`: `str`.

Atributo `erro`: `str`.

Atributo `duracao_segundos`: `float`.

Atributo `saida_truncada`: `bool`.

Atributo `erro_truncado`: `bool`.

Métodos públicos

Referência para leitura

```
codigo_saida(self) -> 'int'
```

Alias de leitura em português corrente para `codigo`.

Referência para leitura

```
exigir_sucesso(self) -> "'ResultadoComando'"
```

Retorna este resultado ou produz uma falha Coral em código não zero.

Referência para leitura

```
linhas_erro(self) -> 'tuple[str, ...]'
```

Linhas do erro capturado sem modificar `erro`.

Referência para leitura

```
linhas_saida(self) -> 'tuple[str, ...]'
```

Linhas da saída capturada sem modificar `saida`.

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
resultado_operacao(self) -> 'ResultadoOperacao'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
tempo_decorrido(self) -> 'float'
```

Alias de leitura para a duração medida na execução.

`coral.sistema.TempoLimiteExcedido`

Timeout de processo, compatível com subprocess.TimeoutExpired da linha 1.4.1.

Assinatura

Assinatura para consulta

```
TempoLimiteExcedido(comando: 'tuple[str, ...]', timeout: 'float | None', mensagem:
  'str')
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
comando	tuple[str, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
timeout	float | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.sistema.ambiente_selecionado`

Copia somente as variáveis explicitamente solicitadas que estejam presentes.

Assinatura

Assinatura para consulta

```
ambiente_selecionado(nomes: 'Iterable[Any]') -> 'dict[str, str]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nomes	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

dict[str, str]

`coral.sistema.argumentos_programa`

Cópia dos argumentos do programa em execução, sem o caminho do próprio programa.

Assinatura

Assinatura para consulta

```
argumentos_programa() -> 'tuple[str, ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

tuple[str, ...]

`coral.sistema.caminho`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
caminho(*partes: 'Any') -> 'Path'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
partes	Any	VAR_POSITIONAL	não	sem padrão

Retorno declarado

Path

`coral.sistema.caminho_executavel`

Caminho do executável que hospeda o runtime, quando o processo o informa.

Assinatura

Assinatura para consulta

```
caminho_executavel() -> 'Path | None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

Path | None

`coral.sistema.definir_variavel_ambiente`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
definir_variavel_ambiente(nome: 'str', valor: 'Any') -> 'None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
valor	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

`coral.sistema.diagnosticar_sistema`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
diagnosticar_sistema() -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

dict[str, Any]

`coral.sistema.executar_comando`

Executa um processo sem shell e com captura opcionalmente limitada.

Quando `limite_saida_bytes` é informado, stdout e stderr recebem o mesmo teto individual. O excedente continua sendo drenado para não bloquear o processo, mas não é mantido em memória. Timeout tem prioridade sobre truncamento e código de saída.

Assinatura

Assinatura para consulta

```
executar_comando(programa: 'str', argumentos: 'Iterable[Any]' = (), *, pasta: 'str |
  Path | None' = None, timeout: 'float | None' = None, ambiente: 'Mapping[str, Any] |
  None' = None, entrada: 'str | None' = None, relogio: 'FonteTempo | None' = None,
  codificacao: 'str | None' = None, erros_decodificacao: 'str' = 'estrito',
  limite_saida_bytes: 'int | None' = None) -> 'ResultadoComando'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
programa	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
argumentos	Iterable[Any]	POSITIONAL_OR_KEYWORD	não	()
pasta	str | Path | None	KEYWORD_ONLY	não	None
timeout	float | None	KEYWORD_ONLY	não	None
ambiente	Mapping[str, Any] | None	KEYWORD_ONLY	não	None
entrada	str | None	KEYWORD_ONLY	não	None
relogio	FonteTempo | None	KEYWORD_ONLY	não	None
codificacao	str | None	KEYWORD_ONLY	não	None
erros_decodificacao	str	KEYWORD_ONLY	não	'estrito'
limite_saida_bytes	int | None	KEYWORD_ONLY	não	None

Retorno declarado

ResultadoComando

`coral.sistema.informacoes`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
informacoes() -> 'InformacoesSistema'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

InformacoesSistema

`coral.sistema.localizar_programa`

Resolve um executável no ambiente atual sem iniciá lo.

Assinatura

Assinatura para consulta

```
localizar_programa(nome: 'str') -> 'Path | None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

Path | None

`coral.sistema.pasta_atual`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
pasta_atual() -> 'Path'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

Path

`coral.sistema.pasta_temporaria`

Pasta temporária configurada para o processo, sem criar arquivos.

Assinatura

Assinatura para consulta

```
pasta_temporaria() -> 'Path'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

Path

`coral.sistema.pasta_usuario`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
pasta_usuario() -> 'Path'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

Path

`coral.sistema.processo_atual`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
processo_atual() -> 'dict[str, Any]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

dict[str, Any]

`coral.sistema.programa_disponivel`

Indica se `` `localizar_programa` `` consegue resolver o executável.

Assinatura

Assinatura para consulta

```
programa_disponivel(nome: 'str') -> 'bool'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

bool

`coral.sistema.quantidade_processadores`

Quantidade lógica observada pelo runtime, sem estimar quotas efetivas.

Assinatura

Assinatura para consulta

```
quantidade_processadores() -> 'int | None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

int | None

`coral.sistema.remover_variavel_ambiente`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
remover_variavel_ambiente(nome: 'str') -> 'bool'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

bool

`coral.sistema.terminal_interativo`

Informa se entrada, saída ou erro estão conectados a um terminal.

Assinatura

Assinatura para consulta

```
terminal_interativo(canal: 'str') -> 'bool'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
canal	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

bool

`coral.sistema.variaveis_ambiente`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
variaveis_ambiente() -> 'dict[str, str]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão

Retorno declarado

dict[str, str]

`coral.sistema.variavel_ambiente`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
variavel_ambiente(nome: 'str', padrao: 'str | None' = None) -> 'str | None'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
padrao	str | None	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

str | None

`coral.sistema.variavel_ambiente_obrigatoria`

Lê uma variável obrigatória sem expor valores em mensagens de erro.

Assinatura

Assinatura para consulta

```
variavel_ambiente_obrigatoria(nome: 'str') -> 'str'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

str

Consultar a implementação canônica de coral.sistema

Implementação canônica de `coral.sistema`

Código Python

```
"""Contrato único de caminhos usado pelas APIs públicas da Coral.

Texto recebido por uma operação local é interpretado como caminho nativo do
sistema atual. Objetos ``PurePath`` de outro estilo continuam sendo caminhos
portáteis, mas não são usados silenciosamente em operações locais.
"""
from __future__ import annotations

import os
from pathlib import Path, PurePath, PurePosixPath, PureWindowsPath
from typing import Any


CaminhoAceito = str | os.PathLike[str]


def estilo_caminho(valor: PurePath) -> str:
    if isinstance(valor, PureWindowsPath):
        return "windows"
    return "posix"


def estilo_nativo() -> str:
    return "windows" if os.name == "nt" else "posix"



def normalizar_caminho(
    valor: CaminhoAceito,
    *,
    expandir_usuario: bool = False,
    resolver: bool = False,
) -> Path:
    """Converte uma entrada para ``Path`` nativo com regras explícitas.

    Caminhos puros de outro sistema são recusados em operações locais. Isso
    evita transformar silenciosamente ``C:\\dados`` em um caminho relativo em
    um host POSIX, ou o inverso.
    """

    if isinstance(valor, PurePath) and estilo_caminho(valor) != estilo_nativo():
        raise ValueError(
            f"caminho de estilo {estilo_caminho(valor)} não pode ser usado "
            f"como caminho local em sistema {estilo_nativo()}"
        )
    try:
        caminho = Path(os.fspath(valor))
    except (TypeError, ValueError) as erro:
        raise TypeError("caminho precisa ser texto ou objeto compatível com
          os.PathLike") from erro
    if expandir_usuario:
        caminho = caminho.expanduser()
    if resolver:
        caminho = caminho.resolve()
    return caminho


def juntar_caminho(*partes: Any, expandir_usuario: bool = True) -> Path:
    if not partes:
        caminho = Path()
```

```
    else:
        caminho = Path(*(os.fspath(p) if isinstance(p, os.PathLike) else str(p) for p in
           partes))
    return caminho.expanduser() if expandir_usuario else caminho


def caminho_portatil(texto: str, estilo: str, *, concreto: bool = True) -> PurePath:
    """Reconstrói a categoria caminho sem fingir compatibilidade entre SOs."""

    estilo = str(estilo).casefold()
    if estilo not in {"posix", "windows"}:
        raise ValueError(f"estilo de caminho desconhecido: {estilo!r}")
    puro: PurePath = PureWindowsPath(texto) if estilo == "windows" else
      PurePosixPath(texto)
    if concreto and estilo == estilo_nativo():
        return Path(puro)
    return puro


__all__ = [
    "CaminhoAceito",
    "normalizar_caminho",
    "juntar_caminho",
    "estilo_caminho",
    "estilo_nativo",
    "caminho_portatil",
]


def _arquivo_chamador(base: CaminhoAceito | None = None) -> Path:
    if base is not None:
        caminho = normalizar_caminho(base, resolver=True)
        return caminho if caminho.suffix else caminho / "__coral_contexto__.py"
    import inspect
    este = Path(__file__).resolve()
    for quadro in inspect.stack()[1:]:
        candidato = Path(quadro.filename).resolve()
        if candidato != este:
            return candidato
    return Path.cwd().resolve() / "__coral_contexto__.py"


def _achar_raiz_contexto(arquivo: Path) -> tuple[Path, bool]:
    atual = arquivo.parent if arquivo.suffix else arquivo
    for pasta in (atual, *atual.parents):
        if (pasta / ".coral_build_manifest.json").is_file():
            return pasta, True
        if (pasta / "coral.toml").is_file():
            return pasta, False
    raise FileNotFoundError("raiz de projeto Coral não encontrada a partir do módulo
      atual")


def raiz_projeto(*, base: CaminhoAceito | None = None) -> Path:
    """Retorna a raiz portátil do projeto fonte ou do build movido."""
    raiz, _ = _achar_raiz_contexto(_arquivo_chamador(base))
    return raiz


def caminho_modulo(
    relativo: CaminhoAceito | None = None,
```

```
    *,
    base: CaminhoAceito | None = None,
) -> Path:
    """Retorna a pasta do módulo chamador, opcionalmente acrescida de um caminho."""
    arquivo = _arquivo_chamador(base)
    pasta = arquivo.parent
    return pasta if relativo is None else pasta / normalizar_caminho(relativo)


def caminho_recurso(relativo: CaminhoAceito, *, base: CaminhoAceito | None = None) ->
  Path:
    """Resolve um recurso declarado tanto na fonte quanto em um build movido."""
    arquivo = _arquivo_chamador(base)
    raiz, em_build = _achar_raiz_contexto(arquivo)
    rel = normalizar_caminho(relativo)
    if rel.is_absolute() or ".." in rel.parts:
        raise ValueError("caminho de recurso deve ser relativo e permanecer dentro do
          projeto")
    return raiz / ("_recursos" if em_build else "") / rel


__all__ += ["raiz_projeto", "caminho_modulo", "caminho_recurso"]

"""Contratos transversais pequenos do runtime Coral.

A camada comum não substitui APIs de domínio. Ela fornece a gramática
arquitetural compartilhada do ciclo 1.5 para erro operacional, resultado e
diagnóstico, preservando os nomes públicos existentes em cada biblioteca.
"""
from __future__ import annotations

from dataclasses import asdict, dataclass, field
from typing import Any, Iterable, Mapping

CONTRATO = "coral.comum/1"
CONTRATO_DIAGNOSTICO = "coral.diagnostico.operacional/1"


class ErroCoral(Exception):
    """Raiz pública dos erros controlados pela linguagem e bibliotecas Coral."""


class ErroOperacaoCoral(RuntimeError, ErroCoral):
    """Falha operacional de um recurso Coral ou de seu backend."""


class ErroDependenciaCoral(ErroOperacaoCoral):
    """Uma capacidade opcional não pode operar por ausência de dependência."""


@dataclass(frozen=True)
class FalhaOperacao:
    codigo: str
    mensagem: str
    dominio: str
    tipo: str = "operacional"
    causa: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def de_excecao(cls, exc: BaseException, *, codigo: str, dominio: str, tipo: str =
```

```
      "operacional", detalhe: Mapping[str, Any] | None = None) -> "FalhaOperacao":
        causa = exc.__cause__ if exc.__cause__ is not None else exc
        return cls(codigo, str(exc), dominio, tipo, type(causa).__name__, dict(detalhe
          or {}))

    def para_dict(self) -> dict[str, Any]:
        return asdict(self)


@dataclass(frozen=True)
class ResultadoOperacao:
    """Representação transversal sem substituir resultados ricos de domínio."""
    ok: bool
    valor: Any = None
    falha: FalhaOperacao | None = None
    metadados: Mapping[str, Any] = field(default_factory=dict)

    @classmethod
    def sucesso(cls, valor: Any = None, **metadados: Any) -> "ResultadoOperacao":
        return cls(True, valor=valor, metadados=metadados)

    @classmethod
    def erro(cls, falha: FalhaOperacao, **metadados: Any) -> "ResultadoOperacao":
        return cls(False, falha=falha, metadados=metadados)

    def para_dict(self) -> dict[str, Any]:
        return {
            "contrato": CONTRATO,
            "ok": self.ok,
            "valor": self.valor,
            "falha": self.falha.para_dict() if self.falha is not None else None,
            "metadados": dict(self.metadados),
        }


@dataclass(frozen=True)
class DiagnosticoOperacional:
    id: str
    dominio: str
    ok: bool
    mensagem: str
    capacidade: str | None = None
    sugestao: str | None = None
    detalhe: Mapping[str, Any] = field(default_factory=dict)

    @property
    def nivel(self) -> str:
        return "info" if self.ok else "erro"

    def para_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "dominio": self.dominio,
            "ok": self.ok,
            "nivel": self.nivel,
            "mensagem": self.mensagem,
            "capacidade": self.capacidade,
            "sugestao": self.sugestao,
            "detalhe": dict(self.detalhe),
        }
```

```
def diagnostico(id: str, dominio: str, ok: bool, mensagem: Any, *, capacidade: str |
  None = None, sugestao: str | None = None, detalhe: Mapping[str, Any] | None = None) ->
  DiagnosticoOperacional:
    return DiagnosticoOperacional(str(id), str(dominio), bool(ok), str(mensagem),
      capacidade, sugestao, dict(detalhe or {}))


def agregar_diagnosticos(itens: Iterable[DiagnosticoOperacional | Mapping[str, Any]], *,
   origem: str = "coral") -> dict[str, Any]:
    normalizados: list[dict[str, Any]] = []
    for item in itens:
        if isinstance(item, DiagnosticoOperacional):
            normalizados.append(item.para_dict())
            continue
        d = dict(item)
        if "id" not in d:
            d["id"] = str(d.get("nome", "diagnostico"))
        if "dominio" not in d:
            d["dominio"] = origem
        d["ok"] = bool(d.get("ok"))
        d.setdefault("nivel", "info" if d["ok"] else "erro")
        d.setdefault("mensagem", str(d.get("detalhe", "")))
        d.setdefault("capacidade", None)
        d.setdefault("sugestao", None)
        detalhe = d.get("detalhe")
        if not isinstance(detalhe, Mapping):
            d["detalhe"] = {"valor": detalhe}
        normalizados.append(d)
    return {
        "contrato": CONTRATO_DIAGNOSTICO,
        "origem": origem,
        "ok": all(x["ok"] for x in normalizados),
        "quantidade": len(normalizados),
        "itens": normalizados,
    }


__all__ = [
    "CONTRATO", "CONTRATO_DIAGNOSTICO", "ErroCoral", "ErroOperacaoCoral",
    "ErroDependenciaCoral", "FalhaOperacao", "ResultadoOperacao",
    "DiagnosticoOperacional", "diagnostico", "agregar_diagnosticos",
]

"""Integração segura e portátil do Coral com o sistema operacional."""
from __future__ import annotations

from dataclasses import asdict, dataclass
import codecs
import locale
import os
from pathlib import Path
import platform
import shutil
import signal
import subprocess
import sys
import tempfile
import threading
import time
from typing import Any, BinaryIO, Iterable, Mapping
```

```
from .comum import ErroOperacaoCoral, FalhaOperacao, ResultadoOperacao, diagnostico
from .caminhos import juntar_caminho, normalizar_caminho
from .tempo_eventos import FonteTempo, RelogioMonotonico, validar_duracao


class ErroSistema(ErroOperacaoCoral):
    """Falha operacional do sistema hospedeiro traduzida para a superfície Coral."""


class ProgramaNaoEncontrado(ErroSistema, FileNotFoundError):
    """Executável ausente, compatível com o FileNotFoundError exposto até a 1.4.2."""


class TempoLimiteExcedido(ErroSistema, subprocess.TimeoutExpired):
    """Timeout de processo, compatível com subprocess.TimeoutExpired da linha 1.4.1."""

    def __init__(self, comando: tuple[str, ...], timeout: float | None, mensagem: str):
        # RuntimeError aparece antes de TimeoutExpired no MRO; inicializamos a
        # mensagem pública Coral e preservamos os atributos historicamente
        # consultáveis por consumidores Python.
        ErroSistema.__init__(self, mensagem)
        self.cmd = comando
        self.timeout = timeout
        self.output = None
        self.stderr = None

    def __str__(self) -> str:
        # ``subprocess.TimeoutExpired`` define um ``__str__`` em inglês. Como
        # esta exceção é uma superfície pública Coral, preservamos a mensagem
        # já normalizada em português sem perder ``cmd`` e ``timeout``.
        return str(self.args[0]) if self.args else "Tempo limite de processo excedido."


class FalhaSistemaOperacional(ErroSistema, OSError):
    """Demais falhas do SO, mantendo compatibilidade com captura por OSError."""


class ComandoFalhou(ErroSistema):
    """Código de saída não zero preservando o resultado completo do processo."""

    def __init__(self, resultado: "ResultadoComando"):
        self.resultado = resultado
        self.codigo = resultado.codigo
        self.stderr = resultado.erro
        mensagem = resultado.erro.strip() or f"Processo terminou com código
          {resultado.codigo}."
        super().__init__(f"O comando falhou com código {resultado.codigo}: {mensagem}")


@dataclass(frozen=True)
class InformacoesSistema:
    sistema: str
    versao: str
    arquitetura: str
    maquina: str
    processador: str
    python: str
    pid: int
    pasta_atual: str
```

```
    def para_dict(self) -> dict[str, Any]:
        return asdict(self)


@dataclass(frozen=True)
class ResultadoComando:
    comando: tuple[str, ...]
    codigo: int
    saida: str
    erro: str
    duracao_segundos: float
    saida_truncada: bool = False
    erro_truncado: bool = False

    @property
    def sucesso(self) -> bool:
        return self.codigo == 0

    def para_dict(self) -> dict[str, Any]:
        return asdict(self) | {"sucesso": self.sucesso}

    def linhas_saida(self) -> tuple[str, ...]:
        """Linhas da saída capturada sem modificar ``saida``."""
        return tuple(self.saida.splitlines())

    def linhas_erro(self) -> tuple[str, ...]:
        """Linhas do erro capturado sem modificar ``erro``."""
        return tuple(self.erro.splitlines())

    def codigo_saida(self) -> int:
        """Alias de leitura em português corrente para ``codigo``."""
        return self.codigo

    def tempo_decorrido(self) -> float:
        """Alias de leitura para a duração medida na execução."""
        return self.duracao_segundos

    def exigir_sucesso(self) -> "ResultadoComando":
        """Retorna este resultado ou produz uma falha Coral em código não zero."""
        if not self.sucesso:
            raise ComandoFalhou(self)
        return self

    def resultado_operacao(self) -> ResultadoOperacao:
        if self.sucesso:
            return ResultadoOperacao.sucesso(self, dominio="sistema",
              codigo_saida=self.codigo)
        falha = FalhaOperacao(
            "sistema.comando.codigo_saida", self.erro or f"Processo terminou com código
              {self.codigo}.",
            "sistema", "resultado", detalhe={"codigo": self.codigo, "comando":
              list(self.comando)},
        )
        return ResultadoOperacao.erro(falha, codigo_saida=self.codigo)


def informacoes() -> InformacoesSistema:
    return InformacoesSistema(
        sistema=platform.system() or os.name,
        versao=platform.version(),
        arquitetura=platform.machine(),
```

```
        maquina=platform.node(),
        processador=platform.processor(),
        python=platform.python_version(),
        pid=os.getpid(),
        pasta_atual=str(Path.cwd()),
    )


def variavel_ambiente(nome: str, padrao: str | None = None) -> str | None:
    return os.environ.get(str(nome), padrao)


def definir_variavel_ambiente(nome: str, valor: Any) -> None:
    os.environ[str(nome)] = str(valor)


def remover_variavel_ambiente(nome: str) -> bool:
    return os.environ.pop(str(nome), None) is not None


def variaveis_ambiente() -> dict[str, str]:
    return dict(os.environ)


def ambiente_selecionado(nomes: Iterable[Any]) -> dict[str, str]:
    """Copia somente as variáveis explicitamente solicitadas que estejam presentes."""
    if isinstance(nomes, (str, Path)):
        nomes = (nomes,)
    selecionado: dict[str, str] = {}
    for nome in nomes:
        chave = str(nome)
        if not chave:
            raise ValueError("nomes de variáveis de ambiente não podem ser vazios")
        if chave in os.environ:
            selecionado[chave] = os.environ[chave]
    return selecionado


def variavel_ambiente_obrigatoria(nome: str) -> str:
    """Lê uma variável obrigatória sem expor valores em mensagens de erro."""
    chave = str(nome)
    if not chave:
        raise ValueError("nome da variável de ambiente não pode ser vazio")
    if chave not in os.environ:
        raise ErroSistema(f"A variável de ambiente obrigatória {chave!r} não está
          definida.")
    return os.environ[chave]


def localizar_programa(nome: str) -> Path | None:
    """Resolve um executável no ambiente atual sem iniciá lo."""
    candidato = str(nome).strip()
    if not candidato:
        raise ValueError("nome deve identificar um programa")
    resolvido = shutil.which(candidato)
    return Path(resolvido).resolve() if resolvido else None


def programa_disponivel(nome: str) -> bool:
    """Indica se ``localizar_programa`` consegue resolver o executável."""
    return localizar_programa(nome) is not None
```

```
def quantidade_processadores() -> int | None:
    """Quantidade lógica observada pelo runtime, sem estimar quotas efetivas."""
    quantidade = os.cpu_count()
    return quantidade if quantidade is None or quantidade > 0 else None


def pasta_temporaria() -> Path:
    """Pasta temporária configurada para o processo, sem criar arquivos."""
    return Path(tempfile.gettempdir())


def terminal_interativo(canal: str) -> bool:
    """Informa se entrada, saída ou erro estão conectados a um terminal."""
    chave = str(canal).strip().casefold()
    canais = {
        "entrada": sys.stdin, "stdin": sys.stdin,
        "saida": sys.stdout, "saída": sys.stdout, "stdout": sys.stdout,
        "erro": sys.stderr, "stderr": sys.stderr,
    }
    if chave not in canais:
        raise ValueError("canal deve ser entrada, saida ou erro")
    fluxo = canais[chave]
    isatty = getattr(fluxo, "isatty", None)
    if not callable(isatty):
        return False
    try:
        return bool(isatty())
    except (OSError, ValueError):
        return False


def argumentos_programa() -> tuple[str, ...]:
    """Cópia dos argumentos do programa em execução, sem o caminho do próprio
      programa."""
    return tuple(str(item) for item in sys.argv[1:])


def caminho_executavel() -> Path | None:
    """Caminho do executável que hospeda o runtime, quando o processo o informa."""
    bruto = str(getattr(sys, "executable", "") or "").strip()
    if not bruto:
        return None
    try:
        return Path(bruto).resolve()
    except OSError:
        return Path(bruto)


def pasta_atual() -> Path:
    return Path.cwd()


def pasta_usuario() -> Path:
    return Path.home()


def caminho(*partes: Any) -> Path:
    return juntar_caminho(*partes, expandir_usuario=True)
```

```
def _validar_limite_saida_bytes(valor: int | None) -> int | None:
    if valor is None:
        return None
    if isinstance(valor, bool) or not isinstance(valor, int) or valor < 0:
        raise ValueError("limite_saida_bytes deve ser inteiro maior ou igual a zero ou
          nulo")
    return valor


def _resolver_codificacao(codificacao: str | None) -> str:
    nome = codificacao or locale.getpreferredencoding(False) or "utf-8"
    try:
        return codecs.lookup(nome).name
    except LookupError as exc:
        raise ValueError(f"codificação desconhecida: {nome!r}") from exc


def _resolver_erros_decodificacao(politica: str) -> str:
    mapa = {
        "estrito": "strict", "strict": "strict",
        "substituir": "replace", "replace": "replace",
        "ignorar": "ignore", "ignore": "ignore",
    }
    chave = str(politica).strip().casefold()
    if chave not in mapa:
        raise ValueError("erros_decodificacao deve ser estrito, substituir ou ignorar")
    return mapa[chave]


@dataclass
class _CapturaLimitada:
    dados: bytearray
    truncada: bool = False


def _drenar_fluxo(fluxo: BinaryIO, captura: _CapturaLimitada, limite: int | None) ->
  None:
    try:
        while True:
            trecho = fluxo.read(65536)
            if not trecho:
                break
            if limite is None:
                captura.dados.extend(trecho)
                continue
            restante = max(0, limite - len(captura.dados))
            if restante:
                captura.dados.extend(trecho[:restante])
            if len(trecho) > restante:
                captura.truncada = True
    finally:
        try:
            fluxo.close()
        except OSError:
            pass


def _escrever_entrada(fluxo: BinaryIO, dados: bytes) -> None:
    try:
        fluxo.write(dados)
```

```
        fluxo.flush()
    except (BrokenPipeError, OSError):
        pass
    finally:
        try:
            fluxo.close()
        except OSError:
            pass


def _opcoes_grupo_processos() -> dict[str, Any]:
    if os.name == "posix":
        return {"start_new_session": True}
    if os.name == "nt":
        return {"creationflags": getattr(subprocess, "CREATE_NEW_PROCESS_GROUP", 0)}
    return {}


def _encerrar_arvore_processos(processo: subprocess.Popen[bytes]) -> None:
    if processo.poll() is not None:
        return
    if os.name == "posix":
        try:
            os.killpg(processo.pid, signal.SIGKILL)
        except (ProcessLookupError, PermissionError, OSError):
            processo.kill()
    elif os.name == "nt":
        try:
            subprocess.run(
                ["taskkill", "/PID", str(processo.pid), "/T", "/F"],
                stdout=subprocess.DEVNULL,
                stderr=subprocess.DEVNULL,
                check=False,
                shell=False,
            )
        except OSError:
            processo.kill()
    else:
        processo.kill()


def executar_comando(
    programa: str,
    argumentos: Iterable[Any] = (),
    *,
    pasta: str | Path | None = None,
    timeout: float | None = None,
    ambiente: Mapping[str, Any] | None = None,
    entrada: str | None = None,
    relogio: FonteTempo | None = None,
    codificacao: str | None = None,
    erros_decodificacao: str = "estrito",
    limite_saida_bytes: int | None = None,
) -> ResultadoComando:
    """Executa um processo sem shell e com captura opcionalmente limitada.

    Quando ``limite_saida_bytes`` é informado, stdout e stderr recebem o mesmo
    teto individual. O excedente continua sendo drenado para não bloquear o
    processo, mas não é mantido em memória. Timeout tem prioridade sobre
    truncamento e código de saída.
    """
```

```
    if programa is None or not str(programa).strip():
        raise ValueError("programa deve identificar um executável")
    if isinstance(argumentos, (str, Path)):
        argumentos = (argumentos,)
    timeout = validar_duracao(timeout, "timeout", permitir_none=True)
    limite = _validar_limite_saida_bytes(limite_saida_bytes)
    encoding = _resolver_codificacao(codificacao)
    erros_texto = _resolver_erros_decodificacao(erros_decodificacao)
    cmd = (str(programa), *(str(x) for x in argumentos))
    env = None
    if ambiente is not None:
        env = os.environ.copy()
        env.update({str(k): str(v) for k, v in ambiente.items()})
    relogio_medicao = relogio or RelogioMonotonico()
    inicio = relogio_medicao.agora()
    captura_saida = _CapturaLimitada(bytearray())
    captura_erro = _CapturaLimitada(bytearray())
    processo: subprocess.Popen[bytes] | None = None
    try:
        processo = subprocess.Popen(
            cmd,
            cwd=str(normalizar_caminho(pasta, expandir_usuario=True)) if pasta is not
              None else None,
            env=env,
            stdin=subprocess.PIPE if entrada is not None else None,
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            shell=False,
            **_opcoes_grupo_processos(),
        )
        assert processo.stdout is not None and processo.stderr is not None
        leitores = [
            threading.Thread(target=_drenar_fluxo, args=(processo.stdout, captura_saida,
               limite), daemon=True),
            threading.Thread(target=_drenar_fluxo, args=(processo.stderr, captura_erro,
              limite), daemon=True),
        ]
        for leitor in leitores:
            leitor.start()
        escritor = None
        if entrada is not None:
            assert processo.stdin is not None
            escritor = threading.Thread(
                target=_escrever_entrada,
                args=(processo.stdin, entrada.encode(encoding, errors=erros_texto)),
                daemon=True,
            )
            escritor.start()
        try:
            codigo = processo.wait(timeout=timeout)
        except subprocess.TimeoutExpired as exc:
            _encerrar_arvore_processos(processo)
            try:
                processo.wait(timeout=5)
            except subprocess.TimeoutExpired:
                processo.kill()
                processo.wait()
            for leitor in leitores:
                leitor.join(timeout=2)
            if escritor is not None:
                escritor.join(timeout=1)
```

```
            raise TempoLimiteExcedido(
                cmd, timeout, f"O comando {programa!r} excedeu o tempo limite de
                  {timeout} segundos."
            ) from exc
        for leitor in leitores:
            leitor.join()
        if escritor is not None:
            escritor.join(timeout=1)
    except FileNotFoundError as exc:
        raise ProgramaNaoEncontrado(
            f"Não foi possível executar {programa!r}: programa não encontrado."
        ) from exc
    except TempoLimiteExcedido:
        raise
    except UnicodeError as exc:
        raise ErroSistema(
            f"Falha de codificação textual ao executar {programa!r}: "
            "a entrada ou a saída não pôde ser processada pela codificação configurada."
        ) from exc
    except OSError as exc:
        if processo is not None and processo.poll() is None:
            _encerrar_arvore_processos(processo)
        raise FalhaSistemaOperacional(
            f"Falha do sistema ao executar {programa!r}: {exc}"
        ) from exc

    try:
        saida = bytes(captura_saida.dados).decode(encoding, errors=erros_texto)
        erro = bytes(captura_erro.dados).decode(encoding, errors=erros_texto)
    except UnicodeError as exc:
        raise ErroSistema(
            f"Falha de codificação textual ao executar {programa!r}: "
            "a saída não pôde ser processada pela codificação configurada."
        ) from exc
    return ResultadoComando(
        comando=cmd,
        codigo=codigo,
        saida=saida,
        erro=erro,
        duracao_segundos=max(0.0, relogio_medicao.agora() - inicio),
        saida_truncada=captura_saida.truncada,
        erro_truncado=captura_erro.truncada,
    )


def diagnosticar_sistema() -> dict[str, Any]:
    info = informacoes()
    item = diagnostico(
        "sistema.runtime", "sistema", True, "Integração básica com o sistema
          disponível.",
        capacidade="sistema", detalhe=info.para_dict(),
    )
    return {"disponivel": True, "informacoes": info.para_dict(), "diagnostico":
      item.para_dict()}


def processo_atual() -> dict[str, Any]:
    executavel = caminho_executavel()
    return {
        "pid": os.getpid(),
        "python": sys.executable,
```

```
        "executavel": str(executavel) if executavel is not None else None,
        "programa": str(sys.argv[0]) if sys.argv else None,
        "argumentos": tuple(sys.argv),
        "argumentos_programa": argumentos_programa(),
        "pasta": str(Path.cwd()),
    }


__all__ = [
    "ErroSistema", "ProgramaNaoEncontrado", "TempoLimiteExcedido",
      "FalhaSistemaOperacional",
    "ComandoFalhou", "InformacoesSistema", "ResultadoComando", "informacoes",
      "diagnosticar_sistema",
    "variavel_ambiente", "variavel_ambiente_obrigatoria", "definir_variavel_ambiente",
    "remover_variavel_ambiente", "variaveis_ambiente", "ambiente_selecionado",
      "localizar_programa",
    "programa_disponivel", "quantidade_processadores", "pasta_temporaria",
      "terminal_interativo",
    "argumentos_programa", "caminho_executavel", "pasta_atual", "pasta_usuario",
      "caminho",
    "executar_comando", "processo_atual",
]

"""Tempo, eventos e cancelamento compartilhados do runtime Coral.

Este módulo materializa a segunda parte da coesão 1.5 sem substituir as APIs
históricas de ``coral.regras``. Ele fornece fontes de tempo injetáveis,
validação uniforme de durações, eventos síncronos/enfileirados/propagados e um
token cooperativo de cancelamento reutilizável por bibliotecas e ferramentas.
"""
from __future__ import annotations

from collections import deque
from dataclasses import dataclass, field
import math
import time
from typing import Any, Callable, Mapping, Protocol, runtime_checkable

from .comum import ErroOperacaoCoral

CONTRATO = "coral.tempo_eventos/1"


def validar_duracao(valor: Any, nome: str = "duração", *, permitir_none: bool = False,
                    permitir_zero: bool = True) -> float | None:
    """Normaliza uma duração pública para segundos finitos não negativos."""
    if valor is None and permitir_none:
        return None
    if isinstance(valor, bool):
        raise ValueError(f"{nome} deve ser um número finito maior ou igual a zero")
    try:
        numero = float(valor)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ValueError(f"{nome} deve ser um número finito maior ou igual a zero") from
           exc
    if not math.isfinite(numero) or numero < 0 or (numero == 0 and not permitir_zero):
        operador = "maior que zero" if not permitir_zero else "maior ou igual a zero"
        raise ValueError(f"{nome} deve ser um número finito {operador}")
    return numero
```

```
def normalizar_dt(valor: Any, *, compatibilidade_negativo_zero: bool = True) -> float:
    """Normaliza avanço de simulação.

    A linha 1.2 já definia que ``dt`` negativo não volta temporizadores. Por
    compatibilidade, a política padrão converte valores negativos finitos em
    zero; novos consumidores podem optar por rejeitá-los.
    """
    if isinstance(valor, bool):
        raise ValueError("avanço de tempo deve ser um número finito")
    try:
        numero = float(valor)
    except (TypeError, ValueError, OverflowError) as exc:
        raise ValueError("avanço de tempo deve ser um número finito") from exc
    if not math.isfinite(numero):
        raise ValueError("avanço de tempo deve ser um número finito")
    if numero < 0:
        if compatibilidade_negativo_zero:
            return 0.0
        raise ValueError("avanço de tempo não pode ser negativo")
    return numero


@runtime_checkable
class FonteTempo(Protocol):
    def agora(self) -> float: ...
    def passo(self) -> float: ...


class RelogioMonotonico:
    """Relógio real baseado em ``perf_counter`` e apropriado para durações."""
    def __init__(self, fonte: Callable[[], float] = time.perf_counter):
        self._fonte = fonte
        self._ultimo = float(fonte())

    def agora(self) -> float:
        return float(self._fonte())

    def passo(self) -> float:
        atual = self.agora()
        dt = normalizar_dt(atual - self._ultimo)
        self._ultimo = atual
        return dt


class RelogioSimulado:
    """Relógio controlado pelo programa, sem dormir nem consultar o sistema."""
    def __init__(self, inicial: float = 0.0):
        self._agora = float(validar_duracao(inicial, "tempo inicial"))
        self._ultimo = self._agora

    def agora(self) -> float:
        return self._agora

    def avancar(self, dt: Any) -> float:
        self._agora += normalizar_dt(dt, compatibilidade_negativo_zero=False)
        return self._agora

    def passo(self) -> float:
        dt = normalizar_dt(self._agora - self._ultimo,
          compatibilidade_negativo_zero=False)
        self._ultimo = self._agora
```

```
        return dt

    def definir(self, instante: Any) -> float:
        novo = float(validar_duracao(instante, "instante"))
        if novo < self._agora:
            raise ValueError("o relógio simulado não pode retroceder")
        self._agora = novo
        return novo


class OperacaoCancelada(ErroOperacaoCoral):
    """Operação cooperativa interrompida antes de uma fase segura."""


class TokenCancelamento:
    def __init__(self):
        self._cancelado = False
        self._motivo: str | None = None

    @property
    def cancelado(self) -> bool:
        return self._cancelado

    @property
    def motivo(self) -> str | None:
        return self._motivo

    def cancelar(self, motivo: str = "Operação cancelada pelo usuário.") -> None:
        self._cancelado = True
        self._motivo = str(motivo)

    def verificar(self) -> None:
        if self._cancelado:
            raise OperacaoCancelada(self._motivo or "Operação cancelada.")


@dataclass
class Evento:
    nome: str
    args: tuple[Any, ...] = ()
    kwargs: Mapping[str, Any] = field(default_factory=dict)
    origem: str | None = None
    sequencia: int = 0
    instante: float | None = None
    cancelavel: bool = False
    cancelado: bool = False
    motivo_cancelamento: str | None = None
    consumido: bool = False
    metadados: Mapping[str, Any] = field(default_factory=dict)
    causa_sequencia: int | None = None

    def __post_init__(self):
        self.nome = str(self.nome)
        self.args = tuple(self.args)
        self.kwargs = dict(self.kwargs)
        self.origem = None if self.origem is None else str(self.origem)
        self.metadados = dict(self.metadados)

    def __getattr__(self, nome: str) -> Any:
        payload = self.__dict__.get("kwargs", {})
        if nome in payload:
```

```
            return payload[nome]
        raise AttributeError(nome)

    def cancelar(self, motivo: str | None = None) -> None:
        if not self.cancelavel:
            raise ValueError("este evento não é cancelável")
        self.cancelado = True
        self.motivo_cancelamento = None if motivo is None else str(motivo)

    def consumir(self) -> None:
        self.consumido = True

    def estado_inspecao(self) -> dict[str, Any]:
        """Estado observável do evento sem executar ouvintes nem alterar a fila."""
        return {
            "nome": self.nome,
            "args": self.args,
            "kwargs": dict(self.kwargs),
            "origem": self.origem,
            "sequencia": self.sequencia,
            "instante": self.instante,
            "cancelavel": self.cancelavel,
            "cancelado": self.cancelado,
            "motivo_cancelamento": self.motivo_cancelamento,
            "consumido": self.consumido,
            "metadados": dict(self.metadados),
            "causa_sequencia": self.causa_sequencia,
        }


@dataclass
class _InscricaoEvento:
    funcao: Callable[..., Any]
    prioridade: int
    ordem: int
    receber_evento: bool = False
    max_execucoes: int | None = None
    cooldown: float = 0.0
    execucoes: int = 0
    ultimo_disparo: float | None = None


_PRIORIDADES_EVENTO = {
    "baixa": 0,
    "normal": 100,
    "alta": 200,
    "critica": 300,
    "crítica": 300,
}


def _normalizar_prioridade_evento(prioridade: str | int) -> int:
    if isinstance(prioridade, bool):
        raise ValueError("prioridade de evento desconhecida: use baixa, normal, alta ou
          crítica")
    if isinstance(prioridade, int):
        return prioridade
    nome = str(prioridade).strip().casefold()
    if nome not in _PRIORIDADES_EVENTO:
        raise ValueError(
            f"prioridade de evento desconhecida: {prioridade!r}; use baixa, normal, alta
```

```
                ou crítica"
            )
        return _PRIORIDADES_EVENTO[nome]


class Eventos:
    """Barramento local compatível com ``coral.regras.Eventos``.

    ``emitir`` mantém a chamada histórica de ouvintes via ``*args`` e
    ``**kwargs``. Inscrições novas podem receber o objeto :class:`Evento`
    completo e declarar prioridade sem alterar consumidores antigos.
    """
    def __init__(
        self,
        *,
        pai: "Eventos | None" = None,
        relogio: FonteTempo | None = None,
        fonte_instante: Callable[[], float] | None = None,
    ):
        self._ouvintes: dict[str, list[_InscricaoEvento]] = {}
        self._fila: deque[Evento] = deque()
        self._pai = pai
        self._relogio = relogio
        self._fonte_instante = fonte_instante
        self._sequencia = 0
        self._ordem_inscricao = 0
        self._pilha_processamento: list[Evento] = []

    def estado_inspecao(self) -> dict[str, Any]:
        return {
            "pendentes": len(self._fila),
            "fila": tuple(self._fila),
            "tipos_ouvintes": tuple(sorted(self._ouvintes)),
            "quantidade_ouvintes": sum(len(itens) for itens in self._ouvintes.values()),
            "ouvintes_por_tipo": {
                nome: len(itens)
                for nome, itens in sorted(self._ouvintes.items())
            },
            "sequencia": self._sequencia,
            "possui_pai": self._pai is not None,
            "possui_fonte_instante": self.possui_fonte_instante,
        }

    @property
    def pendentes(self) -> int:
        return len(self._fila)

    @property
    def pai(self) -> "Eventos | None":
        return self._pai

    @property
    def relogio(self) -> FonteTempo | None:
        return self._relogio

    def definir_fonte_instante(self, fonte: Callable[[], float] | None) -> None:
        self._fonte_instante = fonte

    def _instante_atual(self) -> float | None:
        if self._relogio is not None:
            return float(self._relogio.agora())
```

```
        if self._fonte_instante is not None:
            return float(self._fonte_instante())
        return None

    @property
    def possui_fonte_instante(self) -> bool:
        return self._relogio is not None or self._fonte_instante is not None

    def definir_pai(self, pai: "Eventos | None") -> None:
        if pai is self:
            raise ValueError("um barramento de eventos não pode propagar para si mesmo")
        atual = pai
        while atual is not None:
            if atual is self:
                raise ValueError("ciclo de propagação de eventos não permitido")
            atual = atual.pai
        self._pai = pai

    def quando(
        self,
        nome: str,
        funcao: Callable[..., Any],
        *,
        prioridade: str | int = "normal",
        receber_evento: bool = False,
        max_execucoes: int | None = None,
        cooldown: float = 0.0,
    ):
        valor_prioridade = _normalizar_prioridade_evento(prioridade)
        if max_execucoes is not None:
            if isinstance(max_execucoes, bool) or not isinstance(max_execucoes, int) or
              max_execucoes <= 0:
                raise ValueError("máximo de execuções de evento deve ser um inteiro
                  positivo ou nulo")
        try:
            valor_cooldown = float(validar_duracao(cooldown, "cooldown de evento"))
        except ValueError as exc:
            raise ValueError("cooldown de evento deve ser um número finito maior ou
              igual a zero") from exc
        if valor_cooldown > 0 and self._relogio is None and self._fonte_instante is
          None:
            raise ValueError("cooldown de evento exige uma fonte de tempo")
        self._ordem_inscricao += 1
        inscricao = _InscricaoEvento(
            funcao=funcao,
            prioridade=valor_prioridade,
            ordem=self._ordem_inscricao,
            receber_evento=bool(receber_evento),
            max_execucoes=max_execucoes,
            cooldown=valor_cooldown,
        )
        self._ouvintes.setdefault(str(nome), []).append(inscricao)
        return funcao

    def remover(self, nome: str, funcao: Callable[..., Any]) -> bool:
        ouvintes = self._ouvintes.get(str(nome))
        if not ouvintes:
            return False
        for indice, inscricao in enumerate(ouvintes):
            if inscricao.funcao is funcao or inscricao.funcao == funcao:
                del ouvintes[indice]
```

```
                if not ouvintes:
                    self._ouvintes.pop(str(nome), None)
                return True
        return False

    def _novo_evento(
        self,
        nome: str,
        args: tuple[Any, ...],
        kwargs: Mapping[str, Any],
        origem: str | None,
        *,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
    ) -> Evento:
        self._sequencia += 1
        instante = self._instante_atual()
        return Evento(
            str(nome),
            tuple(args),
            dict(kwargs),
            None if origem is None else str(origem),
            self._sequencia,
            instante=instante,
            cancelavel=bool(cancelavel),
            metadados={} if metadados is None else dict(metadados),
            causa_sequencia=(self._pilha_processamento[-1].sequencia if
              self._pilha_processamento else None),
        )

    def _snapshot_ouvintes(self, nome: str) -> tuple[_InscricaoEvento, ...]:
        ouvintes = tuple(self._ouvintes.get(nome, ()))
        return tuple(sorted(ouvintes, key=lambda item: (-item.prioridade, item.ordem)))

    def _emitir_local(self, evento: Evento) -> int:
        ouvintes = self._snapshot_ouvintes(evento.nome)
        for inscricao in ouvintes:
            if inscricao.max_execucoes is not None and inscricao.execucoes >=
              inscricao.max_execucoes:
                continue
            if inscricao.cooldown > 0 and inscricao.ultimo_disparo is not None:
                if evento.instante is None:
                    raise ValueError("cooldown de evento exige instante lógico")
                decorrido = float(evento.instante) - inscricao.ultimo_disparo
                if decorrido < inscricao.cooldown and not math.isclose(
                    decorrido, inscricao.cooldown, rel_tol=1e-12, abs_tol=1e-15
                ):
                    continue
            if inscricao.receber_evento:
                inscricao.funcao(evento)
            else:
                inscricao.funcao(*evento.args, **dict(evento.kwargs))
            inscricao.execucoes += 1
            if evento.instante is not None:
                inscricao.ultimo_disparo = float(evento.instante)
        return len(ouvintes)

    def emitir(self, nome: str, *args, **kwargs) -> int:
        evento = self._novo_evento(nome, args, kwargs, None)
        return self._emitir_local(evento)
```

```
    def emitir_evento(
        self,
        nome: str,
        *args,
        origem: str | None = None,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
        **kwargs,
    ) -> Evento:
        evento = self._novo_evento(
            nome,
            args,
            kwargs,
            origem,
            cancelavel=cancelavel,
            metadados=metadados,
        )
        self._emitir_local(evento)
        return evento

    def enfileirar(self, nome: str, *args, origem: str | None = None, **kwargs) ->
      Evento:
        evento = self._novo_evento(nome, args, kwargs, origem)
        self._fila.append(evento)
        return evento

    def enfileirar_evento(
        self,
        nome: str,
        *args,
        origem: str | None = None,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
        **kwargs,
    ) -> Evento:
        evento = self._novo_evento(
            nome,
            args,
            kwargs,
            origem,
            cancelavel=cancelavel,
            metadados=metadados,
        )
        self._fila.append(evento)
        return evento

    def processar(self, limite: int | None = None) -> tuple[Evento, ...]:
        if limite is not None and (isinstance(limite, bool) or not isinstance(limite,
          int) or limite < 0):
            raise ValueError("limite de eventos deve ser inteiro não negativo ou nulo")
        processados: list[Evento] = []
        maximo = len(self._fila) if limite is None else min(limite, len(self._fila))
        for _ in range(maximo):
            evento = self._fila.popleft()
            self._pilha_processamento.append(evento)
            try:
                self._emitir_local(evento)
            finally:
                self._pilha_processamento.pop()
            processados.append(evento)
        return tuple(processados)
```

```
    def _emitir_propagado_objeto(self, evento: Evento) -> int:
        total = self._emitir_local(evento)
        if self._pai is not None and not evento.consumido:
            total += self._pai._emitir_propagado_objeto(evento)
        return total

    def emitir_propagado(self, nome: str, *args, origem: str | None = None, **kwargs) ->
       int:
        evento = self._novo_evento(nome, args, kwargs, origem)
        return self._emitir_propagado_objeto(evento)

    def emitir_evento_propagado(
        self,
        nome: str,
        *args,
        origem: str | None = None,
        cancelavel: bool = False,
        metadados: Mapping[str, Any] | None = None,
        **kwargs,
    ) -> Evento:
        evento = self._novo_evento(
            nome,
            args,
            kwargs,
            origem,
            cancelavel=cancelavel,
            metadados=metadados,
        )
        self._emitir_propagado_objeto(evento)
        return evento


__all__ = [
    "CONTRATO", "validar_duracao", "normalizar_dt", "FonteTempo",
    "RelogioMonotonico", "RelogioSimulado", "OperacaoCancelada",
    "TokenCancelamento", "Evento", "Eventos",
]
```

Biblioteca `coral.texto`

`aparar(...)` · módulo `coral.texto`

O que faz

Retira espaços em branco nas extremidades do texto.

Assinatura

Assinatura para consulta

```
aparar(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe aparar
mostre aparar("  Coral  ")
```

Resultado

Saída esperada

```
Coral
```

Forma natural

`apare o texto de " Coral "` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`comeca_com(...)` · módulo `coral.texto`

O que faz

Verifica se o texto começa com o prefixo informado.

Assinatura

Assinatura para consulta

```
comeca_com(valor, prefixo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
prefixo	qualquer valor convertido com str	sim	sem padrão	Texto procurado no início.

Retorno

Booleano.

Exemplo

Código Coral

```
de coral.texto importe comeca_com
mostre comeca_com("coral", "co")
```

Resultado

Saída esperada

```
True
```

Forma natural

`"coral" começa com "co"` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`contar(...)` · módulo `coral.texto`

O que faz

Conta ocorrências de um trecho.

Assinatura

Assinatura para consulta

```
contar(valor, trecho, inicio=0, fim=None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
trecho	qualquer valor convertido com str	sim	sem padrão	Texto cujas ocorrências serão contadas.
inicio	valor convertível com int	não	0	Índice inicial da busca.
fim	valor convertível com int ou nulo	não	None	Limite final da busca.

Retorno

Quantidade inteira.

Exemplo

Código Coral

```
de coral.texto importe contar
mostre contar("banana", "a")
```

Resultado

Saída esperada

```
3
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`dividir(...)` · módulo `coral.texto`

O que faz

Divide o texto em partes.

Assinatura

Assinatura para consulta

```
dividir(valor, separador=None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
separador	texto não vazio ou nulo	não	None	Texto que separa as partes.

Retorno

Lista de textos.

Exemplo

Código Coral

```
de coral.texto importe dividir
mostre dividir("a,b", ",")
```

Resultado

Saída esperada

```
['a', 'b']
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`formatar(...)` · módulo `coral.texto`

O que faz

Substitui placeholders simples pelos valores fornecidos, sem avaliar expressões.

Assinatura

Assinatura para consulta

```
formatar(modelo, valores=None, **campos)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
modelo	qualquer valor	sim	sem padrão	Modelo convertido para texto; placeholders devem ser identificadores simples.
valores	dicionário ou nulo	não	None	Valores dos campos; nulo começa sem campos.
campos	argumentos nomeados variáveis	não	sem padrão	Campos adicionais, que substituem as chaves do dicionário com mesmo nome.

Retorno

Texto formatado.

Exemplo

Código Coral

```
de coral.texto importe formatar
mostre formatar("Olá, {nome}. Nota {nota:.1f}", {"nome": "Ana", "nota": 7})
```

Resultado

Saída esperada

```
Olá, Ana. Nota 7.0
```

Erros

TypeError para valores que não é dicionário nem nulo; KeyError para campo ausente; ValueError para placeholder não simples, conversão !r/!s/!a ou especificação inválida.

Atenção

Não permite acesso a atributos, índices ou avaliação de expressões. Especificações de formato seguem format do Python.

`juntar(...)` · módulo `coral.texto`

O que faz

Reúne textos com um separador.

Assinatura

Assinatura para consulta

```
juntar(separador, valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
separador	qualquer valor convertido com str	sim	sem padrão	Texto que separa as partes.
valores	iterável de valores convertidos com str	sim	sem padrão	Coleção de entrada.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.texto importe juntar
mostre juntar(",", ["a", "b"])
```

Resultado

Saída esperada

```
a,b
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`linhas(...)` · módulo `coral.texto`

O que faz

Obtém as linhas de um texto.

Assinatura

Assinatura para consulta

```
linhas(valor, *, manter_quebras=False)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
manter_quebras	booleano	não	False	Se verdadeiro, mantém os caracteres de quebra de linha nas partes devolvidas.

Retorno

Lista de textos.

Exemplo

Código Coral

```
de coral.texto importe linhas
mostre linhas("a\nb")
```

Resultado

Saída esperada

```
['a', 'b']
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`localizar(...)` · módulo `coral.texto`

O que faz

Procura um trecho no texto.

Assinatura

Assinatura para consulta

```
localizar(valor, trecho, inicio=0)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
trecho	qualquer valor convertido com str	sim	sem padrão	Texto procurado a partir do índice inicial.
inicio	valor convertível com int	não	0	Índice inicial da busca.

Retorno

Índice inteiro; menos um quando não encontrado.

Exemplo

Código Coral

```
de coral.texto importe localizar
mostre localizar("coral", "ra")
```

Resultado

Saída esperada

```
2
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`maiusculas(...)` · módulo `coral.texto`

O que faz

Produz a versão em maiúsculas do texto.

Assinatura

Assinatura para consulta

```
maiusculas(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe maiusculas
mostre maiusculas("Coral")
```

Resultado

Saída esperada

```
CORAL
```

Forma natural

`maiúsculas de "Coral"` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`minusculas(...)` · módulo `coral.texto`

O que faz

Produz a versão em minúsculas do texto.

Assinatura

Assinatura para consulta

```
minusculas(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe minusculas
mostre minusculas("Coral")
```

Resultado

Saída esperada

```
coral
```

Forma natural

`minúsculas de "Coral"` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`normalizar_unicode(...)` · módulo `coral.texto`

O que faz

Normaliza a representação Unicode com a forma escolhida.

Assinatura

Assinatura para consulta

```
normalizar_unicode(valor, forma='NFC')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
forma	texto NFC, NFD, NFKC ou NFKD	não	'NFC'	Forma de normalização Unicode.

Retorno

Texto normalizado.

Exemplo

Código Coral

```
de coral.texto importe normalizar_unicode
mostre normalizar_unicode("Coral")
```

Resultado

Saída esperada

```
Coral
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`preencher(...)` · módulo `coral.texto`

O que faz

Preenche um texto até a largura solicitada com o alinhamento indicado.

Assinatura

Assinatura para consulta

```
preencher(valor, largura, caractere=' ', alinhamento='direita')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor	sim	sem padrão	Valor convertido para texto com str.
largura	valor convertível para inteiro	sim	sem padrão	Largura mínima em caracteres; não conta largura visual no terminal.
caractere	texto de um caractere	não	' '	Caractere usado para preencher.
alinhamento	texto	não	'direita'	direita/fim, esquerda/inicio/início ou centro/centralizado/centralizada.

Retorno

Texto novo; se a largura já foi atingida, o texto não é truncado.

Exemplo

Código Coral

```
de coral.texto importe preencher
mostre preencher("7", 3, "0")
```

Resultado

Saída esperada

```
007
```

Erros

ValueError para caractere com tamanho diferente de um, alinhamento desconhecido ou largura não convertível; TypeError para largura incompatível.

Atenção

direita é o alinhamento do conteúdo: os caracteres de preenchimento entram à esquerda.

`remover_prefixo(...)` · módulo `coral.texto`

O que faz

Retira o prefixo se estiver presente.

Assinatura

Assinatura para consulta

```
remover_prefixo(valor, prefixo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
prefixo	qualquer valor convertido com str	sim	sem padrão	Texto procurado no início.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe remover_prefixo
mostre remover_prefixo("coral", "co")
```

Resultado

Saída esperada

```
ral
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`remover_sufixo(...)` · módulo `coral.texto`

O que faz

Retira o sufixo se estiver presente.

Assinatura

Assinatura para consulta

```
remover_sufixo(valor, sufixo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
sufixo	qualquer valor convertido com str	sim	sem padrão	Texto procurado no final.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe remover_sufixo
mostre remover_sufixo("coral", "al")
```

Resultado

Saída esperada

```
cor
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`substituir(...)` · módulo `coral.texto`

O que faz

Produz um texto substituindo ocorrências do trecho informado.

Assinatura

Assinatura para consulta

```
substituir(valor, antigo, novo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
antigo	qualquer valor convertido com str	sim	sem padrão	Trecho a substituir.
novo	qualquer valor convertido com str	sim	sem padrão	Texto que ocupa o lugar do trecho encontrado.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe substituir
mostre substituir("casa", "a", "o")
```

Resultado

Saída esperada

```
coso
```

Forma natural

`substitua "a" por "o" em "casa"` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`termina_com(...)` · módulo `coral.texto`

O que faz

Verifica se o texto termina com o sufixo informado.

Assinatura

Assinatura para consulta

```
termina_com(valor, sufixo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
sufixo	qualquer valor convertido com str	sim	sem padrão	Texto procurado no final.

Retorno

Booleano.

Exemplo

Código Coral

```
de coral.texto importe termina_com
mostre termina_com("coral", "al")
```

Resultado

Saída esperada

```
True
```

Forma natural

`"coral" termina com "al"` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

Consultar a implementação canônica de coral.texto

Implementação canônica de `coral.texto`

Código Python

```
from __future__ import annotations

import re
import string
import unicodedata


def maiusculas(valor): return str(valor).upper()
def minusculas(valor): return str(valor).lower()
def aparar(valor): return str(valor).strip()
def dividir(valor, separador=None): return str(valor).split(separador)
def juntar(separador, valores): return str(separador).join(map(str, valores))
def substituir(valor, antigo, novo): return str(valor).replace(str(antigo), str(novo))
def comeca_com(valor, prefixo): return str(valor).startswith(str(prefixo))
def termina_com(valor, sufixo): return str(valor).endswith(str(sufixo))


def localizar(valor, trecho, inicio=0):
    return str(valor).find(str(trecho), int(inicio))


def contar(valor, trecho, inicio=0, fim=None):
    texto = str(valor)
    if fim is None:
        return texto.count(str(trecho), int(inicio))
    return texto.count(str(trecho), int(inicio), int(fim))


def normalizar_unicode(valor, forma="NFC"):
    forma = str(forma).upper()
    if forma not in {"NFC", "NFD", "NFKC", "NFKD"}:
        raise ValueError("forma Unicode precisa ser NFC, NFD, NFKC ou NFKD")
    return unicodedata.normalize(forma, str(valor))


def linhas(valor, *, manter_quebras=False):
```

```
    return str(valor).splitlines(keepends=bool(manter_quebras))


def remover_prefixo(valor, prefixo):
    texto = str(valor); prefixo = str(prefixo)
    return texto[len(prefixo):] if texto.startswith(prefixo) else texto


def remover_sufixo(valor, sufixo):
    texto = str(valor); sufixo = str(sufixo)
    return texto[:-len(sufixo)] if sufixo and texto.endswith(sufixo) else texto


def preencher(valor, largura, caractere=" ", alinhamento="direita"):
    texto = str(valor); largura = int(largura); caractere = str(caractere)
    if len(caractere) != 1:
        raise ValueError("caractere de preenchimento precisa ter exatamente um
          caractere")
    modo = str(alinhamento).strip().casefold()
    if modo in {"direita", "fim"}:
        return texto.rjust(largura, caractere)
    if modo in {"esquerda", "inicio", "início"}:
        return texto.ljust(largura, caractere)
    if modo in {"centro", "centralizado", "centralizada"}:
        return texto.center(largura, caractere)
    raise ValueError("alinhamento precisa ser esquerda, direita ou centro")


def formatar(modelo, valores=None, **campos):
    """Formata placeholders simples sem avaliar expressões ou atributos."""
    dados = {}
    if valores is not None:
        if not isinstance(valores, dict):
            raise TypeError("valores precisa ser um dicionário")
        dados.update(valores)
    dados.update(campos)
    partes = []
    formatter = string.Formatter()
    for literal, campo, especificacao, conversao in formatter.parse(str(modelo)):
        partes.append(literal)
        if campo is None:
            continue
        if not campo.isidentifier():
            raise ValueError("placeholder precisa ser um nome simples")
        if conversao is not None:
            raise ValueError("conversões !r, !s e !a não são permitidas")
        if campo not in dados:
            raise KeyError(campo)
        partes.append(format(dados[campo], especificacao))
    return "".join(partes)


__all__ = [
    "maiusculas", "minusculas", "aparar", "dividir", "juntar", "substituir",
    "comeca_com", "termina_com", "localizar", "contar", "normalizar_unicode", "linhas",
    "remover_prefixo", "remover_sufixo", "preencher", "formatar",
]

"""Superfície pública de coral.texto."""
from .stdlib.texto import *  # noqa: F401,F403
```

Biblioteca `coral.tipos`

`coral.tipos.TIPOS_CORAL`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

```
{'inteiro': 'int', 'decimal': 'float', 'número': 'float', 'numero': 'float', 'texto': 'str', 'booleano': 'bool', 'lógico': 'bool', 'logico': 'bool', 'lista': 'list', 'dicionário': 'dict', 'dicionario': 'dict', 'conjunto': 'set', 'tupla': 'tuple', 'qualquer': 'object', 'nulo': 'None'}
```

`coral.tipos.TipoCoral`

TipoCoral(nome: 'str', tipos_python: 'tuple[type, ...]' = (), aliases: 'tuple[str, ...]' = ())

Assinatura

Assinatura para consulta

```
TipoCoral(nome: 'str', tipos_python: 'tuple[type, ...]' = (), aliases: 'tuple[str, ...]'
    = ()) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
tipos_python	tuple[type, ...]	POSITIONAL_OR_KEYWORD	não	()
aliases	tuple[str, ...]	POSITIONAL_OR_KEYWORD	não	()

Retorno declarado

None

Atributo `nome`: `str`.

Atributo `tipos_python`: `tuple[type, ...]`.

Atributo `aliases`: `tuple[str, ...]`.

Métodos públicos

Referência para leitura

```
aceita(self, valor: 'Any') -> 'bool'
```

Método público sem descrição própria no runtime.

`e_tipo(...)` · **módulo** `coral.tipos`

O que faz

Testa se o valor pertence à categoria informada.

Assinatura

Assinatura para consulta

```
e_tipo(valor: 'Any', esperado: 'str | type | TipoCoral') -> 'bool'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor	sim	sem padrão	Valor sobre o qual a operação atua.
esperado	texto de tipo, classe Python ou TipoCoral	sim	sem padrão	Nome, tipo ou descritor aceito pelo teste de tipo.

Retorno

Booleano.

Exemplo

Código Coral

```
de coral.tipos importe e_tipo
mostre e_tipo(verdadeiro, "inteiro")
```

Resultado

Saída esperada

```
False
```

Forma natural

`valor for do tipo inteiro` e `animal for do tipo Animal` usam o mesmo contrato. A negação é `animal não for do tipo Animal`.

Erros

Um nome de classe ausente ou ocultado por outro valor pode receber diagnóstico de análise. A API não transforma um nome textual arbitrário numa classe do programa.

Atenção

Booleano não é inteiro implícito. Classes são passadas como objetos reais, não como texto. Subclasses pertencem aos ancestrais válidos, inclusive na herança múltipla. Não valida profundamente coleções parametrizadas.

Veja também

Cap. 17, seção Consultar classes do programa; Cap. 16, seção Exemplo: uma célula que guarda conteúdo.

`nome_tipo(...)` · módulo `coral.tipos`

O que faz

Obtém o nome Coral da categoria do valor.

Assinatura

Assinatura para consulta

```
nome_tipo(valor: 'Any') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor, classe Python ou TipoCoral	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.tipos importe nome_tipo
mostre nome_tipo(12)
```

Resultado

Saída esperada

```
inteiro
```

Forma natural

`o nome do tipo de 12` consulta o mesmo contrato.

Atenção

Booleano não é inteiro implícito. Não é um validador profundo de coleções parametrizadas.

`tipo_de(...)` · módulo `coral.tipos`

O que faz

Obtém o descritor da categoria do valor.

Assinatura

Assinatura para consulta

```
tipo_de(valor: 'Any') -> 'TipoCoral'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

TipoCoral; o atributo nome contém o nome da categoria.

Exemplo

Código Coral

```
de coral.tipos importe tipo_de
mostre tipo_de(12).nome
```

Resultado

Saída esperada

```
inteiro
```

Forma natural

`(o tipo de 12).nome` consulta o mesmo contrato.

Atenção

Booleano não é inteiro implícito. Não é um validador profundo de coleções parametrizadas.

`coral.tipos.traduzir_tipo`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
traduzir_tipo(tipo: 'str') -> 'str'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
tipo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

str

Consultar a implementação canônica de coral.tipos

Implementação canônica de `coral.tipos`

Código Python

```
from __future__ import annotations

from dataclasses import dataclass
from datetime import date, datetime, timedelta
from pathlib import PurePath
from typing import Any

TIPOS_CORAL = {
    "inteiro": "int",
    "decimal": "float",
    "número": "float",
    "numero": "float",
    "texto": "str",
    "booleano": "bool",
    "lógico": "bool",
    "logico": "bool",
    "lista": "list",
    "dicionário": "dict",
    "dicionario": "dict",
    "conjunto": "set",
    "tupla": "tuple",
    "qualquer": "object",
    "nulo": "None",
}



def traduzir_tipo(tipo: str) -> str:
    from .tipos_estruturais import analisar_tipo, emitir_tipo_python
    return emitir_tipo_python(analisar_tipo(tipo))


@dataclass(frozen=True)
class TipoCoral:
    nome: str
    tipos_python: tuple[type, ...] = ()
    aliases: tuple[str, ...] = ()

    def __str__(self) -> str:
        return self.nome

    def aceita(self, valor: Any) -> bool:
        if self.nome == "nulo":
            return valor is None
        if self.nome == "inteiro":
            return isinstance(valor, int) and not isinstance(valor, bool)
        if self.nome == "decimal":
            return isinstance(valor, float)
        if self.nome == "data":
            return isinstance(valor, date) and not isinstance(valor, datetime)
        return bool(self.tipos_python) and isinstance(valor, self.tipos_python)


_TIPOS_RUNTIME = (
    TipoCoral("booleano", (bool,), ("lógico", "logico")),
    TipoCoral("inteiro", (int,)),
    TipoCoral("decimal", (float,), ("número", "numero")),
    TipoCoral("texto", (str,)),
    TipoCoral("lista", (list,)),
    TipoCoral("dicionário", (dict,), ("dicionario",)),
```

```
    TipoCoral("conjunto", (set,)),
    TipoCoral("tupla", (tuple,)),
    TipoCoral("bytes", (bytes, bytearray)),
    TipoCoral("data e hora", (datetime,), ("data_hora", "data-hora")),
    TipoCoral("data", (date,)),
    TipoCoral("duração", (timedelta,), ("duracao",)),
    TipoCoral("caminho", (PurePath,)),
    TipoCoral("nulo"),
)
_TIPOS_POR_NOME = {
    nome.casefold(): tipo
    for tipo in _TIPOS_RUNTIME
    for nome in (tipo.nome, *tipo.aliases)
}


def tipo_de(valor: Any) -> TipoCoral:
    for tipo in _TIPOS_RUNTIME:
        if tipo.aceita(valor):
            return tipo
    return TipoCoral(type(valor).__name__, (type(valor),))


def nome_tipo(valor: Any) -> str:
    if isinstance(valor, TipoCoral):
        return valor.nome
    if isinstance(valor, type):
        for tipo in _TIPOS_RUNTIME:
            if valor in tipo.tipos_python:
                return tipo.nome
        return valor.__name__
    return tipo_de(valor).nome


def e_tipo(valor: Any, esperado: str | type | TipoCoral) -> bool:
    if isinstance(esperado, TipoCoral):
        return esperado.aceita(valor)
    if isinstance(esperado, type):
        if esperado is int:
            return isinstance(valor, int) and not isinstance(valor, bool)
        return isinstance(valor, esperado)
    tipo = _TIPOS_POR_NOME.get(str(esperado).strip().casefold())
    return tipo.aceita(valor) if tipo is not None else False


__all__ = [
    "TIPOS_CORAL",
    "traduzir_tipo",
    "TipoCoral",
    "tipo_de",
    "nome_tipo",
    "e_tipo",
]
```

Biblioteca `coral.web`

`coral.web.html`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`<módulo coral.web.html>`

`coral.web.http`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`<módulo coral.web.http>`

`coral.web.servidor`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`<módulo coral.web.servidor>`

`coral.web.url`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`<módulo coral.web.url>`

Consultar a implementação canônica de coral.web

Implementação canônica de `coral.web`

Código Python

```
"""Ferramentas Web locais e determinísticas da Coral.

A base Web contém HTML e URL locais. O cliente HTTP e o servidor de desenvolvimento
  permanecem em submódulos separados.
Importar ``coral.web`` continua sem abrir conexão, criar socket de escuta ou iniciar
thread de servidor.
"""
from . import html, url

__all__ = ["html", "url", "http", "servidor"]
```

Biblioteca `coral.web.html`

`coral.web.html.DocumentoHTML`

Documento HTML completo com idioma e UTF 8 declarados.

Assinatura

Assinatura para consulta

```
DocumentoHTML(titulo: 'str', corpo: 'FragmentoHTML', idioma: 'str' = 'pt-BR') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
titulo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
corpo	FragmentoHTML	POSITIONAL_OR_KEYWORD	sim	sem padrão
idioma	str	POSITIONAL_OR_KEYWORD	não	'pt-BR'

Retorno declarado

None

Atributo `titulo`: `str`.

Atributo `corpo`: `FragmentoHTML`.

Atributo `idioma`: `str`.

`coral.web.html.ElementoHTML`

Elemento validado com atributos e filhos em ordem estável.

Assinatura

Assinatura para consulta

```
ElementoHTML(nome: 'str', atributos: 'tuple[tuple[str, Any], ...]' = (), filhos:
  'tuple[Any, ...]' = ()) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
atributos	tuple[tuple[str, Any], ...]	POSITIONAL_OR_KEYWORD	não	()
filhos	tuple[Any, ...]	POSITIONAL_OR_KEYWORD	não	()

Retorno declarado

None

Atributo `nome`: `str`.

Atributo `atributos`: `tuple[tuple[str, Any], ...]`.

Atributo `filhos`: `tuple[Any, ...]`.

`coral.web.html.ErroHTML`

Falha de composição ou serialização HTML controlada pela Coral.

Assinatura

Assinatura para consulta

```
ErroHTML(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.web.html.FragmentoHTML`

Sequência de nós sem elemento contêiner adicional.

Assinatura

Assinatura para consulta

```
FragmentoHTML(filhos: 'tuple[Any, ...]') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
filhos	tuple[Any, ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `filhos`: `tuple[Any, ...]`.

`coral.web.html.documento`

Cria documento completo; título e corpo seguem a mesma política de escape.

Assinatura

Assinatura para consulta

```
documento(titulo: 'Any', corpo: 'Any', idioma: 'str' = 'pt-BR') -> 'DocumentoHTML'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
titulo	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
corpo	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
idioma	str	POSITIONAL_OR_KEYWORD	não	'pt-BR'

Retorno declarado

DocumentoHTML

`coral.web.html.elemento`

Cria um elemento validado; texto em `` `filhos` `` será escapado ao renderizar.

Assinatura

Assinatura para consulta

```
elemento(nome: 'str', atributos: 'Any' = None, filhos: 'Any' = None) -> 'ElementoHTML'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
nome	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
atributos	Any	POSITIONAL_OR_KEYWORD	não	None
filhos	Any	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

ElementoHTML

`coral.web.html.escapar_atributo`

Escapa um valor para atributo entre aspas duplas.

Esta operação é codificação contextual, não validação de URL. Por exemplo, `` `javascript:...` `` continua sendo um esquema perigoso depois do escape.

Assinatura

Assinatura para consulta

```
escapar_atributo(valor: 'Any') -> 'str'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

str

`coral.web.html.escapar_texto`

Escapa um valor para o contexto de texto de um elemento HTML.

Assinatura

Assinatura para consulta

```
escapar_texto(valor: 'Any') -> 'str'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

str

`coral.web.html.fragmento`

Agrupa nós preservando a ordem sem criar marcação adicional.

Assinatura

Assinatura para consulta

```
fragmento(filhos: 'Any' = None) -> 'FragmentoHTML'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
filhos	Any	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

FragmentoHTML

`coral.web.html.renderizar`

Serializa um nó ou valor em memória, sem efeitos externos.

Assinatura

Assinatura para consulta

```
renderizar(no: 'Any') -> 'str'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
no	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

str

`coral.web.html.tabela`

Monta tabela a partir de registros comuns com colunas explícitas.

Mapeamentos e objetos com atributos são aceitos. Nenhuma célula é tratada como HTML confiável: todo valor comum é escapado durante a serialização.

Assinatura

Assinatura para consulta

```
tabela(registros: 'Iterable[Any]', colunas: 'Any') -> 'ElementoHTML'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
registros	Iterable[Any]	POSITIONAL_OR_KEYWORD	sim	sem padrão
colunas	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

ElementoHTML

Consultar a implementação canônica de coral.web.html

Implementação canônica de `coral.web.html`

Código Python

```
from __future__ import annotations

from typing import Any

from .comum import ErroCoral, ErroOperacaoCoral


class ErroCoralRuntime(ErroCoral):
    """Base dos erros controlados expostos pelo runtime base da Coral."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroEntrada(ErroCoralRuntime):
    """Falha ao obter entrada para um programa Coral."""


class FimDeEntrada(ErroEntrada, EOFError):
    """A fonte de entrada terminou antes de produzir outra linha."""



class ErroConversao(ErroCoralRuntime, ValueError):
    """Um valor não pôde ser convertido para o tipo Coral solicitado."""

    def __init__(
        self,
        valor: Any,
        destino: str,
        mensagem: str | None = None,
        *,
        causa: BaseException | None = None,
    ):
        self.valor = valor
        self.destino = destino
        super().__init__(
            mensagem or f"não foi possível converter o valor para {destino}",
            causa=causa,
        )


class ErroFormato(ErroCoralRuntime, ValueError):
    """Dados ou texto não obedecem ao formato esperado."""


class ErroValor(ErroCoralRuntime, ValueError):
    """O tipo é aceito, mas o valor não é válido para a operação."""


class ErroOperacao(ErroOperacaoCoral):
    """Falha operacional com preservação opcional da causa técnica."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa
```

```
class ErroArquivo(ErroOperacao):
    """Falha em uma operação com arquivo ou caminho."""


__all__ = [
    "ErroCoralRuntime",
    "ErroEntrada",
    "FimDeEntrada",
    "ErroConversao",
    "ErroArquivo",
    "ErroFormato",
    "ErroValor",
    "ErroOperacao",
]

"""Composição de HTML local com escape por contexto.

A superfície desta versão trabalha somente com uma árvore de nós simples.
Texto comum nunca é interpretado como marcação e a serialização não escreve em
disco, não abre navegador e não acessa a rede.
"""
from __future__ import annotations

from collections.abc import Iterable, Mapping, Sequence
from dataclasses import dataclass
from html import escape as _escape_html
import re
from typing import Any

from ..erros import ErroValor


class ErroHTML(ErroValor):
    """Falha de composição ou serialização HTML controlada pela Coral."""


_NOME_ELEMENTO = re.compile(r"^[A-Za-z][A-Za-z0-9:-]*$")
_NOME_ATRIBUTO = re.compile(r"^[A-Za-z_:][A-Za-z0-9_.:-]*$")
_IDIOMA = re.compile(r"^[A-Za-z]{2,8}(?:-[A-Za-z0-9]{1,8})*$")
_ELEMENTOS_VAZIOS = frozenset(
    {"area", "base", "br", "col", "embed", "hr", "img", "input", "link", "meta",
      "source", "track", "wbr"}
)
_ATRIBUTOS_URL = frozenset({"href", "src", "action", "formaction", "poster", "cite"})
_ESQUEMAS_PERIGOSOS = frozenset({"javascript", "vbscript", "data"})


@dataclass(frozen=True, slots=True)
class ElementoHTML:
    """Elemento validado com atributos e filhos em ordem estável."""

    nome: str
    atributos: tuple[tuple[str, Any], ...] = ()
    filhos: tuple[Any, ...] = ()


@dataclass(frozen=True, slots=True)
class FragmentoHTML:
    """Sequência de nós sem elemento contêiner adicional."""
```

```
    filhos: tuple[Any, ...]


@dataclass(frozen=True, slots=True)
class DocumentoHTML:
    """Documento HTML completo com idioma e UTF 8 declarados."""

    titulo: str
    corpo: FragmentoHTML
    idioma: str = "pt-BR"


def escapar_texto(valor: Any) -> str:
    """Escapa um valor para o contexto de texto de um elemento HTML."""
    if valor is None:
        return ""
    return _escape_html(str(valor), quote=False)


def escapar_atributo(valor: Any) -> str:
    """Escapa um valor para atributo entre aspas duplas.

    Esta operação é codificação contextual, não validação de URL. Por exemplo,
    ``javascript:...`` continua sendo um esquema perigoso depois do escape.
    """
    if valor is None:
        return ""
    return _escape_html(str(valor), quote=True)


def _validar_nome_elemento(nome: Any) -> str:
    if not isinstance(nome, str) or not _NOME_ELEMENTO.fullmatch(nome.strip()):
        raise ErroHTML("nome de elemento HTML inválido")
    return nome.strip().lower()


def _validar_nome_atributo(nome: Any) -> str:
    if not isinstance(nome, str) or not _NOME_ATRIBUTO.fullmatch(nome.strip()):
        raise ErroHTML("nome de atributo HTML inválido")
    resultado = nome.strip().lower()
    if resultado.startswith("on"):
        raise ErroHTML("atributos de evento inline não são aceitos pela base HTML
          segura")
    return resultado


def _validar_url_em_atributo(nome: str, valor: Any) -> None:
    if nome not in _ATRIBUTOS_URL or valor in (None, False, True):
        return
    texto = str(valor).strip()
    # Fragmentos, caminhos relativos e URLs sem esquema explícito são dados
    # legítimos. Quando um esquema aparece, recusamos os que executam conteúdo.
    candidato = texto.split(":", 1)
    if len(candidato) == 2:
        # Navegadores podem tolerar tabulações e quebras de linha no trecho do
        # esquema. Normalizamos apenas esse prefixo para que ``java\nscript:``
        # não contorne a política de atributos URL.
        esquema = re.sub(r"[\x00-\x20\x7f]+", "", candidato[0])
        if re.fullmatch(r"[A-Za-z][A-Za-z0-9+.-]*", esquema):
            if esquema.casefold() in _ESQUEMAS_PERIGOSOS:
                raise ErroHTML(f"esquema de URL perigoso no atributo {nome}")
```

```
def _normalizar_atributos(atributos: Any) -> tuple[tuple[str, Any], ...]:
    if atributos is None:
        return ()
    if isinstance(atributos, Mapping):
        itens = tuple(atributos.items())
    else:
        if isinstance(atributos, (str, bytes, bytearray)):
            raise ErroHTML("atributos devem ser mapeamento ou pares nome valor")
        try:
            itens = tuple(atributos)
        except TypeError as exc:
            raise ErroHTML("atributos devem ser mapeamento ou pares nome valor") from
              exc

    resultado: list[tuple[str, Any]] = []
    vistos: set[str] = set()
    for item in itens:
        if not isinstance(item, Sequence) or isinstance(item, (str, bytes, bytearray))
          or len(item) != 2:
            raise ErroHTML("cada atributo deve ser um par nome valor")
        nome = _validar_nome_atributo(item[0])
        if nome in vistos:
            raise ErroHTML(f"atributo duplicado: {nome}")
        vistos.add(nome)
        valor = item[1]
        _validar_url_em_atributo(nome, valor)
        resultado.append((nome, valor))
    return tuple(resultado)


def _normalizar_filhos(filhos: Any) -> tuple[Any, ...]:
    if filhos is None:
        return ()
    if isinstance(filhos, (ElementoHTML, FragmentoHTML, DocumentoHTML, str, bytes,
      bytearray)):
        return (filhos,)
    if isinstance(filhos, Mapping):
        # Um registro não vira implicitamente uma coleção de chaves.
        return (filhos,)
    try:
        return tuple(filhos)
    except TypeError:
        return (filhos,)


def elemento(nome: str, atributos: Any = None, filhos: Any = None) -> ElementoHTML:
    """Cria um elemento validado; texto em ``filhos`` será escapado ao renderizar."""
    nome_ok = _validar_nome_elemento(nome)
    filhos_ok = _normalizar_filhos(filhos)
    if nome_ok in _ELEMENTOS_VAZIOS and filhos_ok:
        raise ErroHTML(f"elemento vazio {nome_ok} não pode receber filhos")
    return ElementoHTML(nome_ok, _normalizar_atributos(atributos), filhos_ok)


def fragmento(filhos: Any = None) -> FragmentoHTML:
    """Agrupa nós preservando a ordem sem criar marcação adicional."""
    return FragmentoHTML(_normalizar_filhos(filhos))
```

```
def documento(titulo: Any, corpo: Any, idioma: str = "pt-BR") -> DocumentoHTML:
    """Cria documento completo; título e corpo seguem a mesma política de escape."""
    if not isinstance(idioma, str) or not _IDIOMA.fullmatch(idioma.strip()):
        raise ErroHTML("idioma HTML inválido")
    return DocumentoHTML(str(titulo), fragmento(corpo), idioma.strip())


def _renderizar_atributos(atributos: tuple[tuple[str, Any], ...]) -> str:
    partes: list[str] = []
    for nome, valor in atributos:
        if valor is None or valor is False:
            continue
        if valor is True:
            partes.append(nome)
        else:
            partes.append(f'{nome}="{escapar_atributo(valor)}"')
    return (" " + " ".join(partes)) if partes else ""


def _renderizar_no(no: Any) -> str:
    if isinstance(no, DocumentoHTML):
        titulo = escapar_texto(no.titulo)
        corpo = "".join(_renderizar_no(item) for item in no.corpo.filhos)
        idioma = escapar_atributo(no.idioma)
        return (
            "<!doctype html>\n"
            f'<html lang="{idioma}">\n'
            "<head>\n"
            '<meta charset="utf-8">\n'
            f"<title>{titulo}</title>\n"
            "</head>\n"
            f"<body>{corpo}</body>\n"
            "</html>"
        )
    if isinstance(no, FragmentoHTML):
        return "".join(_renderizar_no(item) for item in no.filhos)
    if isinstance(no, ElementoHTML):
        attrs = _renderizar_atributos(no.atributos)
        if no.nome in _ELEMENTOS_VAZIOS:
            return f"<{no.nome}{attrs}>"
        conteudo = "".join(_renderizar_no(item) for item in no.filhos)
        return f"<{no.nome}{attrs}>{conteudo}</{no.nome}>"
    if isinstance(no, bytes):
        try:
            no = no.decode("utf-8")
        except UnicodeDecodeError as exc:
            raise ErroHTML("bytes usados como texto HTML devem estar em UTF 8") from exc
    if isinstance(no, bytearray):
        return _renderizar_no(bytes(no))
    return escapar_texto(no)


def renderizar(no: Any) -> str:
    """Serializa um nó ou valor em memória, sem efeitos externos."""
    return _renderizar_no(no)


def _normalizar_colunas(colunas: Any) -> tuple[tuple[str, str], ...]:
    if isinstance(colunas, Mapping):
        itens = tuple(colunas.items())
    else:
```

```
        if isinstance(colunas, (str, bytes, bytearray)):
            itens = (colunas,)
        else:
            try:
                itens = tuple(colunas)
            except TypeError as exc:
                raise ErroHTML("colunas deve listar campos ou pares campo rótulo") from
                  exc
    if not itens:
        raise ErroHTML("tabela requer ao menos uma coluna")

    resultado: list[tuple[str, str]] = []
    vistos: set[str] = set()
    for item in itens:
        if isinstance(item, str):
            campo, rotulo = item, item
        elif isinstance(item, Sequence) and not isinstance(item, (bytes, bytearray)) and
           len(item) == 2:
            campo, rotulo = item[0], item[1]
        else:
            raise ErroHTML("cada coluna deve ser um campo ou par campo rótulo")
        if not isinstance(campo, str) or not campo.strip():
            raise ErroHTML("nome de campo da tabela não pode ser vazio")
        campo = campo.strip()
        if campo in vistos:
            raise ErroHTML(f"coluna duplicada: {campo}")
        vistos.add(campo)
        resultado.append((campo, str(rotulo)))
    return tuple(resultado)


def _valor_registro(registro: Any, campo: str, indice: int) -> Any:
    if isinstance(registro, Mapping):
        if campo not in registro:
            raise ErroHTML(f"registro {indice} não possui campo {campo}")
        return registro[campo]
    if not hasattr(registro, campo):
        raise ErroHTML(f"registro {indice} não possui campo {campo}")
    return getattr(registro, campo)


def tabela(registros: Iterable[Any], colunas: Any) -> ElementoHTML:
    """Monta tabela a partir de registros comuns com colunas explícitas.

    Mapeamentos e objetos com atributos são aceitos. Nenhuma célula é tratada
    como HTML confiável: todo valor comum é escapado durante a serialização.
    """
    if isinstance(registros, (str, bytes, bytearray, Mapping)):
        raise ErroHTML("registros deve ser uma coleção de registros")
    try:
        linhas = tuple(registros)
    except TypeError as exc:
        raise ErroHTML("registros deve ser uma coleção de registros") from exc
    colunas_ok = _normalizar_colunas(colunas)

    cabecalho = elemento(
        "thead",
        filhos=elemento(
            "tr",
            filhos=tuple(elemento("th", filhos=rotulo) for _, rotulo in colunas_ok),
        ),
```

```
    )
    corpo_linhas: list[ElementoHTML] = []
    for indice, registro in enumerate(linhas):
        celulas = tuple(
            elemento("td", filhos=_valor_registro(registro, campo, indice))
            for campo, _ in colunas_ok
        )
        corpo_linhas.append(elemento("tr", filhos=celulas))
    corpo = elemento("tbody", filhos=tuple(corpo_linhas))
    return elemento("table", filhos=(cabecalho, corpo))


__all__ = [
    "ErroHTML",
    "ElementoHTML",
    "FragmentoHTML",
    "DocumentoHTML",
    "escapar_texto",
    "escapar_atributo",
    "elemento",
    "fragmento",
    "documento",
    "renderizar",
    "tabela",
]
```

Biblioteca `coral.web.http`

`coral.web.http.ErroHTTP`

Falha de transporte ou política do cliente HTTP da Coral.

Assinatura

Assinatura para consulta

```
ErroHTTP(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.web.http.ErroLimiteHTTP`

Resposta excedeu um limite declarado pelo programa.

Assinatura

Assinatura para consulta

```
ErroLimiteHTTP(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.web.http.ErroRedirecionamentoHTTP`

Cadeia de redirecionamento viola o contrato configurado.

Assinatura

Assinatura para consulta

```
ErroRedirecionamentoHTTP(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.web.http.LIMITE_RESPOSTA_PADRAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`8388608`

`coral.web.http.MAX_REDIRECIONAMENTOS_PADRAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`5`

`coral.web.http.RespostaHTTP`

Resposta HTTP materializada com corpo preservado em bytes.

Assinatura

Assinatura para consulta

```
RespostaHTTP(status: 'int', cabecalhos: 'tuple[tuple[str, str], ...]', corpo_bytes:
  'bytes', url: 'str', metodo: 'str', redirecionamentos: 'int' = 0) -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
status	int	POSITIONAL_OR_KEYWORD	sim	sem padrão
cabecalhos	tuple[tuple[str, str], ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
corpo_bytes	bytes	POSITIONAL_OR_KEYWORD	sim	sem padrão
url	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
metodo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
redirecionamentos	int	POSITIONAL_OR_KEYWORD	não	0

Retorno declarado

None

Atributo `status`: `int`.

Atributo `cabecalhos`: `tuple[tuple[str, str], ...]`.

Atributo `corpo_bytes`: `bytes`.

Atributo `url`: `str`.

Atributo `metodo`: `str`.

Atributo `redirecionamentos`: `int`.

Métodos públicos

Referência para leitura

```
cabecalho(self, nome: 'str', padrao: 'str | None' = None) -> 'str | None'
```

Obtém o último valor de um cabeçalho sem alterar a resposta.

Referência para leitura

```
json(self) -> 'Any'
```

Decodifica JSON sem transformar erro de formato em erro de transporte.

Referência para leitura

```
texto(self, codificacao: 'str' = 'utf-8', *, erros: 'str' = 'strict') -> 'str'
```

Decodifica o corpo sob demanda usando codificação explícita.

`coral.web.http.TEMPO_LIMITE_PADRAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`10.0`

`coral.web.http.codificar_formulario`

Codifica formulário `application/x-www-form-urlencoded` em UTF 8.

Campos repetidos e sua ordem são preservados quando a entrada é uma sequência de pares. O resultado é bytes prontos para envio explícito.

Assinatura

Assinatura para consulta

```
codificar_formulario(campos: 'Any') -> 'bytes'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
campos	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

bytes

`coral.web.http.enviar`

Atalho para POST; redirecionamentos exigem opt-in por padrão.

Assinatura

Assinatura para consulta

```
enviar(url: 'str', corpo: 'Any', cabecalhos: 'Any' = None, tempo_limite: 'float' = 10.0,
    **opcoes: 'Any') -> 'RespostaHTTP'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
url	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
corpo	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
cabecalhos	Any	POSITIONAL_OR_KEYWORD	não	None
tempo_limite	float	POSITIONAL_OR_KEYWORD	não	10.0
opcoes	Any	VAR_KEYWORD	não	sem padrão

Retorno declarado

RespostaHTTP

`obter(...)` **· módulo** `coral.web.http`

O que faz

Faz um pedido HTTP GET com limites explícitos.

Assinatura

Assinatura para consulta

```
obter(url: 'str', cabecalhos: 'Any' = None, tempo_limite: 'float' = 10.0, **opcoes:
  'Any') -> 'RespostaHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
url	str	sim	sem padrão	URL do recurso HTTP.
cabecalhos	Any	não	None	Cabeçalhos adicionais da resposta.
tempo_limite	float	não	10.0	Tempo limite finito da operação, em segundos.
opcoes	Any	não	sem padrão	Opções nomeadas aceitas pelo contrato chamado, conforme a referência de ServidorHTTP ou requisitar nesta parte.

Retorno

RespostaHTTP, com status, cabeçalhos e corpo em bytes; a interpretação do conteúdo é solicitada por métodos da resposta.

Exemplo

Código Coral

```
# Exemplo de HTTP no loopback
de coral.web.servidor importe criar_servidor, resposta_texto
de coral.web.http importe obter

crie a função pagina com requisicao
    retorne resposta_texto("Coral local")
fim

defina servidor como criar_servidor()
execute servidor.adicionar_rota("GET", "/", pagina)
execute servidor.iniciar()
tente
    defina resposta como obter(servidor.url_base + "/", tempo_limite=2,
      limite_resposta=4096)
    mostre resposta.status
    mostre resposta.texto()
finalmente
    execute servidor.encerrar()
fim
```

Resultado

Saída esperada

```
200
Coral local
```

Erros

ErroHTTP para falha de transporte ou limites no cliente. O servidor de desenvolvimento não substitui um servidor de produção.

Atenção

Este exemplo completo usa somente o loopback e encerra o servidor no bloco finalmente. HTTP local não comprova TLS ou segurança de um serviço público. Consulte o Volume II, Parte II, seções coral.web.http e coral.web.servidor.

`coral.web.http.requisitar`

Executa uma requisição HTTP limitada e retorna resposta inspecionável.

Respostas HTTP 3xx/4xx/5xx não são convertidas em exceção de transporte. O corpo permanece em bytes e só é decodificado por `texto ou json`.

Assinatura

Assinatura para consulta

```
requisitar(metodo: 'str', url: 'str', cabecalhos: 'Any' = None, corpo: 'Any' = None,
  tempo_limite: 'float' = 10.0, *, codificacao_corpo: 'str | None' = None,
  limite_resposta: 'int' = 8388608, seguir_redirecionamentos: 'bool | None' = None,
  max_redirecionamentos: 'int' = 5, proxy: 'Any' = None, contexto_tls: 'ssl.SSLContext |
  None' = None) -> 'RespostaHTTP'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
metodo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
url	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
cabecalhos	Any	POSITIONAL_OR_KEYWORD	não	None
corpo	Any	POSITIONAL_OR_KEYWORD	não	None
tempo_limite	float	POSITIONAL_OR_KEYWORD	não	10.0
codificacao_corpo	str | None	KEYWORD_ONLY	não	None
limite_resposta	int	KEYWORD_ONLY	não	8388608
seguir_redirecionamentos	bool | None	KEYWORD_ONLY	não	None
max_redirecionamentos	int	KEYWORD_ONLY	não	5
proxy	Any	KEYWORD_ONLY	não	None
contexto_tls	ssl.SSLContext | None	KEYWORD_ONLY	não	None

Retorno declarado

RespostaHTTP

Consultar a implementação canônica de coral.web.http

Implementação canônica de `coral.web.http`

Código Python

```
from __future__ import annotations

from typing import Any

from .comum import ErroCoral, ErroOperacaoCoral


class ErroCoralRuntime(ErroCoral):
    """Base dos erros controlados expostos pelo runtime base da Coral."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroEntrada(ErroCoralRuntime):
    """Falha ao obter entrada para um programa Coral."""


class FimDeEntrada(ErroEntrada, EOFError):
    """A fonte de entrada terminou antes de produzir outra linha."""



class ErroConversao(ErroCoralRuntime, ValueError):
    """Um valor não pôde ser convertido para o tipo Coral solicitado."""

    def __init__(
        self,
        valor: Any,
        destino: str,
        mensagem: str | None = None,
        *,
        causa: BaseException | None = None,
    ):
        self.valor = valor
        self.destino = destino
        super().__init__(
            mensagem or f"não foi possível converter o valor para {destino}",
            causa=causa,
        )


class ErroFormato(ErroCoralRuntime, ValueError):
    """Dados ou texto não obedecem ao formato esperado."""


class ErroValor(ErroCoralRuntime, ValueError):
    """O tipo é aceito, mas o valor não é válido para a operação."""


```

```
class ErroOperacao(ErroOperacaoCoral):
    """Falha operacional com preservação opcional da causa técnica."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroArquivo(ErroOperacao):
    """Falha em uma operação com arquivo ou caminho."""


__all__ = [
    "ErroCoralRuntime",
    "ErroEntrada",
    "FimDeEntrada",
    "ErroConversao",
    "ErroArquivo",
    "ErroFormato",
    "ErroValor",
    "ErroOperacao",
]

"""Cliente HTTP geral e limitado da Coral 1.6.8.

O módulo só realiza acesso à rede quando uma função de requisição é chamada.
Importá-lo não abre sockets, não inicia servidor e não consulta recursos externos.
"""
from __future__ import annotations

from collections.abc import Mapping, Sequence
from dataclasses import dataclass
import codecs
import json as _json
import re
import socket
import ssl
from typing import Any
from urllib.error import HTTPError, URLError
from urllib.request import (
    HTTPRedirectHandler,
    HTTPSHandler,
    ProxyHandler,
    Request,
    build_opener,
)

from ..erros import ErroFormato, ErroOperacao, ErroValor
from .url import ErroURL, codificar_consulta, decompor


TEMPO_LIMITE_PADRAO = 10.0
LIMITE_RESPOSTA_PADRAO = 8 * 1024 * 1024
MAX_REDIRECIONAMENTOS_PADRAO = 5

_TOKEN_HTTP = re.compile(r"^[!#$%&'*+.^_`|~0-9A-Za-z-]+$")


class ErroHTTP(ErroOperacao):
    """Falha de transporte ou política do cliente HTTP da Coral."""
```

```
class ErroLimiteHTTP(ErroHTTP):
    """Resposta excedeu um limite declarado pelo programa."""


class ErroRedirecionamentoHTTP(ErroHTTP):
    """Cadeia de redirecionamento viola o contrato configurado."""


@dataclass(frozen=True, slots=True)
class RespostaHTTP:
    """Resposta HTTP materializada com corpo preservado em bytes."""

    status: int
    cabecalhos: tuple[tuple[str, str], ...]
    corpo_bytes: bytes
    url: str
    metodo: str
    redirecionamentos: int = 0

    @property
    def sucesso(self) -> bool:
        return 200 <= self.status < 300

    def cabecalho(self, nome: str, padrao: str | None = None) -> str | None:
        """Obtém o último valor de um cabeçalho sem alterar a resposta."""
        alvo = str(nome).casefold()
        encontrados = [valor for chave, valor in self.cabecalhos if chave.casefold() ==
          alvo]
        return encontrados[-1] if encontrados else padrao

    def texto(self, codificacao: str = "utf-8", *, erros: str = "strict") -> str:
        """Decodifica o corpo sob demanda usando codificação explícita."""
        try:
            codecs.lookup(codificacao)
        except LookupError as exc:
            raise ErroFormato(f"codificação HTTP desconhecida: {codificacao}",
              causa=exc) from exc
        try:
            return self.corpo_bytes.decode(codificacao, errors=erros)
        except (UnicodeDecodeError, LookupError) as exc:
            raise ErroFormato(
                f"corpo HTTP não pode ser decodificado como {codificacao}", causa=exc
            ) from exc

    def json(self) -> Any:
        """Decodifica JSON sem transformar erro de formato em erro de transporte."""
        try:
            # json.loads aceita bytes e reconhece UTF 8/16/32 e BOMs suportados.
            return _json.loads(self.corpo_bytes)
        except (UnicodeDecodeError, _json.JSONDecodeError) as exc:
            raise ErroFormato("resposta HTTP não contém JSON válido", causa=exc) from
              exc


class _RedirecionadorLimitado(HTTPRedirectHandler):
    def __init__(self, *, seguir: bool, maximo: int):
        super().__init__()
        self.seguir = seguir
        self.maximo = maximo
```

```
        self.quantidade = 0

    def redirect_request(self, req, fp, code, msg, headers, newurl):  # noqa: ANN001
        if not self.seguir:
            return None
        if self.quantidade >= self.maximo:
            raise ErroRedirecionamentoHTTP(
                f"requisição excedeu {self.maximo} redirecionamentos"
            )
        try:
            partes = decompor(newurl)
        except ErroURL as exc:
            raise ErroRedirecionamentoHTTP("redirecionamento aponta para URL inválida",
              causa=exc) from exc
        if partes.esquema not in {"http", "https"}:
            raise ErroRedirecionamentoHTTP(
                "redirecionamento HTTP só pode apontar para http ou https"
            )
        self.quantidade += 1
        return super().redirect_request(req, fp, code, msg, headers, newurl)


def _validar_numero_positivo(valor: Any, nome: str, *, permitir_zero: bool = False) ->
  float:
    if isinstance(valor, bool):
        raise ErroValor(f"{nome} deve ser numérico")
    try:
        numero = float(valor)
    except (TypeError, ValueError) as exc:
        raise ErroValor(f"{nome} deve ser numérico") from exc
    minimo_ok = numero >= 0 if permitir_zero else numero > 0
    if not minimo_ok or numero == float("inf") or numero != numero:
        relacao = "maior ou igual a zero" if permitir_zero else "maior que zero"
        raise ErroValor(f"{nome} deve ser finito e {relacao}")
    return numero


def _validar_inteiro_nao_negativo(valor: Any, nome: str) -> int:
    if isinstance(valor, bool) or not isinstance(valor, int) or valor < 0:
        raise ErroValor(f"{nome} deve ser inteiro maior ou igual a zero")
    return valor


def _normalizar_metodo(metodo: Any) -> str:
    if not isinstance(metodo, str):
        raise ErroValor("método HTTP deve ser texto")
    resultado = metodo.strip().upper()
    if not resultado or not _TOKEN_HTTP.fullmatch(resultado):
        raise ErroValor("método HTTP inválido")
    return resultado


def _normalizar_url(endereco: Any) -> str:
    if not isinstance(endereco, str):
        raise ErroValor("URL HTTP deve ser texto")
    texto = endereco.strip()
    try:
        partes = decompor(texto)
    except ErroURL as exc:
        raise ErroValor("URL HTTP inválida", causa=exc) from exc
    if partes.esquema not in {"http", "https"}:
```

```
        raise ErroValor("cliente HTTP aceita somente esquemas http e https")
    if not partes.host:
        raise ErroValor("URL HTTP precisa declarar um host")
    # Credenciais embutidas são fáceis de vazar em logs e erros. O perfil base
    # usa cabeçalhos explícitos quando autenticação for realmente necessária.
    if "@" in partes.autoridade.rsplit("]", 1)[-1] and partes.autoridade.count("@"):
        raise ErroValor("URL HTTP não deve conter credenciais embutidas")
    return texto


def _normalizar_cabecalhos(cabecalhos: Any) -> tuple[tuple[str, str], ...]:
    if cabecalhos is None:
        return ()
    if isinstance(cabecalhos, Mapping):
        itens = tuple(cabecalhos.items())
    else:
        if isinstance(cabecalhos, (str, bytes, bytearray)):
            raise ErroValor("cabeçalhos HTTP devem ser mapeamento ou pares nome valor")
        try:
            itens = tuple(cabecalhos)
        except TypeError as exc:
            raise ErroValor("cabeçalhos HTTP devem ser mapeamento ou pares nome valor")
              from exc

    saida: list[tuple[str, str]] = []
    for item in itens:
        if (
            not isinstance(item, Sequence)
            or isinstance(item, (str, bytes, bytearray))
            or len(item) != 2
        ):
            raise ErroValor("cada cabeçalho HTTP deve ser um par nome valor")
        nome = str(item[0]).strip()
        valor = str(item[1]).strip()
        if not _TOKEN_HTTP.fullmatch(nome):
            raise ErroValor("nome de cabeçalho HTTP inválido")
        if "\r" in valor or "\n" in valor or "\x00" in valor:
            raise ErroValor(f"valor inválido para cabeçalho HTTP {nome}")
        saida.append((nome, valor))
    return tuple(saida)


def _normalizar_corpo(corpo: Any, codificacao_corpo: str | None) -> bytes | None:
    if corpo is None:
        return None
    if isinstance(corpo, bytes):
        return corpo
    if isinstance(corpo, bytearray):
        return bytes(corpo)
    if isinstance(corpo, memoryview):
        return corpo.tobytes()
    if isinstance(corpo, str):
        if not codificacao_corpo:
            raise ErroValor(
                "corpo HTTP em texto exige codificacao_corpo explícita, por exemplo
                  'utf-8'"
            )
        try:
            codecs.lookup(codificacao_corpo)
            return corpo.encode(codificacao_corpo)
        except LookupError as exc:
```

```
            raise ErroValor(f"codificação de corpo desconhecida: {codificacao_corpo}")
              from exc
    raise ErroValor("corpo HTTP deve ser bytes, texto ou vazio")


def _normalizar_proxy(proxy: Any) -> dict[str, str]:
    if proxy is None:
        return {}
    if isinstance(proxy, str):
        return {"http": proxy, "https": proxy}
    if not isinstance(proxy, Mapping):
        raise ErroValor("proxy HTTP deve ser URL textual ou mapeamento por esquema")
    resultado: dict[str, str] = {}
    for esquema, endereco in proxy.items():
        chave = str(esquema).casefold().strip()
        if chave not in {"http", "https"}:
            raise ErroValor("proxy só pode ser configurado para http ou https")
        valor = _normalizar_url(str(endereco))
        resultado[chave] = valor
    return resultado


def _cabecalhos_resposta(resposta: Any) -> tuple[tuple[str, str], ...]:
    mensagem = getattr(resposta, "headers", None)
    if mensagem is None:
        return ()
    return tuple((str(nome), str(valor)) for nome, valor in mensagem.items())


def _ler_corpo_limitado(resposta: Any, limite: int) -> bytes:
    tamanho = resposta.headers.get("Content-Length") if getattr(resposta, "headers",
      None) else None
    if tamanho is not None:
        try:
            declarado = int(tamanho)
        except ValueError:
            declarado = None
        if declarado is not None and declarado > limite:
            raise ErroLimiteHTTP(
                f"resposta HTTP declara {declarado} bytes, acima do limite de {limite}"
            )
    corpo = resposta.read(limite + 1)
    if len(corpo) > limite:
        raise ErroLimiteHTTP(f"resposta HTTP excedeu o limite de {limite} bytes")
    return corpo


def _resposta_materializada(
    resposta: Any,
    *,
    metodo: str,
    redirecionamentos: int,
    limite_resposta: int,
) -> RespostaHTTP:
    corpo = _ler_corpo_limitado(resposta, limite_resposta)
    status = int(getattr(resposta, "status", resposta.getcode()))
    url_final = str(getattr(resposta, "url", resposta.geturl()))
    return RespostaHTTP(
        status=status,
        cabecalhos=_cabecalhos_resposta(resposta),
        corpo_bytes=corpo,
```

```
        url=url_final,
        metodo=metodo,
        redirecionamentos=redirecionamentos,
    )


def requisitar(
    metodo: str,
    url: str,
    cabecalhos: Any = None,
    corpo: Any = None,
    tempo_limite: float = TEMPO_LIMITE_PADRAO,
    *,
    codificacao_corpo: str | None = None,
    limite_resposta: int = LIMITE_RESPOSTA_PADRAO,
    seguir_redirecionamentos: bool | None = None,
    max_redirecionamentos: int = MAX_REDIRECIONAMENTOS_PADRAO,
    proxy: Any = None,
    contexto_tls: ssl.SSLContext | None = None,
) -> RespostaHTTP:
    """Executa uma requisição HTTP limitada e retorna resposta inspecionável.

    Respostas HTTP 3xx/4xx/5xx não são convertidas em exceção de transporte.
    O corpo permanece em bytes e só é decodificado por ``texto`` ou ``json``.
    """
    metodo_ok = _normalizar_metodo(metodo)
    url_ok = _normalizar_url(url)
    timeout = _validar_numero_positivo(tempo_limite, "tempo_limite")
    limite = _validar_inteiro_nao_negativo(limite_resposta, "limite_resposta")
    max_redirs = _validar_inteiro_nao_negativo(max_redirecionamentos,
      "max_redirecionamentos")
    corpo_bytes = _normalizar_corpo(corpo, codificacao_corpo)
    headers = list(_normalizar_cabecalhos(cabecalhos))

    if corpo_bytes is not None and codificacao_corpo:
        if not any(nome.casefold() == "content-type" for nome, _ in headers):
            headers.append(("Content-Type", f"text/plain; charset={codificacao_corpo}"))

    if seguir_redirecionamentos is None:
        # Métodos de leitura seguem redirecionamento por padrão. Métodos com
        # potencial de efeito exigem opt-in para evitar repetição implícita.
        seguir = metodo_ok in {"GET", "HEAD", "OPTIONS"}
    elif isinstance(seguir_redirecionamentos, bool):
        seguir = seguir_redirecionamentos
    else:
        raise ErroValor("seguir_redirecionamentos deve ser verdadeiro ou falso")

    redirecionador = _RedirecionadorLimitado(seguir=seguir, maximo=max_redirs)
    proxy_handler = ProxyHandler(_normalizar_proxy(proxy))
    contexto = contexto_tls if contexto_tls is not None else
      ssl.create_default_context()
    if not isinstance(contexto, ssl.SSLContext):
        raise ErroValor("contexto_tls deve ser ssl.SSLContext")

    opener = build_opener(proxy_handler, HTTPSHandler(context=contexto), redirecionador)
    pedido = Request(url_ok, data=corpo_bytes, method=metodo_ok)
    for nome, valor in headers:
        pedido.add_header(nome, valor)

    try:
        with opener.open(pedido, timeout=timeout) as resposta:
```

```
            return _resposta_materializada(
                resposta,
                metodo=metodo_ok,
                redirecionamentos=redirecionador.quantidade,
                limite_resposta=limite,
            )
    except HTTPError as exc:
        # HTTPError também é uma resposta válida para status não 2xx e para
        # redirecionamentos que a política decidiu não seguir.
        try:
            return _resposta_materializada(
                exc,
                metodo=metodo_ok,
                redirecionamentos=redirecionador.quantidade,
                limite_resposta=limite,
            )
        finally:
            exc.close()
    except ErroHTTP:
        raise
    except (socket.timeout, TimeoutError) as exc:
        raise ErroHTTP("requisição HTTP excedeu o tempo limite", causa=exc) from exc
    except ssl.SSLError as exc:
        raise ErroHTTP("falha de verificação ou negociação TLS", causa=exc) from exc
    except URLError as exc:
        motivo = getattr(exc, "reason", None)
        if isinstance(motivo, (socket.timeout, TimeoutError)):
            raise ErroHTTP("requisição HTTP excedeu o tempo limite", causa=exc) from exc
        if isinstance(motivo, ssl.SSLError):
            raise ErroHTTP("falha de verificação ou negociação TLS", causa=exc) from exc
        raise ErroHTTP("falha de transporte HTTP", causa=exc) from exc
    except OSError as exc:
        raise ErroHTTP("falha de transporte HTTP", causa=exc) from exc


def obter(
    url: str,
    cabecalhos: Any = None,
    tempo_limite: float = TEMPO_LIMITE_PADRAO,
    **opcoes: Any,
) -> RespostaHTTP:
    """Atalho para GET com as mesmas regras de :func:`requisitar`."""
    return requisitar("GET", url, cabecalhos, None, tempo_limite, **opcoes)


def enviar(
    url: str,
    corpo: Any,
    cabecalhos: Any = None,
    tempo_limite: float = TEMPO_LIMITE_PADRAO,
    **opcoes: Any,
) -> RespostaHTTP:
    """Atalho para POST; redirecionamentos exigem opt-in por padrão."""
    return requisitar("POST", url, cabecalhos, corpo, tempo_limite, **opcoes)


def codificar_formulario(campos: Any) -> bytes:
    """Codifica formulário ``application/x-www-form-urlencoded`` em UTF 8.

    Campos repetidos e sua ordem são preservados quando a entrada é uma
    sequência de pares. O resultado é bytes prontos para envio explícito.
```

```
    """
    return codificar_consulta(campos).encode("ascii")


__all__ = [
    "ErroHTTP",
    "ErroLimiteHTTP",
    "ErroRedirecionamentoHTTP",
    "RespostaHTTP",
    "TEMPO_LIMITE_PADRAO",
    "LIMITE_RESPOSTA_PADRAO",
    "MAX_REDIRECIONAMENTOS_PADRAO",
    "requisitar",
    "obter",
    "enviar",
    "codificar_formulario",
]

"""Manipulação local de URLs e consultas para a Coral 1.6.6.

Todas as operações deste módulo são puramente textuais. Nenhuma função resolve
DNS, abre socket, realiza requisição ou consulta recurso externo.
"""
from __future__ import annotations

from collections.abc import Iterable, Mapping, Sequence
from dataclasses import dataclass
import re
from typing import Any
from urllib.parse import parse_qsl, urlencode, urljoin, urlsplit, urlunsplit

from ..erros import ErroValor


class ErroURL(ErroValor):
    """URL ou consulta não obedece ao contrato esperado."""


@dataclass(frozen=True, slots=True)
class EnderecoURL:
    """Componentes textuais de uma URL com esquema explícito."""

    esquema: str
    autoridade: str
    host: str | None
    porta: int | None
    caminho: str
    consulta: str
    fragmento: str

    @property
    def parametros(self) -> tuple[tuple[str, str], ...]:
        """Consulta decodificada preservando ordem e chaves repetidas."""
        return ler_consulta(self.consulta)

    def reconstruir(self) -> str:
        """Reconstrói a URL sem acessar a rede."""
        return urlunsplit((self.esquema, self.autoridade, self.caminho, self.consulta,
          self.fragmento))
```

```
_PERCENTUAL_INVALIDO = re.compile(r"%(?![0-9A-Fa-f]{2})")
_ESQUEMA = re.compile(r"^[A-Za-z][A-Za-z0-9+.-]*$")


def _texto(valor: Any, contexto: str) -> str:
    if isinstance(valor, bytes):
        try:
            return valor.decode("utf-8")
        except UnicodeDecodeError as exc:
            raise ErroURL(f"{contexto} em bytes deve estar em UTF 8") from exc
    if isinstance(valor, bytearray):
        return _texto(bytes(valor), contexto)
    if valor is None:
        return ""
    return str(valor)


def _pares_parametros(parametros: Any) -> tuple[tuple[str, str], ...]:
    if isinstance(parametros, Mapping):
        bruto: list[tuple[Any, Any]] = []
        for chave, valor in parametros.items():
            if isinstance(valor, (list, tuple)):
                bruto.extend((chave, item) for item in valor)
            else:
                bruto.append((chave, valor))
        itens = tuple(bruto)
    else:
        if isinstance(parametros, (str, bytes, bytearray)):
            raise ErroURL("parâmetros devem ser mapeamento ou sequência de pares")
        try:
            itens = tuple(parametros)
        except TypeError as exc:
            raise ErroURL("parâmetros devem ser mapeamento ou sequência de pares") from
              exc

    resultado: list[tuple[str, str]] = []
    for item in itens:
        if not isinstance(item, Sequence) or isinstance(item, (str, bytes, bytearray))
          or len(item) != 2:
            raise ErroURL("cada parâmetro deve ser um par chave valor")
        chave = _texto(item[0], "chave de consulta")
        valor = _texto(item[1], "valor de consulta")
        resultado.append((chave, valor))
    return tuple(resultado)


def codificar_consulta(parametros: Any) -> str:
    """Codifica pares ordenados em UTF 8 preservando chaves repetidas."""
    pares = _pares_parametros(parametros)
    return urlencode(pares, doseq=False, encoding="utf-8", errors="strict")


def ler_consulta(texto: Any) -> tuple[tuple[str, str], ...]:
    """Lê consulta em UTF 8, preservando ordem, vazios e duplicatas."""
    bruto = _texto(texto, "consulta")
    if bruto.startswith("?"):
        bruto = bruto[1:]
    if _PERCENTUAL_INVALIDO.search(bruto):
        raise ErroURL("consulta contém escape percentual inválido")
    try:
        pares = parse_qsl(
```

```
            bruto,
            keep_blank_values=True,
            strict_parsing=False,
            encoding="utf-8",
            errors="strict",
        )
    except (UnicodeDecodeError, ValueError) as exc:
        raise ErroURL("consulta não contém codificação UTF 8 válida") from exc
    return tuple((str(chave), str(valor)) for chave, valor in pares)


def decompor(endereco: Any) -> EnderecoURL:
    """Decompõe uma URL com esquema explícito sem realizar acesso externo."""
    texto = _texto(endereco, "endereço").strip()
    if not texto:
        raise ErroURL("endereço não pode ser vazio")
    try:
        partes = urlsplit(texto)
    except ValueError as exc:
        raise ErroURL("endereço URL inválido") from exc
    if not partes.scheme or not _ESQUEMA.fullmatch(partes.scheme):
        raise ErroURL("endereço deve possuir esquema explícito")
    try:
        porta = partes.port
        host = partes.hostname
    except ValueError as exc:
        raise ErroURL("porta da URL é inválida") from exc
    return EnderecoURL(
        esquema=partes.scheme.casefold(),
        autoridade=partes.netloc,
        host=host,
        porta=porta,
        caminho=partes.path,
        consulta=partes.query,
        fragmento=partes.fragment,
    )


def juntar(base: Any, referencia: Any) -> str:
    """Resolve referência contra uma base usando apenas regras textuais de URL."""
    base_texto = _texto(base, "URL base").strip()
    referencia_texto = _texto(referencia, "referência").strip()
    # A base precisa ser uma URL real e explícita; a referência pode ser relativa.
    decompor(base_texto)
    try:
        resultado = urljoin(base_texto, referencia_texto)
    except ValueError as exc:
        raise ErroURL("não foi possível juntar a URL base e a referência") from exc
    # A resolução deve continuar produzindo um endereço com esquema explícito.
    decompor(resultado)
    return resultado


__all__ = [
    "ErroURL",
    "EnderecoURL",
    "codificar_consulta",
    "ler_consulta",
    "juntar",
    "decompor",
]
```

Biblioteca `coral.web.servidor`

`coral.web.servidor.ErroLimiteCorpoHTTP`

Corpo de requisição excedeu o limite declarado pelo servidor.

Assinatura

Assinatura para consulta

```
ErroLimiteCorpoHTTP(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.web.servidor.ErroRotaHTTP`

Definição de rota não obedece ao contrato literal do servidor.

Assinatura

Assinatura para consulta

```
ErroRotaHTTP(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.web.servidor.ErroServidorHTTP`

Falha operacional do servidor HTTP local da Coral.

Assinatura

Assinatura para consulta

```
ErroServidorHTTP(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.web.servidor.HOST_PADRAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`'127.0.0.1'`

`coral.web.servidor.LIMITE_ARQUIVO_PADRAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`8388608`

`coral.web.servidor.LIMITE_CORPO_PADRAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`1048576`

`coral.web.servidor.PORTA_PADRAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`0`

`coral.web.servidor.RequisicaoHTTP`

Requisição materializada antes da chamada ao tratador da rota.

Assinatura

Assinatura para consulta

```
RequisicaoHTTP(metodo: 'str', caminho: 'str', consulta: 'tuple[tuple[str, str], ...]',
  cabecalhos: 'tuple[tuple[str, str], ...]', corpo_bytes: 'bytes', cliente: 'str') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
metodo	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
caminho	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
consulta	tuple[tuple[str, str], ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
cabecalhos	tuple[tuple[str, str], ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
corpo_bytes	bytes	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
cliente	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `metodo`: `str`.

Atributo `caminho`: `str`.

Atributo `consulta`: `tuple[tuple[str, str], ...]`.

Atributo `cabecalhos`: `tuple[tuple[str, str], ...]`.

Atributo `corpo_bytes`: `bytes`.

Atributo `cliente`: `str`.

Métodos públicos

Referência para leitura

```
cabecalho(self, nome: 'str', padrao: 'str | None' = None) -> 'str | None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
formulario(self) -> 'tuple[tuple[str, str], ...]'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
json(self) -> 'Any'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
texto(self, codificacao: 'str' = 'utf-8', *, erros: 'str' = 'strict') -> 'str'
```

Método público sem descrição própria no runtime.

`coral.web.servidor.RespostaServidorHTTP`

Resposta pronta para serialização pelo servidor local.

Assinatura

Assinatura para consulta

```
RespostaServidorHTTP(status: 'int', cabecalhos: 'tuple[tuple[str, str], ...]',
  corpo_bytes: 'bytes') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
status	int	POSITIONAL_OR_KEYWORD	sim	sem padrão

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
cabecalhos	tuple[tuple[str, str], ...]	POSITIONAL_OR_KEYWORD	sim	sem padrão
corpo_bytes	bytes	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `status`: `int`.

Atributo `cabecalhos`: `tuple[tuple[str, str], ...]`.

Atributo `corpo_bytes`: `bytes`.

`coral.web.servidor.ServidorHTTP`

Servidor HTTP pequeno, explícito e orientado a desenvolvimento local.

Assinatura

Assinatura para consulta

```
ServidorHTTP(host: 'str' = '127.0.0.1', porta: 'int' = 0, *, limite_corpo: 'int' =
  1048576, tempo_limite_corpo: 'float' = 10.0, raiz_arquivos: 'Any' = None,
  permitir_externo: 'bool' = False)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
host	str	POSITIONAL_OR_KEYWORD	não	'127.0.0.1'
porta	int	POSITIONAL_OR_KEYWORD	não	0
limite_corpo	int	KEYWORD_ONLY	não	1048576
tempo_limite_corpo	float	KEYWORD_ONLY	não	10.0
raiz_arquivos	Any	KEYWORD_ONLY	não	None
permitir_externo	bool	KEYWORD_ONLY	não	False

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
adicionar_arquivo(self, caminho_rota: 'str', caminho_arquivo: 'Any', *, limite_arquivo:
  'int' = 8388608, cabecalhos: 'Any' = None) -> "'ServidorHTTP'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
adicionar_rota(self, metodo: 'str', caminho: 'str', tratador: 'TratadorHTTP') ->
  "'ServidorHTTP'"
```

Método público sem descrição própria no runtime.

Referência para leitura

```
encerrar(self) -> 'None'
```

Método público sem descrição própria no runtime.

Referência para leitura

```
iniciar(self) -> "'ServidorHTTP'"
```

Método público sem descrição própria no runtime.

`coral.web.servidor.TEMPO_LIMITE_CORPO_PADRAO`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Valor

`10.0`

`criar_servidor(...)` · **módulo** `coral.web.servidor`

O que faz

Cria a configuração de um servidor HTTP pequeno para desenvolvimento local. Não começa a escuta até iniciar.

Assinatura

Assinatura para consulta

```
criar_servidor(host: 'str' = '127.0.0.1', porta: 'int' = 0, **opcoes: 'Any') ->
  'ServidorHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
host	str	não	'127.0.0.1'	Endereço de escuta; o padrão limita o servidor ao loopback.
porta	int	não	0	Porta de escuta; zero pede uma porta efêmera livre.
opcoes	Any	não	sem padrão	Opções nomeadas aceitas pelo contrato chamado, conforme a referência de ServidorHTTP ou requisitar nesta parte.

Retorno

ServidorHTTP.

Exemplo

Código Coral

```
# Exemplo de HTTP no loopback
de coral.web.servidor importe criar_servidor, resposta_texto
de coral.web.http importe obter

crie a função pagina com requisicao
    retorne resposta_texto("Coral local")
fim

defina servidor como criar_servidor()
execute servidor.adicionar_rota("GET", "/", pagina)
execute servidor.iniciar()
tente
    defina resposta como obter(servidor.url_base + "/", tempo_limite=2,
      limite_resposta=4096)
    mostre resposta.status
    mostre resposta.texto()
finalmente
    execute servidor.encerrar()
fim
```

Resultado

Saída esperada

```
200
Coral local
```

Erros

ErroHTTP para falha de transporte ou limites no cliente. O servidor de desenvolvimento não substitui um servidor de produção.

Atenção

Este exemplo completo usa somente o loopback e encerra o servidor no bloco finalmente. HTTP local não comprova TLS ou segurança de um serviço público. Consulte o Volume II, Parte II, seções coral.web.http e coral.web.servidor.

`coral.web.servidor.resposta_arquivo`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
resposta_arquivo(raiz: 'Any', caminho_relativo: 'Any', status: 'int' = 200, cabecalhos:
  'Any' = None, *, limite_arquivo: 'int' = 8388608) -> 'RespostaServidorHTTP'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
raiz	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
caminho_relativo	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
status	int	POSITIONAL_OR_KEYWORD	não	200
cabecalhos	Any	POSITIONAL_OR_KEYWORD	não	None
limite_arquivo	int	KEYWORD_ONLY	não	8388608

Retorno declarado

RespostaServidorHTTP

`coral.web.servidor.resposta_bytes`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
resposta_bytes(corpo: 'Any', status: 'int' = 200, cabecalhos: 'Any' = None, *,
  tipo_conteudo: 'str' = 'application/octet-stream') -> 'RespostaServidorHTTP'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
corpo	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
status	int	POSITIONAL_OR_KEYWORD	não	200
cabecalhos	Any	POSITIONAL_OR_KEYWORD	não	None
tipo_conteudo	str	KEYWORD_ONLY	não	'application/octet-stream'

Retorno declarado

RespostaServidorHTTP

`coral.web.servidor.resposta_grafico`

Adapta um gráfico Coral para resposta HTML sem iniciar servidor.

Plotly continua opcional e é carregado somente nesta chamada por meio de `coral.graficos.para_html. A resposta recebe Cache-Control: no-store` por padrão porque o HTML autossuficiente incorpora os próprios dados.

Assinatura

Assinatura para consulta

```
resposta_grafico(grafico: 'Any', status: 'int' = 200, cabecalhos: 'Any' = None, *,
  limite_pontos: 'int' = 200000) -> 'RespostaServidorHTTP'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
grafico	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
status	int	POSITIONAL_OR_KEYWORD	não	200
cabecalhos	Any	POSITIONAL_OR_KEYWORD	não	None
limite_pontos	int	KEYWORD_ONLY	não	200000

Retorno declarado

RespostaServidorHTTP

`coral.web.servidor.resposta_html`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
resposta_html(conteudo: 'Any', status: 'int' = 200, cabecalhos: 'Any' = None) ->
  'RespostaServidorHTTP'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
conteudo	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
status	int	POSITIONAL_OR_KEYWORD	não	200
cabecalhos	Any	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

RespostaServidorHTTP

`coral.web.servidor.resposta_json`

Símbolo público exportado pelo runtime. Seu contrato de uso é apresentado no código canônico incorporado nesta parte.

Assinatura

Assinatura para consulta

```
resposta_json(valor: 'Any', status: 'int' = 200, cabecalhos: 'Any' = None) ->
  'RespostaServidorHTTP'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
valor	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
status	int	POSITIONAL_OR_KEYWORD	não	200
cabecalhos	Any	POSITIONAL_OR_KEYWORD	não	None

Retorno declarado

RespostaServidorHTTP

`resposta_texto(...)` · módulo `coral.web.servidor`

O que faz

Cria uma resposta HTTP cujo corpo é texto, com tipo de conteúdo e codificação definidos.

Assinatura

Assinatura para consulta

```
resposta_texto(texto: 'Any', status: 'int' = 200, cabecalhos: 'Any' = None, *,
  codificacao: 'str' = 'utf-8') -> 'RespostaServidorHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
texto	Any	sim	sem padrão	Texto a enviar no corpo da resposta.
status	int	não	200	Código HTTP da resposta.
cabecalhos	Any	não	None	Cabeçalhos adicionais da resposta.
codificacao	str	não	'utf-8'	Codificação de texto usada na resposta.

Retorno

RespostaServidorHTTP.

Exemplo

Código Coral

```
# Exemplo de HTTP no loopback
de coral.web.servidor importe criar_servidor, resposta_texto
de coral.web.http importe obter

crie a função pagina com requisicao
    retorne resposta_texto("Coral local")
fim

defina servidor como criar_servidor()
execute servidor.adicionar_rota("GET", "/", pagina)
execute servidor.iniciar()
tente
    defina resposta como obter(servidor.url_base + "/", tempo_limite=2,
      limite_resposta=4096)
    mostre resposta.status
    mostre resposta.texto()
finalmente
    execute servidor.encerrar()
fim
```

Resultado

Saída esperada

```
200
Coral local
```

Erros

ErroHTTP para falha de transporte ou limites no cliente. O servidor de desenvolvimento não substitui um servidor de produção.

Atenção

Este exemplo completo usa somente o loopback e encerra o servidor no bloco finalmente. HTTP local não comprova TLS ou segurança de um serviço público. Consulte o Volume II, Parte II, seções coral.web.http e coral.web.servidor.

Consultar a implementação canônica de coral.web.servidor

Implementação canônica de `coral.web.servidor`

Código Python

```
from __future__ import annotations

from typing import Any

from .comum import ErroCoral, ErroOperacaoCoral


class ErroCoralRuntime(ErroCoral):
    """Base dos erros controlados expostos pelo runtime base da Coral."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroEntrada(ErroCoralRuntime):
    """Falha ao obter entrada para um programa Coral."""


class FimDeEntrada(ErroEntrada, EOFError):
    """A fonte de entrada terminou antes de produzir outra linha."""



class ErroConversao(ErroCoralRuntime, ValueError):
    """Um valor não pôde ser convertido para o tipo Coral solicitado."""

    def __init__(
        self,
        valor: Any,
        destino: str,
        mensagem: str | None = None,
        *,
        causa: BaseException | None = None,
    ):
        self.valor = valor
        self.destino = destino
        super().__init__(
            mensagem or f"não foi possível converter o valor para {destino}",
            causa=causa,
        )


class ErroFormato(ErroCoralRuntime, ValueError):
    """Dados ou texto não obedecem ao formato esperado."""


class ErroValor(ErroCoralRuntime, ValueError):
    """O tipo é aceito, mas o valor não é válido para a operação."""


class ErroOperacao(ErroOperacaoCoral):
    """Falha operacional com preservação opcional da causa técnica."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa

```

```
class ErroArquivo(ErroOperacao):
    """Falha em uma operação com arquivo ou caminho."""


__all__ = [
    "ErroCoralRuntime",
    "ErroEntrada",
    "FimDeEntrada",
    "ErroConversao",
    "ErroArquivo",
    "ErroFormato",
    "ErroValor",
    "ErroOperacao",
]

"""Composição de HTML local com escape por contexto.

A superfície desta versão trabalha somente com uma árvore de nós simples.
Texto comum nunca é interpretado como marcação e a serialização não escreve em
disco, não abre navegador e não acessa a rede.
"""
from __future__ import annotations

from collections.abc import Iterable, Mapping, Sequence
from dataclasses import dataclass
from html import escape as _escape_html
import re
from typing import Any

from ..erros import ErroValor


class ErroHTML(ErroValor):
    """Falha de composição ou serialização HTML controlada pela Coral."""


_NOME_ELEMENTO = re.compile(r"^[A-Za-z][A-Za-z0-9:-]*$")
_NOME_ATRIBUTO = re.compile(r"^[A-Za-z_:][A-Za-z0-9_.:-]*$")
_IDIOMA = re.compile(r"^[A-Za-z]{2,8}(?:-[A-Za-z0-9]{1,8})*$")
_ELEMENTOS_VAZIOS = frozenset(
    {"area", "base", "br", "col", "embed", "hr", "img", "input", "link", "meta",
      "source", "track", "wbr"}
)
_ATRIBUTOS_URL = frozenset({"href", "src", "action", "formaction", "poster", "cite"})
_ESQUEMAS_PERIGOSOS = frozenset({"javascript", "vbscript", "data"})


@dataclass(frozen=True, slots=True)
class ElementoHTML:
    """Elemento validado com atributos e filhos em ordem estável."""

    nome: str
    atributos: tuple[tuple[str, Any], ...] = ()
    filhos: tuple[Any, ...] = ()


@dataclass(frozen=True, slots=True)
class FragmentoHTML:
    """Sequência de nós sem elemento contêiner adicional."""
```

```
    filhos: tuple[Any, ...]


@dataclass(frozen=True, slots=True)
class DocumentoHTML:
    """Documento HTML completo com idioma e UTF 8 declarados."""

    titulo: str
    corpo: FragmentoHTML
    idioma: str = "pt-BR"


def escapar_texto(valor: Any) -> str:
    """Escapa um valor para o contexto de texto de um elemento HTML."""
    if valor is None:
        return ""
    return _escape_html(str(valor), quote=False)


def escapar_atributo(valor: Any) -> str:
    """Escapa um valor para atributo entre aspas duplas.

    Esta operação é codificação contextual, não validação de URL. Por exemplo,
    ``javascript:...`` continua sendo um esquema perigoso depois do escape.
    """
    if valor is None:
        return ""
    return _escape_html(str(valor), quote=True)


def _validar_nome_elemento(nome: Any) -> str:
    if not isinstance(nome, str) or not _NOME_ELEMENTO.fullmatch(nome.strip()):
        raise ErroHTML("nome de elemento HTML inválido")
    return nome.strip().lower()


def _validar_nome_atributo(nome: Any) -> str:
    if not isinstance(nome, str) or not _NOME_ATRIBUTO.fullmatch(nome.strip()):
        raise ErroHTML("nome de atributo HTML inválido")
    resultado = nome.strip().lower()
    if resultado.startswith("on"):
        raise ErroHTML("atributos de evento inline não são aceitos pela base HTML
          segura")
    return resultado


def _validar_url_em_atributo(nome: str, valor: Any) -> None:
    if nome not in _ATRIBUTOS_URL or valor in (None, False, True):
        return
    texto = str(valor).strip()
    # Fragmentos, caminhos relativos e URLs sem esquema explícito são dados
    # legítimos. Quando um esquema aparece, recusamos os que executam conteúdo.
    candidato = texto.split(":", 1)
    if len(candidato) == 2:
        # Navegadores podem tolerar tabulações e quebras de linha no trecho do
        # esquema. Normalizamos apenas esse prefixo para que ``java\nscript:``
        # não contorne a política de atributos URL.
        esquema = re.sub(r"[\x00-\x20\x7f]+", "", candidato[0])
        if re.fullmatch(r"[A-Za-z][A-Za-z0-9+.-]*", esquema):
            if esquema.casefold() in _ESQUEMAS_PERIGOSOS:
                raise ErroHTML(f"esquema de URL perigoso no atributo {nome}")
```

```
def _normalizar_atributos(atributos: Any) -> tuple[tuple[str, Any], ...]:
    if atributos is None:
        return ()
    if isinstance(atributos, Mapping):
        itens = tuple(atributos.items())
    else:
        if isinstance(atributos, (str, bytes, bytearray)):
            raise ErroHTML("atributos devem ser mapeamento ou pares nome valor")
        try:
            itens = tuple(atributos)
        except TypeError as exc:
            raise ErroHTML("atributos devem ser mapeamento ou pares nome valor") from
              exc

    resultado: list[tuple[str, Any]] = []
    vistos: set[str] = set()
    for item in itens:
        if not isinstance(item, Sequence) or isinstance(item, (str, bytes, bytearray))
          or len(item) != 2:
            raise ErroHTML("cada atributo deve ser um par nome valor")
        nome = _validar_nome_atributo(item[0])
        if nome in vistos:
            raise ErroHTML(f"atributo duplicado: {nome}")
        vistos.add(nome)
        valor = item[1]
        _validar_url_em_atributo(nome, valor)
        resultado.append((nome, valor))
    return tuple(resultado)


def _normalizar_filhos(filhos: Any) -> tuple[Any, ...]:
    if filhos is None:
        return ()
    if isinstance(filhos, (ElementoHTML, FragmentoHTML, DocumentoHTML, str, bytes,
      bytearray)):
        return (filhos,)
    if isinstance(filhos, Mapping):
        # Um registro não vira implicitamente uma coleção de chaves.
        return (filhos,)
    try:
        return tuple(filhos)
    except TypeError:
        return (filhos,)


def elemento(nome: str, atributos: Any = None, filhos: Any = None) -> ElementoHTML:
    """Cria um elemento validado; texto em ``filhos`` será escapado ao renderizar."""
    nome_ok = _validar_nome_elemento(nome)
    filhos_ok = _normalizar_filhos(filhos)
    if nome_ok in _ELEMENTOS_VAZIOS and filhos_ok:
        raise ErroHTML(f"elemento vazio {nome_ok} não pode receber filhos")
    return ElementoHTML(nome_ok, _normalizar_atributos(atributos), filhos_ok)


def fragmento(filhos: Any = None) -> FragmentoHTML:
    """Agrupa nós preservando a ordem sem criar marcação adicional."""
    return FragmentoHTML(_normalizar_filhos(filhos))
```

```
def documento(titulo: Any, corpo: Any, idioma: str = "pt-BR") -> DocumentoHTML:
    """Cria documento completo; título e corpo seguem a mesma política de escape."""
    if not isinstance(idioma, str) or not _IDIOMA.fullmatch(idioma.strip()):
        raise ErroHTML("idioma HTML inválido")
    return DocumentoHTML(str(titulo), fragmento(corpo), idioma.strip())


def _renderizar_atributos(atributos: tuple[tuple[str, Any], ...]) -> str:
    partes: list[str] = []
    for nome, valor in atributos:
        if valor is None or valor is False:
            continue
        if valor is True:
            partes.append(nome)
        else:
            partes.append(f'{nome}="{escapar_atributo(valor)}"')
    return (" " + " ".join(partes)) if partes else ""


def _renderizar_no(no: Any) -> str:
    if isinstance(no, DocumentoHTML):
        titulo = escapar_texto(no.titulo)
        corpo = "".join(_renderizar_no(item) for item in no.corpo.filhos)
        idioma = escapar_atributo(no.idioma)
        return (
            "<!doctype html>\n"
            f'<html lang="{idioma}">\n'
            "<head>\n"
            '<meta charset="utf-8">\n'
            f"<title>{titulo}</title>\n"
            "</head>\n"
            f"<body>{corpo}</body>\n"
            "</html>"
        )
    if isinstance(no, FragmentoHTML):
        return "".join(_renderizar_no(item) for item in no.filhos)
    if isinstance(no, ElementoHTML):
        attrs = _renderizar_atributos(no.atributos)
        if no.nome in _ELEMENTOS_VAZIOS:
            return f"<{no.nome}{attrs}>"
        conteudo = "".join(_renderizar_no(item) for item in no.filhos)
        return f"<{no.nome}{attrs}>{conteudo}</{no.nome}>"
    if isinstance(no, bytes):
        try:
            no = no.decode("utf-8")
        except UnicodeDecodeError as exc:
            raise ErroHTML("bytes usados como texto HTML devem estar em UTF 8") from exc
    if isinstance(no, bytearray):
        return _renderizar_no(bytes(no))
    return escapar_texto(no)


def renderizar(no: Any) -> str:
    """Serializa um nó ou valor em memória, sem efeitos externos."""
    return _renderizar_no(no)


def _normalizar_colunas(colunas: Any) -> tuple[tuple[str, str], ...]:
    if isinstance(colunas, Mapping):
        itens = tuple(colunas.items())
    else:
```

```
        if isinstance(colunas, (str, bytes, bytearray)):
            itens = (colunas,)
        else:
            try:
                itens = tuple(colunas)
            except TypeError as exc:
                raise ErroHTML("colunas deve listar campos ou pares campo rótulo") from
                  exc
    if not itens:
        raise ErroHTML("tabela requer ao menos uma coluna")

    resultado: list[tuple[str, str]] = []
    vistos: set[str] = set()
    for item in itens:
        if isinstance(item, str):
            campo, rotulo = item, item
        elif isinstance(item, Sequence) and not isinstance(item, (bytes, bytearray)) and
           len(item) == 2:
            campo, rotulo = item[0], item[1]
        else:
            raise ErroHTML("cada coluna deve ser um campo ou par campo rótulo")
        if not isinstance(campo, str) or not campo.strip():
            raise ErroHTML("nome de campo da tabela não pode ser vazio")
        campo = campo.strip()
        if campo in vistos:
            raise ErroHTML(f"coluna duplicada: {campo}")
        vistos.add(campo)
        resultado.append((campo, str(rotulo)))
    return tuple(resultado)


def _valor_registro(registro: Any, campo: str, indice: int) -> Any:
    if isinstance(registro, Mapping):
        if campo not in registro:
            raise ErroHTML(f"registro {indice} não possui campo {campo}")
        return registro[campo]
    if not hasattr(registro, campo):
        raise ErroHTML(f"registro {indice} não possui campo {campo}")
    return getattr(registro, campo)


def tabela(registros: Iterable[Any], colunas: Any) -> ElementoHTML:
    """Monta tabela a partir de registros comuns com colunas explícitas.

    Mapeamentos e objetos com atributos são aceitos. Nenhuma célula é tratada
    como HTML confiável: todo valor comum é escapado durante a serialização.
    """
    if isinstance(registros, (str, bytes, bytearray, Mapping)):
        raise ErroHTML("registros deve ser uma coleção de registros")
    try:
        linhas = tuple(registros)
    except TypeError as exc:
        raise ErroHTML("registros deve ser uma coleção de registros") from exc
    colunas_ok = _normalizar_colunas(colunas)

    cabecalho = elemento(
        "thead",
        filhos=elemento(
            "tr",
            filhos=tuple(elemento("th", filhos=rotulo) for _, rotulo in colunas_ok),
        ),
```

```
    )
    corpo_linhas: list[ElementoHTML] = []
    for indice, registro in enumerate(linhas):
        celulas = tuple(
            elemento("td", filhos=_valor_registro(registro, campo, indice))
            for campo, _ in colunas_ok
        )
        corpo_linhas.append(elemento("tr", filhos=celulas))
    corpo = elemento("tbody", filhos=tuple(corpo_linhas))
    return elemento("table", filhos=(cabecalho, corpo))


__all__ = [
    "ErroHTML",
    "ElementoHTML",
    "FragmentoHTML",
    "DocumentoHTML",
    "escapar_texto",
    "escapar_atributo",
    "elemento",
    "fragmento",
    "documento",
    "renderizar",
    "tabela",
]
```

```
"""Servidor HTTP local de desenvolvimento da Coral 1.6.9.

O módulo não abre portas durante a importação. Um servidor só começa a aceitar
conexões depois de :meth:`ServidorHTTP.iniciar` ou da entrada em seu contexto.
Por padrão, ele escuta exclusivamente em ``127.0.0.1`` e usa porta efêmera.
"""
from __future__ import annotations

from collections.abc import Callable, Mapping, Sequence
from dataclasses import dataclass
import codecs
import ipaddress
import json as _json
import mimetypes
from pathlib import Path
import re
import socket
import threading
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from typing import Any
from urllib.parse import unquote_to_bytes, urlsplit

from ..erros import ErroFormato, ErroOperacao, ErroValor
from .html import renderizar as renderizar_html
from .url import ler_consulta


HOST_PADRAO = "127.0.0.1"
PORTA_PADRAO = 0
LIMITE_CORPO_PADRAO = 1 * 1024 * 1024
LIMITE_ARQUIVO_PADRAO = 8 * 1024 * 1024
TEMPO_LIMITE_CORPO_PADRAO = 10.0

_TOKEN_HTTP = re.compile(r"^[!#$%&'*+.^_`|~0-9A-Za-z-]+$")
_PERCENTUAL_INVALIDO = re.compile(r"%(?![0-9A-Fa-f]{2})")
```

```
class ErroServidorHTTP(ErroOperacao):
    """Falha operacional do servidor HTTP local da Coral."""


class ErroRotaHTTP(ErroValor):
    """Definição de rota não obedece ao contrato literal do servidor."""


class ErroLimiteCorpoHTTP(ErroServidorHTTP):
    """Corpo de requisição excedeu o limite declarado pelo servidor."""


@dataclass(frozen=True, slots=True)
class RequisicaoHTTP:
    """Requisição materializada antes da chamada ao tratador da rota."""

    metodo: str
    caminho: str
    consulta: tuple[tuple[str, str], ...]
    cabecalhos: tuple[tuple[str, str], ...]
    corpo_bytes: bytes
    cliente: str

    def cabecalho(self, nome: str, padrao: str | None = None) -> str | None:
        alvo = str(nome).casefold()
        encontrados = [valor for chave, valor in self.cabecalhos if chave.casefold() ==
          alvo]
        return encontrados[-1] if encontrados else padrao

    def texto(self, codificacao: str = "utf-8", *, erros: str = "strict") -> str:
        try:
            codecs.lookup(codificacao)
        except LookupError as exc:
            raise ErroFormato(f"codificação de requisição desconhecida: {codificacao}",
              causa=exc) from exc
        try:
            return self.corpo_bytes.decode(codificacao, errors=erros)
        except UnicodeDecodeError as exc:
            raise ErroFormato(
                f"corpo da requisição não pode ser decodificado como {codificacao}",
                  causa=exc
            ) from exc

    def json(self) -> Any:
        try:
            return _json.loads(self.corpo_bytes)
        except (UnicodeDecodeError, _json.JSONDecodeError) as exc:
            raise ErroFormato("corpo da requisição não contém JSON válido", causa=exc)
              from exc

    def formulario(self) -> tuple[tuple[str, str], ...]:
        tipo = (self.cabecalho("Content-Type") or "").split(";",
          1)[0].strip().casefold()
        if tipo != "application/x-www-form-urlencoded":
            raise ErroFormato(
                "formulário HTTP requer Content-Type application/x-www-form-urlencoded"
            )
        try:
            texto = self.corpo_bytes.decode("ascii")
```

```
        except UnicodeDecodeError as exc:
            raise ErroFormato("formulário HTTP deve usar codificação percentual UTF 8")
              from exc
        return ler_consulta(texto)


@dataclass(frozen=True, slots=True)
class RespostaServidorHTTP:
    """Resposta pronta para serialização pelo servidor local."""

    status: int
    cabecalhos: tuple[tuple[str, str], ...]
    corpo_bytes: bytes


TratadorHTTP = Callable[[RequisicaoHTTP], RespostaServidorHTTP]


def _validar_status(status: Any) -> int:
    if isinstance(status, bool) or not isinstance(status, int) or not 100 <= status <=
      599:
        raise ErroValor("status HTTP deve ser inteiro entre 100 e 599")
    return status


def _normalizar_metodo(metodo: Any) -> str:
    if not isinstance(metodo, str):
        raise ErroRotaHTTP("método da rota deve ser texto")
    resultado = metodo.strip().upper()
    if not resultado or not _TOKEN_HTTP.fullmatch(resultado):
        raise ErroRotaHTTP("método da rota HTTP inválido")
    return resultado


def _normalizar_caminho_rota(caminho: Any) -> str:
    if not isinstance(caminho, str):
        raise ErroRotaHTTP("caminho da rota deve ser texto")
    resultado = caminho.strip()
    if not resultado.startswith("/"):
        raise ErroRotaHTTP("rota HTTP deve começar com /")
    if "?" in resultado or "#" in resultado:
        raise ErroRotaHTTP("rota literal não deve conter consulta nem fragmento")
    if _PERCENTUAL_INVALIDO.search(resultado):
        raise ErroRotaHTTP("rota contém escape percentual inválido")
    try:
        decodificado = unquote_to_bytes(resultado).decode("utf-8")
    except UnicodeDecodeError as exc:
        raise ErroRotaHTTP("rota deve possuir codificação UTF 8 válida") from exc
    if not decodificado.startswith("/") or "\x00" in decodificado:
        raise ErroRotaHTTP("rota HTTP inválida")
    return decodificado


def _normalizar_cabecalhos(cabecalhos: Any) -> tuple[tuple[str, str], ...]:
    if cabecalhos is None:
        return ()
    if isinstance(cabecalhos, Mapping):
        itens = tuple(cabecalhos.items())
    else:
        if isinstance(cabecalhos, (str, bytes, bytearray)):
            raise ErroValor("cabeçalhos da resposta devem ser mapeamento ou pares")
```

```
        try:
            itens = tuple(cabecalhos)
        except TypeError as exc:
            raise ErroValor("cabeçalhos da resposta devem ser mapeamento ou pares") from
               exc
    saida: list[tuple[str, str]] = []
    for item in itens:
        if (
            not isinstance(item, Sequence)
            or isinstance(item, (str, bytes, bytearray))
            or len(item) != 2
        ):
            raise ErroValor("cada cabeçalho deve ser um par nome valor")
        nome = str(item[0]).strip()
        valor = str(item[1]).strip()
        if not _TOKEN_HTTP.fullmatch(nome):
            raise ErroValor("nome de cabeçalho HTTP inválido")
        if any(c in valor for c in ("\r", "\n", "\x00")):
            raise ErroValor(f"valor inválido para cabeçalho HTTP {nome}")
        if nome.casefold() in {"content-length", "connection"}:
            raise ErroValor(f"cabeçalho {nome} é controlado pelo servidor")
        saida.append((nome, valor))
    return tuple(saida)


def _normalizar_corpo(corpo: Any) -> bytes:
    if isinstance(corpo, bytes):
        return corpo
    if isinstance(corpo, bytearray):
        return bytes(corpo)
    if isinstance(corpo, memoryview):
        return corpo.tobytes()
    raise ErroValor("corpo de resposta HTTP deve ser bytes")


def resposta_bytes(
    corpo: Any,
    status: int = 200,
    cabecalhos: Any = None,
    *,
    tipo_conteudo: str = "application/octet-stream",
) -> RespostaServidorHTTP:
    corpo_bytes = _normalizar_corpo(corpo)
    headers = list(_normalizar_cabecalhos(cabecalhos))
    if tipo_conteudo and not any(nome.casefold() == "content-type" for nome, _ in
      headers):
        headers.append(("Content-Type", str(tipo_conteudo)))
    return RespostaServidorHTTP(_validar_status(status), tuple(headers), corpo_bytes)


def resposta_texto(
    texto: Any,
    status: int = 200,
    cabecalhos: Any = None,
    *,
    codificacao: str = "utf-8",
) -> RespostaServidorHTTP:
    try:
        codecs.lookup(codificacao)
        corpo = str(texto).encode(codificacao)
    except LookupError as exc:
```

```
        raise ErroValor(f"codificação de resposta desconhecida: {codificacao}") from exc
    return resposta_bytes(
        corpo,
        status,
        cabecalhos,
        tipo_conteudo=f"text/plain; charset={codificacao}",
    )


def resposta_html(
    conteudo: Any,
    status: int = 200,
    cabecalhos: Any = None,
) -> RespostaServidorHTTP:
    corpo = renderizar_html(conteudo).encode("utf-8")
    return resposta_bytes(
        corpo,
        status,
        cabecalhos,
        tipo_conteudo="text/html; charset=utf-8",
    )


def resposta_grafico(
    grafico: Any,
    status: int = 200,
    cabecalhos: Any = None,
    *,
    limite_pontos: int = 200_000,
) -> RespostaServidorHTTP:
    """Adapta um gráfico Coral para resposta HTML sem iniciar servidor.

    Plotly continua opcional e é carregado somente nesta chamada por meio de
    ``coral.graficos.para_html``. A resposta recebe ``Cache-Control: no-store``
    por padrão porque o HTML autossuficiente incorpora os próprios dados.
    """
    from ..graficos import para_html

    html = para_html(grafico, limite_pontos=limite_pontos)
    headers = list(_normalizar_cabecalhos(cabecalhos))
    if not any(nome.casefold() == "cache-control" for nome, _ in headers):
        headers.append(("Cache-Control", "no-store"))
    if not any(nome.casefold() == "x-content-type-options" for nome, _ in headers):
        headers.append(("X-Content-Type-Options", "nosniff"))
    return resposta_bytes(
        html.encode("utf-8"),
        status,
        headers,
        tipo_conteudo="text/html; charset=utf-8",
    )


def resposta_json(
    valor: Any,
    status: int = 200,
    cabecalhos: Any = None,
) -> RespostaServidorHTTP:
    try:
        corpo = _json.dumps(
            valor,
            ensure_ascii=False,
```

```
            separators=(",", ":"),
            allow_nan=False,
        ).encode("utf-8")
    except (TypeError, ValueError) as exc:
        raise ErroFormato("valor não pode ser serializado como JSON HTTP fiel",
          causa=exc) from exc
    return resposta_bytes(
        corpo,
        status,
        cabecalhos,
        tipo_conteudo="application/json; charset=utf-8",
    )


def _validar_raiz(raiz: Any) -> Path:
    if raiz is None:
        raise ErroValor("servidor não possui raiz de arquivos declarada")
    try:
        caminho = Path(raiz).expanduser().resolve(strict=True)
    except (OSError, RuntimeError) as exc:
        raise ErroValor("raiz de arquivos não existe ou não pode ser resolvida") from
          exc
    if not caminho.is_dir():
        raise ErroValor("raiz de arquivos deve apontar para uma pasta")
    return caminho


def _resolver_arquivo_sob_raiz(raiz: Path, relativo: Any) -> Path:
    if not isinstance(relativo, (str, Path)):
        raise ErroValor("caminho de arquivo deve ser texto ou Path")
    texto = str(relativo)
    candidato_rel = Path(texto)
    if candidato_rel.is_absolute():
        raise ErroValor("arquivo servido deve usar caminho relativo à raiz declarada")
    try:
        candidato = (raiz / candidato_rel).resolve(strict=True)
    except (OSError, RuntimeError) as exc:
        raise ErroValor("arquivo solicitado não existe sob a raiz declarada") from exc
    try:
        candidato.relative_to(raiz)
    except ValueError as exc:
        raise ErroValor("caminho de arquivo tenta sair da raiz declarada") from exc
    if not candidato.is_file():
        raise ErroValor("caminho servido deve apontar para um arquivo regular")
    return candidato


def resposta_arquivo(
    raiz: Any,
    caminho_relativo: Any,
    status: int = 200,
    cabecalhos: Any = None,
    *,
    limite_arquivo: int = LIMITE_ARQUIVO_PADRAO,
) -> RespostaServidorHTTP:
    raiz_ok = _validar_raiz(raiz)
    caminho = _resolver_arquivo_sob_raiz(raiz_ok, caminho_relativo)
    if isinstance(limite_arquivo, bool) or not isinstance(limite_arquivo, int) or
      limite_arquivo < 0:
        raise ErroValor("limite_arquivo deve ser inteiro maior ou igual a zero")
    try:
```

```
        tamanho = caminho.stat().st_size
    except OSError as exc:
        raise ErroServidorHTTP("não foi possível inspecionar o arquivo servido",
          causa=exc) from exc
    if tamanho > limite_arquivo:
        raise ErroServidorHTTP(
            f"arquivo possui {tamanho} bytes, acima do limite de {limite_arquivo}"
        )
    try:
        corpo = caminho.read_bytes()
    except OSError as exc:
        raise ErroServidorHTTP("não foi possível ler o arquivo servido", causa=exc) from
           exc
    tipo, _ = mimetypes.guess_type(caminho.name)
    return resposta_bytes(corpo, status, cabecalhos, tipo_conteudo=tipo or
      "application/octet-stream")


def _host_local(host: str) -> bool:
    texto = host.strip().casefold()
    if texto == "localhost":
        return True
    try:
        return ipaddress.ip_address(texto).is_loopback
    except ValueError:
        return False


def _validar_host(host: Any, permitir_externo: bool) -> str:
    if not isinstance(host, str) or not host.strip():
        raise ErroValor("host do servidor deve ser texto não vazio")
    resultado = host.strip()
    if not permitir_externo and not _host_local(resultado):
        raise ErroValor(
            "servidor HTTP local só aceita loopback por padrão; use
              permitir_externo=True conscientemente"
        )
    return resultado


def _validar_porta(porta: Any) -> int:
    if isinstance(porta, bool) or not isinstance(porta, int) or not 0 <= porta <= 65535:
        raise ErroValor("porta HTTP deve ser inteiro entre 0 e 65535")
    return porta


def _validar_limite_corpo(limite: Any) -> int:
    if isinstance(limite, bool) or not isinstance(limite, int) or limite < 0:
        raise ErroValor("limite_corpo deve ser inteiro maior ou igual a zero")
    return limite


def _validar_tempo_limite(valor: Any) -> float:
    if isinstance(valor, bool):
        raise ErroValor("tempo_limite_corpo deve ser numérico positivo")
    try:
        numero = float(valor)
    except (TypeError, ValueError) as exc:
        raise ErroValor("tempo_limite_corpo deve ser numérico positivo") from exc
    if numero <= 0 or numero != numero or numero == float("inf"):
        raise ErroValor("tempo_limite_corpo deve ser finito e maior que zero")
```

```
    return numero


class _ServidorInterno(ThreadingHTTPServer):
    daemon_threads = True
    allow_reuse_address = True

    def __init__(self, endereco: tuple[str, int], dono: "ServidorHTTP"):
        self.dono = dono
        super().__init__(endereco, _TratadorInterno)


class _TratadorInterno(BaseHTTPRequestHandler):
    protocol_version = "HTTP/1.1"
    server_version = "CoralHTTP/1.6"
    sys_version = ""

    @property
    def _dono(self) -> "ServidorHTTP":
        return self.server.dono  # type: ignore[attr-defined]

    def log_message(self, format: str, *args: Any) -> None:  # noqa: A002
        # Servidor de desenvolvimento não escreve em stderr implicitamente.
        return None

    def _caminho_e_consulta(self) -> tuple[str, tuple[tuple[str, str], ...]]:
        partes = urlsplit(self.path)
        bruto = partes.path or "/"
        if _PERCENTUAL_INVALIDO.search(bruto):
            raise ErroFormato("caminho da requisição contém escape percentual inválido")
        try:
            caminho = unquote_to_bytes(bruto).decode("utf-8")
        except UnicodeDecodeError as exc:
            raise ErroFormato("caminho da requisição não possui UTF 8 válido") from exc
        return caminho, ler_consulta(partes.query)

    def _ler_corpo(self) -> bytes:
        transferencia = (self.headers.get("Transfer-Encoding") or "").strip().casefold()
        if transferencia and transferencia != "identity":
            raise ErroFormato("servidor básico não aceita Transfer-Encoding; use
              Content-Length")
        tamanho_txt = self.headers.get("Content-Length")
        if tamanho_txt is None:
            return b""
        try:
            tamanho = int(tamanho_txt)
        except ValueError as exc:
            raise ErroFormato("Content-Length da requisição é inválido") from exc
        if tamanho < 0:
            raise ErroFormato("Content-Length da requisição não pode ser negativo")
        if tamanho > self._dono.limite_corpo:
            raise ErroLimiteCorpoHTTP(
                f"corpo da requisição declara {tamanho} bytes, acima do limite de
                  {self._dono.limite_corpo}"
            )
        self.connection.settimeout(self._dono.tempo_limite_corpo)
        try:
            corpo = self.rfile.read(tamanho)
        except (socket.timeout, TimeoutError) as exc:
            raise ErroServidorHTTP("leitura do corpo da requisição excedeu o tempo
              limite", causa=exc) from exc
```

```
        if len(corpo) != tamanho:
            raise ErroFormato("corpo da requisição terminou antes do Content-Length
              declarado")
        return corpo

    def _enviar(self, resposta: RespostaServidorHTTP, *, somente_cabecalhos: bool =
      False) -> None:
        self.send_response(resposta.status)
        vistos = {nome.casefold() for nome, _ in resposta.cabecalhos}
        for nome, valor in resposta.cabecalhos:
            self.send_header(nome, valor)
        if "content-length" not in vistos:
            self.send_header("Content-Length", str(len(resposta.corpo_bytes)))
        self.send_header("Connection", "close")
        self.end_headers()
        if not somente_cabecalhos and resposta.corpo_bytes:
            self.wfile.write(resposta.corpo_bytes)
        self.close_connection = True

    def _erro(self, status: int, mensagem: str) -> None:
        self._enviar(resposta_texto(mensagem, status=status))

    def _despachar(self, *, somente_cabecalhos: bool = False) -> None:
        metodo = self.command.upper()
        try:
            caminho, consulta = self._caminho_e_consulta()
            corpo = self._ler_corpo()
        except ErroLimiteCorpoHTTP:
            self._erro(413, "corpo da requisição excede o limite configurado")
            return
        except (ErroFormato, ErroValor):
            self._erro(400, "requisição HTTP inválida")
            return
        except ErroServidorHTTP:
            self._erro(408, "não foi possível ler a requisição no prazo configurado")
            return

        rota = self._dono._rotas.get((metodo, caminho))
        if rota is None and metodo == "HEAD":
            rota = self._dono._rotas.get(("GET", caminho))
        if rota is None:
            metodos = sorted({m for m, p in self._dono._rotas if p == caminho})
            if metodos:
                resposta = resposta_texto(
                    "método não permitido para esta rota",
                    status=405,
                    cabecalhos={"Allow": ", ".join(metodos)},
                )
                self._enviar(resposta, somente_cabecalhos=metodo == "HEAD")
            else:
                self._erro(404, "rota não encontrada")
            return

        requisicao = RequisicaoHTTP(
            metodo=metodo,
            caminho=caminho,
            consulta=consulta,
            cabecalhos=tuple((str(k), str(v)) for k, v in self.headers.items()),
            corpo_bytes=corpo,
            cliente=str(self.client_address[0]),
        )
```

```
        try:
            resposta = rota(requisicao)
            if not isinstance(resposta, RespostaServidorHTTP):
                raise TypeError("tratador deve retornar RespostaServidorHTTP")
        except Exception as exc:  # noqa: BLE001
            self._dono._ultimo_erro = exc
            self._erro(500, "erro interno no tratador da rota")
            return
        self._enviar(resposta, somente_cabecalhos=somente_cabecalhos or metodo ==
          "HEAD")

    def do_GET(self) -> None:
        self._despachar()

    def do_HEAD(self) -> None:
        self._despachar(somente_cabecalhos=True)

    def do_POST(self) -> None:
        self._despachar()

    def do_PUT(self) -> None:
        self._despachar()

    def do_PATCH(self) -> None:
        self._despachar()

    def do_DELETE(self) -> None:
        self._despachar()

    def do_OPTIONS(self) -> None:
        self._despachar()


class ServidorHTTP:
    """Servidor HTTP pequeno, explícito e orientado a desenvolvimento local."""

    def __init__(
        self,
        host: str = HOST_PADRAO,
        porta: int = PORTA_PADRAO,
        *,
        limite_corpo: int = LIMITE_CORPO_PADRAO,
        tempo_limite_corpo: float = TEMPO_LIMITE_CORPO_PADRAO,
        raiz_arquivos: Any = None,
        permitir_externo: bool = False,
    ):
        if not isinstance(permitir_externo, bool):
            raise ErroValor("permitir_externo deve ser verdadeiro ou falso")
        self.host = _validar_host(host, permitir_externo)
        self.porta_solicitada = _validar_porta(porta)
        self.limite_corpo = _validar_limite_corpo(limite_corpo)
        self.tempo_limite_corpo = _validar_tempo_limite(tempo_limite_corpo)
        self.raiz_arquivos = _validar_raiz(raiz_arquivos) if raiz_arquivos is not None
          else None
        self.permitir_externo = permitir_externo
        self._rotas: dict[tuple[str, str], TratadorHTTP] = {}
        self._servidor: _ServidorInterno | None = None
        self._thread: threading.Thread | None = None
        self._ultimo_erro: BaseException | None = None

    @property
```

```
    def ativo(self) -> bool:
        return self._servidor is not None and self._thread is not None and
          self._thread.is_alive()

    @property
    def porta(self) -> int:
        if self._servidor is None:
            return self.porta_solicitada
        return int(self._servidor.server_address[1])

    @property
    def url_base(self) -> str:
        if self._servidor is None:
            raise ErroServidorHTTP("servidor ainda não foi iniciado")
        host = self.host
        if ":" in host and not host.startswith("["):
            host = f"[{host}]"
        return f"http://{host}:{self.porta}"

    @property
    def ultimo_erro(self) -> BaseException | None:
        return self._ultimo_erro

    def adicionar_rota(self, metodo: str, caminho: str, tratador: TratadorHTTP) ->
      "ServidorHTTP":
        if self.ativo:
            raise ErroServidorHTTP("rotas não podem ser alteradas enquanto o servidor
              está ativo")
        metodo_ok = _normalizar_metodo(metodo)
        caminho_ok = _normalizar_caminho_rota(caminho)
        if not callable(tratador):
            raise ErroRotaHTTP("tratador da rota deve ser chamável")
        chave = (metodo_ok, caminho_ok)
        if chave in self._rotas:
            raise ErroRotaHTTP(f"rota duplicada: {metodo_ok} {caminho_ok}")
        self._rotas[chave] = tratador
        return self

    def adicionar_arquivo(
        self,
        caminho_rota: str,
        caminho_arquivo: Any,
        *,
        limite_arquivo: int = LIMITE_ARQUIVO_PADRAO,
        cabecalhos: Any = None,
    ) -> "ServidorHTTP":
        raiz = _validar_raiz(self.raiz_arquivos)
        # Validar já no registro impede publicar uma rota que só falharia no primeiro
          acesso.
        _resolver_arquivo_sob_raiz(raiz, caminho_arquivo)

        def servir(_: RequisicaoHTTP) -> RespostaServidorHTTP:
            return resposta_arquivo(
                raiz,
                caminho_arquivo,
                cabecalhos=cabecalhos,
                limite_arquivo=limite_arquivo,
            )

        return self.adicionar_rota("GET", caminho_rota, servir)
```

```
    def iniciar(self) -> "ServidorHTTP":
        if self._servidor is not None:
            raise ErroServidorHTTP("servidor HTTP já foi iniciado")
        try:
            servidor = _ServidorInterno((self.host, self.porta_solicitada), self)
        except OSError as exc:
            raise ErroServidorHTTP("não foi possível abrir a porta HTTP solicitada",
              causa=exc) from exc
        thread = threading.Thread(
            target=servidor.serve_forever,
            name=f"coral-http-{servidor.server_address[1]}",
            daemon=True,
        )
        self._servidor = servidor
        self._thread = thread
        thread.start()
        return self

    def encerrar(self) -> None:
        servidor = self._servidor
        thread = self._thread
        if servidor is None:
            return
        try:
            servidor.shutdown()
        finally:
            servidor.server_close()
            if thread is not None and thread is not threading.current_thread():
                thread.join(timeout=2.0)
            self._servidor = None
            self._thread = None

    def __enter__(self) -> "ServidorHTTP":
        return self.iniciar()

    def __exit__(self, exc_type, exc, tb) -> None:  # noqa: ANN001
        self.encerrar()


def criar_servidor(
    host: str = HOST_PADRAO,
    porta: int = PORTA_PADRAO,
    **opcoes: Any,
) -> ServidorHTTP:
    """Cria servidor sem abrir a porta; chame ``iniciar`` ou use ``com``."""
    return ServidorHTTP(host, porta, **opcoes)


__all__ = [
    "ErroServidorHTTP",
    "ErroRotaHTTP",
    "ErroLimiteCorpoHTTP",
    "RequisicaoHTTP",
    "RespostaServidorHTTP",
    "ServidorHTTP",
    "HOST_PADRAO",
    "PORTA_PADRAO",
    "LIMITE_CORPO_PADRAO",
    "LIMITE_ARQUIVO_PADRAO",
    "TEMPO_LIMITE_CORPO_PADRAO",
    "resposta_bytes",
```

```
    "resposta_texto",
    "resposta_html",
    "resposta_grafico",
    "resposta_json",
    "resposta_arquivo",
    "criar_servidor",
]

"""Manipulação local de URLs e consultas para a Coral 1.6.6.

Todas as operações deste módulo são puramente textuais. Nenhuma função resolve
DNS, abre socket, realiza requisição ou consulta recurso externo.
"""
from __future__ import annotations

from collections.abc import Iterable, Mapping, Sequence
from dataclasses import dataclass
import re
from typing import Any
from urllib.parse import parse_qsl, urlencode, urljoin, urlsplit, urlunsplit

from ..erros import ErroValor


class ErroURL(ErroValor):
    """URL ou consulta não obedece ao contrato esperado."""


@dataclass(frozen=True, slots=True)
class EnderecoURL:
    """Componentes textuais de uma URL com esquema explícito."""

    esquema: str
    autoridade: str
    host: str | None
    porta: int | None
    caminho: str
    consulta: str
    fragmento: str

    @property
    def parametros(self) -> tuple[tuple[str, str], ...]:
        """Consulta decodificada preservando ordem e chaves repetidas."""
        return ler_consulta(self.consulta)

    def reconstruir(self) -> str:
        """Reconstrói a URL sem acessar a rede."""
        return urlunsplit((self.esquema, self.autoridade, self.caminho, self.consulta,
          self.fragmento))


_PERCENTUAL_INVALIDO = re.compile(r"%(?![0-9A-Fa-f]{2})")
_ESQUEMA = re.compile(r"^[A-Za-z][A-Za-z0-9+.-]*$")


def _texto(valor: Any, contexto: str) -> str:
    if isinstance(valor, bytes):
        try:
            return valor.decode("utf-8")
        except UnicodeDecodeError as exc:
            raise ErroURL(f"{contexto} em bytes deve estar em UTF 8") from exc
```

```
    if isinstance(valor, bytearray):
        return _texto(bytes(valor), contexto)
    if valor is None:
        return ""
    return str(valor)


def _pares_parametros(parametros: Any) -> tuple[tuple[str, str], ...]:
    if isinstance(parametros, Mapping):
        bruto: list[tuple[Any, Any]] = []
        for chave, valor in parametros.items():
            if isinstance(valor, (list, tuple)):
                bruto.extend((chave, item) for item in valor)
            else:
                bruto.append((chave, valor))
        itens = tuple(bruto)
    else:
        if isinstance(parametros, (str, bytes, bytearray)):
            raise ErroURL("parâmetros devem ser mapeamento ou sequência de pares")
        try:
            itens = tuple(parametros)
        except TypeError as exc:
            raise ErroURL("parâmetros devem ser mapeamento ou sequência de pares") from
              exc

    resultado: list[tuple[str, str]] = []
    for item in itens:
        if not isinstance(item, Sequence) or isinstance(item, (str, bytes, bytearray))
          or len(item) != 2:
            raise ErroURL("cada parâmetro deve ser um par chave valor")
        chave = _texto(item[0], "chave de consulta")
        valor = _texto(item[1], "valor de consulta")
        resultado.append((chave, valor))
    return tuple(resultado)


def codificar_consulta(parametros: Any) -> str:
    """Codifica pares ordenados em UTF 8 preservando chaves repetidas."""
    pares = _pares_parametros(parametros)
    return urlencode(pares, doseq=False, encoding="utf-8", errors="strict")


def ler_consulta(texto: Any) -> tuple[tuple[str, str], ...]:
    """Lê consulta em UTF 8, preservando ordem, vazios e duplicatas."""
    bruto = _texto(texto, "consulta")
    if bruto.startswith("?"):
        bruto = bruto[1:]
    if _PERCENTUAL_INVALIDO.search(bruto):
        raise ErroURL("consulta contém escape percentual inválido")
    try:
        pares = parse_qsl(
            bruto,
            keep_blank_values=True,
            strict_parsing=False,
            encoding="utf-8",
            errors="strict",
        )
    except (UnicodeDecodeError, ValueError) as exc:
        raise ErroURL("consulta não contém codificação UTF 8 válida") from exc
    return tuple((str(chave), str(valor)) for chave, valor in pares)
```

```
def decompor(endereco: Any) -> EnderecoURL:
    """Decompõe uma URL com esquema explícito sem realizar acesso externo."""
    texto = _texto(endereco, "endereço").strip()
    if not texto:
        raise ErroURL("endereço não pode ser vazio")
    try:
        partes = urlsplit(texto)
    except ValueError as exc:
        raise ErroURL("endereço URL inválido") from exc
    if not partes.scheme or not _ESQUEMA.fullmatch(partes.scheme):
        raise ErroURL("endereço deve possuir esquema explícito")
    try:
        porta = partes.port
        host = partes.hostname
    except ValueError as exc:
        raise ErroURL("porta da URL é inválida") from exc
    return EnderecoURL(
        esquema=partes.scheme.casefold(),
        autoridade=partes.netloc,
        host=host,
        porta=porta,
        caminho=partes.path,
        consulta=partes.query,
        fragmento=partes.fragment,
    )


def juntar(base: Any, referencia: Any) -> str:
    """Resolve referência contra uma base usando apenas regras textuais de URL."""
    base_texto = _texto(base, "URL base").strip()
    referencia_texto = _texto(referencia, "referência").strip()
    # A base precisa ser uma URL real e explícita; a referência pode ser relativa.
    decompor(base_texto)
    try:
        resultado = urljoin(base_texto, referencia_texto)
    except ValueError as exc:
        raise ErroURL("não foi possível juntar a URL base e a referência") from exc
    # A resolução deve continuar produzindo um endereço com esquema explícito.
    decompor(resultado)
    return resultado


__all__ = [
    "ErroURL",
    "EnderecoURL",
    "codificar_consulta",
    "ler_consulta",
    "juntar",
    "decompor",
]
```

Biblioteca `coral.web.url`

`coral.web.url.EnderecoURL`

Componentes textuais de uma URL com esquema explícito.

Assinatura

Assinatura para consulta

```
EnderecoURL(esquema: 'str', autoridade: 'str', host: 'str | None', porta: 'int | None',
  caminho: 'str', consulta: 'str', fragmento: 'str') -> None
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
esquema	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
autoridade	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
host	str | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
porta	int | None	POSITIONAL_OR_KEYWORD	sim	sem padrão
caminho	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
consulta	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
fragmento	str	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

None

Atributo `esquema`: `str`.

Atributo `autoridade`: `str`.

Atributo `host`: `str | None`.

Atributo `porta`: `int | None`.

Atributo `caminho`: `str`.

Atributo `consulta`: `str`.

Atributo `fragmento`: `str`.

Métodos públicos

Referência para leitura

```
reconstruir(self) -> 'str'
```

Reconstrói a URL sem acessar a rede.

`coral.web.url.ErroURL`

URL ou consulta não obedece ao contrato esperado.

Assinatura

Assinatura para consulta

```
ErroURL(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
mensagem	str	POSITIONAL_OR_KEYWORD	sim	sem padrão
causa	BaseException | None	KEYWORD_ONLY	não	None

Retorno declarado

Sem anotação no runtime. A implementação canônica a seguir define os caminhos de retorno; não inferir um tipo a partir da ausência de anotação.

Métodos públicos

Referência para leitura

```
add_note
```

Exception.add_note(note) -- add a note to the exception

Referência para leitura

```
with_traceback
```

Exception.with_traceback(tb) -- set self.__traceback__ to tb and return self.

`coral.web.url.codificar_consulta`

Codifica pares ordenados em UTF 8 preservando chaves repetidas.

Assinatura

Assinatura para consulta

```
codificar_consulta(parametros: 'Any') -> 'str'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
parametros	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

str

`coral.web.url.decompor`

Decompõe uma URL com esquema explícito sem realizar acesso externo.

Assinatura

Assinatura para consulta

```
decompor(endereco: 'Any') -> 'EnderecoURL'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
endereco	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

EnderecoURL

`coral.web.url.juntar`

Resolve referência contra uma base usando apenas regras textuais de URL.

Assinatura

Assinatura para consulta

```
juntar(base: 'Any', referencia: 'Any') -> 'str'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
base	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão
referencia	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

str

`coral.web.url.ler_consulta`

Lê consulta em UTF 8, preservando ordem, vazios e duplicatas.

Assinatura

Assinatura para consulta

```
ler_consulta(texto: 'Any') -> 'tuple[tuple[str, str], ...]'
```

Parâmetro	Tipo declarado	Modo	Obrigatório	Padrão
texto	Any	POSITIONAL_OR_KEYWORD	sim	sem padrão

Retorno declarado

tuple[tuple[str, str], ...]

Consultar a implementação canônica de coral.web.url

Implementação canônica de `coral.web.url`

Código Python

```
from __future__ import annotations

from typing import Any

from .comum import ErroCoral, ErroOperacaoCoral


class ErroCoralRuntime(ErroCoral):
    """Base dos erros controlados expostos pelo runtime base da Coral."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa


class ErroEntrada(ErroCoralRuntime):
    """Falha ao obter entrada para um programa Coral."""


class FimDeEntrada(ErroEntrada, EOFError):
    """A fonte de entrada terminou antes de produzir outra linha."""



class ErroConversao(ErroCoralRuntime, ValueError):
    """Um valor não pôde ser convertido para o tipo Coral solicitado."""

    def __init__(
        self,
        valor: Any,
        destino: str,
        mensagem: str | None = None,
        *,
        causa: BaseException | None = None,
    ):
        self.valor = valor
        self.destino = destino
        super().__init__(
            mensagem or f"não foi possível converter o valor para {destino}",
            causa=causa,
        )


class ErroFormato(ErroCoralRuntime, ValueError):
    """Dados ou texto não obedecem ao formato esperado."""


class ErroValor(ErroCoralRuntime, ValueError):
    """O tipo é aceito, mas o valor não é válido para a operação."""


class ErroOperacao(ErroOperacaoCoral):
    """Falha operacional com preservação opcional da causa técnica."""

    def __init__(self, mensagem: str, *, causa: BaseException | None = None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.causa = causa

```

```
class ErroArquivo(ErroOperacao):
    """Falha em uma operação com arquivo ou caminho."""


__all__ = [
    "ErroCoralRuntime",
    "ErroEntrada",
    "FimDeEntrada",
    "ErroConversao",
    "ErroArquivo",
    "ErroFormato",
    "ErroValor",
    "ErroOperacao",
]

"""Manipulação local de URLs e consultas para a Coral 1.6.6.

Todas as operações deste módulo são puramente textuais. Nenhuma função resolve
DNS, abre socket, realiza requisição ou consulta recurso externo.
"""
from __future__ import annotations

from collections.abc import Iterable, Mapping, Sequence
from dataclasses import dataclass
import re
from typing import Any
from urllib.parse import parse_qsl, urlencode, urljoin, urlsplit, urlunsplit

from ..erros import ErroValor


class ErroURL(ErroValor):
    """URL ou consulta não obedece ao contrato esperado."""


@dataclass(frozen=True, slots=True)
class EnderecoURL:
    """Componentes textuais de uma URL com esquema explícito."""

    esquema: str
    autoridade: str
    host: str | None
    porta: int | None
    caminho: str
    consulta: str
    fragmento: str

    @property
    def parametros(self) -> tuple[tuple[str, str], ...]:
        """Consulta decodificada preservando ordem e chaves repetidas."""
        return ler_consulta(self.consulta)

    def reconstruir(self) -> str:
        """Reconstrói a URL sem acessar a rede."""
        return urlunsplit((self.esquema, self.autoridade, self.caminho, self.consulta,
          self.fragmento))


_PERCENTUAL_INVALIDO = re.compile(r"%(?![0-9A-Fa-f]{2})")
_ESQUEMA = re.compile(r"^[A-Za-z][A-Za-z0-9+.-]*$")
```

```
def _texto(valor: Any, contexto: str) -> str:
    if isinstance(valor, bytes):
        try:
            return valor.decode("utf-8")
        except UnicodeDecodeError as exc:
            raise ErroURL(f"{contexto} em bytes deve estar em UTF 8") from exc
    if isinstance(valor, bytearray):
        return _texto(bytes(valor), contexto)
    if valor is None:
        return ""
    return str(valor)


def _pares_parametros(parametros: Any) -> tuple[tuple[str, str], ...]:
    if isinstance(parametros, Mapping):
        bruto: list[tuple[Any, Any]] = []
        for chave, valor in parametros.items():
            if isinstance(valor, (list, tuple)):
                bruto.extend((chave, item) for item in valor)
            else:
                bruto.append((chave, valor))
        itens = tuple(bruto)
    else:
        if isinstance(parametros, (str, bytes, bytearray)):
            raise ErroURL("parâmetros devem ser mapeamento ou sequência de pares")
        try:
            itens = tuple(parametros)
        except TypeError as exc:
            raise ErroURL("parâmetros devem ser mapeamento ou sequência de pares") from
              exc

    resultado: list[tuple[str, str]] = []
    for item in itens:
        if not isinstance(item, Sequence) or isinstance(item, (str, bytes, bytearray))
          or len(item) != 2:
            raise ErroURL("cada parâmetro deve ser um par chave valor")
        chave = _texto(item[0], "chave de consulta")
        valor = _texto(item[1], "valor de consulta")
        resultado.append((chave, valor))
    return tuple(resultado)


def codificar_consulta(parametros: Any) -> str:
    """Codifica pares ordenados em UTF 8 preservando chaves repetidas."""
    pares = _pares_parametros(parametros)
    return urlencode(pares, doseq=False, encoding="utf-8", errors="strict")


def ler_consulta(texto: Any) -> tuple[tuple[str, str], ...]:
    """Lê consulta em UTF 8, preservando ordem, vazios e duplicatas."""
    bruto = _texto(texto, "consulta")
    if bruto.startswith("?"):
        bruto = bruto[1:]
    if _PERCENTUAL_INVALIDO.search(bruto):
        raise ErroURL("consulta contém escape percentual inválido")
    try:
        pares = parse_qsl(
            bruto,
            keep_blank_values=True,
```

```
            strict_parsing=False,
            encoding="utf-8",
            errors="strict",
        )
    except (UnicodeDecodeError, ValueError) as exc:
        raise ErroURL("consulta não contém codificação UTF 8 válida") from exc
    return tuple((str(chave), str(valor)) for chave, valor in pares)


def decompor(endereco: Any) -> EnderecoURL:
    """Decompõe uma URL com esquema explícito sem realizar acesso externo."""
    texto = _texto(endereco, "endereço").strip()
    if not texto:
        raise ErroURL("endereço não pode ser vazio")
    try:
        partes = urlsplit(texto)
    except ValueError as exc:
        raise ErroURL("endereço URL inválido") from exc
    if not partes.scheme or not _ESQUEMA.fullmatch(partes.scheme):
        raise ErroURL("endereço deve possuir esquema explícito")
    try:
        porta = partes.port
        host = partes.hostname
    except ValueError as exc:
        raise ErroURL("porta da URL é inválida") from exc
    return EnderecoURL(
        esquema=partes.scheme.casefold(),
        autoridade=partes.netloc,
        host=host,
        porta=porta,
        caminho=partes.path,
        consulta=partes.query,
        fragmento=partes.fragment,
    )


def juntar(base: Any, referencia: Any) -> str:
    """Resolve referência contra uma base usando apenas regras textuais de URL."""
    base_texto = _texto(base, "URL base").strip()
    referencia_texto = _texto(referencia, "referência").strip()
    # A base precisa ser uma URL real e explícita; a referência pode ser relativa.
    decompor(base_texto)
    try:
        resultado = urljoin(base_texto, referencia_texto)
    except ValueError as exc:
        raise ErroURL("não foi possível juntar a URL base e a referência") from exc
    # A resolução deve continuar produzindo um endereço com esquema explícito.
    decompor(resultado)
    return resultado


__all__ = [
    "ErroURL",
    "EnderecoURL",
    "codificar_consulta",
    "ler_consulta",
    "juntar",
    "decompor",
]
```

PARTE III: FERRAMENTAS

Objetivo desta parte

Consultar execução, verificação, instalação e integração com o editor.

Como usar esta referência

Comandos de terminal devem ser executados no sistema, fora de um arquivo .coral.

CLI Coral

O comando `coral` reúne execução, inspeção do compilador, projetos, testes, documentação, editor e diagnósticos da linha 1.3.

Fluxos do dia a dia

Terminal

```
coral programa.coral
coral verificar programa.coral
coral lint programa.coral
coral formatar programa.coral
coral testar
coral documentar
coral explicar programa.coral
coral ensinar programa.coral
coral naturalidade .
```

Diagnóstico e release 1.3

Terminal

```
coral --numerico-check
coral --sistema-info
coral --hardware-check
coral --laboratorio-check
coral --contratos-1-3
coral --conformidade-1-3
coral --gate-1-3
coral --release-check-1-3
```

Inspeção do compilador

Terminal

```
coral --tokens-expressao "1 mais 2 vezes 3"
coral --ast-expressao "1 mais 2 vezes 3"
coral --ast-programa programa.coral
coral --semantica programa.coral
coral --simbolos programa.coral
coral --mapa-linhas programa.coral
```

Cache

Terminal

```
coral --cache-info programa.coral
coral --sem-cache programa.coral
coral --limpar-cache programa.coral
```

Referência completa

Este capítulo mantém apenas os comandos necessários ao fluxo cotidiano. A listagem integral das opções da CLI fica concentrada no seção **Referência do CLI** desta parte, evitando duplicar no percurso didático um bloco extenso de documentação de referência. Para conferir a instalação atual, use também `coral --help`.

VS Code

A extensão 0.77.0 distribuída com a Coral 1.7.6 reúne realce, snippets, formatação, LSP, DAP, Test Explorer, Project Explorer, criação de projetos, Modo Aprender, Coral Ensinar, diagnóstico, tokens semânticos, navegação e integração com componentes e Coral Manager.

Resolução do runtime

A ordem de preferência é explícita: configuração do workspace/usuário quando fornecida, instalação persistente quando disponível e mecanismos de fallback controlados. **Coral: Mostrar runtime selecionado** apresenta a decisão efetiva.

Edição tolerante

Durante digitação, o LSP reduz cascatas de erros em cabeçalhos e blocos parciais, mantém diagnósticos versionados e reanalisa o documento completo quando necessário para preservar consistência.

Componentes

Coral: Verificar suporte opcional e **Coral: Instalar suporte opcional** delegam ao gerenciador da própria Coral. A extensão não implementa uma segunda política de `pip`.

Coral Manager

A extensão pode abrir o Manager e disparar diagnóstico da instalação, mantendo editor e runtime sobre o mesmo contrato de manutenção.

Depuração

O DAP Coral é implementado pelo runtime. F5 no VS Code inicia uma sessão que compila o projeto, preserva mapa de linhas e expõe pausa, pilha, variáveis e avaliação.

Breakpoints e módulos locais

A linha 1.3.11 eliminou uma limitação importante: breakpoints podem ser registrados em módulos `.coral` locais antes do launch e permanecem válidos quando a execução atravessa imports do projeto.

Navegação

`continue`, `next`, `stepIn` e `stepOut` possuem contratos automatizados. `stackTrace`, `scopes`, `variables` e `evaluate` permitem inspecionar o estado disponível na pausa.

O que ainda depende de ambiente gráfico

A lógica do protocolo é testada automaticamente. O comportamento completo da interface de uma janela real do VS Code em Windows é verificado pela suíte de campo e só recebe estado de aprovado mediante evidência externa.

Diagnóstico e logs

A linha 1.4 possui três superfícies complementares: Coral Manager, Coral Setup e os assistentes autocontidos.

Coral Manager

`coral manager` é a API operacional principal. Ele diagnostica, planeja instalação, atualização, reparo e remoção. Operações mutáveis só aplicam mudanças com confirmação explícita pelo parâmetro apropriado.

Coral Setup

`coral setup` é uma interface gráfica construída sobre `coral.manager/1`. O modo `coral setup --diagnostico` permite testar o contrato sem abrir janela.

Assistentes

`Instalar_Coral.pyz` e `Diagnosticar_Extensao.pyz`, com lançadores `.sh` e `.cmd`, permanecem como entradas autocontidas. O diagnosticador coleta evidências de runtime, DAP, ensino, VSIX e, quando possível, Extension Host.

Logs

A extensão possui o canal **Coral: Log** e exportação de log. A suíte Windows/VS Code cria um ZIP único de evidências tanto em sucesso quanto em falha, incluindo resumo JSON, eventos, stdout/stderr e registros relevantes do editor.

Atualizar uma instalação existente para a 1.7.4

1. Faça backup de projetos e configurações pessoais.
2. Use a distribuição completa 1.7.4 ou um patch cuja base seja declarada pelo fluxo de release; não aplique patches sobre uma base diferente da indicada.
3. Execute `Coral_Setup` ou `coral manager atualizar` para obter o plano da mudança.
4. Revise o plano antes de usar a forma com `--aplicar`.
5. Confirme `coral --versao` e `coral --self-check`. Os gates amplos de release pertencem ao processo de desenvolvimento e congelamento, não ao uso cotidiano.

A atualização não instala dependências opcionais silenciosamente. Perfis e componentes continuam sob o contrato do Manager e o modo Personalizado permanece disponível.

Referência do CLI

Use `coral --help` para a lista exata da versão instalada. Os fluxos principais estão organizados abaixo:

Tarefa	Comando
Executar arquivo	`coral arquivo.coral`
Verificar	`coral verificar arquivo.coral`
Lint	`coral lint arquivo.coral`
Formatar	`coral formatar arquivo.coral`
Testar projeto	`coral testar`
Documentar	`coral documentar`
REPL	`coral repl`
Versão	`coral --versao`
Instalação	`coral diagnostico`
VS Code	`coral vscode-check`
Jogos	`coral jogos-check`
NumPy	`coral --numerico-check`
Sistema	`coral --sistema-info`
Hardware	`coral --hardware-check`
Laboratório	`coral --laboratorio-check`

Tarefa	Comando
Modo Aprender	`coral explicar arquivo.coral`
Ensinar	`coral ensinar arquivo.coral`
Naturalidade	`coral naturalidade .`
Contratos 1.3	`coral --contratos-1-3`
Conformidade 1.3	`coral --conformidade-1-3`
Gate 1.3	`coral --gate-1-3`
Auditoria de release	`coral --release-check-1-3`

Ajuda da CLI

Referência para leitura

```
usage: coral [-h] [--versao] [--repl] [--mostrar-python]
             [--compilar ARQUIVO.py] [--verificar] [--lint] [--formatar]
             [--formatar-em-lugar] [--checar-formatacao]
             [--ast-expressao EXPRESSAO] [--tokens-expressao EXPRESSAO]
             [--ast-programa] [--ast-json] [--semantica] [--simbolos]
             [--mapa-linhas] [--diagnosticos-json] [--conformidade]
             [--conformidade-1-1] [--conformidade-1-2] [--conformidade-1-3]
             [--auditar-exemplos] [--robustez] [--robustez-intensa] [--corpus]
             [--sem-cache] [--cache-info] [--limpar-cache] [--ambiente]
             [--ambiente-json] [--self-check] [--lsp] [--dap] [--api-info]
             [--projeto-info] [--projeto-validar] [--projeto-executar]
             [--projeto-construir] [--dependencias] [--testar [CAMINHO]]
             [--testes-json [CAMINHO]] [--filtro-teste NOME] [--caso-teste N]
             [--documentar [DESTINO]] [--contratos-stdlib] [--contratos-1-3]
             [--contratos-1-3-json] [--gate-1-1] [--gate-rc-1-1]
             [--vscode-check] [--vscode-check-json] [--gate-vscode]
             [--gate-1-2] [--gate-1-3] [--gate-1-4] [--gate-1-4-9]
             [--gate-1-4-10] [--gate-1-4-11] [--gate-1-5]
             [--release-check-1-3] [--release-check-1-3-json]
             [--validar-vsix ARQUIVO] [--listar-templates]
             [--novo-projeto NOME] [--template-projeto ID]
             [--destino-projeto PASTA] [--jogos-check] [--numerico-check]
             [--sistema-info] [--sistema-info-json] [--hardware-check]
             [--hardware-check-json] [--laboratorio-check] [--explicar]
             [--explicar-json] [--naturalidade] [--naturalidade-json]
             [--diagnostico-instalacao] [--diagnostico-instalacao-json]
             [arquivo]

Executa, analisa, formata e traduz programas escritos na linguagem Coral.

positional arguments:
  arquivo               arquivo .coral

options:
  -h, --help            show this help message and exit
  --versao              mostra a versão do Coral
  --repl                abre o console interativo do Coral
  --mostrar-python      mostra o Python gerado antes da execução
  --compilar ARQUIVO.py
                        gera o Python correspondente sem executar
```

```
  --verificar           apenas verifica tradução e sintaxe
  --lint                analisa o arquivo e mostra diagnósticos
  --formatar            imprime o código formatado
  --formatar-em-lugar   formata o arquivo no próprio caminho
  --checar-formatacao   retorna erro se o arquivo/diretório precisar de
                        formatação
  --ast-expressao EXPRESSAO
                        mostra AST e Python de uma expressão Coral
  --tokens-expressao EXPRESSAO
                        mostra os tokens de uma expressão Coral
  --ast-programa        mostra a AST estrutural de um arquivo Coral
  --ast-json            mostra a AST estrutural em JSON
  --semantica           executa somente a análise semântica estrutural
  --simbolos            mostra a tabela de símbolos e escopos
  --mapa-linhas         mostra o mapa de linhas Python → Coral
  --diagnosticos-json   emite diagnósticos estáticos em JSON
  --conformidade        executa a suíte normativa de conformidade Coral 1.0
  --conformidade-1-1    executa a suíte normativa candidata do Coral 1.1
  --conformidade-1-2    executa a suíte normativa da linha Coral 1.2
  --conformidade-1-3    executa a suíte normativa candidata da linha Coral 1.3
  --auditar-exemplos    confirma que exemplos usam apenas o backend estrutural
  --robustez            executa testes determinísticos de robustez do
                        compilador
  --robustez-intensa    executa o gate pesado de robustez da linha 1.0
  --corpus              executa o corpus de programas Coral de estabilização
  --sem-cache           desativa o cache persistente desta compilação
  --cache-info          mostra informações do cache de compilação
  --limpar-cache        remove o cache persistente associado ao
                        arquivo/diretório atual
  --ambiente            mostra informações de plataforma e runtime
  --ambiente-json       mostra o ambiente em JSON
  --self-check          verifica rapidamente a instalação do Coral
  --lsp                 inicia o Coral Language Server experimental via stdio
  --dap                 inicia o Coral Debug Adapter via stdio
  --api-info            mostra a versão e a superfície da API pública Coral
  --projeto-info        mostra informações do coral.toml encontrado
  --projeto-validar     valida o coral.toml e os caminhos declarados
  --projeto-executar    constrói e executa o projeto coral.toml
  --projeto-construir   compila todos os módulos do projeto
  --dependencias        mostra o grafo de módulos Coral locais
  --testar [CAMINHO]    executa testes Coral
  --testes-json [CAMINHO]
                        executa testes Coral e retorna JSON
  --filtro-teste NOME   executa teste(s) por nome; pode repetir a opção, ou
                        usar grupo:NOME
  --caso-teste N        executa somente o caso parametrizado N; use com um
                        filtro de teste
  --documentar [DESTINO]
                        gera documentação do projeto Coral
  --contratos-stdlib    executa os contratos normativos da biblioteca padrão
                        Coral
  --contratos-1-3       executa os contratos públicos de comportamento da
                        Coral 1.3
  --contratos-1-3-json  emite os contratos públicos da Coral 1.3 em JSON
  --gate-1-1            executa o gate integrado da linha Coral 1.1
  --gate-rc-1-1         executa os gates de release candidate Coral 1.1
  --vscode-check        diagnostica a integração local/remota com VS Code
  --vscode-check-json   diagnostica a integração VS Code em JSON
  --gate-vscode         executa o gate de compatibilidade VS Code da linha
                        Coral 1.2
  --gate-1-2            executa o gate integrado da linha Coral 1.2
```

```
  --gate-1-3            executa o gate integrado da linha Coral 1.3
  --gate-1-4            verifica os 20 critérios de prontidão para Coral 1.4
  --gate-1-4-9          executa o gate específico da Coral 1.4.9
  --gate-1-4-10         executa o gate de fechamento da Coral 1.4.10
  --gate-1-4-11         executa o gate visual da Coral 1.4.11
  --gate-1-5            executa o gate final de promoção da Coral 1.5.0
  --release-check-1-3   audita o candidato/release Coral 1.3
  --release-check-1-3-json
                        audita a release Coral 1.3 em JSON
  --validar-vsix ARQUIVO
                        valida estruturalmente um pacote VSIX Coral
  --listar-templates    lista templates oficiais de projeto
  --novo-projeto NOME   cria um novo projeto Coral
  --template-projeto ID
                        template usado por --novo-projeto
  --destino-projeto PASTA
                        pasta de destino para --novo-projeto
  --jogos-check         verifica a integração opcional Pygame CE
  --numerico-check      verifica a integração opcional NumPy do Coral 1.3
  --sistema-info        mostra informações do sistema operacional para Coral
                        1.3
  --sistema-info-json   mostra informações do sistema operacional em JSON
  --hardware-check      diagnostica recursos e dependências de hardware do
                        Coral 1.3
  --hardware-check-json
                        diagnostica hardware em JSON
  --laboratorio-check   mostra capacidades do laboratório reprodutível Coral
                        1.3
  --explicar            executa o programa no Modo Aprender e explica os
                        passos
  --explicar-json       executa o Modo Aprender e retorna o rastreio em JSON
  --naturalidade        audita se o arquivo ou projeto preserva o estilo
                        natural e idiomático do Coral
  --naturalidade-json   emite a auditoria de naturalidade em JSON
  --diagnostico-instalacao
                        verifica instalação persistente e execução fora do
                        projeto
  --diagnostico-instalacao-json
                        diagnóstico de instalação em JSON

Manutenção: coral diagnostico --completo; coral reparar --help. Coral Manager:
coral manager --help. Coral Setup: coral setup --help. Perfis: coral perfis
--help. Componentes: coral componentes --help.
```

Configurações

coral.toml

Configuração TOML

```
[projeto]
nome = "meu-app"
versao = "0.1.0"

[coral]
versao = ">=1.7.4"

[entrada]
arquivo = "principal.coral"

[modulos]
caminhos = ["."]

[testes]
caminhos = ["testes"]
```

Instalação persistente e VS Code

A instalação gera `instalacao.json` e `vscode-settings.json`. Um exemplo de configuração explícita:

Configuração JSON

```
{
  "coral.pythonPath": "/caminho/para/coral/ambiente/bin/python",
  "coral.pyzPath": "/caminho/para/coral/coral.pyz",
  "coral.executablePath": "",
  "coral.format.enable": true,
  "coral.inlayHints.enable": true,
  "coral.trace.server": "off"
}
```

Quando esses caminhos não são definidos explicitamente, a extensão 0.77.0 pode consultar o manifesto persistente da instalação do usuário. Use **Coral: Mostrar runtime selecionado** para conferir a decisão.

launch.json

Configuração JSON

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "coral",
      "request": "launch",
      "name": "Coral: Depurar arquivo",
      "program": "${file}",
      "cwd": "${workspaceFolder}",
      "args": [],
      "stopOnEntry": false
    }
  ]
}
```

Solução de problemas

Sintoma	Ação sugerida
`coral` não é encontrado	Reabra o terminal, execute o instalador e consulte `coral diagnostico`.
Versão antiga	Execute `coral --versao`, confira `instalacao.json` e use **Coral: Mostrar runtime selecionado**.
F5 pede outro debugger	Abra um `.coral`, recarregue a janela e confirme Coral Language 0.77.0.
Run funciona e F5 não	Exporte **Coral: Log** e execute `Diagnosticar_Extensao`.
Breakpoint não é verificado	Use linha executável do arquivo de entrada; breakpoints internos em módulos locais continuam limitados.
LSP não inicia	Execute `coral vscode-check` e confira o runtime selecionado.
NumPy ausente	Instale NumPy no Python da instalação Coral e execute `coral --numerico-check`.
Estatística vazia falha	É contrato preservado na 1.5.5; forneça ao menos um valor. Variância amostral exige dois.
Experimento com zero repetições falha	Use `repeticoes >= 1`; zero é inválido por contrato.
pyserial ausente	Instale pyserial no ambiente Coral antes de usar portas seriais.
Porta serial falha	Execute `coral --hardware-check`; a serial física depende de evidência em dispositivo real.
Pygame ausente	Instale `pygame-ce` no Python do ambiente Coral e execute `coral jogos-check`.
Tela não abre em CI ou SSH	Use `BackendNulo` ou execute em ambiente gráfico.
Programa externo não inicia	`executar_comando` não usa shell; passe programa e argumentos separadamente.
Diagnósticos aparecem enquanto digita	Atualize para 0.77.0; a edição tolerante reduz cascatas, mas alguns limites estruturais continuam.

Antes de pedir ajuda

Terminal

```
coral --versao
coral --self-check
coral diagnostico
coral vscode-check
coral --gate-1-3
coral --release-check-1-3
```

Se o problema estiver no editor, exporte **Coral: Log**. Para instalação, VSIX ou DAP, execute o assistente externo e compartilhe o ZIP de logs.

Instalação

A distribuição completa separa responsabilidades em seis áreas principais. Código fonte, instalação, testes, documentação, exemplos e Livro Oficial não competem pelo mesmo diretório. Histórico técnico permanece dentro das áreas correspondentes, sem recriar uma segunda árvore ativa.

Referência para leitura

```
Coral/
|--- 00_LEIA_PRIMEIRO.md
|--- Coral.code-workspace
|--- Instalacao/
|   |--- Runtime/
|   |--- Extensao_VSCode/
|   |--- Assistentes/
|   `--- Ferramentas/
|--- Documentacao/
|   |--- Usuario/
|   |--- Desenvolvimento/
|   |--- Identidade_Visual/
|   `--- Historico/
|--- Projeto/
|   |--- coral/
|   |--- tools/
|   |--- vscode-coral/
|   |--- configuracao/
|   `--- auditoria/
|--- Exemplos/
|--- Testes/
|   |--- Rapidos/
|   |--- Integracao/
|   |--- Slow/

|   |--- Historicos/
|   |--- Campo/
|   `--- Executar_Testes_Reais.coral
`--- Livro/
    |--- Coral_1.7.6_Volume_I_Fonte.md
    |--- Coral_1.7.6_Volume_I.pdf
    |--- Exemplos/
    `--- recursos/
```

Artefatos correntes

Artefato	Arquivo	Papel
Runtime Coral	`Instalacao/Runtime/coral-1.7.6.pyz`	aplicação Python zip portátil
Extensão VS Code	`Instalacao/Extensao_VSCode/coral-language-0.77.0.vsix`	editor, LSP, DAP, testes e experiência de estudo
Fonte da extensão	`Instalacao/Extensao_VSCode/coral-vscode-0.77.0-source.zip`	auditoria e desenvolvimento da extensão
Coral Manager	`coral manager`	diagnóstico, instalação, atualização, reparo e remoção
Coral Setup	`Instalacao/Coral_Setup.*`	interface que consome o mesmo contrato do Manager
Livro Oficial	`Livro/Coral_1.7.6_Volume_I.pdf`	volume didático da coleção oficial
Livro Oficial, Volume II	`Livro/Coral_1.7.6_Volume_II.pdf`	referência e arquitetura da coleção oficial
Testes reais	`Testes/Executar_Testes_Reais.coral`	evidência assistida em máquina física

Dependências

O núcleo exige **Python 3.10 ou superior**. NumPy, Matplotlib, Plotly, psutil, pyserial e Pygame CE continuam opcionais e são administráveis pelo sistema oficial de componentes e perfis. A extensão declara compatibilidade com **VS Code ^1.85.0**.

Há quatro caminhos oficiais, todos apoiados pela mesma base de runtime.

Coral Setup, recomendado para uso comum

Abra `Instalacao/Coral_Setup.cmd` no Windows ou `Instalacao/Coral_Setup.sh` em sistemas POSIX. A interface usa o contrato `coral.manager/1`: não existe um segundo instalador escondido na GUI.

Coral Manager, recomendado para automação e diagnóstico

Terminal

```
python Instalacao/Runtime/coral-1.7.6.pyz manager diagnosticar --json
python Instalacao/Runtime/coral-1.7.6.pyz manager instalar --runtime coral-1.7.6.pyz
  --perfil estudante --json
python Instalacao/Runtime/coral-1.7.6.pyz manager instalar --runtime coral-1.7.6.pyz
  --perfil estudante --aplicar --json
```

Operações mutáveis produzem primeiro um plano. A alteração é aplicada apenas quando o usuário pede explicitamente.

Runtime portátil

Terminal

```
python3 Instalacao/Runtime/coral-1.7.6.pyz --versao
python3 Instalacao/Runtime/coral-1.7.6.pyz meu_programa.coral
```

Desenvolvimento do compilador

Terminal

```
python -m venv .venv
# Linux/macOS
source .venv/bin/activate
# Windows PowerShell
.\.venv\Scripts\Activate.ps1
python -m pip install -e .
python -m coral --self-check
```

A instalação persistente continua disponível pelos assistentes em `Instalacao/Assistentes`.

4.1 Requisitos

O executável Linux x86_64 inclui o pacote Coral com Python e pip privados. O modo portátil continua aceitando Python 3.10 ou superior quando o usuário escolhe executar o PYZ diretamente.

4.2 Setup ou assistente

Para instalar a versão atual pelo launcher da distribuição, execute:

Terminal

```
bash Instalacao/Coral_Setup.sh
```

O pacote `Projeto/dist/nativos/coral-setup_1.7.0_amd64.deb` é histórico da 1.7.0 e não deve ser confundido com um instalador nativo novo da 1.7.4. Os launchers `Instalacao/Coral_Setup.sh` e os assistentes de terminal continuam disponíveis para o fluxo portátil e de desenvolvimento.

4.3 Validar

Terminal

```
coral --versao
coral --self-check
coral --gate-1-3
coral --gate-1-4
coral diagnostico --completo
```

4.4 VS Code

O VSIX corrente fica em `Instalacao/Extensao_VSCode/coral-language-0.77.0.vsix`. Instale pela interface **Extensions: Install from VSIX...** ou pelo CLI do editor.

4.5 Componentes opcionais

Use `coral componentes listar`, `coral componentes verificar` e `coral componentes instalar`. Perfis como estudante, científico, hardware, jogos e completo agrupam capacidades sem transformar dependências opcionais em requisitos do núcleo.

Realce semântico na 1.7.4

O LSP fornece Semantic Tokens usando a resolução compartilhada de símbolos e interfaces. Distingue categorias comprováveis e posições de declaração, referência e tipo. O cliente VS Code usa `coral.semanticColoring.enable`; o tema decide as cores. TextMate é a base de fallback para trechos incompletos ou não resolvidos.

A análise de dependências considera importações também dentro de funções e mantém os caminhos das dependências mesmo quando um arquivo contém erros de sintaxe. Mudanças num contrato de arquivo aberto provocam nova publicação de diagnósticos nos consumidores abertos e invalidam o cache de tokens afetado.

Editor na Coral 1.7.6

Consultas de pertencimento oferecem autocomplete restrito a tipos e classes válidas. As mesmas identidades alimentam navegação, hover e Semantic Tokens, com coordenadas UTF 16 do protocolo. Em expressões compostas, o contexto especial de tipo termina no identificador consultado; o restante volta ao completamento comum.

Chamadas multilinha oferecem ajuda de assinatura para funções, métodos e construtores, incluindo construtores herdados. O editor conserva o alvo e o argumento ativo durante uma edição incompleta. O compilador usa o parser estrito. As opções de fechamento e realce existentes continuam disponíveis.

Ambientes reproduzíveis da 1.7.6

`coral manager ambiente` oferece `resolver`, `preparar`, `verificar` e `sincronizar`. O manifesto é lido por um contrato central; a resolução usa pip isolado e aceita apenas wheels. `coral.lock`, no formato `coral.projeto.resolucao/1`, registra requisitos normalizados, identidade do runtime, alvo, grafo selecionado, origem e hashes. Alterar requisitos, catálogo, runtime ou alvo invalida a resolução adotada.

A aplicação prepara uma nova geração sob trava e só altera o ponteiro ativo depois de verificar instalação, arquivos e isolamento. O registro `coral.projeto.ambiente/1` identifica essa geração. A verificação não importa código do usuário para decidir se a instalação é

válida. Gerações anteriores permanecem disponíveis; a remoção automática não faz parte deste contrato.

`--simular` não resolve nem instala. `--offline` usa artefatos já disponíveis. Rede exige `--permitir-rede`. `--cancel-arquivo CAMINHO` cancela em uma fronteira segura antes de publicar lock ou ativar geração; não encerra imediatamente um pip em andamento. O runtime verifica capacidades do pip e espaço estimado antes de aplicar. A estimativa é informativa e não reserva espaço nem mede quotas.

A CLI e o launcher de depuração selecionam o executor de projeto com uma ponte que evita relançamento recursivo. API Python de execução chamada diretamente, Windows, rede pública e operação física do VS Code não foram qualificados neste checkpoint. Essas limitações não são convertidas em garantias pela existência dos comandos.

PARTE IV: IMPLEMENTAÇÃO E ARQUITETURA

Objetivo desta parte

Entender como a fonte Coral chega à execução em Python.

Como usar esta referência

Leia as camadas em sequência. AST, análise semântica e backend são etapas distintas do compilador.

Fundação em Python

Coral é implementada em Python e **transpila** programas `.coral` para Python. Isso permite aproveitar portabilidade, biblioteca padrão, ecossistema, mecanismos de exceção, asyncio e empacotamento Python sem obrigar a superfície da linguagem a parecer Python.

Camadas de distribuição

- `Projeto/pyproject.toml` define o pacote `coral-linguagem`, com versão obtida de `coral.__version__`, requer Python >=3.10 e registra o comando `coral = coral.cli:main`.
- `coral-1.7.6.pyz` é um **Python zip application**: um arquivo ZIP executável por Python que contém o pacote e um ponto de entrada.
- a instalação persistente cria um `venv`, copia o `.pyz` e cria launchers.
- a extensão VS Code não contém um segundo compilador; ela chama o mesmo runtime por CLI/LSP/DAP.

O que transpilar significa aqui

Referência para leitura

```
fonte .coral
    ↓
parser de programa + parser de expressões
    ↓
AST Coral
    ↓
análise semântica
    ↓
backend estrutural
    ↓
Python gerado + mapa de linhas
    ↓
compile/exec do Python
```

O Python gerado é uma representação de backend. Ele pode ser inspecionado com `--mostrar-python` ou salvo com `--compilar`, mas não é a API editorial principal da linguagem.

Modelo de segurança

Coral **não é um sandbox**. Um programa Coral, inclusive código trazido por `python:`, executa com as permissões do processo Python. Nunca execute código Coral não confiável supondo isolamento.

Interoperabilidade

`python:` permanece o escape explícito:

Código Coral

```
python: print("interoperabilidade direta")
```

Saída esperada

```
interoperabilidade direta
```

Imports também permitem usar módulos Python quando a sintaxe Coral os referencia. Para a biblioteca oficial, aliases como `matematica` e `arquivos` são resolvidos para `coral.stdlib` quando apropriado; módulos locais têm prioridade. Quando há colisão, o namespace explícito `coral.json`, por exemplo, remove a ambiguidade.

Extras científicos da 1.3

O `pyproject.toml` registra extras opcionais `numerico`, `sistema`, `hardware`, `jogos` e `completo`. O núcleo permanece instalável sem essas dependências. Isso mantém o compilador leve e permite que um curso introdutório não dependa de NumPy, Pygame ou acesso serial.

Pipeline do compilador

Caminho real de compilação

`coral.api.compilar` e `traduzir_codigo` convergem em `coral.tradutor.compilar_validado`. A função executa análise semântica e depois geração estrutural. Nesta implementação, a semântica analisa o programa e o backend volta a analisá lo; não há ainda uma única AST compartilhada entre todas as etapas.

Módulo	Responsabilidade
`coral/parser_programa.py`	Reconhecer instruções, blocos e construções naturais a partir das linhas
`coral/lexico.py`	Tokenizar expressões, preservando literais
`coral/parser_expressoes.py`	Precedência e composição da AST de expressões
`coral/ast_programa.py`	Nós de programa, blocos e localização de fonte

Módulo	Responsabilidade
`coral/nos.py`	Nós de expressão e sua representação Python
`coral/semantica.py`	Diagnósticos contextuais, como retorno fora de função e await fora de contexto assíncrono
`coral/backend_estrutural.py`	Gerar Python por nós estruturais e produzir mapa de linhas
`coral/runtime.py`	Operações auxiliares usadas pelo programa gerado
`coral/diagnosticos.py`	Normalizar diagnósticos para CLI e ferramentas
`coral/simbolos.py`	Símbolos e escopos para ferramentas de linguagem

O parser de programa usa reconhecedores de linha, incluindo expressões regulares, para construir nós próprios. Isso não equivale a um fallback de tradução textual: o caminho oficial precisa reconhecer a construção estruturalmente. O lexer de expressões não é uma etapa única aplicada previamente ao programa inteiro.

Expressões e interoperabilidade

Chamadas naturais são declaradas e analisadas deterministicamente. `python:` é escape explícito. `traduzir_expressao` mantém espaços internos de literais na 1.2.1; a normalização textual anterior alterava dados do usuário. O arquivo `tradutor.py` continua sendo uma fachada pública e contém auxiliares; seu nome não indica a permanência do tradutor legado 0.x.

Execução e projetos

O CLI compila e executa Python no processo do runtime. `projeto.py` lê `coral.toml`, coleta módulos e produz `.coral_build`. A 1.2.1 rejeita diretórios de saída que contenham fontes, configuração ou raiz do projeto, e rejeita módulos distintos que gerariam o mesmo arquivo Python, antes de limpar a saída.

O cache em `cache.py` depende da fonte, nome de arquivo, versão Coral e formato de serialização. É uma otimização: entradas inválidas são descartadas e recompiladas. Escritas usam temporários individuais com substituição final do arquivo.

Editor

A extensão `Projeto/vscode-coral/extension.js` inicia `coral --lsp` para linguagem e `coral --dap` para depuração. O cliente LSP está em `coral-lsp-client.js`. `lsp.py` implementa os recursos de linguagem; `dap.py` usa o mapa de linhas para conectar a execução Python à fonte Coral. Test Explorer e Modo Aprender consomem interfaces do runtime.

Bibliotecas

`regras.py` fornece motor, condições, ações, eventos e temporizadores. `mundo.py` representa entidades e relações; a busca BFS é bidirecional por padrão apesar de as relações serem dirigidas. `rpg.py` oferece rolagens e turnos. `jogos/base.py` define o contrato do backend e o ciclo de vida; `pygame_backend.py` realiza as operações gráficas e `BackendNulo` registra comandos sem abrir janela.

Limites arquiteturais

A análise semântica não é uma prova de correção completa nem um sistema obrigatório de tipos estáticos. A gramática EBNF 1.0 permanece histórica; a linha 1.6.1 mantém uma referência sintática versionada apoiada pelo parser e por testes sentinela. LSP e DAP reutilizam as estruturas correntes do compilador. A execução Coral tem os acessos do processo Python e não oferece isolamento de código não confiável.

Referências internas: `Projeto/coral/tradutor.py`, `Projeto/coral/api.py`, `Projeto/coral/parser_programa.py`, `Projeto/coral/backend_estrutural.py` e os testes correspondentes em `Testes/`.

Módulos centrais e tamanho aproximado nesta distribuição

Módulo	Linhas
`coral/ast_programa.py`	238
`coral/backend_estrutural.py`	588
`coral/cache.py`	143
`coral/cli.py`	821
`coral/dap.py`	552
`coral/diagnosticos.py`	84
`coral/lexico.py`	93
`coral/lsp.py`	1223
`coral/nos.py`	207
`coral/parser_expressoes.py`	736
`coral/parser_programa.py`	835
`coral/projeto.py`	361
`coral/runtime.py`	7
`coral/semantica.py`	184
`coral/simbolos.py`	191
`coral/tradutor.py`	214

A fachada pública `tradutor.py` permanece por compatibilidade de API, mas o caminho estável é estrutural. O nome histórico do arquivo não significa que o compilador 0.x de substituições textuais continue como fallback oficial.

Lexer e expressões

Expressões são analisadas por uma camada própria, separada do reconhecimento de blocos do programa. `coral.lexico` tokeniza, preservando strings; `coral.parser_expressoes` aplica precedência e constrói nós de `coral.nos`.

Nós de expressão atuais

`NoExpressao`, `Literal`, `Nome`, `Lista`, `Tupla`, `Conjunto`, `Dicionario`, `ArgumentoChamada`, `Chamada`, `Lambda`, `Atributo`, `Indice`, `Fatia`, `Aguarde`, `Unario`, `Binario`, `Comparacao`, `CompreensaoLista`, `CompreensaoConjunto`, `CompreensaoDicionario`.

Precedência conceitual

Da menor para a maior ligação:

1. `ou`
2. `e`
3. comparações (`for igual a`, `for maior que`, etc.)
4. adição/subtração
5. multiplicação/divisão
6. potência
7. operadores unários
8. chamada, atributo, índice e fatia

O parser aceita formas naturais e algumas formas simbólicas para interoperabilidade. A identidade editorial favorece palavras em português quando a construção oficial existe.

AST, não execução

O parser produz objetos como `Literal`, `Nome`, `Binario`, `Comparacao`, `Chamada`, `Atributo` e `Indice`. Isso permite LSP, análise, formatação e backend raciocinarem sobre estrutura em vez de depender de `eval` ou de substituições de texto.

Programa, blocos e AST

`coral.parser_programa` percorre as linhas do arquivo, reconhece construções por regras determinísticas e produz nós de `coral.ast_programa`.

Nós principais

`InstrucaoSimples`, `Bloco`, `RamoCondicional`, `Condicional`, `RamoTente`, `Tente`, `Caso`, `Combinar`, `FormaChamadaNatural`, `RegraDeclarativa`, `EventoDeclarativo`, `TemporizadorDeclarativo`, `Programa`.

O parser usa reconhecedores de linha, inclusive expressões regulares. Isso não é equivalente a um tradutor textual: uma construção só entra no caminho oficial depois de ser classificada e representada como nó estruturado.

Delimitador de bloco

Coral usa `fim` como delimitador explícito. A indentação é parte do estilo oficial e das ferramentas, mas `fim` é o sinal estrutural principal.

Chamadas naturais

Uma função pode declarar uma frase de chamada:

Código Coral

```
crie a função somar com a e b chamada como "some {a} com {b}"
    retorne a mais b
fim

defina total como some 4 com 7
```

Este exemplo não imprime texto no terminal.

O template é validado no parser: placeholders precisam corresponder a parâmetros e templates ambíguos/duplicados são rejeitados. Não há interpretação livre por IA.

Regras declarativas

`RegraDeclarativa`, `EventoDeclarativo` e `TemporizadorDeclarativo` são nós próprios. A forma longa de regra exige a sequência `regra`, `quando`, `então`, corpo e `fim`, com erros P130/P131 quando etapas obrigatórias estão ausentes.

Semântica e símbolos

Depois do parse, `coral.semantica` verifica regras contextuais que não são apenas sintaxe.

Exemplos de erros:

Código	Exemplo de categoria
S101	retorno/produção fora de função
S102	`pare` fora de laço
S103	`continue` fora de laço
S104	`aguarde` fora de contexto assíncrono
S105	laço assíncrono fora de função assíncrona
S106	contexto assíncrono fora de função assíncrona
S107	método/propriedade fora de classe
S110	teste em escopo inválido
S120-S123	restrições contextuais de regras/eventos/temporizadores

O parser usa códigos Pxxx para problemas estruturais, e o linter usa Cxxx para estilo/geração, incluindo:

- `C101`: tabulação em vez de espaços.
- `C102`: linha acima de 150 caracteres.
- `C201`: uso de `python:` reduz portabilidade.
- `C202`: captura genérica de exceção.
- `C203`: importação com `*`.

A tabela de símbolos em `coral.simbolos` registra escopos e localizações e alimenta recursos de IDE como definição, referências, rename, completion e hierarquias.

Dívida arquitetural conhecida

Na arquitetura atual, a análise semântica parseia o programa e o backend estrutural volta a parsear antes de gerar Python. Ainda não há uma única instância de AST compartilhada por todas as etapas. O comportamento é testado, mas essa duplicação é uma oportunidade futura de simplificação interna.

Backend e mapas de linhas

`coral.backend_estrutural` gera Python a partir dos nós do programa. Durante a geração, mantém um mapa entre linhas Python e linhas Coral.

Esse mapa é essencial para:

- erros de execução que apontam para o arquivo Coral;
- breakpoints e stack frames do DAP;
- Modo Aprender;
- diagnósticos e ferramentas de inspeção.

Inspecione:

Terminal

```
coral --mostrar-python programa.coral
coral --mapa-linhas programa.coral
coral --compilar programa.py programa.coral
```

O módulo `coral.runtime` contém operações auxiliares usadas pelo Python gerado. `coral.regras`, `mundo`, `rpg` e `jogos` são runtimes públicos especializados, não macros textuais.

Cache

`coral.cache` fornece cache persistente endereçado por conteúdo e versão, com escrita por arquivo temporário e substituição final. O parser de expressões também usa cache em memória. Cache é otimização: dados inválidos devem ser descartados e recompilados.

Terminal

```
coral --cache-info
coral --limpar-cache
coral --sem-cache programa.coral
```

Módulos e construção

`coral.projeto` lê `coral.toml`, descobre módulos locais, monta o grafo de importações, detecta ciclos e constrói saída em `.coral_build`.

Módulos locais têm prioridade sobre aliases curtos da stdlib. A construção rejeita situações perigosas como um diretório de saída contendo a própria fonte/configuração e módulos diferentes que colidiriam no mesmo arquivo Python gerado.

Exemplo de coral.toml

Configuração TOML

```
[projeto]
nome = "meu-projeto"
versao = "0.1.0"

[coral]
versao = ">=1.7.4"

[entrada]
arquivo = "principal.coral"

[modulos]
caminhos = ["."]

[testes]
caminhos = ["testes"]
```

Comandos relacionados:

Terminal

```
coral projeto-info
coral dependencias
coral construir
coral executar
coral testar
coral documentar
```

LSP

O LSP separa inteligência da linguagem da interface do editor. Coral inicia o servidor com:

Terminal

```
coral --lsp
```

O processo conversa por **JSON-RPC sobre stdio**. O cliente do VS Code é implementado no próprio projeto, sem dependência runtime de `vscode-languageclient`.

Capacidades declaradas pelo servidor 1.5.5

- sincronização de documento
- hover
- go to definition
- references
- document symbols
- workspace symbols
- rename com prepare
- completion
- signature help
- formatação de documento
- range formatting
- on type formatting

- document highlights
- selection ranges
- call hierarchy
- type hierarchy
- code actions
- semantic tokens
- folding
- inlay hints
- document links
- CodeLens

Ciclo simplificado

Referência para leitura

```
VS Code
  | JSON-RPC / stdio
  v
coral-lsp-client.js
  |
  v
coral --lsp
  |
  |-- parser / semântica
  |-- símbolos / índice de workspace
  |-- formatador / linter
  `-- documentação / tipos / chamadas naturais
```

O uso de LSP reduz duplicação: a mesma análise pode, em princípio, ser consumida por outros editores que implementem o protocolo.

Edição tolerante e cache na 1.3

O LSP diferencia o diagnóstico tolerante usado enquanto a pessoa digita do diagnóstico estrito aplicado ao salvar. Cabeçalhos e construções parcialmente reconhecidas podem ser tratados como edição incompleta, reduzindo falsos positivos durante a escrita.

O índice de símbolos mantém cache limitado por conteúdo e tamanho. Documentos acima do limite configurado são processados sem permanecer no cache. Essa estratégia não equivale a um parser incremental completo; é uma otimização controlada com recuperação parcial.

DAP

O debugger é iniciado por:

Terminal

```
coral --dap
```

O VS Code conversa com `coral.dap` pelo **Debug Adapter Protocol**. O adaptador prepara o arquivo, compila Coral, usa o mapa de linhas e controla a execução Python.

Capacidades atuais

- configurationDone
- breakpoints condicionais
- hit conditions
- log points
- breakpoint locations
- exceções não tratadas/todas
- exceptionInfo
- setVariable
- stackTrace
- scopes
- variables
- evaluate
- continue
- step over
- step in
- step out
- pause
- terminate/disconnect

Fluxo

Referência para leitura

```
VS Code Run and Debug
   | DAP (JSON)
   v
coral-debug.js
   | stdio
   v
coral --dap
   |
   |-- compilação Coral -> Python
   |-- mapa de linhas
   |-- breakpoints
   |-- inspeção de variáveis
   `-- controle de execução
```

Limites honestos

O adaptador anuncia `supportsStepBack: false` e `supportsRestartRequest: true`. O comando de restart reinicia a sessão preservando a configuração de lançamento aplicável. Breakpoints podem ser associados aos módulos Coral conhecidos pelo projeto e são traduzidos pelo mapa de linhas correspondente; fontes que não pertencem ao projeto continuam sendo rejeitadas ou marcadas como não verificadas em vez de receber suporte fictício. Retroceder a execução continua fora do contrato da 1.6.1.

Extensão e VSIX

A extensão está em `Projeto/vscode-coral/` e é empacotada como VSIX. O manifesto atual declara:

Campo	Valor
Nome	Coral Language
ID	`coral-lang.coral-language`
Versão	0.77.0
VS Code	`^1.85.0`
Runtime dependencies	nenhuma
Dev dependency	`@vscode/vsce ^3.0.0`

Arquivos de execução principais

- `extension.js`: ativação, comandos, tarefas, Project Explorer, Test Explorer e integração geral.
- `coral-runtime.js`: descoberta e invocação do runtime Coral.
- `coral-lsp-client.js`: cliente LSP JSON-RPC autocontido.
- `coral-debug.js`: configuração e ponte de depuração.
- `coral-teaching.js`: estudo automático estático.
- `coral-log.js`: canal **Coral: Log** e exportação de logs.

Há ainda gramática TextMate, snippets, configuração da linguagem, ícones e recursos de UI.

VSIX

A distribuição inclui `coral-language-0.67.0.vsix`. O VS Code permite instalar VSIX pela Command Palette/Extensions e por `code --install-extension caminho.vsix`. Instalações via VSIX têm atualização automática desativada por padrão segundo a documentação do editor; uma atualização da Coral deve portanto considerar runtime **e** extensão.

Instalação e Manager

A instalação possui uma única autoridade de mutação: o contrato `coral.manager/1`. O **Coral Setup** implementa `coral.setup/1` como interface gráfica para esse mesmo contrato; launchers e VS Code delegam às mesmas operações, sem manter uma segunda tabela de pacotes ou chamar `pip` diretamente.

Componentes e perfis

O núcleo exige apenas Python 3.10 ou superior. NumPy, Matplotlib, Plotly, psutil, pyserial e Pygame CE permanecem opcionais e são administrados pelo sistema oficial de **componentes e perfis**. Perfis como Estudante, Científico, Hardware, Jogos e Completo agrupam intenções comuns; o modo Personalizado continua disponível para escolher capacidades individualmente.

Uma operação mutável produz primeiro um plano. Instalação, atualização, reparo ou remoção só alteram o ambiente quando o usuário solicita explicitamente a aplicação. Diagnóstico e planejamento nunca instalam dependências como efeito colateral.

Ciclo mínimo comprovável

A prova automatizada de instalação usa um destino isolado e o perfil Estudante. Ela percorre planejamento, instalação, diagnóstico, launcher, reparo, atualização e remoção sem alterar PATH global. Isso prova o contrato operacional sem fingir validar propriedades visuais de uma janela real.

Setup 1.5

O visual consolidado deriva do redesign da 1.4.11: identidade da cobra Coral, hierarquia mais limpa, indicador das cinco etapas, navegação por teclado, tamanho mínimo e suporte a redução de movimento. O Setup não possui um motor de instalação próprio; toda mutação continua passando pelo Manager.

`coral setup --diagnostico` informa disponibilidade de Tkinter e contexto gráfico. Em Linux sem DISPLAY/Wayland e em qualquer ambiente onde a janela real não possa ser exercitada, a evidência visual permanece `nao_executado`, não “aprovada”.

Launchers correntes

Windows e Linux apontam para `Instalacao/Runtime/coral-1.7.6.pyz`. Os assistentes de instalação e diagnóstico são reconstruídos a partir da mesma árvore da release.

Princípios de manutenção

1. Português corrente, não mera tradução lexical

Coral deve ser escrito e lido como um subconjunto controlado do português. A forma preferida é:

Código Coral

```
defina idade como 19
se idade for maior ou igual a 18 então
    mostre "Acesso permitido"
senão
    mostre "Acesso negado"
fim
```

Saída esperada

```
Acesso permitido
```

A linguagem evita fazer o usuário pensar primeiro em Python para depois traduzir. Operadores simbólicos e chamadas convencionais continuam disponíveis quando necessários à interoperabilidade, mas não definem a identidade do Coral.

2. Naturalidade controlada

Português totalmente livre seria ambíguo. Coral aceita formas deliberadamente limitadas, documentadas e determinísticas. A leitura deve soar natural sem depender de IA ou de adivinhação da intenção do programador.

3. Legibilidade antes de concisão

Entre duas sintaxes equivalentes, a API oficial deve preferir a que comunica intenção com menos conhecimento prévio de convenções de outras linguagens.

Código Coral

```
defina pontos como 10
adicione 5 a pontos
```

Este exemplo não imprime texto no terminal.

é preferível, na documentação introdutória, a uma mutação simbólica equivalente.

4. APIs oficiais devem ter uma superfície Coral

Bibliotecas podem usar classes e métodos internamente, mas templates, tutoriais e exemplos devem oferecer uma camada que permaneça coerente com a leitura da linguagem. Chamadas como `objeto.metodo(...)` são válidas e importantes para interoperabilidade, porém não devem ser a única forma de ensinar os recursos centrais da biblioteca oficial.

5. Previsibilidade acima de adivinhação

Chamadas naturais são declarativas (`chamada como`) e regras/eventos possuem gramática formal. A mesma entrada deve sempre ter a mesma interpretação.

6. Python como plataforma, não como aparência

Coral transpila para Python para aproveitar runtime, bibliotecas, portabilidade e ecossistema. Essa decisão de implementação não deve obrigar o código Coral a parecer código Python.

7. Interoperabilidade progressiva

O prefixo `python:` permanece como escape explícito. Ele é uma ferramenta de integração, não a forma recomendada para funcionalidades que já possuem equivalente Coral.

8. AST e semântica próprias

A sintaxe oficial deve ser reconhecida pelo parser e representada na AST Coral. Recursos novos não devem ser introduzidos como simples substituições textuais frágeis.

9. Ensino sem infantilização

Coral busca reduzir barreiras linguísticas e conceituais sem esconder que programação possui estado, fluxo, funções, objetos, erros e abstrações. O objetivo é tornar esses conceitos observáveis e legíveis, não fingir que eles não existem.

10. Critério para novos recursos

Um recurso candidato deve responder positivamente a estas perguntas:

1. É determinístico?
2. É legível em português sem conhecer sua tradução Python?
3. É consistente com construções já existentes?
4. Pode ser diagnosticado pelo compilador em termos Coral?
5. Pode ser documentado e testado sem depender do backend Python?

Se uma nova API importante só puder ser demonstrada com chamadas Pythonicas, ela ainda não atingiu a experiência ideal da biblioteca oficial Coral.

Checklist de implementação

Ao adicionar sintaxe nova:

1. defina a forma portuguesa e casos ambiguamente parecidos;
2. adicione testes de parser antes do backend;
3. represente a construção em AST ou em um tipo estrutural existente adequado;
4. acrescente diagnósticos semânticos quando houver restrições de contexto;
5. gere Python no backend estrutural;

6. preserve mapa de fonte;
7. acrescente formatação, highlighting/snippet/LSP quando relevante;
8. atualize conformidade e naturalidade;
9. documente a forma natural como principal e a API OO/Python como avançada;
10. execute gates e testes de regressão.

Uma nova API que só pode ser ensinada como `objeto.metodo(...)` ainda não atingiu a experiência Coral ideal quando poderia existir uma frase determinística melhor.

11. Separar estabilidade de experimentação

Na 1.5.5, essa regra também vale para comunicação serial, Pygame, áudio, janela, entrada real e recursos dependentes do sistema: contratos automatizados cobrem o que o ambiente permite, enquanto `Testes/Executar_Testes_Reais.coral` registra a evidência complementar em máquina física. Uma funcionalidade não deve ganhar o rótulo de estável apenas porque compila ou passa por mocks.

Autoridade documental da base estável

A coleção oficial é formada pelos Volumes I e II da 1.7.6, com fontes e PDFs registrados em `Livro/EDICAO.json`. O livro único 1.7.0 permanece como edição histórica. A identificação oficial da coleção foi definida pelo responsável do projeto; ela não elimina as lacunas editoriais registradas na validação. O histórico de evolução é preservado principalmente pelo changelog e pela documentação de release; cópias editoriais antigas não precisam permanecer indefinidamente na árvore principal. O índice corrente, os manuais operacionais e o README da distribuição devem apontar para esta edição.

Resolução e build incremental na 1.7.4

`coral.contratos_importados` centraliza a resolução de contratos públicos importados. O compilador, a análise semântica, a tipagem e o editor recebem interfaces como contexto, sem executar código do usuário para descobrir assinaturas.

O hash de contrato transitivo incorpora interfaces públicas das dependências e reexportações. O manifesto de build identifica a versão do contrato para invalidar caches anteriores. Um build limpo e um incremental devem produzir comportamento equivalente. A inferência permanece conservadora quando não existe evidência suficiente.

Documentos, snapshots e índice seletivo na 1.7.5

`_lsp_documentos` administra documentos abertos, URI normalizada, versões e gerações. Eventos antigos são rejeitados. O fechamento retira o overlay e restaura a leitura do disco. `_lsp_snapshots` mantém uma LRU com até 128 entradas e 32 MiB de fontes; o limite em bytes não representa um teto de heap total. Consumidores recebem cópias defensivas dos resultados mutáveis.

A chave de snapshot considera fonte, opções, identidade de projeto, interfaces de dependências e versão do contrato. `_lsp_indice` mantém relações diretas e reversas, resumos compactos de símbolos e contratos adquiridos. Uma edição de corpo preserva os consumidores quando o contrato já conhecido permanece igual; a primeira edição sem contrato aquecido pode invalidar transitivamente de modo conservador.

Criação, remoção e mudança de manifesto atualizam o projeto afetado. Raízes aninhadas são separadas, overlays são reaplicados depois de reconstruções e ciclos não causam recursão infinita. Navegação e renomeação respeitam identidade, aliases, reexportações e sombras locais; uma referência ambígua não é renomeada por coincidência textual.

`lsp.py` mantém a fachada e o transporte públicos. Os serviços não executam código do usuário para descobrir tipos. O protocolo permanece síncrono. As medições registradas em `Projeto/MEDICAO_EDITOR_1_7_5.json` comparam processos Python; não medem a latência percebida do VS Code nem qualificam equipamentos físicos.

PARTE V: COMPATIBILIDADE, LIMITES E INTEROPERABILIDADE

Objetivo desta parte

Integrar ferramentas sem depender de detalhes internos.

Como usar esta referência

Prefira a API pública. Confira compatibilidade, limites e o glossário ao interpretar um contrato.

Coral mantém uma API pública estável da família 1.x para ferramentas que precisam analisar ou compilar sem invocar a CLI.

`coral.api.info_api()` informa:

Configuração JSON

```
{
  "api": "1.0",
  "funcoes": [
    "parse_programa",
    "parse_expressao",
    "analisar",
    "compilar",
    "formatar",
    "info_api"
  ],
  "tipos": [
    "ResultadoAnalise",
    "ResultadoCompilacao",
    "DiagnosticoEstruturado",
    "ErroCoral",
    "ErroDeTraducao"
  ]
}
```

Funções

- `parse_programa(codigo)` - obtém AST estrutural de programa.
- `parse_expressao(texto)` - obtém AST de expressão.
- `analisar(codigo, nome_fonte=...)` - devolve diagnósticos estruturados.
- `compilar(codigo, nome_fonte=...)` - devolve resultado de compilação e Python gerado.
- `formatar(codigo)` - aplica o formatador Coral.
- `info_api()` - descreve versão e superfície da API.

Tipos públicos

`ResultadoAnalise`, `ResultadoCompilacao`, `DiagnosticoEstruturado`, `ErroCoral` e `ErroDeTraducao`.

Ferramentas externas devem preferir essa API à importação de módulos internos como `parser_programa` ou `backend_estrutural`, pois estes podem evoluir entre releases sem a mesma promessa de estabilidade.

Glossário

- **AST**: árvore sintática abstrata do programa.
- **Backend**: etapa que transforma a representação estrutural em Python executável.
- **DAP**: Debug Adapter Protocol usado pelo debugger.
- **LSP**: Language Server Protocol usado pelo servidor de linguagem.
- **PYZ**: aplicação Python empacotada como ZIP executável.
- **VSIX**: pacote de extensão do Visual Studio Code.
- **Superfície natural**: forma idiomática e determinística em português para uma operação Coral.
- **Contrato público**: comportamento verificado automaticamente e tratado como parte da superfície estável.
- **BackendNulo**: backend de jogos que permite testar o loop sem abrir janela.
- **Coral Manager**: orquestrador público da instalação e manutenção.
- **Coral Setup**: interface gráfica que consome o contrato do Manager.

Limites conhecidos

1. Coral depende de Python 3.10 ou superior; não é uma compilação nativa independente.
2. O código executado possui as permissões do processo Python; não há sandbox de segurança geral.
3. A análise semântica é contextual e não constitui prova completa de correção.
4. O parser do editor possui recuperação parcial, mas não é um parser incremental total.
5. Pygame CE, NumPy, psutil e pyserial continuam dependências externas opcionais de áreas específicas.
6. Medições de tempo e memória do laboratório são locais e não devem ser tratadas como resultados universais.
7. Windows nativo e a interface gráfica real do VS Code dependem da suíte de campo para evidência no host do usuário.
8. Arduino e ESP32 físicos dependem de validação em dispositivo real; o backend serial simulado não substitui esse teste.

Referências internas

- `Projeto/README.md` e `Documentacao/Desenvolvimento/Projeto/INDICE.md`.
- `Documentacao/Desenvolvimento/Projeto/PRINCIPIOS.md`, `ARQUITETURA_COMPILADOR.md` e `GRAMATICA.md`.

- `Documentacao/Desenvolvimento/Projeto/CORAL_SETUP_MANAGER.md`, `TUTORIAL_VSCODE.md` e `VALIDACOES_DE_CAMPO_1_4.md`.
- `Documentacao/Desenvolvimento/Projeto/PRONTIDAO_CORAL_1_4.md` e `PIPELINE_RELEASE.md`.
- `Projeto/vscode-coral/package.json`, cliente LSP e adaptador DAP.
- `Testes/` e `Testes/Fixtures/runtimes/` para regressões históricas e `Testes/Executar_Testes_Reais.coral` para evidência complementar em host real.

Referências externas oficiais

- Python: `https://docs.python.org/`
- Visual Studio Code: `https://code.visualstudio.com/docs`
- Language Server Protocol: `https://microsoft.github.io/language-server-protocol/`
- Debug Adapter Protocol: `https://microsoft.github.io/debug-adapter-protocol/`
- NumPy: `https://numpy.org/doc/`
- Matplotlib: `https://matplotlib.org/stable/`
- Plotly: `https://plotly.com/python/`
- psutil: `https://psutil.readthedocs.io/`
- pySerial: `https://pyserial.readthedocs.io/`
- Pygame CE: `https://pyga.me/docs/`

Encerramento

A Coral 1.7.6 parte de uma arquitetura fechada na 1.6.0, preserva a organização em seis áreas e mantém português corrente controlado, bibliotecas coesas, ferramentas integradas, instalação unificada e testes proporcionais ao impacto. Esta edição sincroniza o Livro com o runtime, preserva a geometria e visualização científica 3D da 1.6.12 e integra registros comuns e experimentos com Gráficos e Web, preservando a apresentação HTML interativa introduzida na 1.6.13. A linha 1.6.x não mantém uma frente pública dedicada a quatro dimensões espaciais. A implementação em Python continua sendo uma escolha de plataforma; a identidade da linguagem é definida por seus contratos, sua sintaxe e sua forma de explicar programas em português.

Compatibilidade da 1.7.4

A qualificação da release é autoritativa em Linux. Windows não bloqueia a publicação e só volta ao escopo quando solicitado. Dependências científicas e gráficas continuam opcionais; a ausência de um backend não torna o núcleo indisponível.

A análise ampliada de membros e tipos entre módulos pertence à linha seguinte. `programa principal` é opcional; programas lineares e funções comuns chamadas principal continuam válidos. Anotações estruturais não inserem coerção ou validação profunda em tempo de execução.

A revisão editorial tem identidade própria. Ela não substitui a evidência de execução dos artefatos binários congelados. Consulte `EDICAO.json` para os volumes e o relatório `VALIDACAO.json` para os gates editoriais desta revisão.

Compatibilidade da 1.7.5

Consultas nativas, `e_tipo`, chamadas compactas, scripts lineares e `programa principal` conservam seus contratos. A consulta natural de classes é uma forma preferencial de escrever código comum, não uma comparação nominal. O novo corpo de chamada é opcional e preserva a ordem dos argumentos.

A coleção oficial passa a ter dois volumes na versão 1.7.5. A constante histórica `coral.EDICAO_LIVRO` permanece em 1.7.0 para consumidores do antigo formato de livro único; ela não seleciona a coleção oficial. Novos consumidores devem usar `Livro/EDICAO.json`, chave `colecao`, e percorrer seus dois volumes. Os geradores usam a versão do runtime para a coleção e conservam o modo legado explícito.